

INTERNATIONAL STANDARD

IEC
61158-5

Third edition
2003-05

**Digital data communications
for measurement and control –
Fieldbus for use in industrial
control systems –**

**Part 5:
Application layer service definition**



Reference number
IEC 61158-5:2003(E)

Publication numbering

As from 1 January 1997 all IEC publications are issued with a designation in the 60000 series. For example, IEC 34-1 is now referred to as IEC 60034-1.

Consolidated editions

The IEC is now publishing consolidated versions of its publications. For example, edition numbers 1.0, 1.1 and 1.2 refer, respectively, to the base publication, the base publication incorporating amendment 1 and the base publication incorporating amendments 1 and 2.

Further information on IEC publications

The technical content of IEC publications is kept under constant review by the IEC, thus ensuring that the content reflects current technology. Information relating to this publication, including its validity, is available in the IEC Catalogue of publications (see below) in addition to new editions, amendments and corrigenda. Information on the subjects under consideration and work in progress undertaken by the technical committee which has prepared this publication, as well as the list of publications issued, is also available from the following:

- **IEC Web Site** (www.iec.ch)

- **Catalogue of IEC publications**

The on-line catalogue on the IEC web site (http://www.iec.ch/searchpub/cur_fut.htm) enables you to search by a variety of criteria including text searches, technical committees and date of publication. On-line information is also available on recently issued publications, withdrawn and replaced publications, as well as corrigenda.

- **IEC Just Published**

This summary of recently issued publications (http://www.iec.ch/online_news/justpub/jp_entry.htm) is also available by email. Please contact the Customer Service Centre (see below) for further information.

- **Customer Service Centre**

If you have any questions regarding this publication or need further assistance, please contact the Customer Service Centre:

Email: custserv@iec.ch
Tel: +41 22 919 02 11
Fax: +41 22 919 03 00

INTERNATIONAL STANDARD

IEC 61158-5

Third edition
2003-05

Digital data communications for measurement and control – Fieldbus for use in industrial control systems –

Part 5: Application layer service definition

© IEC 2003 — Copyright - all rights reserved

No part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from the publisher.

International Electrotechnical Commission, 3, rue de Varembé, PO Box 131, CH-1211 Geneva 20, Switzerland
Telephone: +41 22 919 02 11 Telefax: +41 22 919 03 00 E-mail: inmail@iec.ch Web: www.iec.ch



Commission Electrotechnique Internationale
International Electrotechnical Commission
Международная Электротехническая Комиссия

PRICE CODE **XW**

For price, see current catalogue

CONTENTS

FOREWORD.....	24
0 Introduction.....	26
0.1 General.....	26
0.2 Nomenclature for references within this standard	26
1 Scope	27
2 Normative references.....	27
3 Terms and definitions.....	28
3.1 ISO/IEC 7498-1 terms	28
3.2 ISO/IEC 8822 terms	29
3.3 ISO/IEC 9545 terms	29
3.4 ISO/IEC 8824 terms	29
3.5 Fieldbus Data Link Layer terms	29
3.6 Fieldbus Application Layer specific definitions	30
3.7 Abbreviations and symbols	41
3.8 Conventions	42
4 Concepts	45
4.1 Overview	45
4.2 Architectural relationships	46
4.3 Fieldbus Application Layer structure.....	48
4.4 Fieldbus Application Layer naming and addressing	60
4.5 Architecture summary.....	60
4.6 FAL service procedures.....	61
4.7 Common FAL attributes	62
4.8 Common FAL service parameters.....	62
4.9 APDU size	63
5 Data type ASE	63
5.1 General.....	63
5.2 Formal definition of data type objects	66
5.3 FAL defined data types.....	67
5.4 Data type ASE service specification	102
6 Type 1 communication model specification.....	102
6.1 Concepts.....	102
6.2 ASEs.....	102
6.3 ARs	246
6.4 Summary of FAL classes	270
6.5 Permitted FAL services by AREP role.....	271
7 Type 2 communication model specification.....	273
7.1 Concepts.....	273
7.2 ASEs	281
7.3 AR.....	346
7.4 Summary of FAL classes	382
7.5 Permitted FAL services by AR type	383
8 Type 3 communication model specification.....	383

8.1	DP concepts	383
8.2	ASEs	401
8.3	Summary of AL classes	734
8.4	Permitted AL services by AREP role	735
8.5	Conformance classes	739
8.6	Application characteristics	739
9	Type 4 communication model specification	740
9.1	Concepts	740
9.2	Variable ASE	747
9.3	Application relationship ASE	766
10	Type 5 communication model specification	772
10.1	Concepts	772
10.2	ASEs	793
10.3	FDA sessions	828
10.4	Summary of FAL Type 9 and Type 5 classes	837
10.5	Permitted FAL Type 9 and Type 5 services by AREP role	838
11	Type 6 communication model specification	840
11.1	Application relationship (AR) and Application relationship endpoint (AREP) characteristics	840
11.2	Relationship of Type 6 to Type 1	842
11.3	Permitted services by AREP role	846
12	Type 7 communication model specification	848
12.1	Concepts	848
12.2	ASEs	864
12.3	ARs	1033
13	Type 8 communication model specification	1053
13.1	Concepts	1053
13.2	ASEs	1055
13.3	Application relationships	1070
13.4	Permitted FAL services by AREP role	1071
14	Type 9 communication model specification	1072
14.1	Concepts	1072
14.2	Common parameters	1072
14.3	ASEs	1073
14.4	ARs	1163
14.5	Summary of classes	1166
14.6	Permitted services by AREP role	1166
15	Type 10 communication model specification	1168
15.1	Type 10 Concepts	1168
15.2	ASE Data Types	1174
15.3	ASEs	1175
15.4	ARs	1289
15.5	Summary of FAL classes	1292
15.6	Summary of FAL services	1292

Figure 1 – Relationship of IEC 61158-3 to other fieldbus layers and to users of the fieldbus Application Service	26
Figure 2 – Relationship to the OSI basic reference model	46
Figure 3 – Architectural positioning of the fieldbus Application Layer	47
Figure 4 – Client/server interactions	49
Figure 5 – Pull model interactions	50
Figure 6 – Push model interactions	51
Figure 7 – APOs services conveyed by the FAL	52
Figure 8 – Application entity structure	54
Figure 9 – Example FAL ASEs	55
Figure 10 – FAL management of objects	56
Figure 11 – ASE service conveyance	57
Figure 12 – Defined and established AREPs	59
Figure 13 – FAL architectural components	61
Figure 14 – Data type class hierarchy example	64
Figure 15 – The AR ASE conveys APDUs between APs	133
Figure 16 – 1-to-1 AR establishment	145
Figure 17 – 1-to-many AR establishment	145
Figure 18 – Event model overview	185
Figure 19 – Residence timeliness	261
Figure 20 – Synchronized timeliness	262
Figure 21 – Residence timeliness	267
Figure 22 – Synchronized timeliness	268
Figure 23 – Overview of ASE's and object classes	275
Figure 24 – Addressing format using MAC, class, instance and attribute IDs	275
Figure 25 – Static Assembly state transition diagram	293
Figure 26 – Dynamic Assembly state transition diagram	294
Figure 27 – Example of Find_Next_Object_Instance service	323
Figure 28 – Context of transport services within the connection model	349
Figure 29 – Application-to-application view of data transfer	349
Figure 30 – Data flow diagram for a link producer	350
Figure 31 – Data flow diagram for a link consumer	351
Figure 32 – Triggers	352
Figure 33 – Binding transport instances to the producer and consumer of a transport connection that does not have a reverse data path	353
Figure 34 – Binding transport instances to the producers and consumers of a transport connection that does have a reverse data path	353
Figure 35 – Binding transport instances to the producer and consumers of a multipoint connection when the transport connection does not have a reverse data path	354
Figure 36 – Binding transport instances to the producers and consumers of a multipoint connection when the transport connection does have reverse data paths	354
Figure 37 – Example of DP communication with a single controlling device	386
Figure 38 – Example of DP communication with several controlling devices	386
Figure 39 – Example of DP communication between field devices	386
Figure 40 – DP-slave model (modular DP-slave)	389
Figure 41 – DP-slave model (compact DP-slave)	389
Figure 42 – Overview of application processes	390

Figure 43 – DP-slave model (modular DP-slave).....	391
Figure 44 – Application Service Elements (ASEs)	393
Figure 45 – Application Process with application Objects (APOs).....	394
Figure 46 – Access to a remote APO	395
Figure 47 – Access to a remote APO for publisher/subscriber association	396
Figure 48 – Example of one AR with two AREPs	397
Figure 49 – Relation of a simple process data object to the real object	402
Figure 50 – Relation of a combined process data object to the real objects.....	405
Figure 51 – Sequence of an isochronous DP cycle with one DP-master (class 1).....	433
Figure 52 – Additional time relationships in a DP system operating in isochronous mode....	434
Figure 53 – DP system with optimised isochronous DP cycle	436
Figure 54 – Buffered synchronised isochronous mode at the DP-master (class 1).....	437
Figure 55 – Enhanced synchronised isochronous mode at the DP-master (class 1)	438
Figure 56 – Input, output and PLL state machine interaction	438
Figure 57 – PLL state diagram	444
Figure 58 – OUTPUT state diagram	448
Figure 59 – INPUT state diagram	452
Figure 60 – Treatment of an alarm in the DP system.....	481
Figure 61 – Load Region state diagram for erasable memory.....	568
Figure 62 – Load region state diagram for non erasable memory	568
Figure 63 – Function invocation state diagram	595
Figure 64 – System architecture	605
Figure 65 – Assignment of communication relationship to application relationship	612
Figure 66 – MS0 application relationship	618
Figure 67 – Output buffer model of a DP-slave without sync functionality.....	619
Figure 68 – Output buffer model of a DP-slave with sync functionality.....	619
Figure 69 – Input buffer model of a DP-slave without freeze functionality.....	620
Figure 70 – Input buffer model of a DP-slave with freeze functionality.....	620
Figure 71 – MS1 application relationship	621
Figure 72 – MS2 application relationship	622
Figure 73 – Example of inter-network communication	623
Figure 74 – Example without inter-network addressing.....	623
Figure 75 – First example with inter-network addressing	624
Figure 76 – Second example with inter-network addressing	624
Figure 77 – MS3 application relationship	625
Figure 78 – MM1 application relationship	626
Figure 79 – MM2 application relationship	627
Figure 80 – Cycle time of the DP system	740
Figure 81 – FAL AE	741
Figure 82 – Summary of the FAL architecture	744
Figure 83 – FAL service procedure overview.....	745
Figure 84 – Time sequence diagram for the confirmed services	746
Figure 85 – Time sequence diagram for unconfirmed services	747
Figure 86 – VCR initiation	778
Figure 87 – Misordered message handling.....	784
Figure 88 – FF SM port message processing order	785

Figure 89 – FF FDA port message processing order	785
Figure 90 – FF TCP connection message processing order	786
Figure 91 – Session endpoint message processing order.....	786
Figure 92 – FDA LAN redundancy port message processing order	786
Figure 93 – Message processing by receiving entity	787
Figure 94 – Organisation of the ASEs and ARs	848
Figure 95 – Object model of the MPS ASE	868
Figure 96 – Time-out evaluation net.....	881
Figure 97 – Asynchronous promptness status evaluation net	885
Figure 98 – Synchronous promptness status evaluation net	886
Figure 99 – Punctual promptness status evaluation net	888
Figure 100 – Asynchronous refreshment status evaluation net.....	891
Figure 101 – Synchronous refreshment status evaluation net.....	892
Figure 102 – Punctual refreshment status evaluation net	894
Figure 103 – A_Readloc service procedure.....	897
Figure 104 – A_Writeloc service procedure.....	898
Figure 105 – A_Update service procedure	900
Figure 106 – A_Readfar service procedure	902
Figure 107 – A_Writefar service procedure	904
Figure 108 – A_Sent service procedure	905
Figure 109 – A_Received service procedure	906
Figure 110 – A_Read service procedure	908
Figure 111 – A_Read service state machine	909
Figure 112 – A_Write service procedure	910
Figure 113 – A_Write service state machine	911
Figure 114 – Model of a resynchronised variable	914
Figure 115 – Principles for resynchronisation of a produced variable	915
Figure 116 – Resynchronisation mechanism state machine for a produced variable.....	917
Figure 117 – Asynchronous refreshment private mechanism evaluation net.....	918
Figure 118 – Asynchronous refreshment public mechanism evaluation net	919
Figure 119 – Synchronous refreshment private mechanism evaluation net.....	920
Figure 120 – Synchronous refreshment public mechanism evaluation net	921
Figure 121 – Punctual refreshment private mechanism evaluation net	922
Figure 122 – Punctual refreshment public mechanism evaluation net.....	923
Figure 123 – Principles for the resynchronisation of a consumed variable.....	924
Figure 124 – Resynchronisation mechanism state machine for consumed variable	926
Figure 125 – Asynchronous promptness public mechanism evaluation net.....	927
Figure 126 – Asynchronous promptness private mechanism evaluation net	928
Figure 127 – Synchronous promptness public mechanism evaluation net.....	929
Figure 128 – Synchronous promptness private mechanism evaluation net	931
Figure 129 – Punctual promptness public mechanism evaluation net	932
Figure 130 – Punctual promptness private mechanism evaluation net.....	933
Figure 131 – Spatial consistency list variables interchange mechanism	936
Figure 132 – Spatial consistency – consistency variable interchange mechanism	936
Figure 133 – Spatial consistency – list recovery mechanism	937
Figure 134 – Spatial consistency – validity of the spatial consistency status	937

Figure 135 – Object model of a variable list	938
Figure 136 – A_Readlist service procedure	943
Figure 137 – Consistency variable value evaluation net	949
Figure 138 – Consistency interchange timing diagram	950
Figure 139 – Recovery mechanism evaluation net	951
Figure 140 – Recovery interchange timing diagram	951
Figure 141 – Flowchart of the sub-MMS environment management state	958
Figure 142 – Domain management state chart	988
Figure 143 – Domain upload flowchart	991
Figure 144 – Domain download sequence diagram	992
Figure 145 – Domain upload sequence diagram	992
Figure 146 – Program invocation state chart	1004
Figure 147 – A_Associate service procedure	1042
Figure 148 – A_Release service procedure	1045
Figure 149 – A_Abort service procedure	1046
Figure 150 – A_Data service procedure	1048
Figure 151 – A_Unidata service procedure	1051
Figure 152 – Associated mode service state chart	1052
Figure 153 – Non-associated mode service state chart	1053
Figure 154 – Architectural positioning of the fieldbus Application Layer	1053
Figure 155 – VFD model	1073
Figure 156 – Abstract model of an automation system (VFD)	1073
Figure 157 – Source OD/remote OD	1080
Figure 158 – Put OD state machine	1093
Figure 159 – Transaction object state machine	1100
Figure 160 – Context test of two features-supported with different bitstring length	1107
Figure 161 – Overview of event	1127
Figure 162 – Event state machine	1133
Figure 163 – Domain GenericDownload/Download state machine (server)	1147
Figure 164 – Domain Upload state machine (server)	1149
Figure 165 – State diagram	1161
Figure 166 – FAL ASEs Communication Architecture	1170
Figure 167 – Runtime object model	1171
Figure 168 – Relationship between Engineering and Runtime	1172
Figure 169 – Navigation in the runtime object model	1173
Figure 170 – Operating state block diagram	1198
Figure 171 – Device status model for the common diagnosis	1199
Figure 172 – ACCO ASE structure	1223
Figure 173 – Productive operation of data connections	1224
Figure 174 – Productive operation of event connections	1225
Figure 175 – Quality Code transfer – standard behavior	1233
Figure 176 – Quality Code with communication fault	1234
Figure 177 – Quality Code when an connection is cleared	1235
Figure 178 – Quality Code when an connection is deactivated	1235
Figure 179 – Quality Code during the transfer of "incorrect" connection data	1236
Figure 180 – Quality Code for provider in "Ready" state	1236

Figure 181 — Quality Code when clearing an object from the provider.....	1237
Figure 182 — Quality Code when a connection is forced.....	1237
Figure 183 — Quality Code at QoS violation	1238
Figure 184 — Quality Code for initial value	1238
Figure 185 — Productive operation of data connections.....	1242
Figure 186 — Productive operation of event connections.....	1243
Figure 187 — Failure of the provider in productive operation	1246
Figure 188 — Failure of the consumer	1247
Figure 189 — Failure of the provider when changing the connection status	1248
Figure 190 — Failure of the provider when changing the interconnection status	1249
Figure 191 — Failure of the provider when clearing connections.....	1250
Figure 192 — Information levels	1250
Figure 193 — ACCO ASE status model for the common diagnosis.....	1251
Figure 194 — ACCO ASE status model for the detailed diagnosis	1251
Figure 195 — Structure of the transmitted connection data	1274
Table 1 — PERSISTDEF	72
Table 2 — VARTYPE	73
Table 3 — ITEMQUALITYDEF.....	74
Table 4 — STATEDEF	77
Table 5 — GROUPEXCEPTIONDEF.....	77
Table 6 — ACCESSRIGHTSDEF.....	78
Table 7 — HRESULT	78
Table 8 — UUID.....	85
Table 9 — Data type names for value.....	100
Table 10 — UUID.....	102
Table 11 — Create service parameters	103
Table 12 — Delete service parameters.....	104
Table 13 — Get Attributes service parameters	106
Table 14 — Set Attributes service parameters.....	108
Table 15 — Begin Set Attributes	110
Table 16 — End Set Attributes	111
Table 17 — Subscribe service parameters	120
Table 18 — Identify	123
Table 19 — Get status	124
Table 20 — Status notification.....	125
Table 21 — Initiate.....	126
Table 22 — Terminate.....	129
Table 23 — Conclude.....	131
Table 24 — Reject	131
Table 25 — Conveyance of service primitives by AREP role.....	134
Table 26 — Valid combinations of AREP roles involved in an AR	134
Table 27 — AR-Unconfirmed Send.....	140
Table 28 — AR-Confirmed Send.....	142
Table 29 — AR-Establish service	144
Table 30 — Valid combinations of AREP classes to be related	146

Table 31 – AR-DeEstablish service	147
Table 32 – AR-Abort	148
Table 33 – AR-Compel service.....	149
Table 34 – AR-Get Buffered Message service.....	150
Table 35 – AR-Schedule Communication service	151
Table 36 – AR-Cancel Scheduled Sequence service	152
Table 37 – AR-Status.....	153
Table 38 – AR-XON-OFF	154
Table 39 – AR-Remote Read service	155
Table 40 – AR-Remote Write service	156
Table 41 – Read service parameters.....	165
Table 42 – Read List service parameters	168
Table 43 – Write service parameters.....	170
Table 44 – Write List service parameters	172
Table 45 – Information Report service	174
Table 46 – Information Report List service	176
Table 47 – Exchange service parameters	178
Table 48 – Exchange List service parameters	182
Table 49 – Acknowledge Event	194
Table 50 – Acknowledge Event List service parameters	195
Table 51 – Enable event	197
Table 52 – Event Notification service parameters.....	198
Table 53 – Enable Event List	200
Table 54 – Notification recovery service parameters	201
Table 55 – Get Event Summary service parameters.....	202
Table 56 – Get Event Summary List service parameters	204
Table 57 – Query Event Summary List service parameters	208
Table 58 – Initiate Load service parameters.....	216
Table 59 – Terminate Load service parameters.....	218
Table 60 – Push Segment service parameters	219
Table 61 – Pull Segment service parameters	220
Table 62 – Discard service parameters	221
Table 63 – Pull Upload Sequencing of service primitives	223
Table 64 – Pull Upload service parameter constraints	223
Table 65 – Pull Upload state table	224
Table 66 – Pull Download Sequencing of service primitives	225
Table 67 – Pull Download service parameter constraints	225
Table 68 – Pull Download state table	226
Table 69 – Push Download sequencing of service primitives.....	227
Table 70 – Push Download service parameter constraints	228
Table 71 – Push Download state table	228
Table 72 – Start service parameters	235
Table 73 – Stop service parameters.....	236
Table 74 – Resume service parameters	237
Table 75 – Reset service parameters	238
Table 76 – Kill service parameters	239

Table 77 – Action Invoke service parameters	240
Table 78 – Action Return service parameters	241
Table 79 – State transitions for a function invocation object	243
Table 80 – FAL class summary	271
Table 81 – Services by AREP role	271
Table 82 – Common elements	278
Table 83 – ST language elements	279
Table 84 – Type conversion operations	279
Table 85 – Values of implementation-dependent parameters	280
Table 86 – Extensions to IEC 61131-3	281
Table 87 – Static Assembly state event matrix	293
Table 88 – Dynamic Assembly state event matrix	294
Table 89 – Message Router object Forward_Open parameters	297
Table 90 – Status codes	299
Table 91 – Format of extended status	301
Table 92 – Get_Attribute_All service parameters	302
Table 93 – Set_Attribute_All service parameters	303
Table 94 – Get_Attribute_List service parameters	305
Table 95 – Set_Attribute_List service parameters	307
Table 96 – Reset service parameters	309
Table 97 – Start service parameters	311
Table 98 – Stop service parameters	313
Table 99 – Create service parameters	315
Table 100 – Delete service parameters	317
Table 101 – Get_Attribute_Single service parameters	318
Table 102 – Set_Attribute_Single service parameters	320
Table 103 – Find_Next_Object_Instance service parameters	322
Table 104 – NOP service parameters	324
Table 105 – Apply_Attributes service parameters	325
Table 106 – Save service parameters	327
Table 107 – Restore service parameters	329
Table 108 – CM_Open service parameters	337
Table 109 – CM_Close service parameters	339
Table 110 – CM_Unconnected_Send service parameters	340
Table 111 – CM_Get_Connection_Data service parameters	342
Table 112 – CM_Search_Connection_Data service parameters	344
Table 113 – CM_Get_Connection_Data service parameters	345
Table 114 – How production trigger, transport class, and CM_RPI determine when data is produced	348
Table 115 – Transport classes	358
Table 116 – UCMM_Create service parameters	375
Table 117 – UCMM_Delete service parameters	376
Table 118 – UCMM_Write service parameters	377
Table 119 – UCMM_Abort service parameters	379
Table 120 – TR_Write service parameters	379
Table 121 – TR_Trigger service parameters	380

Table 122 – TR_Packet_arrived service parameters	380
Table 123 – TR_Ack_received service parameters	381
Table 124 – TR_Verify service parameters	381
Table 125 – TR_Status_updated service parameters	382
Table 126 – FAL class summary	382
Table 127 – FAL services by AR type	383
Table 128 – Requirements and features of fieldbus DP	385
Table 129 – Status values of the service primitives	401
Table 130 – Access Rights MS1	403
Table 131 – Access Rights MS2	404
Table 132 – Access Rights MS1	406
Table 133 – Access Rights MS2	406
Table 134 – SCL matching rules	407
Table 135 – Read	408
Table 136 – Write	409
Table 137 – Data transport	410
Table 138 – Format (simple input data description)	414
Table 139 – Consistency (simple input data description)	414
Table 140 – Format (simple output data)	416
Table 141 – Consistency (simple output data)	416
Table 142 – Format (extended input data)	417
Table 143 – Consistency (extended input data)	417
Table 144 – Format (extended output data)	419
Table 145 – Consistency (extended output data)	419
Table 146 – Set Input	420
Table 147 – Read Input	421
Table 148 – Get Input	422
Table 149 – New Input	422
Table 150 – Set Output	424
Table 151 – Final	424
Table 152 – Read Output	425
Table 153 – Get Output	426
Table 154 – Clear Flag	426
Table 155 – New Flag	426
Table 156 – New Output	426
Table 157 – Clear Flag	427
Table 158 – Global Control	427
Table 159 – Clear Command	428
Table 160 – Sync Command	428
Table 161 – Freeze Command	428
Table 162 – New publisher data	428
Table 163 – Get publisher data	429
Table 164 – New Flag	429
Table 165 – SYNCH	430
Table 166 – SYNCH Delayed	430
Table 167 – DX Finished	431

Table 168 – SYNCH Event.....	431
Table 169 – Status.....	431
Table 170 – Primitives issued by the AL to the PLL state machine	440
Table 171 – Primitives issued by the user to the PLL state machine	440
Table 172 – Allowed values of status	440
Table 173 – Primitives issued by the user to the input state machine	440
Table 174 – Primitives issued by the user to the output state machine	441
Table 175 – Primitives issued by the PLL to the output state machine	441
Table 176 – Primitives issued by the output to the PLL state machine	441
Table 177 – Primitives issued by the PLL to the input state machine.....	441
Table 178 – Primitives issued by the output to the input state machine	441
Table 179 – Primitives issued by the output state machine to the AL	442
Table 180 – Primitives issued by the AL to the output state machine	442
Table 181 – Primitives issued by the input state machine to the AL	442
Table 182 – Primitives issued by the AL to the input state machine	442
Table 183 – PLL state table	445
Table 184 – OUTPUT state table	449
Table 185 – INPUT state table	453
Table 186 – Identifier status.....	455
Table 187 – Channel type	456
Table 188 – IO type	456
Table 189 – Error type	457
Table 190 – Status type	457
Table 191 – Status specifier	458
Table 192 – Status specifier	459
Table 193 – Module status.....	459
Table 194 – Status specifier	460
Table 195 – Link status.....	460
Table 196 – Link error.....	461
Table 197 – Set Slave Diag	462
Table 198 – Ext Diag Flag.....	462
Table 199 – Get Slave Diag	464
Table 200 – Read Slave Diag	473
Table 201 – New Slave Diag.....	481
Table 202 – Alarm type	483
Table 203 – Add Ack.....	483
Table 204 – Alarm specifier	483
Table 205 – Alarm notification.....	484
Table 206 – Alarm Ack.....	485
Table 207 – Prm data type.....	490
Table 208 – Supported feature.....	499
Table 209 – Supported profile feature	499
Table 210 – Role	500
Table 211 – Check user Prm.....	502
Table 212 – Prm structure.....	503
Table 213 – MS1 Command.....	505

Table 214 – Check user Prm result	507
Table 215 – Status values.....	507
Table 216 – Check Ext user Prm.....	508
Table 217 – Check Ext user Prm result	511
Table 218 – Status values.....	511
Table 219 – Check Cfg	512
Table 220 – Check Cfg result.....	513
Table 221 – Status values.....	513
Table 222 – Set Cfg	514
Table 223 – Get Cfg	514
Table 224 – Set Slave Add	515
Table 225 – Initiate	516
Table 226 – Abort	519
Table 227 – Instance	519
Table 228 – MS0 init DP-slave.....	520
Table 229 – MS1 init DP-slave.....	520
Table 230 – MS2 init DP-slave.....	520
Table 231 – DP-slave started.....	521
Table 232 – Alarm limit	521
Table 233 – DP-slave stopped	522
Table 234 – Reset DP-slave	522
Table 235 – DP-slave fault.....	522
Table 236 – Application ready DP-slave.....	522
Table 237 – Start subscriber	523
Table 238 – Stop subscriber	523
Table 239 – Publisher active.....	524
Table 240 – Status.....	524
Table 241 – Init DP-master CI1	525
Table 242 – DP-master CI1 started	526
Table 243 – Alarm limit	526
Table 244 – DP-master CI1 stopped	526
Table 245 – Reset DP-master CI1	527
Table 246 – DP-master CI1 fault	527
Table 247 – DP-master CI1 reject	527
Table 248 – Set mode DP-master CI1	528
Table 249 – DP-master CI1 mode changed	529
Table 250 – Load bus Par DP-master CI1	529
Table 251 – Mark DP-master CI1	530
Table 252 – Abort DP-master CI1	531
Table 253 – Read value DP-master CI1	531
Table 254 – Delete SC DP-master CI1	531
Table 255 – DP-master CI1 event	532
Table 256 – Init DP-master CI2.....	532
Table 257 – Reset DP-master CI2.....	533
Table 258 – DP-master CI2 fault	534
Table 259 – DP-master CI2 reject	534

Table 260 – DP-master CI2 closed	534
Table 261 – DP-master CI2 event	535
Table 262 – USIF state	536
Table 263 – Data rate	539
Table 264 – USIF state	540
Table 265 – Isochronous mode	540
Table 266 – Slave type	543
Table 267 – Alarm mode	543
Table 268 – Get Master Diag	546
Table 269 – MDiag identifier	546
Table 270 – Start Seq	547
Table 271 – Area code (start seq)	548
Table 272 – Download	549
Table 273 – Upload	550
Table 274 – End Seq	551
Table 275 – Act Para Brct	551
Table 276 – Area code (Act Para Brct)	552
Table 277 – Act param	552
Table 278 – Area code (Act param)	553
Table 279 – Activate	553
Table 280 – Access rights MS1	555
Table 281 – Access rights MS2	556
Table 282 – Load region state	556
Table 283 – Initiate load	558
Table 284 – Default values for the parameter Intersegment Request Timeout	558
Table 285 – Push segment	559
Table 286 – Pull segment	561
Table 287 – Terminate load	562
Table 288 – Primitives issued by the user to the Load Region state machine	564
Table 289 – Primitives issued by the Load Region state machine to the user	564
Table 290 – Primitives issued by the Function Invocation to the Load Region state machine	565
Table 291 – Primitives issued by the Load Region to the Function Invocation state machine	565
Table 292 – Load Region state definitions	565
Table 293 – Load Region function table	566
Table 294 – Load Region state table for erasable memory	569
Table 295 – Load Region state table for non erasable memory	578
Table 296 – Access rights MS1	582
Table 297 – Access rights MS2	582
Table 298 – Function Invocation state	583
Table 299 – Load Region object in use	583
Table 300 – Access rights MS1	584
Table 301 – Access rights MS2	584
Table 302 – Load Region object in use	585
Table 303 – Start	586

Table 304 – Stop	587
Table 305 – Resume	587
Table 306 – Reset	588
Table 307 – Get FI state	589
Table 308 – Call	590
Table 309 – Primitives issued by the user to the Function Invocation state machine	592
Table 310 – Primitives issued by the Function Invocation state machine to the user	592
Table 311 – Primitives issued by the Load Region to the Function Invocation state machine	593
Table 312 – Primitives issued by the Function Invocation to the Load Region state machine	593
Table 313 – Function Invocation state definitions	593
Table 314 – Function definitions	594
Table 315 – Function Invocation state table	595
Table 316 – CS status	607
Table 317 – Summertime	607
Table 318 – Synchronisation active	608
Table 319 – Announcement hour	609
Table 320 – Summertime	609
Table 321 – Accuracy	609
Table 322 – Set time	610
Table 323 – Sync interval violation	611
Table 324 – Parameter of Initiate service without inter-network addressing	624
Table 325 – Parameter of Initiate service with inter-network addressing (first example)	624
Table 326 – Parameter of Initiate service with inter-network addressing (second example)	625
Table 327 – AR type	630
Table 328 – Sync supported	632
Table 329 – Freeze supported	632
Table 330 – Group identifier	634
Table 331 – DPV1 enabled	634
Table 332 – Fail safe	635
Table 333 – WD base	635
Table 334 – No Add change	637
Table 335 – Alarm mode supported	639
Table 336 – Isochronous mode supp	643
Table 337 – Isochronous mode	643
Table 338 – Alarm mode	644
Table 339 – Time device type	645
Table 340 – S_SAP_index	648
Table 341 – D_addr	648
Table 342 – Service_activate	649
Table 343 – Role_in_service	649
Table 344 – Indication_mode	650
Table 345 – Max_DLSDU_length_req_low	650
Table 346 – Max_DLSDU_length_req_high	651
Table 347 – Max_DLSDU_length_ind_low	651

Table 348 – Max_DLSDU_length_ind_high	652
Table 349 – S_SAP_index	656
Table 350 – D_SAP_index	657
Table 351 – D_addr	657
Table 352 – Service_activate	658
Table 353 – Role_in_service	658
Table 354 – Indication_mode	659
Table 355 – Max_DLSDU_length_req_low	659
Table 356 – Max_DLSDU_length_req_high	659
Table 357 – Max_DLSDU_length_ind_low	660
Table 358 – Max_DLSDU_length_ind_high	660
Table 359 – Sync	661
Table 360 – Freeze	661
Table 361 – DPV1 enabled	663
Table 362 – Fail safe	663
Table 363 – Enable publisher	663
Table 364 – WD base	663
Table 365 – Alarm mode	665
Table 366 – Fail safe	673
Table 367 – S_SAP_index	680
Table 368 – D_SAP_index	680
Table 369 – D_addr	681
Table 370 – Service_activate	681
Table 371 – Role_in_service	681
Table 372 – Max_DLSDU_length_req_low	681
Table 373 – Max_DLSDU_length_req_high	682
Table 374 – Max_DLSDU_length_ind_low	682
Table 375 – Max_DLSDU_length_ind_high	682
Table 376 – DLL init DP-slave	683
Table 377 – Load ARL DP-slave	683
Table 378 – Get ARL DP-slave	689
Table 379 – Set ARL isochronous mode	694
Table 380 – Load ARL DP-master CI1	695
Table 381 – Get ARL DP-master CI1	697
Table 382 – ARL Slave update DP-master CI1	699
Table 383 – Load ARL DP-master CI2	700
Table 384 – Get ARL DP-master CI2	701
Table 385 – Load CRL DP-slave	702
Table 386 – Load CRL DXB link entries	703
Table 387 – Get CRL DP-slave	704
Table 388 – Load CRL DP-master CI1	706
Table 389 – Get CRL DP-master CI1	717
Table 390 – CRL Slave activate	728
Table 391 – CRL Slave new Prm	729
Table 392 – CRL Slave new Prm data	730
Table 393 – Load CRL DP-master CI2	732

Table 394 – Get CRL DP-master CI2	733
Table 395 – Fieldbus AL class summary	735
Table 396 – Assignment of the services to DP-masters and DP-slaves	736
Table 397 – Support of AR types in the different DP-device types.....	737
Table 398 – Support of services at the different AREPs respectively CREPs	737
Table 399 – Conformance classes DP-master (class 1)	739
Table 400 – Conformance classes DP-master (class 2)	739
Table 401 – REQUEST service parameters	762
Table 402 – RESPONSE service parameters	763
Table 403 – Error codes by source	764
Table 404 – Reserve REP service parameters	764
Table 405 – Free AREP service parameters	765
Table 406 – Get REP attribute service parameters.....	765
Table 407 – Set REP attribute service parameters	766
Table 408 – AR Send service parameters	769
Table 409 – AR Acknowledge service parameters.....	770
Table 410 – AR Get Attributes service parameters.....	770
Table 411 – AR Set Attributes service parameters	771
Table 412 – Scope of Invoke Id.....	782
Table 413 – Types of misordering detectable by message numbers.....	783
Table 414 – Delivery of misordered message types on publisher/subscriber VCRs	783
Table 415 – Statistics gathered per VCR	783
Table 416 – Determination of misordering type at a subscriber VCR	784
Table 417 – Mapping of received messages to primitives.....	784
Table 418 – Mapping of received primitives to messages.....	785
Table 419 – Defined network addresses	788
Table 420 – Use of network addresses	788
Table 421 – Use of endpoint selectors in server VCRs.....	789
Table 422 – Use of endpoint selectors in publisher VCRs	789
Table 423 – Use of endpoint selectors in source VCRs	789
Table 424 – Network address and port numbers for device annunciation	791
Table 425 – Network address and port numbers for set/clear assignment info and clear address	791
Table 426 – Network address and port numbers for SM identify.....	791
Table 427 – Network address and port numbers for SM find tag	792
Table 428 – Network address and port numbers for clients and servers (part 1).....	792
Table 429 – Network address and port numbers for clients and servers (part 2).....	792
Table 430 – Network address and port numbers for publishers and subscribers.....	792
Table 431 – Network address and port numbers for report distribution.....	792
Table 432 – Network address and port numbers for LAN redundancy get and put information.....	792
Table 433 – Network address and port numbers for LAN redundancy diagnostics	792
Table 434 – VCR types	794
Table 435 – Use of VCR user id.....	795
Table 436 – Use of FDA address	795
Table 437 – Initiate	797

Table 438 – Find tag query service parameters.....	802
Table 439 – SMK IDs	803
Table 440 – Find tag reply service parameters.....	804
Table 441 – Identify service parameters	807
Table 442 – Annunciate service parameters.....	810
Table 443 – Set assignment info service parameters	812
Table 444 – Clear assignment info service parameters	815
Table 445 – Clear address service parameters	817
Table 446 – Diagnostic message service	822
Table 447 – Get redundancy info service	823
Table 448 – Put redundancy info service	825
Table 449 – Get redundancy statistics service	827
Table 450 – Open session service	834
Table 451 – Idle session service	837
Table 452 – FAL class summary	838
Table 453 – Services by AREP role	839
Table 454 – Services by AREP role	847
Table 455 – Access protection.....	863
Table 456 – Binary time coding.....	879
Table 457 – Asynchronous promptness events and actions	885
Table 458 – Synchronous promptness events and actions	886
Table 459 – Punctual promptness events and actions	888
Table 460 – Asynchronous refreshment events and actions	891
Table 461 – Synchronous refreshment events and actions.....	892
Table 462 – Punctual refreshment events and actions	895
Table 463 – A_Readloc service parameters	896
Table 464 – A_Writeloc service parameters	897
Table 465 – A_Update service parameters.....	899
Table 466 – A_Readfar service parameters	901
Table 467 – A_Writefar service parameters	903
Table 468 – A_Sent service parameters.....	905
Table 469 – A_Received service parameters	906
Table 470 – A_Read service parameters	907
Table 471 – A_Write service parameters.....	909
Table 472 – Asynchronous refreshment private mechanism events and actions	918
Table 473 – Asynchronous refreshment public mechanism events and actions	919
Table 474 – Synchronous refreshment private mechanism events and actions.....	920
Table 475 – Synchronous refreshment public mechanism events and actions	921
Table 476 – Punctual refreshment private mechanism events and actions	923
Table 477 – Punctual refreshment public mechanism events and actions.....	924
Table 478 – Asynchronous promptness public mechanism events and actions.....	927
Table 479 – Asynchronous promptness private mechanism events and actions	928
Table 480 – Synchronous promptness public mechanism events and actions.....	930
Table 481 – Synchronous promptness privatemechanism events and actions	931
Table 482 – Punctual promptness public mechanism events and actions	933
Table 483 – Punctual promptness privatemechanism events and actions.....	934

Table 484 – A_Readlist service parameters	942
Table 485 – Confirmed initiate service parameters.....	962
Table 486 – Detailed structure of the extension calling parameter.....	963
Table 487 – Detailed structure of the init request detail parameter.....	964
Table 488 – Detailed structure of the extension called parameter	965
Table 489 – Detailed structure of the init request detail parameter.....	966
Table 490 – Conclude service parameter	967
Table 491 – Unconfirmed abort service parameters	969
Table 492 – Unconfirmed reject service parameters.....	970
Table 493 – Confirmed status service parameters.....	972
Table 494 – Unconfirmed unsolicited status service parameter	972
Table 495 – Confirmed identify service parameters.....	973
Table 496 – Confirmed get name list service paramaters	974
Table 497 – Access group attribute description for domain object	976
Table 498 – Access rights attribute description for domain object	977
Table 499 – Confirmed delete domain service parameters	977
Table 500 – Confirmed initiate download sequence service parameters.....	978
Table 501 – Confirmed download segment service parameters	980
Table 502 – Confirmed terminate download sequence service parameters.....	981
Table 503 – Confirmed initiate upload sequence service parameters	982
Table 504 – Confirmed upload segment service parameters	983
Table 505 – Confirmed terminate upload sequence service parameters	984
Table 506 – Confirmed get domain attributes service parameters	985
Table 507 – Access group attribute details for program invocation object	994
Table 508 – Access rights attribute details for program invocation object.....	994
Table 509 – Confirmed create program invocation service parameters.....	995
Table 510 – Confirmed delete program invocation service parameters.....	997
Table 511 – Confirmed start service parameters	998
Table 512 – Confirmed stop service parameters	999
Table 513 – Confirmed resume service parameters	1000
Table 514 – Confirmed reset service parameters	1001
Table 515 – Confirmed kill service parameters.....	1002
Table 516 – Access group attribute details for variable object.....	1006
Table 517 – Access rights attribute details for variable object.....	1007
Table 518 – Access group attribute details for variable list object	1008
Table 519 – Access right attribute details for variable list objects.....	1008
Table 520 – Confirmed read service parameters	1009
Table 521 – Confirmed write service parameters	1011
Table 522 – Unconfirmed information report service parameters.....	1012
Table 523 – Confirmed define variable-list service parameters.....	1013
Table 524 – Confirmed delete variable-list service parameters.....	1014
Table 525 – Confirmed get variable access attributes service parameters.....	1015
Table 526 – Confirmed get variable-list attributes service parameters.....	1017
Table 527 – Data type specification	1018
Table 528 – Variable access specification.....	1019
Table 529 – Variable access description attribute details	1020

Table 530 – Path selection parameters	1021
Table 531 – Access group attribute detail for event object	1024
Table 532 – Access rights attribute details for event object	1024
Table 533 – Unconfirmed event notification service parameters	1025
Table 534 – Event type parameter details	1025
Table 535 – Confirmed acknowledged event notification service parameter	1027
Table 536 – Confirmed alter event condition monitoring service parameters	1028
Table 537 – Confirmed get alarm summary service parameters	1030
Table 538 – Confirmed get event condition attributes service parameters	1032
Table 539 – Classification of service quality parameters	1035
Table 540 – Identification parameters	1039
Table 541 – List of MCS AR ASE services	1040
Table 542 – A_Associate service parameters	1040
Table 543 – A_Release service parameters	1045
Table 544 – A_Abort service parameters	1046
Table 545 – A_Data service parameters	1047
Table 546 – A_Unidata service parameters	1048
Table 547 – Get Attributes service parameters	1056
Table 548 – Initiate service parameters	1059
Table 549 – Conveyance of service primitives by AREP role	1060
Table 550 – Valid combinations of AREP roles involved in an AR	1060
Table 551 – AR-Unconfirmed Send service parameters	1062
Table 552 – AR-Establish service parameters	1063
Table 553 – AR-Data-Send-Acknowledge service parameters	1063
Table 554 – Read service parameters	1066
Table 555 – Write service parameters	1066
Table 556 – Information Report service parameters	1067
Table 557 – Start service parameters	1068
Table 558 – Stop service parameters	1068
Table 559 – Resume service parameters	1069
Table 560 – Reset service parameters	1069
Table 561 – Services by AREP role	1072
Table 562 – Logical status	1075
Table 563 – Status	1076
Table 564 – Unsolicited status	1077
Table 565 – Identify	1078
Table 566 – Structure of the object dictionary	1081
Table 567 – Structure of the static list of types	1081
Table 568 – Structure of the static object dictionary	1081
Table 569 – Structure of the dynamic list of variable lists	1082
Table 570 – Structure of the dynamic list of program invocations	1082
Table 571 – Empty object dictionary	1086
Table 572 – Get OD service parameters	1089
Table 573 – Initiate Put OD service parameters	1091
Table 574 – Put OD service parameters	1092
Table 575 – Terminate Put OD service parameters	1093

Table 576 – Put OD state transitions.....	1095
Table 577 – Attribute FMS features supported	1097
Table 578 – Transaction object state transitions	1100
Table 579 – Initiate service parameters	1101
Table 580 – Failure reasons	1102
Table 581 – Abort service parameters.....	1103
Table 582 – User abort reasons	1104
Table 583 – APO ASE abort reasons	1104
Table 584 – Reject service parameters	1105
Table 585 – Reject APDU reasons	1105
Table 586 – Compatibility of the local context to the remote context	1106
Table 587 – Unconfirmed Send service parameters	1109
Table 588 – Confirmed Send service parameters	1110
Table 589 – AR-Abort service parameters.....	1110
Table 590 – Compel service parameters	1111
Table 591 – Get Buffered Message service parameters	1112
Table 592 – AR-Status service parameters	1113
Table 593 – Simple variable access group membership	1115
Table 594 – Simple variable access rights membership	1115
Table 595 – Array variable access group membership	1117
Table 596 – Array variable access rights membership.....	1117
Table 597 – Variable list access group membership.....	1119
Table 598 – Variable list access rights membership.....	1119
Table 599 – Read service parameters.....	1122
Table 600 – Write service parameters.....	1123
Table 601 – Information Report service parameters	1124
Table 602 – Define Variable List service parameters	1125
Table 603 – Delete Variable List service parameters	1126
Table 604 – Event access group membership	1128
Table 605 – Event access rights membership	1129
Table 606 – Event Notification service parameters.....	1130
Table 607 – Acknowledge Event Notification service parameters	1131
Table 608 – Alter Event Condition Monitoring service parameters.....	1132
Table 609 – Event state transitions	1133
Table 610 – Domain access group membership	1134
Table 611 – Domain access rights membership	1134
Table 612 – GenericInitiateDownloadSequence	1136
Table 613 – GenericDownloadSegment	1137
Table 614 – GenericTerminateDownloadSequence	1138
Table 615 – InitiateDownloadSequence	1139
Table 616 – DownloadSegment	1140
Table 617 – TerminateDownloadSequence	1141
Table 618 – RequestDomainDownload.....	1142
Table 619 – InitiateUploadSequence.....	1143
Table 620 – UploadSegment.....	1144
Table 621 – TerminateUploadSequence	1145

Table 622 – RequestDomainUpload	1146
Table 623 – Domain GenericDownload/Download state machine (server)	1147
Table 624 – Domain Upload state machine (server)	1149
Table 625 – Program invocation access group membership	1151
Table 626 – Program invocation access group membership	1151
Table 627 – Create Program Invocation service parameters	1153
Table 628 – Delete Program Invocation service parameters	1154
Table 629 – Start service parameters	1155
Table 630 – Stop service parameters	1156
Table 631 – Resume service parameters	1157
Table 632 – Reset service parameters	1158
Table 633 – Kill service parameters	1159
Table 634 – Program invocation state machine	1161
Table 635 – Class summary	1166
Table 636 – Services by AREP role	1167
Table 637 – QueryInterface	1176
Table 638 – AddRef	1177
Table 639 – Release	1178
Table 640 – GetTypeInfoCount	1179
Table 641 – GetTypeInfo	1180
Table 642 – GetIDsOfNames	1181
Table 643 – Invoke	1182
Table 644 – get_Producer	1186
Table 645 – get_Product	1187
Table 646 – get_SerialNo	1188
Table 647 – get_ProductionDate	1189
Table 648 – Revision	1190
Table 649 – get_LogicalDevice	1191
Table 650 – get_Count	1192
Table 651 – BrowseItems	1193
Table 652 – Save	1194
Table 653 – get_Name	1199
Table 654 – get_Producer	1200
Table 655 – get_Product	1201
Table 656 – get_SerialNo	1202
Table 657 – get_ProductionDate	1203
Table 658 – Revision	1204
Table 659 – get_ACCO	1205
Table 660 – get_RTAuto	1206
Table 661 – get_State	1207
Table 662 – Activate	1208
Table 663 – Deactivate	1209
Table 664 – Reset	1210
Table 665 – AdviseState	1211
Table 666 – UnadviseState	1212
Table 667 – get_Time	1213

Table 668 – put_Time	1214
Table 669 – get_Count	1215
Table 670 – BrowseItems	1216
Table 671 – GroupError	1217
Table 672 – AdviseGroupError.....	1218
Table 673 – UnadviseGroupError.....	1219
Table 674 – PingFactor values.....	1221
Table 675 – QoS Types and Values	1226
Table 676 – Quality Codes.....	1233
Table 677 – Quality Code priority table	1239
Table 678 – Error codes for the ACCO ASE detailed diagnosis	1252
Table 679 – AddConnections	1255
Table 680 – RemoveConnections.....	1257
Table 681 – ClearConnections	1258
Table 682 – SetActivationState	1259
Table 683 – GetInfo	1260
Table 684 – GetIDs.....	1261
Table 685 – GetConnections.....	1262
Table 686 – ReviseQoS	1263
Table 687 – get_PingFactor	1265
Table 688 – put_PingFactor	1266
Table 689 – get_CDBCookie.....	1267
Table 690 – Connect.....	1268
Table 691 – Disconnect	1269
Table 692 – DisconnectMe.....	1271
Table 693 – SetActivation	1272
Table 694 – Ping.....	1273
Table 695 – OnDataChanged.....	1274
Table 696 – Data Type Names for Values	1276
Table 697 – ReadItems	1277
Table 698 – WriteItems.....	1278
Table 699 – WriteItemsQCD	1279
Table 700 – GroupError	1280
Table 701 – AdviseGroupError.....	1281
Table 702 – UnadviseGroupError.....	1282
Table 703 – Allowed Data Types.....	1284
Table 704 – get_Name.....	1285
Table 705 – Revision	1286
Table 706 – get_Count	1287
Table 707 – BrowseItems	1288
Table 708 – CoCreateInstance.....	1290
Table 709 – Call	1291
Table 710 – Fieldbus AL class summary	1292
Table 711 – Assignment of the services to client and server	1292

INTERNATIONAL ELECTROTECHNICAL COMMISSION

**DIGITAL DATA COMMUNICATIONS FOR MEASUREMENT AND CONTROL –
FIELD BUS FOR USE IN INDUSTRIAL CONTROL SYSTEMS –****Part 5: Application Layer Service definition**

FOREWORD

- 1) The IEC (International Electrotechnical Commission) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of the IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, the IEC publishes International Standards. Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. The IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of the IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested National Committees.
- 3) The documents produced have the form of recommendations for international use and are published in the form of standards, technical specifications, technical reports or guides and they are accepted by the National Committees in that sense.
- 4) In order to promote international unification, IEC National Committees undertake to apply IEC International Standards transparently to the maximum extent possible in their national and regional standards. Any divergence between the IEC Standard and the corresponding national or regional standard shall be clearly indicated in the latter.
- 5) The IEC provides no marking procedure to indicate its approval and cannot be rendered responsible for any equipment declared to be in conformity with one of its standards.
- 6) Attention is drawn to the possibility that some of the elements of this International Standard may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 61158-5 has been prepared by subcommittee 65C: Digital communications, of IEC technical committee 65: Industrial-process measurement and control.

This third edition cancels and replaces the second edition published in 2000. This edition constitutes a technical revision.

The text of this standard is based on the following documents:

FDIS	Report on voting
65C/292/FDIS	65C/300/RVD

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This edition includes the following significant changes from the prior edition:

- a) updates to Types 1 through 5, 7 and 8;
- b) addition of Types 6, 9 and 10.

This publication has been drafted in accordance with ISO/IEC Directives, Part 2.

The committee has decided that the contents of this publication will remain unchanged until 2007. At this date, the publication will be

- reconfirmed;
- withdrawn;
- replaced by a revised edition, or
- amended.

IEC 61158 consists of the following parts, under the general title *Digital data communications for measurement and control – Fieldbus for use in industrial control systems*:

Part 1: Overview and guidance for the IEC 61158 series

Part 2: Physical Layer specification and service definition

Part 3: Data Link Service definition

Part 4: Data Link Protocol specification

Part 5: Application Layer Service definition

Part 6: Application Layer protocol specification

The contents of the corrigendum of July 2004 have been included in this copy.

0 Introduction

0.1 General

This part of IEC 61158 is one of a series produced to facilitate the interconnection of automation system components. It is related to other standards in the set as defined by the “three-layer” fieldbus Reference Model, which is based in part on the Basic Reference Model for Open Systems Interconnection. Both Reference Models subdivide the area of standardization for interconnection into a series of layers of specification, each of manageable size.

The Application Service is provided by the Application Protocol making use of the services available from the Data Link or other immediately lower layer. This part of the IEC 61158 series defines the Application Service characteristics that any immediately higher-level protocols may exploit. The relationship between the International Standards for fieldbus Application Service, fieldbus Application Protocol and Systems Management is illustrated in Figure 1.

NOTE Systems Management, as used in this standard, is a local mechanism for managing the layer protocols

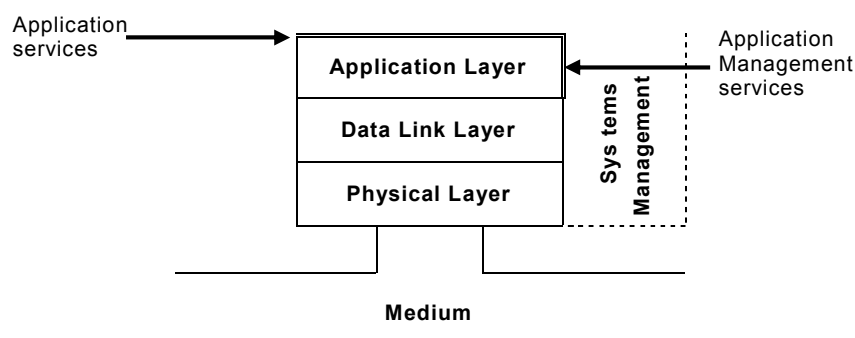


Figure 1 – Relationship of IEC 61158-3 to other fieldbus layers and to users of the fieldbus Application Service

This application layer standard does not specify individual implementations or products, nor does it constrain the implementations of application entities and interfaces within the industrial automation system.

This application layer standard does not contain test specifications used to demonstrate compliance with IEC 61158-5 and IEC 61158-6.

0.2 Nomenclature for references within this standard

Clauses, including annexes, can be referenced in their entirety, including any subordinate subclauses, as “clause N” or “Annex N”, where N is the number of the clause or letter of the annex.

Subclauses can be referenced in their entirety, including any subordinate subclauses, as “N.M” or “N.M.P” and so forth, depending on the level of the subclause, where N is the number of the subclause or letter of the annex, and M, P and so forth represent the successive levels of subclause up to and including the subclause of interest.

When a clause or subclause contains one or more subordinate subclauses, the text between the clause or subclause heading and its first subordinate subclause can be referenced in its entirety as “N.0” or “N.M.0” or “N.M.P.0” and so forth, where N, M and P are as above. Stated differently, a reference ending with “.0” designates the text and figures between a clause or subclause header and its first subordinate subclause.

DIGITAL DATA COMMUNICATIONS FOR MEASUREMENT AND CONTROL – FIELDBUS FOR USE IN INDUSTRIAL CONTROL SYSTEMS –

Part 5: Application Layer Service definition

1 Scope

The fieldbus Application Layer (FAL) provides user programs with a means to access the fieldbus communication environment. In this respect, the FAL can be viewed as a “window between corresponding application programs.”

The FAL is an Application Layer Communication Standard designed to support the conveyance of time-critical and non-time-critical application requests and responses among devices in an automation environment. The term “time-critical” is used to represent the presence of an application time-window, within which one or more specified actions are required to be completed with some defined level of certainty.

This standard specifies the structure and services of the IEC fieldbus Application Layer. It is specified in conformance with the OSI Basic Reference Model (ISO/IEC 7498) and the OSI Application Layer Structure (ISO/IEC 9545).

FAL services and protocols are provided by FAL application-entities (AE) contained within the application processes. The FAL AE is composed of a set of object-oriented Application Service Elements (ASEs) and a Layer Management Entity (LME) that manages the AE. The ASEs provide communication services that operate on a set of related application process object (APO) classes. One of the FAL ASEs is a management ASE that provides a common set of services for the management of the instances of FAL classes.

This part of IEC 61158 specifies interactions between remote applications in terms of

- an abstract model for defining application resources (objects) capable of being manipulated by users via the use of FAL Services,
- the primitives (interactions between the FAL and the FAL user) associated with each FAL Service;
- the parameters associated with each primitive;
- the interrelationship between and the valid sequences of the primitives for each service.

Several models of communications are specified in this document. Each model is specified as a communication “type”. Each type has its own separate clause in the document.

Although these services specify, from the perspective of applications, how request and responses are issued and delivered, they do not include a specification of what the requesting and responding applications are to do with them. That is, the behavioral aspects of the applications are not specified; only a definition of what requests and responses they can send/receive is specified. This permits greater flexibility to the FAL users in standardizing such object behavior. In addition to these services, some supporting services are also defined in this standard to provide access to the FAL to control certain aspects of its operation.

2 Normative references

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 61131, *Programmable controllers*

IEC 61131-1:1992, *Part 1: Programmable controllers — General information*

IEC 61131-3:1993, *Part 3: Programmable controllers — Programming languages*

IEC 61158, *Digital data communications for measurement and control – Fieldbus for use in industrial control systems*

IEC 61158-3:2003, *Digital data communications for measurement and control – Fieldbus for use in industrial control systems — Part 3: Data Link Layer service definition*

IEC 61158-4:2003, *Digital data communications for measurement and control – Fieldbus for use in industrial control systems — Part 4: Data Link Layer Protocol Specification*

IEC 61158-6:2003, *Digital data communications for measurement and control – Fieldbus for use in industrial control systems — Part 6: Application Layer Protocol Specification*

ISO/IEC 646:1991, *Information technology – ISO 7-bit coded character set for information interchange*

ISO/IEC 7498, *Information technology – Open Systems Interconnection – Basic Reference Model*

ISO/IEC 7498-1:1994, *Information technology – Open Systems Interconnection – Basic Reference Model — Part 1: The Basic Model*

ISO/IEC 7498-3:1997, *Information technology – Open Systems Interconnection – Basic Reference Model — Part 3: Naming and addressing*

ISO/IEC 8822:1994, *Information technology – Open Systems Interconnection – Presentation service definition*

ISO/IEC 8824:1990, *Information technology – Open Systems Interconnection – Specification of Abstract Syntax Notation One (ASN.1)*

ISO/IEC 8859-1:1998, *Information technology – 8-bit single-byte coded graphic character sets – Part 1: Latin alphabet No. 1*

ISO/IEC 9545:1994, *Information technology – Open Systems Interconnection – Application Layer structure*

ISO/IEC 10646-1:1993, *Information technology – Universal Multiple-Octet Coded Character Set (UCS) – Architecture and Basic Multilingual Plane*

ISO/IEC 10731:1994, *Information technology – Open Systems Interconnection – Basic Reference Model – Conventions for the definition of OSI services*

IEEE Std 754:1985 (R1990), *IEEE Standard for Binary Floating Point Arithmetic*

3 Terms and definitions

For the purposes of this document, the following terms as defined in these publications apply:

3.1 ISO/IEC 7498-1 terms

- a) application entity
- b) application process
- c) application protocol data unit
- d) application service element
- e) application entity invocation

- f) application process invocation
- g) application transaction
- h) real open system
- i) transfer syntax

3.2 ISO/IEC 8822 terms

For the purposes of this document, the following terms as defined in ISO/IEC 8822 apply:

- a) abstract syntax
- b) presentation context

3.3 ISO/IEC 9545 terms

For the purposes of this document, the following terms as defined in ISO/IEC 9545 apply:

- a) application-association
- b) application-context
- c) application context name
- d) application-entity-invocation
- e) application-entity-type
- f) application-process-invocation
- g) application-process-type
- h) application-service-element
- i) application control service element

3.4 ISO/IEC 8824 terms

For the purposes of this document, the following terms as defined in ISO/IEC 8824 apply:

- a) object identifier
- b) type

3.5 Fieldbus Data Link Layer terms

For the purposes of this document, the following terms as defined in IEC 61158-3 and IEC 61158-4 apply.

- a) DL-Time
- b) DL-Scheduling-policy
- c) DLCEP
- d) DLC
- e) DL-connection-oriented mode
- f) DLPDU
- g) DLSDU
- h) DLSAP
- i) fixed tag
- j) generic tag
- k) link
- l) MAC ID
- m) network address

- n) node address
- o) node
- p) tag
- q) scheduled
- r) unscheduled

3.6 Fieldbus Application Layer specific definitions

For the purposes of this part of IEC 61158, the following terms and definitions apply.

3.6.1 access protection

limitation of the usage of an application object to one client

3.6.2 active connection control object

instance of a certain FAL class that abstracts the interconnection facility (as Consumer and Provider) of an automation device

3.6.3 address assignment table

mapping of the client's internal I/O-Data object storage to the decentralised input and output data objects

3.6.4 allocate

take a resource from a common area and assign that resource for the exclusive use of a specific entity

3.6.5 application

function or data structure for which data is consumed or produced

3.6.6 application layer interoperability

capability of application entities to perform coordinated and cooperative operations using the services of the FAL

3.6.7 application objects

multiple object classes that manage and provide a run time exchange of messages across the network and within the network device

3.6.8 application process

part of a distributed application on a network, which is located on one device and unambiguously addressed

3.6.9 application process identifier

distinguishes multiple application processes used in a device

3.6.10 application process object

component of an application process that is identifiable and accessible through an FAL application relationship

NOTE Application process object definitions are composed of a set of values for the attributes of their class (see the definition for Application Process Object Class Definition). Application process object definitions may be accessed remotely using the services of the FAL Object Management ASE. FAL Object Management services can be used to load or update object definitions, to read object definitions, and to dynamically create and delete application objects and their corresponding definitions.

3.6.11 application process object class

a class of application process objects defined in terms of the set of their network-accessible attributes and services

3.6.12 application relationship

cooperative association between two or more application-entity-invocations for the purpose of exchange of information and coordination of their joint operation. This relationship is activated either by the exchange of application-protocol-data-units or as a result of preconfiguration activities

3.6.13 application relationship application service element

application-service-element that provides the exclusive means for establishing and terminating all application relationships

3.6.14 application relationship endpoint

context and behavior of an application relationship as seen and maintained by one of the application processes involved in the application relationship

NOTE Each application process involved in the application relationship maintains its own application relationship endpoint.

3.6.15 attribute

description of an externally visible characteristic or feature of an object

NOTE The attributes of an object contain information about variable portions of an object. Typically, they provide status information or govern the operation of an object. Attributes may also affect the behaviour of an object. Attributes are divided into class attributes and instance attributes.

3.6.16 behaviour

indication of how an object responds to particular events

3.6.17 bit-no

designates the number of a bit in a bitstring or an octet

3.6.18 channel

single physical or logical link of an input or output application object of a server to the process

3.6.19 channel related diagnosis

information concerning a specific element of an input or output application object, provided for maintenance purposes

EXAMPLE: validity of data

3.6.20 class

a set of objects, all of which represent the same kind of system component

NOTE A class is a generalisation of an object; a template for defining variables and methods. All objects in a class are identical in form and behaviour, but usually contain different data in their attributes.

3.6.21 class attributes

attribute that is shared by all objects within the same class

3.6.22 class code

unique identifier assigned to each object class

3.6.23 class specific service

service defined by a particular object class to perform a required function which is not performed by a common service

NOTE A class specific object is unique to the object class which defines it.

3.6.24 client

- a) object which uses the services of another (server) object to perform a task
- b) initiator of a message to which a server reacts

3.6.25 communication objects

components that manage and provide a run time exchange of messages across the network

EXAMPLES: Connection Manager object, Unconnected Message Manager (UCMM) object, and Message Router object

3.6.26 configuration check

comparison of the expected I/O-Data object structuring of the client with the real I/O-Data object structuring to the server in the start-up phase

3.6.27 configuration data base

interconnection information maintained by the ACCO ASE

3.6.28 configuration fault

an unacceptable difference between the expected I/O-Data object structuring and the real I/O-Data object structuring, as detected by the server

3.6.29 configuration identifier

representation of a portion of I/O Data of a single input- and/or output-module of a server

3.6.30 connection

logical binding between application objects that may be within the same or different devices

NOTE 1 Connections may be either point-to-point or multipoint.

NOTE 2 The logical link between sink and source of attributes and services at different custom interfaces of RT-Auto ASEs is referred to as interconnection. There is a distinction between data and event interconnections. The logical link and the data flow between sink and source of automation data items is referred to as data interconnection. The logical link and the data flow between sink (method) and source (event) of operational services is referred to as event interconnection.

3.6.31 connection channel

description of a connection between a sink and a source of data items

3.6.32 connection ID (CID)

identifier assigned to a transmission that is associated with a particular connection between producers and consumers, providing a name for a specific piece of application information

3.6.33 connection path

an octet stream that defines the application object to which a connection instance applies

3.6.34 connection point

buffer which is represented as a subinstance of an Assembly object

3.6.35 consume

act of receiving data from a producer

3.6.36 consumer

node or sink that is receiving data from a producer

3.6.37 consuming application

application that consumes data

3.6.38 consumerID

unambiguous identifier within the scope of the ACCO assigned by the consumer to recognize the internal data of a configured interconnection sink

3.6.39 control commands

action invocations transferred from client to server to clear outputs, freeze inputs and/or synchronise outputs

3.6.40 conveyance path

unidirectional flow of APDUs across an application relationship

3.6.41 cyclic

repetitive in a regular manner

3.6.42 data consistency

means for coherent transmission and access of the input- or output-data object between and within client and server

3.6.43 data marshaling

the encoding of parameters of the FAL service primitives with respect to their interface definition

<Type 10> NOTE This is part of the abstract ORPC model.

3.6.44 dedicated AR

AR used directly by the FAL User

NOTE On Dedicated ARs, only the FAL Header and the user data are transferred.

3.6.45 default DL-address

value 126 as an initial value for DL-address, which has to be changed (e.g. by assignment of an DL-address via the fieldbus) before operation with a DP-master (class 1)

3.6.46 device

physical hardware connected to the link

NOTE A device may contain more than one node.

3.6.47 device profile

a collection of device dependent information and functionality providing consistency between similar devices of the same device type

3.6.48 diagnosis information

all data available at the server for maintenance purposes

3.6.49 diagnosis information collection

system diagnosis information that is assembled at the client side

3.6.50 DP-master (class 1)

a controlling device which controls several DP-slaves (field devices)

NOTE This is usually a programmable controller or a distributed control system.

3.6.51 DP-master (class 2)

controlling device which manages configuration data (parameter sets) and diagnosis data of a DP-master (Class 1), and that additionally can perform all communication capabilities of a DP-master (Class 1)

3.6.52 DP-slave

field device that can be assigned to one DP-master (Class 1) as a provider for cyclic I/O data exchange; in addition acyclic functions and alarms could be provided

3.6.53 dynamic AR

AR that requires the use of the AR establishment procedures to place it into an established state

3.6.54 end node

producing or consuming node

3.6.55 endpoint

one of the communicating entities involved in a connection

3.6.56 engineering

abstract term that characterizes the client application or device responsible for configuring an automation system via interconnecting data items

3.6.57 error

discrepancy between a computed, observed or measured value or condition and the specified or theoretically correct value or condition

3.6.58 error class

general grouping for related error definitions and corresponding error codes

3.6.59 error code

identification of a specific type of error within an error class

3.6.60 event

an instance of a change of conditions

3.6.61 FAL subnet

subnetworks composed of one or more data link segments, identified by a subset of the network address

NOTE FAL subnets are permitted to contain bridges but not routers.

3.6.62 FIFO variable

a Variable Object class, composed of a set of homogeneously typed elements, where the first written element is the first element that can be read

NOTE On the fieldbus only one, complete element can be transferred as a result of one service invocation.

3.6.63 frame

denigrated synonym for DLPDU

3.6.64 freeze

function at the DP-slaves for simultaneous data transfer between the input data object and the process

3.6.65 group

- a) <general> a general term for a collection of objects. Specific uses:
- b) <addressing> when describing an address, an address that identifies more than one entity
- c) <Type 3> set of DP-slaves which perform a Freeze or Sync function

3.6.66 interface

- (a) shared boundary between two functional units, defined by functional characteristics, signal characteristics, or other characteristics as appropriate
- (b) collection of FAL class attributes and services that represents a specific view on the FAL class

3.6.67 interface definition language

syntax and semantics of describing service parameters in a formal way

NOTE This description is the input for the ORPC model, especially for the ORPC wire protocol.

3.6.68 interface pointer

key attribute that unambiguously addresses an object interface instance

3.6.69 invocation

act of using a service or other resource of an application process

NOTE Each invocation represents a separate thread of control that may be described by its context. Once the service completes, or use of the resource is released, the invocation ceases to exist. For service invocations, a service that has been initiated but not yet completed is referred to as an outstanding service invocation. Also for service invocations, an Invoke ID may be used to unambiguously identify the service invocation and differentiate it from other outstanding service invocations.

3.6.70 I/O data

object designated to be transferred cyclically for the purpose of processing

3.6.71 ident number

DP-master (Class 1) or DP-slave device type

3.6.72 identifier related diagnosis

information dedicated to modules for maintenance purpose

3.6.73 index

address of an object within an application process

3.6.74 instance

the actual physical occurrence of an object within a class that identifies one of many objects within the same object class

EXAMPLE Clifornia is an instance of the object class state.

NOTE The terms object, instance, and object instance are used to refer to a specific instance.

3.6.75 instance attributes

attribute that is unique to an object instance and not shared by the object class

3.6.76 instantiated

object that has been created in a device

3.6.77 logical device

a certain FAL class that abstracts a software component or a firmware component as an autonomous self-contained facility of an automation device

3.6.78 Lpacket (or Link packet)

a piece of application information that contains a size, control byte, tag, and link data

NOTE Peer Data Link Layers use Lpackets to send and receive service data units from higher layers in the OSI stack.

3.6.79 manufacturer ID

identification of each product manufacturer by a unique number

3.6.80 management information

network-accessible information that supports managing the operation of the fieldbus system, including the application layer

NOTE Managing includes functions such as controlling, monitoring, and diagnosing.

3.6.81 master parameter set

the configuration and parameterisation data of all DP-slaves that are assigned to the corresponding DP-master and the bus parameters

3.6.82 member

piece of an attribute that is structured as an element of an array

3.6.83 message router

object within a node that distributes messaging requests to appropriate application objects

3.6.84 method

<object> a synonym for an operational service which is provided by the server ASE and invoked by a client

3.6.85 module

- a) <general> hardware or logical component of a physical device
- b) <Type 3> addressable unit inside the DP-slave

3.6.86 multipoint connection

connection from one node to many

NOTE Multipoint connections allow messages from a single producer to be received by many consumer nodes.

3.6.87 network

a set of nodes connected by some type of communication medium, including any intervening repeaters, bridges, routers and lower-layer gateways

3.6.88 object

abstract representation of a particular component within a device, usually a collection of related data (in the form of variables) and methods (procedures) for operating on that data that have clearly defined interface and behaviour

3.6.89 object remote procedure call

model for object oriented or component based remote method invocation

3.6.90 object specific service

service unique to the object class which defines it

3.6.91 originator

client responsible for establishing a connection path to the target

3.6.92 peer

role of an AR endpoint in which it is capable of acting as both client and server

3.6.93 physical device

<general> an automation or other network device

<Type10> a certain FAL class that abstracts the hardware facilities of an automation device

3.6.94 point-to-point connection

connection that exists between exactly two application objects

3.6.95 pre-defined AR endpoint

AR endpoint that is defined locally within a device without use of the create service

NOTE Pre-defined ARs that are not pre-established are established before being used

3.6.96 pre-established AR endpoint

AR endpoint that is placed in an established state during configuration of the AEs that control its endpoints

3.6.97 process data

object(s) which are already pre-processed and transferred acyclically for the purpose of information or further processing

3.6.98 produce

act of sending data to be received by a consumer

3.6.99 producer

node that is responsible for sending data

3.6.100 property

- a) <general> a general term for descriptive information about an object
- b) <Type 10> a synonym for ASE attributes which are readable or writeable via operational ASE services

NOTE These services are generally named "get_<Attribute Name>" or "set_<Attribute Name>" and correspond with the IDL keywords "propget" and "propput" used in Type 10 clauses of IEC 61158-6.

3.6.101 provider

source of a data connection

3.6.102 providerID

an unambiguous identifier within the scope of the ACCO assigned by the provider to recognize the internal data of a configured interconnection source

3.6.103 publisher

role of an AR endpoint that transmits APDUs onto the fieldbus for consumption by one or more subscribers

NOTE A publisher may not be aware of the identity or the number of subscribers and it may publish its APDUs using a dedicated AR.

3.6.104 publishing manager

role of an AR endpoint in which it issues one or more confirmed service request APDUs to a publisher to request the publisher to publish a specified object. Two types of publishing managers are defined by this standard, pull publishing managers and push publishing managers, each of which is defined separately

3.6.105 pull publisher

type of publisher that publishes an object in response to a request received from its pull publishing manager

3.6.106 pull publishing manager

type of publishing manager that requests that a specified object be published in a corresponding response APDU

3.6.107 push publisher

type of publisher that publishes an object in an unconfirmed service request APDU

3.6.108 push publishing manager

type of publishing manager that requests that a specified object be published using an unconfirmed service

3.6.109 pull subscriber

type of subscriber that recognizes received confirmed service response APDUs as published object data

3.6.110 push subscriber

type of subscriber that recognizes received unconfirmed service request APDUs as published object data

3.6.111 quality code aware

attribute of the RT-Auto class that indicates that an RT-Auto object uses a status code for its data items

3.6.112 quality code

additional status information of a data item

3.6.113 quality code unaware

opposite of quality code aware

3.6.114 real configuration

input and output data structure of the DP-slave, including definition of data consistency

3.6.115 resource

a processing or information capability of a subsystem

3.6.116 route endpoint

object container containing Variable Objects of a variable class

3.6.117 RT-auto

an FAL class that abstracts the automation function as a process-related component of an automation device

3.6.118 runtime object model

objects that exist in a device together with their interfaces and methods that are accessible

3.6.119 serial number

<Type 2> a unique 32-bit integer value assigned by each manufacturer to every device having Type 2 communication capabilities

NOTE The Manufacturer ID and serial number jointly form a unique identifier for each device.

3.6.120 server

- a) role of an AREP in which it returns a confirmed service response APDU to the client that initiated the request
- b) object which provides services to another (client) object

3.6.121 service

operation or function than an object and/or object class performs upon request from another object and/or object class

3.6.122 slot

address of a module within a DP-slave

3.6.123 subscriber

role of an AREP in which it receives APDUs produced by a publisher

3.6.124 sync

function at the DP-slaves for simultaneous data transfer between the output data object and the process

3.6.125 target

end-node to which a connection is established

3.6.126 unconnected message manager (UCMM)

component within a node that transmits and receives unconnected explicit messages and sends them directly to the Message Router object

3.6.127 unconnected service

messaging service which does not rely on the set up of a connection between devices before allowing information exchanges

3.7 Abbreviations and symbols

ACCO	Active Connection Control Object
AE	Application Entity
AL	Application Layer
ALME	Application Layer Management Entity
ALP	Application Layer Protocol
APO	Application Object
AP	Application Process
APDU	Application Protocol Data Unit
API	Application Process Identifier
AR	Application Relationship
AREP	Application Relationship End Point
ASCII	American Standard Code for Information Interchange
ASE	Application Service Element
CID	Connection ID
CIM	Computer Integrated Manufacturing
CIP	Control and Information Protocol
CM_API	Actual Packet Interval
CM_RPI	Requested Packet Interval
Cnf	Confirmation
COR	Connection originator
CR	Communication Relationship
CREP	Communication Relationship End Point
DL-	(as a prefix) Data Link-
DLC	Data Link Connection
DLCEP	Data Link Connection End Point
DLL	Data Link Layer
DLM	Data Link-management
DLSAP	Data Link Service Access Point
DLSDU	DL-service-data-unit
DNS	Domain Name Service
DP	Decentralised Peripherals
FAL	Fieldbus Application Layer
FIFO	First In First Out
HMI	Human-Machine Interface
ID	Identifier
IDL	Interface Definition Language
IEC	International Electrotechnical Commission
Ind	Indication
IP	Internet Protocol
ISO	International Organization for Standardization
LDev	Logical Device

LME	Layer Management Entity
O2T	Originator to target (connection characteristics)
O⇒T	Originator to target (connection characteristics)
ORPC	Object Remote Procedure Call
OSI	Open Systems Interconnect
PDev	Physical Device
PDU	Protocol Data Unit
PL	Physical Layer
QoS	Quality of Service
QC	Quality Code
REP	Route Endpoint
Req	Request
Rsp	Response
RT	Runtime
SAP	Service Access Point
SCL	Security Level
SDU	Service Data Unit
SEM	State event matrix
SMIB	System Management Information Base
SMK	System Management Kernel
STD	State transition diagram, used to describe object behaviour
S-VFD	Simple Virtual Field Device
T2O	Target to originator (connection characteristics)
T⇒O	Target to originator (connection characteristics)
VAO	Variable Object

3.8 Conventions

3.8.1 Overview

The FAL is defined as a set of object-oriented ASEs. Each ASE is specified in a separate subclause. Each ASE specification is composed of two parts, its class specification, and its service specification.

The class specification defines the attributes of the class. The attributes are accessible from instances of the class using the Object Management ASE services specified in Clause 5 of this standard. The service specification defines the services that are provided by the ASE.

3.8.2 General conventions

This standard uses the descriptive conventions given in ISO/IEC 10731

3.8.3 Conventions for class definitions

Class definitions are described using templates. Each template consists of a list of attributes for the class. The general form of the template is shown below:

FAL ASE:	ASE Name
CLASS:	Class Name
CLASS ID:	#

PARENT CLASS: Parent Class Name

ATTRIBUTES:

1	(o)	Key Attribute:	numeric identifier
2	(o)	Key Attribute:	name
3	(m)	Attribute:	attribute name(values)
4	(m)	Attribute:	attribute name(values)
4.1	(s)	Attribute:	attribute name(values)
4.2	(s)	Attribute:	attribute name(values)
4.3	(s)	Attribute:	attribute name(values)
5.	(c)	Constraint:	constraint expression
5.1	(m)	Attribute:	attribute name(values)
5.2	(o)	Attribute:	attribute name(values)
6	(m)	Attribute:	attribute name(values)
6.1	(s)	Attribute:	attribute name(values)
6.2	(s)	Attribute:	attribute name(values)

SERVICES:

1	(o)	OpsService:	service name
2.	(c)	Constraint:	constraint expression
2.1	(o)	OpsService:	service name
3	(m)	MgtService:	service name

- (1) The "FAL ASE:" entry is the name of the FAL ASE that provides the services for the class being specified.
- (2) The "CLASS:" entry is the name of the class being specified. All objects defined using this template will be an instance of this class. The class may be specified by this standard, or by a user of this standard.
- (3) The "CLASS ID:" entry is a number that identifies the class being specified. This number is unique within the FAL ASE that will provide the services for this class. When qualified by the identity of its FAL ASE, it unambiguously identifies the class within the scope of the FAL. The value "NULL" indicates that the class cannot be instantiated. Class IDs between 1 and 255 are reserved by this standard to identify standardized classes. They have been assigned to maintain compatibility with existing national standards. CLASS IDs between 256 and 2048 are allocated for identifying user defined classes.
- (4) The "PARENT CLASS:" entry is the name of the parent class for the class being specified. All attributes defined for the parent class and inherited by it are inherited for the class being defined, and therefore do not have to be redefined in the template for this class.

NOTE The parent-class "TOP" indicates that the class being defined is an initial class definition. The parent class TOP is used as a starting point from which all other classes are defined. The use of TOP is reserved for classes defined by this standard.

- (5) The "ATTRIBUTES" label indicate that the following entries are attributes defined for the class.
 - a) Each of the attribute entries contains a line number in column 1, a mandatory (m) / optional (o) / conditional (c) / selector (s) indicator in column 2, an attribute type label in column 3, a name or a conditional expression in column 4, and optionally a list of enumerated values in column 5. In the column following the list of values, the default value for the attribute may be specified.
 - b) Objects are normally identified by a numeric identifier or by an object name, or by both. In the class templates, these key attributes are defined under the key attribute.

- c) The line number defines the sequence and the level of nesting of the line. Each nesting level is identified by period. Nesting is used to specify
 - i) fields of a structured attribute (4.1, 4.2, 4.3),
 - ii) attributes conditional on a constraint statement (5). Attributes may be mandatory (5.1) or optional (5.2) if the constraint is true. Not all optional attributes require constraint statements as does the attribute defined in (5.2).
 - iii) the selection fields of a choice type attribute (6.1 and 6.2).
- (6) The "SERVICES" label indicates that the following entries are services defined for the class.
 - a) An (m) in column 2 indicates that the service is mandatory for the class, while an (o) indicates that it is optional. A (c) in this column indicates that the service is conditional. When all services defined for a class are defined as optional, at least one has to be selected when an instance of the class is defined.
 - b) The label "OpsService" designates an operational service (1).
 - c) The label "MgtService" designates an management service (2).
 - d) The line number defines the sequence and the level of nesting of the line. Each nesting level is identified by period. Nesting within the list of services is used to specify services conditional on a constraint statement.

3.8.4 Conventions for service definitions

3.8.4.1 General

The service model, service primitives, and time-sequence diagrams used are entirely abstract descriptions; they do not represent a specification for implementation.

3.8.4.2 Service parameters

Service primitives are used to represent service user/service provider interactions (ISO/IEC 10731). They convey parameters which indicate information available in the user/provider interaction. In any particular interface, not all parameters need be explicitly stated.

The service specifications of this standard uses a tabular format to describe the component parameters of the ASE service primitives. The parameters which apply to each group of service primitives are set out in tables. Each table consists of up to five columns for the

- 1) Parameter name,
- 2) request primitive,
- 3) indication primitive,
- 4) response primitive, and
- 5) confirm primitive.

One parameter (or component of it) is listed in each row of each table. Under the appropriate service primitive columns, a code is used to specify the type of usage of the parameter on the primitive specified in the column:

- M parameter is mandatory for the primitive
- U parameter is a User option, and may or may not be provided depending on dynamic usage of the service user. When not provided, a default value for the parameter is assumed.
- C parameter is conditional upon other parameters or upon the environment of the service user.

— (blank) parameter is never present.

S parameter is a selected item.

Some entries are further qualified by items in brackets. These may be

- a) a parameter-specific constraint:
“(=)” indicates that the parameter is semantically equivalent to the parameter in the service primitive to its immediate left in the table.
- b) an indication that some note applies to the entry:
“(n)” indicates that the following note "n" contains additional information pertaining to the parameter and its use.

3.8.4.3 Service procedures

The procedures are defined in terms of

- the interactions between application entities through the exchange of fieldbus Application Protocol Data Units, and
- the interactions between an application layer service provider and an application layer service user in the same system through the invocation of application layer service primitives.

These procedures are applicable to instances of communication between systems which support time-constrained communications services within the fieldbus Application Layer.

4 Concepts

4.1 Overview

The fieldbus is intended to be used in factories and process plants to interconnect primary automation devices (e.g. sensors, actuators, local display devices, annunciators, programmable logic controllers, small single loop controllers, and stand-alone field controls) with control and monitoring equipment located in control rooms.

Primary automation devices are associated with the lowest levels of the industrial automation hierarchy and perform a limited set of functions within a definite time window. Some of these functions include diagnostics, data validation, and handling of multiple inputs and outputs.

These primary automation devices, also termed field devices, are located close to the process fluids, the fabricated part, the machine, the operator and the environment. This use positions the fieldbus at the lowest levels of the Computer Integrated Manufacturing (CIM) architecture.

Some of the expected benefits in using fieldbus are reduction in wiring, increase in amount of data exchanged, wider distribution of control between the primary automation devices and the control room equipment, and the satisfaction of time critical constraints.

This subclause describes fundamentals of the FAL. Detailed descriptive information about each of the FAL ASEs can be found in the “Overview” subclause of each of the Communication Model specifications.

4.2 Architectural relationships

4.2.1 Relationship to the Application Layer of the OSI basic reference model

The functions of the FAL have been described according to OSI layering principles. However, its architectural relationship to the lower layers is different, as shown in Figure 2.

- The FAL includes OSI functions together with extensions to cover time-critical requirements. The OSI Application Layer Structure standard (ISO/IEC 9545) was used as a basis for specifying the FAL.
- The FAL directly uses the services of the underlying layer. The underlying layer may be the data link layer or any layer in between. When using the underlying layer, the FAL may provide functions normally associated with the OSI Middle Layers for proper mapping onto the underlying layer.

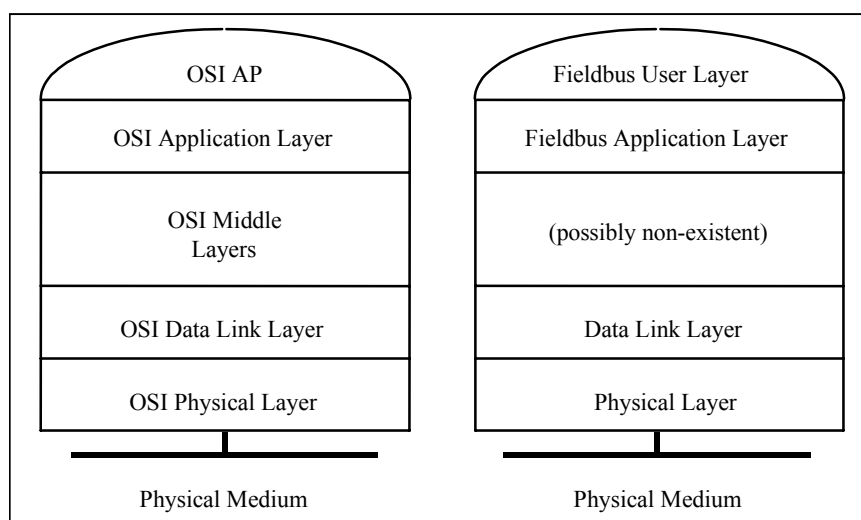


Figure 2 – Relationship to the OSI basic reference model

4.2.2 Relationships to other fieldbus entities

4.2.2.1 General

The fieldbus Application Layer (FAL) architectural relationships, as illustrated in Figure 3, have been designed to support the interoperability needs of time-critical systems distributed within the fieldbus environment.

Within this environment, the FAL provides communications services to time-critical and non-time-critical applications located in fieldbus devices.

In addition, the FAL directly uses the Data Link Layer to transfer its application layer protocol data units. It does this using a set of data transfer services and a set of supporting services used to control the operational aspects of the Data Link Layer.

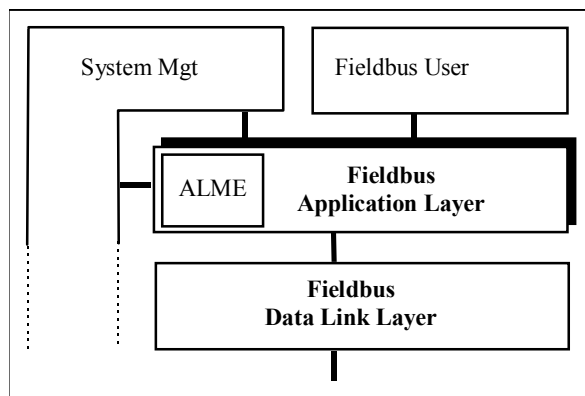


Figure 3 – Architectural positioning of the fieldbus Application Layer

4.2.2.2 Use of the fieldbus Data Link Layer

The fieldbus Application Layer (FAL) provides network access to fieldbus APs. It interfaces directly to the fieldbus Data Link Layer for transfer of its APDUs.

The Data Link Layer provides various types of services to the FAL for the transfer of data between Data Link endpoints (e.g. DLSAPs, DLCEPs).

4.2.2.3 Support for fieldbus applications

Fieldbus applications are represented to the network as application processes (APs). APs are the components of a distributed system that may be individually identified and addressed.

Each AP contains an FAL application entity (AE) that provides network access for the AP. That is, each AP communicates with other APs through its AE. In this sense, the AE provides a window of visibility into the AP.

APs contain identifiable components that are also visible across the network. These components are represented to the network as Application Process Objects (APO). They may be identified by one or more key attributes. They are located at the address of the application process that contains them.

The services used to access them are provided by APO-specific application service elements (ASEs) contained within the FAL. These ASEs are designed to support user, function block, and management applications.

4.2.2.4 Support for system management

The FAL services can be used to support various management operations, including management of fieldbus systems, applications, and the fieldbus network.

4.2.2.5 Access to FAL layer management entities

One layer management entity (LME) may be present in each FAL entity on the network. FALMEs provide access to the FAL for system management purposes.

The set of data accessible by the System Manager is referred to as the System Management Information Base (SMIB). Each fieldbus Application Layer Management Entity (FALME) provides the FAL portion of the SMIB. How the SMIB is implemented is beyond the scope of this standard.

4.3 Fieldbus Application Layer structure

4.3.1 Overview

The structure of the FAL is a refinement of the OSI Application Layer Structure (ISO/IEC 9545). As a result, the organization of this subclause is similar to that of ISO/IEC 9545. Certain concepts presented here have been refined from ISO/IEC 9545 for the fieldbus environment.

The FAL differs from the other layers of OSI in two principal aspects:

- OSI defines a single type of application layer communications channel, the association, to connect APs to each other. The FAL defines the Application Relationship (AR), of which there are several types, to permit application processes (APs) to communicate with each other.
- The FAL uses the DLL to transfer its PDUs and not the presentation layer. Therefore, there is no explicit presentation context available to the FAL. Between the same pair (or set) of data link service access points the FAL protocol may not be used concurrently with other application layer protocols.

4.3.2 Fundamental concepts

The operation of time critical real open systems is modeled in terms of interactions between time-critical APs. The FAL permits these APs to pass commands and data between them.

Cooperation between APs requires that they share sufficient information to interact and carry out processing activities in a coordinated manner. Their activities may be restricted to a single fieldbus segment, or they may span multiple segments. The FAL has been designed using a modular architecture to support the messaging requirements of these applications.

Cooperation between APs also sometimes requires that they share a common sense of time. The FAL or the Data Link Layer (IEC 61158-3 and IEC 61158-4) may provide for the distribution of time to all devices. They also may define local device services that can be used by APs to access the distributed time.

The remainder of this subclause describes each of the modular components of the architecture and their relationships with each other. The components of the FAL are modeled as objects, each of which provides a set of FAL communication services for use by applications. The FAL objects and their relationships are described below. The detailed specifications of FAL objects and their services are provided in the following clauses of this standard. IEC 61158-6 specifies the protocols necessary to convey these object services between applications.

4.3.3 Fieldbus application processes

4.3.3.1 Definition of the fieldbus AP

In the fieldbus environment, an application may be partitioned into a set of components and distributed across a number of devices on the network. Each of these components is referred to as a fieldbus Application Process (AP). A fieldbus AP is a variation of an Application Process as defined in ISO OSI Reference Model (ISO/IEC 7498). Fieldbus APs may be unambiguously addressed by at least one individual Data Link Layer service access point address. Unambiguously addressed, in this context, means that no other AP may simultaneously be located by the same address. This definition does not prohibit an AP from being located by more than one individual or group data link service access point address.

4.3.3.2 Communication services

Fieldbus APs communicate with each other using confirmed and unconfirmed services (ISO/IEC 10731). The services defined in this standard for the FAL specify the semantics of

the services as seen by the requesting and responding APs. The syntax of the messages used to convey the service requests and responses is defined in IEC 61158-6. The AP behavior associated with the services is specified by the AP.

Confirmed services are used to define request/response exchanges between APs.

Unconfirmed services, in contrast, are used to define the unidirectional transfer of messages from one AP to one or more remote APs. From a communications perspective, there is no relationship between separate invocations of unconfirmed services as there is between the request and response of a confirmed service.

4.3.3.3 AP interactions

4.3.3.3.1 General

Within the fieldbus environment, APs may interact with other APs as necessary to achieve their functional objectives. No constraints are imposed by this standard on the organization of these interactions or the possible relationships that may exist between them.

For example, in the fieldbus environment, interactions may be based on request/response messages sent directly between APs, or on data/events sent by one AP for use by others. These two models of interactions between APs are referred to as client/server and publisher/subscriber interactions.

The services supported by an interaction model are conveyed by application relationship endpoints (AREPs) associated with the communicating APs. The role that the AREP plays in the interaction (e.g. client, server, peer, publisher, subscriber) is defined as an attribute of the AREP.

4.3.3.3.2 Client/server interactions

Client/server interactions are characterized by a bi-directional data flow between a client AP and one or more server APs. Figure 4 illustrates the interaction between a single client and a single server. In this type of interaction, the client may issue a confirmed or unconfirmed request to the server to perform some task. If the service is confirmed then the server will always return a response. If the service is unconfirmed, the server may return a response using an unconfirmed service defined for this purpose.

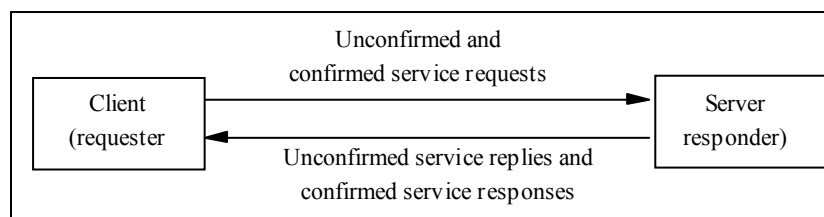


Figure 4 – Client/server interactions

4.3.3.3.3 Publisher/subscriber interactions

4.3.3.3.3.1 General

Publisher/subscriber interactions, on the other hand, involve a single publisher AP, and a group of one or more subscriber APs. This type of interaction has been defined to support variations of two models of interaction between APs, the "pull" model and the "push" model. In both models, the setup of the publishing AP is performed by management and is outside the scope of this standard.

4.3.3.3.2 Pull model interactions

In the "pull" model, the publisher receives a request to publish from a remote publishing *manager*, and broadcasts (or multicasts) its response across the network. The publishing manager is responsible only for initiating publishing by sending a request to the publisher.

Subscribers wishing to receive the published data listen for responses transmitted by the publisher. In this fashion, data is "pulled" from the publisher by requests from the publishing manager.

Confirmed FAL services are used to support this type of interaction. Two characteristics of this type of interaction differentiate it from the other types of interaction. First, a typical confirmed request/response exchange is performed between publishing manager and the publisher. However, the underlying conveyance mechanism provided by the FAL returns the response not just to the publishing manager, but also to all subscribers wishing to receive the published information. This is accomplished by having the Data Link Layer transmit the response to a group address, rather than to the individual address of the publishing manager. Therefore, the response sent by the publisher contains the published data and is multicast to the publishing manager and to all subscribers.

The second difference occurs in the behavior of the subscribers. Pull model subscribers, referred to as pull subscribers, are capable of accepting published data in confirmed service responses without having issued the corresponding request. Figure 5 illustrates these concepts.

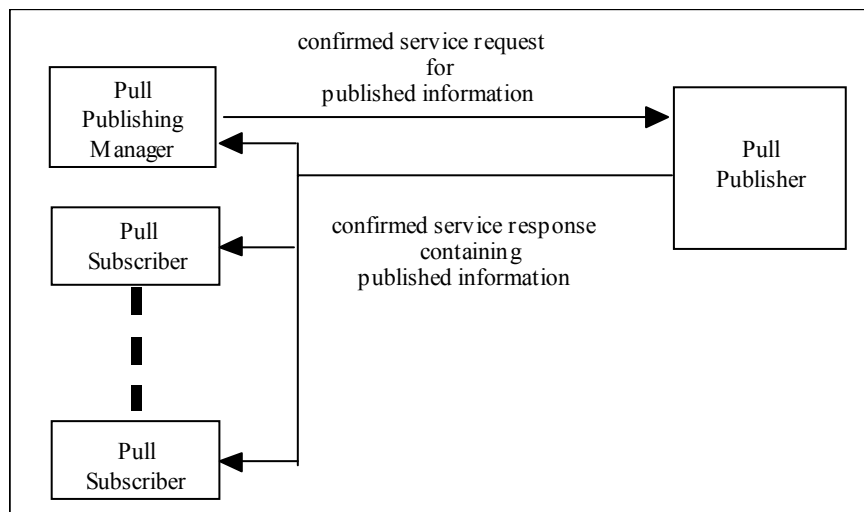


Figure 5 – Pull model interactions

4.3.3.3.3 Push model interactions

In the "push" model, two services may be used, one confirmed and one unconfirmed. The confirmed service is used by the subscriber to request to join the publishing. The response to this request is returned to the subscriber, following the client/server model of interaction. This exchange is only necessary when the subscriber and the publisher are located in different APs.

The unconfirmed service used in the Push Model is used by the publisher to distribute its information to subscribers. In this case, the publisher is responsible for invoking the correct unconfirmed service at the appropriate time and for supplying the appropriate information. In this fashion, it is configured to "push" its data onto the network.

Subscribers for the Push Model receive the published unconfirmed services distributed by publishers. Figure 6 illustrates the concept of the Push Model.

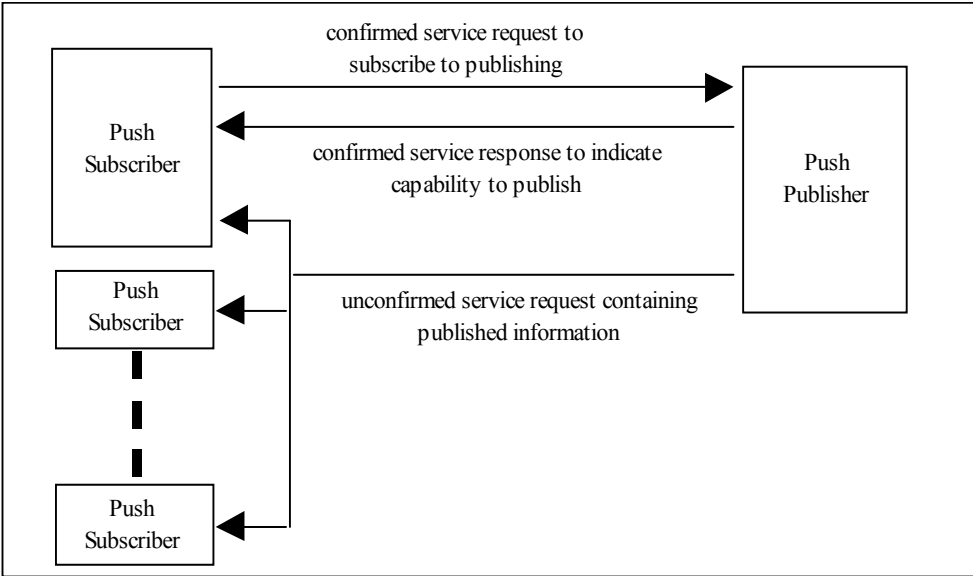


Figure 6 – Push model interactions

4.3.3.3.4 Timeliness of published information

To support the perishable nature of published information, the FAL may support four types of timeliness defined for publisher/subscriber interactions. Each make it possible for subscribers of published data to determine if the data they are receiving is up-to-date or “stale”. These types are realized through mechanisms within the Data Link Layer (DLL). Each is described briefly below. For a more detailed description, refer to IEC 61158-3 and IEC 61158-4.

Type	Description
Transparent	This type of timeliness allows the user application process to determine the timeliness quality of data that it generates and have the timeliness quality accompany the information when it is transferred across the network. In this type of timeliness, the network provides no computation or measurement of timeliness. It merely conveys the timeliness quality provided with the data by the user application process.
Residence	When the FAL submits data from the publishing AP to the DLL for transmission, the DLL starts a timer. If the timer expires before the data has been transmitted, the DLL marks the buffer as “not timely” and conveys this timeliness information with the data.
Synchronized	This type of timeliness requires the coordination of two pieces of published information. One is the data to be published and the other is a special “sync mark”. When the sync mark is received from the network a timer starts in each of the participating stations. Subsequently, when data is received for transmission by the DLL at the publishing station, or when the transmitted data is received from the network at a subscribing station, the DLL timeliness attribute for the data is set to TRUE. It remains TRUE until the reception of the next sync mark or until the timer expires. Data received after the timer expires but before the next sync mark does not cause the timeliness attribute to be reset to TRUE. It is only reset to TRUE if data is received within the time window after receipt of the sync mark. Data transmitted by the publisher station with the timeliness attribute set to FALSE maintains the setting of FALSE at each of the subscribers, regardless of their timer operation.
Update	This type of timeliness requires the coordination of the same two pieces of published information defined for <i>synchronized</i> timeliness. In this type, the sync mark also starts a timer in each of the participating stations. Like <i>synchronized</i> timeliness, expiration of the timer always causes the timeliness attribute to be set to FALSE. Unlike <i>synchronized</i> timeliness, receipt of new data at any time (not just within the time window started with the receipt of a sync mark) causes the timeliness attribute to be set to TRUE.

4.3.3.4 AP structure

The internals of APs may be represented by one, or more Application Process Objects (APOs) and accessed through one or more Application Entities (AEs). AEs provide the communication capabilities of the AP. For each fieldbus AP, there is one and only one FAL AE. APOs are the network representation of application specific capabilities (user application process objects) of an AP that are accessible through its FAL AE.

4.3.3.5 AP class

An AP class is a definition of the attributes and services of an AP. The standard class definition for APs is defined in this standard. User defined classes also may be specified. Class identifiers (described in 3.8.2) are assigned from a set reserved for this purpose.

4.3.3.6 AP type

As described above in the previous subclauses, APs are defined by instantiating an AP class. Each AP definition is composed of the attributes and services selected for the AP from those defined by its AP class. In addition, an AP definition contains values for one or more of the attributes selected for it. When two APs share the same definition, that definition is referred to as an AP type. Thus, an AP type is a generic specification of an AP that may be used to define one or more APs.

4.3.4 Application process objects

4.3.4.1 Definition of APO

An application process object (APO) is a network representation of a specific aspect of an AP. Each APO represents a specific set of information and processing capabilities of an AP that are accessible through services of the FAL. APOs are used to represent these capabilities to other APs in a fieldbus system.

From the perspective of the FAL, an APO is modeled as a network accessible object contained within an AP or within another APO (APOs may contain other APOs). APOs provide the network definition for objects contained within an AP that are remotely accessible. The definition of an APO includes an identification of the FAL services that can be used by remote APs for remote access. The FAL services, as shown in Figure 7, are provided by the FAL communications entity of the AP, known as the FAL Applications Entity (FAL AE).

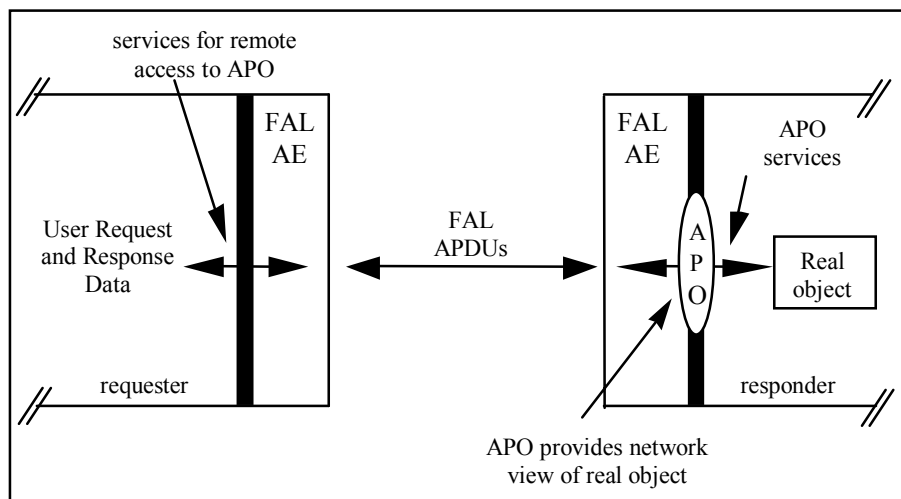


Figure 7 – APOs services conveyed by the FAL

In Figure 7, remote APs acting as clients may access the real object by sending requests through the APO that represents the real object. Local aspects of the AP convert between the network view (the APO) of the real object and the internal AP view of the real object.

To support the publisher/subscriber model of interaction, information about the real object can be published through its APO. Remote APs acting as subscribers see the APO view of the published information instead of having to know any of the real object specific details.

4.3.4.2 APO classes

An APO Class is a generic specification for a set of APOs, each of which is described by the same set of attributes and accessed using the same set of services.

APO Classes provide the mechanism for standardizing network visible aspects of APs. Each standard APO Class definition specifies a particular set of network accessible AP attributes and services. IEC 61158-6 specifies the syntax and the procedures used by the FAL protocol to provide remote access to the attributes and services of an APO Class.

Standard APO Classes are specified by this standard for the purpose of standardizing remote access to APs. User defined classes may also be specified.

User defined classes are defined as subclasses of standardized APO Classes or of other user-defined classes. They may be defined by identifying new attributes or by indicating that optional attributes for the parent class are mandatory for the subclass. The conventions for defining classes defined in 3.8.2 may be used for this purpose. The method for registering or otherwise making these new class definitions available for public use is beyond the scope of this standard.

4.3.4.3 APOs as instances of APO classes

APO Classes are defined in this standard using templates. These templates are used not only to define APO Classes, but also to specify the instances of a class.

Each APO defined for an AP is an instance of an APO Class. Each APO provides the network view of a real object contained in an AP. An APO is defined by

- (1) selecting the attributes from its APO Class template that are to be accessible from the real object,
- (2) assigning values to one or more attributes indicated as key in the template. Key attributes are used to identify the APO;
- (3) assigning values to zero, one, or more non-key attributes for the APO. Non-key attributes are used to characterize the APO;
- (4) selecting the services from the template that may be used by remote APs to access the real object.

Subclause 3.8.2 of this standard specifies the conventions for class templates. These conventions provide for the definition of mandatory, optional, and conditional attributes and services.

Mandatory attributes and services are required to be present in all APOs of the class. Optional attributes and services may be selected, on an APO by APO basis, for inclusion in an APO. Conditional attributes and services are defined with an accompanying constraint statement. Constraint statements specify the conditions that indicate whether or not the attribute is to be present in an APO.

4.3.4.4 APO types

APO types provide the mechanism for defining standard APOs.

As described above in the previous subclauses, APOs are defined by instantiating an APO class. Each APO definition is composed of the attributes and services selected for the APO from those defined by its APO class. In addition, an APO definition contain values for one or more of the attributes selected for it. When two APOs share the same definition, except for the key attribute settings, that definition is referred to as an APO type. Thus, an APO type is a generic specification of an APO that may used to define one or more APOs.

4.3.5 Application entities

4.3.5.1 Definition of FAL AE

An application entity provides the communication capabilities for a single AP. An FAL AE provides a set of services and the supporting protocols to enable communications between APs in a fieldbus environment. The services provided by FAL AEs are grouped into Application Service Elements (ASE), such that the FAL services provided to an AP are defined by the ASEs its FAL AE contains. Figure 8 illustrates this concept.

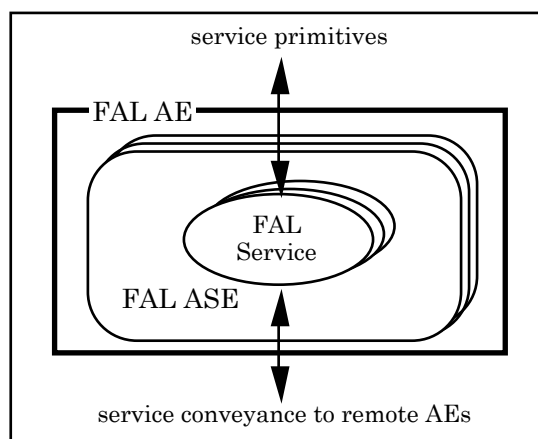


Figure 8 – Application entity structure

4.3.5.2 AE type

Application entities that provide the same set of ASEs are of the same AE-type. Two AEs that share a common set of ASEs are capable of communicating with each other.

4.3.6 Fieldbus application service elements

4.3.6.1 General

An application service element (ASE), as defined in ISO/IEC 9545, is a set of application functions that provide a capability for the interworking of application-entity-involutions for a specific purpose. ASEs provide a set of services for conveying requests and responses to and from application processes and their objects. AEs, as defined above, are represented by a collection of ASE involutions within the AE.

4.3.6.2 FAL services

FAL Services convey functional requests/responses between APs. Each FAL service is defined to convey requests and responses for access to a real object modeled as an FAL accessible object.

The FAL defines both confirmed and unconfirmed services. Confirmed service requests are sent to the AP containing the real object. An invocation of a confirmed service request may be identified by a user supplied Invoke ID. This Invoke ID is returned in the response by the AP containing the real object. When present, it is used by the requesting AP and its FAL AE to associate the response with the appropriate request.

Unconfirmed services may be sent from the AP containing the real object to send information about the object. They also may be sent to the AP containing the real object to access the real object. Both types of unconfirmed services may be defined for the FAL.

4.3.6.3 Definition of FAL ASEs

4.3.6.3.1 General

A modular approach has been taken in the definition of FAL ASEs. The ASEs defined for the FAL are also object-oriented. In general, ASEs provide a set of services designed for one specific object class or for a related set of classes. Common object management ASEs, when present, provide a common set of management services applicable to all classes of objects.

To support remote access to the AP, the Application Relationship ASE is defined. It provides services to the AP for defining and establishing communication relationships with other APs, and it provides services to the other ASEs for conveying their service requests and responses.

Each FAL ASE defines a set of services, APDUs, and procedures that operate on the classes that it represents. Only a subset of the ASE services may be provided to meet the needs of an application. Profiles may be used to define such subsets. Definition of profiles is beyond the scope of this standard.

APDUs are sent and received between FAL ASEs that support the same services. Each FAL AE contains, at a minimum, the AR ASE and at least one other ASE. Figure 9 illustrates an example set of the FAL ASEs and their architectural relationships. All APO ASEs follow this example.

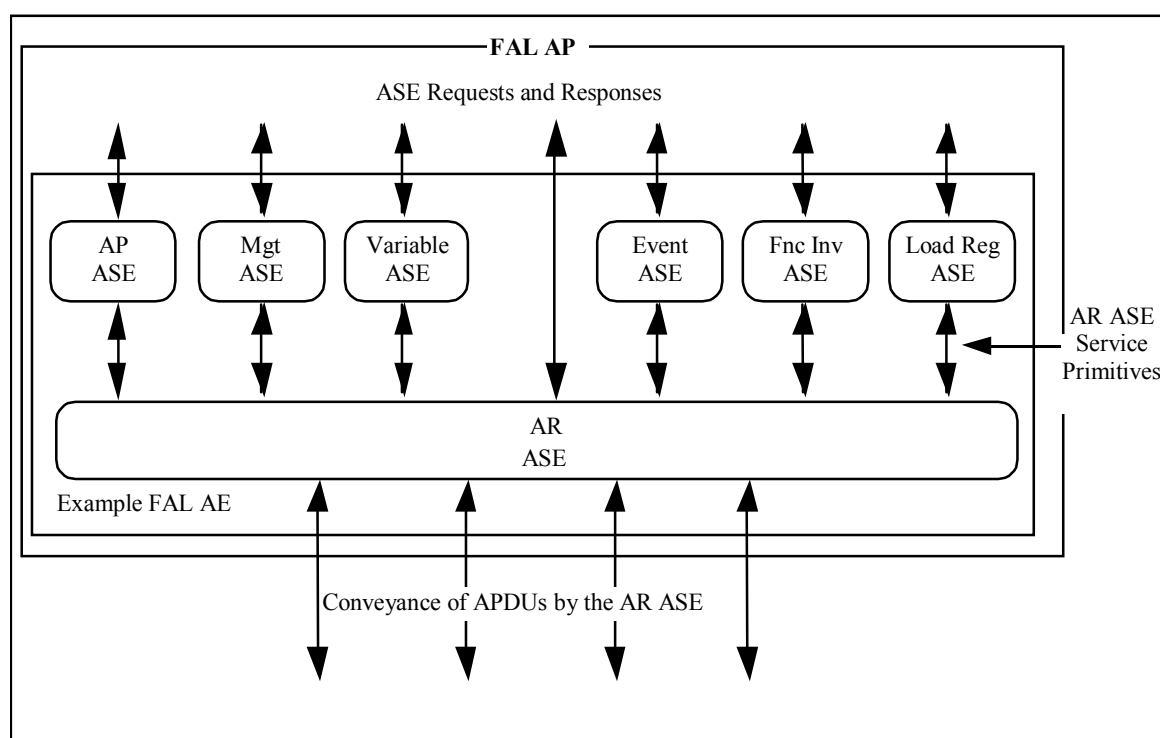


Figure 9 – Example FAL ASEs

4.3.6.3.2 Object management ASE

A special object management ASE may be specified for the FAL to provide services for the management of objects. Its services are used to access object attributes, and create and delete object instances. These services are used to manage network visible AP objects accessed through the FAL. The specific operational services that apply to each object type are specified in the definition of the ASE for the object type. Figure 10 illustrates the integration of management and operational services for an object within an AP.

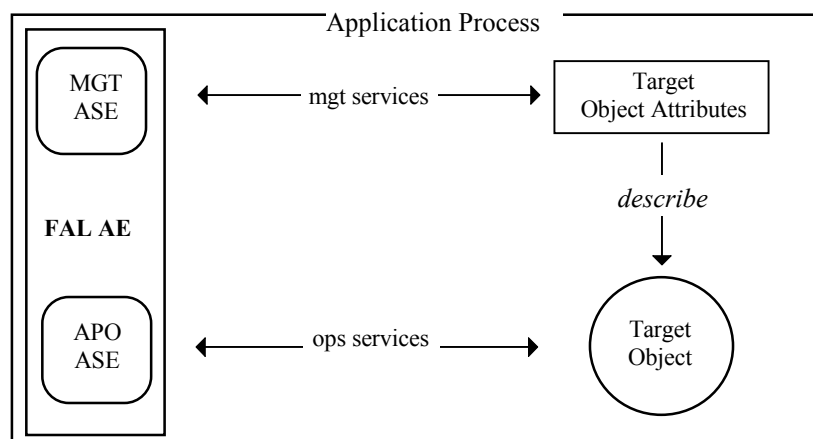


Figure 10 – FAL management of objects

4.3.6.3.3 AP ASE

An AP ASE may be specified for the identification and control of FAL APs. The attributes defined by the AP ASE specify characteristics of the AP about its manufacturer, and list its contents and capabilities.

4.3.6.3.4 APO ASEs

The FAL specifies a set of ASEs with services defined for accessing the APOs of an AP. The APO ASEs defined for the FAL are defined by each Communication Model.

4.3.6.3.5 AR ASE

An AR ASE is specified to establish and maintain application relationships (ARs) that are used to convey FAL APDUs between/among APs. ARs represent application layer communication channels between APs. AR ASEs are responsible for providing services at the endpoints of ARs. AR ASE services may be defined for establishing, terminating, and aborting ARs, for conveying APDUs for the AE, and for indicating the local status of the AR to the user. In addition, local services may be defined for accessing certain aspects of AR endpoints.

4.3.6.4 FAL service conveyance

FAL APO ASEs provide services to convey requests and responses between service users and real objects.

To accomplish the task of conveying service requests and responses, three types of activities for the sending user and three corresponding types for the receiving user are defined. At the sending user, they accept service requests and responses to be conveyed. Second, they select the type of FAL APDU that will be used to convey the request or response and encode the service parameters into its body portion. Then they submit the encoded APDU body to the AR ASE for conveyance.

At the receiving user, they receive encoded APDU bodies from the AR ASE. They decode the APDU bodies and extract the service parameters conveyed by them. To conclude the conveyance, they deliver the service request or response to the user. Figure 11 illustrates these concepts.

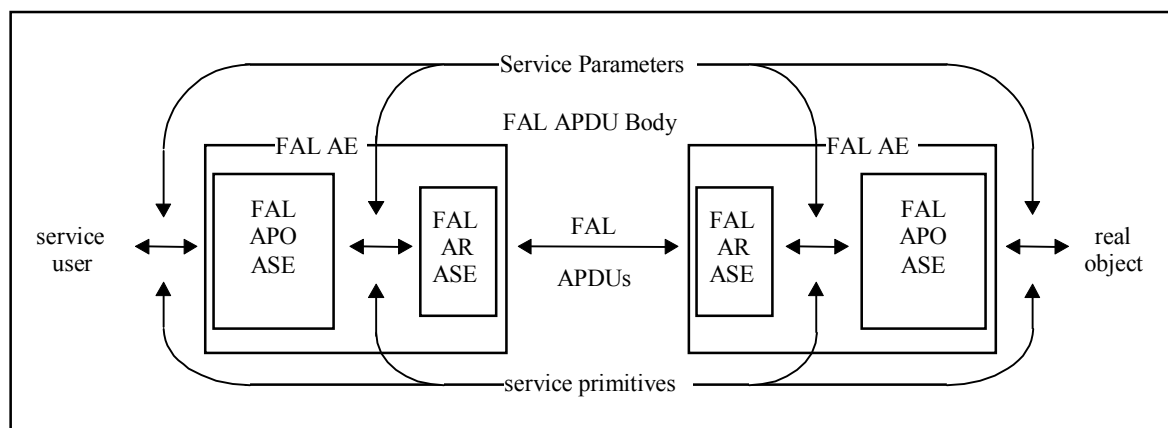


Figure 11 – ASE service conveyance

4.3.6.5 FAL presentation context

The presentation context in the OSI environment is used to distinguish the APDUs of one ASE from another, and to identify the transfer syntax rules used to encode each APDU. However, the fieldbus communications architecture does not include the presentation layer. Therefore, an alternate mechanism is provided for the FAL by each of the specific types of Communication Models.

4.3.7 Application relationships

4.3.7.1 Definition of AR

ARs represent communication channels between APs. They define how information is communicated between APs. Each AR is characterized by how it conveys ASE service requests and responses from one AP to another. These characteristics are described below.

4.3.7.2 AR-endpoints

ARs are defined as a set of cooperating APs. The AR ASE in each AP manages an endpoint of the AR, and maintains its local context. The local context of an AR endpoint is used by the AR ASE to control the conveyance of APDUs on the AR.

4.3.7.3 AR-endpoint classes

ARs are composed of a set of endpoints of compatible classes. AR endpoint classes are used to represent AR endpoints that convey APDUs in the same way. Through the standardization of endpoint classes, ARs for different models of interaction can be defined.

4.3.7.4 AR cardinality

ARs characterize communications between APs. One of the characteristics of an AR is the number of AR endpoints in the AR. ARs that convey services between two APs have a cardinality of 1-to-1. Those that convey services from one AP to a number of APs have a cardinality of 1-to-many. Those that convey services from/to multiple APs have a cardinality of many-to-many.

4.3.7.5 Accessing objects through ARs

ARs provide access to APs and the objects within them through the services of one or more ASEs. Therefore, one characteristic is the set of ASE services that may be conveyed to and from these objects by the AR. The list of services that can be conveyed by the AR are selected from those defined for the AE.

4.3.7.6 AR conveyance paths

ARs are modeled as one or two conveyance paths between AR endpoints. Each conveyance path conveys APDUs in one direction between one or more AR endpoints. Each receiving AR endpoint for a conveyance path receives all APDUs transmitting on the AR by the sending AR endpoint.

4.3.7.7 AREP roles

Because APs interact with each other through endpoints, a basic determinant of their compatibility is the role that they play in the AR. The role defines how an AREP interacts with other AREPs in the AR.

For example, an AREP may operate as a client, a server, a publisher, or a subscriber. When an AREP interacts with another AREP on a single AR as both a client and a server, it is defined to have the role of “peer”.

Certain roles may be capable of initiating service requests, while others may be capable only of responding to service requests. This part of the definition of a role identifies the requirement for an AR to be capable of conveying requests in either direction, or only in one direction.

4.3.7.8 AREP buffers and queues

AREPs may be modeled as a queue or as a buffer. APDUs transferred over a queued AREP are delivered in the order received for conveyance. The transfer of APDUs over a buffered AREP is different. In this case, an APDU to be conveyed by the AR ASE is placed in a buffer for transfer. When the Data Link Layer gains access to the network, it transmits the contents of the buffer.

When the AR ASE receives another conveyance request, it replaces the previous contents of the buffer whether or not they were transmitted. Once an APDU is written into a buffer for transfer, it is preserved in the buffer until the next APDU to be transmitted replaces it. While in the buffer, an APDU may be read more than once without deleting it from the buffer or changing its contents.

At the receiving end, the operation is similar. The receiving endpoint places a received APDU into a buffer for access by the AR ASE. When a subsequent APDU is received, it overwrites the previous APDU in the buffer whether or not it was read. Reading the APDU from the buffer is not destructive — it does not destroy or change the contents of the buffer, allowing the contents to be read from the buffer one or more times.

4.3.7.9 User-triggered and scheduled conveyance

Another characteristic of an AREP is when they convey service requests and responses. AREPs that convey them upon submission by the user are called user-triggered. Their conveyance is asynchronous with respect to network operation.

AREPs that convey requests and responses at predefined intervals, regardless of when they are received for transfer are termed scheduled. Scheduled AREPs may be capable of indicating when transferred data was submitted late for transmission, or when it was submitted on time, but transmitted late.

4.3.7.10 AREP timeliness

AREPs convey APDUs between applications using the services of the Data Link Layer. When the timeliness capabilities are defined for an AREP and supported by the Data Link Layer, the AREP forwards the timeliness indicators provided by the Data Link Layer. These timeliness indicators make it possible for subscribers of published data to determine if the data they are receiving is up-to-date or “stale”.

To support these types of timeliness, the publishing AREP establishes a publisher data link connection reflecting the type of timeliness configured for it by management. After connection establishment, the AREP receives user data and submits it to the DLL for transmission, where timeliness procedures are performed. When the Data Link Layer has the opportunity to transmit the data, it transmits the current timeliness status with the data.

At the subscriber AREP, a data link connection is opened to receive published data that reflects the type of timeliness configured for it by management. The Data Link Layer computes the timeliness of received data and then delivers it to the AREP. The data is then delivered to the user AP through the appropriate ASE.

4.3.7.11 Definition and creation of AREPs

AREP definitions specify instances of AREP classes. AREPs may be predefined or they may be defined using a “create” service if their AE supports this capability.

AREPs may be pre-defined and pre-established, or they may be pre-defined and dynamically established. Figure 12 depicts these two cases. AREPs also may require both dynamic definition and establishment or they may be dynamically defined such that they may be used without any establishment (they are defined in an established state).

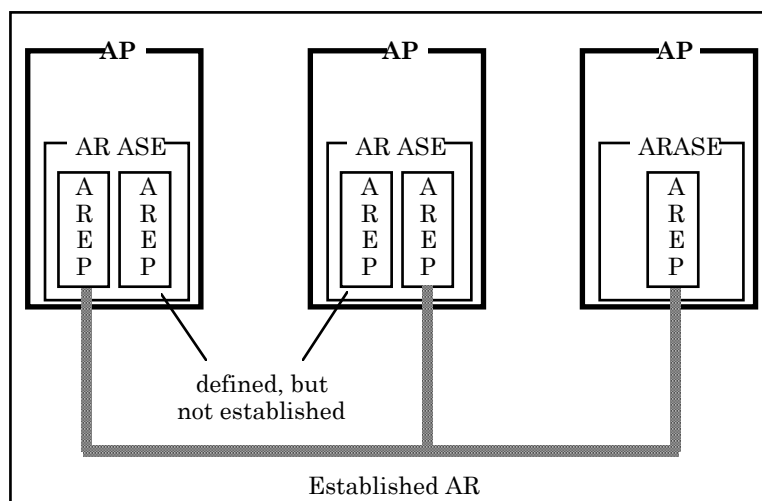


Figure 12 – Defined and established AREPs

4.3.7.12 AR establishment and termination

ARs may be established either before the operational phase of the AP or during its operation. When established during the operation of an AP, the AR is established through the exchange of AR APDUs.

Once an AR has been established, an AR may be terminated gracefully or it may be aborted, depending on the capabilities of the AR.

4.4 Fieldbus Application Layer naming and addressing

4.4.1 General

This subclause refines the principles defined in ISO 7498-3 that involve the identification (naming) and location (addressing) of APOs referenced through the fieldbus Application Layer.

This subclause defines how names and numeric identifiers are used to identify APOs accessible through the FAL. This subclause also indicates how addresses from underlying layers are used to locate APs in the fieldbus environment.

4.4.2 Identifying objects accessed through the FAL

4.4.2.1 General

APOs accessed through the FAL are identified independent of their location. That is, if the location of the AP that contains the APO changes, the APO may still be referenced using the same set of identifiers.

Identifiers for APs and APOs within the FAL are defined as key attributes in the class definitions for APOs. Within these APO definitions, two types of key attributes are commonly used, names and numeric identifiers.

4.4.2.2 Names

Names are string-oriented identifiers. They are defined to permit APs and APOs to be named within the system where they are used. Therefore, although the scope of the name of an APO is specific to the AP in which it resides, the assignment of the name is administered within the system in which it is configured.

Names may be descriptive, although they do not have to be. Descriptive names make it possible to provide meaningful information, such as its use, about the object they name.

Names may also be coded. Coded names make it possible to identify an object using a short, compressed form of a name. They are typically simpler to transfer and process, but not as easy to understand as descriptive names.

4.4.2.3 Numeric identifiers

Numeric identifiers are identifiers whose values are numbers. They are designed for efficient use within the fieldbus system, and may be assigned for efficient access to APOs by their AP.

4.4.3 Addressing APs accessed through the FAL

Fieldbus addresses represent the network locations of APs. Addresses relevant to the FAL are the addresses of the underlying layers that are used to locate the AREPs of an AP.

4.5 Architecture summary

The summary of the FAL architecture is presented in this Clause. Figure 13 illustrates the major components of the FAL architecture and how they relate to each other.

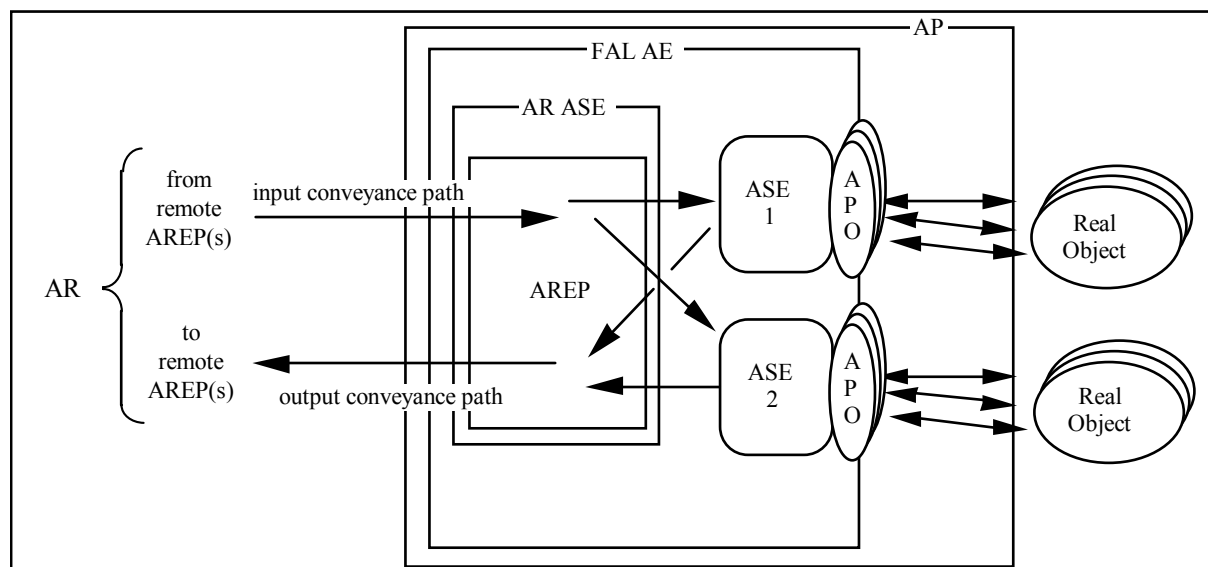


Figure 13 – FAL architectural components

Figure 13 depicts an AP that communicates through the FAL AE. The AP represents its internal real objects as APOs for remote access to them. Two ASEs are shown that provide the remote access services to their related APOs. The AR ASE contains a single AREP that conveys service requests and responses for the ASEs to one or more remote AREPs located in remote APs.

4.6 FAL service procedures

4.6.1 FAL confirmed service procedures

The requesting user submits a confirmed service request primitive to its FAL. The appropriate FAL ASE builds the related confirmed service request APDU Body and conveys it on the specified AR.

Upon receipt of the confirmed service request APDU Body, the responding ASE decodes it. If a protocol error did not occur, the ASE delivers a confirmed service indication primitive to its user.

If the responding user is able to successfully process the request, the user returns a confirmed service response (+) primitive.

If the responding user is unable to successfully process the request, the service fails and the user issues a confirmed service response (-) primitive indicating the reason.

The responding ASE builds a confirmed service response APDU Body for a confirmed service response (+) primitive or a confirmed service error APDU Body for a confirmed service response (-) primitive and conveys it on the specified AR.

Upon receipt of the returned APDU Body the initiating ASE delivers a confirmed service confirmation primitive to its user which specifies success or failure, and the reason for failure if a failure occurred.

4.6.2 FAL unconfirmed service procedures

The requesting user submits an unconfirmed service request primitive to its FAL AE. The appropriate FAL ASE builds the related unconfirmed service request APDU Body and conveys it on the specified AR.

Upon receipt of the unconfirmed request APDU Body, the receiving ASE(s) participating in the AR delivers the appropriate unconfirmed service indication primitive to its user. Timeliness parameters are included in the indication primitive if the AR that conveyed the APDU Body supports timeliness.

4.7 Common FAL attributes

In the specifications of the FAL classes that follow, many classes use the following attributes. Therefore, these attributes are defined here instead of with the other attributes for each of the classes, except for the Data Type class.

ATTRIBUTES:

- | | | | |
|---|-----|----------------|--------------------|
| 1 | (o) | Key Attribute: | Numeric Identifier |
| 2 | (o) | Key Attribute: | Name |
| 3 | (o) | Attribute: | User Description |
| 4 | (o) | Attribute: | Object Revision |

Numeric Identifier

This optional key attribute specifies the numeric id of the object. It is used as a shorthand reference by the FAL protocol to identify the object. There are three possibilities for identification purposes: numeric identifier or name or both. This attribute is required for the Data Type model.

Name

This optional key attribute specifies the name of the object. There are three possibilities for identification purposes: numeric identifier or name or both.

User description

This optional attribute contains user defined descriptive information about the object.

Object Revision

This optional attribute specifies the revision level of the object. It is a structured attribute composed of major and minor revision numbers. If Object Revision is supported, it contains both a Major Revision and a Minor Revision with a value range 0 to 15 for each. The use of major/minor fields is intended to provide the following features:

Major Revision

The Major Revision field contains the major revision value for the object. A change to the major revision indicates that interoperability is affected by the change.

Minor Revision

The Minor Revision field contains the minor revision value for the object. A change to the minor revision indicates that interoperability was not affected by the change -- that is users of the object will continue to be capable of interoperating with the object when its minor revision is changed, provided that the major revision remains the same.

4.8 Common FAL service parameters

In the specifications of the FAL services that follow, many services use the following parameters. Therefore, they are defined here instead of with the other parameters for each of the services.

AREP

This parameter contains sufficient information to locally identify the AREP to be used to convey the service. This parameter may use a key attribute of the AREP to identify the application relationship. When an AREP supports multiple contexts (established using the Initiate service) at the same time, the AREP parameter is extended to identify the context as well as the AREP.

Invoke ID

This parameter identifies this invocation of the service. It is used to associate service requests with responses. Therefore, no two outstanding service invocations may be identified by the same Invoke ID value.

FAL ASE/FAL Class

This parameter specifies the FAL ASE (e.g. AP, AR, Variable, Data Type, Event, Function Invocation, Load Region) and the FAL Class within the ASE (e.g. AREP, Variable List, Notifier, Action).

Numeric ID

This parameter is the numeric identifier of the object.

Error Info

This parameter provides error information for service errors. It is returned in confirmed service response(-) primitives. It is composed of the following elements.

Error Class

This parameter indicates the general class of error. Valid values are specified in the definition of Error Code parameter, below.

Error Code

This parameter identifies the specific service error.

Additional Code

This optional parameter identifies the error encountered when processing the request specific to the object being accessed. When used, the value submitted in the response primitive is delivered unchanged in the confirmation primitive.

Additional Detail

This optional parameter contains user data that accompanies the negative response. When used, the value submitted in the response primitive is delivered unchanged in the confirmation primitive.

4.9 APDU size

APDU size is Communication Model dependent.

5 Data type ASE**5.1 General****5.1.1 Overview**

Fieldbus data types specify the machine independent syntax for application data conveyed by FAL services. The fieldbus application layer supports the definition and transfer of both basic and constructed data types. Encoding rules for the data types specified in this Clause are provided in IEC 61158-6.

Basic types are atomic types that cannot be decomposed into more elemental types. Constructed types are types composed of basic types and other constructed types. Their complexity and depth of nesting is not constrained by this standard.

Data types are defined as instances of the data type class, as shown in Figure 14. Only a subset of the data types defined in this Clause are shown in this figure. Defining new types is accomplished by providing a numeric id and supplying values for the attributes defined for the data type class.

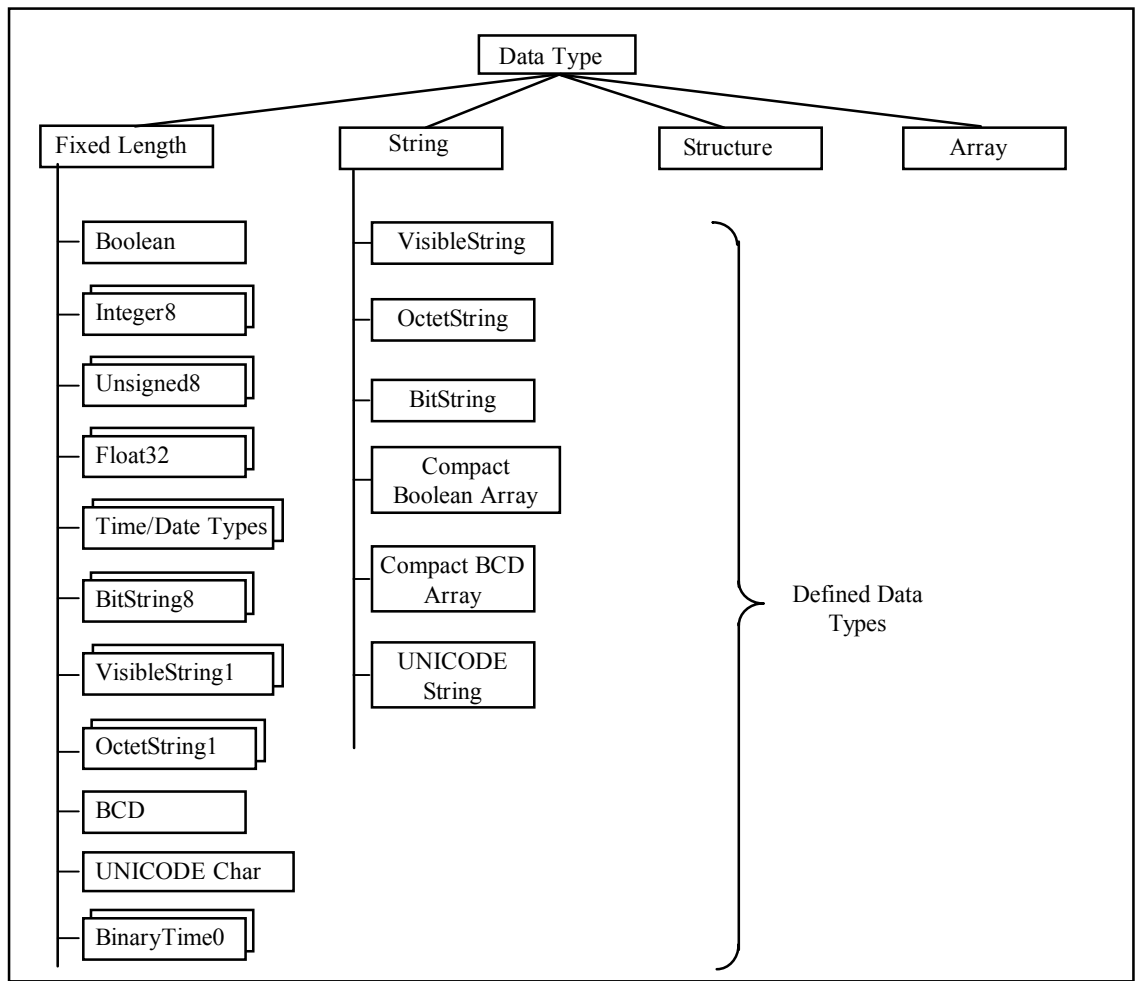


Figure 14 – Data type class hierarchy example

The data type definitions in Figure 14 are represented as a class/format/instance structure beginning with data type class entitled "Data Type". The formats for data types are defined by the data type class and are represented in Figure 14.

The basic data classes are used to define fixed length and bitstring data types. Standard types taken from ISO/IEC 8824 are referred to as *simple* data types. Other standard basic data types are defined specifically for fieldbus applications and are referred to as *specific types*.

The constructed types specified in this standard are strings, arrays and structures. There are no standard types defined for arrays and structures.

5.1.2 Basic type overview

Most basic types are defined from a set of ISO/IEC 8824 types (simple types). Some ISO/IEC 8824 types have been extended for fieldbus specific use (specific types).

Simple types are ISO/IEC 8824 universal types. They are defined in this standard to provide them with fieldbus class identifiers.

Specific types are basic types defined specifically for use in the fieldbus environment. They are defined as simple class subtypes.

Basic types have a constant length. Two variations are defined, one for defining data types whose length is an integral number of octets, and one for defining data types whose length is bits.

NOTE Boolean, Integer, OctetString, VisibleString, and UniversalTime are defined in this standard for the purpose of assigning fieldbus class identifiers to them. This standard does not change their definitions as specified in ISO/IEC 8824.

5.1.3 Fixed length type overview

The length of Fixed length types is an integral number of octets.

5.1.4 Constructed type overview

5.1.4.1 Strings

A string is composed of an ordered set, variable in number, of homogeneously typed fixed-length elements.

5.1.4.2 Arrays

An array is composed of an ordered set of homogeneously typed elements. This standard places no restriction on the data type of array elements, but it does require that each element be of the same type. Once defined, the number of elements in an array may not be changed.

5.1.4.3 Structures

A structure is made of an ordered set of heterogeneously typed elements called fields. Like arrays, this standard does not restrict the data type of fields. However, the fields within a structure do not have to be of the same type.

5.1.4.4 Nesting level

This standard permits arrays and structures to contain arrays and structures. It places no restriction on the number of nesting levels allowed. However, the FAL services defined to access data provide for partial access to the level negotiated by the Initiate service. The default number of levels for partial access is one.

When an array or structure contains constructed elements, access to a single element in its entirety is always provided. Access to subelements of the constructed element is also provided, but only when explicitly negotiated during AR establishment, or when explicitly preconfigured on pre-established ARs.

NOTE For example, suppose that a data type named "employee" is defined to contain the structure "employee name", and "employee name" is defined to contain "last name" and "first name". To access the "employee" structure, the FAL permits independent access to the entire structure and to the first level field "employee name". Without explicitly negotiating partial access to more than one level, independent access to "last name" or "first name" would not be possible; their values could only be accessed together as a unit through access to "employee" or "employee name".

5.1.5 Specification of user defined data types

Users may find it necessary to define custom data types for their own applications. User defined types are supported by this standard as instances of data type classes.

User defined types are specified in the same manner that all FAL objects are specified. They are defined by providing values for the attributes specified for their class.

5.1.6 Transfer of user data

User data is transferred between applications by the FAL protocol. All encoding and decoding are performed by the FAL user.

The rules for encoding user data in FAL protocol data units is data type dependent. These rules are defined in IEC 61158-6. User-defined data types for which there are no encoding rules are transferred as a variable-length sequence of octets. The format of the data within the octet string is defined by the user.

5.2 Formal definition of data type objects

5.2.1 Data type class

5.2.1.1 Template

The data type class specifies the root of the data type class tree. Its parent class "top" indicates the top of the FAL class tree.

FAL ASE:		DATA TYPE ASE
CLASS:	DATA TYPE	
CLASS ID:		5 (FIXED LENGTH & STRING), 6 (STRUCTURE), 12 (ARRAY)
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m)	Key Attribute: Data Type Numeric Identifier
2	(o)	Key Attribute: Data Type Name
3	(m)	Attribute: Format (FIXED LENGTH, STRING, STRUCTURE, ARRAY)
4	(c)	Constraint: Format = FIXED LENGTH STRING
4.1	(m)	Attribute: Octet Length
5	(c)	Constraint: Format = STRUCTURE
5.1	(m)	Attribute: Number of Fields
5.2	(m)	Attribute: List of Fields
5.2.1	(o)	Attribute: Field Name
5.2.2	(m)	Attribute: Field Data Type
6	(c)	Constraint: Format = ARRAY
6.1	(m)	Attribute: Number of Array Elements
6.2	(m)	Attribute : Array Element Data Type

5.2.1.2 Attributes

Data Type Numeric Identifier

This attribute identifies the numeric identifier of the related data type.

Data Type Name

This optional attribute identifies the name of the related data type.

Format

This attribute identifies the data type as a fixed-length, string, array, or data structure.

Octet Length

This conditional attribute defines the representation of the dimensions of the associated type object. It is present when the value of the format attribute is "FIXED LENGTH" or "STRING". For FIXED LENGTH data types, it represents the length in octets. For STRING data types, it represents the length in octets for a single element of a string.

Number of Fields

This conditional attribute defines the number of fields in a structure. It is present when the value of the format attribute is "STRUCTURE".

List of Fields

This conditional attribute is an ordered list of fields contained in the structure. Each field is specified by its number and its type. Fields are numbered sequentially from 0 (zero) in the order in which they occur. Partial access to fields within a structure is supported by identifying

the field by number. This attribute is present when the value of the format attribute is "STRUCTURE".

Field Name

This conditional, optional attribute specifies the name of the field. It may be present when the value of the format attribute is "STRUCTURE".

Field Data Type

This conditional attribute specifies the data type of the field. It is present when the value of the format attribute is "STRUCTURE". This attribute may itself specify a constructed data type either by referencing a constructed data type definition by its numeric id, or by embedding a constructed data type definition here. When embedding a description, the Embedded Data Type description shown below is used.

Number of Array Elements

This conditional attribute defines the number of elements for the array type. Array elements are indexed starting at "0" through "n-1" where the size of the array is "n" elements. This attribute is present when the value of the format attribute is "ARRAY".

Array Element Data Type

This conditional attribute specifies the data type for the elements of an array. All elements of the array have the same data type. It is present when the value of the format attribute is "ARRAY". This attribute may itself specify a constructed data type either by referencing a constructed data type definition by its numeric id, or by embedding a constructed data type definition here. When embedding a description, the Embedded Data Type description shown below is used.

Embedded Data Type Description

This attribute is used to recursively define embedded data types within a structure or array. The template below defines its contents. The attributes shown in the template are defined above in the data type class, except for the Embedded Data Type attribute, which is a recursive reference to this attribute. It is used to define nested elements.

ATTRIBUTES:

1	(m)	Attribute:	Format(FIXED LENGTH, STRING, STRUCTURE, ARRAY)
2	(c)	Constraint:	Format = FIXED LENGTH STRING
2.1	(m)	Attribute:	Data Type Numeric ID value
2.2	(m)	Attribute:	Octet Length
3	(c)	Constraint:	Format = STRUCTURE
3.1	(m)	Attribute:	Number of Fields
3.2	(m)	Attribute:	List of Fields
3.2.1	(m)	Attribute:	Embedded Data Type Description
4	(c)	Constraint:	Format = ARRAY
4.1	(m)	Attribute:	Number of Array Elements
4.2	(m)	Attribute:	Embedded Data Type Description

5.3 FAL defined data types

5.3.1 Fixed length types

5.3.1.1 Boolean types

5.3.1.1.1 Boolean

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	1
2	Data Type Name	=	Boolean

3	Format	=	FIXED LENGTH
4.1	Octet Length	=	1

This data type expresses a Boolean data type with the values TRUE and FALSE.

5.3.1.1.2 BOOL

This IEC 61131-3 type is the same as Boolean.

5.3.1.1.3 VT_BOOLEAN

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	VT_BOOLEAN
4	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This data type expresses a Boolean data type with the values TRUE (-1) and FALSE (0) (see Interger16).

5.3.1.2 Bitstring types

5.3.1.2.1 BitString8

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	22
2	Data Type Name	=	Bitstring8
3	Format	=	FIXED LENGTH
5.1	Octet Length	=	1

This type contains 1 element of type BitString.

5.3.1.2.2 BYTE

This IEC 61131-3 type is the same as Bitstring8.

5.3.1.2.3 BitString16

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	23
2	Data Type Name	=	Bitstring16
3	Format	=	FIXED LENGTH
5.1	Octet Length	=	2

5.3.1.2.4 WORD

This IEC 61131-3 type is the same as Bitstring16.

5.3.1.2.5 BitString32

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	24
2	Data Type Name	=	Bitstring32
3	Format	=	FIXED LENGTH
5.1	Octet Length	=	4

5.3.1.2.6 DWORD

This IEC 61131-3 type is the same as Bitstring32.

5.3.1.2.7 BitString64**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	57
2	Data Type Name	=	Bitstring64
3	Format	=	FIXED LENGTH
5.1	Octet Length	=	8

5.3.1.2.8 LWORD

This IEC 61131-3 type is the same as Bitstring64.

5.3.1.3 Currency types**5.3.1.3.1 currency****CLASS:** Data Type**ATTRIBUTES:**

2	Data Type Name	=	currency
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This data type defines a signed 64-bit integer in units of 1/10,000 (or 1/100 of a cent). A currency number stored as an 8-byte, two's complement integer, scaled by 10,000 to give a fixed-point number with 15 digits to the left of the decimal point and 4 digits to the right. This representation provides a range of $\pm 922337203685477,5807$. This data type is useful for calculations involving money, or for any fixed-point calculation where accuracy is particularly important.

5.3.1.4 Date types**5.3.1.4.1 BinaryDate****CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	11
2	Data Type Name	=	BinaryDate
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	7

This data type is composed of six elements of unsigned values and expresses calendar date and time. The first element is an Unsigned16 data type and gives the fraction of a minute in milliseconds. The second element is an Unsigned8 data type and gives the fraction of an hour in minutes. The third element is an Unsigned8 data type and gives the fraction of a day in hours. The fourth element is an Unsigned8 data type. Its upper three (3) bits give the day of the week and its lower five (5) bits give the day of the month. The fifth element is an Unsigned8 data type and gives the month. The last element is Unsigned8 data type and gives the year.

5.3.1.4.2 BinaryDate2000**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	51
2	Data Type Name	=	BinaryDate2000
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This data type is composed of six elements of unsigned values and expresses calendar date and time. The first element is an Unsigned16 data type and gives the fraction of a minute in milliseconds. The second element is an Unsigned8 data type and gives the fraction of an hour

in minutes. The third element is an Unsigned8 data type and gives the fraction of a day in hours. The fourth element is an Unsigned8 data type. Its upper three (3) bits give the day of the week and its lower five (5) bits give the day of the month. The fifth element is an Unsigned8 data type and gives the month. The last element is Unsigned16 data type and gives the year.

5.3.1.4.3 Date

CLASS:		Data Type
ATTRIBUTES:		
1	Data Type Numeric Identifier	= 50
2	Data Type Name	= Date
3	Format	= FIXED LENGTH
4.1	Octet Length	= 7

This data type is composed of six elements of unsigned values and expresses calendar date and time. The first element is an Unsigned16 data type and gives the fraction of a minute in milliseconds. The second element is an Unsigned8 data type and gives the fraction of an hour in minutes. The third element is an Unsigned8 data type and gives the fraction of a day in hours with the most significant bit indicating Standard Time or Daylight Saving Time. The fourth element is an Unsigned8 data type. Its upper three (3) bits give the day of the week and its lower five (5) bits give the day of the month. The fifth element is an Unsigned8 data type and gives the month. The last element is Unsigned8 data type and gives the year. The values 0 ... 50 correspond to the years 2000 to 2050, the values 51 ... 99 correspond to the years 1951 to 1999.

5.3.1.4.4 DATE

CLASS:		Data Type
ATTRIBUTES:		
1	Data Type Numeric Identifier	= not used
2	Data Type Name	= DATE
3	Format	= FIXED LENGTH
4.1	Octet Length	= 2

This IEC 61131-3 type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of two octets. It expresses the date as a number of days, starting from 1972.01.01 (January 1st, 1972), the start of the Coordinated Universal Time (UTC) era, until 2151.06.06 (June 6th, 2151), i.e. a total range of 65536 days.

5.3.1.4.5 date

This data type is the same as Float64.

The data type date has a resolution in the range of one nanosecond. It is valid for dates between 1 January 0100 and 31 December 9999. The value 0,0 has been defined for 30 December 1899, 00:00. The integer part of the value represents the days after 30 December 1899 (for dates before this day, the corresponding value is negative); the fractional part defines the time at that day.

5.3.1.4.6 TimeOfDay

CLASS:		Data Type
ATTRIBUTES:		
1	Data Type Numeric Identifier	= 12
2	Data Type Name	= TimeOfDay
4	Format	= FIXED LENGTH
4.1	Octet Length	= 6

This data type is composed of two elements of unsigned values and expresses the time of day and the date. The first element is an Unsigned32 data type and gives the time after the midnight in milliseconds. The second element is an Unsigned16 data type and gives the date counting the days from January 1, 1984.

5.3.1.4.7 TimeOfDay with date indication

This data type is the same as the TimeOfDay data type defined above.

5.3.1.4.8 TimeOfDay without date indication

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 52
2 Data Type Name	= TimeOfDay without date indication
4 Format	= FIXED LENGTH
4.1 Octet Length	= 4

This data type is composed of one element of an unsigned value and expresses the time of day. The element is an Unsigned32 data type and gives the time after the midnight in milliseconds.

5.3.1.4.9 TIME_OF_DAY

This IEC 61131-3 type is the same as TimeOfDay without date indication.

5.3.1.4.10 TimeDifference

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 13
2 Data Type Name	= TimeDifference
3 Format	= FIXED LENGTH
4.1 Octet Length	= 4 or 6

This data type is composed of two elements of unsigned values that express the difference in time. The first element is an Unsigned32 data type that provides the fractional portion of one day in milliseconds. The optional second element is an Unsigned16 data type that provides the difference in days.

5.3.1.4.11 TimeDifference with date indication

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 53
2 Data Type Name	= TimeDifference with date indication
3 Format	= FIXED LENGTH
4.1 Octet Length	= 6

This data type is composed of two elements of unsigned values that express the difference in time. The first element is an Unsigned32 data type that provides the fractional portion of one day in milliseconds. The second element is an Unsigned16 data type that provides the difference in days.

5.3.1.4.12 TimeDifference without date indication

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 54
2 Data Type Name	= TimeDifference without date indication

This data type is composed of one element of an unsigned value that express the difference in time. The element is an Unsigned32 data type that provides the fractional portion of one day in milliseconds.

1	Data Type Numeric Identifier	=	21
2	Data Type Name	=	Time Value
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

1	Data Type Numeric Identifier	=	16
2	Data Type Name	=	UniversalTime
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	12

1	Data Type Numeric Identifier	=	17
2	Data Type Name	=	FieldbusTime
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	7

CLASS:	Data Type		
ATTRIBUTES:			
2	Data Type Name	=	PERSISTDEF
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

Value
CBAVolatile
CBAPendingPersistent
CBAPersistent

5.3.1.5.2 VARTYPE**CLASS:** Data Type**ATTRIBUTES:**

2 Data Type Name = VARTYPE
 3 Format = FIXED LENGTH
 4.1 Octet Length = 2

The VARTYPE specifies the data type that governs the interpretation of the data. The allowed values are shown in Table 2.

Table 2 – VARTYPE

Value	Data type
VT_EMPTY	no value
VT_NULL	null value (no valid data)
VT_BOOL	VT_BOOLEAN
VT_I1	char
VT_I2	short
VT_I4	long
VT_UI1	unsigned char
VT_UI2	unsigned short
VT_UI4	unsigned long
VT_R4	float
VT_R8	double
VT_CY	Currency
VT_DATE	date
VT_BSTR	address of a BSTR
VT_SAFEARRAY_BOOL	address of a SAFEARRAY (VT_BOOLEAN)
VT_SAFEARRAY_I1	address of a SAFEARRAY (char)
VT_SAFEARRAY_I2	address of a SAFEARRAY (short)
VT_SAFEARRAY_I4	address of a SAFEARRAY (long)
VT_SAFEARRAY_UI1	address of a SAFEARRAY (unsigned char) specified.
VT_SAFEARRAY_UI2	address of a SAFEARRAY (unsigned short) specified.
VT_SAFEARRAY_UI4	address of a SAFEARRAY (unsigned long) specified.
VT_SAFEARRAY_R4	address of a SAFEARRAY (float)
VT_SAFEARRAY_R8	address of a SAFEARRAY (double)
VT_SAFEARRAY_CY	address of a SAFEARRAY (Currency)
VT_SAFEARRAY_DATE	address of a SAFEARRAY (date)
VT_SAFEARRAY_BSTR	address of a SAFEARRAY (BSTR)
VT_DISPATCH	Interface Pointer to an IDispatch interface
VT_UNKNOWN	Interface Pointer to an IUnknown interface
VT_USERDEFINED	address of an userdefined struct
VT_ERROR	A HRESULT is specified.
NOTE All defined structured data types shall have the type code VT_USERDEFINED.	

5.3.1.5.3 ITEMQUALITYDEF**CLASS:** Data Type**ATTRIBUTES:**

2 Data Type Name = ITEMQUALITYDEF
 3 Format = FIXED LENGTH
 4.1 Octet Length = 1

This data type contains the status information of the related data. It consists of three portions: Quality, Substatus and Limits. There are four states of quality (Bad - the value is not useful; Uncertain - the quality of the value is less than normal, but the value may still be useful; Good (Non Cascade) - the quality of the value is good, possible alarm conditions may be indicated by the substatus; Good (Cascade) - the value may be used in control), a set of sub-status values for each quality, and four states of the limits (OK - the value is free to move; low limited (LL) - the value has acceded its low limits, high limited (HL) - the value has acceded its high limits; constant (C) - the value cannot move, no matter what the process does). The allowed values are shown in Table 3.

Table 3 – ITEMQUALITYDEF

Quality	Value	Description
Bad	BadNonSpecific	There is no specific reason why the value is bad. Used for propagation.
	BadNonSpecificLL	and low limited
	BadNonSpecificHL	and high limited
	BadNonSpecificC	and constant
	BadConfigurationError	Set if the value is not useful because there is some other problem with the block, depending on what a specific producer can detect.
	BadConfigurationErrorLL	and low limited
	BadConfigurationErrorHL	and high limited
	BadConfigurationErrorC	and constant
	BadNotConnected	Set if this input is required to be connected and is not connected.
	BadNotConnectedLL	and low limited
	BadNotConnectedHL	and high limited
	BadNotConnectedC	and constant
	BadDeviceFailure	Set if the source of the value is affected by a device failure.
	BadDeviceFailureLL	and low limited
	BadDeviceFailureHL	and high limited
	BadDeviceFailureC	and constant
	BadSensorFailure	Set if the device can determine this condition. The limits define which direction has been exceeded.
	BadSensorFailureLL	and low limited
	BadSensorFailureHL	and high limited
	BadSensorFailureC	and constant
	BadLastKnownValue	Set if this value had been set by communication, which has now failed.
	BadLastKnownValueLL	and low limited
	BadLastKnownValueHL	and high limited
	BadLastKnownValueC	and constant
	BadCommFailure	Set if there has never been any communication with this value since it was last Out of Service.
	BadCommFailureLL	and low limited
	BadCommFailureHL	and high limited
	BadCommFailureC	and constant
	BadOutOfService	The value is not reliable because the block is not being evaluated, and may be under construction by a configuration tool. It is set if the block mode is O/S.
	BadOutOfServiceLL	and low limited
	BadOutOfServiceHL	and high limited
	BadOutOfServiceC	and constant
Uncertain	UncertainNonSpecific	There is no specific reason why the value is uncertain. Used for propagation.
	UncertainNonSpecificLL	and low limited
	UncertainNonSpecificHL	and high limited
	UncertainNonSpecificC	and constant
	UncertainLastUsableValue	Whatever was writing this value has stopped doing so. This is used for fail safe handling.
	UncertainLastUsableValueLL	and low limited
	UncertainLastUsableValueHL	and high limited
	UncertainLastUsableValueC	and constant
	UncertainSubstituteSet	Predefined value is used instead of the calculated one. This is used for fail safe handling.
	UncertainSubstituteSetLL	and low limited
	UncertainSubstituteSetHL	and high limited
	UncertainSubstituteSetC	and constant
	UncertainInitialValue	Value of volatile parameters during and after the reset of the device or a parameter.

Table 3 (continued)

Quality	Value	Description
	UncertainInitialValueLL	and low limited
	UncertainInitialValueHL	and high limited
	UncertainInitialValueC	and constant
	UncertainSensorNotAccurate	Set if the value is at one of the sensor limits. The limits define which direction has been exceeded. Also set if the device can determine that the sensor has reduced accuracy (e.g. degraded analyzer), in which case no limits are set.
	UncertainSensorNotAccurateLL	and low limited
	UncertainSensorNotAccurateHL	and high limited
	UncertainSensorNotAccurateC	and constant
	UncertainEngineeringUnitsExceeded	Set if the value lies outside of the range of values defined for this parameter. The limits define which direction has been exceeded.
	UncertainEngineeringUnitsExceededLL	and low limited
	UncertainEngineeringUnitsExceededHL	and high limited
	UncertainEngineeringUnitsExceededC	and constant
	UncertainSubNormal	Set if a value derived from multiple values has less than the required number of Good sources.
	UncertainSubNormalLL	and low limited
	UncertainSubNormalHL	and high limited
	UncertainSubNormalC	and constant
	UncertainConfigurationError	Set if there is some inconsistency regarding the parameterization or configuration, depending on what a specific producer can detect.
	UncertainConfigurationErrorLL	and low limited
	UncertainConfigurationErrorHL	and high limited
	UncertainConfigurationErrorC	and constant
	UncertainSimulatedValue	Set when the process value is written by the operator while the block is in manual mode.
	UncertainSimulatedValueLL	and low limited
	UncertainSimulatedValueHL	and high limited
	UncertainSimulatedValueC	and constant
	UncertainSensorCalibration	Set during the active calibration process together with the current measured value.
	UncertainSensorCalibrationLL	and low limited
	UncertainSensorCalibrationHL	and high limited
	UncertainSensorCalibrationC	and constant
Good	GoodNonCascOk	No error or special condition is associated with this value.
Non Cascade	GoodNonCascOkC	and constant
	GoodNonCascActiveUpdateEvent	Set if the value is good and the block has an active Update Event.
	GoodNonCascActiveUpdateEventLL	and low limited
	GoodNonCascActiveUpdateEventHL	and high limited
	GoodNonCascActiveUpdateEventC	and constant
	GoodNonCascActiveAdvisoryAlarm	Set if the value is good and the block has an active Alarm with a priority less than 8.
	GoodNonCascActiveAdvisoryAlarmLL	and low limited
	GoodNonCascActiveAdvisoryAlarmHL	and high limited
	GoodNonCascActiveAdvisoryAlarmC	and constant
	GoodNonCascActiveCriticalAlarm	Set if the value is good and the block has an active Alarm with a priority greater than or equal to 8.
	GoodNonCascActiveCriticalAlarmLL	and low limited
	GoodNonCascActiveCriticalAlarmHL	and high limited
	GoodNonCascActiveCriticalAlarmC	and constant
	GoodNonCascUnackUpdateEvent	Set if the value is good and the block has an unacknowledged Update Event.
	GoodNonCascUnackUpdateEventLL	and low limited
	GoodNonCascUnackUpdateEventHL	and high limited

Table 3 (continued)

Quality	Value	Description
	GoodNonCascUnackUpdateEventC	and constant
	GoodNonCascUnackAdvisoryAlarm	Set if the value is good and the block has an unacknowledged Alarm with a priority less than 8.
	GoodNonCascUnackAdvisoryAlarmLL	and low limited
	GoodNonCascUnackAdvisoryAlarmHL	and high limited
	GoodNonCascUnackAdvisoryAlarmC	and constant
	GoodNonCascUnackCriticalAlarm	Set if the value is good and the block has an unacknowledged Alarm with a priority greater than or equal to 8.
	GoodNonCascUnackCriticalAlarmLL	and low limited
	GoodNonCascUnackCriticalAlarmHL	and high limited
	GoodNonCascUnackCriticalAlarmC	and constant
	GoodNonCascInitialFailSafe	This value is from a block that wants its following output block (e.g. AO) to go to Fail Safe.
	GoodNonCascInitialFailSafeLL	and low limited
	GoodNonCascInitialFailSafeHL	and high limited
	GoodNonCascInitialFailSafeC	and constant
	GoodNonCascMaintenanceRequired	The device works still without failure but service support will be necessary soon. This may be detected e.g. by a Transducer Block of a value of pH meter.
	GoodNonCascMaintenanceRequiredLL	and low limited
	GoodNonCascMaintenanceRequiredHL	and high limited
	GoodNonCascMaintenanceRequiredC	and constant
Good	GoodCascOk	No error or special condition is associated with this value.
Cascade	GoodCascOkC	and constant
	GoodCascInitializationAcknowledge	The value is an initialized value from a source (cascade input, remote-cascade in, and remote-output in parameters).
	GoodCascInitializationAcknowledgeLL	and low limited
	GoodCascInitializationAcknowledgeHL	and high limited
	GoodCascInitializationAcknowledgeC	and constant
	GoodCascInitializationRequest	The value is an initialization value for a source (back calculation input parameter), because the lower loop is broken or the mode is wrong.
	GoodCascInitializationRequestLL	and low limited
	GoodCascInitializationRequestHL	and high limited
	GoodCascInitializationRequestC	and constant
	GoodCascNotInvited	The value is from a block which does not have a target mode that would use this input. This covers all cases other than Fail Safe Active, Local Override, and Not Selected. The target mode can be the next permitted mode of higher priority in the case of shedding a supervisory computer.
	GoodCascNotInvitedLL	and low limited
	GoodCascNotInvitedHL	and high limited
	GoodCascNotInvitedC	and constant
	GoodCascDoNotSelect	The value is from a block which should not be selected, due to conditions in or above the block.
	GoodCascDoNotSelectLL	and low limited
	GoodCascDoNotSelectHL	and high limited
	GoodCascDoNotSelectC	and constant
	GoodCascLocalOverride	The value is from a block that has been locked out by a local key switch or is a Complex AO/DO with interlock logic active. The failure of normal control shall be propagated to a PID block for alarm and display purposes. This also implies Not Invited.
	GoodCascLocalOverrideLL	and low limited
	GoodCascLocalOverrideHL	and high limited
	GoodCascLocalOverrideC	and constant

Table 3 (continued)

Quality	Value	Description
	GoodCasInitiateFailSafe	The value is from a block that wants its downstream output block (e.g. AO) to go to Fail Safe. This is determined by a block option to initiate Fail Safe if the status of the primary input and/or cascade input goes Bad.
	GoodCasInitiateFailSafeLL	and low limited
	GoodCasInitiateFailSafeHL	and high limited
	GoodCasInitiateFailSafeC	and constant

5.3.1.5.4 STATEDEF**CLASS:** Data Type**ATTRIBUTES:**

- 2 Data Type Name = STATEDEF
 3 Format = FIXED LENGTH
 4.1 Octet Length = 2

The allowed values are shown in Table 4.

Table 4 – STATEDEF

Value
CBANonExistent
CBAInitializing
CBAReady
CBAOperating
CBADefect

5.3.1.5.5 GROUPEERRORDEF**CLASS:** Data Type**ATTRIBUTES:**

- 2 Data Type Name = GROUPEERRORDEF
 3 Format = FIXED LENGTH
 4.1 Octet Length = 2

The allowed values are shown in Table 5.

Table 5 – GROUPEERRORDEF

Value
CBANonAccessible
CBAOkay
CBAProblem
CBAUnknown

5.3.1.5.6 ACCESSRIGHTSDEF**CLASS:** Data Type**ATTRIBUTES:**

- 2 Data Type Name = ACCESSRIGHTSDEF
 3 Format = FIXED LENGTH
 4.1 Octet Length = 2

The allowed values are shown in Table 6.

Table 6 – ACCESSRIGHTSDEF

Value
CBANoAccess
CBAReadAccess
CBAWriteAccess
CBAFullAccess

5.3.1.6 Handle types

5.3.1.6.1 HRESULT

CLASS: Data Type

ATTRIBUTES:

2 Data Type Name = HRESULT
 3 Format = FIXED LENGTH
 4.1 Octet Length = 4

The allowed values are shown in Table 7. The defined HRESULT values may be extended by user specific values. If necessary the coding scheme as defined in IEC 61158-6 type 10 shall be applied. In particular, the severity code to indicate success or error shall be used.

Table 7 – HRESULT

Value	Description
CBA_E_MALFORMED	The identifier is malformed (too long, unallowed characters, no separation characters, syntactically invalid)
CBA_E_UNKNOWNOBJECT	The object specified in the item identifier is not known.
CBA_E_UNKNOWNMEMBER	The member specified in the item identifier is not known.
CBA_E_TYPERISMATCH	The type specified does not match the expected type.
CBA_E_INVALIDENUMVALUE	The enumeration value is invalid.
CBA_E_INVALIDID	The ConsumerID or ProviderID is not valid.
CBA_E_INVALIDEPSILON	The Epsilon type or value is not valid.
CBA_E_INVALIDSUBSTITUTE	The Substitute type or value is not valid.
CBA_E_INVALIDCONNECTION	It is not allowed to specify a connection from an identifier to itself.
CBA_E_INVALIDCOOKIE	The Cookie value is not valid.
CBA_E_TIMEVALUEUNSUPPORTED	The time value is unsupported.
CBA_E_QOSTYPEUNSUPPORTED	The QoS type is unsupported.
CBA_E_QOSVALUEUNSUPPORTED	The QoS value is unsupported.
CBA_E_PERSISTRUNNING	While the Save service is running, no changes are allowed to the configuration data base (try again in a few seconds).
CBA_E_INUSE	The destination specified in the item identifier is already connected to some other provider.
CBA_E_NOTAPPLICABLE	The operation is currently not applicable.
CBA_E_NONACCESSIBLE	The item is not accessible.
CBA_E_DEFECT	Hardware defect detected, replacement needed.
CBA_S_PERSISTPENDING	The value requested is currently not persistent.
CBA_S_ESTABLISHING	The connection is not yet established.
CBA_S_NOCONNECTION	There is no connection from the provider to this specific consumer.
CBA_S_VALUEBUFFERED	The value was only buffered and has no immediate effect on the process (e.g. device is in state CBAReady).
CBA_S_VALUEUNCERTAIN	The value requested is currently uncertain.
E_OUTOFMEMORY	Insufficient memory to complete the call.
E_INVALIDARG	One ore more arguments are invalid.
E_NOTIMPL	The service is not implemented.
E_FAIL	Unspecified error.
E_NOINTERFACE	No such interface supported.
RPC_S_PROCNUM_OUT_OF_RANGE	The procedure number is out of range.
RPC_E_INVALID_OXID	The object exporter was not found.
RPC_E_INVALID_OID	The specified object was not found or recognized.
RPC_E_INVALID_SET	The object exporter set was not found.

Table 7 (continued)

Value	Description
RPC_E_INVALID_OBJECT	The requested object does not exist.
RPC_E_VERSION_MISMATCH	The ORPC version on the client and server machine does not match.
DISP_E_BADINDEX	Invalid index.
DISP_E_BADPARAMCOUNT	The number of elements provided to DISPPARAMS is different from the number of arguments accepted by the method or property.
DISP_E_BADVARTYPE	One of the arguments is not a valid variant type.
DISP_E_EXCEPTION	The application needs to raise an exception. In this case, the structure passed in pExcepInfo should be filled in.
DISP_E_MEMBERNOTFOUND	The requested member does not exist, or the call to Invoke tried to set the value of a read-only property.
DISP_E_NONAMEDARGS	This implementation of IDispatch does not support named arguments
DISP_E_OVERFLOW	One of the arguments could not be coerced to the specified type.
DISP_E_PARAMNOTFOUND	One of the parameter DISPIDs does not correspond to a parameter on the method. In this case, puArgErr should be set to the first argument that contains the error.
DISP_E_TYPEMISMATCH	One or more of the arguments could not be coerced. The index within rgvarg of the first parameter with the incorrect type is returned in the puArgErr parameter.
DISP_E_UNKNOWNINTERFACE	The interface identifier passed in riid is not IID_NULL.
DISP_E_UNKNOWNLCID	Unknown language.
DISP_E_UNKNOWNNAME	Unknown name.
DISP_E_PARAMNOTOPTIONAL	A required parameter was omitted.
TYPE_E_ELEMENTNOTFOUND	Element not found.
S_OK	Success, "everything worked".
S_FALSE	Success but with additional results, "function worked and the result is false".

5.3.1.7 Numeric types

5.3.1.7.1 BCD

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|--------------|
| 1 | Data Type Numeric Identifier | = | 35 |
| 2 | Data Type Name | = | Unsigned8 |
| 3 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 1 |

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of one octet. In this type, the least significant four bits are used to express a BCD value which is between zero and nine inclusive. The most significant four bits are unused.

5.3.1.7.2 Floating Point types

5.3.1.7.2.1 Float32

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|--------------|
| 1 | Data Type Numeric Identifier | = | 8 |
| 2 | Data Type Name | = | Float32 |
| 4 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 4 |

This type has a length of four octets. The format for float32 is that defined by ANSI/IEEE 754 as single precision.

5.3.1.7.2.2 Floating point

This data type is the same as Float32.

5.3.1.7.2.3 REAL

This IEC 61131-3 type is the same as Float32.

5.3.1.7.2.4 float

This data type is the same as Float32.

5.3.1.7.2.5 Float64

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 15
2 Data Type Name	= Float64
3 Format	= FIXED LENGTH
4.1 Octet Length	= 8

This type has a length of eight octets. The format for float64 is that defined by ANSI/IEEE 754 as double precision.

5.3.1.7.2.6 LREAL

This IEC 61131-3 type is the same as Float64.

5.3.1.7.2.7 double

This data type is the same as Float64.

5.3.1.7.3 Integer types

5.3.1.7.3.1 Integer8

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 2
2 Data Type Name	= Integer8
3 Format	= FIXED LENGTH
4.1 Octet Length	= 1

This integer type is a two's complement binary number with a length of one octet.

5.3.1.7.3.2 SINT

This IEC 61131-3 type is the same as Integer8.

5.3.1.7.3.3 char

This data type is the same as Integer8.

5.3.1.7.3.4 Integer16

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 3
2 Data Type Name	= Integer16

3 Format = FIXED LENGTH

4.1 Octet Length = 2

This integer type is a two's complement binary number with a length of two octets.

5.3.1.7.3.5 INT

This IEC 61131-3 type is the same as Integer16.

5.3.1.7.3.6 short

This data type is the same as Integer16.

5.3.1.7.3.7 Integer32

CLASS: **Data Type**

ATTRIBUTES:

1	Data Type Numeric Identifier	=	4
2	Data Type Name	=	Integer32
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This integer type is a two's complement binary number with a length of four octets.

5.3.1.7.3.8 DINT

This IEC 61131-3 type is the same as Integer32.

5.3.1.7.3.9 long

This data type is the same as Integer32.

5.3.1.7.3.10 Integer64

CLASS: **Data Type**

ATTRIBUTES:

1	Data Type Numeric Identifier	=	55
2	Data Type Name	=	Integer64
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This integer type is a two's complement binary number with a length of eight octets.

5.3.1.7.3.11 LINT

This IEC 61131-3 type is the same as Integer64.

5.3.1.7.4 Unsigned types

5.3.1.7.4.1 Unsigned8

CLASS: **Data Type**

ATTRIBUTES:

1	Data Type Numeric Identifier	=	5
2	Data Type Name	=	Unsigned8
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	1

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of one octet.

5.3.1.7.4.2 USINT

This IEC 61131-3 type is the same as Unsigned8.

5.3.1.7.4.3 unsigned char

This data type is the same as Unsigned8.

5.3.1.7.4.4 Unsigned16

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 6
2 Data Type Name	= Unsigned16
3 Format	= FIXED LENGTH
4.1 Octet Length	= 2

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of two octets.

5.3.1.7.4.5 UINT

This IEC 61131-3 type is the same as Unsigned16.

5.3.1.7.4.6 unsigned short

This data type is the same as Unsigned16.

5.3.1.7.4.7 Unsigned32

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 7
2 Data Type Name	= Unsigned32
3 Format	= FIXED LENGTH
4.1 Octet Length	= 4

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of four octets.

5.3.1.7.4.8 UDINT

This IEC 61131-3 type is the same as Unsigned32.

5.3.1.7.4.9 unsigned long

This data type is the same as Unsigned32.

5.3.1.7.4.10 Unsigned64

CLASS:	Data Type
ATTRIBUTES:	
1 Data Type Numeric Identifier	= 56

2	Data Type Name	=	Unsigned64
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of eight octets.

5.3.1.7.4.11 ULINT

This IEC 61131-3 type is the same as Unsigned64.

5.3.1.8 OctetString character types

5.3.1.8.1 OctetString1

CLASS:	Data Type
ATTRIBUTES:	
1	Data Type Numeric Identifier = 30
2	Data Type Name = OctetString1
3	Format = FIXED LENGTH
4.1	Octet Length = 1

This type has a length of one octet.

5.3.1.8.2 OctetString2

CLASS:	Data Type
ATTRIBUTES:	
1	Data Type Numeric Identifier = 31
2	Data Type Name = OctetString2
3	Format = FIXED LENGTH
4.1	Octet Length = 2

This type has a length of two octets.

5.3.1.8.3 OctetString4

CLASS:	Data Type
ATTRIBUTES:	
1	Data Type Numeric Identifier = 32
2	Data Type Name = OctetString4
3	Format = FIXED LENGTH
4.1	Octet Length = 4

This type has a length of four octets.

5.3.1.8.4 OctetString8

CLASS:	Data Type
ATTRIBUTES:	
1	Data Type Numeric Identifier = 33
2	Data Type Name = OctetString8
3	Format = FIXED LENGTH
4.1	Octet Length = 8

This octet string type has a length of eight octets.

5.3.1.8.5 OctetString16

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|---------------|
| 1 | Data Type Numeric Identifier | = | 34 |
| 2 | Data Type Name | = | OctetString16 |
| 3 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 16 |

This type has a length of 16 octets.

5.3.1.8.6 UUID

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----------|--------------------------|---|----------------|
| 2 | Data Type Name | = | UUID |
| 3 | Format | = | STRUCTURE |
| 5.1 | Number of Fields | = | 4 |
| 5.2.1 | Field Name | = | Data1 |
| 5.2.2 | Field Data Type | = | unsigned long |
| 5.2.3 | Field Name | = | Data2 |
| 5.2.4 | Field Data Type | = | unsigned short |
| 5.2.5 | Field Name | = | Data3 |
| 5.2.6 | Field Data Type | = | unsigned short |
| 5.2.7 | Field Name | = | Data4 |
| 5.2.8.1 | Format | = | ARRAY |
| 5.2.8.4.1 | Number of Array Elements | = | 8 |
| 5.2.8.4.2 | Array Element Data Type | = | unsigned char |

This data type defines a 16 octet fixed length data type. The semantic is specified by the used ORPC model and beyond the scope of this standard.

This data type is structured as follows.

Data1

This field contains the first eight hexadecimal digits of the UUID.

Data2

This field contains the first group of four hexadecimal digits of the UUID.

Data3

This field contains the second group of four hexadecimal digits of the UUID.

Data4

This field contains an array of eight elements. The first two elements contain the third group of four hexadecimal digits of the UUID. the remaining six elements contain the final 12 hexadecimal digits of the UUID.

Predefined values for TYPE 10 items are shown in Table 8.

Table 8 – UUID

Value	Description
UUID_NULL	
UUID_IUnknown	Identifies the IUnknown interface uniquely.
UUID_IDispatch	Identifies the IDispatch interface uniquely.
UUID_ICBAPhysicalDevice	Identifies the ICBAPhysicalDevice interface uniquely.
UUID_ICBABrowse	Identifies the ICBABrowse interface uniquely.
UUID_ICBAPersist	Identifies the ICBAPersist interface uniquely.
UUID_ICBALogicalDevice	Identifies the ICBALogicalDevice interface uniquely.
UUID_ICBAState	Identifies the ICBAState interface uniquely.
UUID_ICBATime	Identifies the ICBATime interface uniquely.
UUID_ICBAGroupError	Identifies the ICBAGroupError interface uniquely.
UUID_ICBAAccoMgt	Identifies the ICBAAccoMgt interface uniquely.
UUID_ICBAAccoServer	Identifies the ICBAAccoServer interface uniquely.
UUID_ICBAAccoCallback	Identifies the ICBAAccoCallback interface uniquely.
UUID_ICBAAccoSync	Identifies the ICBAAccoSync interface uniquely.
UUID_ICBARTAuto	Identifies the ICBARTAuto interface uniquely.
UUID_PhysicalDevice	Identifies the Physical Device class uniquely.
UUID_LogicalDevice	Identifies the Logical Device class uniquely.
UUID_ACCO	Identifies the ACCO class uniquely.
UUID_RTAuto	Identifies the RTAuto class uniquely.

5.3.1.9 Pointer types**5.3.1.9.1 Interface pointer****CLASS:** Data Type**ATTRIBUTES:**

2	Data Type Name	=	Interface Pointer
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This data type defines a 4 octet fixed length data type.

5.3.1.9.2 LPWSTR**CLASS:** Data Type**ATTRIBUTES:**

2	Data Type Name	=	LPWSTR
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This data type defines a reference to an UnicodeString.

5.3.1.10 Time types**5.3.1.10.1 BinaryTime0****CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	40
2	Data Type Name	=	BinaryTime0
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of two octets. The unit of time for this type is 10 µs.

5.3.1.10.2 BinaryTime1

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	41
2	Data Type Name	=	BinaryTime1
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of two octets. The unit of time for this type is 100 μ s.

5.3.1.10.3 BinaryTime2

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	42
2	Data Type Name	=	BinaryTime2
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of two octets. The unit of time for this type is 1 ms.

5.3.1.10.4 BinaryTime3

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	43
2	Data Type Name	=	BinaryTime3
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of two octets. The unit of time for this type is 10 ms.

5.3.1.10.5 BinaryTime4

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	44
2	Data Type Name	=	BinaryTime4
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of four octets. The unit of time for this type is 10 μ s.

5.3.1.10.6 BinaryTime5

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	45
2	Data Type Name	=	BinaryTime5
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of four octets. The unit of time for this type is 100 μ s.

5.3.1.10.7 BinaryTime6

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	46
2	Data Type Name	=	BinaryTime6
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of four octets. The unit of time for this type is 1 ms.

5.3.1.10.8 BinaryTime7

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	47
2	Data Type Name	=	BinaryTime7
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of six octets. The unit of time for this type is 1 ms.

5.3.1.10.9 BinaryTime8

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	48
2	Data Type Name	=	BinaryTime8
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	6

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of six octets. The unit of time for this type is 10 μ s.

5.3.1.10.10 BinaryTime9

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	49
2	Data Type Name	=	BinaryTime9
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	6

This type is a binary number. The most significant bit of the most significant byte is always used as the most significant bit of the binary number; no sign bit is included. This binary time type has a length of six octets. The unit of time for this type is 100 μ s.

5.3.1.10.11 TIME

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	TIME
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This IEC 61131-3 type is a two's complement binary number with a length of four octets. The unit of time for this type is 1 ms.

5.3.1.10.12 ITIME

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	ITIME
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This IEC 61131-3 type extension is a two's complement binary number with a length of two octets. The unit of time for this type is 1 ms.

5.3.1.10.13 FTIME

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	FTIME
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	4

This IEC 61131-3 type extension is a two's complement binary number with a length of four octets. The unit of time for this type is 1 µs.

5.3.1.10.14 LTIME

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	LTIME
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This IEC 61131-3 type extension is a two's complement binary number with a length of eight octets. The unit of time for this type is 1 µs.

5.3.1.10.15 NetworkTime

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	58
2	Data Type Name	=	NetworkTime
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This data type is composed of an integer value and of an unsigned value that express the network time.

The first element is an Unsigned32 data type that provides the network time in seconds since 1900.01.01 00:00:00(UTC) for network time greater/equal 1984.01.01 00:00:00 (UTC) and

less than or 2036.07.02 06:28:16(UTC), or in seconds since 2036.07.02 06:28:16(UTC) for Network time greater or equal 2036.07.02 06:28:16(UTC).

The second element is an Unsigned32 data type that provides the fractional portion of seconds in $1/2^{32}$ s).

5.3.1.10.16 NetworkTimeDifference

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	59
2	Data Type Name	=	NetworkTimeDifference
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	8

This data type is composed of an integer value and of an unsigned value that express the difference in network time. The first element is an Integer32 data type that provides the network time difference in seconds. The second element is an Unsigned32 data type that provides the fractional portion of seconds in $1/2^{32}$ s.

5.3.1.11 VisibleString character types

5.3.1.11.1 UNICODE char

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	36
2	Data Type Name	=	UnicodeChar
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type is defined as a single character in the UNICODE string type.

5.3.1.11.2 VisibleString1

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	25
2	Data Type Name	=	VisibleString1
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	1

This type is defined as a single character in the ISO VisibleString type.

5.3.1.11.3 VisibleString2

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	26
2	Data Type Name	=	VisibleString2
3	Format	=	FIXED LENGTH
4.1	Octet Length	=	2

This type contains two elements of type VisibleString.

5.3.1.11.4 VisibleString4

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|----------------|
| 1 | Data Type Numeric Identifier | = | 27 |
| 2 | Data Type Name | = | VisibleString4 |
| 3 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 4 |

This type contains four elements of type VisibleString.

5.3.1.11.5 VisibleString8

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|----------------|
| 1 | Data Type Numeric Identifier | = | 28 |
| 2 | Data Type Name | = | VisibleString8 |
| 3 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 8 |

This type contains eight elements of type VisibleString.

5.3.1.11.6 VisibleString16

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|-----------------|
| 1 | Data Type Numeric Identifier | = | 29 |
| 2 | Data Type Name | = | VisibleString16 |
| 3 | Format | = | FIXED LENGTH |
| 4.1 | Octet Length | = | 16 |

This type contains 16 elements of type VisibleString.

5.3.2 String types

5.3.2.1 BitString

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|-----------|
| 1 | Data Type Numeric Identifier | = | 14 |
| 2 | Data Type Name | = | Bitstring |
| 3 | Format | = | STRING |
| 5.1 | Octet Length | = | 1 to n |

This string type is defined as a series of BitString8 elements.

5.3.2.2 CompactBooleanArray

CLASS: Data Type

ATTRIBUTES:

- | | | | |
|-----|------------------------------|---|---------------------|
| 1 | Data Type Numeric Identifier | = | 37 |
| 2 | Data Type Name | = | CompactBooleanArray |
| 3 | Format | = | STRING |
| 6.1 | Octet Length | = | 1 |

In this type, each bit value of 0 (zero) represents the Boolean value FALSE and each bit value of 1 represents the Boolean value TRUE.

5.3.2.3 CompactBCDArray

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	38
2	Data Type Name	=	CompactBCDArray
3	Format	=	STRING
4.1	Octet Length	=	1

This type is used to pack an ordered series of BCD values, two per octet, into an ordered series of octets. The first octet contains the most significant BCD value in its most significant quartet. If the number of BCD values is odd, the least significant quartet of the final octet is set to the reserved value "1111".

5.3.2.4 OctetString

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	10
2	Data Type Name	=	OctetString
3	Format	=	STRING
4.1	Octet Length	=	1 to n

An OctetString is an ordered sequence of octets, numbered from 1 to n. For the purposes of discussion, octet 1 of the sequence is referred to as the first octet. IEC 61158-6 defines the order of transmission.

5.3.2.5 UNICODEString

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	39
2	Data Type Name	=	UnicodeString
3	Format	=	STRING
4.1	Octet Length	=	2

This type is defined as the UNICODE string type.

5.3.2.6 VisibleString

CLASS: Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	9
2	Data Type Name	=	VisibleString
3	Format	=	STRING
4.1	Octet Length	=	1 to n

This type is defined as the ISO/IEC 646 string type.

5.3.3 Structure types

5.3.3.1 ADDCONNECTIONIN

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	ADDCONNECTIONIN
3	Format	=	STRUCTURE
5.1	Number of Fields	=	5
5.2.1	Field Name	=	ProviderItem
5.2.2	Field Data Type	=	LPWSTR
5.2.3	Field Name	=	ConsumerItem

5.2.4	Field Data Type =	LPWSTR
5.2.5	Field Name =	Persistence
5.2.6	Field Data Type =	PERSISTDEF
5.2.7	Field Name =	SubstituteValue
5.2.8	Field Data Type =	VARIANT
5.2.9	Field Name =	Epsilon
5.2.10	Field Data Type =	VARIANT

This data type defines the ADDCONNECTIONIN data type.

This data type is structured as follows.

ProviderItem

This field contains the name of the source data item.

ConsumerItem

This field contains the name of the sink data item.

Persistence

This field describes the required persistence of the connection information. The values CBAVolatile and CBAPersistent are allowed.

SubstituteValue

This field specifies the substitute value according the data type of the connection item.

Epsilon

This field specifies the hysteresis as absolute change of the value.

5.3.3.2 ADDCONNECTIONOUT

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name =	ADDCONNECTIONOUT
3	Format =	STRUCTURE
5.1	Number of Fields =	3
5.2.1	Field Name =	ConsumerID
5.2.2	Field Data Type =	unsigned long
5.2.3	Field Name =	Version
5.2.4	Field Data Type =	unsigned short
5.2.5	Field Name =	ErrorState
5.2.6	Field Data Type =	HRESULT

This data type defines the ADDCONNECTIONOUT data type.

This data type is structured as follows.

ConsumerID

This field contains the identifier of the consumer.

Version

This field contains the version number of the connection.

ErrorState

This field contains the error condition of the connection.

5.3.3.3 BSTR

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name =	BSTR
3	Format =	STRUCTURE
5.1	Number of Fields =	2
5.2	List of Fields	
5.2.1	Field Name =	ByteCount
5.2.2	Field Data Type =	unsigned long
5.2.3	Field Name =	UnicodeString
5.2.4	Field Data Type =	UnicodeString

This data type defines an UnicodeString with preceding byte count value. The count contains the number of octets, not UNICODE characters, in the string. This count does not include the terminating zero character (2 octets).

5.3.3.4 CONNECTIN**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	CONNECTIN
3	Format	=	STRUCTURE
5.1	Number of Fields	=	4
5.2	List of Fields		
5.2.1	Field Name	=	ProviderItem
5.2.2	Field Data Type	=	LPWSTR
5.2.3	Field Name	=	DataType
5.2.4	Field Data Type	=	VARTYPE
5.2.5	Field Name	=	Epsilon
5.2.6	Field Data Type	=	VARIANT
5.2.7	Field Name	=	ConsumerID
5.2.8	Field Data Type	=	unsigned long

This data type defines the CONNECTIN data type.

This data type is structured as follows.

ProviderItem

This field contains the name of the source data item.

DataType

This field contains the type code for the data item.

Epsilon

This field specifies the hysteresis as absolute change of the value.

ConsumerID

This field contains the identifier of the consumer.

5.3.3.5 CONNECTOUT**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	CONNECTOUT
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	ProviderID
5.2.2	Field Data Type	=	unsigned long
5.2.3	Field Name	=	ErrorState
5.2.4	Field Data Type	=	HRESULT

This data type defines the CONNECTOUT data type.

This data type is structured as follows.

ProviderID

This field contains the identifier of the provider.

ErrorState

This field contains the error condition of the interconnection.

5.3.3.6 EXCEPTINFO**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	EXCEPTINFO
3	Format	=	STRUCTURE
5.1	Number of Fields	=	9
5.2.1	Field Name	=	wCode
5.2.2	Field Data Type	=	unsigned short
5.2.3	Field Name	=	wReserved
5.2.4	Field Data Type	=	unsigned short
5.2.5	Field Name	=	bstrSource
5.2.6	Field Data Type	=	BSTR
5.2.7	Field Name	=	bstrDescription
5.2.8	Field Data Type	=	BSTR
5.2.9	Field Name	=	bstrHelpFile

5.2.10	Field Data Type =	BSTR
5.2.11	Field Name =	dwHelpContext
5.2.12	Field Data Type =	unsigned long
5.2.13	Field Name =	pvReserved
5.2.14	Field Data Type =	unsigned long
5.2.15	Field Name =	pfnDeferredFillIn
5.2.16	Field Data Type =	unsigned long
5.2.17	Field Name =	scode
5.2.18	Field Data Type =	long

This data type defines the EXCEPINFO data type to describe an exception that occurred during the Invoke service.

This data type is structured as follows.

wCode

This field contains an error code identifying the error. Error codes should be greater than 1000. Either this field or the scode field shall be filled in; the other shall be set to 0.

wReserved

This field is reserved and should be set to 0.

bstrSource

This field contains a textual, human-readable name of the source of the exception. Typically, this is an application name. This field should be filled in by the implementor of the IDispatch interface.

bstrDescription

This field contains a textual, human-readable description of the error intended for the customer. If no description is available, use Null.

bstrHelpFile

This field contains the fully qualified drive, path, and file name of a Help file with more information about the error. If no Help is available, use Null.

dwHelpContext

This field contains the Help context ID of the topic within the Help file. This field should be filled in if and only if the bstrHelpFile field is not Null.

pvReserved

This field is reserved and shall be set to Null.

pfnDeferredFillIn

This field contains the address of a function that takes an EXCEPINFO structure as an argument and returns an HRESULT value. If deferred, fill-in is not desired, this field should be set to Null.

scode

This field contains a return value describing the error. Either this field or the wCode field shall be filled in; the other shall be set to 0.

5.3.3.7 DATE_AND_TIME

CLASS:

Data Type

ATTRIBUTES:

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	DATE_AND_TIME
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2.1	Field Name	=	Time_Of_Day_Element
5.2.2	Field Data Type	=	TIME_OF_DAY
5.3.1	Field Name	=	Date_Element
5.3.2	Field Data Type	=	DATE

This IEC 61131-3 type extension is a structure which expresses both the date (as a number of days starting from January 1st, 1972 until June 6th, 2151), and the time of day as a number of ms starting from midnight.

5.3.3.8 FILETIME**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	FILETIME
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	LowDateTime
5.2.2	Field Data Type	=	unsigned long
5.2.3	Field Name	=	HighDateTime
5.2.4	Field Data Type	=	unsigned long

This data type is a 64 bit time value representing the number of 100-nanosecond intervals since January 1, 1601.

5.3.3.9 GETIDOUT**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	GETIDOUT
3	Format	=	STRUCTURE
5.1	Number of Fields	=	4
5.2.1	Field Name	=	ConsumerID
5.2.2	Field Data Type	=	unsigned long
5.2.3	Field Name	=	State
5.2.4	Field Data Type	=	VT_BOOLEAN
5.2.5	Field Name	=	Version
5.2.6	Field Data Type	=	unsigned short
5.2.7	Field Name	=	ErrorState
5.2.8	Field Data Type	=	HRESULT

This data type defines the GETIDOUT data type.

This data type is structured as follows.

ConsumerID

This field contains the identifier of the consumer.

State

This field contains the state of the connection. The value TRUE indicates an active and the value FALSE an inactive connection.

Version

This field contains the version number of the connection.

ErrorState

This field contains the error condition of the connection.

5.3.3.10 GETCONNECTIONOUT**CLASS: Data Type****ATTRIBUTES:**

2	Data Type Name	=	GETCONNECTIONOUT
3	Format	=	STRUCTURE
5.1	Number of Fields	=	10
5.2.1	Field Name	=	Provider
5.2.2	Field Data Type	=	LPWSTR
5.2.3	Field Name	=	ProviderItem
5.2.4	Field Data Type	=	LPWSTR
5.2.5	Field Name	=	ConsumerItem
5.2.6	Field Data Type	=	LPWSTR
5.2.7	Field Name	=	SubstituteValue
5.2.8	Field Data Type	=	VARIANT
5.2.9	Field Name	=	Epsilon
5.2.10	Field Data Type	=	VARIANT
5.2.11	Field Name	=	QoSType
5.2.12	Field Data Type	=	unsigned short
5.2.13	Field Name	=	QoSValue
5.2.14	Field Data Type	=	unsigned short
5.2.15	Field Name	=	State

5.2.16	Field Data Type =	VT_BOOLEAN
5.2.17	Field Name =	Persistence
5.2.18	Field Data Type =	PERSISTDEF
5.2.19	Field Name =	Version
5.2.20	Field Data Type =	unsigned short
5.2.21	Field Name =	ErrorState
5.2.22	Field Data Type =	HRESULT

This data type defines the GETCONNECTIONOUT data type.

This data type is structured as follows.

Provider

This field contains the name of the providers LDev (Format: PDev!LDev).

ProviderItem

This field contains the name of the source data item.

ConsumerItem

This field contains the name of the sink data item.

SubstituteValue

This field specifies the substitute value according the data type of the connection item.

Epsilon

This field specifies the hysteresis as absolute change of the value.

QoSType

This field contains the quality of service type.

QoSValue

This field contains the quality of service qualifier.

State

This field contains the state of the connection. The value TRUE indicates an active and the value FALSE an inactive connection.

Persistence

This field describes the required persistence of the connection information. The values CBAVolatile and CBAPersistent are allowed.

Version

This field contains the version number of the connection.

ErrorState

This field contains the error condition of the connection.

5.3.3.11 QualifiedOctetString2

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	QualifiedOctetString2
5	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	Value
5.2.2	Field Data Type	=	Octet String(2)
5.2.3	Field Name	=	Status
5.2.4	Field Data Type	=	Unsigned8

This data type defines the QualifiedOctetString2.

This data type is structured as follows.

Value

This field contains the value as Octet String with the length of 2.

Status

This field describes the status of the value as a qualifier.

5.3.3.12 QualifiedFloat32

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	QualifiedFloat32
5	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	Value
5.2.2	Field Data Type	=	Float32
5.2.3	Field Name	=	Status
5.2.4	Field Data Type	=	Unsigned8

This data type defines the QualifiedFloat32.

This data type is structured as follows.

Value

This field contains the value as Float32.

Status

This field describes the status of the value as a qualifier.

5.3.3.13 QualifiedUnsigned8

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	QualifiedUnsigned8
5	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	Value
5.2.2	Field Data Type	=	Unsigned8
5.2.3	Field Name	=	Status
5.2.4	Field Data Type	=	Unsigned8

This data type defines the QualifiedUnsigned8.

This data type is structured as follows.

Value

This field contains the value as Unsigned8.

Status

This field describes the status of the value as a qualifier.

5.3.3.14 READITEMOUT

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	READITEMOUT
3	Format	=	STRUCTURE
5.1	Number of Fields	=	4
5.2.1	Field Name	=	Value
5.3.2	Field Data Type	=	VARIANT
5.4.3	Field Name	=	QualityCode
5.5.4	Field Data Type	=	ITEMQUALITYDEF
5.6.5	Field Name	=	TimeStamp
5.7.6	Field Data Type	=	FILETIME
5.8.7	Field Name	=	ErrorState
5.9.8	Field Data Type	=	HRESULT

This data type defines the READITEMOUT data type.

This data type is structured as follows.

Value

This field contains the value itself with the format according to the appropriate data type.

QualityCode

This field contains the Quality Code of the data value.

TimeStamp

This field contains the time stamp of the value.

ErrorState

This field contains the error condition of the connection.

5.3.3.15 SAFEARRAY

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	RGSABOUND
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2	List of Fields		
5.2.1	Field Name	=	Elements
5.2.2	Field Data Type	=	unsigned long
5.2.3	Field Name	=	Left Bound
5.2.4	Field Data Type	=	long

This data type is a helper describing one array element with boundary information needed to define the SAFEARRAY below.

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	SAFEARRAY
3	Format	=	STRUCTURE
5.1	Number of Fields	=	6
5.2	List of Fields		
5.2.1	Field Name	=	Dims
5.2.2	Field Data Type	=	unsigned short
5.2.3	Field Name	=	Features
5.2.4	Field Data Type	=	unsigned short
5.2.5	Field Name	=	Elements
5.2.6	Field Data Type	=	unsigned long
5.2.7	Field Name	=	Locks
5.2.8	Field Data Type	=	unsigned long
5.2.9	Field Name	=	Data Address
5.2.10	Field Data Type	=	unsigned long
5.2.11	Field Name	=	Rgsabound
5.2.12	Format	=	ARRAY
5.2.12.1	Number of Array Elements	=	Dims
5.2.12.2	Array Element Data Type	=	RGSABOUND

This data type expresses a one- or multi-dimensional array of a single data type. (However, this single data type can be a VARIANT, allowing you arrays of mixed types.) The reason these arrays are called safe arrays is because they contain bounds information, allowing to check the index or indices against the bounds before accessing the data in the array. The lower bound of the array does not have to be zero, so the safe array has to store its lower bound as well as the size. Finally, safe arrays allow locking (and unlocking) so it can be sure the pointer to the data you get is valid.

This data type is structured as follows.

Dims

This field contains the dimension of the SAFEARRAY.

Features

This field contains flags for allocation type and data type of the SAFEARRAY.

Elements

This field contains the size of a single element of the SAFEARRAY.

Locks

This field contains the lock counter of the SAFEARRAY.

Data Address

This field contains the address of the actual data of the SAFEARRAY.

Rgsabound

This field contains an array with the information about boundaries for each dimension of the SAFEARRAY. Therefore, the number of array elements is according to the dimension of the safe array

5.3.3.16 SHORT_STRING**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	SHORT_STRING
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2.1	Field Name	=	Charcount_Element
5.2.2	Field Data Type	=	USINT
5.3.1	Field Name	=	Stringcontents_Element
5.3.2	Field Data Type	=	OctetString

This IEC 61131-3 type extension is composed of two elements. Charcount_Element gives the current number of characters in the Stringcontents_Element (one USINT per character).

5.3.3.17 STRING**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	STRING
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2.1	Field Name	=	Charcount_Element
5.2.2	Field Data Type	=	UINT
5.3.1	Field Name	=	Stringcontents_Element
5.3.2	Field Data Type	=	OctetString

This IEC 61131-3 type is composed of two elements. Charcount_Element gives the current number of characters in the Stringcontents_Element (one USINT per character).

5.3.3.18 STRING2**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	STRING2
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2.1	Field Name	=	Charcount_Element
5.2.2	Field Data Type	=	UINT
5.3.1	Field Name	=	String2contents_Element
5.3.2	Field Data Type	=	OctetString

This IEC 61131-3 data type extension is composed of two elements. Charcount_Element gives the current number of characters in the String2contents_Element (one UINT per character).

5.3.3.19 STRINGN**CLASS:** Data Type**ATTRIBUTES:**

1	Data Type Numeric Identifier	=	not used
2	Data Type Name	=	STRINGN
3	Format	=	STRUCTURE
5.1	Number of Fields	=	3
5.2.1	Field Name	=	Charsize_Element

5.2.2	Field Data Type	=	UINT
5.3.1	Field Name	=	Charcount_Element
5.3.2	Field Data Type	=	UINT
5.4.1	Field Name	=	StringNcontents_Element
5.4.2	Field Data Type	=	OctetString

This IEC 61131-3 type extension is composed of three elements. Charsize_Element gives the size of a character in StringNcontents_Element (N = number of USINT). Charcount_Element gives the current number of characters in the StringNcontents_Element (N USINT per character).

5.3.3.20 VARIANT

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	VARIANT
3	Format	=	STRUCTURE
5.1	Number of Fields	=	5
5.2	List of Fields		
5.2.1	Field Name	=	Tag
5.2.2	Field Data Type	=	VARTYPE
5.2.3	Field Name	=	Padding1
5.2.4	Field Data Type	=	unsigned short
5.2.5	Field Name	=	Padding2
5.2.6	Field Data Type	=	unsigned short
5.2.7	Field Name	=	Padding3
5.2.8	Field Data Type	=	unsigned short
5.2.9	Field Name	=	Value
5.2.10	Field Data Type	=	see Table 9

This data type expresses a VARIANT that containing possible values of data types according to Table 9. The Tag field contains the numeric identifier of the data type to identify the data type of the value. The overall size of the VARIANT is 16 octets.

Table 9 – Data type names for value

Data Type
VT_BOOLEAN
char
short
long
unsigned char
unsigned short
unsigned long
float
double
date
currency
Address of a BSTR
Address of a SAFEARRAY
Address of a userdefined struct
Interface Pointer
HRESULT

5.3.3.21 WRITEITEMIN

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	WRITEITEMIN
3	Format	=	STRUCTURE
5.1	Number of Fields	=	2
5.2.1	Field Name	=	Item
5.2.2	Field Data Type	=	LPWSTR
5.2.3	Field Name	=	Value
5.2.4	Field Data Type	=	VARIANT

This data type defines the WRITEITEMIN data type.

This data type is structured as follows.

Item

This field contains the name of the data item.

Value

This field contains the value itself with the format according to the appropriate data type.

5.3.3.22 WRITEITEMQCDIN

CLASS: Data Type

ATTRIBUTES:

2	Data Type Name	=	WRITEITEMQCDIN
3	Format	=	STRUCTURE
5.1	Number of Fields	=	3
5.2.1	Field Name	=	WriteItem
5.2.2	Field Data Type	=	WRITEITEMIN
5.2.3	Field Name	=	QualityCode
5.2.4	Field Data Type	=	ITEMQUALITYDEF
5.2.5	Field Name	=	TimeStamp
5.2.6	Field Data Type	=	FILETIME

This data type defines the WRITEITEMQCDIN data type.

This data type is structured as follows.

WriteItem

This field contains the name and value of the data item.

QualityCode

This field contains the Quality Code of the data value.

TimeStamp

This field contains the time stamp of the value.

5.3.3.23 DISPPARAMS

2	Data Type Name	=	UUID
3	Format	=	STRUCTURE
5.1	Number of Fields	=	4
5.2.1	Field Name	=	Data1
5.2.2	Field Data Type	=	unsigned long
5.2.3	Field Name	=	Data2
5.2.4	Field Data Type	=	unsigned short
5.2.5	Field Name	=	Data3
5.2.6	Field Data Type	=	unsigned short
5.2.7	Field Name	=	Data4
5.2.8.1	Format	=	ARRAY
5.2.8.4.1	Number of Array Elements	=	8
5.2.8.4.2	Array Element Data Type	=	unsigned char

This data type defines a 16 octet fixed length data type. The semantic is specified by the used ORPC model and beyond the scope of this standard.

This data type is structured as follows.

Data1

This field contains the first eight hexadecimal digits of the UUID.

Data2

This field contains the first group of four hexadecimal digits of the UUID.

Data3

This field contains the second group of four hexadecimal digits of the UUID.

Data4

This field contains an array of eight elements. The first two elements contain the third group of four hexadecimal digits of the UUID. the remaining six elements contain the final 12 hexadecimal digits of the UUID.

Predefined values for TYPE 10 items are shown in Table 10.

Table 10 – UUID

Value	Description
UUID_NULL	
UUID_IUnknown	Identifies the IUnknown interface uniquely.
UUID_IDispatch	Identifies the IDispatch interface uniquely.
UUID_ICBAPhysicalDevice	Identifies the ICBAPhysicalDevice interface uniquely.
UUID_ICBABrowse	Identifies the ICBABrowse interface uniquely.
UUID_ICBAPersist	Identifies the ICBAPersist interface uniquely.
UUID_ICBALogicalDevice	Identifies the ICBALogicalDevice interface uniquely.
UUID_ICBAState	Identifies the ICBAState interface uniquely.
UUID_ICBATime	Identifies the ICBATime interface uniquely.
UUID_ICBAGroupError	Identifies the ICBAGroupError interface uniquely.
UUID_ICBAAccoMgt	Identifies the ICBAAccoMgt interface uniquely.
UUID_ICBAAccoServer	Identifies the ICBAAccoServer interface uniquely.
UUID_ICBAAccoCallback	Identifies the ICBAAccoCallback interface uniquely.
UUID_ICBAAccoSync	Identifies the ICBAAccoSync interface uniquely.
UUID_ICBARTAuto	Identifies the ICBARTAuto interface uniquely.
UUID_PhysicalDevice	Identifies the Physical Device class uniquely.
UUID_LogicalDevice	Identifies the Logical Device class uniquely.
UUID_ACCO	Identifies the ACCO class uniquely.
UUID_RTAuto	Identifies the RTAuto class uniquely.

5.4 Data type ASE service specification

There are no operational services defined for the type object.

6 Type 1 communication model specification

6.1 Concepts

There are no additional concepts.

6.2 ASEs

6.2.1 Object management ASE

6.2.1.1 Overview

The FAL ASE Models each specify one or more related FAL APO classes as a collection of attributes and services. The FAL services defined for a class are used to manipulate, control, or access APOs of the class. The services which are supported by a specific APO are specified by the ListOfManagementServices attribute of the APO.

To dynamically create or delete the network view of an APO, or to access its attribute values, this FAL ASE defines common FAL APO management services. Each APO Class definition specifies which of these services may be included in the definitions of APOs of the class. These services operate using the client/server model.

6.2.1.2 FAL management model specification

The management services specified by the FAL Management Model operate on FAL APOs and their attributes. FAL APO classes are defined within the FAL ASEs that are specified in the remaining clauses of this standard. Each FAL APO Class defined is described by a set of attributes and a set of services.

6.2.1.3 FAL management model services

6.2.1.3.1 Supported services

Management services are defined for managing APs and APOs. For simplicity APs and APOs are referred to as objects in the remainder of this subclause.

Create and Delete services are specified for a class if the network view of an object of the class can be dynamically created or deleted. Get Attributes and Set Attributes services are specified to indicate when its attributes can be read or written. The attributes of an APO which can be set are specified as part of the attribute definition.

Because unexpected changes to the object definitions within an AP can affect operation of the AP, the Begin Set Attributes and End Set Attributes services are defined. The Begin Set Attributes service is used to request the remote AP for permission to change its object definitions. The End Set Attributes service is used to indicate to the remote AP that changes to its object definitions are complete and that it can resume normal operation.

6.2.1.3.2 Create service

6.2.1.3.2.1 Service overview

This confirmed service is used to make an object visible and accessible across the network. As part of the service, initial values for the object's attributes may be specified. If all attribute values to be initialized cannot be conveyed in a single APDU, then the Set Attributes service can be used once the object has been created for attributes that may be updated using the Set Attributes service. The scope of visibility for created objects is Application Process specific.

The List of Supported Attributes parameter is used to indicate to the requester the attributes which were actually created for the object. If initial values were supplied for attributes that were not created, the List of Supported Attributes can be used to determine the ones which were not created and initialized according to the request. This capability represents a form of negotiation when creating an object.

NOTE Whether or not an AP actually creates the object is a local matter. The intent of the service is to request that it be prepared for remote access.

6.2.1.3.2.2 Service parameters

The service parameters for this service are shown in Table 11.

Table 11 – Create service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
FAL ASE / FAL Class	M	M(=)		
List of Attribute IDs and Initial Values	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Numeric ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

List of Attribute IDs and Initial Values

This parameter identifies and specifies the initial value of one or more of the object's attributes to be set during the creation process. At least one of the object's key attributes shall be present. Values for mandatory and optional attributes may be supplied. The AP that creates the object determines the complement of attributes supported for the object.

NOTE Initial values are only required for mandatory attributes if the AP creating the object does not have the ability to supply an initial value on its own. How it acquires these values on its own is beyond the scope of this standard.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.1.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.1.3.3 Delete service

6.2.1.3.3.1 Service overview

This confirmed service is used to make an object that is currently visible and accessible not visible and not accessible across the network.

NOTE Whether or not the real object is actually deleted or simply removed from network accessibility is a local matter.

6.2.1.3.3.2 Service parameters

The service parameters for this service are shown in Table 12.

Table 12 – Delete service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
FAL ASE / FAL Class	M	M(=)		
Key Attribute	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

FAL ASE/FAL Class

This parameter specifies the FAL ASE (AP, AR, Variable, Data Type, Event, Function Invocation, Load Region) and the FAL Class within the ASE (e.g. AREP, Variable List, Notifier, Action).

Key Attribute

This parameter identifies the object by one of its key attributes.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.1.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

Once deleted, an object is no longer accessible using the services of the FAL.

6.2.1.3.4 Get attributes service**6.2.1.3.4.1 Service overview**

This confirmed service is used to read the current values of a list of attributes for one or more objects of one or more classes defined for the AP. It operates in a "best effort" fashion, such that the service succeeds if the value for at least one requested attribute for one requested object is returned.

6.2.1.3.4.2 Service parameters

The service parameters for this service are shown in Table 13.

Table 13 – Get Attributes service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
List of Objects and their Attributes	S	S(=)		
Key Attribute	M	M(=)		
List of Requested Attributes	M	M(=)		
Data Type Requested	C	C(=)		
Range of Objects and their Attributes	S	S(=)		
Starting Numeric ID	M	M(=)		
List of Requested Attributes	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
More			M	M(=)
List of Responses			M	M(=)
List of Supported Attributes			C	C(=)
Error Status			S	S(=)
List of Attribute Values			S	S(=)
Data Type Description			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

List of Objects and their Attributes

This selection type parameter provides a list that identifies each object, its class, and the attributes that are being requested. Three values for identifying attributes are possible, all attributes, mandatory attributes only, or a list of individual attributes. It is present if the user is requesting attributes for a list of objects.

Key Attribute

This parameter identifies the object for which attributes are being requested using one of the object's key attributes.

List of Requested Attributes

This parameter identifies one or more of the attributes for which values are being requested.

Range of Objects and their Attributes

This selection type parameter provides a range of numeric identifiers for objects of a single class, and optionally for a series of objects of different classes, and the attributes that are being requested. Three values for identifying attributes are possible, all attributes, mandatory

attributes only, or a list of individual attributes. It is present if the user is requesting attributes for objects within a given range of numeric ids.

Starting Numeric ID

This parameter specifies the first in a range of numeric ids. The attributes being requested are for all objects beginning with and after this numeric id. The value ALL may be used to indicate that attributes for all objects for the specified class are to be returned.

NOTE Multiple requests may have to be issued with successively higher numeric ids when not all the desired object definitions can be returned in a single response.

List of Requested Attributes

This parameter identifies one or more of the attributes for which values are being requested.

Data type Requested

This conditional parameter is present only when the attributes of a variable object are being requested. It indicates whether the data type description is to be returned with requested variable attributes.

Result(+)

This selection type parameter indicates that the service request succeeded.

More

This Boolean parameter indicates, when FALSE, that responses for all requested attributes are being returned. If the value is TRUE, then only responses for a proper subset of the attributes are being returned. This situation will occur only when not all values will fit into a single response APDU. When it does occur, it is necessary for the user to submit an additional request for the missing attributes. Partial values for attributes are never returned.

List of Responses

This parameter contains a status indicator or a list of attribute values for each object specified in the request. The responses in the list are returned in the same order that the objects were specified in the request. The object associated with each response is only explicitly identified if the value of one of its key attributes was requested. If all responses could not be returned in a single APDU, the More parameter is set.

List of Supported Attributes

This parameter identifies the attributes supported by the responder for the specified object. It is present if the specified object exists.

Error Status

This parameter is returned if the get failed for all of the attributes of an object. It indicates the most probable reason why the operation failed using the error class and error code defined for the Get Attributes service.

List of Attribute Values

This parameter is returned if the Get succeeded for any one or the requested attributes. This parameter contains a list of attribute values, each individually identified and complete, for the object requested.

Data Type Description

This conditional parameter is present only when data type description for a variable object has been requested. It contains the complete data type description for the variable.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.1.3.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Get Attributes Service is a "best effort". Best effort means that the service succeeds if at least one value is returned. If the user is able to get one or more of the specified attribute values, and if they can be conveyed in a single APDU, they are returned in a Get Attributes Service response (+) primitive.

If they cannot be conveyed in a single APDU, the user returns those that can and sets the more flag in the Get Attributes Service response (+) primitive. The requester is then responsible for submitting additional requests identifying the remaining attributes to be retrieved.

If the user is unable to get the value for at least one attribute for one object in the list, the service fails and the user issues a Get Attributes Service response (-) primitive indicating the reason.

6.2.1.3.5 Set attributes service

6.2.1.3.5.1 Service overview

This confirmed service is used to update the current value of a list of attributes for one or more objects. This service operates in an all or nothing fashion — if all the requested attributes for all the objects cannot be set, the service fails.

NOTE This service may be used to load entire object definitions by indicating that all mandatory attributes are to be updated.

6.2.1.3.5.2 Service parameters

The service parameters for this service are shown in Table 14.

Table 14 – Set Attributes service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
List of Object Definitions	M	M(=)		
FAL ASE / FAL Class	M	M(=)		
Key Attribute	M	M(=)		
List of Attribute Updates	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
List of Supported Attributes			C	C(=)

Argument

The argument contains the parameters of the service request.

List of Object Updates

This parameter contains the list of objects and their attributes that are to be updated.

FAL ASE/FAL Class

This parameter specifies the FAL ASE (AP, AR, Variable, Data Type, Event, Function Invocation, Load Region) and the FAL Class within the ASE (e.g. AREP, Variable List, Notifier, Action).

Key Attribute

This parameter identifies the object by one of its key attributes.

List of Attribute Updates

This parameter identifies one or more of the attributes that are to be updated and their values.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

List of Supported Attributes

This conditional parameter identifies the attributes that are supported by the object. It is present when a request was received to update unsupported attributes.

6.2.1.3.5.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Set Attributes Service is an "all or nothing" service. All or nothing means that the service succeeds only if all the requested updates succeed.

6.2.1.3.6 Begin set attributes service**6.2.1.3.6.1 Service overview**

The Begin Set Attributes prepares the AP for updates to its object definitions, such that the updates are free of interaction or not free of interaction. The AP returns a successful response to indicate to the requester that it may begin updating object definitions.

6.2.1.3.6.2 Service parameters

The service parameters for this service are shown in Table 15.

Table 15 – Begin Set Attributes

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Consequence	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)

Argument

The argument contains the parameters of the service request.

Consequence

This parameter states whether the intended modifications are free of interaction for the other communication partners, not free of interaction with updates to individual object definitions, or not free of interaction with completely new load of all object definitions.

loading free of interaction

reload, not free of interaction

new load, not free of interaction

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.1.3.6.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.1.3.7 End set attributes service

6.2.1.3.7.1 Service overview

The End Set Attributes service terminates the process of updating object definitions. It is used to indicate to the AP that all object definition updates have been completed.

6.2.1.3.7.2 Service parameters

The service parameters for this service are shown in Table 16.

Table 16 – End Set Attributes

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Numeric ID			C	C(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Numeric ID

This parameter gives the Numeric ID of the object, for which the generation was not successful.

6.2.1.3.7.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.2 Application process ASE

6.2.2.1 Overview

This standard models a fieldbus Application Process (AP). Fieldbus Application Processes represent the information and processing resources of a system that can be accessed through FAL services.

The Application Service Element in the FAL that provides these services is called an Application Process ASE. In the AP ASE, the AP is modeled and accessed as an APO with a standardized and predefined identifier.

6.2.2.2 AP class specification

6.2.2.2.1 Formal model

The AP class specifies the attributes and services defined for application processes. Its parent class "top" indicates the top of the FAL class tree. The numeric identifier for the AP Object is always 0. The use of this reserved value permits management services to be used for AP objects without knowing their names.

FAL ASE: **AP ASE**

CLASS: AP

CLASS ID: 16

PARENT CLASS: TOP

IDENTIFY, GET STATUS, and STATUS NOTIFICATION SERVICE ATTRIBUTES:

1	(m)	Attribute:	List Of AP Service Attributes
1.1	(m)	Attribute:	Manufacturer Identifier
1.1.1	(m)	Attribute:	Vendor Name
1.1.2	(m)	Attribute:	Model Identifier
1.1.3	(m)	Attribute:	Vendor Revision
1.1.4	(o)	Attribute:	Serial Number
1.2	(o)	Attribute:	AP Descriptor Reference
1.3	(m)	Attribute:	Network Access State
1.3.1	(m)	Attribute:	Service Access Status
1.3.2	(m)	Attribute:	Operational Status

SYSTEM MANAGEMENT ATTRIBUTES:

2	(m)	Attribute:	List Of System Management Attributes
2.1	(m)	Attribute:	List Of AR Endpoint Info
2.1.1	(m)	Attribute:	Numeric Id
2.1.2	(m)	Attribute:	Configured Max FAL PDU Size Sending
2.1.3	(m)	Attribute:	Configured Max FAL PDU Size Receiving
2.1.4	(m)	Attribute:	Configured Max Outstanding Services Requesting
2.1.5	(m)	Attribute:	Configured Max Outstanding Services Responding
2.1.6	(m)	Attribute:	List of Supported Services
2.1.7	(o)	Attribute:	Configured Max Data Structure Nesting Level
2.2	(m)	Attribute:	List Of In Use AR Endpoint Info
2.2.1	(m)	Attribute:	Context State
2.2.2	(m)	Attribute:	Negotiated Max FAL PDU Size Sending
2.2.3	(m)	Attribute:	Negotiated Max FAL PDU Size Receiving
2.2.4	(m)	Attribute:	Negotiated Max Outstanding Services Requesting
2.2.5	(m)	Attribute:	Negotiated Max Outstanding Services Responding
2.2.6	(m)	Attribute:	Outstanding Services Requesting Counter
2.2.7	(m)	Attribute:	Outstanding Services Responding Counter
2.2.8	(o)	Attribute:	Negotiated Max Data Structure Nesting Level
2.2.9	(o)	Attribute:	Negotiated List of Supported Services

OBJECT MANAGEMENT ATTRIBUTES:

3	(m)	Attribute:	List Of Managed AP Attributes
3.1	(m)	Attribute:	List Of Supported APO Classes and Objects
3.1.1	(m)	Attribute:	List Header
3.1.1.1	(o)	Attribute:	Numeric Identifier = 0
3.1.1.2	(m)	Attribute:	ROM / RAM Flag (TRUE, FALSE)
3.1.1.3	(m)	Attribute:	Max Name Length
3.1.1.4	(m)	Attribute:	Access Protection Supported (TRUE, FALSE)
3.1.1.5	(m)	Attribute:	Version Of List
3.1.1.6	(m)	Attribute:	List Local Reference
3.1.2	(c)	Constraint	Data Type Supported
3.1.2.1	(o)	Attribute:	Numeric Identifier = 1
3.1.2.2	(m)	Attribute:	Number Of Data Type Objects
3.1.2.3	(m)	Attribute:	Local Reference
3.1.3	(c)	Constraint	Variable Load Region Event Supported
3.1.3.1	(o)	Attribute:	Static Object Numeric Identifier

3.1.3.2	(m)	Attribute:	Number Of Static Objects (Variables + Load Region + Events)
3.1.3.3	(m)	Attribute:	Static Object Local Reference
3.1.4	(c)	Constraint	Variable List Supported
3.1.4.1	(o)	Attribute:	Numeric Identifier
3.1.4.2	(m)	Attribute:	Number Of Variable List Objects
3.1.4.3	(m)	Attribute:	Variable List Local Reference
3.1.5	(c)	Constraint	Function Invocation Supported
3.1.5.1	(o)	Attribute:	Function Invocation Numeric Identifier
3.1.5.2	(m)	Attribute:	Number Of Function Invocation Objects
3.1.5.3	(m)	Attribute:	Function Invocation Local Reference
3.1.6	(c)	Constraint	Dynamic Load Region
3.1.6.1	(o)	Attribute:	Load Region Numeric Identifier
3.1.6.2	(m)	Attribute:	Number of Load Region Objects
3.1.6.3	(m)	Attribute:	Load Region Local Reference
3.2	(o)	Attribute:	List Of DLSAP Addresses
3.3	(o)	Attribute:	List of Event Summary Selection Criteria

SUPPORTED ATTRIBUTES:

5	(o)	Attribute:	List Of Supported Attributes
---	-----	------------	------------------------------

SERVICES:

1	(o)	Ops Service:	Subscribe
2	(o)	Ops Service:	Identify
3	(o)	Ops Service:	Get Status
4	(o)	Ops Service:	Status Notification
5	(o)	Ops Service:	Initiate
6	(o)	Ops Service:	Terminate
7	(o)	Ops Service:	Conclude
8	(o)	Ops Service:	Reject

6.2.2.2.2 Identify, get status, and status notification service attributes**List Of AP Service Attributes**

The following attributes are accessible using the Identify, Get Status, and Status Notification Services. They are not otherwise accessible.

Manufacturer Identifier

The Manufacturer Identifier is composed of the Vendor Name, Model Identifier, Vendor Revision, and optionally, the Serial Number of the AP. These attributes may be read using the Identify service.

Vendor Name

This attribute contains the name of the manufacturer of the AP.

Model Identifier

This attribute specifies the model of the AP.

Vendor Revision

This attribute specifies the revision level of the AP as defined by the manufacturer.

Serial Number

This optional attribute specifies the serial number of the AP.

NOTE The serial number may be used as a qualifier to uniquely identify an AP.

AP Descriptor Reference

This attribute is a reference to a generic description of the AP. The descriptor is an identifier for the characteristics of the AP independent of its use. Two APs, therefore, may share the same generic description. The value, and the permissible set of values for this attribute is user defined. This attribute is used in the Initiate service.

Network Access State

The Network Access State attribute defines the ability of the AP to communicate. Two components are specified in this standard, Service Access Status and Operational Status. These attributes may be read using the Get Status service. The AP may choose to report them without being requested using the Status Notification service.

Service Access Status

This attribute contains information about the state of the communication capabilities of the AP.

READY FOR COMMUNICATION All services may be used normally.

LIMITED NUMBER OF SERVICES Limited number of services may be supported in some cases (e.g. for small devices).

LOADING-NON-INTERACTING Object Definitions are in the process of being loaded locally. Therefore, the Begin Set Attributes service may not be used until the local load is complete and the state has been changed.

LOADING-INTERACTING Object Definitions are in the process of being loaded over the network (using the Set Attributes service). On the AR used for the loading, only the following services should be used:

Abort	Get Attributes
Status	Set Attributes
Identify	End Set Attributes

OperationalStatus

This attribute gives the operational state of the real AP, as follows.

OPERATIONAL

PARTIALLY OPERATIONAL

NOT OPERATIONAL

NEEDS MAINTENANCE

6.2.2.2.3 System management attributes

List Of System Management Attributes

The following attributes are accessible through System Management. They are used by all ASEs of the AP to enforce confirmed service flow control (see the state machines of IEC 61158-6). They are not otherwise accessible.

List of AR Endpoint Info

This attribute specifies the numeric identifiers and other configuration information (the AP Context attributes) of AREPs associated with the AP.

Numeric ID

This attribute specifies the numeric id for the ARep.

Configured Max FAL PDU Size Sending

For non-segmenting ARs, this attribute specifies the configured maximum number of octets that can be sent from this ARep. Its value is equal to the maximum Data Link Service Data Unit size minus one that may be sent from this ARep. For segmenting ARs, it specifies the maximum number of 256-byte segments that can be sent on this ARep.

Configured Max FAL PDU Size Receiving

For non-segmenting ARs, this attribute specifies the configured maximum number of octets that can be received on this ARep. Its value is equal to the maximum Data Link Service Data Unit size minus one that may be received on this ARep. For segmenting ARs, it specifies the maximum number of 256-byte segments that can be received on this ARep.

Configured Max Outstanding Services Requesting (ConfigMaxOsReq)

This attribute specifies the maximum number of outstanding confirmed services allowed in the client role of this ARep. Its value is configured before establishment of the AR. This attribute is used on client-server and peer-to-peer ARs (i.e. QUB) and corresponds to the number of sending transaction state machines.

Configured Max Outstanding Services Responding (ConfigMaxOsRsp)

This attribute specifies the maximum number of outstanding confirmed services allowed for this ARep as a server. Its value is configured before establishment of the AR. This attribute is used on client-server and peer-to-peer ARs (i.e., QUB) and corresponds to the number of receiving transaction state machines.

List of Supported Services

This attribute specifies the services that the FAL supports as a requester and as a responder. Support as a requester indicates that the FAL provides the ability for the AP to initiate confirmed and unconfirmed request primitives and to receive corresponding confirmed service confirmation primitives from the FAL. Support as a responder indicates that the FAL provides the ability for the AP to deliver confirmed and unconfirmed service indication primitives to the AP and receive confirmed service response primitives from the AP.

Configured Max Data Structure Nesting Level (ConfigMaxDSNestingLevel)

This optional attribute specifies the maximum number of nesting levels configured for the AR. It is present only when more than one nesting level is supported. That is, the default value for this parameter is one.

List of In-Use AR Endpoint Info

This attribute specifies dynamic information for the AREPs associated with the AP which are participating in an established AR-I or which are involved in the AR establishment process.

Context State

This attribute specifies the state of the ARep as seen by the AP. The values are:

CLOSED

OPEN

OPENING-REQUESTING

OPENING-RESPONDING

Negotiated Max FAL PDU size sending (NegMaxPduSnd)

For non-segmenting ARs, this attribute specifies the negotiated maximum number of octets that can be sent from this AREP. Its value is equal to the maximum Data Link Service Data Unit size minus one that may be sent from this AREP. For segmenting ARs, it specifies the negotiated maximum number of 256-byte segments that can be sent on this AREP.

Negotiated Max FAL PDU size receiving (NegMaxPduRcv)

For non-segmenting ARs, this attribute specifies the negotiated maximum number of octets that can be received on this AREP. Its value is equal to the maximum Data Link Service Data Unit size minus one that may be received on this AREP. For segmenting ARs, it specifies the maximum number of 256-byte segments that can be received on this AREP.

Negotiated Max outstanding services requesting (NegMaxOsReq)

This attribute specifies the maximum number of outstanding confirmed services allowed for this AREP as a client. Its value is negotiated during establishment of the AR to represent the lower value of the local Configured Max Outstanding Services Requesting attribute and the remote Configured Max Outstanding Services Responding attribute.

Negotiated Max outstanding services responding (NegMaxOsRsp)

This attribute specifies the maximum number of outstanding confirmed services allowed for this AREP as a server. Its value is negotiated during establishment of the AR to represent the lower value of the local Configured Max Outstanding Services Responding attribute and the remote Configured Max Outstanding Services Requesting attribute.

Outstanding services requesting counter (OsReqCtr)

This attribute contains the number of currently outstanding confirmed services on this AR as a requester.

Outstanding services responding counter (OsRspCtr)

This attribute contains the number of currently outstanding confirmed services on this AR as a responder.

Negotiated max data structure nesting level (NegConfigMaxDSNestingLevel)

This optional attribute specifies the maximum number of nesting levels negotiated for the AR. It is present only when more than one nesting level is supported. Its value is negotiated as the lower value configured for the two communicating APs.

Negotiated list of supported services

This optional attribute specifies the negotiated services that the FAL supports as a requester and as a responder on this AR. Support as a requester indicates that the FAL provides the ability for the AP to initiate confirmed and unconfirmed request primitives and to receive corresponding confirmed service confirmation primitives from the FAL. Support as a responder indicates that the FAL provides the ability for the AP to deliver confirmed and unconfirmed service indication primitives to the AP and receive confirmed service response primitives from the AP.

6.2.2.2.4 Object management attributes

List Of Managed AP Attributes

This attribute contains the AP attributes that are accessible using the services of the Management ASE.

List of Supported APO Classes and Objects

This attribute contains the Object Definitions maintained by the AP.

List Header

This attribute describes the characteristics of the Object Definitions maintained by the AP.

Numeric ID

This attribute is the Numeric ID of the List Header. Its value is always 0.

ROM / RAM Flag

This attribute, when TRUE, indicates that modifications to the Object Definitions are allowed.

Max Name Length

This attribute specifies the maximum length in octets for names of objects defined for this AP.

Access Protection Supported

This attribute, when TRUE, indicates that Access Protection is supported.

Version Of List

This attribute indicates the current version of this list. Each update to the list of object definitions, either locally initiated or through the use of the SetAttributes service causes the version number to be incremented.

List Local Reference

This attribute is a reference to the list that has local significance, such as an address. The value FFFFFFFF hex indicates that no local reference is available.

XXX Object Class Supported Constraint

This constraint selects object classes specified by XXX. The attributes following the constraint apply to Object Definitions of the selected object class.

XXX Numeric ID

This attribute is the Numeric ID of the first object in a series of objects of the class(es) selected by the constraint. All objects of the selected class(es) are contained in the series. The first Numeric ID for Data Types is always 1.

Number of XXX Entries

This attribute specifies the number of objects in the series.

XXX Local Reference

This attribute is a reference to the beginning of object definitions that describe the objects in the series. The reference has local significance, such as an address. The value FFFFFFFF hex indicates that no local reference is available.

List of Supported APO Classes and Objects

This attribute contains the Object Definitions maintained by the AP.

List of DLSAP Addresses

This attribute specifies the available DLSAP-Addresses usable by the AP to locate its AREPs.

NOTE DLSAP Address structure is defined in IEC 61158-3 and IEC 61158-4.

List of APO Classes Supported

This attribute specifies the classes of APOs supported by the AP.

List of Event Summary Selection Criteria

This optional attribute specifies a set of Boolean selection criteria that the AP is capable of using to select events when performing event summary processing. A set of selection criteria is specified by this standard. When an AP is defined, criteria specified for this attribute may be selected from this set, or they may defined specifically for the AP. This list of criteria specifies

ENABLED When TRUE, event reporting for event objects is enabled

When FALSE, event reporting for event objects is disabled

DETECTED When TRUE, event occurrence has been detected

When FALSE, event occurrence has not been detected

ACKED When TRUE, event acknowledgment has been received

When FALSE, event acknowledgment is outstanding

ACTIVE When TRUE, event detection is active for the event object

When FALSE, event detection is not active for the event object

NOTE Additional values may be specified by this standard in the future or they may be user defined.

6.2.2.2.5 Supported attributes

List of Supported Attributes

This optional attribute specifies the attributes supported by the object. This list contains, at a minimum, the mandatory attributes for the class of the object.

6.2.2.2.6 Services

Subscribe

This optional service is used by push subscribers to indicate to push publishers their requirements for receiving published data.

Identify

This optional service is used to request manufacturer information from the AP.

Get Status

This optional service is used to request the network visible state from the AP.

Status Notification

This optional service is used by the AP to report its network visible state.

Initiate

This optional service is used to open the AP context for an Application Relationship.

Terminate

This optional service is used to abruptly close the AP context for an Application Relationship.

Conclude

This service is used to gracefully close the AP context for an Application Relationship.

Reject

This service is initiated internally by the AP ASE to indicate that a confirmed service request APDU has been received with a protocol error.

6.2.2.3 Application process ASE service specification**6.2.2.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Subscribe

Identify

Get Status

Status Notification

Initiate

Terminate

Conclude

Reject

6.2.2.3.2 Subscribe service**6.2.2.3.2.1 Service overview**

This confirmed service is used by a push subscriber wishing to dynamically request a push publisher for the publication of selected data. The service response contains sufficient information for the subscriber to assign an AREP to receive the data.

Upon receiving a Subscribe service indication, a push publisher may initiate the publication of an object, or modify the publication constraints of a published object. The actions and policies taken by an AP to satisfy a Subscribe indication are determined by the AP itself, and are outside the scope of this standard.

This service is also used by a push subscriber to indicate to the push publisher that it is no longer subscribing to the published object.

NOTE 1 There may or may not be other subscribers who may also receive the requested published data.

NOTE 2 Publishers do not necessarily need to receive such a request to publish. For example, a publisher could be pre-configured with the knowledge of what to publish.

6.2.2.3.2.2 Service primitives

The service parameters for this service are shown in Table 17.

Table 17 – Subscribe service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Application Relationship	M	M(=)		
Invoke ID	M	M(=)		
Joining	S	S(=)		
Published Object	M	M(=)		
Schedule Interval	M	M(=)		
Maximum ARs	U	U(=)		
Maximum Subscriber AP Permissible Jitter	U	U(=)		
Desired Publishing Start Time	U	U(=)		
Earliest Publishing Start Time	U	U(=)		
Latest Publishing Start Time	U	U(=)		
Leaving	S	S(=)		
List Of Leaving AREPs	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Data Type			C	C(=)
Data Length			C	C(=)
Dedicated			C	C(=)
List Of AR Information			C	C(=)
Publishing AREP ID			M	M(=)
DL Mapping Reference			M	M(=)
Scheduled Start Time			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Joining

This parameter is selected when an AP wants to subscribe to a published object. The Subscribe service is sent by the potential subscriber, using the selected AR, to the publisher of the published object.

Published Object

This parameter specifies one of the key attributes of the published object.

Schedule Interval

This parameter specifies the increment in the fieldbus time between successive starting times for the scheduled sequence. The time granularity is the same as that defined for the underlying Data Link Layer (see IEC 61158-3).

Maximum ARs

This optional parameter specifies the maximum number of ARs on which the subscriber is willing to receive data. The default value for this parameter is one (1).

NOTE In certain circumstances, a Publisher AP may be requested to publish the same object on more than one AR. A subscriber AP may receive the same published object from a Publisher AP on more than one AR. The AR on which a Subscribe service is conducted may be different from the AR used to publish the data.

Maximum Subscriber AP Permissible Jitter

This optional parameter specifies the single upper bound on the two definitions of scheduling jitter on which the subscriber AP is willing to receive the data:

- a) the difference in starting times of a given scheduled sequence within successive subscriber AP cycles, and,
- b) the difference in starting times of a given subscriber AP cycle relative to a DLL cycle.

The value of this parameter is always less than the value of the Cycle Interval parameter. If this parameter is not specified, the jitter will be determined by the publisher. The granularity may be the same as defined for the Data Link Layer (see IEC 61158-3).

Desired Publishing Start Time

This optional parameter specifies the fieldbus time at which publishing should start.

Earliest Publishing Start Time

This optional parameter specifies the earliest fieldbus time at which publishing may start.

Latest Publishing Start Time

This optional parameter specifies the latest fieldbus time at which the publishing is to start.

Leaving

This parameter is to be selected if a subscriber wants to stop its subscription to a specific object through a given AREP. The Subscribe service is sent to the Publisher AP using the AR used for joining.

List of Leaving AREPs

This parameter specifies the push publisher's AREP(s) that the subscribing AP wants to leave.

Result(+)

This selection type parameter indicates that the service request succeeded.

Data Type

This conditional parameter identifies the data type of the variable. This parameter is present if the subscriber is attempting to join a relationship.

Data Length

This conditional parameter specifies the length in octets of the data value to be published. This parameter is present if the subscriber is attempting to join a relationship and if the length is not defined for the data type of the published object.

Dedicated

This conditional parameter, when TRUE, indicates that the publishing relationship is dedicated. This parameter is present if the subscriber is attempting to join a relationship.

List of AR Information

This conditional parameter specifies the information necessary for the subscriber to construct its local subscriber AREP(s). This parameter is present if the subscriber is attempting to join a relationship. If present, this list is not empty.

Publishing AREP ID

This parameter specifies the numeric identifier for the AREP used for publishing.

DL Mapping Reference

This parameter identifies the DL mapping for the publishing AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Scheduled start time

This parameter specifies the actual publication start time.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.2.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

If the request indicates that a subscriber wishes to "join", the publishing AP locally determines if it can accommodate the specifics of the request. If it can, it may (1) reconfigure existing AR endpoint(s), or (2) allocate resources and create a new publishing AR endpoint(s), or (3) allocate bandwidth in the network schedule if necessary.

If the request indicates that a subscriber wishes to "leave", the AP locally determines if it can cease publishing over the indicated AR and still accommodate its other existing subscribers. If so, the AP may locally free up the AREP(s) and cancel the related network schedule(s).

If a confirmation (+) was returned for a "join" operation, the FAL user in the AP receiving the response APDU may configure or may create its local AREP(s) that will be used to receive the published variable.

If a confirmation (+) was returned for a "leave" operation, the FAL user in the AP may locally free up the resources associated with the AREP(s) used for subscription

NOTE It is a local matter for the FAL user to determine if the schedule specified by the publisher is sufficient. If insufficient, the local user should issue a Subscribe service primitive to inform the publisher that it is leaving.

6.2.2.3.3 Identify service

6.2.2.3.3.1 Service overview

Information to identify an AP is read with this service.

NOTE The attribute ProfileNumber of the AP is transmitted with the Initiate service.

6.2.2.3.3.2 Service primitives

The service parameters for this service are shown in Table 18.

Table 18 – Identify

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Vendor Name			M	M(=)
Model Identifier			M	M(=)
Vendor Revision			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Vendor Name

This parameter contains the value of the Vendor Name attribute of the AP.

Model Identifier

This parameter contains the value of the Model Identifier attribute of the AP.

Vendor Revision

This parameter contains the value of the Vendor Revision attribute of the AP.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.2.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.2.3.4 Get status service

6.2.2.3.4.1 Service overview

The device/user state is read with the Get Status service.

6.2.2.3.4.2 Service primitives

The service parameters for this service are shown in Table 19.

Table 19 – Get status

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Service Access Status			M	M(=)
Operational Status			M	M(=)
Local Detail			U	U(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service Access Status

This parameter contains the value of the Service Access Status attribute of the AP.

Operational Status

This parameter contains the value of the Operational Status attribute of the AP.

Local Detail

This parameter contains the user defined state of the application.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.2.3.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.2.3.5 Status notification service

6.2.2.3.5.1 Service overview

The Status Notification service is used by the AP to report its state spontaneously.

6.2.2.3.5.2 Service primitives

The service parameters for this service are shown in Table 20.

Table 20 – Status notification

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Service Access Status	M	M(=)
Operational Status	M	M(=)
LocalDetail	U	U(=)

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and the Remote Address Configuration Type is FREE. It gives the remote address to which the Status Notification should be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and the Remote Address Configuration Type is FREE. This parameter indicates the remote address from which the indicated Status Notification is to be sent.

Service Access Status

This parameter contains the value of the Service Access Status attribute of the AP.

Operational Status

This parameter contains the value of the Operational Status attribute of the AP.

Local Detail

This parameter contains the user defined state of the application.

6.2.2.3.5.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.2.2.3.6 Initiate service

6.2.2.3.6.1 Service overview

This service is used to establish a context between communicating APs for the exchange information on a single AR. The Initiate service negotiates the supported FAL services, the maximum outstanding services permitted, the maximum PDU length and the current version of the Object Definitions. The data structure nesting level is also negotiated if the local and remote APs support data structure nesting levels greater than one. The supported FAL services, the maximum outstanding services permitted, the maximum PDU length and data structure nesting level are AP system management attributes configured for the AREP. This service may be used to dynamically create the AREP to be used if the AREP does not exist.

6.2.2.3.6.2 Service primitives

The service parameters for this service are shown in Table 21.

Table 21 – Initiate

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Version Object Definitions Calling	M	M(=)		
AP Descriptor Calling	M	M(=)		
Access Protection Supported Calling	M	M(=)		
Password Calling	M	M(=)		
Access Groups Calling	M	M(=)		
Destination DL-Address	C			
DL-Mapping	C	C		
Nesting Level Calling	U	U(=)		
User Detail	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Version OD Called			M	M(=)
AP Descriptor Called			M	M(=)
Access Protection Supported Called			M	M(=)
Password Called			M	M(=)
Access Groups Called			M	M(=)
DL-Mapping			C	C
Nesting Level Called			U	U(=)
User Detail			U	U(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)
Max PDU Length Sending Called				C
Max PDU Length Receiving Called				C
FAL Features Supported Called				C

Argument

This parameter carries the parameters of the service invocation.

Version Object Definitions Calling

This parameter specifies the version of the client's Object Definitions. Its value is null if the client does not contain Object Definitions.

AP Descriptor Calling

This parameter contains the value of the calling AP's Descriptor Reference attribute if it has one. If it does not, its value is null.

Access Protection Supported Calling

This parameter contains the value of the calling AP's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password Calling

This parameter specifies the password to be used for the access to all objects of the server on this AR. If access with password is not to be used on this AR, its value is null.

Access Groups Calling

This parameter specifies the client's membership in specific access groups. The membership applies for the access to all objects of the server on this AR.

Destination DL-Address

This parameter exists only when the corresponding AR supports it and the Remote Address Configuration Type is FREE. It gives the remote address to which the connection shall be established.

DL-Mapping

This conditional parameter contains the DL-Mapping attributes for the AREP to be opened. It is used only when the AREP to be opened is not defined.

Nesting Level Calling

This optional parameter indicates the nesting level supported by the requester.

User Detail

This optional parameter contains user information.

Result(+)

This selection type parameter indicates that the service request succeeded.

Version Object Definition Called

This parameter shall specify the version of the server's Object Definitions.

AP Descriptor Called

This parameter contains the value of the called AP's Descriptor Reference attribute.

Access Protection Supported Called

This parameter contains the value of the called AP's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password Called

This parameter specifies the password to be used for the access to all objects of the client on this AR. If access with password is not to be used on this AR, its value is null.

Access Groups Called

This parameter specifies the server's membership in specific access groups. The membership applies for the access to all objects of the client on this AR.

DL-Mapping

This conditional parameter contains the negotiated DL-Mapping attributes for the AREP to be opened. It is used only when the AREP to be opened is not defined.

Nesting Level Called

This optional parameter indicates the nesting level supported by the responder.

User Detail

This optional parameter contains user information.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter shall provide the reason for the failure.

Reason	Meaning
Max FAL PDU Size Insufficient	The maximum PDU length is not sufficient for the communication.
Service Not Supported	The requested service or option is not supported by the server.
User Initiate Denied	The FAL user refuses to establish the connection.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already an AR established with the same password, or the password is not valid.
AP Descriptor incompatible	The descriptor is not supported by the server.
Other	Reason other than any of those identified above.

Max PDU Length Sending Called

This parameter shall specify the maximum length of the FAL PDU to be sent that may be handled on this AR.

It shall be transmitted by the called FAL and shall only be part of the Initiate.cnf primitive.

Max PDU Length Receiving Called

This parameter shall specify the maximum length of the FAL PDU to be received that may be handled on this AR.

It shall be transmitted, if supported, by the called FAL and shall only be part of the Initiate.cnf primitive.

FAL Services Supported Called

This parameter shall identify the optional AL services and the options supported by the server (see AR List).

It shall be transmitted, if supported, by the called FAL and shall only be part of the Initiate.cnf primitive.

6.2.2.3.6.3 Service procedure

This service operates through a queue.

The requesting user submits the service request primitive to its FAL AE and starts an associated timer to monitor the request. The AP ASE builds the Initiate Request APDU Body and conveys it on the specified AR using the AR Establish Service request primitive. It also creates a transaction state machine.

Upon receipt of the Initiate Request APDU Body in an AR Establish service indication primitive, the responding AP ASE decodes it. If a protocol error was encountered while decoding the received APDU Body, the ASE uses the AR Abort Service to abort the AR. If a protocol error did not occur, the ASE delivers an Initiate service indication primitive to its user.

If the responding user is able to successfully process the request, the user returns an Initiate service response (+) primitive.

If the responding user is unable to successfully process the request, the service fails and the user issues an Initiate service response (-) primitive indicating the reason.

The responding AP ASE builds an Initiate service response APDU Body for a response (+) primitive or an Initiate service error APDU Body for a response (-) primitive and conveys it on the specified AR using an AR Establish service response primitive.

Upon receipt of the returned APDU Body in an AR Establish service confirmation (+ or -) primitive the initiating ASE delivers an Initiate service confirmation primitive to its user which specifies success or failure, and the reason for failure if a failure occurred. It also cancels the corresponding transaction state machine.

However, if the AP's transaction timer expires before the corresponding response or error APDU is received, the AP may take corrective actions, which may include instructing its local ASE to cancel the transaction state machine. Such instruction to the ASE would occur through a locally defined interface.

6.2.2.3.7 Terminate service

6.2.2.3.7.1 Service overview

This service shall be used to release an existing AR between APs, or to terminate a subscriber/receiver AREP's involvement in an AR.

6.2.2.3.7.2 Service primitives

The service parameters for this service are shown in Table 22.

Table 22 – Terminate

Parameter name	Req	Ind
Argument	M	M
AREP	M	M(=)
Locally Generated		M
Terminate Identifier	M	M(=)
Reason Code	M	M(=)
Terminate Detail	U	U(=)

Argument

This parameter carries the parameters of the service invocation.

Locally Generated

This parameter shall indicate whether the termination was generated locally or by the communication partner.

The value false is not allowed if the Terminate Identifier parameter has the value FAL and the Reason Code parameter has the value AR Error.

Terminate Identifier

This parameter shall indicate where the reason for the termination has been detected. Possible values are:

FAL USER
FAL APO ASE
FAL AR ASE
DLL

Reason Code

This parameter shall specify the reason for the termination.

If the Terminate Identifier parameter has the value USER, the following values shall apply.

	Meaning
Disconnection	The connection is released by the FAL user.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already an AR established with the same password, or the password is not valid.
Descriptor incompatible	The server's descriptor is not supported by the client.
Limited Services Permitted	The AP is in the Logical Status LIMITED-SERVICES-PERMITTED.

If the Abort Identifier parameter has the value FAL, the following values shall apply:

Reason	Meaning
AR Error	Faulty AR
User Error	Improper, unknown or faulty service primitive received from the FAL user
FAL PDU Error	Unknown or faulty FAL PDU received from the AR ASE
Connection State Conflict AR ASE	Improper AR ASE service primitive
AR ASE Error	Unknown or faulty AR ASE service primitive
PDU Size	PDU length exceeds maximum PDU length
Feature Not Supported	SERVICE-REQ PDU received from the AR ASE and service or option not supported as a server (see FAL Features Supported attribute in the AR)
Invoke ID Error Response	Confirmed service.rsp received from the FAL user and Invoke ID does not exist or CONFIRMED_SERVICE-RSP_PDU received from the AR ASE and Invoke ID does not exist
Max Services Overflow	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Outstanding Services Counter Receiving $\geq$ Max Outstanding Services Receiving
Connection State Conflict Service Error	INITIATE-REQ_PDU received from the AR ASE The service in the response does not match the service in the indication or the service in the confirmation does not match the service in the request.
Invoke ID Error Request	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Invoke ID already exists.

If the Terminate Identifier parameter has the value AR ASE or DLL, the Reason Code parameter shall be supplied by the AR ASE.

Terminate Detail

This optional parameter contains additional information about the abort reason (max. 16 octets). In case of error reports from the application, the meaning is defined in the profile.

6.2.2.3.7.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.2.2.3.8 Conclude service

6.2.2.3.8.1 Service overview

This service shall be used to gracefully release an existing AR between APs.

6.2.2.3.8.2 Service primitives

The service parameters for this service are shown in Table 23.

Table 23 – Conclude

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Info			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.2.3.8.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.2.3.9 Reject service

6.2.2.3.9.1 Service overview

This service is used to inform the remote endpoint that a protocol error has been detected. It is generated by the AP ASE if an APDU has been received with an invalid service type code. When another APO ASE detects a protocol error, it internally requests the AP ASE to generate a Reject PDU. This service is not used to indicate that a service error has been detected by the user.

6.2.2.3.9.2 Service primitives

The service parameters for this service are shown in Table 24.

Table 24 – Reject

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Reject Code	M	M(=)
Original FAL Service Type	M	M(=)
Original Invoke ID	M	M(=)
Original PDU Body	U	U(=)

Argument

The argument contains the parameters of the service request.

Reject Code

This parameter indicates the reason for rejection. The values of this may indicate:

- Unsupported Service
- AR Not Established
- AR Not Defined
- Max Outstanding Requests Exceeded
- Max PDU Size Exceeded
- Duplicate Invoke ID

Original FAL Service Type

This parameter contains the service type of the rejected service request.

Original Invoke ID

This parameter contains the invoke id of the rejected service request.

Original PDU Body

This optional parameter contains some or all of the data of the APDU that was rejected. The amount of data included is determined by the user.

6.2.2.3.9.3 Service procedure

The Reject service is a service that operates through a queue or buffer. It may be initiated only by the FAL AP ASE.

If any APO ASE receives a confirmed request APDU that contains a protocol error, the AP ASE builds a Reject Request APDU and returns it on the specified AR.

Upon receipt of the Reject Request APDU, the receiving AP ASE delivers an AR-Reject.indication primitive through the FAL ASE that issued the confirmed send request primitive, and it issues a negative confirmation to the requesting user.

6.2.3 Application relationship ASE**6.2.3.1 Overview****6.2.3.1.1 General**

In a distributed system, application processes communicate with each other by exchanging application layer messages across well-defined application layer communications channels. These communication channels are modeled in the fieldbus Application Layer as application relationships (ARs).

ARs are responsible for conveying messages between applications according to specific communications characteristics required by time-critical systems. Different combinations of these characteristics lead to the definition of different types of ARs. The characteristics of ARs are defined formally as attributes of AR Endpoint classes.

The messages that are conveyed by ARs are FAL service requests and responses. Each is submitted to the AR ASE for transfer by an FAL Application Service Element (ASE) that represents the class of the APO being accessed. Figure 15 illustrates this concept.

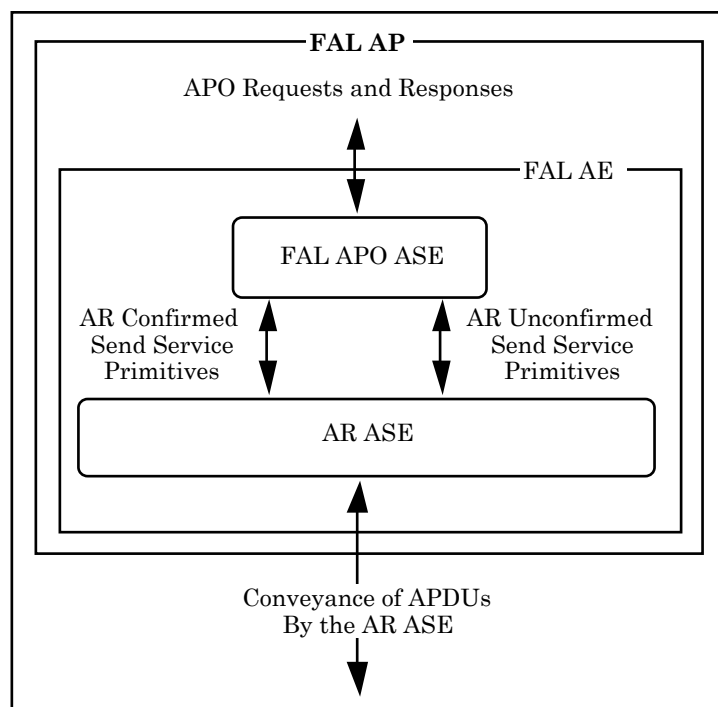


Figure 15 – The AR ASE conveys APDUs between APs

Depending on the type of AR, APDUs may be sent to one or more destination application processes connected by the AR. Other characteristics of the AR determine how APDUs are to be transferred. These characteristics are described below.

6.2.3.1.2 Endpoint context

Each AP involved in an AR contains an endpoint of the AR. Each AR endpoint is defined within the AE of the AP. Its definition, when combined with the definitions of the other endpoints defines an AR. To ensure communications compatibility among or between endpoints, each endpoint definition contains a set of compatibility-related characteristics. These characteristics need to be configured appropriately for each endpoint for the AR to operate properly.

Endpoint definitions also contain a set of characteristics that describe the operation of the AR. These characteristics, when combined with those used to specify compatibility, define the context of the endpoint. The endpoint context is used by the AR ASE to manage the operation of the endpoint and the conveyance of APDUs. The characteristics which comprise the endpoint context are described next.

6.2.3.1.2.1 Endpoint role

The role of an AREP determines the permissible behavior of an AP at the AREP. An AREP role may be that of a client, server, peer (client and/or server), push or pull publisher, push or pull subscriber, or push or pull publishing manager.

NOTE See concepts subclause for description of clients and servers, and the push and pull models for publishers and subscribers.

Table 25 and Table 26 summarize the characteristics and combinations of each of the AREP roles.

Table 25 – Conveyance of service primitives by AREP role

	Client	Server	Peer	Push Publisher	Push Subscriber	Pull Publ Mgr	Pull Publisher	Pull Subscriber
Send Service Req	X		X	X		X		
Recv Service Req		X	X		X		X	
Send Service Rsp		X	X				X	
Recv Service Rsp	X		X			X		X

Table 26 – Valid combinations of AREP roles involved in an AR

AREP Roles by Interaction Model	Client	Server	Peer	Push Publisher	Push Subscriber	Pull Publ Mgr	Pull Publisher	Pull Subscriber
Client/server								
Client		Yes	Yes					
Server			Yes					
Peer			Yes					
Publisher/subscriber								
Push Publisher					Yes			
Push Subscriber								
Pull Publ Mgr							Yes	
Pull Publisher								Yes
Pull Subscriber								

6.2.3.1.2.2 Dedicated AR endpoints

AR endpoints that are dedicated provide their services directly to the FAL User. Although their behavior is the same as non-dedicated endpoints, the APDUs they convey contain no service specific protocol control information other than the AR control field.

FAL Users configured for dedicated ARs construct and transfer their data without using the services of the other FAL ASEs. The format and contents of the user data conveyed over dedicated ARs are known only through configuration.

6.2.3.1.2.3 Cardinality

The cardinality of an AR specifies, from the point of view of a client or publisher endpoint, how many remote application processes are involved in an AR. Cardinality is never expressed from the viewpoint of a server or a subscriber.

When expressed from the viewpoint of a client or peer endpoint, ARs are always 1-to-1. Clients are never capable of issuing a request and waiting for responses from multiple servers.

When expressed from the viewpoint of a publisher endpoint, they indicate that multiple subscribers are supported. These ARs are 1-to-many. ARs that are 1-to-many provide for communications between one application and a group of (one or more) applications. They are often referred to as multi-party and multi-cast. To support the “pull” model of publishing, these ARs provide a management path between the publishing manager and the publisher.

In 1-to-many ARs, the publisher endpoint identifies the remote endpoints using a group identifier, rather than identifying each one individually as is done with 1-to-1 ARs. This permits subscribers to join and leave ARs without disrupting the publisher endpoint context because their individual identities are not known to the publisher endpoint. However, the publisher application may be aware of their participation, but that information is not part of the AR endpoint context.

6.2.3.1.2.4 Conveyance model

6.2.3.1.2.4.1 General

The conveyance model defines how APDUs are sent between endpoints of an AR. Three characteristics are used to define these transfers:

- conveyance paths
- trigger policy, and
- conveyance policy.

6.2.3.1.2.4.2 Conveyance paths

The purpose of AR ASEs is to transfer information between AR endpoints. This information transfer occurs over the conveyance paths of an AR. A conveyance path represents a one-way communication path used by an endpoint for input or output.

To support the role of the application process, the endpoint is configured with either one or two conveyance paths. Endpoints that only send, or only receive, are configured with either a send or receive conveyance path, and those that do both are configured with both. ARs with a single conveyance path are called uni-directional, and those with two conveyance paths are called bi-directional.

Uni-directional ARs are capable of conveying service requests only. To convey service responses, a bi-directional AR is necessary. Therefore, uni-directional ARs support the transfer of unconfirmed services in one direction only, while bi-directional ARs support the transfer of unconfirmed and confirmed services initiated by only one endpoint, or by both endpoints.

6.2.3.1.2.4.3 Trigger policy

Trigger policy indicates when APDUs are transmitted by the Data Link Layer over the network.

The first type is referred to as user-triggered. User-triggered AREPs submit FAL APDUs to the Data Link Layer for transmission at its earliest opportunity.

The second type is referred to as network-scheduled. Network scheduled AREPs submit FAL APDUs to the Data Link Layer for transmission according to a schedule configured by management.

This network scheduling mechanism can be either cyclic or one-time.

Network-scheduled ARs may also convey requests and responses asynchronously if the underlying layer has available bandwidth through use of the compel service. These transfers occur in addition to the scheduled transfers, and without being triggered from the network schedule.

6.2.3.1.2.4.4 Conveyance policy

Conveyance policy indicates whether APDUs are transferred according to a buffer model or a queue model. These models describe the method of conveying APDUs from sender to receiver.

Buffered ARs contain conveyance paths that have a single buffer at each endpoint. Updates to the source buffer are conveyed to the destination buffer according to the trigger policy of the AR. Updates to either buffer replace its contents with new data. In buffered ARs, unconveyed or undelivered data that has been replaced is lost. Additionally, data contained in a buffer may be read multiple times without destroying its contents.

Queued ARs contain conveyance paths that are represented as a queue between endpoints. Queued ARs convey data using a FIFO queue. Queued ARs are not overwritten; new entries are queued until they can be conveyed and delivered.

If a queue is full, a new message will not be placed into the queue.

NOTE The AR conveyance services are described abstractly in such a way that they are capable of being implemented to operate using buffers or queues. These services may be implemented in a number of ways. For example, they may be implemented such that the capability is provided to load the buffer/queue, and subsequently post it for transfer by the underlying Data Link Layer. Or, these services may be implemented such that these capabilities are combined so that the buffer/queue may be loaded and transferred in a single request. On the receiving side, these services may be implemented by delivering the data when it is received, or by indicating its receipt and allowing the user to retrieve it in a separate operation. Another option is to require the user to detect that the buffer or queue has been updated.

6.2.3.1.3 Underlying communications services

6.2.3.1.3.1 General

The AR ASE conveys FAL APDUs using the capabilities of the underlying Data Link Layer. Several characteristics are used to describe these capabilities. This subclause provides a description of each. These characteristics are specific to the Data Link Mapping defined in IEC 61158-6. Their precise specification can be found there.

6.2.3.1.3.2 Connectionless and connection-oriented services

The underlying layer may support AR endpoints by providing connectionless or connection-oriented services. Endpoints configured to operate over connection-oriented services may be capable of opening and closing connections if the connections they use are not pre-configured to be open.

NOTE Connectionless and connection-oriented are DL terms. Refer to IEC 61158-3 and IEC 61158-4 for definition.

6.2.3.1.3.3 Buffered and queued services

The underlying layer may support AR endpoints by providing buffered or queued services. These services can be used to implement buffers or queues required by certain classes of endpoints.

6.2.3.1.3.4 Cyclic and acyclic transfers

The underlying layer may support AR endpoints by providing cyclic or acyclic services.

The underlying layer may also provide for the acyclic transfer of APDUs concurrently with cyclic transfer of APDUs between the same endpoints.

6.2.3.1.3.5 Quality of service

The underlying layer may provide more than one quality of service. When this occurs, the AREP shall be configured to use the quality of service appropriate for the APDUs it conveys. Quality of service parameters for AREPs are contained in the data link mappings in IEC 61158-6.

6.2.3.1.3.6 Segmentation

The FAL has the capability of segmenting user data conveyed by it. In addition, when the underlying layer provides segmentation, too, the maximum length of data conveyed by the FAL in a single APDU will increase accordingly.

6.2.3.1.3.7 Address format

The underlying layer may support more than one address format. When it does, all endpoints of an AR shall be mapped to compatible address formats. DL-mappings for AREPs are specified in IEC 61158-6 for the Data Link Layer (IEC 61158-3).

6.2.3.1.3.8 Timeliness

AR endpoints may be defined to support timeliness information about their transfers as calculated by the Data Link Layer. This timeliness information is received from the Data Link Layer and forwarded to the FAL user as a parameter of the associated FAL service. Publisher and subscriber AR endpoints support transparent, residence, synchronized, and update timeliness. These types of timeliness are described in the definitions of each of these endpoint types below. The mechanism used by the Data Link Layer to calculate timeliness of published data is defined in IEC 61158-3 and IEC 61158-4. The attributes defined by the AR Model to support timeliness are defined in the DLL Mapping Attributes for the Publisher/Subscriber AREPs in IEC 61158-6.

6.2.3.1.3.9 Duplicate FAL PDU detection

AR endpoints may be defined to support detection of the receipt of duplicate FAL APDUs. Alternatively, an indication may be received from the Data Link Layer to support this capability and forwarded to the FAL user as a parameter of the associated FAL service. The attributes defined by the AR Model to support this capability are defined in the DLL Mapping Attributes in IEC 61158-6.

6.2.3.1.4 AR establishment

For an AR endpoint to be used by an application process, the corresponding AR has to be active. When an AR is activated, it is referred to as "established".

AR establishment can occur in one of three ways. First, ARs can be pre-established. Pre-established means that the AE that maintains the endpoint context is created when the AP is connected to the network. In this case, communications among the applications involved in the AR may take place without first having to explicitly establish the AR. Any AR may be defined to be pre-established.

Second, ARs can be pre-defined, but not pre-established. Pre-defined means that the characteristics of the AR are known to each endpoint, but that their contexts have not been synchronized for operation. In this case, the use of the FAL Establish service is required to synchronize them and open them for data transfer.

Third, ARs can require dynamic definition and establishment. In this case, definitions have to be created for each of the AREPs using the Create service. After creation, they are established using the Establish service if they were not created as "established".

6.2.3.1.5 Application relationship classes

AREPs are defined with a combination of characteristics to form different classes of ARs.

6.2.3.2 Application relationship endpoint class specifications

6.2.3.2.1 Formal model

The AR endpoint formal model defines the characteristics common to all AR endpoints. This class is not capable of being instantiated. It is present only for the inheritance of its attributes and services by its subclasses, each specified in a separate subclause of this standard.

All AR endpoint attributes are accessible through system Management (see the future IEC 61158-7), and not through the Object Management ASE services defined in this standard.

FAL ASE: AR ASE

CLASS: AR ENDPOINT

CLASS ID: 32

PARENT CLASS: TOP

SYSTEM MANAGEMENT ATTRIBUTES:

- | | | | |
|---|-----|------------|------------------------------|
| 1 | (m) | Attribute: | Local AP |
| 2 | (m) | Attribute: | FAL Revision |
| 3 | (m) | Attribute: | Dedicated (TRUE, FALSE) |
| 4 | (o) | Attribute: | Transfer Syntax |
| 5 | (o) | Attribute: | List Of Supported Attributes |

SERVICES:

- | | | | |
|----|-----|------------|------------------------------|
| 1 | (o) | OpService: | AR-Unconfirmed Send |
| 2 | (o) | OpService: | AR-Confirmed Send |
| 3 | (o) | OpService: | AR-Establish |
| 4 | (o) | OpService: | AR-DeEstablish |
| 5 | (o) | OpService: | AR-Abort |
| 6 | (o) | OpService: | AR-Compel |
| 7 | (o) | OpService: | AR-Get Buffered Message |
| 8 | (o) | OpService: | AR-Schedule Communication |
| 9 | (o) | OpService: | AR-Cancel Scheduled Sequence |
| 10 | (o) | OpService: | AR-Get DL-Time |
| 11 | (o) | OpService: | AR-Status |
| 12 | (o) | OpService: | AR-XON-OFF |
| 13 | (o) | OpService: | AR-RemoteRead |
| 14 | (o) | OpService: | AR-RemoteWrite |

6.2.3.2.2 System management attributes

Local AP

This attribute identifies the AP attached or configured to use the AREP using a local reference.

FAL Revision

This specifies the revision level of the FAL protocol used by this endpoint. The revision level is in the AR header of all FAL-PDUs transmitted.

Dedicated

The attribute specifies, when TRUE, that the endpoint is dedicated. When TRUE, the services of the AR ASE are accessed directly by the FAL User.

Transfer Syntax

This optional attribute identifies the encoding rules to be used on the AR. When not present, the default FAL Transfer Syntax of this standard is used.

List of Supported Attributes

This optional attribute specifies the attributes supported by the object. This list contains, at a minimum, the mandatory attributes for the class of the object.

6.2.3.2.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be selected.

AR-Unconfirmed Send

This optional service is used to send an unconfirmed service.

AR-Confirmed Send

This optional service is used to send a confirmed service.

AR-Establish

This optional service is used to establish (open) an AR.

AR-DeEstablish

This optional service is used by client or peer users to request the graceful termination of a 1-to-1 AR.

AR-Abort

This optional service is used to abort (abruptly terminate) an AR.

AR-Compel

This optional service is used by the FAL user to request the AR ASE to convey a message which has been deferred until explicitly released.

AR-Get Buffered Message

This optional service is used to request the AR to retrieve a message which is being maintained in a buffer in the local Data Link Layer.

AR-Schedule Communication

This optional service is used to to schedule a sequence of DL-COMPELs for a particular relationship.

AR-Cancel Scheduled Sequence

This optional service is used to cancel an existing sequence which has previously been scheduled.

AR-Get DL-Time

This optional service is used to access the DL-Time service of the Data Link Layer.

AR-Status

This optional service is used to indicate to the AR user that an internal event of the AREP, such as the transmission of a network-triggered buffer, has occurred.

AR-XON-OFF

This optional service causes the flow of data on a specified AR to be suspended or resumed.

AR-RemoteRead

This optional service is used to request the AR to retrieve a message which is being maintained in a buffer in the remote Data Link Layer.

AR-RemoteWrite

This optional service is used to to trigger a data exchange, after writing a value in a local buffer.

6.2.3.3 Application relationship ASE service specifications

6.2.3.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are

- AR-Unconfirmed Send
- AR-Confirmed Send
- AR-Establish
- AR-DeEstablish
- AR-Abort
- AR-Compel
- AR-Get Buffered Message
- AR-Schedule Communication
- AR-Cancel Scheduled Sequence
- AR-Status
- AR-XON-OFF
- AR-RemoteRead
- AR-RemoteWrite

The services AR-Confirmed Send, AR-Establish, and AR-DeEstablish contain the FAL PDU Body as part of the Result parameter in the response and confirmation primitives. The FAL PDU Body may contain either the positive or negative responses returned by the FAL User transparently to the AR ASE. Therefore, these services have a single Result parameter instead of separate Result(+) and Result(-) parameters that are commonly used to convey the positive and negative responses returned by the FAL User.

6.2.3.3.2 AR-unconfirmed send service

6.2.3.3.2.1 Service overview

This service is used to send AR-Unconfirmed request APDUs for FAL APO ASEs. The AR-Unconfirmed Send service may be requested at either endpoint of a one-to-one bi-directional AR, at the server endpoint of a one-to-one uni-directional AR, or at the publisher endpoint of a 1-to-many AR.

NOTE This service is described abstractly in such a way that it is capable of operating with ARs that convey FAL APDUs through buffers or queues. This service may be implemented in such a way that the capability is provided to load the buffer/queue, and subsequently post it for transfer by the underlying Data Link Layer. Alternatively, this service may be implemented such that these capabilities are combined so that the user may load the buffer/queue and request its transfer in a single operation.

6.2.3.3.2.2 Service primitives

The service parameters for this service are shown in Table 27.

Table 27 – AR-Unconfirmed Send

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Remote DLSAP Address	C	C
FAL Service Type	M	M(=)
FAL APDU Body	M	M(=)
Timeliness		C
Duplicate FAL PDU Body		C

Argument

The argument contains the parameters of the service request.

Remote DLSAP Address

This conditional parameter contains the destination DLSAP address in the request and the source DLSAP address in the indication. It is present if the AREP is configured with a Remote Address Configuration Type of FREE.

FAL Service Type

This parameter contains the type of the service being conveyed.

FAL APDU Body

This parameter contains the service dependent body for the APDU.

Timeliness

This conditional parameter indicates both the local and remote timeliness status of the FAL APDU Body contained in the AR-Unconfirmed Send request primitive. This parameter is present if Timeliness is supported in the DL Mapping Attributes of the AREP. The values associated with this parameter are defined in the description of the DLL Mapping Attributes in IEC 61158-6.

Duplicate FAL PDU Body

This conditional parameter indicates whether or not the receipt of a duplicate FAL PDU has been detected by the Data Link Layer. It is present if supported in the DL Mapping Attributes of the AREP.

6.2.3.3.2.3 Service procedure

The AR-Unconfirmed Send Service is a service that operates through a queue or buffer.

The requesting FAL ASE submits an AR-Unconfirmed send.request primitive to its AR ASE. The AR ASE builds an AR-Unconfirmed Send request APDU. If the local AREP for the specified AR supports timeliness, the AR ASE indicates the publishing and transmission timeliness in the APDU.

If the AREP is queued, the AR ASE queues the APDU for submission to the lower layer. If the AR is buffered, the AR ASE replaces the previous contents of the buffer with the APDU contained in the service primitive.

If the AREP is user-triggered, the AR ASE immediately requests the lower layer to transfer the APDU. If the AR is network-scheduled, the AR ASE requests the DL to transfer the data at the scheduled time. The Data Link mapping indicates how the AR ASE coordinates its requests to transmit the data with the Data Link Layer.

NOTE The transmission schedule is managed by the underlying layer, not the AR-ASE. Refer to IEC 61158-3 and IEC 61158-4 for further details.

Upon receipt of the AR-Unconfirmed Send request APDU, the receiving AR ASE delivers an AR-Unconfirmed send.indication primitive to the appropriate FAL ASE as indicated by the FAL Service Type Parameter. If the receiving AREP supports timeliness, the AR ASE includes the timeliness parameters received from the Data Link Layer in the indication primitive.

6.2.3.3.3 AR-confirmed send service

6.2.3.3.3.1 Service overview

This service is used to send confirmed request and response APDUs for FAL APO ASEs. The AR-Confirmed Send service may be requested at Client and Peer endpoints of one-to-one bi-directional ARs.

NOTE This service is described abstractly in such a way that it is capable of operating with ARs that convey FAL APDUs through buffers or queues. This service may be implemented in such a way that the capability is provided to load the buffer/queue, and subsequently post it for transfer by the underlying Data Link Layer. Alternatively, this service may be implemented such that these capabilities are combined so that the user may load the buffer/queue and request its transfer in a single operation.

6.2.3.3.3.2 Service primitives

The service parameters for this service are shown in Table 28.

Table 28 – AR-Confirmed Send

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Destination DL-Address	C	C		
FAL Service Type	M	M(=)		
FAL APDU Body	M	M(=)		
Result			M	M(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Source DL-Address			C	C
FAL Service Type			M	M(=)
FAL APDU Body			M	M(=)
Timeliness				C

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This conditional parameter contains the Destination DL-address. It is present only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE.

NOTE Not all confirmed services support this parameter. The use of this parameter when using confirmed services is a local matter.

FAL Service Type

This parameter contains the type of the service being conveyed.

FAL APDU Body

This parameter contains the service dependent body for the APDU.

Result

This selection type parameter indicates that the service request succeeded.

Source DL-Address

This conditional parameter contains the Source DL-address. It is present only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE.

NOTE Not all confirmed services support this parameter. The use of this parameter with confirmed services is a local matter.

FAL Service Type

This parameter contains the type of the service being conveyed.

FAL APDU Body

This parameter contains the service dependent body for the APDU.

Timeliness

This conditional parameter indicates the timeliness of the update into the FAL. This parameter is present if Timeliness is supported in the DL Mapping Attributes of the AREP. The values associated with this parameter are defined in the description of the DLL Mapping Attributes in IEC 61158-6.

6.2.3.3.3 Service procedure

The AR-Confirmed Send Service is a service that operates through a queue or buffer.

The requesting FAL ASE submits an AR-Confirmed Send request primitive to its AR ASE. The AR ASE creates a transaction state machine to control the invocation of the service.

The AR ASE builds an AR-Confirmed Send request APDU. If the local AREP for the specified AR supports timeliness, the AR ASE indicates the publishing and transmission timeliness in the APDU.

If the AREP is queued, the AR ASE queues the APDU for submission to the lower layer. If the AR is buffered, the AR ASE replaces the previous contents of the buffer with the APDU contained in the service primitive.

If the AREP is user-triggered, the AR ASE immediately requests the lower layer to transfer the APDU. If the AR is network-scheduled, the AR ASE requests the DL to transfer the data at the scheduled time. The Data Link mapping indicates how the AR ASE coordinates its requests to transmit the data with the Data Link Layer.

NOTE The transmission schedule is managed by the underlying layer, not the AR-ASE. Refer to IEC 61158-3 and IEC 61158-4 for further details.

Upon receipt of the AR-Confirmed Send request APDU, the receiving AR ASE delivers an AR-Confirmed Send indication primitive to the appropriate FAL ASE as indicated by the FAL Service Type Parameter. If the receiving AREP supports timeliness, the AR ASE includes the timeliness parameters received from the Data Link Layer in the indication primitive.

The responding FAL ASE submits a confirmed send response primitive to its AR ASE. The AR ASE builds a confirmed send response APDU.

If the AREP is queued, the AR ASE queues the APDU for submission to the lower layer. If the AR is buffered, the AR ASE replaces the previous contents of the buffer with the APDU contained in the service primitive.

If the AREP is user-triggered, the AR ASE immediately requests the lower layer to transfer the APDU. If the AR is network-scheduled, the AR ASE requests the DL to transfer the data at the scheduled time. The data link mapping indicates how the AR ASE coordinates its requests to transmit the data with the Data Link Layer.

Upon receipt of the confirmed send response APDU, the receiving AR ASE uses the Invoke ID contained in the response APDU to associate the response with the appropriate request and cancel the associated transaction state machine. The AR ASE delivers an AR-Confirmed Send confirmation primitive to the requesting FAL ASE. If the receiving AREP supports timeliness, the AR ASE includes the timeliness parameters received from the Data Link Layer in the confirmation primitive.

If the timer expires before the sending AR ASE receives the response APDU, the AR ASE cancels the associated transaction state machine and delivers an AR-Confirmed Send confirmation (-) primitive to the requesting FAL ASE.

6.2.3.3.4 AR-establish service

6.2.3.3.4.1 Service overview

This confirmed service operates in a pair-wise manner between two AR endpoints to synchronize their contexts and activate them for the transfer of APDUs.

The endpoint context may be created during establishment or prior to establishment through System Management. Once defined, the Set Attributes and Get Attributes services can be used to update and further coordinate endpoint contexts.

6.2.3.3.4.2 Service primitives

The service parameters for this service are shown in Table 29.

Table 29 – AR-Establish service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Remote DL-address	C	C(=)		
User Data	U	U(=)		
Result			M	M(=)
AREP			M	M(=)
User Data			U	U(=)

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AREP to be established.

Remote DL-address

This conditional parameter identifies the remote DL-address. If the AR to be established is supported by a DL-Connection, then the DL-address is a DLCEP-address. If not, the DL-address is a DLSAP-address. It is present when the AREP Remote Address Configuration Type attribute is FREE.

User Data

This optional parameter contains user supplied data that is to be conveyed with the service request.

Result

This parameter indicates that the service request succeeded or failed.

AREP

This parameter contains sufficient information to locally identify the AREP to be established.

User Data

This optional parameter contains user supplied data that is to be conveyed with the service response.

6.2.3.3.4.3 Service procedure

6.2.3.3.4.3.1 1-to-1 AR establishment

When used in support of 1-to-1 AREPs, the AR-Establish service causes Establish PDUs to be exchanged between calling and called DLSAPs as shown in Figure 16.

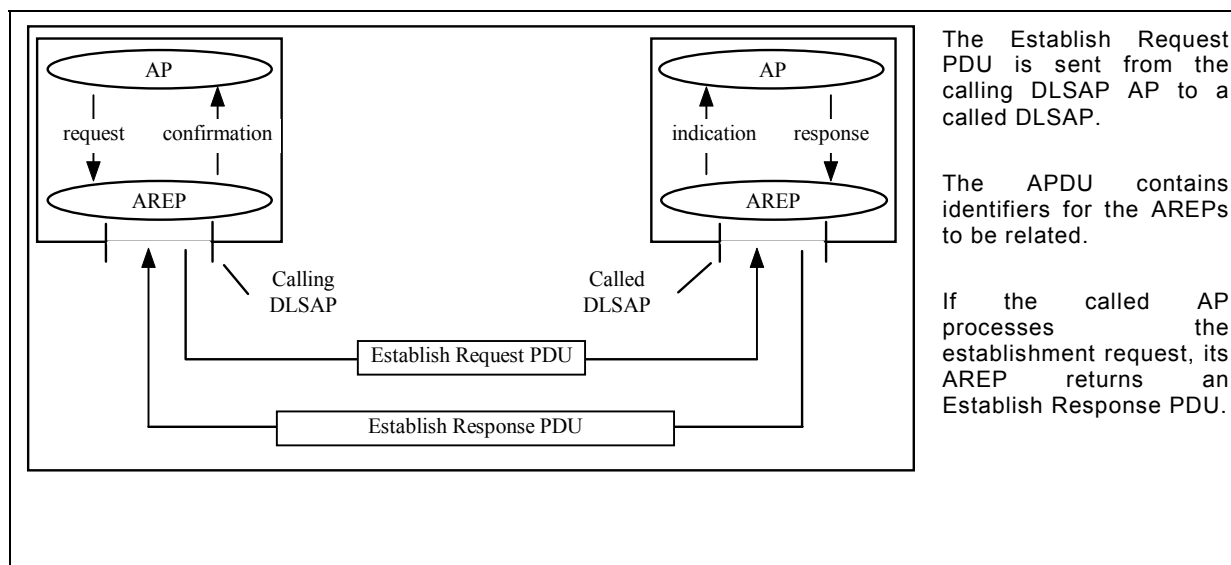


Figure 16 – 1-to-1 AR establishment

Upon receipt of an AR-Establish request service primitive, the calling FAL issues an Establish Request APDU to the called AP ASE that handles establishment of an AR. Upon receipt of an Establish indication primitive, the called AP ASE examines the parameters specified in it and returns the appropriate response in the AR-Establish response primitive.

6.2.3.3.4.3.2 1-to-many AR establishment

When used in support of 1-to-Many AREPs, the AR-Establish service causes local AREPs to be established independently, as shown in Figure 17.

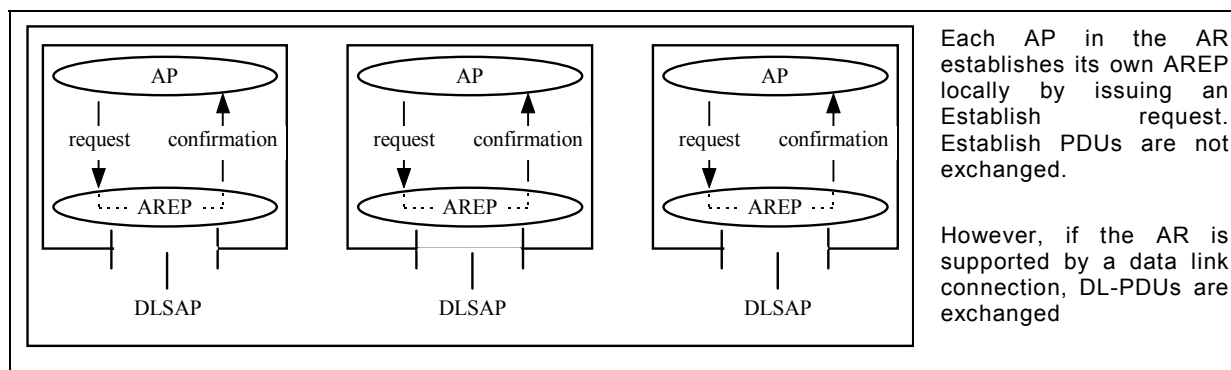


Figure 17 – 1-to-many AR establishment

Upon receipt of an AR-Establish request service primitive, the calling FAL issues an AR-Establish confirmation service primitive to the calling AP indicating whether or not it was able to establish the AREP. If one of the following cases is found, it was not able to establish the AREP:

- 1) the requested AREP is not specified within the FAL, or
- 2) resources are not available to establish the requested AREP, or
- 3) for AREPs supported by a data link connection, the Data Link Layer was not able to establish the data link connection.

6.2.3.3.4.3.3 Compatible AREP classes

Table 30 provides possible combinations of the AREP classes which may be related with the AR-Establish service. In this table the SERVER column contains a "-" to indicate that Server AREPs do not initiate AR establishment.

Table 30 – Valid combinations of AREP classes to be related

		Calling AREP Role		
		PEER	CLIENT	SERVER
Called AREP Role	PEER	YES	YES	--
	CLIENT	NO	NO	--
	SERVER	YES	YES	--

If the called AP decides to establish an AR, it issues an AR-Establish response primitive. The FAL returns the AR parameter which is used to refer to the AR being formed from the called service user. If the AR being established uses the Connection Oriented Data Link Layer and its explicit establishment is required, it is established before an Establish Response APDU is returned. The called AP then returns an Establish Response APDU with the Result(+) parameter to the calling AR_ASE. The calling AR_ASE issues an AR-Establish confirmation primitive in which the AR parameter is specified.

6.2.3.3.4.3.4 Conflict resolution

The normal establishment of ARs follows the general time-sequence for confirmed services. In the cases where each endpoint of an AR concurrently issues an AR-Establish request, a conflict arises. The algorithms for resolving conflicts of this type are specified in detail in IEC 61158-6.

6.2.3.3.5 AR-deEstablish service

6.2.3.3.5.1 Service overview

This confirmed service is invoked by client or peer users to request the graceful termination of a 1-to-1 application relationship. Use of the AR-DeEstablish service causes the endpoints of the AR to be closed. They may be reopened using the AR-Establish service. This service may be used on any open AREP, whether or not the AR was pre-established or dynamically established using the AR-Establish service.

When a request to de-establish an AR is made, the local AR ASE conveys the DeEstablish Request PDU to the remote endpoint of the AR, and accepts no additional request PDUs from the user unless an AR-Abort is requested. Upon receipt of a DeEstablish Response PDU, it clears all outstanding service requests, closes the endpoint context, and informs the user using the confirm service primitive.

The remote AR ASE informs its user when an AR-DeEstablish Request PDU is received using the indication primitive for the service. The user returns a response primitive after responding to the service indications it received. When the remote AR ASE receives the response, it closes the endpoint context, and returns an AR- DeEstablish Response PDU.

6.2.3.3.5.2 Service primitives

The service parameters for this service are shown in Table 31. The use of this service is for future study.

Table 31 – AR-DeEstablish service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.5.3 Service procedure

The AR-DeEstablish Service is a service that operates through a queue or buffer. Its service procedure is for future study.

6.2.3.3.6 AR-abort service

6.2.3.3.6.1 Service overview

The service is used by the FAL User to abruptly terminate an AR. It is always successful; the receiver of an Abort request or Abort request APDU always aborts the AR. This service may be used on any open AREP, whether or not the AR was pre-established or dynamically established using the establish service.

The AR-Abort Service is used to instruct the AR ASE to abruptly terminate all activity on an AREP and place it in the Closed state. Receipt of an AR-Abort request primitive causes the AR ASE to immediately close the AREP context and issue an Abort Request APDU to the remote AREPs. Receipt of an Abort Request APDU causes the AR ASE to immediately close the AR AREP context and deliver an AR-Abort request indication primitive to the user. The immediate close of each endpoint context causes all outstanding service requests to be cleared. All subsequent service primitives and APDUs received by the AR ASE for the aborted AR are discarded except for those of the AR-Establish service.

The AR-Abort service may be requested at either AREP of a one-to-one relationship, or at the publisher endpoint of a 1-to-many or 1-to-all AR. Subscribers are not capable of aborting ARs with publishers, although they may close their own endpoints through local means.

The AR ASE may also initiate the abort service when it detects unrecoverable communication failures. In this case, the AR ASE delivers an AR-Abort indication primitive informing the user of the failure and closes the endpoint context.

6.2.3.3.6.2 Service primitives

The service parameters for this service are shown in Table 32.

Table 32 – AR-Abort

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Locally Generated		M
Originator	M	M(=)
Reason Code	M	M(=)
Additional Detail	U	U(=)

Argument

This parameter carries the information associated with the AR-Abort service.

Locally Generated

This parameter specifies if the abort was locally generated, or not.

Originator

This parameter identifies the originator for the abort. Its valid values are DLL, FAL, or FAL-USER. The value DLL cannot be used in the request primitive.

Reason Code

This parameter indicates the reason for the Abort. It may be supplied by either the provider or the user. One reason is defined: AR ASE error. Other reason code values may be supplied by the Data Link Layer, or by the user.

Additional Detail

This optional parameter contains user data that accompanies the indication. When used, the value submitted in the request primitive is delivered unchanged in the indication primitive.

6.2.3.3.6.3 Service procedure

The abort service is a service that operates through a queue or buffer.

If the user wishes to abort an AR, it submits an abort request primitive to its FAL AR ASE. If an AR ASE detects an unrecoverable local failure or a communication failure, it delivers an abort indication primitive to the endpoint user.

The FAL AR ASE builds an abort request APDU and conveys it on the specified AR if it is capable of doing so. It also transitions the AREP state to CLOSED

Upon receipt of the Abort Request APDU, each remote FAL AR ASE transitions its AREP to CLOSED and delivers an abort indication primitive to its user.

6.2.3.3.7 AR-compel service

6.2.3.3.7.1 Service overview

This service is used by the FAL user to request the AR ASE to convey a message which has been deferred until explicitly released.

NOTE The services that operate on ARs are described abstractly in such a way that they are capable of operating with ARs that convey FAL APDUs through buffers or queues. These services may be implemented such that the capability is provided to load the buffer/queue, and subsequently post it for transfer by the underlying Data Link Layer using this service.

6.2.3.3.7.2 Service primitives

The service parameters for this service are shown in Table 33.

Table 33 – AR-Compel service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Schedule ID	U			
Result (+)				S
AREP				M
Status				M
Result (-)				S
AREP				M
Error Info				M

Argument

The argument contains the parameters of the service request.

Schedule ID

This optional parameter specifies the Data Link Layer sequence to be compelled for network scheduled ARs. The scheduling sequence is part of the DL-Mapping defined in IEC 61158-6.

Result(+)

This selection type parameter indicates that the service request succeeded.

Status

This parameter indicates the result of the service request. The following three status codes provided by the DLL are defined:

success

failure – inappropriate request;

failure – reason unspecified.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.7.3 Service procedure

The AR-Compel service is a service that operates through a queue or buffer.

The requesting user submits an AR compel.request primitive to its FAL AR ASE. The FAL AR ASE issues the corresponding Data Link Layer service request to the Data Link Layer.

6.2.3.3.8 AR-get buffered message service

6.2.3.3.8.1 Service overview

This local service is used by an application process to request the AR ASE to retrieve a message which is being maintained in a buffer in the local Data Link Layer.

This service does not result in the conveyance of an APDU. It is provided so that the FAL user may access a buffer through the FAL AR.

6.2.3.3.8.2 Service primitives

The service parameters for this service are shown in Table 34.

Table 34 – AR-Get Buffered Message service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Result(+)				S
AREP				M
Decoded buffer data				M
Local timeliness				C
Remote timeliness				C
Duplicate FAL PDU Body				C
Result (-)				S
AREP				M
Error Info				M

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Decoded Buffer Data

This parameter contains the user data in the FAL APDU read from the buffer.

Local Timeliness

This conditional parameter, if it is present and True, indicates that the Decoded Buffer Data parameter has met the receiving timeliness criteria defined for the DLL. If its value is False, at least one of the timeliness criteria has not been met. It is present if supported in the DL Mapping Attributes of the AREP.

Remote Timeliness

This conditional parameter, if it is present and True, indicates that the Decoded Buffer Data parameter has met the Publisher's and transmitting DL's timeliness criteria. If its value is False, at least one of the timeliness criteria has not been met. It is present if supported in the DL Mapping Attributes of the AREP.

Duplicate FAL PDU Body

This conditional parameter indicates whether or not the receipt of a duplicate FAL PDU has been detected by the Data Link Layer. It is present if supported in the DL Mapping Attributes of the AREP.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.8.3 Service procedure

This service requests the FAL to return the current contents of the buffer in a confirmation(+) primitive. If the buffer is empty, a confirmation(-) primitive is returned.

6.2.3.3.9 AR-schedule communication service

6.2.3.3.9.1 Service overview

This local service provides the AL user with the ability to schedule a sequence of DL-COMPELs for a particular relationship. It maps directly to the DL-Schedule-Sequence service and has no effect on the AR state machines.

This service does not result in the conveyance of an APDU. It may, however, result in the transfer of Data Link Layer PDUs.

6.2.3.3.9.2 Service primitives

The service parameters for this service are shown in Table 35.

Table 35 – AR-Schedule Communication service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Invoke ID	M			
Schedule Information	M			
Result (+)				S
AREP				M
Invoke ID				M
Sequence ID				M
Result (-)			S	S
AREP			M	M
Invoke ID			M	M
Error Info			M	M

Argument

The argument contains the parameters of the service request.

Schedule Info

This specifies the Data Link Layer scheduling information for the scheduled communication.

Result(+)

This selection type parameter indicates that the service request succeeded.

Sequence ID

This parameter provides a short-hand identifier for the requested scheduled communication.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.9.3 Service procedure

This service requests the communication stack to create a scheduled sequence.

6.2.3.3.10 AR-cancel scheduled sequence service

6.2.3.3.10.1 Service overview

This local service provides the AL user with the ability to cancel an existing sequence which has previously been scheduled. It maps directly to the DL-Cancel-Schedule service and has no effect on the AR state machines.

This service does not result in the conveyance of an APDU. It may, however, result in the conveyance of Data Link Layer PDUs.

6.2.3.3.10.2 Service primitives

The service parameters for this service are shown in Table 36.

Table 36 – AR-Cancel Scheduled Sequence service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Invoke ID	M			
Sequence ID	M			
Result (+)				S
AREP				M
Invoke ID				M
Result (-)				S
AREP				M
Invoke ID				M
Error Info				M

Argument

The argument contains the parameters of the service request.

Sequence ID

This parameter specifies the schedule of the sequence to be cancelled.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.10.3 Service procedure

This service requests the communication stack to cancel an existing scheduled sequence.

6.2.3.3.11 AR-get DL-time service

6.2.3.3.11.1 Description

This service is defined for the FAL to provide the FAL User with access to the DL-Time service of the Data Link Layer. To maintain compatibility with the DL-Time service, the service and parameters definitions are not defined in this standard. See IEC 61158-3 for the definition of the DL-Time service.

6.2.3.3.12 AR-status service

6.2.3.3.12.1 Service overview

This local service provides the AL user notification of a status change of the AREP.

6.2.3.3.12.2 Service primitives

The service parameters for this service are shown in Table 37.

Table 37 – AR-Status

Parameter name	Req	Ind	Rsp	Cnf
Argument		M		
AREP		M		
Status code		M		

Argument

This parameter carries the parameters of the service invocation.

Status Code

This specifies the status change being reported. The following status codes are defined

- lower layer reset;
- buffer received;
- buffer transmitted;
- transmission not timely;
- lower layer lost schedule -- rescheduling required;
- local confirmation.

6.2.3.3.12.3 Service procedure

This service indicates that a significant event, as defined by the status code parameter, occurred in the communication stack.

6.2.3.3.13 AR-XON-OFF service

6.2.3.3.13.1 Service overview

This local service causes the flow of data on a specified AR to be suspended or resumed.

6.2.3.3.13.2 Service primitives

The service parameters for this service are shown in Table 38.

Table 38 – AR-XON-OFF

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Invoke ID	M	M(=)
XON-OFF	M	M(=)

Argument

The argument contains the parameters of the service request.

XON-OFF

This parameter contains user supplied data that can be conveyed with the service request. User data can be "ON" or "OFF".

6.2.3.3.13.3 Service procedure

The AR-XON-OFF Service is a service that operates through a queue.

The requesting FAL ASE submits an AR-XON-OFF.request primitive to its AR ASE. The AR ASE builds an AR-XON-OFF request APDU.

NOTE The transmission schedule is managed by the underlying layer, not the AR-ASE. Refer to IEC 61158-3 and IEC 61158-4 for further details.

Upon receipt of the AR-XON-OFF request APDU, the receiving AR ASE delivers an AR-XON-OFF.indication primitive to the appropriate FAL ASE as indicated by the FAL Service Type Parameter.

6.2.3.3.14 AR remote read service

6.2.3.3.14.1 Service overview

This service provides the application process to request the AR ASE to retrieve a message which is being maintained in a buffer in the remote Data Link Layer.

6.2.3.3.14.2 Service primitives

The service parameters for this service are shown in Table 39.

Table 39 – AR-Remote Read service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Priority	M			
Result (+)				S
AREP				M
Decoded Buffer Data				M
Local Timeliness				C
Remote Timeliness				C
Result (-)				S
AREP				M
Error info				M

Argument

The argument contains the parameters of the service request.

Priority

This argument is locally used and defined by the user to allow two data flows

Result(+)

This selection type parameter indicates that the service request succeeded.

Decoded Buffer Data

This parameter contains the user data in the FAL APDU read from the buffer.

Local timeliness

This conditional parameter, if it is present and True, indicates that the Decoded Buffer Data parameter has met the receiving timeliness criteria defined for the DLL. If its value is False, at least one of the timeliness criteria has not been met. It is present if supported in the DL Mapping Attributes of the AREP.

Remote timeliness

This conditional parameter, if it is present and True, indicates that the Decoded Buffer Data parameter has met the Publisher's and transmitting DL's timeliness criteria. If its value is False, at least one of the timeliness criteria has not been met. It is present if supported in the the DL Mapping Attributes of the AREP.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.3.14.3 Service procedure

This service requests the FAL to return the current contents of the remote buffer and give the value of the contents in a confirmation(+) primitive. If the buffer is empty, a confirmation(-) primitive is returned.

6.2.3.3.15 AR-remote write service

6.2.3.3.15.1 Service overview

This service provides the AL user with the ability to trigger a data exchange, after writing a value in a local buffer.

6.2.3.3.15.2 Service primitives

The service parameters for this service are shown in Table 40.

Table 40 – AR-Remote Write service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Priority	M			
Decoded Buffer Data	M			
Result (+)				S
AREP				M
Result (-)				S
AREP				M
Error Info				M

Argument

The argument contains the parameters of the service request.

Priority

This argument is locally used and defined by the user to allow two data flows.

Decoded Buffer Data

This argument is used to contain the APDU to be written.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.3.4 Service procedure

This service requests the FAL to write the contents of the local buffer and to trigger the data transfer. A confirmation(+) primitive is sent when the service succeeds. If the service fails, a confirmation(-) primitive is returned.

6.2.4 Variable ASE

6.2.4.1 Overview

In the fieldbus environment, application processes contain data that remote applications are able to read and write. The variable ASE defines the network visible attributes of application data and provides a set of services used to read, write, and report their values. Common FAL management services are used to create and delete variable objects and to access their attributes.

Two classes of variable APOs are defined by the Variable Model, individual variables and variable lists. Individual variables are used to specify access to application data of any FAL-defined or user-defined data type. Services defined to access individual variables can be used to access one or more entire variables or one or more entries (per variable) in an array or one or more fields (per variable) in a structure.

Variable lists are used to group variables for access purposes. The same services used to access individual variables can be used to access a variable list.

Two types of operation are supported by variable services, "best effort" and "atomic". The "best effort" services succeed when at least one of the referenced values can be accessed. The "atomic" services succeed when all of the referenced values can be accessed, and fail if any one of the values cannot be accessed.

When supported by the appropriate type of application relationship, the Variable Model service can be used to support two different access models, the client/server model and the publisher/subscriber model. The client/server model is characterized by a client application sending a read or write request to a server application that responds accordingly. The server's activity is stimulated by clients on the network; if there are no requests, the server generates no responses.

The publisher/subscriber model is different. It is characterized by a data producer publishing its data onto the network. Subscribers wishing to acquire the published data join the application relationship used to publish it and listen for the data as it is transmitted.

Two models are provided to support this publisher/subscriber activity, the "pull" and the "push". In the "pull" model, the publishing manager pulls the data from the publisher by issuing a read request to it. The publisher responds by multicasting a sequence of read responses to the publishing manager and to the subscribers.

In the "push" model, the publishing manager is always co-located with the publisher. Subscribers are able to indicate to the publishing manager how they want the data published. Through local interfaces, the publishing manager controls the activity of the publisher and the characteristics of the application relationships used to distribute the data. In this case, the published data is transmitted onto the network using the Information Report unconfirmed service.

Subscribers in both models receive the published data through a local copy of the data maintained by the FAL. As the data is received from the network, the local copy is updated and made available to the subscriber. When the subscriber wishes to access the data, it accesses the local copy, instead of issuing a read request to the remote variable as would happen in the client/server model. If the appropriate AREP attributes are set, timeliness information will be included with the data.

The FAL AR ASE supports both publisher/subscriber models by multi-casting the published data on buffered, network scheduled application relationships. The exact characteristics of the transfers are controlled by the attribute settings of the application relationship.

The formal model of the variable model is presented next, followed by a description of its services. IEC 61158-6 describes the abstract syntax and procedures for its protocol.

6.2.4.2 Requirements for access to variables

The structure of a variable value is defined by its data type. A variable's data type may be of any valid standardized or user defined data type. Standardized types and the facilities for defining user types are specified by the Data Type Model.

As the Data Type Model permits nesting of data structures and arrays to more than one level, the services defined for variable access permit partial access to any nesting level. Partial access means independent access to several components of each structure or array referenced. That is, if constructed type "A" contains constructed type "B", then partial access is provided to "B", and also to components of "B".

6.2.4.3 Variable model class specification

6.2.4.3.1 Simple variable class specification

6.2.4.3.1.1 Simple variable formal model

FAL ASE: VARIABLE ASE

CLASS: SIMPLE VARIABLE

CLASS ID: 7

PARENT CLASS: TOP

ATTRIBUTES:

- | | | | |
|-----|-----|----------------|--|
| 1 | (o) | Key Attribute: | Symbolic Address |
| 2 | (m) | Attribute: | Data Type |
| 3 | (m) | Attribute: | Length |
| 4 | (c) | Constraint: | Data Type Format = STRING |
| 4.1 | (o) | Attribute: | Variable Length Conveyance (TRUE, FALSE) -- see note below |
| 5 | (m) | Attribute: | Access Privilege |
| 5.1 | (m) | Attribute: | Password |
| 5.2 | (m) | Attribute: | Access Groups |
| 5.3 | (m) | Attribute: | Access Rights |
| 6 | (m) | Attribute: | Local Detail |

SERVICES:

- | | | | |
|---|-----|-------------|-------------------------|
| 1 | (o) | OpsService: | Read |
| 2 | (o) | OpsService: | Write |
| 3 | (o) | OpsService: | Read List |
| 4 | (o) | OpsService: | Write List |
| 5 | (o) | OpsService: | Information Report |
| 6 | (o) | OpsService: | Information Report List |
| 7 | (o) | OpsService: | Exchange |
| 8 | (o) | OpsService: | Exchange List |

NOTE The constraint is TRUE when a variable is being defined as an array.

6.2.4.3.1.2 Attributes

Symbolic Address

This optional key attribute specifies a symbolic reference for the variable. This attribute provides a second name space for defining variables that is separate from that of the variable name to ensure that duplication of names in separate name spaces do not create a problem. The symbolic address may be used to assign variable names independent of how they are named within a given system.

A space (" ") character may not be embedded in the middle of a symbolic address. However, using it as leading or trailing fillers is permitted. Such fillers are not to be interpreted as part of the symbolic address. The use of leading or trailing fillers permits fixed-length character string usage.

NOTE For example, a device manufacturer may use the symbolic address to name the data acquired from a local input channel as "port 1". A user of that data may create a second variable for the data using the variable name "temperature".

Data Type

This attribute is the numeric identifier of a FIXED-LENGTH or STRING data type.

Length

This attribute indicates the length of simple variables data types. For variables with STRING data types, this attribute is used to define the maximum length of the variable in octets. For variables with the FIXED-LENGTH data type, this attribute is used to reflect the length of the variable as specified by the data type.

Variable Length Conveyance

This conditional Boolean attribute, when TRUE, indicates that only the valid octets of the string are conveyed. When FALSE, all octets of the string are conveyed, even if they are not part of the string value. This attribute is present only for variables with the STRING data type.

Access Privilege

This attribute specifies the access controls defined for this variable. It is composed of the following:

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute identifies which of the eight user-defined access groups are defined for the variable. More than one access group may be defined.

Access Rights

This attribute defines the type of access defined for the variable. Valid values are:

- Right to Write for Access Groups
- Right to Read for Access Groups
- Right to Write for the registered Password
- Right to Read for the registered Password
- Right to Write for all Communication Partners
- Right to Read for all Communication Partners

Local Detail

This attribute contains local information.

6.2.4.3.1.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be supported.

Read

This optional service may be used to read a single variable object or variable list object. This service may be used in both the client/server model and the publisher/subscriber pull model.

Write

This optional service may be used to update a single variable object or variable list object. This service may be used only in the client/server model.

Read List

This optional service may be used to read multiple variable objects or components of multiple variable objects or a single variable list object. The number of referenced variables/components or variable objects in the list is a device constructor matter. Each of the referenced variables/components or variables in the list is accessed and returned in a “best effort” fashion, such that at least one value is to be returned for the service to succeed. This service may be used only in the client/server model.

Write List

This optional service may be used to update multiple variable objects or single components of multiple variable objects or a single variable list object. The number of referenced variables/components or variable objects in the list is a device constructor matter. Each of the referenced variables/components or variables in the list are updated in a “best effort” fashion, such that at least one value is to be updated for the service to succeed. This service may be used only in the client/server model.

Information Report

This optional service is an unconfirmed service that may be used to report the value of a variable or variable list object. This service may be used in both the client/server model and the publisher/subscriber push model.

Information Report List

This optional unconfirmed service may be used to report multiple variable objects or single components of multiple variable objects or a single variable list object. The number of referenced variables/components or variable objects in the list is a device constructor matter. This service may be used in both the client/server model and the push publisher/subscriber model.

Exchange

This optional service is a confirmed service that may be used to write the value of a remote variable or variable list and read the value of another variable or variable list in one operation. This service may be used in the client/server model. The relationship between the specified variables and User Layer Exchange Function is outside the scope of this standard.

Exchange List

This optional confirmed service may be used to update multiple variable objects or single components of multiple variable objects or a single variable list object and to read multiple variable objects or single components of multiple variable objects or a single variable list object in one operation. The number of referenced variables/components or variable objects in the list is a device constructor matter. Each of the referenced variables/components or variables in the list is updated/accessed in a “best effort” fashion, such that at least one value is to be updated/returned for the service to succeed. This service may be used only in the client/server model. The relationship between the specified variables and a User Layer Exchange Function is outside the scope of this standard.

6.2.4.3.2 Array variable class specification

6.2.4.3.2.1 Formal model

FAL ASE:	VARIABLE ASE
CLASS:	ARRAY VARIABLE
CLASS ID:	8
PARENT CLASS:	TOP
ATTRIBUTES:	
1	(o) Key Attribute: Symbolic Address
2	(m) Attribute: Data Type
2.1	(s) Attribute: Data Type ID
2.2	(s) Attribute: Embedded Data Type

3	(c)	Constraint:	Data Type Format = ARRAY FIXED-LENGTH STRING
3.1	(o)	Attribute:	Element Length
4	(c)	Constraint:	Data Type Format = STRING
4.1	(o)	Attribute:	Variable Length Conveyance (TRUE, FALSE)
5	(o)	Attribute:	Number of Elements
6	(m)	Attribute:	Access Privilege
6.1	(m)	Attribute:	Password
6.2	(m)	Attribute:	Access Groups
6.3	(m)	Attribute:	Access Rights
7	(m)	Attribute:	Local Detail

SERVICES:

1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
3	(o)	OpsService:	Read List
4	(o)	OpsService:	Write List
5	(o)	OpsService:	Information Report
6	(o)	OpsService:	Information Report List
7	(o)	OpsService:	Exchange
8	(o)	OpsService:	Exchange List

6.2.4.3.2.2 Attributes**Symbolic Address**

See the description of this attribute defined for the Simple Variable class above.

Data Type

This attribute specifies the data type for elements of the array.

Data Type ID

This attribute is the numeric identifier of a data type associated with the array. If the format of the data type is ARRAY, this attribute is the data type of the array. If the data type is FIXED-LENGTH or STRING, this attribute is the data type of an array element.

Embedded Data Type

This attribute is used to embed the data type description in the array definition. The contents for this attribute are shown in the Data Type model.

Element Length

This conditional attribute indicates the length of an array element with an ARRAY, FIXED-LENGTH or STRING data type. For array elements with STRING data types, this attribute is used to define the length of an array element in octets. For variables with an ARRAY or FIXED-LENGTH data type, this attribute is used to reflect the length of an array element as specified by the data type.

Variable-length Conveyance

This conditional Boolean attribute, when TRUE, indicates that only the valid octets of the string are conveyed. When FALSE, all octets of the string are conveyed, even if they are not part of the string value. This attribute is present only for arrays with a STRING data type.

Number of Elements

This attribute specifies the number of array elements for variable objects that are defined as arrays. Array variables may be defined in one of two ways. Each of the ways, and the use of this attribute for each are shown below.

Method	Attribute Use
Reference to a data type with the format of ARRAY:	Reflects the number of elements defined for the array data type
Reference to a data type with the format of FIXED-LENGTH or STRING	Specifies the number of elements defined for the array variable

Access Privilege

See the description of this attribute and its components defined in the Simple Variable class above.

Local Detail

This attribute contains local information.

6.2.4.3.2.3 Services

See the description of the services defined in the Simple Variable class above.

6.2.4.3.3 Record variable class specification

6.2.4.3.3.1 Formal model

FAL ASE:	VARIABLE ASE
CLASS:	RECORD VARIABLE
CLASS ID:	9
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (o)	Key Attribute: Symbolic Address
2 (m)	Attribute: Data Type
2.1 (s)	Attribute: Data Type ID
2.2 (s)	Attribute: Embedded Data Type
3 (m)	Attribute: Access Privilege
3.1 (m)	Attribute: Password
3.2 (m)	Attribute: Access Groups
3.3 (m)	Attribute: Access Rights
4 (m)	Attribute: List of Local Detail
SERVICES:	
1 (o)	OpsService: Read
2 (o)	OpsService: Write
3 (o)	OpsService: Read List
4 (o)	OpsService: Write List
5 (o)	OpsService: Information Report
6 (o)	OpsService: Information Report List
7 (o)	OpsService: Exchange
8 (o)	OpsService: Exchange List

6.2.4.3.3.2 Attributes

Symbolic Address

See the description of this attribute defined for the Simple Variable class above.

Data Type

This attribute specifies the data type for elements of the array.

Data Type ID

This attribute is the numeric identifier of a STRUCTURE data type.

Embedded Data Type

This attribute is used to embed the data type description in the record definition. The contents for this attribute are shown in the Data Type model.

Access Privilege

See the description of this attribute defined for the Simple Variable class above.

Password

See the description of this attribute defined for the Simple Variable class above.

Access Groups

See the description of this attribute defined for the Simple Variable class above.

Access Rights

See the description of this attribute defined for the Simple Variable class above.

List of Local Detail

This attribute contains the local detail for each element (field) of the record.

6.2.4.3.3.3 Services

See the description of the services defined for the Simple Variable class above.

6.2.4.3.4 Variable list class specification

6.2.4.3.4.1 Formal model

FAL ASE:		VARIABLE ASE	
CLASS:	VARIABLE LIST		
CLASS ID:	10		
PARENT CLASS:	TOP		
ATTRIBUTES:			
1	(m)	Attribute:	Number of Entries
2	(m)	Attribute:	List Of Variables
3	(c)	Attribute:	Deletable
4	(m)	Attribute:	Access Privilege
4.1	(m)	Attribute:	Password
4.2	(m)	Attribute:	Access Groups
4.3	(m)	Attribute:	Access Rights
SERVICES:			
1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
3	(o)	OpsService:	Read List
4	(o)	OpsService:	Write List
5	(o)	OpsService:	Exchange
6	(o)	OpsService:	Exchange List
7	(o)	OpsService:	Information Report
8	(o)	OpsService:	Information Report List

6.2.4.3.4.2 Attributes

Number of Entries

This attribute contains the number of variables in the list.

List of Variables

This attribute identifies the variables (only simple, array or record variable objects) by Key Attribute that are contained in the list.

Deletable

This attribute indicates, when TRUE, that the variable list can be deleted using the Delete service. The value of this attribute is always TRUE for objects dynamically created with the Object Management ASE Create Service.

Access Privilege

This attribute specifies the access controls defined for this variable list. It is composed of the following:

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute identifies which of the eight user-defined access groups are defined for the variable list. More than one access group may be defined.

Access Rights

This attribute defines the type of access defined for the variable list. Valid values are:

- Right to Delete for Access Groups
- Right to Write for Access Groups
- Right to Read for Access Groups
- Right to Delete for the registered Password
- Right to Write for the registered Password
- Right to Read for the registered Password
- Right to Delete for all Communication Partners
- Right to Write for all Communication Partners
- Right to Read for all Communication Partners

6.2.4.3.4.3 Services

See the description of the services defined for the Simple Variable class above.

6.2.4.4 Variable ASE service specification

6.2.4.4.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Read
- Write
- Read List
- Write List
- Information Report
- Information Report List
- Exchange
- Exchange List

The exact nature of the operation of variable services is determined, in part, by the application relationship over which they operate.

6.2.4.4.2 Read service

6.2.4.4.2.1 Service overview

This confirmed service may be used to read the value of a variable object or variable list object. It may be used with application relationships configured to support the client/server model, or it may be used with application relationships configured to support the publisher/subscriber "pull" model.

6.2.4.4.2.2 Service primitives

The service parameters for this service are shown in Table 41.

Table 41 – Read service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variable Specifier	M	M(=)		
Key Attribute	S	S(=)		
Key Attribute and Component ID	S	S(=)		
Numeric Address and Data Length	S	S(=)		
Data Type Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Best Effort Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List of Access Results			M	M(=)
Error Status			S	S(=)
Returned Data			S	S(=)
Value			M	M(=)
Data Type			C	C(=)
Object Revision			C	C(=)
Timeliness			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Variable Specifier

This selector type parameter specifies the variable, variable list, or an element of an array or record variable.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component Identifier

This parameter identifies the field of a constructed data type variable by a combination of one of its key attributes and the field identifier of a data structure or an index into an array. This parameter may identify a component with a nesting level greater than one if supported by the AP.

Numeric Address/Data Length

This parameter provides the numeric address and the data length of the data to be read. The mapping between this parameter and the actual memory address in a real system is a local matter.

Data Type Requested

This optional parameter indicates that the data type of each variable should be returned with the data.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired. If the Object Revision is requested, but is not supported for the specified object, the service fails.

Best Effort Requested

This conditional, optional parameter is used to request to perform a “best effort” read of a variable list. In the “best effort” read, the service succeeds if at least one variable in the variable list can be read.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Access Result

This parameter contains the responses returned by the remote AP. If “best effort” was requested for a variable list, then a list of responses is returned. The order of the response in the list is the same as the order of the variables in the variable list.

Error Status

This selection type parameter is when “best effort” was requested for a variable list and one of the variables in the list could not be read. It indicates the reason for the read failure.

Returned Data

This selection type parameter contains the data returned by the remote AP. It is always present if “best effort” was not requested for a variable list.

Value

This parameter contains the value read. For each of the variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated together in the order that they appear in the list. If any of the variables in a variable list could not be read, the service fails.

Data Type

This optional parameter indicates that the data type of each value is returned.

Object Revision

This conditional parameter contains the object revision of the specified object. It is present if the object revision was requested.

Timeliness

This conditional parameter indicates the Data Link Layer timeliness status for the referenced variable, or variable list object. This parameter is present if it is supported on the specified AR.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Read Service is an "all or nothing" service. All or nothing means that the service succeeds only if the requested value, or values in the case of a variable list object, are read and returned.

If the object specified to read is a variable list object, the value, data type (when requested), and object revision (when requested) of the variables in the variable list are returned.

A timeliness parameter is included in the indication primitive if the AR that conveyed the APDU Body supports timeliness.

6.2.4.4.3 Read list service**6.2.4.4.3.1 Service overview**

This confirmed service is used to read the values of multiple variables or a single variable list object. It may be used with application relationships configured to support the client/server model.

It operates in a "best effort" fashion, such that the service succeeds if the value for at least one variable or one variable in a variable list is returned, otherwise it fails. When the variable was successfully accessed, the returned data shall be the variable value otherwise it shall be an error status.

6.2.4.4.3.2 Service primitives

The service parameters for this service are shown in Table 42.

Table 42 – Read List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Data Type Requested	U	U(=)		
List Of Variable Specifiers	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and Component ID(s)	S	S(=)		
List Of Numeric Addresses and Data Lengths	S	S(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List of Access Results			M	M(=)
List of Data Types			C	C(=)
List of Data			M	M(=)
Error Status			S	S(=)
Value			S	S(=)
Value with Object Revision			S	S(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Data Type Requested

This optional parameter is used to request that the type description of the List of data parameter be returned. A value of TRUE indicates that the type description is desired. A value of FALSE indicates that type description is not desired. This request can be used only with List of Variable Specifiers selection.

List of Variable Specifiers

This parameter individually identifies a list of variables and/or components of variables to be read or one variable list to be read. The nesting level of the components to be read can be equal to or greater than one.

Key Attribute

This selection type parameter specifies a Variable or Variable list Identifier key attribute values.

Key Attribute And Component ID(s)

This parameter identifies the components of a constructed data type variable by a combination of one of its key attributes and the components path ID(s).

List Of Numeric Addresses and Data Lengths

This parameter provides the numeric address and the data length of the variable objects to be read. The mapping between this parameter and the actual memory address in a real system is a local matter. This selection is allowed to access only the data of variable object

NOTE The Numeric Address parameters may be useful to specify a variable or a certain memory location in a called service user. These addresses are not global. One example of the use of the parameters is to specify a pointer to a target memory location.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired for all variables (if supported). A value of FALSE indicates that object revision is not desired for any variable.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Access Results

This parameter contains an indicator for each variable accessed. If an access succeeds the indicator shall be TRUE otherwise it shall be FALSE.

List of Data Types

This optional parameter provides the type description of the values in the List of Data parameter.

List of Data

This parameter contains one or more error statuses, variable values, or variable values with object revision read. The error statuses or values with/without object revision are concatenated in the order in which they were described in the request. For variable lists this is always the order of the list definition. For each variable or a variable in a variable list successfully accessed, the returned data shall be the value or the value with object revision of the variable object (if requested). Otherwise it shall be an error status. The returned data shall also be an error status if the object revision is requested and the variable object accessed does not support the object revision attribute.

Error Status

This parameter indicates the most probable reason why the operation failed using the error class and error code defined for the read service.

Value

This parameter contains a value read. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Value with Object Revision

This parameter contains a value read plus the object revision of the variable object. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Read List Service is a "best effort". Best effort means that the service succeeds if at least one value is read.

If the user is not able to read at least one of the values, the service fails and the user issues a Read List Service response (-) primitive indicating the reason.

When requested, the data type descriptions are returned. When requested, the object revision attribute value is returned concatenated with the value of the variable.

6.2.4.4.4 Write service

6.2.4.4.4.1 Service overview

This confirmed service is used to write the value of a variable. It is not used with the publisher/subscriber model.

6.2.4.4.4.2 Service primitives

The service parameters for this service are shown in Table 43.

Table 43 – Write service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)	M(=)	M(=)
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variable Specifier	M	M(=)		
Key Attribute	S	S(=)		
Key Attribute and Component ID	S	S(=)		
Numeric Address and Data Length	S	S(=)		
Value	M	M(=)		
Data Type	U	U(=)		
Object Revision	U	U(=)		
Best Effort Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List of Error Status			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Variable Specifier

This parameter specifies the variable, variable list, or an element of an array or record variable.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component Identifier

This parameter identifies the field of a constructed data type variable by a combination of one of its key attributes and the field identifier of a data structure or an index into an array. This parameter may identify a component with a nesting level greater than one if supported by the AP.

Numeric Address/Data Length

This parameter provides the numeric address and the data length of the data to be written. The mapping between this parameter and the actual memory address in a real system is a local matter.

Value

This parameter contains the value to write. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated together in the order that they appear in the list.

NOTE If any variable in a variable list object cannot be updated, none of the variables in the variable list object will be updated, and the write will fail.

Data Type

This optional parameter contains the data type of the values in the Value parameter.

Object Revision

This optional parameter contains the expected object revision of the variable object. When present, it indicates that the AP containing the variable is to compare the value of this parameter to the Object Revision of the variable before applying the write.

Best Effort Requested

This conditional, optional parameter is used to request to perform a “best effort” write of a variable list. In the “best effort” write, the service succeeds if at least one variable in the variable list can be written.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Status

This parameter contains more than one status if a “best effort” write was requested. The status indicates success or the reason for failure. The order in the list is the same as the order of the variables in the variable list.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Write Service is an “all or nothing” service. All or nothing means that the service succeeds only if the requested value, or values in the case of a variable list object, are successfully written.

6.2.4.4.5 Write list service

6.2.4.4.5.1 Service overview

This confirmed service is used to write the values of multiple variables or variables in a single variable list object. The number of variables referenced or defined in the list is a device local matter. It may be used with application relationships configured to support the client/server model.

It operates in a "best effort" fashion, such that the service succeeds if the value for at least one variable or variable in a variable list is written, otherwise it fails.

6.2.4.4.5.2 Service primitives

The service parameters for this service are shown in Table 44.

Table 44 – Write List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
List of Variable Specifiers	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and Component ID(s)	S	S(=)		
List of Numeric Addresses and Data Lengths	S	S(=)		
List of Data Types	U	U(=)		
List of Data	M	M(=)		
Value	S	S(=)		
Value with Object Revision	S	S(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List of Variable Data Access Statuses			M	M(=)
List of Error Statuses			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

List of Variable Specifiers

This parameter individually identifies multiple variables and/or components of variables to be written or a single variable list to be written. The nesting level of the components to be written can be equal to or greater than one.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component ID(s)

This parameter identifies components of a constructed data type variable by a combination of one of its key attributes and the component path ID(s).

List of Numeric Addresses and Data Lengths

This parameter provides the numeric address and the data length of the data to be written. The mapping between this parameter and the actual memory address in a real system is a local matter.

List of Data Types

This optional parameter provides the type description of the values in the List of Data parameter.

List of Data

This parameter contains one or more values or one or more values with object revision to write. The values with/without object revision are concatenated in the order in which they are described in the preceding parameters. For variable lists this is always the order of the list definition.

Value

This parameter contains a value to write. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Value with Object Revision

This parameter contains a value to write plus the expected object revision of the variable object. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Variable Data Access Statuses

This parameter contains an indicator for each variable accessed. If an access succeeds, the indicator shall be TRUE, otherwise it shall be FALSE.

List of Error Statuses

This parameter contains an error status for each individual variable or variable in variable list for which the access failed. The order of the error statuses is the same as the order of the objects identified in the service request.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.5.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Write List Service is a "best effort" service. Best effort means that the service succeeds if at least one value is written.

If the responding user is able to write the value for at least one of the requested variables, the user returns a Write List Service response (+) primitive.

If the user is not able to write at least one of the values, the user issues a Write List Service response (-) primitive indicating the reason.

6.2.4.4.6 Information report service

6.2.4.4.6.1 Service overview

This unconfirmed service is used by an application process to report the value of a variable to receiver(s) designated by the AR.

6.2.4.4.6.2 Service primitives

The service parameters for this service are shown in Table 45.

Table 45 – Information Report service

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Variable Specifier	C	C(=)
Key Attribute	S	S(=)
Key Attribute and Component ID	S	S(=)
Numeric Address and Data Length	S	S(=)
Value	M	M(=)
Object Revision	U	U(=)
Data Type	U	U(=)
Timeliness		C
Duplicate FAL PDU Body		C

Argument

This parameter carries the parameters of the service invocation.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It gives the remote address to which the requested Information Report should be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It identifies the source address from which the Information Report is to be sent.

Variable Specifier

This selector type parameter specifies the variable, variable list, or an element of an array or record variable.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component Identifier

This parameter identifies the field of a constructed data type variable by a combination of one of its key attributes and the field identifier of a data structure or an index into an array. This parameter may identify a component with a nesting level greater than one if supported by the AP.

Numeric Address and Data Length

This parameter provides the numeric address and data length reported.

Value

This parameter contains the value. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated together in the order that they appear in the list.

Object Revision

This optional parameter contains the expected object revision of the variable object. When present, it indicates that the calling service user wishes the server to check the current Object Revision parameter of the variable to be updated. If supported, the server only updates the value of a variable in the list if its object revision attribute is equivalent to the value of this parameter.

Data Type

This optional parameter contains the data type of the values in the Value parameter.

Timeliness

This conditional parameter indicates the Data Link Layer timeliness status for the referenced variable, or variable list object. It is present if timeliness is supported on the specified AR.

Duplicate FAL PDU Body

This conditional parameter indicates whether or not the receipt of a duplicate FAL PDU has been detected by the Data Link Layer. It is present if supported in the DL Mapping Attributes of the AREP.

6.2.4.4.6.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

A timeliness parameter is included in the indication primitive if the AR that conveyed the APDU Body supports timeliness.

6.2.4.4.7 Information report list service**6.2.4.4.7.1 Service overview**

This unconfirmed service is used by an application process to report a list of values or a list of values with object revision to receiver(s) designated by the AR. The number of variables referenced or defined in the list is a device local matter.

6.2.4.4.7.2 Service primitives

The service parameters for this service are shown in Table 46.

Table 46 – Information Report List service

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
List of Variable Specifiers	S	S(=)
Key Attribute	S	S(=)
Key Attribute and Component IDs	S	S(=)
List of Numeric Addresses and Data Lengths	S	S(=)
List of Data Types	U	U(=)
List of Data	M	M(=)
Value	S	S(=)
Value with Object Revision	S	S(=)
Timeliness		C
Duplicate FAL PDU Body		C

Argument

This parameter carries the parameters of the service invocation.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It gives the remote address to which the requested Information Report List is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated Information Report List is to be sent.

List of Variable Specifiers

This parameter individually identifies multiple variables and/or components of variables to be written or a single variable list to be written. The nesting level of the components to be written can be equal to or greater than one.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component ID(s)

This parameter identifies components of a constructed data type variable by a combination of one of its key attributes and the components path ID(s).

List of Numeric Addresses and Data Lengths

This parameter provides the numeric address and the data length of the data to be written. The mapping between this parameter and the actual memory address in a real system is a local matter.

List of Data Types

This optional parameter provides the type description of the values in the List of Data parameter.

List of Data

This parameter contains one or more values or one or more values with object revision to write. The values with/without object revision are concatenated in the order in which they are described in the preceding parameters. For variable lists this is always the order of the list definition.

Value

This parameter contains a value to write. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Value with Object Revision

This parameter contains a value to write plus the expected object revision of the variable object. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Timeliness

This conditional parameter indicates the Data Link Layer timeliness for the list of values. It is present if timeliness is defined for the specified AR.

Duplicate FAL PDU Body

This conditional parameter indicates whether or not the receipt of a duplicate FAL PDU has been detected by the Data Link Layer. It is present if supported in the DL Mapping Attributes of the AREP.

6.2.4.4.7.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

If the object specified to be reported is a variable list object, the values or the values with object revision of the variables in the variable list are concatenated in the List of Data parameter in the order that the variables appear in the variable list.

A timeliness parameter is included in the indication primitive if the AR that conveyed the APDU Body supports timeliness.

6.2.4.4.8 Exchange service**6.2.4.4.8.1 Service overview**

This confirmed service is used to write the value of one variable or variable list object, and read the value of another variable or a variable list object in a single operation. The order of processing the read and write by the responding user is not specified by the FAL.

6.2.4.4.8.2 Service primitives

The service parameters for this service are shown in Table 47.

Table 47 – Exchange service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variable to Write	M	M(=)		
Specifier	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and Element ID	S	S(=)		
Numeric Address and Data Length	S	S(=)		
Value	M	M(=)		
Data Type	U	U(=)		
Object Revision	U	U(=)		
Variable to Read	M	M(=)		
Specifier	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and ElementID	S	S(=)		
Numeric Address and Data Length	S	S(=)		
Data Type Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Write Result			M	M(=)
Read Response			M	M(=)
Error Status			S	S(=)
Returned Value			S	S(=)
Value			M	M(=)
Data type			C	C(=)
Object Revision			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Specifier for Variable to Write

This parameter specifies the variable, variable list, or variable field to be updated.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Element Identifier

This parameter identifies the field of a constructed data type variable by a combination of one of its key attributes and the field identifier of a data structure or an index into an array. This parameter may identify an element with a nesting level greater than one if supported by the AP.

Numeric Address/Data Length

This parameter provides the numeric address and the data length of the data to be written. The mapping between this parameter and the actual memory address in a real system is a local matter.

Value to Write

This parameter contains the value to write. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated together in the order that they appear in the list.

NOTE If any variable in a variable list cannot be updated, none of the variables in the variable list object will be updated, and the write will fail.

Data Type

This optional parameter contains the data type of the values to write in the Value parameter.

Object Revision

This optional parameter contains the expected object revision of the variable object to be updated. When present, it indicates that the AP containing the variable is to compare the value of this parameter to the Object Revision of the variable before applying the write.

Variable Specifier

This selector type parameter specifies the variable, variable list, or variable field to be read.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Element Identifier

This parameter identifies the field of a constructed data type variable by a combination of one of its key attributes and the field identifier of a data structure or an index into an array. This parameter may identify an element with a nesting level greater than one if supported by the AP.

Numeric Address/Data Length

This parameter provides the numeric address and the data length of the data to be read. The mapping between this parameter and the actual memory address in a real system is a local matter.

Data Type Requested

This optional parameter indicates that the data type of each variable should be returned with the data.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired. If the Object Revision is requested, but is not supported for the specified object, the service fails.

Result (+)

The Result (+) parameter indicates that the service request succeeded.

Write Status

This parameter indicates whether or not the variable was updated as requested. If it was not updated, it indicates the most probable reason why the operation failed using the error class and error code defined for the write service.

Read Response

This parameter provides the data value corresponding to the Variable Specifier to Read parameter in the request primitive. This parameter indicates the result of the read operation.

Error Status

This parameter is used to indicate that the access to the specified object was not successful. It indicates the most probable reason why the operation failed using the error class and error code defined for the read service.

Returned Value

This parameter contains the value of the referenced variable or variable list object. If the identified variable was a variable list object, then this parameter will contain a concatenated set of values for each variable in the list.

Value

This parameter contains the value read. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated together in the order that they appear in the list.

NOTE Because there are no identifiers in a concatenated list of values for a variable list object, a read failure for any variable in the variable list object causes the service to fail.

Data Type

This optional parameter indicates that the data type of each value returned.

Object Revision

This conditional parameter contains the object revision of the specified object. It is present if the object revision was requested.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.8.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Exchange Service is a "best effort" service. Best effort means that the service succeeds if either of the requested read or write operations succeeds.

If a variable list was specified for the variable to write, the value and object revision (when present) of each variable in the variable list are concatenated in the Exchange Request APDU Body in the order that the variables appear in the variable list.

If a variable list was specified for the variable to read, the value and object revision (when present) of each variable in the variable list are concatenated in the Exchange Response APDU Body in the order that the variables appear in the variable list.

6.2.4.4.9 Exchange list service

6.2.4.4.9.1 Service overview

This confirmed service is used to write multiple variable objects or single components of multiple variable objects or a single variable list object and to read multiple variable objects or single components of multiple variable objects or a single variable list object in a single operation. The order of processing the read and write by the responding user is not specified by the FAL.

6.2.4.4.9.2 Service primitives

The service parameters for this service are shown in Table 48.

Table 48 – Exchange List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variables to Write	M	M(=)		
List of Variable Specifiers	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and Component IDs	S	S(=)		
List of Numeric Addresses and Data Lengths	S	S(=)		
List of Data Types	U	U(=)		
List of Data	M	M(=)		
Value	S	S(=)		
Value with Object Revision	S	S(=)		
Variables to Read	M	M(=)		
List of Variable Specifiers	S	S(=)		
Key Attribute	S	S(=)		
Key Attribute and Component IDs	S	S(=)		
List of Numeric Addresses and Data Lengths	S	S(=)		
Data Type Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List of Write Access Results			M	M(=)
List of Error Statuses			C	C(=)
List of Read Access Results			M	M(=)
List of Data Types			C	C(=)
List of Data			M	M(=)
Error Status			S	S(=)
Value			S	S(=)
Value With Object Revision			S	S(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Variables to Write

This parameter contains information related to the variables to be updated.

List of Variable Specifiers

This parameter individually identifies multiple variables and/or components of variables to be written or a single variable list to be written. The nesting level of the components to be written can be equal to or greater than one.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component ID(s)

This parameter identifies the component of a constructed data type variable by a combination of one of its key attributes and the components path ID(s).

List of Numeric Addresses and Data Lengths

This parameter provides the numeric address and the data length of the data to be written. The mapping between this parameter and the actual memory address in a real system is a local matter.

List of Data Types

This optional parameter provides the type description of the values in the List of Data parameter.

List of Data

This parameter contains one or more values or one or more values with object revision to write. The values with/without object revision are concatenated in the order in which they are described in the preceding parameters. For variable lists this is always the order of the list definition.

Value

This parameter contains a value to write. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Value with Object Revision

This parameter contains a value to write plus the expected object revision of the variable object. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Variables to Read

This parameter carries information related to the variables to be read.

List of Variable Specifiers

This parameter individually identifies multiple variables and/or components of variables to be written or a single variable list to be read. The nesting level of the components to be read can be equal to or greater than one.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Key Attribute and Component ID(s)

This parameter identifies the component of a constructed data type variable by a combination of one of its key attributes and the component path ID(s).

List of Numeric Addresses and Data Lengths

This parameter provides the numeric address and the data length of the data to be read. The mapping between this parameter and the actual memory address in a real system is a local matter.

Data Type Requested

This optional parameter is used to request that the data type be returned. A value of TRUE indicates that data type is desired. A value of FALSE indicates that data type is not desired.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired.

Result (+)

The Result (+) parameter indicates that the service request succeeded.

List of Write Access Results

This parameter contains an indicator for each variable accessed. If an access succeeds the indicator shall be TRUE otherwise it shall be FALSE.

List of Error Statuses

This parameter contains an error status for each individual variable or variable in variable list for which the access failed. The order of the error statuses is the same as the order of the objects identified in the service request.

List of Read Access Results

This parameter contains an indicator for each variable accessed. If an access succeeds the indicator shall be TRUE otherwise it shall be FALSE.

List of Data Types

This optional parameter provides the type description of the values in the List of Data parameter.

List of Data

This parameter contains one or more error statuses, values or values with object revision read. The error statuses or values with/without object revision are concatenated in the order in which they were described in the request. For variable lists this is always the order of the list definition. For each variable or a variable in a variable list successfully accessed, the returned data shall be the value or the value with object revision of the variable object (if requested). Otherwise it shall be an error status. The returned data shall also be an error status if the object revision is requested and the variable object accessed does not support the object revision attribute.

Error Status

This parameter indicates the most probable reason why the operation failed using the error class and error code defined for the read service.

Value

This parameter contains a value read. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Value with Object Revision

This parameter contains a value read plus the object revision of the variable object. For variables, this parameter contains the value of the variable. For variable lists, this parameter contains the values of each of the variables in the list concatenated in the order in which they appear in the list.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.4.4.9.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Exchange Service is a "best effort" service. Best effort means that the service succeeds if either of the requested read or write operations succeeds.

The write operation succeeds if the access to any variables of the list or any variable in the variable list succeeds.

The read operation succeeds if any variable accessed in the list or variables in the variable list succeeds.

6.2.5 Event ASE

6.2.5.1 Overview

The FAL event model defines three event related objects, event objects for the definition of events and their messages, event notifiers for the distribution of event messages, and event lists for the acquisition of event summary information from a group of event objects.

Event objects are used to define messages used to report event occurrences. Event messages contain information that identifies and describes occurrences of events. To simplify the processing of event messages, event objects are defined to generate messages using one of four defined formats.

NOTE For example, event objects may be defined that generate event messages containing a time-tag and/or a message count for event occurrences.

Notifiers are responsible for collecting event messages from event objects, and distributing one or more in a single invocation of the FAL event notification service. The number of event messages that may be submitted in a single service invocation is limited by the maximum APDU size that can be transferred by the AR. Figure 18 illustrates this concept.

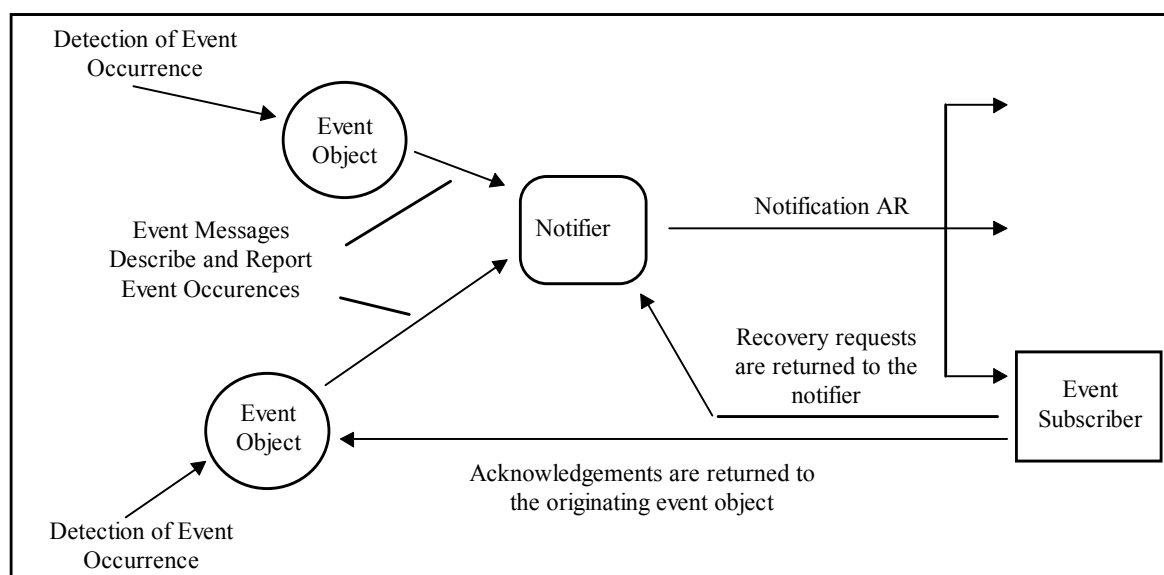


Figure 18 – Event model overview

To provide flexibility in the event notification process notifiers may be defined to use one of four types of event notifications to carry event messages. Once defined, a notifier always uses the same type of event notification to carry one or more different types of event messages.

If an application process fails to receive one or more event notifications, a notification recovery service is provided for it to request a retransmission from the notifier.

In addition, the get event summary service is provided to permit an application process to query one or more event objects to select those that meet certain criteria pertaining to the current state of the event objects. An event list object is defined that allows event objects to be grouped together to simplify the query process.

The AR endpoint class used to distribute event notifications is specified by one of the notifier attributes. An event attribute is used to specify which AR distributes the notifications.

Queued ARs are most often used because they prevent event notifications from being overwritten before they are read. When the purpose of the notification is to signal a change in the state of a field device AP, buffered ARs may be used.

NOTE When no events have occurred during a specified period of time notifiers may choose to generate a special "heartbeat" event notification that indicates to subscribers that the notifier is still alive. These special notifications look like the last normal notification sent except that they do not contain the event messages. Heartbeat notifications are defined to permit event subscribers to determine if they have missed an event notification.

In this model, application processes are responsible for providing the functions for event, notifier, and event list objects, and the FAL is responsible for providing communication services designed specifically for them. The application process detects events, builds event messages and aggregates them together. It distributes the aggregated set using the FAL event notification service. At the receiving end, subscriber application processes can use one of two event acknowledgment services to acknowledge event occurrences.

The remainder of this subclause specifies the class definitions and services for event, notifier, and event list objects.

6.2.5.2 Event model class specification

6.2.5.2.1 Event class specification

6.2.5.2.1.1 Formal model

FAL ASE:		EVENT ASE	
CLASS:	EVENT		
CLASS ID:		4	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1	(m)	Attribute:	Event Status
1.1	(o)	Attribute:	Active (TRUE,FALSE)
1.2	(o)	Attribute:	Detected (TRUE,FALSE)
1.3	(m)	Attribute:	Enabled (TRUE,FALSE)
1.4	(o)	Attribute:	Acknowledged (TRUE,FALSE)
2	(m)	Attribute:	Message Type (SIMPLE, REPORTED EVENT COUNT, EVENT DETECTION TIME, COMPOSITE, SIMPLE WITH DATA, REPORTED EVENT COUNT WITH DATA, EVENT DETECTION TIME WITH DATA, COMPOSITE WITH DATA)
3	(m)	Attribute:	Access Privilege
3.1	(m)	Attribute:	Password
3.2	(m)	Attribute:	Access Groups
3.3	(m)	Attribute:	Access Rights

- | | | | |
|-------|-----|-------------|---|
| 4 | (o) | Attribute: | Last Reported Event Info |
| 4.1 | (o) | Attribute: | Count |
| 4.2 | (o) | Attribute: | Detection Time |
| 5 | (c) | Constraint: | Message Type = SIMPLE WITH DATA
REPORTED EVENT COUNT WITH DATA
EVENT DETECTION TIME WITH DATA
COMPOSITE WITH DATA) |
| 5.1 | (c) | Attribute: | Event Data Specifier |
| 5.1.1 | (s) | Attribute: | Variable ID |
| 5.1.2 | (s) | Attribute: | Data Type ID and Length |
| 6 | (c) | Constraint: | Event Status.Acknowledgment Supported = TRUE |
| 6.1 | (o) | Attribute: | Acknowledgment Data Specifier |
| 6.1.1 | (s) | Attribute: | Variable ID |
| 6.1.2 | (s) | Attribute: | Data Type ID and Length |
- SERVICES:**
- | | | | |
|---|-----|------------|--------------------------|
| 1 | (o) | OpService: | Acknowledge Event |
| 2 | (o) | OpService: | Acknowledge Event List |
| 3 | (o) | OpService: | Enable Event |
| 4 | (o) | OpService: | Enable Event List |
| 5 | (o) | OpService: | Get Event Summary |
| 6 | (o) | OpService: | Get Event Summary List |
| 7 | (o) | OpService: | Query Event Summary List |

6.2.5.2.1.2 Attributes

Event Status

This attribute indicates the status of the event. The status is indicated as a combination of Boolean attributes: active, detected, enabled, acknowledgment support, and acknowledged.

Active

This optional attribute indicates, when TRUE, that events are capable of being detected. If FALSE, event occurrences are not detected.

NOTE The notifier object also has an enable attribute used to control whether or not the event notifications are to be generated.

Detected

This optional attribute indicates, when TRUE, that the conditions for detecting an event occurrence are TRUE.

Enabled

This attribute indicates, when TRUE, that event messages are to be reported to the related notifier upon detection of event occurrences. If FALSE, event messages are not to be generated and reported when event occurrences are detected.

NOTE The notifier object also has an enable attribute used to control whether or not the event notifications are to be generated.

Acknowledged

This optional attribute indicates, when TRUE, that all event messages generated for this event object have been acknowledged. This attribute will also be TRUE if no event messages have been generated or if the event object does not support acknowledgment. The value FALSE indicates that the event object is waiting for an acknowledgment.

Message Type

This attribute specifies which type of event message is generated when an event occurs. The defined messages types and their structure are:

Type	Contents
SIMPLE MSG	Event Numeric ID
REPORTED EVENT COUNT	Event Numeric ID Last Detected Event Info.Count
EVENT DETECTION TIME	Event Numeric ID Last Reported Event Info.Detection Time
COMPOSITE	Event Numeric ID Last Reported Event Info.Count Last Reported Event Info.Detection Time
SIMPLE MSG WITH DATA	Event Numeric ID Event Data Identified by the Event Data Specifier
REPORTED EVENT COUNT WITH DATA	Event Numeric ID Last Detected Event Info.Count Event Data Identified by the Event Data Specifier
EVENT DETECTION TIME WITH DATA	Event Numeric ID Last Reported Event Info.Detection Time Event Data Identified by the Event Data Specifier
COMPOSITE WITH DATA	Event Numeric ID Last Reported Event Info.Count Last Reported Event Info.Detection Time Event Data Identified by the Event Data Specifier

Access Privilege

This attribute specifies the access controls defined for this event. It is composed of the following:

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute identifies which of the eight user-defined access groups are defined for the event. More than one access group may be defined.

Access Rights

This attribute defines the type of access defined for the event. Valid values are:

- Right to Acknowledge for registered Password
- Right to Enable/Disable for registered Password
- Right to Acknowledge for the Access Groups
- Right to Enable/Disable for the Access Groups
- Right to Acknowledge for all Communication Partners
- Right to Enable/Disable for all Communication Partners

Last Reported Event Info

This optional attribute contains the count and time of detection of the last reported event.

Count

This optional attribute counts the number of reported event occurrences. It is incremented by one for each event occurrence detected and reported. When the event object is disabled, detected event occurrences are not counted.

When the value of this attribute rollover, it rollover to 1, skipping 0. The value 0 is always used to indicate that no event messages have been generated. If the message type attribute is set to REPORTED EVENT COUNT or COMPOSITE, its updated value is included in event messages.

NOTE The presence of this attribute is independent of whether it is contained in an event message. Its presence indicates this attribute is accessible using the FAL management services. FAL is not involved with incrementing the value of this attribute. Therefore, the FAL does not know when its value rollover, or if it rollover properly.

Detection Time

This optional attribute specifies the time that the last reported event was detected. If the message type attribute is set to EVENT DETECTION TIME or COMPOSITE its value is included in event messages.

NOTE The presence of this attribute is independent of whether it is contained in an event message. Its presence indicates this attribute is accessible using the FAL management services.

Event Data Specifier

This conditional-type attribute specifies user data to be included in an event message. Event data is specified by either identifying a variable object to be incorporated into an event message, or by simply specifying its data type and length. This attribute is present when the Message Type indicates that event data is to be included in event messages.

Variable ID

This selection type attribute identifies a variable object or a variable list object contained in the same AP as the event whose value is to be incorporated into an event message.

Data Type ID and Length

This selection type attribute identifies the data type and length of the user data that is to be incorporated into an event message. Depending on the data type, the length may be defined by the data type.

Acknowledgment Data Specifier

This optional attribute specifies user data to be included in an acknowledgment message. Acknowledgment data is specified by either identifying a variable object to be incorporated into an acknowledgment message, or by simply specifying its data type.

Variable ID

This selection type attribute identifies a variable object or a variable list object contained in the same AP as the event whose value is to be incorporated into an acknowledgment message.

Data Type ID and Length

This selection type attribute identifies the data type and length of the user data that is to be incorporated into an acknowledgment message.

6.2.5.2.1.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be selected.

Acknowledge Event

This optional confirmed service is provided to permit a subscriber of an event to acknowledge receipt of an event message.

Acknowledge Event List

This optional confirmed service is provided to permit a subscriber of an event to acknowledge receipt of a list of event messages generated by one or more specified event objects.

Enable Event

This optional service is provided to permit a subscriber of an event to enable an event.

Enable Event List

This optional service is provided to permit a subscriber of an event to enable a list of events.

Get Event Summary

This optional service provides the ability to retrieve summary information for one event.

Get Event Summary List

This optional service provides the ability to retrieve summary information for a list of events.

Query Event Summary List

This optional service provides the ability to query a set of events for summary information based on selection criteria.

6.2.5.2.2 Event list class specification

6.2.5.2.2.1 Formal model

FAL ASE:		EVENT ASE
CLASS:	EVENT LIST	
CLASS ID:		17
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m) Attribute:	Number of Entries
2	(m) Attribute:	List Of Events
SERVICES:		
1	(o) OpService:	Get Event Summary List
2	(o) OpService:	Query Event Summary List

6.2.5.2.2.2 Attributes

Number of Entries

This attribute contains the number of events in the list.

List of Events

This attribute identifies the event objects are contained in the event list.

6.2.5.2.2.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be selected.

Get Event Summary List

This optional service provides the ability to retrieve summary information for a list of events.

Query Event Summary

This optional service provides the ability to query a set of events for summary information based on selection criteria.

6.2.5.2.3 Notifier class specification

6.2.5.2.3.1 Formal model

This subclause provides formal class definitions for notifier objects. Notifiers provide the ability to send event notifications composed of event occurrence messages from one or more event objects.

FAL ASE:		EVENT ASE
CLASS:	NOTIFIER	
CLASS ID:		18
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m) Attribute:	Enabled (TRUE,FALSE)
2	(m) Attribute:	Notification AREP Class
3	(m) Attribute:	Notification AREP
4	(m) Attribute:	Notification Type (BASIC, SEQUENCED, TIME OF NOTIFICATION, COMPOUND)
5	(o) Attribute:	Contained Message Type (SIMPLE, REPORTED EVENT COUNT, EVENT DETECTION TIME, COMPOSITE)
6	(o) Attribute:	Contained Event Data (TRUE, FALSE)
7	(o) Attribute:	Last Notification Sequence Number
8	(o) Attribute:	List Of Events
SERVICES:		
1	(m) OpsService:	Event Notification
2	(o) OpsService:	Notification Recovery

6.2.5.2.3.2 Attribute

Enabled

When TRUE, this attribute indicates that the notifier object is enabled. When enabled the notifier object groups event messages from one or more event objects together and submits them in a single invocation of the event notification service to the FAL for conveyance.

When FALSE, the notifier object is disabled. When disabled, the notifier is inactive. It does not issue any event notification service requests.

Notification AREP Class

This attribute identifies the class of the AREP required to convey event notifications.

Notification AREP

This attribute identifies the AREP configured to convey event notifications. This AREP is also the AREP used for reporting the event notifications generated as the result of an event recovery request.

Notification Type

This attribute specifies the type of notification issued by the notifier. The defined notification types are:

Type Contents

BASIC Notifier ID
 SEQUENCED Notifier ID
 Sequence Number
 TIME OF NOTIFICATION Notifier ID
 Notification Time-Tag
 COMPOUND Notifier ID
 Sequence Number
 Time-Tag

Contained Message Type

This attribute specifies the types of event messages that may be contained in notifications issued by the notifier. The event message types are defined by the event object. When an event notification is constructed by this notifier object, the types of event messages that may be contained in the event notification are defined by this attribute. The defined types are:

Contained Message Type	Description
SIMPLE	All event messages in all notifications are of type SIMPLE.
REPORTED EVENT COUNT	All event messages in all notifications are of type REPORTED EVENT COUNT.
EVENT DETECTION TIME	All event messages in all notifications are of type EVENT DETECTION TIME.
COMPOSITE	All event messages in all notifications are of type COMPOSITE.
SIMPLE WITH DATA	All event messages in all notifications are of type SIMPLE.
REPORTED EVENT COUNT WITH DATA	All event messages in all notifications are of type REPORTED EVENT COUNT WITH DATA.
EVENT DETECTION TIME WITH DATA	All event messages in all notifications are of type EVENT DETECTION TIME WITH DATA.
COMPOSITE WITH DATA	All event messages in all notifications are of type COMPOSITE WITH DATA.

Contained Event Data

This attribute, when TRUE, indicates that event messages conveyed by the notifier may contain event data. Event messages for each of the event message types may contain data. This attribute indicates whether or not event messages that are defined for an event object to contain data are capable of being transferred in an event notification generated by this notifier object.

Last Notification Sequence Number

The conditional attribute contains the last sequence number used. It is incremented for each event notification service invocation. When the value of this attribute rolls-over, it rolls-over to 1, skipping 0. The value 0 is always used to indicate that no event notifications have been generated. It is mandatory when the value of notification type is SEQUENCE or COMPOUND.

NOTE The FAL does not check to see if the roll-over has been performed properly.

List of Events

This optional attribute identifies the events that are configured for this notifier.

6.2.5.2.3.3 Services**Event Notification**

This service is provided to permit notifier object to report the occurrence of one or more events in a single service invocation.

Notification Recovery

This optional service is provided to permit one or more event notification subscribers to request that the notifier resend one or more event notifications that it has retained. If this service is supported, then the Sequence Number attribute is required.

6.2.5.3 Event ASE service specification**6.2.5.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Acknowledge Event
- Acknowledge Event List
- Enable Event
- Enable Event List
- Event Notification
- Notification Recovery
- Get Event Summary
- Get Event Summary List
- Query Event Summary

6.2.5.3.2 Acknowledge event**6.2.5.3.2.1 Service overview**

This service allows the acknowledgment of a reported event.

6.2.5.3.2.2 Service primitives

The service parameters for this service are shown on Table 49.

Table 49 – Acknowledge Event

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Reported Event Count	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Reported Event Count

This parameter is used to select a previously generated event message by the reported event count.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The association between the event identified in this service and the corresponding event object in the receiving AP is performed by the FAL user.

6.2.5.3.3 Acknowledge event list service

6.2.5.3.3.1 Service overview

This confirmed service is used by an AP receiving an event message to acknowledge receipt of one or more event messages. The service contains an acknowledgment policy parameter that indicates for each specified event message, whether only that event message is to be acknowledged or whether all event messages that were generated by the same event object up to and including the specified event are to be acknowledged. The Acknowledge Event List is a best effort service.

6.2.5.3.3.2 Service primitives

The service parameters for this service are shown in Table 50.

Table 50 – Acknowledge Event List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Acknowledgment Policy	C	C(=)		
Messages to Ack	M	M(=)		
List Of Event ID	M	M(=)		
List Of Message Specifier	U	U(=)		
Reported Event Count	U	U(=)		
Event Detection Time	U	U(=)		
List Of Acknowledgment Data	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List Of Access Result			M	M(=)
List Of Error			C	C(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Acknowledgment Policy

This conditional parameter indicates whether only the specified message is to be acknowledged or whether all messages up to and including the specified message are to be acknowledged. It is present only when the List of Messages to Ack parameter identifies a single message.

Messages to Ack

This parameter contains the messages of all events to be acknowledged.

List of Event ID

This parameter identifies the events being acknowledged by this service.

List of Message Specifier

This optional parameter specifies which messages are to be acknowledged for each event by this service invocation. If this parameter is not present, all messages for the event are to be acknowledged.

If the acknowledgment policy indicates ONLY THE SPECIFIED then only the message identified by this parameter is to be acknowledged.

If the acknowledgment policy indicates UP TO AND INCLUDING, then the message identified by this parameter, and all messages generated before it, are to be acknowledged.

Reported Event Count

This optional parameter is used to select a previously generated event message by the reported event count.

Event Detection Time

This optional parameter is used to select a previously generated event message by the time of the event detection.

List of Acknowledgment Data

This optional parameter contains the user data to be included in an event acknowledgment in addition to that used to identify the event occurrence. The type of this data is specified by the Acknowledgement Data Specifier attribute of the Event Object.

Result(+)

This selection informs the client that the server was able to successfully acknowledge at least one event object among those appearing in the request.

List of Access Result

This parameter contains the failure/success result of all the event objects acknowledged.

List of Error

This parameter indicates the reason for failure for each failed acknowledgement.

Result(-)

This selection indicates that the service request failed.

6.2.5.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The association between the events referenced by the Event Acknowledgement service and the corresponding event object in the receiving AP is performed by the FAL user.

6.2.5.3.4 Enable event service

6.2.5.3.4.1 Service overview

This service allows the enabling or disabling of the Event Object.

6.2.5.3.4.2 Service primitives

The service parameters for this service are shown in Table 51.

Table 51 – Enable event

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Enable Flag	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Key Attribute

This parameter identifies the variable or variable list by one of its key attributes.

Enable Flag

This Boolean parameter, when TRUE, indicates that the event object is to be enabled, and when FALSE, that the event object is to be disabled.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.5.3.5 Event notification service

6.2.5.3.5.1 Service overview

This unconfirmed service is used by a notifier of an FAL AP to notify other APs that one or more events have occurred. When this service is used without including a list of event messages, the event notification is regarded as a “heartbeat” event notification. This service is unconfirmed.

6.2.5.3.5.2 Service primitives

The service parameters for this service are shown in Table 52.

Table 52 – Event Notification service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Notifier ID	C	C(=)
Sequence Number	U	U(=)
Notification Time	U	U(=)
List of Event Messages	U	U(=)
List of Event Key Attributes	M	M(=)
List of Event Data Types	C	C(=)
List of EventMessage Contents	C	C(=)
Reported Event Count	C	C(=)
Event Detection Time	C	C(=)
Event Data	C	C(=)

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the destination address to which the requested Event Notification is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated Event Notification is to be sent.

Notifier ID

This conditional parameter identifies the notifier issuing the event notification. It is present if the AP has more than one notifier defined for it.

Sequence Number

This optional parameter is the sequence number for the event notification. It may be used for notification recovery purposes.

Notification Time

This optional parameter is the time tag for the event notification.

List of Event Messages

This optional parameter contains the list of event messages that are to be reported. It may contain messages from one or more event objects, each containing the same set of parameters (specified by the Contained Message Type attribute of the Notifier object). The contents of each message are specified by its event object and shall be consistent with that specified for the Notifier object. When this parameter is not present, the event notification is treated as a “heartbeat” notification.

List of Event Key Attributes

This parameter identifies each of the specific events being acknowledged by this service.

List of Event Data Types

This conditional, optional parameter indicates the data type of each of the event data parameters. This parameter may be present only if the event data parameter is present. If the event data parameter is present, this parameter may be present, but is not required to be.

List of Event Message Contents

This conditional, optional parameter contains the event messages generated by the event objects.

Reported Event Count

This conditional parameter reports the event count for the occurrence being reported. This parameter is present only if it is defined for the message type of the specified event object and if that message type is supported by the specified notifier.

Event Detection Time

This optional parameter reports the time of the event detection. This parameter is present only if it is defined for the message type of the specified event object and if that message type is supported by the specified notifier.

Event Data

This conditional parameter contains user data to be included in an event message in addition to that used to identify the event occurrence. This parameter is present only if it is defined for the specified event object and if it is supported by the specified notifier.

6.2.5.3.5.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.2.5.3.6 Enable event list service**6.2.5.3.6.1 Service overview**

This service allows the enabling or disabling of the Event Object.

6.2.5.3.6.2 Service primitives

The service parameters for this service are shown in Table 53.

Table 53 – Enable Event List

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
List Of Key Attributes	M	M(=)		
List Of Enable Flags	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
List Of Enable Result			M	M(=)
List Of Error			C	C(=)
Result(-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

List of Key Attributes

This parameter identifies each event by one of its key attributes.

List of Enable Flags

This parameter, indicates for each event, whether the corresponding event object is to be enabled (when the respective event flag is TRUE), or to be disabled (when the respective event flag is FALSE).

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Enable Result

This list of Boolean parameter indicates for each event that was to be enabled or disabled, if the Enable or Disable action was successful (Enable Result = TRUE) or failed (Enable Result = FALSE).

List of Error

This list of error parameter provides information on the kind of error occurred for each event whose Enable or Disable Action failed.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.6.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service. The Enable Eventlist is a “best effort” service.

6.2.5.3.7 Notification recovery service

6.2.5.3.7.1 Service overview

This unconfirmed service is used to request that a specified number of retained event notifications be returned. Notifications are returned using the event notification service.

6.2.5.3.7.2 Service primitives

The service parameters for this service are shown in Table 54.

Table 54 – Notification recovery service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Notifier ID	M	M(=)
Number To Recover	U	U(=)
Sequence Number	U	U(=)

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the destination address to which the requested notification recovery request is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated notification recovery request is to be sent.

Notifier ID

This parameter identifies the notifier to which this service is directed.

Number to recover

This optional parameter specifies the number of event notifications to be re-sent. If this number is greater than the actual number stored, then only the available notifications will be sent. The default for this is one.

Sequence Number

This optional parameter specifies the sequence number of the oldest event notification to be re-sent. If not present, the last notification sent is being requested. It may be used only when the specified notifier uses sequence numbers.

6.2.5.3.7.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

Recovery is performed by the responding AP by re-issuing Event Notification service requests containing the notification messages to be recovered.

6.2.5.3.8 Get event summary service

6.2.5.3.8.1 Service overview

This confirmed service is used to request summary information for a specified event object.

6.2.5.3.8.2 Service primitives

The service parameters for this service are shown in Table 55.

Table 55 – Get Event Summary service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Event ID	M	M(=)		
Recovered Message Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Event Status			M	M(=)
Active			M	M(=)
Detected			M	M(=)
Enabled			M	M(=)
Acknowledgment Supported			M	M(=)
Acknowledged			M	M(=)
Object Revision			C	C(=)
Recovered Message			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Event ID

This mandatory parameter identifies the event object for which the event summary is being requested. An event list object cannot be specified because the members of the list may be dynamically changed by the AP, making it necessary to identify each event in the response.

Recovered Message Requested

This optional parameter specifies, when TRUE, that the last message generated by the event is to be returned.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired. If the Object Revision is requested, but is not supported for the specified object, the service fails.

Result(+)

This selection indicates that the service request succeeded.

Event Status

This parameter contains the values of the event status attribute. It indicates the status of the event. The status is indicated as a combination of Boolean parameters: active, detected, enabled, acknowledgment support, and acknowledged.

Detected

This Boolean parameter indicates, when TRUE, that the conditions for detecting an event occurrence are TRUE.

Active

This Boolean parameter indicates, when TRUE, that events are capable of being detected. If FALSE, event occurrences are not detected.

Enabled

This Boolean parameter indicates, when TRUE, that event messages are to be reported to the related notifier upon detection of event occurrences. If FALSE, event messages are not to be generated and reported when event occurrences are detected.

Acknowledgment Supported

This Boolean parameter indicates, when TRUE, that this event object supports the acknowledgment of event messages.

Acknowledged

This Boolean parameter indicates, when TRUE, that all event messages generated for this event object have been acknowledged. This attribute will also be TRUE if no event messages have been generated or if the event object does not support acknowledgment. The value FALSE indicates that the event object is waiting for an acknowledgment.

Recovered Message

This conditional parameter is present if the service request specified message recovery was requested. When present, this parameter contains the recovered event message.

Object Revision

This conditional parameter contains the object revision of the specified object. It is present if the object revision was requested.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.8.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Get Event Summary Service is an "all or nothing" service that operates through a queue or buffer. All or nothing means that the service succeeds only if the requested summary is returned.

6.2.5.3.9 Get event summary list service**6.2.5.3.9.1 Service overview**

This confirmed service is used to request summary information for a list of specified event object.

6.2.5.3.9.2 Service primitives

The service parameters for this service are shown in Table 56.

Table 56 – Get Event Summary List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Summary Requests	M	M(=)		
Requested Events	M	M(=)		
All Events	S	S(=)		
List of Events	S	S(=)		
Recovered Message Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Event Summaries			M	M(=)
List Of Event ID			C	C(=)
List Of Access Result			M	M(=)
List Of Response			M	M(=)
Error Status			S	S(=)
Event Summary			S	S(=)
Event Status			M	M(=)
Active			M	M(=)
Detected			M	M(=)
Enabled			M	M(=)
Acknowledgment Supported			M	M(=)
Acknowledged			M	M(=)
Object Revision			C	C(=)
Recovered Message			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Summary Requests

This mandatory parameter specifies a list of events, and the summary information for each, that is being requested. Each entry in the list may identify either an event or an event list object. Each entry in the list also identifies the summary information that is being requested. If an entry identifies an event list, then the summary information requested will apply to all events in the event list.

Requested Events

This mandatory parameter indicates that summaries are being requested for all events, or it contains a list that identifies the event objects for which event summaries are being requested.

All Events

This selection type parameter indicates that summaries are being requested for all events.

List of Events

This selection type parameter is a list that contains the identifiers of the events and/or event lists for which event summaries are being requested. When selected, the list shall contain at least one entry.

Recovered Message Requested

This optional parameter specifies, when TRUE, that the last message generated by the event is to be returned.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired. If the Object Revision is requested, but is not supported for the specified object, the service fails.

Result(+)

This selection indicates that the service request succeeded.

Event Summaries

This parameter contains a response for each requested event. The order of the information is the same as the order in which the objects were referenced in the service request.

List of Event ID

This conditional parameter identifies each event for which an event summary or error status is returned.

List of Access Result

This list of Boolean parameters provides for each event for which an event summary was requested an information, whether the access was successful (Access Result = TRUE) or not (Access Result = FALSE).

List of Response

This parameter contains an error status indicator or the event summary for each requested event. The order of the information is the same as the order in which the objects were referenced in the service request.

Error Status

This selection type parameter specifies if the condition which triggers the reporting of the event is active. A value of TRUE indicates that the underlying condition which

triggers the event is active. A value of FALSE indicates that the underlying condition which triggers the event is not active.

Event Summary

This selection type parameter contains the summary information for the specified event.

Event Status

This parameter contains the values of the event status attribute. It indicates the status of the event. The status is indicated as a combination of Boolean parameters: active, detected, enabled, acknowledgment support, and acknowledged.

Detected

This Boolean parameter indicates, when TRUE, that the conditions for detecting an event occurrence are TRUE.

Active

This Boolean parameter indicates, when TRUE, that events are capable of being detected. If FALSE, event occurrences are not detected.

Enabled

This Boolean parameter indicates, when TRUE, that event messages are to be reported to the related notifier upon detection of event occurrences. If FALSE, event messages are not to be generated and reported when events occurrences are detected.

Acknowledgment Supported

This Boolean parameter indicates, when TRUE, that this event object supports the acknowledgment of event messages.

Acknowledged

This Boolean parameter indicates, when TRUE, that all event messages generated for this event object have been acknowledged. This attribute will also be TRUE if no event messages have been generated or if the event object does not support acknowledgment. The value FALSE indicates that the event object is waiting for an acknowledgment.

Recovered Message

This conditional parameter is present if the service request specified message recovery was requested. When present, this parameter contains the recovered event message.

Object Revision

This conditional parameter contains the object revision of the specified object. It is present if the object revision was requested.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.9.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Get Event Summary List Service is a "best effort" service that operates through a queue or buffer. Best effort means that the service succeeds if at least one event summary is returned. If the user is not able to read at least one of the values, the service fails and the user issues a Get Event Summary List Service response (-) primitive indicating the reason.

6.2.5.3.10 Query event summary list service

6.2.5.3.10.1 Service overview

This confirmed service provides the means for an application to request that all events, an event list, or a list of events be searched according to the criterion provided. The list of valid criteria supported by the responding application process is defined by the List of Event Summary Selection Criteria attribute of the responding AP. A supported criterion may define either an individual event characteristic, or a combination of event characteristics.

NOTE The selection criteria attribute defined for an event may be based on an event characteristic such as its status, whether it is enabled or disabled, or whether it has been acknowledged. It is also possible to specify user-defined criteria such as the events related to a particular operating unit or a piece of equipment.

6.2.5.3.10.2 Service primitives

The service parameters for this service are shown in Table 57.

Table 57 – Query Event Summary List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Events to Search	M	M(=)		
All Events	S	S(=)		
List of Events	S	S(=)		
Selection Criterion	M	M(=)		
Recovered Message Requested	U	U(=)		
Object Revision Requested	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Event Summaries			M	M(=)
List Of Event ID			C	C(=)
List Of Access Result			M	M(=)
List Of Response			M	M(=)
Event Status			M	M(=)
Active			M	M(=)
Detected			M	M(=)
Enabled			M	M(=)
Acknowledgment Supported			M	M(=)
Acknowledged			M	M(=)
Object Revision			C	C(=)
Recovered Message			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Events to Search

This parameter identifies the event objects that are to be searched. The events are identified by specifying all events, an event list, or a list of events.

All Events

This selection type parameter indicates that all events are to be searched.

List of Events

This selection type parameter specifies events that are to be searched. The events may be specified as a list of event objects, a list of event list objects, or a combination of the two. When selected, the list shall contain at least one entry.

Selection Criterion

This parameter identifies the criterion to use in the selection process. The value used shall be contained in the List of Event Summary Selection Criteria attribute of the AP containing the events to be searched.

Recovered Message Requested

This optional parameter specifies, when TRUE, that the last message generated by the event is to be returned.

Object Revision Requested

This optional parameter is used to request that the value of the object revision attribute be returned. A value of TRUE indicates that object revision is desired. A value of FALSE indicates that object revision is not desired. If the Object Revision is requested, but is not supported for the specified object, the service fails.

Result(+)

This selection indicates that the service request succeeded.

Event Summaries

This parameter contains the summary information for each selected event

List of Event ID

This conditional attribute is present when the selection criterion is not “none” or when the selection criterion is “none” and the specified events are “all” or a “list of events”.

List of Access Result

This list of Boolean parameters provides for each event for which an event summary is returned an information, whether the access was successful (Access Result = TRUE) or not (Access Result = FALSE).

List of Response

This parameter contains an error status indicator or the event summary for each returned event summary. The order of the information is the same as the order in which the objects were referenced in the service request.

Event status

This parameter specifies, when TRUE, that the conditions for detecting an event are currently TRUE. This parameter has the value FALSE if the conditions are FALSE or if the event object is not active.

Active

This parameter specifies, when TRUE, that event detection is active for the event. When active, the event object detects when events occur. When inactive, it does not.

Enabled

This parameter specifies the value of the Enabled attribute of the identified Event object.

Unacknowledged

This parameter, when TRUE, specifies that the event object does not have outstanding event occurrences for which it is awaiting acknowledgments, or that it does not support acknowledgment. If the Event is waiting for acknowledgments this parameter has a value of FALSE.

Recovered Message

This conditional parameter is present if the service request specified message recovery was requested. When present, this parameter contains the recovered event message.

Object Revision

This conditional parameter is present if the service request specified the object revision was requested. When present, this parameter contains the object revision of the event object selected.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.5.3.10.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Query Event Summary Service is a "best effort" service that operates through a queue or buffer. Best effort means that the service succeeds if at least one event summary is returned. If the user is not able to obtain the summary for at least one event, the service fails and the user issues a Query Event Summary Service response (-) primitive indicating the reason.

6.2.6 Load region ASE

6.2.6.1 Overview

A Load Region represents an unstructured memory area whose contents may be uploaded (read) or downloaded (written). Unstructured in this context means that the memory area is represented only as an ordered sequence of octets. No other structure is apparent.

Load regions may represent an unnamed volatile memory area, such as that implemented by dynamic computer memory, or a named non-volatile memory object, such as a file. The contents of a load region are referred to as a load image. Load images may contain programs or data. The transfer of a load image to or from a load region is performed using the load process.

To maintain integrity, only one load process for a load region is permitted at a time. The Load Region State attribute of the load region is used to indicate whether the load region is empty, being downloaded, or loaded (see the definition of the attribute for a complete list of the states). Load regions may be cleared using the Discard service.

The Upload State attribute indicates the progress of an upload. This attribute is defined to separate the state of the contents of the load region from the operation of having the contents uploaded (dumped).

This load region model provides services that permit an AP to initiate the download or upload of one of its load regions. It identifies the load region and whether to upload or download as parameters of the request. A third parameter indicates whether the transfer of load image segments is to be accomplished using the Pull Segment service or the Push Segment service.

NOTE The Push and Pull Segment services have no relationship to the Push and Pull AREP types.

Load image segments are pulled by having the receiver of the load image issue requests for them. The AP that contains the load image responds by returning the requested segment, indicating with a parameter when the final segment of the load image has been returned.

Load image segments are pushed by having the AP containing the load image send individual segments to the receiver and wait for a response that indicates whether the segment was received. In this case, the sender indicates when the last segment has been transferred.

After the last segment has been received, the AP that initiated the transfer ends the load process by issuing a terminate request to the remote AP. The terminate request may also be used by either AP when an AP determines that the load process cannot be completed successfully.

6.2.6.2 Load region model specification

6.2.6.2.1 Formal model

FAL ASE:		LOAD REGION ASE	
CLASS:	LOAD REGION		
CLASS ID:		2	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1	(m)	Attribute:	Load Region Size
2	(m)	Attribute:	Access Privilege
2.1	(m)	Attribute:	Password
2.2	(m)	Attribute:	Access Groups
2.3	(m)	Attribute:	Access Rights
3	(m)	Attribute:	Local Address
4	(m)	Attribute:	Load Region State
5	(m)	Attribute:	Upload State
6	(m)	Attribute:	Number of Related Objects In Use
7	(o)	Attribute:	List of Related Objects
7.1	(m)	Attribute:	ClassID
7.2	(m)	Attribute:	Numeric ID
7.2	(m)	Attribute:	In Use Flag (TRUE, FALSE)
8	(o)	Attribute:	Contents Size
9	(o)	Attribute:	Load Image Name
10	(o)	Attribute:	Fixed Length Segments (TRUE/FALSE) Default: FALSE
11	(c)	Constraint:	Fixed Length Segments = TRUE
11.1	(o)	Attribute:	Fixed Segment Length
12	(o)	Attribute:	Intersegment Request Timeout
13	(o)	Attribute:	Sharable
14	(o)	Attribute:	Local Detail
SERVICES:			
1	(m)	Ops Service:	Initiate Load
2	(o)	Ops Service:	Push Segment
3	(o)	Ops Service:	Pull Segment
4	(m)	Ops Service:	Terminate Load
5	(o)	Ops Service:	Discard

6.2.6.2.2 Attributes

Load Region Size

This attribute specifies the maximum size of the Load Region in octets.

Access Privilege

This attribute specifies the access controls defined for this load region. It is composed of the following:

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute identifies which of the eight user-defined access groups are defined for the load region. More than one access group may be defined.

Access Rights

This attribute defines the type of access defined for the load region. Valid values are:

- Right to Use in a FI for the Access Groups
- Right to Write for the Access Groups
- Right to Read for the Access Groups
- Right to Use in a FI for the registered Password
- Right to Write for the registered Password
- Right to Read for the registered Password
- Right to Use in a FI for all Communication Partners
- Right to Write for all Communication Partners
- Right to Read for all Communication Partners

Local Address

This attribute is a locally significant address of the load region. The value FFFFFFFF hex indicates that no local address is available.

Load Region State

This attribute specifies the state of the Load Region Object. The values of this attribute are:

NOT DOWNLOADABLE	This state indicates that the Load Region is not capable of being downloaded.
DOWNLOADABLE	This state indicates that the Load Region is empty, but is capable of being downloaded.
DOWNLOADING	This state indicates that the download sequence has been initiated.
DOWNLOAD FAILURE	This transient state indicates that the download sequence has failed, but has not yet been terminated.
DOWNLOAD SUCCESS	This transient state indicates that the download sequence has succeeded, but has not yet been terminated.
LOADED	This state indicates that the download sequence has terminated successfully.

IN-USE	This state indicates that the Load Region is loaded and is currently being used.
--------	--

Download State attribute constraints:

A Load Region object whose contents are stored in non-erasable memory cannot support downloading transfers. Its valid state values are LOADED and IN USE

A Load Region object whose contents are stored in erasable memory supports downloading transfers. Its valid state values are DOWNLOADABLE, DOWNLOADED, DOWNLOADING, DOWNLOADING_FAILURE, DOWNLOADING_SUCCESS, LOADED and IN_USE,

NOTE The state NOT DOWNLOADABLE state is reserved for Load Regions whose contents are stored in erasable memory, but who have encountered a temporary condition that does not allow them to be downloaded. contents of Load Region objects located in an Application Process paying client role.

Upload State

This attribute specifies the state of the Upload Process of the Load Region.

NOT UPLOADABLE	This state indicates that the Load Region is not capable of being uploaded.
UPLOADABLE	This state indicates that the Load Region is capable of being uploaded.
UPLOADING	This state indicates that the upload sequence has been initiated.
COMPLETING UPLOAD	This transient state indicates that the upload sequence has completed, but has not yet been terminated.

Upload State attribute constraints:

A Load Region object whose contents are stored in erasable memory or not support Uploading transfers. Its valid state values are UPLOADABLE, UPLOADING, and COMPLETING UPLOADING

NOTE The NOT UPLOADABLE state is reserved Load Regions whose contents are stored in erasable memory, but who have encountered a temporary condition that does not allow them to be uploaded.

Number of Related Objects in Use

This attribute indicates the number of related Action objects or Function Invocation objects which currently use the contents of the Load Region object.

This attribute is incremented each time a related Function Invocation object enters the RUNNING state or a related Action object is invoked.

This attribute is decremented each time a related Function Invocation object leaves the RUNNING state or a related Action object completes execution.

If the attribute sharable = TRUE, then several Function Invocation objects and Action objects may share the Load Region contents; otherwise only one Function Invocation or Action object can be attached and use the contents of the object

List of Related Objects

This optional attribute specifies the Function Invocation and Action objects sharing the contents of the Load Region. Each Function Invocation object or Action object is identified by specifying its class identifier and its numeric identifier.

This list is respectively increased/decreased each time a related Function Invocation object or Action object is created or deleted.

Class ID

This attribute is the class identifier for the related object.

Numeric ID

This attribute is the numeric identifier for the related object.

In Use Flag

This attribute is TRUE if the related object is a Function Invocation object in the RUNNING state or if the related object is an Action object that is currently executing.

Contents Size

This optional attribute specifies the length of the data contained in the Load Region. The length is indicated in octets. If the state of the load region is DOWNLOADABLE, then the value of this attribute is zero.

Load Image Name

This optional attribute specifies the name of load image contained in the load region.

Fixed-Length Segments

This optional attribute specifies, when TRUE, that the load region supports the upload and download of fixed length segments only. The default value for this attribute is FALSE.

Fixed-Segment Length

This conditional attribute specifies the length in octets of the fixed-length-segments uploaded to and downloaded from this load region. The attribute is present when the Fixed Length Segments attribute is TRUE.

Intersegment Request Timeout

This optional attribute specifies the timeout period for receiving segment requests, in seconds.

Sharable

This attribute indicates, when TRUE, that the Load Region object contents may be used by multiple Function Invocation/Action objects. The default value for this attribute is TRUE.

Local Detail

This attribute contains additional information, such as a list of capabilities and/or the number of octets to be transferred.

6.2.6.2.3 Services

Initiate Load

This service is used by an application to initiate the download or upload of a Load Region. A parameter of the service indicates whether the load image will be transferred into the Load Region using the Pull Segment or from the Load Region using the Push Segment service.

Push Segment

This optional service is used to transfer load segments in the request/indication primitives. Although this service is optional, at least one of the segment transfer services, Push Segment or Pull Segment, is required.

Pull Segment

This optional service is used to request that a load segment be returned in the response/confirmation primitives. Although this service is optional, at least one of the segment transfer services, Push Segment or Pull Segment, is required.

Terminate Load

This service is used by an AP to terminate the load process.

Discard

This optional service is used to clear the contents of the load region.

6.2.6.3 Load region service specification**6.2.6.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Initiate Load
- Terminate Load
- Push Segment
- Pull Segment
- Discard

6.2.6.3.2 Initiate load service**6.2.6.3.2.1 Service overview**

This confirmed service is used to request the download or upload of a load region. A parameter of the service indicates whether the load image is to be transferred by the requester using the Pull Segment or Push Segment service. When a download is being requested for a load region in a remote AP (identified by the called load region key attribute) that does not yet exist, the responding AP may either dynamically create the load region and return a positive response, or return a negative response that indicates that the load region does not exist.

6.2.6.3.2.2 Service primitives

The service parameters for this service are shown in Table 58.

Table 58 – Initiate Load service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Load Region Key Attribute	M	M(=)		
Calling	S	S(=)		
Called	S	S(=)		
Load Type	M	M(=)		
Load Service To Use	M	M(=)		
Load Service Initiator	M	M(=)		
Additional Information	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Additional Detail			U	U(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Called Size			C	C(=)

Argument

The argument contains the parameters of the service request.

Load Region Key Attribute

This parameter identifies the load region to be downloaded or to be uploaded.

Calling

This selection type parameter identifies the load region of the requester. It is selected if the requester is the server.

Called

This selection type parameter identifies the load region of the responder. It is selected if the requester is the client.

Load Type

This parameter specifies whether the load is an UPLOAD (out of the load region) or a DOWNLOAD (into the load region).

Load Service to Use

This parameter specifies that the load region is to be transferred from the remote AP using either the Pull Segment or the Push Segment service.

Load Service Initiator

This parameter specifies, when TRUE, that the load service identified by the previous parameter is to be initiated by the AP initiating this service. When FALSE, the remote AP is to initiate the load service.

Additional Information

This optional parameter contains additional information, such as a file name and/or the number of octets to be transferred. It may be present if the requester is the server.

Result(+)

This selection type parameter indicates that the service request succeeded.

Additional Detail

This optional parameter contains additional information, such as a list of capabilities and/or the number of octets to be transferred. It may be present if the responder is the server.

Result(-)

This selection type parameter indicates that the service request failed.

Called Size

This conditional parameter specifies the maximum size in octets that the responding AP is prepared to transfer. It is present if the error was due to a size problem.

6.2.6.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.6.3.3 Terminate load service**6.2.6.3.3.1 Service overview**

This confirmed service is used to terminate the load process. It may be used upon successful completion of the load, or to abort a load in progress. It may be requested by either endpoint.

If the load process has completed, it is always issued by the AP that issued the initiate service that started the load process. If the load process is being aborted,

- a) the load region sending the image returns to the UPLOADABLE Upload State;
- b) the load region receiving the image deletes the incomplete load from its load region and returns to the DOWNLOADABLE State.

6.2.6.3.3.2 Service primitives

The service parameters for this service are shown in Table 59.

Table 59 – Terminate Load service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Load Region Key Attribute	M	M(=)		
Load Type	M	M(=)		
Load Service Used	M	M(=)		
Terminate Reason	C	C(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Terminate Reason			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Load Region Key Attribute

This parameter specifies one of the key attributes of the Load Region for which the load is being terminated.

Load Type

This parameter indicates the type of load being terminated. Valid values are UPLOAD and DOWNLOAD.

Load Service Used

This parameter indicates the type of load service used to transfer load segments. Valid values are PUSH and PULL.

Terminate Reason

This conditional parameter indicates the success or failure of the load process. It is present if the server issues the terminate request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Terminate Reason

This conditional parameter indicates the success or failure of the load process. It is present if the client issues the terminate request to terminate a download.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.6.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.6.3.4 Push segment service

6.2.6.3.4.1 Service overview

This confirmed service is used after the load process has been initiated to transfer a single load image segment in its request and indication primitives. The response and confirmation primitives are used to convey the success or failure of the segment transfer. This service may be used to support uploads and downloads.

6.2.6.3.4.2 Service primitives

The service parameters for this service are shown in Table 60.

Table 60 – Push Segment service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Load Type	M	M(=)		
Load Region Key Attribute	M	M(=)		
Segment Number	U	U(=)		
Load Data	M	M(=)		
More Follows	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Load Type

This parameter specifies whether the load is an UPLOAD (out of the load region) or a DOWNLOAD (into the load region).

Load Region Key Attributes

This parameter specifies one of the key attributes of the Load Region whose image is to be transferred.

NOTE The load region is local if this service is being used for an upload. The load region is remote if this service is being used for a download.

Segment Number

This optional parameter identifies the number of the segment which is being transferred. Segment numbers are ascending integers beginning with the value one (1).

Load Data

This parameter contains the data to be loaded.

More Follows

This parameter indicates whether or not any additional data remains to be transmitted.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.6.3.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.6.3.5 Pull segment service

6.2.6.3.5.1 Service overview

This confirmed service is used after the load process has been initiated to request that a specified load image segment be returned in the response and confirmation primitives. This service may be used to support uploads and downloads.

6.2.6.3.5.2 Service primitives

The service parameters for this service are shown in Table 61.

Table 61 – Pull Segment service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Load Type	M	M(=)		
Load Region Key Attribute	M	M(=)		
Segment Number	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Load Data			M	M(=)
More Follows			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Load Type

This parameter specifies whether the load is an UPLOAD (out of the load region) or a DOWNLOAD (into the load region).

Load Region Key Attribute

This parameter specifies one of the key attributes of the Load Region whose image is to be transferred.

NOTE The load region is remote if this service is being used for an upload. The load region is local if this service is being used for a download.

Segment Number

This optional parameter identifies the number of the segment which is being requested. Segment numbers are ascending integers beginning with the value one (1).

Result(+)

This selection type parameter indicates that the service request succeeded.

Load Data

This parameter contains the data to be downloaded or uploaded.

More Follows

This parameter indicates whether or not any additional data remains to be transmitted.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.6.3.5.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.6.3.6 Discard service

6.2.6.3.6.1 Service overview

This confirmed service is used to request that the contents of a load region be discarded.

6.2.6.3.6.2 Service primitives

The service parameters for this service are shown in Table 62.

Table 62 – Discard service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Load Region Key Attribute	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Load Region Key Attribute

This parameter specifies one of the key attributes of the Load Region object whose contents are to be discarded.

Result(+)

This selection indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.2.6.3.6.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.6.4 Load region state machines

6.2.6.4.1 General

This standard defines state machines for three types of transfers.

Pull_Uploading:	The load region contents are retrieved from a Load Region object, segment by segment, by a Client AP.
Pull_Downloading:	A load image contained in a Client AP is copied into a Load Region object, segment by segment, at the request of the server AP that contains the load region. That is, the Server AP requests the client to send the segments of the load image. This kind of transfer is allowed for erasable load regions. Its state diagram and transitions table give additional information related to a non-erasable load region contents.
Push_Downloading:	A load image contained in a client AP are sent to the Load Region object, segment by segment, by the client AP. Upon receipt of each segment, the Server AP that contains the load region copies it into the load region. This kind of transfer is allowed for erasable load regions. Its state diagram and transitions table give additional information related to a non-erasable load region contents.

Other kinds of transfer are beyond the scope of this standard.

6.2.6.4.2 Pull upload state machine

6.2.6.4.2.1 Description

This kind of transfer uses the following services: Initiate Load, Pull Segment and Terminate Load

A Server may initiate an uploading transfer. In this case it issues an Initiate_Load.Reg service primitive to a Client. This primitive tells the Client to initiate uploading transfer for a specified Load Region. Once the client has pulled the contents, it issues an Initiate_Load.Rsp(+/-) to the Server.

A Client may initiate an uploading transfer. In this case it issues an Initiate_Load.Reg primitive to a Server. This primitive tells the Server that the contents of a specified load region

object will be pulled. Once the client has pulled the contents, it ends the transfer by issuing a Terminate_Load.Reg to the Server.

6.2.6.4.2.2 Sequencing of service primitives

Table 63 summarizes the sequencing of service primitives. In the table, T1 and T2 show different options for initiating the upload, one by the server AP (T1) and one by the client AP (T2).

Table 63 – Pull Upload Sequencing of service primitives

FAL User(Client)	FAL User(Server) Owner of called Load Region
Initiate_Load.Ind ←---(T 1)-----	Initiate_Load.Reg
Initiate_Load.Reg -----(T2)----→	Initiate_Load.Ind (Transfer started)
Initiate_Load.Cnf ←---(T2)-----	Initiate_Load.Rsp
Pull_Segment.Reg -----(T3)----→	Pull_Segment.Ind (Segments Transferred)
Pull_Segment.Cnf ←---(T3)-----	Pull_Segment.Rsp
Terminate_Load.Reg -----(T4)----→	Terminate_Load.Ind (Transfer ended)
Terminate_Load.Cnf ←---(T4)-----	Terminate_Load.Rsp
Initiate_Load.Rsp -----(T1)----→	Initiate_Load.Cnf

6.2.6.4.2.3 Service parameter value constraints

Table 64 summarizes the service parameter constraints.

Table 64 – Pull Upload service parameter constraints

Transition Number	Parameter constraints
T1	Initiate_Load Calling Load Region Key attribute selected, Load Type : = upload, Service To Use : = Pull_Segment, Load Service Initiator : = FALSE
T2	Initiate_Load Called Load Region Key Attribute selected, Load Type : = upload Service To Use : = Pull_Segment Load Service Initiator : = TRUE
T3	Pull_Segment Load Type : = upload, Segment Number : = ordered 1 to n if any More Follows : = TRUE/FALSE with FALSE for the last segment
T4	Terminate_Load Load Type : = upload, Load Service Used : = Pull_Segment

6.2.6.4.2.4 Upload state transition table

Table 65 gives the upload state transitions.

Table 65 – Pull Upload state table

Transition	Current State	Event/Action	Next State
1	UPLOADABLE	Initiate_Load.Ind &&Load Region State of this object is in state DOWNLOADABLE II LOADED II IN USE Initiate_Load.Rsp(+) the load region image is segmented in accordance with fixed-length attribute constraints into n segments Segments Left To Transfer : = n -- (local counter)	UPLOADING
2	UPLOADING	Pull_Segment.Ind && Segments Left To Transfer > 1 &&service succeeds(e.g. compliance with Intersegment Req Time Out attribute constraint if supported) Pull_Segment.Rsp(+) Segment Number : = Segments Left To Transfer (if any) with More Follows : = TRUE Segments Left To Transfer : = -1	UPLOADING
3 Exist if supports: Intersegmnt Req Time Out	UPLOADING	Pull_Segment.Ind &&Segments Left To Transfer = any value &&service fails(e.g. no compliance with Intersegment Req Time Out attribute constraint if supported) Pull_Segment.Rsp(-)	UPLOADING
4	UPLOADING	Pull_Segment.Ind &&Segments Left To Transfer=1 (last segment) &&service succeeds(e.g. compliance with Intersegment Req Time Out attribute constraint) Pull_Segment.Rsp(+) Segment Number : = Segments Left To Transfer (if any) More Follows : = FALSE Segments Left To Transfer : = -1	COMPLETING UPLOAD
5	UPLOADING	Terminate_Upload.Ind Terminate_Upload.Rsp(-) Error Info : = object state conflict	UPLOADABLE
6	UPLOADING or COMPLETING UPLOAD	Abort.Ind	UPLOADABLE
7	COMPLETING UPLOAD	Terminate_Upload.Ind Terminate_Upload.Rsp(+)	UPLOADABLE
8	UPLOADABLE	Initiate_Load.Ind && Load Region State attribute is not in state: DOWNLOADABLE II LOADED II IN USE Initiate_Load.Rsp(-) with Error Info : = object constraint conflict	UPLOADABLE

6.2.6.4.3 Pull download state machine

6.2.6.4.3.1 Description

This kind of transfer uses the following services: Initiate Load, Pull Segment and Terminate Load.

A Server may initiate a downloading transfer. In this case it issues an `Initiate_Load.Reg` service primitive to a Client. This primitive tells the Client to initiate downloading transfer for a specified Load Region object. Once the Server has pulled the contents, the Client issues an `Initiate_Load.Rsp(+/-)` to the Server.

A Client may initiate the downloading transfers. In this case it issues an `Initiate_Load.req` primitive to a Server. This primitive tells the Server to pull the contents of a specified load region object. Once the Server has pulled the contents, it ends the transfer by issuing a `Terminate_Load.Reg` to the Client

6.2.6.4.3.2 Sequencing of service primitives

Table 66 summarizes the sequencing of service primitives.

Table 66 – Pull Download Sequencing of service primitives

FAL User(a Client) Owner of a calling Object	FAL User(a Server) Owner of called Object
<code>Initiate_Load.Ind</code> ←----(T 1)-----	<code>Initiate_Load.Reg</code>
<code>Initiate_Load.Reg</code> ----->(T2)-----	<code>Initiate_Load.Ind</code> (Transfer started)
<code>Initiate_Load.Cnf</code> ←----(T2)-----	<code>Initiate_Load.Rsp</code>
<code>Pull_Segment.Ind</code> <----->(T3)-----	<code>Pull_Segment.Reg</code> (Transfer continued)
<code>Pull_Segment.Rsp</code> ----->(T3)-----	<code>Pull_Segment.Cnf</code>
<code>Terminate_Load.Ind</code> <----->(T4)-----	<code>Terminate_Load.Reg</code> (Transfer ended)
<code>Terminate_Load.Rsp</code> ----->(T4)-----	<code>Terminate_Load.Cnf</code>
<code>Initiate_Load.Rsp</code> ----->(T1)-----	<code>Initiate_Load.Cnf</code>

6.2.6.4.3.3 Service parameter constraints

Table 67 summarizes the service parameter constraints.

Table 67 – Pull Download service parameter constraints

Transition Number	Parameter constraints
T1	<code>Initiate_Load</code> : Calling Load Region Key attribute selected, Load Type : = download, Service To Use : = <code>Pull_Segment</code> , Load Service Initiator : = TRUE
T2	<code>Initiate_Load</code> Called Load Region Key Attribute selected, Load Type : = download Service To Use : = <code>Pull_Segment</code> Load Service Initiator : = FALSE
T3	<code>Pull_Segment</code> Load Type : = download, Segment Number : = ordered 1 to n if any More Follows : = TRUE/FALSE with FALSE for the last segment
T4	<code>Terminate_Load</code> Load Type : = download, Load Service Used : = <code>Pull_Segment</code> Terminate Reason contained in the request

6.2.6.4.3.4 Download state transition table

Table 68 gives the download state transitions.

Table 68 – Pull Download state table

Transition	Current State	Event/Action	Next State
1	DOWNLOADABLE	Initiate_Load.Ind &&Upload State = UPLOADABLE Initiate_Load.Rsp(+) Pull_Segment.Req	DOWNLOADING
2	DOWNLOADABLE	Initiate_Load.Ind &&Upload State <>UPLOADABLE Initiate_Load.Rsp(-) Error Info : = object constraint conflict	DOWNLOADABLE
3	DOWNLOADING	Pull_Segment.Cnf(+) &&More Follows=TRUE Store received data Pull_Segment.Req	DOWNLOADING
4	DOWNLOADING	Pull_Segment.Cnf(+) &&More Follows=FALSE Store received data Terminate_Load.Req Terminate Reason : = success	DOWNLOAD SUCCESS
5	DOWNLOADING	Pull_Segment.Cnf(-) Discard received data Terminate_Load.Req Terminate Reason : = failure	DOWNLOAD FAILURE
6	DOWNLOADING, DOWNLOAD FAILURE, DOWNLOAD SUCCESS	Abort.Ind Discard received data	DOWNLOADABLE
7	DOWNLOAD FAILURE	Terminate_Load.Cnf(+/-) Discard received data	DOWNLOADABLE
8	DOWNLOAD SUCCESS	Terminate_Load.Cnf(-) Discard received data	DOWNLOADABLE
9	DOWNLOAD SUCCESS	Terminate_Load.Cnf(+) Update Contents Size attribute if present	LOADED
10	LOADED	Initiate_Load.Ind &&Upload state attribute in state UPLOADABLE &&service succeeds(e.g erasable load region contents) Initiate_Load.Rsp(+)	DOWNLOADING
11	LOADED	Discard.Ind &&Upload State attribute in state UPLOADABLE &&service succeeds(e.g. erasable load region contents) Discard.Rsp(+)	DOWNLOADABLE
12	IN USE	Number of Related object In Use attribute incremented &&its new value > 1 Set Related Object In Use flag value to TRUE if present (Start/Resume)	IN USE
13	IN USE	Number of Related Object In Use attribute decremented &&its new value >0	IN USE

Transition	Current State	Event/Action	Next State
		Set Related Object In Use flag value to FALSE if any (Reset/Stop/Kill)	
14	LOADED	Initiate_Load.Ind &&Upload State attribute in state <> UPLOADABLE If Service fails(e.g. non erasable load region contents) Initiate_Load.Rsp(-) Error Info : = object constraint conflict	LOADED
15	LOADED	Discard.Ind && Upload State attribute in state <> UPLOADABLE If Service fails(e.g. non erasable load region contents) Discard.Rsp(-) Error Info : = object constraint conflict	LOADED
16	LOADED	Number of Related Object In Use attribute incremented &&its new value=1 Set Related Object In Use flag value to TRUE if any (Start/Stop, Related objects are FI)	IN USE
17	IN USE	Number of Related Object In Use Attribute decremented &&its new value=0 Set Related Object In Use flag value to FALSE if any (Reset/Stop/Kill, Related objects are FI)	LOADED

6.2.6.4.4 Push download state machine

6.2.6.4.4.1 Description

This kind of transfer uses the following services: Initiate Load, Pull Segment and Terminate Load

A Server may initiate a downloading transfer. In this case it issues an Initiate_Load.Reg service primitive to a Client. This primitive tells the Client to initiate downloading transfer for a specified Load Region object. Once the Client has pushed the contents, it issues an Initiate_Load.Rsp(+/-) to the Server.

A Client may initiate a downloading transfer. In this case it issues an Initiate_Load.Reg service primitive to a Server. This primitive tells the Server that the contents of a specified load region object will be pushed. Once the Client has pushed the contents, it ends the transfer by issuing a Terminate_Load.req to the Server

6.2.6.4.4.2 Sequencing of service primitives

Table 69 summarizes the sequencing of service primitives.

Table 69 – Push Download sequencing of service primitives

FAL User(a Client) Owner of a calling Object	FAL User(a Server) Owner of called Object
Initiate_Load.Ind ←---(T 1)----	Initiate_Load.Reg
Initiate_Load.Reg ----- (T2) ----→	Initiate_Load.Ind (Transfer started)
Initiate_Load.Cnf ←---(T2)----	Initiate_Load.Rsp
Push_Segment.Reg ----- (T3) ----→	Push_Segment.Ind (Transfer continued)
Push_Segment.Cnf ←---(T3)----	Push_Segment.Rsp
Terminate_Load.Reg ----- (T4) ----→	Terminate_Load.Ind (Transfer ended)
Terminate_Load.Cnf ←---(T4)----	Terminate_Load.Rsp
Initiate_Load.Rsp ----- (T1) ----→	Initiate_Load.Cnf

6.2.6.4.4.3 Service parameter constraints

Table 70 summarizes the service parameter constraints.

Table 70 – Push Download service parameter constraints

Transition Number	Parameter constraints
T1	Initiate_Load: Calling Load Region Key Attribute selected, Load Type : = download, Service To Use : = Push_Segment, Load Service Initiator : = FALSE
T2	Initiate_Load Called Load Region Key Attribute selected, Load Type : = upload Service To Use : = Push_Segment Load Service Initiator : = TRUE
T3	Pull_Segment Load Type : = upload, Segment Number : = ordered 1 to n if any More Follows : = TRUE/FALSE with FALSE for the last segment
T4	Terminate_Load Load Type : = upload, Load Service Used : = Pull_Segment Terminate Reason contained in the response

6.2.6.4.4.4 Download state transition table

Table 71 gives the upload state transitions.

Table 71 – Push Download state table

Transition	Current State	Event	Next State
1	DOWNLOADABLE	Initiate_Load.Ind &&Upload state attribute in state UPLOADABLE Initiate_Load.Rsp(+)	DOWNLOADING
2	DOWNLOADABLE	Initiate_Load.Ind &&Upload state attribute in state <>UPLOADABLE Initiate_Load.Rsp(-) with Error Info : = object constraint conflict	DOWNLOADABLE
3	DOWNLOADING	Push_Segment.Ind &&More Follows=TRUE &&compliance with Inter Segment Req Time Out constraint Push_Segment.Rsp(+)	DOWNLOADING
4	DOWNLOADING	Push_Segment.Ind &&More Follows=FALSE &&compliance with Inter Segment Req Time Out constraint Push_Segment.Rsp(+)	DOWNLOAD SUCCESS
5 exist if supports Inter Segment Req Time Out	DOWNLOADING	Push_Segment.Ind &&More Follows=TRUE II FALSE &&no compliance with Inter Segment Req Time Out constraint Push.Segment.Rsp(-) Discard received data	DOWNLOAD FAILURE
6	DOWNLOADING, DOWNLOAD FAILURE, DOWNLOAD SUCCESS	Abort.Ind Discard received data	DOWNLOADABLE

Transition	Current State	Event	Next State
7	DOWNLOAD FAILURE	Terminate_Load.Ind Terminate_Load.Rsp(+/-) Terminate Reason : = success/failure	DOWNLOADABLE
8	DOWNLOAD SUCCESS	Terminate_Load.Ind &&service fails Terminate_Load.Rsp(-) Terminate Reason : = failure Discard received data	DOWNLOADABLE
9	DOWNLOAD SUCCESS	Terminate_Load.Ind &&service succeeds(e.g. all segments received) Update Contents Size attribute if present Terminate_Load.Rsp(+) Terminate Reason : = success	LOADED
10	LOADED	Initiate_Load.Ind &&Upload State attribute in state UPLOADABLE &&service succeeds(e.g. erasable load region contents) Initiate_Load.Rsp(+)	DOWNLOADING
11	LOADED	Discard.Ind &&Upload State attribute in state UPLOADABLE &&service succeeds(e.g. erasable load region contents) Discard.Rsp(+)	DOWNLOADABLE
12	IN USE	Number of Related object In Use attribute incremented &&its new value > 1 Set Related Object In Use flag value to TRUE if any (Start/Resume)	IN USE
13	IN USE	Number of Related Object In Use attribute decremented &&its new value >0 Set Related Object In Use flag value to FALSE if any (Reset/Stop/Kill)	IN USE
14	LOADED	Initiate_Load.Ind &&Upload State attribute in state <> UPLOADABLE Service fails(e.g. non erasable load region contents) Initiate_Load.Rsp(-) Error Info : = object constraint conflict	LOADED
15	LOADED	Discard.Ind &&Upload State attribute in state <> UPLOADABLE Service fails(e.g. non erasable load region contents) Discard.Rsp(-) Error Info : = object constraint conflict	LOADED
16	LOADED	Number of Related Object In Use attribute incremented &&its new value=1 Set Related Object In Use flag value to TRUE if any (Start/Stop, Related objects are FI)	IN USE
17	IN USE	Number of Related Object In Use Attribute decremented &&its new value=0 Set Related Object In Use flag value to FALSE if any (Reset/Stop/Kill, Related objects are FI)	LOADED

6.2.7 Function invocation ASE

6.2.7.1 Overview

The FAL function invocation model defines two objects, the stateless action object and the state-oriented function invocation object.

Stateless function invocations, referred to as actions, run to completion when invoked and cannot be interrupted. In addition, some atomic actions return a value in response to being invoked, while others do not. Those that do may be used to model software *functions*, and those that do not may be used to model software *procedures*.

State-oriented function invocations, on the other hand, may be controlled during their execution. Services are defined to suspend, resume, or abort them once they have been invoked. They do not return a value in response to being started. If it is necessary to abort a function invocation once it has been started, then it shall be defined as state-oriented.

State-oriented function invocations represent the network view of a specific invocation of a user function. If the user function can be invoked more than once simultaneously, then separate function invocation objects are needed to represent each invocation.

State-oriented function invocations may be used to model software processes or user operations that may be started and controlled.

NOTE An example of a state-oriented function invocation that may be used to model a user operation is the “playback” operation of a video recorder. Once it has been started, the playback operation may be stopped (paused) and later resumed.

To support the concept of software process modeling, the definition of state-oriented functional invocations includes optional attributes that can be used to relate them to load region objects.

The remainder of this subclause specifies the class definitions and services for the state-oriented *function invocation* object and the stateless *action* object.

6.2.7.2 Function invocation model class specifications

6.2.7.2.1 Function invocation class specification

6.2.7.2.1.1 Class overview

The function invocation class models the state-oriented function invocation. It may be used to model software processes or user functions whose operation may be controlled.

6.2.7.2.1.2 Formal model

FAL ASE:		FUNCTION INVOCATION ASE	
CLASS:		FUNCTION INVOCATION	
CLASS ID:		3	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1	(m)	Attribute:	Access Privilege
1.1	(m)	Attribute:	Password
1.2	(m)	Attribute:	Access Groups
1.3	(m)	Attribute:	Access Rights
2	(m)	Attribute:	Deletable (TRUE,FALSE)
1	(m)	Attribute:	Reusable (TRUE,FALSE) Default : TRUE
2	(m)	Attribute:	Function Invocation State
4	(m)	Attribute:	Number of Related Objects In Use

3	(o)	Attribute:	List of Related Objects
3.1	(o)	Attribute:	ClassID
3.2	(o)	Attribute:	Numeric ID
5.2	(m)	Attribute:	In Use Flag (TRUE, FALSE)
4	(c)	Constraint:	Start Service Supported
4.1	(o)	Attribute:	Start Service Argument Data Type
5	(c)	Constraint:	Stop Service Supported
5.1	(o)	Attribute:	Stop Service Argument Data Type
6	(c)	Constraint:	Resume Service Supported
6.1	(o)	Attribute:	Resume Service Argument Data Type
7	(c)	Constraint:	Reset Service Supported
7.1	(o)	Attribute:	Reset Service Argument Data Type
8	(c)	Constraint:	Kill Service Supported
8.1	(o)	Attribute:	Kill Service Argument Data Type
9	(o)	Attribute:	Last Execution Argument

SERVICES:

1	(o)	OpsService:	Start
2	(o)	OpsService:	Stop
3	(o)	OpsService:	Resume
4	(o)	OpsService:	Reset
5	(o)	OpsService:	Kill

6.2.7.2.1.3 Attributes**Access Privilege**

This attribute specifies the access controls defined for this function invocation. It is composed of the following:

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute identifies which of the eight user-defined access groups are defined for the function invocation. More than one access group may be defined.

Access Rights

This attribute defines the type of access defined for the function invocation. Valid values are:

- Right to Delete for Access Groups(see note)
- Right to Start for Access Groups
- Right to Stop for Access Groups
- Right to Resume for Access Groups
- Right to Reset for Access Groups
- Right to Kill for Access Groups
- Right to Delete for the Registered Password (see note)
- Right to Start for the Registered Password
- Right to Stop for the Registered Password
- Right to Resume for the Registered Password
- Right to Reset for the Registered Password
- Right to Kill for the Registered Password

- Right to Delete for all Communication Partners (see note)
- Right to Start for all Communication Partners
- Right to Stop for all Communication Partners
- Right to Resume for all Communication Partners
- Right to Reset for all Communication Partners
- Right to Kill for all Communication Partners

NOTE The right to delete has no meaning if the function invocation object is not deletable. This right requires the use of the Object Management Delete service, and not an operational service of the Function Invocation class.

Deletable

This attribute indicates, when TRUE, that the function invocation can be deleted using the Delete service. The value of this attribute is always TRUE for objects dynamically created with the Object Management ASE Create Service.

Reusable

This attribute indicates, when TRUE, that the function invocation may be executed more than once.

Function Invocation State

This attribute indicates the current state of the function invocation. An enumerated set of values has been defined, and may be extended for local use within the AP to describe transient states of the Function Invocation. These transient state values for this attribute, however, are not visible over the network. The transitions between states and the events that cause transitions are user-defined.

UNRUNNABLE This state indicates that the function invocation is not executing and may not be executed.

IDLE This state indicates that the function invocation is not executing, but is capable of being executed.

RUNNING This state indicates that the function invocation is executing.

STOPPED This state indicates that the execution of a function invocation has been suspended.

STARTING This transient state indicates that the function invocation is being prepared for execution. Function invocations in this state are "ready to run".

STOPPING This transient state indicates that the execution of the function invocation is in the process of being stopped and its context saved.

RESUMING This transient state indicates that the function invocation is in the process of being returned to the executing state from a previously saved context.

RESETTING This transient state indicates that the function invocation is being reset to its initial context and removed from execution.

Number of Related Objects in Use

This attribute indicates the number of related Load Region objects defined for this function invocation that are currently in the IN USE state. This attribute is incremented/decremented each time a related Load Region Object State Attribute goes/leaves the IN USE state. The Load Region State Machine gives the constraints for these transitions.

List of Related Objects

This optional attribute specifies the Load Region objects related to this function invocation.

Class ID

This attribute is the class identifier for the related object.

Numeric ID

This attribute is the numeric identifier for the related object.

In-use flag

This attribute indicates, when TRUE, that the related object is currently in the IN USE state. This attribute is updated each time the related Load Region Object state attribute goes/leaves the IN USE state. The Load Region State Machine gives the constraints for these transitions

Start Service Argument Data Type

This attribute identifies the data type for each of the execution arguments for the start service supported by this function invocation. This attribute is present if the function invocation supports the start service. If the start service is supported but has no execution argument, the value of this attribute is NULL.

Stop Service Argument Data Type

This attribute identifies the data type for the execution argument for the stop service supported by this function invocation. This attribute is present if the function invocation supports the stop service. If the stop service is supported but has no execution argument, the value of this attribute is NULL.

Resume Service Argument Data Type

This attribute identifies the data type for the execution argument for the resume service supported by this function invocation. This attribute is present if the function invocation supports the resume service. If the resume service is supported but has no execution argument, the value of this attribute is NULL.

Reset Service Argument Data Type

This attribute identifies the data type for the execution argument for the reset service supported by this function invocation. This attribute is present if the function invocation supports the reset service. If the reset service is supported but has no execution argument, the value of this attribute is NULL.

Kill Service Argument Data Type

This attribute identifies the data type for the execution argument for the kill service supported by this function invocation. This attribute is present if the function invocation supports the kill service. If the kill service is supported but has no execution argument, the value of this attribute is NULL.

Last Execution Argument

This optional attribute contains the value for the Execution Argument that was last sent by the Client.

6.2.7.2.1.4 Services**Start**

This optional service is used to request the function invocation to start executing. If this service is not present, it is assumed that the function invocation will be started using local means.

Stop

This optional service is used to request the function invocation to stop executing.

Resume

This optional service is used to request the function invocation to continue its execution.

Reset

This optional service is used to request the function invocation to return to its initial context and state.

Kill

This optional service is used to request that the current execution of the function invocation be aborted so that it may not be resumed. Once killed, the context of the function execution is lost.

6.2.7.2.2 Action class specification

6.2.7.2.2.1 Class overview

The Action Class is used to define stateless function invocations. Stateless function invocations run to completion when invoked. Their execution may not be aborted using the services of the FAL. They may return a result using the Return service.

6.2.7.2.2.2 Formal model

FAL ASE:		FUNCTION INVOCATION ASE
CLASS:	ACTION	
CLASS ID:		19
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m) Attribute:	Reusable (TRUE,FALSE) Default : TRUE
2	(o) Attribute:	List of Related Objects
3.1	(m) Attribute:	ClassID
3.2	(m) Attribute:	Numeric ID
3.2	(m) Attribute:	In Use Flag (TRUE, FALSE)
4	(o) Attribute:	Action Invoke Service Argument Data Type
5	(c) Constraint:	Action Return Service Supported
5.1	(o) Attribute:	Action Return Service Argument Data Type
SERVICES:		
1	(m) OpsService:	Action Invoke
2	(o) OpsService:	Action Return

6.2.7.2.2.3 Attributes

Reusable

This attribute indicates, when TRUE, that the action may be executed more than once.

List of Related Objects

This optional attribute specifies the Load Region objects related to this function invocation.

Class ID

This attribute is the class identifier for the related object.

Numeric ID

This attribute is the numeric identifier for the related object.

In-use Flag

This attribute indicates, when TRUE, that the related object is currently in the IN USE state. This attribute is updated each time the Load Region Object State attribute goes/leaves the IN USE state. The Load Region State Machine gives the constraints for these transitions.

Action Invoke Service Argument Data Type

This attribute identifies the data type for the execution argument for the action invoke service supported by this action. This attribute is present if the action supports the action invoke service. If the action invoke service is supported but has no execution argument, the value of this attribute is NULL.

Action Return Service Argument Data Type

This attribute identifies the data type for the execution argument for the action invoke service supported by this action. This attribute is present if the action supports the action invoke service. If the action invoke service is supported but has no execution argument, the value of this attribute is NULL.

6.2.7.2.2.4 Services

Action Invoke

This mandatory service is used to call the action object with an argument list.

Action Return

This optional service is used by the action object to return the results of its invocation.

6.2.7.3 Function invocation model service specifications

6.2.7.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Start
Stop
Resume
Reset
Kill
Action Invoke
Action Return

6.2.7.3.2 Start service

6.2.7.3.2.1 Service overview

This confirmed service is used to request that a function invocation be started.

6.2.7.3.2.2 Service primitives

The service parameters for this service are shown in Table 72.

Table 72 – Start service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Key attribute

This parameter specifies one of the key attributes of the function invocation object.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the function invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function Invocation State

This conditional parameter contains the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.2.7.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.7.3.3 Stop service

6.2.7.3.3.1 Service overview

This confirmed service is used to stop a function invocation retaining its context so that it may be resumed.

6.2.7.3.3.2 Service primitives

The service parameters for this service are shown in Table 73.

Table 73 – Stop service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Key Attribute

This parameter specifies one of the key attributes of the function invocation object.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the function invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function Invocation State

This conditional parameter contains the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.2.7.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.7.3.4 Resume service**6.2.7.3.4.1 Service overview**

This confirmed service is used to resume the execution of a function invocation that has been stopped. Execution is resumed using the context saved when the function invocation was stopped.

6.2.7.3.4.2 Service primitives

The service parameters for this service are shown in Table 74.

Table 74 – Resume service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Key Attribute

This parameter specifies one of the key attributes of the function invocation object.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the function invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function Invocation State

This conditional parameter contains the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.2.7.3.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.7.3.5 Reset service

6.2.7.3.5.1 Service overview

This confirmed service is used to reset a function invocation with its initial context. Its initial context is defined as the context of the function invocation set by its initialization procedures.

6.2.7.3.5.2 Service primitives

The service parameters for this service are shown in Table 75.

Table 75 – Reset service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Invoke ID

This parameter identifies this invocation of the service. It is used to associate service requests with responses. Therefore, no two outstanding service invocations may be identified by the same Invoke ID value.

Key Attribute

This parameter specifies one of the key attributes of the function invocation object.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the function invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function Invocation State

This conditional parameter contains the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.2.7.3.5.3 Service procedure

The Confirmed Service Procedure specified In Clause 4 applies to this service.

6.2.7.3.6 Kill service**6.2.7.3.6.1 Service overview**

This confirmed service is used to abort the execution of a function invocation so that it may not be restarted or resumed.

6.2.7.3.6.2 Service primitives

The service parameters for this service are shown in Table 76.

Table 76 – Kill service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Key Attribute

This parameter specifies one of the key attributes of the function invocation object.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the function invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function Invocation State

This conditional parameter contains the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.2.7.3.6.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.2.7.3.7 Action invoke service

6.2.7.3.7.1 Service overview

This unconfirmed service is used to invoke the execution of an action object.

6.2.7.3.7.2 Service primitives

The service parameters for this service are shown in Table 77.

Table 77 – Action Invoke service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Key Attribute	M	M(=)
Action Invocation ID	M	M(=)
Execution Argument	U	U(=)
Timeliness		C
Duplicate FAL PDU Body		C

Argument

This parameter carries the parameters of the service invocation.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the destination address to which the requested Action Invoke is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated Action Invoke is to be sent.

Key Attribute

This parameter specifies one of the key attributes of the action object.

Action Invocation ID

This parameter identifies a specific invocation of the action object. It permits correlation of the invocation and its result as reported with a Return service.

Execution Argument

This optional parameter carries the execution argument. This parameter is present when its data type is defined in the List of Service Argument Data Types for the action object.

Timeliness

This conditional parameter indicates the Data Link Layer timeliness status for the transfer of this service request. This parameter is present if it is supported on the specified AR.

Duplicate FAL PDU Body

This conditional parameter indicates whether or not the receipt of a duplicate FAL PDU has been detected by the Data Link Layer. It is present if supported in the DL Mapping Attributes of the AREP.

6.2.7.3.7.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.2.7.3.8 Action return service**6.2.7.3.8.1 Service overview**

This unconfirmed service is used return the results from an invocation of an action object.

6.2.7.3.8.2 Service primitives

The service parameters for this service are shown in Table 78.

Table 78 – Action Return service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Action Invocation ID	M	M(=)
Action Success	S	S(=)
Return Value	U	U(=)
Action Failure	S	S(=)
Error Info	M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the destination address to which the requested Action Return is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated Action Return is to be sent.

Action Invocation ID

This parameter identifies the specific invocation of the action object whose results are being returned.

Action Success

This selection type parameter is present if the action was successful.

Return Value

This optional parameter carries the results of this invocation of the action.

Action Failure

This selection type argument is present if the action was not successful.

6.2.7.3.8.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

The AP containing the action submits an Action Return Service request primitive to return the results of a previous invocation of the action. An action invocation identifier is included in the request to identify the particular invocation. This identifier is used only by the receiving user, and is only conveyed by the FAL

6.2.7.4 Function invocation state machine

Table 79 gives the State Transitions for a Function Invocation object. The state “NON EXISTENT” indicates that the object does not exist. It is not defined as a valid value for the function invocation state attribute because attributes are created and deleted with the object (and therefore have no value when the object does not exist).

Table 79 – State transitions for a function invocation object

Transition	Current State	Events	Next State
1	NON EXISTENT	Create.Ind && List of Attribute IDs and Initial Values parameter = List of Load Region Object IDs && Specified Load Region objects available For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Create.Rsp(+)	Idle
2	NON EXISTENT	Create.Ind && List of Attribute IDs and Initial Values parameter <> List of Load Region Object IDs Specified Load Region objects not available Create.Rsp(-)	NON EXISTENT
3	IDLE	Delete.Ind && Deleteable=TRUE Delete.Rsp(+)	NON EXISTENT
4	IDLE	Delete.Ind && Deleteable=FALSE Delete.Rsp(-)	IDLE
5	IDLE	Start.Ind	STARTING
6	STARTING	Start succeeds: e.g. Related Load Region Object in state LOADED or IN-USE For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Start.Rsp(+)	RUNNING
7	STARTING	Start fails, non destructive: e.g. Related Load Region Object not in state LOADED or IN USE Start.Rsp(-)	IDLE
8	STARTING	Start fails, destructive Start.Rsp(-)	UN-RUNNABLE
9	RUNNING	Stop.Ind	STOPPING
10	STOPPING	Stop succeeds: e.g. Related Load Region Object in state IN USE For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Stop.Rsp(+)	STOPPED
11	STOPPING	Stop Execution fails, non destructive Stop.Rsp(-)	RUNNING

Table 79 (continued)

Transition	Current State	Events	Next State
12	STOPPING	Stop fails, destructive For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Stop.Rsp(-)	UN-RUNNABLE
13	STOPPED	Resume.Ind	RESUMING
14	RESUMING	Resume succeeds: e.g. Related Load Region Object in state Loaded or In-Use For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Resume.Rsp(+)	RUNNING
15	RESUMING	Resume fails, non destructive: e.g. Load Region not in state LOADED or IN-USE Resume.Rsp(-)	STOPPED
16	RESUMING	Resume fails, destructive For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Resume.Rsp(-)	UNRUNNABLE
17	STOPPED	Reset.Ind	RESETTING
18	RESETTING	Reset succeeds, reusable=TRUE Reset.Rsp(+)	IDLE
19	RESETTING	Reset succeeds, reusable = false Reset.Rsp(+)	UNRUNNABLE
20	RESETTING	Reset fails, non destructive Reset.Rsp(-)	STOPPED
21	RESETTING	Reset fails, destructive Reset.Rsp(-)	UNRUNNABLE
22	RESETTING	Kill.Ind Kill.Rsp(+)	UNRUNNABLE
23	STOPPING	Kill.Ind For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Kill.Rsp(+)	UNRUNNABLE
24	RUNNING	End of Program && Reusable=TRUE	IDLE

Table 79 (continued)

Transition	Current State	Events	Next State
		For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	
25	RUNNING	End of Program && Reusable = false For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	UNRUNNABLE
26	RUNNING	Program stopped For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	STOPPED
27	STOPPED	Delete.Ind && this Function Invocation is not deletable Delete.Rsp(-)	STOPPED
28	STOPPED	Kill.Ind Kill.Rsp(+)	UNRUNNABLE
29	STOPPED	Delete.Ind && this Function Invocation is deletable Delete.Rsp(+)	NON EXISTENT
30	RESUMING	Kill.Ind Kill.Rsp(+)	UNRUNNABLE
31	RUNNING	Kill.Ind For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Kill.Rsp(+)	UNRUNNABLE
32	STARTING	Kill.Ind Kill.Rsp(+)	UNRUNNABLE
33	IDLE	Kill.Ind Kill.Rsp(+)	UNRUNNABLE
34	UNRUNNABLE	Delete.Ind Delete.Rsp(+)	NON EXISTENT

6.3 ARs

6.3.1 Queued user-triggered unidirectional (QUU) AR endpoint class specification

6.3.1.1 Class overview

This class is defined to support the on-demand queued distribution of unconfirmed services to one or more application processes. The behavior of this type of AR can be described as follows.

An AR ASE user wishing to convey a request APDU submits an AR ASE Service Data Unit to the sending endpoint of the AR. The AREP sending the request APDU submits it to its underlying layer for transfer. The underlying layer sends it at its next opportunity. The AREP receiving the request APDU from its underlying layer delivers it to the AR ASE user in the order that it was received.

The following summarizes the characteristics of this AREP class.

Roles:	REPORT SOURCE REPORT SINK
Cardinality:	1-to-n
Timeliness:	No

6.3.1.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Uni-directional AR Endpoint
CLASS ID:	36
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1. (m) Attribute:	Role (REPORT SOURCE, REPORT SINK)
2. (m) Attribute:	AREP State (OPEN, CLOSED)
3. (m) Attribute:	Remote Address Configuration Type (FREE, LINKED)
4. (m) Attribute:	DL Mapping Reference
SERVICES:	
1. (o) OpsService:	Unconfirmed Send

6.3.1.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

REPORT SOURCE Endpoints of this type distribute reports to multiple endpoints using unconfirmed services operating over connectionless data link services.

REPORT SINK Endpoints of this type receive reports from source endpoints using unconfirmed services operating over connectionless data link services.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured.

- | | |
|--------|--|
| FREE | The value of “FREE” indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically in the Unconfirmed service request. |
| LINKED | The value of “LINKED” indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6). |

DL Mapping Reference

This attribute is a reference to the underlying Data Link Layer mapping. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.1.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR.

6.3.2 Queued user-triggered bi-directional connection-oriented (QUB-Co) AR endpoint class specification

6.3.2.1 Class overview

This class is defined to support the on-demand exchange of confirmed services between two application processes. Unconfirmed services are not supported by this type of AR. It uses connection-oriented data link services for the exchanges. The behavior of these classes is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user in the order that it was received.

For a confirmed service request the AREP receiving the request APDU accepts the corresponding response APDU from its AR ASE user and queues it to the underlying layer for transfer.

The AREP that issued the request APDU receives the response APDU from its underlying layer and queues it for delivery to its AR ASE user in the order that it was received. It also stops its associated service response timer.

The following summarizes the characteristics of this AREP class.

Roles:	Client Server Peer
Cardinality:	1-to-1
Timeliness:	No

6.3.2.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional Connection-Oriented AREP
CLASS ID:	34
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1. (m) Attribute:	Role (CLIENT, SERVER, PEER)
2. (m) Attribute:	AREP State
3. (m) Attribute:	Remote Address Configuration Type (FREE, LINKED)
4. (m) Attribute:	Maximum Outstanding Requests Calling
5. (m) Attribute:	Maximum Outstanding Requests Called
6. (m) Attribute:	Initiator (TRUE, FALSE)
7. (o) Attribute:	Remote AP Name
8. (m) Attribute:	Transmit DL Mapping Reference
9. (m) Attribute:	Receive DL Mapping Reference
SERVICES:	
1. (o) OpsService:	Confirmed Send
2. (o) OpsService:	Establish

6.3.2.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER) Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.

CLIENT Endpoints of this type issue confirmed service Request-APDUs to servers and receive confirmed service Response-APDUs.

SERVER Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs to them.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

- | | |
|---------------|--|
| FREE | The value of "FREE" indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically in the Unconfirmed service request. |
| LINKED | The value of "LINKED" indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6). |

Max Outstanding Requests Calling

This conditional attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Max Outstanding Requests Called

This conditional attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Remote AP Name

This optional attribute specifies the name of the AP attached at the remote AREP.

Transmit DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Receive DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the receive conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.2.4 Services**Confirmed Send**

This optional service is used to send a confirmed service on an AR.

Establish

This service is used to establish an AR.

6.3.3 Queued user-triggered bi-directional connectionless (QUB-CL) AR endpoint class specification**6.3.3.1 Class overview**

This class is defined to support the on-demand exchange of confirmed and unconfirmed services between two or more application processes. This class uses connectionless data link services for the exchanges. The Data Link Layer transfers may be either acknowledged or unacknowledged. The behavior of this class is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user in the order that it was received.

For a confirmed service request the AREP receiving the request APDU accepts the corresponding response APDU from its AR ASE user and queues it to the underlying layer for transfer.

The AREP that issued the request APDU receives the response APDU from its underlying layer and queues it for delivery to its AR ASE user in the order that it was received. It also stops its associated service response timer.

The following summarizes the characteristics of this AREP class.

Roles:	Client Server Peer
Cardinality:	1-to-n
Timeliness:	No

6.3.3.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional Connectionless AREP
CLASS ID:	45
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1	(m) Attribute: Role (CLIENT, SERVER, PEER)
2	(m) Attribute: AREP State
3	(m) Attribute: Remote Address Configuration Type (FREE, LINKED)
4	(m) Attribute: Maximum Outstanding Requests Calling
5	(m) Attribute: Maximum Outstanding Requests Called
6	(m) Attribute: Initiator (TRUE, FALSE)
7	(o) Attribute: Remote AP Name
8	(m) Attribute: Transmit DL Mapping Reference
9	(m) Attribute: Receive DL Mapping Reference
SERVICES:	
1	(o) OpsService: Confirmed Send
2	(o) OpsService: Unconfirmed Send
3	(o) OpsService: Establish
4	(o) OpsService: Abort

6.3.3.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER) Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.

CLIENT Endpoints of this type issue confirmed and unconfirmed service Request-APDUs to servers and receive confirmed service Response-APDUs.

SERVER Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs to them. They may also issue unconfirmed service Request APDUs to clients.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

FREE	The value of “FREE” indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically.
LINKED	The value of “LINKED” indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6).

Max Outstanding Requests Calling

This conditional attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Max Outstanding Requests Called

This conditional attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Remote AP Name

This optional attribute specifies the name of the AP attached at the remote AREP.

Transmit DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Receive DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the receive conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.3.4 Services

Confirmed Send

This optional service is used to send a confirmed service on an AR.

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR.

Establish

This service is used to establish an AR.

Abort

This service is used to abort an AR.

6.3.4 Queued user-triggered bi-directional with flow control (QUB-FC) AR endpoint class specification

6.3.4.1 Class overview

This class is defined to support asynchronous operation of the client/server model using queues. It provides for the conveyance of confirmed and unconfirmed services using application layer flow control for unconfirmed services and uses connection-oriented data link services for the exchanges. The data link priority for the transfers is specified separately for each transfer.

The behavior of this class is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity with exception of unconfirmed acknowledged services. The latter will be ignored if the outstanding service counter has reached the negotiated maximum.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user, in the order that it was received. If the received APDU contains an unconfirmed acknowledged request, the AREP issues the appropriated indication to the AR ASE user and a UCA_AckPDU to the underlying layer for transfer.

For a confirmed service request the AREP receiving the request APDU accepts the corresponding response APDU from its AR ASE user and queues it to the underlying layer for transfer.

The AREP that issued the request APDU receives the response APDU from its underlying layer and queues it for delivery to its AR ASE user in the order that it was received and decrements the appropriate flow control counter.

Additionally the AREP monitors the connection in cases if no service from the AR ASE user is outstanding by sending an IdlePDU to the peer instance. The TCTimer (Transmit Client Timer) and the TSTimer (Transmit Server Timer) monitor the sending of IdlePDUs on both connection endpoints. The RCTimer (Receive Client Timer) and RSTimer (Receive Server Timer) monitor the liveness of the connection on both connection endpoints. The RCTimer or RSTimer expire if no PDU (also no IdlePDU) has been received during a certain amount of time and causes the termination of the AREP connection.

Furthermore, the AR ASE user at server site is able to interrupt or continue the data flow by issuing the AR-XON-XOFF service.

The following summarizes the characteristics of this AREP class.

Roles	Client Server Peer
Cardinality	1-to-1
Timeliness	No

6.3.4.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional with Flow Control AREP
CLASS ID:	46
PARENT CLASS:	AR Endpoint

NETWORK MANAGEMENT ATTRIBUTES:

1	(m)	Attribute:	Role (CLIENT, SERVER, PEER)
2	(m)	Attribute:	AREP State
3	(m)	Attribute:	Remote Address Configuration Type (FREE, LINKED)
4	(m)	Attribute:	Maximum Outstanding Requests Calling
5	(m)	Attribute:	Maximum Outstanding Requests Called
6	(m)	Attribute:	Max Outstanding Unconfirmed Requests Client (maxUCSC)
7	(m)	Attribute:	Max Outstanding Unconfirmed Requests Server (maxUCSS)
8	(m)	Attribute:	Maximum Outstanding Requests Client (maxOSCC)
9	(m)	Attribute:	Maximum Outstanding Requests Server (maxOSCS)
10	(m)	Attribute:	Initiator (TRUE, FALSE)
11	(o)	Attribute:	Remote AP Name
12	(m)	Attribute:	Transmit DL Mapping Reference
13	(m)	Attribute:	Receive DL Mapping Reference
14	(m)	Attribute:	CIU

SERVICES:

1	(o)	OpsService:	Confirmed Send
2	(o)	OpsService:	Unconfirmed Send
3	(o)	OpsService:	Establish
4	(o)	OpsService:	Abort

6.3.4.3 Network management attributes**Role**

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER) Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.

CLIENT Endpoints of this type issue confirmed and unconfirmed service Request-APDUs to servers and receive confirmed service Response-APDUs.

SERVER Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs to them. They may also issue unconfirmed service Request APDUs to clients.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

FREE	The value of "FREE" indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically in the Unconfirmed service request.
LINKED	The value of "LINKED" indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6).

Max outstanding Requests Calling

This attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Max Outstanding Requests Called

This attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Max Outstanding Unconfirmed Requests Client (maxUCSC)

This attribute indicates the maximum number of requests that the AR Endpoint may be expecting from its local user.

Max Outstanding Unconfirmed Requests Server (maxUCSS)

This attribute indicates the maximum number of requests that the AREP may be expecting from the remote AREP.

Maximum Outstanding Requests Client (maxOSCC)

This attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Maximum Outstanding Requests Server (maxOSCS)

This attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Remote AP Name

This optional attribute specifies the name of the AP attached at the remote AREP.

Transmit DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Receive DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the receive conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

CIU

This attribute (CIU - Control Interval User-triggered) contains the control interval for monitoring of a user triggered connection. The attribute is set by network management and used by the ARPM. If this attribute contains a value not 0 then the ARPM transmits an Idle PDU while AR ASE user has not passed any service during the control interval (CIU/3). If CIU expires then the connections will be terminated.

6.3.4.4 Services**Confirmed Send**

This optional service is used to send a confirmed service on an AR.

Establish

This service is used to establish an AR.

Abort

This service is used to disconnect an AR.

Unconfirmed Send

This service is used to send an unconfirmed service on an AR.

6.3.5 Queued user-triggered bi-directional with segmentation(QUB-Seg) AR endpoint class specification

6.3.5.1 Class overview

This class is defined to support the on-demand exchange of confirmed and unconfirmed services between two application processes. It uses connection-oriented datalink services for the exchanges. It supports segmentation of APDUs within the Application Layer. The data link priority for the transfers is specified separately for each transfer direction.

An AR ASE service data unit (if its size is greater than a single transferable element) can be split into several APDUs which are sent to its underlying layer to be transferred. The underlying layer sends each segment at the next opportunity. The AREP receiving the request APDU segments from its underlying layer reassembles the APDU and delivers it to the AR ASE. Delivery of multiple APDUs is done in the order in which they are completely received.

The following summarizes the characteristics of this AREP class.

Roles:	Client Server Peer
Cardinality:	1-to-1
Timeliness:	No

6.3.5.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional Segmentation AREP
CLASS ID:	45
PARENT CLASS:	AR Endpoint

NETWORK MANAGEMENT ATTRIBUTES:

1. (m) Attribute: Role (CLIENT, SERVER, PEER)
2. (m) Attribute: AREP State
3. (m) Attribute: Remote Address Configuration Type (FREE, LINKED)
4. (m) Attribute: Maximum Outstanding Requests Calling
5. (m) Attribute: Maximum Outstanding Requests Called
6. (m) Attribute: Initiator (TRUE, FALSE)
7. (o) Attribute: Remote AP Name
8. (m) Attribute: Transmit DL Mapping Reference
9. (m) Attribute: Receive DL Mapping Reference

SERVICES:

1. (o) OpsService: Confirmed Send
2. (o) OpsService: Unconfirmed Send
3. (o) OpsService: Establish
4. (o) OpsService: Reject
5. (o) OpsService: Abort

6.3.5.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER)	Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.
CLIENT	Endpoints of this type issue confirmed service Request-APDUs to servers and receive confirmed service Response-APDUs and unconfirmed service Request PDU.
SERVER	Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs and unconfirmed service Request PDU to them. They may also issue unconfirmed service Request APDUs to clients.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

FREE	The value of "FREE" indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically.
LINKED	The value of "LINKED" indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6).

Max Outstanding Requests Calling

This conditional attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Max Outstanding Requests Called

This conditional attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Remote AP Name

This optional attribute specifies the name of the AP attached at the remote AREP.

Transmit DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Receive DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the receive conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.5.4 Services

Confirmed Send

This optional service is used to send a confirmed service on an AR.

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR.

Establish

This service is used to establish an AR.

Abort

This service is used to Abort an AR.

Reject

This provider initiated service is used to signal the detection of a protocol error.

6.3.6 Buffered user-triggered bi-directional (BUB) AR endpoint class specification

6.3.6.1 Class overview

This class is defined to support the on-demand exchange of confirmed and of unconfirmed services between two application processes using buffers.

The behavior of this type of AR can be described as follows.

An AR ASE user wishing to convey a request or response APDU submits it to its AREP as an AR ASE Service Data Unit. The AREP sending the request or response APDU writes it into the Data Link Layer buffer, completely replacing the existing contents of the buffer. The Data Link Layer transfers the buffer contents at the earliest transfer opportunity. The buffer may also be transferred whenever a request to transmit (AR Compel) is received, either locally or from the network.

The AREP receiving the request or response APDU from its underlying layer, buffers it for delivery. If the APDU is a response, the endpoint stops its associated timer.

If the AREP that sent the APDU receives another APDU before the buffer contents are transmitted, the buffer contents will be replaced with the new APDU, and the previous APDU will be lost. When the buffer contents are transmitted, the AR ASE notifies the user of the transmission.

At the receiving endpoint, the APDU is received from the network and written into the buffer, completely replacing the existing contents of the buffer. The endpoint notifies the user that the APDU has arrived and delivers it to the user according to the local user interface. If the APDU has not been delivered before the next APDU arrives, it will be replaced by the next APDU and lost.

An FAL user receiving the buffered transmission may later request to receive the currently buffered APDU.

The following summarizes the characteristics of this AREP class.

Roles:	Client Server Peer
Cardinality:	1-to-1
Timeliness:	No

6.3.6.2 Formal model

FAL ASE:		AR ASE	
CLASS:	Buffered User-triggered Bi-directional AREP		
CLASS ID:	42		
PARENT CLASS:	AR Endpoint		
NETWORK MANAGEMENT ATTRIBUTES:			
1	(m)	Attribute:	Role (CLIENT, SERVER, PEER)
2	(m)	Attribute:	AREP State
3	(m)	Attribute:	Confirmed Services Flag(TRUE/FALSE)
4	(m)	Attribute:	Initiator (TRUE, FALSE)
5	(m)	Attribute:	Transmit DL Mapping Reference
6	(m)	Attribute:	Receive DL Mapping Reference
7	(c)	Constraint:	Confirmed Services = TRUE
7.1	(m)	Attribute:	Max Outstanding Requests Calling
7.2	(m)	Attribute:	Max Outstanding Requests Called
8	(m)	Attribute:	Remote AP Name
SERVICES:			
1	(o)	OpsService:	Unconfirmed Send
2	(o)	OpsService:	AR Compel
3	(o)	OpsService:	Confirmed Send
4	(o)	OpsService:	Establish
5	(o)	OpsService:	Get Buffered Message

6.3.6.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER) Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.

CLIENT Endpoints of this type issue confirmed and unconfirmed service Request-APDUs to servers and receive confirmed service Response-APDUs.

SERVER Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs to them. They may also issue unconfirmed service Request APDUs to clients.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Confirmed Services Flag

This attribute specifies, when TRUE, that this endpoint supports the conveyance of confirmed service requests.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Transmit DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL-Mapping References for AREPs are specified in IEC 61158-6.

Receive DL Mapping Reference

This attribute provides the mapping to the underlying layer for the receive conveyance path for this AREP. DL-Mapping References for AREPs are specified in IEC 61158-6.

Max Outstanding Requests Calling

This conditional attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP. This attribute is present only if this endpoint supports confirmed services (confirmed services = TRUE).

Max Outstanding Requests Called

This conditional attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user. This attribute is present only if this endpoint supports confirmed services (Confirmed Services = TRUE).

Remote AP Name

This attribute specifies the AP attached to the remote AREP.

6.3.6.4 Services**Unconfirmed Send**

This optional service is used to send an unconfirmed service on an AR.

AR Compel

This optional service is used to cause an APDU locally or remotely buffered for transmission in the Data Link Layer to be transmitted at the next available opportunity.

Confirmed Send

This optional service is used to send a confirmed service on an AR.

Establish

This service is used to establish an AR.

Get Buffered Message

This local service is used to retrieve an APDU from the buffer used by an AR.

6.3.7 Buffered network-scheduled uni-directional (BNU) AR endpoint class specification**6.3.7.1 Class overview**

This class is defined to support the “push” model for scheduled, buffered distribution of unconfirmed services to one or more application processes.

The behavior of this type of AR can be described as follows.

An AR ASE user wishing to convey a request or response APDU submits it as an AR ASE Service Data Unit to its AREP for distribution. The AREP sending the request or response APDU writes it into the Data Link Layer buffer, completely replacing the existing contents of the buffer. The Data Link Layer transfers the buffer contents at the next scheduled transfer opportunity. The buffer is also transferred whenever a request to transmit (AR Compel) is received, either locally or from the network.

NOTE Unscheduled requests to transmit have no effect on the behavior of this type of AR. They are treated as "out of band" with respect to the AR behavior.

If the AREP that sent the APDU receives another APDU before the buffer contents are transmitted, the buffer contents will be replaced with the new APDU, and the previous APDU will be lost. When the buffer contents are transmitted, the AR ASE notifies the user of the transmission.

At the receiving endpoint, the APDU is received from the network and is immediately written into the buffer, completely overwriting the existing contents of the buffer. The endpoint notifies the user that the APDU has arrived and delivers it to the user according to the local user interface. If the APDU has not been delivered before the next APDU arrives, it will be overwritten by the next APDU and lost.

An FAL user receiving the buffered transmission may later request to receive the currently buffered APDU.

This type of AR also supports, as an option, the ability of the AR ASE to determine publishing, transmission, or reception timeliness. For publishing timeliness, the sending AR ASE will mark the buffer if it has not been updated within a specified update interval. For reception timeliness, the receiving AR ASE will mark the buffer if it has not been received within a specified reception interval. Both publishing and reception timeliness information are given. For transmission timeliness, the sending AR ASE determines if the buffer has been transmitted within a specified interval. If the transmission does not occur, the FAL user is notified and the buffer contents are updated to indicate the lack of transmission timeliness.

Residence timeliness is supported by this type of AREP. Figure 19 illustrates this type of timeliness.

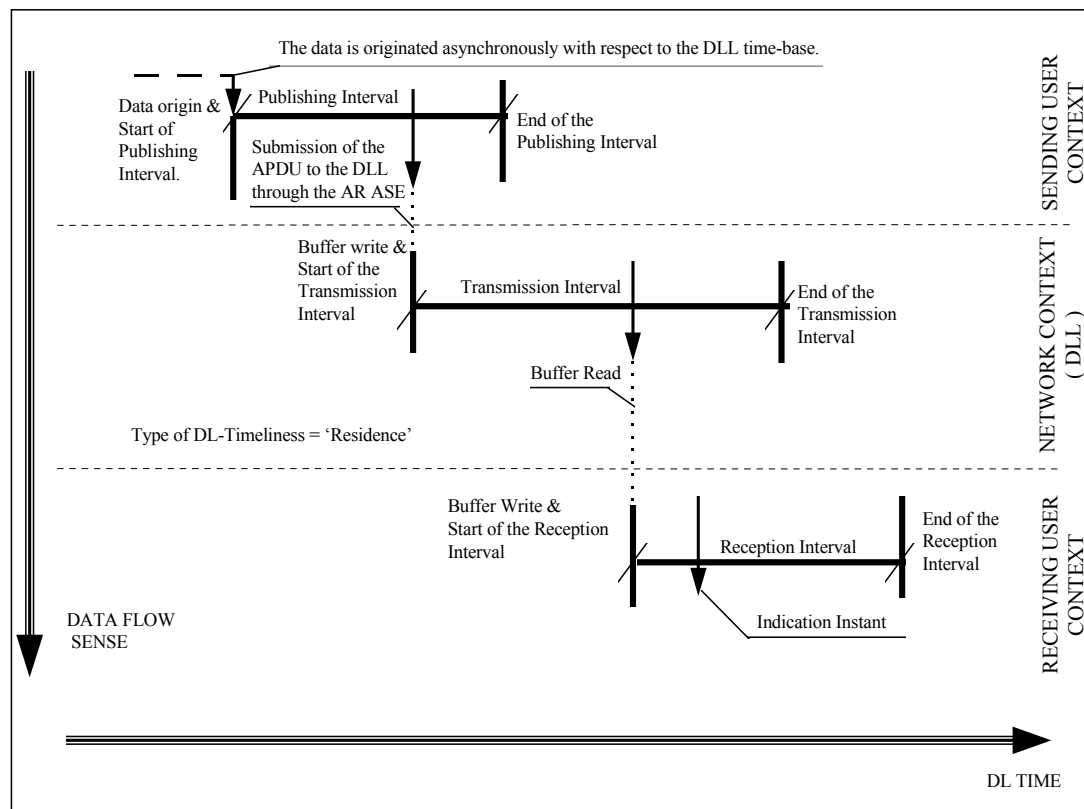


Figure 19 – Residence timeliness

This type of timeliness requires the use of a data link timer associated with data buffer (transmit or receive). The timer is started when the data is written into the transmit buffer by the publisher AREP or received from the network into the subscriber buffer. The length of the time interval is configured for the AREP by management.

The timeliness indicator for this type of AREP is computed as follows:

Publisher AREP Data is timely if it is transmitted within the timeliness window after it is placed into the data link buffer for transmission. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent transmissions of the same data from the buffer will indicate its current status.

Subscriber AREP Data is timely if it is received as timely by the Data Link Layer and is read from the receive buffer by the AREP within the timeliness window that was started when the data was received. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent reads from the buffer of the same data will indicate its current status.

Synchronized timeliness is supported by this type of AREP. Figure 20 illustrates this type of timeliness.

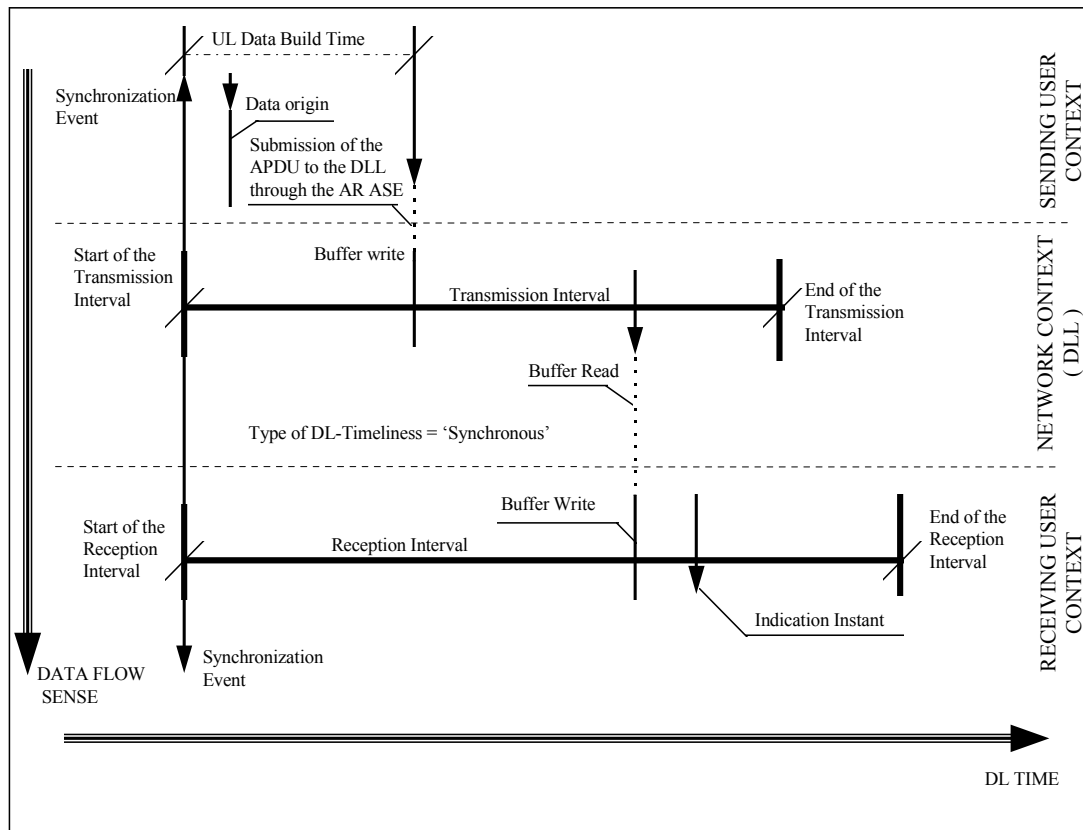


Figure 20 – Synchronized timeliness

This type of timeliness requires the use of two supporting data link connections. One is used to publish the data and the other is used to distribute a “sync mark” within the Data Link Layer to each of the stations participating as either a publisher or a subscriber. The sync mark causes a timer associated with data buffer (transmit or receive) to be started within the Data Link Layer. The length of the time interval is configured for the AREP by management.

The timeliness indicator for this type of AREP is computed as follows:

Publisher AREP Data is timely if it is placed into the buffer within the time window after the receipt of a “sync mark”. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent transmissions of the same data from the buffer will indicate the current status of the buffer.

Subscriber AREP Data is timely if it is received as timely by the Data Link Layer and is received within the time window. When the timeliness window expires, the data in the buffer is regarded as NOT Timely. Data received after the expiration of the window, but before the next sync mark is also considered NOT Timely. Subsequent reads from the buffer of the same data will indicate it current status.

The following summarizes the characteristics of this AREP class.

Roles: Push Publisher
Push Subscriber

Cardinality: 1-to-Many

Timeliness: Optional

6.3.7.2 Formal model

FAL ASE: **AR ASE**

CLASS: Buffered Network-Scheduled Uni-directional AR Endpoint

CLASS ID: 38

PARENT CLASS: AR Endpoint

NETWORK MANAGEMENT ATTRIBUTES:

- | | | | |
|-----|-----|------------|--|
| 1 | (m) | Attribute: | Role (PUSH PUBLISHER, PUSH SUBSCRIBER) |
| 2 | (m) | Attribute: | AREP State |
| 2 | (c) | Constraint | Role == PUSH SUBSCRIBER |
| 2.1 | (o) | Attribute: | Duplicate Pdu Detection Supported |
| 3 | (m) | Attribute: | DL Mapping Reference |

SERVICES:

- | | | | |
|---|-----|------------|---------------------------|
| 1 | (o) | OpService: | Unconfirmed Send |
| 2 | (o) | OpService: | AR Compel |
| 3 | (o) | OpService: | Get Buffered Message |
| 4 | (o) | OpService: | Schedule Communication |
| 5 | (o) | OpService: | Cancel Scheduled Sequence |

6.3.7.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PUSH-PUBLISHER Endpoints of this type publish their data issuing unconfirmed service Request-APDUs.

PUSH-SUBSCRIBER Endpoints of this type receive data published in confirmed service Response-APDUs.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

DL Mapping Reference

For PUSH-PUBLISHER AREPs, this attribute specifies the mapping to the transmit conveyance path. For PUSH-SUBSCRIBER AREPs, this attribute specifies the mapping to the receive conveyance path. DL mapping attributes for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.7.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR.

AR Compel

This optional service is used to cause an APDU locally buffered for transmission (for PUSH-PUBLISHER AREPs) or remotely buffered for transmission (for PUSH-SUBSCRIBER AREPs) in the Data Link Layer to be transmitted at the next available opportunity.

Get Buffered Message

This local service is used to retrieve an APDU from the buffer used by an AR.

Schedule Communication

This local service provides the AL user with the ability to schedule a sequence of DL-COMPELs for the AREP.

Cancel Scheduled Sequence

This local service provides the AL user with the ability to cancel an existing sequence which has previously been scheduled for the AREP.

6.3.8 Buffered network-scheduled bi-directional (BNB) AR endpoint class specification

6.3.8.1 Class overview

This class is defined to support the cyclic exchange of confirmed services (only Read) and on-demand exchange of unconfirmed services between two application processes using buffers. The cycle is given by the network.

The behavior of this type of AR can be described as follows.

An AR ASE user wishing to convey a Read request or unconfirmed request APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU writes it into the Data Link Layer buffer, completely replacing the existing contents of the buffer. Furthermore, a copy of the APDU is stored at a local buffer within the AREP if the Data Link Layer buffer is current used by a precede request. The Data Link Layer transfers the buffer contents once at the earliest transfer by opportunity when a request to transmit (AR Compel) is received from the network.

At the receiving endpoint, the APDU is received from the network, it is written into the buffer, completely replacing the existing contents of the buffer. The endpoint notifies the user that the confirmed APDU (Read) has arrived and delivers it to the user according to the local user interface. Furthermore, a copy of the APDU is stored at a local buffer within the AREP. The following response from the AR ASE user is written by the AREP into the Data Link Layer buffer, completely replacing the existing contents of the buffer. The Data Link Layer transfers the buffer contents once at the earliest transfer opportunity when a request to transmit (AR Compel) is received from the network. The AR ASE user is notified again with the restored request when the response is sent over the network. This causes the AR ASE user to response again the current requested variable in a cyclic manner.

At the requesting endpoint, the APDU containing the response received from the network, it is written to the buffer, completely replacing the existing contents of the buffer. The endpoint notifies the user that the APDU has arrived and delivers it to the user according to the local user interface as the confirmation. If the AR ASE user wishes to convey a Read request again the response is generated locally from the Data Link Layer buffer. The content of the buffer is cyclic updated by the network with the related mechanism at the receiving endpoint.

The AREP receiving an unconfirmed request APDU from its underlying layer, queues it for delivery to its AR ASE user in the order that it was received.

Additionally the AREP monitors the connection by observing the network activities.

The following summarizes the characteristics of this AREP class.

Roles:	Peer
Cardinality:	1-to-1
Timeliness:	No

6.3.8.2 Formal model

FAL ASE: **AR ASE**

CLASS: Buffered Network-Triggered Bi-directional AR Endpoint

CLASS ID: 44

PARENT CLASS: AR Endpoint

NETWORK MANAGEMENT ATTRIBUTES:

- | | | | |
|---|-----|------------|--|
| 1 | (m) | Attribute: | Role (CLIENT, SERVER, PEER) |
| 2 | (m) | Attribute: | AREP State |
| 3 | (m) | Attribute: | Remote Address Configuration Type (FREE, LINKED) |
| 4 | (m) | Attribute: | Initiator (TRUE/FALSE) |
| 5 | (m) | Attribute: | Receive DL Mapping Reference |
| 6 | (m) | Attribute: | Transmit DL Mapping Reference |
| 7 | (o) | Attribute: | Remote AP Name |
| 8 | (m) | Attribute: | CIN |

SERVICES:

- | | | | |
|---|-----|------------|------------------|
| 1 | (o) | OpService: | Unconfirmed Send |
| 2 | (o) | OpService: | Confirmed Send |
| 3 | (o) | OpService: | Establish |
| 4 | (o) | OpService: | Abort |

6.3.8.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

CLIENT, SERVER, PEER () Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type shall indicate whether or not they are to be the initiator of the AR establishment process.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Remote Address Configuration Type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

- | | |
|--------|--|
| FREE | The value of "FREE" indicates that the destination of the FAL-PDUs from the Source AREP is provided dynamically in the request service. |
| LINKED | The value of "LINKED" indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6). |

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Receive DL Mapping Reference

This attribute provides a reference to the underlying Data Link Layer mapping for the conveyance path from the Publishing Manager. DL mapping attributes for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Transmit DL Mapping Reference

This conditional attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path. It is present only for PULL PUBLISHERs and PULL PUBLISHING MANAGERs. DL mapping attributes for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

Remote AP Name

This optional attribute specifies the name of the AP attached at the remote AREP.

CIN

This attribute (CIN - Control Interval Network triggered) indicates the control interval for monitoring a network triggered connection.

6.3.8.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR.

Confirmed Send

This optional service is used to send a confirmed service on an AR.

Establish

This service is used to establish an AR.

DeEstablish

This service is used to de-establish (gracefully terminate) an AR.

Abort

This service is used to terminate an AR.

6.3.9 Buffered network-scheduled and unscheduled uni-directional (BNU-Mp) AR endpoint class specification

6.3.9.1 Class overview

This class is defined to support the “push” model for scheduled, buffered distribution of unconfirmed services to one or more application processes.

The behavior of this type of AR can be described as follows.

The push model is used for the scheduled traffic, as follows.

An AR ASE user wishing to broadcast new data, uses an AR ASE Service Data Unit with a special encoding (see IEC 61158-6) to its AREP for distribution. The AREP sending the data writes it into the Data Link Layer buffer, completely replacing the existing contents of the buffer. The Data Link Layer transfers the buffer contents at the next scheduled opportunity. When the buffer contents are transmitted, the AR ASE notifies the user of the transmission.

If the AREP that sent data receives from the AR ASE user another data before the buffer contents are transmitted, the buffer contents will be replaced with the new data and the previous data will be lost.

At the receiving endpoint, the data is received from the network, it is immediately written into the buffer, completely overwriting the existing contents of the buffer. The endpoint notifies the user that the data has arrived and delivers it to the user. Extra local readings of the contents of the buffer are possible. If the data has not been delivered before the next data arrives, it will be overwritten by the next data and lost.

The pull model is used for the on-demand traffic, as follows.

An AR ASE user wishing to send or read the remote value of the buffer, uses the Remote-Write or Remote-Read services with an AR ASE Service Data Unit specially encoded (see IEC 61158-6). An AR ASE user wishing to broadcast data on-demand writes it into the Data Link Layer buffer replacing the existing contents of the buffer. The Data Link Layer transfers the buffer contents after soliciting a transmission opportunity. The transmission of the buffer is reported to the user as the confirmation of the service. At the receiving endpoint, the data is received from the network and is immediately written into the buffer, completely overwriting the existing contents of the buffer. The endpoint notifies the user that the data has arrived and delivers it to the user. Extra local readings of the contents of the buffer are possible.

An AR ASE user wishing to read the remote value of a buffer, asks the Data Link Layer to solicit a transmission opportunity to get the value of the remote buffer. The transmission of the buffer is reported to the user as the confirmation of the service. At the receiving endpoint, the data is received from the network and is immediately written into the buffer, completely overwriting the existing contents of the buffer. The endpoint notifies the user that the data has arrived and delivers it to the user. Extra local readings of the contents of the buffer are possible.

This type of AR also supports, as an option, the ability of the AR ASE to determine publishing, transmission, or reception timeliness. For publishing timeliness, the sending AR ASE will mark the buffer if it has not been updated within a specified update interval. For reception timeliness, the receiving AR ASE will mark the buffer if it has not been received within a specified reception interval. Both publishing and reception timeliness information are given. For transmission timeliness, the sending AR ASE determines if the buffer has been transmitted within a specified interval. If the transmission does not occur, the FAL user is notified and the buffer contents are updated to indicate the lack of transmission timeliness.

Residence timeliness is supported by this type of AREP. Figure 21 illustrates this type of timeliness.

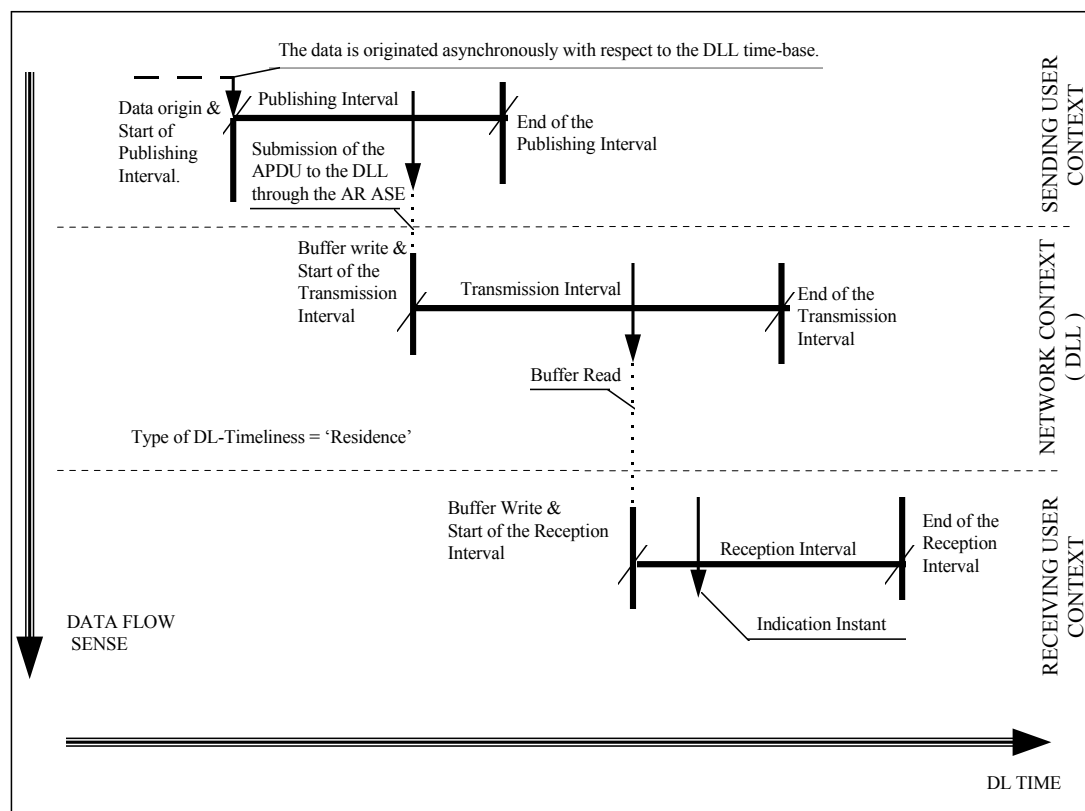


Figure 21 – Residence timeliness

This type of timeliness requires the use of a data link timer associated with data buffer (transmit or receive). The timer is started when the data is written into the transmit buffer by the publisher AREP or received from the network into the subscriber buffer. The length of the time interval is configured for the AREP by management.

The timeliness indicator for this type of AREP is computed as follows.

Publisher AREP Data is timely if it is transmitted within the timeliness window after it is placed into the data link buffer for transmission. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent transmissions of the same data from the buffer will indicate its current status.

Subscriber AREP Data is timely if it is received as timely by the Data Link Layer and is read from the receive buffer by the AREP within the timeliness window that was started when the data was received. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent reads from the buffer of the same data will indicate its current status.

Synchronized timeliness is supported by this type of AREP. Figure 22 illustrates this type of timeliness.

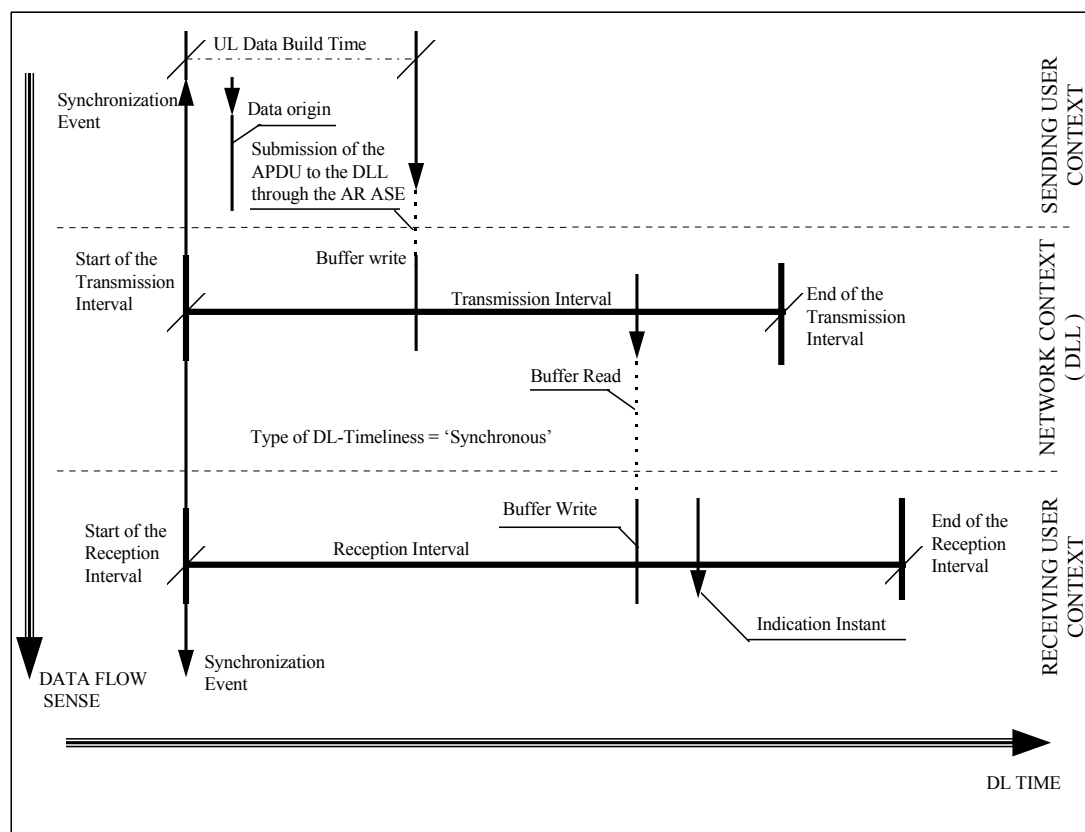


Figure 22 – Synchronized timeliness

This type of timeliness requires the use of two supporting data link connections. One is used to publish the data and the other is used to distribute a “sync mark” within the Data Link Layer to each of the stations participating as either a publisher or subscriber. The sync mark causes a timer associated with data buffer (transmit or receive) to be started within the Data Link Layer. The length of the time interval is configured for the AREP by management.

The timeliness indicator for this type of AREP is computed as follows.

Publisher AREP data is timely if it is placed into the buffer within the time window after the receipt of a “sync mark”. When the timeliness window expires, the data in the buffer is regarded as NOT timely. Subsequent transmissions of the same data from the buffer will indicate the current status of the buffer.

Subscriber AREP Data is timely if it is received as timely by the Data Link Layer and is received within the time window. When the timeliness window expires, the data in the buffer is regarded as NOT Timely. Data received after the expiration of the window, but before the next sync mark, is also considered NOT Timely. Subsequent reads from the buffer of the same data will indicate its current status.

The following summarizes the characteristics of this AREP class.

Roles:	Push Publisher Push Subscriber
Cardinality:	1-to-Many
Timeliness:	Optional

6.3.9.2 Formal model

FAL ASE: **AR ASE**

CLASS: Buffered Network-Scheduled Uni-directional AR Endpoint

CLASS ID: 39

PARENT CLASS: AR Endpoint

NETWORK MANAGEMENT ATTRIBUTES:

- | | | | |
|---|-----|------------|---|
| 1 | (m) | Attribute: | Role (PUSH PUBLISHER, PUSH SUBSCRIBER, PULL PUBLISHER, PULL SUBSCRIBER) |
| 2 | (m) | Attribute: | AREP State |
| 3 | (m) | Attribute: | DL Mapping Reference |

SERVICES:

- | | | | |
|---|-----|------------|---------------------------|
| 1 | (o) | OpService: | Unconfirmed Send |
| 2 | (o) | OpService: | AR Compel |
| 3 | (o) | OpService: | Get Buffered Message |
| 4 | (o) | OpService: | Schedule Communication |
| 5 | (o) | OpService: | Cancel Scheduled Sequence |
| 6 | (o) | OpService: | Remote Read |
| 7 | (o) | OpService: | Remote Write |

6.3.9.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PUSH-PUBLISHER Endpoints of this type publish their data issuing unconfirmed service Request-APDUs.

PUSH-SUBSCRIBER Endpoints of this type receive data published in confirmed service Response-APDUs.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

DL Mapping Reference

For PUSH-PUBLISHER AREPs, this attribute specifies the mapping to the transmit conveyance path. For PUSH-SUBSCRIBER AREPs, this attribute specifies the mapping to the receive conveyance path. DL mapping attributes for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

6.3.9.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR. The structure of the PDU is specified and described in the corresponding DL-Mapping State Machine.

AR Compel

This optional service is used to cause an APDU locally buffered for transmission (for PUSH-PUBLISHER AREPs) or remotely buffered for transmission (for PUSH-SUBSCRIBER AREPs) in the Data Link Layer to be transmitted at the next available opportunity.

Get Buffered Message

This local service is used to retrieve an APDU from the buffer used by an AR.

Schedule Communication

This local service provides the AL user with the ability to schedule a sequence of DL-COMPELs for the AREP.

Cancel Scheduled Sequence

This local service provides the AL user with the ability to cancel an existing sequence which has previously been scheduled for the AREP.

Remote Read

This service (based on the Pull Subscriber model) allows a user to read the contents of a remote buffer. The structure of the PDU is described and specified in the corresponding DL-Mapping State Machine in the protocol document.

Remote Write

This service (based on the Pull Subscriber model) allows a user to write the contents of a remote buffer. The structure of the PDU is specified and described in the corresponding DL-Mapping State Machine in the protocol document.

6.4 Summary of FAL classes

This subclause contains a summary of the defined FAL Classes. The Class ID values have been assigned to be compatible with existing standards.

Table 80 provides a summary of the classes.

Table 80 – FAL class summary

FAL ASE	Class	Class ID
Application Process	Application Process	16
Data Type	Fixed Length & String Data Type	5
	Structure Data Type	6
	Array Data Type	12
Application Relationship	AREP	32
	QUB-Co	34
	QUU	36
	BNU	38
	BNU Mp	39
	BUU (for future study)	40
	BUB	42
	BNB	44
	QUB-CI	45
	QUB-FC	46
	QUB-Seg	48
Variable	Fixed Length & String Variable	7
	Array Variable	8
	Data Structure Variable	9
	Variable List	10
Event	Event	4
	Event List	17
	Notifier	18
Load Region	Load Region	2
Function Invocation	Function Invocation	3
	Action	19

6.5 Permitted FAL services by AREP role

Table 81 below defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 81 – Services by AREP role

	Unc	Cnf	Client	Server	Push Publ	Push Subsc	Pull Publ Mgr	Pull Publ	Pull Subsc	Report Src	Report Sink
FAL Services			req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv
Mgt ASE											
Create		X	X	X							
Delete		X	X	X							
Get Attributes		X	X	X							
Get Attribute List		X	X	X							
Set Attributes		X	X	X							
Begin Set Attributes		X	X	X							
End Set Attributes		X	X	X							
AP ASE											
Subscribe		X	X	X							
Identify		X	X	X							
Get Status		X	X	X							
Status Notification	X				X	X				X	X
Initiate		X	X	X							
Terminate		X	X	X							

	Unc	Cnf	Client	Server	Push Publ	Push Subsc	Pull Publ Mgr	Pull Publ	Pull Subsc	Report Src	Report Sink
			req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv
FAL Services											
Conclude		X	X	X							
Reject		X	X								
AR ASE											
AR-Confirmed Send			X	X							
AR-Unconfirmed Send			X X	X X	X	X					
AR-Establish			X	X							
AR-De-Establish			X	X							
AR-Abort			X X	X X	X	X X				X	X
AR Compel			X	X		X					
AR-Get Buffered Msg			X	X		X					
AR-Schedule					X	X					
Communication											
AR-Cancel Schedule					X	X					
Sequence											
AR-Status			X	X							
AR-XON-OFF			X	X							
AR Remote Read							X	X	X		
AR Remote Write							X	X	X		
Variable ASE											
Read		X	X	X			X X	X			
Read List		X	X	X							
Write		X	X	X							
Write List		X	X	X							
Information Report	X				X	X				X	X
Information Report List	X				X	X				X	X
Exchange		X	X	X							
ExchangeList		X	X	X							
Event ASE											
Unconfirmed Ack	X				X	X				X	X
Events											
Confirmed Ack Event		X	X	X							
Ack Event List		X	X	X							
Enable Event		X	X	X							
Enable Event List		X	X	X							
Get Event Summary		X	X	X							
Get Event Summary		X	X	X							
List											
Query Event Summary		X	X	X							
List											
Event Notification	X				X	X				X	X
Notification Recovery	X				X	X				X	X
Load Region ASE											
Initiate Load		X	X	X							
Push Segment		X	X	X							
Pull Segment		X	X	X							
Terminate Load		X	X	X							
Discard		X	X	X							
Function Invocation ASE											
Start		X	X	X							
Stop		X	X	X							
Resume		X	X	X							
Reset		X	X	X							
Kill		X	X	X							
Action Invoke	X									X	X
Action Return	X									X	X

7 Type 2 communication model specification

7.1 Concepts

7.1.1 General

This is a serial communication system for communication between devices that wish to exchange both time critical and messaging type application information. These devices include simple I/O devices, such as sensors/actuators as well as complex control devices such as robots, programmable logic controllers, welders, process controllers, etc.

Unlike some general purpose communication systems that rely on the destination delivery model, this network uses a variant of the publisher/subscriber push model, called the producer/consumer model. The producer/consumer model allows the exchange of time critical application information between a sending device (i.e. the producer) and many receiving devices (i.e. the consumers) without the need to send the data separately to each destination. This is accomplished by attaching a unique identifier to each piece of application information that is being produced onto the network medium. Any device that requires a specific piece of application information simply filters the data on the network medium for the appropriate identifier. Many devices can receive the same piece of application information from a single producing device.

The Data Link Layer provides a high degree of protocol efficiency by utilising an implied token passing mechanism. This mechanism allows all devices equal access to the network without the network overhead associated with passing a “token” to each device granting it permission to send data. The protocol utilises a time based scheduling mechanism which provides network devices with deterministic and predictable access to the medium while preventing network collisions. This scheduling mechanism allows time critical data, which is required on a periodic, repeatable and predictable basis, to be produced on a predefined schedule without the loss of efficiency associated with continuously requesting or “polling” for the required data. The protocol supports an additional mechanism which allows data that is not time critical in nature or which is only required on an occasional basis to utilise any available network time. This unscheduled data is transmitted after the production of the time critical data has been completed and before the beginning of the next scheduled production of time critical data.

7.1.2 General concepts

Most of the general concepts described in Clause 4 apply, with some restrictions or additions as noted:

- the Application Layer includes those functionalities from the intermediate layers which are necessary for proper mapping onto the Data Link Layer,
- Client/Server relationships can be one-to-one, but also one-to-many (see AR model in 7.3.1),
- a variant of the Publisher/Subscriber Push model, the Producer/Consumer model, is used as the base of all AR's (see AR model in 7.3.1),
- an overview of the ASE/APO types and their relationships is provided in 7.1.3,
- AR follow a model which is detailed in 7.3.1.
- specific naming and addressing is specified in 7.1.4,
- common FAL attributes and parameters from Clause 4 are not used; instead they are specified in relevant Type-specific subclauses.
- there are no Abort or Reject services, only negative result confirmations, so the error procedure described in Annex A does not apply.

7.1.3 Relationships between ASE's

Every node shall contain as a minimum instance one of the following ASE object classes in its application layer :

Object Management ASE :

- Identity (identification and general information about the device)
- Message Router (messaging connection point for communications within the device)

Connection Manager ASE :

- Connection Manager (establishment and maintenance of connections)

AR ASE :

- Unconnected Message Manager (queued messaging services on a single link)

Other application layer objects may be implemented in some nodes only :

Object Management ASE :

- Assembly Object (binds data from multiple objects to be sent through one connection)
- additional application-specific objects constructed according to the general Type 2 formal model

AR ASE :

- Transports (connected messaging and data services)

Yet more objects belong to system management, primarily concerned with the Data Link layer:

- ControlNet Object (station management interfaces to lower layers, required in all devices using Type 2 Data Link Layer),
- Keeper Object (holds and distributes attributes of all devices on the link, one required per link, with optional backup(s) – used in conjunction with Type 2 Data Link Layer),
- Scheduling Object (holds information on link schedules, one required in each connection originator – used in conjunction with Type 2 Data Link Layer),
- TCP/IP Interface Object (provides the mechanism to configure the TCP/IP interface of a device, required in all devices with a TCP/IP interface),
- Ethernet Link Object (maintains link-specific counters and status information for an Ethernet port, required in all devices with an Ethernet port).

NOTE These lower layers Management objects are described in IEC 61158-4.

The ASE and object interactions for a device using both Application and Data Link Type 2 protocols are illustrated in Figure 23:

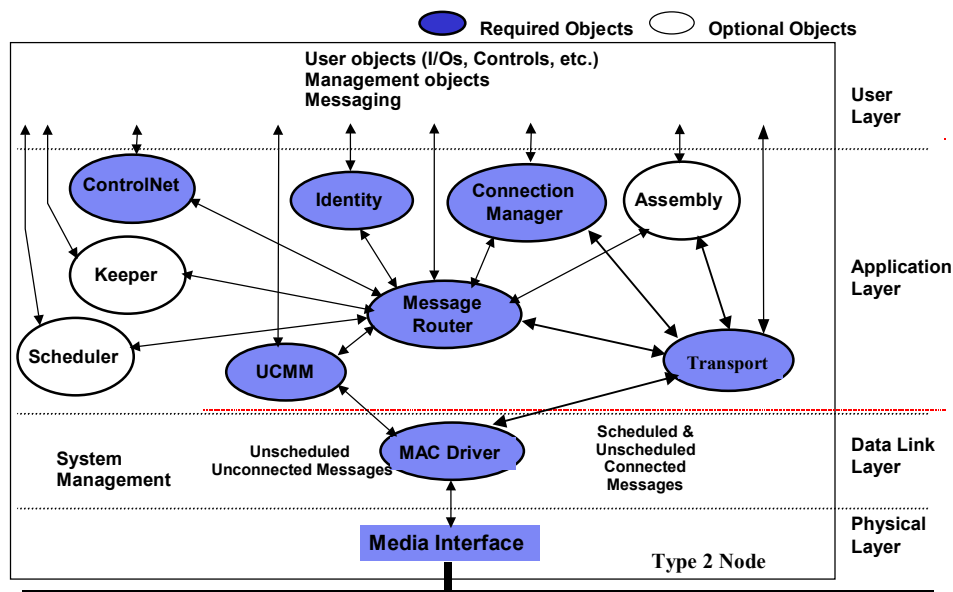


Figure 23 – Overview of ASE's and object classes

7.1.4 Naming and addressing

The information presented here provides a common basis for logically addressing separate physical components across the network. These addressing terms are used in the specifications. Figure 24 should be referenced in the following discussion.

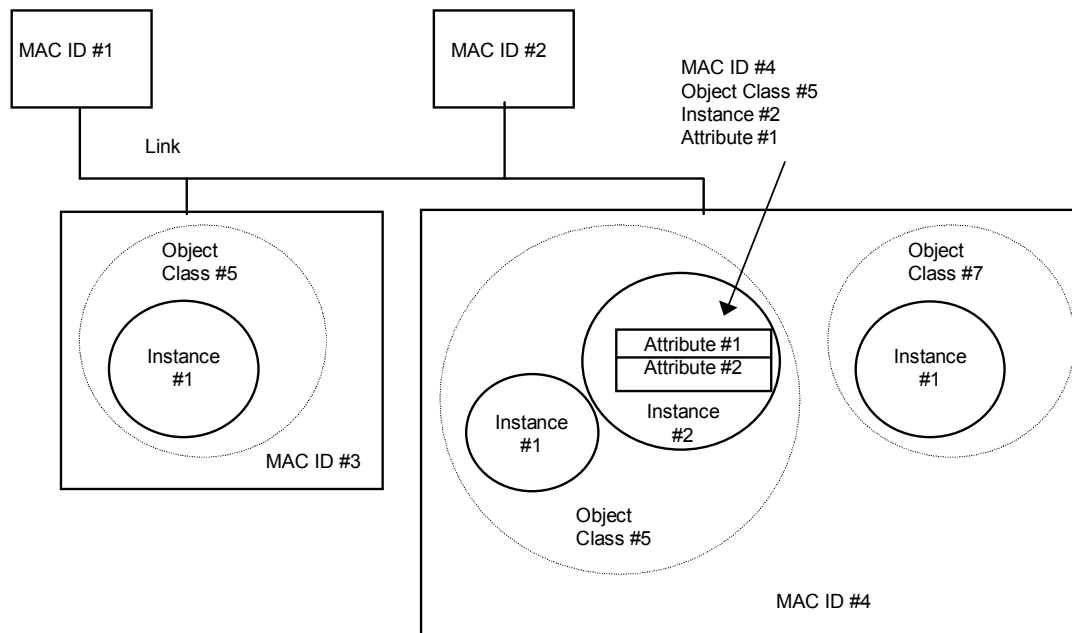


Figure 24 – Addressing format using MAC, class, instance and attribute IDs

The term “node” is used to refer to that portion of a device which contains a link interface and responds to a single MAC ID. The term “device” refers to a complete physical device connecting to the network. A device is able to contain multiple nodes.

The Class ID is a unique integer identification value assigned to each object class accessible from the network. The object class may be referenced with this Class ID. In all IEC 61158 specifications for Type 2, class code is synonymous with Class ID.

The Instance ID is an integer identification value assigned to an object instance when it is created that identifies it among all Instances of the same Class. This integer is unique within the <Node:Class> in which it resides.

The Attribute ID is an integer identification value that is unique among all other attributes of that object.

The address of attribute number 1 of instance 2 in class 5 in the node with a MAC ID of 4 is MAC ID#4: Object Class #5: Instance #2: Attribute #1 as shown in Figure 24. This nomenclature is referred to as Class/Instance/Attribute addressing.

Information on how to address an object through multiple links or using a name is provided in IEC 61158-6 (Path definition).

7.1.5 Data types

7.1.5.1 Data type general specification

The specification of a data type shall be comprised of the range of values that variables of the type may take on and the operations performed on these variables.

Data shall be made up of elementary (primitive) data types. These elementary data types shall be used to construct derived (constructed) data types.

NOTE 1 Elementary and derived data type specifications correspond to the notation of IEC 61131-3. In addition, since function blocks as defined in IEC 61131-3 have both an associated data structure and a set of defined operations, these elements are also specified below as additional data types.

The derived data types shall be :

- directly derived;
- enumerated;
- subrange;
- structured;
- array.

NOTE 2 These derived data types are defined in IEC 61131-3, 1.3 and 2.3.3. The means of specifying these data types and their default initial values is defined in IEC 61131-3, 2.3.3.1 and 2.3.3.2. The usage of variables of these data types is defined in IEC 61131-3, 2.3.3.3.

7.1.5.2 Supported elementary data types

The following basic data types defined in Clause 5 are supported:

BOOL
SINT
INT
DINT
LINT
USINT
UINT
UDINT
ULINT
REAL
LREAL
ITIME
TIME
FTIME
LTIME
DATE
TIME_OF_DAY or TOD
DATE_AND_TIME or DT
STRING
STRING2
STRINGN
SHORT_STRING
BYTE
WORD
DWORD
LWORD

7.1.5.3 Compliance with IEC 61131-3

7.1.5.3.1 Compliance statement

NOTE 1 This subclause provides information with respect to only the data types and associated operations defined in IEC 61158-5 for Type 2.

NOTE 2 IEC 61131-3, 1.5.1, defines the requirements which are met by programmable controller systems claiming compliance to IEC 61131-3. This provides the data type-related information to be included in the documentation of functional units which support the data types defined in IEC 61158-5. Subsets or extensions of this documentation are provided as appropriate to the specific compliant functional unit.

An implementation shall comply with the requirements of IEC 61131-3 for language features as specified in Table 82 through Table 84.

Table 82 – Common elements

IEC 61131-3 Table/feature	Feature description
10/1	BOOL data type
10/2	SINT data type
10/3	INT data type
10/4	DINT data type
10/5	LINT data type
10/6	USINT data type
10/7	UINT data type
10/8	UDINT data type
10/9	ULINT data type
10/10	REAL data type
10/11	LREAL data type
10/12	TIME data type
10/13	DATE data type
10/14	TIME_OF_DAY or TOD data type
10/15	DATE_AND_TIME or DT data type
10/16	STRING data type
10/17	BYTE data type
10/18	WORD data type
10/19	DWORD data type
10/20	LWORD data type
12/1	Directly derived data types
12/2	Enumerated data types
12/3	Subrange data types
12/4	Array data types
12/5	Structured data types
13	Standard default initial values
2.5.1.3	User-declared functions (no table entry)
22/1	Type conversions (see Table 84)
22/2	TRUNC function
22/3	BCD_TO_** functions
22/4	*_TO_BCD functions
23/1-11	Standard functions of one numeric variable: ABS, SQRT, LN, LOG, EXP, SIN, COS, TAN, ASIN, ACOS, ATAN
24/12n-18n	Standard named arithmetic functions: ADD, MUL, SUB, DIV, MOD, EXPT, MOVE
24/ 12s-15s, 17s,18s	Standard symbolic arithmetic functions: +, *, -, /, **, :=
25/1-4	Standard bit string functions: SHL, SHR, ROR, ROL
26/5s-8s	Standard named bitwise Boolean functions: AND, OR, XOR, NOT
26/5s-7s	Standard symbolic bitwise Boolean functions: &, >=1, =2k+1
27/1-4	Standard selection functions: SEL, MAX, MIN, LIMIT, MUX
28/5n-10n	Standard named comparison functions: GT, GE, EQ, LE, LT, NE
28/5s-10s	Standard symbolic comparison functions: >, >=, =, <=, <, <>
29/1-9	Standard character string functions: LEN, LEFT, RIGHT, MID, CONCAT, INSERT, DELETE, REPLACE, FIND
30/1-14	Standard functions of time data types: (note) ADD, SUB, MUL, DIV, CONCAT, DATE_AND_TIME_TO_TIME_OF_DAY, TIME_OF_DAY_TO_DATE_AND_TIME
31/1-4	Standard functions of enumerated data types: SEL, MUX, EQ, NE
32	Standard access mechanisms to function block I/O parameters
33/8a,8b,9a,9b	User-defined function blocks per IEC 61131-3, 2.5.2.2, with graphical or textual rising or falling edge input options
34/1-3	Standard bistable function blocks: SR, RS, SEMA
35/1,2	Standard edge detection function blocks: R_EDGE, F_EDGE
36/1-3	Standard counter function blocks: CTU, CTD, CTUD
37/1,2a,3a, 4	Standard timer function blocks: TP, TON, TOF, RTC
55/1-17	Standard operators: (), function evaluation, **, -, NOT, *, /, MOD, +, -, <, >, <=, >=, =, <>, &, AND, XOR, OR
NOTE IEC 61131-3, Table 30, limits the data types to which these operations apply.	

Table 83 – ST language elements

IEC 61131-3 Table/feature	Feature description
55/1-17	Standard operators: (), function evaluation, **, −, NOT, *, /, MOD, +, −, <, >, <=, >=, =, <>, &, AND, XOR, OR
56/2	Function block invocation and output usage

Table 84 – Type conversion operations

Operation	Result	Error conditions
ANY_BIT_TO_ANY_BIT	See note 4	None
ANY_BIT_TO_ANY_INT	OUT _{min} + Sbk2k (note 5)	Result > OUT _{max}
ANY_BIT_TO_BOOL	FALSE if IN = 0 TRUE otherwise	None
ANY_BIT_TO_STRING	See note 6	None
ANY_DATE_TO_STRING	See note 7	None
ANY_INT_TO_BOOL	FALSE if IN = 0 TRUE otherwise	None
ANY_NUM_TO_ANY_INT	IN (note 8)	(IN > OUT _{max}) or (IN < OUT _{min})
ANY_NUM_TO_ANY_REAL	IN	See note 9
ANY_NUM_TO_DATE	See note 10	
ANY_NUM_TO_STRING	See note 11	
ANY_NUM_TO_TIME	See note 12	
ANY_NUM_TO_TOD	See note 13	
ANY_REAL_TO_BOOL	FALSE if IN = 0,0 TRUE otherwise	None
BOOL_TO_ANY_BIT	0 if IN = FALSE	None
BOOL_TO_ANY_INT	1 if IN = TRUE	
BOOL_TO_ANY_REAL	0.0 if IN = FALSE 1.0 if IN = TRUE	None
BOOL_TO_STRING	'FALSE' if IN = FALSE 'TRUE' if IN = TRUE	None
DATE_TO_ANY_NUM	See note 14	Result > OUT _{max}
STRING_TO_ANY	Converted data	See note 15
TIME_TO_ANY_NUM	See note 16	Result > OUT _{max}
TOD_TO_ANY_NUM	See note 17	Result > OUT _{max}

NOTE 1 Use of the generic data types ANY_NUM, ANY_REAL, ANY_INT, and ANY_BIT defined in IEC 61131-3, 2.3.2, is intended to imply a family of conversions. For instance, the conversion BOOL_TO_ANY_REAL is intended to imply BOOL_TO_REAL and BOOL_TO_LREAL.

NOTE 2 IN refers to the value of the input variable of the type conversion function.

NOTE 3 OUT_{min} and OUT_{max} refer to the minimum and maximum values of the output data type of the conversion function, as defined in Clause 5.

NOTE 4 In conversions of bit string types, if the number of bits in the input variable IN is less than the number of the bits in the output variable OUT, the bits of the input is copied to the corresponding least significant bits of the result and the remainder of the result is zero-filled. If the number of bits of IN is greater than the number of bits of OUT, the least significant bits of IN is copied to the corresponding bits of the result. For instance:

BYTE_TO_WORD(16#FF) = 16#00FF
and
WORD_TO_BYTE(16#0FF0) = 16#F0

NOTE 5 Bit numbering in this expression is as specified in IEC 61158-6.

NOTE 6 The result of conversion of a bit string variable to type STRING consists of a string containing the base 16 literal representation of the variable value, as defined in IEC 61131-3, 2.2.1, in characters taken from the ISO 646 character set.

NOTE 7 The result of conversion of a date and/or time of day variable to type STRING consists of a string containing the literal representation of the variable value, as defined in IEC 61131-3, 2.2.1, in characters taken from the ISO 646 character set.

NOTE 8 Conversion of REAL and LREAL to integer types is accomplished by rounding as specified in IEEE Std 754, 5.4.

NOTE 9 Rounding errors may occur if the number of significant bits in the input variable is larger than the number of significant bits in the output floating-point representation. Also, range errors of

the type noted for ANY_NUM_TO_ANY_INT may occur in LREAL_TO_REAL.

NOTE 10 Conversion of a variable of a numeric type to DATE has the same result as conversion of the variable to UINT, with the result being interpreted as the number of days since 1972-01-01.

NOTE 11 Conversion of a variable of a numeric type to type STRING consists of a string containing the literal representation of the variable value, as defined in IEC 61131-3, 2.2.1, in characters taken from the ISO 646 character set.

NOTE 12 Conversion of a variable of a numeric type to TIME has the same result as conversion of the variable to DINT, with the result being interpreted as a duration in milliseconds.

NOTE 13 Conversion of a variable of a numeric type to TOD has the same result as conversion of the variable to UDINT, with the result being interpreted as a time since midnight in milliseconds.

NOTE 14 Conversion of a variable of type DATE to a numerical type is the same as the conversion of a variable of type UINT to the corresponding numerical type, with the result being the numerical equivalent of the days since 1972-01-01.

NOTE 15 It is an error if the STRING data to be converted is not in the format for external representation of the output data type as specified in IEC 61131-3, 2.2, or if the result of the conversion is outside the range {OUT_{min}..OUT_{max}}.

NOTE 16 Conversion of a variable of type TIME to a numerical type is the same as the conversion of a variable of type DINT to the corresponding numerical type, with the result being the numerical equivalent of the corresponding time interval expressed in milliseconds.

NOTE 17 Conversion of a variable of type TOD (TIME_OF_DAY) to a numerical type is the same as the conversion of a variable of type UDINT to the corresponding numerical type, with the result being the numerical equivalent of the time since midnight expressed in milliseconds.

7.1.5.3.2 Implementation dependant parameters

The values of implementation dependent parameters from IEC 61131-3, Table D.1, shall be as shown in Table 85.

NOTE Values of other implementation dependent parameters are defined in other standards or in the specifications of individual functional units as appropriate.

Table 85 – Values of implementation-dependent parameters

Subclause of IEC 61131-3	Parameter	Value
2.2.3.1	Range of values of duration	Same as LINT in microseconds
2.3.1	Range of values for variables of type TIME	Same as DINT in milliseconds
	Precision of representation of seconds in types TIME_OF_DAY and DATE_AND_TIME	1 millisecond
2.3.3.1	Maximum number of enumerated values	256
2.3.3.2	Default maximum length of STRING variables	256
	Maximum allowed length of STRING variables	65536
2.4.1.2	Maximum number of subscripts	8
	Maximum range of subscript values	0 – 255
	Maximum number of levels of structures	8
2.5.1.5	Maximum inputs of extensible functions	8
2.5.1.5.1	Effects of type conversions on accuracy	As defined in Table 84
2.5.1.5.2	Accuracy of functions of one variable	As defined in IEEE 754
2.5.2.3.3	PVmin, PVmax of counters	0, 65535

7.1.5.3.3 Language extensions

The extensions to IEC 61131-3 defined shall be as listed in Table 86. When these extensions are used in a particular device, the subclause references in this table shall be replaced by references to the descriptions of the corresponding extensions in the functional unit's documentation.

Table 86 – Extensions to IEC 61131-3

Subclause	Description
2.1.1	ITIME data type FTIME data type LTIME data type STRING2 data type STRINGN data type SHORT_STRING data type
2.1.4	Structured bit string types
2.2	Operations on STRING2 variables
2.2.4.1	Numbered bit string access
2.2.4.2	Structured bit string access

7.2 ASEs

7.2.1 Object management ASE

7.2.1.1 Overview

The FAL Object Management ASE is used to access all network accessible application elements or system management elements.

An access rule is associated with each attribute of an object class, which specifies how a requester can access this attribute. The definitions for access rules are as follows :

- Settable (Set) – The attribute may be accessed by one of the Set_Attribute services (if the behaviour of the device does not require a Set_Attribute service, then a device implementation of that object is not required to implement the attribute as settable);
- Settable attributes are also Gettable and may be accessed by Get_Attribute services;
- Gettable (Get) – The attribute may be accessed by one of the Get_Attribute services, but shall not be modified by any of the Set_Attribute services.

7.2.1.2 FAL management model class specification

7.2.1.2.1 General formal model

7.2.1.2.1.1 Class definition

The base AP class below specifies the common attributes and services defined for all other AP classes, including those defined by the user (application specific).

(*) in front of an attribute or a service means that this attribute/service is either mandatory or optional, based on some constraints defined in the attribute/service description.

FAL ASE: FAL Management ASE

CLASS: Base_Object

CLASS ID: <tbid>

PARENT CLASS: TOP

ACCESS ATTRIBUTES:

1 (m) Key Attribute: Object Instance number

2 (o) Key Attribute: Symbolic name

SYSTEM MANAGEMENT ATTRIBUTES (CLASS ATTRIBUTES) :

1 (*) Attribute: Revision

2 (o) Attribute: Max Instance

3 (o) Attribute: Number of Instances

4 (o) Attribute: Optional attribute list

4.1 (m) Attribute: Number of attributes

4.2 (m) Attribute: Optional attributes

- 5 (o) Attribute: Optional service list
- 5.1 (m) Attribute: Number services
- 5.2 (m) Attribute: Optional services
- 6 (o) Attribute: Maximum ID Number Class Attributes
- 7 (o) Attribute: Maximum ID Number Instance Attributes

OBJECT MANAGEMENT ATTRIBUTES (INSTANCE ATTRIBUTES) :

- 1 (o) Attribute: Attr1

SYSTEM MANAGEMENT SERVICES:

- 1 (o) Mgt Service: Get_Attribute_All
- 2 (o) Mgt Service: Set_Attribute_All
- 3 (o) Mgt Service: Get_Attribute_List
- 4 (o) Mgt Service: Set_Attribute_List
- 5 (o) Mgt Service: Reset
- 6 (o) Mgt Service: Start
- 7 (o) Mgt Service: Stop
- 8 (o) Mgt Service: Create
- 9 (o) Mgt Service: Delete
- 10 (o) Mgt Service: Get_Attribute_Single
- 11 (o) Mgt Service: Set_Attribute_Single
- 12 (o) Mgt Service: Find_Next_Object_Instance
- 13 (o) Mgt Service: Apply_Attributes
- 14 (o) Mgt Service: Save
- 15 (o) Mgt Service: Restore

OBJECT MANAGEMENT SERVICES:

- 1 (o) Ops Service: Get_Attribute_All
- 2 (o) Ops Service: Set_Attribute_All
- 3 (o) Ops Service: Get_Attribute_List
- 4 (o) Ops Service: Set_Attribute_List
- 5 (o) Ops Service: Reset
- 6 (o) Ops Service: Start
- 7 (o) Ops Service: Stop
- 8 (o) Ops Service: Create
- 9 (o) Ops Service: Delete
- 10 (o) Ops Service: Get_Attribute_Single
- 11 (o) Ops Service: Set_Attribute_Single
- 12 (o) Ops Service: NOP
- 13 (o) Ops Service: Apply_Attributes
- 14 (o) Ops Service: Save
- 15 (o) Ops Service: Restore

OBJECT SPECIFIC SERVICES:

- 1 (o) OpsService: Service1

7.2.1.2.1.2 Access attributes

These internal attributes uniquely identify an object instance within a device. The corresponding values are used as part of the path in service requests to specify the target object instance.

Object Instance number

Unique number associated with a given object instance. Instance number 0 (zero) means that access to the object class itself is requested.

Symbolic name

Optional name which may be associated with a given object instance.

7.2.1.2.1.3 System management attributes (class attributes)

Attributes at the class level. An attribute whose value is shared by all objects within the same class shall be referred to as a Class Attribute.

Seven predefined Class Attribute IDs shall be reserved for class object definition. The seven predefined/reserved class attributes shall have the definitions listed below. Because these attributes are reserved, class Attribute ID numbers 1 through 7 shall always be reserved. Therefore, if a class attribute is added to an object specification, it shall start with Attribute ID #8.

If a Class Attribute is optional, then a default value or a special case processing method shall be defined such that the Client (Requester) can process the error message that occurs when accessing those objects that choose not to implement the class attribute.

All the common class attributes have an access rule of Get only.

Revision

Revision of this object. The starting value assigned to this attribute is one (1). If updates that require an increase in this value are made, then the value of this attribute increases by 1.

If the value is 1, then this attribute shall be optional in implementation. If the value is greater than 1, then this attribute shall be mandatory. The Revision of an object shall specify the interface to that object, which shall encompass all of the items in the object specification, including services, attributes, connections and behaviour.

Max Instance

Maximum instance number of an object currently created in this class level in the device. The largest instance number of an object at this class hierarchy level created in the device.

Number of Instances

Number of object instances currently created at this class level in the device. The number of object instances at this class hierarchy level.

Optional attribute list

List of optional instance attributes utilised in an object class implementation. A list of attribute numbers specifying the optional attributes implemented in the device for this class.

Number of attributes

Number of attributes in the optional attribute list.

Optional attributes

List of optional attribute numbers.

Optional service list

List of optional services utilised in an object class implementation. A list of service codes specifying the optional services implemented in the device for this class.

If an optional service is implemented in a class, and the Optional Service List class attribute is also implemented in the class, then the service shall be included in the Optional Service List.

Number services

Number of services in the optional service list.

Optional services

List of optional service codes.

Maximum ID Number Class Attributes

The Attribute ID number of the last class attribute of the class definition implemented in the device.

NOTE This allows to simplify auto determination of class implementation by a remote terminal.

Maximum ID Number Instance Attributes

The Attribute ID number of the last instance attribute of the class definition implemented in the device.

NOTE This allows to simplify auto determination of class implementation by a remote terminal.

7.2.1.2.1.4 Object management attributes

Attributes at the instance level

An Instance Attribute shall be unique to an object instance and not shared by the object class. Instance ID = 0 shall be a special case. A service directed to Instance ID = 0 shall be applied to the class, not to a particular instance.

Each instance shall have a unique set of attributes. Instance attribute ID's may be numbered from 1 onwards without any correlation to the Attribute ID's of the class of which the object is an instance. There shall be no reserved Instance Attributes.

7.2.1.2.1.5 System management (class level) and object management (instance level) services

All the common services behave the same way, whether used as System Management or Object Management services.

Get_Attribute_All

The Get_Attribute_All service shall return the contents of all attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Set_Attribute_All

The Set_Attribute_All service shall modify the contents of all attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Get_Attribute_List

The Get_Attribute_List service shall return the contents of the selected gettable attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Set_Attribute_List

The Set_Attribute_List service shall update the contents of the selected attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Reset

The Reset service shall invoke the Reset service of the specified APO object class or instance.

Start

The Start service shall invoke the Start service of the specified APO object class or instance.

Stop

The Stop service shall invoke the Stop service of the specified APO object class or instance.

Create

The Create service shall result in the instantiation of a new object instance within the specified object class.

Delete

The Delete service shall delete an object instance of the specified object class.

Get_Attribute_Single

The Get_Attribute_Single service shall return the contents of the specified attribute of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Set_Attribute_Single

The Set_Attribute_Single service shall modify the contents of the specified attribute of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

Find_Next_Object_Instance

The Find_Next_Object_Instance service shall be supported at the APO object class level only. It shall cause the specified APO object class to search for and return a list of Instance IDs associated with existing object instances. Existing objects are those that are currently accessible from the link.

NOP

The NOP service shall cause the receiving APO object instance to return a *No Operation* response, without carrying out any other internal action.

Apply_Attributes

The Apply_Attributes service shall cause attribute values whose use is pending to become actively used in the specified APO object class or instance.

Save

The Save service shall copy the contents of an APO object class or instance attributes to a location accessible by the Restore service.

Restore

The Restore service shall restore the contents of an APO object class or instance attributes from a storage location accessible by the Save service.

7.2.1.2.1.6 Object specific services

Object-specific services shall be unique services supported only by the class of objects in which they are defined, whereas common services may be used in many objects. Object-specific service codes shall be unique only within the class in which they are defined.

Object-specific services sent to the Class Level of an object shall be called Object-specific System Management (Class Level) Services. Object-specific services sent to the Instance Level of an object shall be called Object-specific Object Management (Instance Level) Services.

7.2.1.2.2 Identity formal model

7.2.1.2.2.1 Class definition

The Identity ASE shall provide identification and general information about the device. Instance one of the Identity Object shall be present in all devices.

The first instance shall identify the whole device. It shall be used for electronic keying and by applications wishing to determine what nodes are on the network. Other instances are optional. They may be provided by a device to give additional information about a device and its subsystems.

(*) in front of an attribute or a service means that this attribute/service is either mandatory or optional, based on some constraints defined in the attribute/service description.

FAL ASE:	FAL Management ASE
CLASS:	Identity_Object
CLASS ID:	<tbid>
PARENT CLASS:	Base_Object
ACCESS ATTRIBUTES:	
1	(m) Key Attribute: Object Instance number
2	(o) Key Attribute: Symbolic name
SYSTEM MANAGEMENT ATTRIBUTES (CLASS ATTRIBUTES) :	
1	(o) Attribute: Revision = 1
2	(*) Attribute: Max Instance
6	(o) Attribute: Maximum ID Number Class Attributes
7	(o) Attribute: Maximum ID Number Instance Attributes
OBJECT MANAGEMENT ATTRIBUTES (INSTANCE ATTRIBUTES) :	
1	(m) Attribute: Manufacturer ID
2	(m) Attribute: Device Type
3	(m) Attribute: Product Code
4	(m) Attribute: Revision
4.1	(m) Attribute: Major Revision
4.2	(m) Attribute: Minor Revision
5	(m) Attribute: Status
5.1	(m) Attribute: Owned
5.2	(m) Attribute: Configured
5.3	(m) Attribute: Additional Status
5.4	(m) Attribute: Minor Recoverable Fault
5.5	(m) Attribute: Minor Unrecoverable Fault
5.6	(m) Attribute: Major Recoverable Fault
5.7	(m) Attribute: Major Unrecoverable Fault
6	(m) Attribute: Serial Number
7	(m) Attribute: Product Name
SYSTEM MANAGEMENT SERVICES:	
1	(o) Mgt Service: Get_Attribute_All
5	(o) Mgt Service: Reset
10	(o) Mgt Service: Get_Attribute_Single
12	(o) Mgt Service: Find_Next_Object_Instance
OBJECT MANAGEMENT SERVICES:	
1	(m) Ops Service: Get_Attribute_All
5	(m) Ops Service: Reset
10	(o) Ops Service: Get_Attribute_Single

7.2.1.2.2.2 System management attributes (class attributes)

Max Instance

If multiple subcomponents can be identified using instances of the Identity Object, then this attribute shall be **mandatory**, else it shall be **optional**. The numerically lowest available integer shall be assigned as the Instance Identifier of a newly created Identity Object Instance.

7.2.1.2.2.3 Object management attributes

The following instance attributes shall be used to identify with certainty the appropriate device targeted by a connection originator :

1. Manufacturer;
2. Device Type;
3. Product Code;
4. Revision.

This collection of attributes, when kept by the connection originator, shall be referred to as a device's "electronic key".

Manufacturer ID

Identification of each manufacturer by number. Manufacturer IDs uniquely identify a particular manufacturer. The value zero shall not be used.

This instance attribute has an access rule of Get only.

Device Type

Indication of general type of product. In order to allow interoperability and interchangeability, a Device Type shall be used to identify similar devices which exhibit the same behaviour, produce and/or consume the same set of data, and contain the same set of configurable attributes. The formal definition of this information is known as a Device Profile : all devices with the same Device Type number shall meet the minimum requirements and implement the common options specified in the Device Profile for that device type. A listing of the Device Type ranges can be found in IEC 61158-6.

This instance attribute has an access rule of Get only.

Product Code

Identification of a particular product of an individual manufacturer. The manufacturer assigned Product Code shall identify a particular product within a device type. Each manufacturer shall assign this code to each of its products. The Product Code typically maps to one or more catalogue/model numbers. Products shall have different codes if their configuration and/or runtime options are different. Such devices present a different logical view to the network. The value zero is not valid.

This instance attribute has an access rule of Get only.

Revision

Revision of the item that this instance of the Identity Object represents. The Revision attribute, which consists of Major and Minor Revisions, shall identify the Revision of the item the Identity Object is representing.

The value zero shall not be valid for either the Major and Minor Revision fields of a compliant product. A damaged product may return zero to indicate unknown revision if its storage of revision become corrupt.

NOTE 1 The Major and Minor Revision are typically displayed as "major.minor", minor revisions being displayed as three digits with leading zeros as necessary.

This instance attribute has an access rule of Get only.

Major Revision

The major revision shall be incremented by the manufacturer when there is a significant change to the 'fit, form, or function' of the product. Any changes that effect the configuration choices available to the user require incrementing the major revision.

Minor Revision

The minor revision may be used to identify changes in a product that do not effect user configuration choices, e.g. bug fixes, hardware component change, labelling change, etc. All firmware changes shall, at a minimum, update the minor revision.

Status

Summary status of device. This attribute shall represent the current status of the entire device. Its value changes as the state of the device changes. The detailed format of this Status attribute is described in IEC 61158-6.

NOTE 2 The events that constitute a fault (recoverable or unrecoverable) shall be determined by the device implementers.

This instance attribute has an access rule of Get only.

Owned

A (TRUE) shall indicate the device (or an object within the device) has an owner.

Configured

A (TRUE) shall indicate the application of the device has been configured to do something different than the "out-of-box" default. This shall not include configuration of the communications.

Additional Status

This attribute is used to provide more detailed information about device status, and can have one of the following values :

- Self-Testing (power-up) or State Unknown
- Firmware Update in progress
- Communication Fault (lost connection)
- Unkeyed, Awaiting Connection
- Non-Volatile Configuration Bad
- Major Fault
- Connected, Active
- Idle (program mode type)

Minor Recoverable Fault

A (TRUE) shall indicate the device detected a problem with itself, which is thought to be recoverable. The problem shall not cause the device to go into one of the faulted states.

NOTE 3 For example, an analogue input device is sensing an input that exceeds the configured maximum input value.

Minor Unrecoverable Fault

A (TRUE) shall indicate the device detected a problem with itself, which is thought to be unrecoverable. The problem shall not cause the device to go into one of the faulted states.

NOTE 4 For example, the device's battery backed RAM requires a battery replacement. The device shall continue to function properly until the first time power is cycled.

Major Recoverable Fault

A (TRUE) shall indicate the device detected a problem with itself, which caused the device to go into the “Major Recoverable Fault” state.

NOTE 5 For example, the device's configuration is incorrect or incomplete.

Major Unrecoverable Fault

A (TRUE) indicates the device detected a problem with itself, which caused the device to go into the “Major Unrecoverable Fault” state.

A device may not be able to communicate in the Major Unrecoverable Fault state. Therefore, it might not be able to report a Major Unrecoverable Fault. It shall not process a Reset service. The only exit from a Major Unrecoverable Fault shall be to cycle the device power.

NOTE 6 For example, the device failed its ROM checksum process.

Serial Number

Serial number of device. This attribute provides a number used in conjunction with the Manufacturer ID to form a unique identifier for each device. Each manufacturer shall be responsible for guaranteeing the uniqueness of the serial number across all of its devices.

This instance attribute has an access rule of Get only.

Product Name

Human readable identification. This text string may represent a short description of the product/product family represented by the "Product Code" attribute. The same product code may have a variety of product name strings.

This instance attribute has an access rule of Get only.

7.2.1.2.2.4 System management (class level) and object management (instance level) services

Object-specific details of the common services are provided below for the Identity object.

Reset

The Object Specific Data of the request primitive will contain a dedicated parameter to specify the type of Reset requested by the user. Its detailed format and corresponding Reset options are specified in IEC 61158-6.

When the Identity ASE (Object) receives a Reset request, it shall :

1. determine if it can provide the type of reset requested;
2. respond to the request;
3. attempt to perform the type of reset requested.

The Reset service shall not be processed when the device is in the Major Unrecoverable Fault state.

7.2.1.2.3 Assembly formal model**7.2.1.2.3.1 Class definition**

Each piece of data (whether real time or configuration data) communicated by a device may be represented by an attribute of one of the device internal objects. Communicating multiple pieces of data (attributes) across a single connection may require that the attributes be grouped or assembled together into a single block for optimisation purpose. Instances of the

Assembly object class shall perform this grouping. Individual components are referenced in the definitions below as "members" of the Assembly.

An Assembly object represents a buffer that binds attributes of multiple objects, allowing data to or from each object to be sent or received over a single connection. Assembly objects are used to produce and/or consume data to/from the network. An instance of the Assembly object can both produce and consume data from the network if designed to do so.

Assembly ASEs (objects) shall be either dynamic or static.

- Dynamic assemblies shall use assemblies member lists created and managed by the user. The member list may be altered by adding or deleting members;
- Static assemblies shall use member lists defined by the device profile or by the product implementers. The Instance number, number of members, and member list shall be fixed.

NOTE Static assemblies can usually be implemented entirely in ROM.

Instances of the Assembly object shall be divided into ranges specified as part of the device profiles.

The behaviour of the Assembly object shall differ by the type of Assembly. A Dynamic Assembly member list shall be created and managed by the user of the device. The manager of the Dynamic Assembly shall specify and maintain the member list. A Static Assembly member list shall be defined by the device manufacturer. It shall not be modified.

To provide for the ability to create and delete objects, as well as change member lists, Dynamic Assemblies shall support additional services which are not supported by Static Assemblies.

(*) in front of an attribute or a service means that this attribute/service is either mandatory or optional, based on some constraints defined in the attribute/service description.

FAL ASE:	FAL Management ASE
CLASS:	Assembly_Object
CLASS ID:	<tbid>
PARENT CLASS:	Base_Object
ACCESS ATTRIBUTES:	
1	(m) Key Attribute: Object Instance number
2	(o) Key Attribute: Symbolic name
3	(m) Attribute: Assembly type (STATIC, DYNAMIC)
SYSTEM MANAGEMENT ATTRIBUTES (CLASS ATTRIBUTES) :	
1	(m) Attribute: Revision = 2
2	(o) Attribute: Max Instance
OBJECT MANAGEMENT ATTRIBUTES (INSTANCE ATTRIBUTES) :	
1	(*) Attribute: Number of Members in List
2	(*) Attribute: Member List
2.1	(m) Attribute: Member Data Size
2.2	(m) Attribute: Member Path Size
2.3	(m) Attribute: Member Path
3	(m) Attribute: Data
4	(o) Attribute: Size
SYSTEM MANAGEMENT SERVICES:	
8	(c) Constraint: Assembly type = DYNAMIC
8.1	(m) Mgt Service: Create
9	(c) Constraint: Assembly type = DYNAMIC
9.1	(o) Mgt Service: Delete

10 (m) Mgt Service: Get_Attribute_Single

OBJECT MANAGEMENT SERVICES:

9 (c) Constraint: Assembly type = DYNAMIC

9.1 (m) Ops Service: Delete

10 (m) Ops Service: Get_Attribute_Single

11 (*) Ops Service: Set_Attribute_Single

7.2.1.2.3.2 System management attributes (class attributes)

No specific requirement for this object.

7.2.1.2.3.3 Object management attributes

Access rules (Get/Set) for the instance attributes are further limited by the operating state of the Assembly object. These additional constraints are detailed in the State Machine subclause.

Number of Members in List

This instance attribute is **optional** for a Static Assembly, **mandatory** for a Dynamic Assembly.

Number of members in "Member List" attribute below.

This instance attribute has an access rule of Get only.

Member List

This instance attribute is **optional** for a Static Assembly, **mandatory** for a Dynamic Assembly.

The member list is an array of each member size and origin (internal path). Each member in the Member List shall contain the entries defined below.

This instance attribute has an access rule of Get only for a Static Assembly, of Set for a Dynamic Assembly.

Member Data Size

Size of member data (in bits).

Member Path Size

Size of Member Path (in bytes). If no path is specified, then a value of zero should be used.

Member Path

Internal path to/from data for this member (packed format). See IEC 61158-6 for the format of packed path.

The following rules shall apply to the "Member List" attribute.

When an empty path (Member Path Size = 0) is used in an Assembly member list, the Assembly shall insert/discard the number of bits as specified in the Member Data Size sub-attribute field when producing/consuming. The Assembly object shall insert if it is producing and discards if it is consuming. The Assembly shall use a value of zero for all produced data which has been inserted.

NOTE 1 The use of an empty consumed path allows, for example, data destined for multiple nodes to be sent in a single message since each node can be configured to discard data in the message not intended for it.

The empty path shall be supported for all dynamic assemblies. Static assemblies may also contain empty paths.

No checking shall be done by the Assembly Object at any time to verify that the size of the member data is correct for the given member path. It shall be the responsibility of the Assembly member to properly handle too much or too little data. The Assembly shall be required to deliver the configured number of bits to the member.

No padding shall be done by the Assembly Object itself to align data from each Assembly member on a byte, word, or other boundary. Empty paths may be used to provide padding if desired.

Data

All of the member data packed into one array.

NOTE 2 This data may contain many different data types. For efficiency it is best to keep this data word aligned by packing it on word boundaries and adding padding as needed. This can be accomplished by using "empty paths" (Member Path Size = 0).

This instance attribute has an access rule of Set.

Size

Number of bytes in "Data" attribute.

This instance attribute has an access rule of Get only.

7.2.1.2.3.4 System management (class level) and object management (instance level) services

Delete

At the instance level (object management), the Delete service shall delete the specified Assembly object instance and releases all associated resources. At the class level (system management), this service deletes all existing Assembly instances.

Set_Attribute_Single

The Set_Attribute_Single service shall modify the contents of the specified attribute of the specified Assembly object instance. This service is **optional** for a Static Assembly, **mandatory** for a Dynamic Assembly. This service may be used to add or remove a member to/from the Assembly Member List of a Dynamic Assembly.

7.2.1.2.3.5 Assembly state machines

Actual usage of the supported services (e.g. Get_Attribute_Single and Set_Attribute_Single) is limited by the operating state of the Assembly object. These constraints are detailed in the Assembly State Machines below.

Static Assembly State Machine

Figure 25 and Table 87 illustrate the behaviour of Static Assembly objects.

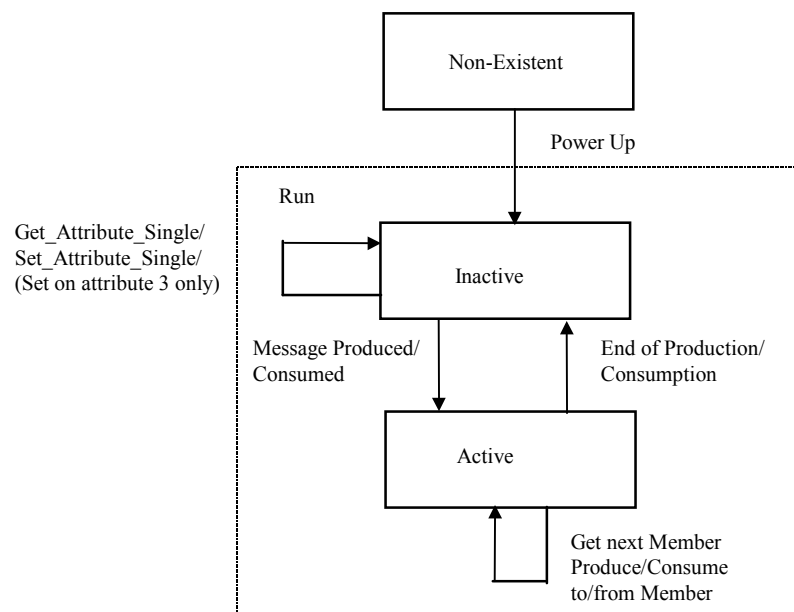


Figure 25 – Static Assembly state transition diagram

Table 87 – Static Assembly state event matrix

Event	Static Assembly Object state		
	Non-existent	Inactive	Active
Power Up	Transition to Inactive	Not applicable	Not applicable
Get_Attribute_Single	Error : Object does not exist.	Validate/service the request. Return response	Error : Object is not able to perform service in its current mode/state
Set_Attribute_Single	Error : Object does not exist.	Validate/service the request. Return response	Error : Object is not able to perform service in its current mode/state
Message produced/ consumed	Error : Object does not exist.	Transition to Active. Begin producing/consuming from/to each member in list.	Error : Object is not able to perform service in its current mode/state
End of production/ consumption	Error : Object does not exist.	Error : Object is not able to perform service in its current mode/state	Transition to Inactive

For all attributes of a Static Assembly object, the Get_Attribute_Single and Set_Attribute_Single services (when supported), are only available in the Inactive state.

Dynamic Assembly State Machine

Figure 26 and Table 88 illustrate the behaviour of Dynamicic Assembly objects.

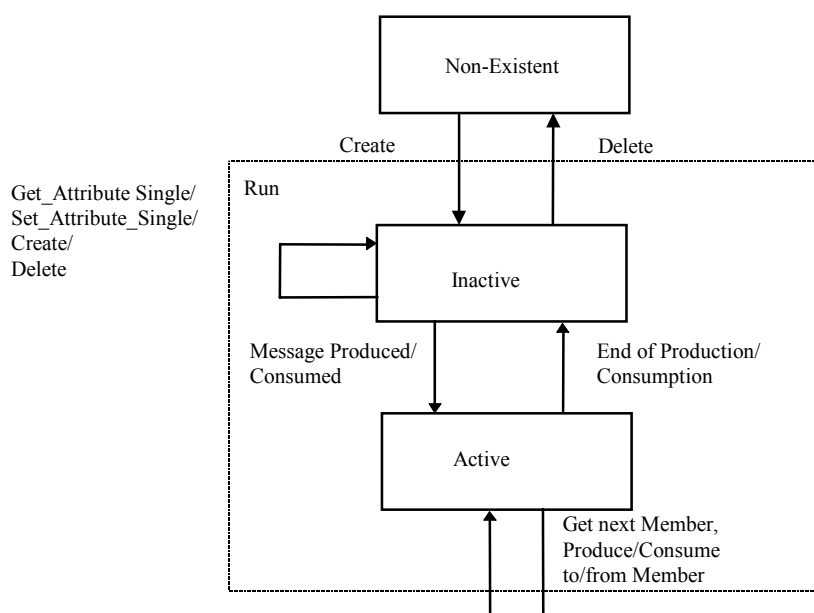


Figure 26 – Dynamic Assembly state transition diagram

Table 88 – Dynamic Assembly state event matrix

Event	Dynamic Assembly Object state		
	Non-existent	Inactive	Active
Create	Class instantiates an Assembly Object. Transition to Inactive	Not applicable	Not applicable
Delete	Error : Object does not exist.	Release all associated resources. Transition to Non-Existent	Error : Object is not able to perform service in its current mode/state
Get_Attribute_Single	Error : Object does not exist.	Validate/service the request. Return response	Error : Object is not able to perform service in its current mode/state
Set_Attribute_Single	Error : Object does not exist.	Validate/service the request. Return response	Error : Object is not able to perform service in its current mode/state
Message produced/ consumed	Error : Object does not exist.	Transition to Active. Begin producing/consuming from/to each member in list.	Error : Object is not able to perform service in its current mode/state
End of production/ consumption	Error : Object does not exist.	Error : Object is not able to perform service in its current mode/state	Transition to Inactive

For all attributes of a Dynamic Assembly object, the Get_Attribute_Single and Set_Attribute_Single services (when supported), are only available in the Inactive state.

7.2.1.2.3.6 Specific requirements for a connection to the Assembly object

The Assembly object shall be unique among all objects in that its connection points and instances are the same. For example, connection point 4 of the Assembly object shall be the same as instance 4 of the Assembly object. Accessing data with a path specifying "class=Assembly, instance=VV, attribute=3" shall return the same data as a path specifying "class= Assembly, connection point=VV, attribute=3".

When the Assembly object is the target of a connection, one instance plus two connection points shall be specified by the connection path in the Forward_Open service.

The format of the connection path shall be "class=Assembly, instance=XX, connection point=YY, connection point=ZZ" where XX is the instance, and YY/ZZ are the connection points. Any data segment in the path shall be used to set the data attribute of instance XX. Connection point (instance) YY shall be the consumer ($O \Rightarrow T$) for the connection. Likewise, connection point (instance) ZZ shall be the producer ($T \Rightarrow O$).

Using this format, three instances shall be specified for a connection to the Assembly object:

- configuration,
- producer,
- consumer.

For example, the following two connection paths specify the same producer instance 8. These two connections can be made simultaneously provided that the connection requested specifies multipoint.

- class=Assembly, instance= 5, connection point=4, connection point=8
- class=Assembly, instance= 3, connection point=2, connection point=8.

7.2.1.2.4 Message router formal model

7.2.1.2.4.1 Class definition

The Message Router ASE (Object) shall provide a formal messaging connection point through which a client may address a service to any object class or instance residing in the physical device. Every node shall contain instance 1 of the Message Router object.

The Message Router ASE (Object) shall receive all incoming messages (service indications addressed to application or user objects within the device). The source of these messages may be either the Unconnected Message Manager (UCMM), or an active connection to the Message Router object.

The first part of the message (path information) shall then be parsed to determine the target object class. Interpretation of the class instance shall be performed on every service received by the Message Router. Any class instance that cannot be interpreted by the implementation of a Message Router in a device shall cause an error response to be returned with the "Object Not Found" status.

The service indication shall then be routed to the target object for actual processing. If the service is directed to the Message Router itself (system and object management services addressing the Message Router), then the Message Router shall process the service request and respond accordingly.

When the service response has been received from the target object (or generated by the Message Router itself), then it shall be routed back to the source of the service request. All connected service responses shall be routed back across the connection from which the service request was received. All unconnected service responses (UCMM) shall be returned via the same UCMM transaction that provided the request.

NOTE Practically, the Message Router acts as an internal switching board, which retains all the necessary routing information for messages. Therefore, the target objects do not need to be aware of this information (matching of service indications and responses is achieved via a Local ID provided by the Message Router).

FAL ASE:	FAL Management ASE
CLASS:	Message_Router_Object
CLASS ID:	<tbd>
PARENT CLASS:	Base_Object

ACCESS ATTRIBUTES:

- 1 (m) Key Attribute: Object Instance number
- 2 (o) Key Attribute: Symbolic name

SYSTEM MANAGEMENT ATTRIBUTES (CLASS ATTRIBUTES) :

- 1 (o) Attribute: Revision = 1
- 4 (o) Attribute: Optional attribute list
- 4.1 (m) Attribute: Number of attributes
- 4.2 (m) Attribute: Optional attributes
- 5 (o) Attribute: Optional service list
- 5.1 (m) Attribute: Number services
- 5.2 (m) Attribute: Optional services
- 6 (o) Attribute: Maximum ID Number Class Attributes
- 7 (o) Attribute: Maximum ID Number Instance Attributes

OBJECT MANAGEMENT ATTRIBUTES (INSTANCE ATTRIBUTES) :

- 1 (o) Attribute: Object_List
- 1.1 (m) Attribute: Number
- 1.2 (m) Attribute: Classes
- 2 (o) Attribute: Number available

SYSTEM MANAGEMENT SERVICES:

- 1 (o) Mgt Service: Get_Attribute_All
- 3 (o) Mgt Service: Get_Attribute_List
- 10 (*) Mgt Service: Get_Attribute_Single

OBJECT MANAGEMENT SERVICES:

- 1 (o) Ops Service: Get_Attribute_All
- 3 (o) Ops Service: Get_Attribute_List
- 10 (*) Ops Service: Get_Attribute_Single

7.2.1.2.4.2 System management attributes (class attributes)

No specific requirement for this object.

7.2.1.2.4.3 Object management attributes

Object_List

List of object class codes supported by the device via the Message Router.

This instance attribute has an access rule of Get only.

Number

Number of supported classes in the Classes attribute below.

Classes

List of class codes supported by the device

Number available

Maximum number of connections supported by the Message Router

This instance attribute has an access rule of Get only.

7.2.1.2.4.4 System management (class level) and object management (instance level) services

Get_Attribute_Single

This service is **mandatory** if any attributes are implemented, else it is **optional**.

7.2.1.2.4.5 Specific requirements of the Message Router object

The Message Router object shall have one state, the Run state. It shall always be running (after power-up) and shall be fixed by device design. More information concerning the Message Router object can be found in IEC 61158-6 (definition of Path to access object element within a device).

The Message Router object shall support 1 or more connections to them. A Forward_Open with the parameters listed in Table 89 shall also be supported.

Table 89 – Message Router object Forward_Open parameters

Parameter	Parameter Value
O⇒T priority	low
O⇒T connection type	point-to-point
O⇒T fixed/variable	variable
O⇒T size	up to 504 bytes
T⇒O priority	low
T⇒O connection type	point-to-point
T⇒O fixed/variable	variable
T⇒O size	up to 504 bytes
Client/server	server
Trigger mode	ignore
Transport class	class 3 (verified)

7.2.1.3 FAL management model ASE service specification

7.2.1.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Get_Attribute_All
 Set_Attribute_All
 Get_Attribute_List
 Set_Attribute_List
 Reset
 Start
 Stop
 Create
 Delete
 Get_Attribute_Single
 Set_Attribute_Single
 Find_Next_Object_Instance
 NOP

Apply_Attributes

Save

Restore

7.2.1.3.2 FAL service common parameters

The following service parameters are common to all FAL services.

AREP

This parameter contains sufficient information to locally identify the entity to be used to convey the service : this entity could be local or the UCMM or a transport connection (AR). This parameter may use a dedicated identifier associated with a local entity, the identifier of a UCMM transaction record previously created, or the transport identifier returned by the connection establishment process and associated with the selected AR. The confirmation primitive should use the same value as the one sent in the corresponding request primitive.

Receiver/Server Local ID (id)

This parameter is locally generated by the Message Router ASE of the responding node, and identifies locally this invocation of the service. It is used to associate service responses with indications. Therefore, no two outstanding service invocations may be identified by the same ID (id) value.

Path

In the request, this parameter specifies the FAL APO or FAL APO element that is the destination of the service request. The path shall contain multiple segments which can indicate the network route to the node containing the FAL APO (in the case of multiple links), the identification of the APO element within the target node (Routing Path), and optional additional information for the target APO (Additional Path).

In the indication, this parameter shall contain only those segments beyond the logical class segment from the service request, i.e. the optional additional information for the target APO (AdditionalPath).

See IEC 61158-6 for more details on the Path structure and contents.

Port ID

This parameter shall indicate on which port of the device the service indication arrived.

Service status

This parameter provides information on the result of service execution. It is returned in all confirmed service response primitives (+ and -). It is composed of the following elements.

Status Code (mandatory)

This parameter shall indicate whether the service was successfully processed or not. If an error occurred, it indicates the type of error. For some errors, further identification of the error may be provided in the Extended Status parameter below. Available status codes are listed in 7.2.1.3.3.

Extended Status (conditional)

This conditional parameter shall further identify the error encountered when processing the request specific to the object being accessed. Actual usage of this Extended Status is dependant on the object class or service, but its format is fixed for each status code. When used, the values submitted in the response primitive are delivered unchanged in the confirmation primitive. Available extended status codes are described in 7.2.1.3.3.

7.2.1.3.3 FAL service status codes

Table 90 specifies the range of possible status codes, and indicates whether an associated Extended Status is defined.

Table 90 – Status codes

Name	Description and meaning of status code	Extended Status
Success	Service was successfully performed by the object specified.	None
Connection failure	A connection related service failed along the connection path.	Conditional
Resource unavailable	Resources needed for the object to perform the requested behaviour were unavailable. Further object specific information shall be supplied in the object specific status field of the response.	None
Invalid value in object specific data parameter of a service request	A portion of the data supplied as a object specific data parameter of a service was invalid. The verification of the data shall be specified in the object specification of the object reporting the error.	None
Path segment error	The path segment identifier or the segment syntax was not understood by the processing node. The word offset to the first segment of the path that is not understood shall be supplied in the first word of the object specific status field of the response. The offset shall be zero based and calculated from the first word following the path size in the message. Path processing shall stop when a path segment error is encountered.	Conditional
Path destination unknown	The path is referencing an object class, instance or structure element that is not known or is not contained in the processing node. The word offset to the first segment component that references something that is unknown or not present in the processing node shall be supplied in the first word of the object specific status field of the response. The offset shall be zero based and calculated from the first word following the path size in the message. Path processing shall stop when a path destination unknown error is encountered.	Conditional
Partial transfer	Only part of the expected data was transferred.	None
Connection lost	The messaging connection was lost.	None
Unimplemented service	The service requested was not implemented or defined for this class or object instance.	None
Invalid attribute value	The value of an attribute of the object or class is invalid. The object specific status shall report the attribute number and the status code of the first attribute refusing data.	Conditional
Attribute list error	An attribute in the Get_Attribute_List or Set_Attribute_List response has a non zero status.	None
Already in requested mode/state	The object is already in the mode/state being requested by the service. The object specific status shall report the current status of the object.	Conditional
Object is not able to perform service in its current mode/state	The object can not perform the requested service in its current mode/state. The object specific status shall report the current status of the object.	Conditional
Object already exists	The requested instance of object to be created already exists.	None
Attribute value not settable	The object attribute is not a settable attribute. The object specific status shall report the number of the attribute refusing data.	None
Access permission does not allow service	The access permissions do not allow the object to perform the service. The access permissions available to the object shall be reported in the extended status. See Table 91 for details concerning extended status.	None

Name	Description and meaning of status code	Extended Status
Device's mode/state does not allow object to perform service	The device containing the object does not allow the object to perform the service in the current mode/state of the device. The object specific status shall report the device current status, e.g. a controller may have a key switch which when set to the "hard run" state causes Service Requests to several different objects to fail (i.e. program edits) causing this status code to be returned.	Conditional
Response data too large	The data to be transmitted in the response buffer is larger than the allocated response buffer; therefore, no data was transferred.	None
Fragmentation of a primitive value	The service specified an operation that is going to fragment a primitive data value, i.e. half a REAL data type.	None
Not enough data	The service did not supply enough data to perform the specified operation.	None
Undefined attribute	The attribute specified is not defined for the class or object.	None
Too much data	The service supplied more data than was expected (depending on the service and the object, the service might still be processed).	None
Object does not exist	The object specified does not exist in the device.	None
Service fragmentation sequence not currently in progress	The fragmentation sequence for this service is not currently active for this data.	None
No stored attribute data	The attribute data of this object was not saved prior to the requested service.	None
Store operation failure	The attribute data of this object was not saved due to some failure during the attempt.	Conditional
Routing failure, request packet too large for network	The service request packet was too large for transmission on a network in the path to the destination. The routing device was forced to abort the service.	None
Routing failure, response packet too large for network	The service response packet was too large for transmission on a network in the path from the destination. The routing device was forced to abort the service.	None
Missing attribute list entry data	The service did not supply an attribute in a list of attributes that was needed by the service to perform the requested behaviour.	None
Invalid attribute value list	The service is returning the list of attributes supplied with status information for those attributes that were invalid.	Conditional
Embedded service error	An embedded service resulted in an error.	None
Connection Related Failure	A service failed because of an error condition related to the processing of a connection related service. This might occur during connected and unconnected messaging. The same extended status codes used for General Status Code 0x01 are returned for this error's extended status.	None
Write once value or medium already written	An attempt was made to write to a write once medium (e.g. WORM drive, PROM) that has already been written, or to modify a value that cannot be changed once established.	None
Invalid Response Received	An invalid response is received (e.g. response service code does not match the request service code, or response message is shorter than the minimum expected response size). This status code can serve for other causes of invalid replies.	None
Key Failure in path	The Key Segment which was included as the first segment in the path does not match the destination module. The object specific status shall indicate which part of the key check failed.	Conditional
Path Size Invalid	The size of the path which was sent with the Service Request is either not large enough to allow the Request to be routed to an object or too much routing data was included.	None
Unexpected attribute in list	An attempt was made to set an attribute that cannot be set at this time.	None
Invalid Member ID	The Member ID specified in the request does not exist in the specified Class/Instance/Attribute.	None
Member not settable	A request to modify a non-modifiable member was received.	None
Reserved object/class specific	This range of status codes has been reserved for use by object and class specific services.	Conditional

Valid Extended Status formats are specified in Table 91.

Table 91 – Format of extended status

Name	Format of extended status	Meaning of Extended Status
Connection failure	extended_error_code	Additional error code specific to the connection process. List of additional codes is provided in IEC 61158-6.
Path segment error	path_segment_offset	Word offset from the beginning of the path of the incorrect segment
Path destination unknown	path_segment_offset	Word offset from the beginning of the path the first segment that is referencing a non-existent object or class
Invalid attribute value	attribute_number attribute_status	Number of the attribute refusing the data value status code explaining reason for invalid attribute value
Already in requested mode/state	object_status	Status/state/mode information of the object
Object is not able to perform service in its current mode/state	object_status	Status/state/mode information of the object
Device's mode/state does not allow object to perform service	device_status	Status/state/mode information of the device
Store operation failure	storage_status	Status/state/mode information of the storage media
Invalid attribute value list	attribute_count status_of_attributes [] attribute_number attribute_status	Number of attribute values being returned List of attribute status Number of attribute Status of attribute
Key Failure in path	key_element	Code indicating which part of the key check failed

7.2.1.3.4 Get_Attribute_All**7.2.1.3.4.1 Service overview**

The Get_Attribute_All service shall return the contents of all attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.4.2 Service primitives

The service parameters for this service are shown in Table 92.

Table 92 – Get_Attribute_All service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Attribute Data			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request. There are no specific parameters for this service.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Attribute Data

This parameter returns a stream of information containing all of the attributes. APO classes which support this service shall define the format of this parameter. No fragmentation shall be allowed.

If supported, the structure of the Attribute Data information shall adhere to the Get_Attribute_All response structure as defined by the object class specification. The object class specification shall provide a detailed definition of the format of the data to be sent in the class and instance response messages.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Response data too large (no extended status)

7.2.1.3.4.3 Service procedure

No specific service procedure.

7.2.1.3.5 Set_Attribute_All**7.2.1.3.5.1 Service overview**

The Set_Attribute_All service shall modify the contents of the attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.5.2 Service primitives

The service parameters for this service are shown in Table 93.

Table 93 – Set_Attribute_All service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Attribute Data	M	M(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Attribute Data

This parameter specifies a stream of information containing all of the attributes. APO classes which support this service shall define the format of this parameter.

If supported, the structure of the Attribute Data information in the service request shall adhere to the Set_Attribute_All request structure as defined by the object class specification. The object class specification shall provide a detailed definition of the format of the data to be sent in the request message.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Invalid attribute value (mandatory extended status, see Table 91)
- Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.5.3 Service procedure

If the ability to modify an attribute changes based on the state of the object, the object specification shall specify when its attributes may be written.

Attributes shall be modified only if all attribute specific value verifications (e.g. range checks, etc.) are successful. If an error is detected, the Set_Attribute_All response shall returned an appropriate error response message. If all verification checks pass, then the attributes shall be modified and a Set_Attribute_All success response shall be returned.

7.2.1.3.6 Get_Attribute_List**7.2.1.3.6.1 Service overview**

The Get_Attribute_List service shall return the contents of the selected gettable attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.6.2 Service primitives

The service parameters for this service are shown in Table 94.

Table 94 – Get_Attribute_List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Attribute Count	M	M(=)		
Attribute List	M	M(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Attribute Count			M	M(=)
Attribute Data			M	M(=)
Attribute Identifier			M	M(=)
Attribute Status			M	M(=)
Attribute Value			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Attribute Count

This parameter indicates the number of attribute identifiers in the Attribute List parameter below.

Attribute List

This parameter contains the list of the top-level attribute identifiers for the attributes that are being requested from the specified class or object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Attribute Count

This parameter indicates the number of attributes actually returned in the Attribute Data parameter below. This count can be different if the requested data does not fit entirely in the response buffer (see Service Procedure below).

Attribute Data

This parameter returns a stream of information containing all or part of the requested attributes. Attributes shall be retrieved and packed in the sequence specified in the request.

Attribute Identifier

This parameter contains the attribute identifier corresponding to the value being returned in Attribute Value.

Attribute Status

This parameter indicates the individual status information for the requested attribute. It can either mirror one of the common service status codes, or be specific to this object/class attribute.

Attribute Value

This parameter returns a stream of information containing all data for the requested attribute. Individual attribute data shall fit into the response buffer in its entirety (without fragmentation of individual attributes). The service shall access as many attributes as can fit into the response buffer.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Attribute list error (no extended status)

7.2.1.3.6.3 Service procedure

The attributes shall be specified using a list of attribute identifiers, which shall be supplied as a service parameter. If there is not enough space for an attribute's data in the response buffer, service processing shall terminate and the Attribute Count parameter shall be set to the number of attributes actually packed in the buffer. The client application (the requester), shall verify the value of the Attribute Count parameter in the response.

NOTE If necessary, the client application can submit another Get_Attribute_List service request for the attribute data remaining from its original list.

7.2.1.3.7 Set_Attribute_List

7.2.1.3.7.1 Service overview

The Set_Attribute_List service shall update the contents of the selected attributes of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.7.2 Service primitives

The service parameters for this service are shown in Table 95.

Table 95 – Set_Attribute_List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Attribute Count	M	M(=)		
Attribute Data	M	M(=)		
Attribute Identifier	M	M(=)		
Attribute Value	M	M(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Attribute Count			M	M(=)
Attribute Status List			M	M(=)
Attribute Identifier			M	M(=)
Attribute Status			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Attribute Count

This parameter indicates the number of attributes to be updated, and listed in the Attribute Data parameter below.

Attribute Data

This parameter contains a stream of information containing the actual list of attributes to be updated and the corresponding update values.

Attribute Identifier

This parameter contains the attribute identifier corresponding to the attribute to be updated with the contents of Attribute Value. The client shall be unable to specify sub-elements of an attribute.

Attribute Value

This parameter contains a stream of information containing all data for the target attribute. The data for an individual attribute shall be placed into the request buffer in its entirety (without fragmentation).

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Attribute Count

This parameter indicates the number of attribute status returned in the Attribute Status List parameter below.

Attribute Status List

This parameter returns a stream of information containing status information for each attribute listed in the service request.

Attribute Identifier

This parameter contains the attribute identifier corresponding to the status being returned in Attribute Status.

Attribute Status

This parameter indicates the individual status information for the specified attribute. It can either mirror one of the common service status codes, or be specific to this object/class attribute.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Attribute list error (no extended status)
- Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

Missing attribute list entry data

7.2.1.3.7.3 Service procedure

The service shall be terminated if the response buffer is unable to accept the status of the next attribute.

Individual set operations shall not be interdependent (the request shall be treated as a series of independent set requests).

7.2.1.3.8 Reset

7.2.1.3.8.1 Service overview

The Reset service shall invoke the Reset service of the specified APO object class or instance.

NOTE Typically this would cause a transition to a default state or mode.

7.2.1.3.8.2 Service primitives

The service parameters for this service are shown in Table 96.

Table 96 – Reset service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Invalid value in object specific data parameter of a service request (no extended status)

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.8.3 Service procedure

If the execution of the Reset service request would place the object/device in a state that enables it to respond to the requester, then the response shall not be returned until the service has completed. If the execution of the Reset service would place the object/device in a state in which it is not able to respond, the object/device shall respond prior to executing the Reset service.

7.2.1.3.9 Start

7.2.1.3.9.1 Service overview

The Start service shall invoke the Start service of the specified APO object class or instance.

NOTE Typically this would place an object into a running state/mode.

7.2.1.3.9.2 Service primitives

The service parameters for this service are shown in Table 97.

Table 97 – Start service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Already in requested mode/state (mandatory extended status, see Table 91)

Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.9.3 Service procedure

If supported by an object class or instance, the effect of this service shall be as documented in that object specification.

7.2.1.3.10 Stop

7.2.1.3.10.1 Service overview

The Stop service shall invoke the Stop service of the specified APO object class or instance.

NOTE Typically this would place an object into a stopped or idle state/mode.

7.2.1.3.10.2 Service primitives

The service parameters for this service are shown in Table 98.

Table 98 – Stop service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Already in requested mode/state (mandatory extended status, see Table 91)

Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.10.3 Service procedure

The effect of this service shall be documented in the individual object specification.

7.2.1.3.11 Create

7.2.1.3.11.1 Service overview

The Create service shall result in the instantiation of a new object instance within the specified object class.

7.2.1.3.11.2 Service primitives

The service parameters for this service are shown in Table 99.

Table 99 – Create service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Instance ID			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Instance ID

This mandatory parameter indicates the integer value assigned to identify the newly created object instance.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Invalid attribute value (mandatory extended status, see Table 91)
- Already in requested mode/state (mandatory extended status, see Table 91)
- Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)
- Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)
- Missing attribute list entry data (no extended status)
- Invalid attribute value list (mandatory extended status, see Table 91)

7.2.1.3.11.3 Service procedure

If this service is supported, the object instance shall be created and initialised in accordance with the object specification.

Any error shall result in the object instance not being created.

7.2.1.3.12 Delete

7.2.1.3.12.1 Service overview

The Delete service shall delete an object instance of the specified object class.

7.2.1.3.12.2 Service primitives

The service parameters for this service are shown in Table 100.

Table 100 – Delete service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.12.3 Service procedure

All resources shall be deallocated and returned to the system.

7.2.1.3.13 Get_Attribute_Single

7.2.1.3.13.1 Service overview

The Get_Attribute_Single service shall return the contents of the specified attribute of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.13.2 Service primitives

The service parameters for this service are shown in Table 101.

Table 101 – Get_Attribute_Single service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Attribute Data			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request. There are no specific parameters for this service.

Path

This parameter shall include a path segment specifying the attribute number.

NOTE See IEC 61158-6 for a description of path segments.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Attribute Data

This parameter contains the requested attribute data.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Attribute value not settable (no extended status)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

Object does not exist (no extended status)

7.2.1.3.13.3 Service procedure

No specific service procedure.

7.2.1.3.14 Set_Attribute_Single**7.2.1.3.14.1 Service overview**

The Set_Attribute_Single service shall modify the contents of the specified attribute of the specified object class (APO System Management attributes) or instance (APO Object Management attributes).

7.2.1.3.14.2 Service primitives

The service parameters for this service are shown in Table 102.

Table 102 – Set_Attribute_Single service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Attribute Data	M	M(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request. There are no specific parameters for this service.

Path

This parameter shall include a path segment specifying the attribute number.

NOTE See IEC 61158-6 for a description of path segments.

Attribute Data

This parameter contains the value to which the specified attribute is to be modified. The attribute data shall be validated prior to the modification taking place.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Attribute value not settable (no extended status)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

Object does not exist (no extended status)

7.2.1.3.14.3 Service procedure

No specific service procedure.

7.2.1.3.15 Find_Next_Object_Instance**7.2.1.3.15.1 Service overview**

The Find_Next_Object_Instance service shall be supported at the APO object class level only. It shall cause the specified APO object class to search for and return a list of Instance IDs associated with existing object instances. Existing objects are those that are currently accessible from the link.

7.2.1.3.15.2 Service primitives

The service parameters for this service are shown in Table 103.

Table 103 – Find_Next_Object_Instance service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Receiver/Server Local ID	M	M(=)		
Path size	M	M(=)		
Path	M	M(=)		
Port ID		M		
Maximum Returned Values	M	M(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Number of List Members			M	M(=)
Instance ID List			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Path

This parameter shall include a path segment specifying an Instance ID, which will be used to determine the starting point for the search.

Maximum Returned Values

This parameter indicates the maximum number of Instance ID values to be returned in the response message.

Result(+)

This selection type parameter indicates that the service request succeeded.

Number of List Members

This parameter contains the actual number of Instance ID's returned in this response message. This number of Instance ID's shall be less than or equal to the maximum specified in the request.

Instance ID List

This parameter contains the list of returned Instance ID's.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Object is not able to perform service in its current mode/state

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.15.3 Service procedure

The object class shall utilise the value specified in the Instance ID of the request message to determine the starting point for the search as described below :

1. If the Instance ID in the request message is zero (0), then the class shall start with the numerically lowest Instance ID;
2. If the Instance ID in the request message is not zero (0), then the class shall start with the next Instance ID whose value is numerically greater than the specified Instance ID.
3. If the Instance ID in the request message is greater than or equal to the numerically highest Instance ID within the class, then the value zero (0) shall be returned as Instance ID.

The class shall return a list of Instance IDs associated with existing objects, beginning at the starting point and continuing in ascending Instance ID value order. It shall return Instance ID value zero (0) to indicate that the end of the list has been reached.

Figure 27 provides a general example of this service.

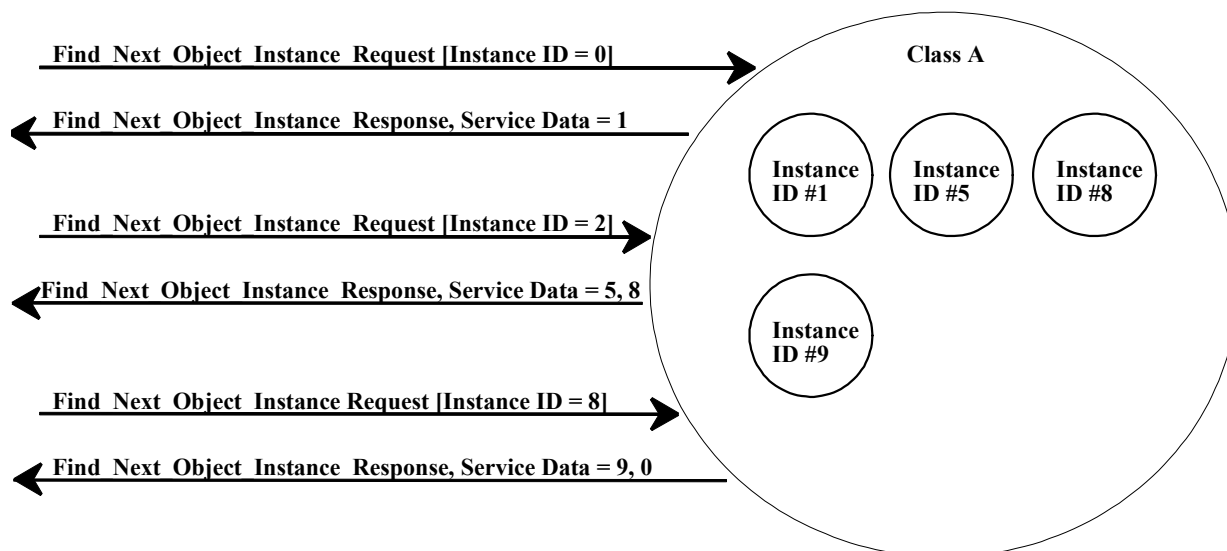


Figure 27 – Example of Find_Next_Object_Instance service

7.2.1.3.16 NOP

7.2.1.3.16.1 Service overview

The NOP service shall cause the receiving APO object instance to return a *No Operation* response. The receiving APO object instance shall not carry out any other internal action.

NOTE This service may be used to test whether or not a particular object is still present and responding without causing a state change.

7.2.1.3.16.2 Service primitives

The service parameters for this service are shown in Table 104.

Table 104 – NOP service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

7.2.1.3.16.3 Service procedure

If the object instance to which the request is delivered supports the NOP service, then a response whose status indicates success shall be returned. If the object does not support the NOP service, then a response indicating an error was detected shall be returned.

7.2.1.3.17 Apply_Attributes**7.2.1.3.17.1 Service overview**

The Apply_Attributes service shall cause attribute values whose use is pending to become actively used in the specified APO object class or instance.

7.2.1.3.17.2 Service primitives

The service parameters for this service are shown in Table 105.

Table 105 – Apply_Attributes service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

Path segment error (mandatory extended status, see Table 91)

Path destination unknown (mandatory extended status, see Table 91)

Connection lost (no extended status)

Unimplemented service (no extended status)

Invalid attribute value (mandatory extended status, see Table 91)

Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)

Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)

7.2.1.3.17.3 Service procedure

Data for pending attributes shall be verified before using them.

7.2.1.3.18 Save

7.2.1.3.18.1 Service overview

The Save service shall copy the contents of an APO object class or instance attributes to a location accessible by the Restore service.

7.2.1.3.18.2 Service primitives

The service parameters for this service are shown in Table 106.

Table 106 – Save service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)
- Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)
- Store operation failure (mandatory extended status, see Table 91)

7.2.1.3.18.3 Service procedure

The Save service shall report success only after the copy has been completed and verified.

7.2.1.3.19 Restore

7.2.1.3.19.1 Service overview

The Restore service shall restore the contents of an APO object class or instance attributes from a storage location accessible by the Save service. Attribute data shall be copied from a storage area to the currently active memory area used by the object class or instance.

7.2.1.3.19.2 Service primitives

The service parameters for this service are shown in Table 107.

Table 107 – Restore service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Additional Path	U	U(=)		
Port ID		M		
Object Specific Data	C	C(=)		
Result (+)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)
Object Specific Data			C	C(=)
Result (-)			S	S(=)
AREP				M
Receiver/Server Local ID			M	
Service status			M	M(=)

Argument

The argument contains the parameters of the service request.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific parameters. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service status

This parameter shall indicate success.

Object Specific Data

This conditional parameter, if specified, contains object class/instance specific information. If an object class/instance utilises this field, then the object class/instance specification shall detail its format.

Result(-)

This selection type parameter indicates that the service request failed.

Service status

This parameter shall indicate an error among the following choices :

- Path segment error (mandatory extended status, see Table 91)
- Path destination unknown (mandatory extended status, see Table 91)
- Connection lost (no extended status)
- Unimplemented service (no extended status)
- Invalid attribute value (mandatory extended status, see Table 91)
 - Object is not able to perform service in its current mode/state (mandatory extended status, see Table 91)
 - Device's mode/state does not allow object to perform service (mandatory extended status, see Table 91)
 - No stored attribute data (no extended status)

7.2.1.3.19.3 Service procedure

Attribute data shall be verified before performing the copy from the “storage area” to the “actively used” area.

If the ability to modify an attribute changes based on the state of the object, the object specification shall provide a detailed description of how this service is effected. This service may only be supported in a state where all attributes are modifiable. The object may ignore the data associated with a currently non-modifiable attribute.

Attributes shall be modified only if all attribute specific value verifications (e.g. range checks, etc.) are successful. The first attribute failing verification shall be specified in the Extended Status element of the Service Status parameter of the response message.

If any other error is detected, then the response shall be returned with a non-zero status code.

If all verification checks pass, then the attributes shall be modified and a Restore success response shall be returned.

7.2.2 Connection manager ASE

7.2.2.1 Overview

The Connection Manager ASE (Object) is used to manage the establishment and maintenance of communication connections. Every device shall implement instance 1 of the Connection Manager object.

7.2.2.2 Connection manager class specification

7.2.2.2.1 Connection manager formal model

7.2.2.2.1.1 Class definition

FAL ASE:	FAL Management ASE
CLASS:	Connection_Manager_Object
CLASS ID:	<td>
PARENT CLASS:	Base_Object

ACCESS ATTRIBUTES:

- 1 (m) Key Attribute: Object Instance number
- 2 (o) Key Attribute: Symbolic name

SYSTEM MANAGEMENT ATTRIBUTES (CLASS ATTRIBUTES) :

- 1 (o) Attribute: Revision = 1
- 2 (o) Attribute: Max Instance
- 4 (o) Attribute: Optional attribute list
- 4.1 (m) Attribute: Number of attributes
- 4.2 (m) Attribute: Optional attributes

OBJECT MANAGEMENT ATTRIBUTES (INSTANCE ATTRIBUTES) :

- 1 (o) Attribute: OpenReqs
- 2 (o) Attribute: OpenFormat Rejects
- 3 (o) Attribute: OpenResource Rejects
- 4 (o) Attribute: OpenOther Rejects
- 5 (o) Attribute: CloseReqs
- 6 (o) Attribute: CloseFormat Rejects
- 7 (o) Attribute: CloseOther Rejects
- 8 (o) Attribute: ConnTimeouts
- 9 (o) Attribute: Connection Entry List
- 9.1 (m) Attribute: NumConnEntries
- 9.2 (m) Attribute: ConnOpenBits
- 11 (o) Attribute: CpuUtilization
- 12 (o) Attribute: MaxBuffSize
- 13 (o) Attribute: BufSize Remaining

SYSTEM MANAGEMENT SERVICES:

- 1 (o) Mgt Service: Get_Attribute_All
- 3 (o) Mgt Service: Get_Attribute_List
- 10 (o) Mgt Service: Get_Attribute_Single

OBJECT MANAGEMENT SERVICES:

- 1 (o) Ops Service: Get_Attribute_All
- 2 (o) Ops Service: Set_Attribute_All
- 3 (o) Ops Service: Get_Attribute_List
- 4 (o) Ops Service: Set_Attribute_List
- 10 (o) Ops Service: Get_Attribute_Single
- 11 (o) Ops Service: Set_Attribute_Single

OBJECT SPECIFIC SERVICES:

- 1 (m) OpsService: CM_Open
- 2 (m) OpsService: CM_Close
- 3 (o) OpsService: CM_Unconnected_Send
- 4 (o) OpsService: CM_Get_Connection_Data
- 5 (o) OpsService: CM_Search_Connection_Data
- 6 (o) OpsService: CM_Get_Object_Owner

7.2.2.2.1.2 System management attributes (class attributes)

No specific requirement for this object.

7.2.2.2.1.3 Object management attributes**OpenReqs**

Number of Open requests received including Null Open requests.

This instance attribute has an access rule of Set.

OpenFormat Rejects

Number of Open requests rejected by this node due to format errors.

This instance attribute has an access rule of Set.

OpenResource Rejects

Number of Open requests rejected by this node.

This instance attribute has an access rule of Set.

OpenOther Rejects

Number of Open requests rejected or timed out by downstream nodes.

This instance attribute has an access rule of Set.

CloseReqs

Number of Close requests received.

This instance attribute has an access rule of Set.

CloseFormat Rejects

Number of Close requests rejected by this node due to format errors.

This instance attribute has an access rule of Set.

CloseOther Rejects

Number of Close requests rejected or timed out by downstream nodes.

This instance attribute has an access rule of Set.

ConnTimeouts

Number of connections which have been timed out by this node after they were opened.

This instance attribute has an access rule of Set.

Connection Entry List

List of connections.

This instance attribute has an access rule of Get only.

NumConnEntries

Number of entries in the ConnOpenBits Attribute.

ConnOpenBits

List of connection data which may be individually queried. Each entry represents a possible connection, and has one of two states : No Connection or Connection Established. More information on a connection can be obtained using a dedicated service.

CpuUtilization

CPU utilization in tenths of a percent.

This instance attribute has an access rule of Get only.

MaxBuffSize

Amount of buffer space originally available (buffer size is in bytes).

This instance attribute has an access rule of Get only.

BufSize Remaining

Amount of buffer space available at this time (buffer size is in bytes).

This instance attribute has an access rule of Get only.

7.2.2.2.1.4 System management (class level) and object management (instance level) services

No specific requirement for this object.

7.2.2.2.1.5 Object specific services

These Connection Manager services are available only through the Unconnected Message Manager AR (UCMM). They are used either to establish/de-establish an application connection, or to send messages through routers without an established connection.

CM_Open

This service is used by connection originator users to establish an application connection.

CM_Close

This service is used by connection originator users to de-establish an application connection.

CM_Unconnected_Send

The CM_Unconnected_Send service shall allow an application to send a message to a device through multiple links without first setting up a connection. This optional service is mandatory for originator devices and devices that route messages between links.

CM_Get_Connection_Data

This service is used for diagnostics of a network, to retrieve connection characteristics.

CM_Search_Connection_Data

This service is used for diagnostics of a network, to retrieve connection characteristics.

CM_Get_Object_Owner

This service is used to determine the owner of a redundant connection.

7.2.2.3 Connection manager ASE service specification

7.2.2.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

CM_Open

CM_Close

CM_Unconnected_Send

CM_Get_Connection_Data

CM_Search_Connection_Data

CM_Get_Object_Owner

7.2.2.3.2 Connection manager common service parameters

The following service parameters are common to all Connection Manager services.

NOTE 1 Complete specification of these parameters is provided in IEC 61158-6.

Originator Local ID

This parameter is locally generated by the originator node, and identifies locally this invocation of the service. It is used to associate service confirmations with requests. Therefore, no two outstanding service invocations may be identified by the same ID value.

Target Local ID

This parameter is locally generated by the target node, and identifies locally this invocation of the service. It is used to associate service responses with indications. Therefore, no two outstanding service invocations may be identified by the same ID value.

Path

This parameter specifies the FAL APO or FAL APO element that is the target of a connection establishment (or de-establishment) request, or an unconnected message request. The path shall contain multiple segments which can indicate the network route to the node containing the FAL APO (in the case of multiple links), the identification of the APO element within the target node (Routing Path), and optional additional information for the target APO (Additional Path). This Additional Path parameter is passed on to the target application, and could be a data segment used to configure the application, the transport class, and the trigger.

Transport identifier

The Transport identifier parameter is generated locally by the originator Connection Manager, and shall serve as a further reference for the user to the newly established connection.

Transaction_priority

The Transaction_priority parameter shall determine which Data Link Layer priority to use for the messages that convey the service requests and responses, and shall be one of LOW or HIGH.

NOTE 2 The definition of DLL priority is specified in IEC 61158-4.

Transaction_timeout

The Transaction_timeout parameter shall determine the time to wait for service completion. The units for Transaction_timeout shall be milliseconds.

O2T_ConnParm / T2O_ConnParm

The O2T parameter shall specify the characteristics of the originator to target (O⇒T) communication on the new connection. The T2O parameter shall specify the characteristics of the target to originator (T⇒O) communication.

CM_RPI

The Requested Packet Interval (CM_RPI) shall specify the expected time between packets, i.e. how frequently the originating application requires the transmission of data from the target application. It shall be expressed as a number of microseconds.

Type

The Type parameter shall indicate the type of the connection, and shall be one of NULL, MULTIPOINT, or POINT2POINT. A value of NULL indicates that the specified direction (O⇒T or T⇒O) shall have no transport associated. A value of MULTIPOINT indicates that other connections may later participate in the transport. Conversely, a value of POINT2POINT indicates that this connection shall not allow any other transport to participate.

Priority

The Priority parameter shall determine which Data Link Layer priority to use for the new transports, and shall be one of LOW, HIGH, or SCHEDULED.

Variable

The Variable parameter specifies whether every application data which traverses the new transport shall be the same size or not, and shall be either TRUE (variable size) or FALSE (fixed size).

Size

The Size parameter shall specify the size (in bytes) of the largest application data packet for this connection.

Redundant Owner

The Redundant Owner parameter shall specify whether more than one owner may be permitted to make a connection simultaneously.

CM_RPI_multiplier

The product of the CM_RPI_multiplier parameter and the actual packet interval (CM_API) shall specify the time-out for the transport. If the application has not used the transport within this time-out, the transport shall close, which shall subsequently close the connection.

Trigger

The trigger parameter shall specify the trigger mode, and shall be one of CYCLIC, CHANGE_OF_STATE, or APPLICATION.

NOTE 3 This parameters shall be used to configure the transport(s). Its meaning to the transports is described in 7.3.1.

Trans_class

The Trans_class parameter shall specify the transport class selected, and shall be one of NULL, DUPLICATE_DETECTION, ACKNOWLEDGED, VERIFIED, NON-BLOCKING, NON-BLOCKING_FRAGMENTING, or MULTIPOINT_FRAGMENTING

NOTE 4 This parameters shall be used to configure the transport(s). Its meaning to the transports is described in 7.3.1.

Is_server

The Is_server parameter shall specify if the new transport shall be of server (TRUE) or a client (FALSE).

NOTE 5 This parameters shall be used to configure the transport(s). Its meaning to the transports is described in 7.3.1.

Manufacturer_ID

Manufacturer ID of the device which has requested establishment of the connection (connection originator).

Originator_Ser_Num

Serial number of the device which has requested establishment of the connection (connection originator).

Connection Serial Number

The Connection Serial Number parameter is a value selected by the originator Connection Manager to uniquely identify a connection within the originator device.

Service status

This parameter provides information on the result of service execution. It is returned in all Connection Manager service response/confirmation primitives. It has the same definition as the Service Status parameter of the FAL Management services. Corresponding available Status Codes and Extended Status formats are detailed in 7.2.2.3.3.

O2T_CM_API / T2O_CM_API

The O2T_CM_API and T2O_CM_API shall specify the actual packet interval that shall be used for the new connection, i.e. how frequently the connection produces its data. It shall be expressed in microseconds. These values may be different than the requested packet intervals, but shall always be equal or smaller than the requested interval.

Remaining Path Size

In the failure response, the remaining_path parameter specifies the “pre-stripped” size. This is the size of the path when the node first receives the request and has not yet started processing it. A target node may instead return 0 (zero) for this parameter. Associated with the Service Status (Status Code and Extended Status), this parameter allows to identify the type and location of any errors found during connection establishment or de-establishment.

Response Data

If the target has any application specific response data, it shall be returned in the Response Data parameter.

7.2.2.3.3 Connection manager service status codes

Specific status codes for the Connection Manager are detailed in IEC 61158-6.

7.2.2.3.4 CM_Open**7.2.2.3.4.1 Service overview**

The CM_Open service shall be used by connection originator users to establish an application connection to a specified target object instance or instance element. The CM_Open request contains the parameters to select connection types, transports and to specify the requested packet interval in both the originator to target and target to originator direction.

7.2.2.3.4.2 Service primitives

The service parameters for this service are shown in Table 108.

Table 108 – CM_Open service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Additional Path	U	U(=)		
Transport identifier	M			
Transaction_priority	M			
Transaction_timeout	M			
O2T_ConnParm	M	M(=)		
CM_RPI	M	M(=)		
Type	M	M(=)		
Priority	M	M(=)		
Variable	M	M(=)		
Size	M	M(=)		
T2O_ConnParm	M	M(=)		
CM_RPI	M	M(=)		
Type	M	M(=)		
Priority	M	M(=)		
Variable	M	M(=)		
Size	M	M(=)		
CM_RPI_multiplier	M	M(=)		
Trigger	M	M(=)		
Trans_class	M	M(=)		
Is_server	M	M(=)		
Manufacturer_ID		M		
Originator_Ser_Num		M		
Connection Serial Number		M		
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Transport identifier				M
O2T_CM_API			M	M(=)
T2O_CM_API			M	M(=)
Remaining Path Size			M	M(=)
Response Data			U	U(=)

Argument

The argument contains the parameters of the service request.

Transport identifier

The transport identifier parameter in the request shall allow a previously opened connection to be reused (this is the same identifier as the one returned in the CM_Open confirmation for this first connection). If Transport identifier is zero, then a new transport shall be created.

NOTE Reusing a transport allows an originator to establish multipoint connections in the O⇒T direction.

Result

This parameter indicates whether the service request succeeded or failed.

7.2.2.3.4.3 Service procedure

The CM_Open request primitive, sent from the originating application, first triggers the connection establishment process into the local Connection Manager, before being forwarded to the remote target Connection Manager.

The corresponding CM_open indication primitive shall then be sent to the target application from the target Connection Manager, after some local processing. The application shall reply to the CM_open indication primitive with a CM_open response whether the application accepts the connection or not.

After the connection has been established, or if the connection attempt failed, a CM_open confirm primitive shall be sent from the local originator Connection Manager back to the originating application. The confirmation shall indicate the status of the connection and shall provide information about the connection including the Transport Identifier parameter, which shall serve as a reference to the newly established connection.

7.2.2.3.5 CM_Close

7.2.2.3.5.1 Service overview

The CM_Close service shall be used by connection originator users to de-establish ("close") an application connection previously established to a target object instance or instance element, and to de-allocate the resources associated with the connection. The service request shall indicate the connection to close by using the original path and the Transport identifier. The originating application shall save both the connection path required to establish the connection, and the Transport identifier returned by the CM_open confirmation.

7.2.2.3.5.2 Service primitives

The service parameters for this service are shown in Table 109.

Table 109 – CM_Close service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Additional Path	U	U(=)		
Transport identifier	M			
Transaction_priority	M			
Transaction_timeout	M			
Manufacturer_ID		M		
Originator_Ser_Num		M		
Connection Serial Number		M		
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Transport identifier				M
Remaining Path Size			M	M(=)
Response Data			U	U(=)

Argument

The argument contains the parameters of the service request.

Result

This parameter indicates whether the service request succeeded or failed.

7.2.2.3.5.3 Service procedure

The CM_close request primitive, sent from the originator application, first triggers the connection de-establishment process into the local Connection Manager, before being forwarded to the remote target Connection Manager. of a connection to de-establish the connection and de-allocate the resources associated with the connection. The service shall be sent from the originator application to the local Connection Manager.

The corresponding CM_close indication primitive shall then be sent from the target Connection Manager to the target application, after some local processing.

The CM_close response shall be sent from the target application to the target Connection Manager to indicate the success of the CM_close request. A successful CM_close response primitive shall be returned when the de-establishment has been acknowledged by the target application. Once the response indicating success has been received, the connection shall be closed. The target application may optionally pass back application specific information in the close response. This application response information shall be object specific.

A target application shall not refuse to close a connection but may not be able to find the specified connection. Once the CM_close_response is received, the specified connection shall be closed or shall not be present. The object specific status words shall contain information about the reason for the failure. The remaining connection path size shall contain the path size from the point at which the close connection failed.

The CM_close confirmation shall be sent back from the local Connection Manager to the originating application to indicate the status of the close connection request.

A successful CM_close confirmation shall be returned when the close has been acknowledged by all the nodes involved in the original connection. The confirmation shall indicate the Transport identifier of the closed connection. Once the confirmation indicating success has been received, the connection shall be closed. The target application may optionally pass back application specific information in the close response. This information shall be object specific.

In case of a failure response, the Service status parameter shall contain information about the reason for the failure. The Remaining Path Size shall contain the path size from the point at which the close connection failed. This Remaining Path Size shall be the “pre-stripped” size. This shall be the size of the path when the node first receives the request and has not yet started processing it. A target node may return either the “pre-stripped” size or 0 for the Remaining Path Size.

7.2.2.3.6 CM_unconnected_Send

7.2.2.3.6.1 Service overview

The CM_Unconnected_Send service shall allow an application to send a message to a device through multiple links without first setting up a connection. This optional service is mandatory for originator devices and devices that route messages between links.

7.2.2.3.6.2 Service primitives

The service parameters for this service are shown in Table 110.

Table 110 – CM_Unconnected_Send service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Additional Path	U	U(=)		
Transaction_priority	M			
Transaction_timeout	M			
OM_Service Code	M	M(=)		
OM_Service Request Parameters	M	M(=)		
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Remaining Path Size			M	M(=)
OM_Service Response Parameters			M	M(=)

Argument

The argument contains the parameters of the service request.

OM_Service Code

This parameter indicate the code of the Object Management service to be performed by the target element.

OM_Service Request Parameters

This parameter contains the specific arguments parameters of the Object Management service to be performed by the target element.

Result

This parameter indicates whether the service request succeeded or failed.

OM_Service Response Parameters

This parameter contains the specific result parameters of the Object Management service to be performed by the target element.

7.2.2.3.6.3 Service procedure

The CM_Unconnected_Send service shall use the Connection Manager object in each intermediate node to forward the message and to remember the return path. The UCMM of each link shall be used to forward the request from Connection Manager to Connection Manager just as it is for the Forward_Open service; however, no connection shall be built. The CM_Unconnected_Send service shall be sent to the local Connection Manager and shall be sent between intermediate nodes. When an intermediate node removes the last port segment, the message shall be formatted as a UCMM message and sent to the port and link address of the last segment.

NOTE The target node never sees the CM_Unconnected_Send service but only a standard message arriving via the UCMM.

The CM_Unconnected_Send response shall be generated by the last intermediate node from the UCMM response generated by the target node or by an intermediate node as the result of a UCMM time-out, a problem with the embedded message, or a problem with the Unconnected Service Request itself. The packet shall be routed from intermediate node to intermediate node using the information stored when the CM_Unconnected_Send request was processed. The response shall contain status information about the request and a response generated by the target node.

7.2.2.3.7 CM_Get_Connection_Data**7.2.2.3.7.1 Service overview**

The CM_Get_Connection_Data service may be used for diagnostics of a network. This service shall return the parameters associated with a specified connection.

7.2.2.3.7.2 Service primitives

The service parameters for this service are shown in Table 111.

Table 111 – CM_Get_Connection_Data service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Target transport identifier	M	M(=)		
Transaction_priority	M			
Transaction_timeout	M			
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Target transport identifier			M	M(=)
Connection state			M	M(=)
Originator port			M	M(=)
Target port			M	M(=)
Connection Serial Number			M	M(=)
Manufacturer_ID			M	M(=)
Serial_Number			M	M(=)
Originator O2T_CID			M	M(=)
Target O2T_CID			M	M(=)
O2T_CM_RPI_multiplier			M	M(=)
Originator O2T_CM_RPI			M	M(=)
Originator O2T_CM_API			M	M(=)
Originator T2O_CID			M	M(=)
Target T2O_CID			M	M(=)
T2O_CM_RPI_multiplier			M	M(=)
Originator T2O_CM_RPI			M	M(=)
Originator T2O_CM_API			M	M(=)

Argument

The argument contains the parameters of the service request.

Target transport identifier

This parameter identifies the connection for which data is requested. This number may be different from device to device even for the same connection. This number corresponds to the offset into the Connection Manager attribute that enumerates the status of the connections.

Result

This parameter indicates whether the service request succeeded or failed.

Serial_Number

This parameter contains the Serial Number of the target node.

7.2.2.3.7.3 Service procedure

No specific service procedure.

7.2.2.3.8 CM_Search_Connection_Data**7.2.2.3.8.1 Service overview**

The CM_Search_Connection_Data service may be used for diagnostics of a network. This service shall return the parameters associated with a specified connection within a device, identified by vendor and serial number.

7.2.2.3.8.2 Service primitives

The service parameters for this service are shown in Table 112.

Table 112 – CM_Search_Connection_Data service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Transaction_priority	M			
Transaction_timeout	M			
Manufacturer_ID	M	M(=)		
Serial Number	M	M(=)		
Connection Serial Number	M	M(=)		
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Target transport identifier			M	M(=)
Connection state			M	M(=)
Originator port			M	M(=)
Target port			M	M(=)
Connection Serial Number			M	M(=)
Manufacturer_ID			M	M(=)
Serial Number			M	M(=)
Originator O2T_CID			M	M(=)
Target O2T_CID			M	M(=)
O2T_CM_RPI_multiplier			M	M(=)
Originator O2T_CM_RPI			M	M(=)
Originator O2T_CM_API			M	M(=)
Originator T2O_CID			M	M(=)
Target T2O_CID			M	M(=)
T2O_CM_RPI_multiplier			M	M(=)
Originator T2O_CM_RPI			M	M(=)
Originator T2O_CM_API			M	M(=)

Argument

The argument contains the parameters of the service request.

Serial_Number

This parameter contains the Serial Number of the target node.

Result

This parameter indicates whether the service request succeeded or failed. Its contents are identical to the Get_Connection_Data primitives.

7.2.2.3.8.3 Service procedure

No specific service procedure.

7.2.2.3.9 CM_Get_Object_Owner

7.2.2.3.9.1 Service overview

The CM_Get_Object_Owner service shall return data about the connection(s) that own(s) a particular object. It shall be implemented in any device that accepts redundant connections.

7.2.2.3.9.2 Service primitives

The service parameters for this service are shown in Table 113.

Table 113 – CM_Get_Connection_Data service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M			
Originator Local ID	M			
Target Local ID		M		
Path	M			
Routing Path	M			
Transaction_priority	M			
Transaction_timeout	M			
Result				
Originator Local ID				M
Target Local ID			M	
Service status			M	M(=)
Number of connections			M	M(=)
Number claiming ownership			M	M(=)
Number ready for ownership			M	M(=)
Last action			M	M(=)
Connection Serial Number			M	M(=)
Originator Manufacturer_ID			M	M(=)
Originator_Ser_Num			M	M(=)

Argument

The argument contains the parameters of the service request.

Path

This parameter shall be the internal path from the Message Router in the target node to the selected object. It shall be the same path as would have appeared in a CM_Forward_Open request for this object, except that corresponding Additional Path electronic key, network, and data segments shall be removed before matching the paths.

Result

This parameter indicates whether the service request succeeded or failed.

Number of connections

This parameter shall be the number of connections currently open to the specified path.

Number claiming ownership

This parameter shall be the number of connections that are currently claiming ownership.

Number ready for ownership

This parameter shall be the number of connections currently ready for ownership.

Last action

This parameter shall indicate mode of owning connection as follows. If the owning connection is in the idle mode, the Last action field shall equal 1. If the connection is in the run mode, the Last action field shall equal 2. If there is currently no owner, the Last action field shall equal 0. Values of the Last action field from 0x03 through 0xFE shall be reserved. If an implementation does not support one of these fields, the field shall equal 0xFF.

Connection Serial Number**Originator Manufacturer_ID****Originator_Ser_Num**

These parameters shall be from the CM_Forward_Open request whose connection is currently the owner of the object. If no owner currently exists, these parameters shall each be set to zero.

7.2.2.3.9.3 Service procedure

No specific service procedure.

7.3 ARs**7.3.1 Overview****7.3.1.1 General**

One of the roles of the Application Layer is to establish and maintain connections. A connection is similar to a telephone circuit. When a call is placed, the telephone system selects a path for your call and sets up each switching station in the route to handle it. As long as the call continues, the resulting virtual circuit remains open, carrying data or voice traffic. In the telephone system, a single call can travel over multiple, different type links, and the format of the data can change from link to link, but the connection appears the same to both the caller and the party called. Sound in one end becomes sound out the other.

A connected message assumes previously negotiated resources and parameters at its source, its destination(s), and any intermediate transit points. These resources are referenced by a unique connection identifier and do not need to be contained in each message, only the connection ID is needed to identify the message and refer to its related parameters, thus giving significant savings in message efficiency.

An unconnected message provides a means to communicate on the local link without previously negotiated resources at the destination so it shall carry full destination ID details, internal data descriptors and full source ID details if a reply is requested. Unconnected messages are used mainly to create connections.

The unconnected service is provided by the Unconnected Message Manager (UCMM). Messages received through the UCMM are forwarded to the Message Router (MR), which direct them to the appropriate internal object for execution. Connections are established by specific unconnected messages sent through the Connection Manager (CM) using UCMM services. Connections may be established either to the Message Router (for messaging purpose), or directly to an application object. Connection target is specified using a

connection path, and may be either on the local or a remote link, through several intermediate router nodes. Once a connection is established with an application object, the UCMM, MR and CM are no longer required, since data will be exchanged directly with the connected object, based on the corresponding connection ID. Connected messages sent to the Message Router will be forwarded to the appropriate internal object for execution.

The connected and unconnected protocols specified within this IEC 61158-5 are collectively known as the control and information protocol (CIP).

7.3.1.2 Message router

The Message Router object shall provide a messaging interface through which a client may address a service to any object class or instance residing in the device. Every node shall contain instance 1 of the Message Router object.

7.3.1.3 Application connections

7.3.1.3.1 Connections

An application connection is a logical connection between two applications, and is made at the application layer of the communication model. An application connection is built on one or two transport connections. The originating application is responsible for creating and binding the client transport instance to the originating application, and the target application is responsible for creating and binding the server transport instance to the target application. An application connection includes the transport connection which, in turn, includes the producers and consumers involved in the network connection.

Since the unidirectional class 0 and 1 transports require a single network connection, two unidirectional transports can be established within a single connection. The two unidirectional transports do not have to be co-ordinated at either of the applications, but shall be linked in any of the routers.

The application connection can also include additional parameters or data. A target application, for example, may need information on electronic keying, ownership verification, or the detailed configuration to establish the connection. Such information is sent as a structure in a data segment as part of the connection path. The data segment is forwarded by the intermediate nodes and delivered to the target application as additional segments in the CM_open indication service primitive. The data segment shall be interpreted by the target application.

NOTE The target application may check the module type in the electronic key, check or save the ownership information, and use the configuration information to set up the device. Success and reply information or failure and a reason for the failure can be returned in the message reply. The format of the data segment is a function of the target application.

7.3.1.3.2 Production trigger, transport class and CM_RPI

A client application uses the transport class, trigger type and CM_RPI to determine when to sample and send data.

The system supports three types of triggers for the production of data: cyclic, change of state, and application-triggered. These triggers only apply to the client application.

The triggers are used in conjunction with the transport class and CM_RPI to control when data are sampled and sent on the link. Table 114 shows how these factors relate to each other in determining when data are produced.

Table 114 – How production trigger, transport class, and CM_RPI determine when data is produced

Trigger type	Transport class	When data are sampled and sent	When data are repeated
Cyclic	0 – Null	At the CM_RPI	
	1 – Duplicate Detection	At the CM_RPI	
	2 – Acknowledged	At the CM_RPI	At 1/4 the CM_RPI or faster if not acknowledged
	3 – Verified	At the CM_RPI	At 1/4 the CM_RPI or faster if not verified
	4 – Non-Blocking	Trigger type not supported	Trigger type not supported
	5 – Fragmenting	Trigger type not supported	Trigger type not supported
	6 - Multipoint Fragmenting	Trigger type not supported	Trigger type not supported
Change of State	0 – Null	When data changes	At the CM_RPI
	1 – Duplicate Detection	When data changes	At the CM_RPI
	2 – Acknowledged	When data changes	At 1/4 the CM_RPI or faster until acknowledged, and then at the CM_RPI
	3 – Verified	When data changes	At 1/4 the CM_RPI or faster until verified, and then at the CM_RPI
	4 – Non-Blocking	When application places on queue	At 1/4 the CM_RPI or faster until acknowledged, and then at the CM_RPI
	5 – Fragmenting	When application places on queue	At 1/4 the CM_RPI or faster until acknowledged, and then at the CM_RPI
	6 - Multipoint Fragmenting	When application generates Write event	At 1/4 the CM_RPI or faster until verified, and then at the CM_RPI
Application	0 – Null	Determined by application	
	1 – Duplicate Detection	Determined by application	
	2 – Acknowledged	Determined by application	At 1/4 the CM_RPI or faster if not acknowledged
	3 – Verified	Determined by application	At 1/4 the CM_RPI or faster if not verified
	4 – Non-Blocking	When application places on queue	At 1/4 the CM_RPI or faster if until acknowledged, and then when application generates Trigger event
	5 – Fragmenting	When application places on queue	At 1/4 the CM_RPI or faster if until acknowledged, and then when application generates Trigger event
	6 - Multipoint Fragmenting	When application generates Trigger event	At 1/4 the CM_RPI or faster if until verified, and then when application generates Trigger event

7.3.1.3.3 Polling

Although related to triggering, polling is a server-only function. The system supports two types of polling: asynchronous and synchronous.

Asynchronous polling uses transport class 2 (acknowledged). When the server application receives the poll request, the current data buffer is sent as the acknowledgement. This method requires the application to continuously sample data and update the buffer. The application sampling the data does not know which sample is sent in response to [via] a poll request.

The synchronous poll shall use transport class 3 (verified). When the server application is notified of the poll request, it shall sample new data, which it returns as the verification.

7.3.1.4 Transport connections

7.3.1.4.1 General

A transport connection is a logical binding between two transport instances that is made to enable two applications to transfer data over network connections. A transport instance is a specific implementation of a transport function. Every transport connection is built on one or two network connections. Figure 28 uses the communication model to show the context in which transport connections are made.

NOTE 1 Input, Download, Upload and Scanner functions are examples of users of the transport services, they are not described in IEC 61158.

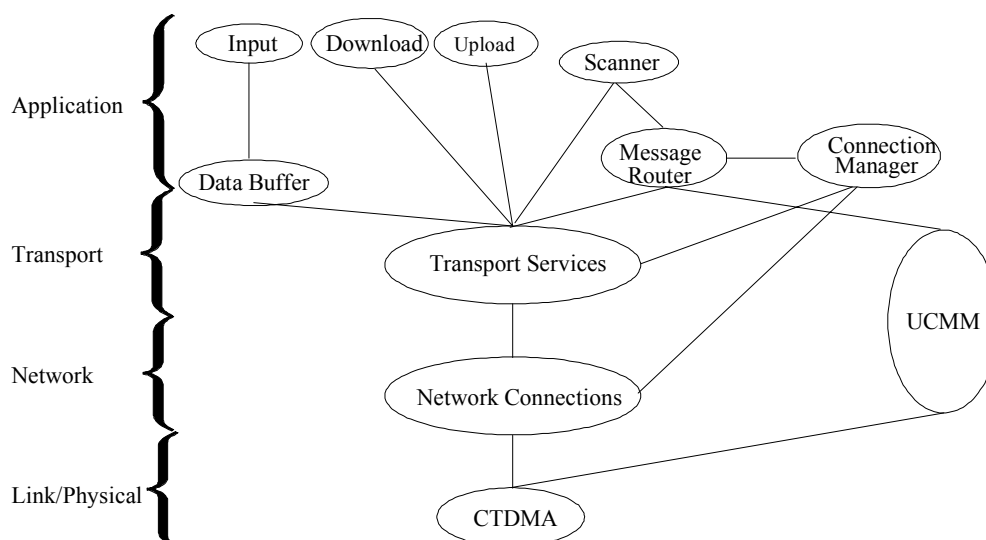


Figure 28 – Context of transport services within the connection model

In Figure 29, application A interfaces to a client transport instance to trigger the production of data over the network connection, and application B interfaces to a server transport instance to be notified that data has been written to the transport protocol data unit (T-PDU) buffer. A T-PDU comprises one packet of data as well as its link, network, and transport headers. A T-PDU buffer is a memory location in which one T-PDU can be stored.

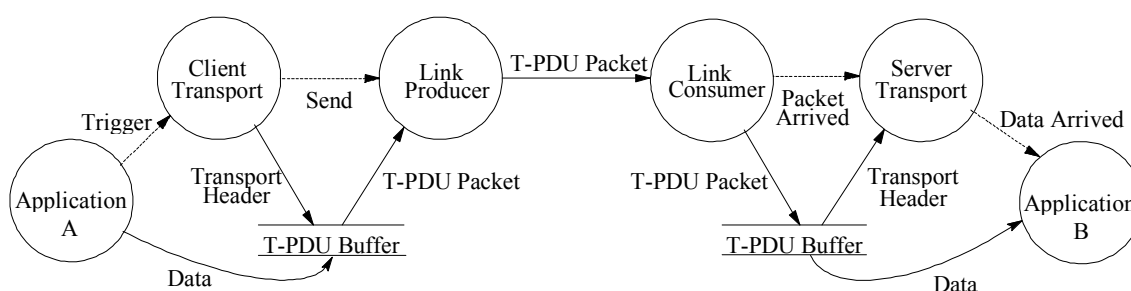


Figure 29 – Application-to-application view of data transfer

Data is not directly written to or read from the transport instances: data is sent to or removed from the T-PDU buffers. The client and server transport instances are responsible only for managing the transport headers of the data packets in the T-PDU buffers. See text on transport classes in IEC 61158-6 for additional details on the responsibilities of the client and server transport instances.

NOTE 2 Producers, consumers, and transport instances do not have public interfaces. They are accessible only through the Connection Manager that creates them.

7.3.1.4.2 Components of transport connections

7.3.1.4.2.1 Components

A transport connection includes the transport instance(s), the client- and server-side T-PDU buffer(s), and the network connection(s) involved. The network connection, in turn, includes the link producer(s) and link consumer(s). The originating application is responsible for creating all of the above components, and for creating bindings between those components that result in the desired transport connection being created.

7.3.1.4.2.2 Network connections

A logical binding between a producer and consumer shall form a network connection. The Connection Manager shall establish this connection. When a producer wants to send a message, it shall provide just the production ID. The production ID shall imply the path that the Connection Manager specified. Because of this implied route, no addressing or routing information other than the production ID is used at run time.

There is no queuing of data along a network connection. Overwrites shall be permitted at all intermediate nodes. Those applications that require queuing shall use an appropriate transport class to deliver this behaviour.

Link Producer

The link producer shall be responsible for fetching data from the data buffer and producing it on the data link after the send event has been received. Data buffer management schemes such as overwrite, double buffered, triple buffered are not excluded, but since they are implementation specific, they are not specified here.

Behaviour of the link producer is shown in Figure 30.

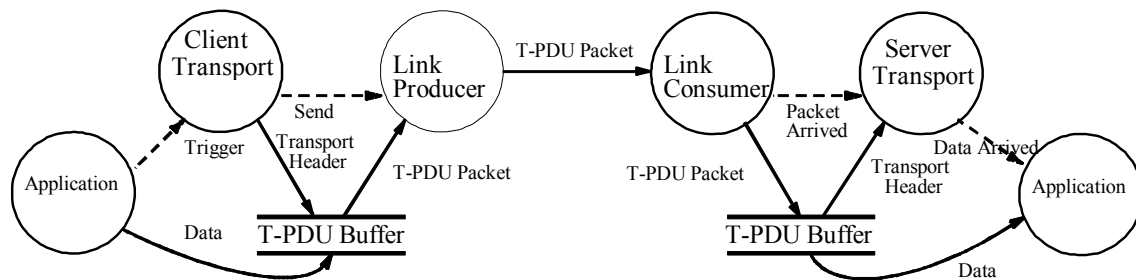


Figure 30 – Data flow diagram for a link producer

Class object description of the Link Producer is detailed below.

FAL ASE:		Link Producer ASE
CLASS:	Link_Producer	
CLASS ID:	<tbid>	
PARENT CLASS:	TOP	
STATIC ATTRIBUTES:		
1	(m) Attribute:	Production ID
2	(m) Attribute:	Priority = SCHEDULED HIGH LOW
3	(m) Attribute:	Connection Size
4	(m) Attribute:	Destination Type = POINT2POINT MULTIPOINT
5	(m) Attribute:	State
SERVICES:		
1	(m) OpsService:	Create
2	(o) OpsService:	Delete
3	(m) OpsService:	Send

Create

Create instance of a link producer.

Delete

Delete instance of a link producer.

Send

Produce the T-PDU buffer.

Link Consumer

The link consumer is responsible for receiving T-PDU packets, storing them in the T-PDU buffer and notifying the server transport class that a packet has arrived. Data buffer management schemes such as overwrite, double buffered, triple buffered are not excluded, but since they are implementation specific, they are not specified here.

Behaviour of the link producer is shown in Figure 31.

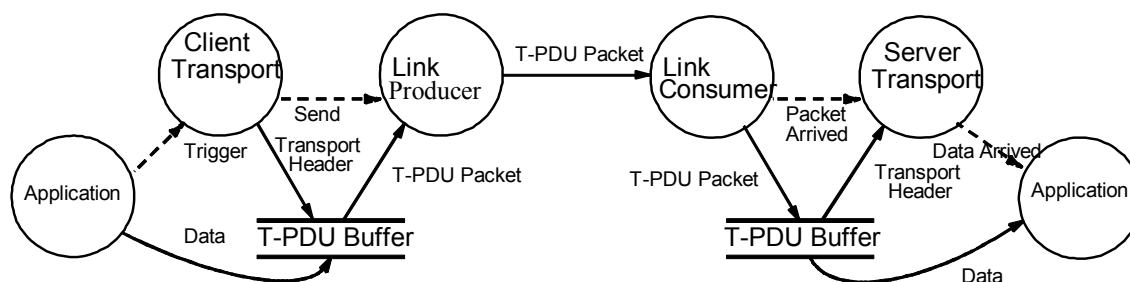


Figure 31 – Data flow diagram for a link consumer

Class object description of the Link Consumer is detailed below.

FAL ASE:		Link Consumer ASE
CLASS:	Link_Consumer	
CLASS ID:	<tbid>	
PARENT CLASS:	TOP	
STATIC ATTRIBUTES:		
1	(m) Attribute:	Consumption ID
2	(m) Attribute:	Priority = SCHEDULED HIGH LOW
3	(m) Attribute:	Connection Size
4	(m) Attribute:	Destination Type = POINT2POINT MULTIPOINT
5	(m) Attribute:	State
SERVICES:		
1	(m) OpService:	Create
2	(o) OpService:	Delete
3	(m) OpService:	Send

Create

Create instance of a link consumer.

Delete

Delete instance of a link consumer.

7.3.1.4.2.3 Triggers

A trigger shall cause data to be sent. This trigger can be anything that makes sense from an application point of view, such as a clock on a module, a network trigger, a scheduler, an asynchronous event or a timer, as shown in Figure 32.

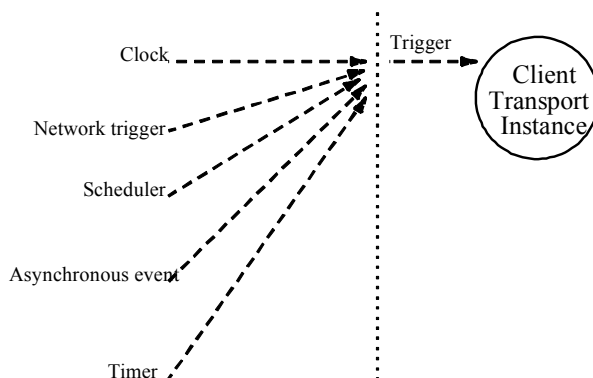


Figure 32 – Triggers

It is important to note that the writing of data to the buffer and triggering of that data are separate functions. An application may write to a buffer more often than it triggers data or trigger data more often than it writes to the buffer. Applications shall write data to the data buffer before it is initially triggered to ensure that valid data is sent.

7.3.1.4.3 Creating transport connections

After the application has bound the producer to the consumer at the network level of the communication model, the application creates the transport connection by binding the producer, consumer, and T-PDU buffer(s) to the transport instances at the transport level. An application may support as many transport connections as resources permit.

The simplest transport functions use a single network connection, which can be either point-to-point or multipoint; the more complex transport functions also use one or more point-to-point return network connections. The transport functions correspond to the transport classes. Each transport class provides a specified level of transport services.

7.3.1.4.4 Binding transports to producers and consumers of network connections

The binding rules followed to create transport connections are as follows:

- A client transport instance is bound to exactly one producer instance;
- A client transport instance can be bound to zero, one, or many consumer instances;
- A server transport instance can be bound to zero or one producer instances;
- A server transport instance is bound to exactly one consumer instance;
- A server transport instance can be bound to a producer instance and a consumer instance, but can not be bound to a producer instance without also being bound to a consumer instance.

The following examples demonstrate application of the binding rules to different kinds of transport connections that include network connections with different destination types.

- For a point-to-point connection that does not have a reverse data path, the producer is bound to the client transport instance and the consumer is bound to the server transport instance as shown in Figure 33:

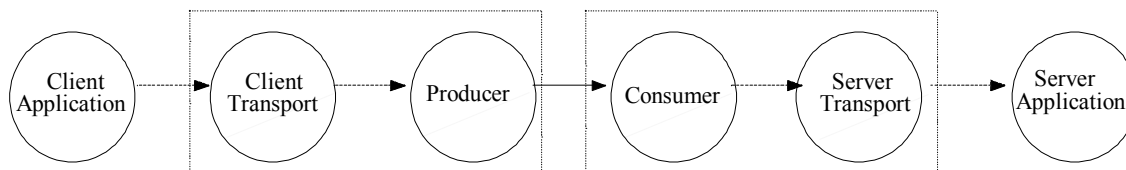
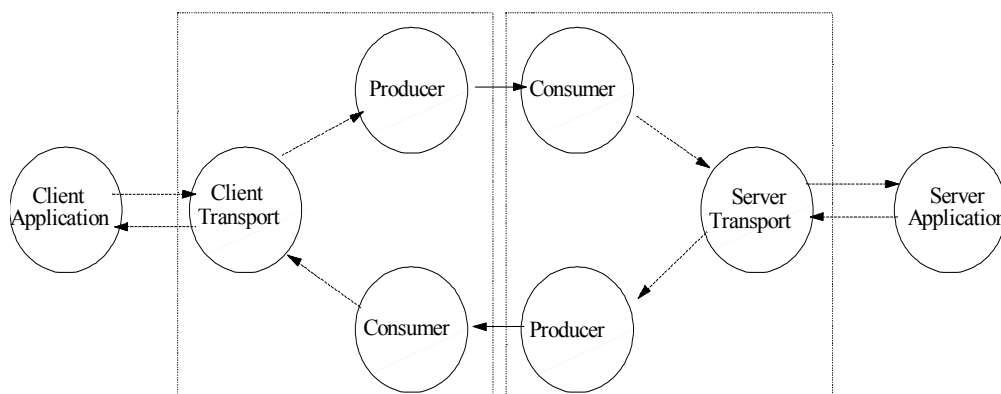


Figure 33 – Binding transport instances to the producer and consumer of a transport connection that does not have a reverse data path

- For a point-to-point connection that does have a reverse data path:
 - In the client-to-server direction, the producer is bound to the client transport instance, and the consumer is bound to the server transport instance.
 - In the server-to-client direction, the producer is bound to the server transport instance, and the consumer is bound to the client transport instance.

Figure 34 illustrates this scenario.



NOTE: In this configuration, the transport connection includes:

- 1) the client and server transport instances
- 2) the network connections, including the producers and consumers in both the client-to-server and server-to-client directions.

Figure 34 – Binding transport instances to the producers and consumers of a transport connection that does have a reverse data path

- For a multipoint connection that does not have a reverse data path, the producer is bound to the client transport instance, and each consumer is bound to a separate server transport instance. It is important to note that there is only one client transport instance, and there are as many server transport instances as there are client-to-server network connections.

Figure 35 illustrates this scenario.

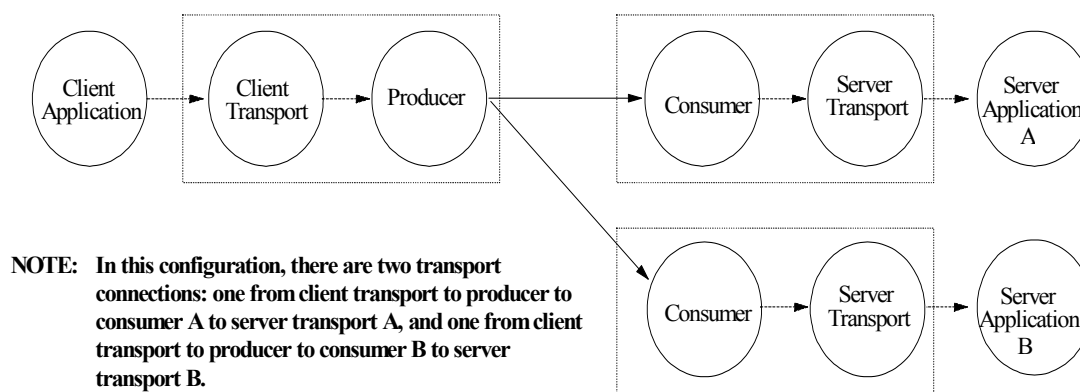


Figure 35 – Binding transport instances to the producer and consumers of a multipoint connection when the transport connection does not have a reverse data path

- For a multipoint connection that does have a reverse data path:
 - In the client-to-server direction, the producer is bound to the client transport instance, and each consumer is bound to a separate server transport instance.
 - In the server-to-client direction, each producer is bound to a separate server transport instance, and each consumer is bound to the client transport instance. It is important to note that there is only one client transport instance, and there are as many server transport instances as there are client-to-server network connections.

Figure 36 illustrates this scenario.

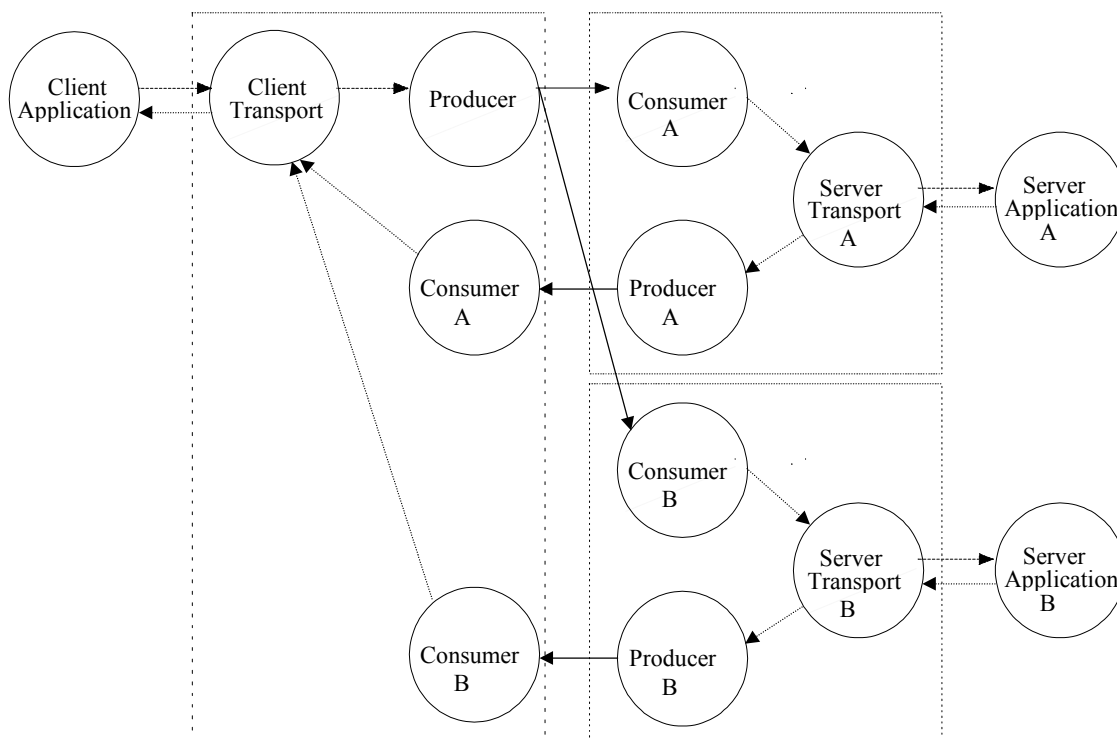


Figure 36 – Binding transport instances to the producers and consumers of a multipoint connection when the transport connection does have reverse data paths

7.3.1.4.5 Transport type

7.3.1.4.5.1 General

The transport type shall determine the target behaviour concerning the relationship between different connections each sharing a producer, and shall be one of LISTEN_ONLY, INPUT_ONLY, EXCLUSIVE_OWNER or REDUNDANT_OWNER.

7.3.1.4.5.2 Listen only

If a connection has a transport type of listen only, it shall be dependent on another connection for its existence. Devices that wish to listen to multicast data without providing configuration or scheduling information may use this type of transport. If the connection on which a listen only connection depends is closed or times out, the listen only connection shall also be closed.

Unlike exclusive owner and redundant owner connections, listen only connections need not have a method to specify the RUN/IDLE state of the originator.

7.3.1.4.5.3 Input only

If a connection has a transport type of input only, it shall not be dependent on any other connection for its existence. For scheduled input only connection, the Forward_Open path shall contain a schedule segment. No $O \Rightarrow T$ data shall be sent; therefore, a target may accept many input only connections. A specific implementation may limit the number of input only connections it accepts.

Unlike exclusive owner and redundant owner connections, input only connections shall not have a method to specify the RUN/IDLE state of the originator.

7.3.1.4.5.4 Exclusive owner

If a connection has a transport type of exclusive owner, it shall not be dependent on any other connection for its existence. For scheduled exclusive owner connections, the Forward_Open path shall contain a schedule segment. $O \Rightarrow T$ data that controls outputs may be present; therefore, a target may only accept one exclusive owner connection. In addition, the target may accept listen only and input only connections that use the same multicast $T \Rightarrow O$ data.

The term connection owner shall refer to the connection originator whose $O \Rightarrow T$ packets are being consumed by the target object. The term owning connection shall refer to the connection associated with connection owner.

NOTE Exclusive owner connections are the most common transport type for controller to I/O connections since they allow the controller to receive inputs and control outputs on the same connection.

7.3.1.4.5.5 Redundant owner

General

The redundant owner connection shall allow multiple separate originator applications to each establish an independent, identical connection to the transport of a target application. The target transport shall in turn send events to the target application so that the redundant owner connection appears as a single, exclusive owner connection to the target application. At most one of the originator applications shall have its data applied to the target application. The target transport shall multicast the target application data to each of the originator applications.

NOTE 1 The redundant owner connection may commonly be used in applications where up time is at a premium. In these applications, multiple originator applications can each establish a connection to a target application (typically an output application). Should an originator application fail or otherwise give up control, another originator application can quickly have its data applied to the target application without having to establish a connection with the target.

Establishing the Connection

A redundant owner connection and an exclusive owner connection shall not both be established to a target application at the same time. Multiple redundant owner connections may be established to the same target simultaneously provided that the following fields of the Forward_Open request are identical and the connection paths match : O2T_RPI, O2T_connection_parameters, T2O_RPI, T2O_connection_parameters, xport_type_and_trigger. The connection path shall match including any data segments.

The target shall return a status code of "connection failure" and an extended status of "redundant connection mismatch" if any of these fields do not match.

The target transport shall only send the CM_open indication to the target application when the first CM_Forward_Open PDU is received. Subsequent redundant Forward_Open PDUs shall not cause a CM_open indication to be sent to the target application.

NOTE 2 The Redundant Owner element of the connection parameters in the CM_Open request specifies whether the connection is an exclusive owner or redundant owner connection (see 7.2.2.3.2).

7.3.2 UCMM AR formal model

7.3.2.1 Description

The UCMM shall provide an unconnected request/response message service, limited to a single link that supports multiple outstanding messages. The required number of outstanding messages shall be implementation specific. The UCMM shall be present in all nodes, and shall support at least one outstanding message.

NOTE Most I/O modules need only support a single message while controllers typically support multiple outstanding messages. Setting up a connection requires communication between devices. This initial communication requires an unconnected message service. The UCMM, which only operates on a single link, is different from other transports. That is, while connected transport types use a single buffer to store packets, with new packets overwriting old ones, UCMM packets are queued at each node instead. Multiple connections can, therefore, be established at the same time. This reduces the overall time needed to set up all the connections in a system.

All incoming UCMM requests shall be internally forwarded to the Message Router, who is responsible for sending them to the target object instance or element, based on the path specified in the request.

7.3.2.2 Formal class definition

FAL ASE:	AR ASE
CLASS:	UCMM_Transaction_Record
CLASS ID:	<td>
PARENT CLASS:	TOP
ACCESS ATTRIBUTES:	
1	(m) Key Attribute: Record Number
STATIC ATTRIBUTES:	
1	(m) Attribute: Fixed Tag
2	(m) Attribute: Max_Retry
3	(m) Attribute: Retry Timeout
DYNAMIC ATTRIBUTES:	
1	(m) Attribute: Record State
2	(m) Attribute: Sequence Number
3	(m) Attribute: Retry Count
4	(m) Attribute: Response Timeout

SERVICES:

1	(m)	OpsService:	UCMM_Create
2	(o)	OpsService:	UCMM_Delete
3	(m)	OpsService:	UCMM_Write
4	(m)	OpsService:	UCMM_Abort

7.3.2.3 Key attributes**Record Number**

Unique number associated with a given UCMM transaction record.

7.3.2.4 Static attributes

These attributes have fixed values once the Transaction Record is created.

Fixed Tag

DLL fixed tag service used to exchange messages on this transaction record

Max_Retry

Maximum number of retries for this transaction record.

Retry Timeout

If no response or acknowledgement has been received back after the transmission of a UCMM packet (request or response) within the delay specified by the Retry Timeout, transmission of the packet should be repeated. The Retry Timeout value shall be fixed for the link at a value which guarantees that both the client and server nodes have an opportunity to transmit an unscheduled DLL packet (see IEC 61158-3).

7.3.2.5 Dynamic attributes

These attributes change values based on the current operating state of the Transaction Record.

Record State

Current state of the Transaction Record. This shall be one of INACTIVE, RUNNING or WAITING-FOR-RESPONSE.

Sequence Number

Sequence Number shall be used for duplicate detection. It shall be incremented for every unique message on a given Transaction Record ; however it shall not be incremented on a retry.

The Sequence Number shall be set to zero at initialisation. The sequence number value shall be maintained in the INACTIVE state, to be used when the record transitions to running.

Retry Count

Current number of retries for a given message. When Retry count reaches Max_Retry, the current transaction shall be automatically aborted.

Response Timeout

The Response Timeout attribute indicates the maximum duration of the transaction in microseconds. If no response is received before the time-out expires, the transaction shall be automatically aborted. This value is provided when the UCMM_Write service is invoked.

1 (m) Key Attribute: Transport identifier

STATIC ATTRIBUTES:

- | | | | |
|---|-----|------------|-----------------|
| 1 | (m) | Attribute: | Transport class |
| 2 | (o) | Attribute: | Max_Retry |
| 3 | (o) | Attribute: | Retry Timeout |

DYNAMIC ATTRIBUTES:

- | | | | |
|---|-----|------------|--------------------------|
| 1 | (m) | Attribute: | Transport State |
| 2 | (o) | Attribute: | Sequence Count/Number |
| 3 | (o) | Attribute: | Previous Sequence Number |
| 4 | (o) | Attribute: | Command |
| 5 | (o) | Attribute: | Previous Command |
| 6 | (o) | Attribute: | Retry Count |
| 7 | (o) | Attribute: | Response Timeout |

SERVICES:

- | | | | |
|---|-----|-------------|-------------------|
| 1 | (o) | OpsService: | TR_Write |
| 2 | (o) | OpsService: | TR_Trigger |
| 3 | (o) | OpsService: | TR_Packet_arrived |
| 4 | (o) | OpsService: | TR_Ack_received |
| 5 | (o) | OpsService: | TR_Verify |
| 6 | (o) | OpsService: | TR_Status_updated |

7.3.3.2.2 Key attribute**Transport identifier**

The Transport identifier is generated locally by the originator Connection Manager upon connection establishment, and serves as a further reference for the user to the established connection, i.e. this transport instance.

7.3.3.2.3 Static attributes

These attributes have fixed values once the transport has been created as part of the connection establishment process. They are configured by the Connection Manager. These attributes are defined for management. It is not required for them to be visible to the communications system as accessible attributes.

Trans_class

The Trans_class attribute shall specify the transport class selected, and shall be one of NULL (also called "class 0" in both IEC 61158-5 and IEC 61158-6), DUPLICATE_DETECTION ("class 1"), ACKNOWLEDGED ("class 2"), VERIFIED ("class 3"), NON-BLOCKING ("class 4"), NON-BLOCKING_FRAGMENTING ("class 5"), or MULTIPOINT_FRAGMENTING ("class 6").

Max_Retry

Maximum number of retries for this transport instance.

Retry Timeout

If no response or acknowledgement has been received back after the transmission of a transport packet within the delay specified by the Retry Timeout, transmission of the packet should be repeated.

7.3.3.2.4 Dynamic attributes

These attributes change values based on the current operating state of the transport instance.

Transport State

Current state of the transport instance. *This shall be one of NON-EXISTENT, IDLE, READY-TO-RUN, RUNNING, WAIT-FOR-ACKNOWLEDGE or WAIT-FOR-VERIFY.*

Sequence Count/Sequence Number

Sequence Count/Number shall be used for duplicate detection. It shall be incremented for every unique message on a given transport instance ; however it shall not be incremented on a retry.

The Sequence Count/Number shall be set to zero at initialisation.

Previous Sequence Number

This attribute shall save the Sequence Count/Number used by the previous message on this transport instance (used for duplicate detection).

Command

This attribute identifies the type of message sent on this transport instance, and shall be one of:

- NULL, ONLY, FIRST, MIDDLE, LAST for messages from the sender,
- ACK, NAK, NOT-INITIALISED for messages from the receiver.

Previous Command

This parameter shall save the Command value used by the previous message on this transport instance.

Retry Count

Current number of retries for a given message. When Retry count reaches Max_Retry, the current transaction shall be aborted.

Response Timeout

The Response Timeout attribute indicates the maximum duration of the transaction. If no response is received before the time-out expires, the transaction shall be aborted.

7.3.3.2.5 Services**TR_Write**

The TR_Write service shall be used by a Client/Sender or Server application to put application data into a Transport PDU buffer (transport classes 0 to 3) or queue (transport classes 4 to 6) for further transmission.

TR_Trigger

The TR_Trigger service initiates the actual transfer of Transport PDU from a Client/Sender application. It shall be invoked when the data in the T-PDU buffer is to be sent.

TR_Packet_arrived

This service is invoked at a Server/Receiver application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU buffer or queue.

TR_Ack_received

This service is invoked at a Client application to indicate that an acknowledgement has been received. New data (sent together with the acknowledgement) may be available in the T-PDU buffer.

TR_Verify

The TR_Verify service shall be used by a Server application to send back a verification response, after reception and processing of a request message from a Client application. This verification response may or may not contain data for the Client. This service is also invoked at a Client application to indicate that a verification has been received, and that new data (sent together with the verification) may be available in the T-PDU buffer.

TR_Status_updated

This service is invoked at a Sender application to indicate that status (success/error) for a previous request has been received, and is available in the dedicated Status queue.

NOTE As for all objects, the services Create, Delete, Start, Stop and Reset are available for these transport AR's, but they are not exposed here since they are not directly accessible from the user application (only from the Connection Manager).

7.3.3.3 Class 0 transport ARs**7.3.3.3.1 Description**

Transport class 0 (Null transport type) is the simplest transport class, and can use either a point-to-point or multipoint connection. Since transport class 0 does not detect duplicate data packets, the target application is responsible for detecting them.

Class 0 is typically used to transmit or receive inputs on a multipoint connection or outputs on a point-to-point connection. Possible uses of transport class 0 include sampled inputs, standard outputs, cyclic block transfers, diagnostic events, and communication between controllers and operator interface devices.

7.3.3.3.2 Class 0 client transport AR**7.3.3.3.2.1 Formal class definition**

This is the definition of the Class 0 Client Transport (Null transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Client_0
CLASS ID:	<tbid>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1 (m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:	
1 (m) Attribute:	Transport class = NULL
DYNAMIC ATTRIBUTES:	
1 (m) Attribute:	Transport State = NON-EXISTENT IDLE RUNNING
SERVICES:	
1 (m) OpsService:	TR_Write
2 (m) OpsService:	TR_Trigger

7.3.3.3.2.2 Attributes

No specific requirement for this object.

7.3.3.3.2.3 Services**TR_Write**

The TR_Write service shall be used by the Client application to put application data into a Transport PDU buffer for further transmission.

TR_Trigger

The TR_Trigger service initiates the actual transfer of Transport PDU from the Client application. It shall be invoked when the data in the T-PDU buffer is to be sent.

7.3.3.3.3 Class 0 server transport AR

7.3.3.3.3.1 Formal class definition

This is the definition of the Class 0 Server Transport (Null transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Server_0
CLASS ID:	<tbid>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1	(m) Key Attribute: Transport identifier
STATIC ATTRIBUTES:	
1	(m) Attribute: Transport class
DYNAMIC ATTRIBUTES:	
1	(m) Attribute: Transport State = NON-EXISTENT IDLE RUNNING
SERVICES:	
3	(m) OpsService: TR_Packet_arrived

7.3.3.3.3.2 Attributes

No specific requirement for this object.

7.3.3.3.3.3 Services

TR_Packet_arrived

This service is invoked at the Server application to indicate that a new packet has been received, but it will not detect duplicate packets in this case. The new packet is available in the T-PDU buffer.

7.3.3.4 Class 1 transport ARs

7.3.3.4.1 Description

Transport class 1 (Duplicate Detection transport type), like class 0, shall allow either a point-to-point or multipoint connection. Unlike class 0, the class 1 transport allows to detect the delivery of duplicate data packets.

NOTE Class 1 is preferred to class 0 for targets because the target application does not have to perform duplicate detection, which reduces the overhead required in those end nodes that support the change of state transport trigger. Class 1 permits an efficient change of state trigger, since was designed to allow change of state data transfers.

Applications could use this transport class to distinguish between new samples and old samples that were sent to maintain the connection. To maintain the connection, a timer may be used to trigger the production of the current data in the T-PDU buffer. This may allow applications to reduce their overhead by only processing new data samples.

Possible uses of transport class 1 include sampled inputs, standard outputs, cyclic block transfers, diagnostic events, and communication between controllers and operator interface devices.

7.3.3.4.2 Class 1 client transport AR

7.3.3.4.2.1 Formal class definition

This is the definition of the Class 1 Client Transport (Duplicate Detection transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Client_1
CLASS ID:	<tdb>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1 (m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:	
1 (m) Attribute:	Transport class = DUPLICATE_DETECTION
DYNAMIC ATTRIBUTES:	
1 (m) Attribute:	Transport State = NON-EXISTENT IDLE RUNNING
2 (m) Attribute:	Sequence Count/Number
SERVICES:	
1 (m) OpsService:	TR_Write
2 (m) OpsService:	TR_Trigger

7.3.3.4.2.2 Attributes

No specific requirement for this object.

7.3.3.4.2.3 Services

TR_Write

The TR_Write service shall be used by the Client application to put application data into a Transport PDU buffer for further transmission.

TR_Trigger

The TR_Trigger service initiates the actual transfer of Transport PDU from the Client application. It shall be invoked when the data in the T-PDU buffer is to be sent.

7.3.3.4.3 Class 1 server transport AR

7.3.3.4.3.1 Formal class definition

This is the definition of the Class 1 Server Transport (Duplicate Detection transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Server_1
CLASS ID:	<tdb>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1 (m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:	
1 (m) Attribute:	Transport class = DUPLICATE_DETECTION
DYNAMIC ATTRIBUTES:	
1 (m) Attribute:	Transport State = NON-EXISTENT IDLE READY-TO-RUN RUNNING
2 (m) Attribute:	Sequence Count/Number
3 (m) Attribute:	Previous Sequence Number
SERVICES:	

3 (m) OpsService: TR_Packet_arrived

7.3.3.4.3.2 Attributes

No specific requirement for this object.

7.3.3.4.3.3 Services

TR_Packet_arrived

This service is invoked at the Server application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU buffer.

7.3.3.5 Class 2 transport ARs

7.3.3.5.1 Description

Transport class 2 (Acknowledged transport type) starts with the behaviour in class 1 and adds a return connection that acknowledges the delivery of a packet. The producer connection from the client node can be point to point or multipoint. The connection from the server to the client is point to point.

This transport class allows the application to be notified that the server has received the transmitted packet. This acknowledgement tells the client application only that the data arrived. This does not mean that the server application has read the buffer or that a new sample can be sent without overwriting the buffer.

The main advantage of this class over class 3 (Verified) is that the acknowledgement is returned immediately after the packet arrives at the consumer, while the verify class requires the application to initiate the return verification. The delays associated with receiving the acknowledgement are small, approximately twice the time that is required for a one way transmission.

Transport class 2 uses two connections: the originator-to-target connection, which can be either point-to-point or multipoint; and the return connection, which shall be point-to-point. A multipoint connection using transport class 2 shall have a point-to-point return connection corresponding to each client-to-server connection. The acknowledgement notifies the client transport instance that the consuming node of the network connection has received the packet.

If a server transport instance does not acknowledge receipt of the packet, the client transport instance triggers the client application to retransmit the most recently sent packet; if the client-to-server connection is one of the network connections of a multipoint connection, data is resent over all of the network connections of the multipoint connection. The server application can return data to the client application along with its acknowledgement.

Possible uses of transport class 2 include master/slave applications and communication between controllers and operator interface devices.

7.3.3.5.2 Class 2 client transport AR

7.3.3.5.2.1 Formal class definition

This is the definition of the Class 2 Client Transport (Acknowledged transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Client_2
CLASS ID:	<tbid>

PARENT CLASS: Generic_Transport

ACCESS ATTRIBUTES:

1 (m) Key Attribute: Transport identifier

STATIC ATTRIBUTES:

1 (m) Attribute: Transport class = ACKNOWLEDGED

2 (m) Attribute: Max_Retry

3 (m) Attribute: Retry Timeout

DYNAMIC ATTRIBUTES:

1 (m) Attribute: Transport State = NON-EXISTENT | IDLE | RUNNING
| WAIT-FOR-ACKNOWLEDGE

2 (m) Attribute: Sequence Count/Number

6 (m) Attribute: Retry Count

7 (m) Attribute: Response Timeout

SERVICES:

1 (m) OpsService: TR_Write

2 (m) OpsService: TR_Trigger

4 (m) OpsService: TR_Ack_received

7.3.3.5.2.2 Attributes

No specific requirement for this object.

7.3.3.5.2.3 Services

TR_Write

The TR_Write service shall be used by the Client application to put application data into a Transport PDU buffer for further transmission.

TR_Trigger

The TR_Trigger service initiates the actual transfer of Transport PDU from the Client application. It shall be invoked when the data in the T-PDU buffer is to be sent.

TR_Ack_received

This service is invoked at the Client application to indicate that an acknowledgement has been received. New data (sent together with the acknowledgement) may be available in the T-PDU buffer.

7.3.3.5.3 Class 2 server transport AR

7.3.3.5.3.1 Formal class definition

This is the definition of the Class 2 Server Transport (Acknowledged transport type).

FAL ASE: AR ASE

CLASS: Transport_Server_2

CLASS ID: <tbd>

PARENT CLASS: Generic_Transport

ACCESS ATTRIBUTES:

1 (m) Key Attribute: Transport identifier

STATIC ATTRIBUTES:

1 (m) Attribute: Transport class = ACKNOWLEDGED

DYNAMIC ATTRIBUTES:

- | | | | |
|---|-----|------------|--|
| 1 | (m) | Attribute: | Transport State = NON-EXISTENT IDLE READY-TO-RUN RUNNING |
| 2 | (m) | Attribute: | Sequence Count/Number |
| 3 | (m) | Attribute: | Previous Sequence Number |

SERVICES:

- | | | | |
|---|-----|-------------|-------------------|
| 1 | (m) | OpsService: | TR_Write |
| 3 | (m) | OpsService: | TR_Packet_arrived |

7.3.3.5.3.2 Attributes

No specific requirement for this object.

7.3.3.5.3.3 Services

TR_Write

The TR_Write service shall be used by the Server application to put application data into a Transport PDU buffer for further transmission together with the acknowledgement.

TR_Packet_arrived

This service is invoked at the Server application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU buffer.

7.3.3.6 Class 3 transport ARs

7.3.3.6.1 Description

Transport class 3 (Verified transport type) starts with the behaviour in class 1 and adds a return connection that verifies that the application has received the packet. The sequence count that shall be returned with the verify is the same as the sequence count sent with the data. This return connection is used to identify the verification message.

This transport class allows the application to be notified that the server application has responded to the transmitted packet. This verification tells the client application not only that the data arrived but that the server application has read the buffer and that a new sample can be sent without overwriting the buffer.

The main advantage of this class over class 2 (Acknowledgement) is that verification conveys an application response. The disadvantage is that the delay in receiving the verification can be very difficult to predict.

Transport class 3 uses two connections: the originator-to-target connection, which can be either point-to-point or multipoint; and the return connection, which shall be point-to-point. A multipoint connection using transport class 3 shall have a point-to-point return connection corresponding to each client-to-server connection. The verification notifies the client application that the server application has received and read the transmitted data. If a server application does not verify receipt of the packet, the client application retransmits the most recently sent packet; if the client-to-server connection is a multipoint one, the data is retransmitted over all of the network connections of the multipoint connection.

Possible uses of transport class 3 include change-of-state inputs and outputs, asynchronous [independent] block transfers, event acknowledgements, communication between controllers and operator interface devices, uploads, downloads, and messaging.

7.3.3.6.2 Class 3 client transport AR

7.3.3.6.2.1 Formal class definition

This is the definition of the Class 3 Client Transport (Verified transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Client_3
CLASS ID:	<tbid>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1 (m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:	
1 (m) Attribute:	Transport class = VERIFIED
2 (m) Attribute:	Max_Retry
3 (m) Attribute:	Retry Timeout
DYNAMIC ATTRIBUTES:	
1 (m) Attribute:	Transport State = NON-EXISTENT IDLE RUNNING WAIT-FOR-VERIFY
2 (m) Attribute:	Sequence Count/Number
6 (m) Attribute:	Retry Count
7 (m) Attribute:	Response Timeout
SERVICES:	
1 (m) OpsService:	TR_Write
2 (m) OpsService:	TR_Trigger
5 (m) OpsService:	TR_Verify

7.3.3.6.2.2 Attributes

No specific requirement for this object.

7.3.3.6.2.3 Services

TR_Write

The TR_Write service shall be used by the Client application to put application data into a Transport PDU buffer for further transmission.

TR_Trigger

The TR_Trigger service initiates the actual transfer of Transport PDU from the Client application. It shall be invoked when the data in the T-PDU buffer is to be sent.

TR_Verify

This service is invoked at the Client application to indicate that a verification has been received, and that new data (sent together with the verification) may be available in the T-PDU buffer.

7.3.3.6.3 Class 3 server transport AR

7.3.3.6.3.1 Formal class definition

This is the definition of the Class 3 Server Transport (Verified transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Server_3
CLASS ID:	<tbid>
PARENT CLASS:	Generic_Transport

ACCESS ATTRIBUTES:

1 (m) Key Attribute: Transport identifier

STATIC ATTRIBUTES:

1 (m) Attribute: Transport class = VERIFIED

DYNAMIC ATTRIBUTES:

1 (m) Attribute: Transport State = NON-EXISTENT | IDLE | READY-TO-RUN
| WAIT-FOR-VERIFY | RUNNING

2 (m) Attribute: Sequence Count/Number

3 (m) Attribute: Previous Sequence Number

SERVICES:

1 (m) OpsService: TR_Write

3 (m) OpsService: TR_Packet_arrived

5 (m) OpsService: TR_Verify

7.3.3.6.3.2 Attributes

No specific requirement for this object.

7.3.3.6.3.3 Services

TR_Write

The TR_Write service shall be used by the Server application to put application data into a Transport PDU buffer for further transmission together with the verification.

TR_Packet_arrived

This service is invoked at the Server application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU buffer.

TR_Verify

The TR_Verify service shall be used by the Server application to send back a verification response, after reception and processing of a request message from the Client application. This verification response may or may not contain data for the Client.

7.3.3.7 Class 4 transport ARs

7.3.3.7.1 Description

Transport class 4 (Non-blocking transport type) is similar to having two class 2 transports, one in each direction, except that the class 4 uses two Network Connections instead of four. Transport class 4 can transfer application information in both directions. The delivery is acknowledged by the receiving transport. It uses a sequence number to identify which T-PDU is being acknowledged. Retries are performed.

The main advantage of this class over using two Class 2 (Acknowledgement) transports is that class 4 uses fewer resources. The main advantage of class 4 over class 3 is that it does not block the application from sending a second request before the first response is received. This pipelining can improve the system performance in some circumstances. In addition, class 4 supports bi-directional, asynchronous data transmission, without imposing request-response type serialisation.

This transport class notifies the application that the server application has responded to the transmitted packet. This transport class shall automatically send the next item in the queue when it receives an acknowledge that the previous one has been acknowledged. The application shall be notified if the network connection fails.

Transport class 4 uses two connections: the originator-to-target connection, which shall be point-to-point; and the return connection, which also shall be point-to-point.

The receiving transport initiates and sends the acknowledgement after the consuming node of the network connection has written the packet to the T-PDU buffer, after the transport has read the packet's transport header, and after the transport has placed a copy of the data into a queue for the application. The receiver's acknowledgement signals the sending transport that the receiver-side T-PDU receive buffer is ready to accept the next data packet.

The limit to the number of application requests which can be sent before an application response is received is determined by the design of the target application. It shall typically be limited by the amount of memory which the receiving transport can allocate for holding the outstanding messages. An error shall be returned to the sending transport when a transmission attempts to exceed the limit.

Transport class 4 shall not be used with messaging. Messaging does not provide a way to match requests with responses. Use class 3 for messaging without fragmentation.

7.3.3.7.2 Class 4 sender transport AR

7.3.3.7.2.1 Formal class definition

This is the definition of the Class 4 Sender Transport (Non-blocking transport type).

FAL ASE:		AR ASE
CLASS:	Transport_Sender_4	
CLASS ID:	<tbid>	
PARENT CLASS:	Generic_Transport	
ACCESS ATTRIBUTES:		
1	(m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:		
1	(m) Attribute:	Transport class = NON-BLOCKING
2	(m) Attribute:	Max_Retry
3	(m) Attribute:	Retry Timeout
DYNAMIC ATTRIBUTES:		
1	(m) Attribute:	Transport State = NON-EXISTENT IDLE READY WAITING
2	(m) Attribute:	Sequence Count/Number
4	(m) Attribute:	Command
6	(m) Attribute:	Retry Count
7	(m) Attribute:	Response Timeout
SERVICES:		
1	(m) OpsService:	TR_Write
6	(m) OpsService:	TR_Status_updated

NOTE The TR_Trigger service is not listed here since it is only triggered internally by the transport instance timer, in order to send a retry or a "keep alive" message.

7.3.3.7.2.2 Attributes

No specific requirement for this object.

7.3.3.7.2.3 Services

TR_Write

The TR_Write service shall be used by the Sender application to put application data into a Transport PDU queue for further transmission.

TR_Status_updated

This service is invoked at a Sender application to indicate that status (success/error) for a previous request has been received, and is available in the dedicated Status queue.

7.3.3.7.3 Class 4 receiver transport AR

7.3.3.7.3.1 Formal class definition

This is the definition of the Class 4 Receiver Transport (Non-blocking transport type).

FAL ASE:	AR ASE
CLASS:	Transport_Receiver_4
CLASS ID:	<tbid>
PARENT CLASS:	Generic_Transport
ACCESS ATTRIBUTES:	
1 (m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:	
1 (m) Attribute:	Transport class = NON-BLOCKING
DYNAMIC ATTRIBUTES:	
1 (m) Attribute:	Transport State = NON-EXISTENT IDLE INITIAL READY
2 (m) Attribute:	Sequence Count/Number
3 (m) Attribute:	Previous Sequence Number
4 (m) Attribute:	Command
5 (m) Attribute:	Previous Command
SERVICES:	
3 (m) OpService:	TR_Packet_arrived

7.3.3.7.3.2 Attributes

No specific requirement for this object.

7.3.3.7.3.3 Services

TR_Packet_arrived

This service is invoked at the Receiver application to indicate that a new packet has been received. The new packet is available in the T-PDU queue.

7.3.3.8 Class 5 transport ARs

7.3.3.8.1 Description

Transport class 5 (Non-blocking, Fragmenting transport type) is similar to class 4, except that class 5 adds Fragmentation and Reassembly at the transport layer. Transport class 5 can transfer application information in both directions. The delivery of each fragment is acknowledged by the receiving transport. A sequence number identifies which T-PDU is being acknowledged. Retries are performed.

This transport class notifies the application that the server application has responded to the transmitted packet. This transport class shall automatically send the next item in the queue when it receives an acknowledgement that the previous one has been acknowledged. The application shall be notified if the network connection fails.

Transport Class 5 uses two connections: the originator-to-target connection, which shall be point-to-point, and the return connection, which also shall be point-to-point.

The receiving transport initiates and sends the acknowledgement after the consuming node of the network connection has written the packet to the T-PDU buffer, after the transport has read the packet's transport header, and after the transport has placed a copy of the data into a queue for the application. The receiver's acknowledgement signals the sending transport that the receiver-side T-PDU receive buffer is ready to accept the next data packet.

The limit to the number of application requests which can be sent before an application response is received is determined by the design of the target application. It shall typically be limited by the amount of memory. An error shall be returned when a transmission attempts to exceed the limit.

Transport class 5 shall not be used with messaging. Messaging does not provide a way to match requests with responses. Use class 3 for messaging without fragmentation.

The fragmentation protocols assume that per-connection send order is preserved throughout the system; overwrite is permissible, but changing the order of the messages is not.

Possible uses of transport class 5 include exchanging messages and responses up to 65536 bytes between two applications.

7.3.3.8.2 Class 5 sender transport AR

7.3.3.8.2.1 Formal class definition

This is the definition of the Class 5 Sender Transport (Non-blocking, Fragmenting transport type).

FAL ASE:		AR ASE
CLASS:	Transport_Sender_5	
CLASS ID:	<tbid>	
PARENT CLASS:	Generic_Transport	
ACCESS ATTRIBUTES:		
1	(m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:		
1	(m) Attribute:	Transport class = NON-BLOCKING_FRAGMENTING
2	(m) Attribute:	Max_Retry
3	(m) Attribute:	Retry Timeout
DYNAMIC ATTRIBUTES:		
1	(m) Attribute:	Transport State = NON-EXISTENT IDLE READY WAITING
2	(m) Attribute:	Sequence Count/Number
4	(m) Attribute:	Command
6	(m) Attribute:	Retry Count
7	(m) Attribute:	Response Timeout
SERVICES:		
1	(m) OpsService:	TR_Write
6	(m) OpsService:	TR_Status_updated

NOTE The TR_Trigger service is not listed here since it is only triggered internally by the transport instance timer, in order to send a retry or a "keep alive" message.

7.3.3.8.2.2 Attributes

No specific requirement for this object.

7.3.3.8.2.3 Services

TR_Write

The TR_Write service shall be used by the Sender application to put application data into a Transport PDU queue for further transmission.

TR_Status_updated

This service is invoked at a Sender application to indicate that status (success/error) for a previous request has been received, and is available in the dedicated Status queue.

7.3.3.8.3 Class 5 receiver transport AR

7.3.3.8.3.1 Formal class definition

This is the definition of the Class 5 Receiver Transport (Non-blocking, Fragmenting transport type).

FAL ASE:		AR ASE
CLASS:	Transport_Receiver_5	
CLASS ID:	<tbid>	
PARENT CLASS:	Generic_Transport	
ACCESS ATTRIBUTES:		
1	(m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:		
1	(m) Attribute:	Transport class = NON-BLOCKING_FRAGMENTING
DYNAMIC ATTRIBUTES:		
1	(m) Attribute:	Transport State = NON-EXISTENT IDLE INITIAL ASSEMBLING READY
2	(m) Attribute:	Sequence Count/Number
3	(m) Attribute:	Previous Sequence Number
4	(m) Attribute:	Command
5	(m) Attribute:	Previous Command
SERVICES:		
3	(m) OpsService:	TR_Packet_arrived

7.3.3.8.3.2 Attributes

No specific requirement for this object.

7.3.3.8.3.3 Services

TR_Packet_arrived

This service is invoked at the Receiver application to indicate that a new packet has been received. The new packet is available in the T-PDU queue.

7.3.3.9 Class 6 transport ARs

7.3.3.9.1 Description

Transport class 6 (Multipoint, Fragmenting transport type) is similar to class 3. Transport class 6 can transfer application information in both directions. Class 6 supports multicasting in the Client to Server direction. Class 6 adds Fragmentation and Reassembly at the transport layer, but only in the Client to Server (multicast) direction.

The delivery of each fragment is acknowledged by the receiving transport. A sequence number identifies the T-PDU being acknowledged. Retries are performed.

Class 6 is NOT a superset of class 5. Class 6 is blocking; that is, the next application request is not delivered until the current one is acknowledged. Functionally, class 6 shall be thought of as a fragmenting version of class 3, rather than as a multicasting version of class 5. However, class 6 reuses the transport header format from class 5, instead of the class 3 transport header.

This transport class notifies the application that the server application has responded to the transmitted packet. The application shall be notified if the network connection fails.

Transport class 6 uses multiple network connections: the Client-to-Server connection, which may be multicast; and the return connections (one per Server), which shall be point-to-point.

One significant difference between class 3 and class 6, which results from class 6 doing fragmentation: the “write” event causes the first (or only) fragment to be transmitted; the “trigger” event is only a timer function which is used to signal retries or keep-alive transmissions.

The fragmentation protocols assume that per-connection send order is preserved throughout the system; overwrite is permissible, but changing the order of the messages is not.

Possible uses of transport class 6 include delivering event notification messages (up to 65536 bytes) to multiple servers.

7.3.3.9.2 Class 6 client transport AR

7.3.3.9.2.1 Formal class definition

This is the definition of the Class 6 Client Transport (Multipoint, Fragmenting transport type).

FAL ASE:		AR ASE
CLASS:	Transport_Client_6	
CLASS ID:	<td>	
PARENT CLASS:	Generic_Transport	
ACCESS ATTRIBUTES:		
1	(m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:		
1	(m) Attribute:	Transport class = MULTIPOINT_FRAGMENTING
2	(m) Attribute:	Max_Retry
3	(m) Attribute:	Retry Timeout
DYNAMIC ATTRIBUTES:		
1	(m) Attribute:	Transport State = NON-EXISTENT IDLE READY WAITING
2	(m) Attribute:	Sequence Count/Number
4	(m) Attribute:	Command
6	(m) Attribute:	Retry Count
7	(m) Attribute:	Response Timeout
SERVICES:		
1	(m) OpsService:	TR_Write
5	(m) OpsService:	TR_Verify

NOTE The TR_Trigger service is not listed here since it is only triggered internally by the transport instance timer, in order to send a retry or a “keep alive” message.

7.3.3.9.2.2 Attributes

No specific requirement for this object.

7.3.3.9.2.3 Services

TR_Write

The TR_Write service shall be used by the Client application to put application data into a Transport PDU queue for further transmission.

TR_Verify

This service is invoked at the Client application to indicate that a verification has been received, and that new data (sent together with the verification) may be available in the T-PDU queue.

7.3.3.9.3 Class 6 server transport AR

7.3.3.9.3.1 Formal class definition

This is the definition of the Class 6 Server Transport (Multipoint, Fragmenting transport type).

FAL ASE:		AR ASE
CLASS:	Transport_Server_6	
CLASS ID:	<tdb>	
PARENT CLASS:	Generic_Transport	
ACCESS ATTRIBUTES:		
1	(m) Key Attribute:	Transport identifier
STATIC ATTRIBUTES:		
1	(m) Attribute:	Transport class = MULTIPOINT_FRAGMENTING
2	(m) Attribute:	Max_Retry
3	(m) Attribute:	Retry Timeout
DYNAMIC ATTRIBUTES:		
1	(m) Attribute:	Transport State = NON-EXISTENT IDLE INITIAL ASSEMBLING WAITING READY
2	(m) Attribute:	Sequence Count/Number
3	(m) Attribute:	Previous Sequence Number
4	(m) Attribute:	Command
5	(m) Attribute:	Previous Command
6	(m) Attribute:	Retry Count
7	(m) Attribute:	Response Timeout
SERVICES:		
1	(m) OpsService:	TR_Write
3	(m) OpsService:	TR_Packet_arrived
5	(m) OpsService:	TR_Verify

7.3.3.9.3.2 Attributes

No specific requirement for this object.

7.3.3.9.3.3 Services

TR_Write

The TR_Write service shall be used by the Server application to put application data into a Transport PDU queue for further transmission together with the verification.

TR_Packet_arrived

This service is invoked at the Server application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU queue.

TR_Verify

The TR_Verify service shall be used by the Server application to send back a verification response, after reception and processing of a request message from the Client application. This verification response may or may not contain data for the Client.

7.3.4 AR ASE services**7.3.4.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

UCMM_Create
 UCMM_Delete
 UCMM_Write
 UCMM_Abort
 TR_Write
 TR_Trigger
 TR_Packet_arrived
 TR_Ack_received
 TR_Verify
 TR_Status_updated

7.3.4.2 Common parameters**Record Number**

This parameter is the unique number associated with the UCMM transaction record to be used for the current transaction.

Transport identifier

The Transport identifier is generated locally by the originator Connection Manager upon connection establishment, and serves as a further reference for the user to the established connection, i.e. this transport instance.

7.3.4.3 UCMM_Create**7.3.4.3.1 Service overview**

The UCMM_Create service request shall attempt to create a UCMM transaction record.

7.3.4.3.2 Service primitives

The service parameters for this service are shown in Table 116.

Table 116 – UCMM_Create service parameters

Parameter name	Req	Cnf
Argument	M	
Fixed Tag	M	
Max_Retry	M	
Result		M
Record Number		M
Service Status		M

Argument

The argument contains the parameters of the service request.

Fixed Tag

DLL fixed tag service used to exchange messages on this transaction record (regular UCMM or management UCMM).

Max_Retry

Maximum number of retries for this transaction record.

Result

This selection type parameter indicates whether the service request succeeded or failed.

Record Number

This parameter is the unique number returned by the UCMM and associated with the newly created UCMM transaction record.

Service Status

This parameter provides information on the result of service execution. It shall be one of SUCCESS or CANNOT_CREATE.

7.3.4.3.3 Service procedure

The UCMM_Create service request from the user is processed locally by the UCMM. This service may also be invoked by internal objects (e.g. the Connection Manager).

A transaction record shall be created before any message can be exchanged through the UCMM. Only one message shall be outstanding for a given transaction record. If multiple outstanding messages are required by the user application, then multiple transaction records are required.

7.3.4.4 UCMM_Delete

7.3.4.4.1 Service overview

The UCMM_Delete service shall delete an existing transaction record.

7.3.4.4.2 Service primitives

The service parameters for this service are shown in Table 117.

Table 117 – UCMM_Delete service parameters

Parameter name	Req
Argument	M
Record Number	M

Argument

The argument contains the parameters of the service request.

Record Number

This parameter is the unique number associated with the UCMM transaction record to be deleted.

7.3.4.4.3 Service procedure

The UCMM_Delete service request from the user is processed locally by the UCMM. This service may also be invoked by internal objects (e.g. the Connection Manager).

7.3.4.5 UCMM_Write

7.3.4.5.1 Service overview

The UCMM_Write service shall be used to send a message over an existing (already created) transaction record.

7.3.4.5.2 Service primitives

The service parameters for this service are shown in Table 118.

Table 118 – UCMM_Write service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
Record Number	M	M(=)		
Dest_ID	M			
Source_ID		M		
UCMM_service	M			
Response Timeout	M			
Transaction_Priority	M			
App_Data	M	M(=)		
Result			M	M
Record Number			M	M(=)
Service Status				M
App_Data			M	M(=)

Argument

The argument contains the parameters of the service request.

Dest_ID

This is the MAC ID of the target (server) device for this transaction.

Source_ID

This is the MAC ID of the originating (client) device for this transaction.

UCMM_Service

The UCMM_Service parameter shall specify the delivery mechanism to use for this message and shall be one of :

- Request_With_Acknowledge (request with retry until acknowledged)
- Request_With_No_Response (request with no acknowledgement and no response)
- Request_With_No_ACK (request with retry until response – no acknowledgement)
- Request_With_No_Retry (request with no retry, and response with no retry)

NOTE 1 These values correspond to request command codes contained in the UCMM header (see IEC 61158-6).

Response Timeout

The Response Timeout attribute indicates the maximum duration of the transaction in microseconds. If no response is received before the time-out expires, the transaction shall be automatically aborted, and the Service Status parameter of the confirmation shall be TIMEOUT.

Transaction_Priority

The Transaction_priority parameter shall determine which Data Link Layer priority to use for the messages, and shall be one of LOW or HIGH.

NOTE 2 The definition of DLL priority is specified in IEC 61158-4.

App_Data

The App_Data parameter contains application data (service request parameters). The corresponding data length shall be less than 504 bytes since any UCMM packet shall fit into a single Lpacket.

Result

This selection type parameter indicates whether the service request succeeded or failed.

Service Status

This parameter provides information on the result of service execution. It shall be one of :

SUCCESS
RECORD_NOT_CREATED
PACKET_TOO_BIG
NO_ACKNOWLEDGE
ABORTED
TIMEOUT

App_Data

The App_Data parameter contains application data (service response parameters). The corresponding data length shall be less than 504 bytes since any UCMM packet shall fit into a single Lpacket.

7.3.4.5.3 Service procedure

The UCMM_Write request shall send a message to the device on the local link specified by the Dest_ID parameter. When the message arrives at the target device, this shall result in a UCMM_Write indication at the Message Router in that device (the Message Router shall receive all UCMM_Write indications). The Message Router, in turn, shall forward the message to the appropriate target object. When the object responds, the Message Router shall forward the response via the UCMM_Write response primitive. When the response arrives at the device which originated the request, a UCMM_Write confirmation shall be returned to the application object which sent the request.

7.3.4.6 UCMM_Abort

7.3.4.6.1 Service overview

The UCMM_Abort service shall be used either by the client or the server to abort a transaction.

7.3.4.6.2 Service primitives

The service parameters for this service are shown in Table 119.

Table 119 – UCMM_Abort service parameters

Parameter name	Req
Argument	M
Record Number	M

Argument

The argument contains the parameters of the service request.

Record Number

This parameter is the unique number associated with the UCMM transaction record for which the pending transaction is to be aborted.

7.3.4.6.3 Service procedure

The UCMM_Abort service request from the user is processed locally by the UCMM. This service may also be invoked by internal objects.

When called by the client, the UCMM_Abort service shall abort the transaction corresponding to the specified transaction record, which shall remove the packet from the local DLL queue if it has not yet been transmitted. When called by the server, this request shall abort the transaction, which shall suppress the response from being sent. This service need not have a corresponding confirmation since the confirmation of the original request shall serve this purpose by returning a status of ABORTED.

7.3.4.7 TR_Write**7.3.4.7.1 Service overview**

The TR_Write service shall be used by a Client/Sender or Server application to put application data into a Transport PDU buffer (transport classes 0 to 3) or queue (transport classes 4 to 6) for further transmission. The data written into the Transport PDU buffer or queue has no fixed specification or format.

NOTE Usage of PDU buffers and queues is described in IEC 61158-6.

7.3.4.7.2 Service primitives

The service parameters for this service are shown in Table 120.

Table 120 – TR_Write service parameters

Parameter name	Req
Argument	M
Transport identifier	M
App_Data	M

Argument

The argument contains the parameters of the service request.

App_Data

The App_Data parameter contains application data (service request/response parameters or user formatted data).

7.3.4.7.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.3.4.8 TR_Trigger

7.3.4.8.1 Service overview

The TR_Trigger service initiates the actual transfer of Transport PDU from a Client/Sender application. It shall be invoked when the data in the T-PDU buffer is to be sent. Depending on the type of trigger selected at connection establishment for this transport instance, this service can be requested either directly by the user application, or internally by a timer associated with the connection.

7.3.4.8.2 Service primitives

The service parameters for this service are shown in Table 121.

Table 121 – TR_Trigger service parameters

Parameter name	Req
Argument	M
Transport identifier	M

Argument

The argument contains the parameters of the service request.

7.3.4.8.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.3.4.9 TR_Packet_arrived

7.3.4.9.1 Service overview

This service is invoked at a Server/Receiver application to indicate that a new packet has been received, and whether it is a duplicate or not. The new packet is available in the T-PDU buffer or queue.

7.3.4.9.2 Service primitives

The service parameters for this service are shown in Table 122.

Table 122 – TR_Packet_arrived service parameters

Parameter name	Ind
Argument	M
Transport identifier	M
Data_Arrived	M
Duplicate_Arrived	M

Argument

The argument contains the parameters of the service indication.

Data_Arrived / Duplicate_Arrived

The Data_Arrived and Duplicate_Arrived parameters indicate whether the new packet contains new data, or whether it is just a duplicate from a previously received packet.

NOTE A Duplicate Arrived event is normally not of interest to the application; however, it might be useful in triggering a watchdog timer to indicate that the client's application is alive and triggering.

7.3.4.9.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.3.4.10 TR_Ack_received

7.3.4.10.1 Service overview

This service is invoked at a Client application to indicate that an acknowledgement has been received. New data (sent together with the acknowledgement) may be available in the T-PDU buffer.

7.3.4.10.2 Service primitives

The service parameters for this service are shown in Table 123.

Table 123 – TR_Ack_received service parameters

Parameter name	Cnf
Result	M
Transport identifier	M

Result

This parameter provides information for the receiving application.

7.3.4.10.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.3.4.11 TR_Verify

7.3.4.11.1 Service overview

The TR_Verify service shall be used by a Server application to send back a verification response, after reception and processing of a request message from a Client application. This verification response may or may not contain data for the Client. This service is also invoked at a Client application to indicate that a verification has been received, and that new data (sent together with the verification) may be available in the T-PDU buffer.

7.3.4.11.2 Service primitives

The service parameters for this service are shown in Table 124.

Table 124 – TR_Verify service parameters

Parameter name	Rsp	Cnf
Result	M	M
Transport identifier	M	M

Result

This parameter provides information for the receiving application.

7.3.4.11.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.3.4.12 TR_Status_updated

7.3.4.12.1 Service overview

This service is invoked at a Sender application to indicate that status (success/error) for a previous request has been received, and is available in the dedicated Status queue.

7.3.4.12.2 Service primitives

The service parameters for this service are shown in Table 125.

Table 125 – TR_Status_updated service parameters

Parameter name	Cnf
Result	M
Transport identifier	M

Result

This parameter provides information for the receiving application.

7.3.4.12.3 Service procedure

Since they are dependant on the transport class selected, service procedures for this service will be detailed in IEC 61158-6.

7.4 Summary of FAL classes

This annex contains a summary of the defined FAL Classes. The Class ID values have been assigned to be compatible with existing standards.

Table 126 – FAL class summary

FAL ASE	Class	Class ID
OM ASE	General	tbd
	Identity	tbd
	Assembly	tbd
	Message Router	tbd
CM ASE	Connection Manager	tbd
AR ASE	UCMM – Client	tbd
	UCMM – Server	tbd
	Generic Transport	tbd
	Transport Class 0 – Client	tbd
	Transport Class 0 – Server	tbd
	Transport Class 1 – Client	tbd
	Transport Class 1 – Server	tbd
	Transport Class 2 – Client	tbd
	Transport Class 2 – Server	tbd
	Transport Class 3 – Client	tbd
	Transport Class 3 – Server	tbd
	Transport Class 4 – Sender	tbd
	Transport Class 4 – Receiver	tbd
	Transport Class 5 – Sender	tbd
	Transport Class 5 – Receiver	tbd
	Transport Class 6 – Client	tbd
	Transport Class 6 – Server	tbd

7.5 Permitted FAL services by AR type

Table 127 below defines the valid combinations of services and AR types (which service APDUs can be sent or received by AR with the specified type). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 127 – FAL services by AR type

	Unc	Cnf	Client / Sender		Server / Receiver		UCMM	TR0 - TR6
FAL Services			req	rcv	req	rcv		
OM ASE								
Get_Attribute_Single		X	X		X		X	TR3
Get_Attributes_List		X	X		X		X	TR3
Get_Attributes_All		X	X		X		X	TR3
Set_Attribute		X	X		X		X	TR3
Set_Attributes_List		X	X		X		X	TR3
Set_Attributes_All		X	X		X		X	TR3
Create		X	X		X		X	TR3
Delete		X	X		X		X	TR3
Start		X	X		X		X	TR3
Stop		X	X		X		X	TR3
Reset		X	X		X		X	TR3
Find_Next_Instance		X	X		X		X	TR3
NOP		X	X		X		X	TR3
Apply_Attributes		X	X		X		X	TR3
Save		X	X		X		X	TR3
Restore		X	X		X		X	TR3
CM ASE								
CM_Open		X	X		X		X	
CM_Close		X	X		X		X	
CM_Unconnected_Send		X	X		X		X	
CM_Get_Connection_Data		X	X		X		X	
CM_Search_Connection_Data		X	X		X		X	
CM_Get_Object_Owner		X	X		X		X	
AR ASE								
UCMM_Create		X	X					
UCMM_Delete	X		X					
UCMM_Write		X	X		X		X	
UCMM_Abort	X		X		X			
TR_Write	X		X		X			TR0 – TR6
TR_Trigger	X		X					TR0 – TR3
TR_Verify	X			X	X			TR3, TR6
TR_Packet_Arrived	X				X			TR0 – TR6
TR_Ack_Received	X			X				TR2
TR_Status_Update	X			X	X			TR4, TR5

8 Type 3 communication model specification

8.1 DP concepts

8.1.1 User requirements of the fieldbus DP system

The typical automation system consists of one or several programmable controllers connected by I/O-systems to the machine/process. The I/O-system is mostly implemented as a set of single point connections with a well defined electrical interface such as 24V digital signals or 4 to 20mA analogue signals. The overall structure of the programmable controller is defined in IEC 61131-1 and widely used.

To switch from I/O-systems plugged in a programmable controller to decentralised I/O-systems a serial link is necessary not only for the exchange of process data, but also to transmit diagnosis, configuration data and parameters.

Furthermore many applications in factory automation require very short reaction times.

Therefore the user interface in the programmable controller for the exchange of process data should be designed in a way to allow direct access to these variables without specific function calls.

In addition a reliable data transfer is required for industrial applications.

The integration of simple field devices requires a simple protocol architecture to allow low cost implementations.

To support the integration of smart field devices acyclic communication services and alarms are needed.

In order to achieve an automation system consisting of devices from different manufacturers strict interoperability of all components is necessary.

In accordance with the IEC 61131-1 structure the decentralised structure should be invisible to an existing control program.

For efficient communication between DP-slaves, as required in motion control applications, the fieldbus provides a Publisher/Subscriber communication mechanism.

The fieldbus provides a mechanism for the synchronisation of the application of the DP-master (Class 1) and the DP-slaves with the communication cycle for highly synchronised real time applications.

For timestamping and other actions with precise timing requirements a mechanism to synchronise clocks is offered.

Field devices requires a mechanism to upload (Pull) and/or to download (Push) unstructured LR Data and a mechanism to invoke predefined functions within the DP-slave.

Fieldbus DP provides mechanism to build redundant DP-slaves, which may operate in control systems with several master implementations.

8.1.2 Features of fieldbus DP

Combining the structure defined in IEC 61131-1 with fieldbus DP the following advantages are achieved:

- Changes in the plant cause only small changes in wiring
- Critical signals can be transferred over short distances to the remote field device (sensor/actuator) and to/from the programmable controller over long distances via fieldbus DP with high accuracy
- Parameterisation and diagnosis can be done without additional wiring
- Commissioning of subparts without additional installations

Table 128 gives an overview of the requirements and features of a fieldbus DP system.

Table 128 – Requirements and features of fieldbus DP

Requirement	Feature of fieldbus DP
Short reaction time	Exchange of more than 1000 Inputs and Outputs with 32 field devices in less than 10 ms
Automation system consists of one or several programmable controller	Mono-master or Multi-master Operation
Simple field devices	Simple protocol, low cost communication interface
Reliable data transfer	Hamming Distance 4 guaranteed; Error recovery mechanism; Use of proven transmission technology;
Excellent diagnosis	Various diagnosis in Masters as well as in Slaves
User interface with direct access to the input and output data	Buffer interface for the input and output data
Smart field devices	Acyclic communication which provides a flexible and enhanced addressing scheme of data within field devices. The possibility to transfer an alarm from the field device to the programmable controller with an explicit acknowledgement.
Interoperability	Precise and complete definitions including the definition of the system behaviour
Efficient communication between DP-slaves for motion control applications	The Publisher/Subscriber communication mechanism allows direct communication between DP-slaves without interactions from the User of the DP-master (Class 1).
Synchronisation of the application of the DP-master (Class 1) and the DP-slaves	The Isochronous Mode provides a mechanism for the synchronisation of the application of the DP-master (Class 1) and the DP-slaves with the communication cycle. The cycle jitter is less than 1 µs for data rates above 1,5 Mbit/s
Mechanism to load unstructured data blocks	To load unstructured data blocks fieldbus DP provides the Load Region mechanism.
Invocation of predefined functions in the DP-slave	With the Function Invocation model predefined functions within the DP-slave may be invoked.
Synchronisation of clocks	With the Time Synchronisation an uniform Network Time (UTC) with an accuracy of less than 1 ms can be achieved.

8.1.3 DP communication model

8.1.3.1 Associations

The fieldbus communication model supports the communication of field devices with a single controlling device (e.g. programmable controller or distributed control system) via associations (see Figure 37). Furthermore, the communication between several controlling devices and their assigned field devices is also supported (see Figure 38). In addition to these basic communication models, fieldbus supports also the cyclic communication between field devices (see Figure 39).

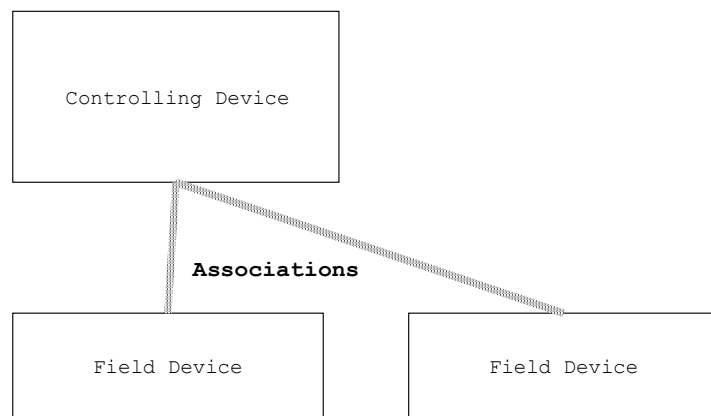


Figure 37 – Example of DP communication with a single controlling device

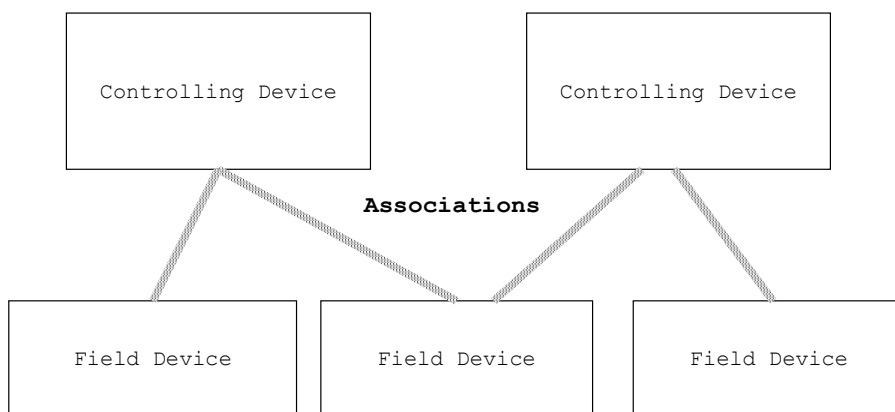


Figure 38 – Example of DP communication with several controlling devices

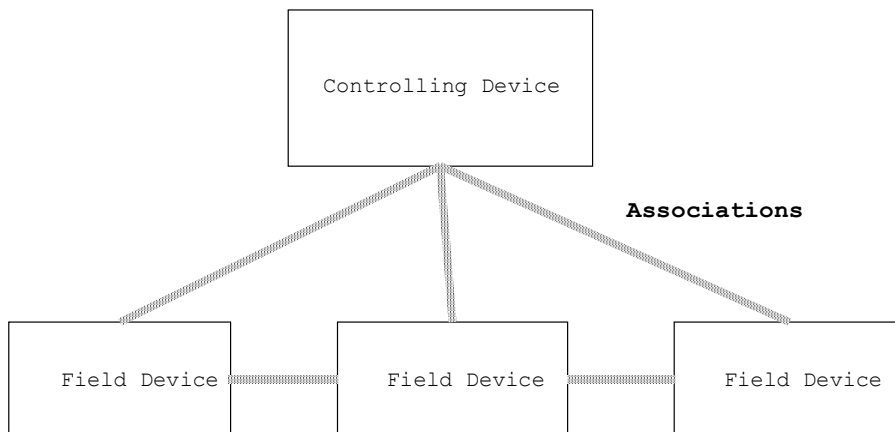


Figure 39 – Example of DP communication between field devices

8.1.3.2 Device types

8.1.3.2.1 DP-master (class 1)

The DP-master (Class 1) is a controlling device which is associated with one or more DP-slaves (field devices). The DP-master (Class 1) performs one or more of the following basic functionalities:

- cyclic exchange of I/O data with related DP-slaves
- diagnosis with the DP-slaves
- configuration of DP-slaves
- treatment of configuration and diagnosis requests of a DP-master (Class 2)

The extended functionalities of a DP-master (Class 1) are:

- acyclic access to process data of the DP-slaves
- treatment of alarms from DP-slaves
- support of the Isochronous Mode for synchronisation purposes with DP-slaves
- support of the DXB mechanism for cyclic data exchange between DP-slaves
- support of the up- and/or download of LR Data with DP-slaves
- invocation of predefined functions within DP-slaves
- support for clock synchronisation of the DP-slave's and other DP-master's clocks

8.1.3.2.2 DP-master (class 2)

The DP-master (Class 2) is a controlling device which manages provision of configuration data (parameter sets) and collection of Diagnosis data for/from a DP-master (Class 1). Additionally the DP-master (Class 2) can perform all basic and some extended communication capabilities of a DP-master (Class 1) to a DP-slave.

8.1.3.2.3 DP-slave

8.1.3.2.3.1 General

The DP-slave is a field device and performs the following activities depending on the functionality.

The basic functionalities are:

- cyclic exchange of I/O data with the assigned DP-master
- diagnosis with the assigned DP-master
- treatment of configuration requests of a DP-master

The extended functionalities of a DP-slave are:

- provision of acyclic access to process data for DP-masters
- provision of alarms to the assigned DP-master
- support of the Isochronous Mode for synchronisation purposes with the DP-master (Class 1)
- cyclic data exchange between DP-slaves using the Publisher/Subscriber communication
- support of the up- and/or download of LR Data
- support of predefined functions which may be invoked by the DP-master (Class 1) or the DP-master (Class 2)

- provision to synchronise the local clock with a DP-master's clock
- support of DP-slave Redundancy

DP-slaves are composed of one or more modules. These modules usually reflect the hardware components of the field device.

8.1.3.2.3.2 Module

The DP Application Layer uses modules to reflect the structuring of components (e.g. hardware units, logical units) inside a DP-slave. A Module may have one or more channels which represent the real structure of the Input and/or Output Data. These channels may be a further subdivision of the Input and/or Output Data object.

Each module will be addressed by a slot number (1 to 254). Numbering is without gaps, ascending, beginning with 1. If a slot is not occupied with a module, an empty slot will be registered under the corresponding slot number in the configuration. The slot number "zero" (Slot = 0) refers to the DP-slave itself. Modules contain data which is index addressable. The concrete use of slot number and index to address data within the device is manufacturer specific.

Furthermore to each module a Configuration Identifier has to be assigned. Numbering is without gaps, ascending, beginning with 0. If a slot is not occupied with a module, a Cfg Identifier with the Input and Output Data length equal zero has to be assigned in the configuration (empty slot).

A module provides one or several of the following data elements:

- Diagnosis
- I/O Data
- Alarm
- Process Data
- LR Data

In modular DP-slaves modules determine the actual configuration of the field device (see Figure 40). The number of modules is fixed according to the device specification (maximum number of modules is 244).

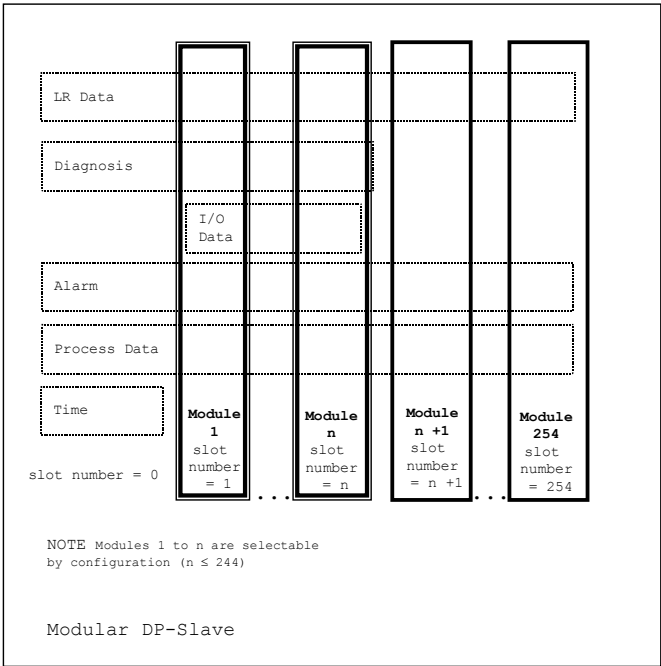


Figure 40 – DP-slave model (modular DP-slave)

In compact DP-slaves with respect to the device specification the number of modules is fixed and therefore also the configuration (for example only one module, see Figure 41).

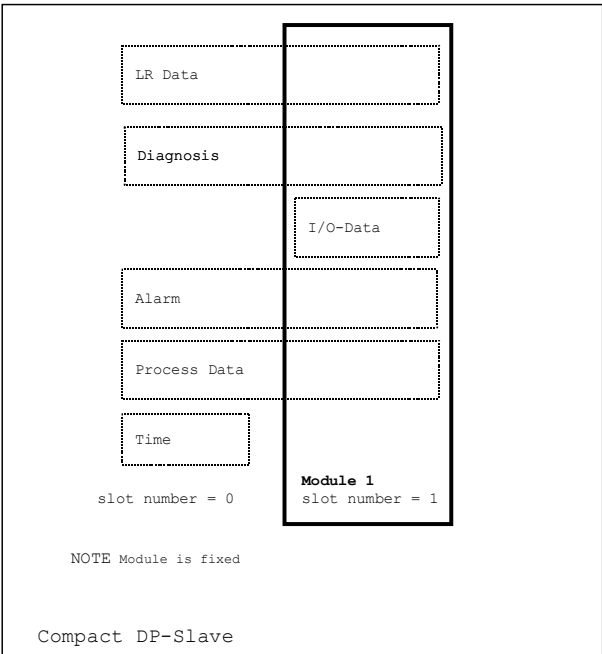


Figure 41 – DP-slave model (compact DP-slave)

8.1.3.3 Device attributes

Device attributes indicate that besides the basic functionality, extended functionality will be supported additionally.

These additional functionalities can imply certain rules for already defined data structures. A device attribute indicates that the device supports additional communication relationships or different communication behaviour.

8.1.3.4 Application process

8.1.3.4.1 Overview

In the fieldbus DP environment, an application may be partitioned and distributed to a number of devices on the network. Each of these partitions is referred to as an application process (AP)(see Figure 42). A device may have several APs. In this case each individual AP is uniquely identified by an AP Identifier (API).

Every DP-slave shall have a Default AP with API = 0. The Default AP may be related to I/O data, to diagnosis and alarms, to configuration, to process data and to LR Data. All other DP-slave APs shall be related to process data only.

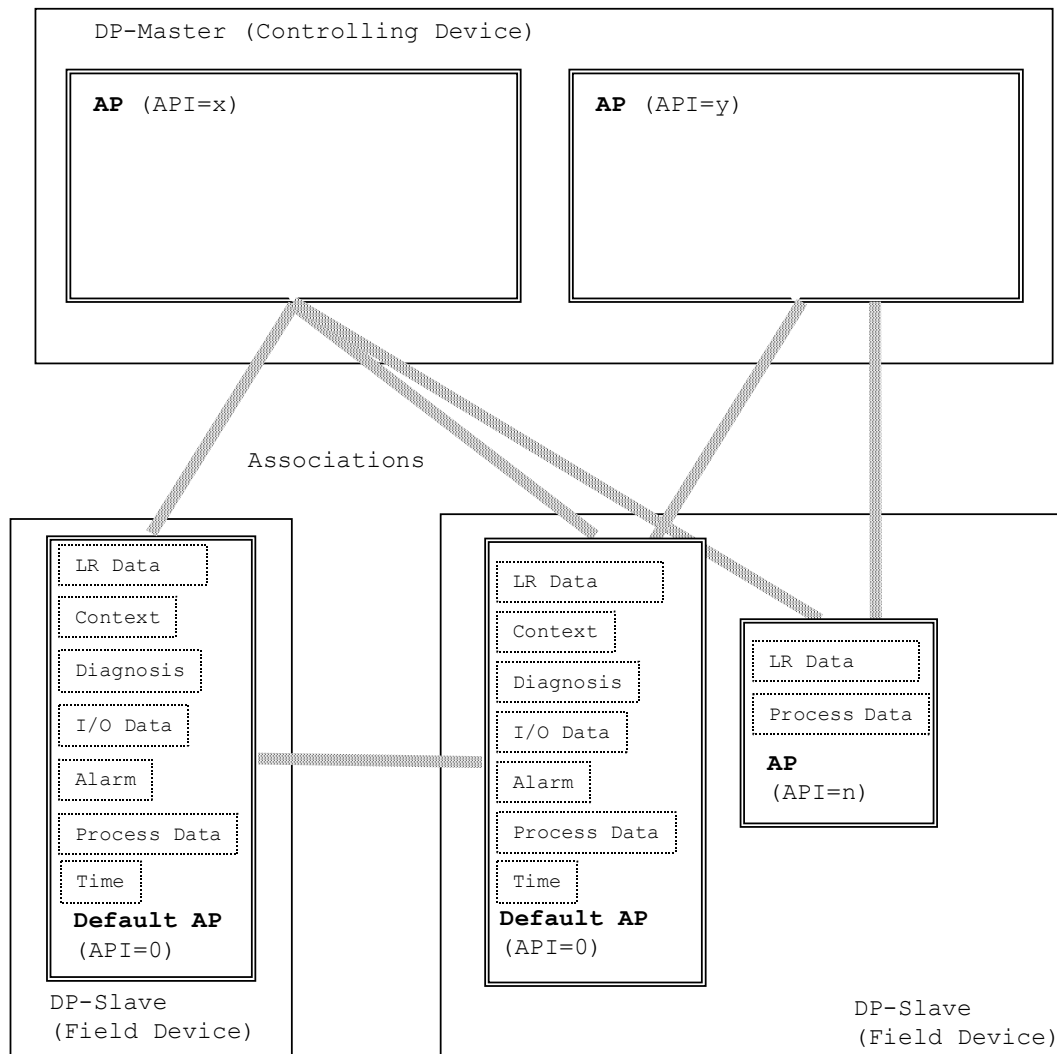


Figure 42 – Overview of application processes

An AP may be distributed to several modules. Each module within an application process is identified uniquely by a slot number (Slot = 1 to 254). The slot number "zero" (Slot = 0) refers to the AP itself within the field device.

The Context shall be related to the Default AP itself and to all its modules containing I/O Data.

The Diagnosis information may only be related to the Default AP itself and to all its modules containing I/O Data.

The I/O Data shall be related to modules of the Default AP and not to the AP itself. The number of modules with I/O Data is limited and depends on the configuration and I/O Data structure of the modules (see 8.1.3.2.3.2).

The Alarm data may be related to the Default AP itself and to all its modules.

The Process Data may be related to all APs and to all their modules.

The LR Data may be related to all APs and to all their modules.

The Time shall be related to the Default AP itself and not to its modules.

Figure 43 shows the relationship between the APs, the data elements and the modules of a DP-slave.

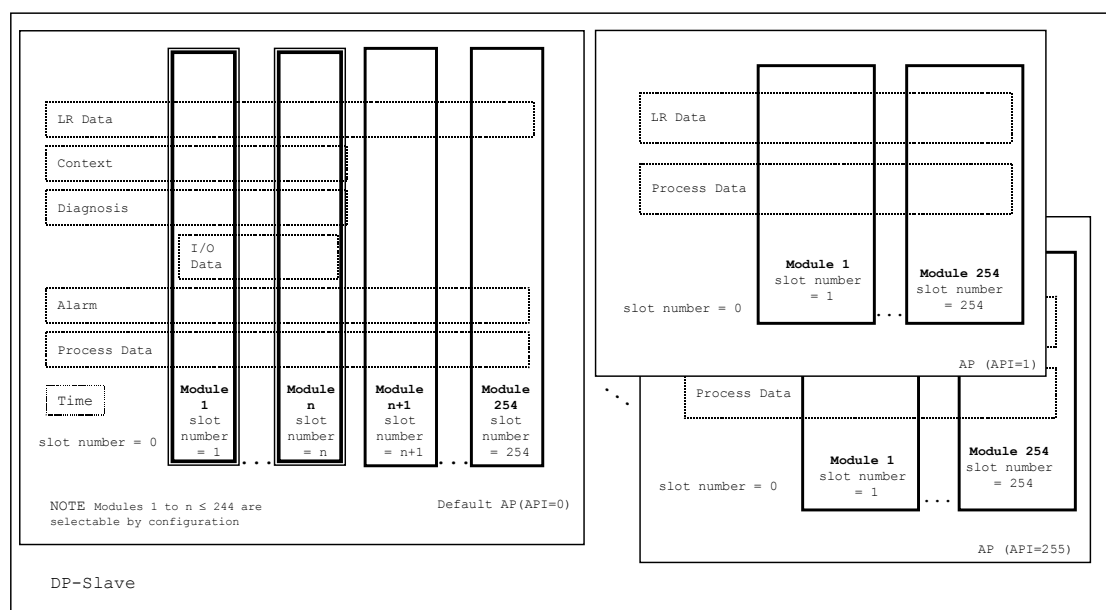


Figure 43 – DP-slave model (modular DP-slave)

The DP-master (Class 1) shall have only its Default AP. The DP-master (Class 1) shall communicate with the Default AP of a DP-slave only.

The DP-master (Class 2) may have several APs. Communicating with a DP-slave the DP-master (Class 2) may be able to address all APs of a DP-slave.

8.1.3.4.2 Application service element

An application service element (ASE), as defined in ISO/IEC 9545, is a set of application functions that provide a capability for the interworking of application processes for a specific purpose. ASEs provide a set of services for conveying requests and responses to and from application processes and their objects.

The DP Application Layer offers the following ASEs (see Figure 44):

I/O Data ASE

The I/O Data ASE provides a set of services to convey I/O data cyclically. These data always belong to those modules that have been configured in terms of the Context ASE. A multicast service provides the synchronisation of the I/O data and reports the operational state of the assigned DP-master. Optionally the I/O Data ASE offers the possibility to share the Input Data of one DP-slave with other DP-slaves.

Process Data ASE

The Process Data ASE provides a set of services to convey acyclically data. The application of the DP-master requests each transmission individually. The Process Data ASE can be related to all APs of a DP-slave.

Diagnosis ASE

The Diagnosis ASE provides services for the DP-master to fetch Diagnosis information from a DP-slave which is related to the Default AP or to its modules that are part of the configuration.

Alarm ASE

The Alarm ASE provides a set of services to convey alarms or status messages issued by the DP-slave. The assigned DP-master acknowledges the Alarms. Status messages will not have to be acknowledged by the DP-master.

Management ASE

The Management ASE provides a set of services to manage a DP-master (Class 1) by a DP-master (Class 2). The DP-master (Class 1) receives the bus parameter set or the slave parameter set. Afterwards the received sets will be activated by the DP-master (Class 2). The DP-master (Class 1) will provide the system diagnosis information if requested.

Context ASE

The Context ASE provides a set of services to

- parameterise and configure the Default AP itself and its modules containing I/O data
- transfer switchover commands for redundant structures and commands to reset MS1 or change operation mode to the DP-slave
- set the device address of a DP-slave
- to provide the real configuration of the DP-slave to be read by a DP-master (Class 2)
- and to establish or release an association between an individual AP of a DP-master (Class 2) and an individual AP of the DP-slave for acyclic communication. This association exclusively belongs to the Process Data ASE.

Load Region ASE

The Load Region ASE provides a set of services to convey LR Data acyclically. The application of the DP-master requests each transmission individually. The Load Region ASE can be related to all APs of a DP-slave.

Function Invocation ASE

The Function Invocation ASE provides a set of services to invoke stateless and/or state-oriented Function Invocation objects. The application of the DP-master has to invoke each function individually. The Function Invocation ASE can be related to all APs of a DP-slave.

Time ASE

The Time ASE provides a set of services to synchronise the clocks of several or all devices on a fieldbus segment. The application of the DP-master (Class 1) requests each synchronisation individually. The Time ASE can be related to the Default AP of a DP-slave.

Application Relationship ASE

The Application Relationship ASE (AR ASE) provides a description model for the separate AR types. This includes their transfer characteristics as well as their current communication states.

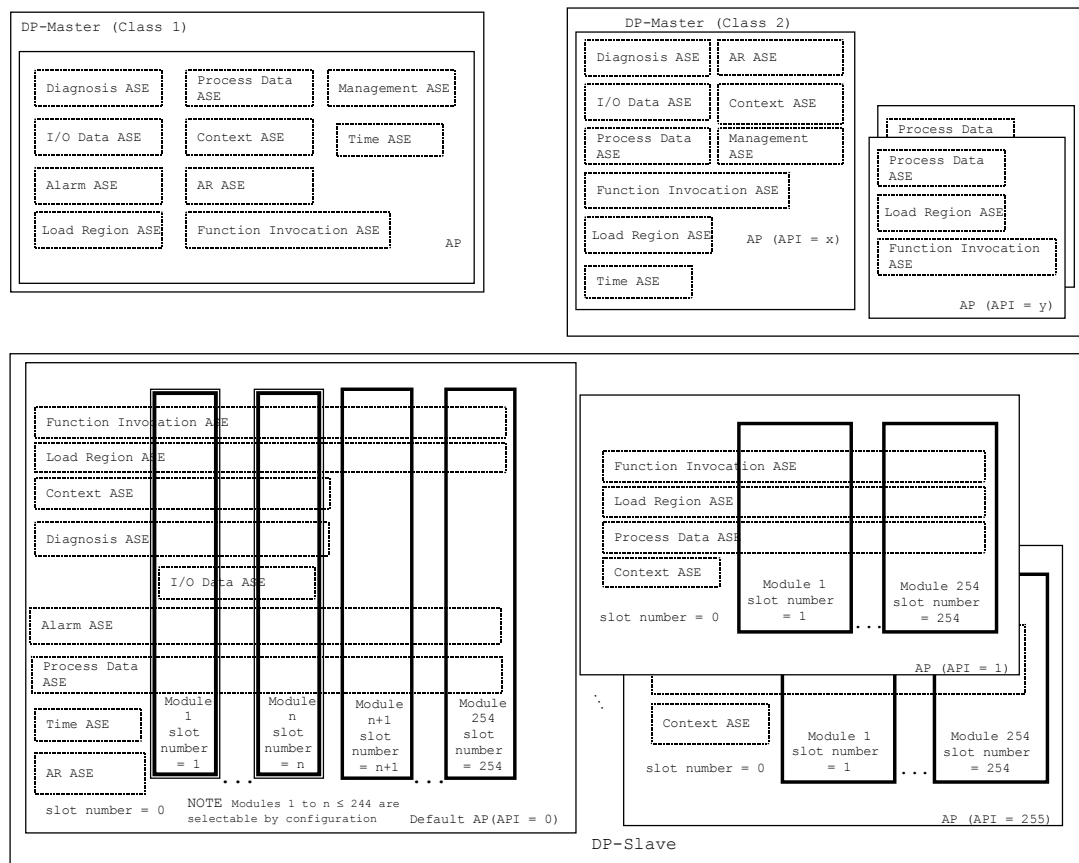


Figure 44 – Application Service Elements (ASEs)

8.1.3.4.3 Application objects

An application object (APO) is a network representation of a specific aspect of an AP. Each APO represents a set of information and processing capabilities of an AP that are accessible through services of the DP Application Layer. APOs are used to represent these capabilities to other APs in a fieldbus DP system.

In order to permit an AP to communicate with an AP of another device, APOs have to be available. Application objects (virtual objects) represent existing process objects (real objects), that an application process has made visible and accessible to the communication (see Figure 45).

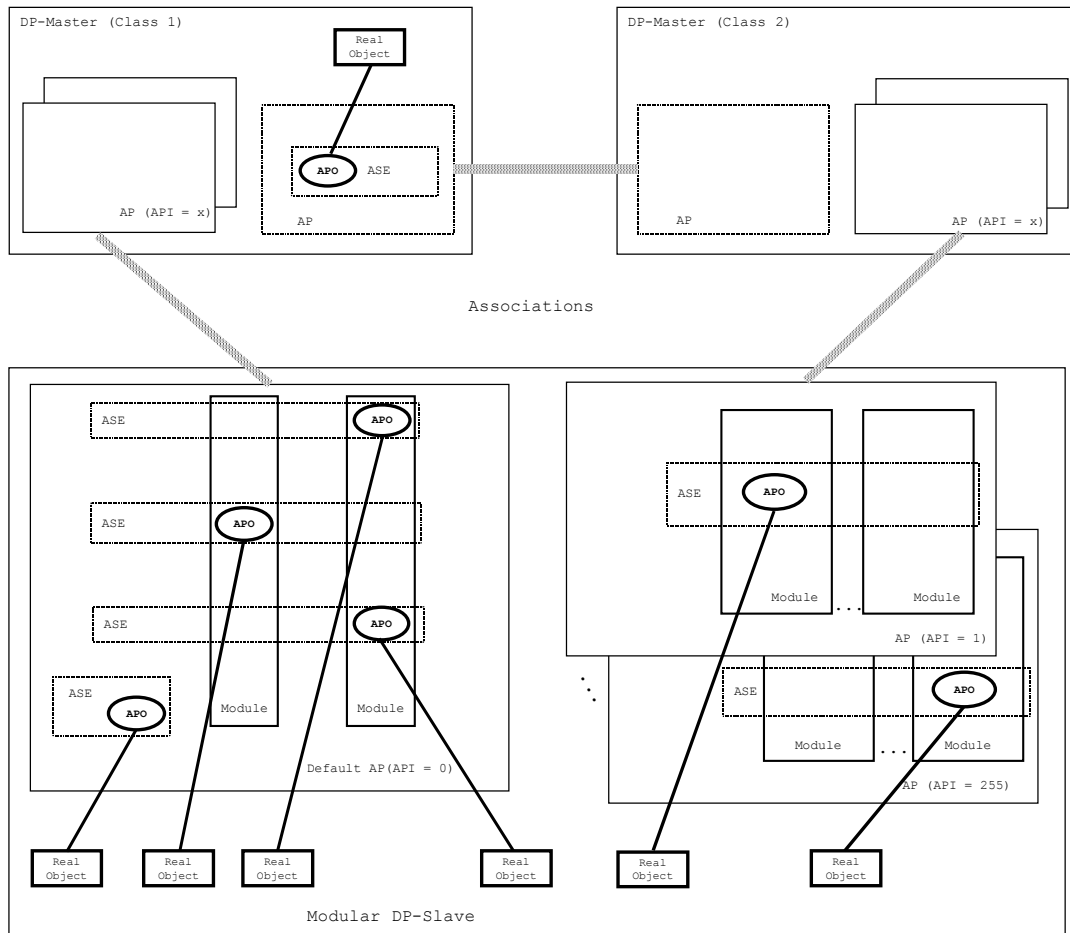


Figure 45 – Application Process with application Objects (APOs)

In Figure 46, a remote AP acting as Client may access the real object by sending requests through the APO that represents the real object. Local aspects of the AP convert between the network view (the APO) of the real object and the internal AP view of the real object.

Within the AP an APO is identified by slot and index. The address space which is defined by slot and index can be used by several APs.

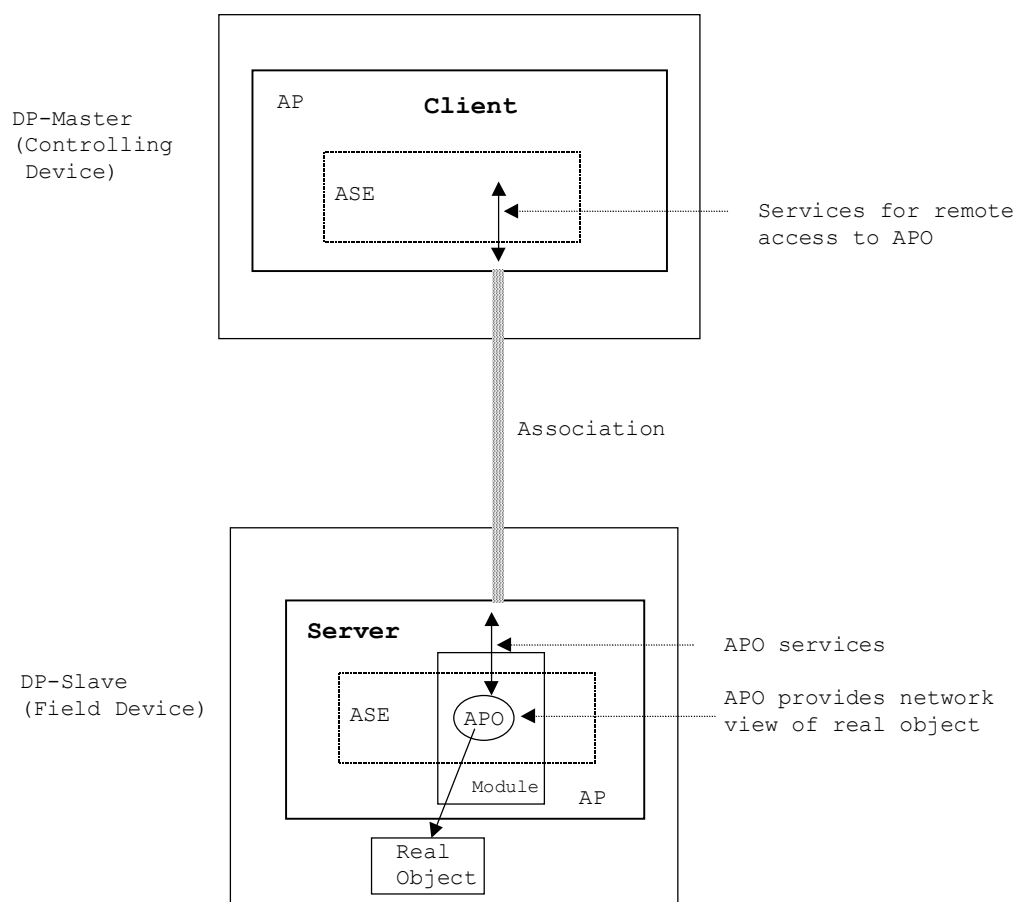


Figure 46 – Access to a remote APO

In Figure 47, a Client/Server and a Publisher/Subscriber association is shown. The Client may access the real object by sending requests through the APO that represents the real object. The Subscriber may subscribe to the whole or to a part of the remote APO. The Publisher/Subscriber association is always related to the Default AP and the I/O Data ASE. Local aspects of the AP convert between the network view (the APO) of the real object and the internal AP view of the real object.

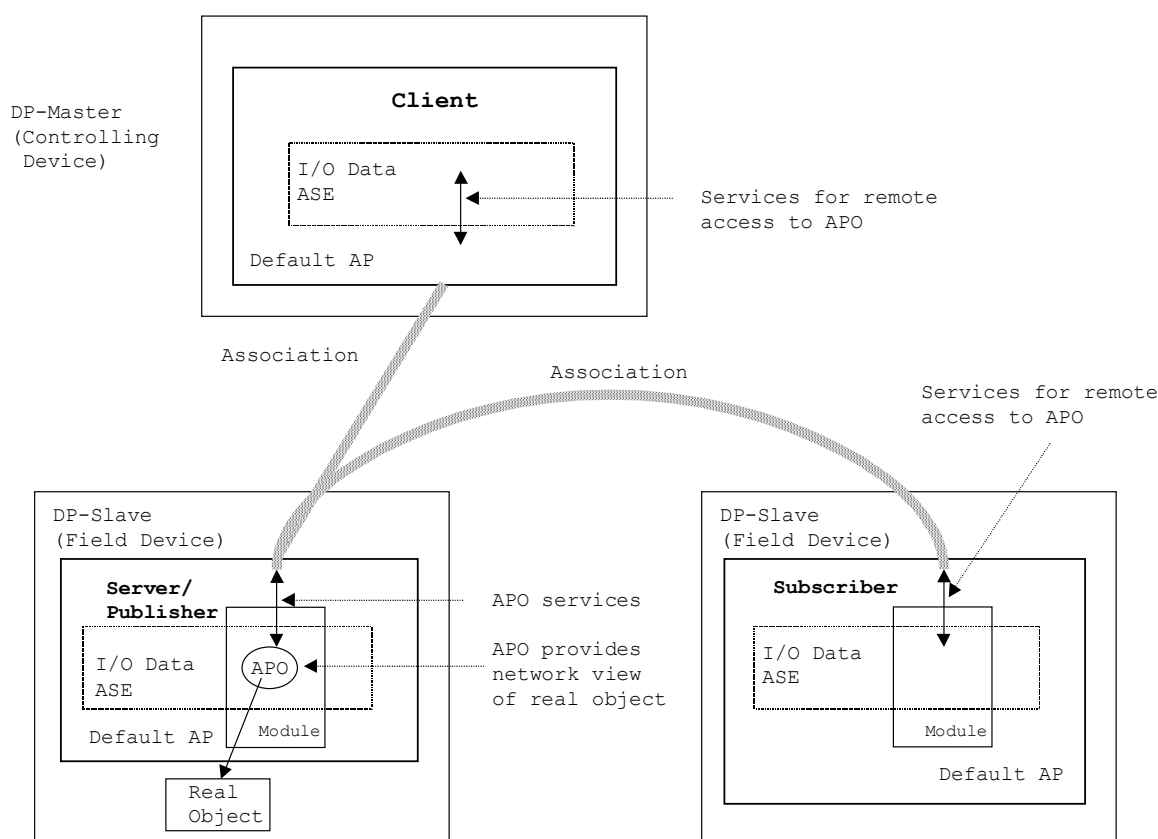


Figure 47 – Access to a remote APO for publisher/subscriber association

8.1.3.5 Application relationship

8.1.3.5.1 General

An Application Relationship (AR) is a co-operative relationship between two or more APs for the purpose of exchange of information and co-ordination of their joint operation (see Figure 48). This relationship is activated by the exchange of Application Protocol Data Units (APDU). Fieldbus DP uses different types of ARs which distinguish in their conveyance characteristic.

8.1.3.5.2 Application relationship endpoint

An AP has access to the communication using application relationship endpoints (AREP, see ISO 7498). One or more AREPs are fixed and uniquely assigned to an AP. These endpoints are addressed by the AP through an identifier (AREP ID). These identifiers are device specific and not defined by the communication itself. ARs are defined as a set of co-operating AREPs. Between two APs one or more ARs may exist, each one having unique AREPs. Figure 48 shows an example of one AR with two AREPs.

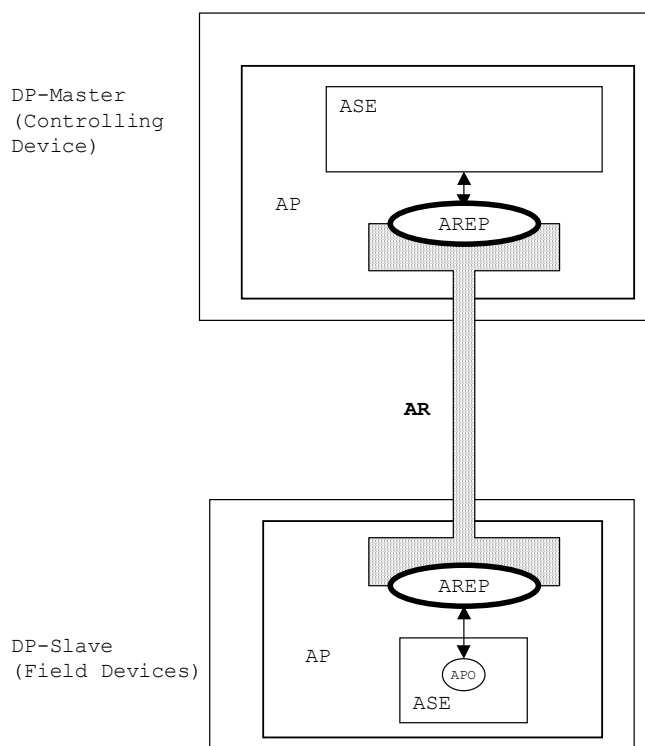


Figure 48 – Example of one AR with two AREPs

8.1.3.5.3 Overview of application relationships

The DP Application Layer offers the following AR types:

MS0: application relationship between the application process of one DP-master (Class 1) and all related DP-slaves and optional between the AP of one or several DP-master (Class 2) and all related DP-slaves and optional between the AP of one or several DP-slaves with all related DP-slaves for the following purposes:

- cyclic exchange of the I/O data with the DP-master (Class 1)
- cyclic exchange of the Input Data between DP-slaves (DXB)
- acyclic data transfer for parameterisation, configuration and diagnosis (DP-master (Class 1))
- acyclic transfer of commands to a set of field devices (DP-master (Class 1))
- cyclic transfer of synchronisation messages to a set of field devices (DP-master (Class 1))
- acyclic read of the I/O data (DP-master (Class 2))
- acyclic read of configuration information (DP-master (Class 2))
- acyclic read of diagnosis information (DP-master (Class 2))
- acyclic write of remanent parameter (DP-master (Class 2))

MS1: connection-oriented relationship between the application processes of one DP-master (Class 1) and of one related DP-slave for the following purposes:

- acyclic read and write of variables
- acyclic transfer of alarms
- up- and/or download of LR Data
- invocation of stateless and/or state-oriented functions

MS2: connection-oriented relationship between the application processes of one DP-master (Class 2) and of one related DP-slave for the purpose:

- acyclic read and write of variables
- up- and/or download of LR Data
- invocation of stateless and/or state-oriented functions

MS3: connectionless relationship between the application processes of one DP-master and of a set of related DP-slaves for the purpose:

- synchronisation of time

MM1: connectionless relationship between the application processes of one configuration device (Master Class 2) and of one related controlling device (Master Class 1) for the following purposes:

- upload and download for configuration information
- upload of diagnosis information
- activation of the previously transferred configuration

MM2: connectionless relationship between the application processes of one configuration device (Master Class 2) and of a set of related controlling devices (Master Class 1) for the purpose:

- activation of the previously transferred configurations

8.1.4 Dynamical behaviour of fieldbus DP

Before commissioning a DP system, all stations shall have been assigned an unique address. In the case of DP-slaves, this address may be set up using the MS0 AR (see 8.2.5.3.9).

All DP-slaves that have not yet been assigned an individual address, shall start with the default address 126. Only one device with this address shall be allowed on the network. A DP-master address shall not be set to the default address. A DP-master (Class 2) may access a DP-slave via the default address and may assign a specific address to that device.

For security reasons, the DP-master (Class 1) shall not exchange I/O data with a DP-slave whose address is 126.

To exchange I/O data a DP-master (Class 1) shall have a valid Master Parameter Set which may be loaded and activated by the DP-master (Class 2) via the MM1 AR and MM2 AR. The Master Parameter Set consists of several sets of configuration data and parameters, each dedicated to a DP-slave. This set of data shall contain all necessary information (for the DP system) to describe a DP-slave. Additionally, the Master Parameter Set shall include the bus parameters as well as the address assignment table which assigns each individual remote I/O data a unique address in the I/O space of the DP-master's User.

If there exists a valid Master Parameter Set in a DP-master (Class 1), this device starts to check whether the DP-slaves dedicated to that DP-master are present or not (this will be done with a read of the diagnosis of a DP-slave). After an appropriate answer the DP-master (Class 1) will set the parameters and check the configuration of that DP-slave. A DP-slave will accept a check configuration request only from the Master who sets the parameters.

After submitting these two functions correctly, a DP-master (Class 1) checks the status of the DP-slave by reading the Diagnosis information. If the configuration check was successful and parameterisation was correct, the DP-master (Class 1) will enter the user data exchange mode. A DP-slave accepts data exchange requests only from a DP-master (Class 1) which has previously submitted parameters and configuration. If a diagnosis event occurred in the application process of a DP-slave, this event is signalled by sending a response with high priority. The DP-master (Class 1) will read the diagnosis and inform its local User.

The DP-master (Class 1) sends information about his own operation mode (CLEAR, OPERATE) to his dedicated DP-slaves for synchronisation. This will be done at every change of the operation mode of the DP-master (Class 1) and additionally at regular intervals (Data Control Time/2).

Only in the data exchange mode the extended functionalities like the acyclic read and write of variables, the acyclic transfer of alarms, the up- and/or download of LR Data, the invocation of stateless and/or state-oriented functions, the DXB functionality and the Isochronous Mode are possible if supported by both the DP-master (Class 1) and the DP-slave. Errors on the acyclic communication (MS1 AR) also influence the cyclic data transfer (MS0 AR) and vice versa. This means that both communication relationships will be aborted. Each alarm has to be acknowledged by the assigned DP-master (Class 1).

Every DP-master (Class 2) can read the diagnosis information, the configuration, the input and output data (via the MS0 AR) of every DP-slave although the DP-slave is under control of a DP-master (Class 1). A DP-master (Class 2) can take a DP-slave under its control. In this case the DP-slave will stop the data exchange mode with the DP-master (Class 1). In this case the DP-master (Class 1) starts to read the diagnosis cyclically. If the DP-master (Class 2) has finished its communication with this DP-slave, the DP-slave will set the field "Master Add(ress)" to invalid. This event triggers the DP-master (Class 1) to gain control over this DP-slave again. As described above it will send parameters and configuration first.

If supported by both the DP-master (Class 2) and the DP-slave the extended functionality like the acyclic read and write of variables, the up- and/or download of LR Data and the invocation of stateless and/or state-oriented functions is possible (via the MS2 AR). The MS2 AR is independent of the MS0 AR and MS1 AR.

Fieldbus DP defines a redundancy model with the following features:

- One slave redundancy model for all redundancy structures (even for non-redundant systems)
- Scalability: Independent master, slave, and line redundancy
- Easy engineering: No additional user efforts, no complex tools necessary
- Complete monitoring of all components
- No influence on bus load and timing
- High reliability
- Short switch over time
- No loss of data during fault tolerance

A redundant DP-slave may consist of two communication interfaces with special redundancy extensions and an independent redundancy communication channel between these two communication interfaces. One of the slave interface modules acts as the primary and one as the backup slave.

The redundancy model of fieldbus DP is based on a switchover command, commands to stop, start and reset the MS1 AR, provided by the MS0 AR via the Check User Prm service to the application of the DP-slave and a Diagnosis object (Red Status) which provides information about the state of the redundant DP-slave if it is primary or backup and if a previous provided command (switchover, start, stop reset of the MS1 AR) has been processed.

During the processing of a switchover command or a stop, start or reset command the MS0 AR remains in the data exchange mode. Only the MS1 AR will be influenced, therefore a seamless takeover of a DP-slave may be accomplished.

The communication with a redundant DP-slave will be done in the following way:

MS0 AR:

The primary slave uses the MS0 AR (cyclic communication) to send and receive the I/O data. In addition the primary slave sends all diagnosis information about itself and the backup slave.

The backup slave may also send and receive values for the I/O data via the MS0 AR, but these values are not relevant and shall be ignored by the backup slave. The backup slave may transmit its own diagnosis information, but, it shall also be part of the primary diagnosis.

MS1 AR:

Only the primary slave will receive and transmit MS1 AR services. When the backup slave receives a MS1 AR service, it shall respond negatively.

MS2 AR:

Both the primary and the backup slave may receive acyclic MS2 AR services. Both slave interface modules shall process these services to allow an individual communication with the specific device. If values will be sent with MS2 AR services, these data may be sent to one slave interface module only and shall be exchanged with the other using the redundancy communication channel between the two communication interfaces. Only data, which will be held in both slave interface modules, shall be exchanged to keep them consistent.

The features for redundancy could be used for systems with backup masters as well and provide means for continuous cyclic operation.

8.1.5 Fieldbus DP status values

Table 129 shows the Status Values of the service primitives.

Table 129 – Status values of the service primitives

Short Name	Meaning
ABT_FE	Invalid request APDU received
ABT_IA	Invalid additional address information
ABT_IV	Invalid service from the User
ABT_OC	Waiting for DL-DATA-REPLY.cnf
ABT_RE	Invalid response APDU received
ABT_SE	Sequence error; service not allowed in this state
ABT_STO	Send Timeout requested was too small
ABT_TO	Timeout of the connection
AD	Access denied
DH	Positive acknowledgement for sent data, reply data with high priority available
DI	Data Incomplete
DS	Local-DL/PHY Entity is not in logical token-ring or disconnected from line
EA	Area too large (Up-/Download)
FE	Format-Error in a Request-APDU
GE	Group Error, invalid group at service request (Global Control)
IP	Invalid Parameter
IV	Invalid parameters in request
LE	Data-block-length too large (Up-/Download)
LR	Resources local not available or not sufficient
LS	Service at local DLSAP or local DLSAP not activated.
NA	Negative ack, no reaction from remote station
NC	Master Parameter Set not compatible
NE	Area non-existent
NI	Service not implemented
NO	Service in this state not possible
NR	No Response data
RDH	Response data high and no resource for sent data
RDL	Response data low and no resource for sent data
RE	Format-Error in a Response-APDU
RR	Resources of the remote-DL Entity not sufficient or not available
RS	Service or remote-address at remote-DLSAP or remote-DLSAP not activated - remote-station is no DP-Station - remote-station is not yet ready for these Service - remote-station is associated with an other Requester - optional service not available
SC	Sequence Conflict
SV	Sequence violation (second Time Master active)
SE	Sequence Error
TO	Function-Timeout expired
UE	Remote-DMPM / DL interface error

8.2 ASEs

8.2.1 Process data ASE

8.2.1.1 Overview

In the DP Application Layer environment, application processes contain data that remote applications are able to read and write. The Process Data ASE defines attributes of general purpose Process Data objects and provides a set of services used to read, and write their values. A Process Data ASE is specific for an AP and every AP may contain one Process Data ASE. Process Data objects within an AP are addressed by Slot Numbers and Indices. The Process Data object can be read and written partially or entirely.

The DP-masters (Class 1) and the DP-master (Class 2) are able to read or write the value of the Process Data object within the Default AP of a DP-slave. Process Data objects outside the Default AP of a DP-slave can only be accessed by the DP-master (Class 2).

The DP-master (Class 2) has to use the Context ASE (see 8.2.5.3.10) to establish an association to get access to the Process Data objects of a DP-slave. The Process Data object in combination with the Context ASE provides the possibility of access protection (see 8.2.1.3).

The DP-master (Class 1) can access the values of the Process Data object within the Default AP after the DP-slave has entered the data exchange mode. This association provides no access protection for the accessible Process Data object.

The access to Process Data objects is performed according to the Client/Server access model. The Client/Server model is characterised by a Client application sending a read or write request to a Server application that responds accordingly. The Server's activity is stimulated by Clients on the network.

The formal model of the Process Data ASE is presented by the Process Data class specification, containing a description of its attributes, services as well as invocations, followed by a detailed service specification.

8.2.1.2 Process data class specification

8.2.1.2.1 General

The Process Data ASE defines two Process Data object types:

- Simple objects
- Combined objects

Simple objects may have a one to one relation to real objects. Combined objects may have a one to two relation to real objects. Combined objects may be read and written with one service interaction. This type of objects may be used for communication which requires consistency between read and write or high protocol efficiency.

8.2.1.2.2 Simple process data class specification

8.2.1.2.2.1 Template

A Simple Process Data object is always related to one real object as shown in Figure 49.

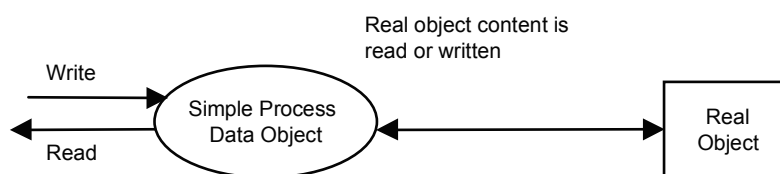


Figure 49 – Relation of a simple process data object to the real object

A Process Data object is described by the following template:

DP ASE:	Process Data ASE
CLASS:	Simple Process Data
CLASS ID:	not used
PARENT CLASS:	TOP

ATTRIBUTES:

1. (m) Key Attributes: Identifier
2. (m) Attribute: Process Data Description
3. (m) Attribute: Partial Access
4. (m) Attribute: Access Rights MS1
5. (m) Attribute: Access Rights MS2
6. (c) Constraint: Access Rights MS2 = Right to Read or Right to Read/Write
- 6.1. (m) Attribute: List of SCL Read
- 6.1.1. (m) Attribute: SCL
7. (c) Constraint: Access Rights MS2 = Right to Write or Right to Read/Write
- 7.1. (m) Attribute: List of SCL Write
- 7.1.1. (m) Attribute: SCL
8. (o) Attribute: Process Data Name
9. (o) Attribute: Local Detail

SERVICES:

1. (o) OpsService: Read
2. (o) OpsService: Write

8.2.1.2.2.2 Attributes**Identifier**

This key attribute is composed of API, Slot Number and Index to define to which Application Process Identifier, module and to which index within the specified module the Simple Process Data object belongs. This Identifier shall not be used by another Action object, Function Invocation object, Load Region object or Process Data object.

Attribute Type: Unsigned8, Unsigned8, Unsigned8

Allowed values: 0 to 254, 0 to 254, 0 to 254

Process Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Partial Access

This attribute defines whether a partial access to the Simple Process Data object is supported or not. Partial access equal TRUE means that the content of the Simple Process Data object is read or written from the first octet up to the octet defined by the length given in the service. The length given in the service shall be less or equal the value of the attribute length in case of a write access. For a read access the entire object will be read if the length given in the service is greater than the length of the object. Partial access equal FALSE means that the content of the Simple Process Data object can only be read or written entirely. That implies that the length given in the service shall be equal to the length of the object.

Attribute Type: Boolean

Access Rights MS1

This attribute defines the type of access rights defined for a MS1 AR for the Simple Process Data object.

The allowed values are shown in Table 130.

Table 130 – Access Rights MS1

Value	Meaning
0	Right to Write
1	Right to Read
2	Right to Read/Write
3	No Access

Access Rights MS2

This attribute defines the type of access rights defined for a MS2 AR for the Simple Process Data object.

The allowed values are shown in Table 131.

Table 131 – Access Rights MS2

Value	Meaning
0	Right to Write
1	Right to Read
2	Right to Read/Write
3	No Access

List of SCL Read

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for read access protection to this Simple Process Data object. The rules for accepting or rejecting a read access are described in 8.2.1.3.

Attribute Type: Unsigned8

List of SCL Write

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for write access protection to this Simple Process Data object. The rules for accepting or rejecting a write access are described in 8.2.1.3.

Attribute Type: Unsigned8

Process Data Name

This optional attribute defines the name of the Process Data object.

Attribute Type: Visible String(32)

Local Detail

This optional attribute contains additional local information about the Simple Process Data object.

Attribute Type: Octet String

8.2.1.2.2.3 Invocation of the simple process data object

For the invocation of the Simple Process Data object the following rules apply:

- Simple Process Data objects shall not exceed the total length of 240 octets. The attribute Process Data description has to be set accordingly.
- At least one service shall be allowed for the access of the object.
- At least one AR shall have access to the object.
- The access rights have to be set accordingly to the allowed service(s).

8.2.1.2.3 Combined process data class specification**8.2.1.2.3.1 Template**

A Combined Process Data object is always related to two real objects as shown in Figure 50.

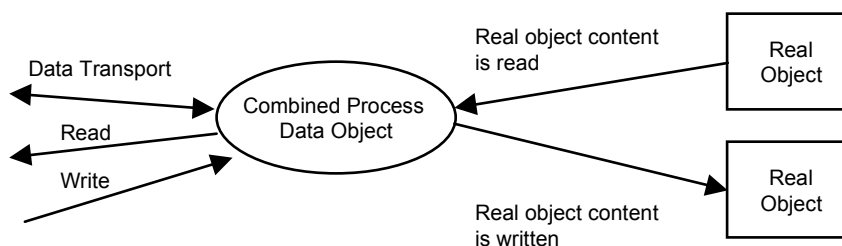


Figure 50 – Relation of a combined process data object to the real objects

A Combined Process Data object is described by the following template:

DP ASE:		Process Data ASE
CLASS:	Combined Process Data	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attributes: Identifier
2.	(m)	Attribute: Read Process Data Description
3.	(m)	Attribute: Read Partial Access
4.	(c)	Constraint: Access Rights MS2= Right to Read or Right to Read/Write
4.1.	(m)	Attribute: List of SCL Read
4.1.1.	(m)	Attribute: SCL
5.	(m)	Attribute: Write Process Data Description
6.	(m)	Attribute: Write Partial Access
7.	(c)	Constraint: Access Rights MS2 = Right to Write or Right to Read/Write
7.1.	(m)	Attribute: List of SCL Write
7.1.1.	(m)	Attribute: SCL
8.	(m)	Attribute: Access Rights MS1
9.	(m)	Attribute: Access Rights MS2
10.	(o)	Attribute: Process Data Name
11.	(o)	Attribute: Local Detail
SERVICES:		
1.	(o)	OpService: Data Transport
2.	(o)	OpService: Read
3.	(o)	OpService: Write

8.2.1.2.3.2 Attributes

Identifier

This key attribute is composed of API, Slot Number and Index to define to which Application Process Identifier, module and index within the specified module the Combined Process Data object belongs. This Identifier shall not be used by another Action object, Function Invocation object, Load Region object or Process Data object.

Attribute Type: Unsigned8, Unsigned8, Unsigned8

Allowed values: 0 to 254, 0 to 254, 0 to 254

Read Process Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. of the Process Data ASE. For data description the template given in 5.2.1 is used.

Read Partial Access

This attribute defines whether a partial read access to the Combined Process Data object is supported or not. Partial access equal TRUE means that the content of the Combined Process Data object is read from the first octet up to the octet defined by the length given in the service. The entire object will be read if the length given in the service is greater than the length of the object. Partial access equal FALSE means that the content of the Combined

Process Data object can only be read entirely. That implies that the length given in the service shall be equal to the length of the object.

Attribute Type: Boolean

List of SCL Read

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for read access protection to this Combined Process Data object. The rules for accepting or rejecting a read access are described in 8.2.1.3.

Attribute Type: Unsigned8

Write Process Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 6.2.1 is used.

Write Partial Access

This attribute defines whether a partial write access to the Combined Process Data object is supported or not. Partial access equal TRUE means that the content of the Combined Process Data object is written from the first octet up to the octet defined by the length given in the service. The length given in the service shall be less or equal the value of the attribute length. Partial access equal FALSE means that the content of the Combined Process Data object can only be written entirely. That implies that the length given in the service shall be equal to the length of the object.

Attribute Type: Boolean

List of SCL Write

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for write access protection to this Combined Process Data object. The rules for accepting or rejecting a write access are described in 8.2.1.3.

Attribute Type: Unsigned8

Access Rights MS1

This attribute defines the type of access rights defined for a MS1 AR for the Combined Process Data object.

The allowed values are shown in Table 132.

Table 132 – Access Rights MS1

Value	Meaning
0	Right to Write
1	Right to Read
2	Right to Read/Write
3	No Access

Access Rights MS2

This attribute defines the type of access rights defined for a MS2 AR for the Combined Process Data object.

The allowed values are shown in Table 133.

Table 133 – Access Rights MS2

Value	Meaning
0	Right to Write
1	Right to Read
2	Right to Read/Write
3	No Access

Process Data Name

This optional attribute defines the name of the Process Data object.

Attribute Type: Visible String(32)

Local Detail

The attribute local detail contains additional local information about the Simple Process Data object.

Attribute Type: Octet String

8.2.1.2.3.3 Invocation of the combined process data object

For the invocation of the Combined Process Data object the following rules apply:

- Combined Process Data objects shall not exceed the total length of 240 octets for both Read Process Data and Write Process Data. The attributes Read Data Description and Write Data Description have to be set accordingly.
- At least one service shall be allowed for the access of the object.
- At least one AR shall have access to the object.
- The access rights have to be set accordingly to the allowed optional service(s).

8.2.1.3 Access protection on process data objects

The Process Data ASE in combination with the Context ASE provides also access protection. This is not a protection against intentional misuse of the communication facilities of a field device but helps to protect a system for accidental erroneous use of Process Data.

The protection is accomplished by two mechanisms:

- the Initiate service will set up a security level for this CR.
- the Process Data object will have a list of SCL values for the read and write service. An access will be accepted only if the SCL of the initiate service matches (see Table 134) with an element of the SCL list for this Process Data object.

This allows the User to select a level for read and write objects. Thus a service device can use a SCL which allows access to almost all objects. A text display may have only read access to specific objects.

Table 134 – SCL matching rules

SCL in Initiate.ind	SCL in Process Data object	Condition	Result
= 0	= 0	-	access allowed
≠ 0	= 0	-	access allowed
= 0	≠ 0	-	access denied
≠ 0	≠ 0	SCL in Initiate.ind <u>not contained in</u> List of SCL at Process Data object	access denied
≠ 0	≠ 0	SCL in Initiate.ind <u>contained in</u> List of SCL at Process Data object	access allowed

8.2.1.4 Process data service specification

8.2.1.4.1 Read

This confirmed service may be used to read the value of a Process Data object. This service shall only be used in conjunction with the MS1 or MS2 AR. Table 135 shows the parameters of the service.

Table 135 – Read

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
Index	M	M(=)		
Length	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Length			M	M(=)
Data			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Decode			M	M(=)
Error Code 1			M	M(=)
Error Code 2			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Process Data object in a specific slot (typically a module).

Index

The parameter Index is used in the destination device for addressing the desired Process Data object.

Length

The parameter Length indicates the number of octets of a Process Data object that has to be read.

Result(+)

This selection type parameter indicates that the service request succeeded.

Data

The parameter Data contains the value of the object which has been read and consists of the number of octets indicated in the Length of the response. This parameter has to be composed of the data types defined in 5.2.1.

Result(-)

This selection type parameter indicates that the service request failed.

Error Decode

This parameter selects one of the following Error schemes: DPV1, FMS, Profile_Specific.

Error Code 1

In case of Error Decode equal DPV1 the Error Code 1 assumes one of the following values:

read error, module failure, version conflict, feature not supported, user specific, invalid index, invalid slot, type conflict, invalid area, state conflict, access denied, invalid range, invalid parameter, invalid type, read constrain conflict, resource busy, resource unavailable.

Error Code 2

The parameter Error Code 2 is user specific.

8.2.1.4.2 Write

This confirmed service may be used to write the value of a Process Data object. This service shall only be used in conjunction with the MS1 or MS2 AR. Table 136 shows the parameters of the service.

Table 136 – Write

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
Index	M	M(=)		
Length	M	M(=)		
Data	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Length			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Decode			M	M(=)
Error Code 1			M	M(=)
Error Code 2			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Process Data object in a specific slot (typically a module).

Index

The parameter Index is used in the destination device for addressing the desired Process Data object.

Length

The parameter Length indicates the number of octets of a Process Data object which has to be written.

Data

The parameter Data contains the value of the Process Data object which has to be written and consists of the number of octets indicated in the Length of the request. This parameter has to be composed of the data types defined in 5.2.1.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Decode

This parameter selects one of the following Error schemes: DPV1, FMS, Profile_Specific.

Error Code 1

In case of Error Decode equal DPV1 the Error Code 1 assumes one of the following values:

write error, module failure, version conflict, feature not supported, user specific, invalid index, write length error, invalid slot, type conflict, invalid area, state conflict, access denied, invalid range, invalid parameter, invalid type, write constrain conflict, resource busy, resource unavailable.

Error Code 2

The parameter Error Code 2 is user specific.

8.2.1.4.3 Data transport

This service is used to write and read the values of a single Combined Process Data object. This service may be used only via MS2 AR. Table 137 shows the parameters of the service.

With this service the data of a Process Data object are transferred

- from the DP-master to the DP-slave,
- from the DP-slave to the DP-master (Class 2) or
- from the DP-master to the DP-slave and back

Table 137 – Data transport

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
Index	M	M(=)		
Length	M	M(=)		
Data	C	C(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Length			M	M(=)
Data			C	C(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Decode			M	M(=)
Error Code 1			M	M(=)
Error Code 2			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired slot (typically a module).

Index

The parameter Index is used in the destination device to address the desired data.

Length

In the Request the parameter Length indicates the number of octets of the parameter "Data" that are transferred from the Client to the Server. In the Response the parameter Length indicates the number of bytes of the parameter "Data" that are transferred from the Server to the Client.

Data

The parameter Data contains the values transferred from the Client to the Server and vice versa and consists of the number of octets indicated in the parameter Length. This parameter shall be composed of the data types defined in 5.2.1.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Decode

This parameter selects one of the following Error schemes: DPV1, FMS, Profile_Specific.

Error Code 1

In case of Error Decode equal DPV1 the Error Code 1 assumes one of the following values:

read error, write error, module failure, version conflict, feature not supported, user specific, invalid index, write length error, invalid slot, type conflict, invalid area, state conflict, access denied, invalid range, invalid parameter, invalid type, read constrain conflict, write constrain conflict, resource busy, resource unavailable.

Error Code 2

The parameter Error Code 2 is user specific.

8.2.2 I/O data ASE**8.2.2.1 Overview**

In the DP Application Layer environment, the default application process of a DP-slave may contain I/O Data objects that shall be cyclically conveyed over the network. Therefore the I/O Data ASE defines attributes of the I/O Data objects and provides a set of services used to get and set the buffer to transport their values. The I/O Data objects are separated in an Input Data object and an Output Data object.

An Input Data object or an Output Data object shall be partitioned in Input Data or Output Data Elements which represent the I/O Data of the modules. The remote Input Data subscribed to in combination with the Output Data represent the real output data of the DP-slave.

Additional services are provided to read acyclically the values of the I/O Data objects and to indicate new values for the Input Data object and the Output Data object. Furthermore services are provided to indicate the presence and to get the content of new subscribed Input Data.

The I/O Data objects are implicitly addressed through the related services. The granularity of input or output data in a Server/Publisher is according to the correspondent MS0 Configuration Elements object. A Client is always related to the whole Input Data and/or Output Data of a Server. A Subscriber may be related to the whole Input Data or a part of the Input Data of a Publisher according to the attribute values of the DXB-Linktable object.

The I/O Data ASE uses the Client/Server and Publisher/Subscriber access model. The Client/Server model is characterised by a Client application conveying the value for the Output Data object to the Server's buffer. The Server application fetches this value by a Get Output service. The receipt of a new value is indicated by the New Output service. The Server application conveys the value of its Input Data object via the Set Input service to the Server's buffer as soon as one of the real object values changes. The Client application gets this value by the Get Input service. The transmission of the Server buffer to the Client buffer and vice versa is done asynchronously to the Get Output and Set Input services.

The Publisher/Subscriber model is characterised by DP-slaves acting as Publisher which publish the values of the Input Data objects and Subscribers which subscribe one or all values of the Input Data object of one or several Publisher (DP-slaves). The Publisher application conveys the value of its Input Data object (Simple or Extended Input Data object) via the Set Input service to the Server's buffer as soon as one of the real object values changes. The Client (DP-master (Class 1)) causes the Publisher to publish the values of the Input Data object so that these values are available at the Client side and at every Subscriber on the network. The Subscriber subscribes to as many Input Data objects as the attribute values of the DXB-Linktable or DXB-Subscribtable indicate. The receipt of new values of subscribed Input Data are indicated by the New Publisher Data service at the Subscriber. The Subscriber application gets these values by the Get Publisher Data service.

It is possible to update only parts of the values of the I/O Data objects according to the given granularity regarding the structure of the objects. Furthermore, it is also possible that a DP-slave has only either an Input Data object or an Output Data object.

Most DP-slaves support only Byte or Word structures for the Input Data object and the Output Data object. This type of devices use the Simple Input and Output Data object. However, complex devices use Floating Point values or Doubleword structures or even combinations of several data types. The extended Input/Output Data object reflect the requirements of these enhanced devices.

The values of the I/O Data objects of all or a group of DP-slaves within a given network may be synchronised by its assigned DP-master with help of the Global Control service.

The synchronisation of the values of the Input Data objects is achieved by the Freeze Command and the synchronisation of the values of the Output Data objects is achieved by the Sync Command. This synchronisation mechanism has no effect on the reception of the subscribed Input Data.

With each Freeze Command the current value of the Input Data object in the MS0 buffer is frozen in the addressed DP-slaves. If the Server application now updates the value of the Input Data object by the Set Input service then this value will not be issued to the buffer until the next Freeze Command occurs. That means, that the addressed DP-slaves will convey the "frozen" values of the Input Data objects. The synchronisation of the Input Data objects will be switched off by a Freeze Command with the value Unfreeze.

With the first Sync Command the current value of the Output Data object in the MS0 buffer is frozen in the addressed DP-slaves. With each further Sync Command the previously transmitted values of the Output Data objects will be updated to the MS0 AR buffer for access with the Get Output service. The synchronisation of the Output Data objects will be switched off by a Sync Command with the value Unsync. A detailed description is given in 8.2.8.3.2.

The values of the subscribed Input Data objects will not be synchronised with the values of the Output Data object in the Subscriber.

The DP-master (Class 1) sends information about its own operation mode to its assigned DP-slaves for synchronisation. This will be done at every change of the operation mode of the DP-master (Class 1) or occurs in a time interval which can be parameterised. If the DP-master's state is CLEAR then the values of the Output Data object shall be set to a value corresponding to a safe state.

The values of the I/O Data objects of all or a group of DP-slaves within a given network may be synchronised by the assigned DP-master (Class 1) by a special synchronisation mechanism (Isochronous Mode).

The synchronisation of the values of the Input Data objects and the synchronisation of the values of the Output Data objects is achieved by the synchronisation message (SYNCH) and fixed time settings related to the synchronisation message within the involved DP-slaves and the DP-master (Class 1).

The values of the subscribed Input Data objects in the Subscriber supporting the Isochronous Mode may also be synchronised by the Isochronous Mode.

The DP-master (Class 1) communicates via the MS0 AR with all Input Data and Output Data objects of the assigned DP-slaves simultaneously. Additionally the Client application may communicate with an Input Data object, an Output Data object, an Input Data Element or an Output Data Element of a single DP-slave.

The DP-master (Class 2) communicates via the MS0 AR with the Input Data and Output Data object of a single DP-slave.

A DP-slave, configured as Subscriber, communicates with the Input Data object(s) of one or more DP-slaves, configured as Publisher and to which he is subscribed to via the MS0 AR.

The mapping of the subscribed Input Data to all or parts of the Output Data Elements of the DP-slave may be done application specific or may be accomplished by the application of the DP-slave with the DXB-Subscribtable in conjunction with the configuration data.

The formal model of the I/O Data ASE is presented next, followed by a description of its services.

8.2.2.2 I/O data class specification

8.2.2.2.1 Simple input data class specification

8.2.2.2.1.1 Template

The Simple Input Data object is described by the following template:

DP ASE:		I/O Data ASE
CLASS: Simple Input Data		
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Implicit
2.	(m)	Attribute: List of Input Data Elements
2.1.	(m)	Attribute: Slot Number
2.2.	(m)	Attribute: Simple Input Data Description
2.2.1.	(m)	Attribute: Length
2.2.2.	(m)	Attribute: Format
2.2.3.	(m)	Attribute: Consistency
2.2.4.	(o)	Attribute: Name
SERVICES:		
1.	(m)	OpsService: Set Input
2.	(o)	OpsService: Read Input
3.	(m)	OpsService: Get Input
4.	(o)	OpsService: New Input
5.	(m)	OpsService: Get Publisher Data
6.	(o)	OpsService: New Publisher Data
7.	(m)	OpsService: Global Control
8.	(o)	OpsService: SYNCH
9.	(o)	OpsService: SYNCH Delayed
10.	(o)	OpsService: DX Finished
11.	(o)	OpsService: SYNCH Event

8.2.2.2.1.2 Attributes

Implicit

The attribute Implicit indicates that the Simple Input Data object is implicitly addressed by the services.

List of Input Data Elements

One Input Data Element is composed of the following list elements:

Slot Number

This attribute defines to which module the Input Data Element belongs.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Simple Input Data Description

This attribute consists of the following attributes.

Length

The attribute Length defines the number of octets of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Format

This attribute defines the format of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 138.

Table 138 – Format (simple input data description)

Value	Meaning
0	Byte
1	Word

Consistency

This attribute defines the Consistency of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 139.

Table 139 – Consistency (simple input data description)

Value	Meaning
0	Consistency over the defined Format of the Input Data Element
1	Consistency over the whole Input Data Element

Name

This attribute defines the name of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Visible String(32)

8.2.2.2.1.3 Invocation of the simple input data object

For the invocation of the Simple Input Data object the following rules apply:

- Simple Input Data objects shall not exceed the total length of 244 octets. The attributes List of Input Data Elements and Length have to be set accordingly.
- Only one Simple Input Data object shall be invoked in a DP-slave.
- A Simple Input Data object shall only be invoked if no Extended Input Data object (see 8.2.2.2.3) has been invoked.

8.2.2.2.2 Simple output data class specification

8.2.2.2.2.1 Template

The Simple Output Data object is described by the following template:

DP ASE:		I/O Data ASE	
CLASS:	Simple Output Data		
CLASS ID:		not used	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1.	(m)	Key Attribute:	Implicit
2.	(m)	Attribute:	List of Output Data Elements
2.1.	(m)	Attribute:	Slot Number
2.2.	(m)	Attribute:	Simple Output Data Description
2.2.1.	(m)	Attribute:	Length
2.2.2.	(m)	Attribute:	Format
2.2.3.	(m)	Attribute:	Consistency
2.2.4.	(o)	Attribute:	Name
SERVICES:			
1.	(m)	OpsService:	Set Output
2.	(o)	OpsService:	Read Output
3.	(m)	OpsService:	Get Output
4.	(o)	OpsService:	New Output
5.	(m)	OpsService:	Global Control
6.	(o)	OpsService:	SYNCH
7.	(o)	OpsService:	SYNCH Delayed
8.	(o)	OpsService:	DX Finished
9.	(o)	OpsService:	SYNCH Event

8.2.2.2.2.2 Attributes

Implicit

The attribute Implicit indicates that the Simple Output Data object is implicitly addressed by the services.

List of Output Data Elements

One Output Data Element is composed of the following list elements:

Slot Number

This attribute defines to which module the Output Data Element belongs.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Simple Output Data Description

This attribute consists of the following attributes.

Length

The attribute Length defines the number of octets of the Output Data Element. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Format

This attribute defines the format of the Output Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 140.

Table 140 – Format (simple output data)

Value	Meaning
0	Byte
1	Word

Consistency

This attribute defines the Consistency of the Output Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 141.

Table 141 – Consistency (simple output data)

Value	Meaning
0	Consistency over the defined Format of the Output Data Element
1	Consistency over the whole Output Data Element

Name

This optional attribute defines the name of the Output Data Element.

Attribute Type: Visible String(32)

8.2.2.2.2.3 Invocation of the simple output data object

For the invocation of the Simple Output Data object the following rules apply:

- Simple Output Data objects shall not exceed the total length of 244 octets. The attributes List of Output Data Elements and Length have to be set accordingly.
- Only one Simple Output Data object shall be invoked in a DP-slave.
- A Simple Output Data object shall only be invoked if no Extended Output Data object (see 8.2.2.2.4) has been invoked.

8.2.2.2.3 Extended input data class specification**8.2.2.2.3.1 Template**

The Extended Input Data object is described by the following template:

DP ASE:		I/O Data ASE
CLASS:	Extended Input Data	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of Input Data Elements
2.1.	(m) Attribute:	Slot Number
2.2.	(m) Attribute:	Selector of Data Type
2.2.1.	(s) Attribute:	Extended Input Data Description
2.2.2.	(s) Attribute:	Simple Input Data Description
2.2.2.1.	(m) Attribute:	Length
2.2.2.2.	(m) Attribute:	Format
2.2.2.3.	(m) Attribute:	Consistency
2.3.	(o) Attribute:	Name
SERVICES:		
1.	(m) OpService:	Set Input
2.	(o) OpService:	Read Input
3.	(m) OpService:	Get Input
4.	(o) OpService:	New Input
5.	(m) OpService:	Get Publisher Data

- 6. (o) OpsService: New Publisher Data
- 7. (m) OpsService: Global Control
- 8. (o) OpsService: SYNCH
- 9. (o) OpsService: SYNCH Delayed
- 10. (o) OpsService: DX Finished
- 11. (o) OpsService: SYNCH Event

8.2.2.2.3.2 Attributes

Implicit

The attribute Implicit indicates that the Extended Input Data object is implicitly addressed by the services.

List of Input Data Elements

One Input Data Element is composed of the following list elements:

Slot Number

This attribute defines to which module the Input Data Element belongs.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Selector of Data Type

This attribute defines whether this module is described according to the Extended Input Data Description or according to the Simple Input Data Description. In case of an Extended Input Data Description, consistency is over the whole Input Data Element.

Extended Input Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Simple Input Data Description

This attribute consists of the following attributes.

Length

The attribute Length defines the number of octets of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Format

This attribute defines the format of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 142.

Table 142 – Format (extended input data)

Value	Meaning
0	Byte
1	Word

Consistency

This attribute defines the Consistency of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 143.

Table 143 – Consistency (extended input data)

Value	Meaning
0	Consistency over the defined Format of the Input Data Element
1	Consistency over the whole Input Data Element

Name

This optional attribute defines the name of the Input Data Element. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Visible String(32)

8.2.2.2.3.3 Invocation of the extended input data object

For the invocation of the Extended Input Data object the following rules apply:

- Extended Input Data objects shall not exceed the total length of 244 octets. The attributes List of Input Data Elements, Input Data Description respectively Format and Length have to be set accordingly.
- Only one Extended Input Data object shall be invoked in a DP-slave.
- An Extended Input Data object shall only be invoked if no Simple Input Data object has been invoked.

8.2.2.2.4 Extended output data class specification

The Extended Output Data object is described by the following template:

DP ASE:		I/O Data ASE
CLASS:	Extended Output Data	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of Output Data Elements
2.1.	(m) Attribute:	Slot Number
2.2.	(m) Attribute:	Selector of Data Type
2.2.1.	(s) Attribute:	Extended Output Data Description
2.2.2.	(s) Attribute:	Simple Output Data Description
2.2.2.1.	(m) Attribute:	Length
2.2.2.2.	(m) Attribute:	Format
2.2.2.3.	(m) Attribute:	Consistency
2.3.	(o) Attribute:	Name
SERVICES:		
1.	(m) OpService:	Set Output
2.	(o) OpService:	Read Output
3.	(m) OpService:	Get Output
4.	(o) OpService:	New Output
5.	(m) OpService:	Global Control
6.	(o) OpService:	SYNCH
7.	(o) OpService:	SYNCH Delayed
8.	(o) OpService:	DX Finished
9.	(o) OpService:	SYNCH Event

8.2.2.2.4.1 Attributes**Implicit**

The attribute Implicit indicates that the Extended Output Data object is implicitly addressed by the services.

List of Output Data Elements

One Output Data Element is composed of the following list elements:

Slot Number

This attribute defines to which module the Output Data Element belongs.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Selector of Data Type

This attribute defines whether this module is described according to the Extended Output Data Description or according to the Simple Output Data Description. In case of an Extended Output Data Description, consistency is over the whole Output Data Element.

Extended Output Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Simple Output Data Description

This attribute consists of the following attributes.

Length

The attribute Length defines the number of octets of the Output Data Element. The value of this attribute is determined by the corresponding configuration data. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Format

This attribute defines the format of the Output Data Element. The value of this attribute is determined by the corresponding configuration data. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 144.

Table 144 – Format (extended output data)

Value	Meaning
0	Byte
1	Word

Consistency

This attribute defines the Consistency of the Output Data Element. The value of this attribute is determined by the corresponding configuration data. The value of this attribute is determined by the corresponding configuration data.

The allowed values are shown in Table 145.

Table 145 – Consistency (extended output data)

Value	Meaning
0	Consistency over the defined Format of the Output Data Element
1	Consistency over the whole Output Data Element

Name

This optional attribute defines the name of the Output Data Element. The value of this attribute is determined by the corresponding configuration data. The value of this attribute is determined by the corresponding configuration data.

Attribute Type: Visible String(32)

8.2.2.2.4.2 Invocation of the extended output data object

For the invocation of the Extended Output Data object the following rules apply:

- Extended Output Data objects shall not exceed the total length of 244 octets. The attributes List of Output Data Elements, Output Data Description respectively Format and Length have to be set accordingly.
- Only one Extended Output Data object shall be invoked in a DP-slave.
- An Extended Output Data object shall only be invoked if no Simple Output Data object has been invoked.

8.2.2.3 I/O data service specification

8.2.2.3.1 Set input

This service shall be used by a DP-slave to pass the value of a Simple Input Data object or an Extended Input Data object to the DP Application Layer. This service shall only be used in conjunction with the MS0 AR.

By this service the application process of the DP-slave updates the local buffer. According to the cyclic buffer to buffer transportation characteristics of the used application relationship the following behaviour is possible:

- The Set Input requests come faster than the contents of the buffers are transported over the network to the Client and Subscribers. In this case not each value of the Input Data object provided with the Set Input service is conveyed over the network. Only the latest value is transported, others are locally overwritten.
- The Set Input requests are issued slower than the contents of the buffers are transported over the network to the Client and Subscribers. In this case each value of the Input Data object provided with the Set Input service is conveyed more than once over the network.
- If the Set Input requests are issued synchronous with the transport of the contents of the buffers each value of the Input Data object provided with the Set Input service is conveyed over the network.

Table 146 shows the parameters of the service.

Table 146 – Set Input

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Input Data	M	
Result(+)		S
AREP		M
Result(-)		S
AREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Input Data

This parameter contains the value of the Input Data object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.2.3.2 Read input

By this service any DP-master (Class 2) can read the value of the Input Data object of a DP-slave. The service will only deliver a positive result if the DP-slave is already in the data exchange mode with its assigned DP-master (Class 1). This service shall only be used in conjunction with the MS0 AR. Table 147 shows the parameters of the service.

Table 147 – Read Input

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Input Data		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Input Data

This parameter contains the requested value of the Input Data object.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provides detailed information about the occurred error.

Allowed values: DS, NA, RS, UE, NR, RE

8.2.2.3.3 Get input

This service shall only be used in conjunction with the MS0 AR. It is used by a DP-master (Class 1) to read:

- the values of all Input Data objects of the assigned DP-slaves if the parameter CREP and Slot Number have not been used in the Get Input service request
- the value of the Input Data object of a single assigned DP-slave if the parameter Slot Number has not been used in the Get Input service request
- the value of an Input Data Element of a single assigned DP-slave if the parameter CREP and Slot Number have been used in the Get Input service request

This service may be acknowledged negatively if the DP-master (Class 1) operates in the Isochronous Mode in the following cases:

- if the service request is made not within the allowed window of interactions of the Enhanced Synchronised Isochronous Mode (SE)
- if the service request is made between the service Set Output with the parameter Final set to TRUE and the next SYNCH indication in case of Buffered Synchronised Isochronous Mode (SE)
- if the addressed DP-slave is deactivated (SC)

Table 148 shows the parameters of the service.

Table 148 – Get Input

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	U	
Slot Number	U	
Result(+)		S
AREP		M
CREP		C
Slot Number		C
Input Data		M
Result(-)		S
AREP		M
CREP		C
Slot Number		C
Status		C

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

If this parameter is present then the Input Data object of a single DP-slave is addressed.

Slot Number

If this parameter is present then an Input Data Element of a single DP-slave is addressed.
If this parameter is used the parameter CREP shall also be present.

Allowed values: 1 to 244

Result(+)

This selection type parameter indicates that the service request succeeded.

Input Data

This parameter contains the selected value.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provides detailed information about the occurred error.

Allowed values: SC, SE

8.2.2.3.4 New input

The New Input service signals to the application of the DP-master (Class 1) that new values of all Input Data objects of a DP-slaves which are related to the MS0 AR are present. This service shall only be used in conjunction with the MS0 AR. Table 149 shows the parameters of the service.

Table 149 – New Input

Parameter name	Ind
Argument	M
AREP	M
CREP	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

CREP

This parameter is the local identifier of of a single DP-slave.

8.2.2.3.5 Set output

By this service the Client application process of the DP-master (Class 1) requests the conveyance of:

- the values of all Output Data objects of the assigned DP-slaves if the parameter CREP and Slot Number have not been used in the Set Output service request
- the value of the Output Data object of a single assigned DP-slave if the parameter Slot Number has not been used in the Set Output service request
- the value of an Output Data Element of a single assigned DP-slave if the parameter CREP and Slot Number have been used in the Set Output service request

This service shall only be used in conjunction with the MS0 AR.

According to the cyclic buffer to buffer transportation characteristics the following behaviour is possible:

- The Set Output requests come faster than the contents of the buffers are transported over the network. In this case only the values provided with the latest Set Output service are conveyed over the network.
- The Set Output requests issued slower than the contents of the buffers are transported over the network. In this case the values provided with each Set Output service are conveyed more than once over the network.
- If the Set Output requests are issued synchronous with the transport of the contents of the buffers the values provided with each Set Output service are conveyed once over the network.

This service may be acknowledged negatively if the DP-master (Class 1) operates in the Isochronous Mode in the following cases:

- if the service request is made not within the allowed window of interactions of the Enhanced Synchronised Isochronous Mode (SE)
- if the service request is made between the service Set Output with the parameter Final set to TRUE and the next SYNCH indication in case of Buffered Synchronised Isochronous Mode (SE)
- if the addressed DP-slave is deactivated (SC)

Table 150 shows the parameters of the service.

Table 150 – Set Output

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	U	
Slot Number	U	
Output Data	M	
Final	M	
Result(+)		S
AREP		M
CREP		C
Slot Number		C
Result(-)		S
AREP		M
CREP		C
Slot Number		C
Status		C

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

If this parameter is present then the Output Data object of a single DP-slave is addressed.

Slot Number

If this parameter is present then an Output Data Element of a single DP-slave is addressed. If this parameter is used the parameter CREP shall also be present.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Output Data

This parameter contains the selected value.

Final

This parameter indicates to the DP Application Layer to change the intermediate buffer of the Input Data and of the Output Data of the DP-slaves. The DP Application Layer changes these buffers after the DX Finished indication and before the next SYNCH indication of the Isochronous DP cycle in case of Buffered Synchronised Isochronous Mode.

The allowed values are shown in Table 151.

Table 151 – Final

Value	Meaning
TRUE	change of the intermediate buffer of the Input Data and of the Output Data shall be done
FALSE	change of the intermediate buffer of the Input Data and of the Output Data is not requested

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provides detailed information about the occurred error.

Allowed values: SC, SE

8.2.2.3.6 Read output

By this service any DP-master (Class 2) can read acyclic the value of the Output Data object of a DP-slave. The service will only deliver a positive result if the DP-slave is already in the data exchange mode with its assigned DP-master (Class 1). This service shall only be used in conjunction with the MS0 AR.

If the DP-master (Class 1) changes the operation mode to CLEAR the DP-slave shall set the value of the Output Data object to a value corresponding to a safe state (e.g. zero, last or default value). In this case the value received with this service may not correspond to the real value of the Output Data object.

Table 152 shows the parameters of the service.

Table 152 – Read Output

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Output Data		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Output Data

This parameter contains the requested value of the Output Data object.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provides detailed information about the occurred error.

Allowed values: DS, NA, RS, UE, NR, RE

8.2.2.3.7 Get output

This service shall be used by a DP-slave to read the value for its Simple Output Data object or Extended Output Data object from the DP Application Layer. This service shall only be used in conjunction with the MS0 AR.

Table 153 shows the parameters of the service.

Table 153 – Get Output

Parameter name	Req	Cnf
Argument AREP	M M	
Result(+) AREP		S M
Output Data		M
Clear Flag		M
New Flag		M
Result(-) AREP		S M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Output Data

This parameter contains the value for the Output Data object.

Clear Flag

This parameter indicates whether the value for the Output Data object is valid or cleared.

The allowed values are shown in Table 154.

Table 154 – Clear Flag

Value	Meaning
0	value for the Output Data object valid (DP-master in the state OPERATE)
1	value for the Output Data cleared (DP-master in the state CLEAR)

New Flag

This parameter indicates whether a new value for the Output Data object is received or not.

The allowed values are shown in Table 155.

Table 155 – New Flag

Value	Meaning
0	no new value received
1	new value received

Result(-)

This selection type parameter indicates that the service request failed.

8.2.2.3.8 New output

This service indicates to the application of the DP-slave that a new value for its Simple Output Data object or Extended Output Data object is present. Table 156 shows the parameters of the service.

Table 156 – New Output

Parameter name	Ind
Argument	M
AREP	M
Clear Flag	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

Clear Flag

This parameter indicates whether the value for the Output Data object is valid or cleared.

The allowed values are shown in Table 157.

Table 157 – Clear Flag

Value	Meaning
0	value for the Output Data object valid (DP-master in the state OPERATE)
1	value for the Output Data cleared (DP-master in the state CLEAR)

8.2.2.3.9 Global control

This service is used by the DP-master (Class 1) for the synchronisation of the values of the Input Data objects (Freeze Command) and the synchronisation of the values of the Output Data objects (Sync Command) of a group or all assigned DP-slaves. This service shall only be used in conjunction with the MS0 AR.

If the DP-master (Class 1) operates in the Isochronous Mode with all or a group of its assigned DP-slaves group 8 of the parameter Group Select of the Global Control service is no longer a valid group value for this parameter.

Furthermore this service is used by the DP-master (Class 1) to send information about its own operation mode (Clear Command) to all assigned DP-slaves.

Table 158 shows the parameters of the service.

Table 158 – Global Control

Parameter name	Req	Ind	Cnf
Argument	M	M(=)	
AREP	M	M(=)	
Clear Command		M	
Sync Command	M	M(=)	
Freeze Command	M	M(=)	
Group Select	M	M(=)	
Result(+)			S
AREP			M
Result(-)			S
AREP			M
Status			M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Clear Command

This parameter contains the Clear Command.

The allowed values are shown in Table 159.

Table 159 – Clear Command

Value	Meaning
0	DP-master (Class 1) in the state OPERATE: The values for the Output Data objects are valid.
1	DP-master (Class 1) in the state CLEAR: The values for the Output Data objects shall be set to a value corresponding to a safe state.

Sync Command

This parameter controls the Sync Operation.

The allowed values shown in Table 160.

Table 160 – Sync Command

Value	Meaning
0	no action
1	Sync
2	Unsync

Freeze Command

This parameter controls the Freeze Operation.

The allowed values are shown in Table 161.

Table 161 – Freeze Command

Value	Meaning
0	no action
1	Freeze
2	Unfreeze

Group Select

This parameter determines which group(s) of assigned DP-slaves shall be addressed. The control command takes effect if the AND operation between the attribute Group Identifier of the ARL object of the DP-slave and the Group Select parameter results in a value different from zero. If the parameter Group Select is zero all assigned DP-slaves are addressed.

If the DP-master (Class 1) operates in the Isochronous Mode group 8 is not allowed for this parameter. The service request shall be confirmed negatively with an appropriate status value (GE).

Result(+)

This selection type parameter indicates that the service request has been transmitted.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provides detailed information about the occurred error.

Allowed values: DS, NO, IV, GE

8.2.2.3.10 New publisher data

This service indicates to the application of the DP-slave that a new value from a subscribed Input Data object is present. Table 162 shows the parameters of the service.

Table 162 – New publisher data

Parameter name	Ind
Argument	M
AREP	M
CREP	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier for the desired AR.

CREP

This parameter is the local identifier of the CR (Publisher) which passed the indication.

8.2.2.3.11 Get publisher data

This service shall be used by the DP-slave (Subscriber) application to read the value from a particular subscribed Input Data object from the DP Application Layer. This service shall only be used in conjunction with the MS0 AR.

Table 163 shows the parameters of the service.

Table 163 – Get publisher data

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	M	
Result(+)		S
AREP		M
CREP		M
Data		M
New Flag		M
Result(-)		S
AREP		M
CREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

This parameter is the local identifier for the desired CR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Data

This parameter contains the value from the subscribed Input Data object.

New Flag

This parameter indicates whether a new value from the subscribed Input Data object is received or not.

The allowed values are shown in Table 164.

Table 164 – New Flag

Value	Meaning
0	no new value received
1	new value received

Result(-)

This selection type parameter indicates that the service request failed.

8.2.2.3.12 SYNCH

The service SYNCH indicates to the application of the DP-master (Class 1) that a new Isochronous DP cycle has been started.

Table 165 shows the parameters of the service.

Table 165 – SYNCH

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

8.2.2.3.13 SYNCH delayed

The service SYNCH Delayed shall be passed to the application of the DP-master (Class 1) only if the last synchronisation message was not sent within the allowed time $\max T_{SH}$. The value of the time $\max T_{SH}$ is contained in the related MS0 AR entry of the ARL of the DP-master (Class 1). The service also delivers the time difference measured according to the following rules:

- the synchronisation message was sent earlier than $|T_{DP} - \max T_{SH}|$
- the synchronisation message was sent later than $|T_{DP} + \max T_{SH}|$

Table 166 shows the parameters of the service.

Table 166 – SYNCH Delayed

Parameter name	Ind
Argument	M
AREP	M
Shift	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

Shift

This parameter contains the value of the measured time difference (shift) in units of t_{BIT} .

Attribute Type: Unsigned32

Allowed values: 1 to $2^{32}-1$

8.2.2.3.14 DX finished

The service DX Finished indicates to the application of the DP-master (Class 1) that the cyclic part (DX) of the Isochronous DP cycle is completed.

Table 167 shows the parameters of the service.

Table 167 – DX Finished

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

8.2.2.3.15 SYNCH event

The service SYNCH Event indicates to the application of the DP-slave if

- the DP-master (Class 1) and the DP Application Layer of the DP-slave has started to operate in the Isochronous Mode
- that a new Isochronous DP cycle has been started
- the DP-master (Class 1) and the DP Application Layer of the DP-slave have stopped to operate in the Isochronous Mode.

Table 168 shows the parameters of the service.

Table 168 – SYNCH Event

Parameter name	Ind
Argument	M
AREP	M
Status	M

Argument

The argument shall convey the service specific parameters of the service indication.

AREP

This parameter is the local identifier of the AR which passed the indication.

Status

This parameter indicates the status of the Isochronous Mode.

The allowed values are shown in Table 169.

Table 169 – Status

Value	Meaning
IsoM Start	start of Isochronous Mode operation
IsoM SYNCH	start of a new Isochronous DP cycle
IsoM Stop	stop of Isochronous Mode operation

8.2.2.4 Behaviour of I/O data objects**8.2.2.4.1 General behaviour of the output data object**

For performance reasons, the value for the Output Data object should be transferred on receipt as fast as possible to the object. The speed and the jitters of this action is a performance parameter of a DP-slave. This parameter determines the synchronisation accuracy.

For highly synchronised DP-slaves, this action should be executed always at the same moment of time after the Sync Command.

8.2.2.4.2 Characteristics of a DP system with isochronous mode functionality

8.2.2.4.2.1 Basic behaviour of input and output data in the isochronous mode

A DP system operating in the Isochronous Mode provides the following features (specified at data rates above 1000 kbit/s and isochronous DP cycles between 500 μ s and 32 ms)

- all DP-Devices taking part in the Isochronous Mode functionality synchronise their internal clock system with the synchronisation message SYNCH with the Isochronous DP cycle
- all DP-Devices taking part in the Isochronous Mode functionality have to be on the same network (Isochronous Mode functionality over multiple networks is not possible; repeaters are allowed)
- the jitter of SYNCH-Event.ind is less than 1 μ s (Jitters within DP-slave is assumed as 0)
- in each DP cycle optionally an acyclic service of full length on the MS1 AR may be included (request or response) and be accomplished if there is enough time to process the service completely before the next Isochronous DP cycle has to be started
- the operation of a DP-master (Class 2) on the same network for configuration or diagnosis purposes is possible (the time to execute those services should be included in the cycle time).

In a DP system with enabled Isochronous Mode functionality DP-slaves with basic functionality can be used in conjunction with synchronised DP-slaves. The Isochronous Mode functionality tolerates and compensates communication errors like:

- faulty synchronisation messages (SYNCH); the synchronisation message may be disturbed at the DP-master (Class 1) at the generation or at the conveyance to the DP-slave or at the recognition at the DP-slave
- the Isochronous Mode is capable to compensate the jitters of the synchronisation message produced by the DP-master (Class 1) and if present by network devices like repeaters; this jitter shall be less than 1 μ s (this includes the jitters within the DP-master (Class 1))
- the deviation of the real Isochronous DP cycle to the ideal, not disturbed Isochronous DP cycle shall not exceed 0,03%
- the SYNCH-Event.ind may be missed due to disturbed telegram within the Isochronous DP; up to three subsequently not in time sent or missing synchronisation messages are allowed for a DP-master (Class 1) operating in the Isochronous Mode
- due to disturbances the synchronisation messages may be destroyed at the conveyance to the DP-slave; the clock recovery mechanism in the DP-slave operating in the Isochronous Mode tolerates up to five subsequently not received synchronisation messages

If a DP system shall operate in the Isochronous Mode the following definitions shall apply.

- The DP-master (Class 1) and the correspondent DP-slaves shall support the Fail Safe mechanism.
- The DP-slave shall operate in the Isochronous Mode only if the parameter Fail Safe and IsoM Req provided by the parameterisation are set to TRUE. Additionally the necessary IsoM Parameter object shall be provided with the Check User Prm or Check Ext User Prm service to the application of the DP-slave to set the Isochronous Mode parameter of the DP-slave.

8.2.2.4.2.2 Telegram sequences of a DP system operating in the isochronous mode

Figure 51 shows the sequences of DP systems operating in the Isochronous Mode. A DP cycle in the Isochronous context is considered from the SYNCH (synchronisation message) to the next SYNCH.

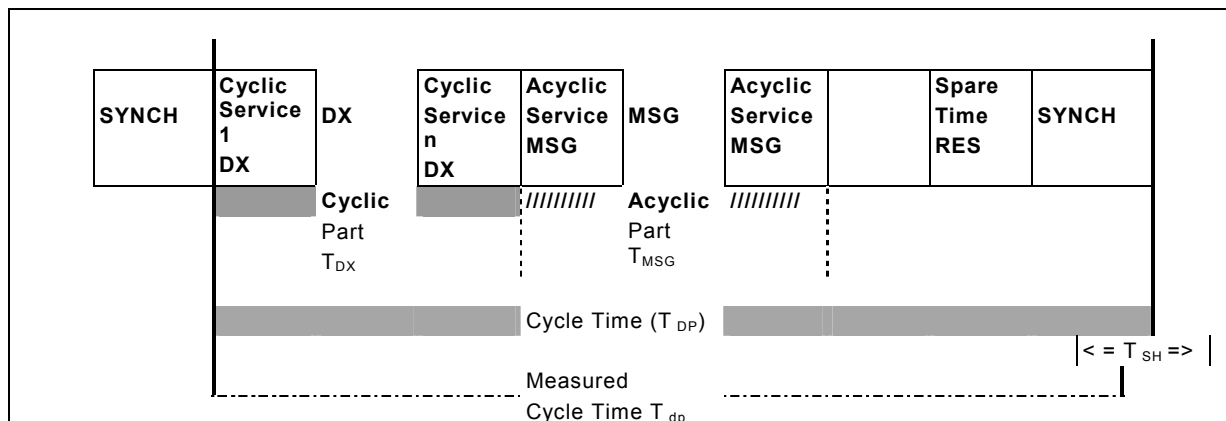


Figure 51 – Sequence of an isochronous DP cycle with one DP-master (class 1)

For the Isochronous Mode the following definitions shall apply.

T_{DX} (Data eXchange Time)

The time T_{DX} may elapse to handle all MS0 communication of one Master (Class 1) of fieldbus DP.

T_{DP} (Data and Parameter exchange Time = Isochronous DP cycle time)

The time T_{DP} may elapse to handle all parts of the Isochronous DP cyclic. The Isochronous DP cycle consists of the following parts:

- SYNCH-Event
- cyclic services (DX)
- acyclic services (MSG)
- spare time (RES)

T_{MSG} (Message Time)

The times T_{MSG} may elapse to handle all acyclic services on the MS1 AR (T_{MSG1}) and MS2 AR (T_{MSG2}) of fieldbus DP as well as all other DL-services with Service_class = low. These acyclic services shall be executed after the cyclic services. To ensure an Isochronous DP cycle this part shall be limited.

T_{SH} (Shift Time)

This time shall be calculated with $T_{\text{measured cycle}} - T_{DP}$. The shift may be caused due to the following reasons:

- delayed sending of the synchronisation message (SYNCH) at the DP-master (Class 1)
- inexact calculation of the time T_{DP} at the DP-master (Class 1)

The maximum value of this time ($\text{max}T_{SH}$) is implementation and configuration dependent. If the time T_{SH} exceeds a certain limit ($\text{max}T_{SH}$) the application in the DP-master (Class 1) will be informed with the SYNCH Delayed service indicating the measured shift.

Jitters

Jitters may be caused due to the following reasons:

(This definition determines the required parts for a PLL- Minimising Jitters influence, capture drifts and compensate signal losses. It shall be used as a base for quality assurance)

- jitters caused by network components like repeaters or fibre optic interfaces
- transmission rate dependent jitters ($\text{jitters} < 1 \cdot T_{BIT}$ for the sender, receiver and the network components between)
- implementation dependent reaction time in the sending and receiving station and the network components between

The following changes of T_{DP} are not part of the Jitters definition

- maximum deviation of T_{DP} in the sending and receiving station ($\text{jitters} < 0,03\% \cdot T_{DP}$)
- losses of clock signalling
- deviation changes due to temperature and ageing (1 ppm within the range of a minute)
- SYNCH is sent too late

RES (Spare time)

This part is defined as the rest of the time between the cyclic part, the acyclic part and the Token handling and the next synchronisation message. The duration of the spare time (RES) depends on the services sent in the actual Isochronous DP cycle.

8.2.2.4.2.3 Time relationships of a DP system operating in isochronous mode

Figure 52 shows the time relationships of the applications of the DP-master (Class 1) and 3 DP-slaves operating in the Isochronous Mode.

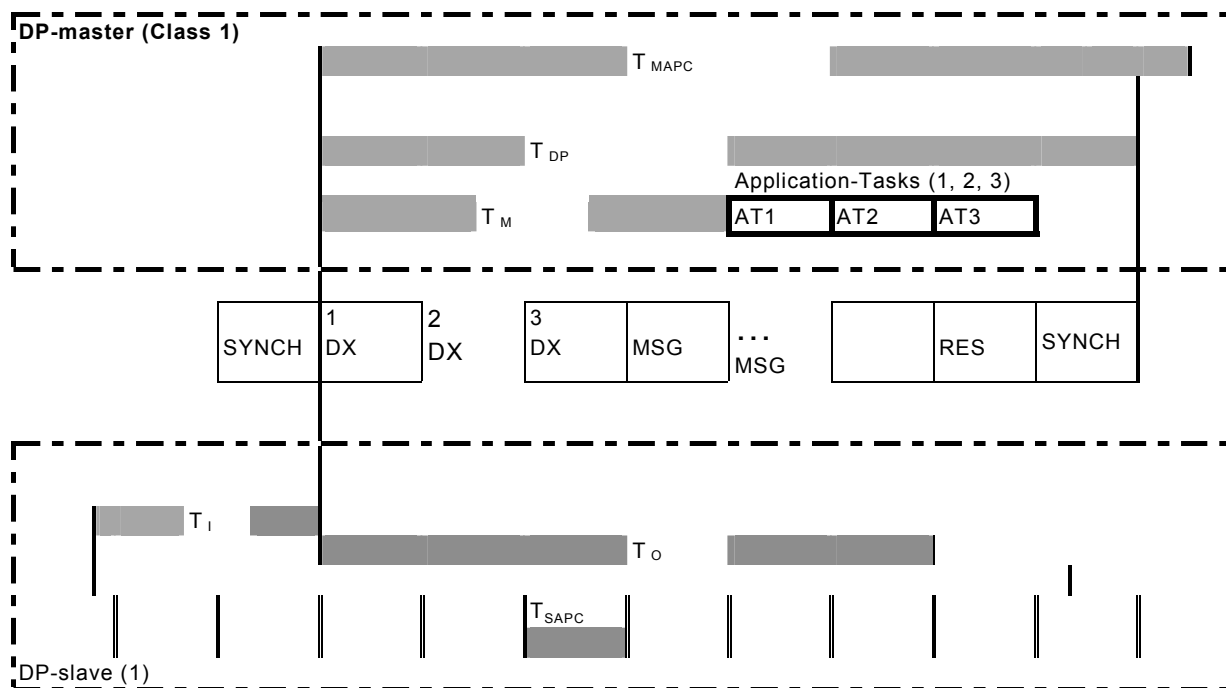


Figure 52 – Additional time relationships in a DP system operating in isochronous mode

For the Isochronous Mode the following time definitions for the application relationships shall be applied.

T_I_MIN (Minimum T_I)

This time parameter indicates the minimum time necessary to get and update the Input Data of an individual DP-slave.

T_I (Input Time)

This time is used to trigger the acquisition and update of the Input Data object in a DP-slave operating in Isochronous Mode and may be calculated by the configuration tool. This time is related to the end of the Isochronous DP cycle.

T_O_MIN (Minimum T_O)

This time parameter indicates the minimum time that is necessary at the end of the cyclic part of the Isochronous DP cycle (T_{DX}) to get and output the Output Data given in units of T_{BASE_IO} of an individual DP-slave.

T_O (Output Time)

This time is used to trigger the output processing of the Output Data object in a DP-slave operating in Isochronous Mode and may be calculated by the configuration tool. This time is related to the beginning of the Isochronous DP cycle.

T_M (Master Time)

This time is used to shift the processing of the application tasks in the DP-master (Class 1) behind the end of the cyclic part of the Isochronous DP cycle (T_{DX}). This parameter may be set by the application in the DP-master (Class 1) according to T_M > T_{DX}. If the time T_M is set to 0 (special case) the DP-master (Class 1) may process input data conveyed a complete Isochronous DP cycle before.

T_{MAPC}, T_{SAPC} (Master Application Cycle Time, Slave Application Cycle Time)

These parameters indicate the time the applications of the DP-master (Class 1) or of the DP-slave need to process an application task completely. The settings of these cycle time parameter may be done device specific. It is allowed to have different settings for the Master Application Cycle Time and Slave Application Cycle Time in each DP-slave using the common time base T_{DP}. The following rules shall apply:

- T_{MAPC} ≥ T_{DP} ≥ T_{SAPC}
- T_{MAPC} = n · T_{DP} (n = 1 to 14; limited by the slave life sign)

The Time relationships of a DP system, especially of the DP-slave, operating in the Isochronous Mode is described only for the so called compact DP-slave. To make the model also suitable for modular DP-slaves additional aspects have to be added. Especially the time relationships have to be extended and some additional time parameters may be necessary.

At least the following constraints have to be addressed:

- at a modular DP-slave the time parameters depend on the plugged in modules.
- each module may add also a time offset (each module type may have an individual time offset)
- the setting of T_I and T_O may be restricted.
- the device itself may add an additional time offset due to the "transfer" time of the I/O data between the Interface device and the modules.

8.2.2.4.2.4 Example of a DP system with optimised isochronous DP cycle

In the following example the application task 1 and 2 in the DP-master (Class 1) may be processed in each Isochronous DP cycle completely.

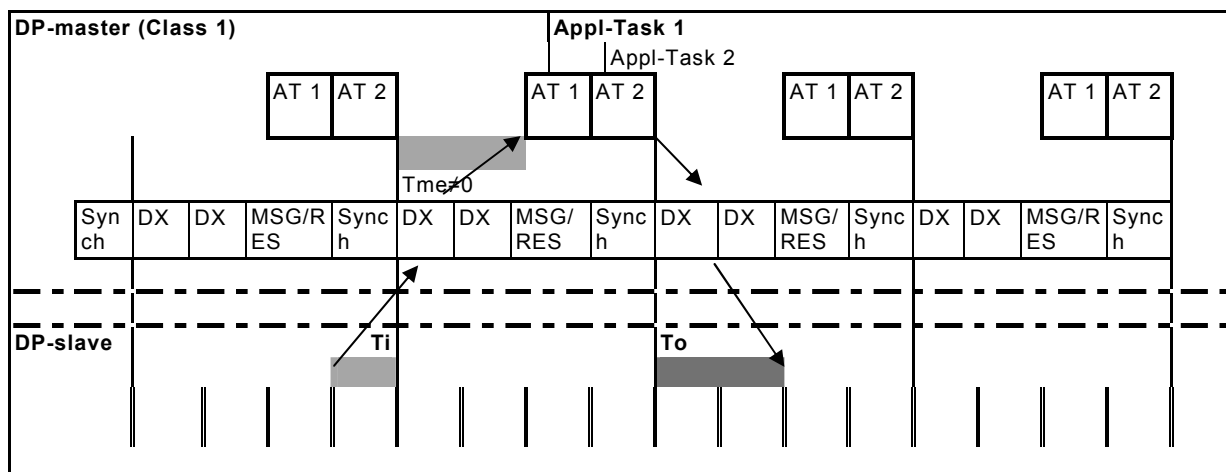


Figure 53 – DP system with optimised isochronous DP cycle

Figure 53 shows a DP System where the sequence of the acquisition and conveyance of the Input Data, processing of the application task and the transfer and output of the Output Data are optimised. This optimisation reduces the necessary cycles to perform a complete control loop to less than 3 Isochronous DP cycles and is characterised by the following phases:

- Phase 1: acquisition and update of the value of the Input Data in the DP-slave
- Phase 2: conveyance of the value of the Input Data to the DP-master (Class 1) and processing of the application task 2 with the actual value of the Input Data and transfer of the new value for the Output Data from the application task 2 to the DP-master (Class 1)
- Phase 3: conveyance of the value for the Output Data to the DP-slave and output of the new Output Data

This model of synchronisation requires enhanced processing capabilities of the DP-master (Class 1) and requires an optimised scheduling in the DP-master (Class 1) and in the DP-slaves.

The dead time in this model of synchronisation is given with: $T_D = T_{DP} + T_I + T_O$

Optimisation in the DP-slave (T_I)

The acquisition and update of the value of the Input Data of all involved DP-slaves should be done at the end of each Isochronous DP cycle as close as possible to ensure to provide the most actual values of the Input Data.

Optimisation in the DP-slave (T_O)

The output of the value of the Output Data of all involved DP-slaves should be done at the end of the cyclic part (DX) of each Isochronous DP cycle as close as possible to ensure to provide the actual values for the Output Data as soon as possible.

Optimisation in the DP-master (Class 1) (T_M)

The processing of the application tasks in the DP-master (Class 1) may be shifted at the end of the cyclic part of the Isochronous DP cycle (T_{DX}) by the parameter T_M . At this time the values of the Input Data of all involved DP-slaves has been update in the DP-master (Class 1). The processing of the application tasks in the DP-master (Class 1) has to be finished within the actual Isochronous DP cycle to provide new values for the Output Data of all involved DP-slaves to be conveyed at the next Isochronous DP cycle.

8.2.2.4.3 Application model of the DP-master (class 1) with isochronous mode functionality

8.2.2.4.3.1 Model variants

Two different models concerning the interface between the DP Application Layer and the application of the DP-master (Class 1) itself are provided by fieldbus DP for the DP-master (Class 1) supporting the Isochronous Mode:

- Buffered Synchronised
- Enhanced Synchronised

The DP Application Layer in conjunction with the Data Link Layer provides the necessary events and messages to synchronise the application of the DP-master (Class 1) with the DP Application Layer for both synchronisation mechanism.

8.2.2.4.3.2 Buffered synchronised isochronous mode

The Buffered Synchronised model provides intermediate buffer to store the Input Data from and the Output Data for the DP-slaves during one or more Isochronous DP cycle. During this period the application may access these buffers to read and update their contents at any time. This mechanism requires less processing performance of the application tasks and less synchronisation between the application and the DP Application Layer.

At first the application shall read the value of the Input Data with the Get Input service and shall update the value of the Output Data of the DP-slaves with the Set Output service. To inform the DP Application Layer to change the intermediate buffer the application has to set the parameter Final of the Set Output service to TRUE at the update of the last DP-slave. The DP Application Layer exchanges the intermediate buffer at the end of the current Isochronous DP cycle. The application shall wait for the next SYNCH indication to start the next application cycle. The application has to ensure the right sequence of interactions.

Figure 54 illustrates the Buffered Synchronised Isochronous Mode at the DP-master (Class 1).

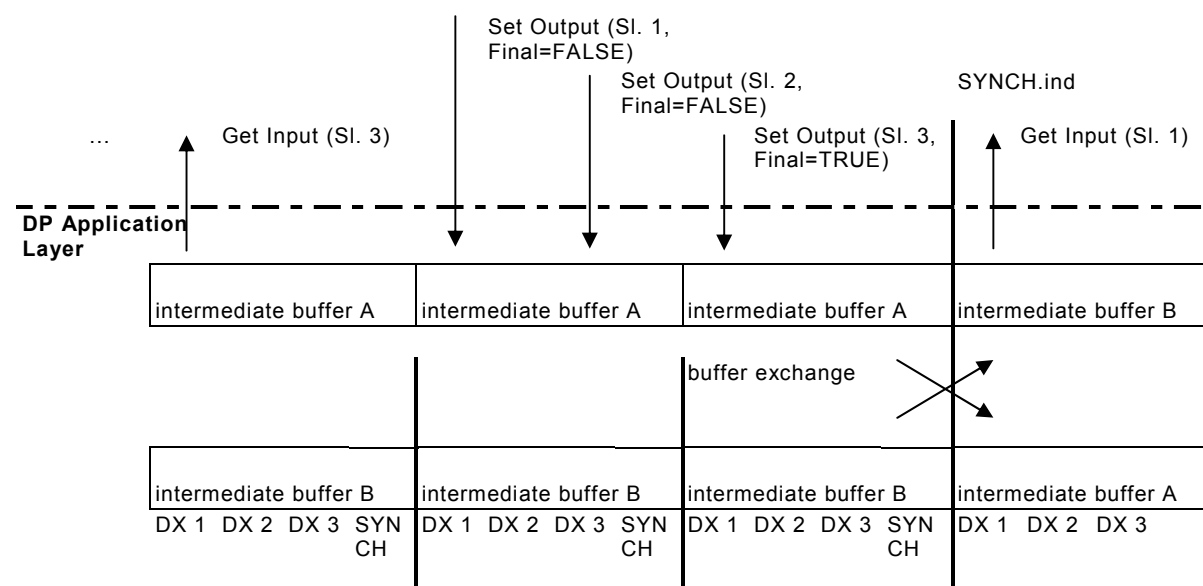


Figure 54 – Buffered synchronised isochronous mode at the DP-master (class 1)

8.2.2.4.3.3 Enhanced synchronised isochronous mode

The Enhanced Synchronised model is characterised by a well defined sequence of interactions between the application and the DP Application Layer of the DP-master (Class 1). The DP Application Layer provides means to supervise the proper sequence of interactions of the application within the Isochronous DP cycle and informs the application in case of erroneous interactions.

Figure 55 illustrates the Mechanism of the Enhanced Synchronised Isochronous Mode.

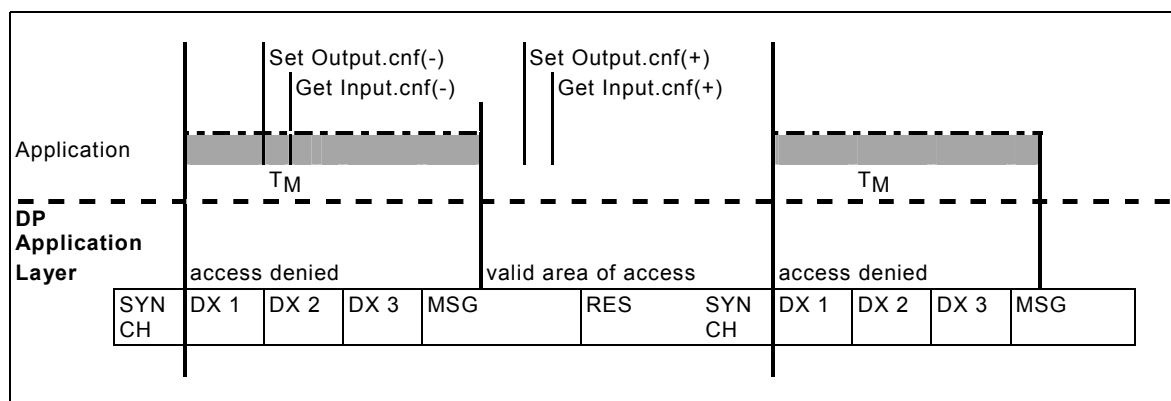


Figure 55 – Enhanced synchronised isochronous mode at the DP-master (class 1)

This mechanism requires enhanced processing performance of the application tasks and a very close synchronisation between the application and the DP Application Layer.

8.2.2.4.4 Output data state machine description for isochronous mode

8.2.2.4.4.1 General

The behaviour of a Output Data object is defined by the Output Data and PLL state machine in case of Isochronous Mode.

There are several State Machines for Input and Output Data objects. Figure 56 shows the interactions of PLL, Input and Output state machine.

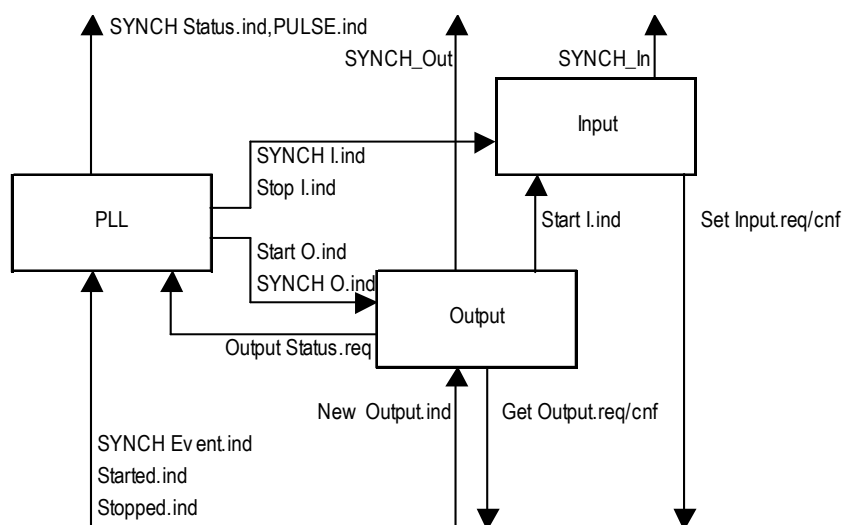


Figure 56 – Input, output and PLL state machine interaction

The Output Data and PLL state machine represents a part of the User functionality and are included to ensure interoperability.

The Output Data and PLL state machine provides events to other parts of the User functionality. The Output Data and PLL state machine have interfaces to the Input Data state machine for co-ordination purposes.

The main functionalities of the Output Data state machine are:

- Transfer the received Output Data to the application on time.
- Check the correct sequence: SYNCH, New Output, read out or the Output Data at To-To_min.
- Check the Masters Sign of Life (MLS).

The main functionalities of the PLL state machine are:

- Jitter compensation of the SYNCH-Event.ind in the DP-slave
- Drift compensation
- Generation of a Pulse Scheme for the local application

The synchronisation primitive (SYNCH) is conveyed by a Global Control service sent as multicast with fixed settings of the parameter Group Select and Control Command Freeze and/or Sync.

The synchronisation (SYNCH) indicated to the application of the DP-slave may jitter, may be lost at the conveyance to the DP-slave or may be received too late because the DP-master (Class 1) has sent the synchronisation message not in time. A PLL (built in hardware or software) shall be used in the DP-slaves to minimise the jitters and phase shifts of the synchronisation message and to compensate the loss of synchronisation messages and may reduce the possible variation of the synchronisation to a value the DP-slave operating in the Isochronous Mode can tolerate. If the variation exceeds a certain limit the application of the DP-slave will leave the Isochronous Mode and will change to a safe state.

The PLL in the DP-slave shall handle the following situations

- The jitters of the synchronisation message at the DP-slave have to be smoothed by the PLL. If the jitters exceeds a certain limit, the PLL will recognise a lost clock.
- Synchronisation messages lost due to bus disturbances (the multicast may not be retried) have to be compensated.
- Drift: After the start-up the Master will send Synchronisation Messages. A DP-slave has to adjust the local time T_{DP} to the Masters T_{DP} . At first SYNCH the Window shall be set to first Window, which is at least $T_{PLL_W} \cdot n_1 + T_{DP} \cdot n_2$ ($n_1 > 1$, $n_2 > 0,0003$). If the following SYNCH are within the Window, the local T_{DP} will be adjusted and the size of the Window shall be reduced until the window has the size of T_{PLL_W} . The PLL shall enter the normal Operating mode now. Additional Drifts due to temperature change or ageing will be adjusted in the normal operating mode. Drift changes or Jitters should result in a small change of the T_{DP} . To minimise changes in T_{DP} drift changes and Jitters shall be limited.
- Phase shifts due to line delay between the different DP-slaves may be compensated, if the output of the PLL will be shifted in relation to the clock. Each DP-slave and the DP-master (Class 1) have to have an individual time shift to be synchronised with the DP-slave which have the longest line delay.

8.2.2.4.4.2 Primitive definitions

8.2.2.4.4.2.1 Primitives exchanged between the AL and the PLL state machine

Table 170 shows primitive the exchanged between the User and the PLL state machine.

Table 170 – Primitives issued by the AL to the PLL state machine

Primitive Name	Source	Associated parameters	Functions
SYNCH Event.ind	AL	AREP Status	
Started.ind	AL	AREP	
Stopped.ind	AL	AREP	

The parameters used with the primitives are described in the Service Specification of the Output Data object (see 8.2.2.3).

8.2.2.4.4.2.2 Primitives exchanged between the user and the PLL state machine

Table 171 show the primitive exchanged between the User and the PLL state machine.

Table 171 – Primitives issued by the user to the PLL state machine

Primitive Name	Source	Associated parameters	Functions
SYNCH Status.ind	PLL	Status	Reports change of Operation Mode, Warnings and Errors
PULSE.ind	PLL		Signal for Application Process to start a new cycle. This signal can be subdivided from T_{DP}. (see Figure 52, T_{SAPC})

The parameter Status used with the primitive SYNCH Status.ind reports the different operation mode changes, warnings and errors. Table 172 shows the allowed values of Status.

Table 172 – Allowed values of status

Value	Source	Meaning
PLL_STOP	PLL	PLL is stopped Device has to be resynchronised
PLL_SYNCH_ERR	PLL	PLL detects Synchronisation Errors and has to be resynchronised
PLL_SYNCHED	PLL	PLL is synchronised with the Masters SYNCH
OUTPUT_SEQUENCE	Output	Output detects Sequence errors (Violation of the sequence: SYNCH > New Output > SYNCH_Out)
OUTPUT_RESETMLS	Output	Output MLS check detects MLS errors and resynchronises MLS
OUTPUT_RUNMLS	Output	Output MLS check detects MLS in sequence

8.2.2.4.4.2.3 Primitives exchanged between the user and the input state machine

Table 173 show the primitive exchanged between the User and the Input state machine.

Table 173 – Primitives issued by the user to the input state machine

Primitive Name	Source	Associated parameters	Functions
SYNCH In	Input		Collects the Values of the Input User Data

8.2.2.4.4.2.4 Primitives exchanged between the user and the output state machine

Table 174 shows the primitive exchanged between the User and the Output state machine.

Table 174 – Primitives issued by the user to the output state machine

Primitive Name	Source	Associated parameters	Functions
SYNCH Out	Output		Pass the Output Buffer to the Output User Data

8.2.2.4.4.2.5 Primitives exchanged between the output state machine and the PLL state machine

Table 175 and Table 176 show the primitive exchanged between the Output state machine and the PLL state machine.

Table 175 – Primitives issued by the PLL to the output state machine

Primitive Name	Source	Associated parameters	Functions
Start_O.ind	PLL		Start Output Data processing
SYNCH_O.ind	PLL		Trigger Signal for Output Data Processing

Table 176 – Primitives issued by the output to the PLL state machine

Primitive Name	Source	Associated parameters	Functions
Output_Status.req	Output	Status	Report Status of Output Data processing

The parameter Status used with the primitive Output_Status.req reports the different warnings and errors.

8.2.2.4.4.2.6 Primitives exchanged between the input state machine and the PLL state machine

Table 177 shows the primitive exchanged between the Input state machine and the PLL state machine.

Table 177 – Primitives issued by the PLL to the input state machine

Primitive Name	Source	Associated parameters	Functions
Stop_I.ind	PLL		Stop Input Data processing
SYNCH_I.ind	PLL		Trigger Signal for Input Data Processing

8.2.2.4.4.2.7 Primitives exchanged between the input state machine and the output state machine

Table 178 shows the primitive exchanged between the Input state machine and the Output state machine.

Table 178 – Primitives issued by the output to the input state machine

Primitive Name	Source	Associated parameters	Functions
Start I.ind	Output		Start Output Data processing

8.2.2.4.4.2.8 Primitives exchanged between the AL and the output state machine

Table 179 shows the primitive exchanged between the Output state machine and AL.

Table 179 – Primitives issued by the output state machine to the AL

Primitive Name	Source	Associated parameters	Functions
Get Output.req	Output	AREP	

Table 180 shows the primitive exchanged between the AL and Output state machine.

Table 180 – Primitives issued by the AL to the output state machine

Primitive Name	Source	Associated parameters	Functions
Get Output.cnf	AL	AREP Output Data Clear Flag New Flag	
New Output.ind	AL	AREP Clear Flag	

8.2.2.4.4.2.9 Primitives exchanged between the input state machine and the AL

Table 181 shows the primitive exchanged between the Input state machine and the AL.

Table 181 – Primitives issued by the input state machine to the AL

Primitive Name	Source	Associated parameters	Functions
Set Input.req	Input	AREP Input Data	

Table 182 shows the primitive exchanged between the Input state machine and the AL.

Table 182 – Primitives issued by the AL to the input state machine

Primitive Name	Source	Associated parameters	Functions
Set Input.cnf	AL	AREP	

8.2.2.4.4.3 Local variable definitions of PLL

AREP

This local variable contains the AREP which represents the application relationships.

W

This local variable contains the current value of the window. The window is used to detect a valid SYNCH Event.ind.

Tdp_n

This local variable contains the current value of the adjusted cycle time. The adaptation is necessary due to clock drifts in a DP-master (class 1).

count_error

This local variable contains the current value of the weighted errors in relation to the SYNCH Event.ind within the window.

RUN

This local variable indicates that the Output State Machine is already started.

cnt

This local variable contains the current value of the segment counter

seg

This local variable contains the current value of the segment time (=Tdp_n/m)

TimerW

This local timer contains the current value of either the time between a SYNCH Event.ind and the beginning of the next window or the time between window start and SYNCH Event.ind. If there is no SYNCH Event.ind within the window, the end of the window substitutes this event. The value of the timer contains the time expired since the last start.

TimerP

This local timer contains the current value of either the time between time between two PULSE.ind. This is used to subdivide.

8.2.2.4.4.4 Local constant definitions of PLL**E_limit**

This local constant limits the number of acceptable synchronisation errors during a time interval.

N_error

This local constant is used to weight the synchronisation errors and the synchronisation success. A factor of 2 means that 2 SYNCH Event.ind are needed to compensate an synchronisation error.

m

This local constant is used to subdivide Tdp_n.

8.2.2.4.4.5 Local function

NOTE Some of these functions are also used by the Output and Input state machine.

inc(x)

if (x=LIMIT) then x:=0 else x:=x+1

inc_LS(x)

if (x=LS_LIMIT) then x:=1 else x:=x+1

reduce_Window(W)

Reduces W as long as $W > T_{pll}$. W shall not be smaller than T_{pll} .

NOTE This function does not decrement W as linear function but can change in a discontinuous way (keeping W for a while and reducing it by dividing the value with 2)

adjust_Tdp(Tdp_n, error)

Adapt Tdp_n to a change of Tdp. To avoid unnecessary changes due to jitters an error should result in a change of Tdp_n smaller than the error value.

8.2.2.4.4.6 PLL state diagram

Figure 57 shows the state diagram of the PLL state machine.

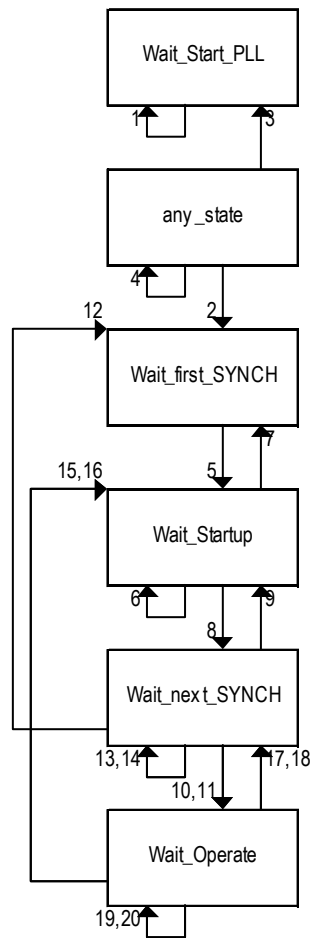


Figure 57 – PLL state diagram

8.2.2.4.4.7 PLL state table

Table 183 shows the states of a PLL state machine.

Table 183 – PLL state table

#	Current State	Event /Condition =>Action	Next State
1	Wait_Start_PLL	SYNCH Event.ind(AREP,Status) => ignore	Wait_Start_PLL
2	any_state	Started.ind(AREP) => W := first_Window Tdp_n := Tdp RUN := 0 count_error := 0	Wait_first_SYNCH
3	any_state	Stopped.ind(AREP) => Status := PLL_STOP SYNCH Status.ind(Status) Stop_I.ind	Wait_Start_PLL
4	any_state	Output Status.req(Status) => SYNCH Status.ind(Status) if (Status = OUTPUT_RESETMLS) Stop_I.ind endif	any_state
5	Wait_first_SYNCH	SYNCH Event.ind(AREP,Status) => start TimerW(Tdp_n-W)	Wait_Startup
6	Wait_Startup	SYNCH Event.ind(AREP,Status) /count_error < e_limit => count_error := count_error + 1	Wait_Startup
7	Wait_Startup	SYNCH Event.ind(AREP,Status) /count_error = e_limit => Status := PLL_SYNCH_ERR W := first_Window Tdp_n := Tdp count_error := 0 start TimerW(Tdp_n-W) SYNCH Status.ind(Status) Stop_I.ind	Wait_first_SYNCH
8	Wait_Startup	TimerW expired => start TimerW(2*W)	Wait_next_SYNCH
9	Wait_next_SYNCH	SYNCH Event.ind(AREP,Status) /W > Tpllw => Tdp_n:= adjust_Tdp(Tdp_n ,(TimerW-W)) W:=reduce_Window(W) start TimerW(Tdp_n-TimerW)	Wait_Startup
10	Wait_next_SYNCH	SYNCH Event.ind(AREP,Status) /W = Tpllw => Tdp_n:= adjust_Tdp(Tdp_n ,(TimerW-W)) count_error := 0start TimerW(Tdp_n-TimerW)	Wait_Operate
11	Wait_next_SYNCH	TimerW expired /count_error < e_limit => count_error := count_error + 1 start TimerW(Tdp-2*W)	Wait_Operate
12	Wait_next_SYNCH	TimerW expired /count_error = e_limit => Status := PLL_SYNCH_ERR W := first_Window Tdp_n := Tdp RUN := 0 count_error := 0 start TimerW(Tdp_n-W) SYNCH Status.ind(Status) Stop_I.ind	Wait_first_SYNCH

#	Current State	Event /Condition =>Action	Next State
13	Wait_next_SYNCH	TimerP expired /cnt < m => cnt:= cnt+1 Start TimerP(Seg) PULSE.ind	Wait_next_SYNCH
14	Wait_next_SYNCH	TimerP expired /cnt = m => PULSE.ind	Wait_next_SYNCH
15	Wait_Operate	SYNCH Event.ind(AREP,Status) /count_error < e_limit => count_error := count_error + 1	Wait_Startup
16	Wait_Operate	SYNCH Event.ind(AREP,Status) /count_error = e_limit => Status := PLL_SYNCH_ERR W := first_Window Tdp_n := Tdp RUN := 0 count_error := 0 start TimerW(Tdp_n-W) SYNCH Status.ind(Status) Stop_l.ind	Wait_Startup
17	Wait_Operate	TimerW expired /RUN = 1 => cnt:=1 Seg = (Tdp_n)/m Start TimerP(W+Delay) Start TimerW(2*W) SYNCH_l.ind SYNCH_O.ind	Wait_next_SYNCH
18	Wait_Operate	TimerW expired /RUN = 0 => Status := PLL_SYNCHED cnt:=1 Seg = (Tdp_n)/m Start TimerP(W+Delay) RUN:= 1 Start TimerW(2*W) Start_O.ind SYNCH_l.ind SYNCH_O.ind SYNCH_Status.ind(Status)	Wait_next_SYNCH
19	Wait_Operate	TimerP expired /cnt < m => cnt:= cnt+1 Start TimerP(Seg) PULSE.ind	Wait_Operate
20	Wait_Operate	TimerP expired /cnt = m => PULSE.ind	Wait_Operate

8.2.2.4.4.8 Local variable definitions of output

AREP

This local variable contains the AREP which represents the application relationships.

wrong_output_update

This local variable contains the current value of the weighted errors of output updates.

MLS_start_counter

This local variable contains the current value of correct MLS at start-up.

MLS_error_counter

This local variable contains the current value of the MLS errors.

TMAPC_counter

This local variable contains the current value of the number of cycles since the last MLS update.

old_MLS

This local variable contains the value of the last MLS.

MLS_State

This local variable contains the value sub-state for MLS calculation.

Possible values are:

- Wait_first_MLS Start-up of MLS
- Wait_next_MLS Wait for next following Start-up MLS
- Wait_MLS Wait for MLS during normal operation

O_Buffer

This local variable contains the values of the Output Data received. It contains a optional variable MLS which contains the live sign. If this variable is not present the State "Check_MLS" has no Action.

TimerTo

This local timer contains the current value of the time between a SYNCH Event.ind and the transfer of the Output_Data to the Application Process.

Timer(To-Tomin)

This local timer contains the current value of the time between a SYNCH Event.ind and the transfer of the Output_Data to the Output State Machine.

8.2.2.4.4.9 Local constant definitions**limit**

This local constant limits the number of acceptable output update errors during a time interval.

n

This local constant is used to weight wrong output updates and successful. A factor of 2 means that 2 successful updates are needed to compensate an erroneous update.

MLS_factor

This local constant is used define the number of cycles between increments of MLS.

n_TMAPC

This local constant is used to limit the errors of MLS.

n_MLS

This local constant is used for start-up to have a reasonable number of correct MLS.

8.2.2.4.4.10 Local function

inc(x)

if (x=LIMIT) then x:=0 else x:=x+1

8.2.2.4.4.11 OUTPUT state diagram

Figure 58 shows the state diagram of the OUTPUT state machine.

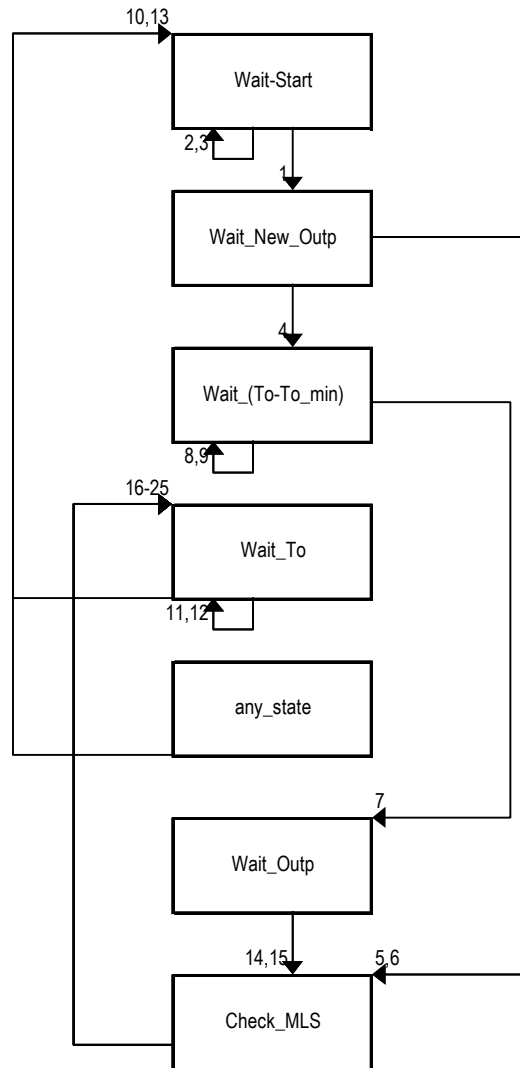


Figure 58 – OUTPUT state diagram

8.2.2.4.4.12 OUTPUT state table

Table 184 shows the states of a OUTPUT state machine.

Table 184 – OUTPUT state table

#	Current State	Event /Condition =>Action	Next State
1	Wait-Start	SYNCH_O.ind => Start Timer(To-To_min) Start TimerTo	Wait_New_Outp
2	Wait-Start	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd < limit => wrong_output_upd= wrong_output_upd+n	Wait-Start
3	Wait-Start	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd = limit => Status := OUTPUT_SEQUENCE Output_Status.req (Status)	Wait-Start
4	Wait_New_Outp	New_Output.ind (AREP, Clear_Flag) => if (wrong_output_upd>0) wrong_output_upd= wrong_output_upd-1	Wait_(To-To_min)
5	Wait_New_Outp	Timer(To-To_min) expired /wrong_output_upd < limit => wrong_output_upd= wrong_output_upd+n	Check_MLS
6	Wait_New_Outp	Timer(To-To_min) expired /wrong_output_upd = limit => Status := OUTPUT_SEQUENCE Output_Status.req (Status)	Check_MLS
7	Wait_(To-To_min)	Timer(To-To_min) expired => Get_Output.req(AREP)	Wait_Outp
8	Wait_(To-To_min)	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd < limit => wrong_output_upd= wrong_output_upd+n	Wait_(To-To_min)
9	Wait_(To-To_min)	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd = limit => Status := OUTPUT_SEQUENCE Output_Status.req (Status)	Wait_(To-To_min)
10	Wait_To	TimerTo expired => SYNCH_Out	Wait-Start
11	Wait_To	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd < limit => wrong_output_upd= wrong_output_upd+n	Wait_To
12	Wait_To	New_Output.ind (AREP, Clear_Flag) /wrong_output_upd = limit => Status := OUTPUT_SEQUENCE Output_Status.req (Status)	Wait_To
13	any_state	Start_O.ind=>MLS_State:= Wait_first_MLSO_Buffer:=Nil(with O_Buffer.MLS:=0)old_MLS:=0MLS_error_counter:=0MAPC_counter:=0MLS_ start_counter:=0	Wait-Start
14	Wait_Outp	Get_Output.cnf (AREP, Output_Data, New_Flag, Clear_Flag) /New_Flag && !Clear_Flag => O_Buffer := Output_Data	Check_MLS
15	Wait_Outp	Get_Output.cnf (AREP, Output_Data, New_Flag, Clear_Flag) /!(New_Flag && !Clear_Flag) =>	Check_MLS
16	Check_MLS	/MLS_State = Wait_first_MLS => MLS_State := Wait_next_MLS Status:=OUTPUT_RESETMLS old_MLS:=O_Buffer.MLS Output_Status.req (Status)	Wait_To

#	Current State	Event /Condition =>Action	Next State
17	Check_MLS	/MLS_State = Wait_next_MLS TMAPC_counter < MLS_factor => MLS_State := Wait_next_MLS inc(TMAPC_counter)	Wait_To
18	Check_MLS	/MLS_State = Wait_next_MLS O_Buffer.MLS <> inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_error_counter < n_TMAPC => MLS_State := Wait_next_MLS inc_LS(old_MLS) TMAPC_counter:=0 MLS_error_counter := MLS_error_counter + n_error	Wait_To
19	Check_MLS	/MLS_State = Wait_next_MLS O_Buffer.MLS <> inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_error_counter >= n_TMAPC => MLS_State := Wait_first_MLS Status:=OUTPUT_RESETMLS TMAPC_counter:=0 MLS_error_counter:=0 old_MLS:=0 MLS_start_counter:=0 Output_Status.req (Status)	Wait_To
20	Check_MLS	/MLS_State = Wait_next_MLS O_Buffer.MLS = inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_start_counter < n_MLS => MLS_State := Wait_next_MLS TMAPC_counter:=0 if (MLS_error_counter >0) MLS_error_counter := MLS_error_counter –1 old_MLS:=MLS inc(MLS_start_counter)	Wait_To
21	Check_MLS	/MLS_State = Wait_next_MLS O_Buffer.MLS = inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_start_counter = n_MLS => MLS_State := Wait_MLS Status:=OUTPUT_RUNMLS TMAPC_counter:=0 MLS_error_counter:=0 old_MLS:=O_Buffer.MLS MLS_start_counter:=0 Output_Status.req (Status) Start_I.ind	Wait_To
22	Check_MLS	/MLS_State = Wait_MLS TMAPC_counter < MLS_factor => MLS_State := Wait_MLS inc(TMAPC_counter)	Wait_To
23	Check_MLS	/MLS_State = Wait_MLS O_Buffer.MLS <> inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_error_counter < n_TMAPC => MLS_State := Wait_MLS inc_LS(old_MLS) TMAPC_counter:=0 MLS_error_counter := MLS_error_counter + n_error	Wait_To

#	Current State	Event /Condition =>Action	Next State
24	Check_MLS	<pre> /MLS_State = Wait_MLS O_Buffer.MLS <> inc_LS(old_MLS) & TMAPC_counter = MLS_factor & MLS_error_counter >= n_TMAPC => MLS_State := Wait_first_MLS Status:=OUTPUT_RESETMLS TMAPC_counter:=0 MLS_error_counter:=0 old_MLS:=0 Output_Status.req (Status) </pre>	Wait_To
25	Check_MLS	<pre> /MLS_State = Wait_MLS O_Buffer.MLS = inc_LS(old_MLS) & TMAPC_counter = MLS_factor => MLS_State := Wait_MLS TMAPC_counter:=0 if (MLS_error_counter >0) MLS_error_counter := MLS_error_counter -1 old_MLS:=O_Buffer.MLS </pre>	Wait_To

8.2.2.4.5 Behaviour of the input data object

For performance reasons, the value of the Input Data object should be transferred on change as fast as possible to the MS0 buffer for transmission. The speed and the jitters of this action is a performance parameter of a DP-slave. This parameter determines the freeze accuracy.

For highly synchronised DP-slaves, this action should be executed always at the same moment of time before the Freeze Command. This requires that a Freeze Command will be sent cyclically.

8.2.2.4.6 Input data state machine description for isochronous mode

8.2.2.4.6.1 Local variable definitions of input

AREP

This local variable contains the AREP which represents the application relationships.

SLS

This local variable contains the value of the Slave life sign.

I_Buffer

This local variable contains the values of the Input Data fetched from the Application Process. It contains a optional variable SLS which contains the live sign. If SLS is not present the Update of SLS is a NULL Operation.

TimerTix

This local timer contains the current value of the time between a SYNCH Event.ind and the transfer of the Input_Data to the AL.

Timer(TDP-Ti)

This local timer contains the current value of the time between a SYNCH Event.ind and the transfer of the Input_Data to the Input State Machine.

8.2.2.4.6.2 INPUT state diagram

Figure 59 shows the state diagram of the INPUT state machine.

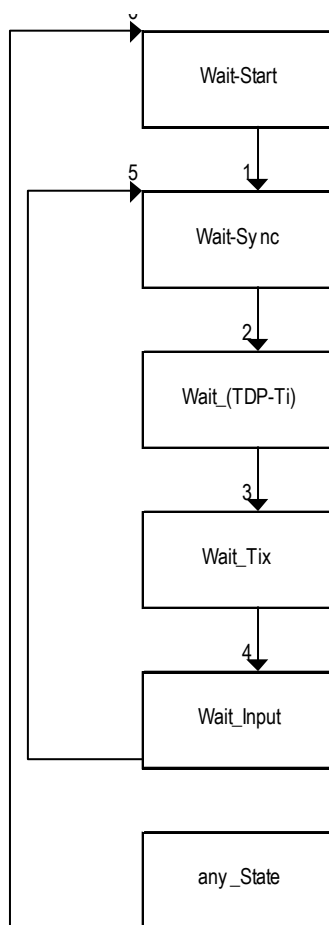


Figure 59 – INPUT state diagram

8.2.2.4.6.3 INPUT state table

Table 185 shows the states of a INPUT state machine.

Table 185 – INPUT state table

#	Current State	Event /Condition =>Action	Next State
1	Wait-Start	Start_I.ind =>	Wait-Sync
2	Wait-Sync	SYNCH_I.ind => Start Timer(TDP-Ti) Start TimerTix	Wait_(TDP-Ti)
3	Wait_(TDP-Ti)	Timer(TDP-Ti) expired => SYNCH_In	Wait_Tix
4	Wait_Tix	TimerTix expired => Input_Data:= I_Buffer Input_Data.SLS:= inc_LS(SLS) Set_Input.req (AREP,Input_Data)	Wait_Input
5	Wait_Input	Set_Input.cnf(AREP) =>	Wait-Sync
6	any_State	Stop_I.ind => SLS:= 0	Wait_Start

8.2.3 Diagnosis ASE

8.2.3.1 Overview

In the DP Application Layer environment, the Diagnosis ASE of a DP-slave contains Diagnosis objects that are used to report the status of the Default Application Process of the DP-slave and the modules which are part of the configuration. The Diagnosis ASE defines attributes of the Diagnosis objects and provides a set of local services used to set and get their values. The following Diagnosis objects are defined:

- Device Related Diagnosis object
- Status object
- Module Status Data object
- DXB-Link Status object
- Identifier Related Diagnosis object
- Channel Related Diagnosis object

The application of a DP-slave may provide the value of one or several Diagnosis objects to the MS0 AR. The DP-slave shall provide only the values of those Diagnosis objects whose values have changed since their last provision. The DP-slave may indicate (with the parameter Ext Diag Overflow) that the values of all Diagnosis objects with a changed value could not be provided with one Set Slave Diag service request. The DP-master (Class 1) shall indicate (with the parameter Ext Diag Overflow) in the Get Slave Diag confirmation that not all received values of the Diagnosis objects from the DP-slave could be provided to the application.

The DP-slave shall indicate (with the parameter Ext Diag Flag) that a relevant diagnosis event has occurred which leads to a change in at least one Diagnosis object (e. g. should be set if one element of the Identifier Related Diagnosis object is set to the value 1). This flag causes an entry in the attribute System Diagnosis of the Master Diag object of the DP-master (Class 1).

The DP-slave may indicate the actual version of the DP-slave with the parameter Revision Number provided by the Set Slave Diag service. The Revision Number may be used by a DP-master (Class 2) to find the related GSD file of the DP-slave.

The Diagnosis model uses the Client/Server access model. The Diagnosis model is based on a buffer to buffer relationship. Thus the Server application updates (sets) the values of the Diagnosis objects in its local buffer for the conveyance. The Client application may get the values of the Diagnosis objects from its local buffer.

The DP-master (Class 1) communicates via the MS0 AR with all Diagnosis objects of the assigned DP-slaves simultaneously. Additionally the Client application may communicate with the Diagnosis objects of a single DP-slave.

The DP-master (Class 2) communicates via the MS0 AR with the Diagnosis objects of a single DP-slave.

Furthermore additional Diagnosis information (attributes of the MS0 AR object) is provided by the DP Application Layer as parameters of the Get Slave Diag service and Read Slave Diag service.

The formal description of the various Diagnosis objects is presented next, followed by a description of its services.

8.2.3.2 Diagnosis class specification

8.2.3.2.1 Device related diagnosis class specification

8.2.3.2.1.1 Template

The Device Related Diagnosis object is described by the following template:

DP ASE:		Diagnosis ASE
CLASS:	Device Related	Diagnosis
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	Device Related Data Description
SERVICES:		
1.	(m) OpService:	Set Slave Diag
2.	(m) OpService:	Get Slave Diag
3.	(o) OpService:	Read Slave Diag
4.	(o) OpService:	New Slave Diag

8.2.3.2.1.2 Attributes

Implicit

The attribute Implicit indicates that the Device Related Diagnosis object is implicitly addressed by the services.

Device Related Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used. The maximum length of this Device Related Data shall not exceed 62 octets.

8.2.3.2.1.3 Invocation of the device related diagnosis object

For the invocation of the Device Related Diagnosis object the following rules shall be considered:

- Device Related Diagnosis objects shall be used only if the device attribute DPV1 = FALSE.
- Device Related Diagnosis objects shall not exceed the total length of 62 octets. The attribute Device Related Data Description has to be set accordingly.
- More than one Device Related Diagnosis object may be invoked in a DP-slave.

8.2.3.2.2 Identifier related diagnosis class specification

8.2.3.2.2.1 Template

This class specifies the state of the modules which are part of the configuration. Each bit is related to a Module (Slot). An element set to FALSE means that no diagnosis of the related module exists or no module is placed. A bit set to TRUE indicates a module failure. If a module failure exists, the values of the Input Data object and/or the Output Data object of this module are set to the default value by the DP-slave.

The Identifier Related Diagnosis object is described by the following template:

DP ASE:		Diagnosis ASE	
CLASS:		Identifier Related Diagnosis	
CLASS ID:		not used	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1.	(m)	Key Attribute:	Implicit
2.	(m)	Attribute:	List of Identifier Status
2.1.	(m)	Attribute:	Identifier Status
SERVICES:			
1.	(m)	OpsService:	Set Slave Diag
2.	(m)	OpsService:	Get Slave Diag
3.	(o)	OpsService:	Read Slave Diag
4.	(o)	OpsService:	New Slave Diag

8.2.3.2.2.2 Attributes

Implicit

The attribute Implicit indicates that the Identifier Related Diagnosis object is implicitly addressed by the services.

List of Identifier Status

This attribute is composed of the following list elements:

Identifier Status

The elements of this list attribute indicates the state of each module. E.g. if the first module has an error the first element of the List of Identifier Status has the value 1.

The allowed values are shown in Table 186.

Table 186 – Identifier status

Value	Meaning
0	Module in operation.
1	Module not in operation.

8.2.3.2.2.3 Invocation of the identifier related diagnosis object

For the invocation of the Identifier Related Diagnosis object the following rules shall be considered:

- Only one Identifier Related Diagnosis object shall be invoked in a DP-slave.
- According to the maximum number of modules, that are part of the configuration, an Identifier Related Diagnosis object shall not exceed the total length of 32 octets.

8.2.3.2.3 Channel related diagnosis class specification

8.2.3.2.3.1 Template

The Channel Related Diagnosis object is described by the following template:

DP ASE:		Diagnosis ASE
CLASS:	Channel Related Diagnosis	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attributes: Identifier
2.	(m)	Attribute: Channel Type
3.	(m)	Attribute: IO Type
4.	(m)	Attribute: Error Type
SERVICES:		
1.	(m)	OpsService: Set Slave Diag
2.	(m)	OpsService: Get Slave Diag
3.	(o)	OpsService: Read Slave Diag
4.	(o)	OpsService: New Slave Diag

8.2.3.2.3.2 Attributes

Identifier

This key attribute is composed of Identifier Number and Channel Number to indicate the source of the Channel Related Diagnosis object.

Allowed values: 0 to 63, 0 to 63

Channel Type

This attribute indicates the channel-size to which the Channel Related Diagnosis object is related.

The allowed values are shown in Table 187.

Table 187 – Channel type

Value	Meaning
0	unspecific
1	1 bit
2	2 bit
3	4 bit
4	byte
5	word
6	2 words

IO Type

This attribute indicates the IO Type of the channel to which the Channel Related Diagnosis object is related.

The allowed values are shown in Table 188.

Table 188 – IO type

Value	Meaning
1	an Input channel
2	an Output channel
3	a Bi-directional channel

Error Type

This attribute indicates the error type of the Channel Related Diagnosis object.

The allowed values are shown in Table 189.

Table 189 – Error type

Value	Meaning
1	short circuit
2	undervoltage
3	overvoltage
4	overload
5	overtemperature
6	line break
7	upper limit value exceeded
8	lower limit value exceeded
9	error
16 to 31	manufacturer specific

8.2.3.2.3.3 Invocation of the channel related diagnosis object

For the invocation of the Channel Related Diagnosis object the following rules shall be considered:

- Several Channel Related Diagnosis objects may be invoked in a DP-slave.

8.2.3.2.4 Status class specification**8.2.3.2.4.1 Template**

The status model allows the transfer of status messages from the DP-slave to the DP-master (Class 1). The status model is slot based.

A Status object is described by the following template:

DP ASE:		Diagnosis ASE
CLASS:	Status	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attributes:	Identifier
2.	(m) Attribute:	Status Specifier
3.	(o) Attribute:	Status Data Description
SERVICES:		
1.	(m) OpService:	Set Slave Diag
2.	(m) OpService:	Get Slave Diag
3.	(o) OpService:	Read Slave Diag
4.	(o) OpService:	New Slave Diag

8.2.3.2.4.2 Attributes**Identifier**

This key attribute is composed of Slot Number and Status Type to indicate the number of the slot and the status type.

Allowed values for Slot Number: 0 to 254

The allowed values for Status Type are shown in Table 190.

Table 190 – Status type

Value	Meaning
1	Status Message
32-126	Manufacturer Specific

Status Specifier

This attribute specifies additional status information.

The allowed values are shown in Table 191.

Table 191 – Status specifier

Value	Meaning
0	no further differentiation
1	status appears
2	status disappears

Status Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

8.2.3.2.4.3 Invocation of the status object

For the invocation of the Status object the following rules apply:

- Status objects shall be used only if the device attribute DPV1 = TRUE
- Several Status objects may be invoked in a DP-slave.
- A Status object shall not exceed the total length of 59 octets. The attribute Status Data Description has to be set accordingly.

8.2.3.2.5 Module status data class specification

8.2.3.2.5.1 Template

This class specifies the status of the modules which are part of the configuration.

The Module Status Data object is described by the following template:

DP ASE:	Diagnosis ASE
CLASS:	Module Status Data
CLASS ID:	not used
PARENT CLASS:	TOP
ATTRIBUTES:	
1.	(m) Key Attribute: Implicit
2.	(m) Attribute: Status Specifier
3.	(m) Attribute: List of Module Status
3.1.	(m) Attribute: Module Status

SERVICES:

1. (m) OpService: Set Slave Diag
2. (m) OpService: Get Slave Diag
3. (o) OpService: Read Slave Diag
4. (o) OpService: New Slave Diag

8.2.3.2.5.2 Attributes**Implicit**

The attribute Implicit indicates that the Module Status Data object is implicitly addressed by the services.

Status Specifier

This attribute specifies additional status information.

The allowed values are shown in Table 192.

Table 192 – Status specifier

Value	Meaning
0	no further differentiation
1	status appears
2	status disappears

List of Module Status

This attribute is composed of the following list elements:

Module Status

This attribute specifies additional status information for each module. The first element of the list contains the status of the first module (Slot Number 1).

The allowed values are shown in Table 193.

Table 193 – Module status

Value	Meaning
0	data valid
1	data invalid - signal error: the data of the corresponding module are not valid due to an error (inputs not accessible or outputs cannot be set, e.g. due to short circuit or overload)
2	data invalid - wrong module: the data of the corresponding module are not valid, due to a wrong module in place
3	data invalid - no module: the data of the corresponding module are not valid because there is no module in place

8.2.3.2.5.3 Invocation of the module status data object

For the invocation of the Module Status Data object the following rules apply:

- The Module Status Data object shall be used only if the device attribute DPV1 = TRUE.
- Only one Module Status Data object shall be invoked in a DP-slave.
- The Module Status Data object shall not exceed the total length of 59 octets.

8.2.3.2.6 DXB-link status class specification**8.2.3.2.6.1 Template**

This class specifies the status of the DXB-Links (Publisher/Subscriber relationships) contained in the DXB-Linktable at the Subscriber.

The DXB-Link Status object shall be used according the following rules:

- the DXB-Link Status object shall be issued at the first time with the Diagnosis information if the DP-slave has received a correct DXB-Linktable and if the DP-slave has entered the data transfer state
- the DXB-Link Status object shall be omitted if the DXB-Linktable was marked as invalid (e. g. receipt of new parameterisation data without a DXB-Linktable)

The DXB-Link Status object is described by the following template:

DP ASE: **Diagnosis ASE**
CLASS: **DXB-Link Status**
CLASS ID: not used
PARENT CLASS: TOP
ATTRIBUTES:
 1. (m) Key Attribute: Implicit
 2. (m) Attribute: Status Specifier
 3. (m) Attribute: List of DXB-Link Status
 3.1. (m) Attribute: Publisher Address
 3.2. (m) Attribute: Link Status
 3.3. (m) Attribute: Link Error
SERVICES:
 1. (m) OpService: Set Slave Diag
 2. (m) OpService: Get Slave Diag
 3. (o) OpService: Read Slave Diag
 4. (o) OpService: New Slave Diag

8.2.3.2.6.2 Attributes

Implicit

The attribute Implicit indicates that the DXB-Link Status object is implicitly addressed by the services.

Status Specifier

This attribute specifies additional status information.

The allowed values are shown in Table 194.

Table 194 – Status specifier

Value	Meaning
0	no further differentiation
1	status appears
2	status disappears

List of DXB-Link Status

This attribute is composed of the following list elements:

Publisher Address

This attribute indicates the DL-address of the corresponding Publisher.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Link Status

This attribute indicates the status of the related DXB-Link.

The allowed values are shown in Table 195.

Table 195 – Link status

Value	Meaning
TRUE	DXB-Link active; at least one subscription of the related Publisher has been performed during the last watchdog control (T _{WD})
FALSE	DXB-Link not active; no subscription of the related Publisher has been performed during the last watchdog control (T _{WD}); the Subscriber may indicate detailed information about the reason with the attribute Link Error

Link Error

This attribute indicates detailed information about the failure of the related DXB-Link.

The allowed values are shown in Table 196.

Table 196 – Link error

Value	Meaning
0	unspecific
1	wrong length; the length of the Input Data of the Publisher is not in alignment with the value of the attribute Input Data Length Publisher of the related DXB-Link Entry

8.2.3.2.6.3 Invocation of the DXB-link status object

For the invocation of the DXB-Link Status object the following rules apply:

- The DXB-Link Status object shall be used only if the device attribute DPV1 is set to TRUE.
- Only one DXB-Link Status object shall be invoked in a DP-slave.
- The DXB-Link Status object shall not exceed the total length of 59 octets. The attribute List of DXB-Link Status has to be set accordingly.

8.2.3.3 Diagnosis service specification**8.2.3.3.1 Set slave diag**

This service shall be used by a DP-slave to pass the values of the Diagnosis objects to the DP Application Layer. The overall transmitted diagnosis information shall not exceed the total length of 238 octets. By this service the Server application process of the DP-slave updates the local buffer. According to the buffer to buffer characteristics of the MS0 AR the following behaviour is possible:

- The Set Slave Diag requests come faster than the contents of the buffers are transported over the network. In this case not each value of the Diagnosis objects provided with the Set Slave Diag service is conveyed over the network. Only the latest values are transported, others are locally overwritten.
- The Set Slave Diag requests issued slower than the contents of the buffers are transported over the network. In this case the values of the Diagnosis objects provided with the Set Slave Diag service are conveyed more than once over the network.
- If the Set Slave Diag requests are issued synchronous with the transport of the contents of the buffers each value of the Diagnosis objects provided with the Set Slave Diag service are conveyed over the network.

This service shall only be used in conjunction with the MS0 AR.

Table 197 shows the parameters of the service.

Table 197 – Set Slave Diag

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Ext Diag Overflow	M	
Ext Diag Flag	M	
Ext Diag Data	C	
Device Related Diagnosis Data	U	
List of Identifier Status	U	
Identifier Status	M	
List of Channel Related Diagnosis Data	U	
Identifier Number	M	
Channel Number	M	
Channel Type	M	
IO Type	M	
Error Type	M	
List of Status Data	U	
Slot Number	M	
Status Type	M	
Status Specifier	M	
Status Data	M	
Module Status Data	U	
Status Specifier	M	
List of Module Status	M	
Module Status	M	
DXB-Link Status	U	
Status Specifier	M	
List of DXB-Link Status	M	
Publisher Address	M	
Link Status	M	
Link Error	M	
Revision Number	U	
Result		S
AREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Ext Diag Flag

This parameter indicates that a relevant diagnosis event has occurred which leads to a change in at least one Diagnosis object (e. g. should be set if one element of the Identifier Related Diagnosis object is set to the value 1).

The allowed values are shown in Table 198.

Table 198 – Ext Diag Flag

Value	Meaning
TRUE	There exists at least one important diagnosis (such as short circuit). An entry has to be present in the Diagnosis objects (Device Related Diagnosis objects, Identifier Related Diagnosis object, Channel Related Diagnosis objects, Status objects, Module Status object).
FALSE	No important diagnosis entry exist. If the attribute value is FALSE, a status message (for example, supply voltage OK) may be present in the Diagnosis Data, or there exists no Diagnosis Data. The meaning of this status message is application specific.

Ext Diag Overflow

This parameter indicates that the values of all Diagnosis objects with a changed value could not been provided with this Set Slave Diag service request.

Allowed values: TRUE, FALSE

NOTE In case of an overflow condition it is allowed to truncate only at Diagnosis object limits.

Ext Diag Data

This parameter is composed of the following items.

Device Related Diagnosis Data

This parameter contains the value of the Device Related Diagnosis object (this parameter shall be used only if DPV1 = FALSE).

List of Identifier Status

This parameter is composed of the following list elements:

Identifier Status

This parameter contains the value of the attribute Identifier Status of the Identifier Related Diagnosis object.

List of Channel Related Diagnosis Data

This parameter is composed of the following list elements:

Identifier Number

This parameter contains the value of the attribute Identifier Number of one Channel Related Diagnosis object.

Channel Number

This parameter contains the value of the attribute Channel Number of one Channel Related Diagnosis object.

Channel Type

This parameter contains the value of the attribute Channel Type of one Channel Related Diagnosis object.

IO Type

This parameter contains the value of the attribute IO Type of one Channel Related Diagnosis object.

Error Type

This parameter contains the value of one Channel Related Diagnosis object.

List of Status Data

This parameter is composed of the following list elements:

Slot Number

This parameter contains the value of the attribute Slot Number of one Status object.

Status Type

This parameter contains the value of the attribute Status Type for one Status object.

Status Specifier

This parameter contains the value of the attribute Status Specifier of one Status object.

Status Data

This parameter contains the value of one Status object.

Module Status Data

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the Module Status Data object.

List of Module Status

This parameter is composed of the following list elements:

Module Status

This parameter contains the value of the attribute Module Status of the Module Status Data object.

DXB-Link Status

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the DXB-Link Status object.

List of DXB-Link Status

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of the DXB-Link Status object.

Link Status

This parameter contains the value of the attribute Link Status of the DXB-Link Status object.

Link Error

This parameter contains the value of the attribute Link Error of the DXB-Link Status object.

Revision Number

This parameter contains the actual value of the Revision Number.

Result

This parameter indicates that the service request succeeded.

8.2.3.3.2 Get slave diag

This service may be used by a DP-master (Class 1) to read the previously received values of the Diagnosis objects of one or all assigned DP-slave which are related to the MS0 AR. Additionally the Get Slave Diag service provides the diagnosis information provided by the DP Application Layer of the DP-slave and the DP-master (Class 1). This service shall only be used in conjunction with the MS0 AR. Table 199 shows the parameters of the service.

Table 199 – Get Slave Diag

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	U	
Result(+)		S
AREP		M
CREP		C
List of Diag Data		M
Prm Req		M
Prm Fault		M
Cfg Fault		M
Not Supported		M
Station Not Ready		M
Stat Diag		M
WD On		M
Sync Mode		M
Freeze Mode		M
Master Add		M
Ident Number		M
Deactivated		M
Station Not Existent		M
Invalid Slave Response		M
Master Lock		M
Ext Diag Overflow		M

Parameter name	Req	Cnf
Ext Diag Flag		M
Ext Diag Data		C
Device Related Diagnosis Data		U
List of Identifier Status		U
Identifier Status		M
List of Channel Related Diagnosis Data		U
Identifier Number		M
Channel Number		M
Channel Type		M
IO Type		M
Error Type		M
List of Status Data		U
Slot Number		M
Status Type		M
Status Specifier		M
Status Data		M
Module Status Data		U
Status Specifier		M
List of Module Status		M
Module Status		M
DXB-Link Status		U
Status Specifier		M
List of DXB-Link Status		M
Publisher Address		M
Link Status		M
Link Error		M
Red Diagnosis		U
Red Status		S
Seq Number		M
Function		M
Primary Request		M
MS1 Command		M
Check Properties		M
Master State Clear		M
Red Status1		M
Role		M
HW Defect		M
Data Exchange		M
Safe State		M
DL Initialised		M
TOH started		M
Red Status2		M
Role		M
HW Defect		M
Data Exchange		M
Safe State		M
DL Initialised		M
TOH started		M
Prm Command Ack		S
Seq Number		M
Function		M
Primary Request		M
MS1 Command		M
Check Properties		M
Master State Clear		M
Red Status1		M
Role		M
HW Defect		M
Data Exchange		M
Master State Clear		M
DL Initialised		M

Parameter name	Req	Cnf
TOH started		M
Red Status2		M
Role		M
HW Defect		M
Data Exchange		M
Master State Clear		M
DL Initialised		M
TOH started		M
Revision Number		U
Result(-)		S
AREP		M
CREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

If this parameter is present then the Diagnosis objects of a single DP-slave are read.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of Diag Data

This parameter is composed of the following list elements:

Prm Req

This parameter contains the value of the attribute Prm Req of the MS0 ARL entry of a DP-slave.

Prm Fault

This parameter contains the value of the attribute Prm Fault of the MS0 ARL entry of a DP-slave.

Cfg Fault

This parameter contains the value of the attribute Cfg Fault of the MS0 ARL entry of a DP-slave.

Not Supported

This parameter contains the value of the attribute Not Supported of the MS0 ARL entry of a DP-slave.

Station Not Ready

This parameter contains the value of the attribute Station Not Ready of the MS0 ARL entry of a DP-slave.

Stat Diag

This parameter contains the value of the attribute Stat Diag of the MS0 ARL entry of a DP-slave.

WD On

This parameter contains the value of the attribute WD On of the MS0 ARL entry of a DP-slave.

Sync Mode

This parameter contains the value of the attribute Sync Mode of the MS0 ARL entry of a DP-slave.

Freeze Mode

This parameter contains the value of the attribute Freeze Mode of the MS0 ARL entry of a DP-slave.

Master Add

This parameter contains the value of the attribute Master Add of the MS0 ARL entry of a DP-slave.

Ident Number

This parameter contains the value of the attribute Ident Number of the MS0 ARL entry of a DP-slave.

Deactivated

This parameter contains the value of the attribute Deactivated of the of the related CRL entry of the correspondent DP-slave of the DP-master (Class 1).

Station Non Existent

This parameter contains the value of the attribute Station Non Existent of the related CRL entry of the correspondent DP-slave of the DP-master (Class 1).

Invalid Slave Response

This parameter contains the value of the attribute Invalid Slave Response of the related CRL entry of the correspondent DP-slave of the DP-master (Class 1).

Master Lock

This parameter indicates whether the DP-slave is locked by a DP-master. If the Master Lock parameter is TRUE the DP-slave is locked by another DP-master. If the parameter is False the DP-slave is not locked by another DP-master (Class 1).

Ext Diag Overflow

This parameter contains the value of the parameter Ext Diag Overflow of the Set Slave Diag service if not all delivered values of the Diagnosis objects are contained in this service confirmation. This parameter contains the value of the attribute Ext Diag Overflow of the related CRL entry of the correspondent DP-slave of the DP-master (Class 1) if not all delivered values of the Diagnosis objects are contained in this service confirmation due to storage problems.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

NOTE In case of an overflow condition it is allowed to truncate only at Diagnosis object limits.

Ext Diag Flag

This parameter contains the value of the parameter Ext Diag Flag of the Set Slave Diag service of a DP-slave.

Ext Diag Data

This parameter is composed of the following subparameters:

Device Related Diagnosis Data

This parameter contains the value of the Device Related Diagnosis object of a DP-slave.

List of Identifier Status

This parameter is composed of the following list elements:

Identifier Status

This parameter contains the value of the attribute Identifier Status of the Identifier Related Diagnosis object of a DP-slave.

List of Channel Related Diagnosis Data

This parameter is composed of the following list elements:

Identifier Number

This parameter contains the value of the attribute Identifier Number of one Channel Related Diagnosis object of a DP-slave.

Channel Number

This parameter contains the value of the attribute Channel Number of one Channel Related Diagnosis object of a DP-slave.

Channel Type

This parameter contains the value of the attribute Channel Type of one Channel Related Diagnosis object of a DP-slave.

IO Type

This parameter contains the value of the attribute IO Type of one Channel Related Diagnosis object of a DP-slave.

Error Type

This parameter contains the value of the attribute Error Type of one Channel Related Diagnosis object of a DP-slave.

List of Status Data

This parameter is composed of the following list elements:

Slot Number

This parameter contains the value of the attribute Slot Number of one Status object of a DP-slave.

Status Type

This parameter contains the value of the attribute Status Type of one Status object of a DP-slave.

Status Specifier

This parameter contains the value of the attribute Status Specifier of one Status object of a DP-slave.

Status Data

This parameter contains the value of one Status object of a DP-slave.

Module Status Data

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the Module Status Data object of a DP-slave.

List of Module Status

This parameter is composed of the following list elements:

Module Status

This parameter contains the value of the attribute Module Status of the Module Status Data object of a DP-slave.

DXB-Link Status

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the DXB-Link Status object of a DP-slave.

List of DXB-Link Status

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of the DXB-Link Status object of a DP-slave.

Link Status

This parameter contains the value of the attribute Link Status of the DXB-Link Status object of a DP-slave.

Link Error

This parameter contains the value of the attribute Link Error of the DXB-Link Status object of a DP-slave.

Red Diagnosis

This parameter is composed of the following subparameters:

Red Status

This selection type parameter indicates that the parameter Red Diagnosis contains the diagnosis information of the redundant communication interfaces of a redundant DP-Slave. This parameter is composed of the following subparameters:

Seq Number

This parameter contains the Sequence Number necessary for the unique identification of the received Red Status.

Allowed values: 0 to 31

Function

This parameter is composed of the following subparameters:

Primary Request

This parameter set to TRUE indicates that a Primary Request was executed at this communication interface. This parameter set to FALSE indicates that no Primary Request was executed at this communication interface.

Allowed values: TRUE, FALSE

MS1 Command

This parameter indicates if a MS1 Command like Start MS1, Stop MS1, Reset MS1 or no MS1 Command was executed at this communication interface.

Allowed values: Start MS1, Stop MS1, Reset MS1, no Action

Check Properties

This parameter set to TRUE indicates that a Check Properties request was executed at this communication interface. This parameter set to FALSE indicates that no Check Properties request was executed at this communication interface.

Allowed values: TRUE, FALSE

Master State Clear

This parameter set to TRUE indicates that the related DP-Master (Class 1) have set the communication interface in the state Clear. This parameter set to FALSE indicates that the related DP-Master (Class 1) have sent no Master State Clear command.

Allowed values: TRUE, FALSE

Red Status1

This parameter contains the redundancy status of the first communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that this communication interface acts as the primary interface. This parameter set to Backup indicates that this communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of this communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that this communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that this communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of this communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Red Status2

This parameter contains the redundancy status of the second (redundant) communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that the other communication interface acts as the primary interface. This parameter set to Backup indicates that the other communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of the other communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that the other communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that the other communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of the other communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Prm Command Ack

This selection type parameter indicates that the parameter Red Diagnosis contains the acknowledgment of a previously received Prm Command. This parameter is composed of the following subparameters:

Seq Number

This parameter contains the Sequence Number necessary for the unique identification of the received Prm Command Ack.

Allowed values: 0 to 31

Function

This parameter is composed of the following subparameters:

Primary Request

This parameter set to TRUE indicates that a Primary Request was executed at this communication interface. This parameter set to FALSE indicates that no Primary Request was executed at this communication interface.

Allowed values: TRUE, FALSE

MS1 Command

This parameter indicates if a MS1 Command like Start MS1, Stop MS1, Reset MS1 or no MS1 Command was executed at this communication interface.

Allowed values: Start MS1, Stop MS1, Reset MS1, no Action

Check Properties

This parameter set to TRUE indicates that a Check Properties request was executed at this communication interface. This parameter set to FALSE indicates that no Check Properties request was executed at this communication interface.

Allowed values: TRUE, FALSE

Master State Clear

This parameter set to TRUE indicates that the related DP-Master (Class 1) have set the communication interface in the state Clear. This parameter set to FALSE indicates that the related DP-Master (Class 1) have sent no Master State Clear command.

Allowed values: TRUE, FALSE

Red Status1

This parameter contains the redundancy status of the first communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that this communication interface acts as the primary interface. This parameter set to Backup indicates that this communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of this communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that this communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that this communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of this communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Red Status2

This parameter contains the redundancy status of the second (redundant) communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that the other communication interface acts as the primary interface. This parameter set to Backup indicates that the other communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of the other communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that the other communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that the other communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of the other communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Revision Number

This parameter contains the actual value of the Revision Number.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.3.3.3 Read slave diag

This service may be used by a DP-master (Class 2) to read acyclic the values of the Diagnosis objects of one DP-slave via a MS0 AR. Additionally the Read Slave Diag service provides the diagnosis information provided by the DP Application Layer of the DP-slave and the DP-master (Class 2).

Table 200 shows the parameters of the service.

Table 200 – Read Slave Diag

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Diag Data		M
Prm Req		M
Prm Fault		M
Cfg Fault		M
Not Supported		M
Station Not Ready		M
Stat Diag		M
WD On		M
Sync Mode		M
Freeze Mode		M
Master Add		M
Ident Number		M
Ext Diag Overflow		M
Ext Diag Flag		M
Ext Diag Data		C
Device Related Diagnosis Data		U
List of Identifier Status		U
Identifier Status		M
List of Channel Related Diagnosis Data		U
Identifier Number		M
Channel Number		M
Channel Type		M
IO Type		M
Error Type		M
List of Status Data		U
Slot Number		M
Status Type		M
Status Specifier		M
Status Data		M
Module Status Data		U
Status Specifier		M
List of Module Status		M
Module Status		M
DXB-Link Status		U
Status Specifier		M
List of DXB-Link Status		M
Publisher Address		M
Link Status		M
Link Error		M
Red Diagnosis		U
Red Status		S
Seq Number		M
Function		M
Primary Request		M
MS1 Command		M
Check Properties		M
Master State Clear		M
Red Status1		M
Role		M
HW Defect		M
Data Exchange		M
Safe State		M
DL Initialised		M
TOH started		M
Red Status2		M

Parameter name	Req	Cnf
Role		M
HW Defect		M
Data Exchange		M
Safe State		M
DL Initialised		M
TOH started		M
Prm Command Ack		S
Seq Number		M
Function		M
Primary Request		M
MS1 Command		M
Check Properties		M
Master State Clear		M
Red Status1		M
Role		M
HW Defect		M
Data Exchange		M
Master State Clear		M
DL Initialied		M
TOH started		M
Red Status2		M
Role		M
HW Defect		M
Data Exchange		M
Master State Clear		M
DL Initialised		M
TOH started		M
Revision Number		U
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Diag Data

This parameter is composed of the following list elements:

Prm Req

This parameter contains the value of the attribute Prm Req of the MS0 ARL entry of the selected DP-slave.

Prm Fault

This parameter contains the value of the attribute Prm Fault of the MS0 ARL entry of the selected DP-slave.

Cfg Fault

This parameter contains the value of the attribute Cfg Fault of the MS0 ARL entry of the selected DP-slave.

Not Supported

This parameter contains the value of the attribute Not Supported of the MS0 ARL entry of the selected DP-slave.

Station Not Ready

This parameter contains the value of the attribute Station Not Ready of the MS0 ARL entry of the selected DP-slave.

Stat Diag

This parameter contains the value of the attribute Stat Diag of the MS0 ARL entry of the selected DP-slave.

WD On

This parameter contains the value of the attribute WD On of the MS0 ARL entry of the selected DP-slave.

Sync Mode

This parameter contains the value of the attribute Sync Mode of the MS0 ARL entry of the selected DP-slave.

Freeze Mode

This parameter contains the value of the attribute Freeze Mode of the MS0 ARL entry of the selected DP-slave.

Master Add

This parameter contains the value of the attribute Master Add of the MS0 ARL entry of the selected DP-slave.

Ident Number

This parameter contains the value of the attribute Ident Number of the MS0 ARL entry of the selected DP-slave.

Ext Diag Overflow

This parameter contains the value of the parameter Ext Diag Overflow of the Set Slave Diag service if not all delivered values of the Diagnosis objects are contained in this service confirmation. This parameter will be set to TRUE by the AP of the DP-master (Class 2) if not all delivered values of the Diagnosis objects are contained in this service confirmation due to storage problems.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

NOTE In case of an overflow condition it is allowed to truncate only at Diagnosis object limits.

Ext Diag Flag

This parameter contains the value of the parameter Ext Diag Flag of the Set Slave Diag service of the selected DP-slave.

Ext Diag Data

This parameter is composed of the following subparameters:

Device Related Diagnosis Data

This parameter contains the value of the Device Related Diagnosis object of the selected DP-slave.

List of Identifier Status

This parameter is composed of the following list elements:

Identifier Status

This parameter contains the value of the attribute Identifier Status of the Identifier Related Diagnosis object of the selected DP-slave.

List of Channel Related Diagnosis Data

This parameter is composed of the following list elements:

Identifier Number

This parameter contains the value of the attribute Identifier Number of one Channel Related Diagnosis object of the selected DP-slave.

Channel Number

This parameter contains the value of the attribute Channel Number of one Channel Related Diagnosis object of the selected DP-slave.

Channel Type

This parameter contains the value of the attribute Channel Type of one Channel Related Diagnosis object of the selected DP-slave.

IO Type

This parameter contains the value of the attribute IO Type of one Channel Related Diagnosis object of the selected DP-slave.

Error Type

This parameter contains the value the attribute Error Type of one Channel Related Diagnosis object of the selected DP-slave.

List of Status Data

This parameter is composed of the following list elements:

Slot Number

This parameter contains the value of the attribute Slot Number of one Status object of the selected DP-slave.

Status Type

This parameter contains the value of the attribute Status Type of one Status object of the selected DP-slave.

Status Specifier

This parameter contains the value of the attribute Status Specifier of one Status object of the selected DP-slave.

Status Data

This parameter contains the value of one Status object of the selected DP-slave.

Module Status Data

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the Module Status Data object of the selected DP-slave.

List of Module Status

This parameter is composed of the following list elements:

Module Status

This parameter contains the value of the attribute Module Status of the Module Status Data object of the selected DP-slave.

DXB-Link Status

This parameter is composed of the following subparameters:

Status Specifier

This parameter contains the value of the attribute Status Specifier of the DXB-Link Status object of a DP-slave.

List of DXB-Link Status

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of the DXB-Link Status object of a DP-slave.

Link Status

This parameter contains the value of the attribute Link Status of the DXB-Link Status object of a DP-slave.

Link Error

This parameter contains the value of the attribute Link Error of the DXB-Link Status object of a DP-slave.

Red Diagnosis

This parameter is composed of the following subparameters:

Red Status

This selection type parameter indicates that the parameter Red Diagnosis contains the diagnosis information of the redundant communication interfaces of a redundant DP-Slave. This parameter is composed of the following subparameters:

Seq Number

This parameter contains the Sequence Number necessary for the unique identification of the received Red Status.

Allowed values: 0 to 31

Function

This parameter is composed of the following subparameters:

Primary Request

This parameter set to TRUE indicates that a Primary Request was executed at this communication interface. This parameter set to FALSE indicates that no Primary Request was executed at this communication interface.

Allowed values: TRUE, FALSE

MS1 Command

This parameter indicates if a MS1 Command like Start MS1, Stop MS1, Reset MS1 or no MS1 Command was executed at this communication interface.

Allowed values: Start MS1, Stop MS1, Reset MS1, no Action

Check Properties

This parameter set to TRUE indicates that a Check Properties request was executed at this communication interface. This parameter set to FALSE indicates that no Check Properties request was executed at this communication interface.

Allowed values: TRUE, FALSE

Master State Clear

This parameter set to TRUE indicates that the related DP-Master (Class 1) have set the communication interface in the state Clear. This parameter set to FALSE indicates that the related DP-Master (Class 1) have sent no Master State Clear command.

Allowed values: TRUE, FALSE

Red Status1

This parameter contains the redundancy status of the first communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that this communication interface acts as the primary interface. This parameter set to Backup indicates that this communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of this communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that this communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that this communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of this communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Red Status2

This parameter contains the redundancy status of the second (redundant) communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that the other communication interface acts as the primary interface. This parameter set to Backup indicates that the other communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of the other communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that the other communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that the other communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of the other communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Prm Command Ack

This selection type parameter indicates that the parameter Red Diagnosis contains the acknowledgment of a previously received Prm Command. This parameter is composed of the following subparameters:

Seq Number

This parameter contains the Sequence Number necessary for the unique identification of the received Prm Command Ack.

Allowed values: 0 to 31

Function

This parameter is composed of the following subparameters:

Primary Request

This parameter set to TRUE indicates that a Primary Request was executed at this communication interface. This parameter set to FALSE indicates that no Primary Request was executed at this communication interface.

Allowed values: TRUE, FALSE

MS1 Command

This parameter indicates if a MS1 Command like Start MS1, Stop MS1, Reset MS1 or no MS1 Command was executed at this communication interface.

Allowed values: Start MS1, Stop MS1, Reset MS1, no Action

Check Properties

This parameter set to TRUE indicates that a Check Properties request was executed at this communication interface. This parameter set to FALSE indicates that no Check Properties request was executed at this communication interface.

Allowed values: TRUE, FALSE

Master State Clear

This parameter set to TRUE indicates that the related DP-Master (Class 1) have set the communication interface in the state Clear. This parameter set to FALSE indicates that the related DP-Master (Class 1) have sent no Master State Clear command.

Allowed values: TRUE, FALSE

Red Status1

This parameter contains the redundancy status of the first communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that this communication interface acts as the primary interface. This parameter set to Backup indicates that this communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of this communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that this communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that this communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of this communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Red Status2

This parameter contains the redundancy status of the second (redundant) communication interface of a redundant DP-Slave. This parameter is composed of the following subparameters:

Role

This parameter set to Primary indicates that the other communication interface acts as the primary interface. This parameter set to Backup indicates that the other communication interface acts as the Backup interface. This parameter set to Power On indicates that the assignment of the roles between the two communication interfaces is not finished yet (e. g. at Power On of the DP-Slave).

Allowed values: Primary, Backup, Power On

HW Defect

This parameter set to TRUE indicates that an HW Defect of the other communication interface has been detected.

Allowed values: TRUE, FALSE

Data Exchange

This parameter set to TRUE indicates that the other communication interface is in the data exchange state.

Allowed values: TRUE, FALSE

Safe State

This parameter set to TRUE indicates that the other communication interface is in the Fail Safe state.

Allowed values: TRUE, FALSE

DL Initialised

This parameter set to TRUE indicates that the DL parameter of the other communication interface has been loaded.

Allowed values: TRUE, FALSE

TOH started

This parameter set to TRUE indicates that the Output Hold Timer of this communication interface has been started.

Allowed values: TRUE, FALSE

Revision Number

This parameter contains the value of the Revision Number of the selected DP-slave.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter contains the reason for the failure.

Allowed values: RE, DS, UE, RS, NA, NR, RR

8.2.3.3.4 New slave diag

This service may be used by a DP-master (Class 1) to indicate to the application that new received values of the Diagnosis objects of one assigned DP-slave which is related to the MS0 AR, is present. Additionally this service may be used to indicate that new diagnosis information provided by the DP Application Layer of the DP-slave or the DP-master (Class 1) is present.

Table 201 shows the parameters of the service.

Table 201 – New Slave Diag

Parameter name	Ind
Argument	M
AREP	M
CREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the related AR.

CREP

This parameter is the local identifier of the related CR (DP-slave).

8.2.4 Alarm ASE

8.2.4.1 Overview

The alarm model allows the transfer of an alarm from the DP-slave to the assigned DP-master (Class 1) and the explicit acknowledgement of the alarm by the DP-master (Class 1) via the MS1 AR.

Figure 60 shows the alarm interactions between the DP-slave (Alarm Notification) and the DP-master (Alarm Ack).

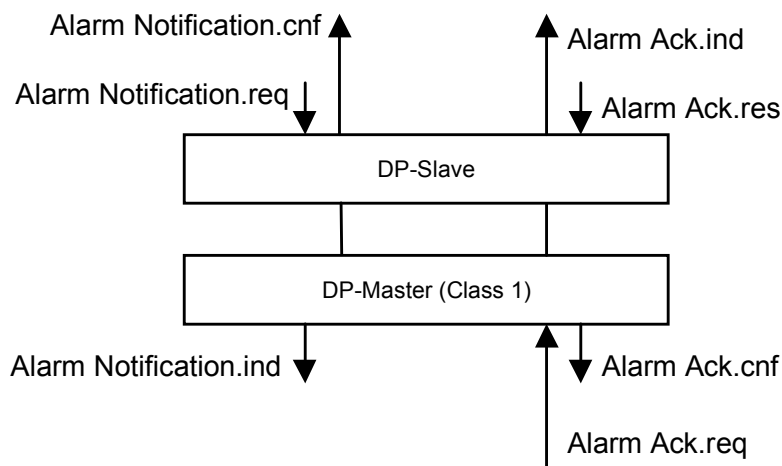


Figure 60 – Treatment of an alarm in the DP system

For the transmission of an alarm the following conditions have to be fulfilled:

- the DP-slave has to be in the data exchange mode
- the MS1 AR has to be open
- the corresponding alarm type has to be enabled
- the limit of active alarms is not exceeded

In addition to the alarm transmission a Channel Related and/or Identifier Related Diagnosis may be issued which identify the exact source of the alarm.

Two alternatives of the alarm handling (see 8.2.10.3.1) are defined:

- only one alarm of a specific alarm type can be active at the same time (Type Mode)
- several alarms (2 to 32) independent of the type can be active at the same time (Sequence Mode)

The DP-master (Class 1) and the DP-slave shall implement the Type Mode and may additionally implement the Sequence Mode if alarms are supported. Alarms will be queued. The alarm remains active until the DP-master (Class 1) explicitly acknowledges the alarm.

The alarm acknowledgement is either performed via the CR, that conveys the Read and Write service, or optionally via a separate CR on the MS1 AR. With this latter feature, the alarms will not be delayed by read and write services.

The following alarm types are defined. They may be extended specific to the manufacturer:

Diagnosis Alarm

A diagnosis alarm signals an event within a module, for instance overtemperature, short circuit, etc.

Process Alarm

A process alarm signals the occurrence of an event in the connected process, for instance upper limit value exceeded.

Pull Alarm

A slot signals the withdrawal of a module.

Plug Alarm

A slot signals the insertion of a module.

Status Alarm

A status alarm signals a change in the state of a module, for instance run, stop or ready.

Update Alarm

An update alarm signals the change of a parameter in a module e.g. by a local operation or a remote access.

8.2.4.2 Alarm class specification

8.2.4.2.1 Template

An Alarm object is described by the following template:

DP ASE:		Alarm ASE
CLASS:	ALARM DATA	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attributes:	Identifier
2.	(m) Attribute:	Add Ack
3.	(m) Attribute:	Alarm Specifier
4.	(o) Attribute:	Alarm Data Description
SERVICES:		
1.	(m) OpService:	Alarm Notification
2.	(m) OpService:	Alarm Ack

8.2.4.2.2 Attributes

Identifier

This key attribute is composed of Slot Number and Alarm Type to indicate the number of the slot and the alarm type.

Allowed values for Slot Number: 0 to 254

The allowed values of Alarm Type are shown in Table 202.

Table 202 – Alarm type

Value	Meaning
1	Diagnosis Alarm
2	Process Alarm
3	Pull Alarm
4	Plug Alarm
5	Status Alarm
6	Update Alarm
32-126	Manufacturer Specific

Add Ack

This attribute indicates to the DP-master (Class 1) that this alarm requires an additional User acknowledgement. This can be done for instance by a write service. The alarm remains active after the Alarm Ack.

The allowed values are shown in Table 203.

Table 203 – Add Ack

Value	Meaning
TRUE	additional acknowledgement required
FALSE	no additional acknowledgement required

Alarm Specifier

This attribute specifies additional alarm information.

The allowed values are shown in Table 204.

Table 204 – Alarm specifier

Value	Meaning
0	no further differentiation
1	Alarm appears and the related module is disturbed
2	Alarm disappears and the related module has no further errors
3	Alarm disappears and the related module is still disturbed

Alarm Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

8.2.4.2.3 Invocation of the alarm data object

For the invocation of the Alarm Data object the following rules apply:

- Alarm Data objects shall be used only if the device attribute DPV1 = TRUE
- Several Alarm Data objects can be invoked in a DP-slave
- For each module only one Alarm Data object of one type shall be invoked
- Alarm Data objects shall not exceed the total length of 59 octets. The attribute Alarm Data Description has to be set accordingly.

8.2.4.3 Alarm service specification

8.2.4.3.1 Alarm notification

This service is used to transfer an alarm notification from the DP-slave to the DP-master (Class 1). This service shall only be used in conjunction with the MS1 AR. Table 205 shows the parameters of the service.

Table 205 – Alarm notification

Parameter name	Req	Ind	Cnf
Argument	M	M(=)	
AREP	M	M(=)	
Slot Number	M	M(=)	
Alarm Type	M	M(=)	
Seq Nr	M	M(=)	
Add Ack	M	M(=)	
Alarm Specifier	M	M(=)	
Alarm Data	U	C(=)	
Result(+)			S
AREP			M
Slot Number			M
Alarm Type			M
Seq Nr			M
Result(-)			S
AREP			M
Slot Number			M
Alarm Type			M
Seq Nr			M
Status			M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

This parameter contains the value of the attribute Slot Number of the Alarm Data object.

Alarm Type

This parameter contains the value of the attribute Alarm Type of the Alarm Data object.

Seq Nr

This parameter is used as an unique identifier for the transaction of an alarm notification by the DP Application Layer.

Allowed values: 0 to 31

Add Ack

This parameter contains the value of the attribute Add Ack of the Alarm Data object.

Alarm Specifier

This parameter contains the value of the attribute Alarm Specifier of the Alarm Data object.

Alarm Data

This parameter contains the value of the Alarm Data object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter contains the reason for the failure.

Allowed values: MSAL1S_Stopped, Not_Enabled, Limit_Expired, Sequence_Nr_Pending, No_Start

8.2.4.3.2 Alarm ack

This service is used by the DP-master (Class 1) to acknowledge the reception of an alarm notification which has been previously received from the DP-slave. This service shall only be used in conjunction with the MS1 AR.

Table 206 shows the parameters of the service.

Table 206 – Alarm Ack

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
Alarm Type	M	M(=)		
Seq Nr	M	M(=)		
Result(+)			S	S
AREP			M	M(=)
Slot Number			M	M(=)
Alarm Type			M	M(=)
Seq Nr			M	M(=)
Result(-)			S	S
AREP			M	M(=)
Slot Number			M	M(=)
Alarm Type			M	M(=)
Seq Nr			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

This parameter contains the value of the attribute Slot Number of the of the previously received Alarm Data object.

Alarm Type

This parameter contains the value of the attribute Alarm Type of the previously received Alarm Data object.

Seq Nr

This parameter contains the sequence number of the previously received alarm notification.

Allowed values: 0 to 31

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.5 Context ASE

8.2.5.1 Overview

The Context ASE provides a set of services to parameterise and configure the Default AP of a DP-slave itself and its modules containing I/O data.

The Application Process of many DP-slaves need parameter (e.g. the range of an analogue signal) from the assigned DP-master (Class 1) to provide the data from the Input Data object and/or for the Output Data object according to the requirements of the AP of the DP-master (Class 1). These DP-slaves provide in the Context ASE a MS0 Parameter object or, for more flexibility, several MS0 Structured Parameter objects. The values for the attributes of these objects will be provided by the DP-master (Class 1) and assigned after validation from the AP of the DP-slave.

The Context ASE provides objects and the related services for the flexible parameterisation of a DP-slave. The following objects are defined:

- MS0 User Parameter object (basic parameterisation object)
- MS0 Structured User Parameter object
- DXB-Linktable object
- DXB-Subscribable object
- IsoM Parameter object
- Time AR Parameter object

The functionality of a DP-slave application implies the necessary parameterisation objects which have to be conveyed at the start-up phase of the DP system from the DP-master (Class 1) to the assigned DP-slaves via the MS0 AR. The application of the DP-slave has to check the parameterisation data sent by the DP-master (Class 1). If the check fails the DP-slave may respond with an appropriate entry in the Diagnosis information.

The MS0 Configuration Elements object describes the module structure, the range of input and output data and contains information about the data consistency. The Context ASE defines a set of services used to set and get their values. The application of the DP-slave will compare the configuration data sent by the DP-master (Class 1) with the local configuration of the DP-slave. If they match the MS0 AR to this DP-slave will be established. The real configuration of the DP-slave may be read by a DP-master (Class 2) with the related service.

The DXB-Subscribable object in conjunction with the MS0 Configuration Elements object provides the information the Subscriber needs to map the output data of the DP-master (Class 1) and/or the subscribed Input Data of one or more Publisher to the data areas of the Output Data Elements of the DP-slave.

The User shall check the contents of the DXB-Subscribable provided with the Check User Prm service or Check Ext User Prm service and the configuration information provided with the Check Cfg service. In case of a positive result of these checks the User shall extract the necessary information from the DXB-Subscribable the DP Application Layer needs for subscription. The User shall load this information with the Load CRL DXB Link Entries service.

For this mapping the following rules shall apply:

The data areas of an Output Data Element may be assigned:

- completely to the DP-master (Class 1) (default) or
- partially to the DP-master (Class 1) and partially to one or more Publisher or
- completely to one or more Publisher.

The lowest level of mapping information contained in the DXB-Subscribtable is one octet, no bit mapping will be supported.

It is allowed that for one or more data areas of an Output Data Element no values are provided from the DP-master (Class 1) or from a Publisher. For these data areas default values (zero) shall be used.

A data area of an Output Data Element shall always be supplied completely either by the DP-master (Class 1) or by a Publisher. It is not allowed to provide values only for a part of a data area of an Output Data Element.

Variations of the data areas (length of the Output Data) which are supplied by the DP-master (Class 1) may be achieved by variations of the Simple or Extended Output Data Description for this Output Data Element.

For those Output Data Elements of a DP-slave where no Output Data are required from the DP-master (Class 1) the attribute Length of the Simple or Extended Output Data Description (no Output Data) shall be set to zero. For these modules no Output Data will be transferred from the DP-master (Class 1).

The data areas of an Output Data Element which are usually assigned to the DP-master (Class 1) should be placed at the beginning of the Output Data Element. In this case no mapping information is necessary in the DXB-Subscribtable (default mapping).

The following checks may be accomplished for each module at the Subscriber depending on the information provided with the Check Cfg service and the information contained in the DXB-Subscribtable.

Case 1:

The length indicated in the Simple or Extended Output Description of a module is set to zero (no Output Data from the DP-master (Class 1)) and the DXB-Subscribtable contains at least one entry for this module then the following rules shall be applied:

- The sum of the length given in the attribute Length Data Area of the entries in the DXB-Subscribtable for the Simple or Extended Output Data Element shall be the length of the Simple or Extended Output Data Element, if there exist an entry for each data area of the Output Data Element.
- Overlapping of the values of the attributes Offset Data Area and Length Data Area of the entries in the DXB-Subscribtable for this Output Data Element is not allowed.
- The DXB-Subscribtable shall not contain any DXB Subscriber Entry of the DP-master (Class 1) (Master Data) for this Output Data Element.
- Not covered data areas of the Output Data Element shall be supplied with default values.

Case 2:

The length indicated in the Simple or Extended Output Description of a module is unequal to zero and smaller than the complete length of the Output Data Element (parts of the Output Data for this Output Data Element will be provided by the DP-master (Class 1)) and the DXB-Subscribtable contains at least one entry for this module then the following rules shall be applied:

- The sum of the values of the attributes Length Data Area of the DXB Subscriber Entries in the DXB-Subscribtable of the DP-master (Class 1) (Master Data) shall be equal the length given in the Simple or Extended Output Data Description of this Output Data Element.
- If the DXB-Subscribtable contains no entries for the DP-master (Class 1) (Master Data) the values for this Output Data Element from the DP-master (Class 1) shall be mapped to the first data areas of this Output Data Element (default mapping).

- The mapping of the subscribed data of the Publisher shall be done according to the entries in the DXB-Subscribtable.
- Overlapping of the values of the attributes Offset Data Area and Length Data Area of the entries in the DXB-Subscribtable for this Output Data Element is not allowed also not with the data areas supplied by the DP-master (Class 1) via the default mapping.
- Not covered data areas of the Output Data Element shall be supplied with default values.

Case 3:

The length indicated in the Simple or Extended Output Description of a module is equal to the length of the Output Data Element (all Output Data for this Output Data Element will be provided by the DP-master (Class 1)) then the following rule shall be applied:

- The DXB-Subscribtable shall contain no entries for this Output Data Element.

The Context ASE also provides the means to change the address of a DP-slave via the MS0 AR. This may be done by a DP-master (Class 2).

The length of an Input Data Element of a module shall not be influenced by the possible variations of assignments of the data areas of the Output Data Element.

The Context ASE is used to establish or release an association between an individual AP of a DP-master (Class 2) and an individual AP of the DP-slave for acyclic communication. This association exclusively belongs to the Process Data ASE.

The formal description of the Context ASE is presented next, followed by a description of its services.

8.2.5.2 Context class specification

8.2.5.2.1 MS0 user parameter class specification

8.2.5.2.1.1 Template

This class specifies the object for the parameter data that may be related to the DP-slave itself and/or to the modules of the DP-slave.

The MS0 User Parameter object is described by the following template:

DP ASE:		Context ASE
CLASS:	MS0 User Parameter	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(o) Attribute:	User Prm Data
2.1.	(o) Attribute:	Device Related User Parameter
2.1.1	(m) Attribute:	Data Description
2.2.	(o) Attribute:	List of Module Related User Parameter
2.2.1	(m) Attribute:	Data Description
SERVICES:		
1.	(m) OpsService:	Check User Prm
2.	(m) OpsService:	Check User Prm Result

8.2.5.2.1.2 Attributes

Implicit

The attribute Implicit indicates that the User Parameter object is implicitly addressed by the services.

User Prm Data

This attribute is an aggregate of the Device and/or List of Module Related User Parameter and is composed of the following elements.

Device Related User Parameter

This attribute is composed of following elements:

Data Description

This attribute contains either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

List of Module Related User Parameter

This attribute is composed of following list elements (each element corresponds to a module).

Data Description

This attribute contains either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

8.2.5.2.1.3 Invocation of the MS0 user parameter object

For the invocation of the MS0 User Parameter object the following rules apply:

- The MS0 User Parameter object shall not exceed the total length of 237 octets (only 234 if the device attribute DPV1 = TRUE). The attributes Data Description and List of Module Related User Parameter have to be set accordingly.
- The MS0 User Parameter object shall exist only if the device attribute DPV1 is set to TRUE and the parameter Prm Structure provided with the Check User Prm service is set to FALSE or if the device attribute DPV1 is set to FALSE.
- Only one MS0 User Parameter object can be invoked in a DP-slave.

8.2.5.2.2 MS0 structured user parameter class specification

8.2.5.2.2.1 Template

In contrast to the MS0 User Parameter object this class defines separate objects for the device itself and for each of its modules. Furthermore these objects allow the unique identification and assignment.

This structuring should be used by enhanced devices with a high variation of parameters which may not be present in every application. This object is mandatory for DP-slaves which support the Isochronous Mode and for the conveyance of the DXB-Linktable.

The MS0 Structured User Parameter object is described by the following template:

DP ASE:		Context ASE
CLASS:	MS0 Structured User Parameter	
CLASS ID:	not used	
PARENT CLASS:	TOP	
ATTRIBUTES:		
1.	(m)	Key Attribute: Identifier
2.	(m)	Attribute: Structured User Prm Data
2.1.	(s)	Attribute: Device Related User Parameter
2.1.1	(m)	Attribute: Data Description
2.2.	(s)	Attribute: Module Related User Parameter
2.2.1	(m)	Attribute: Data Description
SERVICES:		
1.	(m)	OpsService: Check User Prm
2.	(o)	OpsService: Check Ext User Prm

3. (m) OpsService: Check User Prm Result
4. (o) OpsService: Check Ext User Prm Result

8.2.5.2.2.2 Attributes

Identifier

This key attribute is composed of Slot Number and Prm Data Type to indicate the number of the slot and the type of parameter data the specified MS0 Structured User Parameter object contains. If the attribute Slot Number contains the value 0 the following Structured Prm Data are Device Related User Parameter, otherwise they are Module Related User Parameter

Attribute Type of Slot Number: Unsigned8

Allowed values for Slot Number: 0 to 244

Attribute Type of Prm Data Type:

The allowed values of Prm Data Type are shown in Table 207.

Table 207 – Prm data type

Value	Meaning
129	Parameter Data
32–128	Manufacturer Specific

Structured User Prm Data

This attribute contains either Device or Module Related User Parameter indicated by the attribute Identifier and is composed of the following elements.

Device Related User Parameter

This attribute is composed of following elements:

Data Description

This attribute contains either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Module Related User Parameter

This attribute is composed of following elements.

Data Description

This attribute contains either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

8.2.5.2.2.3 Invocation of the MS0 structured user parameter object

For the invocation of the MS0 Structured User Parameter object the following rules apply:

- MS0 Structured User Parameter objects shall exist only if the device attribute DPV1 is set to TRUE.
- The MS0 Structured User Parameter objects may be provided with the Check User Prm service if the parameter Prm Structure provided with the Check User Prm service is set to TRUE.
- The description of all MS0 Structured User Parameter objects shall not exceed the total length of 230 octets if the values are provided with the Check User Prm service. The attribute Data Description has to be set accordingly.
- The description of all MS0 Structured User Parameter objects shall not exceed the total length of 240 octets if the values are provided with the Check Ext User Prm service. The attribute Data Description has to be set accordingly.
- Several MS0 Structured User Parameter objects may be invoked in a DP-slave.
- Only one MS0 Structured User Parameter object of the same type shall be invoked for each slot.

8.2.5.2.3 DXB-linktable class specification

8.2.5.2.3.1 Template

This class specifies the entries of the DXB-Linktable. Each entry specifies a Publisher the Subscriber has to subscribe from.

The DXB-Linktable object is described by the following template:

DP ASE:		Context ASE
CLASS:	DXB-Linktable	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of DXB Link Entries
2.1.	(m) Attribute:	Publisher Address
2.2.	(m) Attribute:	Input Data Length Publisher
2.3.	(m) Attribute:	Source Offset Publisher
2.4.	(m) Attribute:	Length
SERVICES:		
1.	(m) OpsService:	Check User Prm
2.	(o) OpsService:	Check Ext User Prm
3.	(m) OpsService:	Check User Prm Result
4.	(o) OpsService:	Check Ext User Prm Result

8.2.5.2.3.2 Attributes

Implicit

The attribute Implicit indicates that the DXB-Linktable object is implicitly addressed by the services.

List of DXB Link Entries

This attribute is composed of the following list elements.

Publisher Address

This attribute indicates the DL-address of a DP-slave (Publisher) the Subscriber shall subscribe from. The following rules shall apply for this attribute:

A value between 0 and 125 represents a valid Publisher address and therefore an active DXB-Link.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Input Data Length Publisher

This attribute defines the number of octets of the Simple or Extended Input Data object of the related Publisher.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Source Offset Publisher

This attribute defines the beginning (offset) of the subscription within the Simple or Extended Input Data object of the related Publisher given by the attribute Publisher Address.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Length

This attribute defines the number of octets the Subscriber shall subscribe from the related Publisher starting at the offset given in the attribute Source Offset Publisher.

Attribute Type: Unsigned8

Allowed values: 1 to 244

8.2.5.2.3.3 Invocation of the DXB-linktable object

For the invocation of the DXB-Linktable object the following rules apply:

- The DXB-Linktable object shall exist only if the device attribute DPV1 of the DP-slave is set to TRUE.
- Only one entry for each Publisher shall be contained in the DXB-Linktable.
- The values for the DXB-Linktable object may be provided with the Check User Prm service if the parameter Prm Structure provided with the Check User Prm service is set to TRUE.
- The description of the DXB-Linktable object shall not exceed the total length of 230 octets if the values of the DXB-Linktable object are provided with the Check User Prm service. The attribute List of DXB Link Entries has to be set accordingly.
- The description of the DXB-Linktable object shall not exceed the total length of 240 octets if the values of the DXB-Linktable object are provided with the Check Ext User Prm service. The attribute List of DXB Link Entries has to be set accordingly.
- Only one DXB-Linktable object shall be invoked in a DP-slave.
- The DXB-Linktable object shall only be invoked in a DP-slave if no DXB-Subscribabletable object is invoked and vice versa.

8.2.5.2.4 DXB-subscribabletable class specification

8.2.5.2.4.1 Template

This class specifies the entries of the DXB-Subscribabletable. Each entry specifies the source, a Publisher or the DP-master (Class 1), and the related mapping information needed to transfer the correspondent data to the data areas of an Output Data Element of the related module.

The DXB-Subscribabletable object is described by the following template:

DP ASE:		Context ASE
CLASS:	DXB-Subscribabletable	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of DXB Subscriber Entries
2.1.	(s) Attribute:	Publisher Address
2.1.1.	(m) Attribute:	Input Data Length Publisher
2.1.2.	(m) Attribute:	Source Offset Publisher
2.2.	(s) Attribute:	Master Data
2.2.1.	(m) Attribute:	Source Offset Master
2.3.	(m) Attribute:	Dest Slot Number
2.4.	(m) Attribute:	Offset Data Area
2.5.	(m) Attribute:	Length Data Area

SERVICES:

1. (m) OpsService: Check User Prm
2. (o) OpsService: Check Ext User Prm
3. (m) OpsService: Check User Prm Result
4. (o) OpsService: Check Ext User Prm Result

8.2.5.2.4.2 Attributes**Implicit**

The attribute Implicit indicates that the DXB-Subscribertable object is implicitly addressed by the services.

List of DXB Subscriber Entries

This attribute is composed of the following list elements.

Publisher Address

This selection attribute indicates the DL-address of a DP-slave (Publisher) the Subscriber shall subscribe from. The following rules shall apply for this attribute:

A value between 0 and 125 represents a valid Publisher address and therefore an active DXB-Link.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Input Data Length Publisher

This attribute defines the number of octets of the Simple or Extended Input Data object of the related Publisher.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Source Offset Publisher

This attribute defines the beginning (offset) of the subscription within the Simple or Extended Input Data object of the related Publisher given by the attribute Publisher Address.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Master Data

This selection attribute indicates that for this entry the data shall be derived from the related DP-master (Class 1).

Attribute Type: Unsigned8

Allowed values: 128

Source Offset Master

This attribute defines the beginning (offset) within the Output Data of the DP-master (Class 1).

Attribute Type: Unsigned8

Allowed values: 0 to 243

Dest Slot Number

This attribute indicates the Slot Number of the module to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Offset Data Area

This attribute defines the beginning (offset) within the Output Data of the module (data area) to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Length Data Area

This attribute defines the number of octets from the related Publisher the Subscriber shall subscribe to, starting at the offset given in the attribute Source Offset Publisher in case of a Publisher. If the data shall be derived from the DP-master (Class 1) the starting offset is given in the attribute Source Offset Master.

The subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped to the data area of the module given in the attributes Dest Slot Number and Offset Data Area of the related DXB Subscriber Entry.

Attribute Type: Unsigned8

Allowed values: 1 to 244

8.2.5.2.4.3 Invocation of the DXB-subscribable object

For the invocation of the DXB-Subscribable object the following rules apply:

- The DXB-Subscribable object shall exist only if the device attribute DPV1 of the DP-slave is set to TRUE.
- The values for the DXB-Subscribable object may be provided with the Check User Prm service if the parameter Prm Structure provided with the Check User Prm service is set to TRUE.
- The description of the DXB-Subscribable object shall not exceed the total length of 230 octets if the values of the DXB-Subscribable object are provided with the Check User Prm service. The attribute List of DXB Link Entries has to be set accordingly.
- The description of the DXB-Subscribable object shall not exceed the total length of 240 octets if the values of the DXB-Subscribable object are provided with the Check Ext User Prm service. The attribute List of DXB Link Entries has to be set accordingly.
- Only one DXB-Subscribable object shall be invoked in a DP-slave.
- The DXB-Subscribable object shall only be invoked in a DP-slave if no DXB-Linktable object is invoked and vice versa.

8.2.5.2.5 IsoM parameter class specification**8.2.5.2.5.1 Template**

This class specifies the necessary parameter for a DP-slave to operate in the Isochronous Mode.

The IsoM Parameter object is described by the following template:

DP ASE:		Context ASE
CLASS:	IsoM Parameter	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	T _{BASE_DP}
3.	(m) Attribute:	T _{DP}
4.	(m) Attribute:	T _{MAPC}
5.	(m) Attribute:	T _{BASE_IO}

- | | | | |
|-----|-----|------------|--------------|
| 6. | (m) | Attribute: | T_I |
| 7. | (m) | Attribute: | T_O |
| 8. | (m) | Attribute: | T_{DX} |
| 9. | (m) | Attribute: | T_{PLL_W} |
| 10. | (m) | Attribute: | T_{PLL_D} |

SERVICES:

- | | | | |
|----|-----|-------------|---------------------------|
| 1. | (m) | OpsService: | Check User Prm |
| 2. | (o) | OpsService: | Check Ext User Prm |
| 3. | (m) | OpsService: | Check User Prm Result |
| 4. | (o) | OpsService: | Check Ext User Prm Result |

8.2.5.2.5.2 Attributes**Implicit**

The attribute Implicit indicates that the IsoM Parameter object is implicitly addressed by the services.

TBASE_DP

This attribute indicates the time base (in units of $1/12\mu s$) for the time T_{DP} .

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

TDP

This attribute indicates the time T_{DP} (in units of T_{BASE_DP} . This time includes all parts of the Isochronous DP cycle:

- synchronisation message (SYNCH)
- cyclic services (DX)
- acyclic services (MSG)
- token (TOK)
- spare time (RES)

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

TMAPC

This attribute indicates the Master Application Cycle Time (in units of T_{DP}) the application of the DP-master (Class 1) needs to process an application task completely. From Isochronous point of view there is exactly one user task to control the isochronous part of application.

Allowed values: 1 to 14

TBASE_IO

This attribute indicates the time base (in units of $1/12\mu s$) for the times T_I and T_O of the DP-slave.

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

 T_I

This attribute indicates the time T_I (in units of T_{BASE_IO}) for the DP-slave.

Allowed values: 0 (special case), 1 to $2^{16}-1$

 T_O

This attribute indicates the time T_O (in units of T_{BASE_IO}) for the DP-slave.

Allowed values: 0 (special case), 1 to $2^{16}-1$

 T_{DX}

This attribute indicates the time T_{DX} (in units of $1/12\mu s$) for the DP-slave.

Allowed values: 0 (special case), 1 to $2^{32}-1$

T_{PLL_W}

This attribute indicates the time T_{PLL_W} (in units of 1/12μs) for the PLL of the DP-slave.

Allowed values: 1 to 2¹⁶-1 (default value 12)

T_{PLL_D}

This attribute indicates the time T_{PLL_D} (in units of 1/12μs) for the PLL of the DP-slave.

Attribute Type: Unsigned16

Allowed values: 0 to 2¹⁶-1

8.2.5.2.5.3 Invocation of the IsoM parameter object

For the invocation of the IsoM Parameter object the following rules apply:

- The IsoM Parameter object shall exist only if the device attribute DPV1 of the correspondent DP-slave is set to TRUE.
- The values of the IsoM Parameter object may be provided with the Check User Prm service if the parameter Prm Structure provided with the Check User Prm service is set to TRUE.
- The values of the IsoM Parameter object may be provided with the Check Ext User Prm service.
- Only one IsoM Parameter object shall be invoked in a DP-slave.

8.2.5.2.6 Time AR parameter class specification

8.2.5.2.6.1 Template

This class specifies the necessary parameter for a DP-slave to operate as a Time Receiver.

The Time AR Parameter object is described by the following template:

DP ASE:		Context ASE
CLASS:	Time AR Parameter	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	Clock Sync Interval
3.	(o) Attribute:	CS Delay Time
SERVICES:		
1.	(m) OpService:	Check User Prm
2.	(o) OpService:	Check Ext User Prm
3.	(m) OpService:	Check User Prm Result
4.	(o) OpService:	Check Ext User Prm Result

8.2.5.2.6.2 Attributes

Implicit

The attribute Implicit indicates that the Time AR Parameter object is implicitly addressed by the services.

Clock Sync Interval

This attribute contains the value for the attribute clock synchronisation interval of the ARL of the DP-slave operating as time receiver.

Allowed values: 1 to 2¹⁶-1; Default value: 100

Time Base: 10ms

CS Delay Time

This attribute contains the value for the attribute CS Delay Time of the ARL of the DP-slave operating as time receiver.

Default value: 0

Type: Network Time Difference

8.2.5.2.6.3 Invocation of the Time AR parameter object

For the invocation of the Time AR Parameter object the following rules have to be obeyed:

- The Time AR Parameter object shall exist only if the device attribute DPV1 of the correspondent DP-slave is set to TRUE.
- The values of the Time AR Parameter object may be provided with the Check User Prm service if the parameter Prm Structure provided with the Check User Prm service is set to TRUE.
- The values of the Time AR Parameter object may be provided with the Check Ext User Prm service.
- Only one Time AR Parameter object shall be invoked in a DP-slave.

8.2.5.2.7 MS0 configuration elements class specification

8.2.5.2.7.1 Template

The MS0 Configuration Elements object is described by the following template:

DP ASE:		Context ASE
CLASS:	MS0 Configuration Elements	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of Cfg Data
2.1.	(o) Attribute:	Input Data Reference
2.2.	(o) Attribute:	Output Data Reference
2.3.	(o) Attribute:	Manufacturer Specific Data
2.4.	(o) Attribute:	Module Name
SERVICES:		
1.	(m) OpsService:	Set Cfg
2.	(m) OpsService:	Get Cfg
3.	(m) OpsService:	Check Cfg
4.	(m) OpsService:	Check Cfg Result

8.2.5.2.7.2 Attributes

Implicit

The attribute Implicit indicates that the MS0 Configuration Elements object is implicitly addressed by the services.

List of Cfg Data

This attribute is an aggregate of the configuration data of a DP-slave which is described by Cfg Identifier or Special Cfg Identifier. It shall contain at least one Cfg Data entry. A Cfg Data entry consists of the following attributes.

Input Data Reference

Reference to an Input Data Element of a Simple or an Extended Input Data object.

Output Data Reference

Reference to an Output Data Element of a Simple or an Extended Output Data object.

Manufacturer Specific Data

This attribute contains a manufacturer specific addition to the Input and/or Output Data Reference attributes. The contents are device specific or module specific.

Attribute Type: Octet string

Module Name

This attribute defines a name of a list element.

Attribute Type: Visible String(32)

8.2.5.2.7.3 Invocation of the MS0 configuration elements object

For the invocation of the MS0 Configuration Elements object the following rules apply:

- Only one MS0 Configuration Elements object shall be invoked in a DP-slave
- The description of the MS0 Configurations Elements object shall not exceed 244 octets. The attribute List of Cfg Data has to be set accordingly.

8.2.5.2.8 Remanent parameter class specification

8.2.5.2.8.1 Template

A Remanent Parameter object is described by the following template:

DP ASE:	Context ASE
CLASS:	Remanent Parameter
CLASS ID:	not used
PARENT CLASS:	TOP
ATTRIBUTES:	
1. (m)	Key Attribute: Implicit
2. (m)	Attribute: No Add Change
3. (m)	Attribute: New Slave Add
4. (o)	Attribute: Rem Para
4.1 (m)	Attribute: Data Description
SERVICES:	
1. (m)	OpsService: Set Slave Add

8.2.5.2.8.2 Attributes

Implicit

The attribute Implicit indicates that an Remanent Parameter object is implicitly addressed by the service.

No Add Change

This attribute specifies whether it is allowed to change the DP-slave address again. After the initial reset the DP-slave DL-address will be set to the default address (126) and the attribute No Add Change will be set to FALSE. To change this attribute from TRUE to FALSE an initial reset of the DP-slave is necessary.

Allowed values: FALSE, TRUE

New Slave Add

This attribute contains the address of the DP-slave which is to be set.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Rem Para

This attribute contains parameters to be stored permanently.

Data Description

This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

8.2.5.2.8.3 Invocation of Rem Para object

For the invocation of the Remanent Parameter object the following rules apply:

- Only one Remanent Parameter object shall be invoked in a DP-slave.
- The Remanent Parameter object shall not exceed 242 octets. The attribute Data Description has to be set accordingly.

8.2.5.2.9 MS2 user parameter class specification

8.2.5.2.9.1 Template

The MS2 User Parameter object is described by the following template:

DP ASE:		Context ASE
CLASS:	MS2 User Parameter	
CLASS ID:	not used	
PARENT CLASS:	TOP	
ATTRIBUTES:		
1.	(m)	Key Attribute: Implicit
2.	(m)	Attribute: Features Supported
2.1.	(m)	Attribute: Supported Feature
3.	(m)	Attribute: Profile Features Supported
3.1.	(m)	Attribute: Supported Profile Feature
4.	(m)	Attribute: Profile Ident Number
SERVICES:		
1.	(m)	OpsService: Initiate
2.	(m)	OpsService: Abort

8.2.5.2.9.2 Attributes

Implicit

The attribute Implicit indicates that the MS2 User Parameter object is implicitly addressed by the services.

Features Supported

This attribute is composed of the following list elements:

Supported Feature

This attribute reflects the actual features on the MS2 AR of a DP-slave. The DP-master (Class 2) and the DP-slave inform each other about the supported service functionality. The DP-slave and the DP-master (Class 2) have the possibility to adjust their functionality according to the requirements. The User application of the DP-master has to decide if the reported functionality is sufficient or not.

The allowed values are shown in Table 208.

Table 208 – Supported feature

Value	Meaning
1	Read and Write service supported
2 to 16	reserved for further features

Profile Features Supported

This attribute is composed of the following list elements:

Supported Profile Feature

This attribute reflects the actual profile features on the MS2 AR of a DP-slave. The DP-master (Class 2) and the DP-slave inform each other about the supported profile features. Regarding the used profile definition, the profile is identified by the Profile Ident Number. The User application of the DP-master has to decide if the reported functionality is sufficient or not.

The allowed values are shown in Table 209.

Table 209 – Supported profile feature

Value	Meaning
1 to 16	feature to be defined by profiles

Profile Ident Number

The attribute Profile Ident Number contains the unique identifier of a profile definition. All devices using the same profile definition have to use the same Profile Ident Number. The Profile Ident Number will be taken from the pool of Ident Numbers for vendor specific or authorised profiles. The value 0 indicates that no profile is supported.

Attribute type: Unsigned16

8.2.5.2.9.3 Invocation of the MS2 user parameter object

For the invocation of the MS2 user parameter object the following rules have to be obeyed:

- Only one MS2 user parameter object shall be invoked in a DP-slave.

8.2.5.2.10 Red Context class specification

8.2.5.2.10.1 Template

This class specifies the status of the redundant communication interfaces of a DP-slave and the state of a previous provided switchover command.

The following template describes the Red Context object:

DP ASE:		Context ASE
CLASS:	Red Context	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Communication Interface Identifier
2.	(m) Attribute:	Role
SERVICES:		
1.	(m) OpsService:	Check User Prm
2.	(m) OpsService:	Check User Prm Result

8.2.5.2.10.2 Attributes

Communication Interface Identifier

This attribute identifies uniquely the communication interface.

Role

This attribute defines the actual role of the communication interface.

Allowed values: Primary, Backup, Power On

Table 210 – Role

Value	Meaning
Primary	At the primary interface there exists no constraints between the communication interface and the ASE's.
Backup	In case of Backup the following items between the communication interface and the ASE's are active: Diagnosis ASE (all services) Context ASE (Check User Prm, Check Ext User Prm, Check Cfg) Process Data ASE (all services, only via MS2 AR) Load Region ASE (all services, only via MS2 AR) Function Invocation ASE (all services, only via MS2 AR)
Power On	In case of Power On the following items between the communication interface and the ASE's are active: Diagnosis ASE (all services) Context ASE (Check User Prm, Check Ext User Prm)

8.2.5.2.10.3 Invocation of the Red Context object

For the invocation of the Red Context object the following rules have to be obeyed:

- The Red Context object shall be used only if the device attribute DPV1 is set to TRUE.
- For each communication interface only one Red Context object shall be invoked.

8.2.5.3 Context service specification

8.2.5.3.1 Check user Prm

The DP Application Layer of the DP-slave notifies the application when a valid parameterisation is received from the DP-master because this service requests a validation of the User Prm Data or List of Structured Prm Data. A further Check User Prm indication is only issued after the Check User Prm service is completed with a Check User Prm Result service.

The Check User Prm service shall be acknowledged negatively (the parameter Prm OK of the service Check User Prm Result shall be set to FALSE) by the application of the DP-slave if the following conditions are met:

- the parameter Prm Structure is set to FALSE and the parameterisation data (User Prm Data) are not allowed or not acceptable by the application of the DP-slave
- the parameter Prm Structure is set to TRUE and at least one of the MS0 Structured User Parameter object, the DXB-Linktable object, the DXB-Subscribable object, or the IsoM Parameter object does not exist
- the parameter Prm Structure is set to TRUE and at least one value for a MS0 Structured User Parameter object, or for the DXB-Linktable object, or for the DXB-Subscribable object, or for the IsoM Parameter object is not allowed or not acceptable by the application of the DP-slave.

Table 211 shows the parameters of the service.

Table 211 – Check user Prm

Parameter name	Ind
Argument	M
AREP	M
Prm Structure	M
User Prm Data	C
Device Prm Data	U
Module Prm Data	U
List of Structured Prm Data	C
Structured User Prm Data	U
Device Related User Parameter	U
Device Prm Data	M
Module Related User Parameter	U
Slot Number	M
Module Prm Data	M
DXB-Linktable	U
List of DXB Link Entries	M
Publisher Address	M
Input Data Length Publisher	M
Source Offset Publisher	M
Length	M
DXB-Subscribtable	U
List of DXB Subscriber Entries	M
Publisher Address	C
Input Data Length Publisher	M
Source Offset Publisher	M
Master Data	C
Source Offset Master	M
Dest Slot Number	M
Offset Data Area	M
Length Data Area	M
IsoM Parameter	U
TBASE_DP	M
TDP	M
TMAPC	M
TBASE_IO	M
T _I	M
T _O	M
T _{DX}	M
TPLL_W	M
TPLL_D	M
Prm Command	U
Seq Number	M
Primary Request	M
MS1 Command	M
Master State Clear	M
Check Properties	M
Primary Request used	M
MS1 Command used	M
Address Change	M
Address Offset	M
Output Hold Time	M
Time AR Parameter	U
Clock Sync Interval	M
CS Delay Time	U

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the AR.

Prm Structure

This parameter indicates which type of parameterisation objects are provided by the service Check User Prm. This parameter shall only be evaluated if the device attribute DPV1 is set to TRUE.

The allowed values are shown in Table 212.

Table 212 – Prm structure

Value	Meaning
TRUE	The parameter which follow are related to the following objects: MS0 Structured User Parameter DXB-Linktable IsoM Parameter
FALSE	The parameter which follow are only related to the MS0 User Parameter object.

User Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value FALSE. It is composed of one or both of the following subparameters.

Device Prm Data

This parameter contains the value for the Device Related User Parameter of the MS0 User Parameter object.

Module Prm Data

This parameter contains the value for the Module Related User Parameter of the MS0 User Parameter object.

List of Structured Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value TRUE. This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value for the Device Related User Parameter of the MS0 Structured User Parameter object.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value of the attribute Slot Number of the MS0 Structured User Parameter object.

Module Prm Data

This parameter contains the value for the Module Related User Parameter of the MS0 Structured User Parameter object.

DXB-Linktable

This parameter indicates the presence of the DXB-Linktable.

List of DXB Link Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of one DXB-Linktable entry of the DXB-Linktable object.

Input Data Length Publisher

This parameter contains the value of the attribute Input Data Length Publisher of one DXB-Linktable entry of the DXB-Linktable object.

Source Offset Publisher

This parameter contains the value of the attribute Source Offset Publisher of one DXB-Linktable entry of the DXB-Linktable object.

Length

This parameter contains the value of the attribute Length of one DXB-Linktable entry of the DXB-Linktable object.

DXB-Subscribable

This parameter indicates the presence of the DXB-Subscribable.

List of DXB Subscriber Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of one DXB-Subscribable entry of the DXB-Subscribable object.

Input Data Length Publisher

This parameter contains the value of the attribute Input Data Length Publisher of one DXB-Subscribable entry of the DXB-Subscribable object.

Source Offset Publisher

This parameter contains the value of the attribute Source Offset Publisher of one DXB-Subscribable entry of the DXB-Subscribable object.

Master Data

This parameter contains the value of the attribute Master Data of one DXB-Subscribable entry of the DXB-Subscribable object.

Source Offset Master

This parameter contains the value of the attribute Source Offset Master of one DXB-Subscribable entry of the DXB-Subscribable object.

Dest Slot Number

This parameter contains the value of the attribute Dest Slot Number of one DXB-Subscribable entry of the DXB-Subscribable object.

Offset Data Area

This parameter contains the value of the attribute Offset Data Area of one DXB-Subscribable entry of the DXB-Subscribable object.

Length Data Area

This parameter contains the value of the attribute Length Data Area of one DXB-Subscribable entry of the DXB-Subscribable object.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value of the attribute TBASE_DP of the IsoM Parameter object.

TDP

This parameter contains the value of the attribute TDP of the IsoM Parameter object.

TMAPC

This parameter contains the value of the attribute TMAPC of the IsoM Parameter object.

TBASE_IO

This parameter contains the value of the attribute TBASE_IO of the IsoM Parameter object.

T_I

This parameter contains the value of the attribute T_I of the IsoM Parameter object.

T_O

This parameter contains the value of the attribute T_O of the IsoM Parameter object.

T_{DX}

This parameter contains the value of the attribute T_{DX} of the IsoM Parameter object.

T_{PLL_W}

This parameter contains the value of the attribute T_{PLL_W} of the IsoM Parameter object.

T_{PLL_D}

This parameter contains the value of the attribute T_{PLL_D} of the IsoM Parameter object.

Prm Command

This parameter indicates the presence of the Prm Command.

Seq Number

This parameter contains the Sequence Number necessary for the unique identification of the received Prm Command.

Allowed values: 0 to 31

Primary Request

If this parameter is set to TRUE the selected communication interface shall operate as the primary one. All MS1 functions are active and will be processed as if redundancy is not activated. Also all diagnosis informations including channel related, or other status messages, have to be present.

This parameter set to FALSE means that the role of the communication interface (primary, backup) shall not be changed.

Allowed values: TRUE, FALSE

MS1 Command

This parameter defines the desired command for the MS1 AR of the selected communication interface.

The allowed values are shown in Table 213.

Table 213 – MS1 Command

Value	Meaning
no Action	No Command for the MS1 AR is present.
Stop MS1	This parameter specifies when the MS1 AR of the DP-slave shall be stopped by the DP Application Layer. No MS1 AR services are possible any longer.
Start MS1	This parameter specifies when the MS1 AR of the DP-slave shall be (re)started by the DP Application Layer. After the (re)start all MS1 AR services are possible if the role of the communication interface is primary.
Reset MS1	Means that the MS1 AR shall be stopped and then be started afterwards.

Master State Clear

This parameter indicates the actual state of the MS0 AR of the DP-master (class 1). This parameter set to TRUE means that the MS0 AR of the DP-master (class 1) is in the operation mode CLEAR. This parameter set to FALSE means that the MS0 AR of the DP-master (class 1) is in the operation mode OPERATE.

Allowed values: TRUE, FALSE

Check Properties

This parameter indicates whether the DP-slave shall check the properties provided by the Check User Prm service. The parameter set to TRUE indicates that properties shall be checked (e. g. at the start-up of the DP-slave). This parameter set to FALSE indicates that the check of the properties shall be omitted (e. g. at the data exchange mode of the DP-slave).

Allowed values: TRUE, FALSE

Primary Request used

This parameter indicates whether the communication interface of a redundant DP-slave supports primary requests. The DP-slave shall respond negatively (with the parameter Prm OK of the Check User Prm service set to FALSE) if those requests are not supported. In the other case the Check Properties shall be accepted.

Allowed values: TRUE, FALSE

MS1 Command used

This parameter indicates whether the MS1 Commands Start MS1, Stop MS1 and Reset MS1 are supported by the communication interface of a redundant DP-slave. The DP-slave shall respond negatively (with the parameter Prm OK of the Check User Prm service set to FALSE) if those requests are not supported. In the other case the Check Properties shall be accepted.

Allowed values: TRUE, FALSE

Address Change

This parameter indicates whether address changes are requested with the primary request. TRUE indicates that address changes are requested and FALSE indicates that no address changes are requested. The DP-slave shall respond negatively (with the parameter Prm OK of the Check User Prm service set to FALSE) if those requests are not supported. In the other case the Check Properties shall be accepted.

Allowed values: TRUE, FALSE

Address Offset

This parameter indicates if the address offset of 64 will be used at the start-up of the DP-slave or not. The DP-slave shall respond negatively (with the parameter Prm OK of the Check User Prm service set to FALSE) if the address offset is not supported. In the other case the Check Properties shall be accepted.

Allowed values: TRUE, FALSE

Output Hold Time

This parameter indicates the time the redundant DP-slave shall not change its outputs when it performs a redundancy switchover. When the backup communication interface becomes the primary (and vice versa) the User may use the output values previously transmitted from the old primary communication interface or the User has to wait for the first data exchange after the switchover to write new values to its outputs. A possible switchover from backup to primary will always start the Output_Hold_Time.

Timebase: 10ms.

Allowed values: 0 to $2^{16}-1$

Time AR Parameter

This parameter indicates the presence of the Time AR Parameter.

Clock Sync Interval

This parameter contains the value for the attribute Clock Sync Interval of the ARL DP-slave object.

CS Delay Time

This parameter contains the value for the attribute CS Delay Time of the ARL DP-slave object.

8.2.5.3.2 Check user Prm result

This service is used in a DP-slave as a reply to the previous issued Check User Prm service. The Default AP of a DP-slave uses this service to provide the result of the validation of the User Prm Data or of the List of Structured Prm Data. This service shall only be used in conjunction with the MS0 AR.

Table 214 shows the parameters of the service.

Table 214 – Check user Prm result

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Prm OK	M	
Result(+)		S
AREP		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Prm OK

The parameter Prm OK equal TRUE specifies that the User Prm Data or the List of Structured Prm Data conveyed by the previous Check User Prm service are acceptable by the application of the DP-slave. If they are not acceptable for the application of the DP-slave, this parameter shall have the value FALSE. In this case the attributes Prm Req and Prm Fault of the related MS0 ARL Entry will be set to TRUE by the DP Application Layer.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed. The DP-slave shall ignore the DP-masters Parameterisation.

Status

This parameter indicates the cause of the failure.

The allowed values are shown in Table 215.

Table 215 – Status values

Value	Meaning
Reset	Reset still in progress
Not Pending	No Validation pending
New Prm	New User Prm Data or List of Structured Prm Data received during validation
Inv Master Add	A different DP-master has parameterised the DP-slave or the DP-master has released the DP-slave (the validation of the previous User Prm Data or List of Structured Prm Data was useless).

8.2.5.3.3 Check Ext user Prm

This service shall only be used if the MS0 AR offers an additional CR for the conveyance of the parameterisation data and the device attribute DPV 1 is set to TRUE.

The DP Application Layer of the DP-slave notifies the application if additional parameterisation data are received from the DP-master (Class 1). This service requests a validation of the List of Structured Prm Data. A further Check Ext User Prm indication is only issued after the Check Ext User Prm service is completed with a Check Ext User Prm Result service.

The Check Ext User Prm service shall be acknowledged negatively (the parameter Ext Prm OK of the service Check Ext User Prm Result shall be set to FALSE) if at least one parameterisation object is unknown or not allowed or acceptable by the application of the DP-slave.

Table 216 shows the parameters of the service.

Table 216 – Check Ext user Prm

Parameter name	Ind
Argument	M
AREP	M
List of Structured Prm Data	M
Structured User Prm Data	U
Device Related User Parameter	U
Device Prm Data	M
Module Related User Parameter	U
Slot Number	M
Module Prm Data	M
DXB-Linktable	U
List of DXB Link Entries	M
Publisher Address	M
Input Data Length Publisher	M
Source Offset Publisher	M
Length	M
DXB-Subscribertable	U
List of DXB Subscriber Entries	M
Publisher Address	C
Input Data Length Publisher	M
Source Offset Publisher	M
Master Data	C
Source Offset Master	M
Dest Slot Number	M
Offset Data Area	M
Length Data Area	M
IsoM Parameter	U
TBASE_DP	M
TDP	M
TMAPC	M
TBASE_IO	M
T _I	M
T _O	M
T _{DX}	M
T _{PLL_W}	M
T _{PLL_D}	M
Time AR Parameter	U
Clock Sync Interval	M
CS Delay Time	U

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the AR.

List of Structured Prm Data

This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value for the Device Related User Parameter of the MS0 Structured User Parameter object.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value of the attribute Slot Number of the MS0 Structured User Parameter object.

Module Prm Data

This parameter contains the value for the Module Related User Parameter of the MS0 Structured User Parameter object.

DXB-Linktable

This parameter indicates the presence of the DXB-Linktable.

List of DXB Link Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of one DXB-Linktable entry of the DXB-Linktable object.

Input Data Length Publisher

This parameter contains the value of the attribute Input Data Length Publisher of one DXB-Linktable entry of the DXB-Linktable object.

Source Offset Publisher

This parameter contains the value of the attribute Source Offset Publisher of one DXB-Linktable entry of the DXB-Linktable object.

Length

This parameter contains the value of the attribute Length of one DXB-Linktable entry of the DXB-Linktable object.

DXB-Subscribable

This parameter indicates the presence of the DXB-Subscribable.

List of DXB Subscriber Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the attribute Publisher Address of one DXB-Subscribable entry of the DXB-Subscribable object.

Input Data Length Publisher

This parameter contains the value of the attribute Input Data Length Publisher of one DXB-Subscribable entry of the DXB-Subscribable object.

Source Offset Publisher

This parameter contains the value of the attribute Source Offset Publisher of one DXB-Subscribable entry of the DXB-Subscribable object.

Master Data

This parameter contains the value of the attribute Master Data of one DXB-Subscribable entry of the DXB-Subscribable object.

Source Offset Master

This parameter contains the value of the attribute Source Offset Master of one DXB-Subscribable entry of the DXB-Subscribable object.

Dest Slot Number

This parameter contains the value of the attribute Dest Slot Number of one DXB-Subscribable entry of the DXB-Subscribable object.

Offset Data Area

This parameter contains the value of the attribute Offset Data Area of one DXB-Subscribable entry of the DXB-Subscribable object.

Length Data Area

This parameter contains the value of the attribute Length Data Area of one DXB-Subscribable entry of the DXB-Subscribable object.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value of the attribute TBASE_DP of the IsoM Parameter object.

TDP

This parameter contains the value of the attribute TDP of the IsoM Parameter object.

TMAPC

This parameter contains the value of the attribute TMAPC of the IsoM Parameter object.

TBASE_IO

This parameter contains the value of the attribute TBASE_IO of the IsoM Parameter object.

T_I

This parameter contains the value of the attribute T_I of the IsoM Parameter object.

T_O

This parameter contains the value of the attribute T_O of the IsoM Parameter object.

T_{DX}

This parameter contains the value of the attribute T_{DX} of the IsoM Parameter object.

TPLL_W

This parameter contains the value of the attribute TPLL_W of the IsoM Parameter object.

TPLL_D

This parameter contains the value of the attribute TPLL_D of the IsoM Parameter object.

Time AR Parameter

This parameter indicates the presence of the Time AR Parameter.

Clock Sync Interval

This parameter contains the value for the attribute Clock Sync Interval of the ARL DP-slave object.

CS Delay Time

This parameter contains the value for the attribute CS Delay Time of the ARL DP-slave object.

8.2.5.3.4 Check Ext user Prm result

This service is used in a DP-slave as a reply to the previous issued Check Ext User Prm service. The Default AP of a DP-slave uses this service to provide the result of the validation of the List of Structured Prm Data. This service shall only be used in conjunction with the MS0 AR.

Table 217 shows the parameters of the service.

Table 217 – Check Ext user Prm result

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Ext Prm OK	M	
Result(+)		S
AREP		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Ext Prm OK

The parameter Ext Prm OK equal TRUE specifies that the List of Structured Prm Data conveyed by the previous Check Ext User Prm service are acceptable by the application of the DP-slave. If the List of Structured Prm Data are not acceptable for the application of the DP-slave, this parameter shall have the value FALSE. In this case the attributes Prm Req and Cfg Fault of the related MS0 ARL Entry will be set to TRUE by the DP Application Layer.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed. The DP-slave shall ignore the DP-masters Parameterisation.

Status

This parameter indicates the cause of the failure.

The allowed values are shown in Table 218.

Table 218 – Status values

Value	Meaning
Reset	Reset still in progress
Not Pending	No Validation pending
New Prm	New List of Structured Prm Data received during validation
Inv Master Add	A different DP-master has parameterised the DP-slave or the DP-master has released the DP-slave (the validation of the previous List of Structured Prm Data was useless).

8.2.5.3.5 Check Cfg

The DP Application Layer of the DP-slave notifies the application when Cfg Data (Configuration Data) are received from the DP-master because this service requests a validation of the Cfg Data by the application. A further Check Cfg indication is only issued

after the Check Cfg service is completed with a Check Cfg Result service. The DP-slave compares its own configuration data with the Cfg Data received from the DP-master.

The check shall cover the following details:

- module structure
- range of Input and/or Output Data Element
- format
- consistency.

The consistency also affects the DP-slave and the DP-master. The verification of the consistency in the Cfg Data list element shall cause only in the following cases a configuration fault:

- The DP-slave needs consistency for an Input and/or Output Data Element and the DP-master indicates no consistency.
- The DP-slave can not provide consistency for an Input and/or Output Data Element and the DP-master requires consistency for this Input and/or Output Data Element.

DP-slaves may accept different sets of Cfg Data. In this case the Check Cfg shall be successful if one set of configuration data matches. If the service parameter Check Cfg Mode is set to TRUE the check of the Cfg Data may be done in a different User specific way. For example a temporary not available module may be accepted even if the Cfg Data contains the respective Configuration Identifier.

Table 219 shows the parameters of the service.

Table 219 – Check Cfg

Parameter name	Ind
Argument	M
AREP	M
Check Cfg Mode	C
Cfg Data	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the AR.

Check Cfg Mode

This parameter indicates how the DP-slave application has to check the Cfg Data. This parameter shall only be evaluated if the device attribute DPV1 is set TRUE.

Cfg Data

This parameter contains the Cfg Data to be verified by the application of the DP-slave.

8.2.5.3.6 Check Cfg result

This service is used in a DP-slave as a reply to the previous issued Check Cfg service. The Default AP of the DP-slave uses this service to provide the result of the validation of the Cfg Data. This service shall only be used in conjunction with the MS0 AR.

Table 220 shows the parameters of the service.

Table 220 – Check Cfg result

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Cfg OK	M	
Input Data Len	M	
Output Data Len	M	
Result(+)		S
AREP		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Cfg OK

The parameter Cfg OK equal TRUE specifies that the Cfg Data conveyed by the previous Check Cfg service are acceptable by the application of the DP-slave. The application of the DP-slave may update the Real Cfg Data of the MS0 AR with the Set Cfg service. If the Cfg Data are not acceptable for the application of the DP-slave, this parameter shall have the value FALSE. In this case the attributes Prm Req and Cfg fault will be set to TRUE by the DP Application Layer.

Input Data Len

This parameter contains the actual length of the Input Data.

Output Data Len

This parameter contains the actual length of the Output Data.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the cause of the failure.

The allowed values are shown in Table 221.

Table 221 – Status values

Value	Meaning
Reset	Reset still in progress
Not Pending	No Validation of Cfg Data pending
New Cfg	New Cfg Data received during validation
Inv Master Add	A different DP-master has parameterised the DP-slave or the DP-master has released the DP-slave (the validation of the previous Cfg Data was useless).

8.2.5.3.7 Set Cfg

This service is used by the application of the DP-slave to pass configuration data to the DP-Application Layer. This service shall only be used in conjunction with the MS0 AR.

Table 222 shows the parameters of the service.

Table 222 – Set Cfg

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Cfg Data	M	
Result(+)		S
AREP		M
Result(-)		S
AREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Cfg Data

This parameter contains the Cfg Data.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.5.3.8 Get Cfg

This service allows the DP-master (Class 2) to read the configuration data of the DP-slave via the MS0 AR.

Table 223 shows the parameters of the service.

Table 223 – Get Cfg

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Cfg Data		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Cfg Data

This parameter contains the Cfg Data.

Result(-)

This selection type parameter indicates that the service request failed.

Status

The parameter Status indicates the type of failure of the service.

Allowed values: DS, NA, RS, UE, NR, RE

8.2.5.3.9 Set slave add

This service permits a DP-master (Class 2) to change the address of a DP-slave and to pass remanent Slave Data. The Set Slave Add service is only successful if the Ident Number transmitted corresponds with the Ident Number of the DP-slave and the attribute No Add Change (see 8.2.10.3.1) of the DP-slave has the value FALSE. This service shall only be used in conjunction with the MS0 AR.

Table 224 shows the parameters of the service.

Table 224 – Set Slave Add

Parameter name	Req	Ind	Cnf
Argument	M	M(=)	
AREP	M	M(=)	
New Slave Add	M	M(=)	
Ident Number	M	M(=)	
No Add Chg	M	M(=)	
Rem Slave Data	U	C(=)	
Result(+)			S
AREP			M
Result(-)			S
AREP			M
Status			M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

New Slave Add

This parameter contains the value of the attribute New Slave Add of the Remanent Parameter object of the DP-slave which is to be set.

Ident Number

This parameter contains the Ident Number to be checked with the Ident Number of the DP-slave.

No Add Chg

This parameter contains the value of the attribute No Add Change of the Remanent Parameter object. The attribute No Add Change will assume the value of this service parameter in case of success, otherwise No Add Change remains unchanged.

Rem Slave Data

This parameter contains the value of the attribute Rem Para of the Remanent Parameter object.

Result(+)

This selection type parameter indicates that the service request succeeded. This confirmation does not reflect whether the service was successful, i. e. the new values were accepted by the DP-slave. Therefore the DP-master shall check the correct execution of the service by using the Read Slave Diag service with the new DP-slave address.

Result(-)

This selection type parameter indicates that the service request failed.

Status

The parameter Status indicates the type of failure of the service.

Allowed values: DS, NA, RS, RR, UE, RE

8.2.5.3.10 Initiate

This service shall be used by a DP-master (Class 2) to establish a connection to a desired DP-slave. This service shall only be used in conjunction with a MS2 AR.

Table 225 shows the parameters of the service.

Table 225 – Initiate

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Send Timeout	M			
Features Supported	M	M(=)		
Profile Features Supported	M	M(=)		
Profile Ident Number	M	M(=)		
Add Addr Param	M	M(=)		
S-Type	M	M(=)		
S-Len	M	M(=)		
D-Type	M	M(=)		
D-Len	M	M(=)		
S-Addr	M	M(=)		
S-API	M	M(=)		
S-SCL	M	M(=)		
S-Network Address	C	C(=)		
S-MAC Address	C	C(=)		
D-Addr	M	M(=)		
D-API	M	M(=)		
D-SCL	M	M(=)		
D-Network Address	C	C(=)		
D-MAC Address	C	C(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Max Len Data Unit			M	M(=)
Features Supported			M	M(=)
Profile Features Supported			M	M(=)
Profile Ident Number			M	M(=)
Add Addr Param			M	M(=)
S-Type			M	M(=)
S-Len			M	M(=)
D-Type			M	M(=)
D-Len			M	M(=)
S-Addr			M	M(=)
S-API			M	M(=)
S-SCL			M	M(=)
S-Network Address			C	C(=)
S-MAC Address			C	C(=)
D-Addr			M	M(=)
D-API			M	M(=)
D-SCL			M	M(=)
D-Network Address			C	C(=)
D-MAC Address			C	C(=)

Parameter name	Req	Ind	Rsp	Cnf
Result(-)			S	S(=)
AREP			M	M(=)
Error Decode			M	M(=)
Error Code 1			M	M(=)
Error Code 2			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Max Len Data Unit

This parameter contains the value of the attribute Max Len Data Unit of the ARL DP-slave object of a DP-slave.

Send Timeout

This parameter defines the control time for monitoring the MS2 AR requested by the DP-master (Class 2). DP-master and DP-slave inform each other about the supported Send Timeout. The DP-slave adjust its Send Timeout. The DP-master (Class 2) decides whether the reported Send Timeout is sufficient or not.

Features Supported

DP-master and DP-slave inform each other about the supported service functionality. The DP-slave has the possibility to adjust its functionality to the Master's requirements. The User application of the DP-master has to decide if the reported functionality is sufficient or not.

Profile Features Supported

DP-master and DP-slave inform each other about the supported service functionality regarding the used profile definition. The profile is identified by the Profile Ident Number. The meaning of the defined bits is profile or vendor specific.

Profile Ident Number

This parameter identifies a profile definition uniquely. All devices using the same profile definition have to use the same Profile Ident Number. The Profile Ident Number will be taken from the pool of Ident Numbers for vendor specific or authorised profiles. The value 0 indicates that no profile is supported. If the requested profile is supported by the DP-slave, the Profile Ident Number is mirrored in the response. If the requested profile is not supported by the DP-slave, the DP-slave responds negatively or with a Profile Ident Number supported by the DP-slave.

Add Addr Param

Add Addr Param consists of two parts, additional address information of the source and additional address information of the destination. The additional address information contains the parameter API plus an optional Network/MAC address.

D-Addr and S-Addr (API, SCL, Network Address, MAC Address) are related to the direction of the telegrams, which means the source becomes destination and vice versa in the response.

Add Addr Param contains the following subparameters:

S-Type

This subparameter indicates the presence (S-Type = 1) of the optional Network/MAC address in the Add Addr Param of the source.

S-Len

This subparameter indicates the length of the S-Addr subparameter.

D-Type

This subparameter indicates the presence (D-Type = 1) of the optional Network/MAC address in the Add Addr Param of the destination.

D-Len

This subparameter indicates the length of the D-Addr subparameter.

S-Addr

This subparameter contains the additional address information of the source:

S-API

This subparameter identifies the AP of the source.

S-SCL

This subparameter identifies the access level of the source.

S-Network Address

In case of S-Type = 1 this subparameter identifies the network address of the source according to ISO/OSI-Network addresses.

S-MAC Address

In case of S-Type = 1 this subparameter identifies the MAC Address of the source.

D-Addr

This subparameter contains the additional address information of the destination:

D-API

This subparameter identifies the AP of the destination.

D-SCL

This subparameter identifies the access level of the destination.

D-Network Address

In case of D-Type = 1 this subparameter identifies the network address of the destination according to ISO/OSI-Network addresses.

D-MAC Address

In case of D-Type = 1 this subparameter identifies the MAC Address of the destination.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Decode

This parameter selects one of the following Error schemes: DPV1, FMS, Profile_Specific.

Error Code 1

In case of Error Decode equal DPV1 the Error Code 1 assumes one of the following values:

version conflict, feature not supported, user specific, state conflict, invalid range, invalid parameter, invalid type, resource busy, resource unavailable.

Error Code 2

The parameter Error Code 2 is user specific.

8.2.5.3.11 Abort

The DP-master and DP-slave use this service to abort an AR. With the Abort.req the User resets the FSPM. In faulty conditions the FSPM informs the User with an Abort.ind and resets itself. This service shall only be used in conjunction with the MS2 AR.

Table 226 shows the parameters of the service.

Table 226 – Abort

Parameter name	Req	Ind
Argument	M	M
AREP	M	M(=)
Locally Generated		C
Subnet	C	C(=)
Instance	C	C(=)
Reason Code	C	C(=)
Additional Detail		C

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is used as local identifier of the desired AR.

Locally Generated

This parameter only exists in the indication in case of an MS2 AR. It indicates that the Abort has been produced by the remote partner (Locally Generated = FALSE) or in the local station (Locally Generated = TRUE).

Subnet

This parameter is used in case of an MS2 AR to encode information about the location of the source of the Abort.

Instance

This parameter is used in case of an MS2 AR to encode information about the protocol instance that detected a problem causing the Abort.

The allowed values are shown in Table 227.

Table 227 – Instance

Value	Meaning
DL	Data Link Layer
MS2	MS2 AR
User	User

Reason Code

The Reason Code indicates the reason for the Abort in case of an MS2 AR.

Allowed Values: UE, RR, RS, NR, DH, LR, RDL, RDH, DS, NA, ABT_SE, ABT_FE, ABT_TO, ABT_RE, ABT_IV, ABT_STO, ABT_IA, ABT_OC.

Additional Detail

This parameter contains detail information in case of an abort which is caused by an inappropriate Initiate (e.g. Send Timeout to small) in case of an MS2 AR. This parameter is only present if the parameter Reason Code has the value ABT_STO and the parameter Instance equals MS2. In this case the parameter additional detail contains the value of the received Send Timeout of the DP-slave.

8.2.5.3.12 MS0 init DP-slave

This service is used to initialise the MS0 AR of a DP-slave.

Table 228 shows the parameters of the service.

Table 228 – MS0 init DP-slave

Parameter name	Req	Cnf
Argument AREP	M M	
Result AREP		M M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result

This parameter indicates that the service request succeeded.

8.2.5.3.13 MS1 init DP-slave

This service is used to initialise the MS1 AR of a DP-slave.

Table 229 shows the parameters of the service.

Table 229 – MS1 init DP-slave

Parameter name	Req	Cnf
Argument AREP	M M	
Result AREP		M M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result

This parameter indicates that the service request succeeded.

8.2.5.3.14 MS2 init DP-slave

This service is used to initialise all MS2 AR of a DP-slave.

Table 230 shows the parameters of the service.

Table 230 – MS2 init DP-slave

Parameter name	Req	Cnf
Argument AREP	M M	
Result AREP		M M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result

This parameter indicates that the service request succeeded.

8.2.5.3.15 DP-slave started

With this service the supported alarm features on this MS1 AR are indicated. The indication occurs after the DP-slave has entered the data exchange mode and the transfer of alarm and status messages is possible from now on.

Table 231 shows the parameters of the service.

Table 231 – DP-slave started

Parameter name	Ind
Argument	M
AREP	M
Actual Enabled Alarms	M
Alarm Sequence	M
Alarm Limit	C

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Actual Enabled Alarms

This parameter indicates which alarm types are supported by the DP-master (Class 1).

Allowed values: Update Alarm enabled / disabled, Manufacturer Specific Alarm enabled / disabled, Diagnosis Alarm enabled / disabled, Process Alarm enabled / disabled, Pull Plug Alarm enabled / disabled

Alarm Sequence

This parameter indicates whether only one alarm of a specific Alarm Type can be active at one time (Alarm Sequence=FALSE) or several alarms (2 to 32) of the same or different type can be active at one time (Alarm Sequence=TRUE).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Alarm Limit

This parameter indicates the maximum number of parallel alarms.

The allowed values are shown in Table 232.

Table 232 – Alarm limit

Value	Meaning	
0	1 alarm of each type	1 alarm of each type
1	2 alarms in total	2 alarms in total
2	4 alarms in total	4 alarms in total
3	8 alarms in total	8 alarms in total
4	12 alarms in total	12 alarms in total
5	16 alarms in total	16 alarms in total
6	24 alarms in total	24 alarms in total
7	32 alarms in total	32 alarms in total

8.2.5.3.16 DP-slave stopped

In case of a an MS0 AR, this Application Layer service indicates to the User that the DP-slave has left the data exchange mode and that the MS1 AR is closed. As a consequence the transfer of alarms as well as the processing of Write and Read services is no longer possible.

In case of a an MS1 AR, this service indicates to the User that the acyclic communication relationship to the DP-master (Class 1) has been stopped. This means that there is no longer cyclic data exchange, the MS0 AR and MS1 AR of a DP-slave will be set to the initial state. Table 233 shows the parameters of the service.

Table 233 – DP-slave stopped

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.5.3.17 Reset DP-slave

By this service the User of a DP-slave resets the Application Layer.

Table 234 shows the parameters of the service.

Table 234 – Reset DP-slave

Parameter name	Req	Cnf
Argument		

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

8.2.5.3.18 DP-slave fault

By this service the Application Layer indicates to the User of a DP-slave that due to fatal errors all ARs have been reset.

Table 235 shows the parameters of the service.

Table 235 – DP-slave fault

Parameter name	Ind
Argument	

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

8.2.5.3.19 Application ready DP-slave

By this service the User of a DP-slave indicates to the Application Layer that the application is ready. This service shall only be used in conjunction with the MS0 AR.

Table 236 shows the parameters of the service.

Table 236 – Application ready DP-slave

Parameter name	Req
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.5.3.20 Start subscriber

This service is used by the application of the DP-slave to start the subscription of a certain DXB-Link.

Table 237 shows the parameters of the service.

Table 237 – Start subscriber

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	M	
Result(+)		S
AREP		M
CREP		M
Result(-)		S
AREP		M
CREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

This parameter is the local identifier for the desired CR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.5.3.21 Stop subscriber

This service is used by the application of the DP-slave to stop the subscription of a certain DXB-Link.

Table 238 shows the parameters of the service.

Table 238 – Stop subscriber

Parameter name	Req	Cnf
Argument	M	
AREP	M	
CREP	M	
Result(+)		S
AREP		M
CREP		M
Result(-)		S
AREP		M
CREP		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

This parameter is the local identifier for the desired CR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.5.3.22 Publisher active

This service indicates to the application of the Subscriber (DP-slave) the state of the Publisher of a certain DXB-Link. This service will be issued by the DP Application Layer only if the state of the Publisher has been changed since the last watchdog period.

Table 239 shows the parameters of the service.

Table 239 – Publisher active

Parameter name	Ind
Argument	M
AREP	M
CREP	M
Status	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

CREP

This parameter is the local identifier for the desired CR (Publisher).

Status

This parameter indicates the state of the related Publisher.

The allowed values are shown in Table 240.

Table 240 – Status

Value	Meaning
TRUE	the related Publisher is active: during the last watchdog period the subscribed Input Data object of the related Publisher has been received at least one time
FALSE	the related Publisher is not active; during the last watchdog the subscribed Input Data object of the related Publisher has not been received due to the following problems: the value of the attribute Input Data Length Publisher of the related Publisher of the related CRL entry was not in alignment with the actual length of the Input Data object published by the related Publisher the related Publisher was not in the data exchange mode breakdown of the related Publisher

8.2.5.3.23 Init DP-master C11

By this service the User initialises the Application Layer of a DP-master (Class 1). The parameter are described in this specification, part 3 and 4.

Table 241 shows the parameters of the service.

Table 241 – Init DP-master C11

Parameter name	Req	Cnf
Argument	M	
Bus Para	M	
TS	M	
Data rate	M	
TSL	M	
minT _{SDR}	M	
maxT _{SDR}	M	
TQUI	M	
TSET	M	
TTR	M	
G	M	
HSA	M	
max retry limit	M	
Result		M

Argument

The argument shall convey the service specific parameters of the service request.

Bus Para

This parameter is composed of the following subparameters:

TS

This parameter contains the DL-address of this DP-master (Class 1).

Data rate

This parameter contains the data rate of this DP-master (Class 1).

TSL

This parameter contains the slot time for this DP-master (Class 1).

minT_{SDR}

This parameter contains the smallest station delay time for this DP-master (Class 1).

maxT_{SDR}

This parameter contains the largest station delay time for this DP-master (Class 1).

TQUI

This parameter contains the transmitter fall time for this DP-master (Class 1).

TSET

This parameter contains the setup time for this DP-master (Class 1).

TTR

This parameter contains the target rotation time for this DP-master (Class 1).

G

This parameter contains the GAP update factor for this DP-master (Class 1).

HSA

This parameter contains the highest station (DL-) address for this DP-master (Class 1).

max retry limit

This parameter contains the maximum number of retries for this DP-master (Class 1).

Result

This parameter indicates that the service request succeeded.

8.2.5.3.24 DP-master C11 started

In case of a MS0 AR, this Application Layer service indicates to the User of a DP-master (Class 1) that the assigned DP-slave is in the data exchange mode.

In case of a MS1 AR, this Application Layer service indicates that the handling of alarm requests for this DP-slave are possible.

Table 242 shows the parameters of the service.

Table 242 – DP-master CI1 started

Parameter name	Ind
Argument	M
AREP	M
Alarm Limit	C

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR (Address of DP-slave).

Alarm Limit

In case of a an MS1 AR, this parameter indicates the maximum number of parallel alarms.

The allowed values are shown in Table 243.

Table 243 – Alarm limit

Value	Meaning	
0	1 alarm of each type	1 alarm of each type
1	2 alarms in total	2 alarms in total
2	4 alarms in total	4 alarms in total
3	8 alarms in total	8 alarms in total
4	12 alarms in total	12 alarms in total
5	16 alarms in total	16 alarms in total
6	24 alarms in total	24 alarms in total
7	32 alarms in total	32 alarms in total

8.2.5.3.25 DP-master CI1 stopped

This Application Layer service indicates to the User of a DP-master (Class 1) that the related AR has been stopped. In case of a MS0 AR, the MS1 AR has been stopped, too. A new start is necessary to restart MS0 AR and MS1 AR.

Table 244 shows the parameters of the service.

Table 244 – DP-master CI1 stopped

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.5.3.26 Reset DP-master CI1

By this service the User of a DP-master (Class 1) resets the Application Layer.

Table 245 shows the parameters of the service.

Table 245 – Reset DP-master C11

Parameter name	Req	Cnf
Argument	M	
Result		M

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

Result

This parameter indicates that the service request succeeded.

8.2.5.3.27 DP-master C11 fault

By this service the Application Layer indicates to the User of a DP-master (Class 1) that a fatal error has occurred and all ARs have been stopped. Further communication is only possible after a Reset.req of the User.

Table 246 shows the parameters of the service.

Table 246 – DP-master C11 fault

Parameter name	Ind
Argument	

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

8.2.5.3.28 DP-master C11 reject

By this service the Application Layer rejects a previous Read/ Write.req from the User of a DP-master (Class 1) to the MS1 AR, if the MS1 AR was forced to stop.

Table 247 shows the parameters of the service.

Table 247 – DP-master C11 reject

Parameter name	Ind
Argument	M
AREP	M
Reason	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR (Address of DP-slave).

Reason

The parameter Status indicates success or failure of the service.

Allowed values: REJ ABORT, REJ LE, REJ PS, REJ SE, REJ IV

8.2.5.3.29 Set mode DP-master C11

By this service the User of a DP-master (Class 1) can set the operation mode of the MS0 AR. Additionally the User gets information about the actual state of the bus system in case of errors.

Table 248 shows the parameters of the service.

Table 248 – Set mode DP-master C11

Parameter name	Req	Cnf
Argument	M	
AREP	M	
USIF State	M	
Result(+)		M
AREP		M
Bus Accessible		M
Result(-)		
AREP		M
Bus Accessible		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

USIF State

The Application Layer provides four operation modes for the User:

OFFLINE

Communication to all DP participants (DP-master and DP-slave) is stopped. MAC of local DL is in offline state. The Application Layer waits for a signal to start.

STOP

Changing the operation mode from OFFLINE to STOP the bus parameter set is loaded in the DL by the Application Layer. The DL was activated. The responder services for MM1 AR and MM2 AR can be processed. MS0 and MS1 AR will not be activated (No interactions to DP-slaves).

CLEAR

In this operation mode the DP-master (Class 1) attempts to configure the DP-slaves that are assigned to it, and starts the data exchange mode. The input data of the DP-slaves are read and passed to the User. The User's output data are ignored. The assigned DP-slaves receive a value of 0 or no data for their Output Data objects. (depending on the attribute Fail Safe)

OPERATE

The DP-master (Class 1) is in the data exchange mode with the assigned DP-slaves. The inputs from the DP-slaves are transferred to the User and the outputs from the User are passed to the DP-slaves. If the Application Layer leaves the mode OPERATE, all DP-slaves' outputs change in the safe state by means of the service Global Control (Control Command = Clear Data, Group Select = 0).

Result(+)

This selection type parameter indicates that the service request succeeded.

Bus Accessible

This parameter indicates whether the DP-master is able to access the bus (TRUE) or not (FALSE).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Result(-)

This selection type parameter indicates that the service request failed.

8.2.5.3.30 DP-master CI1 mode changed

By this service the Application Layer indicates to the user of a DP-master (Class 1):

- the change of its operation mode from OPERATE to CLEAR if an error occurred and the Error Action Flag is set
- the change of its operation mode from STOP to OFFLINE if an error occurred while setting the bus parameters.

Table 249 shows the parameters of the service.

Table 249 – DP-master CI1 mode changed

Parameter name	Ind
Argument	M
AREP	M
USIF State	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

USIF State

Allowed values: OFFLINE, CLEAR (see 8.2.5.3.29)

8.2.5.3.31 Load bus Par DP-master CI1

By this service the User of a DP-master (Class 1) can load a new bus parameter set. This service is not allowed in the operation mode OFFLINE. During STOP changes to all bus parameters are possible. In the operation modes CLEAR and OPERATE of the Application Layer it is not allowed to load a bus parameter set with changes to critical parameters DL variables Data_rate and TS.

The changed bus parameters are not stored. Thus, all parameters of this service are lost after changing the operation mode of the Application Layer from STOP to OFFLINE. Changes to be stored can only be done within the bus parameter set (Bus Para) of the Init service.

Table 250 shows the parameters of the service.

Table 250 – Load bus Par DP-master CI1

Parameter name	Req	Cnf
Argument	M	
Bus Para	M	
Result(+)		S
Status		M
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Bus Para

This parameter contains the bus parameter set which shall be loaded. The structure is defined in Master CI1 Init service.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

The parameter Status indicates the reason for success or failure of the service.

Allowed values: NO, IV

8.2.5.3.32 Mark DP-master CI1

By this service the User of a DP-master (Class 1) can operate synchronously with the MS0 AR. After the User has sent this service to the Application Layer, the User will receive an acknowledgement from the Application Layer after all activated DP-slaves were polled at least once.

Table 251 shows the parameters of the service.

Table 251 – Mark DP-master CI1

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Dia		M
Result(-)		S
AREP		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Dia

This parameter indicates (if the value is equal TRUE) whether the DP-master has not executed a data exchange sequence to at least one activated DP-slave.

Allowed values: TRUE, FALSE

Result(-)

This selection type parameter indicates that the service request failed.

Status

The parameter Status indicates success or failure of the service. The value NO will be signalled if the Application Layer is in the operation mode OFFLINE or STOP, or a previous Mark.req is still active.

Allowed values: NO

8.2.5.3.33 Abort DP-master CI1

This service allows to release a MS0, MS1 or MM1 AR of a DP-master (Class 1). The indication may be issued by the MS1 AR if the DP-slave responds incorrectly. In this case the MS0 CRs to that DP-slaves will also be released.

Table 252 shows the parameters of the service.

Table 252 – Abort DP-master C11

Parameter name	Req	Cnf
Argument	M	M
AREP	M	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.5.3.34 Read value DP-master C11

The User of a DP-master (Class 1) read any of the DL variables with this service.

Table 253 shows the parameters of the service.

Table 253 – Read value DP-master C11

Parameter name	Req	Cnf
Argument	M	
Variable	M	
Result		M
Value		M
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Variable

The parameter Variable selects the DL Variable whose current value has to be read. See the Type 3 clauses of IEC 61158-3 for a list of all possible DL variables.

Result

This parameter indicates that the service request succeeded.

Value

This parameter contains the actual value of the read DL Variable.

Status

The parameter Status indicates success or failure of the service.

Allowed values: OK, NO, IV

8.2.5.3.35 Delete SC DP-master C11

By this service the DL statistic counters can be cleared (set to zero) by the User of a DP-master (Class 1).

Table 254 shows the parameters of the service.

Table 254 – Delete SC DP-master C11

Parameter name	Req	Cnf
Argument	M	
Address	M	
Result		M

Argument

The argument shall convey the service specific parameters of the service request.

Address

This parameter specifies the DL-address of the statistic counters which have to be cleared.

Attribute Type Unsigend8

Allowed values: 0 to 125

Result

This parameter indicates that the service request succeeded.

8.2.5.3.36 DP-master C11 event

By this service the events of the service DLM-Event are indicated to the User of a DP-master (Class 1) as an Event.

Table 255 shows the parameters of the service.

Table 255 – DP-master C11 event

Parameter name	Ind
Argument	M
Event	M
Add Info	M

Argument

The argument shall convey the service specific parameters of the service request.

Event

This parameter can take the values which are allowed for the DLM-Event service.

Add Info

The parameter indicates the added info of the DL.

8.2.5.3.37 Init DP-master C12

By this service the User initialises the DL-parameter of a DP-master (Class 2).

Table 256 shows the parameters of the service.

Table 256 – Init DP-master C12

Parameter name	Req	Cnf
Argument	M	
Bus Para	M	
TS	M	
Data rate	M	
TSL	M	
minTSDR	M	
maxTSDR	M	
TQUI	M	
TSET	M	
TTR	M	
G	M	
HSA	M	
max retry limit	M	
Result		M

Argument

The argument shall convey the service specific parameters of the service request.

Bus Para

This parameter is composed of the following subparameters:

TS

This parameter contains the DL-address of this DP-master (Class 2).

Data rate

This parameter contains the data rate of this DP-master (Class 2).

TSL

This parameter contains the slot time for this DP-master (Class 2).

minTS DR

This parameter contains the smallest station delay time for this DP-master (Class 2).

maxTS DR

This parameter contains the largest station delay time for this DP-master (Class 2).

TQUI

This parameter contains the transmitter fall time for this DP-master (Class 2).

TSET

This parameter contains the setup time for this DP-master (Class 2).

TTR

This parameter contains the target rotation time for this DP-master (Class 2).

G

This parameter contains the GAP update factor for this DP-master (Class 2).

HSA

This parameter contains the highest station (DL-) address for this DP-master (Class 2).

max retry limit

This parameter contains the maximum number of retries for this DP-master (Class 2).

Result

This parameter indicates that the service request succeeded.

8.2.5.3.38 Reset DP-master CI2

By this service the User of a DP-master (Class 2) resets the Application Layer.

Table 257 shows the parameters of the service.

Table 257 – Reset DP-master CI2

Parameter name	Req	Cnf
Argument		

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

8.2.5.3.39 DP-master CI2 fault

By this service the Application Layer indicates a fatal DL error to the User of a DP-master (Class 2).

Table 258 shows the parameters of the service.

Table 258 – DP-master CI2 fault

Parameter name	Ind
Argument	

Argument

The argument shall convey the service specific parameters of the service request. Nevertheless, in this service there are no specific parameters.

8.2.5.3.40 DP-master CI2 reject

By this service the Application Layer rejects a previous Read, Write, Data Transport service from the User of a DP-master (Class 2).

Table 259 shows the parameters of the service.

Table 259 – DP-master CI2 reject

Parameter name	Ind
Argument	M
AREP	M
Reason	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR (Address of DP-slave).

Reason

The parameter Status indicates success or failure of the service.

Allowed values: REJ ABORT, REJ LE, REJ PS, REJ SE, REJ IV

8.2.5.3.41 DP-master CI2 closed

By this service the Application Layer informs the User of a DP-master (Class 2) that the MS2 AR has been closed.

Table 260 shows the parameters of the service.

Table 260 – DP-master CI2 closed

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.5.3.42 DP-master CI2 event

By this service the events of the DLM are indicated to the User of a DP-master (Class 2) as an Event.

Table 261 shows the parameters of the service.

Table 261 – DP-master CI2 event

Parameter name	Ind
Argument	M
Event	M
Add Info	M

Argument

The argument shall convey the service specific parameters of the service request.

Event

This parameter can take the values which are allowed for the DLM-Event service.

Add Info

The parameter indicates the added info of the DL.

8.2.6 Management ASE**8.2.6.1 Overview**

The Management ASE is used to convey parameters between a DP-master (Class 2) and a DP-master (Class 1). Additionally, the DP-master (Class 2) can read diagnosis information of the DP-master (Class 1) and its assigned DP-slaves. The access of the DP-master (Class 2) to the DP-master (Class 1) is performed according to the Client/Server access model.

8.2.6.2 Management class specification**8.2.6.2.1 Master diag class specification****8.2.6.2.1.1 Template**

A Master Diag object is described by the following template:

DP ASE:	Management ASE
CLASS: Master Diag	
CLASS ID:	not used
PARENT CLASS:	TOP
ATTRIBUTES:	
1. (m) Key Attribute:	Implicit
2. (m) Attribute:	Master Status
2.1. (m) Attribute:	USIF State
2.2. (m) Attribute:	Ident Number
2.3. (m) Attribute:	Hardware Release DP
2.4. (m) Attribute:	Firmware Release DP
2.5. (m) Attribute:	Hardware Release User
2.6. (m) Attribute:	Firmware Release User
3. (m) Attribute:	System Diagnosis
4. (m) Attribute:	Data Transfer List
5. (o) Attribute:	List of DP-slave Diag
5.1 (m) Attribute:	DP-slave Address
5.2 (m) Attribute:	DP-slave Diag
SERVICES:	
1. (o) OpsService:	Get Master Diag

8.2.6.2.1.2 Attributes**Implicit**

The attribute Implicit indicates that the Master Diag object is implicitly addressed by the service.

Master Status

This parameter is composed of the following items.

USIF State

This attribute describes the state of a DP-master (Class 1)

The allowed values are shown in Table 262.

Table 262 – USIF state

Value	Meaning
0	OFFLINE
1	STOP In this state the DP-master (Class 1) does not execute any services at the MS0 and MS1 AR. The MM1 and MM2 AR can be established.
2	CLEAR The DP-master is operating the MS0 AR. The Output Data are in the safe state. The services related with MS1 AR can be executed. The MM1 and MM2 AR can be established.
3	OPERATE The DP-master is operating the MS0 AR. The Output Data are valid. The services related with MS1 AR can be executed. The MM1 and MM2 AR can be established.

Ident Number

This attribute reflects the Ident Number of the DP-master (Class 1).

Attribute Type: Unsigned16

Hardware Release DP

This attribute specifies the hardware release of the DP implementation of the DP-master (Class 1).

Attribute Type: Unsigned8

Firmware Release DP

This attribute specifies the firmware/software release of the DP implementation of the DP-master (Class 1).

Attribute Type: Unsigned8

Hardware Release User

This attribute specifies the hardware release of the User implementation of the DP-master (Class 1).

Attribute Type: Unsigned8

Firmware Release User

This attribute specifies the firmware/software release of the User implementation of the DP-master (Class 1).

Attribute Type: Unsigned8

System Diagnosis

The System Diagnosis reflects the state of all DP-slaves assigned to the DP-master (Class 1). This state itself is a one-bit-information for each DP-slave which is set to the value 1 if its Ext Diag Flag of the Diagnosis information is set or if it does not respond or if it responds incorrectly so that the MS0 AR has to be re-established. The state of a DP-slave is reset to the value 0 after the MS0 AR is successfully re-established or if the DP-slave is deactivated by the AP of the DP-master (Class 1) or the USIF State of the DP-master (Class 1) is STOP.

Attribute Type: List of 0,1

Data Transfer List

The Data Transfer List reflects the status of the data exchange mode of all DP-slaves assigned to the DP-master (Class 1).

This status itself is a one-bit-information for each DP-slave which is set to the value 1 if the data exchange with the assigned DP-slave was successfully or if between two data exchange requests is only one diagnosis request. All other cases reset the status bit-information of the DP-slave to the value 0.

Attribute Type: List of 0,1

List of DP-slave Diag

This attribute is composed of the following list elements.

DP-slave Address

This attribute specifies the DL-address of the DP-slave.

Attribute Type: Unsigned8

Allowed values: 0 to 125

DP-slave Diag

This attribute reflects the diagnosis information of the DP-slave inclusive the information added by the DP Application Layers for each DP-slave.

8.2.6.2.1.3 Invocation of master diag object

For the invocation of the Master Diag object the following rules apply:

- Only one Master Diag object shall be invoked in a DP-master (Class 1)

8.2.6.2.2 Master parameter class specification**8.2.6.2.2.1 Overview specification**

The Master Parameter object consists of one Bus Para element and one or more Slave Para elements. The values for the Master Parameter object may be loaded via the MM1 AR.

The Download of the attribute values for Master Parameter object may be done in one sequence or the attribute values for of the Bus Para element and for the Slave Para elements may be loaded separately.

For the download of the attribute values of Master Parameter object the Application Layer has to be in the operation mode STOP. The operation mode of the Application Layer of the DP-master (Class 1) shall be changed with the Act Param service via the MM1 AR in conjunction with the local service Set Mode DP-master CI1.

If the attribute values for Master Parameter object are loaded in one sequence the values have to be stored first in an intermediate buffer. After reception of the Act Param or Act Para Brct service the User has to transfer the values from the intermediate buffer to the attributes of the Master Parameter object. Afterwards the User shall transfer these values accordingly with the Load ARL, Load CRL (parameter Update set to FALSE) and Load Bus Par DP-master CI1 services. The Act Param service shall be confirmed according to the following rules:

- positive, if all load services are confirmed positively
- negative, if one or more load services are confirmed negatively.

If the attribute values for the Bus Para elements are loaded the values have to be stored first in an intermediate buffer. After reception of the Act Param or Act Para Brct service the User has to transfer the values from the intermediate buffer with the Load Bus Par DP-master CI1 service to the DP Application Layer. The Act Param service shall be confirmed positively, if the previously requested load service are confirmed positively and after the values has been transferred to the attributes of the Master Parameter object.

The Act Param service shall be confirmed negatively, if the previously requested load service are confirmed negatively. In this case the previously transmitted values for the Bus Para elements shall be omitted.

If the attribute values for one Slave Para element are loaded the values have to be stored first in an intermediate buffer. After reception of the Act Param service the User has to transfer the values from the intermediate buffer to the attributes of the Master Parameter object.

Afterwards the User shall transfer these values accordingly with the ARL Slave Update DP-master C11 and the Load CRL (parameter Update set to TRUE) services. The Act Param service shall be confirmed according to the following rules:

- positive, if all load services are confirmed positively
- negative, if one or more load services are confirmed negatively.

Care shall be taken by the User because updating of one Slave Para element may interfere with already loaded Bus Para and may change the system behaviour.

8.2.6.2.2.2 Template

A Master Parameter object is described by the following template:

DP ASE:			Management ASE
CLASS:			Master Parameter
CLASS ID:			not used
PARENT CLASS:			TOP
ATTRIBUTES:			
1.	(m)	Key Attribute:	Implicit
2.	(m)	Attribute:	Bus Para
2.1.	(m)	Attribute:	Bus Para Len
2.2.	(m)	Attribute:	TS
2.3.	(m)	Attribute:	Data rate
2.4.	(m)	Attribute:	T _{SL}
2.5.	(m)	Attribute:	min T _{SDR}
2.6.	(m)	Attribute:	max T _{SDR}
2.7.	(m)	Attribute:	T _{QUI}
2.8.	(m)	Attribute:	T _{SET}
2.9.	(m)	Attribute:	T _{TR}
2.10.	(m)	Attribute:	G
2.11.	(m)	Attribute:	HSA
2.12.	(m)	Attribute:	max retry limit
2.13.	(m)	Attribute:	Error Action Flag
2.14.	(o)	Attribute:	Isochronous Mode
2.15.	(o)	Attribute:	IsoM Freeze
2.16.	(o)	Attribute:	IsoM Sync
2.17.	(m)	Attribute:	Min Slave Interval
2.18.	(m)	Attribute:	Poll Timeout
2.19.	(m)	Attribute:	Data Control Time
2.20.	(o)	Attribute:	Alarm Max
2.21.	(m)	Attribute:	Max User Global Control
2.22.	(m)	Attribute:	Master User Data Len
2.23.	(m)	Attribute:	Master Class2 Name
2.24.	(m)	Attribute:	Master User Data
2.25.	(o)	Attribute:	T _{CT}
2.26.	(o)	Attribute:	maxT _{SH}
3.	(o)	Attribute:	List of Slave Para
3.1.	(m)	Attribute:	Slave DL-address
3.2.	(m)	Attribute:	Slave Para Len
3.3.	(o)	Attribute:	Extra Alarm SAP
3.4.	(o)	Attribute:	DPV1 Data Types
3.5.	(o)	Attribute:	DPV1 Supported
3.6.	(m)	Attribute:	Fail Safe
3.7.	(m)	Attribute:	Active
3.8.	(m)	Attribute:	Slave Type
3.9.	(m)	Attribute:	New Prm
3.10.	(m)	Attribute:	Max Diag Data Len
3.11.	(o)	Attribute:	Max Channel Data Length
3.12.	(m)	Attribute:	Diag Upd Delay
3.13.	(o)	Attribute:	Alarm Mode
3.14.	(m)	Attribute:	NA to Abort
3.15.	(m)	Attribute:	Ignore ACIr
3.16.	(o)	Attribute:	Publisher Flag

- 3.17. (o) Attribute: MS1 Timeout
- 3.18. (o) Attribute: Prm Command supported
- 3.19. (m) Attribute: Prm Data Len
- 3.20. (m) Attribute: Prm Data
- 3.21. (m) Attribute: Cfg Data Len
- 3.22. (m) Attribute: Cfg Data
- 3.23. (m) Attribute: Add Tab Len
- 3.24. (m) Attribute: Add Tab
- 3.25. (m) Attribute: Slave User Data Len
- 3.26. (o) Attribute: Slave User Data
- 3.27. (o) Attribute: Ext User Prm Data Len
- 3.28. (o) Attribute: Ext User Prm Data
- 3.29. (m) Attribute: List of assigned CREPs for the MS0 AR
- 3.29.1. (m) Attribute: CREP
- 3.30. (m) Attribute: Assigned AREP for the MS1 AR
- 3.31. (m) Attribute: Assigned AREP for the MS3 AR
- 4. (m) Attribute: Assigned AREP for the MS0 AR

SERVICES:

- 1. (o) OpService: Start Seq
- 2. (o) OpService: Download
- 3. (o) OpService: Upload
- 4. (o) OpService: End Seq
- 5. (o) OpService: Act Para Brct
- 6. (o) OpService: Act Param

8.2.6.2.2.3 Attributes**Implicit**

This attribute indicates that the Master Parameter object is implicitly addressed by the services.

Bus Para

This combined attribute consists of the following items.

Bus Para Len

This attribute contains the length of Bus Para inclusive the length parameter.

Attribute Type: Unsigned16

Allowed values: 66 to $2^{16}-1$

TS

This attribute contains the own DL-address of the DP-master.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Data rate

This attribute contains the code number of the chosen data rate.

Attribute Type: Unsigned8

The allowed values are shown in Table 263.

Table 263 – Data rate

Value	Meaning
0	9,6kbit/s
1	19,2kbit/s
2	93,75kbit/s
3	187,5kbit/s
4	500 kbit/s
6	1500 kbit/s
7	3000 kbit/s
8	6000 kbit/s
9	12000 kbit/s
10	31,25kbit/s
11	45,45kbit/s

T_{SL}, min T_{SDR}, max T_{SDR}, T_{QUI}, T_{SET}, T_{TR}, G, HSA, max retry limit

These attributes are described in the Type 3 clauses of IEC 61158-3 and IEC 61158-4.

Error Action Flag

This attribute reflects the reaction of the DP-master (Class 1) in case of failed data transfers. This attribute will be evaluated in the operation mode OPERATE and CLEAR of the DP-master (Class 1). If the data transfer to at least one activated DP-slave was impossible for a certain period of time (Data Control Time) and if the Error Action Flag is set, the operation mode will change from OPERATE to CLEAR. If the Error Action Flag is cleared, the DP-master (Class 1) will remain in the operation mode OPERATE despite erroneous data transfers.

If the operation mode of the DP-master (Class 1) shall be changed from CLEAR to OPERATE with the Error Action Flag set to TRUE the data transfer to all activated DP-slaves have to be performed successfully for a certain period of time (Data Control Time)

The allowed values are shown in Table 264.

Table 264 – USIF state

Value	Meaning
0	no change of the operation mode in case of an error
1	change of the operation mode in case of an error

Isochronous Mode

This attribute indicates if the DP-master (Class 1) shall operate in the Isochronous Mode. Two different models concerning the interface between the DP Application Layer and the application of the DP-master (Class 1) itself are provided by fieldbus DP for the DP-master (Class 1), see 8.2.2.4.3.2.

The allowed values are shown in Table 265.

Table 265 – Isochronous mode

Value	Meaning
Not Synchronised	the DP-master (Class 1) shall not operate in the Isochronous Mode
Buffered Synchronised	the DP-master (Class 1) shall operate in the Buffer Synchronised Isochronous Mode
Enhanced Synchronised	the DP-master (Class 1) shall operate in the Enhanced Synchronised Isochronous Mode

IsoM Freeze

This attribute indicates if the synchronisation message (SYNCH) shall be sent with the Control Command Freeze set to TRUE.

Allowed values: TRUE, FALSE

IsoM Sync

This attribute indicates if the synchronisation message (SYNCH) shall be sent with the Control Command Sync set to TRUE.

Allowed values: TRUE, FALSE

Min Slave Interval

This attribute specifies the smallest allowed period of time between two consecutive DP-slave poll cycles. This ensures that the sequence of cyclic service requests from the DP-master (Class 1) can be handled by the DP-slave. This period of time will be complied by the DP-master (Class 1) for every cyclic Master-slave service except for the Global Control service. For the Global Control service the User is responsible for compliance with Min Slave Interval.

Attribute Type: Unsigned16

Allowed values: 1 to 2¹⁶-1

Time Base: 100µs

Poll Timeout

In the case of MM1 AR this attribute specifies the maximum period of time needed to get the response.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 1ms

Data Control Time

This attribute specifies the maximum period of time for an Data Exchange with every activated DP-slave. This Time is used for sending out the Global Control cyclically.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 10ms

Alarm Max

This variable contains the maximum number of alarms per DP-slave, which can be handled by the DP-master. It describes a Master capability. The Master shall be able to handle a minimum number of 7 alarms, because there are existing 777 different alarm classes.

Attribute Type: Unsigned8

Allowed values: 7 to 32

Max User Global Control

This attribute defines the maximum number of Global Control requests, which may be started by the User at the same time. This parameter describes the ability of the DP-master (Class 1). A practicable value for Max User Global Control is 16. For each of the 8 Slave groups one Sync and one Freeze Command may be started at the same time.

Attribute Type: Unsigned8

Allowed values: 1 to 255

Master User Data Len

This attribute contains the length of Master User Data inclusive the length parameter.

Attribute Type: Unsigned16

Allowed values: 34 to $2^{16}-1$

Master Class2 Name

This attribute indicates the name of the DP-master (Class 2) which provided the values for the attributes of this Master Parameter object.

Attribute Type: Visible String(32)

Master User Data

This attribute contains manufacturer specific data which is necessary for the bus parameter set.

Attribute Type: Octet String

TCT

This attribute specifies the time (in units of t_{BIT}) that is necessary to handle a complete Isochronous DP cycle and corresponds to the time T_{DP} .

Allowed values: 1 to $2^{24}-1$

maxTSH

This attribute specifies the maximum allowed time shift (in units of t_{BIT}) of the time T_{DP} measured by the DP-master (Class 1). The application of the DP-master (Class 1) will be informed if the measured Isochronous DP cycle time exceeds this limit by the service SYNCH Delayed indicating the occurred shift.

Allowed values: 1 to 2^8-1

Slave Para

This combined attribute consists of the following items.

Slave DL-address

This attribute reflects the DL-address of the DP-slave. This address is used as an identifier (Area Code) to address the corresponding slave parameter set, e.g. for up- or download purposes.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Slave Para Len

This attribute contains the length of Slave Para inclusive the length parameter. A slave parameter set can be deleted by setting the Slave Para Len to zero.

Attribute Type: Unsigned16

Allowed values: 0 to $2^{16}-1$

Extra Alarm DLSAP

This attribute selects the Channel for the Alarm Ack service.

- If the attribute is set to FALSE the DP-master (Class 1) acknowledges alarms via DLSAP51.
- If the attribute is set to TRUE the DP-master (Class 1) acknowledges alarms via DLSAP50.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

DPV1 Data Types

This attribute reflects whether the DP-slave uses data types.

- If the attribute is set to FALSE the DP-slave uses only basic types (Byte, Word, Array) for the Data Description.
- If the attribute is set to TRUE the DP-slave uses data types for the Data Description.

Attribute Type: Boolean

DPV1 Supported

This attribute reflects whether the DP-slave uses the extended functionality.

- If the attribute is set to FALSE the DP-slave uses no extended functionality.
- If the attribute is set to TRUE the DP-slave uses extended functionality.

Attribute Type: Boolean

Fail Safe

This attribute reflects whether the DP-slave supports the Fail Safe functionality. If the attribute is set to FALSE the DP-slave does not support the Fail Safe functionality. If the attribute is set to TRUE the DP-slave supports the Fail Safe functionality.

Attribute Type: Boolean

Active

This attribute reflects whether the DP-slave has to be activated. If the attribute Active has the value FALSE, the DP-master (Class 1) stops the DP-slave handling to this assigned DP-slave. In this state it is allowed to change the slave parameter set, for example by a DP-master (Class 2) with the Download service. If the Active-Flag has the value TRUE, the DP-slave handling will be done by the DP-master (Class 1) to this assigned DP-slave. Changing of the slave parameter set in this state is limited (see also the description of the attribute New Prm).

Attribute Type: Boolean

Slave Type

This attribute contains a manufacturer specific type denotation for the DP-slave see Table 266.

Attribute Type: Unsigned8

The allowed values are shown in Table 266.

Table 266 – Slave type

Value	Meaning
0	DP-slave
1-15	reserved
16-255	manufacturer specific

New Prm

This attribute reflects whether parameterisation data has to be transferred to the DP-slave in the data exchange mode. If this attribute is set to FALSE the Output Data will be transferred to this DP-slave in the data exchange mode. If this attribute is set to TRUE, for one data cycle parameterisation data will be transferred instead of Output Data. The consistency of the parameterisation data has to be guaranteed by local measures. After transmitting this parameterisation data successfully the DP-master (Class 1) sets this attribute to FALSE and the data exchange mode to the DP-slave will be continued.

Attribute Type: Boolean

Max Diag Data Len

This attribute contains the maximum length of Diag Data for the DP-slave.

Attribute Type: Unsigned8

Allowed values: 6 to 244

Max Channel Data Length

This attribute defines the maximum length of a MS1-APDU (Length parameter and 4 Byte Header) for the corresponding DP-slave.

Attribute Type: Unsigned8

Allowed values: 4 to 244

Diag Upd Delay

The attribute reflects the number of Diag service requests performed in the state DIAG2 of the DP-master (Class 1) while Prm Req is set in Diag Data of the DP-slave (for DP-slaves with reduced performance).

Attribute Type: Unsigned8

Allowed values: 0 to 15 (extendible up to 255)

Alarm Mode

This attribute specifies the maximum number of possible active alarms.

Attribute Type: Unsigned8

The allowed values are shown in Table 267.

Table 267 – Alarm mode

Value	Meaning
0	1 alarm of each type
1	2 alarms in total
2	4 alarms in total
3	8 alarms in total
4	12 alarms in total
5	16 alarms in total
6	24 alarms in total
7	32 alarms in total

NA To Abort

This attribute reflects the behaviour of the DP-master (Class 1) if the assigned DP-slave does not respond at the MS0 AR in the DIAG2 state and the data exchange mode.

- If this attribute is set to FALSE the DP-master continues in case of no response (Status = NA) of the assigned DP-slave.
- If this attribute is set to TRUE the DP-master aborts the MS0 AR in case of no response (Status = NA) of assigned the DP-slave (abort means sending an unlock command and to re-establish the MS0 AR).

Attribute Type: Boolean

Ignore ACIr

This attribute in combination with the Error Action Flag allows a DP-slave specific reaction of the DP-master (Class 1) in case of failed data transfers.

- If the attribute is set to FALSE the DP-master (Class 1) processes the auto-clear function as defined in the Error Action Flag.
- If the attribute is set to TRUE the DP-master (Class 1) ignores the auto-clear function as defined in the Error Action Flag.

Attribute Type: Boolean

Publisher Flag

This attribute indicates if the correspondent DP-slave shall operate as Publisher. If the attribute is set to FALSE the correspondent DP-slave shall not operate as Publisher. If the attribute is set to TRUE the correspondent DP-slave shall be addressed as Publisher by the DP-master (Class 1).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

MS1 Timeout

This attribute contains the value for the Timeout of the acyclic communication on the MS1 AR and specifies the maximum period of time it may take a DP-slave to provide a response. The time contained in this attribute is the maximum time of the assigned DP-slave.

Attribute Type: Unsigned16

Allowed values: 1 to 2¹⁶-1

Time Base: 10ms

Prm Command supported

This attribute reflects whether the DP-slave uses PrmCmd within the extended functionality:

- If the attribute is set to FALSE the DP-slave supports no PrmCmd.
- If the attribute is set to TRUE the DP-slave supports PrmCmd.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Prm Data Len

This attribute contains the length of Prm Data inclusive the length parameter.

Attribute Type: Unsigned8

Allowed values: 9 to 246

Prm Data

This attribute reflects the parameterisation data sent to the corresponding DP-slave by the DP-master (Class 1). The structure of this attribute is described in 8.2.10.4.2.2.

Cfg Data Len

This attribute contains the length of Cfg Data inclusive the length parameter.

Attribute Type: Unsigned8

Allowed values: 3 to 246

Cfg Data

This attribute reflects the configuration information sent to the DP-slave. The structure of this attribute is described in 8.2.10.4.2.2.

Add Tab Len

This attribute contains the length of Add Tab inclusive the length parameter.

Attribute Type: Unsigned16

Allowed values: 2 to $2^{16}-31$

Add Tab

This attribute contains the address assignment table of the DP-slave. In case of a Programmable Logic Controller, this list contains the Programmable Logic Controller addresses of the decentralised addresses.

Slave User Data Len

This attribute contains the length of the Slave User Data inclusive the length parameter.

Attribute Type: Unsigned16

Allowed values: 2 to $2^{16}-31$

Slave User Data

This attribute contains manufacturer specific data that characterise the DP-slave for the DP-master (Class 1).

Attribute Type: Octet String

Ext User Prm Data Len

This attribute contains the length of Ext Prm Data inclusive the length parameter.

Attribute Type: Unsigned8

Allowed values: 2 to 246

Ext User Prm Data

This attribute reflects the extended user parameterisation data sent to the correspondent DP-slave by the DP-master (Class 1). The DP-master (Class 1) will send this data between the parameter data and the configuration information.

List of assigned CREPs for the MS0 AR

This attribute contains the CREPs of the MS0 AR assigned for this DP-slave and is composed of the following list elements:

CREP

This attribute contains one CREP of the MS0 AR.

Assigned AREP for the MS1 AR

This attribute contains the AREP of the MS1 AR assigned for this DP-slave.

Assigned AREP for the MS3 AR

This attribute contains the AREP of the MS3 AR assigned for this DP-slave.

Assigned AREP for the MS0 AR

This attribute contains the AREP of the MS0 AR assigned for all DP-slaves.

8.2.6.2.2.4 Invocation of the master parameter object

For the invocation of the Master Parameter object the following rules apply:

- Only one Master Parameter object can be invoked in a DP-master (Class 1).
- Up to 125 Slave Para elements with an unique value for the attribute Slave DL-address can be contained in a Master Parameter object of a DP-master (Class 1).

8.2.6.3 Management service specification

8.2.6.3.1 Get master diag

With this service a DP-master (Class 2) can read a single element (Master Status, System Diagnosis, Data Transfer List, DP-slave Diag of a single DP-slave) from the Master Diag object of a DP-master (Class 1). This service shall only be used in conjunction with the MM1 AR.

Table 268 shows the parameters of the service.

Table 268 – Get Master Diag

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Mdiag Identifier	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Diagnosis Data			M	M(=)
DP-slave Diag			S	S(=)
System Diagnosis			S	S(=)
Data Transfer List			S	S(=)
Master Status			S	S(=)
Result(-)			S	S(=)
AREP			M	M(=)
Status			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Mdiag Identifier

This parameter specifies the Element of the Master Diag object to be read.

The allowed values are shown in Table 269.

Table 269 – Mdiag identifier

Value	Meaning
0-125	DP-slave Diag of the DP-slave with the corresponding DL-address
126	System Diagnosis
127	Master Status
128	Data Transfer List

Result(+)

This selection type parameter indicates that the service request succeeded.

Diagnosis Data

This parameter contains one of the following element of the Master Diag object as requested.

DP-slave Diag

This parameter contains the Diag Data of the addressed DP-slave.

System Diagnosis

This parameter contains the System Diagnosis of the Master Diag object (see 8.2.6.2.1)

Master Status

This parameter contains the Master Status of the Master Diag object (see 8.2.6.2.1)

Data Transfer List

This parameter contains the Data Transfer List of the Master Diag (see 8.2.6.2.1)

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, RE, TO, FE, NE, IP, AD, EA, LE

8.2.6.3.2 Start seq

With the Start Seq service the beginning of an Upload or Download sequence is signalled by the DP-master (Class 2) in the DP-master (Class 1). In this case the DP-master (Class 1) can set an access protection for the addressed area until the End Seq service signals the end of the sequence. The addressed data can be transmitted in blocks with the services Upload and Download. This service shall only be used in conjunction with the MM1 AR.

Table 270 shows the parameters of the service.

Table 270 – Start Seq

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
AREP	M	M		
Area Code	M	M(=)		
Timeout	M	M(=)		
Result(+)			S	S(=)
AREP			M	M
Max Len Data Unit			C	C
Result(-)			S	S(=)
AREP			M	M
Status			M	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Area Code

The parameter Area Code identifies the element of the Master Parameter object which shall be loaded or read. If the parameter Area Code is set to a value of 255, local access protection for a determined Area Code is not guaranteed for the services following. The local access protection ensures that the User of the DP-master (Class 1) cannot access the partially loaded data during the service sequence.

Attribute Type: Unsigned8

The allowed values are shown in Table 271.

Table 271 – Area code (start seq)

Value	Meaning
0 to 125	Slave Para of the DP-slave with the corresponding DL-address
126	reserved
127	Bus Para
128	reserved
129	statistic counters
130 to 135	reserved
136 to 139	for the transfer of the Master Parameter Set
140 to 254	reserved
255	Start Seq: No local access protection

Timeout

This parameter defines the control time between two successive Upload/Download services. If this time expires the access protection is deactivated. The previously transferred data are invalid.

Attribute Type: Unsigned16

Time Base: 1ms

Result(+)

This selection type parameter indicates that the service request succeeded.

Max Len Data Unit

The parameter Max Len Data Unit defines the maximum possible length of the parameter Data in the following Upload/Download APDUs.

Attribute Type: Unsigned8

Range: 1 to 240

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, TO, FE, RE, NE, AD, IP, NI, SE, SC, EA, LE

8.2.6.3.3 Download

This service is used to transfer the values for the attributes of the Master Parameter object from the DP-master (Class 2) to a DP-master (Class 1). The beginning of the Download sequence can be signalled with the service Start Seq. The actual data set is transmitted in blocks with the Download service in the addressed area. After the transfer is completed the sequence shall be terminated with the End Seq service, if the Download sequence was started with the Start Seq service. This service shall only be used in conjunction with a MM1 AR.

Table 272 shows the parameters of the service.

Table 272 – Download

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
AREP	M	M		
Area Code	M	M		
Add Offset	M	M		
Data	M	M		
Result(+)			S	S(=)
AREP			M	M
Result(-)			S	S(=)
AREP			M	M
Status			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Area Code

The parameter Area Code identifies the element of the Master Parameter object which shall be loaded.

Attribute Type: Unsigned8

Allowed values: see 8.2.6.3.2

Add Offset

This parameter contains the offset to the begin of the area which is specified by the Area Code.

Attribute Type: Unsigned16

Data

This parameter contains the data to be transferred.

Attribute Type: Octet String

Length: 1 to 240

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, TO, FE, RE, NE, AD, EA, LE, SC, NI, NC

8.2.6.3.4 Upload

This service is used to transfer the values of the attributes of the Master Parameter object from the DP-master (Class 1) to a DP-master (Class 2). The Upload sequence can be executed in the same way as the Download sequence. It is possible to transfer single data areas without the services Start Seq and End Seq. In this case the consistency of the transferred data is not guaranteed. This service shall only be used in conjunction with a MM1 AR.

Table 273 shows the parameters of the service.

Table 273 – Upload

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
AREP	M	M		
Area Code	M	M		
Add Offset	M	M		
Data Len	M	M		
Result(+)			S	S(=)
AREP			M	M
Data			C	C(=)
Result(-)			S	S(=)
AREP			M	M
Status			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Area Code

The parameter Area Code identifies the element of the Master Parameter object which shall be read

Attribute Type: Unsigned8

Allowed values: see 8.2.6.3.2

Add Offset

This parameter identifies the offset to the begin of the area which is specified by the Area Code.

Attribute Type: Unsigned16

Data Len

The parameter defines the length of the requested data.

Attribute Type: Unsigned8

Allowed values: 1 to 240

Result(+)

This selection type parameter indicates that the service request succeeded.

Data

This parameter contains the requested data.

Attribute Type: Octet String

Length: 1 to 240

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, TO, FE, RE, NE, EA, LE, NI, SC, AD

8.2.6.3.5 End seq

This service is used to mark the end of a sequence of consistent transfers of data blocks of the Master Parameter object. This service shall only be used in conjunction with a MM1 AR.

Table 274 shows the parameters of the service.

Table 274 – End Seq

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
AREP	M	M		
Result(+)			S	S(=)
AREP			M	M
Result(-)			S	S(=)
AREP			M	M
Status			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, TO, FE, RE, NI, SE, NE, AD, EA, LE, NC

8.2.6.3.6 Act Para Brct

This service is used to activate the previously transferred values for the Master Parameter object by the DP-master (Class 2). This service may be performed simultaneously to one (single) or many (multicast) DP-master (Class 1). In the operation modes CLEAR and OPERATE of the DP-master (Class 1), the activation of the Bus Para element with changed DL parameters data rate or TS will be rejected. This service is only locally confirmed because a possible change of the data rate shall take effect at all addressed DP-masters (Class 1) at the same time. This service shall only be used in conjunction with a MM2 AR.

Table 275 shows the parameters of the service.

Table 275 – Act Para Brct

Parameter name	Req	Ind	Cnf
Argument	M	M	
AREP	M	M	
Area Code	M	M	
Result(+)			S
AREP			M
Result(-)			S
AREP			M
Status			M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Area Code

The parameter Area Code identifies the element of the Master Parameter object which shall be activated.

Attribute Type: Unsigned8

The allowed values are shown in Table 276.

Table 276 – Area code (Act Para Brct)

Value	Meaning
0 to 126	not allowed
127	Bus Para
128 to 129	reserved
130 to 135	reserved
136 to 139	for the activation of the Master parameter object
140 to 254	reserved
255	not allowed

In the presence of several DP-masters (Class 2) it shall be guaranteed that the old and the new bus parameter sets are compatible.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS

8.2.6.3.7 Act param

This service is used by the DP-master (Class 2) for the following purposes:

- to activate or to deactivate a parameterised DP-slave
- to change the operation mode of the DP Application Layer
- to accept and activate the values for the Bus Para Elements of the Master Parameter object.

If the loaded bus parameter set contains a changed data rate or station address for the DP-master (Class 1) the new bus parameter set shall be activated with the Act Para Brct service. This service shall only be used in conjunction with a MM1 AR.

Table 277 shows the parameters of the service.

Table 277 – Act param

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M		
AREP	M	M		
Area Code	M	M(=)		
Activate	M	M(=)		
Result(+)			S	S(=)
AREP			M	M
Result(-)			S	S(=)
AREP			M	M
Status			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier of the desired AR.

Area Code

The parameter Area Code identifies the element of the Master Parameter object which shall be activated/deactivated with the exception of the operation mode.

Attribut Type: Unsigned8

The allowed values are shown in Table 278.

Table 278 – Area code (Act param)

Value	Meaning
0 to 125	Slave Para of the DP-slave with the corresponding DL-address The (Flag/Bit) Active in the slave parameter set of the DP-master (Class 1) is set as specified in the parameter Activate.
126	Reserved
127	Bus Para
128	Operation mode (description see 8.2.5.3.29)
129	Reserved
130 to 135	reserved
136 to 139	activation of the Master Parameter Set
140 to 255	reserved

Activate

The meaning and the allowed values of this parameter are depending on the Area Code.

Attribute Type: Unsigned8

The allowed values are shown in Table 279.

Table 279 – Activate

Area Code	Allowed values for the parameter Activate	
0 to 125: (Slave Para)	0	(= Deactivate)
	128	(= Activate)
127: (Bus Para)	255	(= Activate)
128: (operation mode)	64	(= STOP)
	128	(= CLEAR)
	192	(= OPERATE)

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter provide detailed information about the failure.

Allowed values: DS, NA, RS, RR, UE, TO, FE, RE, NE, AD, IP, SC, NI, DI, EA, LE

8.2.7 Load region ASE**8.2.7.1 Overview**

A Load Region object represents an unstructured memory area whose contents may be uploaded (Pull) or downloaded (Push). Unstructured in this context means that the memory area is represented only as an ordered sequence of octets. No other structure is apparent.

A Load Region object may represent an unnamed volatile memory area, such as that implemented by dynamic computer memory, or a named non-volatile memory object, such as a file. The contents of a Load Region object are referred to as a load image. Load images may contain programs or data. The transfer of the LR Data to or from a Load Region object is performed using the Load Region services.

The Load Region objects shall clearly be separated from the Process Data objects and the Function Invocation objects. No overlapping is allowed.

To maintain integrity, only one download process for a Load Region object is permitted at a time. The Load Region State attribute of the Load Region object is used to indicate whether the Load Region object is empty, being downloaded, or loaded (see the definition of the attribute for a complete list of the states). The Load Region object may be cleared using the Initiate Load service.

The Load Region State attribute also indicates the progress of an upload. Multiple uploads of a Load Region object via different ARs are allowed. This attribute is defined to separate the state of the contents of the Load Region object from the operation of having the contents uploaded.

The Load Region ASE provides services that permit an AP to initiate the upload or download (Pull or Push) of Load Region objects. They identify the Load Region object and whether to Pull or Push as parameters of the initiate load request.

The LR Data may be transferred in segments by using the Pull Segment or the Push Segment service.

LR Data segments are uploaded by the Client issuing Pull Segment requests for them. The AP that contains the LR Data responds by returning the requested segment, indicating with a parameter when the final segment of the LR Data has been returned.

LR Data segments are downloaded by the Client having the AP containing the LR Data image by sending individual segments to the Server and wait for a response that indicates whether the LR Data segment was received. In this case, the Client indicates when the last LR Data segment has been transferred. After the last LR Data segment has been received, the AP that initiated the transfer ends the load process by issuing a Terminate Load request to the remote AP. The Terminate Load request may also be used by an AP if the load process cannot be completed successfully.

To avoid deadlocks at the Server a timeout shall be issued to control the activities of an AR regarding the upload or download procedures.

8.2.7.2 Load region class specification

8.2.7.2.1 Template

The Load Region object is modelled as a Server object. The Load Region object specifies the attributes and services used by Client APs to upload and/or download LR Data. The specification of the Client AP is beyond the scope of this specification.

A Load Region object is described by the following template:

DP ASE:		Load Region ASE
CLASS:	Load Region	
CLASS ID:		not used
PARENT CLASS:		TOP

ATTRIBUTES:

1. (m) Key Attribute: Identifier
2. (m) Attribute: LR Size
3. (m) Attribute: Access Rights MS1
4. (m) Attribute: Access Rights MS2
5. (c) Constraint: Access Rights MS2 = Right to Pull or Right to Pull/Push
- 5.1. (m) Attribute: List of SCL Pull
- 5.1.1. (m) Attribute: SCL
6. (c) Constraint: Access Rights MS2 = Right to Push or Right to Pull/Push
- 6.1. (m) Attribute: List of SCL Push
- 6.1.1. (m) Attribute: SCL
7. (m) Attribute: Load Region State
8. (m) Attribute: Actual LR Size
9. (o) Attribute: Identifier of Related Function Invocation
10. (m) Attribute: Max Segment Length
11. (m) Attribute: Max Response Delay
12. (m) Attribute: Uploadcount
13. (o) Attribute: Load Region Additional Information
14. (o) Attribute: Load Region Name
15. (o) Attribute: Local Detail

SERVICES:

1. (m) OpsService: Initiate Load
2. (o) OpsService: Push Segment
3. (o) OpsService: Pull Segment
4. (m) OpsService: Terminate Load

8.2.7.2.2 Attributes**Identifier**

This key attribute is composed of API, Slot Number and LR Index to define to which Application Process Identifier, module and to which index within the specified module the Load Region object belongs. This Identifier shall not be used by another Action object, Load Region object, Function Invocation object or Process Data object.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

LR Size

This attribute specifies the length of the Load Region object in octets.

Attribute Type: Unsigned32

Allowed values: 1 to 2³²-1

Access Rights MS1

This attribute defines the type of access rights defined for a MS1 AR for the Load Region object.

Attribute Type: Unsigned8

The allowed values are shown in Table 280.

Table 280 – Access rights MS1

Value	Meaning
0	Right to Push
1	Right to Pull
2	Right to Pull/Push
3	No Access

Access Rights MS2

This attribute defines the type of access rights defined for a MS2 AR for the Load Region object.

Attribute Type: Unsigned8

The allowed values are shown in Table 281.

Table 281 – Access rights MS2

Value	Meaning
0	Right to Push
1	Right to Pull
2	Right to Pull/Push
3	No Access

List of SCL Pull

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for Pull access protection for this Load Region object. The rules for accepting or rejecting a Pull access are described in 8.2.1.3 according to the read access.

Attribute Type: Unsigned8

List of SCL Push

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for Push access protection for this Load Region object. The rules for accepting or rejecting a Push access are described in 8.2.1.3 according to the write access protection.

Attribute Type: Unsigned8

Load Region State

This attribute specifies the state of the Load Region object.

Attribute Type: Unsigned8

The allowed values of this attribute are shown in Table 282.

Table 282 – Load region state

Value	Meaning
0	DOWNLOADABLE
1	DOWNLOADING
2	DOWNLOAD SUCCESS
3	LOADED
4	IN-USE

Load Region State attribute constraints:

- A Load Region object whose contents are stored in non-erasable memory cannot support downloading transfers. Its valid state values are LOADED and IN USE.
- A Load Region object whose contents are stored in erasable memory supports downloading transfers. Its valid state values are DOWNLOADABLE, DOWNLOADING, DOWNLOADING SUCCESS, LOADED and IN USE,

Actual LR Size

This attribute specifies the used length of the Load Region object in octets. A Load Region object whose content is cleared shall have a value of zero for this attribute.

Attribute Type: Unsigned32

Allowed values: 0 to 2³²-1

Identifier of Related Function Invocation

This optional attribute is composed of API, Slot Number and FI Index to define to which Application Process Identifier, module and to which index within the specified module the related Function Invocation object belongs.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

Max Segment Length

This attribute specifies the maximum length in octets of each Push Segment or Pull Segment which shall be used for the up- or download of this Load Region object.

Attribute Type: Unsigned8

Allowed values: 1 to 234

Max Response Delay

This attribute specifies the maximum time period (in units of 10ms) the response of an Initiate Load, Pull Segment, Push Segment or Terminate Load service request may be delayed by the Server due to processing of the request. The value of this attribute shall be less than the value of the attribute MS1 Timeout of the ARL DP-master (Class 1) object.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Uploadcount

This attribute indicates the number of Clients which currently upload the contents of the Load Region object. The value of this attribute has to be zero to download the Load Region object. This attribute is incremented each time a Client uploads the Load Region object in parallel. This attribute is decremented each time a Client finishes the upload of the Load Region object.

Attribute Type: Unsigned8

Allowed values: 0 to 255

Load Region Additional Information

This optional attribute contains additional information about the Load Region object, e.g. permitted data formats.

Attribute Type: Octet String

Load Region Name

This optional attribute defines the name of the Load Region object.

Attribute Type: Visible String(32)

Local Detail

This optional attribute contains additional local information about the Load Region object.

Attribute Type: Octet String

8.2.7.2.3 Invocation of the load region object

For the invocation of the Load Region object the following rules apply:

- Load Region objects shall not exceed the total length of $2^{32}-1$ octets. The attribute LR Size has to be set accordingly.
- At least the Push Segment or Pull Segment or both services shall be allowed for the access of the Load Region object.
- At least one AR shall have access to the Load Region object.
- Only one Function Invocation shall be related to a Load Region object.
- The access rights shall be set accordingly to the allowed service(s).

8.2.7.3 Load region service specification**8.2.7.3.1 Initiate load**

This service is used by an AP to initiate the download or upload of a Load Region. A parameter of the service indicates whether the load image will be transferred into the Load Region object using the Pull Segment service or from the Load Region object using the Push Segment service. This service shall only be used in conjunction with the MS1 or MS2 AR. Table 283 shows the parameter of the service.

Table 283 – Initiate load

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
LR Index	M	M(=)		
Load Type	M	M(=)		
Load Image Size	C	C(=)		
Intersegment Request Timeout	M	M(=)		
Additional Information	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Actual LR Size			M	M(=)
Max Response Delay			M	M(=)
Max Segment Length			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Load Region object in a specific slot (typically a module).

LR Index

This parameter specifies the index of the Load Region to be downloaded or to be uploaded.

Load Type

This parameter specifies whether the Load Region object has to be downloaded or uploaded.

Allowed values: Push, Pull

Load Image Size

This conditional parameter indicates the number of octets which has to be downloaded to the Load Region object if the parameter Load Type is equal Push. If the parameter Load Type is equal Pull the parameter Load Image Size is not present. This parameter shall have a value of zero if the content of the Load Region object shall be cleared.

Attribute Type: Unsigned32

Allowed values: 0 to $2^{32}-1$

Intersegment Request Timeout

This parameter specifies the timeout period on the Server side between Pull Segment or Push Segment service requests, in units of 100ms. The default values are shown in Table 284.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Table 284 – Default values for the parameter Intersegment Request Timeout

Baudrate (kbit/s)	≤187,5	500	1500	3000	6000	12000
Intersegment Request Timeout (sec)	40	20	10	10	10	10

Additional Information

This optional parameter specifies additional information for the subsequent load procedure, e.g. the desired data format.

Result(+)

This selection type parameter indicates that the service request succeeded.

Actual LR Size

This parameter contains the value

- of the attribute Actual LR Size of the Load Region object if the parameter Load Type has had the value Pull in the previous request primitive
- of the parameter Load Image Size if the parameter Load Type has had the value Push in the previous request primitive.

Attribute Type: Unsigned32

Allowed values: 0 to $2^{32}-1$

Max Response Delay

This parameter contains the value of the attribute Max Response Delay of the Load Region object. The Client AP shall monitor the Initiate Load, Push Segment, Pull Segment or Terminate Load service request and response sequences with this time parameter.

Max Segment Length

This parameter contains the value of the attribute Max Segment Length of the Load Region object.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, load image size error, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type.

8.2.7.3.2 Push segment

This optional service is used to transfer LR Data to the Load Region object after the download process has been initiated. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 285 shows the parameter of the service.

Table 285 – Push segment

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
LR Index	M	M(=)		
Segment Length	M	M(=)		
Segment Number	M	M(=)		
More Follows	M	M(=)		
Data	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Load Region object in a specific slot (typically a module).

LR Index

This parameter specifies the index of the Load Region object to be downloaded.

Segment Length

This parameter indicates the number of octets of the Load Region object which has to be downloaded.

Attribute Type: Unsigned8

Allowed values: 1 to 234

Segment Number

This parameter indicates the number of the segment which is being transferred. Segment numbers are in ascending order without gaps beginning with the value one.

Attribute Type: Unsigned32

Allowed values: 1 to 2³²-1

More Follows

This parameter indicates whether or not any additional segments remain to be transmitted.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Data

The parameter Data contains the segment of the Load Image which has to be downloaded and consists of the number of octets indicated in the parameter Segment Length of the request. This parameter has to be composed of the data type Octet String.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, length error, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type, feature not supported.

8.2.7.3.3 Pull segment

This optional service is used to transfer LR Data from the Load Region object after the upload process has been initiated. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 286 shows the parameter of the service.

Table 286 – Pull segment

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
LR Index	M	M(=)		
Segment Length	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Segment Length			M	M(=)
Segment Number			M	M(=)
More Follows			M	M(=)
Data			C	C(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Load Region object in a specific slot (typically a module).

LR Index

This parameter specifies the index of the Load Region to be uploaded.

Segment Length

This parameter indicates the number of octets of the Load Region object which shall be uploaded with this request.

Attribute Type: Unsigned8

Allowed values: 1 to 234

Result(+)

This selection type parameter indicates that the service request succeeded.

Segment Length

The value of this parameter is equal to the equivalent parameter of the request if the remaining octets to be uploaded are greater or equal to the requested segment length. Otherwise this parameter contains the number of remaining octets.

Attribute Type: Unsigned8

Allowed values: 0 to 234

Segment Number

This parameter indicates the number of the segment which is being transferred. Segment numbers are in ascending order without gaps beginning with the value one.

Attribute Type: Unsigned32

Allowed values: 1 to 2³²-1

More Follows

This parameter indicates whether or not any additional segments remain to be transmitted.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Data

This conditional parameter is only present if the value of the parameter Segment Length of the response is different from zero and contains the segment of the Load Image which has to be uploaded and consists of the number of octets indicated in the parameter Segment Length of the response. This parameter has to be composed of the data type Octet String.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed value: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type, feature not supported.

8.2.7.3.4 Terminate load

This service is used to terminate the upload or download process. It shall be used upon successful completion, or to abort an upload or download in progress.

If the up- or download process is being aborted with the Terminate Load service, then

- in case of a download the contents of the Load Region object shall be deleted (the attribute Actual LR Size shall be set to zero and the attribute Load Region State shall be set to DOWNLOADABLE);
- in case of an upload the value of the attribute Uploadcount of the Load Region object shall be decremented.

This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 287 shows the parameter of the service.

Table 287 – Terminate load

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
LR Index	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Load Region object in a specific slot (typically a module).

LR Index

This parameter specifies the index of the Load Region object whose upload or download process shall be terminated.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed value: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type.

8.2.7.4 Behaviour of the load region object**8.2.7.4.1 Load region state machine description****8.2.7.4.1.1 Overview**

The behaviour of a Load Region object is defined by the Load Region state machine. Each Load Region object is represented by an own instance of this state machine.

The Load Region state machine represents a part of the User functionality and is included to ensure interoperability. It is assumed that other parts of the User functionality perform the following checks:

- validation of the existence and addressing of the Load Region object
- validation of the access rights
- validation of temporary system restrictions for the use of the Load Region object

and generate appropriate Error Codes if necessary.

Two types of Load Region state machines are defined. One represents the behaviour of Load Region objects whose contents are stored in erasable memory and are therefore capable to be downloaded. The other one represents the behaviour of Load Region objects whose contents are stored in non erasable memory and are therefore not capable to be downloaded.

The Load Region object may be used by Function Invocation objects. Therefore the Load Region state machines provide an interface to the Function Invocation state machines for co-ordination purposes.

The main functionalities of the Load Region state machine are:

- monitoring of the valid sequence of services executed on the Load Region object
- to ensure that a download is only executed if no upload is in progress
- to ensure that an upload or download is only executed if the Load Region object is not used by a Function Invocation object
- to ensure that an upload is only executed if no download is in progress
- supervision of the Client activity.

8.2.7.4.1.2 Primitive definitions**8.2.7.4.1.2.1 Primitives exchanged between the user and the load region state machine**

Table 288 and Table 289 show the primitives exchanged between the User and the Load Region state machine.

Table 288 – Primitives issued by the user to the Load Region state machine

Primitive Name	Source	Associated parameters	Functions
Abort.ind	User	AREP Locally Generated Subnet Instance Reason Code Additional Detail	
DP-slave Stopped.ind	User	AREP	
Initiate Load.ind	User	AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request, Timeout	
Pull Segment.ind	User	AREP, Slot Number, LR Index, Segment Length	
Push Segment.ind	User	AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data	
Terminate Load.ind	User	AREP, Slot Number, LR Index	

Table 289 – Primitives issued by the Load Region state machine to the user

Primitive Name	Source	Associated parameters	Functions
Initiate Load.rsp(+)	Load Region	AREP, Actual LR Size, Max Response Delay, Max Segment Length	
Initiate Load.rsp(-)	Load Region	AREP, Error Code	
Pull Segment.rsp(+)	Load Region	AREP, Segment Length, Segment Number, More Follows, Data	
Pull Segment.rsp(-)	Load Region	AREP, Error Code	
Push Segment.rsp(+)	Load Region	AREP	
Push Segment.rsp(-)	Load Region	AREP, Error Code	
Terminate Load.rsp(+)	Load Region	AREP	
Terminate Load.rsp(-)	Load Region	AREP, Error Code	

The parameters used with the primitives are described in the Service Specification of the Load Region object (see 8.2.7.3).

8.2.7.4.1.2.2 Primitives exchanged between the function invocation and the load region state machine

Table 290 and

Table 291 show the primitives exchanged between the Function Invocation and the Load Region state machines.

Table 290 – Primitives issued by the Function Invocation to the Load Region state machine

Primitive Name	Source	Associated parameters	Functions
DeleteInUse.ind	Function Invocation	FI Identifier, LR Identifier	The Function Invocation indicates with this primitive that the Load Region Object is no longer to be used.
SetInUse.ind	Function Invocation	FI Identifier, LR Identifier	The Function Invocation indicates with this primitive that it wants to make use of the Load Region Object.

Table 291 – Primitives issued by the Load Region to the Function Invocation state machine

Primitive Name	Source	Associated parameters	Functions
DeleteInUse.rsp (+)	Load Region	FI Identifier, LR Identifier	The Load Region responds with this primitive to a previous received DeleteInUse.ind.
SetInUse.rsp (+/-)	Load Region	FI Identifier, LR Identifier	The Load Region responds with this primitive to a previous received SetInUse.ind.

8.2.7.4.1.3 State definitions

Table 292 shows the definitions of the states of the Load Region state machine.

Table 292 – Load Region state definitions

State	Description
DOWNLOADABLE	This state indicates that the Load Region object is empty, but is capable of being downloaded.
DOWNLOADING	This state indicates that the download sequence has been initiated.
DOWNLOAD SUCCESS	This transient state indicates that the download sequence has succeeded, but has not yet been terminated.
LOADED	This state indicates that the download sequence has terminated successfully.
IN-USE	This state indicates that the Load Region object is loaded and is currently being used by a Function Invocation object.

8.2.7.4.1.4 Local variable definitions

AREP List

The AREP List consists of a one dimensional list containing AREP(s) which represent application relationships who currently execute a download or upload sequence.

Segment Number List

The Segment Number List consists of a two dimensional list containing AREP(s), also contained in the AREP List, and the actual Segment Number of the sequence executed on this relationship.

Actual Length List

The Actual Length List consists of a two dimensional list containing AREP(s), also contained in the AREP List, and the Actual Length of the amount of data already exchanged in the sequence executed on this relationship.

FI Identifier

This local variable contains the Identifier (API, Slot Number, FI Index) of the Function Invocation object which currently uses this Load Region object.

8.2.7.4.1.5 Timer definitions

Intersegment Request Timer

With the Intersegment Request Timer the Load Region state machine supervises the Client activity during an upload or download sequence.

8.2.7.4.1.6 Function definitions

The following functions contained in Table 293 are used by both Load Region state machines which describe the behaviour of a Load Region object stored in erasable memory or in non erasable memory.

Table 293 – Load Region function table

Function Name	Description
IsServiceAccepted(Load Type, Load Image Size, Additional Information)	This function checks by calling subfunctions if the service is acceptable and match local object conditions. A) If the results of the called subfunctions are as follows: IsLRSizeOK(Load Image Size) = TRUE IsPushAndUpload() = FALSE IsAddInfoOK(Additional Information) = TRUE than it returns TRUE. B) Otherwise it returns FALSE.
IsPushAndUpload()	This function checks whether an Upload sequence is in progress. A) If the attribute Uploadcount is greater than zero then it returns TRUE. B) Otherwise it returns FALSE.
IsLRSizeOK(Load Image Size)	This function checks whether the Load Image Size is matching the attribute LR Size. A) If the Load Image Size is less or equal LR Size then it returns TRUE. B) Otherwise it returns FALSE.
IsAddInfoOK(Additional Information)	This function checks whether the Additional Information is matching the attribute Load Region Additional Information. A) If the Additional Information is compatible to the attribute Load Region Additional Information then it returns TRUE. B) Otherwise it returns FALSE.
StoreSegmentNumber(AREP, Segment Number)	This function increments the Segment Number with one and stores it AREP related.
GetSegmentNumber(AREP)	This function returns the stored AREP related Segment Number.
CheckSegmentNumber(AREP, Segment Number)	This function compares the Segment Number with the stored AREP related Segment Number. A) If the Segment Number is equal to the stored AREP related Segment Number then it returns TRUE. B) Otherwise it returns FALSE.
StoreActualLength(AREP, Actual Length)	This function stores the Actual Length AREP related.
GetActualLength(AREP)	This function returns the stored AREP related Actual Length.

Function Name	Description
GetActualSegLength(AREP, Segment Length)	This function reads the stored AREP related Actual Length by calling the function GetActualLength and calculates the actual Segment Length according to the following equation: A) If the Actual LR Size-Actual Length is greater or equal to Segment Length and less or equal to Max Segment Length then it returns Segment Length. B) If the Actual LR Size-Actual Length is greater or equal to Segment Length and greater than Max Segment Length then it returns Max Segment Length. C) If the Actual LR Size-Actual Length is less than Segment Length and less or equal to Max Segment Length then it returns Actual LR Size-Actual Length. D) If the Actual LR Size-Actual Length is less than Segment Length and greater Max Segment Length then it returns Max Segment Length. Furthermore this function calculates the new Actual Length according to the following equation: Actual Length = Actual Length + returned value. The new Actual Length is stored AREP related by calling the function StoreActualLength.
CheckActualSegLength(AREP, Segment Length)	This function reads the stored AREP related Actual Length by calling the function GetActualLength and checks the Segment Length according to the following conditions: A) If Segment Length is greater than Max Segment Length then it returns FALSE. B) If Segment Length is greater than Actual LR Size-Actual Length then it returns FALSE. C) Otherwise it returns TRUE.
IsLastSegment(AREP)	This function reads the stored AREP related Actual Length by calling the function GetActualLength and calculates whether it is the last segment to transfer according to the following conditions: A) If the Actual Length is equal to Actual LR Size then it returns FALSE. B) Otherwise it returns TRUE.
StoreAREP(AREP)	This function stores the AREP in the AREP List.
CheckAREP(AREP)	This function checks whether the AREP is contained in the AREP List according to the following conditions: A) If the AREP is contained then it returns TRUE. B) Otherwise it returns FALSE.
DeleteAREP(AREP)	This function deletes the AREP in the AREP List.
StartIntersegReqTimer(AREP, T1)	This function starts or restarts the AREP related Intersegment Request Timer.
StopIntersegReqTimer(AREP)	This function stops the AREP related Intersegment Request Timer.
CheckMoreFollows(AREP; More Follows)	This function reads the stored AREP related Actual Length by calling the function GetActualLength and checks whether it is the last segment to receive according to the following conditions: A) If the Actual Length is equal to Actual LR Size then it returns TRUE. B) Otherwise it returns FALSE.
ClearContent	This function deletes the content of the Load Region object.
StoreFIIdentifier(FI Identifier)	This function stores the FI Identifier.
CheckFIIdentifier(FI Identifier)	This function checks the FI Identifier according to the following conditions: A) If the FI Identifier is equal to the stored FI Identifier then it returns TRUE. B) Otherwise it returns FALSE.
DeleteFIIdentifier(FI Identifier)	This function deletes the stored FI Identifier.
StoreData(Data)	This function stores the Data into the content of the Load Region object.
GetData()	This function gets the Data of the content of the Load Region object.

8.2.7.4.2 Load region state diagrams

8.2.7.4.2.1 Load region state diagram for erasable memory

Figure 61 shows the state diagram of a Load Region for erasable memory.

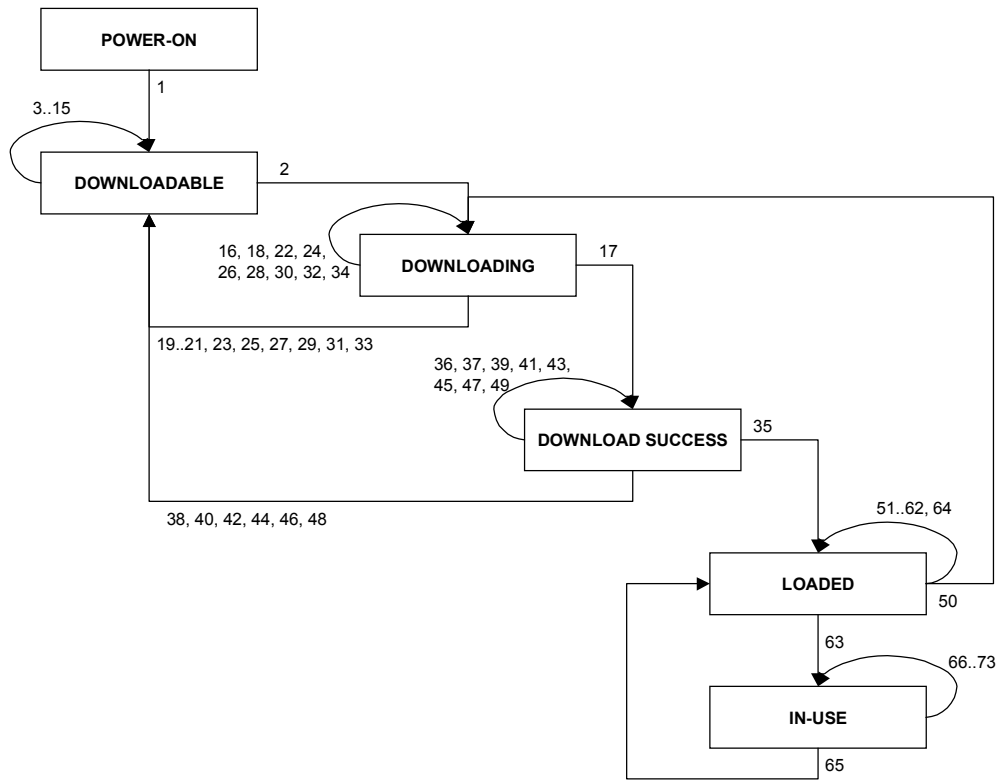


Figure 61 – Load Region state diagram for erasable memory

8.2.7.4.2.2 Load region state diagram for non erasable memory

Figure 62 shows the state diagram of a Load Region for non erasable memory.

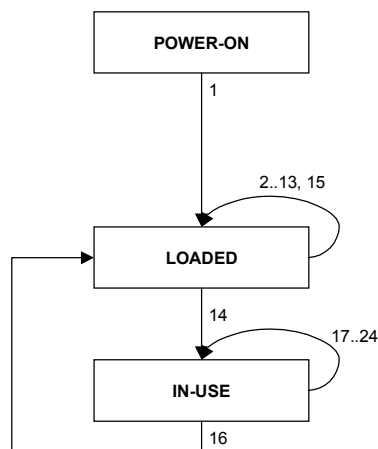


Figure 62 – Load region state diagram for non erasable memory

8.2.7.4.3 Load region state tables

8.2.7.4.3.1 Load region state table for erasable memory

Table 294 shows the state table of a Load Region for erasable memory.

Table 294 – Load Region state table for erasable memory

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	=> Uploadcount:=0	DOWNLOA DABLE
2	DOWNLOA DABLE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) && (IsServiceAccepted(Load Type, Load Image Size, Additional Information)=TRUE) => StoreAREP(AREP) Segment Number:=0 StoreSegmentNumber(AREP, Segment Number) Actual Length:=0 StoreActualLength(AREP, Actual Length) T1:=Intersegment Request Timeout Actual LR Size:=Load Image Size Load Region State:=DOWNLOADING StartIntersegReqTimer(AREP, T1) Initiate Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length)	DOWNLOA DING
3	DOWNLOA DABLE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) && (IsServiceAccepted(Load Type, Additional Information)=TRUE) => T1:=Intersegment Request Timeout Uploadcount:=Uploadcount+1 StoreAREP(AREP) StartIntersegReqTimer(AREP, T1) Initiate Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length)	DOWNLOA DABLE
4	DOWNLOA DABLE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) && (IsServiceAccepted(Load Type, Load Image Size, Additional Information)=FALSE) => if (IsPushAndUpload())=TRUE) Error Code:=state conflict elseif(IsLRSizeOK(Load Image Size)=FALSE) Error Code:=load image size error Endif Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOA DABLE
5	DOWNLOA DABLE	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	DOWNLOA DABLE

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
6	DOWNLOADABLE	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=TRUE => Segment Length:=0 Segment Number:=1 More Follows:=FALSE StartIntersegReqTimer(AREP, T1) Pull Segment.rsp(+) (AREP, Segment Length, Segment Number, More Follows)	DOWNLOADABLE
7	DOWNLOADABLE	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	DOWNLOADABLE
8	DOWNLOADABLE	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	DOWNLOADABLE
9	DOWNLOADABLE	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP) Terminate Load.rsp(+) (AREP)	DOWNLOADABLE
10	DOWNLOADABLE	IntersegReqTimer Expired => AREP:=GetTimerAREP() DeleteAREP(AREP) Uploadcount:=Uploadcount-1 IntersegReqTimerExpired.req(AREP)	DOWNLOADABLE
11	DOWNLOADABLE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	DOWNLOADABLE
12	DOWNLOADABLE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=FALSE =>	DOWNLOADABLE
13	DOWNLOADABLE	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	DOWNLOADABLE
14	DOWNLOADABLE	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=FALSE =>	DOWNLOADABLE
15	DOWNLOADABLE	SetInUse.ind (FI Identifier, LR Identifier) => SetInUse.rsp(-) (FI Identifier, LR Identifier)	DOWNLOADABLE

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
16	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /(CheckAREP(AREP)=TRUE) && (CheckSegmentNumber(AREP, Segment Number)=TRUE) && (CheckActualSegLength(AREP, Segment Length)=TRUE) && (CheckMoreFollows(AREP; More Follows)=TRUE)&& (More Follows=TRUE) => Segment Number:=GetSegmentNumber(AREP) StoreSegmentNumber(AREP, Segment Number) StartIntersegReqTimer(AREP, T1) GetActualSegLength(AREP, Segment Length) StoreData(Data) Push Segment.rsp(+) (AREP)	DOWNLOADING
17	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /(CheckAREP(AREP)=TRUE) && (CheckSegmentNumber(AREP, Segment Number)=TRUE) && (CheckActualSegLength(AREP, Segment Length)=TRUE) && (CheckMoreFollows(AREP; More Follows)=TRUE)&& (More Follows=FALSE) => Segment Number:=GetSegmentNumber(AREP) StoreSegmentNumber(AREP, Segment Number) StartIntersegReqTimer(AREP, T1) GetActualSegLength(AREP, Segment Length) StoreData(Data) Push Segment.rsp(+) (AREP)	DOWNLOADED SUCCESS
18	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /CheckAREP(AREP)=FALSE => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADING
19	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /(CheckAREP(AREP)=TRUE) && (CheckSegmentNumber(AREP, Segment Number)=FALSE) => Error Code:=invalid parameter DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADABLE
20	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /(CheckAREP(AREP)=TRUE) && (CheckSegmentNumber(AREP, Segment Number)=TRUE) && (CheckActualSegLength(AREP, Segment Length)=FALSE) => Error Code:=length error DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADABLE

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
21	DOWNLOADING	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /(CheckAREP(AREP)=TRUE) && (CheckSegmentNumber(AREP, Segment Number)=TRUE) && (CheckActualSegLength(AREP, Segment Length)=TRUE) && (CheckMoreFollows(AREP; More Follows)=FALSE) => Error Code:=invalid parameter DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADABLE
22	DOWNLOADING	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADING
23	DOWNLOADING	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(CheckAREP(AREP)=TRUE) && (Load Type=Push) => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADABLE
24	DOWNLOADING	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(CheckAREP(AREP)=FALSE) && (Load Type=Push) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADING
25	DOWNLOADING	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=TRUE) => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Pull Segment.rsp(-) (AREP, Error Code)	DOWNLOADABLE
26	DOWNLOADING	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=FALSE) => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	DOWNLOADING

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
27	DOWNLOADING	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=TRUE => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Terminate Load.rsp(-) (AREP, Error Code)	DOWNLOADABLE
28	DOWNLOADING	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	DOWNLOADING
29	DOWNLOADING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0	DOWNLOADABLE
30	DOWNLOADING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=FALSE =>	DOWNLOADING
31	DOWNLOADING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0	DOWNLOADABLE
32	DOWNLOADING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=FALSE =>	DOWNLOADING
33	DOWNLOADING	IntersegReqTimer Expired => AREP:=GetTimerAREP() DeleteAREP(AREP) ClearContent Actual LR Size:=0 IntersegReqTimerExpired.req(AREP)	DOWNLOADABLE
34	DOWNLOADING	SetInUse.ind (FI Identifier, LR Identifier) => SetInUse.rsp(-) (FI Identifier, LR Identifier)	DOWNLOADING
35	DOWNLOAD SUCCESS	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) StopIntersegReqTimer(AREP) Terminate Load.rsp(+) (AREP)	LOADED
36	DOWNLOAD SUCCESS	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	DOWNLOAD SUCCESS

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
37	DOWNLOADED SUCCESS	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADED SUCCESS
38	DOWNLOADED SUCCESS	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(CheckAREP(AREP)=TRUE) && (Load Type=Push) => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADED
39	DOWNLOADED SUCCESS	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(CheckAREP(AREP)=FALSE) && (Load Type=Push) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	DOWNLOADED SUCCESS
40	DOWNLOADED SUCCESS	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=TRUE) => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Pull Segment.rsp(-) (AREP, Error Code)	DOWNLOADED
41	DOWNLOADED SUCCESS	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=FALSE) => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	DOWNLOADED SUCCESS
42	DOWNLOADED SUCCESS	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /CheckAREP(AREP)=TRUE => Error Code:=state conflict DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0 Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADED
43	DOWNLOADED SUCCESS	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /CheckAREP(AREP)=FALSE => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	DOWNLOADED SUCCESS

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
44	DOWNLOAD SUCCESS	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0	DOWNLOADABLE
45	DOWNLOAD SUCCESS	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=FALSE =>	DOWNLOAD SUCCESS
46	DOWNLOAD SUCCESS	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) StopIntersegReqTimer(AREP) ClearContent Actual LR Size:=0	DOWNLOADABLE
47	DOWNLOAD SUCCESS	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=FALSE =>	DOWNLOAD SUCCESS
48	DOWNLOAD SUCCESS	IntersegReqTimer Expired => AREP:=GetTimerAREP() DeleteAREP(AREP) ClearContent Actual LR Size:=0 IntersegReqTimerExpired.req(AREP)	DOWNLOADABLE
49	DOWNLOAD SUCCESS	SetInUse.ind (FI Identifier, LR Identifier) => SetInUse.rsp(-) (FI Identifier, LR Identifier)	DOWNLOAD SUCCESS
50	LOADED	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) && (IsServiceAccepted(Load Type, Load Image Size, Additional Information)=TRUE) => StoreAREP(AREP) Segment Number:=0 StoreSegmentNumber(AREP, Segment Number) Actual Length:=0 StoreActualLength(AREP, Actual Length) T1:=Intersegment Request Timeout Actual LR Size:=Load Image Size Load Region State:=DOWNLOADING StartIntersegReqTimer(AREP, T1) Initiate Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length)	DOWNLOADING

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
51	LOADED	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) && (IsServiceAccepted(Load Type, Additional Information)=TRUE) => T1:=Intersegment Request Timeout Uploadcount:=Uploadcount+1 StoreAREP(AREP) Segment Number:=0 StoreSegmentNumber(AREP, Segment Number) Actual Length:=0 StoreActualLength(AREP, Actual Length) StartIntersegReqTimer(AREP, T1) Initiate Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length)	LOADED
52	LOADED	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) && (IsServiceAccepted(Load Type, Load Image Size, Additional Information)=FALSE) => if (IsPushAndUpload())=TRUE) Error Code:=state conflict Elseif(IsLRSizeOK(Load Image Size)=FALSE) Error Code:=load image size error Endif Initiate Load.rsp(-) (AREP, Error Code)	LOADED
53	LOADED	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	LOADED
54	LOADED	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=TRUE => Segment Number:=GetSegmentNumber(AREP) StoreSegmentNumber(AREP, Segment Number) Segment Length:=GetActualSegLength(AREP, Segment Length) More Follows:=IsLastSegment(AREP) StartIntersegReqTimer(AREP, T1) Data:=GetData() Pull Segment.rsp(+) (AREP, Segment Length, Segment Number, More Follows, Data)	LOADED
55	LOADED	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	LOADED
56	LOADED	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP) Terminate Load.rsp(+) (AREP)	LOADED

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
57	LOADED	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	LOADED
58	LOADED	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	LOADED
59	LOADED	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=FALSE =>	LOADED
60	LOADED	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	LOADED
61	LOADED	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=FALSE =>	LOADED
62	LOADED	IntersegReqTimer Expired => AREP:=GetTimerAREP() DeleteAREP(AREP) Uploadcount:=Uploadcount-1 IntersegReqTimerExpired.req(AREP)	LOADED
63	LOADED	SetInUse.ind (FI Identifier, LR Identifier) /Uploadcount=0 => StoreFIIdentifier(FI Identifier) SetInUse.rsp(+) (FI Identifier, LR Identifier)	IN-USE
64	LOADED	SetInUse.ind (FI Identifier, LR Identifier) /Uploadcount>0 => SetInUse.rsp(-) (FI Identifier, LR Identifier)	LOADED
65	IN-USE	DeleteInUse.ind (FI Identifier, LR Identifier) => DeleteFIIdentifier(FI Identifier) DeleteInUse.rsp(+) (FI Identifier, LR Identifier)	LOADED
66	IN-USE	SetInUse.ind (FI Identifier, LR Identifier) /CheckFIIdentifier(FI Identifier)=FALSE => SetInUse.rsp(-) (FI Identifier, LR Identifier)	IN-USE
67	IN-USE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	IN-USE

Table 294 (continued)

#	Current State	Event /Condition =>Action	Next State
68	IN-USE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	IN-USE
69	IN-USE	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	IN-USE
70	IN-USE	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	IN-USE
71	IN-USE	Terminate Load.ind (AREP, Slot Number, LR Index) => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	IN-USE
72	IN-USE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	IN-USE
73	IN-USE	DP-slave Stopped.ind (AREP) =>	IN-USE

8.2.7.4.4 Load region state table for non erasable memory

Table 295 shows the state table of a Load Region for non erasable memory.

Table 295 – Load Region state table for non erasable memory

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	=> Uploadcount:=0	LOADED
2	LOADED	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) => Error Code:= access denied Initiate Load.rsp(-) (AREP, Error Code)	LOADED
3	LOADED	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) && (IsServiceAccepted(Load Type, Additional Information)=TRUE) => T1:=Intersegment Request Timeout Uploadcount:=Uploadcount+1 StoreAREP(AREP) Segment Number:=0 StoreSegmentNumber(AREP, Segment Number) Actual Length:=0 StoreActualLength(AREP, Actual Length) StartIntersegReqTimer(AREP, T1) Initiate Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length)	LOADED

#	Current State	Event /Condition =>Action	Next State
4	LOADED	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	LOADED
5	LOADED	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=TRUE => Segment Number:=GetSegmentNumber(AREP) StoreSegmentNumber(AREP, Segment Number) Segment Length:=GetActualSegLength(AREP, Segment Length) More Follows:=IsLastSegment(AREP) StartIntersegReqTimer(AREP, T1) Data:=GetData() Pull Segment.rsp(+) (AREP, Segment Length, Segment Number, More Follows, Data)	LOADED
6	LOADED	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	LOADED
7	LOADED	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP) Terminate Load.rsp(+) (AREP)	LOADED
8	LOADED	Terminate Load.ind (AREP, Slot Number, LR Index) /(CheckAREP(AREP)=FALSE => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	LOADED
9	LOADED	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	LOADED
10	LOADED	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP)=FALSE =>	LOADED
11	LOADED	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=TRUE => DeleteAREP(AREP) Uploadcount:=Uploadcount-1 StopIntersegReqTimer(AREP)	LOADED
12	LOADED	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP)=FALSE =>	LOADED

#	Current State	Event /Condition =>Action	Next State
13	LOADED	IntersegReqTimer Expired => AREP:=GetTimerAREP() DeleteAREP(AREP) Uploadcount:=Uploadcount-1 IntersegReqTimerExpired.req(AREP)	LOADED
14	LOADED	SetInUse.ind (FI Identifier, LR Identifier) /Uploadcount=0 => StoreFIIdentifier(FI Identifier) SetInUse.rsp(+) (FI Identifier, LR Identifier)	IN-USE
15	LOADED	SetInUse.ind (FI Identifier, LR Identifier) /Uploadcount>0 => SetInUse.rsp(-) (FI Identifier, LR Identifier)	LOADED
16	IN-USE	DeleteInUse.ind (FI Identifier, LR Identifier) => DeleteFIIdentifier(FI Identifier) DeleteInUse.rsp(+) (FI Identifier, LR Identifier)	LOADED
17	IN-USE	SetInUse.ind (FI Identifier, LR Identifier) /CheckFIIdentifier(FI Identifier)=FALSE => SetInUse.rsp(-) (FI Identifier, LR Identifier)	IN-USE
18	IN-USE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, Additional Information) /(Load Type=Push) => Error Code:= access denied Initiate Load.rsp(-) (AREP, Error Code)	IN-USE
19	IN-USE	Initiate Load.ind (AREP, Slot Number, LR Index, Load Type, Intersegment Request Timeout, Additional Information) /(Load Type=Pull) => Error Code:=state conflict Initiate Load.rsp(-) (AREP, Error Code)	IN-USE
20	IN-USE	Push Segment.ind (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) => Error Code:=state conflict Push Segment.rsp(-) (AREP, Error Code)	IN-USE
21	IN-USE	Pull Segment.ind (AREP, Slot Number, LR Index, Segment Length) => Error Code:=state conflict Pull Segment.rsp(-) (AREP, Error Code)	IN-USE
22	IN-USE	Terminate Load.ind (AREP, Slot Number, LR Index) => Error Code:=state conflict Terminate Load.rsp(-) (AREP, Error Code)	IN-USE
23	IN-USE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	IN-USE
24	IN-USE	DP-slave Stopped.ind (AREP) =>	IN-USE

8.2.8 Function invocation ASE

8.2.8.1 Overview

The Function Invocation ASE defines two objects, the stateless Action object and the state-oriented Function Invocation object.

Stateless function invocations, referred to as actions, run to completion when invoked and cannot be interrupted. In addition, some atomic actions return a value in response to being invoked, while others do not. Those that do may be used to model software *functions*, and those that do not may be used to model software *procedures*.

State-oriented function invocations, on the other hand, may be controlled during their execution. Services are defined to stop, resume, or reset them once they have been invoked. They do not return a value in response to being started. If it is necessary to stop a function invocation once it has been started, then it shall be defined as state-oriented.

State-oriented function invocations represent the network view of a specific invocation of a User function. If the User function can be invoked more than once simultaneously, then separate Function Invocation objects are needed to represent each invocation.

State-oriented function invocations may be used to model software processes or user operations that may be started and controlled.

NOTE An example of a state-oriented function invocation that may be used to model a user operation is the "playback" operation of a video recorder. Once it has been started, the playback operation may be stopped (paused) and later resumed.

To support the concept of software process modelling, the definition of functional invocations includes optional attributes that can be used to relate them to Load Region objects.

The Function Invocation objects shall clearly be separated from the Process Data objects and the Load Region objects. No overlapping is allowed.

8.2.8.2 Function invocation model class specifications

8.2.8.2.1 Function invocation class definition

8.2.8.2.1.1 Function invocation formal model

A Function Invocation object is described by the following template:

DP ASE:		Function Invocation ASE
CLASS:	Function Invocation	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Identifier
3.	(m)	Attribute: Access Rights MS1
4.	(m)	Attribute: Access Rights MS2
4.1.	(m)	Attribute: List of SCL
4.1.1.	(m)	Attribute: SCL
7.	(m)	Attribute: Function Invocation State
8.	(o)	Attribute: Load Region Object Identifier
8.1	(c)	Attribute: Load Region Object In Use
9.	(o)	Attribute: Execution Argument Data Description
10.	(o)	Attribute: Function Invocation Name
11.	(o)	Attribute: Local Detail
SERVICES:		
1.	(m)	OpsService: Start
2.	(o)	OpsService: Stop
3.	(o)	OpsService: Resume
4.	(m)	OpsService: Reset
5.	(m)	OpsService: Get FI State

8.2.8.2.1.2 Attributes

Identifier

This key attribute is composed of API, Slot Number and FI Index to define to which Application Process Identifier, module and to which index within the specified module the Function Invocation object belongs. This Identifier shall not be used by another Action object, Function Invocation object, Load Region object or Process Data object.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

Access Rights MS1

This attribute defines the type of access rights defined for a MS1 AR for the Function Invocation object.

Attribute Type: Unsigned8

The allowed values are shown in Table 296.

Table 296 – Access rights MS1

Value	Meaning
2	Right to Start, Stop, Resume, Reset
3	No Access

Access Rights MS2

This attribute defines the type of access rights defined for a MS2 AR for the Function Invocation object.

Attribute Type: Unsigned8

The allowed values are shown in Table 297.

Table 297 – Access rights MS2

Value	Meaning
2	Right to Start, Stop, Resume, Reset
3	No Access

List of SCL

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for access protection to this Function Invocation object. The rules for accepting or rejecting an access are described in 8.2.1.3.

Attribute Type: Unsigned8

Function Invocation State

This attribute specifies the state of the Function Invocation object.

Attribute Type: Unsigned8

The allowed values are shown in Table 298.

Table 298 – Function Invocation state

Value	Meaning
0	IDLE
1	STARTING
2	RUNNING
3	STOPPING
4	STOPPED
5	RESUMING
6	UNRUNNABLE
7	RESETTING
8	WAIT-SET-IN-USE
9	WAIT-SET-IN-USE-2
10	WAIT-DEL-IN-USE
11	WAIT-DEL-IN-USE-2
12	EXEC-ERR
13	EXEC-TERM

Load Region Object Identifier

This optional attribute is composed of API, Slot Number and LR Index to define to which Application Process Identifier, module and to which index within the specified module the related Load Region object belongs.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

Load Region Object In Use

This attribute indicates whether the related Load Region object is actual used by this Function Invocation object.

Attribute Type: Boolean

The allowed values are shown in Table 299.

Table 299 – Load Region object in use

Value	Meaning
FALSE	Load Region object not in use
TRUE	Load Region object in use

Execution Argument Data Description

This optional attribute contains the Data Description for the Parameter Execution Argument of the Start service. This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Function Invocation Name

This optional attribute defines the name of the Function Invocation object.

Attribute Type: Visible String(32)

Local Detail

This optional attribute contains additional local information about the Function Invocation object.

Attribute Type: Octet String

8.2.8.2.1.3 Invocation of the function invocation object

For the invocation of the Function Invocation object the following rules apply:

- At least one AR shall have access to the Function Invocation object.
- Only one Load Region object shall be related to a Function Invocation object.

- The access rights shall be set accordingly to the allowed service(s).
- The optional services shall be selected according to the execution mode (cyclic, non cyclic, ...) of the function invocation.

8.2.8.2.2 Action class definition

8.2.8.2.2.1 Action formal model

An Action object is described by the following template:

DP ASE:		Function Invocation ASE
CLASS:	Action	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Identifier
3.	(m)	Attribute: Access Rights MS1
4.	(m)	Attribute: Access Rights MS2
4.1.	(m)	Attribute: List of SCL
4.1.1.	(m)	Attribute: SCL
8.	(o)	Attribute: Load Region Object Identifier
8.1	(c)	Attribute: Load Region Object In Use
9.	(o)	Attribute: Execution Argument Data Description
10.	(o)	Attribute: Result Argument Data Description
11.	(o)	Attribute: Action Name
12.	(o)	Attribute: Local Detail
SERVICES:		
1.	(m)	OpsService: Call

8.2.8.2.2.2 Attributes

Identifier

This key attribute is composed of API, Slot Number and FI Index to define to which Application Process Identifier, module and to which index within the specified module the Action object belongs. This Identifier shall not be used by another Action object, Function Invocation object, Load Region object or Process Data object.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

Access Rights MS1

This attribute defines the type of access rights defined for a MS1 AR for the Action object.

Attribute Type: Unsigned8

The allowed values are shown in Table 300.

Table 300 – Access rights MS1

Value	Meaning
2	Right to Call
3	No Access

Access Rights MS2

This attribute defines the type of access rights defined for a MS2 AR for the Action object.

Attribute Type: Unsigned8

The allowed values are shown in Table 301.

Table 301 – Access rights MS2

Value	Meaning
2	Right to Call
3	No Access

List of SCL

This attribute is composed of the following list elements:

SCL

The attribute SCL contains one value for access protection to this Action object. The rules for accepting or rejecting an access are described in 8.2.1.3.

Attribute Type: Unsigned8

Load Region Object Identifier

This optional attribute is composed of API, Slot Number and LR Index to define to which Application Process Identifier, module and to which index within the specified module the related Load Region object belongs.

Attribute Type: Unsigned8, Unsigned8, Unsigned16

Allowed values: 0 to 254, 0 to 254, 0 to 61439 (61440 to 65535 reserved)

Load Region Object In Use

This attribute indicates whether the related Load Region object is actual used by this Action object.

Attribute Type: Boolean

The allowed values are shown in Table 302.

Table 302 – Load Region object in use

Value	Meaning
FALSE	Load Region object not in use
TRUE	Load Region object in use

Execution Argument Data Description

This optional attribute contains the Data Description for the Parameter Execution Argument of the request of the Call service. This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Result Argument Data Description

This optional attribute contains the Data Description for the Parameter Execution Argument of the response of the Call service. This attribute may contain either a Simple Data Description or an Array Data Description or a Record Data Description. For data description the template given in 5.2.1 is used.

Action Name

This optional attribute defines the name of the Action object.

Attribute Type: Visible String(32)

Local Detail

This optional attribute contains additional local information about the Action object.

Attribute Type: Octet String

8.2.8.2.2.3 Invocation of the action object

For the invocation of the Action object the following rules apply:

- At least one AR shall have access to the Action object.
- Only one Load Region object shall be related to an Action object.
- The access rights shall be set accordingly to the allowed service(s).

8.2.8.3 Function invocation service specification**8.2.8.3.1 Start service**

This confirmed service is used to request that a Function Invocation be started. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 303 shows the parameters of the service.

Table 303 – Start

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Execution Argument	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Function Invocation object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Function Invocation to be started.

Execution Argument

This optional parameter contains additional information to perform the start of the Function Invocation. It shall be present if the attribute Execution Argument Data Description is defined for the addressed Function Invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type, resource busy.

8.2.8.3.2 Stop service

This confirmed service is used to stop a Function Invocation retaining its context so that it may be resumed. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 304 shows the parameters of the service.

Table 304 – Stop

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Function Invocation object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Function Invocation to be stopped.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type, feature not supported.

8.2.8.3.3 Resume service

This confirmed service is used to resume the execution of a Function Invocation that has been stopped. The execution is resumed using the context saved when the Function Invocation was stopped. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 305 shows the parameters of the service.

Table 305 – Resume

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Execution Argument	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Function Invocation object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Function Invocation to be resumed.

Execution Argument

This optional parameter contains additional information to resume the execution of the Function Invocation. It shall be present if the attribute Execution Argument Data Description is defined for the addressed Function Invocation object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type, feature not supported, resource busy.

8.2.8.3.4 Reset Service

This confirmed service is used to reset a Function Invocation with its initial context. Its initial context is defined as the context of the Function Invocation set by its initialisation procedures. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 306 shows the parameters of the service.

Table 306 – Reset

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Function Invocation object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Function Invocation to be resetted.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: state conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type.

8.2.8.3.5 Get FI state service

This confirmed service is used to read the actual state of a Function Invocation object. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 307 shows the parameters of the service.

Table 307 – Get FI state

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
FI State			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Function Invocation object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Function Invocation to be stopped.

Result(+)

This selection type parameter indicates that the service request succeeded.

FI State

This parameter contains the actual state of the Function Invocation object.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type.

8.2.8.3.6 Call service

This confirmed service is used to invoke the execution of an Action object. This service shall only be used in conjunction with the MS1 or MS2 AR.

Table 308 shows the parameters of the service.

Table 308 – Call

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Slot Number	M	M(=)		
FI Index	M	M(=)		
Execution Argument	U	U(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Result Argument			U	U(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Slot Number

The parameter Slot Number is used in the destination device for addressing the desired Action object in a specific slot (typically a module).

FI Index

This parameter specifies the index of the Action object to be invoked.

Execution Argument

This optional parameter contains additional information to perform the invocation of the Action object. It shall be present if the attribute Execution Argument Data Description is defined for the addressed Action object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result Argument

This optional parameter contains the result of the invocation of the Action object. It shall be present if the attribute Result Argument Data Description is defined for the addressed Action object.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter indicates the reason for the failure.

Allowed values: type conflict, access denied, invalid index, invalid slot, module failure, invalid parameter, invalid type.

8.2.8.4 Behaviour of the function invocation object

8.2.8.4.1 Function invocation state machine description

8.2.8.4.1.1 General

The behaviour of a Function Invocation object is defined by the Function Invocation state machine. Each Function Invocation object is represented by an own instance of this state machine.

The Function Invocation state machine represents a part of the User functionality and is included to ensure interoperability. It is assumed that other parts of the User functionality perform the following checks:

- validation of existence and addressing of the Function Invocation object
- validation of the access rights
- validation of temporary system restrictions for the use of the Function Invocation object
- validation of the correct class of the object

and generate appropriate Error Codes if necessary.

The Function Invocation object may make use of a Load Region object. Therefore the Function Invocation state machine provides an interface to the Load Region state machines for co-ordination purposes.

The main functionalities of the Function Invocation state machine are:

- checking of the validity of the services versus the state of the Function Invocation object
- to ensure that no parallel services are executed on the Function Invocation object
- to ensure that a Function Invocation object is only used if the related Load Region object is available
- supervision of the Client activity.

8.2.8.4.1.2 Primitive definitions

8.2.8.4.1.2.1 Primitives exchanged between the user and the function invocation state machine

Table 309 and Table 310 show the primitive exchanged between the User and the Function Invocation state machine.

Table 309 – Primitives issued by the user to the Function Invocation state machine

Primitive Name	Source	Associated parameters	Functions
Abort.ind	User	AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail	
Call.ind	User	AREP, Slot Number, FI Index, Execution Argument	
DP-slave Stopped.ind	User	AREP	
Reset.ind	User	AREP, Slot Number, FI Index	
Resume.ind	User	AREP, Slot Number, FI Index, Execution Argument	
Start.ind	User	AREP, Slot Number, FI Index, Execution Argument	
Stop.ind	User	AREP, Slot Number, FI Index	
Get FI State.ind	User	AREP, Slot Number, FI Index	

Table 310 – Primitives issued by the Function Invocation state machine to the user

Primitive Name	Source	Associated parameters	Functions
Reset.rsp(+)	Function Invocation	AREP	
Reset.rsp(-)	Function Invocation	AREP, Error Code, Function Invocation State	
Resume.rsp(+)	Function Invocation	AREP	
Resume.rsp(-)	Function Invocation	AREP, Error Code, Function Invocation State	
Start.rsp(+)	Function Invocation	AREP	
Start.rsp(-)	Function Invocation	AREP, Error Code, Function Invocation State	
Stop.rsp(+)	Function Invocation	AREP	
Stop.rsp(-)	Function Invocation	AREP, Error Code, Function Invocation State	
Get FI State.rsp(+)	Function Invocation	AREP, FI State	

The parameters used with the primitives are described in the Service Specification of the Function Invocation object (see 8.2.8.3).

8.2.8.4.1.2.2 Primitives exchanged between the LR and the function invocation state machine

Table 311 and Table 312 show the primitive exchanged between the Load Region state machine and the Function Invocation state machine.

Table 311 – Primitives issued by the Load Region to the Function Invocation state machine

Primitive Name	Source	Associated parameters	Functions
DeleteInUse.cnf(+)	Load Region	FI Identifier, LR Identifier	The Load Region confirms with this primitive positively a previous issued DeleteInUse.req.
SetInUse.cnf(+)	Load Region	FI Identifier, LR Identifier	The Load Region confirms with this primitive positively a previous issued SetInUse.req.
SetInUse.cnf(-)	Load Region	FI Identifier, LR Identifier	The Load Region confirms with this primitive negatively a previous issued SetInUse.req.

Table 312 – Primitives issued by the Function Invocation to the Load Region state machine

Primitive Name	Source	Associated parameters	Functions
DeleteInUse.req	Function Invocation	FI Identifier, LR Identifier	The Function Invocation requests with this primitive the termination of the use of a Load Region object.
SetInUse.req	Function Invocation	FI Identifier, LR Identifier	The Function Invocation requests with this primitive the use of a Load Region object.

8.2.8.4.1.3 State definitions

Table 313 shows the definitions of the states of the Load Region state machine.

Table 313 – Function Invocation state definitions

State	Description
IDLE	This state indicates that the Function Invocation object is not used and can be started.
STARTING	This transient state indicates that the Function Invocation is executing a start procedure.
RUNNING	This state indicates that the object has completed the start-up procedure and executes its normal function.
STOPPING	This transient state indicates that a previous issued Stop is executed.
STOPPED	This state indicates that the Function Invocation is stopped.
RESUMING	This transient state indicates that a Resume command is executed.
UNRUNNABLE	This state indicates that an error occurred.
RESETTING	This transient state indicates that a Reset Command is executed.
WAIT-SET-IN-USE	This transient state indicates that the Function Invocation is executing a start procedure after breakdown of the related Association Relationship.
WAIT-SET-IN-USE-2	This transient state indicates that the Function Invocation is executing a resume procedure after breakdown of the related Association Relationship.
WAIT-DEL-IN-USE	This transient state indicates that the Function Invocation is executing a stop procedure after breakdown of the related Association Relationship.
WAIT-DEL-IN-USE-2	This transient state indicates that the Function Invocation is executing a reset procedure after breakdown of the related Association Relationship.
EXEC-ERR	This transient state indicates that during the execution of a Function Invocation an error has been detected.
EXEC-TERM	This transient state indicates that the execution of a Function Invocation is terminated at the end of the program in case of non cyclic execution mode.

8.2.8.4.1.4 Local variable definitions

AREP

This local variable contains the AREP which represents the application relationships who currently executes a Start, Stop, Resume or Reset service.

LR Identifier

This local variable contains the Identifier (API, Slot Number, LR Index) of the Load Region object which currently is used by this Function Invocation object.

8.2.8.4.1.5 Local event definitions

Execution Error

This local event is generated if during the execution of a Function Invocation an error has been detected. The execution is terminated and the Function Invocation is set to the state UNRUNNABLE.

Execution Terminated

This local event is generated if the execution of a Function Invocation is terminated at the end of the program in case of non cyclic execution modes. The Function Invocation is set to the state IDLE.

8.2.8.4.1.6 Function definitions

The functions contained in Table 314 are used by both Function Invocation state machines which describe the behaviour of a Function Invocation object.

Table 314 – Function definitions

Function Name	Description
IsServiceAccepted(Execution Argument)	This function checks if the received Execution Argument is compatible to the corresponding attribute of the Function Invocation object. A) If the received Execution Argument is compatible then it returns TRUE. B) Otherwise it returns FALSE.
CheckLRIdentifier (LR Identifier)	This function checks if the received LR Identifier is equal to the corresponding attribute of the Function Invocation object. A) If the received LR Identifier is equal then it returns TRUE. B) Otherwise it returns FALSE.
StoreAREP(AREP)	This function stores the received AREP in the variable AREP.
CheckAREP(AREP)	This function checks if the received AREP is equal to the AREP stored in the variable AREP. A) If the received AREP is equal then it returns TRUE. B) Otherwise it returns FALSE.
GetAREP ()	This function reads the AREP stored in the variable AREP.
GetFISState()	This function reads the actual state from the attribute Function Invocation State of the Function Invocation object.

8.2.8.4.2 Function invocation state diagram

Figure 63 shows the state diagram of the Function Invocation state machine.

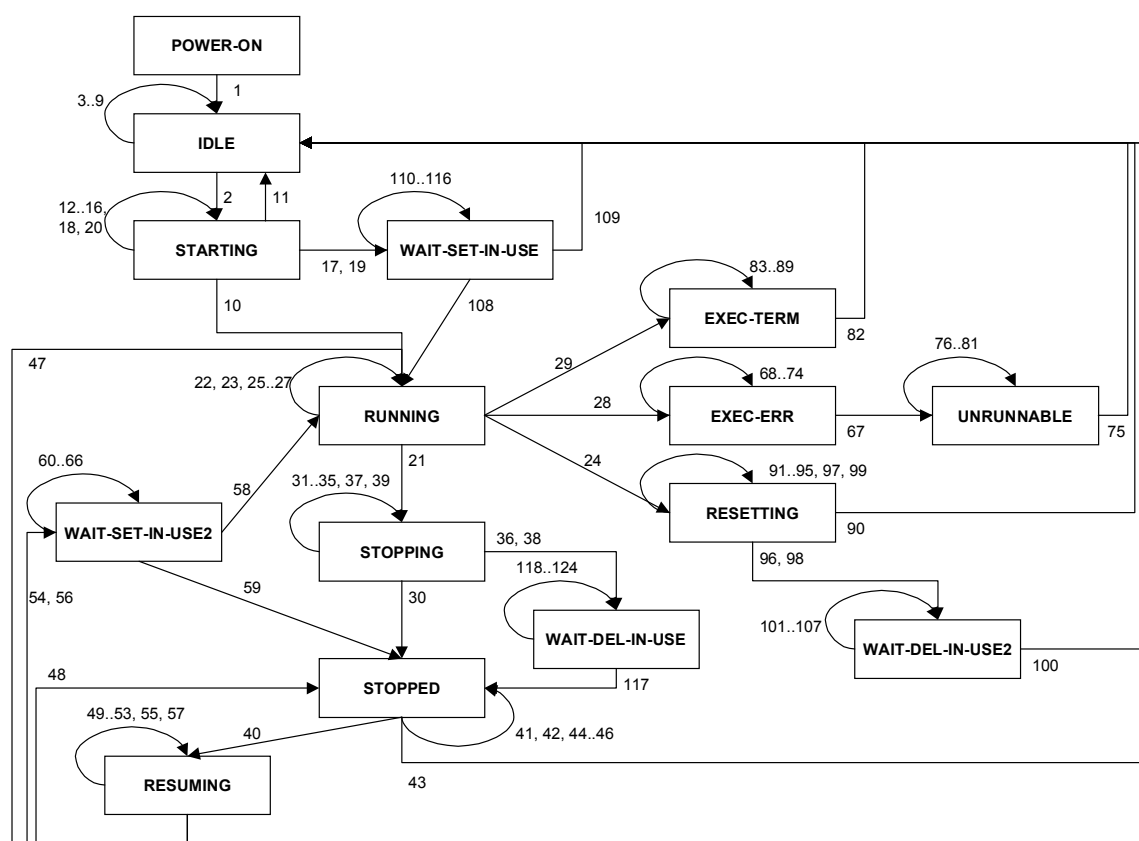


Figure 63 – Function invocation state diagram

8.2.8.4.3 Function invocation state table

Table 315 shows the states of a Function Invocation state machine.

Table 315 – Function Invocation state table

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	=>	IDLE
2	IDLE	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(IsServiceAccepted(Execution Argument) = TRUE) => StoreAREP(AREP) SetInUse.req (FI Identifier, LR Identifier)	STARTING
3	IDLE	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(IsServiceAccepted(Execution Argument) = FALSE) => Error Code := invalid parameter Start.rsp(-) (AREP, Error Code)	IDLE
4	IDLE	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	IDLE

#	Current State	Event /Condition =>Action	Next State
5	IDLE	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	IDLE
6	IDLE	Reset.ind (AREP, Slot Number, FI Index) => Reset.rsp (+) (AREP)	IDLE
7	IDLE	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	IDLE
8	IDLE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	IDLE
9	IDLE	DP-slave Stopped.ind (AREP) =>	IDLE
10	STARTING	SetInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Start.rsp(+) (AREP)	RUNNING
11	STARTING	SetInUse.cnf (-) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Error Code := resource busy Start.rsp(-) (AREP, Error Code)	IDLE
12	STARTING	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	STARTING
13	STARTING	Stop.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	STARTING
14	STARTING	Resume.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	STARTING
15	STARTING	Reset.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	STARTING
16	STARTING	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	STARTING

#	Current State	Event /Condition =>Action	Next State
17	STARTING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = TRUE) =>	WAIT-SET-IN-USE
18	STARTING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = FALSE) =>	STARTING
19	STARTING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = TRUE) =>	WAIT-SET-IN-USE
20	STARTING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = FALSE) =>	STARTING
21	RUNNING	Stop.ind (AREP, Slot Number, FI Index) => StoreAREP(AREP) DeleteInUse.req (FI Identifier, LR Identifier)	STOPPING
22	RUNNING	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	RUNNING
23	RUNNING	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	RUNNING
24	RUNNING	Reset.ind (AREP, Slot Number, FI Index) => DeleteInUse.req (FI Identifier, LR Identifier)	RESETTING
25	RUNNING	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	RUNNING
26	RUNNING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	RUNNING
27	RUNNING	DP-slave Stopped.ind (AREP) =>	RUNNING
28	RUNNING	Execution Error => DeleteInUse.req (FI Identifier, LR Identifier)	EXEC-ERR
29	RUNNING	Execution Terminated => DeleteInUse.req (FI Identifier, LR Identifier)	EXEC-TERM
30	STOPPING	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Stop.rsp(+) (AREP)	STOPPED

#	Current State	Event /Condition =>Action	Next State
31	STOPPING	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	STOPPING
32	STOPPING	Stop.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	STOPPING
33	STOPPING	Resume.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	STOPPING
34	STOPPING	Reset.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	STOPPING
35	STOPPING	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	STOPPING
36	STOPPING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = TRUE) =>	WAIT-DEL-IN-USE
37	STOPPING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = FALSE) =>	STOPPING
38	STOPPING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = TRUE) =>	WAIT-DEL-IN-USE
39	STOPPING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = FALSE) =>	STOPPING
40	STOPPED	Resume.ind (AREP, Slot Number, FI Index) => StoreAREP(AREP) SetInUse.req (FI Identifier, LR Identifier)	RESUMING
41	STOPPED	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	STOPPED
42	STOPPED	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	STOPPED
43	STOPPED	Reset.ind (AREP, Slot Number, FI Index) => Reset.rsp (+) (AREP)	IDLE

#	Current State	Event /Condition =>Action	Next State
44	STOPPED	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	STOPPED
45	STOPPED	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	STOPPED
46	STOPPED	DP-slave Stopped.ind (AREP) =>	STOPPED
47	RESUMING	SetInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Resume.rsp(+) (AREP)	RUNNING
48	RESUMING	SetInUse.cnf (-) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Error Code := resource busy Resume.rsp(-) (AREP, Error Code)	STOPPED
49	RESUMING	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	RESUMING
50	RESUMING	Stop.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	RESUMING
51	RESUMING	Resume.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	RESUMING
52	RESUMING	Reset.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	RESUMING
53	RESUMING	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	RESUMING
54	RESUMING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = TRUE) =>	WAIT-SET-IN-USE-2
55	RESUMING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = FALSE) =>	RESUMING

#	Current State	Event /Condition =>Action	Next State
56	RESUMING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = TRUE) =>	WAIT-SET-IN-USE-2
57	RESUMING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = FALSE) =>	RESUMING
58	WAIT-SET-IN-USE-2	SetInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	RUNNING
59	WAIT-SET-IN-USE-2	SetInUse.cnf (-) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	STOPPED
60	WAIT-SET-IN-USE-2	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE-2
61	WAIT-SET-IN-USE-2	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE-2
62	WAIT-SET-IN-USE-2	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE-2
63	WAIT-SET-IN-USE-2	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	WAIT-SET-IN-USE-2
64	WAIT-SET-IN-USE-2	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	WAIT-SET-IN-USE-2
65	WAIT-SET-IN-USE-2	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	WAIT-SET-IN-USE-2
66	WAIT-SET-IN-USE-2	DP-slave Stopped.ind (AREP) =>	WAIT-SET-IN-USE-2
67	EXEC-ERR	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	UNRUNNABLE
68	EXEC-ERR	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	EXEC-ERR
69	EXEC-ERR	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	EXEC-ERR

#	Current State	Event /Condition =>Action	Next State
71	EXEC-ERR	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	EXEC-ERR
72	EXEC-ERR	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	EXEC-ERR
73	EXEC-ERR	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	EXEC-ERR
74	EXEC-ERR	DP-slave Stopped.ind (AREP) =>	EXEC-ERR
75	UNRUNNABLE	Reset.ind (AREP, Slot Number, FI Index) => Reset.rsp (+) (AREP)	IDLE
76	UNRUNNABLE	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	UNRUNNABLE
77	UNRUNNABLE	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	UNRUNNABLE
78	UNRUNNABLE	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	UNRUNNABLE
79	UNRUNNABLE	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	UNRUNNABLE
80	UNRUNNABLE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	UNRUNNABLE
81	UNRUNNABLE	DP-slave Stopped.ind (AREP) =>	UNRUNNABLE
82	EXEC-TERM	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	IDLE
83	EXEC-TERM	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	EXEC-TERM
84	EXEC-TERM	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	EXEC-TERM
85	EXEC-TERM	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	EXEC-TERM

#	Current State	Event /Condition =>Action	Next State
86	EXEC-TERM	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	EXEC-TERM
87	EXEC-TERM	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	EXEC-TERM
88	EXEC-TERM	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	EXEC-TERM
89	EXEC-TERM	DP-slave Stopped.ind (AREP) =>	EXEC-TERM
90	RESETTING	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) => AREP := GetAREP () Reset.rsp(+) (AREP)	IDLE
91	RESETTING	Start.ind (AREP, Slot Number, FI Index, Execution Argument) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	RESETTING
92	RESETTING	Stop.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	RESETTING
93	RESETTING	Resume.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	RESETTING
94	RESETTING	Reset.ind (AREP, Slot Number, FI Index) /(CheckAREP(AREP) = FALSE) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	RESETTING
95	RESETTING	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	RESETTING
96	RESETTING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = TRUE) =>	WAIT-DEL-IN-USE-2
97	RESETTING	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /(CheckAREP(AREP) = FALSE) =>	RESETTING

#	Current State	Event /Condition =>Action	Next State
98	RESETTING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = TRUE) =>	WAIT-DEL-IN-USE-2
99	RESETTING	DP-slave Stopped.ind (AREP) /(CheckAREP(AREP) = FALSE) =>	RESETTING
100	WAIT-DEL-IN-USE-2	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	IDLE
101	WAIT-DEL-IN-USE-2	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE-2
102	WAIT-DEL-IN-USE-2	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE-2
103	WAIT-DEL-IN-USE-2	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE-2
104	WAIT-DEL-IN-USE-2	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	WAIT-DEL-IN-USE-2
105	WAIT-DEL-IN-USE-2	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	WAIT-DEL-IN-USE-2
106	WAIT-DEL-IN-USE-2	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	WAIT-DEL-IN-USE-2
107	WAIT-DEL-IN-USE-2	DP-slave Stopped.ind (AREP) =>	WAIT-DEL-IN-USE-2
108	WAIT-SET-IN-USE	SetInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	RUNNING
109	WAIT-SET-IN-USE	SetInUse.cnf (-) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	IDLE
110	WAIT-SET-IN-USE	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE
111	WAIT-SET-IN-USE	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE

#	Current State	Event /Condition =>Action	Next State
112	WAIT-SET-IN-USE	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	WAIT-SET-IN-USE
113	WAIT-SET-IN-USE	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	WAIT-SET-IN-USE
114	WAIT-SET-IN-USE	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	WAIT-SET-IN-USE
115	WAIT-SET-IN-USE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	WAIT-SET-IN-USE
116	WAIT-SET-IN-USE	DP-slave Stopped.ind (AREP) =>	WAIT-SET-IN-USE
117	WAIT-DEL-IN-USE	DeleteInUse.cnf (+) (FI Identifier, LR Identifier) /(CheckLRIdentifier (LR Identifier) = TRUE) =>	STOPPED
118	WAIT-DEL-IN-USE	Start.ind (AREP, Slot Number, FI Index, Execution Argument) => Error Code := state conflict Start.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE
119	WAIT-DEL-IN-USE	Stop.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Stop.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE
120	WAIT-DEL-IN-USE	Resume.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Resume.rsp(-) (AREP, Error Code)	WAIT-DEL-IN-USE
121	WAIT-DEL-IN-USE	Reset.ind (AREP, Slot Number, FI Index) => Error Code := state conflict Reset.rsp (-) (AREP, Error Code)	WAIT-DEL-IN-USE
122	WAIT-DEL-IN-USE	Get FI State.ind (AREP, Slot Number, FI Index) => FI State := GetFIState() Get FI State.rsp (+) (AREP, FI State)	WAIT-DEL-IN-USE
123	WAIT-DEL-IN-USE	Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) =>	WAIT-DEL-IN-USE
124	WAIT-DEL-IN-USE	DP-slave Stopped.ind (AREP) =>	WAIT-DEL-IN-USE

8.2.9 Time ASE

8.2.9.1 Overview

The Time ASE specifies the Clock Synchronisation mechanism to synchronise the time used within devices of a network. This time could be used for example to add a time value to an alert information and to sequence messages in a time wise order.

A system consists of one or more Time Master and several Time Receiver. Clock synchronisation is done by the communication system. Adjustment and management of the clock is task of the application.

Time Master tasks are:

- Sending Time Event and Clock Value within Clock Sync Interval
- Resolution of Time Master conflicts
- Report Errors in Clock Synchronisation

Time Receiver tasks are:

- Monitoring and check of the clock via clock sync interval
- Control of clock synchronisation sequence
- Interpretation clock value
- Check status in clock value
- Adjustment of local time (not part of the Clock Synchronisation specification)

The time object specifies two classes:

- Slave Time Class
- Link Time Class

The Slave Time Class comprises the Time Receiver tasks. The Link Time Class comprises Time Master and Time Receiver tasks. But Link Time Class differ from Slave Time Class definition.

The difference is based on a system architecture which provide one or more Time Master in a hierarchical topology and Time Receiver (DP-slaves) which receive a synchronised time value to set a clock within the device, see Figure 64.

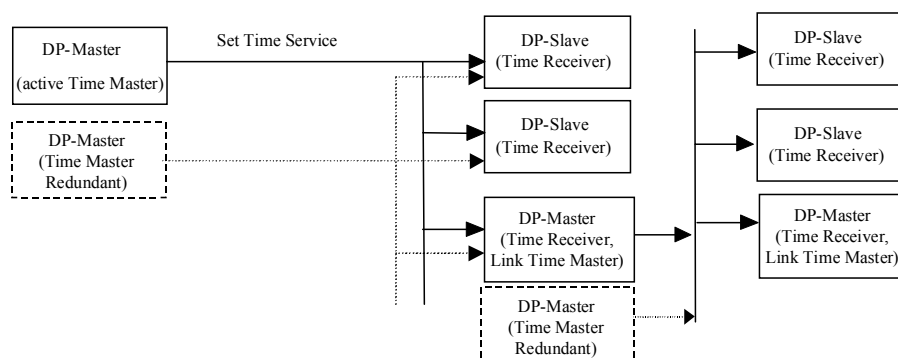


Figure 64 – System architecture

Two services are defined:

- Set Time
- Sync Interval Violation

The Set Time service is requested by the DP-master device and the Sync Interval Violation service through DP-slave devices.

Synchronisation of time between devices on a fieldbus segment and a Time Master application is provided in parallel with other communication functions. The DP-master (Class 1) operates as the active and if required as the redundant Time Master. If a DP-slave supports the Clock Synchronisation mechanism the device attribute DPV1 has to be set to TRUE.

The scheme used is a "backwards time based correction". This results in very few real-time constraints being imposed on a field device. There is no requirement for generating messages at precise instants. Instead, knowledge of when a special Timer event message has been broadcasted is subsequently distributed and used to calculate appropriate clock adjustments.

NOTE The clock synchronisation assumes that there are no delays between the invocation of the Set Time service and the begin of the processing of the related service in the Data Link layer.

The clock synchronisation protocol operates in parallel with the other AL functions. The clock synchronisation interval specifies the rate at which the Time Master initiates the clock synchronisation sequence. The value of this parameter is a system design issue. It shall be the same for all devices on the network.

The value of the clock synchronisation interval is a system design issue. For each station, there is a minimum required clock synchronisation interval given by the desired clock accuracy divided by the clock drift rate. The value for the system should be set to the minimum required value for all stations on the network segment. The value should not be set so large that it causes intolerable start-up delays when devices, such as hand-held terminals, are added to a network.

8.2.9.2 Time class specification

8.2.9.2.1 Slave time class specification

8.2.9.2.1.1 Template

DP ASE:		Time ASE
CLASS:	Slave Time	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	Time Data Description
3.	(o) Attribute:	Local Time Diff
4.	(o) Attribute:	Time Last Rcvd
5.	(o) Attribute:	Last Calc Diff
6.	(m) Attribute:	CS Status
7.	(o) Attribute:	Summertime
SERVICES:		
1.	(m) OpsService:	Set Time
2.	(m) OpsService:	Sync Interval Violation

8.2.9.2.1.2 Attributes

Implicit

The attribute Implicit indicates that the Slave Time object is implicitly addressed by the services.

Time Data Description

This attribute contains a Simple Data Description. Only the Data Type Network Time shall be used. For Data Description the template given in 5.2.1 is used.

Local Time Diff

This optional attribute contains the value that needs to be added to the value of the Slave Time object to obtain the local time.

The default value shall be 0 (no local time used).

Attribute Type: Network Time Difference

Time Last Rcvd

This optional attribute contains the parameter Time Value of the last Set Time service.

Attribute Type: Network Time

Time Master Addr

This optional attribute contains the address of the device from whom the previous Set Time service was received.

Attribute Type: Unsigned8

Allowed Value: 0 to 126

Last Calc Diff

This optional attribute contains the difference between the value of the Slave Time object and the parameter Time Value of the Set Time service plus Delay Factor at the receipt of the last Set Time service.

Attribute Type: Network Time Difference

CS Status

This attribute contains the current status of the synchronisation.

Attribute Type: Boolean

The allowed values are shown in Table 316.

Table 316 – CS status

Value	Meaning
FALSE	Synchronised
TRUE	Not Synchronised Time since last reception of a Set Time service is too long (greater than 3 Clock Sync Intervals).

Summertime

This optional attribute indicates whether the value of the attribute Local Time Diff is based on Summertime or Wintertime.

Attribute Type: Boolean

The allowed values are shown in Table 317.

Table 317 – Summertime

Value	Meaning
FALSE	Wintertime
TRUE	Summertime

8.2.9.2.1.3 Invocation of the slave time object

For the invocation of the Slave Time object the following rules apply:

- Only one Slave Time object shall be invoked in a DP-slave.

8.2.9.2.2 Link time class specification

8.2.9.2.2.1 Template

DP ASE:		Time ASE
CLASS:	Link Time	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Implicit
2.	(m)	Attribute: Time Data Description
3.	(o)	Attribute: Local Time Diff
4.	(o)	Attribute: Time Last Rcvd
5.	(o)	Attribute: Last Calc Diff
6.	(m)	Attribute: Synchronisation Active
7.	(o)	Attribute: Announcement Hour
8.	(o)	Attribute: Summertime
9.	(o)	Attribute: Accuracy
SERVICES:		
1.	(m)	OpsService: Set Time
2.	(m)	OpsService: Sync Interval Violation

8.2.9.2.2.2 Attributes

Implicit

The attribute Implicit indicates that the Link Time object is implicitly addressed by the services.

Time Data Description

This attribute contains a Simple Data Description. Only the Data Type Network Time shall be used. For Data Description the template given in 5.2.1 is used.

Local Time Diff

This optional attribute contains the value that needs to be added to the value of the Link Time object to obtain the local time. It shall be a multiple of 30 minutes, between 0 and 24 hours.

The default value shall be 0 (no local time used).

Attribute Type: Network Time Difference

Time Last Rcvd

This optional attribute contains the parameter Time Value of the last Set Time service.

Attribute Type: Network Time

Last Calc Diff

This optional attribute contains the difference between the value of the Link Time object and the parameter Time Value of the Set Time service plus Delay Factor at the receipt of the last Set Time service.

Attribute Type: Network Time Difference

Synchronisation Active

This attribute indicates whether the value of the Link Time object is synchronised or not. This attribute is set to 0 if either this station has received a Set Time service with the equivalent Parameter set to 0 or if the time since last reception of a Set Time service is too long (greater than 3 Clock Sync Intervals).

Attribute Type: Boolean

The allowed values are shown in Table 318.

Table 318 – Synchronisation active

Value	Meaning
FALSE	Synchronisation Failed
TRUE	Synchronisation Active

Default value: FALSE

Announcement Hour

This optional attribute indicates whether a change from Summertime to Wintertime or vice versa shall be performed with the next change of the hour.

Attribute Type: Boolean

The allowed values are shown in Table 319.

Table 319 – Announcement hour

Value	Meaning
FALSE	No change with the next hour change.
TRUE	Change from Summertime to Wintertime or vice versa with the next hour change.

Summertime

This optional attribute indicates whether the value of the attribute Local Time Diff is based on Summertime or Wintertime.

Attribute Type: Boolean

The allowed values are shown in Table 320.

Table 320 – Summertime

Value	Meaning
FALSE	Wintertime
TRUE	Summertime

Accuracy

This optional attribute indicates the precision of the value of the Link Time object, i.e. which bit positions are relevant. The value will be taken from the Set Time service parameter Accuracy.

Attribute Type: Unsigned8

The allowed values are shown in Table 321.

Table 321 – Accuracy

Value	Meaning
0	< 1 ms
1	< 10 ms
2	< 100 ms
3	< 1 s

Default value: 0

8.2.9.2.2.3 Invocation of the link time object

For the invocation of the Link Time object the following rules apply:

- Only one Link Time object shall be invoked in a Link.

8.2.9.3 Time service specification**8.2.9.3.1 Set time**

This confirmed service shall be used to set the value and some attributes of a Slave Time or Link Time object. This service shall be used only in conjunction with the MS3 AR. This AR provides the facility to address a group of or all present devices on a fieldbus segment (multicast/broadcast) and therefore the corresponding Slave Time or Link Time objects will be set simultaneously. Due to this principle no confirmation of reception can be provided at the requesting side.

Table 322 shows the parameters of the Set Time service.

Table 322 – Set time

Parameter name	Req	Ind	Cnf
Argument	M	M(=)	
AREP	M	M	
Time Value	M	M	
Local Time Diff	M	M(=)	
Summertime	M	M(=)	
Accuracy	M	U(=)	
Synchronisation Active	M	U(=)	
Announcement Hour	M	U(=)	
Result(+)			S
AREP			M
Result(-)			S
AREP			M
Status			M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

Time Value

This Parameter contains the value for the Link Time or Slave Time object.

Local Time Diff

This attribute contains the value for the attribute Local Time Diff of the Link Time or Slave Time object.

Summertime

This attribute contains the value for the attribute Summertime of the Link Time or Slave Time object.

Accuracy

This attribute contains the value for the attribute Accuracy of the Link Time object and shall be present only in the indication if a Link Time object is present.

Synchronisation Active

This attribute contains the value for the attribute Synchronisation Active of the Link Time object and shall be present only in the indication if a Link Time object is present.

Announcement Hour

This attribute contains the value of the attribute Announcement Hour of the Link Time object and shall be present only in the indication if a Link Time object is present.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status:

This parameter provides detailed information about the occurred error.

Allowed values: LS, LR, DS, SV, IV

8.2.9.3.2 Sync interval violation

The Sync Interval Violation service signals to the application of the Time Receiver (DP-slave) that no Set Time service has been received during the last 3 Clock Sync Intervals. This service shall be used only in conjunction with the MS3 AR.

Table 323 shows the parameter of the Sync Interval Violation service.

Table 323 – Sync interval violation

Parameter name	Ind
Argument	M
AREP	M

Argument

The argument shall convey the service specific parameters of the service request.

AREP

This parameter is the local identifier for the desired AR.

8.2.10 AR ASE**8.2.10.1 Overview****8.2.10.1.1 General**

This ASE specifies the structure of the ARs including the necessary DL parameters for the specific CRs. The AR ASE in each device manages an end point of the AR and all related CRs, and maintains their local context.

An AR is characterised by all of the following basic roles of the AREP used in fieldbus DP:

AR cardinality

– one-to-one

In a one-to-one AR an AP communicates with exactly one remote AP.

– one-to-many

In a one-to-many AR the AP of a DP-master (Class 2) communicates with the APs of a group of DP-masters (Class 1). This comprises the simultaneous communication to the defined group of DP-masters (Class 1). Exactly one AP is addressed per device.

– many-to-many

In a many-to-many AR the APs of a DP-master (Class 1) and optional the APs of one or several DP-master (Class 2) and optional the Default-APs of one or several DP-slave communicate with the Default-APs of a group of DP-slaves. This comprises the sequential as well as the simultaneous communication to the defined group of DP-slaves. Exactly one AP is addressed per device.

Device type relations:**Master-slave (MS) communication**

between DP-master (Class 1) and DP-slaves

between DP-master (Class 2) and DP-slaves

Master-master (MM) communication

between DP-master (Class 2) and DP-master (Class 1)

8.2.10.1.2 Communication relationship**8.2.10.1.2.1 General**

An AR is mapped to one or more communication relationships (CRs). A CR carries out the exchange of information based on a set of rules.

8.2.10.1.2.2 Communication relationship endpoint

CRs are defined as a set of co-operating Communication Relationship End Points (CREPs). The local context of an CR endpoint is used to control the conveyance of APDUs on the CR. Figure 65 illustrates the Assignment of Communication Relationship to Application Relationship.

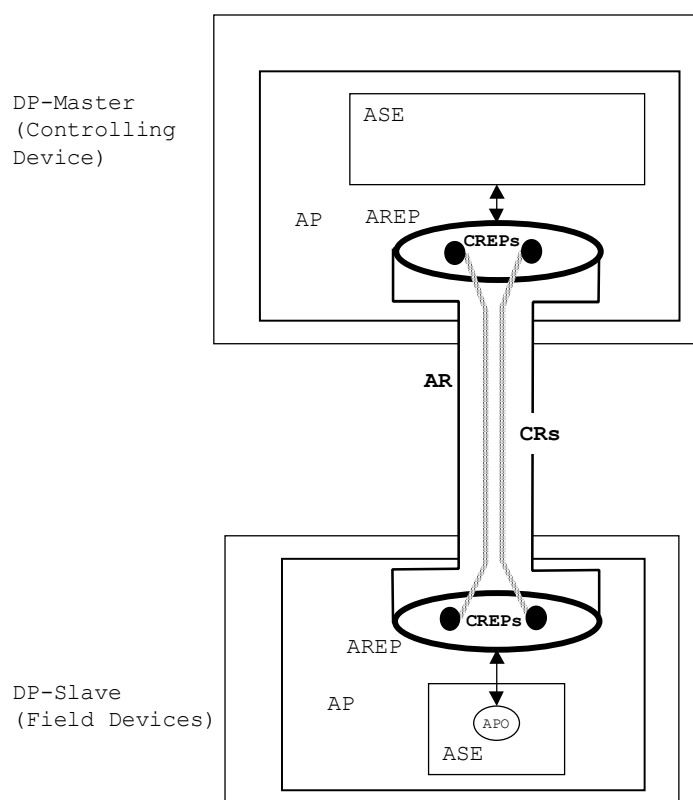


Figure 65 – Assignment of communication relationship to application relationship

8.2.10.1.2.3 Buffer and queues

CREPs may be modelled as a queue or as a buffer. Application data transferred over a queued CREP are delivered in the order received for conveyance. The transfer of application data over a buffered CREP is different. In this case, application data is placed in an application layer buffer for transfer.

When the CREP receives another conveyance request, it replaces the previous contents of the buffer whether or not transmitted. Once application data are written into a buffer for transfer, it is preserved in the buffer until the next application data to be transmitted replaces it. While in the buffer, application data may be read more than once without deleting it from the buffer or changing its contents.

At the receiving end, the operation is similar. The receiving application layer places the received application data into a buffer for access by the different ASEs. When subsequent application data are received, it overwrites the previous application data in the buffer whether or not it was read by the corresponding ASE. Reading the application data from the buffer is not destructive it does not destroy or change the contents of the buffer, allowing the contents to be read from the buffer one or more times.

8.2.10.1.2.4 Communication relationship endpoint roles

An communication relationship is characterised by all of the following basic roles of the CREP used in fieldbus DP:

CR cardinality

– one-to-one:

In a one-to-one communication relationship a CREP communicates with exactly one remote CREP.

– one-to-many:

In a one-to-many relationship one CREP communicates simultaneously with the CREPs of a group of devices. A one-to-many communication relationship does not permit confirmation of the execution of a request.

Connection mode:**connection-oriented**

In a connection-oriented application relationship a logical connection is established between application processes. Three different phases are identified in connection-oriented ARs:

- connection establishment phase
- data transfer phase and
- connection release phase

Exchanging data on a connection shall only be possible after a successful connection establishment. In the connection establishment phase a request to establish the connection is announced at the remote application process (e.g. Initiate service). The request to establish the connection-oriented AR contains the information about the context to be used in the data transfer phase. The remote application informs the requesting application process about the acceptance of the context. If the context is accepted both application processes are in the data transfer phase and are able to communicate with each other according to the agreed context. Connection-oriented AR can be monitored. A connection-oriented AR is dissolved by its release (e.g. Abort service).

connectionless

Connectionless application relationships have neither a connection establishment nor a connection release. They are always in the data transfer phase. It is not possible to monitor the connectionless AR.

Transmission types:**cyclic**

The transmission type "cyclic" means that after an initial request by the application process the application relationship is established and afterwards the data transfer is executed periodically.

acyclic

The transmission type "acyclic" means that a single transfer is executed after each request by the application process.

both

The transmission type "both" means that a cyclic as well as acyclic transfer is supported.

Transmission characteristics**Buffer to Buffer**

The transmission is characterised by a buffer to buffer transfer.

Queue to Queue

The transmission is characterised by a queue to queue transfer.

Buffer to Queue

The transmission is characterised by a buffer to queue transfer.

Queue to Buffer

The transmission is characterised by a queue to buffer transfer.

Service types**Confirmed**

Only confirmed services are allowed.

Unconfirmed

Only unconfirmed services are allowed.

Transmission directions:

Bi-directional

APDUs are transmitted in both direction.

Unidirectional

APDUs are transmitted only in one direction.

Service relations:

Client-Server communication

The services are used according to the Client/Server modell.

Publisher-Subscriber communication

The services are used according to the Publisher/Subscriber modell.

8.2.10.1.2.5 Overview of communication relationships

The DP Application Layer uses the following types of communication relationships:

BBUB (Buffer Buffer Unconfirmed Bidirectional)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connection-oriented
- cyclic
- buffer to buffer
- unconfirmed
- bi-directional
- Client/Server

BBUU-OM (Buffer Buffer Unconfirmed Unidirectional - One to Many)

This type of communication relationship is characterised by the following role attributes:

- one to many
- connection-oriented
- cyclic
- buffer to buffer
- unconfirmed
- unidirectional
- Publisher/Subscriber

BQUU (Buffer Queue Unconfirmed Unidirectional)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connectionless
- acyclic
- buffer to queue
- unconfirmed
- unidirectional
- Client/Server

QQUU-OM (Queue Queue Unconfirmed Unidirectional - One to Many)

This type of communication relationship is characterised by the following role attributes:

- one to many
- connectionless
- acyclic
- queue to queue
- unconfirmed
- unidirectional
- Client/Server

BBUU (Buffer Buffer Unconfirmed Unidirectional)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connectionless
- acyclic
- buffer to buffer
- unconfirmed
- unidirectional
- Client/Server

QQCB-CO (Queue Queue Confirmed Bi-directional - Connection-Oriented)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connection-oriented
- acyclic
- queue to queue
- confirmed
- bidirectional
- Client/Server

QQUU-OO (Queue Queue Unconfirmed Unidirectional - One to One)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connectionless
- acyclic
- queue to queue
- unconfirmed
- unidirectional
- Client/Server

QBCB (Queue Buffer Confirmed Bi-directional)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connectionless
- acyclic
- queue to buffer
- confirmed
- bi-directional
- Client/Server

QQCB-CL (Queue Queue Confirmed Bi-directional - Connectionless)

This type of communication relationship is characterised by the following role attributes:

- one to one
- connectionless
- acyclic
- queue to queue
- confirmed
- bi-directional
- Client/Server

8.2.10.2 Fieldbus ARs

8.2.10.2.1 MS0 application relationship

The MS0 AR reflects the application relationship (see Figure 66) between the application processes of one DP-master (Class 1) and optional the APs of one or several DP-master (Class 2) with the Default AP of all related DP-slaves, additionally the MS0 AR provides the application relationship between the Default-APs of one or several DP-slaves with the Default-APs of a group of DP-slaves for the following purposes:

- cyclic exchange of the I/O data (DP-master (Class 1)) (Client/Server)
- cyclic exchange of the Input Data (DP-slave) (Publisher/Subscriber)
- acyclic data transfer for parameterisation, configuration and diagnosis (DP-master (Class 1))
- acyclic transfer of commands to a set of field devices (DP-master (Class 1))
- cyclic transfer of synchronisation messages to a set of field devices (DP-master (Class 1))
- acyclic read of the I/O data (DP-master (Class 2))
- acyclic read of configuration information (DP-master (Class 2))
- acyclic read of diagnosis information (DP-master (Class 2))
- acyclic write of remanent parameter (DP-master (Class 2))

This type of application relationship is characterised by the following role attributes:

- many to many
- Master-slave communication

The MS0 AR contains the following CR types:

- BBUB (Data Exchange)
- BBUU-OM (DXB)
- BQUU (Prm, Ext Prm and Cfg)
- BBUU (Slave Diag)
- QQUU-OM (Global Control)
- QBCB (Slave Diag, Read Cfg, Read Input, Read Output, Rem Para)

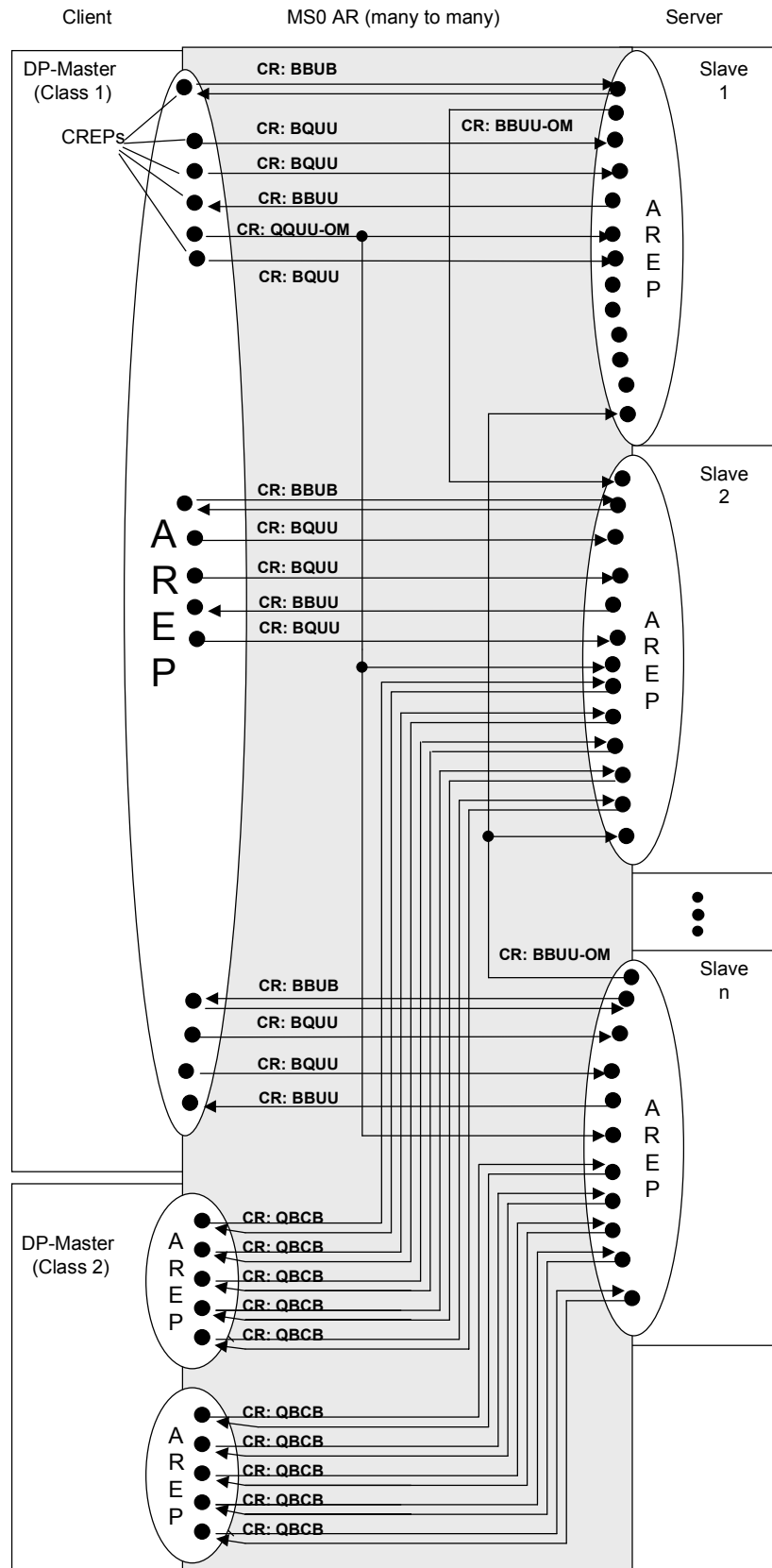


Figure 66 – MS0 application relationship

The DP-Application Layer incorporates buffers to convey the input and output data. These buffers decouple the communication and data processing. For synchronisation purposes two buffers for the Input Data and two buffers for the Outputs are necessary in the DP-Application Layer. If a DP-slave supports no synchronisation capabilities only one buffer is needed in each direction. For the data of the subscribed Input Data only one buffer may be provided by the DP-Application Layer. The detailed model is described in the following subclauses. Figure 67 through Figure 70 illustrate the Input and Output Buffer models of a DP-slave.

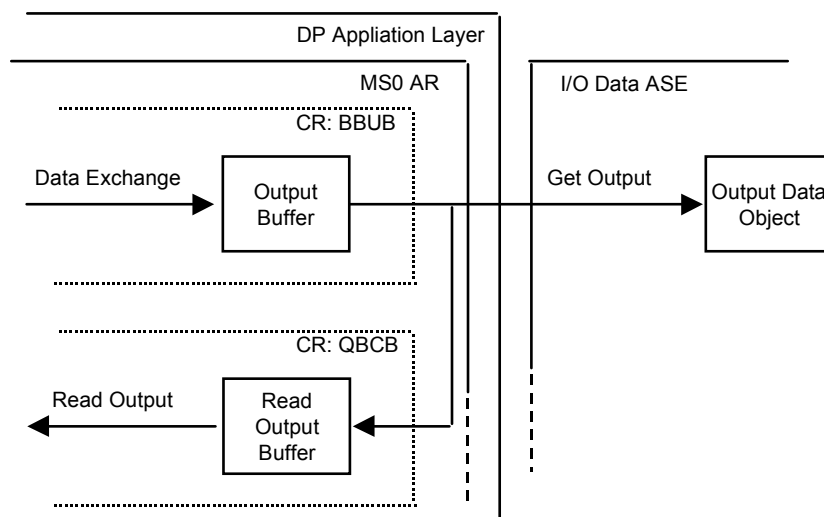


Figure 67 – Output buffer model of a DP-slave without sync functionality

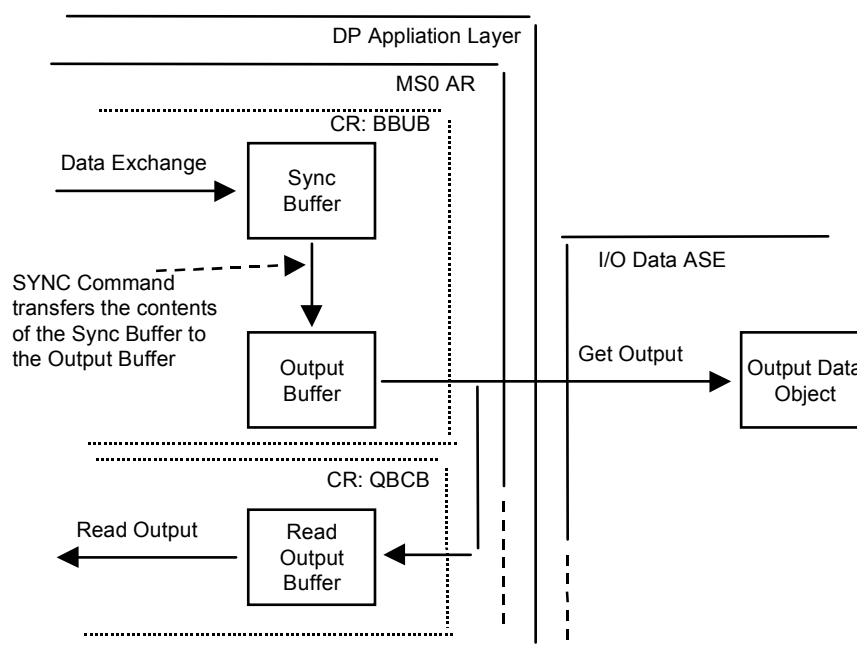


Figure 68 – Output buffer model of a DP-slave with sync functionality

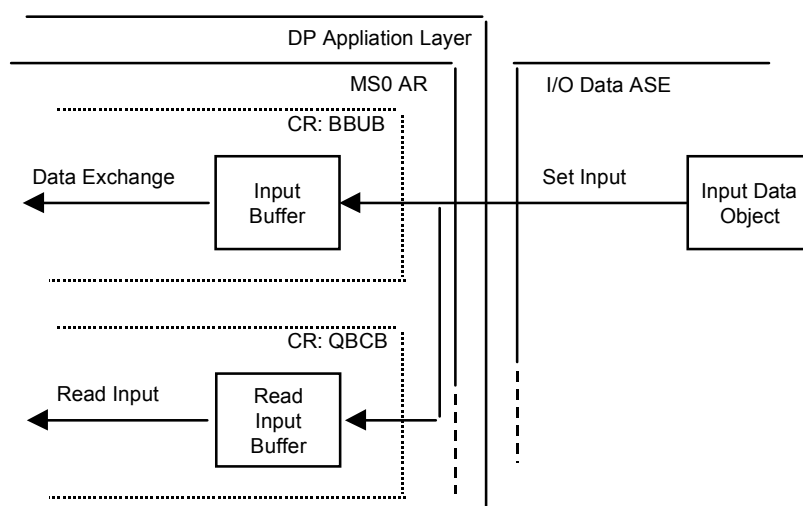


Figure 69 – Input buffer model of a DP-slave without freeze functionality

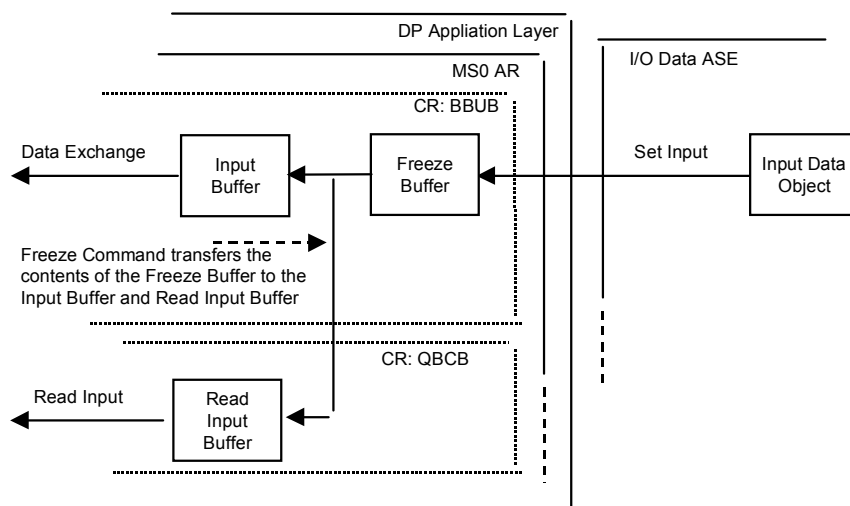


Figure 70 – Input buffer model of a DP-slave with freeze functionality

8.2.10.2.2 MS1 application relationship

The MS1 AR reflects the connection-oriented application relationship (see Figure 71) between the application processes of one DP-master (Class 1) and of one related DP-slave for the following purposes:

- acyclic read and write of variables
- acyclic transfer of alarms
- up- and/or download of LR Data
- invocation of stateless and/or state-oriented functions

This type of application relationship is characterised by the following role attributes:

- one to one
- Master-slave communication

The MS1 AR contains the following CR types:

- QQCB-CO (Process Data, Alarm Ack,)
- QQUU-OO (Alarm)

and optionally the following CR type:

- QQCB-CO (Alarm Ack)

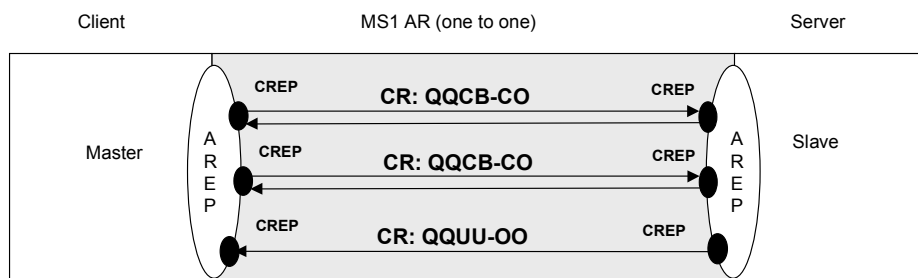


Figure 71 – MS1 application relationship

8.2.10.2.3 MS2 application relationship

8.2.10.2.3.1 General

The MS2 AR reflects the connection-oriented application relationship (see Figure 72) between the application processes of one DP-master (Class 2) and of one related DP-slave for the purpose:

- acyclic read and write of variables
- up- and/or download of LR Data
- invocation of stateless and/or state-oriented functions

This type of application relationship is characterised by the following role attributes:

- one to one
- Master-slave communication

The MS2 AR contains the following CR type:

- QQCB-CO (Process Data)
- QQUU-OO (Resource Manager for Context Management)

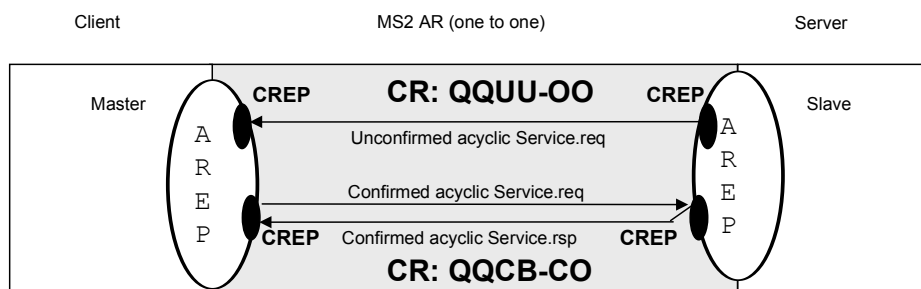


Figure 72 – MS2 application relationship

8.2.10.2.3.2 Communication over multiple interconnected networks

With the Initiate service it is possible to communicate over multiple networks.

The following cases are considered:

- data transfer from one DP-Network to another DP-Network over a link
- data transfer from a None-DP-Network to a DP-Network over a gateway

Within such networks each DP-Device including their applications are addressed with the following parameters:

- Network address
- MAC
- API, SCL

The format of these addresses is defined in the Initiate service description. It is necessary to distinguish between DP-Devices which are either used as a link or a communication endpoint. In the case of the link the Initiate-PDU has to be passed to the addressed device. The Network address 0 is reserved for default addressing purposes and therefore shall not be used as a normal network address.

The Figure 73 shows a cut out of an heterogeneous network. Fieldbus DP covers only the marked area.

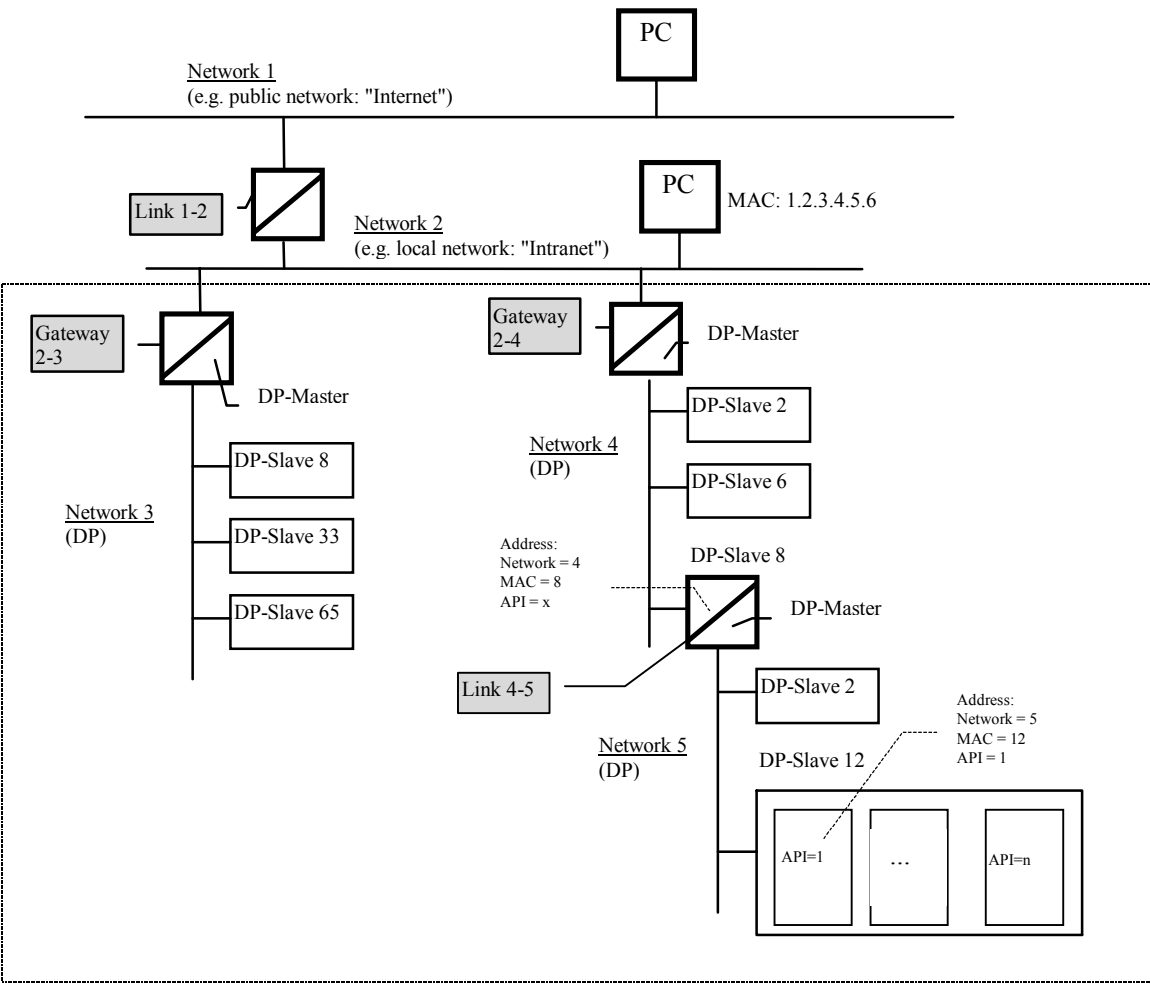


Figure 73 – Example of inter-network communication

The following examples (see Figure 74 through Figure 76) show the exchange of the source- and destination-address parameter of the Initiate.req and the Initiate.res service.

Between the AP 10 of DP-master (Class 2) and AP 1 of the DP-slave a communication relationship will be established. This is done with the Initiate service. After the connection establishment the transfer of data blocks will be realised with the services Read and Write using the parameters Slot Number and Index.

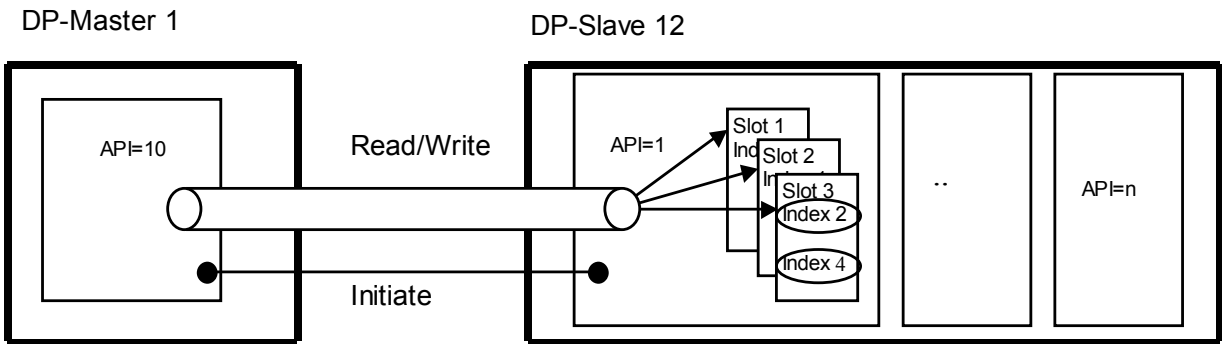


Figure 74 – Example without inter-network addressing

Table 324 shows the parameters of the Initiate service without Inter-Network Addressing.

Table 324 – Parameter of Initiate service without inter-network addressing

Initiate.req (1 -> 12)		Initiate.res (12 -> 1)	
S-Type	0	S-Type	0
S-Len	2	S-Len	2
D-Type	0	D-Type	0
D-Len	2	D-Len	2
S-Addr	API 10, SCL	S-Addr	API 1, SCL
D-Addr	API 1, SCL	D-Addr	API 10, SCL

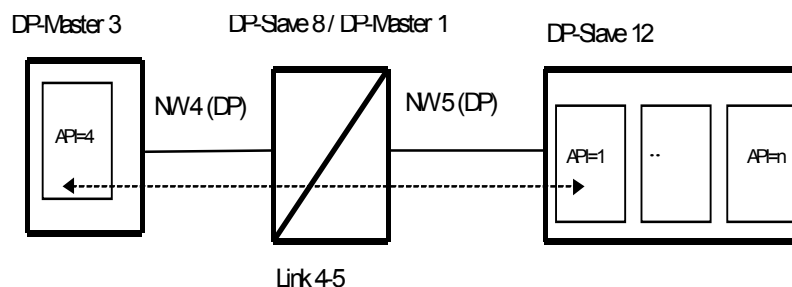


Figure 75 – First example with inter-network addressing

Table 325 shows the parameters of the Initiate service with Inter-Network Addressing.

Table 325 – Parameter of Initiate service with inter-network addressing (first example)

Initiate.req (3 -> 8)		Initiate.res (8 -> 3)	
S-Type	0	S-Type	1
S-Len	2	S-Len	9
D-Type	1	D-Type	0
D-Len	9	D-Len	2
S-Addr	API 4, SCL	S-Addr	API 1, SCL, NW5, 12
D-Addr	API 1, SCL, NW5, 12	D-Addr	API 4, SCL
Initiate.req (1 -> 12)		Initiate.res (12 -> 1)	
S-Type	1	S-Type	0
S-Len	9	S-Len	2
D-Type	0	D-Type	1
D-Len	2	D-Len	9
S-Addr	API 4, SCL, NW4, 3	S-Addr	API 1, SCL
D-Addr	API 1, SCL	D-Addr	API 4, SCL, NW4, 3

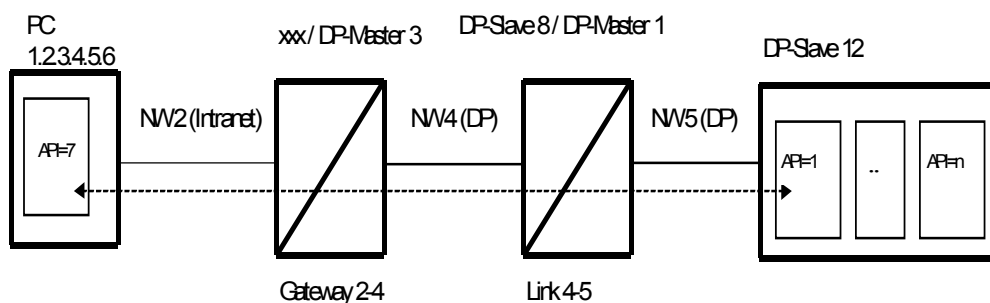


Figure 76 – Second example with inter-network addressing

Table 326 shows the parameters of the Initiate service with Inter-Network Addressing.

Table 326 – Parameter of Initiate service with inter-network addressing (second example)

Initiate-REQ-PDU (3 -> 8)		Initiate-RES-PDU (8 -> 3)	
S-Type	1	S-Type	1
S-Len	14	S-Len	9
D-Type	1	D-Type	1
D-Len	9	D-Len	14
S-Addr	API 7, SCL, NW2, 1.2.3.4.5.6	S-Addr	API 1, SCL, NW5, 12
D-Addr	API 1, SCL, NW5, 12	D-Addr	API 7, SCL, NW2, 1.2.3.4.5.6
Initiate-REQ-PDU (1 -> 12)		Initiate-RES-PDU (12 -> 1)	
S-Type	1	S-Type	0
S-Len	14	S-Len	2
D-Type	0	D-Type	1
D-Len	2	D-Len	14
S-Addr	API 7, SCL, NW2, 1.2.3.4.5.6	S-Addr	API 1, SCL
D-Addr	API 1, SCL	D-Addr	API 7, SCL, NW2, 1.2.3.4.5.6

8.2.10.2.4 MS3 application relationship

The MS3 AR reflects the connectionless relationship (see Figure 77) between the application processes of a DP-master and of a set of related DP-slaves for the purpose:

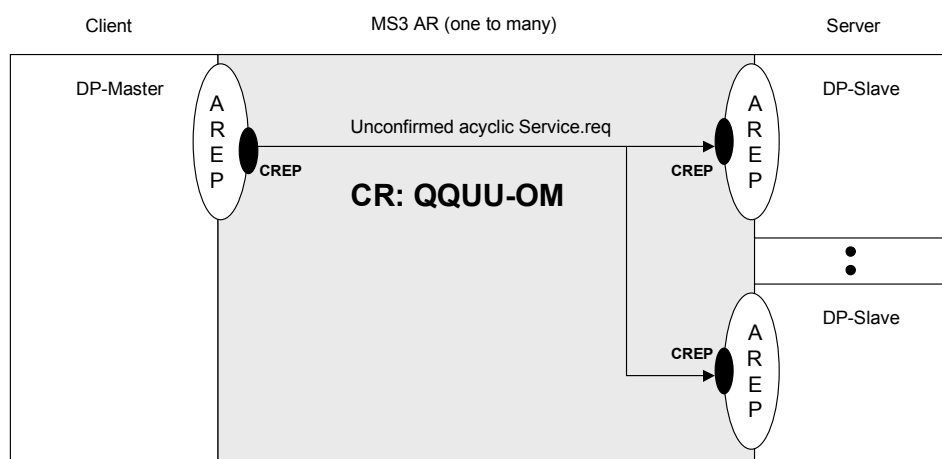
- synchronisation of time

This type of application relationship is characterised by the following role attributes:

- one to many
- Master-slave communication

The MS3 AR contains the following CR type:

- QQUU-OM (Time)

**Figure 77 – MS3 application relationship**

8.2.10.2.5 MM1 application relationship

The MM1 AR reflects the connectionless relationship (see Figure 78) between the application processes of one configuration device (Master Class 2) and of one related controlling device (Master Class 1) for the following purposes:

- upload and download for configuration information
- upload of diagnosis information
- activation of the previously transferred configuration

This type of application relationship is characterised by the following role attributes:

- one to one
- Master-master communication

The MM1 AR contains the following CR type:

- QQCB-CL (MM1(Master Diag/Start Seq/End Seq/Download/Upload/Act Param))

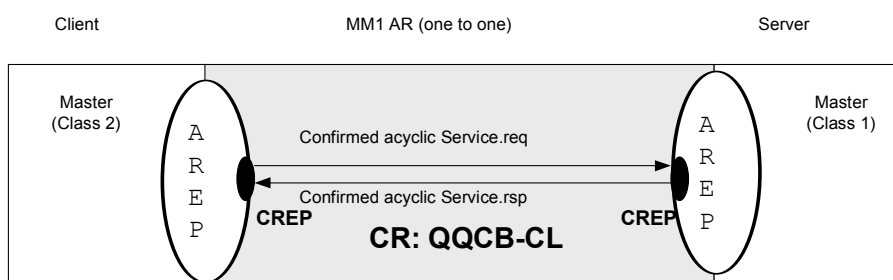


Figure 78 – MM1 application relationship

8.2.10.2.6 MM2 application relationship

The MM2 AR reflects the connectionless relationship (see Figure 79) between the application processes of one configuration device (Master Class 2) and of a set of related controlling devices (Master Class 1) for the purpose:

- activation of the previously transferred configurations

This type of application relationship is characterised by the following role attributes:

- one to many
- Master-master communication

The MM2 AR contains the following CR type:

- QQUU-OM (Act Para Brct)

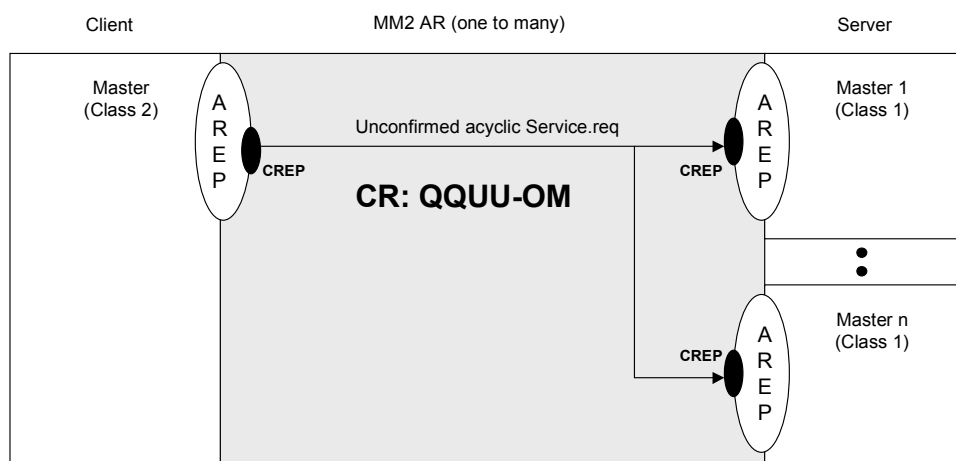


Figure 79 – MM2 application relationship

8.2.10.3 Application relationship list class specification

The ARL Class specification is distinguished between DP-slave, DP-master (Class 1) and DP-master (Class 2).

A static attribute is assigned by a load service and are not changed during operation, whereas a dynamic attribute may set by the load services and may be changed during operation.

8.2.10.3.1 ARL DP-slave class specification

8.2.10.3.1.1 Template

The ARL DP-slave object is described by the following template:

DP ASE:		AR ASE
CLASS:	ARL DP-slave	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(c) Constraint:	at least one entry with AR Type = MS2
2.1.	(m) Attribute:	Min Send Timeout
2.2.	(m) Attribute:	List of Resource Manager Entries
2.2.1.	(m) Attribute:	API
2.2.2.	(m) Attribute:	SCL
2.2.3.	(m) Attribute:	Max Len Data Unit
2.2.4.	(m) Attribute:	Send Timeout
2.2.5.	(m) Attribute:	AREP
2.2.6.	(m) Attribute:	S_SAP_index
2.2.7.	(m) Attribute:	In Use
2.3.	(m) Attribute:	List of S_SAP_indices
2.3.1.	(m) Attribute:	S_SAP_index
3.	(m) Attribute:	List of ARL Entries
3.1.	(m) Attribute:	AREP
3.2.	(m) Attribute:	AR Type
3.3.	(m) Attribute:	List of related CRL Entries
3.3.1.	(m) Attribute:	CREP
3.4.	(c) Constraint:	AR Type = MS0
3.4.1.	(m) Attribute:	Prm Req
3.4.2.	(m) Attribute:	Prm Fault
3.4.3.	(m) Attribute:	Cfg Fault
3.4.4.	(m) Attribute:	Not Supported

- 3.4.5. (m) Attribute: Station Not Ready
- 3.4.6. (m) Attribute: Stat Diag
- 3.4.7. (m) Attribute: Master Add
- 3.4.8. (m) Attribute: Sync Supported
- 3.4.9. (m) Attribute: Sync Mode
- 3.4.10. (m) Attribute: Freeze Supported
- 3.4.11. (m) Attribute: Freeze Mode
- 3.4.12. (m) Attribute: WD Enabled
- 3.4.13. (m) Attribute: WD Fact 1
- 3.4.14. (m) Attribute: WD Fact 2
- 3.4.15. (m) Attribute: Min TSDR
- 3.4.16. (m) Attribute: Ident Number
- 3.4.17. (m) Attribute: Group Identifier
- 3.4.18. (m) Attribute: DPV1 Supported
- 3.4.19. (c) Constraint: DPV1 Supported = TRUE
- 3.4.19.1. (m) Attribute: DPV1 Enabled
- 3.4.19.2. (m) Attribute: Fail Safe Required
- 3.4.19.3. (m) Attribute: Fail Safe
- 3.4.19.4. (m) Attribute: WD Base
- 3.4.20. (m) Attribute: List of Real Cfg Data
- 3.4.20.1. (s) Attribute: Cfg Identifier
- 3.4.20.1.1. (m) Attribute: Type
- 3.4.20.1.2. (m) Attribute: Format
- 3.4.20.1.3. (m) Attribute: Consistency
- 3.4.20.1.4. (m) Attribute: Length
- 3.4.20.2. (s) Attribute: Special Cfg Identifier
- 3.4.20.2.1. (m) Attribute: Type
- 3.4.20.2.2. (c) Constraint: Type = Input or Type = Input-Output
- 3.4.20.2.2.1. (m) Attribute: Input Length
- 3.4.20.2.2.2. (m) Attribute: Input Format
- 3.4.20.2.2.3. (m) Attribute: Input Consistency
- 3.4.20.2.3. (c) Constraint: Type = Output or Type = Input-Output
- 3.4.20.2.3.1. (m) Attribute: Output Length
- 3.4.20.2.3.2. (m) Attribute: Output Format
- 3.4.20.2.3.3. (m) Attribute: Output Consistency
- 3.4.20.2.4. (o) Attribute: List of Data Description
- 3.4.20.2.4.1. (m) Attribute: Data Description
- 3.4.20.2.5. (o) Attribute: Manufacturer Specific Data
- 3.4.21. (o) Attribute: No Add Change
- 3.4.22. (m) Attribute: Publisher Supported
- 3.4.23. (m) Attribute: Isochronous Mode Supported
- 3.4.24. (c) Constraint: Isochronous Mode Supported = TRUE
- 3.4.24.1. (m) Attribute: Isochronous Mode
- 3.4.25. (m) Attribute: Prm Command Supported
- 3.5. (c) Constraint: AR Type = MS1
- 3.5.1. (m) Attribute: Pull Plug Alarm supported
- 3.5.2. (c) Constraint: Pull Plug Alarm supported = TRUE
- 3.5.2.1. (m) Attribute: Pull Plug Alarm enabled
- 3.5.3. (m) Attribute: Process Alarm supported
- 3.5.4. (c) Constraint: Process Alarm supported = TRUE
- 3.5.4.1. (m) Attribute: Process Alarm enabled
- 3.5.5. (m) Attribute: Diagnosis Alarm supported
- 3.5.6. (c) Constraint: Diagnosis Alarm supported = TRUE
- 3.5.6.1. (m) Attribute: Diagnosis Alarm enabled
- 3.5.7. (m) Attribute: Manufacturer Specific Alarm supported
- 3.5.8. (c) Constraint: Manufacturer Specific Alarm supported = TRUE
- 3.5.8.1. (m) Attribute: Manufacturer Specific Alarm enabled
- 3.5.9. (m) Attribute: Status Alarm supported
- 3.5.10. (c) Constraint: Status Alarm supported = TRUE
- 3.5.10.1. (m) Attribute: Status Alarm enabled
- 3.5.11. (m) Attribute: Update Alarm supported
- 3.5.12. (c) Constraint: Update Alarm supported = TRUE
- 3.5.12.1. (m) Attribute: Update Alarm enabled
- 3.5.13. (m) Attribute: Alarm Mode supported
- 3.5.14. (c) Constraint: Alarm Mode supported = TRUE
- 3.5.14.1. (m) Attribute: Alarm Mode enabled

- 3.6. (c) Constraint: AR Type = MS3
 - 3.6.1. (m) Attribute: Clock Sync Interval
 - 3.6.2. (o) Attribute: CS Delay Time
 - 3.7. (o) Attribute: AR Name
- SERVICES:**
- 1. (m) MgtService: Load ARL
 - 2. (o) MgtService: Get ARL
 - 3. (m) MgtService: Set Cfg
 - 4. (o) MgtService: Set ARL Isochronous Mode

8.2.10.3.1.2 Attributes

Implicit

The attribute Implicit indicates that the ARL DP-slave object is implicitly addressed by the service.

Min Send Timeout

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute defines the minimum Send Timeout common for all MS2 ARs supported in this station.

Attribute Type: Unsigned16

Allowed value: 1 to $2^{16}-1$

Time Base: 10ms

List of Resource Manager Entries

This conditional attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute is common for all MS2 ARs supported in this station and composed of the following list elements (for each supported MS2 AR a complete list of elements is needed):

API

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. The attribute API specifies the numeric Identifier for the related Application Process of this MS2 AR.

Attribute Type: Unsigned8

Allowed values: 0 to 255 (0 = Default AP)

SCL

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. The attribute SCL specifies the configured Security Level for the identified Application Process of this MS2 AR.

Attribute Type: Unsigned8

Allowed values: 0 to 255 (0 = no Security Level used)

Max Len Data Unit

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute reflects the upper limit of the parameter Length of the services of the Process Data ASE for this MS2 AR.

Attribute Type: Unsigned8

Allowed values: 0 to 240

Send Timeout

This conditional and dynamic attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute defines the Send Timeout for this MS2 AR. This attribute is set to the maximum of the value of the attribute Min Send Timeout and the field Send Timeout of the previously received Initiate-REQ-PDU for this MS2 AR. This attribute is not influenced by the Load ARL service.

Attribute Type: Unsigned16

Allowed value: 1 to $2^{16}-1$

Time Base: 10ms

AREP

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute specifies the local identifier of this MS2 AR.

Attribute Type: Unsigned8

S_SAP_index

This conditional and dynamic attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute contains the assigned S_SAP_index from the list of S_SAP_indices after reception of an Initiate-REQ-PDU for this MS2 AR. This attribute is not influenced by the Load ARL service.

Attribute Type: Unsigned8

Allowed values: 0 to 48

In Use

This conditional and dynamic attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute specifies the state of this MS2 AR. This attribute is set to TRUE after reception of an Initiate-REQ-PDU and assignment of a S_SAP_index for this MS2 AR. This attribute is set to FALSE after release of this MS2 AR. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: FALSE, TRUE

List of S_SAP_indices

This conditional attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute is common for all MS2 ARs supported in this station and composed of the following list elements:

S_SAP_index

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS2. This attribute specifies an available Responder DLSAP for the MS2 ARs of the DP-slave.

Attribute Type: Unsigned8

Allowed values: 0 to 48

List of ARL Entries

This attribute is composed of the following list elements:

AREP

This static attribute contains the local identifier of this AR.

Attribute Type: Unsigned8

AR Type

This static attribute specifies the type of this AR.

Attribute Type: Unsigned8

The allowed values are shown in Table 327.

Table 327 – AR type

Value	Meaning
0	MS0
1	MS1
2	MS2
3	MS3

List of related CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier for the related CR for this AR.

Attribute Type: Unsigned8

AR Type = MS0

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS0.

Prm Req

This dynamic attribute indicates if the DP-slave has to be reparameterised and reconfigured. This attribute shall be set to TRUE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Prm Fault

This dynamic attribute indicates the result of the check of the parameterisation. This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Cfg Fault

This dynamic attribute indicates the result of the check of the configuration data. This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Not Supported

This dynamic attribute indicates that a functionality requested by the DP-master (Class 1) is not supported on the MS0 AR by the DP-slave. This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Station Not Ready

This dynamic attribute indicates the readiness to enter the data exchange mode with the assigned DP-master (Class 1). This attribute shall be set to TRUE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Stat Diag

This dynamic attribute indicates that the DP-master (Class 1) shall fetch permanently diagnosis from the DP-slave until the application indicates readiness for operation with the Application Ready service. This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine and the Application Ready service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Master Add

This dynamic attribute contains the DL-address of the DP-master (Class 1) which has locked the DP-slave. This attribute shall be set to invalid (= 255) with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Unsigned8

Allowed values: 0 to 126, 255

Sync Supported

This static attribute reflects the ability of a DP-slave to participate in the synchronisation of the Output Data of all or a group of DP-slaves. As the check whether the Sync ability is supported is done during parameterisation phase, errors will be avoided during the data exchange mode. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Table 328 shows the allowed values for this attribute.

Table 328 – Sync supported

Value	Meaning
FALSE	The DP-slave does not support the Sync ability
TRUE	The DP-slave supports the Sync ability

Sync Mode

This dynamic attribute indicates if the Sync mode is activated. This attribute shall always be set to FALSE if the attribute Sync Supported has the value FALSE. If the attribute Sync Supported has the value TRUE the following rules shall apply for the attribute Sync Mode:

- It shall be set to TRUE as soon as the control command Sync is received, which is conveyed by the Global Control service.
- It shall be set to FALSE as soon as the control command Unsync is received which is conveyed by the Global Control service.

This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Freeze Supported

This static attribute reflects the ability of a DP-slave to participate in the synchronisation of the Input Data of all or a group of DP-slaves. As the check whether the Freeze ability is supported is done during parameterisation phase, errors will be avoided during the data exchange mode. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Table 329 shows the allowed values for this attribute.

Table 329 – Freeze supported

Value	Meaning
FALSE	The DP-slave does not support the Freeze ability
TRUE	The DP-slave supports the Freeze ability

Freeze Mode

This dynamic attribute indicates if the Freeze mode is activated. This attribute shall always be set to FALSE if the attribute Freeze Supported has the value FALSE. If the attribute Freeze Supported has the value TRUE the following rules shall apply for the attribute Freeze Mode:

- It shall be set to TRUE as soon as the control command Freeze is received, which is conveyed by the Global Control service.
- It shall be set to FALSE as soon as the control command Unfreeze is received which is conveyed by the Global Control service.

This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine

Attribute Type: Boolean.

WD Enabled

This dynamic attribute indicates if the watchdog control is activated. This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

WD Fact 1, WD Fact 2

These dynamic attributes contain the values for the monitoring of the DP-master (Class 1) at the MS0 AR (watchdog control (T_{WD})). The watchdog control in a DP-slave takes care that, if the DP-master fails, the outputs will be set in the safe state after the expiration of this time. The time is calculated according to equation:

$$T_{WD} = \text{Watchdog Time Base} \times \text{WD Fact 1} \times \text{WD Fact 2}$$

If the attribute DPV1 Supported has the value FALSE the Watchdog Time Base shall be set to 10 ms.

If the attribute DPV1 Supported has the value TRUE the Watchdog Time Base shall be set to the value 1ms if the attribute WD Base has the value TRUE. If the attribute DPV1 Supported has the value TRUE the Watchdog Time Base shall be set to the value 10ms if the attribute WD Base has the value FALSE. These attributes are influenced by the MS0 AR state machine. These attributes are not influenced by the Load ARL service.

Attribute Type: Unsigned8

Allowed values: 1 to 255

Min TSDR

This dynamic attribute contains the value for the Minimum Station Delay Responder time of the DLL. If this Parameter is equal zero, the previous value of the Min TSDR remains unchanged. This attribute is influenced by the MS0 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Unsigned8

Allowed values: 0 to 255

Ident Number

This static attribute contains the Ident Number of the DP-slave. The DP-slave accepts the parameterisation and change of the DL-address only if the transmitted Ident Number in the correspondent service is identical with this attribute. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Unsigned16

Allowed values: 0 to $2^{16}-1$

Group Identifier

This dynamic attribute indicates the group(s) to which this DP-slave has been assigned by the DP-master (Class 1) in the parameterisation. The Group Identifier distinguishes 8 groups. If the Group Identifier is set to No Group, the DP-slave is not assigned to any group. This attribute is influenced by the MS0 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: OctetString(1)

The allowed values are shown in Table 330.

Table 330 – Group identifier

Value	Meaning
Bit 0 = 0	Group 1 has no access
Bit 0 = 1	Group 1 has access
Bit 1 = 0	Group 2 has no access
Bit 1 = 1	Group 2 has access
Bit 2 = 0	Group 3 has no access
Bit 2 = 1	Group 3 has access
Bit 3 = 0	Group 4 has no access
Bit 3 = 1	Group 4 has access
Bit 4 = 0	Group 5 has no access
Bit 4 = 1	Group 5 has access
Bit 5 = 0	Group 6 has no access
Bit 5 = 1	Group 6 has access
Bit 6 = 0	Group 7 has no access
Bit 6 = 1	Group 7 has access
Bit 7 = 0	Group 8 has no access
Bit 7 = 1	Group 8 has access

DPV1 Supported

This static attribute indicates whether the DP-slave supports the extended functionality. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

DPV1 Enabled

This dynamic attribute indicates if the extended functionality is activated. This conditional attribute shall only be present if the attribute DPV1 Supported has the value TRUE. The following rules shall apply for the attribute DPV1 Enabled:

- It shall be set to TRUE if the DP-master (Class 1) requests the DPV1 support in the parameterisation.
- It shall be set to FALSE if the DP-master (Class 1) requests no DPV1 support in the parameterisation.

This attribute shall be set to FALSE with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Table 331 shows the allowed values for this attribute.

Table 331 – DPV1 enabled

Value	Meaning
FALSE	The extended functionality related with the attribute DPV1 Supported is disabled.
TRUE	The extended functionality related with the attribute DPV1 Supported is enabled.

Fail Safe Required

This static attribute indicates if the DP-slave requires the Fail Safe mode for operation. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Fail Safe

This dynamic attribute indicates if the DP-master (Class 1) supports the Fail Safe mode. If the DP-slave requires the Fail Safe mode and the DP-master (Class 1) does not support this mode the MS0 AR state machine shall set the attribute Prm Fault to TRUE in the MS0 AR entry. This attribute is influenced by the MS0 AR state machine. This attribute is not influenced by the Load ARL service.

Table 332 shows the allowed values for this attribute.

Table 332 – Fail safe

Value	Meaning
FALSE	The DP-master (Class 1) does not support the Fail Safe mode.
TRUE	The DP-master (Class 1) supports the Fail Safe mode.

WD Base

This dynamic attribute contains the Time Base value for the watchdog control according to the parameterisation. This attribute is influenced by the MS0 AR state machine. This attribute is not influenced by the Load ARL service.

Table 333 shows the allowed values for this attribute.

Table 333 – WD base

Value	Meaning
FALSE	The Watchdog Time Base is 10ms.
TRUE	The Watchdog Time Base is 1ms.

List of Real Cfg Data

This dynamic attribute is composed of the following list elements:

Cfg Identifier

This dynamic attribute describes the basic configuration format and shall be set with the Load ARL service or with the Set Cfg service. This attribute is composed of the following elements:

Type

This dynamic attribute indicates the type of the related Input/Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Input, Output, Input-Output

Format

This dynamic attribute indicates the format of the related Input/Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word

Consistency

This dynamic attribute indicates the consistency of the related Input/Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word, whole length

Length

This dynamic attribute indicates the length of the related Input/Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: 1 to 32

Special Cfg Identifier

This dynamic attribute describes the special configuration format and is composed of the following elements. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Type

This dynamic attribute indicates the type of the related Input/Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Input, Output, Input-Output

Input Length

This dynamic attribute indicates the length of the related Input Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Input Format

This dynamic attribute indicates the format of the related Input Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word

Input Consistency

This dynamic attribute indicates the consistency of the related Input Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word, whole length

Output Length

This dynamic attribute indicates the length of the related Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Attribute Type: Unsigned8

Allowed values: 1 to 128

Output Format

This dynamic attribute indicates the format of the related Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word

Output Consistency

This dynamic attribute indicates the consistency of the related Output Data Element. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Allowed values: Byte, Word, whole length

List of Data Description

This dynamic attribute is composed of the following list elements. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Data Description

This dynamic attribute contains one of the allowed Data Types of the data description of one element of the Extended Input/Output Data object. The allowed Data Types are given in 5.2.1. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

Manufacturer Specific Data

This dynamic attribute contains the manufacturer specific data for one Special Cfg Identifier. This attribute shall be set accordingly with the Load ARL service. Furthermore the attribute is influenced by the Set Cfg service.

No Add Change

This dynamic attribute indicates whether it is allowed to change the DP-slave address. To change this attribute itself an initial reset of the DP-slave is necessary. After the initial reset the DP-slave takes the default address 126 and the attribute No Add Change will be set to FALSE. This attribute shall be set accordingly with the Load ARL service. This attribute is further influenced by the MS0 AR state machine.

Table 334 shows the allowed values for this attribute.

Table 334 – No Add change

Value	Meaning
FALSE	Change of the address is allowed
TRUE	Change of the address is not allowed

Publisher Supported

This static attribute indicates the capability of a DP-slave to operate as a Publisher. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Isochronous Mode Supported

This static attribute indicates the capability of a DP-slave to operate in the Isochronous Mode. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Isochronous Mode

This dynamic attribute indicates if the DP-slave shall operate in the Isochronous Mode. This attribute shall be set to FALSE with the Load ARL service. After reception of the IsoM Parameter with the Check User Prm or Check Ext User Prm service the User shall set this attribute accordingly with the Set ARL Isochronous Mode service. This shall be done before requesting the services Check User Prm Result or Check Ext User Prm Result.

This conditional attribute shall only be present if the attribute Isochronous Mode Supported is set to TRUE. This attribute is not influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

AR Type = MS1

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS1. This attribute shall furthermore only be present if the attribute DPV1 Supported is set to TRUE in the MS0 AR entry.

Pull Plug Alarm supported

This static attribute indicates the capability of a DP-slave to send a Pull or a Plug Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Pull Plug Alarm enabled

This dynamic attribute indicates if the transmission of the Pull Plug Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Pull Plug Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Process Alarm supported

This static attribute indicates the capability of a DP-slave to send a Process Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Process Alarm enabled

This dynamic attribute indicates if the transmission of the Process Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Process Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Diagnosis Alarm supported

This static attribute indicates the capability of a DP-slave to send a Diagnosis Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Diagnosis Alarm enabled

This dynamic attribute indicates if the transmission of the Diagnosis Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Diagnosis Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Manufacturer Specific Alarm supported

This static attribute indicates the capability of a DP-slave to send a Manufacturer Specific Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Manufacturer Specific Alarm enabled

This dynamic attribute indicates if the transmission of the Manufacturer Specific Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Manufacturer Specific Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Status Alarm supported

This static attribute indicates the capability of a DP-slave to send a Status Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Status Alarm enabled

This dynamic attribute indicates if the transmission of the Status Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Status Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Update Alarm supported

This static attribute indicates the capability of a DP-slave to send a Update Alarm. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Update Alarm enabled

This dynamic attribute indicates if the transmission of the Update Alarm is enabled by the DP-master (Class 1). This conditional attribute shall only be present if the attribute Update Alarm supported is set to TRUE. This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Alarm Mode supported

This static attribute indicates the maximum number of possible active alarms in a DP-slave. This attribute shall be set accordingly with the Load ARL service.

Table 335 shows the allowed values for this attribute.

Table 335 – Alarm mode supported

Value	Meaning
0	1 alarm of each type
1	2 alarms in total
2	4 alarms in total
3	8 alarms in total
4	12 alarms in total
5	16 alarms in total
6	24 alarms in total
7	32 alarms in total

Alarm Mode enabled

This dynamic attribute indicates the maximum number of possible active alarms negotiated with the DP-master (Class 1) on this MS1 AR. The negotiation shall be done according to the following rules:

- If the attribute Alarm Mode is set to the value 0 and the desired Alarm Mode of the DP-master (Class 1) contained in the parameterisation is set to 0 then this attribute shall be set to 0.
- If the attribute Alarm Mode is set to the value 0 and the desired Alarm Mode of the DP-master (Class 1) contained in the parameterisation is set to a value different from 0 then this attribute shall not be set.
- If the attribute Alarm Mode is set to a value different from 0 and the desired Alarm Mode of the DP-master (Class 1) contained in the parameterisation is set to 0 then this attribute shall be set to 0.
- If the attribute Alarm Mode is set to a value different from 0 and the desired Alarm Mode of the DP-master (Class 1) contained in the parameterisation is set to a value different from 0 then this attribute shall be set to the lesser value of both.

This attribute is influenced by the MS1 AR state machine. This attribute is not influenced by the Load ARL service.

Attribute Type: Unsigned8

Allowed values : see Table 335

AR Type = MS3

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS3. This attribute shall furthermore only be present if the attribute DPV1 Supported is set to TRUE in the MS0 AR entry.

Clock Sync Interval

This static attribute contains the value for the clock synchronisation interval.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 10ms

The default value shall be 1s.

CS Delay Time

This static and optional attribute contains the transmission delay of the network (line delay, repeater delay, ...). It should be included for the calculation of the parameter Time Value of the Set Time service.

Attribute Type: Network Time Difference

The default value shall be 0.

AR Name

This static attribute contains the name of the related ARL Entry. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Visible String (32)

8.2.10.3.1.3 Invocation of ARL DP-slave object

For the invocation of the ARL DP-slave object the following rules apply:

- Only one ARL DP-slave object shall be invoked in a DP-slave.
- Only one MS0 Entry shall be contained in the List of ARL Entries
- Only one MS1 Entry shall be contained in the List of ARL Entries
- Only one MS3 Entry shall be contained in the List of ARL Entries
- No MM1 or MM2 Entry shall be contained in the List of ARL Entries

8.2.10.3.2 ARL DP-master (class 1) class specification

8.2.10.3.2.1 Template

The ARL DP-master (Class 1) object is described by the following template:

DP ASE:		AR ASE
CLASS:	ARL DP-master (Class 1)	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(c) Constraint:	at least one entry with AR Type = MS1
2.1.	(m) Attribute:	Alarm Max
3.	(m) Attribute:	List of ARL Entries
3.1.	(m) Attribute:	AREP
3.2.	(m) Attribute:	AR Type
3.3.	(m) Attribute:	List of related CRL Entries
3.3.1.	(m) Attribute:	CREP
3.4.	(c) Constraint:	AR Type = MS0
3.4.1.	(m) Attribute:	Error Action Flag
3.4.2.	(m) Attribute:	Min Slave Interval
3.4.3.	(m) Attribute:	Data Control Time
3.4.4.	(m) Attribute:	Max User Global Control
3.4.5.	(m) Attribute:	Max Diag Data Len
3.4.5.	(m) Attribute:	Isochronous Mode supp
3.4.6.	(c) Constraint:	Isochronous Mode supp = TRUE
3.4.6.1.	(m) Attribute:	Isochronous Mode
3.4.6.2.	(m) Attribute:	IsoM Freeze
3.4.6.3.	(m) Attribute:	IsoM Sync
3.4.6.4.	(m) Attribute:	TCT
3.4.6.5.	(m) Attribute:	maxTSH
3.5.	(c) Constraint:	AR Type = MS1
3.5.2.	(m) Attribute:	Extra Alarm SAP
3.5.3.	(m) Attribute:	Max Channel Data Length
3.5.4.	(m) Attribute:	Alarm Mode

- 3.5.5. (m) Attribute: MS1 Timeout
 - 3.6. (c) Constraint: AR Type = MM1
 - 3.6.1. (m) Attribute: Poll Timeout
 - 3.6.2. (m) Attribute: Ident Number
 - 3.7. (c) Constraint: AR Type = MS3
 - 3.7.1. (m) Attribute: Time Device Type
 - 3.7.2. (c) Constraint: Time Device Type = Time Receiver
 - 3.7.2.1. (m) Attribute: Clock Sync Interval
 - 3.7.2.2. (o) Attribute: CS Delay Time
 - 3.8. (o) Attribute: AR Name
- SERVICES:**
- 1. (m) MgtService: Load ARL
 - 2. (o) MgtService: Get ARL
 - 3. (o) MgtService: ARL Slave Update DP-master CI1

8.2.10.3.2.2 Attributes

Implicit

The attribute Implicit indicates that the ARL DP-slave object is implicitly addressed by the service.

Alarm Max

This conditional and static attribute shall only be present if the List of ARL Entries contains at least one entry with the attribute AR Type set to MS1. It indicates the maximum number of alarms per DP-slave, which can be handled by the DP-master (Class 1). It describes a Master capability and is common for all MS1 ARs. The Master shall be able to handle a minimum number of 7 alarms, because there are existing 7 different alarm types. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned8

Allowed values: 7 to 32

List of ARL Entries

This attribute is composed of the following list elements:

AREP

This static attribute contains the local identifier of this AR.

AR Type

This static attribute specifies the type of this AR.

Allowed values: MS0, MS1, MS3, MM1, MM2

List of related CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier for the related CR for this AR.

AR Type = MS0

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS0.

Error Action Flag

This static attribute indicates how the reaction of the DP-master (Class 1) in case of failed data transfers in the operation mode OPERATE shall be. The reaction shall be performed according to the following rules:

- If the data transfer to at least one selected (dependent on the attribute Ignore ACIr in the List of CRL Entries) DP-slave was erroneous and the Error Action Flag is set to TRUE, the operation mode will change from OPERATE to CLEAR.
- If the Error Action Flag is set to FALSE, the DP-master (Class 1) will remain in the operation mode OPERATE despite erroneous data transfers.

Furthermore this attribute indicates how the transition from the operation mode CLEAR to the operation mode OPERATE shall be performed:

- If the Error Action Flag set to TRUE the transition shall only be performed if the data transfer to the selected DP-slaves has been successfully executed.
- If the Error Action Flag set to FALSE the transition shall be performed independently of a successfully executed data transfer to all activated DP-slaves.

This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Boolean

Allowed values: TRUE, FALSE

Min Slave Interval

This static attribute indicates the smallest allowed period of time between two consecutive DP-slave poll cycles. This ensures that the sequence of cyclic service requests from the DP-master (Class 1) can be handled by the DP-slave. This period of time will be complied by the DP-master (Class 1) at the beginning of each cyclic data transfer. This time interval is not guaranteed for the User Global Control services and for acyclic MS1 AR requests. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 100µs

Data Control Time

This static attribute indicates the period of time in which a Global Control shall be sent indicating the operation mode of the DP-master (Class 1) and at least one successful interaction shall be executed with every activated DP-slave. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 10ms

Max User Global Control

This static attribute indicates the maximum number of Global Control requests, which may be submitted by the User before the Global Control confirmations have been received. This attribute describes the ability of the DP-master (Class 1). A practicable value for Max User Global Control is 16. For each of the 8 Slave groups one Sync and one Freeze Command may be started at the same time. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned8

Allowed values: 1 to 255

Max Diag Data Len

This static attribute indicates the maximum length of Diag Data of all assigned DP-slaves. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned8

Allowed values: 6 to 244

Isochronous Mode supp

This static attribute indicates the capability of the DP-master (Class 1) concerning the Isochronous Mode. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Boolean

The allowed values are shown Table 336.

Table 336 – Isochronous mode supp

Value	Meaning
FALSE	Isochronous Mode not supported by the DP-master (Class 1)
TRUE	Isochronous Mode supported by the DP-master (Class 1)

Isochronous Mode

This attribute indicates if the DP-master (Class 1) shall operate in the Isochronous Mode. Two different models concerning the interface between the DP Application Layer and the application of the DP-master (Class 1) itself are provided by fieldbus DP for the DP-master (Class 1), see 8.2.2.4.3. This attribute shall be set accordingly with the Load ARL service.

The allowed values are shown in Table 337.

Table 337 – Isochronous mode

Value	Meaning
Not Synchronised	the DP-master (Class 1) shall not operate in the Isochronous Mode
Buffered Synchronised	the DP-master (Class 1) shall operate in the Buffer Synchronised Isochronous Mode
Enhanced Synchronised	the DP-master (Class 1) shall operate in the Enhanced Synchronised Isochronous Mode

IsoM Freeze

This attribute indicates if the synchronisation message (SYNCH) shall be sent with the Control Command Freeze set to TRUE. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Boolean

Allowed values: TRUE, FALSE

IsoM Sync

This attribute indicates if the synchronisation message (SYNCH) shall be sent with the Control Command Sync set to TRUE. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Boolean

Allowed values: TRUE, FALSE

TCT

This attribute specifies the time (in units of t_{BIT}) that is necessary to handle a complete Isochronous DP cycle and corresponds to the time T_{DP} . This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned32

Allowed values: 1 to $2^{24}-1$

maxTSH

This attribute specifies the maximum allowed shift (in units of t_{BIT}) of the time T_{DP} measured by the DP-master (Class 1). The application of the DP-master (Class 1) will be informed if the measured Isochronous DP cycle time exceeds this limit by the service SYNCH Delayed indicating the difference. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned8

Allowed values: 1 to 2^8-1

AR Type = MS1

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS1.

Extra Alarm SAP

This static attribute selects the CR for the Alarm Ack service. If the attribute is set to FALSE the DP-master (Class 1) acknowledges alarms via the CR for the read write services. If the attribute is set to TRUE the DP-master (Class 1) acknowledges alarms via a different CR. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Boolean

Allowed values: TRUE, FALSE

Max Channel Data Length

This static attribute indicates the maximum length of the MS1 APDUs (Length parameter and 4 Byte Header) of the corresponding DP-slave. This attribute shall be set accordingly with the Load ARL service.

Attribute Type : Unsigned8

Allowed values: 4 to 244

Alarm Mode

This static attribute indicates the maximum number of possible active alarms of the corresponding DP-slave. This attribute shall be set accordingly with the Load ARL service.

Table 338 shows the allowed values for this attribute.

Table 338 – Alarm mode

Value	Meaning
0	1 alarm of each type
1	2 alarms in total
2	4 alarms in total
3	8 alarms in total
4	12 alarms in total
5	16 alarms in total
6	24 alarms in total
7	32 alarms in total

MS1 Timeout

This attribute contains the value for the Timeout of the acyclic communication on the MS1 AR and specifies the maximum period of time it may take a DP-slave to provide a response. The time contained in this attribute is the maximum time of the assigned DP-slaves.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 10ms

AR Type = MM1

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MM1.

Poll Timeout

This static attribute indicates the maximum period of time in which the requester shall fetch the response. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 1ms

Ident Number

This static attribute contains the Ident Number of the DP-master (Class 1). This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

AR Type = MS3

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MS3.

Time Device Type

This static attribute indicates the Device Type concerning the Time ASE. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Unsigned8

The allowed values are shown in Table 339.

Table 339 – Time device type

Value	Meaning
0	Time Receiver
1	Time Master

Clock Sync Interval

This static and conditional attribute contains the value for the clock synchronisation interval. It shall only be present if the attribute Time Device Type has the value Time Receiver.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 10ms

The default value shall be 1s.

CS Delay Time

This static, conditional and optional attribute contains the transmission delay of the network (line delay, repeater delay, ...). It should be included for the calculation of the parameter Time Value of the Set Time service. It shall only be present if the attribute Time Device Type has the value Time Receiver.

Attribute Type: Network Time Difference

The default value shall be 0.

AR Name

This static attribute contains the name of the related ARL Entry. This attribute shall be set accordingly with the Load ARL service.

8.2.10.3.2.3 Invocation of ARL DP-master (class 1) object

For the invocation of the ARL DP-master (Class 1) object the following rules apply:

- Only one ARL DP-master (Class 1) object shall be invoked in a DP-master (Class 1).
- Only one MS0 Entry shall be contained in the List of ARL Entries.
- Only one MS3 Entry shall be contained in the List of ARL Entries.
- Only one MM1 Entry shall be contained in the List of ARL Entries.
- Only one MM2 Entry shall be contained in the List of ARL Entries.
- No MS2 Entry shall be contained in the List of ARL Entries.

8.2.10.3.3 ARL DP-master (class 2) class specification**8.2.10.3.3.1 Template**

The ARL DP-master (Class 2) object is described by the following template:

DP ASE:	AR ASE
CLASS:	ARL DP-master (Class 2)
CLASS ID:	not used
PARENT CLASS:	TOP

ATTRIBUTES:

1. (m) Key Attribute: Implicit
2. (m) Attribute: List of ARL Entries
- 2.1. (m) Attribute: AREP
- 2.2. (m) Attribute: AR Type
- 2.3. (m) Attribute: List of related CRL Entries
- 2.3.1. (m) Attribute: CREP
- 2.4. (c) Constraint: AR Type = MM1
- 2.4.1 (m) Attribute: Poll Timeout
- 2.5 (o) Attribute: AR Name

SERVICES:

1. (m) MgtService: Load ARL
2. (o) MgtService: Get ARL

8.2.10.3.3.2 Attributes

Implicit

The attribute Implicit indicates that the ARL DP-master (Class 2) object is implicitly addressed by the service.

List of ARL Entries

This attribute is composed of the following list elements:

AREP

This static attribute contains the local identifier of this AR.

AR Type

This static attribute specifies the type of this AR.

Allowed values: MS0, MS2, MM1, MM2

List of related CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier for the related CR for this AR.

AR Type = MM1

This constraint reflects the attributes for this AR if the value of the attribute AR Type is set to MM1.

Poll Timeout

This static attribute indicates the maximum period of time in which the responder shall provide the response. This attribute shall be set accordingly with the Load ARL service.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

Time Base: 1ms

AR Name

This static attribute contains the name of the related ARL Entry. This attribute shall be set accordingly with the Load ARL service.

8.2.10.3.3.3 Invocation of ARL DP-master (class 2) object

For the invocation of the ARL DP-master (Class 2) object the following rules apply:

- Only one ARL DP-master (Class 2) object shall be invoked in a DP-master (Class 2).
- Only one MM1 Entry shall be contained in the List of ARL Entries.
- Only one MM2 Entry shall be contained in the List of ARL Entries.
- No MS1, MS3 Entry shall be contained in the List of ARL Entries.

8.2.10.4 Communication relationship list class specification

8.2.10.4.1 CRL DP-slave class specification

8.2.10.4.1.1 Template

The CRL DP-slave object is described by the following template:

DP ASE:		AR ASE
CLASS:	CRL DP-slave	
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m) Key Attribute:	Implicit
2.	(m) Attribute:	List of CRL Entries
2.1.	(m) Attribute:	CREP
2.2.	(m) Attribute:	CR Type
2.3.	(m) Attribute:	Related AREP
2.4.	(m) Attribute:	S_SAP_index
2.5.	(m) Attribute:	D_addr
2.6.	(m) Attribute:	Service_activate
2.7.	(m) Attribute:	Role_in_Service
2.8.	(c) Constraint:	Service_activate = SRD or MSRD
2.8.1	(m) Attribute:	Indication_mode
2.9.	(m) Attribute:	Max_DLSDU_length_req_low
2.10.	(m) Attribute:	Max_DLSDU_length_req_high
2.11.	(m) Attribute:	Max_DLSDU_length_ind_low
2.12.	(m) Attribute:	Max_DLSDU_length_ind_high
2.13.	(c) Constraint:	CR Type = BBUU-OM & Role_in_Service = Initiator or Both
2.13.1.	(m) Attribute:	Publisher Address
2.13.2.	(m) Attribute:	Input Data Length Publisher
2.13.3.	(m) Attribute:	Source Offset Publisher
2.13.4.	(m) Attribute:	Length
2.14.	(o) Attribute:	CR Name
SERVICES:		
1.	(m) MgtService:	Load CRL
2.	(o) MgtService:	Get CRL
3.	(o) MgtService:	Load CRL DXB Link Entries

8.2.10.4.1.2 Attributes

Implicit

The attribute Implicit indicates that the CRL DP-slave object is implicitly addressed by the service.

List of CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier of this CR. This attribute shall be set accordingly with the Load CRL service.

CR Type

This static attribute indicates the type of this CR. This attribute is dependent of the AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service. For each AR Type the correspondent CRL entries shall be set.

Table 340 shows the allowed values for this attribute.

Related AREP

This static attribute contains the local identifier for the related AR for this CR. This attribute shall be set accordingly with the Load CRL service.

S_SAP_index

This static attribute contains the identifier for the related Service Access Point of the local Data Link Layer. The value for this attribute has to be set correspondent to the attributes CR Type of this CR and to the attribute AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service. Table 340 shows the allowed values for this attribute.

Table 340 – S_SAP_index

AR Type	CR Type	S_SAP_index	Remark
MS0	BBUB	NIL	Mandatory for the MS0 AR
MS0	BBUU-OM	NIL	Optional for the MS0 AR
MS0	BQUU	61, 62	Mandatory for the MS0 AR
MS0	BQUU	53	Optional for the MS0 AR
MS0	BBUU	60	Mandatory for the MS0 AR
MS0	QQUU-OM	58	Mandatory for the MS0 AR
MS0	QBCB	55, 56, 57, 59, 60	Mandatory for the MS0 AR
MS1	QQCB-CO	50	Optional for the MS1 AR
MS1	QQCB-CO	51	Mandatory for the MS1 AR
MS1	QQUU-CO	60	Mandatory for the MS1 AR
MS2	QQCB-CO	0 to 48	Mandatory for each MS2 AR
MS2	QQUU-CO	49	Mandatory for the MS2 AR
MS3	QQUU-OM	CS	Mandatory for the MS3 AR

D_addr

This dynamic attribute contains the DL-address of the remote DP-master or Publisher. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index of this CR and to the attribute AR Type of the related AR. Table 341 shows the allowed values for this attribute used with the Load CRL service. Furthermore this attribute is influenced by the AR state machine dependent of the attribute CR Type of this CR and of the attribute AR Type of the related AR. Table 341 shows the allowed values for this attribute for the modification of the AR state machine.

Table 341 – D_addr

AR Type	CR Type	S_SAP_index	D_addr as loaded with the Load CRL service	D_addr modified by the AR state machine
MS0	BBUB	NIL	None	0 to 125
MS0	BBUU-OM	NIL	None	All
MS0	BQUU	61, 62, 53	All	-
MS0	BBUU	60	All	-
MS0	QQUU-OM	58	None	0 to 125
MS0	QBCB	55, 59, 60	All	-
MS0	QBCB	56, 57	None	All
MS1	QQCB-CO	50, 51	None	0 to 125
MS1	QQUU-CO	60	All	-
MS2	QQCB-CO	0 to 48	None	0 to 125
MS2	QQUU-CO	49	All	-
MS3	QQUU-OM	CS	All	-

Where

None means that the local Service Access Point shall not be activated

All means that the local Service Access Point shall be activated with no access protection

0 to 125 means that the local Service Access Point shall be activated with the corresponding access protection

- means that the AR state machines shall not modify the attribute D_addr

Service_activate

This static attribute indicates the DL services which shall be activated for the S_SAP_index of this CRL entry.

The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 342 shows the allowed values for this attribute.

Table 342 – Service_activate

AR Type	CR Type	S_SAP_index	Service_activate
MS0	BBUB	NIL	(M)SRD
MS0	BBUU-OM	NIL	MSRD
MS0	BQUU	61, 62, 53	SRD
MS0	BBUU	60	SRD
MS0	QQUU-OM	58	SDN
MS0	QBCB	55, 56, 57, 59, 60	SRD
MS1	QQCB-CO	50	SRD
MS1	QQCB-CO	51	SRD
MS1	QQUU-CO	60	SRD
MS2	QQCB-CO	0 to 48	SRD
MS2	QQUU-CO	49	SRD
MS3	QQUU-OM	CS	CS Time Event CS Clock Value

Role_in_service

This dynamic attribute indicates the role which shall be activated for the S_SAP_index of this CRL entry. The following values are specified:

- Initiator: The station initiates the respective service exclusively.
- Responder: The station response to the service exclusively.
- Subscriber: The station receives the reply DLPDU of this service exclusively.
- Both: The station initiates the service and response to the service.
- Responder/Subscriber: The station responds to the service and receives the reply DLPDU of this service

The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 343 shows the allowed values for this attribute.

Table 343 – Role_in_service

AR Type	CR Type	S_SAP_index	Role_in_service as loaded with the Load CRL service	Role_in_service modified by the AR state machine
MS0	BBUB	NIL	None	Responder
MS0	BBUU-OM	NIL	None	Responder, Subscriber, Responder/Subscriber
MS0	BQUU	61, 62, 53	Responder	-
MS0	BBUU	60	Responder	-
MS0	QQUU-OM	58	None	Responder
MS0	QBCB	55, 59, 60	Responder	-
MS0	QBCB	56, 57	None	Responder
MS1	QQCB-CO	50, 51	None	Responder
MS1	QQUU-CO	60	Responder	-
MS2	QQCB-CO	0 to 48	None	Responder
MS2	QQUU-CO	49	Responder	-
MS3	QQUU-OM	CS	Responder	-
Where				
None		means that the local Service Access Point shall not be activated		
-		means that the AR state machines shall not modify the attribute Role_in_service		

Indication_mode

This static attribute shall only be present if the attribute Service_activate of this CRL entry has the value SRD or MSRD. It indicates the Indication_mode which the DLL shall use to convey the indications of the (M)SRD service. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 344 shows the allowed values for this attribute.

Table 344 – Indication_mode

AR Type	CR Type	S_SAP_index	Role_in_service	Indication_mode
MS0	BBUB	NIL	Responder	All
MS0	BBUU-OM	NIL	Subscriber	-
MS0	BBUU-OM	NIL	Responder, Responder/Subscriber	All
MS0	BQUU	61, 62, 53	Responder	All
MS0	BBUU	60	Responder	All
MS0	QBCB	55, 56, 57, 59	Responder	Data
MS0	QBCB	60	Responder	All
MS1	QQCB-CO	50, 51	Responder	Data
MS1	QQUU-CO	60	Responder	All
MS2	QQCB-CO	0 to 48	Responder	Data
MS2	QQUU-CO	49	Respondr	Data
where				
- means that the attribute Indication_mode is not present for this Role_in_service				

Max_DLSDU_length_req_low

This static attribute indicates the maximum length for the DLSDU with low priority as requester. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 345 shows the allowed values for this attribute.

Table 345 – Max_DLSDU_length_req_low

AR Type	CR Type	S_SAP_index	Role_in_service	Max_DLSDU_length_req_low
MS0	BBUB	NIL	Responder	0 to 244
MS0	BBUU-OM	NIL	Subscriber	0
MS0	BBUU-OM	NIL	Responder, Responder/Subscriber	0 to 244
MS0	BQUU	61, 62, 53	Responder	0
MS0	BBUU	60	Responder	6 to 244
MS0	QBCB	55	Responder	0
MS0	QBCB	56, 57, 59	Responder	0 to 244
MS0	QQUU-OM	58	Responder	0
MS1	QQCB-CO	50	Responder	4
MS1	QQCB-CO	51	Responder	4 to 244
MS1	QQUU-CO	60	Responder	6 to 244
MS2	QQCB-CO	0 to 48	Responder	48 to 244
MS2	QQUU-CO	49	Responder	4
MS3	QQUU-OM	CS	Responder	0

Max_DLSDU_length_req_high

This static attribute indicates the maximum length for the DLSDU with high priority as requester. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 346 shows the allowed values for this attribute.

Table 346 – Max_DLSDU_length_req_high

AR Type	CR Type	S_SAP_index	Role_in_service	Max_DLSDU_length_req_high
MS0	BBUB	NIL	Responder	0 to 244
MS0	BBUU-OM	NIL	Subscriber	0
MS0	BBUU-OM	NIL	Responder, Responder/Subscriber	0 to 244
MS0	BQUU	61, 62, 53	Responder	0
MS0	BBUU	60	Responder	0
MS0	QBCB	55, 56, 57, 59	Responder	0
MS0	QQUU-OM	58	Responder	0
MS1	QQCB-CO	50, 51	Responder	0
MS1	QQUU-CO	60	Responder	0
MS2	QQCB-CO	0 to 48	Responder	0
MS2	QQUU-CO	49	Responder	0
MS3	QQUU-CO	CS	Responder	0

Max_DLSDU_length_ind_low

This static attribute indicates the maximum length for the DLSDU with low priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index and Role_in_service of this CR and to the attribute AR Type of the related AR.

Table 347 shows the allowed values for this attribute.

Table 347 – Max_DLSDU_length_ind_low

AR Type	CR Type	S_SAP_index	Role_in_service	Max_DLSDU_length_ind_low
MS0	BBUB	NIL	Responder	0
MS0	BBUU-OM	NIL	Subscriber, Responder/Subscriber	244
MS0	BBUU-OM	NIL	Responder	0
MS0	BQUU	61, 62, 53	Responder	0
MS0	BBUU	60	Responder	0
MS0	QBCB	55	Responder	0
MS0	QBCB	56, 57, 59	Responder	0
MS0	QQUU-OM	58	Responder	0
MS1	QQCB-CO	50	Responder	4
MS1	QQCB-CO	51	Responder	4 to 244
MS1	QQUU-CO	60	Responder	0
MS2	QQCB-CO	0 to 48	Responder	4 to 244
MS2	QQUU-CO	49	Responder	48 to 64
MS3	QQUU-OM	CS	Responder	18

Max_DLSDU_length_ind_high

This static attribute indicates the maximum length for the DLSDU with high priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and S_SAP_index and Role_in_service of this CR and to the attribute AR Type of the related AR.

Table 348 shows the allowed values for this attribute.

Table 348 – Max_DLSDU_length_ind_high

AR Type	CR Type	S_SAP_index	Role_in_service	Max_DLSDU_length_ind_high
MS0	BBUB	NIL	Responder	0 to 244
MS0	BBUU-OM	NIL	Subscriber, Responder/Subscriber	244
MS0	BBUU-OM	NIL	Responder	0
MS0	BQUU	61	Responder	7 to 244
MS0	BQUU	62	Responder	1 to 244
MS0	BQUU	53	Responder	5 to 244
MS0	BBUU	60	Responder	0
MS0	QBCB	55	Responder	4 to 244
MS0	QBCB	56, 57, 59	Responder	0
MS0	QQUU-OM	58	Responder	2
MS1	QQCB-CO	50, 51	Responder	0
MS1	QQUU-CO	60	Responder	0
MS2	QQCB-CO	0 to 48	Responder	0
MS2	QQUU-CO	49	Responder	0
MS3	QQUU-OM	CS	Responder	0

CR Type = BBUU-OM & Role_in_Service = Initiator or Both

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BBUU-OM and the attribute Role_in_Service is set to Subscriber or Responder/Subscriber.

Publisher Address

This dynamic attribute indicates the DL-address of a DP-slave (Publisher) the Subscriber shall subscribe from. The following rules shall apply for this attribute:

A value between 0 and 125 represents a valid Publisher address and therefore an active DXB-Link. The value 127 indicates that this DXB-Link Entry is no longer active.

This attribute shall be set to the value 127 with the Load CRL service. After reception of the DXB-Linktable with the Check User Prm or Check Ext User Prm service, the User shall set this attribute accordingly with the Load CRL DXB Link Entries service.

The MS0 AR state machine set shall set this attribute to the value 127 if the DP-slave acting as Subscriber leaves the data exchange mode.

Attribute Type: Unsigned8

Allowed values: 0 to 125, 127

Input Data Length Publisher

This static attribute indicates the number of octets of the Simple or Extended Input Data object of the related Publisher. This attribute shall only be evaluated by the MS0 AR state machine if the value of the attribute Publisher Address is not equal to 127.

This attribute shall be set with the Load CRL service. After reception of the DXB-Linktable with the Check User Prm or Check Ext User Prm service, the User shall set this attribute accordingly with the Load CRL DXB Link Entries service.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Source Offset Publisher

This static attribute indicates the beginning (offset) of the subscription within the value of the Simple or Extended Input Data object of the related Publisher given with the attribute Publisher Address. This attribute shall only be evaluated by the MS0 AR state machine if the value of the attribute Publisher Address is not equal to 127.

This attribute shall be set with the Load CRL service. After reception of the DXB-Linktable with the Check User Prm or Check Ext User Prm service, the User shall set this attribute accordingly with the Load CRL DXB Link Entries service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Length

This static attribute indicates the number of octets the Subscriber shall subscribe from the related Publisher starting at the offset given in the attribute Source Offset Publisher. This attribute shall only be evaluated by the MS0 AR state machine if the value of the attribute Publisher Address is not equal to 127.

This attribute shall be set with the Load CRL service. After reception of the DXB-Linktable with the Check User Prm or Check Ext User Prm service, the User shall set this attribute accordingly with the Load CRL DXB Link Entries service.

Attribute Type: Unsigned8

Allowed values: 1 to 244

CR Name

This static attribute contains the name of the related CRL Entry. This attribute shall be set accordingly with the Load CRL service.

8.2.10.4.1.3 Invocation of CRL DP-slave object

For the invocation of the CRL DP-slave object the following rule has to be considered:

- Only one CRL DP-slave object shall be invoked in a DP-slave.

8.2.10.4.2 CRL DP-master (class 1) class specification**8.2.10.4.2.1 Template**

The CRL DP-master (Class 1) object is described by the following template:

DP ASE:		AR ASE
CLASS:	CRL DP-master	(Class 1)
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Implicit
2.	(m)	Attribute: List of CRL Entries
2.1.	(m)	Attribute: CREP
2.2.	(m)	Attribute: CR Type
2.3.	(m)	Attribute: Related AREP
2.4.	(m)	Attribute: S_SAP_index
2.5.	(m)	Attribute: D_SAP_index
2.6.	(m)	Attribute: D_addr
2.7.	(m)	Attribute: Service_activate
2.8.	(m)	Attribute: Role_in_service
2.9.	(c)	Constraint: Service_activate = SRD or MSRD
2.9.1.	(m)	Attribute: Indication_mode
2.10.	(m)	Attribute: Max_DLSDU_length_req_low
2.11.	(m)	Attribute: Max_DLSDU_length_req_high
2.12.	(m)	Attribute: Max_DLSDU_length_ind_low
2.13.	(m)	Attribute: Max_DLSDU_length_ind_high
2.14.	(c)	Constraint: CR Type = BQUU & D_SAP_index = 61
2.14.1.	(m)	Attribute: Active
2.14.2.	(m)	Attribute: New Prm
2.14.3.	(m)	Attribute: Prm Data
2.14.3.1.	(m)	Attribute: Lock
2.14.3.2.	(m)	Attribute: Sync
2.14.3.3.	(m)	Attribute: Freeze
2.14.3.4.	(m)	Attribute: WD On
2.14.3.5.	(m)	Attribute: WD Fact 1
2.14.3.6.	(m)	Attribute: WD Fact 2
2.14.3.7.	(m)	Attribute: Min Tsdr
2.14.3.8.	(m)	Attribute: Ident Number
2.14.3.9.	(m)	Attribute: Group Ident
2.14.3.10.	(o)	Attribute: DPV1 Enable
2.14.3.11.	(o)	Attribute: Fail Safe
2.14.3.12.	(o)	Attribute: Enable Publisher

- 2.14.3.13. (o) Attribute: WD Base
- 2.14.3.14. (o) Attribute: Enable Pull Plug Alarm
- 2.14.3.15. (o) Attribute: Enable Process Alarm
- 2.14.3.16. (o) Attribute: Enable Diagnosis Alarm
- 2.14.3.17. (o) Attribute: Enable Manufacturer Specific Alarm
- 2.14.3.18. (o) Attribute: Enable Status Alarm
- 2.14.3.19. (o) Attribute: Enable Update Alarm
- 2.14.3.20. (o) Attribute: Check Cfg Mode
- 2.14.3.21. (o) Attribute: Alarm Mode
- 2.14.3.22. (m) Attribute: Prm Structure
- 2.14.3.23. (c) Constraint: Prm Structure = FALSE
- 2.14.3.23.1 (m) Attribute: User Prm Data
- 2.14.3.23.1.1. (o) Attribute: Device Prm Data
- 2.14.3.23.1.2. (o) Attribute: Module Prm Data
- 2.14.3.24. (c) Constraint: Prm Structure = TRUE
- 2.14.3.24.1. (m) Attribute: List of Structured Prm Data
- 2.14.3.24.1.1. (o) Attribute: Structured User Prm Data
- 2.14.3.24.1.1.1. (o) Attribute: Device Related User Parameter
- 2.14.3.24.1.1.1.1. (m) Attribute: Device Prm Data
- 2.14.3.24.1.1.2. (o) Attribute: Module Related User Parameter
- 2.14.3.24.1.1.2.1. (m) Attribute: Slot Number
- 2.14.3.24.1.1.2.2. (m) Attribute: Module Prm Data
- 2.14.3.24.1.2. (o) Attribute: List of DXB Link Entries
- 2.14.3.24.1.2.1. (m) Attribute: Publisher Address
- 2.14.3.24.1.2.2. (m) Attribute: Input Data Length Publisher
- 2.14.3.24.1.2.3. (m) Attribute: Source Offset Publisher
- 2.14.3.24.1.2.4. (m) Attribute: Length
- 2.14.3.24.1.3. (o) Attribute: IsoM Parameter
- 2.14.3.24.1.3.1. (m) Attribute: T_{BASE_DP}
- 2.14.3.24.1.3.2. (m) Attribute: T_{DP}
- 2.14.3.24.1.3.3. (m) Attribute: T_{MAPC}
- 2.14.3.24.1.3.4. (m) Attribute: T_{BASE_IO}
- 2.14.3.24.1.3.5. (m) Attribute: T_I
- 2.14.3.24.1.3.6. (m) Attribute: T_O
- 2.14.3.24.1.3.7. (m) Attribute: T_{PLL_W}
- 2.14.3.24.1.3.8. (m) Attribute: T_{PLL_D}
- 2.14.3.24.1.4. (o) Attribute: List of DXB Subscriber Entries
- 2.14.3.24.1.4.1. (s) Attribute: Publisher Address
- 2.14.3.24.1.4.2.1. (m) Attribute: Input Data Length Publisher
- 2.14.3.24.1.4.2.2. (m) Attribute: Source Offset Publisher
- 2.14.3.24.1.4.3. (s) Attribute: Master Data
- 2.14.3.24.1.4.3.1. (m) Attribute: Source Offset Master
- 2.14.3.24.1.4.4. (m) Attribute: Dest Slot Number
- 2.14.3.24.1.4.5. (m) Attribute: Offset Data Area
- 2.14.3.24.1.4.6. (m) Attribute: Length Data Area
- 2.14.3.24.1.5. (o) Attribute: Prm Command
- 2.14.3.24.1.6. (c) Constraint: Prm Command supported = TRUE
- 2.14.3.24.1.6.1. (m) Attribute: Seq Number
- 2.14.3.24.1.6.2. (m) Attribute: Primary Request
- 2.14.3.24.1.6.3. (m) Attribute: MS1 Command
- 2.14.3.24.1.6.4. (m) Attribute: Master State Clear
- 2.14.3.24.1.6.5. (m) Attribute: Check Properties
- 2.14.3.24.1.6.5.1. (m) Attribute: Primary Request used
- 2.14.3.24.1.6.5.2. (m) Attribute: MS1 Command used
- 2.14.3.24.1.6.5.3. (m) Attribute: Address Change
- 2.14.3.24.1.6.5.4. (m) Attribute: Address Offset
- 2.14.3.24.1.6.6. (m) Attribute: Output Hold Time
- 2.14.3.24.1.7. (o) Attribute: Time AR Parameter
- 2.14.3.24.1.7.1. (m) Attribute: Clock Sync Interval
- 2.14.3.24.1.7.2. (m) Attribute: CS Delay Time
- 2.14.3.25. (o) Attribute: IsoM Req
- 2.15. (c) Constraint: CR Type = BQUU & D_SAP_index = 62
- 2.15.1. (m) Attribute: List of Cfg Data
- 2.15.1.1. (s) Attribute: Cfg Identifier
- 2.15.1.1.1. (m) Attribute: Type
- 2.15.1.1.2. (m) Attribute: Format
- 2.15.1.1.3. (m) Attribute: Consistency

- 2.15.1.1.4. (m) Attribute: Length
- 2.15.1.2. (s) Attribute: Special Cfg Identifier
- 2.15.1.2.1. (m) Attribute: Type
- 2.15.1.2.2. (c) Constraint: Type = Input or Input-Output
- 2.15.1.2.2.1. (m) Attribute: Input Length
- 2.15.1.2.2.2. (m) Attribute: Input Format
- 2.15.1.2.2.3. (m) Attribute: Input Consistency
- 2.15.1.2.3. (c) Constraint: Type = Output or Input-Output
- 2.15.1.2.3.1. (m) Attribute: Output Length
- 2.15.1.2.3.2. (m) Attribute: Output Format
- 2.15.1.2.3.3. (m) Attribute: Output Consistency
- 2.15.1.2.4. (o) Attribute: List of Data Description
- 2.15.1.2.4.1. (m) Attribute: Data Description
- 2.15.1.2.5. (o) Attribute: Manufacturer Specific Data
- 2.16. (c) Constraint: CR Type = BBUU & D_SAP_index = 60
- 2.16.1. (m) Attribute: Diag Upd Delay
- 2.16.2. (m) Attribute: Ext Diag Overflow
- 2.16.3. (m) Attribute: Deactivated
- 2.16.4. (m) Attribute: Station Non Existent
- 2.16.5. (m) Attribute: Invalid Slave Response
- 2.17. (c) Constraint: CR Type = BBUB & D_SAP_index = NIL
- 2.17.1. (m) Attribute: Fail Safe
- 2.17.2. (m) Attribute: DPV1 Supported
- 2.17.3. (m) Attribute: NA to Abort
- 2.17.4. (m) Attribute: Ignore ACIr
- 2.17.5. (m) Attribute: Output Len
- 2.17.6. (m) Attribute: Exp Inp Len
- 2.17.7. (m) Attribute: Data Transfer Status
- 2.17.8. (m) Attribute: System Diagnosis Status
- 2.17.9. (m) Attribute: Publisher Flag
- 2.18. (c) Constraint: CR Type = BQUU & D_SAP_index = 53
- 2.18.1. (m) Attribute: List of Structured Prm Data
- 2.18.1.1. (o) Attribute: Structured User Prm Data
- 2.18.1.1.1. (o) Attribute: Device Related User Parameter
- 2.18.1.1.1.1. (m) Attribute: Device Prm Data
- 2.18.1.1.2. (o) Attribute: Module Related User Parameter
- 2.18.1.1.2.1. (m) Attribute: Slot Number
- 2.18.1.1.2.2. (m) Attribute: Module Prm Data
- 2.18.1.2. (o) Attribute: List of DXB Link Entries
- 2.18.1.2.1. (m) Attribute: Publisher Address
- 2.18.1.2.2. (m) Attribute: Input Data Length Publisher
- 2.18.1.2.3. (m) Attribute: Source Offset Publisher
- 2.18.1.2.4. (m) Attribute: Length
- 2.18.1.3. (o) Attribute: IsoM Parameter
- 2.18.1.3.1. (m) Attribute: TBASE_DP
- 2.18.1.3.2. (m) Attribute: TDP
- 2.18.1.3.3. (m) Attribute: TMAPC
- 2.18.1.3.4. (m) Attribute: TBASE_IO
- 2.18.1.3.5. (m) Attribute: T_I
- 2.18.1.3.6. (m) Attribute: T_O
- 2.18.1.3.7. (m) Attribute: T_{PLL_W}
- 2.18.1.3.8. (m) Attribute: T_{PLL_D}
- 2.18.1.4. (o) Attribute: List of DXB Subscriber Entries
- 2.18.1.4.1. (s) Attribute: Publisher Address
- 2.18.1.4.2.1. (m) Attribute: Input Data Length Publisher
- 2.18.1.4.2.2. (m) Attribute: Source Offset Publisher
- 2.18.1.4.3. (s) Attribute: Master Data
- 2.18.1.4.3.1. (m) Attribute: Source Offset Master
- 2.18.1.4.4. (m) Attribute: Dest Slot Number
- 2.18.1.4.5. (m) Attribute: Offset Data Area
- 2.18.1.4.6. (m) Attribute: Length Data Area
- 2.18.1.5. (o) Attribute: Time AR Parameter
- 2.18.1.5.1. (s) Attribute: Clock Sync Interval
- 2.18.1.5.2. (s) Attribute: CS Delay Time
- 2.19. (o) Attribute: CR Name

SERVICES:

1. (m) MgtService: Load CRL
2. (o) MgtService: Get CRL
3. (m) MgtService: CRL Slave Activate
4. (m) MgtService: CRL Slave New Prm
5. (m) MgtService: CRL Slave New Prm Data

8.2.10.4.2.2 Attributes**Implicit**

The attribute Implicit indicates that the CRL DP-master (Class 1) object is implicitly addressed by the service.

List of CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier of this CR. This attribute shall be set accordingly with the Load CRL service.

CR Type

This static attribute indicates the type of this CR. This attribute is dependent on the AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service. For each AR Type the correspondent CRL entries shall be set.

Table 349 shows the allowed values for this attribute.

Related AREP

This static attribute contains the local identifier for the related AR for this CR. This attribute shall be set accordingly with the Load CRL service.

S_SAP_index

This static attribute contains the identifier for the related Service Access Point of the local Data Link Layer. The value for this attribute has to be set correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service.

Table 349 shows the allowed values for this attribute.

Table 349 – S_SAP_index

AR Type	CR Type	S_SAP_index	Remark
MS0	BBUB	NIL	Mandatory for the MS0 AR
MS0	BQUU	62	Mandatory for the MS0 AR
MS0	BBUU	62	Mandatory for the MS0 AR
MS0	QQUU-OM	62	Mandatory for the MS0 AR
MS1	QQCB-CO	51	Mandatory for the MS1 AR
MS1	QQUU-OO	62	Mandatory for the MS1 AR
MS3	QQUU-OM	CS	Mandatory for the MS3 AR
MM1	QQCB-CL	54	Mandatory for the MM1 AR
MM2	QQUU-OM	54	Mandatory for the MM2 AR

D_SAP_index

This static attribute contains the identifier for the related Service Access Point of the remote Data Link Layer. The value for this attribute has to be set correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service. Table 350 shows the allowed values for this attribute.

Table 350 – D_SAP_index

AR Type	CR Type	D_SAP_index	Remark
MS0	BBUB	NIL	Mandatory for the MS0 AR
MS0	BQUU	61, 62	Mandatory for the MS0 AR
MS0	BQUU	53	Optional for the MS0 AR
MS0	BBUU	60	Mandatory for the MS0 AR
MS0	QQUU-OM	58	Mandatory for the MS0 AR
MS1	QQCB-CO	50	Optional for each MS1 AR
MS1	QQCB-CO	51	Mandatory for the MS1 AR
MS1	QQUU-OO	60	Mandatory for the MS1 AR
MS3	QQUU-OM	CS	Mandatory for the MS3 AR
MM1	QQCB-CL	54	Mandatory for the MM1 AR
MM2	QQUU-OM	54	Mandatory for the MM2 AR

D_addr

This dynamic attribute contains the DL-address of the remote communication partner. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR. Table 351 shows the allowed values for this attribute used with the Load CRL service. Furthermore this attribute is influenced by the AR state machine dependent of the attribute CR Type of this CR and of the attribute AR Type of the related AR. Table 351 shows the allowed values for this attribute for the modification of the AR state machine.

Table 351 – D_addr

AR Type	CR Type	S_SAP_index	Time Device Type (ARL)	D_addr as loaded with the Load CRL service	D_addr modified by the AR state machine
MS0	BBUB	NIL	--	0 to 126	-
MS0	BQUU	62	--	0 to 126	-
MS0	BBUU	62	--	0 to 126	-
MS0	QQUU-OM	62	--	0 to 127	-
MS1	QQCB-CO	51	--	0 to 126	-
MS1	QQUU-CO	62	--	0 to 126	-
MS3	QQUU-OM	CS	Time Master	127	-
MS3	QQUU-OM	CS	Time Receiver	None	All
MM1	QQCB-CL	54	--	All	0 to 125
MM2	QQUU-OM	54	--	All	0 to 125

Where

All means that the local Service Access Point shall be activated with no access protection
 None means that the local Service Access Point shall not be activated
 0 to 127 means that the attribute D_addr of the local Service Access Point shall be set with the corresponding remote DL-address
 - means that the AR state machines shall not modify the attribute D_addr
 -- means that the value of this attribute is not relevant

Service_activate

This static attribute indicates the DL services which shall be activated for the S_SAP_index of this CRL entry.

The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 352 shows the allowed values for this attribute.

Table 352 – Service_activate

AR Type	CR Type	S_SAP_index	Publisher Flag	Service_activate
MS0	BBUB	NIL	FALSE	SRD
MS0	BBUB	NIL	TRUE	MSRD
MS0	BQUU	62	-	SRD
MS0	BBUU	62	-	SRD
MS0	QQUU-OM	62	-	SDN
MS1	QQCB-CO	51	-	SRD
MS1	QQUU-CO	62	-	SRD
MS3	QQUU-OM	CS	-	CS Time Event CS Clock Value
MM1	QQCB-CL	54	-	SRD
MM2	QQUU-OM	54	-	SDN

Role_in_service

This static attribute indicates the role which shall be activated for the S_SAP_index of this CRL entry. The following values are specified:

- Initiator: The station initiates the respective service exclusively
- Responder: The station response to the service exclusively

The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 353 shows the allowed values for this attribute.

Table 353 – Role_in_service

AR Type	CR Type	S_SAP_index	Time Device Type (ARL)	Role_in_service
MS0	BBUB	NIL	-	Initiator
MS0	BQUU	62	-	Initiator
MS0	BBUU	62	-	Initiator
MS0	QQUU-OM	62	-	Initiator
MS1	QQCB-CO	51	-	Initiator
MS1	QQUU-CO	62	-	Initiator
MS3	QQUU-OM	CS	Time Master	Initiator
MS3	QQUU-OM	CS	Time Receiver	responder
MM1	QQCB-CL	54	-	Responder
MM2	QQUU-OM	54	-	Responder

Indication_mode

This conditional and static attribute shall only be present if the attribute Service_activate of this CRL entry has the value SRD or MSRD and if the attribute Role_in_service of this CRL entry has the value Responder. It indicates the Indication_mode which the DLL shall use to convey the indications of the (M)SRD service. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 354 shows the allowed values for this attribute.

Table 354 – Indication_mode

AR Type	CR Type	S_SAP_index	Indication_mode
MS0	BBUB	NIL	-
MS0	BQUU	62	-
MS0	BBUU	62	-
MS0	QQUU-OM	62	-
MS1	QQCB-CO	51	-
MS1	QQUU-CO	62	-
MM1	QQCB-CL	54	Data
MM2	QQUU-OM	54	-
where - means that the attribute Indication_mode is not present for this CR Type			

Max_DLSDU_length_req_low

This static attribute indicates the maximum length for the DLSDU with low priority as requester/responder. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 355 shows the allowed values for this attribute.

Table 355 – Max_DLSDU_length_req_low

AR Type	CR Type	D_SAP_index	Time Device Type (ARL)	Max_DLSDU_length_req_low
MS0	BBUB	NIL	-	0
MS0	BQUU	61, 62, 53	-	0
MS0	BBUU	60	-	0
MS0	QQUU-OM	58	-	0
MS1	QQCB-CO	50	-	4
MS1	QQCB-CO	51	-	4 to 244
MS1	QQUU-OO	60	-	0
MS3	QQUU-OM	CS	Time Master	17
MS3	QQUU-OM	CS	Time Receiver	0
MM1	QQCB-CL	54		4 to 244
MM2	QQUU-OM	54		0

Max_DLSDU_length_req_high

This static attribute indicates the maximum length for the DLSDU with high priority as requester/responder. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 356 shows the allowed values for this attribute.

Table 356 – Max_DLSDU_length_req_high

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_req_high
MS0	BBUB	NIL	0 to 244
MS0	BQUU	61	7 to 244
MS0	BQUU	62	1 to 244
MS0	BQUU	53	5 to 244
MS0	BBUU	60	0
MS0	QQUU-OM	58	2
MS1	QQCB-CO	50, 51	0
MS1	QQUU-OO	60	0
MS3	QQUU-OM	CS	0
MM1	QQCB-CL	54	0
MM2	QQUU-OM	54	0

Max_DLSDU_length_ind_low

This static attribute indicates the maximum length for the DLSDU with low priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 357 shows the allowed values for this attribute.

Table 357 – Max_DLSDU_length_ind_low

AR Type	CR Type	D_SAP_index	Time Device Type (ARL)	Max_DLSDU_length_ind_low
MS0	BBUB	NIL	-	0 to 244
MS0	BQUU	61,62, 53	-	0
MS0	BBUU	60	-	6 to 244
MS0	QQUU-OM	58	-	0
MS1	QQCB-CO	50	-	4
MS1	QQCB-CO	51	-	4 to 244
MS1	QQUU-OO	60	-	6 to 244
MS3	QQUU-OM	CS	Time Master	0
MS3	QQUU-OM	CS	Time Receiver	17
MM1	QQCB-CL	54		4 to 244
MM2	QQUU-OM	54		2

Max_DLSDU_length_ind_high

This static attribute indicates the maximum length for the DLSDU with high priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 358 shows the allowed values for this attribute.

Table 358 – Max_DLSDU_length_ind_high

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_ind_high
MS0	BBUB	NIL	0 to 244
MS0	BQUU	61,62, 53	0
MS0	BBUU	60	0
MS0	QQUU-OM	58	0
MS1	QQCB-CO	50, 51	0
MS1	QQUU-OO	60	0
MS3	QQUU-OM	CS	0
MM1	QQCB-CL	54	0
MM2	QQUU-OM	54	0

CR Type = BQUU & D_SAP_index = 61

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BQUU and the attribute D_SAP_index is set to 61.

Active

This dynamic attribute indicates the state of interactions to the correspondent DP-slave. The attribute set to FALSE indicates that the DP-master (Class 1) shall stop the interactions to the correspondent DP-slave. In this state it is allowed to change the CRL entries of this DP-slave with the Load CRL service. If this attribute is set to TRUE the DP-master (Class 1) shall start the interactions to the correspondent DP-slave. In this state only the attribute Prm Data may be changed with the CRL Slave New Prm Data service (see also the attribute New Prm). This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave Activate service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

New Prm

This dynamic attribute indicates to the MS0 AR state machine to send new parameterisation data during the data exchange mode to the correspondent DP-slave. The following rules shall be applied:

If the attribute is set to FALSE the DP-master (Class 1) transfers the input, output data to the correspondent DP-slave. If this attribute is set to TRUE and the attribute DPV1 Supported of the related CRL entry has the value FALSE the DP-master (Class 1) transfers for one DP-slave poll cycle new parameterisation data to the correspondent DP-slave. The MS0 AR state machine sets the attribute to FALSE if the transfer was successful.

This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Prm Data

This dynamic attribute is an aggregate of the parameterisation data of the correspondent DP-slave and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Lock

This dynamic attribute is used by the MS0 AR state machine to assign or release the correspondent DP-slave. This attribute is set to LOCK if the attribute Active of this related CRL entry is set to TRUE. This attribute is set to UNLOCK if the attribute Active of this related CRL entry is set to FALSE, or due to failures during the data exchange mode. This attribute shall be set to LOCK with the Load CRL service. This attribute is further influenced by the MS0 AR state machine. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: LOCK, UNLOCK

Sync

This dynamic attribute indicates to the correspondent DP-slave that it shall participate in the synchronisation of the Output Data (Sync mode). The DP-slave shall check whether it supports the requested functionality and if the DP-slave does not support this function it shall respond with an error entry in the Slave Diagnosis (Not Supported). This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Table 359 shows the allowed values for this attribute.

Table 359 – Sync

Value	Meaning
FALSE	Sync ability not requested for this DP-slave
TRUE	Sync ability requested for this DP-slave

Freeze

This dynamic attribute indicates to the correspondent DP-slave that it shall participate in the synchronisation of the Input Data (Freeze mode). The DP-slave shall check whether it supports the requested functionality and if the DP-slave does not support this function it shall respond with an error entry in the Slave Diagnosis (Not Supported). This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Table 360 shows the allowed values for this attribute.

Table 360 – Freeze

Value	Meaning
FALSE	Freeze ability not requested for this DP-slave
TRUE	Freeze ability requested for this DP-slave

WD On

This dynamic attribute indicates if the watchdog control shall be activated at the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

WD Fact 1, WD Fact 2

These dynamic attributes contain the values for the monitoring of the DP-master (Class 1) at the MS0 AR (watchdog control (T_{WD})) at the correspondent DP-slave.

The watchdog control in a DP-slave takes care that, if the DP-master fails, the outputs will be set in the safe state after the expiration of this time. The time is calculated according to equation:

$$T_{WD} = \text{Watchdog Time Base} \times \text{WD Fact 1} \times \text{WD Fact 2}$$

If the attribute DPV1 Supported has the value FALSE the Watchdog Time Base shall be set to 10 ms.

If the attribute DPV1 Supported has the value TRUE the Watchdog Time Base shall be set to the value 1ms if the attribute WD Base has the value TRUE. If the attribute DPV1 Supported has the value TRUE the Watchdog Time Base shall be set to the value 10ms if the attribute WD Base has the value FALSE. These attributes shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: 1 to 255

Min TSDR

This dynamic attribute contains the value for the Minimum Station Delay Responder time of the DLL of the correspondent DP-slave. If this attribute is equal zero, the previous value of the Min TSDR remains unchanged. These attributes shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: 0 to 255

Ident Number

This dynamic attribute contains the Ident Number of the correspondent DP-slave. The DP-slave accepts the parameterisation only if the transmitted Ident Number is identical with the attribute Ident Number of the MS0 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Attribute Type: Unsigned16

Allowed values: 0 to $2^{16}-1$

Group Ident

This dynamic attribute indicates the group(s) to which the correspondent DP-slave shall be assigned by the DP-master (Class 1) in the parameterisation. The Group Ident distinguishes 8 groups. If the Group Ident is set to No Group, the DP-slave is not assigned to any group. The DP-slave stores the value of this attribute in the attribute Group Identifier of the MS0 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: Group 1 to Group 8, No Group and combinations

DPV1 Enabled

This optional and static indicates if the extended functionality shall be activated at the correspondent DP-slave. This attribute shall only be present if the attribute DPV1 Supported in the related MS0 AR entry in the List of ARL Entries of the DP-

slave has the value TRUE. This attribute shall be set accordingly with the Load CRL service.

Table 361 shows the allowed values for this attribute.

Table 361 – DPV1 enabled

Value	Meaning
FALSE	The extended functionality related with the attribute DPV1 Supported is disabled.
TRUE	The extended functionality related with the attribute DPV1 Supported is enabled.

Fail Safe

This optional and static attribute indicates to the correspondent DP-slave if the DP-master (Class 1) supports the Fail Safe mode. If the DP-slave requires the Fail Safe mode and the DP-master (Class 1) does not support this mode the DP-slave shall respond with an error entry in the Slave Diagnosis (Prm Fault). The DP-slave stores the value of this attribute in the attribute Fail Safe of the MS0 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Table 362 shows the allowed values for this attribute.

Table 362 – Fail safe

Value	Meaning
FALSE	The DP-master (Class 1) does not support the Fail Safe mode.
TRUE	The DP-master (Class 1) supports the Fail Safe mode.

Enable Publisher

This optional and static attribute indicates to the correspondent DP-slave that it shall operate as Publisher. The DP-slave shall check whether it supports the requested functionality. If the DP-slave does not support this function it shall respond with an appropriate entry in the Diagnosis information (Prm Fault). This attribute shall be set accordingly with the Load CRL service.

Table 363 shows the allowed values for this attribute.

Table 363 – Enable publisher

Value	Meaning
FALSE	Publisher ability not requested for this DP-slave
TRUE	Publisher ability requested for this DP-slave

WD Base

This optional and static attribute contains the Time Base value for the watchdog control according to the parameterisation for the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute WD Base of the MS0 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Table 364 shows the allowed values for this attribute.

Table 364 – WD base

Value	Meaning
FALSE	The Watchdog Time Base is 10ms.
TRUE	The Watchdog Time Base is 1ms.

Enable Pull Plug Alarm

This optional and static attribute indicates if the transmission of the Pull Plug Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Pull Plug Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: TRUE, FALSE

Enable Process Alarm

This optional and static attribute indicates if the transmission of the Process Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Process Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Enable Diagnosis Alarm

This optional and static attribute indicates if the transmission of the Diagnosis Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Diagnosis Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Enable Manufacturer Specific Alarm

This optional and static attribute indicates if the transmission of the Manufacturer Specific Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Manufacturer Specific Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Enable Status Alarm

This optional and static attribute indicates if the transmission of the Status Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Status Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Enable Update Alarm

This optional and static attribute indicates if the transmission of the Update Alarm shall be enabled at the correspondent DP-slave. The DP-slave stores the value of this attribute in the attribute Update Alarm enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Check Cfg Mode

This optional and static attribute indicates how the validation of the Cfg Data shall be performed in the correspondent DP-slave. The value of this attribute is passed to the application of the DP-slave with the Check Cfg service and influences the check of the Cfg Data (see 8.2.5.3.5). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Alarm Mode

This optional and static attribute indicates the maximum number of possible active alarms between the DP-master (Class 1) and this DP-slave. The DP-slave stores the value of this attribute in the attribute Alarm Mode enabled of the MS1 AR entry in the List of ARL Entries of the DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Table 365 shows the allowed values for this attribute.

Table 365 – Alarm mode

Value	Meaning
0	1 alarm of each type
1	2 alarms in total
2	4 alarms in total
3	8 alarms in total
4	12 alarms in total
5	16 alarms in total
6	24 alarms in total
7	32 alarms in total

Prm Structure

This static attribute indicates which type of parameterisation objects shall be provided by the Check User Prm service at the correspondent DP-slave. The value of this attribute is passed to the application of the DP-slave with the Check User Prm service. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Prm Structure = FALSE

This constraint reflects that the parameter which follow are only related to the MS0 User Parameter object.

User Prm Data

This dynamic attribute contains the value(s) for the MS0 User Parameter object (either Device Related User Parameter or a List of Module Related User Parameter or both) of the correspondent DP-slave and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Device Prm Data

This optional and dynamic attribute contains the value for the Device Related User Parameter of the MS0 User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Module Prm Data

This optional and dynamic attribute contains the value for the Module Related User Parameter of the MS0 User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Prm Structure = TRUE

This constraint reflects that the parameter which follow are related to the MS0 Structured User Parameter object and/or the DXB-Linktable object and/or the IsoM Parameter object.

List of Structured Prm Data

This dynamic attribute is composed of one or more of the following elements.

Structured User Prm Data

This optional and dynamic attribute is composed of one or both of the following elements.

Device Related User Parameter

This static attribute indicates the presence of Device Related User Parameter. This attribute shall be set accordingly with the Load CRL service.

Device Prm Data

This static attribute contains the value for the Device Related User Parameter of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Module Related User Parameter

This static attribute indicates the presence of Module Related User Parameter. This attribute shall be set accordingly with the Load CRL service.

Slot Number

This static attribute contains the value for the attribute Slot Number of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Module Prm Data

This static attribute contains the value for the attribute Module Prm Data of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

List of DXB Link Entries

This optional and static attribute indicates the presence of the DXB-Linktable Entries and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Publisher Address

This static attribute contains the value for the attribute Publisher Address of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Input Data Length Publisher

This static attribute contains the value for the attribute Input Data Length Publisher of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 244

Source Offset Publisher

This static attribute contains the value for the attribute Source Offset Publisher of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Length

This static attribute contains the value for the attribute Length of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 1 to 244

IsoM Parameter

This optional and static attribute indicates the presence of the IsoM Parameter and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

TBASE_DP

This static attribute contains the value for the attribute TBASE_DP of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

TDP

This static attribute contains the value for the attribute TDP of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

TMAPC

This static attribute contains the value for the attribute TMAPC of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 14

TBASE_IO

This static attribute contains the value for the attribute TBASE_IO of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

T_I

This static attribute contains the value for the attribute T_I of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned16

Allowed values: 0 (special case), 1 to $2^{16}-1$

T_O

This static attribute contains the value for the attribute T_O of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned16

Allowed values: 0 (special case), 1 to $2^{16}-1$

T_{DX}

This static attribute contains the value for the attribute T_{DX} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned32

Allowed values: 1 to $2^{32}-1$

T_{PLL_W}

This static attribute contains the value for the attribute T_{PLL_W} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned16

Allowed values: 1 to $2^{16}-1$

T_{PLL_D}

This static attribute contains the value for the attribute T_{PLL_D} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned16

Allowed values: 0 to $2^{16}-1$

List of DXB Subscriber Entries

This optional and static attribute indicates the presence of the DXB-Subscribable Entries and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Publisher Address

This static and selection attribute contains the value for the attribute Publisher Address of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Input Data Length Publisher

This static attribute contains the value for the attribute Input Data Length Publisher of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 244

Source Offset Publisher

This static attribute contains the value for the attribute Source Offset Publisher of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Master Data

This static and selection attribute indicates that for this entry the data shall be derived from the related DP-master (Class 1). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 128

Source Offset Master

This static attribute defines the beginning (offset) within the Output Data of the DP-master (Class 1). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Dest Slot Number

This static attribute indicates the Slot Number of the module to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Offset Data Area

This static attribute defines the beginning (offset) within the Output Data of the module (data area) to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Length Data Area

This attribute defines the number of octets the Subscriber shall subscribe from the related Publisher starting at the offset given in the attribute Source Offset Publisher in case of a Publisher. If the data shall be derived from the DP-master (Class 1) the starting offset is given in the attribute Source Offset Master.

The subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped to the data area of the module given in the attributes Dest Slot Number and Offset Data Area of the related DXB Subscriber Entry.

This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Prm Command

This optional and dynamic attribute indicates the presence of the Prm Command and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Seq Number

This static attribute contains the Seq Number of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: 0 to 31

Primary Request

This static attribute contains the value for the parameter Primary Request of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

MS1 Command

This static attribute contains the value for the parameter MS1 Command of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: no Action, Stop MS1, Start MS1, Reset MS1

Master State Clear

This static attribute contains the value for the parameter Master State Clear of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

Check Properties

This static attribute contains the value for the parameter Check Properties of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

Primary Request used

This static attribute contains the value for the parameter Primary Request used of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

MS1 Command used

This static attribute contains the value for the parameter MS1 Command used of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

Address Change

This static attribute contains the value for the parameter Address Change of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

Address Offset

This static attribute contains the value for the parameter Address Offset of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: TRUE, FALSE

Output Hold Time

This static attribute contains the value for the parameter Output Hold Time of the related Prm Command for the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: 0 to $2^{16}-1$

Time AR Parameter

This optional and dynamic attribute indicates the presence of the Time AR Parameter and consists of the following attributes. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Clock Sync Interval

This static attribute contains the value for the attribute clock synchronization interval of the ARL of the DP-slave operating as time receiver.

Allowed values: 1 to $2^{16}-1$; Default value: 100

Time Base: 10ms

CS Delay Time

This static and optional attribute contains the value for the attribute CS Delay Time of the ARL of the DP-slave operating as time receiver.

Default value: 0

Type: Network Time Difference

IsoM Req

This conditional and dynamic attribute indicates to the DP Application Layer of the correspondent DP-slave to operate in the Isochronous Mode. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

CR Type = BQUU & D_SAP_index = 62

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BQUU and the attribute D_SAP_index is set to 62.

List of Cfg Data

This dynamic attribute is composed of the following list elements:

Cfg Identifier

This selection attribute describes the basic configuration format and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Type

This attribute indicates the type of the related Input/Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Input, Output, Input-Output

Format

This attribute indicates the format of the related Input/Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word

Consistency

This attribute indicates the consistency of the related Input/Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word, whole length

Length

This attribute indicates the length of the related Input/Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 32

Special Cfg Identifier

This selection attribute describes the special configuration format and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Type

This attribute indicates the type of the related Input/Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Input, Output, Input-Output

Input Length

This attribute indicates the length of the related Input Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 128

Input Format

This attribute indicates the format of the related Input Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word

Input Consistency

This attribute indicates the consistency of the related Input Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word, whole length

Output Length

This attribute indicates the length of the related Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 128

Output Format

This attribute indicates the format of the related Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word

Output Consistency

This attribute indicates the consistency of the related Output Data Element. This attribute shall be set accordingly with the Load CRL service.

Allowed values: Byte, Word, whole length

List of Data Description

This attribute contains one of the allowed Data Types of the data description of one element of the Extended Input/Output Data object. The allowed Data Types are given in 5.2.1. This attribute shall be set accordingly with the Load CRL service.

Manufacturer Specific Data

This attribute contains the manufacturer specific data for one Special Cfg Identifier. This attribute shall be set accordingly with the Load CRL service.

CR Type = BBUU & D_SAP_index = 60

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BBUU and the attribute D_SAP_index is set to 60.

Diag Upd Delay

The static attribute indicates the number of Diag service requests performed in the state DIAG2 of the DP-master (Class 1) while Prm Req is set in Diag Data of the correspondent DP-slave. This allows DP-slaves with reduced performance to update the Diag Data during the start-up phase of the DP system. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 to 15 (extendible up to 255)

Ext Diag Overflow

This dynamic attribute shall be set to TRUE if the values of all Diagnosis objects with a changed value could not be provided with one Set Slave Diag service request by the DP-slave or if the DP-master (Class 1) could not provide all received values of the Diagnosis objects from the DP-slave to the application due to storage problems. This attribute shall be set accordingly with the Load CRL service. This attribute is further influenced by the MS0 ARL state machine of the DP-master (Class 1).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Deactivated

This dynamic attribute indicates the state of a DP-slave which is assigned to the DP-master (Class 1). The attribute shall be set to TRUE if the DP-slave is deactivated by the AP of the DP-master (Class 1) or the USIF State of the DP-master (Class 1) is STOP. This attribute shall be set accordingly with the Load CRL service. This attribute is further influenced by the MS0 AR state machine of the DP-master (Class 1).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Station Non Existent

This dynamic attribute indicates the status of an assigned DP-slave. The attribute shall be set to TRUE if the correspondent DP-slave does not respond to any request of the DP-master (Class 1). The attribute shall be set to FALSE if the correspondent DP-slave responds to the requests of the DP-master (Class 1). This attribute shall be set accordingly with the Load CRL service. This attribute is further influenced by the MS0 AR state machine of the DP-master (Class 1).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Invalid Slave Response

This dynamic attribute indicates the status of an assigned DP-slave. The attribute shall be set to TRUE if the correspondent DP-slave does not respond correctly to a request of the DP-master (Class 1) (e. g. the length of the received Input Data is not in alignment with the expected length). The attribute shall be set to FALSE if the correspondent DP-slave responds correctly. This attribute shall be set accordingly with the Load CRL service. This attribute is further influenced by the MS0 AR state machine of the DP-master (Class 1).

Attribute Type: Boolean

Allowed values: TRUE, FALSE

CR Type = BBUB & D_SAP_index = NIL

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BBUB and the attribute D_SAP_index is set to NIL.

Fail Safe

This static attribute indicates if the correspondent DP-slave supports the Fail Safe functionality. If the attribute is set to FALSE the DP-master (Class 1) shall send the value 0 for the Output Data object for this DP-slave in the operation mode CLEAR. If the attribute is set to TRUE the following rules shall apply for the DP-master (Class 1) and the DP-slave:

The DP-master (Class 1) shall convey no values (means Output Data length = 0) for the Output Data object for this DP-slave in the operation mode CLEAR.

The DP-slave shall trigger the application to set the values of the Output Data object in a safe state after receiving no values for the Output Data object during the data exchange mode.

This attribute shall be set accordingly with the Load CRL service.

Table 366 shows the allowed values for this attribute.

Table 366 – Fail safe

Value	Meaning
FALSE	Fail Safe functionality not supported by the DP-slave
TRUE	Fail Safe functionality supported by the DP-slave

DPV1 Supported

This static attribute indicates if the correspondent DP-slave supports the extended functionality. If the attribute is set to FALSE the DP-slave supports no extended functionality. If the attribute is set to TRUE the DP-slave supports the extended functionality. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

NA to Abort

This static attribute indicates the reaction of the DP-master (Class 1) if the correspondent DP-slave does not respond at the MS0 AR in the DIAG2 state and the data exchange mode. If this attribute is set to FALSE the DP-master (Class 1) continues with the interactions even the DP-slave does not respond (Status = NA) to a request. If the attribute is set to TRUE the DP-master (Class 1) releases and re-establish the CRs of the MS0 AR to the correspondent DP-slave if it does not respond (Status = NA) to a request. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Ignore ACIr

This static attribute in combination with the attribute Error Action Flag in the MS0 AR entry of the List of ARL Entries of the DP-master (Class 1) allows a DP-slave specific reaction of the DP-master (Class 1) in case of failed data transfers. If the attribute is set to FALSE the DP-master (Class 1) processes the auto-clear function according to the value of the attribute Error Action Flag. If the attribute is set to TRUE the DP-master (Class 1) ignores the auto-clear function according to the value of the attribute Error Action Flag. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Output Len

This static attribute indicates the length of the data of the Output Data object of the correspondent DP-slave. The value for this attribute shall be derived from the attribute List of Cfg Data of the related CRL entry. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Exp Inp Len

This static attribute indicates the length of the data of the Input Data object of the correspondent DP-slave expected by the DP-master (Class 1). If the length of the Input Data of the Input Data object does not correspond with the value contained in this attribute the DP-master (Class 1) releases and re-establish the CRs of the MS0 AR to the correspondent DP-slave. The value for this attribute shall be derived from the attribute List of Cfg Data of the related CRL entry. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Data Transfer Status

This dynamic attribute indicates the status of the data exchange mode of the correspondent DP-slave. This attribute shall be set to TRUE by the MS0 AR state machine of the DP-master (Class 1) if the data exchange with the assigned DP-slave was successfully or if between two data exchange requests was only one diagnosis request.

In all other cases the attribute shall be set to FALSE. This attribute shall be set to FALSE with the Load CRL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

System Diagnosis Status

This dynamic attribute indicates the state of the correspondent DP-slave assigned to the DP-master (Class 1). This attribute shall be set to TRUE by the MS0 AR state machine of the DP-master (Class 1) if the Ext Diag Flag of the Diagnosis information of the correspondent the DP-slave is set or if the correspondent the DP-slave does not respond or if the correspondent the DP-slave responds incorrectly so that the MS0 AR has to be re-established.

This attribute shall be set to FALSE by the MS0 AR state machine of the DP-master (Class 1) after the MS0 AR is successfully re-established or if the correspondent DP-slave is deactivated by the AP of the DP-master (Class 1) or the operation mode of the DP-master (Class 1) is STOP.

This attribute shall be set to FALSE with the Load CRL service. This attribute is further influenced by the MS0 AR state machine.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

Publisher Flag

This static attribute indicates if the correspondent DP-slave shall operate as Publisher. If the attribute is set to FALSE the correspondent DP-slave shall not operate as Publisher and therefore the DP-master (Class 1) shall address this DP-slave with data exchange requests. If the attribute is set to TRUE the correspondent DP-slave shall operate as Publisher and therefore the DP-master (Class 1) shall address this DP-slave with data exchange requests with changed DL Function (DXB requests). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Boolean

Allowed values: TRUE, FALSE

CR Type = BQUU & D_SAP_index = 53

This constraint reflects the attributes for this CR if the value of the attribute CR Type is set to BQUU and the attribute D_SAP_index is set to 53.

List of Structured Prm Data

This static attribute is composed of one or more of the following elements.

Structured User Prm Data

This optional and static attribute is composed of one or both of the following elements.

Device Related User Parameter

This static attribute indicates the presence of Device Related User Parameter. This attribute shall be set accordingly with the Load CRL service.

Device Prm Data

This static attribute contains the value for the Device Related User Parameter of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Module Related User Parameter

This static attribute indicates the presence of Module Related User Parameter. This attribute shall be set accordingly with the Load CRL service.

Slot Number

This static attribute contains the value for the attribute Slot Number of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Module Prm Data

This static attribute contains the value for the attribute Module Prm Data of the MS0 Structured User Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

List of DXB Link Entries

This optional and static attribute indicates the presence of the DXB-Linktable Entries and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Publisher Address

This static attribute contains the value for the attribute Publisher Address of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 to 125

Input Data Length Publisher

This static attribute contains the value for the attribute Input Data Length Publisher of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 244

Source Offset Publisher

This static attribute contains the value for the attribute Source Offset Publisher of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 to 243

Length

This static attribute contains the value for the attribute Length of one DXB-Linktable entry of the DXB-Linktable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 244

IsoM Parameter

This optional and static attribute indicates the presence of the IsoM Parameter and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

TBASE_DP

This static attribute contains the value for the attribute TBASE_DP of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

TDP

This static attribute contains the value for the attribute TDP of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to $2^{16}-1$

TMAPC

This static attribute contains the value for the attribute TMAPC of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 14

T_{BASE_IO}

This static attribute contains the value for the attribute T_{BASE_IO} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 375, 750, 1500, 3000, 6000, 12000 (the value 1500 is mandatory for standard Isochronous Mode)

T_I

This static attribute contains the value for the attribute T_I of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 (special case), 1 to 2¹⁶-1

T_O

This static attribute contains the value for the attribute T_O of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 (special case), 1 to 2¹⁶-1

T_{DX}

This static attribute contains the value for the attribute T_{DX} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 2³²-1

T_{PLL_W}

This static attribute contains the value for the attribute T_{PLL_W} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 2¹⁶-1

T_{PLL_D}

This static attribute contains the value for the attribute T_{PLL_D} of the IsoM Parameter object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 0 to 2¹⁶-1

List of DXB Subscriber Entries

This optional and static attribute indicates the presence of the DXB-Subscribable Entries and is composed of the following elements. This attribute shall be set accordingly with the Load CRL service.

Publisher Address

This static and selection attribute contains the value for the attribute Publisher Address of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 125

Input Data Length Publisher

This static attribute contains the value for the attribute Input Data Length Publisher of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Allowed values: 1 to 244

Source Offset Publisher

This static attribute contains the value for the attribute Source Offset Publisher of one DXB-Subscribable entry of the DXB-Subscribable object of the correspondent DP-slave. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Master Data

This static and selection attribute indicates that for this entry the data shall be derived from the related DP-master (Class 1). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 128

Source Offset Master

This static attribute defines the beginning (offset) within the Output Data of the DP-master (Class 1). This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 243

Dest Slot Number

This static attribute indicates the Slot Number of the module to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped. This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 0 to 244

Offset Data Area

This static attribute defines the beginning (offset) within the Output Data of the module (data area) to which the subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped. This attribute shall be set accordingly with the Load CRL service

Attribute Type: Unsigned8

Allowed values: 0 to 244

Length Data Area

This attribute defines the number of octets the Subscriber shall subscribe from the related Publisher starting at the offset given in the attribute Source Offset Publisher in case of a Publisher. If the data shall be derived from the DP-master (Class 1) the starting offset is given in the attribute Source Offset Master.

The subscribed values of the Input Data object of the addressed Publisher or the values of the addressed Output Data of the related DP-master (Class 1) shall be mapped to the data area of the module given in the attributes Dest Slot Number and Offset Data Area of the related DXB Subscriber Entry.

This attribute shall be set accordingly with the Load CRL service.

Attribute Type: Unsigned8

Allowed values: 1 to 244

Time AR Parameter

This optional and dynamic attribute indicates the presence of the Time AR Parameter and consists of the following attributes. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Clock Sync Interval

This static attribute contains the value for the attribute clock synchronisation interval of the ARL of the DP-slave operating as time receiver. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Allowed values: 1 to $2^{16}-1$; Default value: 100

Time Base: 10ms

CS Delay Time

This static attribute contains the value for the attribute CS Delay Time of the ARL of the DP-slave operating as time receiver. This attribute shall be set accordingly with the Load CRL service. This attribute may be set dynamically by the CRL Slave New Prm Data service.

Default value: 0

Type: Network Time Difference

CR Name

This static attribute contains the name of the related CRL Entry. This attribute shall be set accordingly with the Load CRL service.

8.2.10.4.2.3 Invocation of CRL DP-master (class 1) object

For the invocation of the CRL DP-master (Class 1) object the following rule has to be considered:

- Only one CRL DP-master (Class 1) object shall be invoked in a DP-master (Class 1).

8.2.10.4.3 CRL DP-master (class 2) class specification**8.2.10.4.3.1 Template**

The CRL DP-master (Class 2) object is described by the following template:

DP ASE:		AR ASE
CLASS:	CRL DP-master	(Class 2)
CLASS ID:		not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1.	(m)	Key Attribute: Implicit
2.	(m)	Attribute: List of CRL Entries
2.1.	(m)	Attribute: CREP
2.2.	(m)	Attribute: CR Type
2.3.	(m)	Attribute: Related AREP
2.4.	(m)	Attribute: S_SAP_index
2.5.	(m)	Attribute: D_SAP_index
2.6.	(m)	Attribute: D_addr
2.8.	(m)	Attribute: Service_activate
2.9.	(m)	Attribute: Role_in_service
2.10.	(m)	Attribute: Max_DLSDU_length_req_low
2.11.	(m)	Attribute: Max_DLSDU_length_req_high
2.12.	(m)	Attribute: Max_DLSDU_length_ind_low
2.13.	(m)	Attribute: Max_DLSDU_length_ind_high
2.14.	(o)	Attribute: CR Name

SERVICES:

1. (m) MgtService: Load CRL
2. (o) MgtService: Get CRL

8.2.10.4.3.2 Attributes**Implicit**

The attribute Implicit indicates that the CRL DP-master (Class 2) object is implicitly addressed by the service.

List of CRL Entries

This attribute is composed of the following list elements:

CREP

This static attribute contains the local identifier of this CR. This attribute shall be set accordingly with the Load CRL service.

CR Type

This static attribute indicates the type of this CR. This attribute is dependent of the AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service. For each AR Type the correspondent CRL entries shall be set.

Table 367 shows the allowed values for this attribute.

Related AREP

This static attribute contains the local identifier for the related AR for this CR. This attribute shall be set accordingly with the Load CRL service.

S_SAP_index

This static attribute contains the identifier for the related Service Access Point of the local Data Link Layer. The value for this attribute has to be set correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service.

Table 367 shows the allowed values for this attribute.

Table 367 – S_SAP_index

AR Type	CR Type	S_SAP_index	Remark
MS0	QBCB	62	mandatory for the MS0 AR
MS2	QQCB-CO	50	mandatory for the MS2 AR
MS2	QQUU-OO	50	mandatory for the MS2 AR
MM1	QQCB-CL	54	mandatory for the MM1 AR
MM2	QQUU-OM	54	mandatory for the MM2 AR

D_SAP_index

This static attribute contains the identifier for the related Service Access Point of the remote Data Link Layer. The value for this attribute has to be set correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR. This attribute shall be set accordingly with the Load CRL service.

Table 368 shows the allowed values for this attribute.

Table 368 – D_SAP_index

AR Type	CR Type	D_SAP_index	Remark
MS0	QBCB	55, 56, 57, 59, 60	mandatory for the MS0 AR
MS2	QQCB-CO	0 to 48	mandatory for the MS2 AR
MS2	QQUU-OO	49	mandatory for the MS2 AR
MM1	QQCB-CL	54	mandatory for the MM1 AR
MM2	QQUU-OM	54	mandatory for the MM2 AR

D_addr

This static attribute contains the DL-address of the remote communication partner. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 369 shows the allowed values for this attribute used with the Load CRL service.

Table 369 – D_addr

AR Type	CR Type	S_SAP_index	D_addr as loaded with the Load CRL service
MS0	QBCB	62	0 to 126
MS2	QQCB-CO	50	0 to 126
MS2	QQUU-OO	50	0 to 126
MM1	QQCB-CL	54	0 to 125
MM2	QQUU-OM	54	0 to 125, 127
where 0 to 125, 127 means that the attribute D_addr of the local Service Access Point shall be set with the corresponding remote DL-address			

Service_activate

This static attribute indicates the DL services which shall be activated for the S_SAP_index of this CRL entry.

The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 370 shows the allowed values for this attribute.

Table 370 – Service_activate

AR Type	CR Type	S_SAP_index	Service_activate
MS0	QBCB	62	SRD
MS2	QQCB-CO	50	SRD
MS2	QQUU-OO	50	SRD
MM1	QQCB-CL	54	SRD
MM2	QQUU-OM	54	SDN

Role_in_service

This static attribute indicates the role which shall be activated for the S_SAP_index of this CRL entry. The following value is specified:

- Initiator: The station initiates the respective service exclusively

The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type of this CR and to the attribute AR Type of the related AR.

Table 371 shows the allowed values for this attribute.

Table 371 – Role_in_service

AR Type	CR Type	S_SAP_index	Role_in_service
MS0	QBCB	62	Initiator
MS2	QQCB-CO	50	Initiator
MS2	QQUU-OO	50	Initiator
MM1	QQCB-CL	54	Initiator
MM2	QQUU-OM	54	Initiator

Max_DLSDU_length_req_low

This static attribute indicates the maximum length for the DLSDU with low priority as requester. The value for this attribute has to be set with the Load CRL service correspondent to the attributes CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 372 shows the allowed values for this attribute.

Table 372 – Max_DLSDU_length_req_low

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_req_low
MS0	QBCB	55, 56, 57, 59, 60	0
MS2	QQCB-CO	0 to 48	4 to 244
MS2	QQUU-OO	49	48 to 64
MM1	QQCB-CL	54	4 to 244

MM2	QQUU-OM	54	2
-----	---------	----	---

Max_DLSDU_length_req_high

This static attribute indicates the maximum length for the DLSDU with high priority as requester. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 373 shows the allowed values for this attribute.

Table 373 – Max_DLSDU_length_req_high

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_req_high
MS0	QBCB	55	4 to 244
MS0	QBCB	56, 57, 59, 60	0
MS2	QQCB-CO	0 to 48	0
MS2	QQUU-OO	49	0
MM1	QQCB-CL	54	0
MM2	QQUU-OM	54	0

Max_DLSDU_length_ind_low

This static attribute indicates the maximum length for the DLSDU with low priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 374 shows the allowed values for this attribute.

Table 374 – Max_DLSDU_length_ind_low

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_ind_low
MS0	QBCB	55	0
MS0	QBCB	56, 57	0 to 244
MS0	QBCB	59	1 to 244
MS0	QBCB	60	6 to 244
MS2	QQCB-CO	0 to 48	48 to 244
MS2	QQUU-OO	49	4
MM1	QQCB-CL	54	4 to 244
MM2	QQUU-OM	54	0

Max_DLSDU_length_ind_high

This static attribute indicates the maximum length for the DLSDU with high priority for indication. The value for this attribute has to be set with the Load CRL service correspondent to the attribute CR Type and D_SAP_index of this CR and to the attribute AR Type of the related AR.

Table 375 shows the allowed values for this attribute.

Table 375 – Max_DLSDU_length_ind_high

AR Type	CR Type	D_SAP_index	Max_DLSDU_length_ind_high
MS0	QBCB	55, 56, 57, 59, 60	0
MS2	QQCB-CO	0 to 48	0
MS2	QQUU-OO	49	0
MM1	QQCB-CL	54	0
MM2	QQUU-OM	54	0

CR Name

This static attribute contains the name of the related CRL Entry. This attribute shall be set accordingly with the Load CRL service.

8.2.10.4.3.3 Invocation of CRL DP-master (class 2) object

For the invocation of the CRL DP-master (Class 2) object the following rule has to be considered:

- Only one CRL DP-master (Class 2) object shall be invoked in a DP-master (Class 2).

8.2.10.5 AR service specification

8.2.10.5.1 DLL init DP-slave

This service is used to initialise the data link layer of a DP-slave.

Table 376 shows the parameters of the service.

Table 376 – DLL init DP-slave

Parameter name	Req	Cnf
Argument	M	
Bus Para	M	
TS	M	
minTSDR	M	
Baudrate	M	
Slot Timer	M	
Result		M

Argument

The argument shall convey the service specific parameters of the service request.

Bus Para

This parameter is composed of the following subparameters:

TS

This parameter contains the DL-address of this DP-slave.

minTSDR

This parameter contains the smallest station delay time of this DP-slave.

Baudrate

This parameter contains the baudrate of this DP-slave.

Slot Time

This parameter contains the slot time of this DP-slave.

Result

This parameter indicates that the service request succeeded.

8.2.10.5.2 Load ARL DP-slave

The service given in Table 377 is used to load the ARL of a DP-slave. For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-slave object.

Table 377 – Load ARL DP-slave

Parameter name	Req	Cnf
Argument	M	
Min Send Timeout	C	
List of Resource Manager Entries	C	
API	C	
SCL	C	
Max Len Data Unit	C	
Send Timeout	C	
AREP	C	
S_SAP_index	C	
In Use	C	
List of S_SAP_indices	C	
S_SAP_index	C	
List of ARL Entries	M	

Parameter name	Req	Cnf
AREP	M	
AR Type	M	
List of related CRL Entries	M	
CREP	M	
Prm Req	C	
Prm Fault	C	
Cfg Fault	C	
Not Supported	C	
Station Not Ready	C	
Stat Diag	C	
Master Add	C	
Sync Supported	C	
Sync Mode	C	
Freeze Supported	C	
Freeze Mode	C	
WD Enabled	C	
WD Fact 1	C	
WD Fact 2	C	
Min TSDR	C	
Ident Number	C	
Group Identifier	C	
DPV1 Supported	C	
DPV1 Enabled	C	
Fail Safe Required	C	
Fail Safe	C	
WD Base	C	
List of Real Cfg Data	M	
Cfg Identifier	S	
Type	M	
Format	M	
Consistency	M	
Length	M	
Special Cfg Identifier	S	
Type	M	
Input Length	C	
Input Format	C	
Input Consistency	C	
Output Length	C	
Output Format	C	
Output Consistency	C	
List of Data Description	U	
Data Description	M	
Manufacturer Specific Data	U	
No Add Change	U	
Publisher Supported	C	
Isochronous Mode Supported	C	
Isochronous Mode	C	
Prm Command Supported	C	
Pull Plug Alarm supported	C	
Pull Plug Alarm enabled	C	
Process Alarm supported	C	
Process Alarm enabled	C	
Diagnosis Alarm supported	C	
Diagnosis Alarm enabled	C	
Manufacturer Specific Alarm supported	C	
Manufacturer Specific Alarm enabled	C	
Status Alarm supported	C	
Status Alarm enabled	C	
Update Alarm supported	C	
Update Alarm enabled	C	
Alarm Mode supported	C	

Parameter name	Req	Cnf
Alarm Mode enabled	C	
Clock Sync Interval	C	
CS Delay Time	C	
AR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Min Send Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

List of Resource Manager Entries

This parameter is composed of the following list elements:

API

This parameter contains the value for the equivalent attribute of the ARL entry.

SCL

This parameter contains the value for the equivalent attribute of the ARL entry.

Max Len Data Unit

This parameter contains the value for the equivalent attribute of the ARL entry.

Send Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

AREP

This parameter contains the value for the equivalent attribute of the ARL entry.

S_SAP_index

This parameter contains the value for the equivalent attribute of the ARL entry.

In Use

This parameter contains the value for the equivalent attribute of the ARL entry.

List of S_SAP_indices

This parameter is composed of the following list elements:

S_SAP_index

This parameter contains the value for the equivalent attribute of the ARL entry.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value for the equivalent attribute of the ARL entry.

List of related CRL Entries

This attribute is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the ARL entry.

Prm Req

This parameter contains the value for the equivalent attribute of the ARL entry.

Prm Fault

This parameter contains the value for the equivalent attribute of the ARL entry.

Cfg Fault

This parameter contains the value for the equivalent attribute of the ARL entry.

Not Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Station Not Ready

This parameter contains the value for the equivalent attribute of the ARL entry.

Stat Diag

This parameter contains the value for the equivalent attribute of the ARL entry.

Master Add

This parameter contains the value for the equivalent attribute of the ARL entry.

Sync Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Sync Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

Freeze Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Freeze Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

WD Enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

WD Fact 1, WD Fact 2

This parameter contains the value for the equivalent attribute of the ARL entry.

Min TSDR

This parameter contains the value for the equivalent attribute of the ARL entry.

Ident Number

This parameter contains the value for the equivalent attribute of the ARL entry.

Group Identifier

This parameter contains the value for the equivalent attribute of the ARL entry.

DPV1 Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

DPV1 Enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Fail Safe Required

This parameter contains the value for the equivalent attribute of the ARL entry.

Fail Safe

This parameter contains the value for the equivalent attribute of the ARL entry.

WD Base

This parameter contains the value for the equivalent attribute of the ARL entry.

List of Real Cfg Data

This parameter is composed of the following list elements:

Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value for the equivalent attribute of the ARL entry.

Format

This parameter contains the value for the equivalent attribute of the ARL entry.

Consistency

This parameter contains the value for the equivalent attribute of the ARL entry.

Length

This parameter contains the value for the equivalent attribute of the ARL entry.

Special Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value for the equivalent attribute of the ARL entry.

Input Length

This parameter contains the value for the equivalent attribute of the ARL entry.

Input Format

This parameter contains the value for the equivalent attribute of the ARL entry.

Input Consistency

This parameter contains the value for the equivalent attribute of the ARL entry.

Output Length

This parameter contains the value for the equivalent attribute of the ARL entry.

Output Format

This parameter contains the value for the equivalent attribute of the ARL entry.

Output Consistency

This parameter contains the value for the equivalent attribute of the ARL entry.

List of Data Description

This attribute is composed of the following list elements:

Data Description

This parameter contains the value for the equivalent attribute of the ARL entry.

Manufacturer Specific Data

This parameter contains the value for the equivalent attribute of the ARL entry.

No Add Change

This parameter contains the value for the equivalent attribute of the ARL entry.

Publisher Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Isochronous Mode Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Isochronous Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

Prm Command Supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Pull Plug Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Pull Plug Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Process Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Process Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Diagnosis Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Diagnosis Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Manufacturer Specific Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Manufacturer Specific Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Status Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Status Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Update Alarm supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Update Alarm enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Alarm Mode supported

This parameter contains the value for the equivalent attribute of the ARL entry.

Alarm Mode enabled

This parameter contains the value for the equivalent attribute of the ARL entry.

Clock Sync Interval

This parameter contains the value for the equivalent attribute of the ARL entry.

CS Delay Time

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value for the equivalent attribute of the ARL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.3 Get ARL DP-slave

The service given in Table 378 is used to get the ARL of a DP-slave. For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-slave object.

Table 378 – Get ARL DP-slave

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
Min Send Timeout		C
List of Resource Manager Entries		C
API		C
SCL		C
Max Len Data Unit		C
Send Timeout		C
AREP		C
S_SAP_index		C
In Use		C
List of S_SAP_indices		C
S_SAP_index		C
List of ARL Entries		M
AREP		M
AR Type		M
List of related CRL Entries		M
CREP		M
Prm Req		C
Prm Fault		C
Cfg Fault		C
Not Supported		C
Station Not Ready		C
Stat Diag		C
Master Add		C
Sync Supported		C
Sync Mode		C
Freeze Supported		C
Freeze Mode		C
WD Enabled		C
WD Fact 1		C
WD Fact 2		C
Min TSDR		C
Ident Number		C
Group Identifier		C
DPV1 Supported		C
DPV1 Enabled		C
Fail Safe Required		C
Fail Safe		C
WD Base		C
List of Real Cfg Data		M
Cfg Identifier		S
Type		M
Format		M
Consistency		M
Length		M
Special Cfg Identifier		S
Type		M
Input Length		C
Input Format		C
Input Consistency		C
Output Length		C
Output Format		C
Output Consistency		C
List of Data Description		U
Data Description		M
Manufacturer Specific Data		U
No Add Change		U
Publisher Supported		C

Parameter name	Req	Cnf
Isochronous Mode Supported		C
Isochronous Mode		C
Prm Command Supported		C
Pull Plug Alarm supported		C
Pull Plug Alarm enabled		C
Process Alarm supported		C
Process Alarm enabled		C
Diagnosis Alarm supported		C
Diagnosis Alarm enabled		C
Manufacturer Specific Alarm supported		C
Manufacturer Specific Alarm enabled		C
Status Alarm supported		C
Status Alarm enabled		C
Update Alarm supported		C
Update Alarm enabled		C
Alarm Mode supported		C
Alarm Mode enabled		C
Clock Sync Interval		C
CS Delay Time		C
AR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Min Send Timeout

This parameter contains the value of the equivalent attribute of the ARL entry.

List of Resource Manager Entries

This parameter is composed of the following list elements:

API

This parameter contains the value of the equivalent attribute of the ARL entry.

SCL

This parameter contains the value of the equivalent attribute of the ARL entry.

Max Len Data Unit

This parameter contains the value of the equivalent attribute of the ARL entry.

Send Timeout

This parameter contains the value of the equivalent attribute of the ARL entry.

AREP

This parameter contains the value of the equivalent attribute of the ARL entry.

S_SAP_index

This parameter contains the value of the equivalent attribute of the ARL entry.

In Use

This parameter contains the value of the equivalent attribute of the ARL entry.

List of S_SAP_indices

This parameter is composed of the following list elements:

S_SAP_index

This parameter contains the value of the equivalent attribute of the ARL entry.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value of the equivalent attribute of the ARL entry.

List of related CRL Entries

This attribute is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the ARL entry.

Prm Req

This parameter contains the value of the equivalent attribute of the ARL entry.

Prm Fault

This parameter contains the value of the equivalent attribute of the ARL entry.

Cfg Fault

This parameter contains the value of the equivalent attribute of the ARL entry.

Not Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Station Not Ready

This parameter contains the value of the equivalent attribute of the ARL entry.

Stat Diag

This parameter contains the value of the equivalent attribute of the ARL entry.

Master Add

This parameter contains the value of the equivalent attribute of the ARL entry.

Sync Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Sync Mode

This parameter contains the value of the equivalent attribute of the ARL entry.

Freeze Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Freeze Mode

This parameter contains the value of the equivalent attribute of the ARL entry.

WD Enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

WD Fact 1, WD Fact 2

This parameter contains the value of the equivalent attribute of the ARL entry.

Min TSDR

This parameter contains the value of the equivalent attribute of the ARL entry.

Ident Number

This parameter contains the value of the equivalent attribute of the ARL entry.

Group Identifier

This parameter contains the value of the equivalent attribute of the ARL entry.

DPV1 Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

DPV1 Enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Fail Safe Required

This parameter contains the value of the equivalent attribute of the ARL entry.

Fail Safe

This parameter contains the value of the equivalent attribute of the ARL entry.

WD Base

This parameter contains the value of the equivalent attribute of the ARL entry.

List of Real Cfg Data

This attribute is composed of the following list elements:

Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value of the equivalent attribute of the ARL entry.

Format

This parameter contains the value of the equivalent attribute of the ARL entry.

Consistency

This parameter contains the value of the equivalent attribute of the ARL entry.

Length

This parameter contains the value of the equivalent attribute of the ARL entry.

Special Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value of the equivalent attribute of the ARL entry.

Input Length

This parameter contains the value of the equivalent attribute of the ARL entry.

Input Format

This parameter contains the value of the equivalent attribute of the ARL entry.

Input Consistency

This parameter contains the value of the equivalent attribute of the ARL entry.

Output Length

This parameter contains the value of the equivalent attribute of the ARL entry.

Output Format

This parameter contains the value of the equivalent attribute of the ARL entry.

Output Consistency

This parameter contains the value of the equivalent attribute of the ARL entry.

List of Data Description

This attribute is composed of the following list elements:

Data Description

This parameter contains the value of the equivalent attribute of the ARL entry.

Manufacturer Specific Data

This parameter contains the value of the equivalent attribute of the ARL entry.

No Add Change

This parameter contains the value of the equivalent attribute of the ARL entry.

Publisher Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Isochronous Mode Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Isochronous Mode

This parameter contains the value of the equivalent attribute of the ARL entry.

Prm Command Supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Pull Plug Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Pull Plug Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Process Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Process Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Diagnosis Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Diagnosis Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Manufacturer Specific Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Manufacturer Specific Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Status Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Status Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Update Alarm supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Update Alarm enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Alarm Mode supported

This parameter contains the value of the equivalent attribute of the ARL entry.

Alarm Mode enabled

This parameter contains the value of the equivalent attribute of the ARL entry.

Clock Sync Interval

This parameter contains the value of the equivalent attribute of the ARL entry.

CS Delay Time

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value of the equivalent attribute of the ARL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.4 Set ARL isochronous mode

The service given in Table 379 is used to set the value of the attribute Isochronous Mode of the ARL of the DP-slave.

Table 379 – Set ARL isochronous mode

Parameter name	Req	Cnf
Argument	M	
Isochronous Mode	M	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Isochronous Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.5 Load ARL DP-master C11

The service given in Table 380 is used to load the ARL of a DP-master (Class 1). For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 1) object.

Table 380 – Load ARL DP-master C11

Parameter name	Req	Cnf
Argument	M	
Alarm Max	C	
List of ARL Entries	M	
AREP	M	
AR Type	M	
List of related CRL Entries	M	
CREP	M	
Error Action Flag	C	
Min Slave Interval	C	
Data Control Time	C	
Max User Global Control	C	
Max Diag Data Len	C	
Isochronous Mode supp	C	
Isochronous Mode	C	
IsoM Freeze	C	
IsoM Sync	C	
TCT	C	
maxTSH	C	
Extra Alarm SAP	C	
Max Channel Data Length	C	
Alarm Mode	C	
MS1 Timeout	C	
Poll Timeout	M	
Ident Number	M	
Time Device Type	C	
Clock Sync Interval	C	
CS Delay Time	C	
AR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Alarm Max

This parameter contains the value for the equivalent attribute of the ARL entry.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value for the equivalent attribute of the ARL entry.

List of related CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the ARL entry.

Error Action Flag

This parameter contains the value for the equivalent attribute of the ARL entry.

Min Slave Interval

This parameter contains the value for the equivalent attribute of the ARL entry.

Data Control Time

This parameter contains the value for the equivalent attribute of the ARL entry.

Max User Global Control

This parameter contains the value for the equivalent attribute of the ARL entry.

Max Diag Data Len

This parameter contains the value for the equivalent attribute of the ARL entry.

Isochronous Mode supp

This parameter contains the value for the equivalent attribute of the ARL entry.

Isochronous Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

IsoM Freeze

This parameter contains the value for the equivalent attribute of the ARL entry.

IsoM Synch

This parameter contains the value for the equivalent attribute of the ARL entry.

TCT

This parameter contains the value for the equivalent attribute of the ARL entry.

maxTSH

This parameter contains the value for the equivalent attribute of the ARL entry.

Extra Alarm SAP

This parameter contains the value for the equivalent attribute of the ARL entry.

Max Channel Data Length

This parameter contains the value for the equivalent attribute of the ARL entry.

Alarm Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

MS1 Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

Poll Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

Ident Number

This parameter contains the value for the equivalent attribute of the ARL entry.

Time Device Type

This parameter contains the value for the equivalent attribute of the ARL entry.

Clock Sync Interval

This parameter contains the value for the equivalent attribute of the ARL entry.

CS Delay Time

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value for the equivalent attribute of the ARL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.6 Get ARL DP-master CI1

The service given in Table 381 is used to get the ARL of a DP-master (Class 1). For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 1) object. For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 2) object.

Table 381 – Get ARL DP-master C11

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
Alarm Max		C
List of ARL Entries		M
AREP		M
AR Type		M
List of related CRL Entries		M
CREP		M
Error Action Flag		C
Min Slave Interval		C
Data Control Time		C
Max User Global Control		C
Max Diag Data Len		C
Isochronous Mode supp		C
Isochronous Mode		C
IsoM Freeze		C
IsoM Sync		C
TCT		C
maxTSH		C
Extra Alarm SAP		C
Max Channel Data Length		C
Alarm Mode		C
MS1 Timeout		C
Poll Timeout		M
Ident Number		M
Time Device Type		C
Clock Sync Interval		C
CS Delay Time		C
AR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Alarm Max

This parameter contains the value of the equivalent attribute of the ARL entry.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value of the equivalent attribute of the ARL entry.

List of related CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the ARL entry.

Error Action Flag

This parameter contains the value of the equivalent attribute of the ARL entry.

Min Slave Interval

This parameter contains the value of the equivalent attribute of the ARL entry.

Data Control Time

This parameter contains the value of the equivalent attribute of the ARL entry.

Max User Global Control

This parameter contains the value of the equivalent attribute of the ARL entry.

Max Diag Data Len

This parameter contains the value of the equivalent attribute of the ARL entry.

Isochronous Mode supp

This parameter contains the value of the equivalent attribute of the ARL entry.

Isochronous Mode

This parameter contains the value of the equivalent attribute of the ARL entry.

IsoM Freeze

This parameter contains the value of the equivalent attribute of the ARL entry.

IsoM Synch

This parameter contains the value of the equivalent attribute of the ARL entry.

TCT

This parameter contains the value of the equivalent attribute of the ARL entry.

maxTSH

This parameter contains the value of the equivalent attribute of the ARL entry.

Extra Alarm SAP

This parameter contains the value of the equivalent attribute of the ARL entry.

Max Channel Data Length

This parameter contains the value of the equivalent attribute of the ARL entry.

Alarm Mode

This parameter contains the value of the equivalent attribute of the ARL entry.

MS1 Timeout

This parameter contains the value of the equivalent attribute of the ARL entry.

Poll Timeout

This parameter contains the value of the equivalent attribute of the ARL entry.

Ident Number

This parameter contains the value of the equivalent attribute of the ARL entry.

Time Device Type

This parameter contains the value of the equivalent attribute of the ARL entry.

Clock Sync Interval

This parameter contains the value of the equivalent attribute of the ARL entry.

CS Delay Time

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value of the equivalent attribute of the ARL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.7 ARL slave update DP-master CI1

The service given in Table 382 is used to update the ARL of a DP-master (Class 1) concerning the entries of one DP-slave. For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 1) object.

Table 382 – ARL Slave update DP-master C11

Parameter name	Req	Cnf
Argument	M	
List of ARL Entries	M	
AREP	M	
AR Type	M	
List of related CRL Entries	M	
CREP	M	
Extra Alarm SAP	C	
Max Channel Data Length	C	
Alarm Mode	C	
MS1 Timeout	C	
Time Device Type	C	
Clock Sync Interval	C	
CS Delay Time	C	
AR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value for the equivalent attribute of the ARL entry.

List of related CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the ARL entry.

Extra Alarm SAP

This parameter contains the value for the equivalent attribute of the ARL entry.

Max Channel Data Length

This parameter contains the value for the equivalent attribute of the ARL entry.

Alarm Mode

This parameter contains the value for the equivalent attribute of the ARL entry.

MS1 Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

Time Device Type

This parameter contains the value for the equivalent attribute of the ARL entry.

Clock Sync Interval

This parameter contains the value for the equivalent attribute of the ARL entry.

CS Delay Time

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value for the equivalent attribute of the ARL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.8 Load ARL DP-master C12

The service given in Table 383 is used to load the ARL of a DP-master (Class 2). For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 2) object.

Table 383 – Load ARL DP-master C12

Parameter name	Req	Cnf
Argument	M	
List of ARL Entries	M	
AREP	M	
AR Type	M	
List of related CRL Entries	M	
CREP	M	
Poll Timeout	M	
AR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value for the equivalent attribute of the ARL entry.

List of related CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the ARL entry.

Poll Timeout

This parameter contains the value for the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value for the equivalent attribute of the ARL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.9 Get ARL DP-master CI2

The service given in Table 384 is used to get the ARL of a DP-master (Class 2). For the use of the conditional parameters in the request primitive refer to the description of the ARL DP-master (Class 2) object.

Table 384 – Get ARL DP-master CI2

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
List of ARL Entries		M
AREP		M
AR Type		M
List of related CRL Entries		M
CREP		M
Poll Timeout		M
AR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of ARL Entries

This parameter is composed of the following list elements:

AREP

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Type

This parameter contains the value of the equivalent attribute of the ARL entry.

List of related CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the ARL entry.

Poll Timeout

This parameter contains the value of the equivalent attribute of the ARL entry.

AR Name

This parameter contains the value of the equivalent attribute of the ARL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.10 Load CRL DP-slave

The service given in Table 385 is used to load the CRL of a DP-slave. For the presence of the conditional parameters in the indication primitive refer to the description of the CRL DP-slave object.

Table 385 – Load CRL DP-slave

Parameter name	Req	Cnf
Argument	M	
List of CRL Entries	M	
CREP	M	
CR Type	M	
Related AREP	M	
S_SAP_index	M	
D_addr	M	
Service_activate	M	
Role_in_Service	M	
Indication_mode	M	
Max_DLSDU_length_req_low	M	
Max_DLSDU_length_req_high	M	
Max_DLSDU_length_ind_low	M	
Max_DLSDU_length_ind_high	M	
List of DXB Link Entries		
Publisher Address	M	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Length	M	
CR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value for the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value for the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value for the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value for the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value for the equivalent attribute of the CRL entry.

Role_in_Service

This parameter contains the value for the equivalent attribute of the CRL entry.

Indication_mode

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value for the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Length

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.11 Load CRL DXB link entries

The service given in Table 386 is used to load the DXB Link Entries for the BBUU-OM CRL entries of a DP-slave. This service is requested by the application of the DP-slave after the reception of the attribute values of its DXB-Linktable object. The DP Application Layer may acknowledge this service negatively if the application has already loaded a DXB-Linktable and the Subscriber State-Machine has been started. This service shall only be used in conjunction with the BBUU-OM CR.

Table 386 – Load CRL DXB link entries

Parameter name	Req	Cnf
Argument	M	
CREP	M	
List of DXB Link Entries	M	
Publisher Address	M	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Length	M	
Result(+)		S
CREP		M
Result(-)		S
CREP		M

Argument

The argument shall convey the service specific parameters of the service request.

CREP

This parameter is used as local identifier of a BBUU-OM CR.

List of DXB Link Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value for the attribute Publisher Address of one BBUU-OM CRL entry.

Input Data Length Publisher

This parameter contains the value for the attribute Input Data Length Publisher of one BBUU-OM CRL entry.

Source Offset Publisher

This parameter contains the value for the attribute Source Offset Publisher of one BBUU-OM CRL entry.

Length

This parameter contains the value for the attribute Length of one DXB-Linktable entry of one BBUU-OM CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

8.2.10.5.12 Get CRL DP-slave

The service given in Table 387 is used to get the CRL of a DP-slave. For the presence of the conditional parameters in the indication primitive refer to the description of the CRL DP-slave object.

Table 387 – Get CRL DP-slave

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
List of CRL Entries		M
CREP		M
CR Type		M
Related AREP		M
S_SAP_index		M
D_addr		M
Service_activate		M
Role_in_Service		M
Indication_mode		M
Max_DLSDU_length_req_low		M
Max_DLSDU_length_req_high		M
Max_DLSDU_length_ind_low		M
Max_DLSDU_length_ind_high		M
List of DXB Link Entries		U
Publisher Address		M
Input Data Length Publisher		M
Source Offset Publisher		M
Length		M
CR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value of the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value of the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value of the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value of the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value of the equivalent attribute of the CRL entry.

Role_in_Service

This parameter contains the value of the equivalent attribute of the CRL entry.

Indication_mode

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value of the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter is composed of the following list elements:

Publisher Address

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Length

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value of the equivalent attribute of the CRL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.13 Load CRL DP-master C11

The service given in Table 388 is used to load the CRL of a DP-master (Class 1). For the use of the conditional parameters in the request primitive refer to the description of the CRL DP-master (Class 1) object.

Table 388 – Load CRL DP-master C11

Parameter name	Req	Cnf
Argument	M	
Update	M	
List of CRL Entries	M	
CREP	M	
CR Type	M	
Related AREP	M	
S_SAP_index	M	
D_SAP_index	M	
D_addr	M	
Service_activate	M	
Role_in_service	M	
Indication_mode	M	
Max_DLSDU_length_req_low	M	
Max_DLSDU_length_req_high	M	
Max_DLSDU_length_ind_low	M	
Max_DLSDU_length_ind_high	M	
Active	C	
New Prm	C	
Prm Data	C	
Lock	M	
Sync	M	
Freeze	M	
WD On	M	
WD Fact 1	M	
WD Fact 2	M	
Min Tsdr	M	
Ident Number	M	
Group Ident	M	
DPV1 Enable	C	
Fail Safe	C	
Enable Publisher	C	
WD Base	C	
Enable Pull Plug Alarm	C	
Enable Process Alarm	C	
Enable Diagnosis Alarm	C	
Enable Manufacturer Specific Alarm	C	
Enable Status Alarm	C	
Enable Update Alarm	C	
Check Cfg Mode	C	
Alarm Mode	C	
Prm Structure	M	
User Prm Data	C	
Device Prm Data	U	
Module Prm Data	U	
List of Structured Prm Data	C	
Structured User Prm Data	U	
Device Related User Parameter	U	
Device Prm Data	M	
Module Related User Parameter	U	
Slot Number	M	
Module Prm Data	M	
List of DXB Link Entries	U	
Publisher Address	M	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Length	M	
IsoM Parameter	U	
TBASE_DP	M	
TDP	M	

Table 388 (continued)

Parameter name	Req	Cnf
T _{MAPC}	M	
T _{BASE_IO}	M	
T _I	M	
T _O	M	
T _{PLL_W}	M	
T _{PLL_D}	M	
List of DXB Subscriber Entries	U	
Publisher Address	C	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Master Data	C	
Source Offset Master	M	
Dest Slot Number	M	
Offset Data Area	M	
Length Data Area	M	
Prm Command	U	
Seq Number	M	
Primary Request	M	
MS1 Command	M	
Master State Clear	M	
Check Properties	M	
Primary Request used	M	
MS1 Command used	M	
Address Change	M	
Address Offset	M	
Output Hold Time	M	
Time AR Parameter	U	
Clock Sync Interval	M	
CS Delay Time	M	
IsoM Req	U	
List of Cfg Data	C	
Cfg Identifier	S	
Type	M	
Format	M	
Consistency	M	
Length	M	
Special Cfg Identifier	S	
Type	M	
Input Length	C	
Input Format	C	
Input Consistency	C	
Output Length	C	
Output Format	C	
Output Consistency	C	
List of Data Description	U	
Data Description	M	
Manufacturer Specific Data	U	
Diag Upd Delay	C	
Ext Diag Overflow	C	
Deactivated	C	
Station Non Existent	C	
Invalid Slave Response	C	
Fail Safe	C	
DPV1 Supported	C	
NA to Abort	C	
Ignore ACIr	C	
Output Len	C	
Exp Inp Len	C	
Data Transfer Status	C	

Table 388 (continued)

Parameter name	Req	Cnf
System Diagnosis Status	C	
Publisher Flag	C	
List of Structured Prm Data	C	
Structured User Prm Data	U	
Device Related User Parameter	U	
Device Prm Data	M	
Module Related User Parameter	U	
Slot Number	M	
Module Prm Data	M	
List of DXB Link Entries	U	
Publisher Address	M	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Length	M	
IsoM Parameter	U	
TBASE_DP	M	
TDP	M	
TMAPC	M	
TBASE_IO	M	
T _I	M	
T _O	M	
TPLL_W	M	
TPLL_D	M	
List of DXB Subscriber Entries	U	
Publisher Address	C	
Input Data Length Publisher	M	
Source Offset Publisher	M	
Master Data	C	
Source Offset Master	M	
Dest Slot Number	M	
Offset Data Area	M	
Length Data Area	M	
Time AR Parameter	U	
Clock Sync Interval	M	
CS Delay Time	M	
CR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Update

This parameter set to FALSE indicates that the complete CRL shall be loaded. This parameter set to TRUE indicates that only the CRL entries of one DP-slave shall be updated.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value for the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value for the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value for the equivalent attribute of the CRL entry.

D_SAP_index

This parameter contains the value for the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value for the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value for the equivalent attribute of the CRL entry.

Role_in_service

This parameter contains the value for the equivalent attribute of the CRL entry.

Indication_mode

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value for the equivalent attribute of the CRL entry.

Active

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

New Prm

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

Prm Data

This conditional parameter is composed of the following subparameters. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

Lock

This parameter contains the value for the equivalent attribute of the CRL entry.

Sync

This parameter contains the value for the equivalent attribute of the CRL entry.

Freeze

This parameter contains the value for the equivalent attribute of the CRL entry.

WD On

This parameter contains the value for the equivalent attribute of the CRL entry.

WD Fact 1

This parameter contains the value for the equivalent attribute of the CRL entry.

WD Fact 2

This parameter contains the value for the equivalent attribute of the CRL entry.

Min Tsdr

This parameter contains the value for the equivalent attribute of the CRL entry.

Ident Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Group Ident

This parameter contains the value for the equivalent attribute of the CRL entry.

DPV1 Enable

This parameter contains the value for the equivalent attribute of the CRL entry.

Fail Safe

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

WD Base

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Pull Plug Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Process Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Diagnosis Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Manufacturer Specific Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Status Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Enable Update Alarm

This parameter contains the value for the equivalent attribute of the CRL entry.

Check Cfg Mode

This parameter contains the value for the equivalent attribute of the CRL entry.

Alarm Mode

This parameter contains the value for the equivalent attribute of the CRL entry.

Prm Structure

This parameter contains the value for the equivalent attribute of the CRL entry.

User Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value FALSE. This parameter is composed of one or both of the following subparameters.

Device Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

List of Structured Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value TRUE. This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter indicates the presence of the DXB-Linktable and is composed of the following list elements.

Publisher Address

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Length

This parameter contains the value for the equivalent attribute of the CRL entry.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value for the equivalent attribute of the CRL entry.

TDP

This parameter contains the value for the equivalent attribute of the CRL entry.

TMAPC

This parameter contains the value for the equivalent attribute of the CRL entry.

TBASE_IO

This parameter contains the value for the equivalent attribute of the CRL entry.

T_I

This parameter contains the value for the equivalent attribute of the CRL entry.

T_O

This parameter contains the value for the equivalent attribute of the CRL entry.

TPLL_W

This parameter contains the value for the equivalent attribute of the CRL entry.

TPLL_D

This parameter contains the value for the equivalent attribute of the CRL entry.

List of DXB Subscriber Entries

This parameter contains the value for the equivalent attribute of the CRL entry.

Publisher Address

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Master Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Master

This parameter contains the value for the equivalent attribute of the CRL entry.

Dest Slot Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Offset Data Area

This parameter contains the value for the equivalent attribute of the CRL entry.

Length Data Area

This parameter contains the value for the equivalent attribute of the CRL entry.

Prm Command

This parameter contains the value for the equivalent attribute of the CRL entry.

Seq Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Primary Request

This parameter contains the value for the equivalent attribute of the CRL entry.

MS1 Command

This parameter contains the value for the equivalent attribute of the CRL entry.

Master State Clear

This parameter contains the value for the equivalent attribute of the CRL entry.

Check Properties

This parameter contains the value for the equivalent attribute of the CRL entry.

Primary Request used

This parameter contains the value for the equivalent attribute of the CRL entry.

MS1 Command used

This parameter contains the value for the equivalent attribute of the CRL entry.

Address Change

This parameter contains the value for the equivalent attribute of the CRL entry.

Address Offset

This parameter contains the value for the equivalent attribute of the CRL entry.

Output Hold Time

This parameter contains the value for the equivalent attribute of the CRL entry.

Time AR Parameter

This parameter contains the value for the equivalent attribute of the CRL entry.

Clock Sync Interval

This parameter contains the value for the equivalent attribute of the CRL entry.

CS Delay Time

This parameter contains the value for the equivalent attribute of the CRL entry.

IsoM Req

This parameter contains the value for the equivalent attribute of the CRL entry.

List of Cfg Data

This conditional parameter is composed of the following list elements. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 62.

Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value for the equivalent attribute of the CRL entry.

Format

This parameter contains the value for the equivalent attribute of the CRL entry.

Consistency

This parameter contains the value for the equivalent attribute of the CRL entry.

Length

This parameter contains the value for the equivalent attribute of the CRL entry.

Special Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Length

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Format

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Consistency

This parameter contains the value for the equivalent attribute of the CRL entry.

Output Length

This parameter contains the value for the equivalent attribute of the CRL entry.

Output Format

This parameter contains the value for the equivalent attribute of the CRL entry.

Output Consistency

This parameter contains the value for the equivalent attribute of the CRL entry.

List of Data Description

This parameter is composed of the following list elements:

Data Description

This parameter contains the value for the equivalent attribute of the CRL entry.

Manufacturer Specific Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Diag Upd Delay

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Ext Diag Overflow

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Deactivated

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Station Non Existent

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Invalid Slave Response

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Fail Safe

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

DPV1 Supported

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

NA to Abort

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Ignore ACIr

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Output Len

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Exp Inp Len

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Data Transfer Status

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

System Diagnosis Status

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Publisher Flag

This conditional parameter contains the value for the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

List of Structured Prm Data

This conditional parameter is only present if the parameter CR Type contains the value BQUU and the parameter D_SAP_index contains the value 53. This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter indicates the presence of the DXB-Linktable and is composed of the following list elements.

Publisher Address

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Length

This parameter contains the value for the equivalent attribute of the CRL entry.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value for the equivalent attribute of the CRL entry.

TDP

This parameter contains the value for the equivalent attribute of the CRL entry.

TMAPC

This parameter contains the value for the equivalent attribute of the CRL entry.

TBASE_IO

This parameter contains the value for the equivalent attribute of the CRL entry.

T_I

This parameter contains the value for the equivalent attribute of the CRL entry.

T_O

This parameter contains the value for the equivalent attribute of the CRL entry.

TPLL_W

This parameter contains the value for the equivalent attribute of the CRL entry.

TPLL_D

This parameter contains the value for the equivalent attribute of the CRL entry.

List of DXB Subscriber Entries

This parameter contains the value for the equivalent attribute of the CRL entry.

Publisher Address

This parameter contains the value for the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value for the equivalent attribute of the CRL entry.

Master Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Source Offset Master

This parameter contains the value for the equivalent attribute of the CRL entry.

Dest Slot Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Offset Data Area

This parameter contains the value for the equivalent attribute of the CRL entry.

Length Data Area

This parameter contains the value for the equivalent attribute of the CRL entry.

Time AR Parameter

This parameter contains the value for the equivalent attribute of the CRL entry.

Clock Sync Interval

This parameter contains the value for the equivalent attribute of the CRL entry.

CS Delay Time

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.14 Get CRL DP-master CI1

The service given in Table 389 is used to get the CRL of a DP-master (Class 1). For the use of the conditional parameters in the request primitive refer to the description of the CRL DP-master (Class 1) object.

Table 389 – Get CRL DP-master CI1

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
List of CRL Entries		M
CREP		M
CR Type		M
Related AREP		M
S_SAP_index		M
D_SAP_index		M
D_addr		M
Service_activate		M
Role_in_service		M
Indication_mode		M
Max_DLSDU_length_req_low		M
Max_DLSDU_length_req_high		M
Max_DLSDU_length_ind_low		M
Max_DLSDU_length_ind_high		M
Active		C
New Prm		C
Prm Data		C
Lock		M
Sync		M
Freeze		M
WD On		M
WD Fact 1		M
WD Fact 2		M
Min Tsdr		M
Ident Number		M
Group Ident		M
DPV1 Enable		C
Fail Safe		C
Enable Publisher		C
WD Base		C
Enable Pull Plug Alarm		C
Enable Process Alarm		C
Enable Diagnosis Alarm		C
Enable Manufacturer Specific Alarm		C
Enable Status Alarm		C
Enable Update Alarm		C
Check Cfg Mode		C
Alarm Mode		C
Prm Structure		M

Table 389 (continued)

Parameter name	Req	Cnf
User Prm Data		C
Device Prm Data		U
Module Prm Data		U
List of Structured Prm Data		C
Structured User Prm Data		U
Device Related User Parameter		U
Device Prm Data		M
Module Related User Parameter		U
Slot Number		M
Module Prm Data		M
List of DXB Link Entries		U
Publisher Address		M
Input Data Length Publisher		M
Source Offset Publisher		M
Length		M
IsoM Parameter		U
TBASE_DP		M
TDP		M
TMAPC		M
TBASE_IO		M
T _I		M
T _O		M
TPLL_W		M
TPLL_D		M
List of DXB Subscriber Entries		U
Publisher Address		C
Input Data Length Publisher		M
Source Offset Publisher		M
Master Data		C
Source Offset Master		M
Dest Slot Number		M
Offset Data Area		M
Length Data Area		M
Prm Command		U
Seq Number		M
Primary Request		M
MS1 Command		M
Master State Clear		M
Check Properties		M
Primary Request used		M
MS1 Command used		M
Address Change		M
Address Offset		M
Output Hold Time		M
Time AR Parameter		U
Clock Sync Interval		M
CS Delay Time		M
IsoM Req		U
List of Cfg Data		C
Cfg Identifier		S
Type		M
Format		M
Consistency		M
Length		M
Special Cfg Identifier		S
Type		M
Input Length		C

Table 389 (continued)

Parameter name	Req	Cnf
Input Format		C
Input Consistency		C
Output Length		C
Output Format		C
Output Consistency		C
List of Data Description		U
Data Description		M
Manufacturer Specific Data		U
Diag Upd Delay		C
Ext Diag Overflow		C
Deactivated		C
Station Non Existent		C
Invalid Slave Response		C
Fail Safe		C
DPV1 Supported		C
NA to Abort		C
Ignore ACIr		C
Output Len		C
Exp Inp Len		C
Data Transfer Status		C
System Diagnosis Status		C
Publisher Flag		C
List of Structured Prm Data		C
Structured User Prm Data		U
Device Related User Parameter		U
Device Prm Data		M
Module Related User Parameter		U
Slot Number		M
Module Prm Data		M
List of DXB Link Entries		U
Publisher Address		M
Input Data Length Publisher		M
Source Offset Publisher		M
Length		M
IsoM Parameter		U
TBASE_DP		M
TDP		M
TMAPC		M
TBASE_IO		M
TI		M
TO		M
TPLL_W		M
TPLL_D		M
List of DXB Subscriber Entries		U
Publisher Address		C
Input Data Length Publisher		M
Source Offset Publisher		M
Master Data		C
Source Offset Master		M
Dest Slot Number		M
Offset Data Area		M
Length Data Area		M
Time AR Parameter		U
Clock Sync Interval		M
CS Delay Time		M
CR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value of the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value of the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value of the equivalent attribute of the CRL entry.

D_SAP_index

This parameter contains the value of the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value of the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value of the equivalent attribute of the CRL entry.

Role_in_service

This parameter contains the value of the equivalent attribute of the CRL entry.

Indication_mode

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value of the equivalent attribute of the CRL entry.

Active

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

New Prm

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

Prm Data

This conditional parameter is composed of the following subparameters. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 61.

Lock

This parameter contains the value of the equivalent attribute of the CRL entry.

Sync

This parameter contains the value of the equivalent attribute of the CRL entry.

Freeze

This parameter contains the value of the equivalent attribute of the CRL entry.

WD On

This parameter contains the value of the equivalent attribute of the CRL entry.

WD Fact 1

This parameter contains the value of the equivalent attribute of the CRL entry.

WD Fact 2

This parameter contains the value of the equivalent attribute of the CRL entry.

Min Tsdr

This parameter contains the value of the equivalent attribute of the CRL entry.

Ident Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Group Ident

This parameter contains the value of the equivalent attribute of the CRL entry.

DPV1 Enable

This parameter contains the value of the equivalent attribute of the CRL entry.

Fail Safe

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

WD Base

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Pull Plug Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Process Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Diagnosis Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Manufacturer Specific Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Status Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Enable Update Alarm

This parameter contains the value of the equivalent attribute of the CRL entry.

Check Cfg Mode

This parameter contains the value of the equivalent attribute of the CRL entry.

Alarm Mode

This parameter contains the value of the equivalent attribute of the CRL entry.

Prm Structure

This parameter contains the value of the equivalent attribute of the CRL entry.

User Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value FALSE. This parameter is composed of one or both of the following subparameters.

Device Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

List of Structured Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value TRUE. This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter indicates the presence of the DXB-Linktable and is composed of the following list elements.

Publisher Address

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Length

This parameter contains the value of the equivalent attribute of the CRL entry.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value of the equivalent attribute of the CRL entry.

TDP

This parameter contains the value of the equivalent attribute of the CRL entry.

TMAPC

This parameter contains the value of the equivalent attribute of the CRL entry.

TBASE_IO

This parameter contains the value of the equivalent attribute of the CRL entry.

T_I

This parameter contains the value of the equivalent attribute of the CRL entry.

T_O

This parameter contains the value of the equivalent attribute of the CRL entry.

TPLL_W

This parameter contains the value of the equivalent attribute of the CRL entry.

TPLL_D

This parameter contains the value of the equivalent attribute of the CRL entry.

List of DXB Subscriber Entries

This parameter contains the value of the equivalent attribute of the CRL entry.

Publisher Address

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Master Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Master

This parameter contains the value of the equivalent attribute of the CRL entry.

Dest Slot Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Offset Data Area

This parameter contains the value of the equivalent attribute of the CRL entry.

Length Data Area

This parameter contains the value of the equivalent attribute of the CRL entry.

Prm Command

This parameter contains the value of the equivalent attribute of the CRL entry.

Seq Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Primary Request

This parameter contains the value of the equivalent attribute of the CRL entry.

MS1 Command

This parameter contains the value of the equivalent attribute of the CRL entry.

Master State Clear

This parameter contains the value of the equivalent attribute of the CRL entry.

Check Properties

This parameter contains the value of the equivalent attribute of the CRL entry.

Primary Request used

This parameter contains the value of the equivalent attribute of the CRL entry.

MS1 Command used

This parameter contains the value of the equivalent attribute of the CRL entry.

Address Change

This parameter contains the value of the equivalent attribute of the CRL entry.

Address Offset

This parameter contains the value of the equivalent attribute of the CRL entry.

Output Hold Time

This parameter contains the value of the equivalent attribute of the CRL entry.

Time AR Parameter

This parameter contains the value of the equivalent attribute of the CRL entry.

Clock Sync Interval

This parameter contains the value of the equivalent attribute of the CRL entry.

CS Delay Time

This parameter contains the value of the equivalent attribute of the CRL entry.

IsoM Req

This parameter contains the value of the equivalent attribute of the CRL entry.

List of Cfg Data

This conditional parameter is composed of the following list elements. It shall only be present if the parameter CREP has the value BQUU and the parameter D_SAP_index has the value 62.

Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value of the equivalent attribute of the CRL entry.

Format

This parameter contains the value of the equivalent attribute of the CRL entry.

Consistency

This parameter contains the value of the equivalent attribute of the CRL entry.

Length

This parameter contains the value of the equivalent attribute of the CRL entry.

Special Cfg Identifier

This selection type parameter indicates the type of Cfg Identifier format.

Type

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Length

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Format

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Consistency

This parameter contains the value of the equivalent attribute of the CRL entry.

Output Length

This parameter contains the value of the equivalent attribute of the CRL entry.

Output Format

This parameter contains the value of the equivalent attribute of the CRL entry.

Output Consistency

This parameter contains the value of the equivalent attribute of the CRL entry.

List of Data Description

This parameter is composed of the following list elements:

Data Description

This parameter contains the value of the equivalent attribute of the CRL entry.

Manufacturer Specific Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Diag Upd Delay

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Ext Diag Overflow

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Deactivated

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Station Non Existent

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Invalid Slave Response

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUU and the parameter D_SAP_index has the value 60.

Fail Safe

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

DPV1 Supported

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

NA to Abort

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Ignore ACIr

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Output Len

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Exp Inp Len

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Data Transfer Status

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

System Diagnosis Status

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

Publisher Flag

This conditional parameter contains the value of the equivalent attribute of the CRL entry. It shall only be present if the parameter CREP has the value BBUB and the parameter D_SAP_index has the value NIL.

List of Structured Prm Data

This conditional parameter is only present if the parameter CR Type contains the value BQUU and the parameter D_SAP_index contains the value 53. This parameter is composed of one or more of the following list elements.

Structured User Prm Data

This parameter is composed of one or both of the following subparameters.

Device Related User Parameter

This parameter indicates the presence of Device Related User Parameter.

Device Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Module Related User Parameter

This parameter indicates the presence of Module Related User Parameter.

Slot Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value of the equivalent attribute of the CRL entry.

List of DXB Link Entries

This parameter indicates the presence of the DXB-Linktable and is composed of the following list elements.

Publisher Address

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Length

This parameter contains the value of the equivalent attribute of the CRL entry.

IsoM Parameter

This parameter indicates the presence of the IsoM Parameter.

TBASE_DP

This parameter contains the value of the equivalent attribute of the CRL entry.

TDP

This parameter contains the value of the equivalent attribute of the CRL entry.

TMAPC

This parameter contains the value of the equivalent attribute of the CRL entry.

TBASE_IO

This parameter contains the value of the equivalent attribute of the CRL entry.

TI

This parameter contains the value of the equivalent attribute of the CRL entry.

TO

This parameter contains the value of the equivalent attribute of the CRL entry.

TPLL_W

This parameter contains the value of the equivalent attribute of the CRL entry.

TPLL_D

This parameter contains the value of the equivalent attribute of the CRL entry.

List of DXB Subscriber Entries

This parameter contains the value of the equivalent attribute of the CRL entry.

Publisher Address

This parameter contains the value of the equivalent attribute of the CRL entry.

Input Data Length Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Publisher

This parameter contains the value of the equivalent attribute of the CRL entry.

Master Data

This parameter contains the value of the equivalent attribute of the CRL entry.

Source Offset Master

This parameter contains the value of the equivalent attribute of the CRL entry.

Dest Slot Number

This parameter contains the value of the equivalent attribute of the CRL entry.

Offset Data Area

This parameter contains the value of the equivalent attribute of the CRL entry.

Length Data Area

This parameter contains the value of the equivalent attribute of the CRL entry.

Time AR Parameter

This parameter contains the value of the equivalent attribute of the CRL entry.

Clock Sync Interval

This parameter contains the value of the equivalent attribute of the CRL entry.

CS Delay Time

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value of the equivalent attribute of the CRL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.15 CRL slave activate

The service given in Table 390 is used to set the value of the attribute Active of the CRL of the DP-master (Class 1).

Table 390 – CRL Slave activate

Parameter name	Req	Cnf
Argument	M	
Activate	M	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Activate

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.16 CRL slave new Prm

The service given in Table 391 is used to set the value of the attribute New Prm of the CRL of the DP-master (Class 1).

Table 391 – CRL Slave new Prm

Parameter name	Req	Cnf
Argument	M	
New Prm	M	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

New Prm

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.17 CRL slave new Prm data

The service given in Table 392 is used to set the values of the attribute Prm Data for DP-slaves with the device attribute DPV1 set to FALSE in the CRL of the DP-master (Class 1). This service is also used to send the Prm Command to a DP-slave with the device attribute DPV1 and Prm Command supported set to TRUE in the CRL of the DP-master (Class 1).

Table 392 – CRL Slave new Prm data

Parameter name	Req	Cnf
Argument	M	
Prm Data	M	
Lock	M	
Sync	M	
Freeze	M	
WD On	M	
WD Fact 1	M	
WD Fact 2	M	
Min Tsdr	M	
Ident Number	M	
Group Ident	M	
User Prm Data	C	
Device Prm Data	U	
Module Prm Data	U	
Prm Command	C	
Seq Number	M	
Primary Request	M	
MS1 Command	M	
Master State Clear	M	
Check Properties	M	
Primary Request used	M	
MS1 Command used	M	
Address Change	M	
Address Offset	M	
Output Hold Time	M	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Prm Data

This parameter is composed of the following subparameters.

Lock

This parameter contains the value for the equivalent attribute of the CRL entry.

Sync

This parameter contains the value for the equivalent attribute of the CRL entry.

Freeze

This parameter contains the value for the equivalent attribute of the CRL entry.

WD On

This parameter contains the value for the equivalent attribute of the CRL entry.

WD Fact 1

This parameter contains the value for the equivalent attribute of the CRL entry.

WD Fact 2

This parameter contains the value for the equivalent attribute of the CRL entry.

Min Tsdr

This parameter contains the value for the equivalent attribute of the CRL entry.

Ident Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Group Ident

This parameter contains the value for the equivalent attribute of the CRL entry.

User Prm Data

This conditional parameter is only present if the parameter Prm Structure contains the value FALSE. This parameter is composed of one or both of the following subparameters.

Device Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Module Prm Data

This parameter contains the value for the equivalent attribute of the CRL entry.

Prm Command

This conditional parameter is composed of the following subparameters.

Seq Number

This parameter contains the value for the equivalent attribute of the CRL entry.

Primary Request

This parameter contains the value for the equivalent attribute of the CRL entry.

MS1 Command

This parameter contains the value for the equivalent attribute of the CRL entry.

Master State Clear

This parameter contains the value for the equivalent attribute of the CRL entry.

Check Properties

This parameter contains the value for the equivalent attribute of the CRL entry.

Primary Request used

This parameter contains the value for the equivalent attribute of the CRL entry.

MS1 Command used

This parameter contains the value for the equivalent attribute of the CRL entry.

Address Change

This parameter contains the value for the equivalent attribute of the CRL entry.

Address Offset

This parameter contains the value for the equivalent attribute of the CRL entry.

Output Hold Time

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.18 Load CRL DP-master CI2

The service given in Table 393 is used to load the CRL of a DP-master (Class 2). For the use of the conditional parameters in the request primitive refer to the description of the CRL DP-master (Class 1) object.

Table 393 – Load CRL DP-master C12

Parameter name	Req	Cnf
Argument	M	
List of CRL Entries	M	
CREP	M	
CR Type	M	
Related AREP	M	
S_SAP_index	M	
D_SAP_index	M	
D_addr	M	
Service_activate	M	
Role_in_service	M	
Max_DLSDU_length_req_low	M	
Max_DLSDU_length_req_high	M	
Max_DLSDU_length_ind_low	M	
Max_DLSDU_length_ind_high	M	
CR Name	U	
Result(+)		S
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value for the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value for the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value for the equivalent attribute of the CRL entry.

D_SAP_index

This parameter contains the value for the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value for the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value for the equivalent attribute of the CRL entry.

Role_in_service

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value for the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value for the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value for the equivalent attribute of the CRL entry.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.2.10.5.19 Get CRL DP-master CI2

The service given in Table 394 is used to get the CRL of a DP-master (Class 2). For the use of the conditional parameters in the request primitive refer to the description of the CRL DP-master (Class 1) object.

Table 394 – Get CRL DP-master CI2

Parameter name	Req	Cnf
Argument	M	
Result(+)		S
List of CRL Entries		M
CREP		M
CR Type		M
Related AREP		M
S_SAP_index		M
D_SAP_index		M
D_addr		M
Service_activate		M
Role_in_service		M
Max_DLSDU_length_req_low		M
Max_DLSDU_length_req_high		M
Max_DLSDU_length_ind_low		M
Max_DLSDU_length_ind_high		M
CR Name		U
Result(-)		S
Status		M

Argument

The argument shall convey the service specific parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of CRL Entries

This parameter is composed of the following list elements:

CREP

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Type

This parameter contains the value of the equivalent attribute of the CRL entry.

Related AREP

This parameter contains the value of the equivalent attribute of the CRL entry.

S_SAP_index

This parameter contains the value of the equivalent attribute of the CRL entry.

D_SAP_index

This parameter contains the value of the equivalent attribute of the CRL entry.

D_addr

This parameter contains the value of the equivalent attribute of the CRL entry.

Service_activate

This parameter contains the value of the equivalent attribute of the CRL entry.

Role_in_service

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_req_high

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_low

This parameter contains the value of the equivalent attribute of the CRL entry.

Max_DLSDU_length_ind_high

This parameter contains the value of the equivalent attribute of the CRL entry.

CR Name

This parameter contains the value of the equivalent attribute of the CRL entry.

Result(-)

This selection type parameter indicates that the service request failed.

Status

This parameter indicates the reason for the failure.

Allowed values: NO, IV

8.3 Summary of AL classes

This subclause contains a summary of the defined fieldbus AL Classes.

Table 395 shows the fieldbus AL Class Summary.

Table 395 – Fieldbus AL class summary

AL ASE	Class
Process Data	Simple Process Data Combined Process Data
I/O Data	Simple Input Data Simple Output Data Extended Input Data Extended Output Data
Diagnosis	Device Related Diagnosis Identifier Related Diagnosis Channel Related Diagnosis Status Module Status
Alarm	Alarm
Context	MS0 User Parameter MS0 Structured User Parameter DXB-Linktable DXB-Subscribabletable Time AR Parameter IsoM Parameter MS0 Configuration Elements Remanent Parameter MS2 User Parameter Red Context
Load Region	Load Region
Function Invocation	Function Invocation Action
Time	Slave Time Link Time
Management	Master Diag Master Parameter
Application Relationship List	ARL DP-slave ARL DP-master (Class 1) ARL DP-master (Class 2)
Communication Relationship List	CRL DP-slave CRL DP-master (Class 1) CRL DP-master (Class 2)

8.4 Permitted AL services by AREP role

Table 396 gives an overview of the requester and responder roles in service execution. Some services are additionally possible. Requester and responder roles in service execution are given separately.

Table 396 – Assignment of the services to DP-masters and DP-slaves

Service	DP-slave				DP-master (Class 1)				DP-master (Class 2)		
	MS0	MS1	MS2	MS3	MS0	MS1	MS3	MM1/ MM2	MS2	MS0	MM1/ MM2
Read		p_R	p_R			I			I		
Write		p_R	p_R			I			I		
Data Transport			p_R						I		
Set Input	LR										
Read Input	i_(R)									I	
Get Input					LR						
New Input					LI						
Set Output					LR						
Read Output	i_(R)									I	
Get Output	LR										
New Output	LI										
Global Control	R				I						
Get Publisher Data	LR										
New Publisher Data	LI										
Publisher Active	LI										
SYNCH					LI						
SYNCH Delayed					LI						
DX Finished					LI						
SYNCH Event	LI										
Set Slave Diag	LR										
Get Slave Diag					LR						
Read Slave Diag	i_(R)									I	
New Slave Diag					LI						
Alarm Notification		I				R					
Alarm Ack		p_R				I					
Initiate			p_R						I		
Abort			R/I						I/R		
Check User Prm	LI										
Check User Prm Result	LR										
Check Ext User Prm	LI										
Check Ext User Prm Result	LR										
Set Cfg	LR										
Get Cfg	i_(R)									I	
Check Cfg	LI										
Check Cfg Result	LR										
Set Slave Add	R									I	
Set Time				R			I				
Sync Interval Violation				LI							
Get Master Diag								p_R			I
Start Seq								p_R			I
Download								p_R			I
Upload								p_R			I
End Seq								p_R			I
Act Para Brct								R			I
Act Param								p_R			I

Table 396 (continued)

where	
I	Initiator of a service
R	Receiver (Responder) of a service
(R)	Receiver (Responder) of a service (but no interaction with the application)
LR	Local service initiated by the application
LI	Local service initiated by the DP-Application Layer
i_	this prefix indicates that the response will be sent immediate, without polling
p_	this prefix indicates that the response will be sent not immediate, polling required

Table 397 shows the support of ARs in the different DP-Device Types.

Table 397 – Support of AR types in the different DP-device types

AR Type	DP-slave	DP-master (Class 1)	DP-master (Class 2)
MS0	M	M	O
MS1	O	O	--
MS2	O	--	O
MS3	O	O	O
MM1	--	M	O
MM2	--	O	O
where			
M	AR Type has to be supported mandatory		
O	AR Type is supported optionally		
--	AR Type not supported		

Table 398 gives an overview of the services which shall be supported at the different AREPs respectively CREPs.

Table 398 – Support of services at the different AREPs respectively CREPs

AR Type	D_SAP_index	CR Type	CR Name	Supported service at the AREP/ CREP		
				DP-slave	DP-master (Class 1)	DP-master (Class 2)
MS0	NIL	BBUB	Data Exchange	Set Input (M)	--	--
				Get Output (M)	--	--
				New Output (O)	--	--
				SYNCH Event (O)	--	--
				--	Set Output (M)	--
				--	Get Input (M)	--
				--	New Input (O)	--
				--	SYNCH (O)	--
				--	SYNCH Delayed (O)	--
				--	DX Finished (O)	--
		BBUU-OM	DXB	Get Publisher Data (O)	--	--
				New Publisher Data (O)	--	--
				Publisher Active (O)	--	--
	62	BQUU	Cfg	Set Cfg (M)	--	--
				Check Cfg (M)	--	--
				Check Cfg Result (M)	--	--
	61	BQUU	Prm	Check User Prm (M)	--	--
				Check User Prm Result (M)	--	--
	53	BQUU	Ext Prm	Check Ext User Prm (O)	--	--
				Check Ext User Prm Result (O)	--	--
	60	BBUU	Slave Diag	Set Slave Diag (M)	--	--
				--	Get Slave Diag (M)	--
				--	New Slave Diag (O)	--
	58	QQUU-OM	Global Control	Global Control (M)	Global Control (M)	--

AR Type	D_SAP_index	CR Type	CR Name	Supported service at the AREP/ CREP		
				DP-slave	DP-master (Class 1)	DP-master (Class 2)
	60	QBCB	Read Diag	Read Slave Diag (M)	--	Read Slave Diag (O)
	59	QBCB	Read Cfg	Get Cfg (M)	--	Get Cfg (O)
	57	QBCB	Read Output	Read Output (M)	--	Read Output (O)
	56	QBCB	Read Input	Read Input (M)	--	Read Input (O)
	55	QBCB	Rem Para	Set Slave Add (O)	--	Set Slave Add (O)
MS1	51	QCOB-CO	Process Data Alarm Ack	Read (O)	Read (O)	--
				Write (O)	Write (O)	--
				Alarm Ack (O)	Alarm Ack (O)	--
				Initiate Load (O)	Initiate Load (O)	--
				Push Segment (O)	Push Segment (O)	--
				Pull Segment (O)	Pull Segment (O)	--
				Terminate Load (O)	Terminate Load (O)	--
				Start (O)	Start (O)	--
				Stop (O)	Stop (O)	--
				Resume (O)	Resume (O)	--
				Reset (O)	Reset (O)	--
				Get FI State (O)	Get FI State (O)	--
				Call (O)	Call (O)	--
	50	QCOB-CO	Alarm Ack	Alarm Ack (O)	Alarm Ack (O)	--
	60	QQUU-OO	Alarm	Alarm Notification (O)	Alarm Notification (O)	--
MS2	49	QQUU-OO	Res Mng	Initiate (O)	--	Initiate (O)
	0 - 48	QCB-CO	Process Data	Abort (O)	--	Abort (O)
				Read (O)	--	Read (O)
				Write (O)	--	Write (O)
				Data Transport (O)	--	Data Transport (O)
				Initiate Load (O)	--	Initiate Load (O)
				Push Segment (O)	--	Push Segment (O)
				Pull Segment (O)	--	Pull Segment (O)
				Terminate Load (O)	--	Terminate Load (O)
				Start (O)	--	Start (O)
				Stop (O)	--	Stop (O)
				Resume (O)	--	Resume (O)
				Reset (O)	--	Reset (O)
				Get FI State (O)	--	Get FI State (O)
				Call (O)	--	Call (O)
MS3	CS	QQUU-OM	CS	Set Time (O)	Set Time (O)	--
				Sync Interval Violation(O)	--	--
MM1	54	QCB-CL	MM	--	Get Master Diag (O)	Get Master Diag (O)
				--	Start Seq (O)	Start Seq (O)
				--	Download (O)	Download (O)
				--	Upload (O)	Upload (O)
				--	End Seq (O)	End Seq (O)
				--	Act Param (O)	Act Param (O)
MM2	54	QQUU-OM	Act Para Brct	--	Act Para Brct (O)	Act Para Brct (O)
where						
		M	mandatory service at this AREP / CREP			
		O	optional service at this AREP / CREP			
		--	non-existent service at this AREP / CREP			

8.5 Conformance classes

Conformance Classes are used to define the minimum functionality of a DP-master. There are no Conformance Classes for DP-slaves. The functionality of the DP-slave determines the necessary conformance class of the DP-master.

For the cyclic data exchange the DP-master (Class 1) has to support the following features:

- min. 2 words consistency
- min. 32 byte data length
- new DPV1 data types; except: Date, Time-Of-Day, Time-Difference (for the Extended Configuration Identifier Format)
- Fail Safe has to be supported
- min. 32 byte Diag Data

For the acyclic communication the DP-master has to support the maximum data length of 244 bytes.

Table 399 and Table 400 show the Conformance Classes of DP-master Class1 and Class 2.

Table 399 – Conformance classes DP-master (class 1)

Conformance Class	Functionality MS1
A	Read Write
B	Read Write Alarm Notification Alarm Ack

Table 400 – Conformance classes DP-master (class 2)

Conformance Class	Functionality MS2
A	Read Write Initiate Abort Data Transport

8.6 Application characteristics

8.6.1 Time behaviour

The following diagram (Figure 80) shows the theoretical system reaction time of a DP-master (Class 1) in dependence of the connected amount of DP-slaves with 2 bytes input- and output-data each.

It is presumed that all configured DP-slaves are in the data exchange mode with no diagnosis message pending. The DP-slaves answer immediately after the minimum Station Delay Timer ($\min T_{SDR} = 11 t_{BIT}$) is expired. At the DP-master (Class 1) T_{SDI} is assumed as 37 bit times.

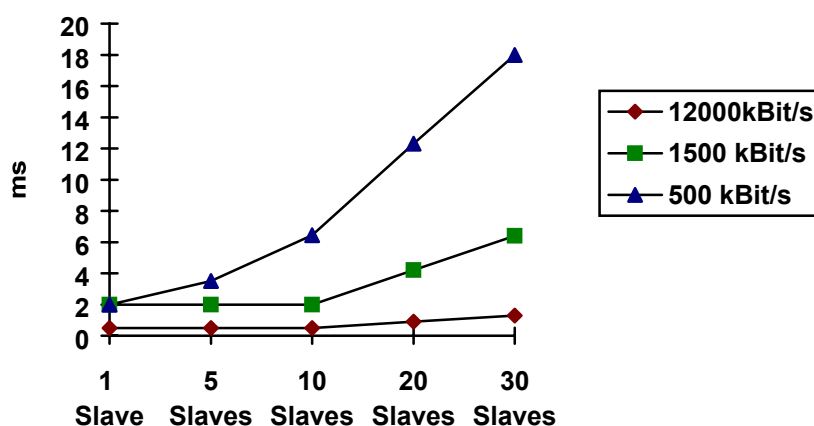


Figure 80 – Cycle time of the DP system

8.6.2 Ident number

The Ident Number is necessary for all DP-Devices except for DP-master (Class 2). For each type of device an individual Ident Number will be needed. This Ident Number is not a series number. If a manufacturer have got a Ident Number for one type of device it is allowed to him to use this number for all produced devices of the same type. At devices of the same type but with differences in the amount of inputs and outputs it is possible to use the same Ident Number for each produced DP-Device. Assumption: The device can be described in the GSD as a modular device.

Each device type shall have an individual Ident Number.

9 Type 4 communication model specification

9.1 Concepts

9.1.1 Overview

The concepts of this model basically follow the common concepts with a few exceptions. The basic principle described in 4.2.2.1 through 4.2.2.3 are applicable for this clause. This Type supports Gateway functionality within the application layer. Part of this can also be recognised from the common concepts. However, to promote readability, some elements from the Clause 4 are repeated in this clause.

9.1.2 Application entities

9.1.2.1 Definition of FAL AE

An FAL AE provides a set of services to enable communications between APs in a fieldbus environment. These services enable a user application in a Client to access user objects in a server.

The functionality of the FAL AE can be divided into two groups.

Application Relation (AR). The AR functionality is responsible for arranging the transportation of APDUs via the network.

Variable Access (VA). The Variable Access functionality is responsible for coding and encoding of APDUs. In a server the VA also convey data directly to / from user objects. How this is done is a local matter and out of scope for this standard.

9.1.2.2 FAL services

The services offered by the FAL AE to the user application are called FAL Services. Services for the AR are called AR ASE services and services for the Variable Access are called Route Endpoint ASE services (REP ASE services).

The FAL services provide access to local AR objects, and provide conveyance of requests and responses for access to real objects modelled as FAL accessible objects. The FAL provides as well confirmed as unconfirmed services. Service requests are conveyed to the AP containing the real object.

Figure 81 illustrates the FAL AE and the architectural relationships.

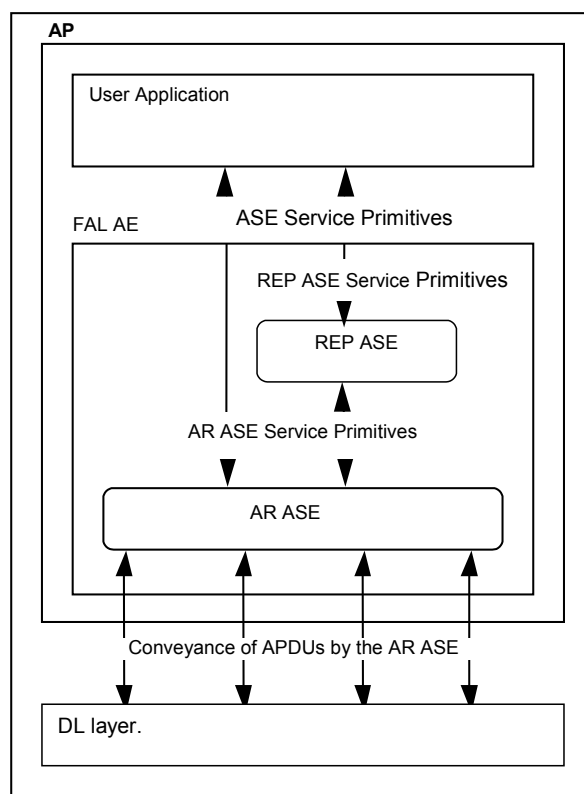


Figure 81 – FAL AE

9.1.2.3 AR ASE

To support access to the remote AP, the Application Relationship ASE is defined. It provides services to the AP for accessing communication-related parameters, and it provides services to the REP ASE for conveying service requests and responses.

The AR ASE provides services at the endpoints of ARs (AREPs).

9.1.2.4 REP ASE

The REP ASE provides a set of services used to read and write network visible attributes of Variable Objects and their application data. As only one ASE type is used remotely, the type is implicit and not transferred.

9.1.2.5 FAL service conveyance and gateway

The FAL AE provides services to convey requests and responses between User Applications and Variable Objects including Gateway functionality.

In a distributed system, application processes communicate with each other by exchanging application layer messages. The AR is defined to support the on-demand exchange of confirmed and unconfirmed services between two application processes. Connectionless data link services are used for the exchanges. The data link layer transfers may be either acknowledged or unacknowledged.

9.1.2.6 Application relationships

9.1.2.6.1 Definition of AR

The AR is responsible for conveying messages between applications. The messages that are conveyed are FAL service requests and responses. Each service request and response is submitted to the AR ASE for transfer. Within the AR only one object type exist, the AR-Endpoint (AREP).

9.1.2.6.2 AR-endpoints

An AR-Endpoint (AREP) represents a service access point to a DLE within the AE. The AR Send Service conveys APDU and route information to the AREP and implicit to the DLE. The AREP is a system management object presenting the user configurable parameters of the related DLE.

This Type has no special system management services, because the attributes of the AREP objects are declared as Variable Objects by the user application. This makes the attributes network visible. The AR Get Attribute and AR Set Attribute services are used by these Variable Objects to access the attribute values.

When an AREP receives an APDU from its DLE (User-triggered) it conveys the APDU to the REP ASE or an AREP (gateway).

9.1.3 Gateway and routing

9.1.3.1 Overview

This Type supports gateway functionality. To identify the destination of a request, a route description is used. This route description holding a complete list of points to pass is sent with the request. On its way to the destination, the route evolves in such a way that it always contains the route back to the requester and the route to the destination. The elements in the route are endpoint and DL addresses.

9.1.3.2 Route endpoint

Within the REP ASE, Route Endpoint objects (REPs) represent endpoints, holding a Route description. An REP represents the starting point for conveying a message. An REP also represents the destination of the APDU.

9.1.3.3 Identification of endpoints

The endpoints are identified by endpoint addresses. These EP addresses are numerical Addresses, taken within an AE from the same name space of numbers, to achieve a unique identification. These EP addresses are used to identify AREPs and REPs.

9.1.3.4 DL addresses

The DLE is identified from the Link by a DL-address. DL-addresses are defined in IEC 61158-4, but as they are part of the route, they are mentioned here. There is a one to one relation between the DL-address of a DLE and the endpoint address of the corresponding AREP.

9.1.3.5 Route

The complete Route contains a Destination-Route and a Source-Route.

The Destination-Route describes how to reach the destination REP. It is a sequence of Endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the receiving DLE, AREP or REP.

The Source Route in the same way describes how to find the way back to the endpoint that initiated the request. It is a sequence of endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the transmitting DLE, AREP or REP.

9.1.3.6 Destination /source route conversion.

A Destination-Route conveyed by an REP is starting with the endpoint address of the local AREP. The next element is the DL-address of the receiving DLE. If the route goes through gateways a pair of addresses shall be added for each gateway, a DL-address and an AREP address. The final address is the endpoint address of the destination REP. The Source address is the endpoint address of the source REP.

The AR Send service removes the first element of the Destination-Route. The DLE inserts its own DL-address in front of the current Source Route before conveying the request to the link.

When a DLE receives an indication from the link, it removes the first element of the Destination Route (its own DL-address). The corresponding AREP inserts its own AREP address in front of the current Source Route before conveying the request to a REP or an AREP.

At the destination REP the Destination-Route holds the endpoint address of the REP, and the Source Route holds the complete route back to the requesting REP.

If any error appears with a request on its way to the destination, the current Source Route can be used for returning an error response.

9.1.4 Architecture summary

A summary of the FAL architecture is presented in Figure 82. It illustrates the major components of the FAL architecture and how they relate to each other.

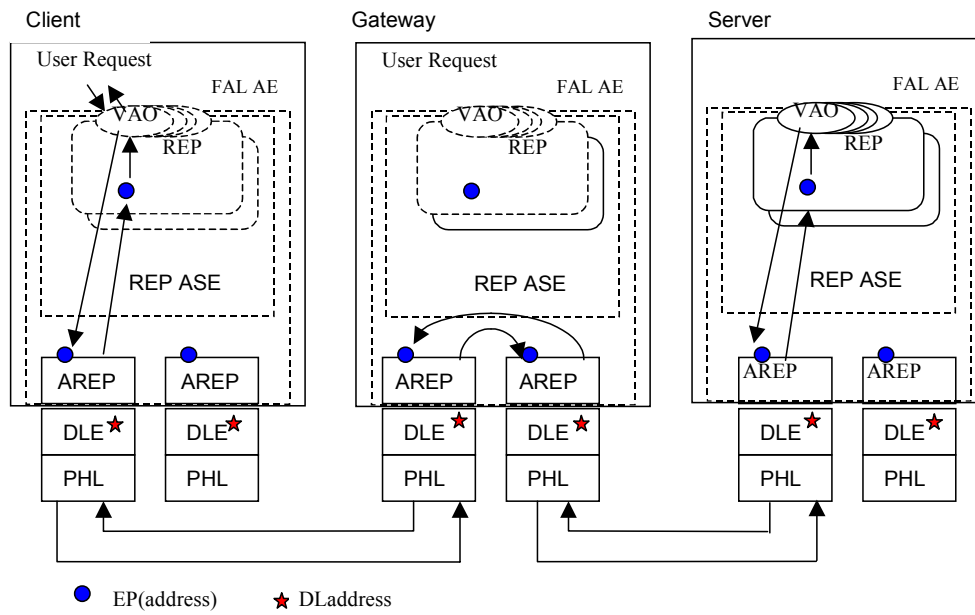


Figure 82 – Summary of the FAL architecture

9.1.5 FAL service procedures and time sequence diagrams

9.1.5.1 Overview

This subclause describes service procedures for FAL confirmed and unconfirmed services. An overview of the FAL service procedures is given in Figure 83.

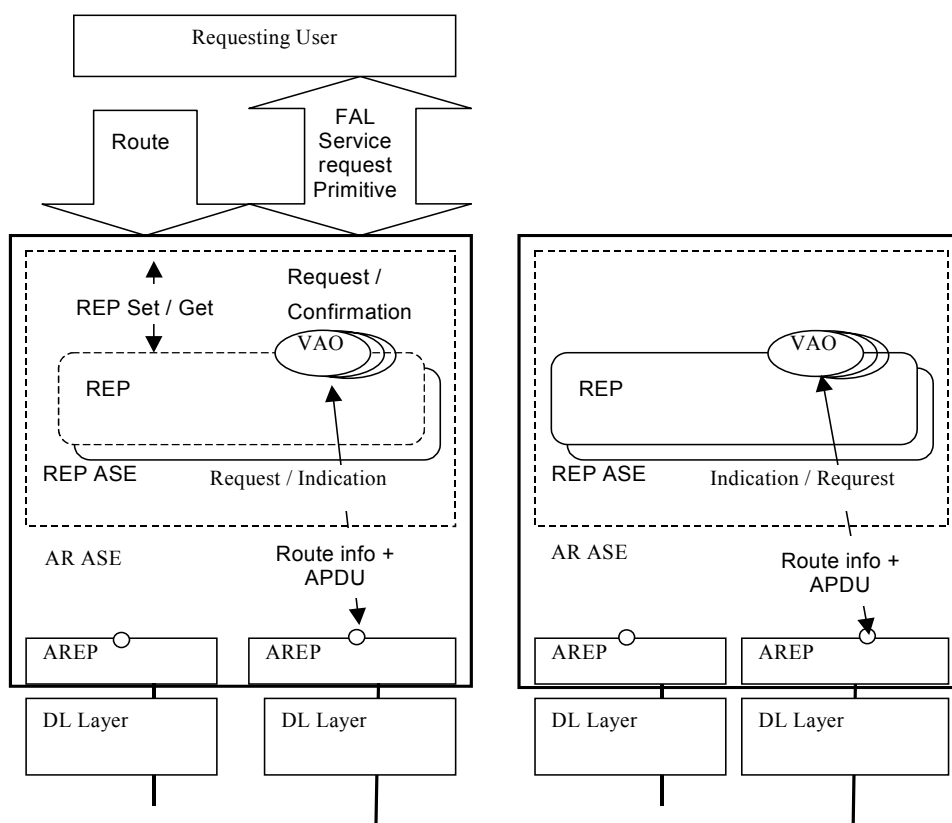


Figure 83 – FAL service procedure overview

9.1.5.2 FAL confirmed service procedures

At requesting AP:

The user application uses the REP ASE service "Reserve REP" to reserve a REP. The user application then uses the REP ASE service "Set REP Attribute" to set attributes of the REP, including the Destination Route to the remote REP.

The requesting user submits a confirmed request primitive to the REP ASE with the REP address as a parameter. The REP ASE builds the related confirmed request APDU and conveys it on the AREP, using an AR Send request primitive. The AREP address is indicated by the first element of the Destination route. The REP ASE starts an associated timer to monitor the request.

The AREP builds a request DLSDU and submits the route info together with the DLSDU to its DLE. The DLE sends the DLSDU at the first opportunity.

At responding server AP:

Upon receipt of the DLSDU, the receiving AREP builds an APDU and delivers the APDU and the received Route to the addressed REP by an AR Send indication primitive.

The REP ASE handles the indication by data exchange with the addressed Variable Object, builds the related unconfirmed response APDU and conveys it on the AREP, using an AR Send request primitive. The received Source route is used as Destination route for the response.

The AREP builds a request DLSDU and submits the route info, together with the DLSDU to its DLE. It is the task of the DL layer to decide whether to “queue” the DLSDU or send it immediately.

If the REP ASE is not able to process the indication and respond within “MaxIndicationDelay”, the REP ASE shall submit an AR Acknowledge request primitive to the receiving AREP, and in this way free the Token. The AR Acknowledge is submitted to the DLE.

At requesting AP receiving the response:

Upon receipt of the response DLSDU, the receiving AREP conveys the response to the REP ASE. The destination address conveyed to the REP ASE holds the endpoint address of the requesting REP. The REP object can now deliver the response to the requesting user application. This is accomplished by the user invocation of the REP ASE RESPONSE Service.

If the timer expires before the requesting REP has received the response, the REP cancels the associated transaction state machine and delivers the Error response “No answer” to the user application.

9.1.5.3 Confirmed service time sequence diagram

Figure 84 shows the time sequence diagram for the confirmed services.

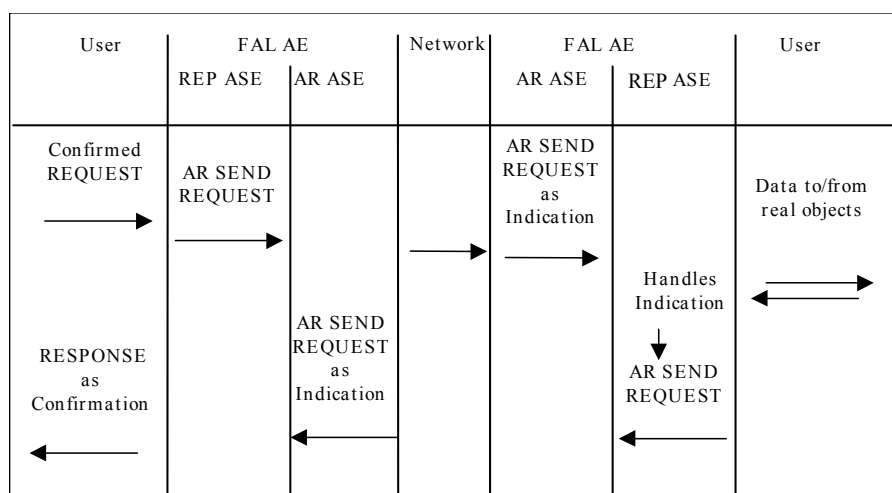


Figure 84 – Time sequence diagram for the confirmed services

9.1.5.4 Unconfirmed request service procedure

At requesting AP:

The user application uses the REP ASE service "Reserve REP" to reserve a REP. The user application then uses the REP ASE service "Set REP Attribute" to set attributes of the REP, including the Destination Route to the remote REP.

The requesting user submits an unconfirmed request primitive to the REP ASE with the REP address as a parameter. The REP ASE builds the related unconfirmed request APDU and conveys it on the AREP, using an AR Send request primitive. The AREP address is indicated by the first element of the Destination route.

The AREP builds a request DLSDU and submits the route info together with the DLSDU to its DLE. The DLE sends the DLSDU at the first opportunity.

At responding server AP:

Upon receipt of the DLSDU, the receiving AREP builds an APDU and delivers that and the received Route to the addressed REP by an AR Send indication primitive.

The REP ASE handles the indication by data exchange with the addressed Variable Object.

9.1.5.5 Unconfirmed service time sequence diagram

Figure 85 shows the time sequence diagram for the unconfirmed services.

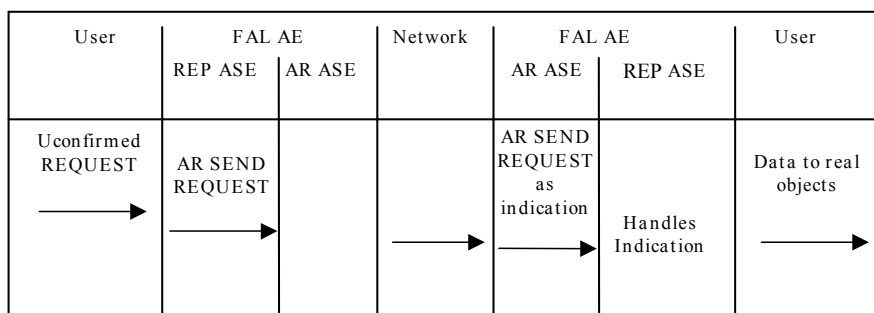


Figure 85 – Time sequence diagram for unconfirmed services

9.1.5.6 Confirmed / unconfirmed service procedure through gateways

Upon receipt of a DLSDU, the receiving AREP delivers an AR Send indication primitive to the local destination. If the endpoint address is a REP address, the APDU + Route is delivered to the REP ASE. If the endpoint address is an AREP address, the AR send service is used to pass it on to the network (Gateway).

For confirmed service Request, an AR Acknowledge is submitted to the “source AREP” unless the response is expected within “MaxIndicationDelay”. “MaxIndicationDelay” is an attribute of the “source AREP”.

9.2 Variable ASE**9.2.1 Variable types****9.2.1.1 Overview**

The Variable ASE specifies the machine independent syntax for application data conveyed by FAL services. The application layer supports the transfer of both basic and constructed variables. The encoding rules for basic variables and constructed variables as specified in this subclause are provided in IEC 61158-6.

Variable Objects (VAOs) are defined as instances of a variable class. The Variable ASE provides access to VAOs. VAOs are identified by a numeric Variable Identifier, and are instances of a variable class.

Basic types are atomic types that cannot be decomposed into more elemental types. Constructed types are types composed of basic types and other constructed types. Their complexity and depth of nesting is free.

Constructed types are packed according to the encoding rules for constructed types. The structure of a constructed type is not network visible. The structure of a constructed type is generated by a compiler and transferred by other means.

All variable classes have the common attributes Variable Identifier, Data Type Identifier, Octet length, Read enable, Write enable and Write protected. All variable classes have the common services Read, Write, Get Variable Attribute and Set Variable Attribute.

This standard recommends using floating point representation of measurement values scaled in SI units.

9.2.1.2 Basic variable types

Basic variable types have a constant length. The length of basic variable types is an integral number of octets. Defining new basic variable types is accomplished by defining new basic variable type classes.

9.2.1.3 Constructed variable types

9.2.1.3.1 Overview

Constructed variable types are needed to completely convey the variety of information present on the fieldbus. Four kinds of constructed variable types are defined in four classes in this standard: Complex, String, BitString and FIFO. Defining new kinds of constructed variable types is accomplished by defining new constructed variable type classes.

On the fieldbus a complete constructed variable can be transferred, or a part of it, e.g. a single field of a complex variable or an element of a string. When transferring only part of a variable, addressing with offset and maybe Bit-no is used. Offset indicates the distance (in octets) from the first octet of the variable to the part to be transferred, Bit-no indicates the bit number in one octet.

9.2.1.3.2 Complex variable type

A Complex variable type defines Arrays and Structures.

An Array is composed of an ordered set of homogeneously typed elements. This standard places no restriction on the type of array elements, but it does require that each element be of the same type. Once defined, the number of elements in an Array may not be changed.

A Structure is made of an ordered set of heterogeneously typed elements called fields. Like Arrays, this standard does not restrict the type of fields. However, the fields within a Structure can be of different types.

9.2.1.3.3 String variable type

A String is composed of an ordered set of an Actual Number of Elements field and a number of homogeneously typed elements. Once defined, the maximum number of elements in a String may not be changed.

9.2.1.3.4 BitString variable type

A BitString is defined as a series of bits. Once defined, the number of elements in a BitString may not be changed. The octet length of a BitString is the integral part of ((the number of elements + 7) divided by 8).

9.2.1.3.5 FIFO variable type

A FIFO queue is composed of a set of homogeneously typed elements. This standard places no restriction on the type of FIFO elements, but it does require that each element be of the same type. The first written element will be the first element that can be read. On the fieldbus only one, complete element can be transferred as a result of one service invocation.

9.2.2 Variable model class specification

9.2.2.1 Variable model

9.2.2.1.1 Formal model

The Variable formal model defines the characteristics common to all Variable classes. This class is not capable of being instantiated. It is present only for the inheritance of its attributes and services of its subclasses.

FAL ASE:		VARIABLE ASE
CLASS:	VARIABLE	
CLASS ID:		1
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m)	Key Attribute: Variable Identifier
2	(o)	Attribute: VariableTypeIdentifier
3	(o)	Attribute: Octet Length
4	(o)	Attribute: Read enable
5	(o)	Attribute: Write enable
6	(o)	Attribute: Write protected
SERVICES:		
1	(m)	OpService: Read
2	(m)	OpService: Write
3	(o)	OpService: Get Variable Attribute
4	(o)	OpService: Set Variable Attribute

9.2.2.1.2 Attributes

Variable Identifier

This attribute identifies the Variable Object instantiating the variable type class.

VariableTypeIdentifier

This optional attribute identifies the variable type as a Boolean, Integer8, Integer16, Integer32, Unsigned8, Unsigned16, Float32, Float64, UNICODE Char, Array, String, Structure or FIFO. The VariableTypeIdentifier is a numerical identifier, defined in IEC 61158-6. In this standard, a descriptive name corresponding to the identifier is stated.

Octet Length

This optional attribute indicates the length in octets of the data of the Variable Object. For all types except FIFO octet length indicates the total number of octets. For FIFO types, octet length indicates the number of octets in one element.

Read enable

The value of this optional attribute indicates, whether the data value of the Variable Object can be read via the fieldbus.

Write enable

The value of this optional attribute indicates, whether the data value of the Variable Object can be updated via the fieldbus.

Write protected

The value of this optional attribute indicates, whether the data value of the Variable Object can only be updated via the fieldbus under certain conditions. The conditions for permitting update of a Write protected VAO are out of scope of this standard.

9.2.2.1.3 Services

Read

This service is used to read the data value of the Variable Object.

Write

This service is used to update the data value of the Variable Object.

Get Variable Attribute

This service is used to read an attribute of the Variable Object.

Set Variable Attribute

This service is used to update an attribute of the Variable Object.

9.2.3 Basic variable type specifications

9.2.3.1 Boolean variable type model

9.2.3.1.1 Formal model

This data type expresses a Boolean variable type with the possible values TRUE and FALSE.

FAL ASE:			VARIABLE ASE
CLASS:	BOOLEAN VARIABLE TYPE		
CLASS ID:	2		
PARENT CLASS:	VARIABLE		
ATTRIBUTES:			
SERVICES:			
1	(m)	OpsService:	And
2	(m)	OpsService:	Or
3	(m)	OpsService:	Test-And-Set

9.2.3.1.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

2	VariableTypeIdentifier	= Boolean
3	Octet Length	= 1

9.2.3.1.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation.

9.2.3.2 Integer8 variable type model

9.2.3.2.1 Formal model

This integer type is a two's complement binary number with a length of one octet.

FAL ASE: VARIABLE ASE

CLASS: INTEGER8 VARIABLE TYPE

CLASS ID: 3

PARENT CLASS: VARIABLE

ATTRIBUTES:

SERVICES:

- | | | | |
|---|-----|-------------|--------------|
| 1 | (m) | OpsService: | And |
| 2 | (m) | OpsService: | Or |
| 3 | (m) | OpsService: | Test-And-Set |

9.2.3.2.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

- | | |
|---|-------------------------------------|
| 2 | VariableTypeIdIdentifier = Integer8 |
| 3 | Octet Length = 1 |

9.2.3.2.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation.

9.2.3.3 Integer16 variable type model

9.2.3.3.1 Formal model

This integer type is a two's complement binary number with a length of two octets.

FAL ASE: VARIABLE ASE

CLASS: INTEGER16 VARIABLE TYPE

CLASS ID: 4

PARENT CLASS: VARIABLE

ATTRIBUTES:

SERVICES:

- | | | | |
|---|-----|-------------|-----|
| 1 | (m) | OpsService: | And |
| 2 | (m) | OpsService: | Or |

9.2.3.3.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

- 2 VariableTypeIdentifier = Integer16
- 3 Octet Length = 2

9.2.3.3.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

9.2.3.4 Integer32 variable type model

9.2.3.4.1 Formal model

This integer type is a two's complement binary number with a length of four octets.

FAL ASE: VARIABLE ASE

CLASS: INTEGER32 VARIABLE TYPE

CLASS ID: 5

PARENT CLASS: VARIABLE

ATTRIBUTES:

SERVICES:

- 1 (m) OpsService: And
- 2 (m) OpsService: Or

9.2.3.4.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

- 2 VariableTypeIdentifier = Integer32
- 3 Octet Length = 4

9.2.3.4.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

9.2.3.5 Unsigned8 variable type model

9.2.3.5.1 Formal model

This integer type is a binary number with a length of one octet. The most significant bit is always used as the most significant bit of the binary number; no sign bit is included.

FAL ASE: VARIABLE ASE

CLASS: UNSIGNED8 VARIABLE TYPE

CLASS ID: 6

PARENT CLASS: VARIABLE

ATTRIBUTES:

SERVICES:

- | | | | |
|---|-----|-------------|--------------|
| 1 | (m) | OpsService: | And |
| 2 | (m) | OpsService: | Or |
| 3 | (m) | OpsService: | Test-And-Set |

9.2.3.5.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

- | | |
|---|------------------------------------|
| 2 | VariableTypeIdentifier = Unsigned8 |
| 3 | Octet Length = 1 |

9.2.3.5.3 Services**And**

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation.

9.2.3.6 Unsigned16 variable type model**9.2.3.6.1 Formal model**

This integer type is a binary number with a length of two octets. The most significant bit of the most significant octet is always used as the most significant bit of the binary number; no sign bit is included.

FAL ASE: VARIABLE ASE

CLASS: UNSIGNED16 VARIABLE TYPE

CLASS ID: 7

PARENT CLASS: VARIABLE

ATTRIBUTES:

SERVICES:

- | | | | |
|---|-----|-------------|-----|
| 1 | (m) | OpsService: | And |
| 2 | (m) | OpsService: | Or |

9.2.3.6.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

- | | |
|---|-------------------------------------|
| 2 | VariableTypeIdentifier = Unsigned16 |
| 3 | Octet Length = 2 |

9.2.3.6.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation.

9.2.3.7 Float32 variable type model

9.2.3.7.1 Formal model

This type has a length of four octets. The format for Float32 is that defined by ANSI/IEEE 754 as single precision.

FAL ASE:	VARIABLE ASE
CLASS:	FLOAT32 VARIABLE TYPE
CLASS ID:	8
PARENT CLASS:	VARIABLE
ATTRIBUTES:	
SERVICES:	

9.2.3.7.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

2	VariableTypeIdentifier = Float32
3	Octet Length = 4

9.2.3.8 Float64 variable type model

9.2.3.8.1 Formal model

This type has a length of eight octets. The format for Float64 is that defined by ANSI/IEEE 754 as double precision.

FAL ASE:	VARIABLE ASE
CLASS:	FLOAT64 VARIABLE TYPE
CLASS ID:	9
PARENT CLASS:	VARIABLE
ATTRIBUTES:	
SERVICES:	

9.2.3.8.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

2	VariableTypeIdentifier = Float64
3	Octet Length = 8

9.2.3.9 UNICODE char variable type model

9.2.3.9.1 Formal model

This type has a length of two octets. It is defined as a single UNICODE character.

FAL ASE:	VARIABLE ASE
CLASS:	UNICODE CHAR VARIABLE TYPE
CLASS ID:	10
PARENT CLASS:	VARIABLE
ATTRIBUTES:	
SERVICES:	

9.2.3.9.2 Attributes

Variable Objects of this class will have the following constant values of the following (parent class) attributes:

2	VariableTypeId	= UNICODE Char
3	Octet Length	= 2

9.2.4 Constructed variable type specifications

9.2.4.1 Complex variable type model

9.2.4.1.1 Formal model

A Complex Variable is an Array or a Structure.

An Array is composed of an ordered set of homogeneously typed elements. This standard places no restriction on the type of array elements, but it does require that each element be of the same type. Once defined, the number of elements in an Array may not be changed.

A Structure is made of an ordered set of heterogeneously typed elements called fields. Like Arrays, this standard does not restrict the type of fields. However, the fields within a Structure can be of different types.

FAL ASE:	VARIABLE ASE
CLASS:	COMPLEX VARIABLE TYPE
CLASS ID:	11
PARENT CLASS:	VARIABLE
ATTRIBUTES:	
SERVICES:	
1	(m) OpsService: And
2	(m) OpsService: Or
3	(m) OpsService: Test-And-Set

9.2.4.1.2 Attributes

Variable Objects of this class will have the following constant value of the following (parent class) attribute:

2	VariableTypeId	= Complex
---	----------------	-----------

9.2.4.1.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation. The operation is legal on subelements of integer or boolean type only.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation. The operation is legal on subelements of integer or boolean type only.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation. The operation is legal on subelements of Integer8, Unsigned8 or boolean type only.

9.2.4.2 String variable type model

9.2.4.2.1 Formal model

A String is composed of an ordered set of an Actual Number of Elements field and a number of homogeneously typed elements. Once defined, the maximum number of elements in a String may not be changed.

FAL ASE:			VARIABLE ASE
CLASS:			STRING VARIABLE TYPE
CLASS ID:			12
PARENT CLASS:			VARIABLE
ATTRIBUTES:			
SERVICES:			
1	(m)	OpsService:	And
2	(m)	OpsService:	Or
3	(m)	OpsService:	Test-And-Set

9.2.4.2.2 Attributes

Variable Objects of this class will have the following constant value of the following (parent class) attribute:

- 2 VariableTypeIdIdentifier = String

9.2.4.2.3 Services

And

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation. The operation is legal on subelements of integer or boolean type only.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation. The operation is legal on subelements of integer or boolean type only.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation. The operation is legal on subelements of Integer8, Unsigned8 or boolean type only.

9.2.4.3 BitString variable type model

9.2.4.3.1 Formal model

A BitString is defined as a series of bits. Once defined, the number of bits in a BitString may not be changed. The octet length of a BitString is the integral part of ((the number of bits + 7) divided by 8).

FAL ASE:		VARIABLE ASE	
CLASS:	BITSTRING VARIABLE TYPE		
CLASS ID:	13		
PARENT CLASS:	VARIABLE		
ATTRIBUTES:			

SERVICES:

- | | | | |
|---|-----|-------------|--------------|
| 1 | (m) | OpsService: | And |
| 2 | (m) | OpsService: | Or |
| 3 | (m) | OpsService: | Test-And-Set |

9.2.4.3.2 Attributes

Variable Objects of this class will have the following constant value of the following (parent class) attribute:

- | | |
|---|------------------------------------|
| 2 | VariableTypeIdentifier = BitString |
|---|------------------------------------|

9.2.4.3.3 Services**And**

This service is used to update the data value of the Variable Object, by storing the result of a logical AND operation. The operation is legal on a subelement (which is a bit) only.

Or

This service is used to update the data value of the Variable Object, by storing the result of a logical OR operation. The operation is legal on a subelement (which is a bit) only.

Test-And-Set

This service is used to read and update the data value of the Variable Object, by returning the result of a logical AND operation, and storing the result of a logical OR operation. The operation is legal on a subelement (which is a bit) only.

9.2.4.4 FIFO variable type model**9.2.4.4.1 Formal model**

A FIFO queue is composed of a set of homogeneously typed elements. This standard places no restriction on the type of FIFO elements, but it does require that each element be of the same type. The first written element will be the first element that can be read. On the fieldbus only one, complete element can be transferred as a result of one service invocation.

FAL ASE: VARIABLE ASE

CLASS:	FIFO VARIABLE TYPE
--------	--------------------

CLASS ID:	14
-----------	----

PARENT CLASS:	VARIABLE
---------------	----------

ATTRIBUTES:

- | | | | |
|---|-----|------------|------------------|
| 1 | (m) | Attribute: | Next Element In |
| 2 | (m) | Attribute: | Next Element Out |
| 3 | (m) | Attribute: | Free elements |
| 4 | (m) | Attribute: | Used elements |
| 5 | (o) | Attribute: | ReRead |
| 6 | (o) | Attribute: | ReWrite |

9.2.4.4.2 Attributes

Variable Objects of this class will have the following constant value of the following (parent class) attribute:

- | | |
|---|-------------------------------|
| 2 | VariableTypeIdentifier = FIFO |
|---|-------------------------------|

Octet length

The octet length of a FIFO indicates the length of a single element.

Next Element In

Next Element In is an Integer16, which holds the index number for the next element to be written to the FIFO.

Next Element Out

Next Element Out is an Integer16, which holds the index number for the next element to be read from the FIFO.

Free elements

This attribute is an Integer16 which holds the current number of free elements in the FIFO (= 0 if full).

Used elements

This attribute is an Integer16 which holds the current number of used elements in the FIFO (= 0 if empty).

Reread

Reread is a Boolean. When the value of Reread is TRUE, a Read will return the previously read element, instead of returning and removing the element indexed by Next Element Out. This facility can be used if a transmission error resulted in a faulty read of an element.

Rewrite

Rewrite is a Boolean. When the value of Rewrite is TRUE, a Write will overwrite the previously written element, instead of inserting a new element. This facility can be used if a transmission error resulted in a faulty write of an element.

9.2.4.4.3 Services**Overview**

The services defined in the Variable ASE provide access to attributes and data of Variable Objects. The services are invoked by a REP ASE REQUEST Service invocation. The REP ASE REQUEST Service conveys the Variable ASE Service parameters. Refer to 10.3.3 for the documentation of these parameters.

Read

This service is used to read (part of) the specified variable. No data is passed with the request. The value of the specified variable is returned in the response.

Write

This service is used to update (part of) the specified variable. The data passed with the request is written into (part of) the specified variable. No data is returned in the response.

And

This service performs a logical AND of (part of) the specified variable and the data passed with the request. The result is stored in (part of) the variable. The service is used to clear individual bits in a variable without first reading the variable and afterwards writing the result back to the variable. No data is returned in the response.

Or

This service performs a logical OR of (part of) the specified variable and the data passed with the request. The result is stored in (part of) the variable. The service is used to set individual bits in a variable without first reading the variable and afterwards writing the result back to the variable. No data is returned in the response.

Test-And-Set

This service performs a logical AND of (part of) the specified variable and the data passed in the request. The result of the AND operation is returned in the response. Then a logical OR operation of (part of) the specified variable and the data passed in the request is performed.

The result of the OR operation is stored in (part of) the variable. The service is used to set individual bits in a variable, and in the same transmission read their state prior to the operation.

Get Variable Attribute

This service is used to read an attribute of the specified Variable Object. The attribute is specified by the offset parameter.

Set Variable Attribute

This service is used to update an attribute of the specified Variable Object. The attribute is specified by the offset parameter.

9.2.5 Route Endpoint ASE

9.2.5.1 Overview

In the fieldbus environment, application processes contain data that remote applications are able to read and write. The data is accessed by means of the REP ASE. This ASE offers services to user applications in clients. The services REQUEST and RESPONSE give the access to real Variable Objects (VAOs) located in a server. In the server the REP ASE conveys data to / from the real Variable Object and returns responses. The services Get REP Attribute, Set REP Attribute, Reserve REP and Free REP are all for local use.

Route Endpoint Description

The Route Endpoint class is defined to support the on-demand exchange of confirmed and unconfirmed services between two application processes. It uses AR ASE services for conveying APDUs.

An REP is an object container, containing Variable Objects of a variable type. It performs the access to variable objects. The Variable Object ID is unique within an REP. The request service addresses a Variable Object within the REP. The Variable Object ID is one of the request service parameters.

An REP in the proxy container role represents a remote REP in a Server. The remote REP is in the real container role. The Destination Route attribute of the REP in the client holds the Route to the real container REP, which it represents.

The REP ASE handles the segmentation when the data length exceeds the MaxDataSize.

When data is segmented, the order of segments is indicated by the offset part of the APDU.

To initiate access of a proxy Variable Object, the user application uses Reserve REP to get a REP currently not in use.

The attributes of the REP are set, including the Destination Route to the remote REP. This is done by the Set REP Attribute service. The requesting user then issues a REQUEST primitive with the REP ID and a Variable Object ID as parameters. The REP ASE builds a request APDU, and passes that along with Route Info in an AR Send request.

Following a successful transmission, if the REQUEST was confirmed, the AREP receives a response APDU and conveys that to the requesting REP by an AR Send indication. The user application can now read the result by issuing a RESPONSE primitive, with the REP ID as a parameter.

Formal Model

CLASS: Route Endpoint

PARENT CLASS: [TOP](#)

1.	(m)	Key Attribute:	Endpoint Address
2.	(m)	Attribute:	Role (Proxy object / Real object)
3.	(m)	Attribute:	REP State
4.	(m)	Attribute:	Priority
5.	(m)	Attribute:	Confirmation
6.	(m)	Attribute:	Destination Route
7.	(m)	Attribute:	Source route
8.	(m)	Attribute:	Progress
9.	(m)	Attribute:	Capabilities
10.	(m)	Attribute:	Flat addressing
11.	(o)	Attribute:	Remote LAN Server ID

1. (m) OpsService: REQUEST
2. (m) OpsService: RESPONSE
3. (m) OpsService: Reserve REP
4. (m) OpsService: Free REP
5. (m) OpsService: Get REP Attribute
6. (m) OpsService: Set REP Attribute

Endpoint Address

This key attribute holds the Endpoint address identifying the REP.

This attribute specifies the role of the REP. The valid values are:

Proxy container	Endpoints of this type are used for sending requests to servers and receive responses from them. The objects contained in REPs of this role are proxy objects for the real objects in the server.
-----------------	---

Real container	are proxy objects for the real objects in the server. Endpoints of this type are used for receiving confirmed and unconfirmed requests from clients and sending responses to them. The objects contained in REPs of this role are the real Variable Objects.
----------------	--

This attribute indicates the state of the REP. The values for this attribute are: IDLE, RESERVED, WAITING FOR RESPONSE, RESPONSE RECEIVED or NOT IN USE.

The DLL may provide the possibility to send high priority APDUs before APDUs of lower priority. The priority only refers to the request on the local link, not requests passing gateways and not the response.

This attribute indicates whether the request has to be confirmed or unconfirmed. The response is always returned unconfirmed. If the Destination Route contains one or more broadcast addresses (126) this attribute shall be set to unconfirmed.

Destination Route

The Destination Route describes the Route to the destination REP. It is a sequence of Endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the next DLE, AREP or REP to receive the APDU. In a request, the Destination Route holds the Route to the REP to respond. In a response, Destination Route holds the Route to the requesting REP.

Source Route

The Source Route describes the Route to the source REP. It is a sequence of endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the DLE, AREP or REP endpoint from where the APDU was conveyed. In a request, the Source Route holds the Route to the requesting REP. In a response, Source Route holds the Route to the responding REP.

Progress

This attribute is only relevant in proxy container REPs. It indicates the progress of a request. It indicates number of segments that has been delivered divided with the total number of segments. This means for non-segmented requests the value is zero while waiting for the response, and one when the response is received. For segmented requests, the value will start from zero and gradually increase till it reaches the value of one.

Capabilities

This attribute defines the capabilities of the REP addressed by Destination Route. It is a local attribute, which is set up by the user application, to reflect the capabilities of the REP containing the real Variable Objects. The value of Capabilities is used by the requesting REP to build the APDU. The attribute indicates, whether the responding REP is capable of handling bit addressing or not.

Flat addressing

This attribute indicates, whether the REP should be seen as a container of individual Variable Objects, or as a container of one, flat memory area. If Flat addressing is selected, the Variable Object ID parameter of the REQUEST service indicates the offset in octets from the beginning of the memory area.

Remote LAN Server ID

This optional attribute specifies the ID of a LAN server. This attribute is only used when the Client is directly connected to a LAN network and uses this LAN network as a transport medium to a fieldbus device/gateway. The ID may be an Internet URL address or a numerical IP/IPX address.

9.2.6 Route endpoint ASE service specification

9.2.6.1 REQUEST service

9.2.6.1.1 Overview

This service is used to initiate access of the value of a Variable Object. It may be used for as well reading as updating the value. The result is the operation (if any) is retrieved by the RESPONSE service.

9.2.6.1.2 Service primitives

The service parameters for this service are shown in Table 401. The Indication parameters are indicating the parameters in the APDU received by the remote REP. These parameters are for local use within the remote REP and its variable objects.

Table 401 – REQUEST service parameters

Parameter name	Req	Ind	Cnf
Argument	M	M	
REP	M	M	
Variable object ID	M	M(=)	
Variable Service	M	M(=)	
Data length	M	M(=)	
Offset/Attribute	C	C(=)	
Bit-no	C	C(=)	
Data	C	C(=)	
Result			
Status			M

Argument

The argument carries the parameters of the REQUEST service invocation.

REP

This parameter is the EP address of the REP holding the route information.

Variable ID

This parameter identifies the Variable Object. The Variable ID indicates the Variable Object on which the service is to be performed.

Variable Service

This parameter holds the Variable service to be performed. (Read, Write, And, Or, Test-And-Set, Get attribute and Set Attribute).

Data Length

This parameter indicates the number of octets of data to be transferred.

Offset/Attribute

This parameter is used when accessing only part of a structured variable or an Attribute of the variable. When accessing part of a structured variable, it indicates the offset in octets to the starting octet of the data block to be transferred, relative to the first octet of the variable. When accessing attributes, it identifies the attribute to be accessed.

Bit-no

This parameter is used to select a bit within a BitString. It indicates the bit number (1-8) within an octet. The Offset/Attribute parameter is used to select the octet.

Data

This parameter holds the data to be transferred.

Status

As a result of the request, Status is returned indicating OK or FAILURE.

9.2.6.2 RESPONSE service

9.2.6.2.1 Overview

This service is used to return the result of a confirmed REQUEST to the requesting user application in a requester.

9.2.6.2.2 Service primitives

The service parameters for this service are shown in Table 402. The response parameters are for local use within the responding REP and its variable object.

Table 402 – RESPONSE service parameters

Parameter name	Rsp	Cnf
Argument	M	M
REP	M	M
Result		
Variable Service	M	M
Data length	M	M(=)
Error status	M	M(=)
Data	C	C(=)

Argument

The argument carries the parameters of the service request.

REP

This parameter holds the ID of the REP holding the route information.

Variable Service

This parameter holds the Variable service that has been performed. (Read, Write, And, Or, Test and Set, Get attribute and Set Attribute).

Data Length

This parameter indicates the number of octets of data that have been transferred.

Error status

This parameter holds error information for errors occurred in any layer local or remote. For details, see Table 403.

The error code can be generated anywhere on the route between client and server, or by the responding Variable Object.

Table 403 – Error codes by source

Error description	Client	Gateway	Server
User Application:			
Instruction Error	X		X
Info length error	X		X
FIFO full or empty			X
Write protection			X
Data format error			X
Variable Object ID error			X
Time Out	X		
Actual data error			X
Historical data error			X
FAL:			
Route error	X	X	X
DL-layer:			
No Response	X	X	
WAIT TOO LONG	X	X	
Out of sync	X	X	
CRC ERROR	X	X	
DLE not Client	X	X	
Physical Layer:			
NET SHORTCIRCUIT	X	X	
OVERRUN/FRAMING ERROR	X	X	
RS-232 HANDSHAKE ERROR	X	X	X

Data

This parameter holds the data that has been transferred.

9.2.6.3 Reserve REP Service

9.2.6.3.1 Overview

This service is used by the user application to reserve an REP currently not in use, and to get the ID of the reserved REP.

9.2.6.3.2 Service Primitives

The service parameters for this service are shown in Table 404.

Table 404 – Reserve REP service parameters

Parameter name	Req	Cnf
Result		
REP		M

REP

This parameter holds the ID of the reserved REP. If no REP is available, zero is returned.

9.2.6.4 Free REP Service

9.2.6.4.1 Overview

This service is used by the user application to free an REP by changing the state of the REP to “not in use”.

9.2.6.4.2 Service Primitives

The service parameters for this service are shown in Table 405.

Table 405 – Free AREP service parameters

Parameter name	Req
Argument	M
REP	M

Argument

The argument carries the parameters of the service request.

REP

This parameter holds the EP address of the reserved REP to free.

9.2.6.5 Get REP Attribute Service

9.2.6.5.1 Overview

This service is used by the user application to read an attribute of a REP locally.

9.2.6.5.2 Service Primitives

The service parameters for this service are shown in Table 406.

Table 406 – Get REP attribute service parameters

Parameter name	Req	Cnf
Argument	M	
REP	M	
Attribute index	M	
Result (+)		
Value		C
Result (-)		
Status		C

Argument

The argument carries the parameters of the service request.

REP

This parameter specifies the REP.

Attribute index

This parameter identifies the attribute to read.

Value

This parameter returns the value of the specified attribute.

Status

This parameter returns error information if the service fails. The possible values are: Illegal attribute index, Illegal REP address.

9.2.6.6 Set REP Attribute Service

9.2.6.6.1 Overview

This service is used by the user application to write a value into an attribute of a REP locally.

9.2.6.6.2 Service Primitives

The service parameters for this service are shown in Table 407.

Table 407 – Set REP attribute service parameters

Parameter name	Req	Cnf
Argument	M	
REP	M	
Attribute index	M	
Result		
Status		M

Argument

The argument carries the parameters of the service request.

REP

This parameter specifies the REP.

Attribute index

This parameter identifies the attribute to update within the specified REP.

Value

This parameter holds the value to be written to the specified attribute.

Status

As a result of the request, Status is returned indicating OK or the reason for failure. The possible values are: OK, Illegal attribute index, Illegal value, Illegal REP address.

9.3 Application relationship ASE

9.3.1 Overview

To support access to the remote AP, the Application Relationship ASE is defined. It provides services to the AP for accessing communication-related parameters, and it provides services to the REP ASE for conveying service requests and responses.

The AR ASE provides services at the endpoints of ARs (AREPs).

9.3.2 Application relationship class specification

9.3.2.1 Application relationship formal model

The functionality of the AR ASE is described in 10.1.

The application ASE defines one class, the AREP class.

This attribute describes the type of Acknowledgement to be used by DLL: Acknowledged or Unacknowledged transfer of Confirmed APDUs, and Acknowledged or Unacknowledged transfer of Unconfirmed APDUs.

MaxIndicationDelay

This attribute indicates to the user application, how long time a Variable Object can use to prepare a response after receiving an indication requiring that. If the Variable Object is unable to prepare a response within MaxIndicationDelay, it shall issue an AR-Acknowledge. The value of MaxIndicationDelay is calculated by the DLE.

Local DLE address (P-NET/IP/IPX address)

This attribute reflects the DL address of the related DLE. The format varies depending on if it is a P-NET address or a LAN address. In some situations it may be a read-only attribute.

MaxRetryTime

This attribute indicates the maximum time the DLE should try to re-transmit a request as a result of Acknowledge responses from the remote Variable Object.

MaxRetries

This attribute indicates the maximum number of re-transmissions carried out by the DLE as a result of transmission errors.

MaxOutstandingRequests

This attribute indicates the maximum number of requests that the AREP may initiate without receiving the related responses.

BaudRate

This attribute specifies the baud rate used by the physical layer. It is conveyed to/from the physical layers by the DLEs.

NumberOfClientDLEs

This attribute only relates to a DLE using the P-NET protocol in the client role. It specifies the number of participants in a token round.

9.3.3 Application relationship ASE service specifications

9.3.3.1 Overview

This subclause contains the definition of the services that are unique to this ASE. The services are:

- AR Send
- AR Acknowledge
- AR Get Attribute
- AR Set Attribute

9.3.3.2 Common parameter definition

Parameters used in more AR ASE services are defined below:

Route info

The complete DL Route contains the following elements:

Destination Route

The Destination-Route describes how to reach the destination REP. It is a sequence of Endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the next DLE, AREP or REP to receive the APDU.

Source Route

The Source Route in the same way describes how to find the way back to the endpoint that initiated the request. It is a sequence of endpoint addresses and DL-addresses. On its way to the destination, the first element always indicates the address of the DLE, AREP or REP endpoint from where the APDU was sent.

Priority

The underlying layer may provide the possibility to send high priority APDUs before APDUs of lower priority. This priority only refers to the request on the local link, not requests passing gateways and not the response.

Confirmation

This parameter indicates whether the request has to be confirmed or unconfirmed. A response is always returned unconfirmed. If the destination route contains one or more broadcast addresses (126) this attribute shall be set to unconfirmed.

Remote LAN Server ID

This optional parameter specifies the ID of a LAN server. This attribute is only used when the Client is directly connected to a LAN network and uses this LAN network as transport medium to a fieldbus device/gateway. The ID may be an Internet URL address or numerical IP/IPX address.

APDU Header

The APDU Header parameter holds a Control/Status subfield indicating the Variable Service, an APDU format subfield, indicating whether the APDU holds an offset, and finally an APDU length subfield, indicating the octet length of the APDU.

APDU Body

This parameter holds the APDU Body to send or receive.

9.3.3.3 AR send service

9.3.3.3.1 Overview

The AR Send service is used to send confirmed or unconfirmed APDUs from one AREP to another.

It is the task of the receiving AREP to convey the APDU to the Endpoint specified by the first element in the destination route. In the gateway situation the AR is using this AR Send service to convey the APDU to an AREP.

9.3.3.3.2 Service primitives

The service parameters for this service are shown in Table 408.

Table 408 – AR Send service parameters

Parameter name	Req	Ind	Cnf
Argument	M	M	
Route info	M	M	
APDU Header	M	M(=)	
APDU Body	C	C(=)	
Result			
Status			M

Argument

The argument carries the parameters of the service request.

Status

As a result of the request, Status is returned indicating OK or the locally detected reason for failure. Status only refers to local conditions. The possible values are: OK, Route Error.

9.3.3.4 AR Acknowledge Service

9.3.3.4.1 Overview

When the Variable ASE in a confirmed request is not able to process the indication and respond within the time limit, "MaxIndicationDelay", it shall submit an AR Send Acknowledge request primitive to the local AREP, and in this way free the Token. "MaxIndicationDelay" is an attribute of the AREP.

9.3.3.4.2 Service Primitives

The service parameters for this service are shown in Table 409.

Table 409 – AR Acknowledge service parameters

Parameter name	Req
Argument	M
Route info	M

Argument

The argument carries the parameters of the service request.

9.3.3.5 AR-Get Attribute Service

9.3.3.5.1 Overview

This confirmed service is used to read the value of attributes of an AREP locally.

9.3.3.5.2 Service Parameters

The service parameters for the AR Get Attribute Service are shown in Table 410.

Table 410 – AR Get Attributes service parameters

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Attribute index	M	
Result (+)		S
Value		C
Result (-)		S
Status		C

Argument

The argument carries the parameters of the service request.

AREP

This parameter identifies the AREP by its endpoint address.

Attribute index

This parameter identifies the attribute.

Result (+)

This selection type parameter indicates that the request succeeded.

Value

This parameter is returned with the requested attribute value if the request succeeded.

Result (-)

This selection type parameter indicates that the request failed.

Status

If the request failed, Status is returned indicating the reason for failure. The possible values are: Illegal attribute index, Illegal AREP address.

9.3.3.6 AR Set Attributes Service

This confirmed service is used to set the current value of attributes of an AREP locally.

9.3.3.6.1 Service Parameters

The service parameters for the AR Set Attribute Service are shown in Table 411.

Table 411 – AR Set Attributes service parameters

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Attribute index	M	
Value	M	
Result		
Status		M

Argument

The argument carries the parameters of the service request.

AREP

This parameter identifies the AREP by its endpoint address. The legal values are 1 to 16 inclusive.

Attribute index

This parameter identifies the attribute to be updated.

Value

This parameter holds the new attribute value.

Status

As a result of the request, Status is returned indicating OK or the reason for failure. The possible values are: OK, Illegal attribute index, Illegal value, Illegal AREP address.

10 Type 5 communication model specification

10.1 Concepts

10.1.1 Overview

This subclause introduces the concept of the FAL application entity (FAL AE). It is referred to as the Field Device Access (FDA) Agent. The FDA Agent is composed of Type 5 specific APO ASEs and ARs and a subset of the Type 9 APO ASEs. A subset of the data types defined in Clause 4 are supported. The Type 9 ASEs, classes, and services supported by Type 5 are defined in 14.3.

The FDA Agent maps the Type 5 and Type 9 ASEs onto a set of underlying connection-oriented and connectionless communication service endpoints modelled as sockets. These services are referred to as socket services in the remainder of this clause.

Socket services are independent of the actual protocols used to provide their services. Any protocols that can be mapped to these services can be used. Sockets are addressed by network address and port number.

Type 5 supports application access to both Type 9 and Type 5 field devices. Access to Type 9 field devices is provided through one class of Type 5 devices called a *Linking Device*. Linking Devices contain a FDA Agent that provides a gateway function between Type 5 and Type 9 networks.

Another class of Type 5 devices is capable of interconnecting a Type 5 network to links that support other data link layer types. These Type 5 devices are called Gateway Devices. The FDA Agent in Gateway Devices provides gateway services that map Type 5 APDUs to their counterparts on the other links.

Another class of Type 5 devices is capable of supporting redundant LAN connections. These Type 5 devices are called LAN Redundancy Capable Devices. The FDA Agent in these devices provides LAN Redundancy ASE services to the LAN Redundancy entity in the device.

The FDA Agent uses a Type 5 MIB to store data. The specification of this MIB is beyond the scope of this standard.

Throughout the remainder of this clause, Type 5 devices are FAL devices that contain a FDA Agent and Type 9 Devices refer to devices that implement the Type 9 FAL and the Type 1 DLL.

10.1.2 Objectives of the FDA agent

The primary objective of the FDA Agent is to convey Type 5 and Type 9 ASE services using socket services. This allows Type 5 and Type 9 field devices, conventional I/O devices, and other I/O devices to be connected through a FDA Agent in a *Linking* or *Gateway Device*.

Another objective of the FDA Agent is to republish Type 9 data from one Type 9 link onto another in multi-ported Type 9 devices that do not support Type 9 data link layer bridging. This allows a FDA Agent in a Linking Device to integrate multiple standalone Type 9 interfaces instead of using a Type 9 bridge.

Another objective of the FDA Agent is to send and receive messages to support applications responsible for adding and deleting Type 5 application objects to/from the network, and for locating application objects on the network.

Another objective of the FDA Agent is to send and receive diagnostic messages to support LAN redundancy applications.

As a result, the FDA Agent enables:

- construction of control systems from Type 5 and Type 9 devices
- linking of Type 5 and Type 9 networks by Linking Devices and
- accessing by remote applications of field devices of any type through socket services using the Type 5 Communications Model.

10.1.3 Data type ASE

10.1.3.1 Data types

The data types supported are a subset of those defined in Clause 4. They are:

- Boolean
- Integer8
- Integer16
- Integer32
- Unsigned8
- Unsigned16
- Unsigned32
- Floating Point
- VisibleString
- OctetString
- Date
- TimeOfDay
- TimeDifference
- BitString
- TimeValue

10.1.3.2 Nesting level

One level of nesting is supported. That is, constructed types may not contain constructed types.

10.1.4 APO ASEs

10.1.4.1 Overview

The FDA Agent supports Type 5 and Type 9 APO ASEs as specified in 14.3. The Type 5 ASEs are the VFD, SMK and LAN Redundancy ASEs. The Type 5 VFD ASE is an adaptation of the Type 9 VFD ASE. The FDA Agent also supports versions of the QUB-CO and QUU Type 9 ARs that are adapted for use over sockets.

10.1.4.2 LAN redundancy ASE

The LAN Redundancy ASE specifies services that support LAN Redundancy entities. A LAN Redundancy Entity in LAN redundancy capable devices uses the services of the FDA Agent to send and receive LAN redundancy ASE APDUs. These APDUs are used by the LAN Redundancy Entity to construct a network status table used to choose routes that avoid network faults.

10.1.4.3 System management kernel (SMK) ASE

10.1.4.3.1 SMK

The SMK entity is the entity in a device that is responsible for system management operation in a device. The SMK ASE defines a set of services that support the conveyance of system management requests and responses to and from SMK entities.

When resident in a Type 5 device, the SMK services are conveyed by Type 5 ARs. When resident in a device of another type, the SMK services may be conveyed by direct access to the data link layer connectionless services.

Like Type 9 ASE services, SMK ASE services can be conveyed between a device connected to a Type 5 network and a device connected to a network that uses an underlying Type 9 Data Link Layer. In this case, the FDA Agent in a Linking Device receives an SMK service request or response from one network and maps it for conveyance by the other.

10.1.4.3.2 SMK ASE services

One SMK ASE service is defined for identifying devices.

Two SMK ASE services are defined for adding and deleting Type 5 Devices and Type 9 devices connected to a Linking Device.

- One of these services is used by the SMK in a new device to announce its presence periodically on the network.
- The other service is used by a configuration application receiving the annunciation to return basic configuration information to the SMK. This allows the SMK to learn its role in the system and begin normal operations.

Two more SMK ASE services are defined for finding devices and named objects on the network.

10.1.4.4 Virtual field device (VFD) ASE

10.1.4.4.1 The VFD object

The Type 9 VFD Object is used in Type 5. In Type 5, all objects and services defined by the Type 9 ASEs are used to access APOs within a VFD, with the following exceptions. The Type 5 VCR replaces the Type 9 VCR and the Type 5 Initiate service replaces the Type 9 Initiate service. All Type 5 and Type 9 VFD services are conveyed on Type 5 networks using the Type 5 syntax and procedures defined in IEC 61158-6.

10.1.4.4.2 Virtual communication relationships (VCRs)

10.1.4.4.2.1 Overview

Type 5 VCRs are similar to the Type 9 VCR (defined by the Type 9 Context Management ASE). Type 5 VCRs provide a Type 9 service interface for access to both Type 5 VFDs and Type 9 VFDs. Type 9 VFDs are accessed through Type 5 AEs in Linking Devices. Three types of VCRs are defined, Client/Server, Publisher/Subscriber, and Report Distribution.

10.1.4.4.2.2 Client/server VCRs

The FDA Agent supports dynamically established client/server VCRs, as follows. Clients send requests to initiate VCRs to the server FAL AE. The server FAL AE dynamically creates a new server VCR and delivers the request to it for processing.

10.1.4.4.2.3 Publisher/subscriber VCRs

Each subscriber VCR endpoint is configured to receive from a specific subscriber AR. Each subscriber VCR endpoint is configured to receive its messages from a single publisher VCR endpoint. The original publisher may exist on a Type 9 link when a Linking Device is used to republish the data onto the Type 5 network. In addition, if the subscriber is located in a Linking Device, it may be configured to republish the data onto a Type 9 link.

When conveyed through a Linking Device, the FAL AE maps Type 9 publisher/subscriber VCRs to Type 5 publisher/subscriber VCRs, receiving published data on one and republishing it on the other.

The FDA Agent can be configured for three types of publishing:

- A local Type 5 application can publish data onto the Type 5 network through the FDA Agent.
- The FDA Agent in a Linking Device can republish data from a Type 9 device onto the Type 5 network.
- The FDA Agent in a Linking Device can republish data from a Type 5 device onto a Type 9 link.

10.1.4.4.2.4 Report distribution VCRs

Report distribution VCRs operate analogously to publisher/subscriber VCRs.

The FDA Agent can be configured for three types of report distribution:

- A local Type 5 application can send reports onto the Type 5 network through the FDA Agent.
- The FDA Agent in a Linking Device forward reports received from devices on a Type 9 link onto the Type 5 network.
- The FDA Agent in a Linking Device can forward reports received from a Type 5 device onto a Type 9 link.

10.1.4.4.3 Initiate service

The Initiate service can be directed to a VFD's generic address. This allows multiple client applications to open concurrent access to the VFD using the same address. The VFD ASE operating at that address creates and a new VCR dynamically to process each received initiate request.

The Initiate service also can be directed to the Management Information Base (MIB) VFD of a device without knowing its generic address. Only the device address is necessary to open the AR to the device.

The Initiate service operates locally for publisher/subscriber VCRs. It opens the VCR for publishing or subscribing.

10.1.5 Application relationships

10.1.5.1 Sessionless transfers

The FDA Agent sends and receives SM and LAN Redundancy messages without FDA sessions using underlying connectionless services. Sessionless transfers are not capable of concatenating messages.

10.1.5.2 FDA sessions and Type 5 VCRs

Type 5 VCRs provide access to VFDs across the Type 5. They are the Type 5 counterpart to Type 9 VCRs. Type 5 VCRs use FDA Sessions to transfer Type 9 services.

FDA sessions provide communications to/from FDA Agents. They are the Type 5 counterpart to Type 9 ARs. They are established for the life of the session to operate over either a TCP connection or over connectionless services, between source and destination ports.

The similarities between Type 9 ARs and FDA Sessions are as follows:

- Type 9 ARs and FDA Sessions support the same VCR models, client/server, publisher/subscriber, and Report Distribution.
- Type 9 devices and FDA Agents each may contain one or more Type 9 AR/FDA Session endpoints.
- Type 9 ARs are identified on the wire by source and destination DLCEPs/DLSAPs. FDA Sessions are identified on the wire by source and destination network addresses (network address plus port number).
- Type 9 SMKs communicate directly with the data link layer using the Unitdata service. FDA SM messages are conveyed in a similar fashion, using connectionless services without using a session.

Type 9 ARs and FDA sessions are negotiated to operate using a specific version of their protocol data units (PDUs)/messages. All PDUs/ messages sent on a Type 9 AR/ session use the same protocol version. FDA Agents that support newer versions also support previous versions for backward compatibility.

- Type 9 ARs and FDA Sessions differ in the following ways.
- A single session is capable of supporting more than one Type 5 VCR. Type 9 ARs are capable of supporting only a single Type 9 VCR.
- Sessions are capable of concatenating multiple messages together into a single PDU. Each DLL PDU conveys only a single Type 9 message (PDU).
- Type 5 connectionless client/server sessions are established using underlying connectionless services. For Type 9 ARs, there is no concept of a connectionless client/server AR.
- Connection-oriented client/server sessions are established over TCP. Once the TCP connection has been established, the client sends an FDA Open Session request message on that connection to open the session. Type 9 ARs operate similarly, but the connection is established at the data link layer, and the data link connect request contains the FAS request to open the AR.

10.1.5.2.1 Client/server

Client/server sessions

Client/server sessions are used to support client/server VCRs. A single client/server session may support more than one VCR. Client/server sessions shall be established before they can be used. They are established by exchanging FDA Open Session request and Response messages between client and server.

If TCP is to be used, the client first sends a TCP connect request to the reserved FDA Port of the server device. The FDA Agent at the server listens at the reserved FDA Port for TCP connect requests, and when one is received it creates a new server session endpoint. It then waits for the next TCP connect request, while the newly created session endpoint waits for an Open Session request.

If underlying connectionless services are to be used, the client sends the Open Session request message to the reserved Type 9 port of the server device. The FDA Agent at the server listens at the reserved FDA Port for Open Session requests, and when one is received it creates a new server session endpoint, allocates a connectionless socket for it with a new port address, and delivers the Open Session request to it. If creation of a new server session endpoint is not possible, a negative response is sent from the FDA Port. The FDA Agent then waits for the next Open Session request.

When the new server session endpoints receives the Open Session request it processes the message. The FDA Open Session request message contains negotiable parameters that allow the server to dynamically configure the server session endpoint and that indicate whether or not the session will be used as a *configuration* session (see 10.1.5.4).

If the request is accepted, the newly created session endpoint returns a positive response. If the request is rejected, the newly created session endpoint returns a negative response, closes the underlying socket, and deletes itself. Responses are returned from the underlying socket, or from the reserved Type 9 port if it was not possible to open a new connectionless socket.

Client/server sessions are terminated by user request, lack of activity, or because of FDA protocol problems.

Client/server VCRs

Client/server VCRs are used to convey requests and responses to and from VFDs. Client/server VCRs may be established between a client application and either a Type 5 or a Type 9 VFD. Type 9 VFDs are accessed by establishing a Type 5 VCR to a linking device. This Type 5 VCR endpoint accesses a Type 9 VCR that connects to the Type 9 VFD. Multiple Type 5 VCRs, over the same or different sessions, may concurrently access a single Type 9 VCR. Type 9 field devices, therefore, do not have to provide individual server VCR endpoints to support each concurrent Type 5 client.

Client/server VCRs shall be established before they can be used. Client/server VCRs are established by exchanging Initiate request and response messages between client and server after a session between them has been opened.

The client sends the Initiate request message to the *generic* VCR endpoint for the server VFD. The generic VCR endpoint is identified in the Initiate by a *generic* selector or by a special value that indicates *NMA Access*. The generic selector value is contained in the VFD List in the MIB for the device. It can also be obtained using the Find Tag Query for the tag of the VFD or for an object contained within the VFD.

The generic VCR endpoint listens at this selector for Initiate requests, and when one is received it creates a new server VCR endpoint and adds an entry in MIB list of *automatic* VCRs. The generic VCR endpoint also delivers the Initiate request to the new VCR endpoint, and then waits for the next Initiate request.

The new VCR endpoint uses address information contained in the request to determine whether the VFD is a local Type 5 VFD, or in the case of a linking device, whether it is a Type 9 VFD. If it is a local Type 5 VFD or if it is a Type 9 management agent VFD in the linking device, then the VCR endpoint delivers an Initiate indication primitive and waits for the response. If the VFD is a Type 9 VFD located in a Type 9 device connected to the linking device, the VCR endpoint submits an Initiate request primitive to the client Type 9 VCR endpoint if the client Type 9 VCR endpoint is not already open. If the client Type 9 VCR endpoint is open, the local Type 5 VCR endpoint locally attaches itself to it.

If the request is accepted by the local VFD or if the Type 9 VCR is opened or already open, the newly created VCR endpoint returns a positive response that contains the OD Index for its new entry in the MIB list of automatic VCRs. This value is used to identify the VCR endpoint in all future messages sent on the VCR. If the request was sent using NMA Access and the underlying session is not a configuration session, then the VCR endpoint rejects all update requests that it receives from the client.

If the Initiate request is rejected, the newly created VCR endpoint returns a negative response, removes its entry in the MIB list of automatic VCRs, and deletes itself.

Client/server VCRs are terminated in the same ways that Type 9 VCRs are terminated. In addition, they are terminated when there is no longer activity on them. The states and processing associated with the VCR initiation and termination are described in 61158-6.

Figure 86 illustrates the use of the session to open Type 5 and Type 9 VCRs.

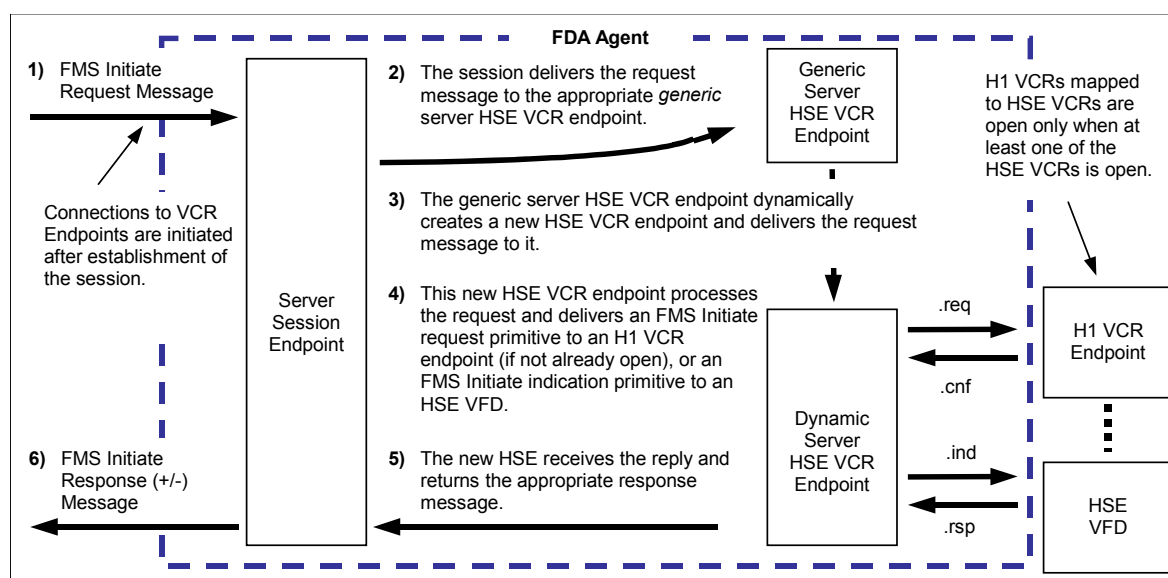


Figure 86 – VCR initiation

10.1.5.2.1.1 Keeping client/server VCRs and their sessions open

Client/server sessions and VCRs close automatically when there is no message traffic for a period of time known as the Inactivity Close Time. To prevent VCRs from terminating due to inactivity, the client VCR endpoint sends a confirmed Idle request message when its Idle timer expires. Its Idle timer is restarted using no more than one third of its inactivity close time each time it sends a message. The server VCR endpoint receiving the Idle request message returns the corresponding Idle Response message immediately. When there are no active VCRs on a session, no Idle messages are sent. This causes the session to timeout and close.

10.1.5.2.2 Publisher/subscriber

10.1.5.2.2.1 General

Publisher/subscriber sessions have been adapted from their counterpart Type 9 ARs. Publisher/subscriber sessions operate as follows:

- Messages are not sent by publisher or subscriber endpoints to establish sessions, in contrast to Type 9 publisher/subscriber connections.
- Published data is multicast using underlying connectionless services to a configured multicast address and to a configured port number.
- Publishers use unconfirmed service messages only to publish data.
- Each Device may be configured with zero, one, or more publisher session endpoints used to transmit published data.
- Subscribers are unknown to the publisher.
- There may be 0, 1, or more subscribers receiving published data.
- Each Device may be configured with zero, one, or more subscriber session endpoints used to receive published data.

Publisher/subscriber VCRs have been adapted from their Type 9 counterpart VCRs. Publisher/subscriber VCRs operate as follows:

- Messages are not sent by publisher or subscriber endpoints to establish VCRs.
- Detection of lost messages, duplicate messages, and late messages is possible, although recovery of lost messages is not possible. However, applications may have their own methods for recovering lost information (e.g. by accessing trend reports).
- Zero, one, or more publisher VCR endpoints may use the same publisher session endpoint to publish their data.
- A publisher VCR endpoint may be configured to publish data for a FBAP.
- A publisher VCR endpoint may be configured to republish data received by a Type 9 subscriber VCR endpoint.
- Each subscriber VCR endpoint receives data from a single publisher VCR endpoint.
- A subscriber VCR endpoint may be configured to deliver received data to a local application. A separate subscriber VCR endpoint is required for each local application that is to receive the published data.
- A subscriber VCR endpoint may be configured to deliver received data to a Type 9 publisher VCR endpoint, causing it to republish Type 5 data onto a Type 9 link. A separate subscriber VCR endpoint is required for each Type 9 link that is to republish the data.

10.1.5.2.2.2 Session and VCR establishment

Publisher/subscriber endpoints are established locally. There are no FDA messages exchanged to establish them.

10.1.5.2.2.3 Timeliness

Applications submit published data to the VCR configured in their link objects. The VCR submits it to its publisher session. The publisher session buffers it for up to the Transmit Delay Time defined for the session prior to submission to Subscriber endpoints deliver the received data to the subscribing applications or to Type 9 for republication (in linking devices) immediately upon receipt. Timeliness of published data is therefore dependent on the publisher Transmit Delay Time, on the latency of the network, and on the delay imposed by the underlying connectionless services and the FDA Agent within the receiving device.

The FDA Message Trailer also contains an optional timestamp that can be delivered to subscribing applications to allow them to determine whether or not the data was published within an acceptable time-window. This timestamp is applied to the message by the session endpoint at the time it submits the message for transfer. Use of this timestamp is configurable.

The timestamp is passed between the FDA Agent and the publishing/subscribing application through local means (they are not part of the Service interface).

10.1.5.2.3 Report distribution

Report Distribution sessions are identical to publisher/subscriber sessions with the following exceptions:

- Report sources correspond to publishers.
- Report sinks correspond to subscribers.
- Reports are identified by the FDA Address in the FDA Message header and by the *index* parameter in the FDA Portion of the message.
- Each report sink session endpoint delivers all received messages to all VCRs associated with it.

10.1.5.2.4 Terminating sessions

Sessions are terminated as a result of:

- Client/server sessions are terminated as a result of inactivity. Inactivity is defined as the failure of a session in the Open state to receive a valid message within the time period specified by the Inactivity Close Time session attribute.
- Client/server sessions are terminated as a result of termination of the underlying TCP connection.
- Client/server sessions are terminated as a result of receipt of an invalid message or a second Open Session request message. These messages are an indication that the data stream has lost synchronization or is communicating with a non-conformant partner. Whether or not subscriber session endpoints close as a result of receipt of an invalid message is implementation dependent.
- User request. User requests to close a session are implementation dependent.

When terminating a session because of inactivity, because of user request, or because of receipt of an invalid message or a second Open Session request message, the FDA Agent performs the steps below, in order. When terminating a session because the underlying TCP connection has been terminated, the FDA Agent performs steps 1, 4, and 5 only.

- 1) Stops accepting requests to send messages on the session,
- 2) Sends Type 9 Abort messages on all of its open Type 5 VCRs,
- 3) Terminates supporting TCP connections, if applicable.
- 4) Delivers all queued confirmation and error primitives, and discards all queued indication primitives,
- 5) Closes the corresponding VCR endpoints and issues a Type 9 Abort indication primitive for each.

NOTE Applications may close VCRs using the Type 9 Abort Service without closing the supporting FDA session.

10.1.5.3 FDA agent initialization

When the FDA Agent starts up, it initializes its server session endpoints using default attribute values. Once initialized, it is ready to send and receive messages for the SMK and for the LAN Redundancy entity. Once SMK has reached its operational state, it enables the FDA Agent to send and receive messages for all other device applications/VFDs.

10.1.5.4 Configuring sessions and VCRs

Before the device can become completely operational, its MIB needs to be configured. FDA session and VCR configuration information is contained in the MIB.

MIB configuration is performed using client/server *configuration* sessions.

Only NMA configuration sessions are allowed to change information in the MIB, in linking device Type 9 interface MIBs, and in Type 9 MIBs of Type 9 devices accessed through the linking device. Only one configuration session to a Type 5 device is allowed to be open at a time. After the configuration session has been opened, separate VCRs are opened to the Type 5 management agent and to each Type 9 management agent using the Initiate service using the NMA Access connect option.

Configuration sessions also provide configuration applications access to VFDs other than NMA VFDs. However, unlike access to the NMA VFDs, other sessions may concurrently write to these non-NMA VFDs.

Non-configuration client/server sessions provide read-only access to NMA VFDs after connecting to them using the Initiate service with the NMA Access connect option. Non-configuration client/server sessions provide read-write access to all other VFDs. The VCR is responsible for filtering update services submitted for MIB updating using these sessions. The following is the list of the update services that are filtered on these VCRs:

- Generic Initiate Download Sequence
- Generic Download Segment
- Generic Terminate Download Sequence
- Initiate Download Sequence
- Download Segment
- Terminate Download
- Request Domain Download
- Write
- Write with Subindex

Client/server sessions and client/server VCRs are defined by default and are not configured, unless they are used to support access from a Type 9 client across the Type 5 network. In this case, only the client endpoints are configured; the server endpoint acquires its attributes dynamically during endpoint establishment, as it does in all other cases.

Publisher/subscriber and Report Distribution Sessions and their associated VCRs are always configured.

10.1.5.5 Conveying services

10.1.5.5.1 General

The FDA Agent conveys FDA, VFD, SM and LAN Redundancy services and Type 9 services. VFD and Type 9 services are used to access VFDs and their objects. SM services are used to access SMKs. LAN Redundancy services are used to support redundant network interfaces. To convey these services the FDA Agent:

- receives service primitives from local applications and/or the Type 9 stack and converts them to FDA messages that it submits to the underlying socket interface.
- receives messages from its underlying socket interface and converts them into service primitives that it delivers to local applications and/or to the Type 9 stack. If a non-configuration session receives an update request message for the MIB, then the receiving VCR endpoint sends a negative response using error class = "access" and error code = "object access denied", instead of delivering the message to the MIB.
- receives SM service requests and responses that it submits to the underlying connectionless services interface.

- receives SM messages from its underlying connectionless services interface and delivers them as SM service indications and confirmations.
- receives LAN Redundancy service requests that it submits to the underlying connectionless services interface.
- receives LAN Redundancy messages from its underlying connectionless services interface and delivers them LAN Redundancy service indications and confirmations.

The following subclauses discuss these topics in more detail.

10.1.5.5.2 Buffered transfers

Session endpoints buffer data in accordance with their Transmit Delay Time attribute. This attribute allows endpoints to concatenate messages into a buffer and transfer them in a single transmission. See 10.1.5.7.4, Concatenating Messages, for a description.

10.1.5.5.3 Matching requests and responses

When the FDA Agent receives a confirmed request message from its underlying socket, it delivers or forwards it to the destination VFD and waits for the response. When the response is received, it forwards the response to the requester.

The Invoke Id carried in the Message Trailer is defined to match the response received from the VFD with the corresponding request. A value is placed into the Invoke Id field of the request and the same value is returned in this field in the response. While a request is outstanding (response not yet received), the client user should not use the same Invoke Id for another request that it sends to the same server using the same protocol identified in the Message Header (e.g. Type 9 or SM). It can be used, however, in requests sent to a different server, or to the same server using a different protocol.

The requester determines each Invoke Id value. The scope of the Invoke Id is specific to the message type, as shown in Table 412.

Table 412 – Scope of Invoke Id

Message Type	Invoke Id is unique within the scope of:
Confirmed SM messages	the requesting socket
FDA Open Session and Initiate messages	the requesting socket
Other Confirmed messages	the associated VCR identified in the FDA Address field of the message header

When a request is received by a linking device that it is to forward to an attached Type 9 link, it solves the following two problems.

First, to ensure uniqueness of the Invoke Id on a Type 9 VCR, the FDA Agent maps Invoke Ids received from different clients onto unique Type 9 VCR Invoke Ids. Once this has been done, responses received on a Type 9 VCR are mapped back to the requesting client, and the originating Invoke Id is recovered. How the FDA Agent implements this mapping is not specified by this document.

Second, it aborts the Type 9 client/server VCR if the response is not returned within a time-out period specified in the MIB. After aborting the Type 9 VCR, it returns a response to the requester using error class = "service" and error code = "response time-out". It then attempts to reestablish the Type 9 VCR. If it is unsuccessful, it aborts all VCRs that are associated with the Type 9 VCR.

10.1.5.5.4 Misordered messages

Misordered messages can be received when underlying connectionless services are used. Misordering means that consecutively received messages are not received in the same order that they were transmitted.

VCRs use message numbers to detect misordering. Message numbers are used to sequentially number each message transmitted by a VCR. Misordering is not detected for SM or LAN Redundancy message transfers.

Message numbers are an optional field defined for the Message Trailer. Their presence in the trailer is configured for publisher/subscriber and report distribution sessions and negotiated for client/server sessions. When they are defined for the session, misordering detection is enabled, regardless of the session endpoint type.

Using the message number, the types of misordering which can be detected are given in Table 413.

Table 413 – Types of misordering detectable by message numbers

Misordering Type	Description
none	the next in-sequence message number is received
duplicate messages	the same message number received more than once
late messages	lower message number received after a higher message number
missed messages	a reasonable (see note) gap in message numbers between consecutively received messages
loss of sync	an unreasonable gap in message numbers between consecutively received messages
NOTE The dividing line between reasonable and unreasonable is termed the "guard band". A gap in received messages numbers that exceeds the guard band indicates that it is unreasonable to believe that the gap represents missed messages. Instead, it is an indicator that synchronization has been lost with the sender. This could occur because the sender has gone through a reset, or that the sender has failed and a backup sender has taken over. The guard band is discussed below.	

All types of misordered messages received on client/server sessions or on report distribution sessions are delivered by the FDA Agent as they are received. This is the same as the delivery order provided when message numbers are not used.

For publisher/subscriber VCRs, the delivery of misordered message types is handled differently, as described in Table 414.

Table 414 – Delivery of misordered message types on publisher/subscriber VCRs

Misordering Type	Receiving subscriber VCR endpoint processing
none	The in-sequence message is delivered
duplicate messages	The duplicate message is discarded
late messages	The late message is discarded
missed messages	The gap is ignored and the received message is delivered
loss of sync	The loss of sync is ignored and the received message is delivered

For all types of VCRs, statistics can be gathered to support network management, as described in Table 415.

Table 415 – Statistics gathered per VCR

Misordering Type	Statistic gathered
none	The number of in-sequence messages received
duplicate messages	The number of duplicate messages received
late messages	The number of late message received
missed messages	The number of messages missed
loss of sync	The number of losses of sync detected

To determine the size of a reasonable gap in received message numbers, a *guard band* is used. The *guard band* provides a lower and upper bound for received message numbers. It is a configured number in the NMIB, and has a default value of 5.

The following algorithm uses the guard band and the value of the message number in the last delivered message (LDM) to determine the misordering type.

- When a subscriber VCR endpoint receives its first message, it delivers it and saves its message number as the initial value of its LDM.
- Following the receipt of the first message, when a subscriber VCR endpoint receives a new message, it determines the type of misordering, using the modulus of the message number for all calculations, as shown in Table 416.

Table 416 – Determination of misordering type at a subscriber VCR

Misordering Type	Calculation
none	Received Message Number = LDM + 1
duplicate messages	Received Message Number = LDM
late messages	$(\text{LDM} - \text{Guard Band}) \leq \text{Received Message Number} < \text{LDM}$
missed messages	$\text{LDM} + 1 < \text{Received Message Number} \leq (\text{LDM} + \text{Guard Band})$
loss of sync	$(\text{LDM} - \text{Guard Band}) > \text{Received Message Number}$ OR $\text{Received Message Number} > (\text{LDM} + \text{Guard Band})$

Figure 87 summarizes the misordering message handling.

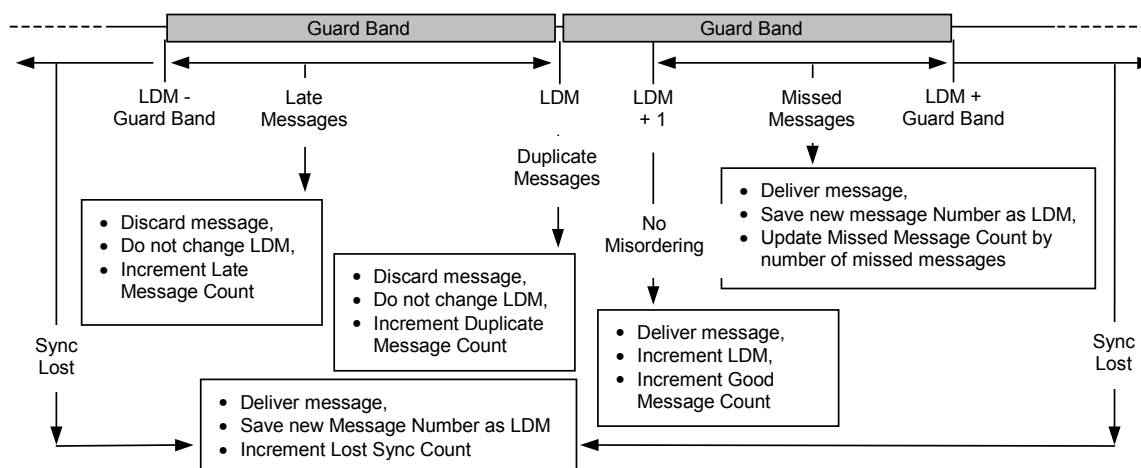


Figure 87 – Misordered message handling

10.1.5.5.5 Converting messages to service primitives

Table 417 specifies the mapping of the received message type to the appropriate primitive type based on the location of the SMK or VFD being accessed.

Table 417 – Mapping of received messages to primitives

Input Message Type	→	Final Destination	FDA Interface Type	Delivered To
SM Request	→	Local SMK	SM Request	Local SMK
SM Response	→	Local SMK	SM Response	Local SMK
SM Request	→	Type 9 SMK	Request Primitive	Local Type 5 SMK
Request Message	→	Local VFD	Indication Primitive	Local VFD
Response Message	→	Local VFD	Confirmation Primitive	Local VFD
Request Message	→	Type 9 VFD	Request Primitive	Type 9 Comm Stack Interface*
Response Message	→	Type 9 VFD	Response Primitive	Type 9 Comm Stack Interface*
NOTE For I/O Gateways, the Type 9 Comm Stack interface abstraction is maintained				

10.1.5.5.6 Converting service primitives to messages

Table 418 specifies the mapping of the received primitive type to the message type based on the location of the SMK or VFD originating the primitive.

Table 418 – Mapping of received primitives to messages

Requester	FDA Interface Type	→	Output Message Type
Local SMK	Request Primitive	→	Request Message
Local SMK	Response Primitive	→	Response Message
Local VFD	Request Primitive	→	Request Message
Local VFD	Response Primitive	→	Response Message
Type 9 Comm Stack Interface*	Indication Primitive	→	Request Message
Type 9 Comm Stack Interface*	Confirmation Primitive	→	Response Message
NOTE For I/O Gateways, the Type 9 Comm Stack interface abstraction is maintained			

10.1.5.5.7 Receiving messages

After initialization, the FDA Agent listens to the SM and FDA Ports and to the other configured ports for FDA Messages. It distinguishes and delivers the following types of messages, as shown in Figure 88 through Figure 92.

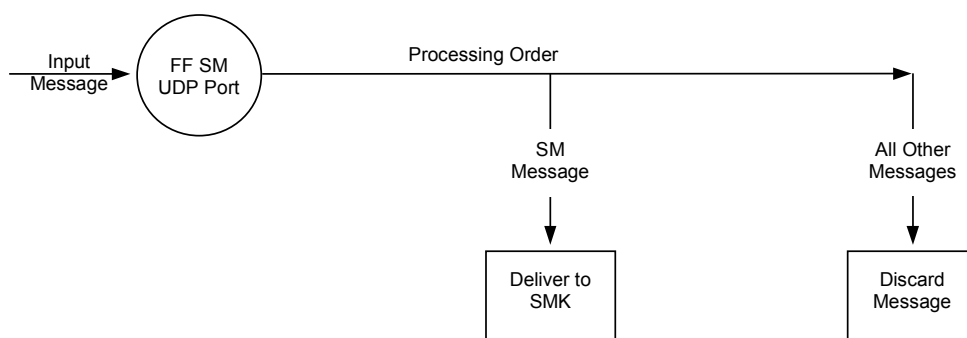


Figure 88 – FF SM port message processing order

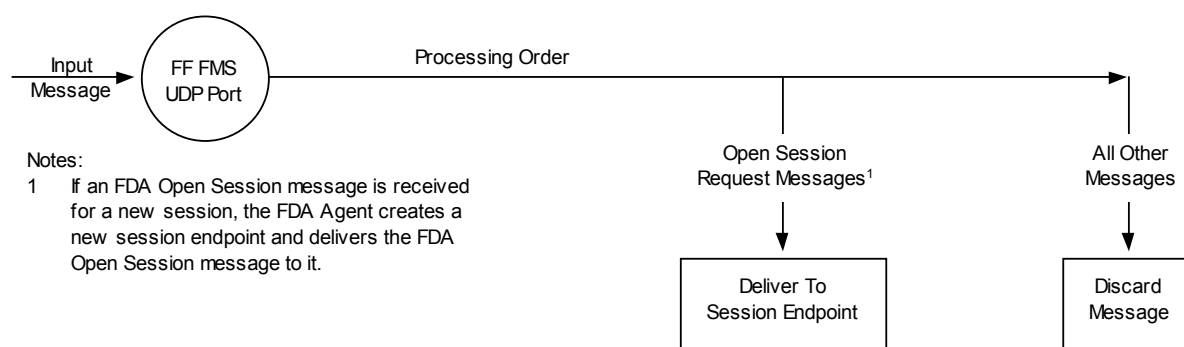


Figure 89 – FF FDA port message processing order

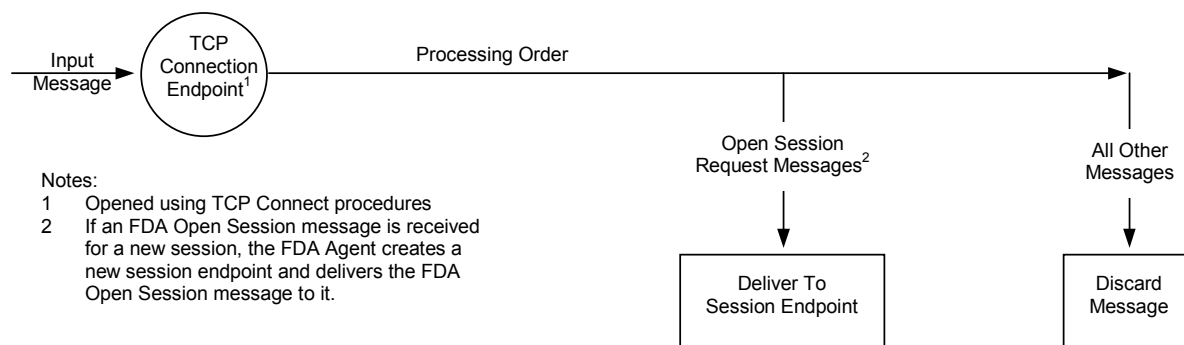


Figure 90 – FF TCP connection message processing order

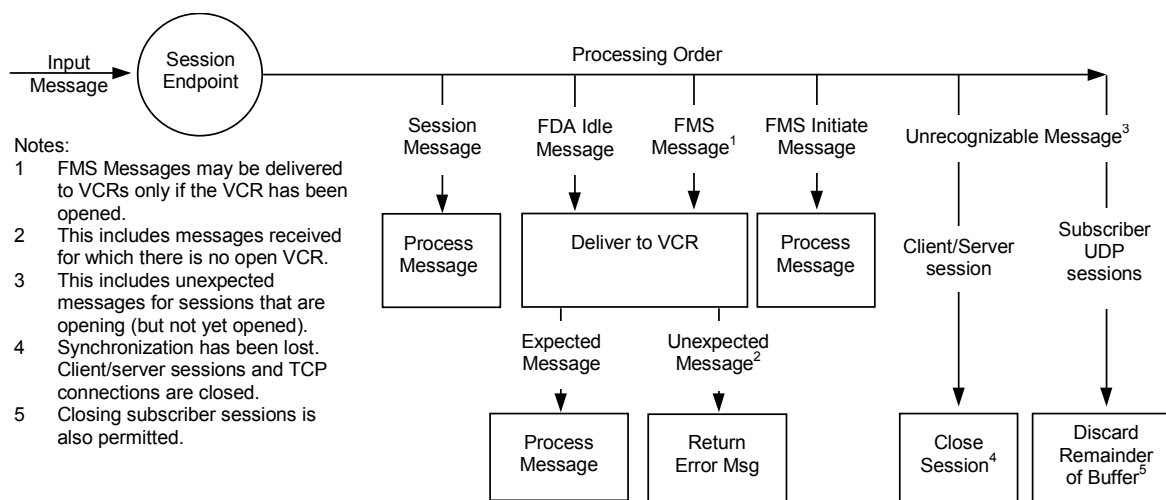


Figure 91 – Session endpoint message processing order

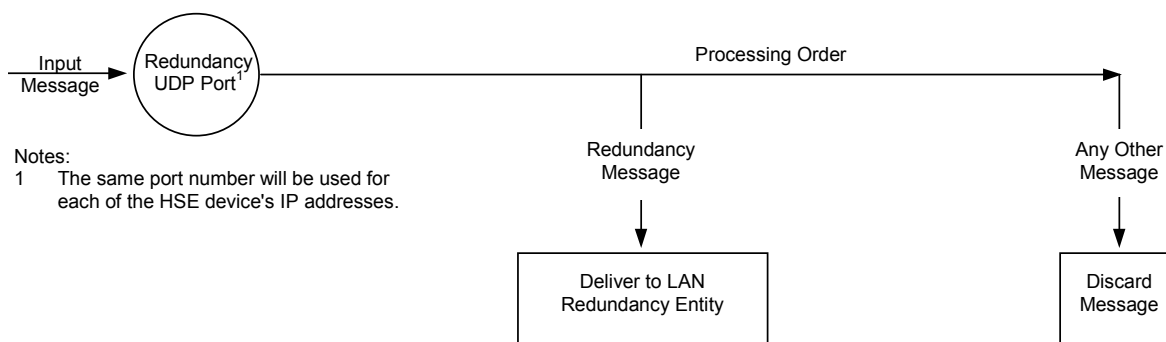
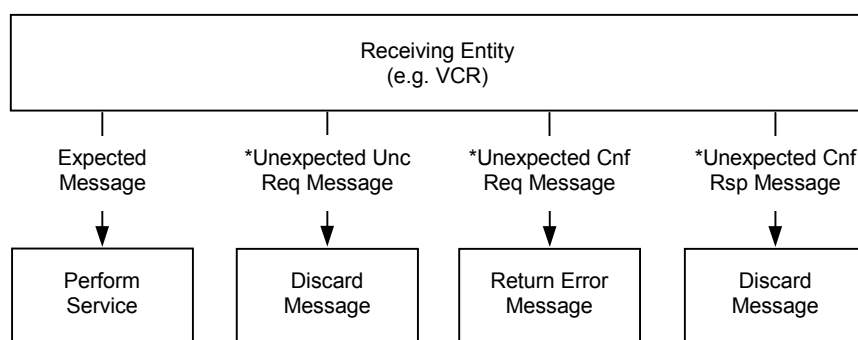


Figure 92 – FDA LAN redundancy port message processing order

Figure 93 summarizes the processing of confirmed and unconfirmed messages by the receiving entity.



* An unexpected message is a message that is valid message that is not expected by the entity in its current state.

Figure 93 – Message processing by receiving entity

10.1.5.5.8 Starting and stopping timers

10.1.5.5.8.1 VCR idle timer

The VCR Idle Timer is used to keep client/server VCRs open in the absence of user requests. The client VCR endpoint restarts this timer each time it sends a message. Server VCR endpoints do not restart this timer.

10.1.5.5.8.2 Inactivity close timer

The Inactivity Close Timer controls how long client and server VCR and session endpoints remain open. Client and server VCR and session endpoints restart this timer each time they receive a message.

10.1.5.5.8.3 Transmit delay timer

The Transmit Delay Timer controls how long a session endpoint waits after accepting a message for transfer to transfer it. The timer is restarted when the first message is placed into the transmit buffer after the buffer has been initialized or after it has been emptied by the previous transmission. It is stopped when the buffer is filled, causing the buffer to be transmitted and then emptied.

10.1.5.6 Identifying Type 5 and Type 9 applications

10.1.5.6.1 General

FDA Messages may be sent or received by either Type 5 or Type 9 applications/SMKs. To identify the sending or receiving application/SMK, a combination of the network address and the FDA Address is used. The FDA Address contains the Link Id and the Selector. Each of these identifiers is discussed below.

10.1.5.6.2 Network addresses and subnet addresses

Network addresses are used as source and destination address parameters for the sending and receiving devices. Source addresses are individual addresses while destination addresses may be individual or multicast addresses. When conveyed within an FDA message, the Network Address uses a 16 byte address format.

Subnet addresses are defined as the first N bits of a device's individual network address, starting with the most significant bit. The number N is configurable and is maintained in the MIB.

For multicast messages, the actual device originating the message is not significant, but the Subnet where the message originated is. Therefore, the Subnet of the sender is used to qualify the FDA Address contained in the message header of multicast messages.

The Network Addresses defined in this specification are listed in Table 419.

Table 419 – Defined network addresses

Address Name	Address Use	Value
Individual	Address of device interface	Assigned by DHCP
Operational Network Address	Address of device interface used in Find Tag Reply messages	Assigned during device assignment
SM Multicast	Multicast address of SMKs	Reserved
Diagnostic Message Interface A Send Address	Multicast address used to send LAN Redundancy Diagnostic messages from network interface A.	Assigned using the LAN Redundancy Put Information service
Diagnostic Message Interface B Send Address	Multicast address used to send LAN Redundancy Diagnostic messages from network interface B.	Assigned using the LAN Redundancy Put Information service
Publisher/Report Source multicast address	Multicast address to which publishers and report sources send their messages	Configured in the MIB

Their use is shown in Table 420.

Table 420 – Use of network addresses

Address Use	Address Type	Used to Identify
Source	Individual	Message sender's subnet and the Device within the subnet
Destination	Individual	Intended receiver's subnet and the Device within the subnet
Destination	Multicast	Group of intended receivers

10.1.5.6.3 The selector

10.1.5.6.3.1 General

The Selector is the low order 16-bits of the FDA Address. It identifies a specific communication endpoint within the identified device or Type 9 Link, or it is used to complete a 32-bit flat address. Each session type uses a different type of selector. The Selector may be hierarchical or flat.

10.1.5.6.3.2 Server selectors

10.1.5.6.3.2.1 General

Server Selectors are used in the FDA Address field of the FDA Message Header in both request and response messages to identify a VCR endpoint used to access the server VFD of a client/server session.

10.1.5.6.3.2.2 Scope

Server VCR Selectors that identify a Type 9 VCR are qualified by the Link Id and are unique within the identified Type 9 link. Server VCR Selectors that identify a Type 5 VCR have a Link Id of 0 and are unique within the device. They are not required to be unique across devices.

10.1.5.6.3.2.3 Use

Server VCR endpoint selectors are used only for Client/Server sessions as shown in Table 421.

Table 421 – Use of endpoint selectors in server VCRs

Session Endpoint Type	Use
Server	Identifies the server VCR endpoint within a Type 5 or Type 9 device. For Type 5 VCR selectors, two values are used. One value is used in the Initiate Request message to identify a generic server VCR endpoint for the associated VFD. The second is the selector for the VCR endpoint that was dynamically created as a result of processing the Initiate request. This selector is the OD index of the Automatic VCR in the MIB.

10.1.5.6.3.3 Publisher/subscriber selectors**10.1.5.6.3.3.1 General**

Publisher Selectors are used in the FDA Address field of the FDA Message Header to identify the publisher data buffer for publisher/subscriber sessions. The index parameter contained in the message identifies the individual parameter being published.

10.1.5.6.3.3.2 Scope

Publisher Selectors are unique within the Link Id value.

10.1.5.6.3.3.3 Use

Publisher Selectors are used only for Publisher/Subscriber sessions as shown in Table 422.

Table 422 – Use of endpoint selectors in publisher VCRs

Session Endpoint Type	Use
Publisher	Identifies the publisher data buffer within a Type 5 or a Type 9 device. They may be hierarchical, or flat within the link id value. When hierarchical, the first byte identifies the Type 9 device and the second identifies the DL-connection endpoint within the Type 9 device.
NOTE Publisher and subscriber VCR endpoints are stored as static VCRs in the MIB. Their OD Indexes are not used as selector values.	

10.1.5.6.3.4 Report distribution selectors**10.1.5.6.3.4.1 General**

Report Source Selectors are used in the FDA Address field of the FDA Message Header to identify the Report Source VFD.

10.1.5.6.3.4.2 Scope

Report Source Selectors are unique within the Link Id value.

10.1.5.6.3.4.3 Use

Report Source Selectors are used only for Report Distribution sessions as shown in Table 423.

Table 423 – Use of endpoint selectors in source VCRs

Session Endpoint Type	Use
Report Source	Identifies the report source VFD within a device or Type 9 link. They may be hierarchical or flat within the link id value. When hierarchical, the first byte identifies the Type 9 device and the second identifies the DL-service access point within the Type 9 device.
NOTE Report source and report sink VCR endpoints are stored as static VCRs in the MIB. Their OD Indexes and are not restricted to 16-bit values, nor are they used as selector values.	

10.1.5.7 Using underlying connectionless and connection-oriented services

10.1.5.7.1 General

Connectionless service use is the default used to support sessions. Connection-oriented services may be used as an option for client/server sessions. The selection of which protocol to use is made by the client or by the configuration application if the client endpoint is configured.

10.1.5.7.2 Connectionless

Connectionless services between two or more FDA Agent session endpoints are message-oriented. Therefore, each message buffer always contains an integral number of FDA messages. FDA messages are never split across message buffers.

Although connectionless services can be used for all types of FDA Agent sessions, they are always used to support FDA Agent sessions that require multicast delivery or that do not require reliable, flow-controlled sequenced message delivery.

10.1.5.7.3 Connection-oriented

Connection-oriented services are used between two and only two FDA Agent session endpoints. Connections are only used to support FDA Agent client/server sessions. That is, they are not used to support the sessionless transfer of SM messages or to support multicast transfers.

Connection-oriented services should be used for FDA Agent sessions that wish to exchange data in a reliable, flow-controlled, and sequenced manner.

10.1.5.7.4 Concatenating messages

Connectionless and connection-oriented services transparently transfer the contents of buffers between applications. Neither inspect the contents or make changes to them.

FDA sessions place their messages into buffers before sending them. The FDA Message structure has been designed to allow sessions to place (concatenate) more than one message into the same buffer for transfer. This can improve network utilization and can reduce the number of interrupts at the receivers. Because FDA SM Messages are not sent on sessions, they are not concatenated into buffers.

The number of messages that can be concatenated together and sent is limited by the size of the sending buffer. Buffers are sent as soon as they are filled. They always are sent with an integral number of messages in them when using underlying connectionless services. When a message is submitted for transfer using underlying connectionless services that would overflow the buffer, the buffer is sent without it, and the message is inserted as the first message in the next buffer to be sent. When underlying connection-oriented services are used, messages may span buffers because the transfer and delivery may be stream oriented.

To prevent buffers from waiting indefinitely to be filled, session endpoints contain an attribute called the Transmit Delay Time. This attribute dictates how long the FDA Agent waits for the buffer to fill. If it is not filled within this time period, the FDA Agent transfers its contents and prepares a new buffer for the next transfer.

10.1.5.7.5 Port numbers

10.1.5.7.5.1 General

Ports identify the access point of an application to the underlying layer. There are three port addresses used by the FDA. One is used exclusively for SM Device Annunciation. It is referred to as the Annunciation Port. The second port is used for SMK access. It is referred to as the SM Port.

The third reserved port is used for receiving FDA Open Session request messages. It is referred to as the FDA Port. Positive responses to Open Session request messages are sent from previously unused ports, and these ports are used to transfer all subsequent session messages. Negative responses to Open Session request messages are sent from the FDA Port.

Data paths between applications are identified by a combination of the source and destination Network Address/Port Number.

10.1.5.7.5.2 Scope

Port numbers are unique within an individual network address. The same port number may be used for connectionless or connection-oriented sockets at the same time without confusion. The underlying protocol qualifies the port number.

10.1.5.7.5.3 Use

FDA Agent Session types use port numbers in conjunction with the network addresses as shown in Table 424 through Table 433. In these tables, the entries in the columns show the addresses to which the endpoint is bound.

Table 424 – Network address and port numbers for device annunciation

Transfer Type	Device SMK annunciates from:		Configuration AP Listens to:
Multicast	Device Network Address Any Unused Port or the Reserved FDA Port	→	Reserved SM Multicast Network Address Reserved Annunciation Port

Table 425 – Network address and port numbers for set/clear assignment info and clear address

Transfer Type	Configuration AP sends and receives using:	→	SMK listens to and responds from:
Connectionless Unicast	Configuration AP Network Address Any Unused Port	←	Device Operational Network Address Reserved SM Port

Table 426 – Network address and port numbers for SM identify

Transfer Type	Requesting AP Sends Request From:		Responding SMK: listens to	responds from
Connectionless Unicast	Requester Network Address	→	Device Operational Network Address Reserved SM Port	Device Operational Network Address
Multicast	Any Unused Port	←	Reserved SM Multicast Network Address Reserved SM Port	Reserved SM Port

Table 427 – Network address and port numbers for SM find tag

Transfer Type	Requesting AP Sends Query From:		Replying SMK: listens to	replies from
Connectionless Unicast	Requester Network Address Any Unused Port	→	Device Operational Network Address	Device Operational Network Address
		←	Reserved SM Port	
Multicast			Reserved SM Multicast Network Address	Reserved SM Port
			Reserved SM Port	

Table 428 – Network address and port numbers for clients and servers (part 1)

Transfer Type	Client AP opens session from:		Server Device: listens to	responds from
Connectionless Unicast	Client Network Address Any Unused Port	→	Device Operational Network Address	Device Operational Network Address
		←	Reserved FDA Port	Any Unused Port (see note)

NOTE Also used as the port for receiving and sending all subsequent messages on the session. Negative responses are returned from the FDA Port if the server is unable to acquire a port for responding.

Table 429 – Network address and port numbers for clients and servers (part 2)

Transfer Type	Client AP opens TCP Connection from:		Server Device: listens to	responds using
Connection-oriented	Client Network Address Any Unused Port	→	Device Operational Network Address	Device Operational Network Address
		←	Reserved FDA Port	The newly established TCP connection (see note)

NOTE The TCP connection is used by both endpoints for sending and receiving all messages on the session, including the FDA Open Session request and response messages.

Table 430 – Network address and port numbers for publishers and subscribers

Transfer Type	Publisher device sends from:		Subscriber device listens to:
Multicast	Publisher Device Operational Network Address Any Unused Port	→	Configured Multicast Network Address Configured Port*

* Multicast addresses and Port Numbers are configured as the destination address/port for publisher endpoints. Subscriber endpoints are configured to receive messages at these addresses/ports. The Configured Port is assigned from the unused range of port numbers. Reserved port numbers or port numbers used for other purposes may not be used.

Table 431 – Network address and port numbers for report distribution

Transfer Type	Report Source device sends from:		Report Sink device listens to:
Multicast	Report Source Device Operational Network Address Any Unused Port	→	Configured Multicast Network Address* Configured Port*

* Multicast addresses and Port Numbers are configured as the destination address/port for report source endpoints. Report sink endpoints are configured to receive messages at these addresses/ports. The Configured Port is assigned from the unused range of port numbers. Reserved port numbers or port numbers used for other purposes may not be used.

Table 432 – Network address and port numbers for LAN redundancy get and put information

Transfer Type	Client AP opens session from:		Server Device: listens to	responds from
Unicast	Client Operational Network Address Any Unused Port	→	Device Operational Network Address	Device Operational Network Address
		←	Assigned LAN Redundancy port	Assigned LAN Redundancy port

Table 433 – Network address and port numbers for LAN redundancy diagnostics

Transfer Type	Device sends from:		Devices listen to:
Multicast	Device Operational Network Address Any Unused Port	→	Assigned Multicast Address
		←	Assigned LAN Redundancy port

10.1.6 Support for configuration management

Configuration data for the FDA Agent is maintained in the MIB. The FDA Agent also uses the bridge data structures in the MIB if it is in a Linking Device and is not supported by Type 9 bridging capabilities. It uses the bridge data structures to perform republishing and report forwarding operations normally performed by the bridge.

10.2 ASEs

10.2.1 Virtual field device ASE

10.2.1.1 Overview

Type 5 VFD is the same as the Type 9 VFD except for the VCR object and the Initiate Service. They replace those found in the Type 9 specification. All other Type 9 objects and services apply. They are not repeated here.

10.2.1.2 Simple virtual field device model class specifications

10.2.1.2.1 Class overview

This subclause provides the formal class definition for the VCR class. VCRs are analogous to Type 9 VCRs.

10.2.1.2.2 VCR formal model

The VCR class specifies the attributes and services of the VCR. Its parent class "top" indicates the top of the FAL class tree.

FAL ASE:		VFD ASE
CLASS:		VCR
CLASS ID:		Not used
PARENT CLASS:		TOP
SYSTEM MANAGEMENT ATTRIBUTES:		
1	(m) Attribute:	VCR Id
2	(m) Attribute:	Related AREP Id
3	(m) Attribute:	VCR Type
4	(m) Attribute:	VCR State
5	(m) Attribute:	VCR User Id
6	(m) Attribute:	FDA Address
7	(c) Constraint	VCR Type = LOCAL SUBSCRIBER TYPE 9 PUBLISHER LOCAL REPORT SINK TYPE 9 REPORT SOURCE
7.1	(m) Attribute:	Subnet Mask
9	(c) Constraint	VCR Type = LOCAL PUBLISHER
9.1	(m) Attribute:	On Change Threshold
7	(c) Constraint	VCR Type = LOCAL CLIENT TYPE 9 SERVER LOCAL SERVER TYPE 9 CLIENT LD TYPE 9 MIB AGENT
7.1	(m) Attribute:	Inactivity Close Time
SERVICES:		
1	(o) Ops Service:	Initiate

10.2.1.2.3 System management attributes

The following attributes are accessible through System Management. They are not otherwise accessible.

Local VCR Id

This attribute identifies the VCR.

Related AREP Id

This attribute is the numeric identifier of the related AREPs used to send or receive messages. This value is 0 when the VCR is used to republish or forward reports between two Type 9 ARs in a linking device. The value 0 signifies that there is no related session endpoint.

VCR Type

This attribute indicates the type of the VCR. Each of the types uses the Related AREP to convey services. Its value are as shown in Table 434.

Table 434 – VCR types

VCR Type	Use
LOCAL CLIENT	The VCR receives confirmed request primitives from the AP.
LOCAL SERVER	The VCR delivers confirmed indication primitives to the AP for processing.
LOCAL PUBLISHER	The VCR publishes data originating in the AP.
LOCAL SUBSCRIBER	The VCR subscribes to published data and delivers it to the AP.
LOCAL REPORT SOURCE	The VCR distributes reports originating in the AP.
LOCAL REPORT SINK	The VCR receives reports and delivers them to the AP.
TYPE 9 SERVER ¹	The VCR is a server on a Type 9 link and a client on a Type 5 network. The VCR receives confirmed indication primitives from the Type 9 link and forwards them onto the Type 5 network.
TYPE 9 CLIENT ¹	The VCR is a client on a Type 9 link and a server on a Type 5 network. The VCR receives confirmed indication primitives from the Type 5 network and forwards them onto a Type 9 link.
TYPE 9 SUBSCRIBER ¹	The VCR is a subscriber on a Type 9 link and a publisher on a Type 5 network. The VCR receives published data from the Type 9 link and republishes it onto the Type 5 network.
TYPE 9 PUBLISHER ¹	The VCR is a publisher on a Type 9 link and a subscriber on a Type 5 network. The VCR receives published data from the Type 5 network and republishes it onto the Type 9 link.
TYPE 9 REPORT SINK ¹	The VCR is a report sink on a Type 9 link and a report source on a Type 5 network. The VCR receives reports from the Type 9 link and forwards them onto the 5 network.
TYPE 9 REPORT SOURCE ¹	The VCR is a report source on a Type 9 link and a report sink on a Type 5 network. The VCR receives reports from the Type 5 network and forwards them onto the Type 9 link.
LD TYPE 9 MIB AGENT ¹	The VCR delivers confirmed indication primitives to the Type MIB Agents for processing (applicable only to linking devices).
NOTE Type 9 VCRs are present only in Linking Devices	

VCR State

This attribute specifies the state of the VCR. Valid states are defined in IEC 61158-6.

VCR User Id

This conditional attribute is the FDA Address that identifies the user of the VCR. The use of this attribute is shown in Table 435.

Table 435 – Use of VCR user id

VCR Type	Contents (represented as an FDA Address):
LOCAL SERVER	The FDA Address contained in the Type 9 Initiate Request used to open the VCR.
LD TYPE 9 MANAGEMENT AGENT	Type 9 link of the Management Agent. The N.S portion of the FDA Address is unused and set to 0.
TYPE 9 SERVER	Type 9 Server VCR used to receive requests from the Type 9 link.
TYPE 9 CLIENT	Type 9 Client VCR used to send requests on the Type 9 link.
TYPE 9 SERVER	Type 9 Server VCR used to receive requests from the Type 9 link.
TYPE 9 PUBLISHER	Type 9 Publisher VCR used for publishing onto the Type 9 link.
TYPE 9 SUBSCRIBER	Type 9 Subscriber VCR used to subscribe to data on the Type 9 link.
TYPE 9 REPORT SOURCE	Type 9 Report Source VCR used for sending reports onto the Type 9 link
TYPE 9 REPORT SINK	Type 9 Report Sink VCR used to receive report distributions from the Type 9 link.

FDA Address

This attribute contains a reference that is conveyed with each service to or from the corresponding VCR endpoint(s). Its values are VCR Type dependent as shown in Table 436.

Table 436 – Use of FDA address

VCR Type	Use
LOCAL CLIENT	The Id of the associated server VCR or Type 9 AREP.
LOCAL SERVER	The VCR Id of the LOCAL SERVER VCR endpoint.
LOCAL PUBLISHER	"Published-Data" reference used to identify the published data buffer. - This address is taken from the 32-bit Type 9 flat DLCEP address space. - The Type 5 Subnet Mask of the publisher qualifies this "Published-Data" reference.
LOCAL SUBSCRIBER	- The reference for the publisher data buffer that is "Subscribed-To". - The Type 5 Subnet Mask of the publisher qualifies this "Subscribed-To" reference.
LOCAL REPORT SOURCE	"Reporting VFD" reference used to identify the VFD sending the report messages. - This address is taken from the 32-bit Type 9 flat DLSAP address space. - The Type 5 Subnet Mask qualifies this "Reporting VFD" reference. - This VFD identifier qualifies the index parameter value contained in the report messages in the service-specific parameters (each source VFD may send one or more report messages, each with a different Index).
LOCAL REPORT SINK	Not Used. A single LOCAL REPORT SINK VCR may receive reports from more than one report source. Therefore, the APDUs received from the related AREP may contain a different FDA Address value.
TYPE 9 SERVER	The numeric id of the Type 9 Server AREP.
TYPE 9 CLIENT	Not Used.
TYPE 9 PUBLISHER	- If the Related AREP Id is non-zero, then this is the <i>FDA Address</i> in received Type 5 publisher APDUs that is "Subscribed-To". The Type 5 Subnet Mask of the publisher qualifies this "Subscribed-To" reference. - If the Related AREP Id is zero, then this value is the Type 9 DLCEP used by the Type 9 Subscriber VCR to receive published data. - The received data is republished onto the Type 9 link using the VCR User Id attribute.
TYPE 9 SUBSCRIBER	Type 9 data link address of the published data buffer.
TYPE 9 REPORT SOURCE	Not Used. A single TYPE 9 REPORT SOURCE VCR may receive reports from more than one report source. Therefore, the APDUs received from the related AREP may contain a different FDA Address value.
TYPE 9 REPORT SINK	Not Used. A single TYPE 9 REPORT SINK VCR may receive reports from more than one report source. Therefore, the reports received from the Type 9 VCR may contain a different Type 9 source address.

Type 5 Subnet Mask

This attribute identifies the subnet of FDA Address and is used to qualify it for service indication primitives received on the VCR.

On Change Threshold

This conditional attribute is present only in LOCAL PUBLISHER VCRs. It specifies the percentage of change to be exceeded before the value of the published data is considered to be changed. If its value is 0, then every value is published using the Type 9 Information Report service, whether or not it has changed. The default value is 0.

If its value is not 0, then changed values are published using the Type 5 Information Report On Change service. The On Change capability operates as follows. The acceptable difference is calculated by multiplying the published data range, specified by the application, by the On Change Threshold percent. If the data range between high and low values were 20 and the On Change Threshold were 10%, the acceptable difference would be 2. When a published variable value is available, the absolute value of its difference with the last published value is calculated. If the new value is 17 and the previous value was 14, then the difference is 3. If this difference is greater than the acceptable difference (3 is greater than 2), then the value is considered to have changed and is submitted for transfer.

If the value has not been transferred for "refresh rate" number of publisher execution cycles, the latest value is transferred. The refresh rate attribute is contained in the MIB. Support for the On Change capability is optional, although support for this attribute is mandatory.

NOTE Whether this capability is implemented is not specified. If it is implemented in the AE, then the AE needs to be able to decode the data portion of the message to determine the value that is being published. If the application implements this capability, then it needs to read the value of this attribute to determine whether or not to submit the data for publishing.

Inactivity Close Time

This attribute controls how long a client VCR endpoint remains open without receiving a message and it also specifies the time interval that the client VCR endpoint uses to control the sending of Idle messages. Its value is inherited from its related AREP.

10.2.1.2.4 Services

Initiate

This optional service is used to open the VCR for an Application Relationship.

10.2.1.3 Virtual field device ASE service specification

10.2.1.3.1 Services supported

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Initiate

10.2.1.3.2 Initiate service

10.2.1.3.2.1 Service overview

This service is used to open to remote VCR endpoint. The remote endpoint can be in a Type 5 device or in a Type 9 device that is connected to the Type 5 device by a linking device.

10.2.1.3.2.2 Service primitives

The service parameters for this service are shown in Table 437.

Table 437 – Initiate

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
FDA Address	M	M(=)		
Connect Option	M	M(=)		
Access Protection Supported Calling	M	M(=)		
Password Calling	M	M(=)		
Access Groups Calling	M	M(=)		
Version OD Calling	M	M(=)		
Profile Number Calling	M	M(=)		
Physical Device Tag	M	M(=)		
Result(+)			S	S(=)
VCR Id			C	C(=)
Version OD Called			M	M(=)
Profile Number Called			M	M(=)
Result(-)			S	S(=)
VCR Id			C	C(=)
Error Class and Code			M	M(=)
Additional Code			U	U(=)
Additional Description			U	U(=)

Argument

This parameter carries the parameters of the service invocation.

FDA Address

This parameter identifies the remote VCR endpoint. Its value is dependent on the value of the Connect Option

Connect Option

This parameter indicates which option is to be used to open the VCR. Its values are:

Connect Option	Description
VCR Selector	This option allows the requesting application to provide the VCR Selector that is to be used to connect to the VFD. If the VFD is a Type 5 VFD, then the VCR Selector is a generic selector for accessing the VFD. If the VFD is a Type 9 VFD, then the VCR Selector is the Id of the Type 9 Client AREP to be used to access the Type 9 device. This Type 9 Client AREP cannot be associated with the standard MIB access VCR endpoint in the Type 9 device.
MIB ¹	This option allows the requesting application to indicate that it desires to connect to a Type 9 or Type 5 MIB VFD. The link id portion of the FDA Address field indicates whether or not the MIB is in the Type 9 or Type 5 device.
FBAP ²	This option allows the requesting application to indicate that it desires to connect to a Type 9 or Type 5 Function Block Application Process (FBAP) VFD. The link id portion of the FDA Address field indicates whether or not the MIB is in the Type 9 or Type 5 device. The node id portion of the FDA Address field identifies the node containing the FBAP, and the selector portion of the FDA Address field is an ordinal number that starts with "1" to indicate which FBAP is to be accessed
NOTE 1 The MIB can only be accessed using this Connect	
NOTE 2 The FBAP can be accessed also using the VCR Selector Connect Option.	

Access Protection Supported Calling

This parameter contains the value of the calling VFD's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password Calling

This parameter specifies the password to be used for the access to all objects of the server on this VCR. If access with password is not to be used on this VCR, its value is null.

Access Groups Calling

This parameter specifies the client's membership in specific access groups. The membership applies for the access to all objects of the server on this VCR.

Version OD Calling

This parameter specifies the version of the client's OD. Its value is null if the client does not contain OD.

Profile Number Calling

This parameter contains the value of the calling VFD's Profile Number attribute if it has one. If it does not, its value is null.

Physical Device Tag

This parameter contains the value of the SMK Physical Device Tag of the device that contains the called VFD if it has one. If it does not, its value is null.

Result(+)

This selection type parameter indicates that the service request succeeded.

VCR Id

This conditional attribute identifies the VCR endpoint from which the response was sent.

Version OD Called

This parameter shall specify the version of the server's OD.

Profile Number Called

This parameter contains the value of the called VFD's Profile Number attribute.

Result(-)

This selection type parameter indicates that the service request failed.

VCR Id

This conditional attribute identifies the VCR endpoint from which the response was sent. It is present when the underlying AREP supports multiple concurrent open VCRs.

Error Class and Code

This parameter shall provide the reason for the failure.

Additional Code

This optional parameter provides an additional code that is associated with the error.

Additional Description

This optional parameter provides 16 octets of additional information that is associated with the error.

10.2.1.3.2.3 Service procedure

This service operates through a queue. The Confirmed Service Procedure specified in 4.6 applies to this service. The service specific processing is as follows:

If the receiving FDA Agent is not able to locate the VCR identified by the FDA Address, or if the VCR is not AP SERVER or TYPE 9 CLIENT, or if the Physical Device Tag in the request does not match the Physical Device Tag of the device's SMK, the FDA Agent returns a negative response containing the FDA Address from the request.

If the request identifies a local VFD, the FDA Agent constructs a new LOCAL SERVER VCR endpoint and delivers the request to it for processing.

If the request identifies a TYPE 9 CLIENT, the FDA Agent constructs a new local TYPE 9 CLIENT and associates it with the Type 9 Client AREP. If the Type 9 Client AREP is not already open, the FDA Agent opens it using the Type 9 Initiate AP ASE service. This permits more than one TYPE 9 CLIENT to be associated with the Type 9 Client AREP.

If the request accepted, the response is returned to the client by the FDA Agent contains the VCR Id of the newly created local VCR. This Id is to be used by the client for subsequent accesses on the VCR.

If the request is rejected, the FDA Agent deletes the VCR object if it created one, and returns a negative response.

If the VCR was opened to a MIB on an AR that does not support update requests, the server FDA Agent issues a negative response if it receives an update request from the client without delivering the service indication to the System Management MIB.

10.2.2 System management kernel ASE

10.2.2.1 Overview

The FAL system management kernel model defines an object that supports a system management agent in device. This system management entity is referred to as the System Management Kernel (SMK) because it provides basic capabilities to add or delete the device from the network and to locate objects within the device.

10.2.2.2 FAL system management kernel class specification

10.2.2.2.1 System management kernel formal model

FAL ASE: **SYSTEM MANAGEMENT KERNEL ASE**

CLASS: **SYSTEM MANAGEMENT KERNEL**

CLASS ID: Not used

PARENT CLASS: TOP

ATTRIBUTES:

- | | | | |
|----|-----|----------------|-------------------------------|
| 1 | (m) | Key Attribute: | Device Id |
| 2 | (m) | Attribute: | Physical Device Tag |
| 3 | (m) | Attribute: | SMK State |
| 4 | (m) | Attribute: | Duplicate Detection State |
| 5 | (m) | Attribute: | LAN Redundancy Socket Address |
| 6 | (m) | Attribute: | Operational Network Address |
| 7 | (m) | Attribute: | Annunciation Repeat Time |
| 8 | (m) | Attribute: | Device Type |
| 9 | (m) | Attribute: | Device Redundancy State |
| 10 | (m) | Attribute: | Device Index |

11	(m)	Attribute:	Max Device Index
12	(m)	Attribute:	SMK Attributes Version Number
13	(m)	Attribute:	Device Version Number
14	(m)	Attribute:	Operational Powerup

SERVICES:

1	(m)	OpsService:	Find Tag Query
2	(m)	OpsService:	Find Tag Reply
3	(m)	OpsService:	Identify
4	(m)	OpsService:	Device Annunciation
5	(m)	OpsService:	Set Assignment Info
6	(m)	OpsService:	Clear Assignment Info
7	(m)	OpsService:	Clear Address

10.2.2.2.2 Attributes

Device Id

This key attribute specifies the vendor specific identification for the device.

Physical Device Tag

This attribute specifies the site administered name (tag) assigned to the device.

SMK State

The attribute specifies the operational state of the SMK. Its values are:

- No Tag

Duplicate Detection State

The attribute specifies whether or not the SMK has detected another device with its Physical Device Tag (PD Tag) or its Device Index. Its values are:

- No Duplicates Detected
- Duplicate Pd Tag Detected
- Duplicate Device Index Detected
- Duplicate Pd Tag and Device Index Detected

LAN Redundancy Socket Address

The attribute specifies the socket address used by the device to receive LAN Redundancy APDUs.

Operational Network Address

This attribute specifies the network address used by the device. More than one address is used when Local Area Network (LAN) redundancy is supported by the device. When more than one address is supported, only one is designated as the Operational Network Address.

Annunciation Repeat Time

This attribute specifies the milliseconds between annunciation messages. Its default value is 15000 (15 seconds).

Device Type

This attribute specifies the type of device and its redundancy capability. The list of valid types includes Type 9 Linking Device, Type 9 Device, Type 5 Device, Gateway Device.

The device's redundancy capability specifies the device's ability to operate as one of a set of redundant devices. All devices in the set are required to have the same value of this attribute configured. Three values are possible, None, External Control, and Automatic.

External Control indicates that the devices in the set are capable of operating as either a primary or a secondary (backup). An external agent is required to synchronise device data, detect a failure of the primary, and after detection, issue a Set Assignment Info service request to one of the secondaries to change its Device Redundancy State from secondary to primary.

Automatic indicates that the device is capable of synchronising with its redundant partners, detecting a fault in the primary, and automatically selecting a new primary to survive the fault.

Device Redundancy State

This attribute specifies the actual device redundancy state of the device. If the device is not participating in redundancy, its value is None. If it is participating as an external controlled device, then its value is either "external control - primary" or "external control - secondary". If it is participating as an automatic device, then its values are "automatic - primary" or "automatic - secondary". This last value indicates that the devices in the set are to select which device is the primary on their own. The Set Assignment Info service can be used to assign the value of this attribute. The assigned value shall be compatible with the Device Type attribute.

Device Index

This attribute is a site administered number that identifies the device. This index is unique within an FAL subnet.

Max Device Index

This attribute defines the maximum value that the device index may have. This effectively indicates the maximum number of devices that may be configured into an FAL subnet. This number is a multiple of 32.

SMK Attributes Version Number

This attribute is the version number for the attribute values that precede this attribute. Each time the preceding attribute's values change, this version number is incremented by 1. This version number is only changed if the value of one of the covered attributes changes. If the device initializes on powerup, the value of the version number is set to zero. If the device supports Operational Powerup and it is true, the annunciation version number may remain the same if the attributes it covers go through the power cycle without changing. The value of SMK State may change if the device supports time synchronization and the status of time sync changes, which causes the value of the annunciation version number to change.

Device Version Number

This attribute is the version number that represents the composite state of the static information in the applications in the device. Each time there is change to the static data of any application in the device, this version number is incremented by 1. Its value initializes to zero if the device is initialized. How the SMK determines that a change has occurred is a local matter.

Operational Powerup

This boolean attribute indicates when true that the SMK data is to be stored in non-volatile memory so that it is retained through a power cycle.

10.2.2.2.3 Services

All services defined for this class are mandatory.

Find Tag Query

This unconfirmed service is to search for an object in the network.

Find Tag Reply

This unconfirmed service is used as the reply to the Find Tag Query service. It returns a list of VCR Identifiers that can be used in the Initiate service to access the queried object.

Identify

This confirmed service is used to request the SMK to identify itself.

Device Annunciation

This unconfirmed service is used to periodically announce the presence of the SMK on the network.

Clear Address

This unconfirmed service is used to request an SMK to clear its network address.

Set Assignment Info

This confirmed service is used to set a selected subset of the SMK attribute values.

Clear Assignment Info

This confirmed service is used to clear a selected subset of the SMK attribute values.

10.2.2.3 System management kernel ASE service specification

10.2.2.3.1 Services supported

SMK Services operate directly over socket services. They do not use ARs.

10.2.2.3.2 Find tag query service

10.2.2.3.2.1 Service overview

This unconfirmed service is used to send a query to one or more SMKs to locate an object on the network.

10.2.2.3.2.2 Service parameters

The service parameters for this service are shown in Table 438.

Table 438 – Find tag query service parameters

Parameter name	Req	Ind
Argument	M	M(=)
Invoke ID	M	M(=)
Source Address		M
Destination Address	M	M(=)
SMK ID	M	M(=)
Query Type	M	M(=)
Physical Device Tag	C	C(=)
VFD Tag	C	C(=)
Application Object Tag	C	C(=)
Element ID	C	C(=)
VFD Reference	C	C(=)
Device Index	C	C(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the service request was sent. It is used by the responder when returning the reply.

Destination Address

This parameter is the address to which the service request is to be sent. This address may be an Individual Device Address or the SMK Multicast Address.

SMK ID

This parameter identifies which SMKs are to respond to the query. The possible values are dependent on the value of the Destination Address. They are as shown in Table 439.

Table 439 – SMK IDs

Destination Address Value	SMK Id Value
Individual Device Address	Device SMK
Individual Device Address	Device SMK and all Type "X" SMKs attached to the addressed device (does not include the Type "X" SMKs contained in the device)
Individual Device Address	Device SMK and all Type "X" SMKs on a specific Type "X" link attached to the addressed device (does not include the Type "X" SMKs contained in the device)
Individual Device Address	Specific Type "X" SMK accessible through the addressed device
SMK Multicast Address	All SMKs
SMK Multicast Address	All SMKs and all Type "X" SMKs (does not include the Type "X" SMKs contained in the device)
SM Multicast Address	Device SMK and all Type "X" SMKs on a specific Type "X" link attached to the addressed device (does not include the Type "X" SMKs contained in the device)

Query Type

This parameter selects the type of query. The following types are mutually exclusive:

- PD Tag query, Primary device
- PD Tag query, Secondary device
- VFD Tag query (also requires PD Tag)
- Application Object Tag query
- Element ID query (also requires Application Object Tag)
- VFD Reference query (also requires PD Tag)
- Device Index query

Physical Device Tag

This conditional type parameter, when present, contains the Physical Device Tag, indicating that the query is to locate a physical device. It is present when the Query Type indicates a Physical Device Tag query or a VFD Tag query.

VFD Tag

This conditional type parameter, when present, contains the VFD Tag, indicating that the query is to locate a VFD. It is present when the Query Type indicates VFD Tag query.

Application Object Tag

This conditional type parameter, when present, contains the tag of an application object, indicating that the query is to locate a tagged object. It is present when the Query Type indicates Application Object Tag query or Element ID query.

Element ID

This conditional parameter, when present, is used to locate an element of an Application Object Tag. When present, the Application Object Tag is also present. It contains a reference to an element of the tagged application object. It is present when the Query Type indicates Element ID query.

VFD Reference

This conditional type parameter, when present, contains a numeric reference to an VFD, indicating that the query is to locate a VFD. It is present when the Query Type indicates VFD Reference query or a VFD Tag query.

Device Index

This conditional type parameter, when present, contains the Device Index, indicating that the query is to locate a device by its Device Index. It is present when the Query Type indicates Device Index query.

10.2.2.3.2.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

10.2.2.3.3 Find tag reply service

10.2.2.3.3.1 Service overview

This unconfirmed service is used to send a reply to the initiator of a Find Tag Query. It contains information that can be used to establish an AR and a VCR to access the queried object.

10.2.2.3.3.2 Service parameters

The service parameters for this service are shown in Table 440.

Table 440 – Find tag reply service parameters

Parameter name	Req	Ind
Argument	M	M(=)
Invoke ID	M	M(=)
Source Address		M
Destination Address	M	M(=)
Duplicate Detection State	M	M(=)
Query Type	M	M(=)
Queried Object Network Address	M	M(=)
Queried Object Link Address	M	M(=)
Queried Object Type "X" Device Address	C	C(=)
Queried Object Device ID	M	M(=)
Queried Object PD Tag	M	M(=)
Queried Object VFD Reference	C	C(=)
Queried Object OD Version Number	C	C(=)
Queried Object Numeric Id	C	C(=)
Duplicate Detection State	M	M(=)
List of VCR References	C	C(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which this service was sent.

Destination Address

This parameter is the address from which the matching Find Tag Query was sent.

Duplicate Detection State

This parameter contains the value of the SMK attribute of the same name. It refers to the SMK of the replying device.

Query Type

This parameter selects the type of query. The following types are mutually exclusive:

- PD Tag query, Primary device
- PD Tag query, Secondary device
- VFD Tag query (also requires PD Tag)
- Function Block Tag query
- Element ID query (also requires FB Tag)
- VFD Reference query (also requires PD Tag)
- Device Index query

Queried Object Network Address

This parameter contains the network address used to access the queried object.

Queried Object Address

This parameter is the link id portion of the address to be used to access the queried object. If the queried object is in a device, then this value is 0. If the queried object is located in a Type "X" device on a Type "X" link that is connected to the responding device, then this value is the Type "X" link address.

Queried Object Type "X" Device Address

This conditional parameter is present if the queried object is located in a Type "X" device on a Type "X" link that is connected to the responding device. It contains the Type "X" device address used to access the Type "X" device.

Queried Object Device ID

This parameter contains the value of the SMK Device ID attribute for the device that contains the queried object.

Queried Object PD Tag

This parameter contains the value of the SMK PD Tag attribute for the device that contains the queried object.

Queried Object VFD Reference

This conditional type parameter, when present, contains a numeric reference of the queried object's VFD. It is present for all queries except for Physical Device Tag queries.

Queried Object OD Version Number

This conditional type parameter, when present, contains the version number of the queried object's OD. It is present for all queries except for Physical Device Tag queries.

Queried Object Numeric Id

This conditional type parameter, when present, contains the numeric id or OD index attribute of the queried object. It is present if the queried object is contained in a VFD.

Duplicate Detection State

This parameter contains the value of the SMK attribute of the same name.

List of VCR References

This conditional type parameter, when present, contains a list of VCR References that can be used with the Type 9 Initiate service to open a VCR to the queried object. It is present for all queries except for Physical Device Tag queries.

10.2.2.3.3 Service procedure

The Unconfirmed Service Procedure specified in Clause 4 applies to this service.

10.2.2.3.4 Identify service

10.2.2.3.4.1 Service overview

This confirmed service is used to obtain identification information from a remote SMK.

10.2.2.3.4.2 Service parameters

The service parameters for this service are shown in Table 441.

Table 441 – Identify service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
SMK ID	M	M(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			M	M(=)
Device ID			M	M(=)
Physical Device Tag			M	M(=)
SMK State			M	M(=)
Device Type			M	M(=)
LAN Redundancy Socket Address			C	C(=)
Annunciation Repeat Time			C	C(=)
Device Redundancy State			C	C(=)
Duplicate Detection State			C	C(=)
Device Index			C	C(=)
Max Device Index			C	C(=)
Operational Network Address			C	C(=)
SMK Attributes Version Number			C	C(=)
Device Version Number			C	C(=)
Version Number List			C	C(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			C	C(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request is sent.

SMK ID

This parameter identifies the SMK that is to respond to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" link interface in a Type 5 linking device, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

SMK ID

This parameter identifies the SMK sending the response. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" link interface in a Type 5 linking device, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Device ID

This parameter contains the value of the SMK attribute of the same name.

Physical Device Tag

This parameter contains the value of the SMK attribute of the same name.

SMK State

This parameter contains the value of the SMK attribute of the same name.

Device Type

This parameter contains the value of the SMK attribute of the same name.

LAN Redundancy Socket Address

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Annunciation Repeat Time

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Device Redundancy State

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Duplicate Detection State

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Device Index

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Max Device Index

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Operational Network Address

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

SMK Attributes Version Number

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Device Version Number

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the responder is a Type 5 device.

Version Number List

This conditional parameter is used by device that connects links of two or more Communication Model Types together. It is present if the responder is a Type 5 device.

If the Type "X" address request parameter is not present, then this parameter contains the link id and version number of each attached link. This Type "X" link version number is incremented each time any device address is added or deleted from the Type "X" network.

If the Type "X" address request parameter identifies a specific link connected to the receiving device, then this parameter contains the Type "X" address and version number associated with each device connected to that link. This Type "X" device address version number is incremented each time that specific device address is added or deleted from the Type "X" network.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

SMK ID

This parameter identifies the SMK sending the response. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" link interface in a Type 5 linking device, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

10.2.2.3.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

10.2.2.3.5 Device annunciation service**10.2.2.3.5.1 Service overview**

This unconfirmed service is used to multicast a notification onto the network that contains basic SMK information. The reserved Annunciation multicast address is used.

10.2.2.3.5.2 Service parameters

The service parameters for this service are shown in Table 442.

Table 442 – Annunciate service parameters

Parameter name	Req	Ind
Argument	M	M(=)
Source Address		M
SMK ID	M	M(=)
Device ID	M	M(=)
Physical Device Tag	M	M(=)
SMK State	M	M(=)
Device Type	M	M(=)
LAN Redundancy Socket Address	M	M(=)
Annunciation Repeat Time	M	M(=)
Device Redundancy State	M	M(=)
Duplicate Detection State	M	M(=)
Device Index	M	M(=)
Max Device Index	M	M(=)
Operational Network Address	M	M(=)
SMK Attributes Version Number	M	M(=)
Device Version Number	M	M(=)
Version Number List	C	C(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Device ID

This parameter contains the value of the SMK attribute of the same name.

Physical Device Tag

This parameter contains the value of the SMK attribute of the same name.

Device Type

This parameter contains the value of the SMK attribute of the same name.

LAN Redundancy Addr

This parameter contains the value of the SMK attribute of the same name.

Annunciation Repeat Time

This parameter contains the value of the SMK attribute of the same name.

SMK State Capability

This parameter contains the value of the SMK attribute of the same name.

Device Redundancy State

This parameter contains the value of the SMK attribute of the same name.

Duplicate Detection State

This parameter contains the value of the SMK attribute of the same name.

Operational Network Address

This parameter contains the value of the SMK attribute of the same name.

Device Index

This parameter contains the value of the SMK attribute of the same name.

Max Device Index

This parameter contains the value of the SMK attribute of the same name.

SMK Attributes Version Number

This parameter contains the value of the SMK attribute of the same name.

Device Version Number

This parameter contains the value of the SMK attribute of the same name.

Version Number List

This conditional parameter is present if the service was requested by a device that connects links of two or more Communication Model Types together and if the Type "X" address request parameter is not present. This parameter contains the version number of each attached link.

10.2.2.3.5.3 Service procedure

The unconfirmed Service Procedure specified in Clause 4 applies to this service.

10.2.2.3.6 Set assignment info service**10.2.2.3.6.1 Service overview**

This service is used to update the attribute values of the SMK addressed by the Destination Address parameter. The SMK receiving the new values updates its attributes, if possible, and returns a response that includes the newly updated attribute values. This service also may be used to assign a new PD Tag and address to a Type "X" device that is connected to a Type 5 device through a Type "X" network.

10.2.2.3.6.2 Service parameters

The service parameters for this service are shown in Table 443.

Table 443 – Set assignment info service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
SMK ID	M	M(=)		
Device ID	M	M(=)		
Physical Device Tag	M	M(=)		
LAN Redundancy Socket Address	C	C(=)		
Device Redundancy State	C	C(=)		
Clear Duplicate Detection State	C	C(=)		
Operational Network Address	C	C(=)		
Device Index	C	C(=)		
Max Device Index	C	C(=)		
Annunciation Repeat Time	C	C(=)		
New Type "X" Address	C	C(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			C	C(=)
Annunciation Repeat Time			C	C(=)
Max Device Index			C	C(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			C	C(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the service was requested.

Destination Address

This parameter is the address to which the request is sent.

SMK ID

This parameter identifies the SMK that is to respond to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Device ID

This parameter contains the value of the SMK attribute of the same name. This value shall match that of the SMK attribute and is not writable.

Physical Device Tag

This parameter contains the value of the SMK attribute of the same name. This value is writable, but only when the device does have a Physical Device Tag. Physical Device Tags can be cleared using the Clear Assignment Info service.

LAN Redundancy Socket Address

This conditional parameter contains the value of the SMK attribute of the same name. This value is writable. It is present if the request is addressed to a Type 5 SMK.

Device Redundancy State

This conditional parameter contains the value of the SMK attribute of the same name. This value is writable. It is present if the request is addressed to a Type 5 SMK.

Clear Duplicate Detection State

This conditional parameter is used to set the value of the SMK Duplicate Detection State attribute to "Duplicate Physical Device Tag Not Detected" and/or "Duplicate Device Index Not Detected". It is present if the request is addressed to a Type 5 SMK.

Operational Network Address

This conditional parameter contains the value of the SMK attribute of the same name. This value is writable. It is present if the request is addressed to a Type 5 SMK.

Device Index

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the request is addressed to a Type 5 SMK.

Max Device Index

This conditional parameter contains the value of the SMK attribute of the same name. It is present if the request is addressed to a Type 5 SMK. The responder may negotiate this value to a smaller value, but no smaller than 500. That is, a value smaller than 500 can be returned only if that value or a smaller value is received in request.

Annunciation Repeat Time

This conditional parameter contains the value of the SMK attribute of the same name. This value is writable. It is selected if the request is addressed to a Type 5 SMK. During device assignment, the FDA Agent may negotiate this value to a larger value no larger than 5000 if the assigned value is less than 5000 and the device is not capable of supporting the assigned value.

New Type "X" Address

This conditional parameter contains a NULL value or an address that is to be assigned to a Type "X" device connected to the Type 5 device through a Type "X" network. It is present if the request is addressed to a Type "X" SMK.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

SMK ID

This parameter identifies the SMK sending the response. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Annunciation Repeat Time

This conditional parameter contains the negotiated value of the SMK attribute of the same name. It is present if the service was requested from a Type 5 device SMK.

Max Device Index

This conditional parameter contains the negotiated value of the SMK attribute of the same name. It is present if the service was requested from a Type 5 device SMK.

Result(-)

This selection type parameter indicates that the service request failed.

Type "X" Address

This conditional parameter contains the Type X address from which the service was requested. It may be the address of the Type "X" link connected to a Type 5 device, or the address of a device on a Type "X" link that is connected to a Type 5 device from which the service was requested. It is present if the service was requested from a Type "X" device.

10.2.2.3.6.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service. The service fails if the Device ID and Device Type parameters do not match those of the addressed SMK.

10.2.2.3.7 Clear assignment info service

10.2.2.3.7.1 Service overview

This service is used to clear a device by clearing SMK and LAN Redundancy assignment information and setting all VFDs to factory defaults. It may be used to clear the PD Tag of a device on a Type "X" link that is connected to a Type 5 device. If the SMK ID indicates that a Type "X" PD Tag is to be cleared, the SMK invokes the appropriate Type "X" SM procedures to clear it.

The configuration application includes the PD Tag (which may be blank) and Device ID to verify that the correct device will receive this service. If either the PD Tag or the Device ID do not match, the device returns a negative response.

10.2.2.3.7.2 Service parameters

The service parameters for this service are shown in Table 444.

Table 444 – Clear assignment info service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
SMK ID	M	M(=)		
Device ID	M	M(=)		
Physical Device Tag	M	M(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			C	C(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
SMK ID			C	C(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the service was requested.

Destination Address

This parameter is the address to which the request is sent.

SMK ID

This parameter identifies the SMK that is to respond to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Device ID

This parameter contains the value of the SMK attribute of the same name. This value shall match that of that SMK attribute.

Physical Device Tag

This parameter contains the value of the SMK attribute of the same name. This value shall match that of that SMK attribute.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

SMK ID

This parameter identifies the SMK sending the response. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

SMK ID

This parameter identifies the SMK sending the response. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

10.2.2.3.7.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service. The service fails if the Device ID and Device Type parameters do not match those of the addressed SMK.

10.2.2.3.8 Clear address service

10.2.2.3.8.1 Service overview

This service is used to clear the address of a Type 5 device, a Type "X" link interface connected to a Type 5 device, or a device on a Type "X" link that is connected to a Type 5 device. After clearing the address of a Type "X" link interface connected to a Type 5 device or a device on a Type "X" link that is connected to a Type 5 device, the responding device SMK returns the response. After accepting the request to clear its own address, the device SMK returns the response, and then clears its address.

This service operates over dynamically established or pre-established client/server sessions.

10.2.2.3.8.2 Service parameters

The service parameters for this service are shown in Table 445.

Table 445 – Clear address service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Destination Address	M	M(=)		
SMK ID	C	C(=)		
Device ID	M	M(=)		
Physical Device Tag	M	M(=)		
InterfaceToClear	C	C(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Destination Address			M	M(=)
SMK ID			C	C(=)
Result (-)			S	S(=)
AREP			M	M(=)
Destination Address			M	M(=)
Invoke ID			M	M(=)
SMK ID			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Destination Address

This parameter is the address to which the service request is to be sent. The request may be addressed to a Type 5 device, to a Type "X" link connected to a Type 5 device, or to a device on a Type "X" link that is connected to a Type 5 device.

SMK ID

This parameter identifies the SMK that is to respond to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Device ID

This parameter contains the value of the SMK attribute of the same name. This value shall match that of that SMK attribute.

Physical Device Tag

This conditional parameter contains the value of the SMK attribute of the same name. This value shall match that of that SMK attribute.

InterfaceToClear

This parameter indicates which of the device's network interfaces are to be cleared. It is present if the request is addressed to a Type 5 SMK.

Result(+)

This selection type parameter indicates that the service request succeeded.

Destination Address

This parameter is the address to which the service response is to be returned.

SMK ID

This parameter identifies the SMK that responds to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

Result(-)

This selection type parameter indicates that the service request failed.

Destination Address

This parameter is the address to which the error service response is to be returned.

SMK ID

This parameter identifies the SMK that responds to the request. The responder may be:

- a Type 5 SMK, or
- a Type "X" SMK of a Type "X" device connected to a Type 5 linking device.

10.2.2.3.8.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

In addition, the Device ID, Physical Device Tag, and Device Type parameter values shall match the corresponding attribute values in the SMK of the device that is to have its address cleared request. If they do not match, a negative response is returned.

10.2.3 LAN redundancy ASE

10.2.3.1 Overview

Devices can have two network interfaces that are used for LAN redundancy. These interfaces are referred to as Network Interface A and Network Interface B. They are managed by a user of the LAN Redundancy ASE referred to as the LAN Redundancy Entity.

The LAN Redundancy Entity exchanges redundancy status information with other LAN Redundancy Entities using the Diagnostic Message Service. The LAN Redundancy Entity of a device selects which network interface to use to send other APDUs based on this status information.

Each LAN Redundancy Entity submits a Diagnostic Message Service request primitive at periodic intervals and the LAN Redundancy ASE constructs two Diagnostic Message Service APDUs and sends them on Network Interface A and Network Interface B concurrently. Each LAN Redundancy Entity receives pairs of service indications from the other devices on the network that participate in LAN Redundancy. This information is used to determine which network interface (A or B) is to be selected to send other APDUs.

10.2.3.2 LAN redundancy class specification

10.2.3.2.1 LAN redundancy formal model

FAL ASE:		LAN Redundancy
CLASS:		LAN Redundancy
CLASS ID:		Not used
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m)	Attribute: LAN Redundancy Attributes Version
2	(m)	Attribute: Number of Network Interfaces
3	(m)	Attribute: Max Sequence Number Difference
4	(m)	Attribute: LAN Redundancy Flags
5	(m)	Attribute: Diagnostic Message Interval
6	(m)	Attribute: Aging Time
7	(m)	Attribute: Diagnostic Message Send Addresses
8	(m)	Attribute: Diagnostic Message Receive Addresses
9	(m)	Attribute: List of Network Interface Status
10	(m)	Attribute: Number of Diagnostic Service Indications Received
11	(m)	Attribute: Number of Diagnostic Message Indications Missed
12	(m)	Attribute: Number of Faults Detected
13	(m)	Constraint: Crossed Cable Detection Enabled = TRUE
13.1	(m)	Attribute: Number of Diagnostic Service Indications Missed
SERVICES:		
1	(m)	OpsService: Diagnostic Message
2	(o)	OpsService: Get Redundancy Information
3	(o)	OpsService: Put Redundancy Information
4	(o)	OpsService: Get Redundancy Statistics

10.2.3.2.2 Attributes

LAN Redundancy Attributes Version

This attribute specifies the version of this object. Each time the value of any of its other attributes changes, the version number is incremented by 1. It survives a power-fail if the APDU contents survive the power-fail; otherwise its value is reset to 0. This is the only time that its value is reset to 0. Version number 0 indicates that this object has not been configured.

Number of Network Interfaces

Specifies the number of network interfaces on this device. A device may have one or two network interfaces. They are labelled Network Interface A and Network Interface B. (Network Interface B is only used if there are two network interfaces.)

Max Sequence Number Difference

This attribute defines the maximum acceptable difference between the sequence number parameters in a pair of Diagnostic Message Service indications received from a single sending device. When the difference exceeds the value of this attribute, a fault in the path from the network interface sending the lower sequence number is detected.

LAN Redundancy Flags

This attribute controls how the Diagnostic Message Service is used by the LAN Redundancy Entity. Its value may indicate one or more of the following:

- Single Multicast APDU Transmission Interface Enabled. The transmission policy for all services with multicast destination addresses except for the Diagnostic Message and SMK Annunciate services.
 - False Transmit on both network interfaces
 - True Transmit on one network interface
- Crossed Cable Detection Enabled.
 - False Do not detect crossed cables
 - True Detect crossed cables
- Single Multicast APDU Reception Interface Enabled. The reception policy for all multicast destination addresses except for SM Multicast address and the LAN Diagnostic Message Receive Addresses for interfaces A and B.
 - False Listen for multicast addresses on both network interfaces
 - True Listen for multicast addresses on one network interface if zero or one fault detected
- Diagnosis using own APDUs Enabled.
 - False Do not use own diagnostic messages for diagnosis
 - True Use own diagnostic messages for diagnosis
- Load Balancing Enabled.
 - False Do not do load balancing
 - True Do load balancing

Diagnostic Message Interval

This attribute specifies the time interval in milliseconds between successive invocations of the Diagnostic Message Service by the LAN Redundancy Entity.

Aging Time

This attribute specifies the time interval used by the LAN Redundancy Entity to remove silent devices from its list of devices that are participating in LAN redundancy. The value of this attribute shall be larger than the value of the *Max Sequence Number Difference* attribute multiplied by the largest value of the parameter *Diagnostic Message Interval* found in Diagnostic Message Service indications.

Diagnostic Message Send Addresses

This attribute defines the destination addresses for sending Diagnostic Message APDUs. One address is defined for each network interface from which Diagnostic Message APDUs are sent. The addresses may be a broadcast, multicast or unicast addresses.

Diagnostic Message Receive Addresses

This attribute defines the addresses for receiving Diagnostic Message APDUs. One address is defined for each network interface on which Diagnostic Message APDUs are received. The addresses may be a broadcast, multicast or unicast addresses.

List of Network Interface Status

This attribute is a list. Each entry in this list represents the status for the path from a remote device from which Diagnostic Messages Service indications may be received. The status is bidirectional for path and indicates whether or not Diagnostic Messages Service indications are being successfully received at each end of the path or whether either end of the path is in a fault state (a fault has been detected and has not cleared).

Number of Diagnostic Message Service Indications Received

This attribute counts the number of Diagnostics Message Service indications received (from all devices). A separate count is maintained for each network interface.

Number of Diagnostic Message Service Indications Missed

This attribute is a count of the number of missed Diagnostics Message Service indications (from all devices) as computed from sequence number gaps. A separate count is maintained for each network interface.

Number of Faults Detected

This attribute is a count of the total number of faults detected from missed Diagnostics Message Service indications. A separate count is maintained for each network interface.

List of Crossed Cable Status

This conditional attribute is a list. This attribute is present only if the value of the *Crossed Cable Detection Enabled* component of the LAN Redundancy Flags attribute is TRUE. Each entry in this list represents the Crossed Cable Status for the path to a remote device from which Diagnostic Messages Service indications may be received. The Crossed Cable Status indicates whether the network interface used by the Diagnostic Message Service for requests, as indicated by the *Network Interface used for Transmission of this Diagnostic Message* parameter in the Diagnostic Message Service, is the same network interface used by the reporting device to receive the messages.

10.2.3.2.3 Services

Diagnostic Message

This service is used to send the same diagnostic message concurrently from each network interface.

Get Redundancy Information

This service is used to retrieve LAN Redundancy attributes.

Set Redundancy Information

This service is used to update LAN Redundancy attributes.

Get Redundancy Statistics

This service is used to retrieve LAN Redundancy statistics.

10.2.3.3 LAN redundancy ASE service specification

10.2.3.3.1 Overview

This subclause contains the definition of the services that is unique to this ASE. LAN Redundancy Services operate directly over socket services. They do not use ARs.

The services defined for this ASE are:

- Diagnostic Message Service
- Get Redundancy Info Service
- Put Redundancy Info Service
- Get Redundancy Statistics Service

10.2.3.3.2 Diagnostic message service

10.2.3.3.2.1 Service overview

This service is used to cause Diagnostic Message Service APDUs to be sent simultaneously from each redundant network interface, labeled Interface A and Interface B. The messages are identical except for the identification of which interface was used to send each APDU.

10.2.3.3.2.2 Service parameters

The service parameters for this service are shown in Table 446.

Table 446 – Diagnostic message service

Parameter name	Req	Ind
Argument	M	M(=)
Source Address		M
Destination Address	M	M(=)
Device Index	M	M(=)
Number of Interfaces	M	M(=)
Transmission Interface	M	M(=)
Diagnostic Message Interval	M	M(=)
SMK PD Tag	M	M(=)
SMK Duplicate Detection State	M	M(=)
List of Network Interface Status	M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request was sent.

Device Index

This value is obtained from the SMK attribute of the same name.

Number of Network Interfaces

This value is derived from the SMK List of Network Addresses attribute. There is one network address per network interface in this list.

Transmission Interface

This parameter is the transmission interface from which the request was sent.

Diagnostic Message Interval

This value is obtained from the LAN Redundancy attribute of the same name.

SMK PD Tag

This parameter contains the value of the SMK PD Tag attribute of the device.

SMK Duplicate Detection State

This parameter contains the value of the SMK Duplicate Detection State attribute of the device.

List of Network Interface Status

This value is obtained from the LAN Redundancy attribute of the same name.

10.2.3.3.2.3 Service procedure

The Unconfirmed Service Procedure specified in Clause 4 applies to this service.

10.2.3.3.3 Get redundancy info service

10.2.3.3.3.1 Service overview

This confirmed service is used to retrieve LAN Redundancy attributes.

10.2.3.3.3.2 Service parameters

The service parameters for this service are shown in Table 447.

Table 447 – Get redundancy info service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
LAN Redundancy Attributes Version			M	M(=)
Number of Network Interfaces			M	M(=)
Max Sequence Number Difference			M	M(=)
LAN Redundancy Flags			M	M(=)
Diagnostic Message Interval			M	M(=)
AgingTime			M	M(=)
Diagnostic Message Interface Send Addresses			M	M(=)
Diagnostic Message Interface Receive Addresses			M	M(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request was sent.

LAN Redundancy Attributes Version

This value contains the value of the LAN Redundancy attribute of the same name.

Number of Network Interfaces

This value contains the value of the LAN Redundancy attribute of the same name.

Max Sequence Number Difference

This value contains the value of the LAN Redundancy attribute of the same name.

LAN Redundancy Flags

This value contains the value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interval

This value contains the value of the LAN Redundancy attribute of the same name.

AgingTime

This value contains the value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Send Addresses

This value contains the value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Receive Addresses

This value contains the value of the LAN Redundancy attribute of the same name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

10.2.3.3.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

10.2.3.3.4 Put redundancy info service

10.2.3.3.4.1 Service overview

This confirmed service is used to retrieve LAN Redundancy attributes. After updating the attributes contained in the request, the responder returns the values and the updated LAN Redundancy Attributes Version number.

10.2.3.3.4.2 Service primitives

The service parameters for this service are shown in Table 448.

Table 448 – Put redundancy info service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
LAN Redundancy Attributes Version			M	M(=)
Number of Network Interfaces			M	M(=)
Max Sequence Number Difference			M	M(=)
LAN Redundancy Flags			M	M(=)
Diagnostic Message Interval			M	M(=)
AgingTime			M	M(=)
Diagnostic Message Interface Send Addresses			M	M(=)
Diagnostic Message Interface Receive Addresses			M	M(=)
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
LAN Redundancy Attributes Version			M	M(=)
Number of Network Interfaces			M	M(=)
Max Sequence Number Difference			M	M(=)
LAN Redundancy Flags			M	M(=)
Diagnostic Message Interval			M	M(=)
AgingTime			M	M(=)
Diagnostic Message Interface Send Addresses			M	M(=)
Diagnostic Message Interface Receive Addresses			M	M(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request was sent.

LAN Redundancy Attributes Version

This value contains the value of the LAN Redundancy attribute of the same name.

Number of Network Interfaces

This value contains the new value of the LAN Redundancy attribute of the same name.

Max Sequence Number Difference

This value contains the new value of the LAN Redundancy attribute of the same name.

LAN Redundancy Flags

This value contains the new value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interval

This value contains the new value of the LAN Redundancy attribute of the same name.

AgingTime

This value contains the new value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Send Addresses

This value contains the new value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Receive Addresses

This value contains the new value of the LAN Redundancy attribute of the same name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

LAN Redundancy Attributes Version

This value contains the updated value of the LAN Redundancy attribute of the same name.

Number of Network Interfaces

This value contains the updated value of the LAN Redundancy attribute of the same name.

Max Sequence Number Difference

This value contains the updated value of the LAN Redundancy attribute of the same name.

LAN Redundancy Flags

This value contains the updated value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interval

This value contains the updated value of the LAN Redundancy attribute of the same name.

AgingTime

This value contains the updated value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Send Addresses

This value contains the updated value of the LAN Redundancy attribute of the same name.

Diagnostic Message Interface Receive Addresses

This value contains the updated value of the LAN Redundancy attribute of the same name.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

10.2.3.3.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

10.2.3.3.5 Get redundancy statistics service**10.2.3.3.5.1 Service overview**

This confirmed service is used to retrieve LAN Redundancy statistics attributes.

10.2.3.3.5.2 Service primitives

The service parameters for this service are shown in Table 449.

Table 449 – Get redundancy statistics service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Number of Diagnostic Message Service Indications Received			M	M(=)
Number of Diagnostic Message Service Indications Missed			M	M(=)
Number of Faults Detected			M	M(=)
List of Crossed Cable Status			M	M(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request was sent.

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

Number of Diagnostic Message Service Indications Received

This value contains the value of the LAN Redundancy attribute of the same name.

Number of Diagnostic Message Service Indications Missed

This value contains the value of the LAN Redundancy attribute of the same name.

Number of Faults Detected

This value contains the value of the LAN Redundancy attribute of the same name.

List of Crossed Cable Status

This value contains the value of the LAN Redundancy attribute of the same name.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

10.2.3.3.5.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

10.3 FDA sessions

10.3.1 Publisher/subscriber session endpoint class specification

10.3.1.1 Class overview

This class is defined to support the on-demand queued distribution of unconfirmed services to one or more application processes. It is the Type 5 counterpart to the Type 9 BNU AREPs. The behaviour of this type of AR can be described as follows.

An AR ASE user wishing to convey an unconfirmed APDU submits an AR ASE Service Data Unit to the sending endpoint of the AR. The AREP sending the request APDU submits it to the underlying layer for transfer. The underlying layer sends it at its next opportunity. The AREP receiving the request APDU from its underlying layer delivers it to selected AR ASE users in the order that it was received. The AR ASE users are selected based on the address information contained in the AR header.

The sending endpoint has a role of PUBLISHER and the receiving endpoints have the role of SUBSCRIBER.

The following summarises the characteristics of this AREP class:

Roles:	PUBLISHER SUBSCRIBER
Cardinality:	1-to-n
Timeliness:	No

10.3.1.2 Formal model

FAL ASE:	AR ASE
CLASS:	Publisher/Subscriber Session Endpoint
CLASS ID:	Not Used
PARENT CLASS:	TOP
NETWORK MANAGEMENT ATTRIBUTES:	
1. (m) Attribute:	Role (PUBLISHER, SUBSCRIBER)
2. (m) Attribute:	AREP State (OPEN, CLOSED)
3. (m) Attribute:	Socket Mapping Reference
SERVICES:	
1. (o) OpsService:	Unconfirmed Send

10.3.1.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PUBLISHER Endpoints of this type send messages to multiple subscriber endpoints using unconfirmed services operating over connectionless socket services.

SUBSCRIBER Endpoints of this type receive reports from source endpoints using unconfirmed services operating over connectionless socket services.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Socket Mapping Reference

This attribute is a reference to the socket mapping to the underlying layer. These mappings are specified in IEC 61158-6.

10.3.1.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR. This service is specified by Type 9.

10.3.2 Report distribution session endpoint class specification

10.3.2.1 Class overview

This class is defined to support the on-demand queued distribution of unconfirmed services to one or more application processes. It is the Type 5 counterpart to the Type 9 QUU AREPs. The behaviour of this type of AR can be described as follows.

An AR ASE user wishing to convey an unconfirmed APDU submits an AR ASE Service Data Unit to the sending endpoint of the AR. The AREP sending the request APDU submits it to the underlying layer for transfer. The underlying layer sends it at its next opportunity. The AREP receiving the request APDU from its underlying layer delivers it to all AR ASE users of the receiving AREP in the order that it was received.

The sending endpoint has a role of REPORT SOURCE and the receiving endpoints have the role of REPORT SINK.

The following summarises the characteristics of this AREP class:

Roles:	REPORT SOURCE REPORT SINK
Cardinality:	1-to-n
Timeliness:	No

10.3.2.2 Formal model

FAL ASE:	AR ASE
CLASS:	Report Distribution Session Endpoint
CLASS ID:	Not Used
PARENT CLASS:	TOP
NETWORK MANAGEMENT ATTRIBUTES:	
1. (m) Attribute:	Role (REPORT SOURCE, REPORT SINK)
2. (m) Attribute:	AREP State (OPEN, CLOSED)
3. (m) Attribute:	Socket Mapping Reference
SERVICES:	
1. (o) OpsService:	Unconfirmed Send

10.3.2.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

REPORT SOURCE endpoints of this type send messages to multiple REPORT SINK endpoints using unconfirmed services operating over connectionless socket services.

REPORT SINK endpoints of this type receive reports from REPORT SOURCE endpoints using unconfirmed services operating over either connectionless.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Socket Mapping Reference

This attribute is a reference to the socket mapping to the underlying layer. These mappings are specified in IEC 61158-6.

10.3.2.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR. This service is specified by Type 9.

10.3.3 Client/server AR endpoint class specification

10.3.3.1 Class overview

This class is defined to support the on-demand exchange of confirmed and unconfirmed services between two application processes. It is the Type 5 counterpart to the Type 9 QUB Co AREP. The behaviour of this type of AR can be described as follows.

An AR ASE user wishing to convey a request or response APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity. Either endpoint may send unconfirmed request or response APDUs. Only the Client AREP may send the OpenSession Request APDU.

The AREP receiving the APDU from its underlying layer, queues it for delivery to its AR ASE user in the order that it was received.

The following summarises the characteristics of this AREP class:

Roles:	Client Server
Cardinality:	1-to-1
Timeliness:	No

10.3.3.2 Formal model

FAL ASE:	AR ASE
CLASS:	Client/server AREP
CLASS ID:	Not Used
PARENT CLASS:	TOP

NETWORK MANAGEMENT ATTRIBUTES:

1. (m) Attribute: Role (CLIENT, SERVER)
2. (m) Attribute: AREP State
3. (m) Attribute: Server Physical Device Tag
4. (m) Attribute: Socket Mapping Reference

SERVICES:

1. (o) OpsService: Confirmed Send
2. (o) OpsService: Unconfirmed Send
3. (o) OpsService: OpenSession

10.3.3.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

CLIENT Endpoints of this type issue confirmed service Request-APDUs to initiate the establishment of the AR. One and only one of the AREPs involved in an AR shall be the initiator.

SERVER Endpoints of this type respond to requests received from the CLIENT to establish the AR.

AREP State

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6.

Server Physical Device Tag

This conditional attribute specifies the Physical Device Tag SMK attribute of the server device. This attribute is used to locate the server on the network. How the server is located is beyond the scope of this standard. This attribute is present only for CLIENT AREPs.

Socket Mapping Reference

This attribute is a reference to the socket mapping to the underlying layer. These mappings are specified in IEC 61158-6.

10.3.3.4 Services

Unconfirmed Send

This optional service is used to send an unconfirmed service on an AR. This service is specified by Type 9.

Confirmed Send

This optional service is used to send a confirmed service on an AR. This service is specified by Type 9.

OpenSession

This service is used to establish an AR.

10.3.4 FDA session ASE service specification

10.3.4.1 Supported services

This subclause contains the definition of the services that is unique to this ASE.

The services defined for this ASE are:

- Open Session Service
- Idle Service

10.3.4.2 Open session service

10.3.4.2.1 Service overview

This service is used to open a Client/Server session between a client session endpoint and a server session endpoint. The source and destination network address pair identifies each session endpoint, where each address is composed of the network device address and a device internal selector.

Client session endpoints are configured with the PD Tag of the server instead of with the network address of the server. They may use the SM Find Tag Query to determine the destination network address or they may obtain it from the Periodic Annunciations sent by the server.

Use of the Find Tag Query service may be necessary also when reopening a session after failure. In this case, the cause of the failure may be failure of the server. If the server is redundant, the secondary will take over, but it will use its own network address. Therefore, the client needs to determine if the network address associated with the server's PD Tag has changed.

Once the network address of the server has been determined, the client sends the FDA Open Session request message to the FDA Selector located at the network device address.

The Version field in the FDA Message Header in the request message indicates the version of the FDA that is being requested for the session. The responder may negotiate the version down.

The Options field in the Message Header in the request message indicates the options that are being requested for the session. The Invoke Id bit is always set. The responder can refuse all other options. Returning 0 in the bit position representing the option indicates refusal.

If the message number option is requested, then the trailer field contains the initial value to be used by both endpoints of the session. This value is returned in the response message. If the requester does not receive a response to the request, and elects to resend the request, it increments the message number in the trailer, while keeping the invoke id the same as the original. This causes the request to be delivered to the session endpoint created if the first request was received and a positive response was returned. If the original request was not received, or it caused a negative response to be returned (that was not received), then the new request is treated like a new request because there is no existing session endpoint to process the message.

The FDA Address in the Message Header is not used and is set to 0.

Successful exchange of Open Session request and response messages results in the establishment of a session that may be used to send request and response messages. If connectionless services are used, the server returns the response from a previously unused connectionless selector. This port is then used to send and receive all subsequent messages for the session.

If the responder is does not have an available Session Index for the new session, then a negative response is returned with error class = "resource" and error code = "max sessions exceeded".

If the responder is does not have the resources to support the new session, then a negative response is returned with error class = "resource" and error code = "object creation failure" or some other appropriate error code within the "resource" error class.

The Open Session request message conveys session attributes for validation and use by the responder. If the responder is able to support certain of these, it is permitted to negotiate them, as defined in the Request Message Parameters in Table 450 below. If the requester is not prepared to operate with the negotiated attributes returned by the responder, it is free to not open the session. It may then reissue an Open Session request with different session attributes, or if using connection-oriented services, close the associated connection to close the session, as appropriate.

If a request is received with invalid or unsupported parameter values, a negative response is returned with an error class of "service" and error code of "parameter-inconsistent" for the offending parameter. This negative response also uses the additional code value of 1, and the additional description to propose an acceptable values for each of the parameters indicated below that can be used to open the session. To convey these values the 16 octets of the additional description is interpreted as 4 Unsigned32 integers instead of as a VisibleString. The location of each parameter value in the 16 octets is specified in the descriptions below.

If the responder receives an Open Session request message for a session on which it has already sent an Open Session response message, it closes the session.

If the responder is does not have an available Session Index for the new session, then a negative response is returned with error class = "resource" and error code = "max sessions exceeded".

If the responder is does not have the resources to support the new session, then a negative response is returned with error class = "resource" and error code = "object creation failure" or some other appropriate error code within the "resource" error class.

Sessions opened for NMA Configuration Use are referred to as configuration sessions. Only one configuration session is allowed at a time. If a request is received to open a configuration session and one is already open, or, optionally, if there is VCR to any Type 5 or Type 9 MIB with update services supported already open, then the configuration session is refused. The FDA Agent returns a negative response with error class = "access" and error code = "config-access-already-open".

Care needs to be taken when negotiating the inactivity time period for configuration sessions that use connectionless services. If the client fails, the server is not permitted to allow another configuration client to establish a session until the negotiated inactivity time period expires.

Opening publisher/subscriber and Report Distribution sessions is local, and does not result in the exchange of FDA Messages. As a result, the Open Session message parameters for these session types are configured into the NMIB instead of negotiated.

10.3.4.2.2 Service primitives

The service parameters for this service are shown in Table 450.

Table 450 – Open session service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
Source Address		M		
Destination Address	M	M(=)		
Session Index	M	M(=)		
Max Buffer Size	M	M(=)		
Max Message Length	M	M(=)		
NMA Configuration Use	M	M(=)		
Inactivity Close Time	M	M(=)		
Transmit Delay Time	M	M(=)		
PD Tag	M	M(=)		
Result (+)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Session Index			M	M(=)
Max Buffer Size			M	M(=)
Max Message Length			M	M(=)
NMA Configuration Use			M	M(=)
Inactivity Close Time			M	M(=)
Transmit Delay Time			M	M(=)
PD Tag			M	M(=)
Result (-)			S	S(=)
Invoke ID			M	M(=)
Source Address				M
Destination Address			M	M(=)
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

Source Address

This parameter is the address from which the request was sent.

Destination Address

This parameter is the address to which the request was sent.

Session Index

The Session Index is the OD Index of the session endpoint description of the client session endpoint, if one exists. If not, 0 is used.

Max Buffer Size

This parameter defines the maximum buffer length in octets (the buffer may contain concatenated FDA Messages) that the sender of this message may send or receive on this session. This parameter can be negotiated down (but not up) by the responder using the Max Buffer Size parameter. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 0. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 0.

Max Message Length

This parameter defines the maximum length in octets of messages to be sent on this session by the sender of this message. It is used by the receiving session endpoint as its Max Message Length attribute. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 4. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 4.

NMA Configuration Use

0 = NMA Configuration Not Permitted

1 = NMA Configuration Permitted

This parameter cannot be negotiated.

Inactivity Close Time

This parameter identifies how long, in seconds, that the session stays open without receiving a message. After experiencing inactivity on the session for this period of time, the session endpoint is closed, along with all VCR activity associated with the session endpoint. The responder can negotiate this value down. The value 0 is not permitted. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 8. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 8.

Transmit Delay Time

This parameter is used to set the Transmit Delay Time attribute in the receiving session endpoint. Its value may not be negotiated. Its value is expressed in milliseconds. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 12. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 12.

PD Tag

PD Tag of the server. If the PD Tag in the request message does not match the PD Tag of the server, then the server rejects the request with error class "access" and error code "object-access-denied".

Result(+)

This selection type parameter indicates that the service request succeeded.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

Session Index

The Session Index is the OD Index of the newly opened session.

Max Buffer Size

This parameter defines the negotiated buffer length in octets (the buffer may contain concatenated FDA Messages) that the sender of this message may send or receive on this session.

Max Message Length

This parameter defines the maximum length in octets of messages to be sent on this session by the sender of this message. It is used by the receiving session endpoint as its Max Message Length attribute.

NMA Configuration Use

0 = NMA Configuration Not Permitted

1 = NMA Configuration Permitted

Inactivity Close Time

This parameter identifies how long, in seconds, that the session stays open without receiving a message. After experiencing inactivity on the session for this period of time, the session endpoint is closed, along with all VCR activity associated with the session endpoint. The responder can negotiate down this value. The value 0 is not permitted.

Transmit Delay Time

This parameter is used to set the Transmit Delay Time attribute in the receiving session endpoint. Its value is expressed in milliseconds.

PD Tag

PD Tag of the server.

Result(-)

This selection type parameter indicates that the service request failed.

Source Address

This parameter is the address from which the response was sent.

Destination Address

This parameter is the address to which the response is sent.

10.3.4.3 Idle service

10.3.4.3.1 Service overview

This service is used by client/server VCR endpoints to keep the VCR alive. Receipt of an Idle message informs the receiver that the sender is present. It may be sent using connectionless services or connection-oriented services. The FDA Address contains the OD Index of the dynamically established server VCR endpoint. If this endpoint does not exist, then an error message is returned with error class = "access" and error code = "unrecognized FDA Address".

10.3.4.3.2 Service primitives

The service parameters for this service are shown in Table 451.

Table 451 – Idle session service

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
Invoke ID	M	M(=)		
VCR Id		M		
Result (+)			S	S(=)
Invoke ID			M	M(=)
VCR Id				M
Result (-)			S	S(=)
Invoke ID			M	M(=)
VCR Id				M
Error Info			M	M(=)

Argument

The argument contains the parameters of the service request.

VCR Id

This parameter identifies the VCR of the service.

Result(+)

This selection type parameter indicates that the service request succeeded.

VCR Id

This parameter identifies the VCR of the service.

Result(-)

This selection type parameter indicates that the service request failed.

VCR Id

This parameter identifies the VCR of the service.

10.4 Summary of FAL Type 9 and Type 5 classes

Table 452 contains a summary of the defined FAL Classes. The Class ID values are defined in the corresponding subclauses of Clause 15 (Type 9).

Table 452 – FAL class summary

FAL ASE	Class	Class ID
VFD	VFD VCR	—
Data Type	Fixed Length & String Data Type Structure Data Type	See Type 9 See Type 9
Object Dictionary	OD Description Object Dictionary	— —
Type 5 AR (FDA Session)	Client/server Publisher/subscriber Report Distribution	— — —
Type 9 Variable	Simple Variable Array Variable Record Variable Variable List	See Type 9 See Type 9 See Type 9 See Type 9
Type 9 Event	Event	See Type 9
Type 9 Load Region	Load Region	See Type 9
Type 9 Function Invocation	Function Invocation	See Type 9
SMK	SMK	—
LAN Redundancy	LAN Redundancy	—

10.5 Permitted FAL Type 9 and Type 5 services by AREP role

Table 453 defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 453 – Services by AREP role

	Unc	Cnf	Client req rcv	Server req rcv	Publ req rcv	Subsc req rcv
FAL Services						
Type 9 Mgt ASE						
Create		X	X X	X X		
Delete		X	X X	X X		
Get Attributes		X	X X	X X		
Get Attribute List		X	X X	X X		
Set Attributes		X	X X	X X		
Begin Set Attributes		X	X X	X X		
End Set Attributes		X	X X	X X		
VFD ASE						
Identify		X	X X	X X		
Get Status		X	X X	X X		
Status Notification	X		X X	X X	X	X
Initiate		X	X X	X X		
Terminate		X	X X	X X		
FDA Session ASE						
Confirmed Send			X X	X X		
Unconfirmed Send			X X	X X	X	X
Open Session			X	X		
Type 9 AR ASE						
AR-Confirmed Send			X X	X X		
AR-Unconfirmed Send			X X	X X	X	X
AR-Associate			X	X		
AR-Abort			X X	X X	X	X X
Type 9 Variable ASE						
Read		X	X X	X X		
Write		X	X X	X X		
Information Report	X		X X	X X	X	X
Type 9 Event ASE						
Confirmed Ack Event		X	X X	X X		
Enable Event		X	X X	X X		
Event Notification	X		X X	X X	X	X
Type 9 Domain ASE						
Initiate Load		X	X X	X X		
Push Segment		X	X X	X X		
Pull Segment		X	X X	X X		
Terminate Load		X	X X	X X		
Type 9 Program Invocation ASE						
Start		X	X X	X X		
Stop		X	X X	X X		
Resume		X	X X	X X		
Reset		X	X X	X X		
Kill			X X	X X		
LAN Redundancy ASE						
Diagnostic Message	X				X	X
SMK ASE						
Find Tag Query	X		X X	X X	X	X
Find Tag Reply	X		X X	X X	X	X
Identify		X	X X	X X		
Annunciate	X				X	X
Device Assignment		X	X X	X X	X	X
Clear Address		X	X X	X X	X	X

11 Type 6 communication model specification

11.1 Application relationship (AR) and Application relationship endpoint (AREP) characteristics

11.1.1 General

The correspondence between AREPs related by an AR is always pre-established by Systems Management.

The following AREP classes are used:

- Queued User-triggered Bi-directional Permanent Connection (QUB-PC) AREP
- Buffered Network-Scheduled Uni-directional Permanent Connection (BNU-PC) AREP

11.1.2 AR-endpoints

ARs are defined as a set of cooperating APs. The AR ASE in each AP manages an endpoint of the AR, and maintains its local context. The local context of an AR endpoint is used by the AR ASE to control the conveyance of APDUs on the AR.

11.1.3 AR-endpoint classes

ARs are composed of a set of endpoints of compatible classes. AREP classes are used to represent AREPs that convey APDUs in the same way. Standardization of endpoint classes allows definition of ARs for different models of interaction.

11.1.4 AR cardinality

ARs characterize communications between APs. One of the characteristics of an AR is the number of AREPs in the AR. QUB-PC AREPs support ARs that convey services between two APs and have a cardinality of 1-to-1. BNU-PC AREPs convey services from one AP to a number of APs and have a cardinality of 1-to-many.

11.1.5 Accessing objects through ARs

ARs provide access to APs and the objects within them through the services of one or more ASEs. One characteristic of AREPs is the set of ASE services that may be conveyed to and from these objects by the corresponding AR. The list of services that can be conveyed by the AR are selected from those defined for the AE.

11.1.6 AR conveyance paths

ARs are modeled as one or two conveyance paths between AREPs. Each conveyance path conveys APDUs in one direction between one or more AREPs. Each receiving AREP for a conveyance path receives all APDUs transmitting on the AR by the sending AREP.

11.1.7 AREP roles

Because APs interact with each other through endpoints, a basic determinant of their compatibility is the role that they play in the AR. The role defines how an AREP interacts with other AREPs in the AR. Endpoint roles are defined to support the client/server and publisher/subscriber models of interaction.

QUB-PC AREPs may operate as a client or a server. BNU-PC AREPs may operate as a publisher, or a subscriber.

Certain roles are capable of initiating service requests, while others are only capable of responding to service requests. This aspect of the definition of a role specifies whether the AR is to be capable of conveying requests in either direction, or only in one direction.

11.1.8 AREP buffers and queues

AREPs may be modeled as a queue or as a buffer provided by the Data Link Layer. APDUs transferred over a queued QUB-PC AREP are delivered in the order received for conveyance.

APDUs conveyed over a buffered BNU-PC AREP are placed in a Data Link Layer buffer for transfer. When the Data Link Layer gains access to the network, it transmits the contents of the buffer. When the AR ASE receives another conveyance request, it replaces the previous contents of the buffer whether or not they were transmitted. Once an APDU is written into a buffer for transfer, it is preserved in the buffer until the next APDU to be transmitted replaces it. While in the buffer, an APDU may be read more than once without deleting it from the buffer or changing its contents.

At the receiving end, the operation is the inverse. The receiving Data Link Layer places a received APDU into a buffer for access by the AR ASE. When a subsequent APDU is received, it overwrites the previous APDU in the buffer whether or not it was read by the AR ASE. Reading the APDU from the buffer is not destructive; it does not destroy or change the contents of the buffer, allowing the contents to be read from the buffer one or more times.

Buffered ARs can be dedicated to one FAL User, typically for the publishing of a single variable (or variable list). When they are, the identity of the variable (or variable list) is known by the context. ARs of this type are known as “dedicated”.

11.1.9 User-triggered and network scheduled conveyance

Another characteristic of an AREP is when they convey service requests and responses. QUB-PC AREPs that convey them upon submission by the user are called user-triggered. Their conveyance is asynchronous with respect to network operation.

BNU-PC AREPs that convey requests and responses at predefined intervals, regardless of when they are received for transfer are termed network-scheduled. The time-intervals used to regulate conveyance are maintained by the Data Link Layer.

11.1.10 Identification of ARs

FAL AEs are operationally accessed through one or more data link DLC identifiers. ARs are identified by the data link identifiers used to configure the AREPs that participate in the AR. The correspondence of these identifiers and DL-addresses is known to the data link bus configuration user.

11.1.11 Definition and establishment of AREPs

AREP definitions specify instances of AREP classes.

AREPs are pre-defined and pre-established.

11.1.12 AR establishment and termination

ARs may be established either before the operational phase of the AP or during its operation. When established during the operation of an AP, the AR Establish service is used. The AR Establish service results in the exchange of AR Establish APDUs.

Once an AR has been established, an AR may be terminated using the Abort service. The Abort service causes the AREP to be closed immediately without the exchange of APDUs.

Instead, the aborted endpoint issues a single abort APDU to the other endpoints participating in the AR that it is no longer capable of sending or receiving APDUs on the AR.

11.2 Relationship of Type 6 to Type 1

11.2.1 General

Type 6 is defined by reference to Type 1.

The Type 6 Application Layer provides an enhanced subset of the services (ASEs) of Type 1 FMS. The subset is defined by the following mechanisms:

- a) None of the Type 1 AREPs are supported. Two new AREPs are defined and supported. These are the QUB-PC 11.2.2.1 and BNU-PC 11.2.2.2 AREPs.
- b) Inclusion of certain ASEs is a conformance class issue 11.2.3
- c) Within supported ASEs, inclusion of certain services is a conformance class issue 11.2.5 and certain other services are never supported 11.2.4.
- d) Certain AREP roles are not supported 11.2.6.
- e) Certain conditional or user parameters and attributes of supported services are not supported 11.2.6.
- f) The Event ASE is enhanced to support notification of the approximate time at which an Event occurred 11.2.8.

11.2.2 Supported AREPs

11.2.2.1 QueuedUserTriggeredBidirectionalPermanentConnection (QUB-PC) AREP

11.2.2.1.1 Description

Corresponding ARs carry both Confirmed Services (CS) and Unconfirmed Services (UCS) over a queued, user triggered, bidirectional AR which utilizes a permanent data link connection

Note1: Event Notifications are carried as an UCS on the QUB-PC AR.

11.2.2.1.2 Formal model

ASE: **AR ASE**
CLASS: **QueuedUser-triggeredBidirectionalPermanentConnectionAREP**
CLASS ID:
PARENT CLASS: **AR Endpoint**
ATTRIBUTES:

1	(m)	Attribute:	Role (Client, Server)
2	(m)	Attribute:	Role = Client
2.1	(m)	Attribute:	Initiator (True, False)
3	(m)	Attribute:	State

SERVICES:

1	(m)	OpsService:	AR Confirmed Send
1	(o)	OpsService:	AR Unconfirmed Send

11.2.2.1.3 Attributes**Role**

This attribute specifies possible role of this end point. The possible values are Client, and Server.

Client	AREPs of this type issue confirmed service request PDUs to Servers and receive their corresponding confirmed service response PDUs. AREPs of this type may also receive unconfirmed service request PDUs from Servers.
Server	AREPs of this type receive confirmed service request PDUs from Clients and issue corresponding confirmed service response PDUs to them. AREPs of this type may also issue unconfirmed service request PDUs to Clients.

Initiator

This conditional attribute is present if the Role attribute has the value of Client. The value of True means that this ARep can issue but cannot receive an Establish request PDU. The value of False means that this ARep can receive but cannot issue an Establish request PDU.

State

This attribute indicates the current state of the Application Relationship Protocol Machine (ARPM) that is defined in detail in Part 6. The possible values for this attribute are specified there.

11.2.2.1.4 Services**AR Confirmed Send**

This service is used to send a confirmed service on the specified AR.

AR Unconfirmed Send

This optional service is used to send an unconfirmed service on the specified AR.

11.2.2.2 BufferedNetworkScheduledUnidirectionalPermanentConnection (BNU-PC) ARep**11.2.2.2.1 Description**

Corresponding ARs carry Unconfirmed Services (UCS) over a buffered, network scheduled, unidirectional AR which utilizes a permanent data link connection

11.2.2.2.2 Formal model

ASE:	AR ASE
CLASS:	QueuedUser-triggeredUnidirectionalPermanentConnectionAREP
CLASS ID:	
PARENT CLASS:	AR Endpoint
ATTRIBUTES:	
1 (m)	Attribute: Role (Publisher,Subscriber)
2 (o)	Attribute: Dedicated
3 (m)	Attribute: State
SERVICES:	
2 (m)	OpService: AR Unconfirmed Send
3 (m)	OpService: AR Get-Buffered-Message
4 (o)	OpService: AR-Status

11.2.2.2.3 Attributes

Role

This attribute specifies possible role of this end point. The possible values are Publisher and Subscriber.

Publisher AREPs of this type publish their data issuing unconfirmed service request-PDUs.

Subscriber AREPs of this type receive data published in unconfirmed service request-PDUs.

Dedicated

This attribute specifies whether the publisher AREP is dedicated to publish one variable or one variable list

State

This attribute indicates the current state of the Application Relationship Protocol Machine (ARPM) that is defined in detail in Part 6. The possible values for this attribute are specified there.

11.2.2.2.4 Services

AR Unconfirmed Send

This service is used to send an unconfirmed service on the specified AR.

AR Get-Buffered-Message

This service is used to retrieve the current contents of the associated buffer.

AR-Status

This optional service is used to report status of the AR.

11.2.3 ASEs whose support is a conformance class issue

ASEs whose support is a conformance class issue are:

- a) Event ASE
- b) Load Region ASE
- c) Function Invocation ASE

11.2.4 Services which are not supported

11.2.4.1 Management (Mgt) ASE

Within the Management ASE, the following services are not supported:

- a) Create
- b) Delete

11.2.4.2 Application Process (AP) ASE

Within the Application Process ASE, the following services are not supported:

- a) Subscribe
- b) Status notification
- c) Conclude
- d) Reject

11.2.4.3 Application Relationship (AR) ASE

Within the Application Relationship ASE, the following services are not supported:

- a) AR-De-Establish
- b) AR Compel
- c) AR Schedule Communications
- d) AR Cancel Schedule Sequence
- e) AR Remote Read
- f) AR Remote Write

11.2.4.4 Event ASE

Within the Event ASE, the following services are not supported:

- a) Unconfirmed Ack Events
- b) Notification Recovery

11.2.4.5 Function Invocation ASE

Within the Function Invocation ASE, the following services are not supported:

- a) Action Invoke
- b) Action Return

11.2.5 Services whose support is a conformance class issue

11.2.5.1 Management ASE

Within the Management ASE, services whose support is a conformance class issue are:

- a) Set Attributes
- b) Begin Set Attributes
- c) End Set Attributes

11.2.5.2 Application Relationship (AR) ASE

Within the Application relationship ASE, services whose support is a conformance class issue are:

- a) AR Unconfirmed Send
- b) AR Status

11.2.5.3 Variable ASE

Within the Variable ASE, services whose support is a conformance class issue are:

- a) Read List
- b) Write List
- c) Information List
- d) Exchange List

11.2.5.4 Event ASE

If the Event ASE is supported, then within the Event ASE, services whose support is a conformance class issue are:

- c) Ack Event List
- d) Get Event Summary List
- e) Query Event Summary List

11.2.6 AREP roles which are not supported

The following AREP roles are not supported:

- a) Pull Publishing Manager
- b) Pull Publisher
- c) Pull Subscriber
- d) ReportSource
- e) ReportSink

11.2.7 Conditional or user parameters and attributes of supported services which are not supported

Conditional or User parameters and attributes of supported services which are not supported are:

- a) Destination DL-Address and Source DL-Address - these parameters are replaced by the LocalDlIdentifier of the sender
- b) Remote DLSAP address - this parameters are replaced by the LocalDlIdentifier of the sender
- c) Local and remote timeliness
- d) Additional detail
- e) Duplicate FAL-PDU body
- f) Duplicate APDU body

11.2.8 Event ASE Enhancement

The Type 1 Event ASE is enhanced by the addition of an optional DL-TIME-offset parameter.

When a user sends an event notification with the EventNotification service, he may optionally transmit the DL-TIME-offset of the Event., which has been determined by the Type 6 DL as defined in 61158-4, as the Notification Time parameter of that service.

11.3 Permitted services by AREP role

Table 454 defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 454 – Services by AREP role

FAL Services	Unc	Cnf	Client	Server	Publisher	Subscriber
			req rcv	req rcv	req rcv	req rcv
Mgt ASE						
Get Attributes		X	X	X		
Get Attribute List		X	X	X		
Set Attributes		X	X	X		
Begin Set Attributes		X	X	X		
End Set Attributes		X	X	X		
AP ASE						
Identify		X	X	X		
Get Status		X	X	X		
Initiate		X	X	X		
Terminate		X	X	X		
AR ASE						
AR-Confirmed Send			X	X		
AR-Unconfirmed Send			X X	X X	X	X
AR-Establish			X	X		
AR-Abort			X X	X X	X	X
AR-Get Buffered Msg			X	X		X
AR-Status			X	X		
AR-XON-OFF			X	X		
Variable ASE						
Read		X	X	X		
Read List		X	X	X		
Write		X	X	X		
Write List		X	X	X		
Information Report	X				X	X
Information Report List	X				X	X
Exchange		X	X	X		
ExchangeList		X	X	X		
Event ASE						
Confirmed Ack Event		X	X	X		
Ack Event List		X	X	X		
Enable Event		X	X	X		
Enable Event List		X	X	X		
Get Event Summary		X	X	X		
Get Event Summary List		X	X	X		
Query Event Summary List		X	X	X		
Event Notification		X	X	X		
Load Region ASE						
Initiate Load		X	X	X		
Push Segment		X	X	X		
Pull Segment		X	X	X		
Terminate Load		X	X	X		
Discard		X	X	X		
Function Invocation ASE						
Start		X	X	X		
Stop		X	X	X		
Resume		X	X	X		
Reset		X	X	X		
Kill		X	X	X		

12 Type 7 communication model specification

12.1 Concepts

12.1.1 AL environment

This general model presents the relationship between the AL environment and the application layer services which is the purpose of this standard.

The AL environment is both process control and discrete parts manufacturing. In those systems, manufacturing devices are controlled by a system that performs the automation functions.

12.1.2 Application services

Application services can be divided into two sets. See Figure 94.

The standard specifies the MPS (Manufacturing Periodic/aperiodic Services) application service set. The other service set corresponds to a subset (subMMS) of that specified in the MMS standard.

MPS services are particularly well suited to the real time requirements of a distributed application in the fact that they support:

- The periodic broadcast update of a distributed database in the various control system devices.
- The elaboration of temporal qualifiers associated with the data base.
- The elaboration of consistency qualifiers associated with sets of data within this database.

SubMMS and MCS are essentially considered to fulfil the requirements for configuration and operating mode management within the distributed application.

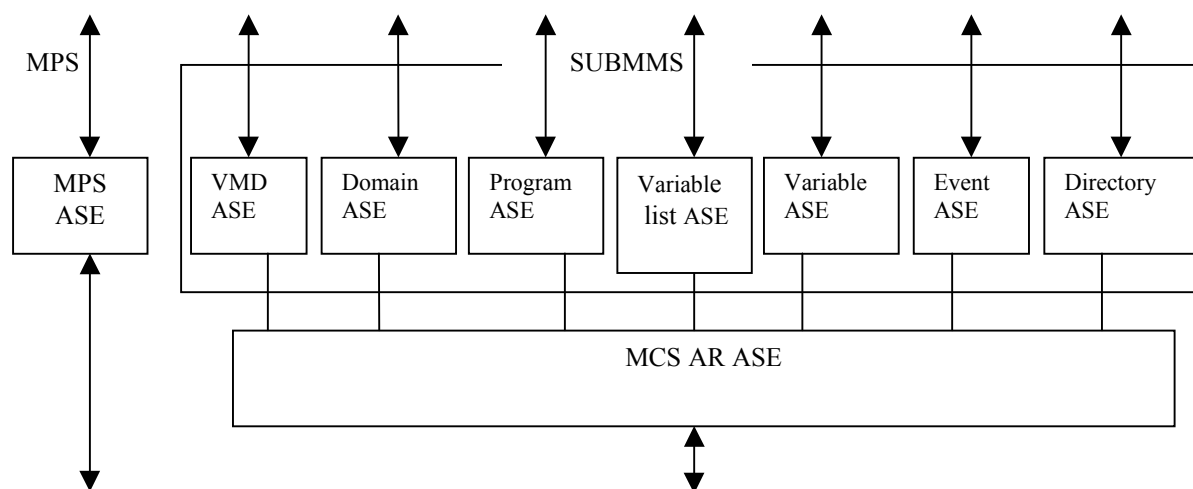


Figure 94 – Organisation of the ASEs and ARs

A distributed application is characterized by various information processing tasks on various devices of a communication system for the requirements of check a production system control application.

Cooperation between the tasks belonging to different devices is modelled in the form of interactions between application processes (APs).

A distributed application is composed of two or more APs, each of the devices of the communication system which can have zero, one or several of them.

As provided for by this standard, the communication between two APs passes via the network.

NOTE The breaking down of a distributed application in several AP's at the level of a device is decided by the user according to his own criteria (Methodology, Functional structuring of the application, Setting into service facilities, etc.)

12.1.3 Application process

12.1.3.1 Application process overview

An AP is modelled by one or several information processing tasks in a device for the requirements of a particular function of the distributed application.

The aspects of an AP, taken into account by the AL, only concern its communication capacities which are modelled by Application Entities (AEs). Each AP can have at the most one AE of a given type.

On the other hand, the cooperation between APs in the framework of a distributed application are performed via the setting up of the relations between AP invocations (APIs) which represent its activities. An AP can, at a given instant, zero, one or several APIs. Besides, each API has one particular utilization of the AE (AEI) which satisfies the communication requirements.

12.1.3.2 AP model

12.1.3.2.1 AP model description

The purpose of the AP model is to show all the aspects of an AP which are taken into account by the AL communication.

NOTE The other aspects concerning the type of information, processing and execution of the latter are not modeled, since they are not involved at the communication level.

Class: APPLICATION PROCESS (AP)

Key Attribute: AP title

Attribute: AP type

Attribute: List of Inherit from Application Entity

Attribute: List of Reference API

12.1.3.2.2 Attributes:

AP title:

A title is attributed to an AP to identify it, among all the other APs, in a non-ambiguous manner in a multisegment AL network. For the requirements of the universal identification of an AP in the AL environment, this AP tile is involved as a constituent of the ISO name proper to AL user site.

This identification can be structured from the name of the device to which it belongs.

NOTE To structure this identification, this AP shall remain permanently in the equipment.

AP type:

The type of an AP helps to designate a set of APs to which it belongs.

NOTE This type is not covered by a title which can be handled in the AL environment.

List of Inherit from Application Entity

The Application Process class inherits an Application Entity class list which gives it the capacity to communicate in the AL environment.

Thus an AP has one or several application entities, which are necessarily of different types.

List of Reference API:

Designates the lists of APIs which model the AP activities. This list can vary dynamically but at a given instant zero, can designate one or several APIs.

12.1.3.3 API model

12.1.3.3.1 API model description

An API model describes the activities of an AP.

Class: APPLICATION PROCESS INVOCATION (API)

Key Attribute: API identification

Attribute: Reference Application Process

Attribute: List of Reference AEI

12.1.3.3.2 Attributes:

API identification:

Identifies an Application Process Invocation in a non-ambiguous manner in the given AP field. It is underlined that the scope of this identification is global for the AP and independent of the rules for naming the AP title.

Reference Application process

The purpose of this attribute is to show that this API belongs to a given AP.

The lifetime of an API is necessarily less than or equal to that of the AP to which it belongs.

List of Reference AEI:

This list designates AE uses which are reserved for the API requirements.

An API can use zero, one or several AEIs for each AE of the AP to which it belongs.

The number of referenced AEIs can vary dynamically in time. Consequently, an API, at a given moment, can have one or several AEIs (Complete definition of an AEI in the following paragraph).

12.1.4 Application entity

An AE represents a set of communication capacities for the requirements of an AP. These communication capacities are defined by a set of Application Service Elements (ASEs). The particular uses of the communication capacities of an AE are represented by AE invocations (AEIs).

An AEI consequently models the communication functions and their state for the particular activities of an API. Communications between APIs take place via AEIs which, as required, may or may not be related by Application Associations (AAs) or Application Relationship (AR) in the AL context.

An AEI in the AL environment gives an API the exclusive possibility of communicating with or without association. An AEI can only participate in one and in only one association application.

The communications without association allow an AEI to exchange information with one or several other AEIs.

An association application allows an AEI to communicate with one or several other AEIs in the framework of a common Context Application (CA).

This context can be negotiated or predefined by configuration for each AEI, for a point to point association which makes use of a pair of AEIs. The context is necessarily prenegotiated for each AEI.

The state of an AEI is thus directly linked to the fact that the latter participates or not in an association and to the state of the association if it participates in a negotiated association.

Generally it can be said that the lifetime of an AEI is equivalent to that of the service exchanged for the communications without association and equivalent to that of the association for communications with associations. However, for a negotiated association, the lifetime of the pair of AEIs is lengthened by their opening and closing transients.

The lifetime of an AEI is controlled by the API which it represents in the AL environment. An API, on the other hand, can have a much longer lifetime than that of its AEIs, equal to that of the AP to which it belongs.

Particular AR are predefined by configuration, these AR are only used by the MPS ASE to exchange plain data (variable). All the parameters relative to the variable are pre-established at the configuration, these parameters never change. Only a new configuration can cancel or modify these particular AR. The identification of this peculiar AR is made by a number unique on the network, this number refer to the whole set of parameters of the AR and the DLL resources implied for the communication. This number similar to a DLCEP Id is called Identifier.

12.1.5 AE model

12.1.5.1 AE model description

Description of the communication capacities offered by an AE.

Class: APPLICATION ENTITY (AE)

Attribute: AE type

Key Attribute: Qualifier

Attribute: List of Inherit from ASE

Attribute: List of Reference AEI

12.1.5.2 Attributes:

Qualifier:

An AE qualifier identifies an AE non-ambiguously in the field of an application process. The combination of this AE qualifier with the AP title forms an AE title which identifies this AE non-ambiguously in the AL environment.

This AE title allows recovery from the directory of the addressing information, the link addresses.

An AE title can have "Synonyms of the AE title" to be distinguished from one or several other AEs by different titles.

An AE can be globally identified with other AEs with the help of an "AE group designation".

AE type:

The AE type designates a set of AEs sharing the same communication characteristics. Various AEs of the same type inherit the same list of application service elements.

List of Inherit from ASE

The Application Entity (AE) class inherits an ASE class list which gives it the specificity of its communication functions. An AE can support one or several ASEs, the latter being necessarily of different types.

List of Reference AEI

Designates the list of AEIs which model the AE uses. This list, which can vary dynamically, can designate at a given moment, zero, one or several AEIs.

12.1.6 AEI model

12.1.6.1 AEI model description

An AEI model is used to describe a particular use of the AE.

Class: APPLICATION ENTITY INVOCATION (AEI)

Key attribute: AEI identification

Attribute: Reference AE

Attribute: Reference API

Attribute: Local AE address

Attribute: Remote AE address

Attribute: Communication mode

[ASSOCIATED; NON-ASSOCIATED]

Constraint: Communication mode = ASSOCIATED

Attribute: Association state

[OPENING;
ESTABLISHED;
CLOSED]

Attribute: Association type

[NEGOTIATED; PRENEGOTIATED]

Attribute: Association type

[NEGOTIATED; PRENEGOTIATED]

Attribute: Local AEI access point

Attribute: Remote AEI access point

Attribute: Inherit from Application context

12.1.6.2 Attributes:

AEI identification:

Allows non-ambiguous identification of an application entity invocation in the sub-field of an AE reserved for an API of a given AP.

The Non-ambiguous designation of an AEI in the environment requires indicating: AP title / API identification / AE qualifier /AEI identification.

Reference AE:

This reference is used to designate the AE object to which this AEI belongs,

The lifetime of an AEI is necessarily less than or equal to that of the AE to which it belongs.

Reference API:

This reference helps to designate the API object which is represented in the AL environment by this

AEI. An AEI can represent one and only one API in the AL environment.

Local AE address:

Corresponds to the link address which allows locating the AE to which this AEI belongs.

This address is completed with the help of directory functions during the creation of this AEI and remains static for its lifetime.

Remote AE address:

Corresponds, either to a link address which allows locating the AE with which this AEI is corresponding, for point to point exchanges, or with a link address which helps to locate an AE group with which this AEI is corresponding for multipoint exchanges.

This address is filled at the AEI level during the opening of an association with one other AEI or when exchanging data in a non-associated mode.

Communication mode [ASSOCIATED; NON-ASSOCIATED]

(The communication mode in the ASSOCIATED state indicated that this AEI allows an API to communicate in a application context which was previously defined or negotiated. The type of exchanges between AEIs, possible in such a communication mode, is point to point or multipoint.

When the communication mode is NON-ASSOCIATED, this AEI allows an API to communicate outside all pre-established context. The nature of the possible exchanges in a non-associated communication mode is point to point or multipoint.

Association state

[OPENING;
ESTABLISHED;
CLOSING]:

In the associated mode, the possibilities of communication of an AEI with another AEI depends on the state of the association. The various states correspond to the phases in the life of an association application:

OPENING:

Corresponds to an AEI which, at the request of the API which it represents, is negotiating an application context with another AEI, for the requirements of an association. In this state, no exchanges can take place outside those reserved for negotiation of the context.

ESTABLISHED:

Corresponds to an AEI which offers the API which it represents, the possibility of communicating with another API via one of its determined AEIs, within the framework of an application context. This application context has been obtained either by negotiation between the AEI pair or by local predefinition at the AEIs.

CLOSING:

Corresponds to an AEI which, at the request of the API which it represents, or with which it had been associated, is being freed from its application context.

Association type

[NEGOTIATED;
PRENEGOTIATED]

This type indicates if the association has been negotiated or not. This attribute, if it takes the PRENEGOTIATED value, imposes on the association to remain exclusively in the ESTABLISHED state.

Local AEI access point:

Each AEI, in the ASSOCIATED mode, can be directly localized by its AEI access point. This addressing information is filled with the help of the directory functions during the creation of an AEI instance and remains static for its full lifetime.

AEI remote access point:

Each AEI, which is in the ASSOCIATED mode, has this addressing information which localizes the AEI with which it is in correspondence for point to point exchanges. This information is filled in for an AEI at the opening of an association.

This information locates the AEIs with which it is in correspondence for multipoint exchanges.

Inherit from Context Application

This inheritance allows designating the object of the application context which conditions communications with this AEI.

The attributes of this application context are valued during installation of an AEI on the basis of the values proposed by the user in the negotiated associated mode and the non associated mode, whereas they are filled by configuration in the prenegotiated associated mode.

12.1.7 Application context

12.1.7.1 Application context overview

A pair of AEIs require common knowledge of the rules which govern their communications. This set of rules is called Context Application. The application context which can be supported by an AEI is necessarily derived from the possibilities offered by the various ASEs possessed by the AE to which it belongs.

12.1.7.2 Application service elements

12.1.7.2.1 ASE description

The possibilities offered by the various ASEs of an AE are defined by the specification (in ASN1) of one or more set of Application Protocol Data Units (APDU) which form abstract syntaxes. Besides, the use of an abstract syntax to dialogue between two AEs requires the implementation of a transfer syntax which defines the rules for encoding of the APDUs on the bus.

NOTE Among the abstract syntaxes associated with the ASE MMS, the general abstract syntax ISO-9506-MMS-1 can be noted as well as other abstract syntaxes such as those meant for digital controls ISO-9506-NCCS-1 and process control ISO-9506-PROCESS-1.

Class: ELEMENT SERVICE APPLICATION

Attribute: ASE type

Attribute: List of Abstract syntax

Attribute: List of Transfer syntax

12.1.7.2.2 Attributes:**ASE type:**

Several ASE types can be met with in the AL environment. This standard is covered by the SubMMSE definition derived from the ASE MMS defined by the ISO.

The MCSE (Messaging Common Service Element) service corresponds to the control ASE which shall necessarily be used to project another ASE application on the link layer messaging. This service element has unique encoding rules which are universally known in the AL messaging environment.

NOTE The MCS encoding rules are not given in ASN1. They are directly given in the representation used for the transfer.

List of Abstract Syntax:

An application service element can have one or several abstract syntaxes. Each of them correspond to a set of APDUs described in an ASN1 module universally identified in the AL environment.

NOTE For example, an abstract syntax defined in ASN1 has the following appearance:

```
<module name> <module universal identifier>
Definitions ::= BEGIN{
IMPORTS <..>, <..>, .....FROM <external module>
EXPORTS <..>, <..>, ...
    PDU module ::= CHOICE {
        {<production name> [N°] <production>} +
    }
END
```

This standard keeps AL SUBMMS-1, derived from the reference core, as one of its abstract syntaxes for the MMS service element.

List of Transfer syntax:

A transfer syntax defines the abstract syntax encoding rules used. An ASE could support several transfer syntaxes within the framework of an AE, which could be used according to the requirements depending on the possibilities of their correspondents.

One of the transfer syntaxes taken into account by this AL environment standard for the implementation of the AL abstract syntax, AL SUBMMS-1, is proposed by the ISO basic encoding rules (BER).

Like the abstract syntaxes, the transfer syntaxes are universally identified in the AL environment.

12.1.7.3 Type of application context**12.1.7.3.1 Application context description**

The application context belonging to an AEI conditions the possible communications with it. In the information contained in the application context, some is specific to the AE architecture, i.e. with the type of ASE resting on MCS as well as with the abstract transfer syntax selected. Other information is specific to the choice performed on the API level and corresponding to the restrictions to the negotiated abstract syntax and to the quantity of communication resources used.

Class: APPLICATION CONTEXT

Attribute: Context name

Key Attribute: Context identification

Attribute: Reference ASE

Attribute: Abstract syntax

Attribute: Transfer syntax

Constraint: Com. mode = ASSOCIATED

Attribute: Application reductions

Attribute: Service quality

12.1.7.3.2 Attributes:

Identification of the context:

Allows identifying an application context in the field of an AE. It thus has an exclusive local role.

Context name:

Used to identify, in the AL environment, the bases of a context consisting of the selected ASE, an abstract syntax and a transfer syntax. This name is used during the association negotiation or during a transfer of data in the non-associated mode.

Reference ASE:

One of the AE application service elements which is selected by this application context.

Abstract syntax:

An abstract syntax among those of the ASE within the framework of the AE, which is selected by this context.

Transfer syntax:

The transfer syntax selected for encoding the ASE PDUs, implemented within the framework of this AE, which is selected by this context.

Application reductions:

The abstract syntax selected at the level of an AEI application context for a given service element can be used in a reduced manner. This possibility is used only for transmission in the associated mode.

Service quality:

The selected service quality.

12.1.7.4 Directory

12.1.7.4.1 Directory overview

In order to communicate mutually, the AEIs make use of directory functions which given them the addressing information which they require. These directory functions are necessary exclusively during the opening of association or during a transmission in the mode without association.

12.1.7.4.2 Addressing model

The link layer offers data transfer services in the non-connected mode and an addressing space allowing location of the application entities. This addressing space allows locating the entities Individually and/or by groups, and on the other hand to locate them with the help of physical or logic addresses. These two types of addresses respectively allow taking into account or not of the network topology in the location of application entities.

The application layer has a double location requirement, that of the AEs during negotiation of the associations or transmission of data without association, and that of the AEIs during transmission of data within the framework of an association.

Consequently, the AE addresses and the AEI access points which need to be located are addressed with the link addresses from the single link addressing space.

In the case of point to point exchanges, the AE addresses and the AEI access points are designated with the help of individual link addresses. In the case of multipoint exchanges, the AE address and the AEI access point locating the transmitter are designated by individual link addresses whereas the AE address and the AEI access point locating the receivers are designated by group link addresses.

The rules allocating the liaison addresses used for the AE addresses and the AEI access points are not defined in this standard.

The abbreviations used to speak about these two uses of link addresses are: ADAE for those which are associated with AE addresses and ADAEI for those which are associated with AEI access points.

NOTE The ADAEIs are allocated by the directory functions of a device to the AEIs, in the sub-set which was attributed to an AE.

The address values contained in this sub-set can be, for example selected by dividing up the addressing space according to the network architecture or well chosen according to other criteria better satisfying the communication requirements of the user application.

Addressing rules:

Rule_1: Each AE is identified by one and only one "AE title" which is in a bi-univocal relation with an ADAE, allowing it to be located in the AL environment.

Rule_2: Each AE can be designated by one or several synonyms, each being in relation with one and only one address.

Rule_3: Each AE can be designated in the framework of one or several "AE group designations", each being in relation with one and only one address.

NOTE It is important to note that biunivocity is not a property of rules 2 and 3.

Rule_3: Each AEI which is identified by an AEI identification in the field of an AE, an API, of a given AP, is in biunivocal relation with an ADAEI, allowing it to be located in the AL environment.

12.1.7.4.3 Directory functions

The projection between the names allowing identification of the various objects (AEs and AEIs) in the application, and the link addresses (respectively ADAE and ADAEI) used to locate them in the AL environment, is managed by the directory service.

For any communication initiative at the level of an AEI, the latter calls the directory functions which give it the addresses necessary for the exchanges.

Initiator Directory Function (IDF):

This function is implemented for an AEI initiating an exchange concerning an association opening or a transmission of data in the non-associated mode.

IDF is used to:

- obtain an ADAE pair allowing location of the local and remote AEs according to their AE titles,
- allocate an ADAEI for the local AEI access point and optionally of another for the remote AEI access point. This allocation only takes place when setting up an association.

<Input parameters> ::=

<Local AE title>
 <Remote AE title
 | Synonym of the AE title
 | Designation of the AE group>
 <Transmission mode>

with <Transmission mode>::=<ASSOCIATED | NON-ASSOCIATED>;

<Output parameter> ::=

<Local ADAE>
 <Remote ADAE>
 [<Local ADAEI>] (*)
 [<Remove ADAE>] (**)

(*) This ADAEI is only returned in the case of an ASSOCIATED" transmission mode.

(**) This ADAEI is returned or not according to the remote AE title.

Responder Directory Function (RDF):

This function is implemented at the level of a called AE, for the opening of an association.

RDF:

Used to allocate an ADAE at the level of the called entity during the response phase for an association opening request.

<Input parameters> ::=

<Local ADAE>
 <Remote ADAE>

<Output parameters> ::=

<Local ADAEI>

NOTE This function is not used on the level of a responder if the initiator of the communication had already performed the allocation by use of the function.

12.1.8 Coordination of the AEIs

12.1.8.1 AEI communication description

The initiative and the type of communications belong to the APIs. The four types of communication are:

- Information exchanges in the non-associated mode.
- Association opening requests.
- Information exchanges in the associated mode.
- Association closing requests

NOTE For each of these types of communication, the AL environment objects are implemented in a specific manner which is covered by the following paragraphs.

12.1.8.2 Information exchanges in the non-associated mode

When an API wishes to initialize a data transfer in the non-associated mode, it proceeds in the following manner:

Exchange scenario:

- It addresses, as applicable:
 - either the source AE identified by its <AE title>, indicating the addressee (either by its <AE title>, or by an <AE synonym title>, or by an <AE group designation> and the <Application context>,
 - or an AEI whose mode is NON-ASSOCIATED, if the latter has been created to allow a series of exchanges.
- The local procedure which takes place at the AE after this request by the API is as follows:

If the API addresses the AE, the following local procedure is executed:

- Creation of an AEI object in the NON-ASSOCIATED mode,
 - Entering of the attributes of this object:
 - <Application context> takes the value indicated by the request,
 - <Remote AE address> takes the value returned by IDF for the non-associated communication mode,
 - <Local AE address>: takes the value returned by ID for the non-associated communication mode.
 - Then whatever the addressing which took place, a data transfer service in the MCSE non-associated mode takes place within the framework of this AEI. The user data encapsulated in this data transfer is that of the MCS user application service which was requested.
 - The AEI thus used could be maintained or revoked after this exchange. In case of revocation, this will be effective immediately if the service is not confirmed or after reception of the response and passage to API of the data contained in it, if the service is confirmed.
- The following procedure is executed at the addressee AE:
 - Depending on whether an AEI does not yet exist or exists already, the following operations are executed or not:
 - Creation of an AEI object in the NON-ASSOCIATED mode.

• Filling of the attributes of this object:

- <Application context>: takes the value indicated in the received PDU, if it is supported by the AE.
- <Remote AE address> takes the value contained in the received PDU.
- <Local AE address> takes the value contained in the PDU.
- Identification of an API object designated implicitly or explicitly in the PDU.
- Passage of the <Application service> contained in the PDU received at the API identified for the execution.
- The AEI thus used can be revoked or preserved at the end of this exchange. In case of revocation, this will be effective immediately if the service is not confirmed or after transmission of the response if it is confirmed.

12.1.8.3 Association opening request

If the API wishes to initialize an association opening request, it proceeds in the following manner:

Opening scenario:

- The API locally addresses the calling AE identified by its <AE title>, indicating:
 - the called <AE title>,
 - the <opening service> (Initiate Rq in MMS) which contains the <Application reductions> of the association, which will be negotiated,
 - the <Application context> description other than the application reductions which will be negotiated at the MCS level.
- The local procedure which will be executed at the AE level after this API request, is the following:
 - Creation of an AEI object in the ASSOCIATED mode in the OPENING state.
 - Entering of the attributes of this object:
 - <Remote AE address; local AEI access point>: take the value returned by IDF.
 - Consideration of the application context requested by the user, and possible reduction of this, if its local AE cannot offer it.
 - A_Associate.Rq service call by MCSE in the framework of this AEI with the application context given by the user as well as the opening request user PDU.
 - On reception of the response, the other AEI attributes can be filled:
 - <Remote AEI access point>: takes the value contained in the PDU.
 - <Application context> takes the value contained in the response which can correspond to a reduction of that requested.
 - Passage of the AEI state to ESTABLISHED,
- The procedure which takes place at the responder AE level on reception of an opening request transmitted by the source procedure is the following:

- Creation of an AEI object in the ASSOCIATED mode in the OPENING state.
- Filling of the attributes of this object:
 - <Application Context>: takes the value indicated in the PDU, or a reduction, if the AE does not support it.
 - <Local AE address> takes the value received in the PDU.
 - <Local AEI access point> can take the optional value contained in the PDU or that returned by the RDF (Responder Directory Function).
 - <Remote AE address; Remote access point> take the values received in the PDU.
- Addressing of the API from its implicit or explicit identification in the PDU.
- Passing to the user of the opening request PDU.
- When the API gives its response, a call of the A_ASSOCIATE.Rp service of MCSE in the framework of this AEI, with the PDU user, as a response to the opening request and possible updating of the context if this has been reduced by the API.
- Passage of the AEI state to ESTABLISHED.

12.1.8.4 Exchange of information in the associated mode

When an API wishes to initialize an associated mode exchange, it proceeds as follows.

Exchange scenario:

- It addresses the AEI locally in the ESTABLISHED state, identified by its <AEI identifier> relative to its area by indicating:
 - the <application service> which shall be executed with the called entity.
- The local procedure which shall be performed at the AEI level is as follows:
 - During reception of the response, for a confirmed service, sending of this to the user.
 - Call of the data transfer service in the MCSE associated mode in the AEI framework, with as user data the PDU associated with the service.
- The procedure executed at the responder AEI on reception of the service request is as follows:
 - Passing of the <Application service> contained in the API PDU.
 - Service call and information transfer in the MCSE associated mode, in the framework of the AEI, with as user data the response PDU associated with this service.

12.1.8.5 Request for termination of an association.

When an API wishes to initialize a request for termination of an association, it proceeds as follows:

Closing scenario:

- It locally addresses the AEI identified by its <AEI identification> in its area, indicating:
 - the <Closing application service>.
- The local procedure which is executed at the AEI initiating this closing is as follows:
 - Setting of the AEI to the CLOSING state (Blocking of the data transfers beyond what has been engaged).
 - MCSE A-RELEASE service call to transmit the closing request service in the framework of this AEI.

- On reception of the positive response, revocation of the AEI object, return to the ESTABLISHED state in the other cases;
- The remote procedure which is performed at the responder AEI level is as follows:
 - Setting of the AEI to the CLOSING state.
 - (Blocking of the data transfers outside those engaged).
 - Passing of the closing request service to the user.
 - At a positive response from the user, revocation of the AEI object and sending of the response to the initiator using the MCSE.

12.1.9 Concepts for sub_MMS

12.1.9.1 General

Sub-MMS communications can be established through negotiated or predefined association as well as without association.

The assignment of access rights to association as well as the protections of objects, allow and the protection of objects authorizes selective access to the objects.

12.1.9.2 Sub-MMS communication channels

1- A Sub-MMS communication can be achieved without association:

These communication can take place in point to point (confirmed/unconfirmed services), in multipoint or in distribution (unconfirmed services).

In this case a service request (other than Initiate, Conclude and Abort) shall be refused if the service is not supported, if the resources are insufficient, if it is not in compliance with the protocol, etc.

The Initiate, Conclude and Abort services have no significance.

This type of communication does not support the access protection mechanism on the objects.

2- A Sub-MMS communication can be performed in a predefined association:

These communication can take place in point to point (confirmed/unconfirmed services), in multipoint or in distribution (unconfirmed services).

In this case a service request (other than Initiate, Conclude and Abort) shall be refused if the service is not supported, if it is not in compliance with the protocol or if the resources reserved for the association are insufficient, etc.

The Initiate, Conclude and Abort services have no significance.

This type of communication does/does not support the access protection mechanism on the objects.

3- A Sub-MMS communication can be performed in a negotiated association:

These communication can take place in point to point (services confirmed/unconfirmed),

In this case a service request (other than Initiate, Conclude and Abort) shall be refused if the service is not supported, if it is not in compliance with the protocol, or if the resources reserved for the association are insufficient, etc.

This type of communication does/does not support the access protection mechanism on the objects.

12.1.9.3 Access protection mechanism

12.1.9.3.1 Protection level introduction

The access to Sub-MMS object can be refused to some clients. For this we define the protection levels assigned to an object and access right assigned to the association.

12.1.9.3.2 Protection level assigned to an object

The definition of objects other than the VMD object includes an OPTIONAL attribute called "ACCESS PROTECTION" which determines if an object is protected or not.

An object which is not protected has its "ACCESS PROTECTION" attribute forced to the value "FALSE".

An object which is protected has its "ACCESS PROTECTION" attribute forced to the value "TRUE".

- 1 – Password,
- 2 – Access Groups,
- 3 – Access Rights.

Password:

The "Password" attribute allows restricting access only to clients holding the password.

Access Groups:

"Access Groups" attribute allows to restrict the access only to clients who belong to one of the groups defined by the attribute.

Access Rights:

The "Access Rights" attribute indicates the various possibilities to obtain access to the object.

It also indicates for each of these access possibilities, the services which can be executed as well as the conditions to be fulfilled, as shown in Table 455.

Table 455 – Access protection

Object class	Executable operations
Domain	Load, Upload, Delete, Connect to a program invocation
Program invocation	Execute, Initialize, Stop, Delete
Variable	Read, Write
Variable-List	Read, Write, Delete
Event	Read, Alter, Acknowledge
Access granted because of password	Conditions to be fulfilled
Yes	a) The association password is identical to that of the object.
No	b) The value of the password of the association is ZERO; condition (a) is not fulfilled
Access granted because of group membership	Conditions to be fulfilled
Yes	a) The group word for the association and the object have at least one identical group No
No	b) The value of the association group word is ZERO; condition (c) is not fulfilled
Access granted to all	Condition to be fulfilled
Yes	e) The "Access Rights" attribute authorizes access to all clients.

12.1.9.3.3 Level of the access rights assigned to an association

It is possible to assign access rights to predefined or negotiated associations. In this case these access rights are characterized by the parameters "Password" and "Access Groups".

The "Password" and "Access Group" parameters are granted implicitly to the predefined association.

The "Password" and "Access Group" parameters are granted to the negotiated association by the initiate service.

The "Password" and "Access Group" parameters thus characterize the Client's access rights.

12.1.9.3.4 Access control mechanisms for protected objects

Access to the objects can be granted:

- a) either when the "Password" attribute of the object is identical to the association "Password" parameter
- b) when the "Access Groups" attribute of the object and the "Access Groups" parameter of the association have at least one group No. in common.
- c) or to all the clients when the "Access Rights" attribute permits.

NOTE No access restriction is imposed for the use of the following services:

- Get Name List,
- Get Domain Attributes,
- Get Program Invocation Attributes,
- Get Variable Access Attributes,
- Get Variable List Attributes,
- Get Event Condition Attributes,
- Get OD Header/Data Type Attributes.

12.1.9.4 Directory management

12.1.9.4.1 General

All the sub-MMS object describers supported by an application form a directory.

The arrangement of the describers in a directory can be either standardized or free.

The standardization of the directory is performed by an object called "OD" (Object Dictionary).

12.1.9.4.2 Use of the "OD" object

It is not mandatory for a device to support the "OD" object.

12.2 ASEs

12.2.1 MPS ASE (periodic/aPeriodic manufacturing services)

12.2.1.1 Overview

In a MPS environment, a certain number of application objects are implemented. These objects come directly from the instantiation of the different classes previously described.

All these application objects representing the network configuration shall be described partially or globally in order to allow:

- the user to have access to the application configuration (e.g.: Access to the variable type),

- the checking of the configuration consistency of the application between several subscribers (e.g.: configuration consistency between the producer and the consumers of a variable).

Moreover, this description is specified in this standard in order to ensure the interoperability in the configuration interchanges between different implementations.

12.2.1.2 Object concepts

12.2.1.2.1 Application objects

The description of periodic/aperiodic MPS services leads to describe all the resources involved in the application layer. This specification defines the static part of the specification.

The specification reactive aspects describe the various service elements actions on the application layer resources:

MPS environment application objects are divided into two sets:

- Objects related to variables,
- Objects related to variable lists.

Variables are abstract elements that associate a transfer syntax and a semantics to a data.

Variables, and related information, are provided to the user by the communication system, either individually, or by lists.

Other variables are provided with an event characteristic to which the user may associate actions relative to the execution of the distributed application.

12.2.1.2.2 Application objects addressing

The key-attributes (A_Name) used to address the MPS application layer objects all belong to the same addressing space.

Each of these names in the addressing space is defined within a maximum length of 16 characters.

The characters belong to the set defined for the "visible string" type of the abstract syntax notation ASN1 (ISO 8824).

Within that set, the following characters are to be used:

Upper case letters (A..Z)

Lower case letters (a..z)

Digits (0..9)

Underscore (_)

Currency symbol (\$)

The "space" character is not to be used.

Upper and lower case letters are considered as different characters.

An A_Name shall not begin with a digit.

The "underscore" character as a second character defines a standard-reserved name.

12.2.1.3 Variable access sub-ASE

12.2.1.3.1 Concepts

12.2.1.3.1.1 MPS environment variables

An MPS environment variable is an abstraction in the MPS environment, in which it is referenced by a unique name or description. The data associated with this variable corresponds to the representation handled by the data link services, which is exchanged on the network.

12.2.1.3.1.2 Network memory

Control system devices provide an AP with a value of each variable concerned an image of which is stored in local network memory.

12.2.1.3.1.3 Producer/consumer/third party concepts

Application entities that recognise a variable are provided with a local image of it.

Within a distributed application, one unique application entity is declared as the producer of the variable value whereas one or several application entities are declared as consumers of that value.

Moreover, some application entities, while being neither producer, nor consumer of the variable value, may have a knowledge of it in order to invoke the updating of its value. Such application entities are called third party entities.

12.2.1.3.1.4 Periodicity and aperiodicity concepts

The exchange of variable value between the producing entity and the consuming entities can be performed periodically or aperiodically.

A periodic updating is initiated by the network and is defined during the configuration step.

The updating period is determined by:

- The user requirements to fulfil the distributed application ,
- constraints related to the network.

Aperiodic updating is initiated by a user, such as the producing entity, a consuming entity, or a third party entity.

12.2.1.3.1.5 Synchronous and asynchronous properties

Application entities handling variables elaborated according to production validity, for the produced variables, and transmission validity for the consumed variables.

Definitions:

Production or transmission validity associated with each variable have an asynchronous character as they are elaborated independently from the network activity.

Production or transmission validity associated with each variable have a synchronous character when they involve synchronization orders issued from the network.

A synchronization order corresponds to the event associated with the emission or the reception of a variable of class SYNCHRONIZATION.

12.2.1.3.2 Variable class specification

12.2.1.3.2.1 Variable class modelisation

A variable in the MPS environment is modelled using the following objects (see Figure 95):

- one *Produced Variable Local Image* object that includes all the attributes necessary to a producer user.
- one or several *Consumed Variable Local Image* object that includes all the attributes necessary to one or several Consumer users.

Those two object types are issued from inheritance from several classes:

The *Produced Variable Local Image* objects are instantiated from the *Produced Variable* class.

The *Produced Variable Class* includes all the attributes proper to a producing entity, and inherits from the *Identified Variable* class. That latter class includes all the attributes that are common to every local image associated with that variable.

Moreover, the *Produced Variable* class optionally inherits from two classes *Refreshment* and *Punctual Refreshment* that include all the attributes necessary to the elaboration of informations relative to the production validity associated with that variable, within the producing application entity.

The Consumed Variable Local Image objects are instantiated from the Consumed Variable class.

The *Consumed Variable* class includes all the variable attributes proper to a consuming entity, and inherits from the *Identified Variable* class. That latter class includes all the attributes common to every local image associated with that variable.

Moreover, the *Consumed Variable* class optionally inherits from two classes *Promptness* and *Punctual Promptness* that includes the attributes necessary to the elaboration of the informations relative to the transmission validity associated with the variable within the consuming application entities.

Each of the classes *Produced Variable* and *Consumed Variable* optionally inherits respectively from a class *Production Resynchronization* and Consumption Resynchronization that include the attributes relative to the double memorisation of a variable in order to allow the providing of the variable:

- to the network by a producing entity,
- To the consuming entities by the network, upon a synchronization order issued from the network.

All the common attributes of a variable are described in a *Variable* generic class.

The variable type is described in a *Type Constructor* class which is referred by the variable and that allow the elaboration of all the type that may exist in the MPS environment.

Concerning the access to a variable, the *Variable Access* class refers to a variable and indicates the access mode that is used.

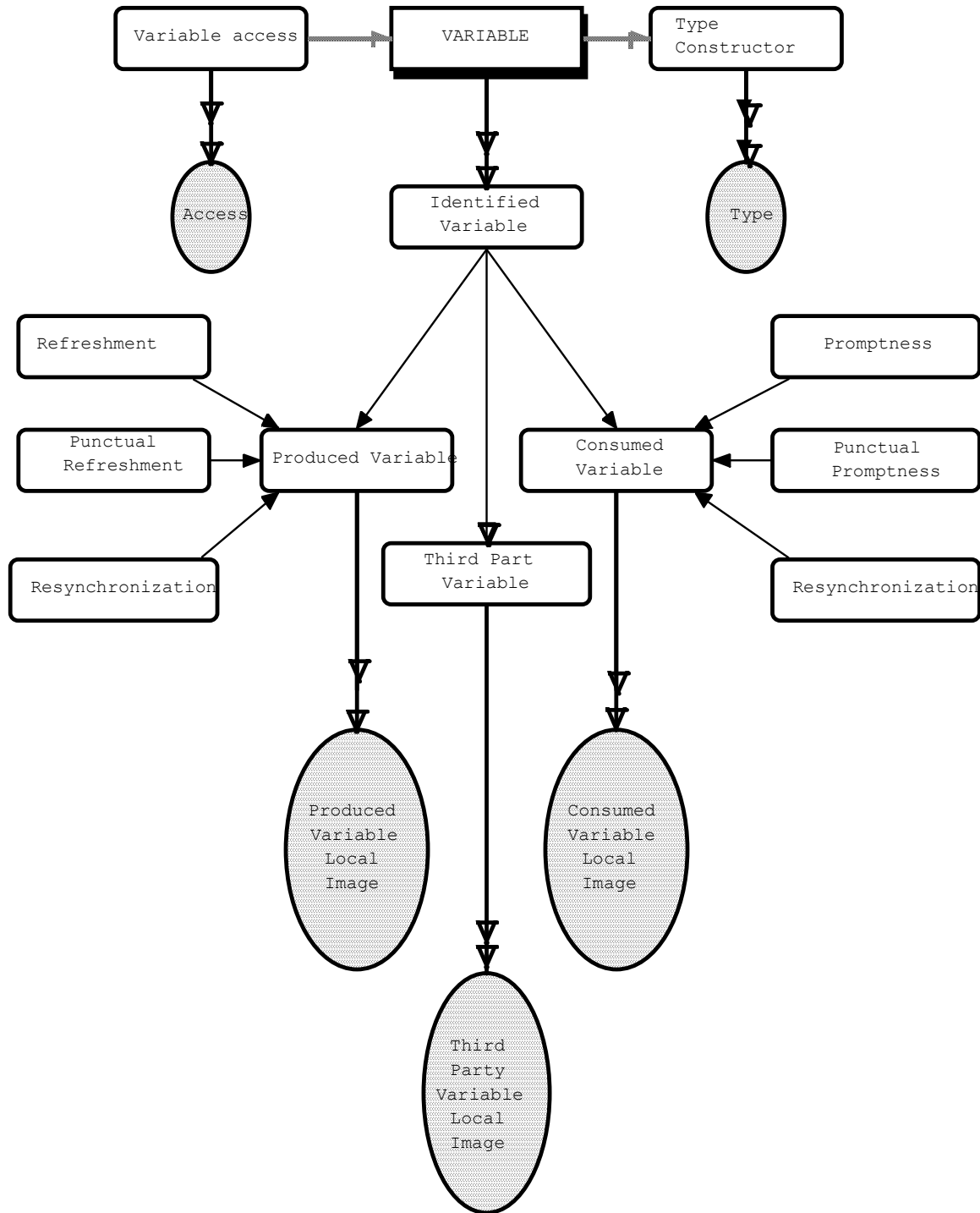


Figure 95 – Object model of the MPS ASE

12.2.1.3.2.2 Variable generic class specification

12.2.1.3.2.2.1 Variable generic class model

FAL ASE:	MPS ASE
CLASS: Variable	
CLASS ID:	not used
PARENT CLASS:	not used
Generic Class:	VARIABLE

Class Attributes:

<i>Key-Attribute</i>	: A_Name
<i>Attribute</i>	: Reference Type Constructor
<i>Attribute</i>	: Transmitted status [TRUE, FALSE]
<i>Constraint</i>	: Transmitted status = TRUE
<i>Attribute</i>	: Significant Status
<i>Attribute</i>	: Identifier
<i>Attribute</i>	: Transmission mode [PERIODIC,APERIODIC]]
<i>Constraint</i>	: Transmission mode = PERIODIC
<i>Attribute</i>	: Network period
<i>Attribute</i>	: Class [NORMAL,SYNCHRONIZATION,DESCRIPTION]
<i>Constraint</i>	: Class = SYNCHRONIZATION
<i>Attribute</i>	: Consistency variable [TRUE, FALSE]
<i>Constraint</i>	: Consistency variable = TRUE
<i>Attribute</i>	: <i>Reference Variable List</i>

Instance Attributes:

<i>Attribute</i>	: Public value
<i>Constraint</i>	: Transmitted status = TRUE
<i>Attribute</i>	: Transmitted status value
<i>Attribute</i>	: Universal services requested [TRUE, FALSE]
<i>Constraint</i>	: Universal services requested = TRUE
<i>Attribute</i>	: Universal services scope [LOCAL,DISTANT]
<i>Constraint</i>	: Universal services scope = DISTANT
<i>Attribute</i>	: Priority [NORMAL,URGENT]

12.2.1.3.2.2.2 Attributes**A_Name:**

This attribute uniquely identifies, a variable at the interface between the application layer and the user. The A_Name of a variable belongs to the MPS addressing space.

Moreover, for a variable for which the Class attribute has the value NORMAL, or SYNCHRONIZATION the A_Name length is limited to 14 characters.

Reference Type Constructor:

This reference indicates the A_Name of a global type associated with the variable.

The type is described by a *Type Constructor* object that supports any variable type that may exist in MPS environment.

Transmitted status:

When this boolean attribute has the value TRUE ,status relative to the variable production validity are broadcast to the respective consumers.

Significant status:

For the variables with information relative to their production validity, it is necessary to specify which status are effectively elaborated by the producer.

This attribute is of type Boolean array , each TRUE element indicates, that the production validity status of the associated the variable is significant.

The associated production validity status is defined for the *Transmitted status Value* array element with the same index

The various elements are:

S1: Elaboration by the producer of a synchronous or asynchronous refreshment status.

S2: Elaboration by the producer of a punctual refreshment status.

Identifier:

This attribute corresponds to a data link address that refers a data link layer object.

There is a one to one correspondence between a variable A_Name and the associated identifier. Moreover, the relationship between an identifier and a variable A_Name is a local issue committed during the network configuration generation step.

Transmission mode:

This attribute defines the exchanges of the value of the variable it is associated with.

The exchanges may be PERIODIC or APERIODIC, depending on the user requirements, and are defined during the network configuration step.

Network period:

When the transmission mode is PERIODIC, this attribute specifies the updating period of the variable by the network.

This period is expressed in milliseconds.

Class:

This attribute permits to associate a functionality to a variable. It may have one of the following values:

NORMAL: NORMAL class variables indicate process representation information.

SYNCHRONIZATION: SYNCHRONIZATION class variable indicates:

- Synchronization information of the distributed application,
- information necessary to the spatial consistency mechanism for a variable list.

DESCRIPTION: DESCRIPTION class variables features the description of all or part of the attributes relative to a variable , to a variable access, to a variable type, or to a variable list.

Consistency variable:

When this attribute has the value TRUE , the described variable corresponds to a consistency variable involved with the spatial consistency mechanism of a variable list.

Reference Variable list:

This reference to the *Variable List* generic class indicates, for a CONSISTENCY class variable, the variables the reception of which, within an application entity, takes part in the elaboration of the consistency variable value when a spatial consistency mechanism is involved

Public value:

This attribute corresponds to the value of the variable provided to the consumers by the producer.

After instantiation, that attributes includes:

- the public value provided to the network
- the last public value received from the network-

regarding that the instance corresponds to a producer's one or a consumer's one.

At a given time, it is to be considered the value available at the consumer, the value transmitted during the last exchange, and the last value received by each consumer.

The last value received by a consumer is not always identical to the last transmitted value because of possible transmission faults.

Transmitted status value:

This attribute corresponds to the array of production validity status , relative to a variable, and transmitted with that variable value. Each status within the array is a boolean.

All the production validity status associated with the variable value are not necessarily significant.

As for the variable public value, that attribute instantiation includes:

- the status values provided to the network,
- the last status values received from the network

regarding that the instance corresponds to a producer's one or a consumer's one.

The handling at a producer and at the consumers of the set consisting of the variable value and its associated status values is atomic.

Status array elements:

S1 Synchronous/asynchronous refreshment status

S2 Punctual refreshment status

NOTE The abstract syntax defines the array of status.

Requested universal services:

When this boolean attribute has the value TRUE , universal services may be invoked for the variable. in consideration.

Universal services scope:

This attribute is used by the variable access extended services, and defines the service category -local or distant- to use for the universal read/write services.

When that attribute has the value LOCAL , universal services that are used correspond to local services; when it has the value DISTANT , they correspond to distant services.

Priority:

This attribute is used by the variable access extended services and defines the priority level used to request variable updating when the universal services correspond to distant services.

When that attribute has the URGENT value, flow control guarantees that an urgent updating request will be processed before a normal request initiated after it.

12.2.1.3.2.3 *Identified Variable* class specification

The *Identified Variable* class is issued from the instantiation of the *Variable* generic class.

12.2.1.3.2.4 *Produced Variable* class specification

12.2.1.3.2.4.1 *Produced variable* class model

This class gathers all the attributes relative to a variable within a producer application layer.

FAL ASE:	MPS ASE
CLASS:	Produced variable
CLASS ID:	not used
PARENT CLASS:	not used
<i>Class:</i> PRODUCED VARIABLE	
<i>Attribute</i>	: <i>Inherit from</i> Identified Variable
<i>Attribute</i>	: Refreshment elaborated [TRUE, FALSE]
<i>Constraint</i>	: Refreshment elaborated = TRUE
<i>Attribute</i>	: <i>Inherit from</i> Refreshment
<i>Attribute</i>	: Punctual refreshment elaborated [TRUE, FALSE]
<i>Constraint</i>	: Punctual refreshment elaborated = TRUE,
<i>Attribute</i>	: <i>Inherit from</i> Punctual Refreshment
<i>Attribute</i>	: Resynchronized Variable [TRUE,FALSE]
<i>Constraint</i>	: Resynchronized Variable = TRUE
<i>Attribute</i>	: <i>Inherit from</i> Production Resynchronization
<i>Attribute</i>	: Emission Indication[TRUE,FALSE]

12.2.1.3.2.4.2 *Attributes*

Inherit from Identified Variable:

Produced Variable class inherits from *Identified Variable* class that provides it with the attributes common to all local copies of the same variable within MPS environment.

Refreshment elaborated:

Refreshment is an information relative to a variable production validity.

This information, which is optionally elaborated, tells the consumer users about the production period of the producer user.

The status associated with this information is elaborated within the producer application layer and provided to the network in the status value associated with the variable value.

When this Boolean attribute has the value TRUE , a refreshment status is elaborated within the producer application entity.

Inherit from Refreshment:

The *Produced Variable* class inherits from the *Refreshment* class.

The *Refreshment* class includes all the attributes relatives to the elaboration of the refreshment information associated with a variable, in order to qualify its production.

Punctual Refreshment elaborated:

Punctual refreshment is an information relative to a variable production validity.

This information, that is optionally elaborated, tells consumer users about the respect by the producer user of a writing operation within a time slot associated with a synchronization order issued from the network.

The status associated with that information is elaborated within the producer application layer and provided to the network in the status value associated with the variable value.

When this boolean attribute has the value TRUE , a punctual refreshment status is elaborated within the producer application entity.

Inherit from Punctual Refreshment:

The Produced Variable class inherits from the Punctual Refreshment class.

The Punctual Refreshment class includes all the attributes relative to the elaboration of the punctual refreshment information associated with the variable, in order to qualify its production.

Resynchronized Variable:

Some users produce variables asynchronously with respect to the network.

These variables can be resynchronized within the application layer by the means of a double buffer.

The private buffer is the one provided to the asynchronous producer user. The public buffer is the one provided to the network.

Resynchronization of such a variable consists of transferring from Private Buffer to Public Buffer upon the reception of a synchronization order from the network.

This attribute supports the chosen option for the variable concerned.

When this boolean attribute has the value TRUE , the concerned variable is resynchronized in production.

Inherit from Production Resynchronization:

The Produced Variable class inherits from the Production Resynchronization class.

The Production Resynchronization class includes all the attributes relative to a variable resynchronization.

Emission Indication:

When it has the value TRUE this boolean attribute indicates that an emission indication (A_SENT.ind) has to be returned to the user once the variable has been sent on the network.

12.2.1.3.2.5 Produced variable local image class specification

Produced Variable Local Image object is issued from the instantiation of the *Produced Variable Class*.

12.2.1.3.2.6 Consumed variable class specification

12.2.1.3.2.6.1 Consumed variable class model

This class gathers all the attributes relative to a variable within a consumer application layer.

FAL ASE:	MPS ASE
CLASS:	Consumed variable
CLASS ID:	not used
PARENT CLASS:	not used
<i>Class: CONSUMED VARIABLE</i>	
<i>Attribute</i>	: <i>Inherit from Identified Variable</i>
<i>Attribute</i>	: Promptness elaborated [TRUE, FALSE]
<i>Constraint</i>	: Promptness elaborated = TRUE
<i>Attribute: Inherit from Promptness</i>	
<i>Attribute</i>	: Punctual Promptness elaborated [TRUE, FALSE]
<i>Constraint</i>	: Punctual Promptness elaborated = TRUE
<i>Attribute: Inherit from Punctual Promptness</i>	
<i>Attribute</i>	: Resynchronized Variable [TRUE, FALSE]
<i>Constraint</i>	: Resynchronized Variable = TRUE
<i>Attribute: Inherit from Consumption resynchronization</i>	
<i>Attribute</i>	: Reception Indication [TRUE, FALSE]

12.2.1.3.2.6.2 Attributes

Inherit from Identified Variable:

Consumed Variable class inherits from *Identified Class* that provides it with all the attributes common to every local copy of the same Identified Variable within MPS environment.

Promptness elaborated:

Promptness is an information relative to a variable transmission validity.

This information, that is optionally elaborated, tells a consumer user about the reception of a variable respecting a consumption period

The status associated with this information is elaborated within the consumer application layer and provided to the consumer user.

When this boolean attribute has the value TRUE , a promptness status is elaborated within the consuming application entity.

Inherit from Promptness:

Consumed Variable class inherits from Promptness class.

Promptness class includes all the attributes relative to the elaboration of a promptness status associated with a consumed variable.

Punctual Promptness elaborated:

Punctual promptness is an information relative to a variable transmission validity.

This information, that is optionally elaborated, tells a consumer user about the reception of a variable within a time slot associated with a synchronization order issued from the network.

The status associated with this information is elaborated in the application layer and provided to the consumer user.

When this boolean attribute has the value TRUE , a punctual promptness status is elaborated within the consuming application entity.

Inherit from Punctual Promptness:

Consumed Variable class inherits from the Punctual Promptness class.

Punctual Promptness class includes all the attributes relative to the elaboration at the consumer level of a punctual promptness status associated with a variable.

Resynchronized Variable:

Some users consume variables asynchronously with respect to the network.

These variables can be resynchronized within the application layer by the means of a double buffer.

The private buffer is the one provided to the asynchronous consuming user. The public buffer is the one receiving the value from the network.

Resynchronization of such a variable consists in transferring the public buffer into the private buffer upon a synchronization order from the network.

This attribute supports the chosen option for the variable concerned in consumption resynchronization.

When this attribute has the value TRUE , the associated variable is resynchronized in consumption.

Inherit from Consumption Resynchronization:

Consumed Variable class inherits from the Consumption resynchronization.

Consumption Resynchronization includes all the attributes relative to a variable resynchronization in consumption.

Reception Indication:

When it has the value TRUE this boolean attribute indicates that a reception_indication (A_RECEIVED.ind) has to be returned to the user once the variable has been received from the network.

12.2.1.3.2.7 Consumed variable local image class specification

Consumed Variable Local Image object is issued from the instantiation of *Consumed Variable* class.

12.2.1.3.2.8 Third party variable class specification

12.2.1.3.2.8.1 Third party variable class model

FAL ASE: **MPS ASE**

CLASS: Third party variable

CLASS ID: not used

PARENT CLASS: not used

Class: THIRD PARTY VARIABLE

Attribute : Inherit from IdentifiedVariable

12.2.1.3.2.8.2 Attributes

Inherit from IdentifiedVariable

Third Party Variable class inherits from *Identified Variable* class this provides it with the attributes necessary to know the variable.

Consequently, this variable is only provided with the pair (A_Name, Identifier) of the identified variable it inherits from.

12.2.1.3.2.9 Third party variable local image object class specification

Third Party Variable Local Image object is issued from instantiation of Third Party Variable class.

12.2.1.3.2.10 Type Constructor class specification

12.2.1.3.2.10.1 Type constructor class model

FAL ASE:	MPS ASE
CLASS:	Type constructor
CLASS ID:	not used
PARENT CLASS:	not used
<i>Class</i>	: TYPE CONSTRUCTOR
<i>Key-Attribute</i>	: A_Name
<i>Attribute</i>	: Construction
	[SIMPLE, ARRAY, STRUCTURE, PREDEFINED, EXPLICIT]
<i>Constraint</i>	: Construction = SIMPLE
<i>Attribute</i>	: Primitive type
<i>Attribute</i>	: Size
<i>Constraint</i>	: Construction = ARRAY
<i>Attribute</i>	: Compressed [TRUE, FALSE]
<i>Attribute</i>	: Dimension
<i>Attribute</i>	: <i>Reference</i> Type Constructor
<i>Constraint</i>	: Construction = STRUCTURE
<i>Attribute</i>	: List of
<i>Attribute</i>	: Named field [TRUE, FALSE]
<i>Constraint</i>	: Named field = TRUE
<i>Attribute</i>	: field name
<i>Attribute</i>	: <i>Reference</i> Type Constructor

12.2.1.3.2.10.2 Attributes

A-Name:

This attribute uniquely identifies an object within the application layer. An_A_Name belongs to the MPS addressing space Moreover, a Type Constructor A_Name shall no be longer than 14 characters.

An A_Name beginning with the characters K_ indicates a type reserved by the standard.

Construction:

The specified type can be of SIMPLE construction, and is then directly derived from the primitive types.

The specified type can be of ARRAY construction, and represents an ordered set of elements of the same type; that latter type can be itself of construction SIMPLE, ARRAY, or STRUCTURE. ARRAY construction type elements are increasingly numbered from zero (0) for the first element.

The specified type can be of STRUCTURE construction, and represents a complex type composed of an ordered set of one or several components. These components may have different types of SIMPLE, ARRAY, or STRUCTURE construction.

The specified type can be of PREDEFINED construction, and is then defined within the standard.

PREDEFINED construction types cannot be used to compose types of ARRAY or STRUCTURE.

The specified type can be of EXPLICIT construction in order to characterise a variable the type of which can change dynamically.

EXPLICIT construction types cannot be used to compose types of ARRAY or STRUCTURE construction.

SIMPLE construction types characteristics

Primitive Type:

Primitive types used.

Size:

This attribute has a value and purpose that depends on the value of the Primitive Type attribute defined above. The size, described for each of the primitive types, for Type Constructor class instances, describes a maximum value to the variable.

ARRAY construction types characteristics

Compressed:

The application layer transfer syntax allows to define, for some ARRAY type variables, compressed encoding rules.

Compressed encoding is specified using a boolean attribute.

When this attribute has the value TRUE, the associated variable value encoding is compressed, each time it is possible depending on the array elements type; when this attribute has the Value FALSE, the associated variable value encoding is not compressed.

Dimension:

This attribute defines the number of elements within the ARRAY construction type.

Reference Type Constructor:

An ARRAY construction type is composed of elements of any one type either of construction SIMPLE, or ARRAY, or STRUCTURE.

This attribute references the "Type" object, instantiated from the *Type Constructor* class associated with the array elements

STRUCTURE construction types characteristics**Named field:**

When a type is **STRUCTURE**, each component composing the type corresponds to the semantic representation of the associated variable. In order to permit a partial access on these components they may be designated with a name. When this attribute has the value **TRUE**, a name is associated with the corresponding component; when it has the value **FALSE**, no name is associated with the component.

Field name:

This attribute is a character string that uniquely identifies a structured variable component.

NOTE The scope of component names is local to a structure.

Reference Type Constructor:

A **STRUCTURE** construction type is composed of components of any type. i.e. **SIMPLE**, **ARRAY**, or **STRUCTURE**.

This attribute denotes that the instance of *Type Constructor* class associated with each component is of type **STRUCTURE**.

PREDEFINED construction types characteristics

PREDEFINED construction types are mostly used for description variables and refer to types that are reserved by the standard.

PREDEFINED construction types cannot be explicitly defined because they need a grammatical extension in order to use optional types, alternative types, or variable length types.

NOTE Grammatical extensions that are involved are specified in ISO 8824 standard.

EXPLICIT construction types characteristics

EXPLICIT construction types allow the associated variables to be provided with an explicit transfer syntax, in order to carry the type semantics with the data.

12.2.1.3.2.10.3 Primitive types

Primitive types that are selected, and their characteristics are mostly specified in the ISO 8824 standard.

NOTE The only access mode available for a primitive type variable is global access.

• BOOLEAN

The definition of this primitive type is the same as the boolean type definition in the ISO 8824 standard. For this type, the *Size* attribute shall be omitted.

• BIT STRING

The definition of this primitive type is the same as the bit string definition in ISO 8824. For this type the *Size* attribute defines the number of bits in the bit string.

• INTEGER

The definition of this primitive type is the same as the integer type definition in ISO 8824 standard. For this type, the *Size* attribute defines the number of bits that are necessary to have a twos complement representation of all possible values.

• UNSIGNED

The definition of this primitive type is the same as the integer type definition in ISO 8824 with the exclusion of negative numbers. For this type, the *Size* attribute defines the number of bits that are necessary to have a twos complement representation of all possible values.

• OCTET STRING

The definition of this primitive type is the same as the octet string type definition in ISO 8824 standard. For this type, the *Size* attribute defines the number of octets in the string.

• VISIBLE STRING

The definition of this primitive type is the same as the visible string type definition in ISO 8824 standard. For this type, the *Size* attribute defines the number of characters in the string.

• GENERALISED TIME

This primitive type is defined with a fourteen-digits visible string (YYYYMMDDHHMMSS); it is equivalent to a particular form of the generalised time type in ISO 8824 standard. For this type, the *Size* attribute shall be omitted.

• FLOATING POINT

This primitive type defines value representation for positive and negative real numbers, including, zero, negative infinite and positive infinite.

The values at the limits (0, $+\infty$, $-\infty$), as well as the various representation formats are defined in ANSI/IEEE 754 standard.

For this type, the *Size* attribute, of boolean type, indicates whether the representation format is simple precision (FALSE) or double precision (TRUE).

• BINARY TIME

This primitive type is defined as the unsigned type of the present standard, the value of which corresponds to a number of elementary times.

For this type, each value of the *Size* attribute defines the value of an elementary time as well as a number of bits used to represent any possible binary time value, as shown in Table 456.

Table 456 – Binary time coding

Size	10 ms	0,1 ms	1 ms	10 ms
16 Bits	0	1	2	3
32 Bits	4	5	6	7
48 Bits	8	9	—	—

• BCD

This type is used to represent a set of values corresponding to the numeric representation of digits.

For this type, the *Size attribute* shall be omitted.

12.2.1.3.3 Basic mechanisms

12.2.1.3.3.1 Time-out class specification

12.2.1.3.3.1.1 Time-out class model

The elaboration, by producing application entities, of information relative to production validity involves time-out resources. Likewise for consuming application entities.

The description of this resource can be modelled with an attribute object, whose purpose is to define visible aspects of time-out used by MPS environment services.

FAL ASE:		MPS ASE
CLASS:	Time-out	
CLASS ID:		not used
PARENT CLASS:		not used
Class:	TIME-OUT	
	<i>Attribute</i>	: Preset
	<i>Attribute</i>	: Current value
	<i>Attribute</i>	: State [RUNNING, IDLE]

12.2.1.3.3.1.2 Attributes

Preset:

Preset corresponds to the duration associated with the time-out when it is armed, whenever the time-out is running or expired.

Current value:

Current value corresponds to the instantaneous value of the time-out.

That value is equal to:

- the period remaining before expiration, when the time-out is running
- "0" when the time-out has expired.

State:

The time-out state indicates whether the time-out is active or not.

The two states are as follows:

RUNNING:Corresponds to a running time-out within the time interval between preset value and expiration.

IDLE: Corresponds to an expired time-out.

NOTE As soon as the time-out current value is forced with a non-zero value, the time-out is automatically set to **RUNNING** state.

12.2.1.3.3.1.3 Time-out mechanism

The dynamic working of a time-out is defined by the evaluation net shown in Figure 96.

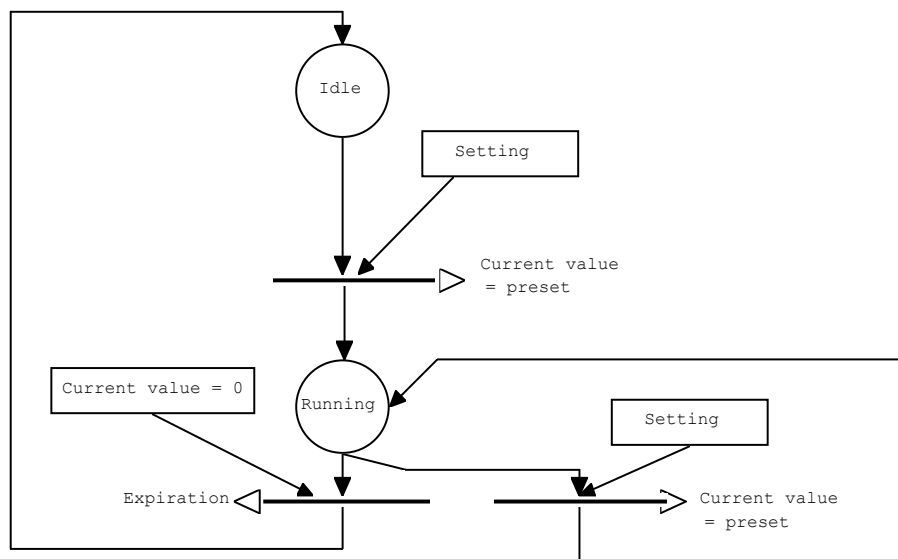


Figure 96 – Time-out evaluation net

12.2.1.3.3.2 Variable addressing

12.2.1.3.3.2.1 Variable addressing description

User access to application variables in MPS environment can be carried out in various ways.

Access to a variable is carried out by means of:

- a variable A_Name, in the case of variable global access,
- an access path in the case of explicit partial access to a structured variable named field or to an array element,
- an access A_Name, in the case of renamed partial access to a variable or to one of its components (Structure field, array element...).

Access A_Name belong all to the same MPS addressing space provided to the user at the interface with the application layer.

12.2.1.3.3.2.2 Access modes

12.2.1.3.3.2.2.1 Global access

This access mode allows the user to globally access a variable.

The access to the variable is made with the Application name of the variable that is accessed. In this case, the application name corresponds to the A_Name attribute of *Produced Variable Local Image*, or *Consumed Variable Local Image*, or *Third Party Variable Local Image*. It belongs to the MPS addressing space provided to the user at the interface between the application layer and the user.

12.2.1.3.3.2.2.2 Explicit partial access

This access mode allows the user the access to a named field of a structured type variable or to an element of an array type variable.

The explicit partial access is made with an access path corresponding to the enumeration of all successive fields in the structure tree or array elements index; This enumeration shall begin from the variable root, to the component that is accessed.

12.2.1.3.3.2.3 Renamed access

Renamed access consists in the projection of an A_Name from the MPS addressing space on a variable A_Name, or on a structured type variable access path, or on an array type variable element.

The names that are used in renamed partial access belong to the same addressing space as variable global names; this means they belong to the addressing space provided to the user at the interface with the application layer.

Consequently, renamed access allows the user access to a structure field or to an array element in the same way as for a global variable.

NOTE This access mode is provided to improve transmission efficiency, by grouping several user defined variables in the same structure.

This means that the transmission efficiency improvement can lead to group in a minimum number of DLPDUs variables exchanged between APs in an elementary distributed application. This is done in order to minimise control data in the data link layer.

Optimisation structures created at the application layer are transparent to the user with the renamed access, but they have to be declared in all the application entities that have to access all or part of the user defined variables they represent.

NOTE on optimisation structure elaboration: Only variables which do not need production validity and for which transmission validity is elaborated with the same value of the consumption period can be grouped within the same optimisation structure.

12.2.1.3.3.2.3 Variable renamed access modelization

12.2.1.3.3.2.3.1 Variable Access class specification

Class: VARIABLE ACCESS

Key-Attribute	: A_Name
Attribute	: Reference Variable
Attribute	: Access Mode [PARTIAL, GLOBAL]
Constraint	: Access Mode = PARTIAL
Attribute	: Access Path

A_Name:

This attribute allows a unique specification of an access name at the interface between the application layer and the user. The A_Name belongs to the MPS addressing space

Moreover, a variable access A_Name shall not be longer than 14 characters.

Reference Variable:

The *Variable Access* class refers to the *Variable* class.

Variable class indicates the application variable with which the access A_Name is associated.

NOTE A variable renamed access name is global, this means that the same access name within several different application entities shall refer to the same application layer variable.

Access Mode:

This attribute indicates which access mode is to be used.

When it has the value TRUE, this attribute indicates a global renamed access to an application layer variable.

When it has the Value FALSE, it indicates a partial access to a field of a structured variable or to an array element.

Access Path:

The access path to a structured variable field or to an array element refers, by increasing depth order structure field names or array element index.

NOTE In order to reach a full specification of the access path to a variable component, it is necessary that all the *Named Field* attributes of the *Type Constructor* objects at the various level of the structure have the value TRUE.

12.2.1.3.3.3 Variable transmission validity specification

12.2.1.3.3.3.1 Variable transmission overview

Consumer users may be provided with information relative to transmission validity for a variable.

Two sets of transmission validity information are defined:

- Promptness; relative to transmission validity regarding a user validity delay for the variable that is received: this validity delay is called the consumption period. Semantics associated with this information depends on the synchronous or asynchronous nature of the promptness
- Punctual promptness; that is relative to the transmission validity regarding a time slot associated with a synchronization order issued from the network.

12.2.1.3.3.3.2 Promptness

12.2.1.3.3.3.2.1 Promptness definitons

Promptness attributes associated with variable are optional.

Promptness is relative to the validity of availability of a variable value provided to the user by the network. This validity is elaborated with a maximum consumption period which is fixed by the user.

In the case of synchronous promptness, the semantics of that validity information takes into account the respect of availability by the network of a synchronization order associated with the variable.

Asynchronous Promptness:

When it has the value TRUE, this boolean information indicates in a consumer entity that the variable value has been updated by the network for a delay that is smaller than the variable consumption period.

Synchronous Promptness:

When it has the value TRUE, this boolean information indicates in a consumer entity that the synchronization order has been received and has been followed by (at least) one updating of the associated variable by the network, and that the updating was within the consumption period.

12.2.1.3.3.3.2.2 Promptness class specification

This class gathers all the attributes relative to the elaboration of a promptness information associated with a variable within a consuming application entity.

Class: PROMPTNESS

<i>Attribute</i>	: Consumption period
<i>Attribute</i>	: <i>Inherit from</i> Time-out
<i>Attribute</i>	: Promptness characteristic [SYNCHRONOUS, ASYNCHRONOUS]
<i>Constraint</i>	: Promptness characteristic = SYNCHRONOUS
<i>Attribute</i>	: <i>Reference</i> (Synchronization) Variable
<i>Attribute</i>	: Promptness status [TRUE,FALSE]

Consumption period:

The consumer user declares a consumption period corresponding with the needs_of the distributed application.

This period corresponds to the minimal period to which the consumer user will access the variable.

Inherit from Time-out:

Promptness class inherits from Time-out class.

Time-out class includes all the attributes relative to a time-out. This resource is mandatory to elaborate an information relative to a variable promptness.

Promptness characteristics:

Promptness may have a characteristic which is proper to the consumer, and that influences the promptness status semantics.

When the characteristic is ASYNCHRONOUS, promptness is elaborated exclusively with attributes proper to the variable.

When the characteristic is SYNCHRONOUS, promptness takes into account not only the variable attributes, but also an event related to the reception of a synchronization order.

Reference (Synchronization) Variable:

This attribute references a *Variable* class for which the *Class* attribute has the value SYNCHRONIZATION. This allows reference to the synchronization order which sets the time-out in the case of synchronous promptness.

Promptness status:

Variable promptness is characterized by a status.

This status is of type boolean and is locally elaborated within each entity having a *Consumed Variable Local Image* associated with the variable.

Promptness status transition conditions from one state to the other are elaborated by the means of:

- Events corresponding to the variable reception or to the occurrence of the synchronization order associated with the variable,
- The expiration of the time-out associated with the variable,
- The state of the time-out associated with the variable for the processing of synchronous promptness.

12.2.1.3.3.2.3 Mechanism

The asynchronous promptness status associated with a variable within a consuming entity is dynamically processed by the mechanism described in Figure 97 and Table 457.

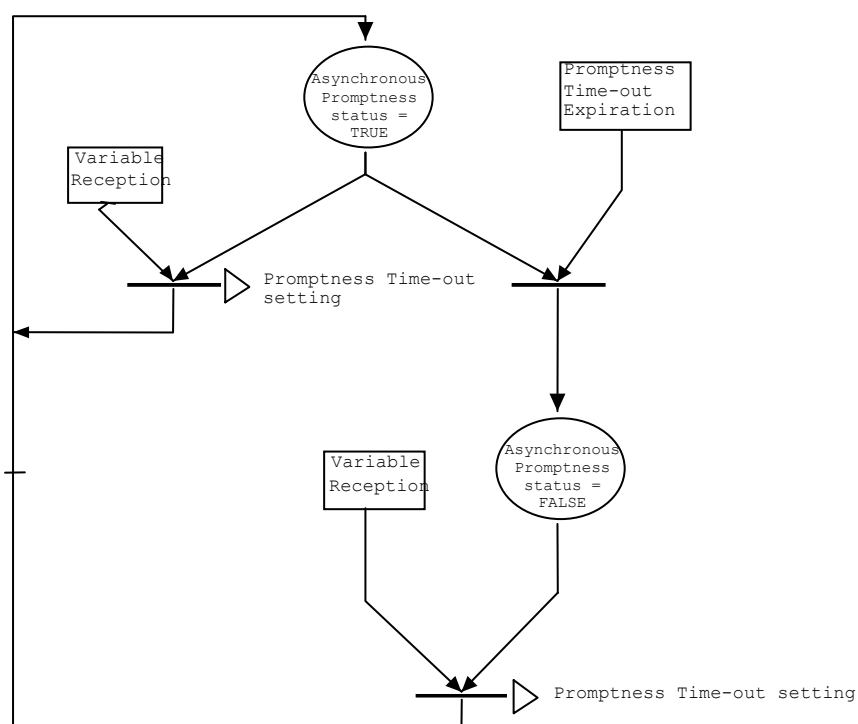


Figure 97 – Asynchronous promptness status evaluation net

Table 457 – Asynchronous promptness events and actions

Initial state: Asynchronous Promptness status = FALSE

Promptness time-out state = IDLE

Requests: Variable Reception:

Variable available from the network

Promptness time-out expiration:

Transition of the time-out associated with the mechanism elaborating the asynchronous promptness status from RUNNING state to IDLE state.

Actions: Promptness time-out setting

Transition of the time-out associated with the mechanism elaborating the asynchronous promptness status from IDLE state to RUNNING state with a preset equal to the *Consumption period* attribute value.

Synchronous promptness status associated with a variable within a consuming application entity is processed by the mechanism described in Figure 98 and Table 458.

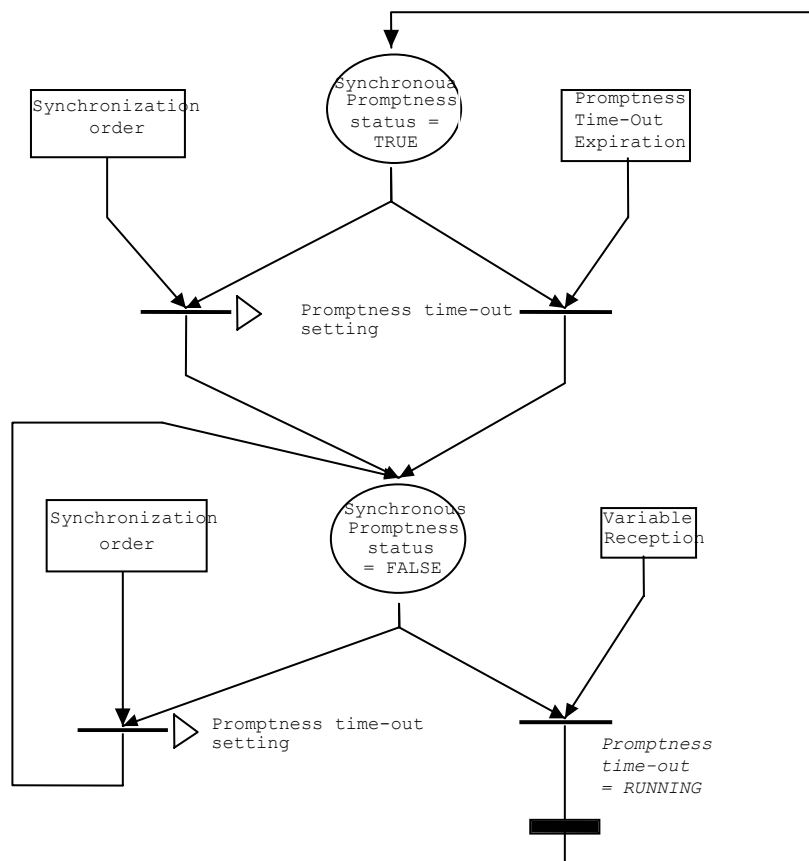


Figure 98 – Synchronous promptness status evaluation net

Table 458 – Synchronous promptness events and actions

Initial state:	<u>Synchronous promptness status = FALSE</u> Promptness time-out state = IDLE
Requests:	<u>Variable reception:</u> Variable available from the network <u>Synchronization order:</u> Occurrence of a synchronization order associated with the synchronous promptness status elaboration mechanism. <u>Promptness time-out expiration:</u> Transition of the time-out associated with the mechanism elaborating the synchronous promptness status from RUNNING state to IDLE state.
Actions:	<u>Promptness time-out setting</u> Transition of the time-out associated with the mechanism elaborating the synchronous promptness status from IDLE state to RUNNING state with a preset equal to the <i>Consumption period</i> attribute value.

12.2.1.3.3.3.3 Punctual promptness

12.2.1.3.3.3.3.1 Punctual promptness definition

Punctual promptness information associated with a variable is optional.

Punctual promptness is relative to a validity of availability of the variable value to the consumer user from the network. That validity is elaborated by the means of a consumption time-slot that is defined during the configuration step. This time-slot is activated upon reception of a synchronization order from the network.

Punctual Promptness:

When it has the value TRUE this boolean informs a consuming application entity that the variable value has been updated by the network (at least) once in the time slot following the reception of a synchronization order associated with the variable. It indicates also that the variable value has not been updated outside the time slot.

12.2.1.3.3.3.2 Punctual Promptness class specification

This class gathers all the attributes relative to the elaboration of a punctual promptness information associated with a variable within a consuming application entity.

Class: PUNCTUAL PROMPTNESS

Attribute	: Consumption time slot
Attribute	: <i>Inherit from</i> Time-out
Attribute	: <i>Reference</i> (Synchronization) Variable
Attribute	: Punctual Promptness Status [TRUE,FALSE]

Consumption time-slot:

Punctual promptness is elaborated by the means of a consumption time slot associated with a variable by a consumer user.

Punctual promptness informs a variable consumer that it has received a new value (at least) once during the time slot associated with the last synchronization order that has been received.

Inherit from Time-out:

Punctual Promptness class inherits from Time-out class.

Time-out class includes all the attributes relative to a time-out (preset, state).

Reference (Synchronization) Variable:

This attribute refers to a *Consumed Variable* class the *Class* attribute of which has the value SYNCHRONIZATION , and that defines the synchronization order that sets the time-out in the case of a punctual promptness.

Punctual promptness status:

A variable punctual promptness is characterized by a status.

That status is of type boolean and is locally elaborated within each entity where a *Consumed Variable Local Image* is declared for the variable.

Promptness status transition conditions from one state to the other are elaborated by the means of:

- Events corresponding to the reception of the variable or to the occurrence of a synchronization order associated with that variable,
- State of the time-out associated with the variable for the punctual promptness processing.

12.2.1.3.3.3.3 Mechanism

Punctual promptness status associated with a variable within a consuming application entity is processed by the mechanism described in Figure 99 and Table 459.

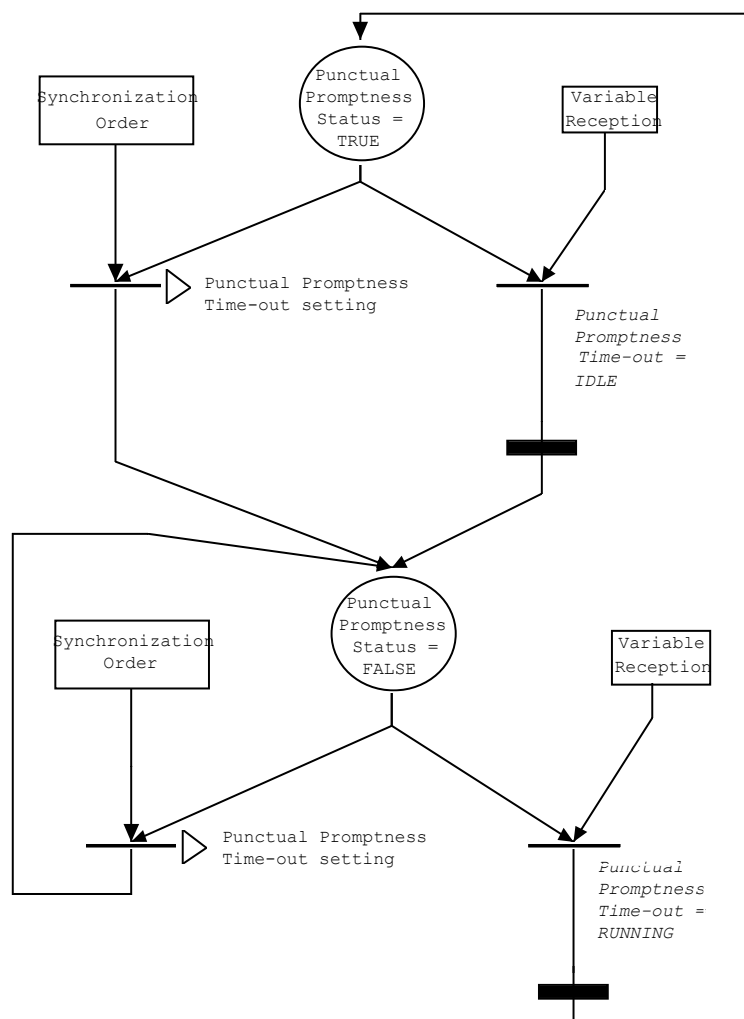


Figure 99 – Punctual promptness status evaluation net

Table 459 – Punctual promptness events and actions

Initial state:	<u>Punctual Promptness status = FALSE</u> Promptness time-out state = IDLE
Requests:	<u>Variable Reception:</u> Variable available from the network <u>Synchronization order:</u> Occurrence of the synchronization order associated with the punctual promptness elaboration mechanism.
Actions:	<u>Punctual Promptness time-out setting</u> Transition of the time-out associated with the mechanism elaborating the punctual promptness status from IDLE state to RUNNING state with a preset equal to the <i>Consumption Time slot</i> attribute value.

12.2.1.3.3.4 Variable production validity specification

12.2.1.3.3.4.1 Variable production validity definition

Production validity may be associated with any variable. Validity information is elaborated by a producing application entity and provided to the consuming entities. Those consuming entities only see a sampled value of the information.

Two categories of production validity information are defined:

- Refreshment, relative to a production validity regarding a maximum delay fixed by the user. Refreshment semantics depends on its characteristic, synchronous or asynchronous, within a producing application entity.
- Punctual refreshment relative to production validity regarding a production time slot associated with a synchronization order issued from the network.

12.2.1.3.3.4.2 Refreshment

12.2.1.3.3.4.2.1 Refreshment definition

This production validity information is elaborated as an option and is characterized with a status.

Refreshment is relative to validity of availability to the network by a producer user of a variable value. This validity is elaborated by the means of a production period that is fixed by the user.

In the case of a synchronous refreshment, the semantics of this validity information denotes the respect of availability by the network of the synchronization order associated with the variable.

That refreshment information, carried with the variable value by the network, is provided to consumer users.

Asynchronous refreshment:

When it has the value TRUE this boolean indicates in a producing application entity that the variable value has been provided to the network by the producer user, with a delay less than the production period.

Synchronous Refreshment:

When it has the value TRUE this boolean indicates within a producing application entity that a synchronization order has been received and followed by a (at least) one sourcing of a new variable value to the network from the producer user with a delay less than the production period.

12.2.1.3.3.4.2.2 Refreshment class specification

This class gathers all the attributes relative to the elaboration of a refreshment information associated with a variable within a producing application entity.

Class: REFRESHMENT

<i>Attribute</i>	: Production period
<i>Attribute</i>	: <i>Inherit from</i> Time-out
<i>Attribute</i>	: Refreshment characteristic [SYNCHRONOUS, ASYNCHRONOUS]
<i>Constraint</i>	: Refreshment characteristic = SYNCHRONOUS
<i>Attribute</i>	: <i>Reference</i> (Synchronization) Variable
<i>Attribute</i>	: Refreshment Status [TRUE, FALSE]

Production period:

The producer user may declare a production period corresponding to the needs of its distributed application.

This period corresponds to the maximum interval during which the producer user will provide the variable to the network.

Inherit from Time-out:

Refreshment class inherits from *Time-out* class.

Time-out class includes all the attributes relative to a time-out. This resource is mandatory to the elaboration of information relative to variable refreshment.

Refreshment characteristic:

Refreshment may be provided with a characteristic proper to a producer and that influences the refreshment status semantics.

When the characteristic is *ASYNCHRONOUS*, refreshment is elaborated exclusively with attributes proper to the variable.

When the characteristic is *SYNCHRONOUS*, refreshment takes into account not only the variable attributes, but an event related to the reception of a synchronization order.

Reference (Synchronization) Variable:

The attribute refers to a *Variable* class, the *Class* attribute of which has the value *SYNCHRONIZATION*. This defines the synchronization order which sets the time-out in the case of a synchronous refreshment.

Refreshment Status:

The variable refreshment provided to a user is characterized by a status.

This status is of type boolean and is locally elaborated within each application entity where a *Produced Variable Local Image* is declared.

The value of the status is associated with the variable value during the exchange on the bus.

Refreshment status is defined by two states, and the transition conditions from one state to the other are elaborated by means of:

- Events corresponding to the production of the variable,
- Events related to the occurrence of the synchronization order associated with the variable,
- Expiration of the time-out associated with the variable,
- State of the Time-out associated with the variable for the processing of synchronous refreshment.

12.2.1.3.3.4.2.3 Mechanism

Asynchronous refreshment status associated with a variable within a producing application entity is elaborated by the mechanism described in Figure 100 and Table 460.

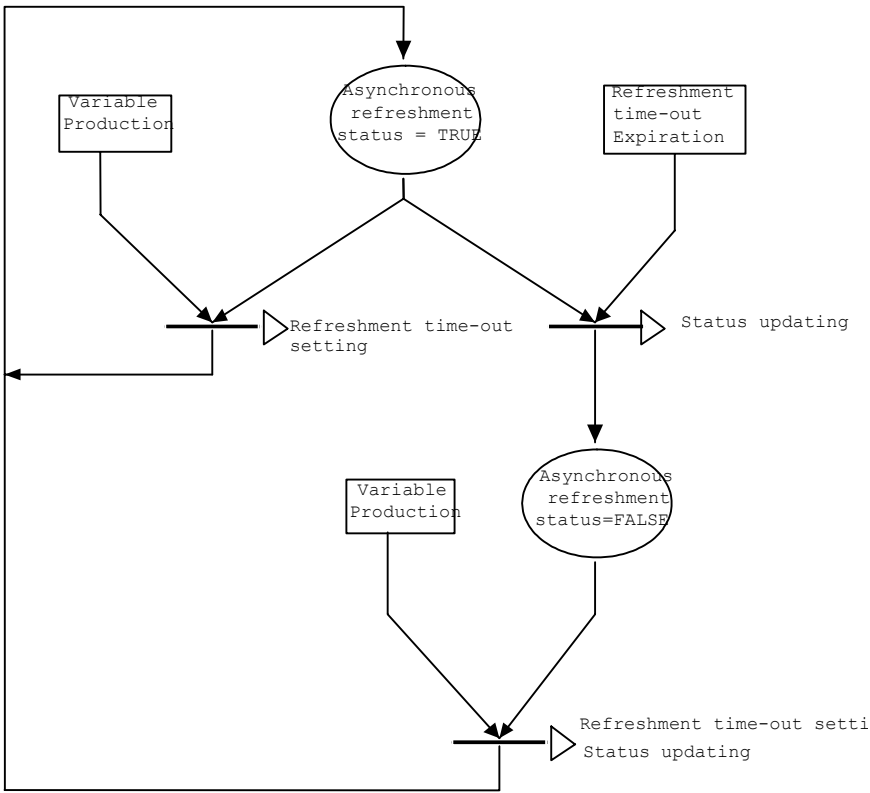


Figure 100 – Asynchronous refreshment status evaluation net

Table 460 – Asynchronous refreshment events and actions

Initial state:	Asynchronous refreshment status = FALSE
	Refreshment time-out state = IDLE
Requests:	<u>Variable Production:</u> A_WRITELOC, A_WRITEFAR or A_WRITE service invocation by the user
	<u>Refreshment time out expiration:</u> Transition of the time-out associated with the asynchronous refreshment evaluation mechanism from the state <i>RUNNING</i> to the state <i>IDLE</i>
Action:	<u>Refreshment time-out setting:</u> Transition of the time-out associated with the asynchronous refreshment mechanism from the state <i>IDLE</i> to the state <i>RUNNING</i> , with a preset equal to the <i>Production Period</i> attribute value.
	<u>Status updating:</u> Providing of the asynchronous refreshment status value to the network.

Synchronous refreshment status associated with a variable within a producing application entity is elaborated by the mechanism described in Figure 101 and Table 461.

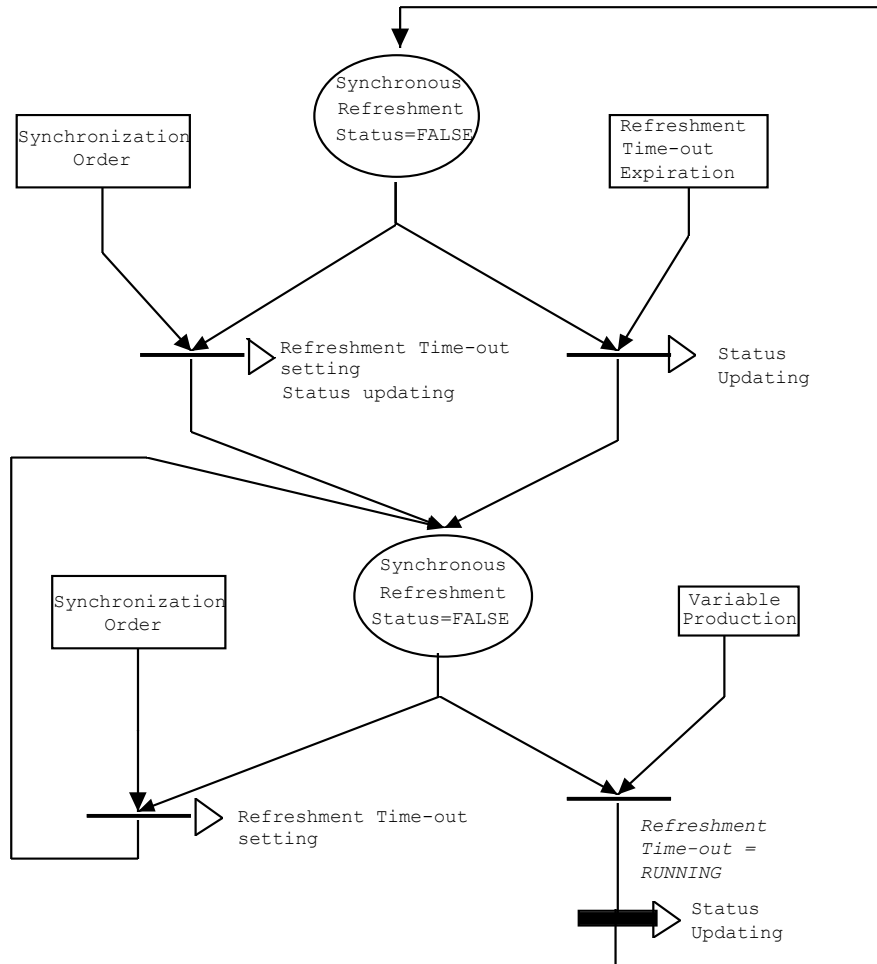


Figure 101 – Synchronous refreshment status evaluation net

Table 461 – Synchronous refreshment events and actions

Initial state:	<u>Synchronous refreshment status = FALSE</u> Refreshment time-out state = IDLE
Requests:	<u>Variable Production:</u> A_WRITELOC, A_WRITEFAR or A_WRITE service invocation by the user <u>Refreshment time out expiration:</u> Transition of the time-out associated with the synchronous refreshment evaluation mechanism from the state <i>RUNNING</i> to the state <i>IDLE</i> <u>Synchronization order:</u> Occurrence of the synchronization order associated with the synchronous refreshment elaboration mechanism.
Action:	<u>Refreshment time-out setting:</u> Transition of the time-out associated with the synchronous refreshment mechanism from the state <i>IDLE</i> to the state <i>RUNNING</i> , with a preset equal to the <i>Production Period</i> attribute value. <u>Status updating:</u> Providing of the asynchronous refreshment status value to the network.

12.2.1.3.3.4.3 Punctual refreshment

12.2.1.3.3.4.3.1 Punctual refreshment definition

Production validity information is optional.

Punctual refreshment is relative to the validity of availability to the network by the producer user of a variable value. This validity is elaborated by the means of a time-slot fixed by the user. This time slot is activated upon the reception of a synchronization order from the network.

Punctual Refreshment:

When it has the value TRUE this boolean indicates within a producing application entity that a variable value has been provided to the network by the producer user (at least) once during the time slot following the reception of a synchronization order. The variable value shall not have been provided outside the time slot.

Punctual refreshment status is associated with the variable value and is provided to the network.

12.2.1.3.3.4.3.2 Punctual Refreshment class specification

This class gathers all the attributes relative to the elaboration of a punctual refreshment information associated with a variable within a producing application entity.

Class: PUNCTUAL REFRESHMENT

Attribute	: Production time slot
Attribute	: <i>Inherit from</i> Time-out
Attribute	: <i>Reference</i> (Synchronization) Variable
Attribute	: Punctual Refreshment Status [TRUE,FALSE]

Production Time slot:

Punctual refreshment is elaborated by the means of a production time-slot associated with a variable by the producer user. Punctual refreshment allows a consumer user to check whether the variable producer guarantees its production within the time slot activated by the last synchronization order issued from the network, and associated with it.

Inherit from Time-out:

Punctual Refreshment class inherits from *time-out* class.

Time-out class includes all the attributes relative to a time-out (preset, state) that characterize the production time slot.

Reference (Synchronization) Variable:

This attribute refers to a *Variable* class, the attribute *Class* of which has the value SYNCHRONIZATION, and that permits to define the synchronization order that sets the time-out in the case of a punctual refreshment.

Punctual Refreshment Status:

Variable punctual refreshment is characterized by a status.

This status is of type boolean and is locally elaborated within each entity owning a *Produced Variable Local Image* object of the variable.

Punctual refreshment status is characterized by two states, the transition conditions between which are elaborated by the means of:

- Events corresponding to the variable production, or to the occurrence of a synchronization order associated with the variable,
- State of the time-out associated with the variable for the processing of the punctual refreshment.

12.2.1.3.3.4.3.3 Mechanism

Punctual refreshment status associated with a variable, within a producing application entity, is elaborated by the mechanism described in Figure 102 and Table 462.

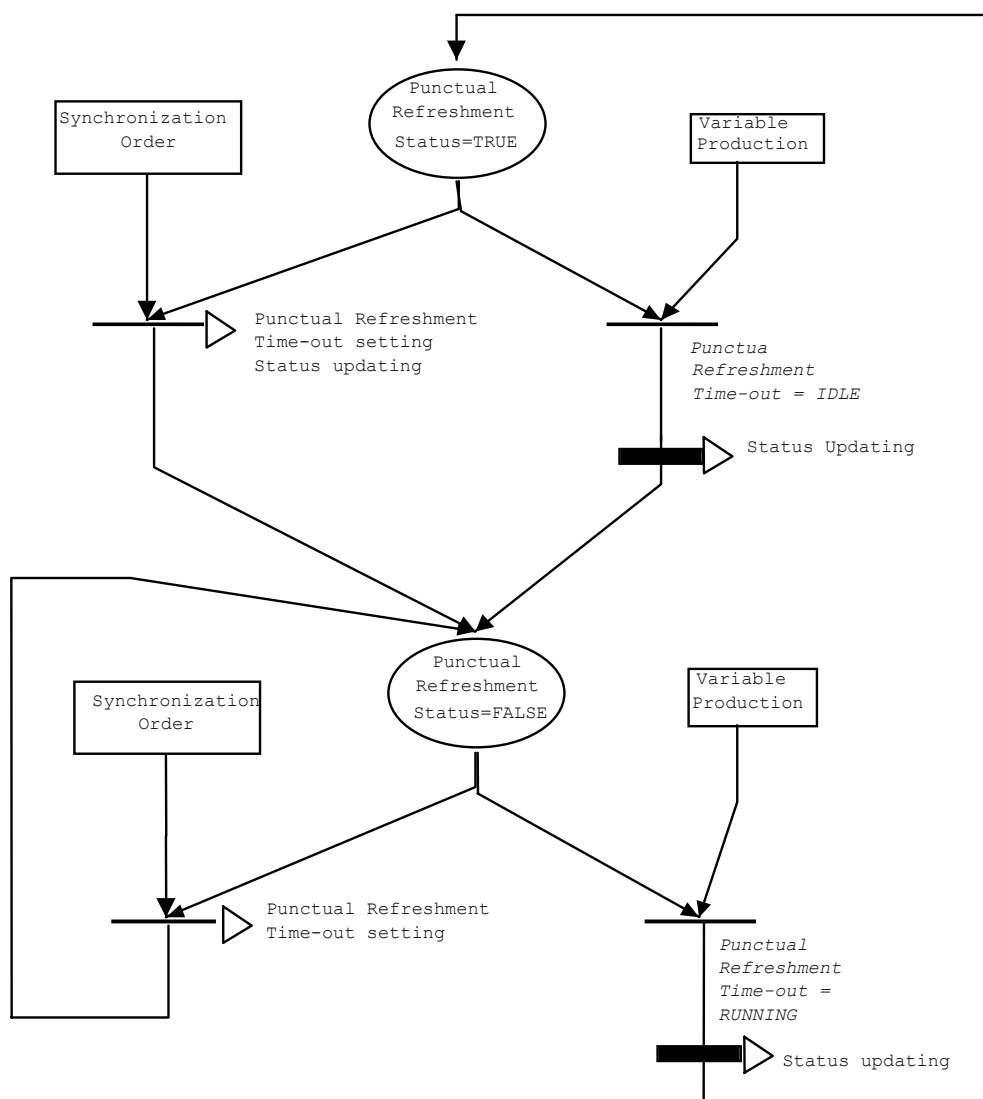


Figure 102 – Punctual refreshment status evaluation net

Table 462 – Punctual refreshment events and actions

Initial state:	<u>Punctual refreshment status = FALSE</u> Refreshment time-out state = IDLE
Requests:	<u>Variable Production:</u> A_WRITELOC, A_WRITEFAR or A_WRITE service invocation by the user <u>Refreshment time out expiration:</u> Transition of the time-out associated with the punctual refreshment evaluation mechanism from the state <i>RUNNING</i> to the state <i>IDLE</i> <u>Synchronization order:</u> Occurrence of the synchronization order associated with the punctual refreshment elaboration mechanism.
Action:	<u>Refreshment time-out setting:</u> Transition of the time-out associated with the punctual refreshment mechanism from the state <i>IDLE</i> to the state <i>RUNNING</i> , with a preset equal to the <i>Production Time Slot</i> attribute value. <u>Status updating:</u> Providing of the asynchronous refreshment status value to the network.

12.2.1.3.4 Basic services**12.2.1.3.4.1 Basic service overview**

The variable access basic services in the MPS environment allow the user to access the associated application objects identified by their specification.

The variable specification corresponds:

- either to the application name for variables accessed in a global way. In this case the application name corresponds to the *A_Name* of the *Consumed Variable Local Image* or *Produced Variable Local Image* or *Third part Variable Local Image* objects.
- or to the partial explicit access name either for structured variables or array type variables partially accessible. This access name is built together with the application variable *A_Name* followed by the access path corresponding to the successive enumeration of field names of the tree of the structure and/or index elements of an array.
- or to the access path in the case of partial or global renamed access on a variable. This name corresponds to the *A_Name* attribute of the Variable Access object

12.2.1.3.4.2 Local read service**12.2.1.3.4.2.1 Functionality**

With this service the user can read a variable value or a field of a structured variable value or the element of an array type variable value which is available in the local communication entity.

12.2.1.3.4.2.2 Service primitives

A local read is performed by using A_READLOC service primitives, as shown in Table 463:

A_READLOC.Rq (Argument): Local read request

A_READLOC.Cnf (Result): Local read confirmation

Table 463 – A_Readloc service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Result (+)		S
Value		M
Refreshment status		C
Refreshment status		C
Promptness status		C
Punctual Promptness status		C
Result (-)		S
Type of error		M

Argument:

Information associated with a local read request.

Variable specification:

Corresponds to the identification of a *Produced Variable Local Image* object in the MPS addressing space.

Result (+):

Valid answer associated with a local read request

Value:

It is the value of the variable, or field, or array element specified in the request. This parameter corresponds to the attribute *Public value* of the *Consumed Variable Local Image* object for the non resynchronized variables. For resynchronized variables this parameter corresponds to the attribute *Private value* of the *Consumed Variable Local Image* object.

Refreshment status:

Punctual Refreshment status:

Information related to the production validity.

This information come from the attribute *Transmitted status value* of the object *Consumed Variable Local Image*.

Promptness status:

Punctual Promptness status:

Information related to the consuming validity.

This information came from the attributes with same name within the *Consumed Variable Local Image* object.

Result (-):

This indicates that the local read service failed.

Type of error:

Various error types may be returned in the confirmation of a read request.

Error_1: The variable value is temporarily not accessible.

Error_2: The variable value is not accessible by means of this service. It corresponds to a variable which is not locally declared, or which is accessible only by production services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

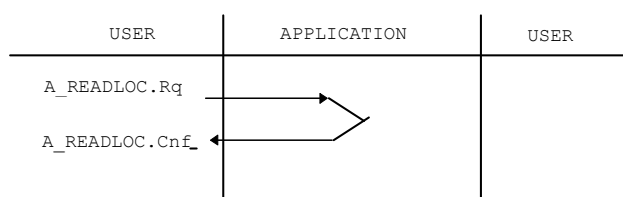
12.2.1.3.4.2.3 Service procedure

Figure 103 – A_Readloc service procedure

This service, shown in Figure 103, ensures the value integrity of the variable or the field or the element handled.

The value integrity of a variable is ensured in a read operation when all the binary elements of the data associated with this value come from the same update.

Furthermore requests on a variable are processed sequentially: a new request may be made only after reception of confirmation upon the previous one.

12.2.1.3.4.3 Local write service**12.2.1.3.4.3.1 Functionality**

With this service the user can write a variable value or a field of a structured variable value or an element of an array type variable value in the local communication entity.

12.2.1.3.4.3.2 Service primitives

A local write is performed using A_WRITELOC service primitive, as shown in Table 464.

A_WRITELOC.Rq (Argument) : Local write request.

A_WRITELOC.Cnf (Result) : Local write confirmation.

Table 464 – A_Writeloc service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Value	M	
Result (+)		S
Result (-)		S
Type of error		M

Argument:

Information associated with a local write request

Variable Specification:

Corresponds to the identification of a *Consumed Variable Local Image* object in the MPS addressing space. The definition of this parameter corresponds to the one shown in paragraph.3.8

Value:

Value to be written to the specified variable specification, or field of a structured variable, or element of an array type variable.

This parameter corresponds to the attribute *Public value* of the *Produced Variable Local Image* object for the non resynchronized variables. For the resynchronized variables this parameter corresponds to the attribute *Private value* of the *Produced Variable Local Image* object.

Result (+):

No parameters

Result (-):

This indicates that the local write service failed.

Type of error:

Various error types may be returned in the confirmation of a read request.

Different error types can be returned with the local write confirmation.

Error_1: The variable value is temporarily not accessible.

Error_2: The variable is not accessible with use of this service. It corresponds to a variable which is not locally declared, or which is accessible only by consuming services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

12.2.1.3.4.3.3 Service procedure

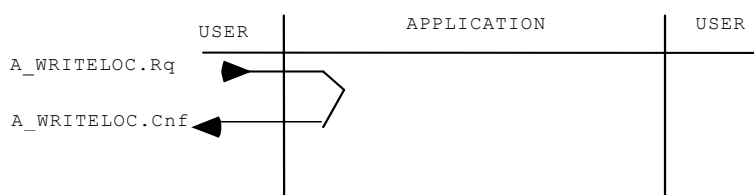


Figure 104 – A_Writeloc service procedure

This service, shown in Figure 104, ensures the integrity of the value of the handled variable.

The integrity of the written variable corresponds to provide the network with a consistent value coming from the same write operation.

All status elaborated by the producing entity and provided to the network shall be consistent with the value of the transmitted variable.

Furthermore this service triggers the mechanism associated with the service elements which elaborate refreshment and punctual refreshment status of the *Produced Variable Local Image* object.

Partial access on a variable when writing a value triggers mechanism of production validity status elaboration associated with this variable.

Furthermore request on variable are processed sequentially; a new request can be made only after reception of confirmation upon the previous one.

12.2.1.3.4.4 Updating service

12.2.1.3.4.4.1 Functionality

This service supports the request (to the bus arbitrator) for the updating of a variable value at the consuming entities level. The variable value is locally available in the producer communication entity level.

This service can be used by the application entity which produces the variable, by an application entity which consumes it, or by an application entity which is neither producer nor consumer (ie: a third party entity).

12.2.1.3.4.4.2 Service primitives

An updating request is performed with the following service primitives, as shown in Table 465.

A_UPDATE.Rq (Argument) : Updating request.

A_UPDATE.Cnf (Result) : Updating confirmation.

Table 465 – A_Update service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Priority	M	
Result (+)		S
Result (-)		S
Type of error		M

Arguments:

Information associated with an updating request.

Variable Specification:

Corresponds to the identification of a *Produced Variable Local Image* object or a *Produced Variable Local Image* object or *Third party Variable Local Image* object in the MPS addressing space

Priority:

This parameter allows the user to choose between two flow control possibilities **Urgent** and **Normal** to request the updating of a variable the name of which has been specified in the request.

The flow control ensures that an urgent request, will trigger an updating before a subsequent normal request.

Result (+):

No parameters.

Result (-):

This indicates that the updating service failed.

Type of error:

Various error types may be returned in the service confirmation:

Error_1: The variable is not declared at the AP level of the request initiator.

Error_2: The request cannot be accepted given the local limits concerning the number of outstanding requests.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

12.2.1.3.4.4.3 Service procedure

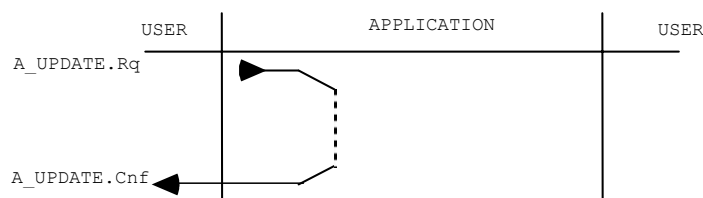


Figure 105 – A_Update service procedure

This service, shown in Figure 105, guarantees that the user will receive a confirmation after a request.

The service confirmation is a positive one when the request is positively confirmed at the data link level.

Requests may be chained together even before receiving all confirmations. However; confirmations are not always issued in the order of requests previously initiated.

For a SYNCHRONIZATION class variable, this service can be initiated only by the variable producer.

NOTE The maximum number of outstanding requests depends on the implementation, and saturation is specified in error codes associated with the confirmation.

12.2.1.3.4.5 Remote read service

12.2.1.3.4.5.1 Functionality

With this service the consumer user of a variable can take the initiative of an information exchange on the bus, before performing a read on the new variable value locally available.

12.2.1.3.4.5.2 Service primitives

Remote read is done with the A_READFAR service primitive, as shown in Table 466.

A_READFAR.Rq (Argument): Remote read request

A_READFAR.Cnf (Result): Remote read confirmation with return of the read value

Table 466 – A_Readfar service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Priority	M	
Result (+)		S
Value		M
Refreshment Status		C
Punctual Refreshment Status		C
Promptness Status		C
Punctual Promptness status		C
Result (-)		S
Type of error		M

Argument:

Information associated with a remote read request.

Variable specification:

Corresponds to the identification of a *Consumed Variable Local Image* object in the MPS addressing space.

Priority:

This parameter allows the user to choose between two flow control possibilities **Urgent** and **Normal** to request the updating of a variable the name of which has been specified in the request.

The flow control guarantees that an urgent request, will trigger an updating before a subsequent normal request.

Result (+):

This result indicates that the requested service has succeeded.

Value:

The value of the variable or field, or a array element specified in the request.

This parameter corresponds the attribute *Public value* of the object *Consumed Variable Local Image* for the non resynchronized variables. For resynchronized variables this parameter corresponds to the attribute *Private value* of the object *Consumed Variable Local Image*.

Refreshment status:**Punctual Refreshment status:**

Information relative to the production validity This information comes from the attribute *Transmitted status value* of the *Consumed Variable Local Image* object.

Promptness status:**Punctual Promptness status:**

Information relative to the consumption validity

This information comes from the attributes with the same name within the object *Consumed variable Local Image*.

Result (-):

This indicates that the requested service failed.

Type of error:

Various error types may be returned in the service confirmation:

Error_1: The variable value is temporarily not accessible.

Error_2: The variable value is not accessible by means of this service. It corresponds to a variable which is not locally declared or which is accessible only by production services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

Error_4: The request cannot be accepted given the local limits concerning the number of outstanding requests.

Error_5: The request has been aborted due to expiration of the time-out associated with the reception waiting of the variable value.

12.2.1.3.4.5.3 Service procedure

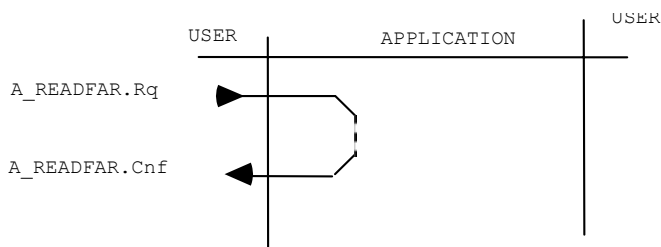


Figure 106 – A_Readfar service procedure

This service, shown in Figure 106, guarantees the integrity of the handled variable value, or field, or array element value.

Requests may be chained together. A new request can be issued without waiting for corresponding confirmations. However, confirmations are not always issued in the same order as the requests.

This service guarantees that the user will receive a confirmation after a request.

The value integrity of a variable value is ensured in a read operation when all the binary elements of the data associated with this value come from the same updating.

12.2.1.3.4.6 Remote write service

12.2.1.3.4.6.1 Functionality

With this service ,the variable producer user may initiate an information exchange on the bus after providing the variable value to the network.

12.2.1.3.4.6.2 Service primitives

Remote write is performed by using the A_WRITEFAR service primitive, as shown in Table 467.

A_WRITEFAR.Rq (Argument): Remote write request

A-WRITEFAR.Cnf (Result): Remote write confirmation

Table 467 – A_Writefar service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Priority	M	M(=)
Value	M	
Result (+)		S
Result (-)		S
Type of error		M

Arguments:

Information associated with a remote request

Variable specification:

Corresponds to the identification of a *Produced Variable Local Image* object in the MPS space

Priority:

This parameter allows to the user to choose between two flow control possibilities **Urgent** and **Normal** to request the updating of a variable which the name of which has been specified in the request.

The flow control ensures that an urgent request, will trigger an updating before a subsequent normal request.

Value:

Upon a remote write the user associates with the variable specification the value he intends to affect it.

This parameter corresponds to the attribute *Public value* of the *Produced Variable Local Image* object for the non resynchronised variables. For the resynchronized variables this parameter corresponds to the attribute *Private value* of the *Produced Variable Local Image* object.

Result (+):

No parameters.

Result (-):

This indicates that the remote write service failed.

Type of error:

Different error types may be returned in the service confirmation:

Error_1: The variable value is temporarily not accessible.

Error_2: The variable is not accessible by means of this service. It corresponds to a variable which is not locally declared or which is accessible only by consuming services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

Error_4: The remote write request cannot be accepted due to local limits concerning the number of outstanding requests. However, the local variable value has been updated.

Error_5: The request has been withdrawn

12.2.1.3.4.6.3 Service procedure

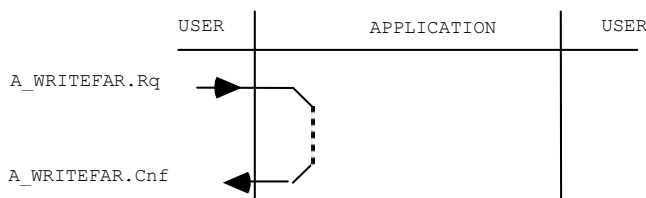


Figure 107 – A_Writefar service procedure

This service, shown in Figure 107, ensures the value integrity of the handled variable or the field, or the element.

Requests may be chained together. A new request can be issued without waiting for corresponding confirmations. However, confirmations are not always issued in the same order as the requests.

This service ensures that the user will receive a confirmation after a request.

The integrity of a written variable value corresponds to an updating of the network with a consistent value coming from only one write operation. This service also ensures the consistency between the produced variable value and statuses related to the production validity which are optionally associated.

Moreover, this service triggers the mechanisms associated with the service elements that elaborate the value of the refreshment status and the punctual refreshment status of the *Produced Variable Local Image* object.

NOTE The value provided to the consuming entities will be the one which was available at the producer level during its network updating.

12.2.1.3.4.7 Transmission indication service

12.2.1.3.4.7.1 Functionality

This service is optional, and indicates to a producer user that a variable has been transmitted on the network.

12.2.1.3.4.7.2 Service primitives

A transmission indication is performed using the A_SENT service primitive, as shown in Table 468.

A_SENT.Ind (Result): Transmission indication of a variable value

Table 468 – A_Sent service parameters

Parameter name	Ind
Result	
Variable specification	M
Result (+)	S
Result (-)	S
Type of error	M

Result:

Variable specification:

Corresponds to the identification of a *Produced Variable Local Image* object in the MPS addressing space.

Result(+):

No parameters

Result(-):

The results indicates that the service failed.

Type of error:

Various error types may be returned by the transmission indication service:

Error_1: The value of the transmitted variable is declared invalid.

12.2.1.3.4.7.3 Service procedure

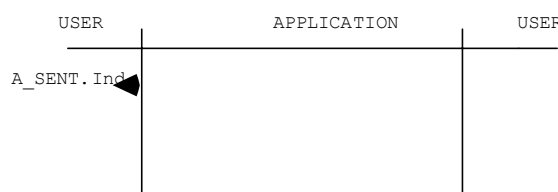


Figure 108 – A_Sent service procedure

This service, shown in Figure 108, informs the producer user of the broadcasting of a variable on the network.

When this variable or some of its fields are locally known at the producer user level by means of names coming from renamed partial access, this service provide an indication to the user for each of the names associated with the same variable.

12.2.1.3.4.8 Reception indication service

12.2.1.3.4.8.1 Functionality

This service informs a consumer user that a variable has been made available by the network.

12.2.1.3.4.8.2 Service primitives

The reception indication is performed by the use of the A_RECEIVED service primitive, shown in Table 469.

A_RECEIVED.Ind(Result): Variable reception indication.

Table 469 – A_Received service parameters

Parameter name	Ind
Result	
Variable specification	M
Result (+)	S
Result (-)	S
Type of error	M

Result:

Variable specification:

Corresponds to the identification of a *Consumed Variable Local Image* object in the MPS addressing space.

Result(+):

No parameters

Result (-):

This indicates that the indication service failed.

Type of error:

Different error type can be returned by the reception indication service:

Error_1: The value of the received variable is declared invalid by the producing application entity.

Error_2: The transmission of this variable failed.

12.2.1.3.4.8.3 Service procedure

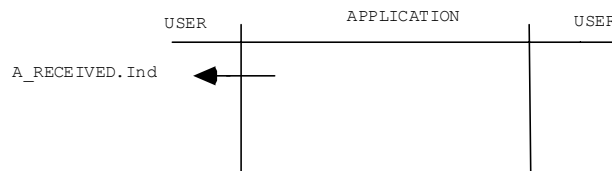


Figure 109 – A_Received service procedure

This service, shown in Figure 109, informs the user about the provision of a variable by the network.

12.2.1.3.5 Enhanced services

12.2.1.3.5.1 Enhanced services definition

Enhanced services are a further level of abstraction and build on the basic services to allow the user to access variables without the need to specify either the type of service (local or remote) or the priority used.

12.2.1.3.5.2 Universal read service.

12.2.1.3.5.2.1 Functionality

This service allows the user to read a variable value without knowing the kind of read service (local or remote) to be used; the service is determined by the value of the attribute *Universal services range* of the class *Identified Variable*.

Furthermore when it is a remote read the priority used by the flow control is determined by the value of the attribute *Priority* of the class *Identified Variable*.

12.2.1.3.5.2.2 Service primitives

The universal read is performed using the A_READ service primitives, shown in Table 470.

A_READ.Rq (Argument): Universal read request

A_READ.Cnf (Result): Universal read confirmation with the read value

Table 470 – A_Read service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Result (+)		S
Value		M
Refreshment Status		C
Punctual Refreshment Status		C
Promptness Status		C
Punctual Promptness status		C
Result (-)		S
Type of error		M

Argument:

Information associated with a universal read request.

Variable specification:

Corresponds to the identification of a *Consumed Variable Local Image* object in the MPS addressing space.

Result (+):

Valid answer associated with an universal read request.

Value:

It is the value of the variable, or field, or array element specified in the request. This parameter corresponds to the attribute *Public value* of the object *Consumed Variable Local Image* for non-resynchronized variables. For resynchronized variables this parameter corresponds to the attribute *Private value* of the object *Consumed Variable Local Image*.

Refreshment status:

Punctual Refreshment status:

Information related to the production validity.

This information is derived from the attribute *Transmitted status value* of the *Consumed Variable Local Image* object.

Promptness status:

Punctual promptness status:

Information related to the consuming validity.

This information is derived from the attributes with same name within the object *Consumed Variable Local Image*.

Result(-):

This indicates that the universal read service failed.

Type of error:

Various error types may be returned in the service confirmation:

Error_1: The variable value is temporarily not accessible.

Error_2: The variable value is not accessible with use of this service. It corresponds to a variable which is not locally declared, or which is accessible only by production services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

Error_4: The request cannot be accepted given the local limits concerning the number of outstanding requests.

12.2.1.3.5.2.3 Service procedure

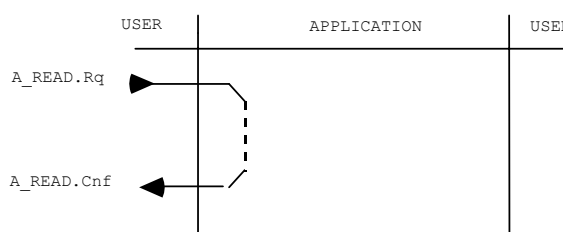


Figure 110 – A_Read service procedure

According to the value of the attribute *Universal services range* of the *Consumed Variable Local Image* object, the invocation of this service implies either the call of the local read service or the call of the remote read service. The universal read service procedure, Figure 110, is the one associated with the invoked service (local or remote).

The priority used in the case of a remote service invocation depends on the value of the attribute *Priority* of the class *Identified variable*.

Figure 111 shows the mechanism for chaining the basic services:

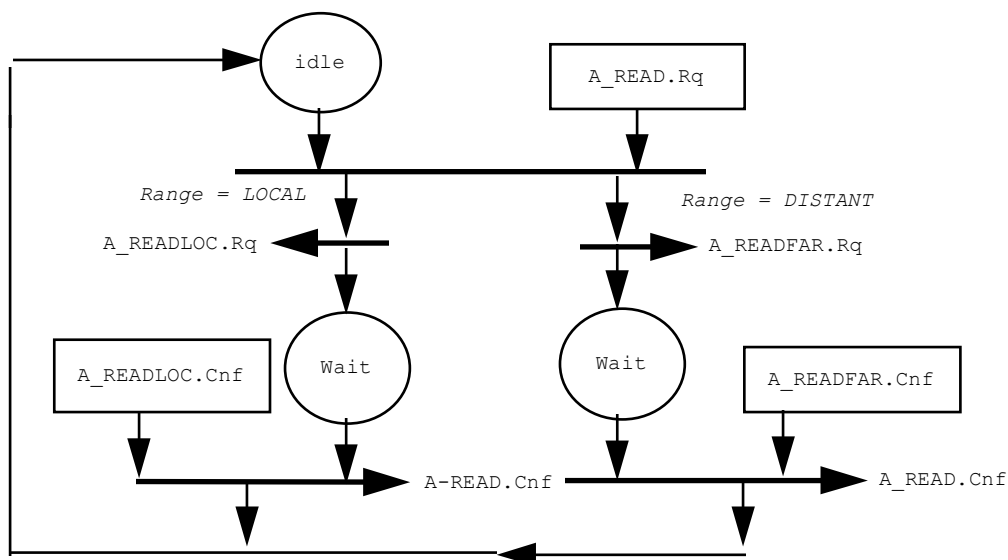


Figure 111 – A_Read service state machine

12.2.1.3.5.3 Universal write service

12.2.1.3.5.3.1 Universal write service functionality

This service allows the user to perform a write of the variable value without having to know the kind of write service (local or remote) to be used. The service is determined by the value of the attribute *Universal service range* of the *Produced Variable Local Image* object.

Furthermore when it is a remote write the priority used by the flow control is determined by the value of the attribute *Priority* of the class *Identified Variable*.

12.2.1.3.5.3.2 Service primitives

The universal write is performed by using the A_WRITE service primitives, shown in Table 471.

A_WRITE.Rq (Argument) : Universal write request

A_WRITE.Cnf (Result) : Universal write confirmation

Table 471 – A_Write service parameters

Parameter name	Req	Cnf
Argument		
Variable specification	M	M(=)
Priority	M	
Result (+)		S
Result (-)		S
Type of error		M

Arguments:

Information associated with a universal write request.

Variable specification:

Corresponds to the identification of a Produced Variable Local Image object in the MPS addressing space.

Value:

It is the value of the variable, or field, or array element specified in the request. This parameter corresponds to the attribute *Public Value* of the *Produced Variable Local Image* object for non-resynchronized variables. For synchronized variables this parameter corresponds to the attribute *Private Value* of the *Produced Variable Local Image* object.

Result(+):

No parameters.

Result(-):

This indicates that the universal write service failed.

Type of error:

Different error types may be returned in the confirmation of a universal write:

Error_1: The variable value is temporarily not accessible.

Error_2: The variable value is not accessible by means of this service. It corresponds to variable which is not locally declared which is accessible only by consuming services.

Error_3: The variable is declared invalid at the network management level with reference to consistency checking of the data base.

Error_4: The request cannot be accepted due to local limits concerning the number of outstanding requests. However, the local variable value has been updated.

Error_5: The request has been withdrawn

12.2.1.3.5.3.3 Service procedure

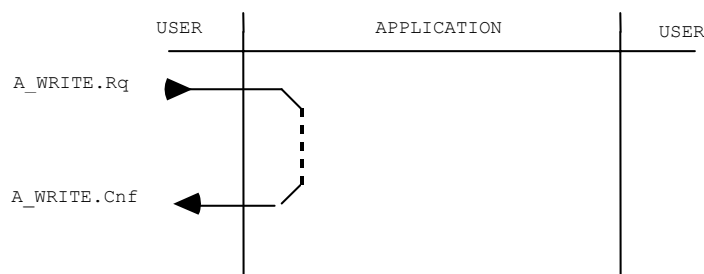


Figure 112 – A_Write service procedure

According to the value of the attribute *Universal services range* of the *Produced Variable Local Image* object, the invocation of this service implies either the call of the local write service or the call of the remote write service. The universal write service procedure, Figure 112, is the one associated with the invoked service (local or remote).

The priority used in the case of a remote service invocation depends on the value of the attribute *Priority* of the class *Identified variable*.

This service triggers the mechanism associated with the service elements that elaborate the refreshment status value and the punctual refreshment status value of the *Produced Variable Local Image* object.

Figure 113 shows the mechanism for chaining the basic services used by the universal write service:

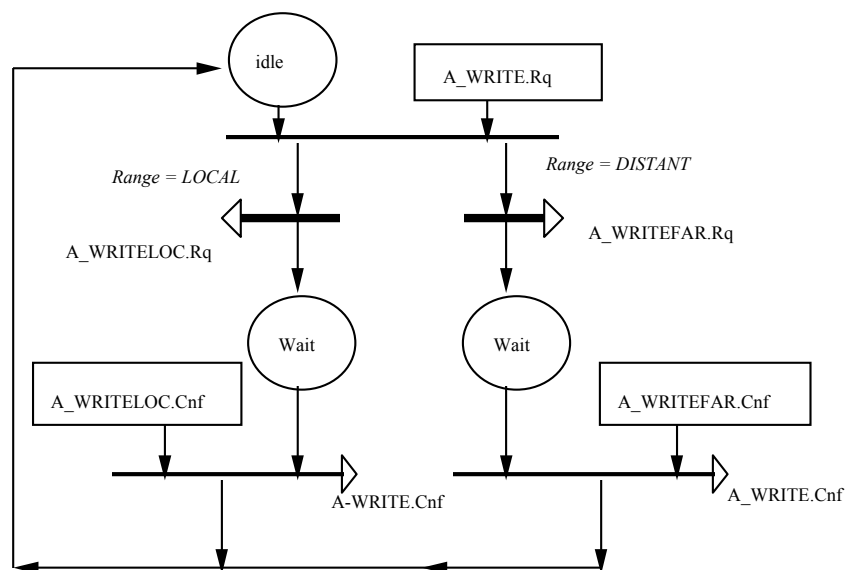


Figure 113 – A_Write service state machine

12.2.1.3.6 Advanced mechanisms

12.2.1.3.6.1 Synchronization

12.2.1.3.6.1.1 Synchronous application concept

The synchronization services provided by the MPS service elements were defined in order to allow the execution of distributed applications.

A distributed application is composed of several Application Processes (AP) in several devices of the control system.

Between the APs which participate in a distributed application, it is necessary to distinguish:

- those which are termed **ASYNCHRONOUS**, because their execution is independent of the network.
- those which are termed **SYNCHRONOUS**, because their execution is associated with indications issued from the network; these indications may be periodic or aperiodic.

Several synchronous APs, the execution of which are associated with the same indications, form a synchronous distributed application.

12.2.1.3.6.1.2 Resynchronization needs

Some APs can only support asynchronous operating mode, i.e. they are managed independent of the communication environment.

NOTE This exclusively asynchronous operating mode is usually the characteristic of APs located on devices whose age or simplicity does not offer the possibility of synchronization.

The resynchronization mechanism permits the sample/hold function of the variable values produced and consumed by an AP asynchronously with respect to the Network. This function allows asynchronous APs to participate in a synchronous distributed application.

12.2.1.3.6.1.3 Synchronization variables

The variables in the MPS environment described in the previous chapter correspond to passive elements which are exchanged on the network.

The synchronization variables are elements associated with actions concerning:

- synchronization processing at the level of the producing application entity and consuming application entity of this synchronization variable.
- procedure execution orders for different APs in a distributed application.

At the application entity level, some variables from the MPS environment take into account the event associated with the transmission of a synchronization variable to elaborate the synchronous validity informations, such as synchronous promptness and synchronous refreshment, or the punctual promptness and the punctual refreshment.

12.2.1.3.6.1.4 Synchronization at transmission and reception

Some services provided to the user may be used to synchronize the distributed application procedures using the network.

Among these services, it is necessary to distinguish those which allow:

- synchronization at emission time to ensure the activation of various local tasks inside a single AP.
- synchronization at reception time to ensure the activation management of various tasks of several APs located in several devices of the control system.

This type of synchronization is used by a synchronous distributed application.

12.2.1.3.6.1.5 Synchronization variable definition.

12.2.1.3.6.1.5.1 Synchronisation model description

A synchronization variable in the MPS environment is modelled in the same manner as variables using the following objects:

- an object *Produced Variable Local Image* including all the attributes required by a synchronization variable in a producing application entity.
- an object *Consumed Variable Local Image* including all attributes required by a synchronization variable in a consuming application entity.

These two types of object correspond to special instantiations of the class *Identified variable* for which the attribute *Class* has the value SYNCHRONIZATION.

An object *Produced Variable Local Image* or *Consumed Variable Local Image* associated with a synchronization variable differs from the one associated with a normal variable since the value of some of its attributes are predetermined.

12.2.1.3.6.1.5.2 Predetermined attribute values of the *Identified variable* class

The *Identified variable* class associated with a synchronization variable is characterised by predetermined values for some of its attributes:

Class:

This attribute takes the value: *SYNCHRONIZATION*

All the other attribute values are not predetermined.

12.2.1.3.6.1.5.3 Predetermined attribute values of *Produced Variable Local Image* object

The *Produced Variable Local Image* object associated with a synchronization variable is characterised by predetermined values for some of its attributes:

Resynchronized variable:

This attribute take the value: *FALSE*

All the other attribute values are not predetermined.

12.2.1.3.6.1.5.4 Predetermined attribute values of *Consumed Variable Local Image* object

The *Consumed Variable Local Image* object associated with a synchronization variable is characterised by predetermined values for some of its attributes:

Resynchronized variable:

This attribute takes the value: *FALSE*

All the other attribute values are not predetermined.

12.2.1.3.6.1.5.5 Access to synchronization variables

The synchronization variables may be handled by a subset of services provided to the user for normal variables.

The services which provide access to synchronization variables are:

- A_WRITELOC / A_WRITEFAR / A_WRITE
- A_READLOC
- A_SENT
- A_RECEIVED
- A_UPDATE

All the others services offered to the user for a normal variable are not authorized for a synchronization variable.

12.2.1.3.6.1.5.6 Synchronization variable type

The synchronization variables may have a value of any type coming from the instantiation of the *Type constructor* class.

However, a synchronization variable used to meet the requirements of the application service element which elaborate a spatial consistency status, shall contain in its value the network exchange number of the list of variables. In this case the variable has a predetermined type.

The exchange number is global data in the MPS environment, produced by a specific application entity. This exchange number qualifies the value of a variable exchanged in a periodic or aperiodic way, relative to the previous exchange.

This exchange number is accessed by the user in the same way as the value of a normal variable, and it is also used by an application service element for the elaboration of the spatial consistency status.

This predetermined type is described by the following instance of the *Type constructor* class.

A_type:	K_UN16
Construction:	SIMPLE
Class of type:	UNSIGNED
Size:	16

12.2.1.3.6.2 Resynchronization

12.2.1.3.6.2.1 Resynchronisation mechanism principle

The resynchronization mechanism is applied to variables of NORMAL class in order to provide a sample/hold function on asynchronous produced or consumed variable values. This resynchronization mechanism requires that the network transmits synchronization orders in order to:

- trigger the sample/hold of the variables produced by the user.
- trigger the sample/hold of the variables consumed by the user.

Definition:

A resynchronization order corresponds to the synchronization order used by the resynchronization mechanism.

The resynchronization mechanism associates a second buffer with a variable at the application entity level, as shown in Figure 114. A resynchronized variable requires:

- A private buffer (TPP) exclusively accessible by the user.
- A public buffer (TPU) accessible by the network.

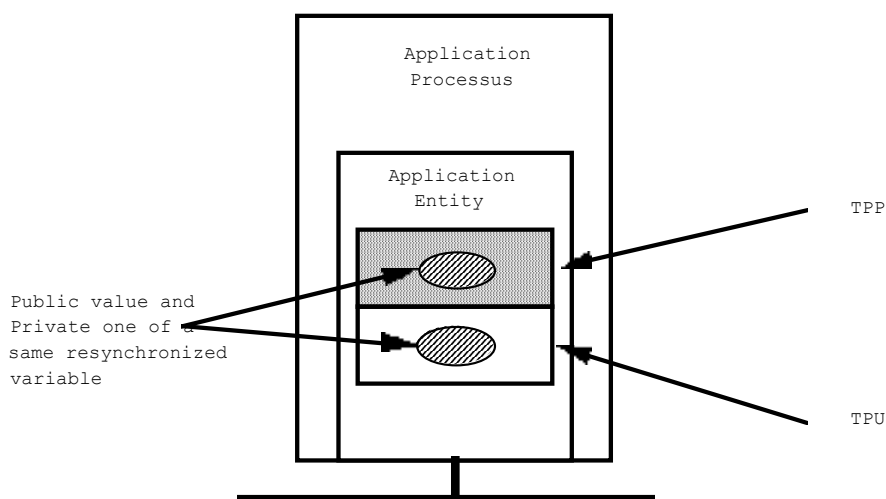


Figure 114 – Model of a resynchronised variable

12.2.1.3.6.2.2 Resynchronization of a produced variable

12.2.1.3.6.2.2.1 Principle

Assume an AP produces a variable in the private buffer (TPP) with a T_p production period. The availability of the value of this variable in the public buffer (TPU) is synchronized by the network by means of a resynchronization order. This synchronization order is broadcast with a period T_r .

The resynchronization mechanism in production holds the value of the produced variable during the time interval between two occurrences of the same resynchronization order.

Figure 115 shows these principles.

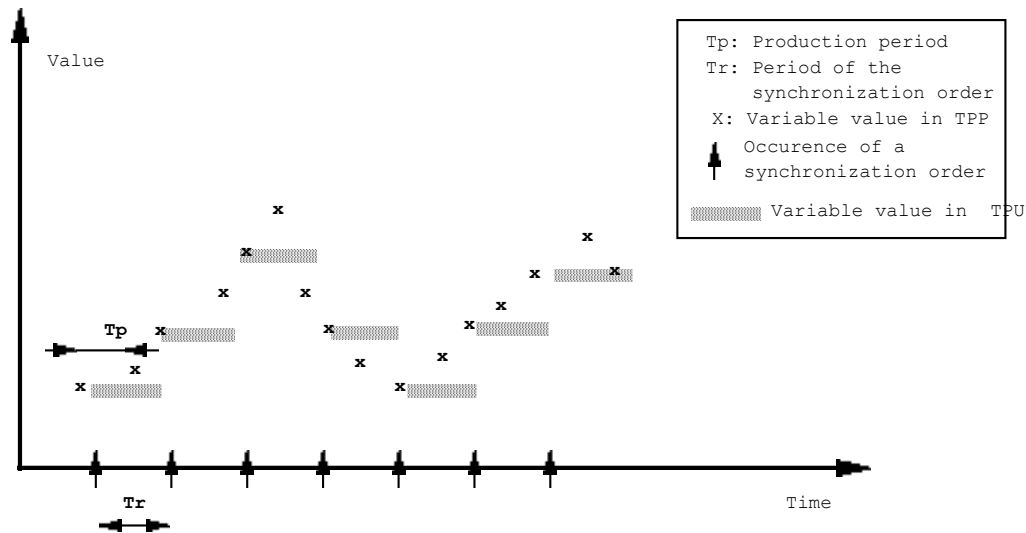


Figure 115 – Principles for resynchronisation of a produced variable

NOTE The resynchronization mechanism of a produced variable is useful only when $T_r \gg T_p$.

The resynchronization service implies the use of extra resources like additional buffers for the private values associated with the public values of the resynchronized variables.

A refreshment status elaboration time out and a punctual refreshment elaboration time out are optionally associated with each public buffer of a resynchronized variable.

The resynchronization service requires the use of a private time-out associated with the private buffer in order to allow the production status to preserve their semantics when the variables are resynchronized.

The elaboration of the production validity status for a resynchronized variable at the producer entity level involves the use of two mechanisms associated with the public and private values of the resynchronized variable.

12.2.1.3.6.2.2.2 Specification of the resynchronization in production class

This class describes additional resources used to resynchronize a produced variable at the application entity level.

The class used to describe a resynchronized variable at the producing entity level inherits from the attributes of the *Resynchronization in production* class.

Class: RESYNCHRONIZATION IN PRODUCTION

- Attribute : Private value
- Constraint : Refreshment elaborated = TRUE
- Attribute : Private Refreshment status
- Attribute : Inherit from Time-out
- Constraint : Punctual Refreshment elaborated = TRUE
- Attribute : Private Punctual Refreshment status
- Attribute : Reference Synchronization variable

Private value:

The resynchronization mechanism principle is based on data which is double_buffered. It is necessary to add a private value to the public value of the variable. The private value may now be written asynchronously.

Refreshment elaborated:

This attribute, both in the private context and in the public context, conditions the existence of a refreshment status associated with a variable.

When this attribute is TRUE, a resynchronized variable is provided with a refreshment status and a private refreshment status.

This conditional attribute is declared in the *Produced variable* class.

Private Refreshment status

This private status corresponds to internal information from the production entity of the resynchronized variable, from which is built the value of the refreshment status which will be transmitted with the public value of the variable.

The characteristic of this private status is SYNCHRONOUS or ASYNCHRONOUS according to the global declaration associated with the resynchronized variable in production which is made in the *Refreshment* class.

In the case of a synchronous private status the synchronization variable used by the private mechanism associated with this status is declared globally in the *Refreshment* class.

Inherit from Time-out:

This set of resources allows the user to maintain the semantics of the refreshment status associated with a resynchronized variable.

In fact, the use of a synchronous or asynchronous refreshment status requires the use of an extra time-out to preserve the whole semantics of this status.

Punctual Refreshment elaborated:

This attribute, both in the private context and in the public context, conditions the existence of a punctual refreshment status associated with a variable.

When this attribute is TRUE, a resynchronized variable is provided with a punctual refreshment status and a private punctual refreshment status.

This conditional attribute is declared in the *Produced variable* class.

Private Punctual Refreshment status:

This private status corresponds to internal information from the production entity of the resynchronized variable, from which is built the value of the refreshment status which will be transmitted with the public value of the variable.

In the case of a private punctual status, the synchronization variable used by the private mechanism associated with this status is globally declared in the *Punctual Refreshment* class.

Reference (Synchronization) variable:

The reference to a synchronization variable indicates the resynchronization order associated with a resynchronized variable.

This resynchronization order conditions the transfer of variable values, production status values and current values of time-outs from the private domain to the public domain. This reference indicates the *A_Name* of the *Produced Variable Local Image* object.

12.2.1.3.6.2.2.3 Mechanism associated with the produced value

The resynchronization mechanism in production of the local variable value is made by the transfer from the private buffer to the public buffer, at the reception of resynchronization order.

The evaluation net of this service element which manages the resynchronization actions within a producing application entity for a resynchronized variable is shown in Figure 116.

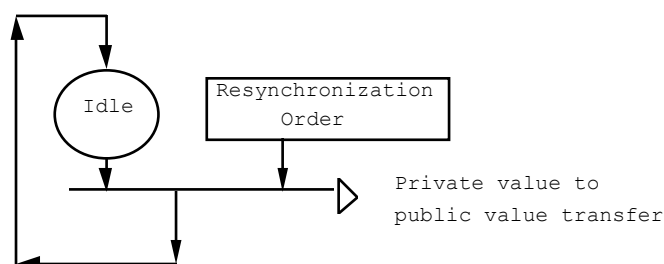


Figure 116 – Resynchronisation mechanism state machine for a produced variable

12.2.1.3.6.2.2.4 Mechanism associated with an asynchronous refreshment

The mechanism which elaborates the asynchronous refreshment status for a resynchronized variable may be described with two evaluation nets one of which describes the private asynchronous refreshment status and the other the asynchronous refreshment status associated with the public value of the variable.

Private mechanism:

The private time-out associated with refreshment is preset with the production period of the variable and set by the user with the act of writing a new private value, as shown in Figure 117 and Table 472.

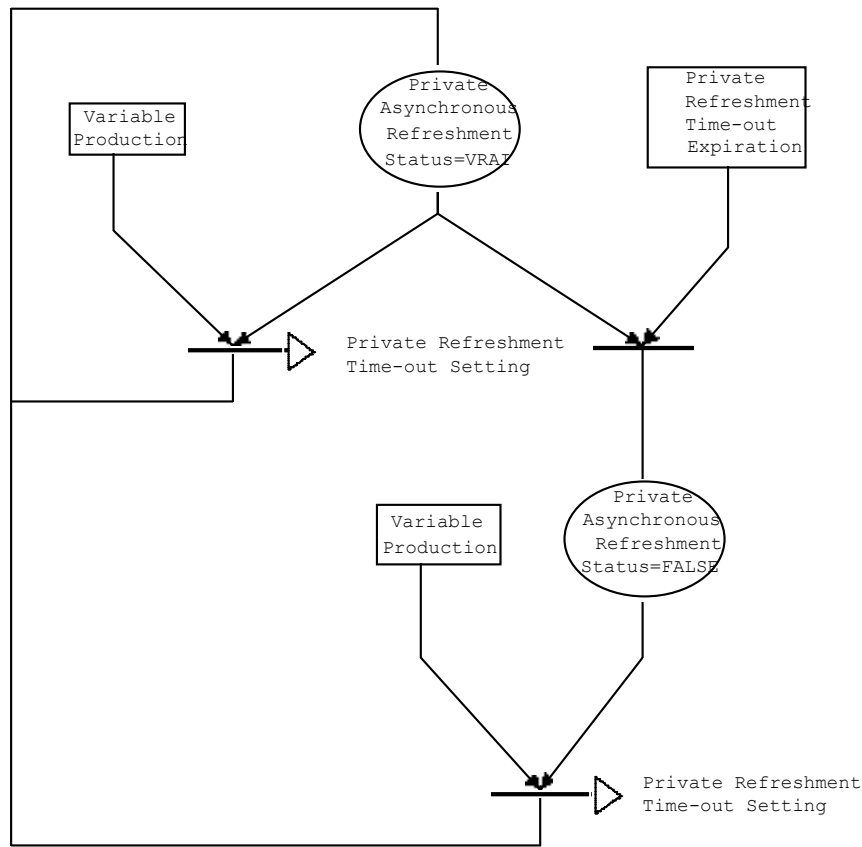


Figure 117 – Asynchronous refreshment private mechanism evaluation net

Table 472 – Asynchronous refreshment private mechanism events and actions

Initial state:	Private asynchronous refreshment status = <i>FALSE</i> State of the private refreshment time-out = <i>IDLE</i>
Requests:	<u>Variable Production:</u> Invocation of the services A_WRITELOC, A_WRITEFAR or A_WRITE by the user <u>Private Refreshment Time-out Expiration:</u> Switch of the private time-out associated with the private asynchronous refreshment status from the <i>RUNNING</i> state to the <i>IDLE</i> state.
Action:	<u>Private Refreshment Time-out Setting</u> Switch of the private time-out associated with the private asynchronous status from the <i>IDLE</i> state to the <i>RUNNING</i> state, with the preset value equal to the Production <i>period</i> attribute value.

Public mechanism:

The public time-out is preset with the current value of the private time-out and set on the resynchronization order associated with this status.

The public mechanism to elaborate the asynchronous refreshment status of a resynchronized variable is represented by the following evaluation net, shown in Figure 118 and Table 473.

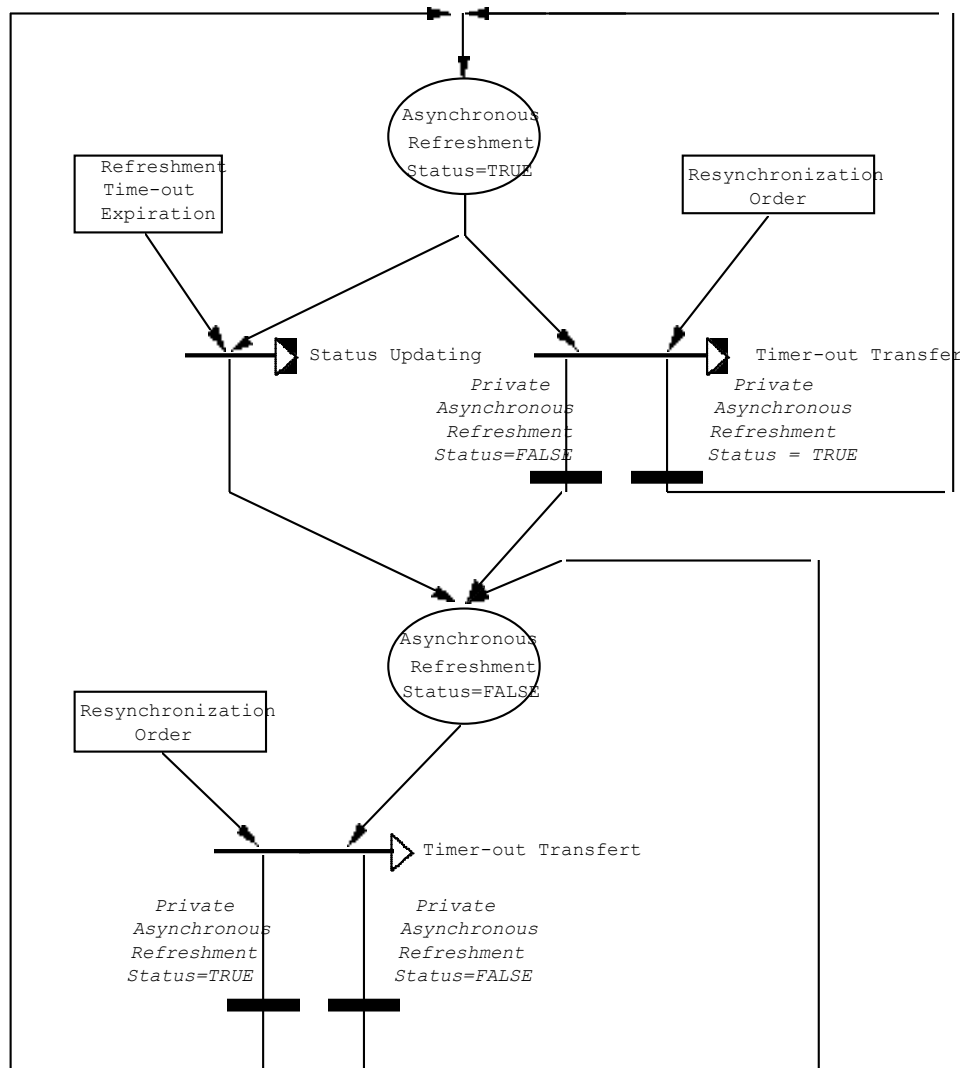


Figure 118 – Asynchronous refreshment public mechanism evaluation net

Table 473 – Asynchronous refreshment public mechanism events and actions

Initial state:	Asynchronous refreshment status = FALSE
	State of the refreshment time-out = <i>IDLE</i>
Request:	<u>Refreshment Time-out Expiration:</u> Evolution of the public time-out associated with the asynchronous refreshment status from the <i>RUNNING</i> state to the <i>IDLE</i> state.
	<u>Resynchronization Order</u> Occurrence of the asynchronization order associated with the public resynchronization mechanism.
Action:	<u>Time-outs Transfer</u> Transfer in the <i>State</i> attribute value and <i>Current attribute value</i> of the private time-out of the private asynchronous refreshment status, of the attributes <i>State</i> and <i>Preset</i> belonging to the public time-out associated with the asynchronous refreshment status.

12.2.1.3.6.2.2.5 Mechanism associated with the synchronous refreshment

The mechanism which elaborates the synchronous refreshment status for a resynchronizes variable may be described with two evaluation nets which one describes the private synchronous refreshment status and the other the synchronous refreshment status associated with the public value of the variable.

Private mechanism:

The private time-out associated with the refreshment is preset with the production period of the variable and set at the receiving time of a new synchronization order associated with this status, as shown in Figure 119 and Table 474.

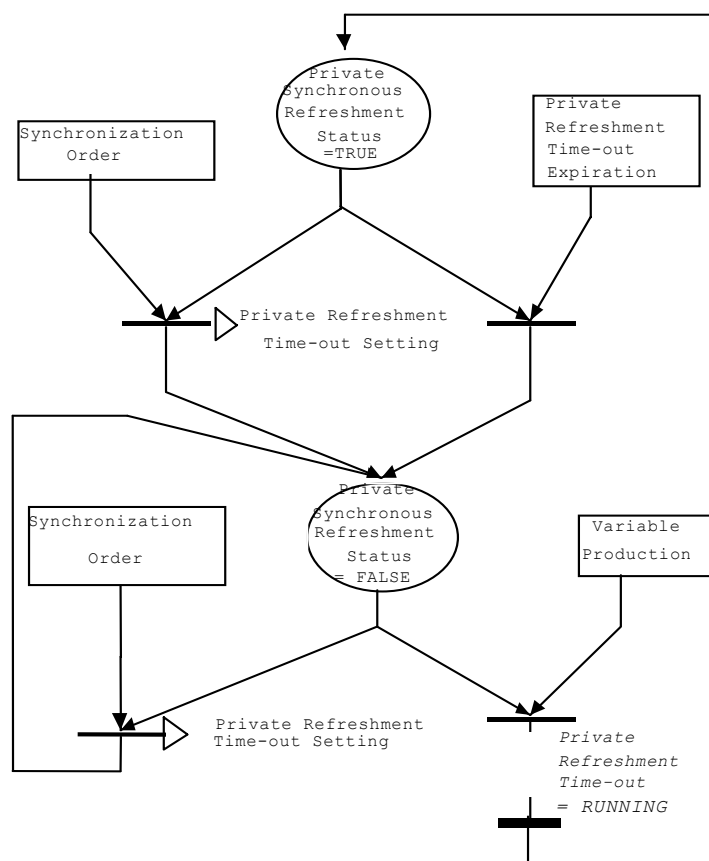


Figure 119 – Synchronous refreshment private mechanism evaluation net

Table 474 – Synchronous refreshment private mechanism events and actions

Initial state:	Private synchronous refreshment status = FALSE
	State of the private refreshment time-out = IDLE
Requests:	<u>Variable Production:</u> Invocation of the services A_WRITELOC, A_WRITEFAR or A_WRITE by the user <u>Private Refreshment Time-out Expiration:</u> Switch of the private time-out associated with the private synchronous status from the RUNNING state to the IDLE state. <u>Synchronisation Order:</u> Occurrence of a synchronization order associated with the public and private synchronous refreshment status.
Action:	<u>Private Refreshment Time-out Setting</u> Switch of the private time-out associated with the private asynchronous status from the <i>OUT</i> state to the <i>SET</i> state, with the preset value equal to the Production <i>period</i> attribute value.

Public mechanism:

The time-out associated with the public mechanism is preset with the current value of the private time-out and set on the occurrence of the resynchronization order.

The public mechanism to elaborate the synchronous refreshment status of a resynchronized variable is represented by the following evaluation net, shown in Figure 120 and Table 475.

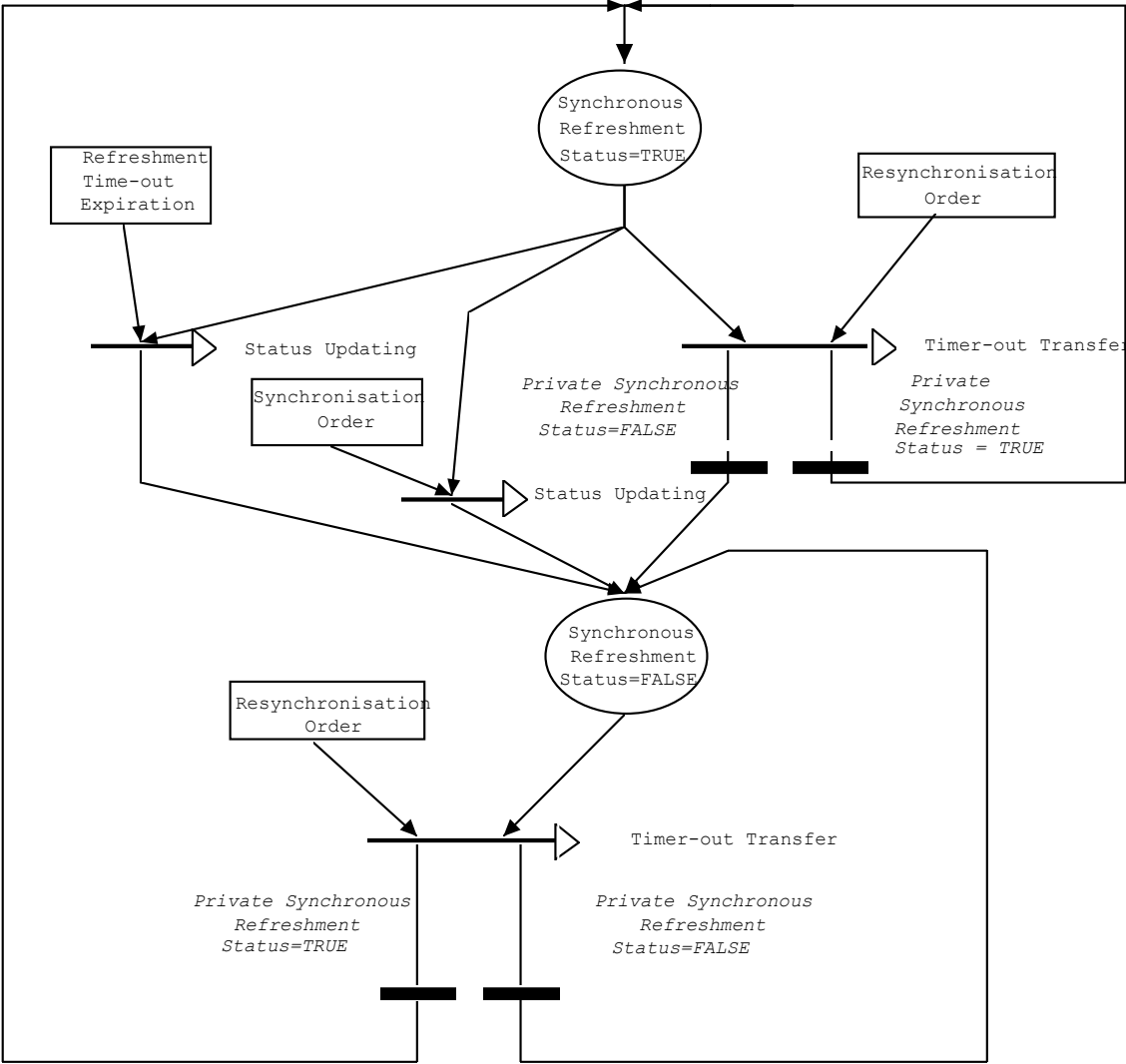


Figure 120 – Synchronous refreshment public mechanism evaluation net

Table 475 – Synchronous refreshment public mechanism events and actions

Initial state:	Synchronous refreshment status = FALSE
	State of the refreshment time-out = IDLE
Requests:	<u>Refreshment Time-out Expiration:</u> Switch of the public time-out associated with the synchronous refreshment status from the RUNNING state to the IDLE state.
	<u>Resynchronization Order:</u> Occurrence of the synchronization order associated with the public resynchronisation mechanism.
	<u>Synchronization Order:</u> Occurrence of the synchronisation order associated with the refreshment status and to the private synchronous refreshment state.
Action:	<u>Time-outs Transfer:</u> Transfer in the <i>State</i> attribute value and <i>Current</i> attribute value of the private time-out of the private asynchronous refreshment status , of the attributes <i>State</i> and <i>Preset</i> belonging to the public time-out associated with the refreshment status.

12.2.1.3.6.2.2.6 Mechanism associated with the punctual refreshment

The mechanism which elaborates the punctual refreshment for a resynchronized variable may be described with two evaluation nets one of which describes the private punctual refreshment status associated with the private variable value, and the other the punctual refreshment status associated with the public variable value.

Private mechanism:

The private time-out associated with the punctual refreshment is preset with the production time slot of the variable and set at the occurrence of the synchronization order associated with this status, as shown in Figure 121 and Table 476

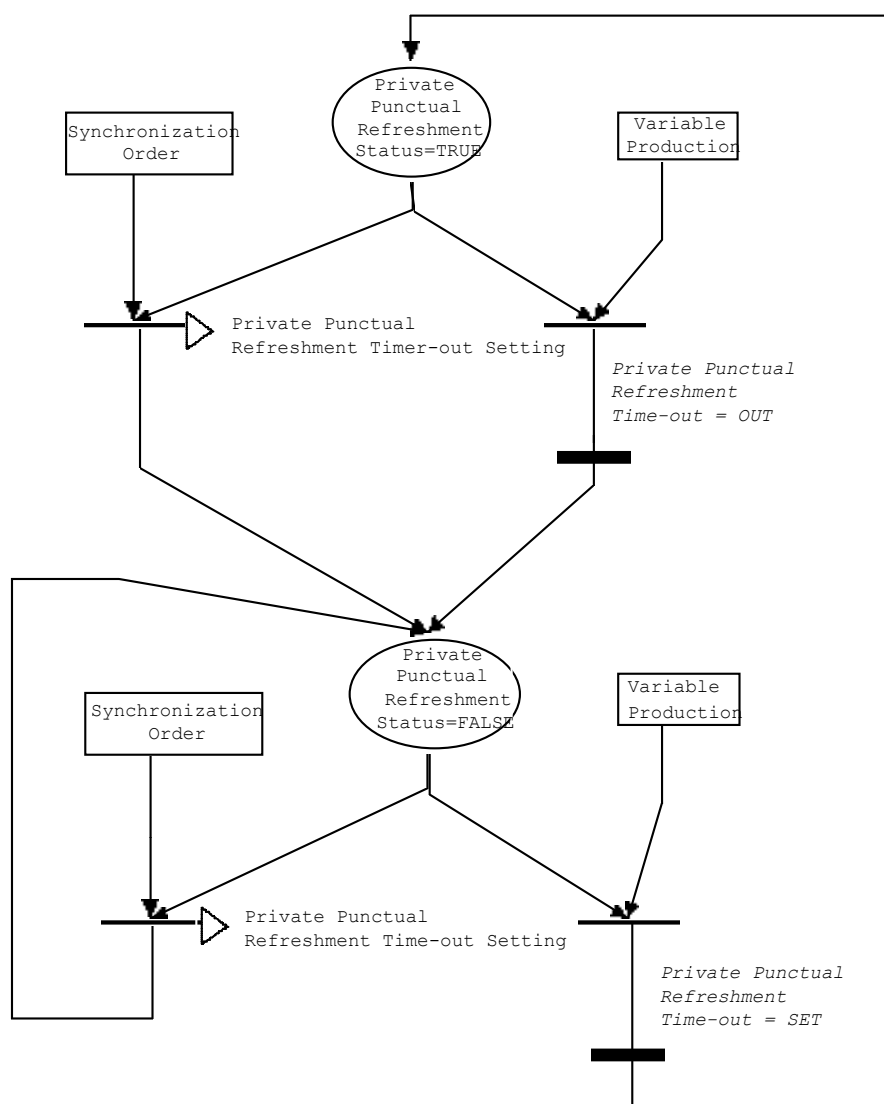


Figure 121 – Punctual refreshment private mechanism evaluation net

Table 476 – Punctual refreshment private mechanism events and actions

Initial state:	Private punctual refreshment status = FALSE
	State of the private punctual refreshment time-out = IDLE
Requests	<u>Variable Production:</u> Invocation of services A_WRITELOC, A_WRITEFAR or A_WRITE by the user
	<u>Synchronization Order:</u> Occurrence of the synchronization order associated with the punctual refreshment status and to the private punctual refreshment.
Action:	<u>Private Punctual Refreshment Time-out Setting</u> Switch of the private time-out associated with the private punctual refreshment status from the IDLE state to the RUNNING state, with the preset value equal to the <i>Production time slot</i> as attribute value.

Public mechanism:

The public mechanism which elaborates the punctual refreshment status of a resynchronized variable is represented by the following evaluation net, shown in Figure 122 and Table 477.

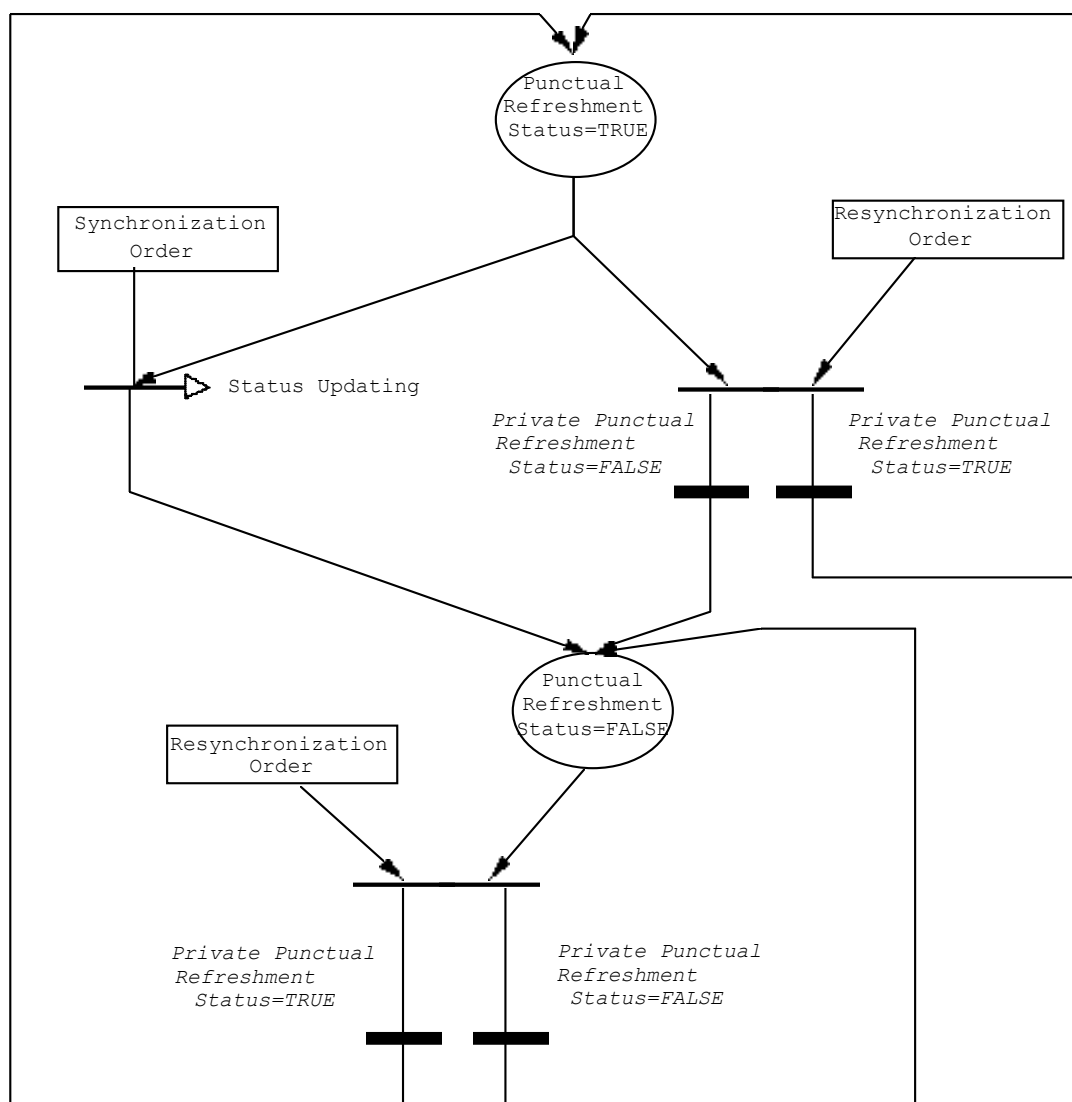
**Figure 122 – Punctual refreshment public mechanism evaluation net**

Table 477 – Punctual refreshment public mechanism events and actions

Initial state	Punctual refreshment status = FALSE
Requests:	<u>Resynchronization Order</u> Occurrence of the synchronization order associated with the public resynchronization mechanism. <u>Synchronization Order:</u> Occurrence of synchronization order associated with the punctual refreshment status and to the private punctual refreshment status.

The elaboration of synchronous refreshment status and punctual refreshment status requires specific precautions with regard to the respective choice of the synchronization variables associated with synchronous status and those associated with the resynchronization.

NOTE The specific choice consisting in using the same synchronization variable results in status that will be FALSE whenever the user performs a write.

The choice of two distinct synchronization variables, which transmission on the bus is separated by a time interval smaller than the production periods or the production time slots, may lead to status FALSE if the resynchronization order arrival time precedes the user writing time.

12.2.1.3.6.2.3 Resynchronization of a consumed variable

12.2.1.3.6.2.3.1 Principle

If the network provides a variable value in the public buffer (TPU) with a T_r transmission period, and at the same time an AP consumes this value in the private buffer (TPP), then the transfer of the variable value from the public buffer to the private buffer is performed on the reception of a resynchronization order with a T_c period.

The consumer resynchronization mechanism holds in the private buffer the same value of the consumed variable during the time interval between two occurrences of the same resynchronization order. Figure 123 shows these principles.

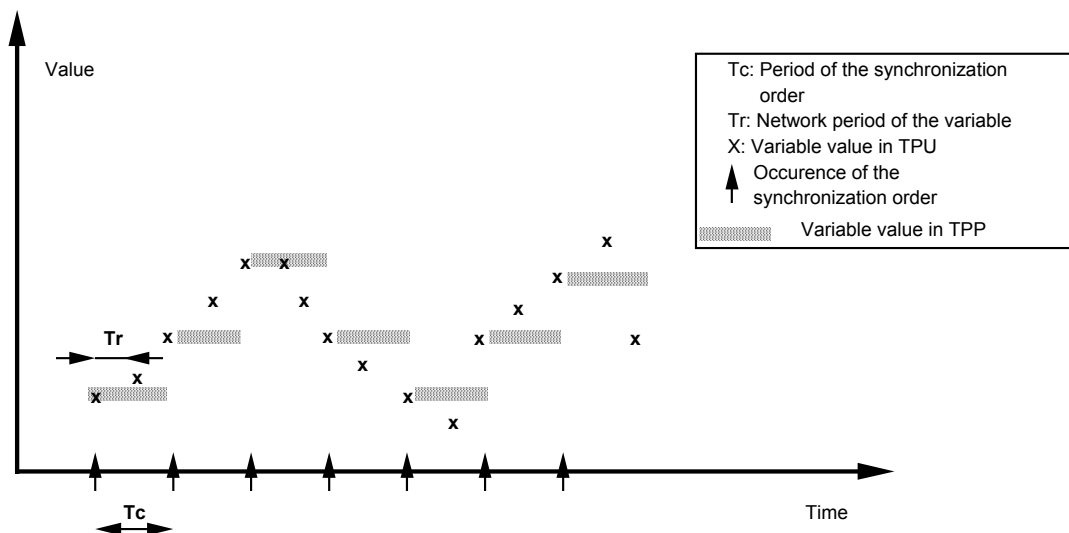


Figure 123 – Principles for the resynchronisation of a consumed variable

NOTE The resynchronization mechanism of a consumed variable is useful only when $T_r \ll T_c$

The resynchronization service implies the use of extra resources like additional buffers for private values associated with the public values of the resynchronized variables.

With each public buffer of a resynchronized variable a time-out of promptness status elaboration time-out, and punctual promptness elaboration time-out, are optionally associated.

The resynchronization service requires the use of a private time-out associated with the private buffer in order to allow the transmission status to preserve their semantics when the variables are resynchronized.

The elaboration of the transmission validity status for a resynchronized variable at the consumer entity level involves the use of two mechanisms associated respectively with the public and private values of the resynchronized variable.

12.2.1.3.6.2.3.2 Specification of the resynchronization in consumption class

This class describes additional resources used to resynchronize a consumed variable at an application entity level.

The class used to describe a resynchronized variable within consuming entity inherits from attributes of the *Resynchronization in consumption* class.

Class: RESYNCHRONIZATION IN CONSUMPTION

Attribute	:	Private value
Constraint	:	Promptness elaborated = TRUE
Attribute	:	Private Promptness status
Attribute	:	Inherit from Time-out
Constraint	:	Punctual Promptness elaborated = TRUE
Attribute	:	Private Punctual Promptness status
Attribute	:	Reference Synchronization variable

Private value:

The resynchronization principle is based on data which is double buffered. It is necessary to add a private value to the public value of the variable. The private value may now be read asynchronously.

Promptness elaborated:

This attribute, both in the private context and in the public context, conditions the existence of a promptness status associated with a variable.

When this attribute is TRUE a resynchronized variable has the use of a promptness status and a private promptness status.

This conditional attribute is declared in the *Consumed variable* class.

Private promptness status

This private status corresponds to internal information from the consuming entity of the resynchronized variable, from which is built the value of the promptness status.

The characteristic of this status is SYNCHRONOUS or ASYNCHRONOUS according to the global declaration associated with resynchronized variable in consumption which is made in the *Promptness* class.

In the case of a private synchronous status the synchronization variable used by the private mechanism associated with this status is declared globally in the *Promptness* class.

Inherit from Time-out:

This set of resources allows the user to maintain the semantics of the promptness status associated with a resynchronized variable.

In fact, the use of a synchronous or asynchronous promptness status requires the use of an extra time-out to preserve the whole semantics of this status.

Punctual Promptness elaborated

This attribute conditions globally in the private context and in the public context the existence of a punctual promptness status associated with a variable.

When this attribute is TRUE, a resynchronized variable has the use of a punctual promptness status and a private punctual promptness status.

This conditional attribute is declared in the *Consumed variable* class.

Private Punctual Promptness status

This private status corresponds to internal information from the consuming entity of the resynchronized variable, from which is built the value of the punctual promptness status.

In the case of a private punctual status the synchronization variable used by the private mechanism associated with this status is declared globally in *Punctual Promptness* class

Reference (Synchronization) variable:

The reference to synchronization variable indicates the resynchronization order associated with a resynchronized variable.

This resynchronization order conditions the transfer of variable values, transmission statuses and current values of time-outs from the private domain to the public domain. This reference indicates the *A_Name* of the *Consumed Variable Local Image* object.

12.2.1.3.6.2.3.3 Mechanism associated with the consumed value

The resynchronization mechanism in consumption of the local variable value is the transfer from the private buffer to the public buffer, on the resynchronization order.

The evaluation net of this service element which manages the resynchronization actions within a consuming application entity for a resynchronized variable is shown in Figure 124.

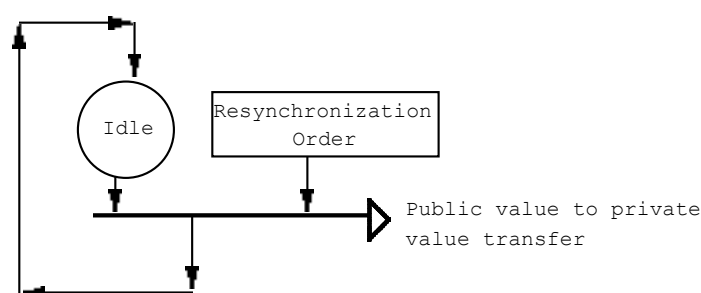


Figure 124 – Resynchronisation mechanism state machine for consumed variable

12.2.1.3.6.2.3.4 Mechanism associated with asynchronous promptness

The mechanism which elaborates the asynchronous promptness status for a resynchronized variable may be described with two evaluation nets one of which describes the private asynchronous promptness and the other the asynchronous promptness status associated with the public value of the variable.

Public mechanism:

The public time-out associated with the promptness is preset with the consumption period of the variable and set at the receiving time of a new public variable value, as shown in Figure 125 and Table 478.

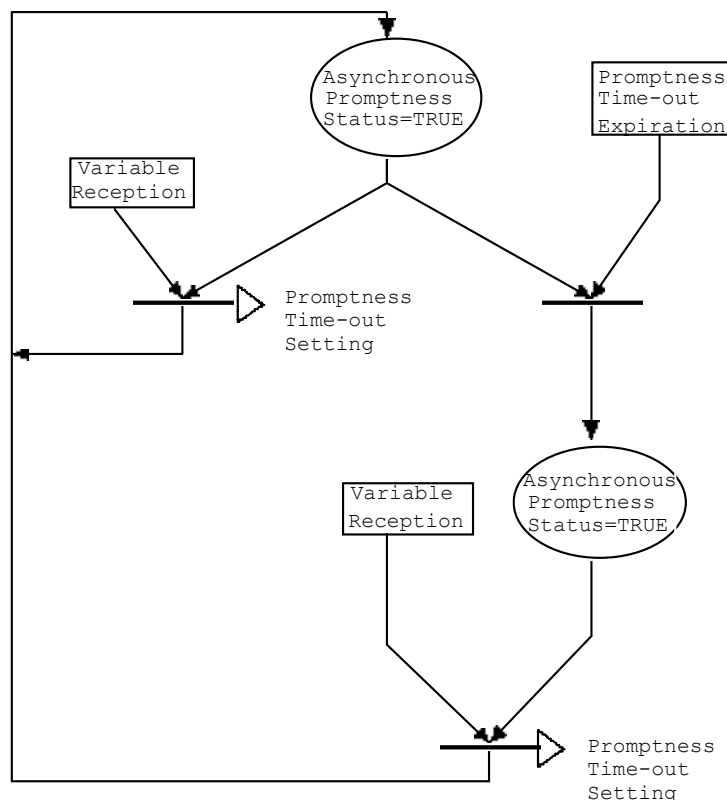


Figure 125 – Asynchronous promptness public mechanism evaluation net

Table 478 – Asynchronous promptness public mechanism events and actions

Initial state:	Asynchronous promptness status = IDLE
	State of the promptness time-out = OUT
Requests:	<u>Variable Reception:</u> Updating of the variable value by the network
	<u>Promptness Time-out Expiration:</u> Switch of the public time-out associated with the public asynchronous promptness status from the RUNNING to the IDLE state.
Actions:	<u>Promptness Time-out Setting:</u> Switch of the public time-out associated with the public asynchronous status from IDLE state to RUNNING state, with the preset value equal to the <i>Consumption period</i> attribute value.

Private mechanism:

The private time-out is preset with the current value of the public time-out and set on the resynchronization order associated with this status.

The mechanism to elaborate the asynchronous private promptness status of a resynchronized variable is represented by the following evaluation net, shown in Figure 126 and Table 479.

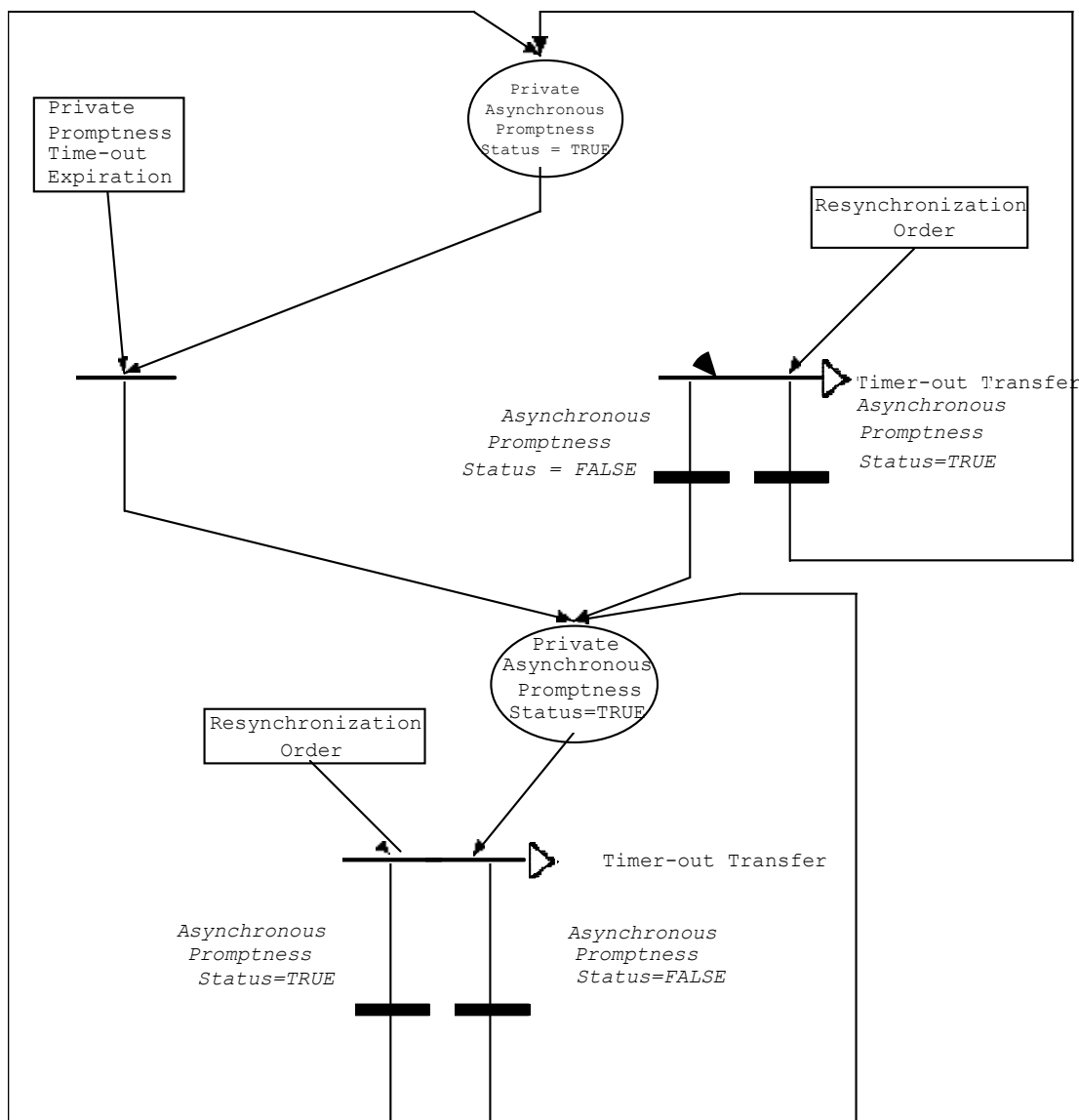


Figure 126 – Asynchronous promptness private mechanism evaluation net

Table 479 – Asynchronous promptness private mechanism events and actions

Initial state:	Private asynchronous promptness status = FALSE State of the private promptness time-out = IDLE
Requests:	<u>Private Promptness Time-out Expiration</u> Switch of the private time-out associated with the private asynchronous promptness status from the RUNNING state to the IDLE state. <u>Resynchronization Order</u> Occurrence of a synchronization order associated with the resynchronization mechanism.
Action:	<u>Time-outs Transfer</u> Transfer in the State attribute value and Current attribute value of the public time-out of the public asynchronous promptness status, of the attributes State and Preset belonging the private time-out associated with the asynchronous promptness status.

12.2.1.3.6.2.3.5 Mechanism associated with synchronous promptness

The mechanism which elaborates the synchronous promptness status for a resynchronized variable may be described with two evaluation nets one of which describes the private synchronous promptness status and the other the synchronous promptness status associated with the public value of the variable.

Public mechanism:

The public time-out associated with the promptness is preset with the consumption period of the variable and set at the receiving time of a new synchronization order, as shown in Figure 127 and Table 480.

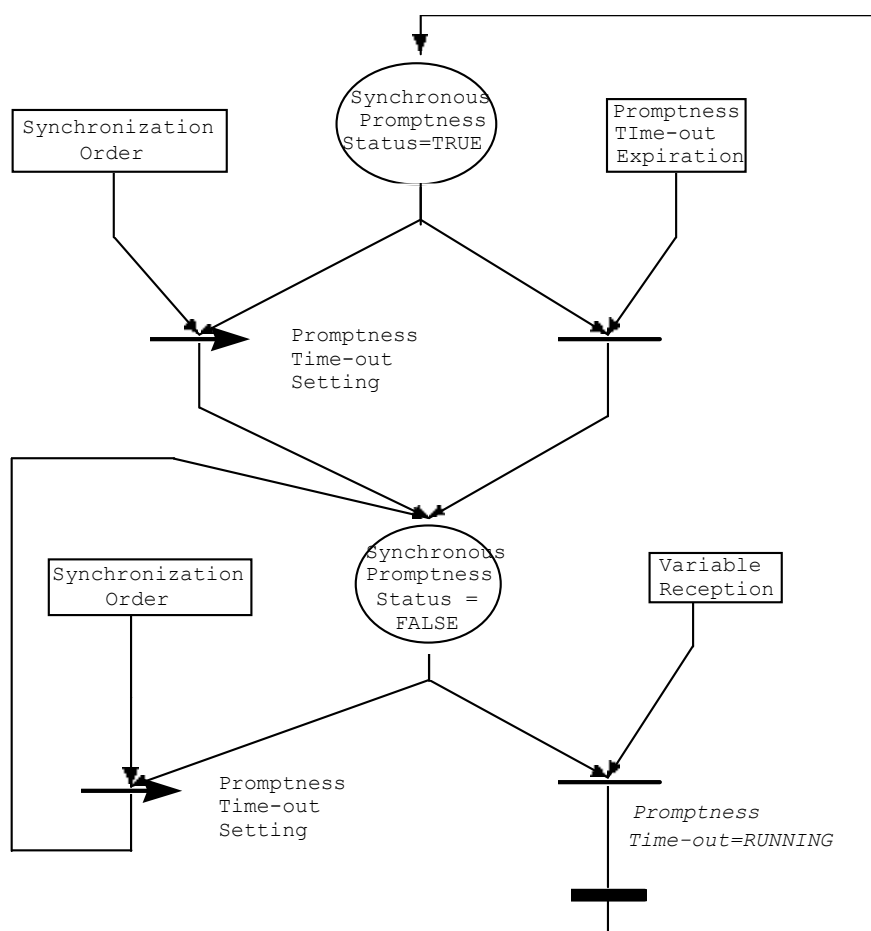


Figure 127 – Synchronous promptness public mechanism evaluation net

Table 480 – Synchronous promptness public mechanism events and actions

Initial state:	Synchronous promptness status = FALSE State of the promptness time-out = IDLE
Requests:	<u>Variable Reception:</u> Updating of the variable value by the network <u>Synchronization Order:</u> Occurrence of a synchronization order associated with the promptness status and to the private synchronous promptness status. <u>Promptness Time-out Expiration:</u> Switch of the public time-out associated with the synchronous promptness status from the RUNNING state to the IDLE state.
Action:	<u>Promptness Time-out Setting:</u> Switch of the public time-out associated with the public synchronous status from IDLE state to the RUNNING state with the preset value equal to the Consumption period attribute value.

Private mechanism:

The private time-out is preset with the current value of the public time-out and set on the Occurrence of the resynchronization order.

The private mechanism to elaborate the synchronous promptness status of a resynchronized variable is represented by the following evaluation net, shown in Figure 128 and Table 481.

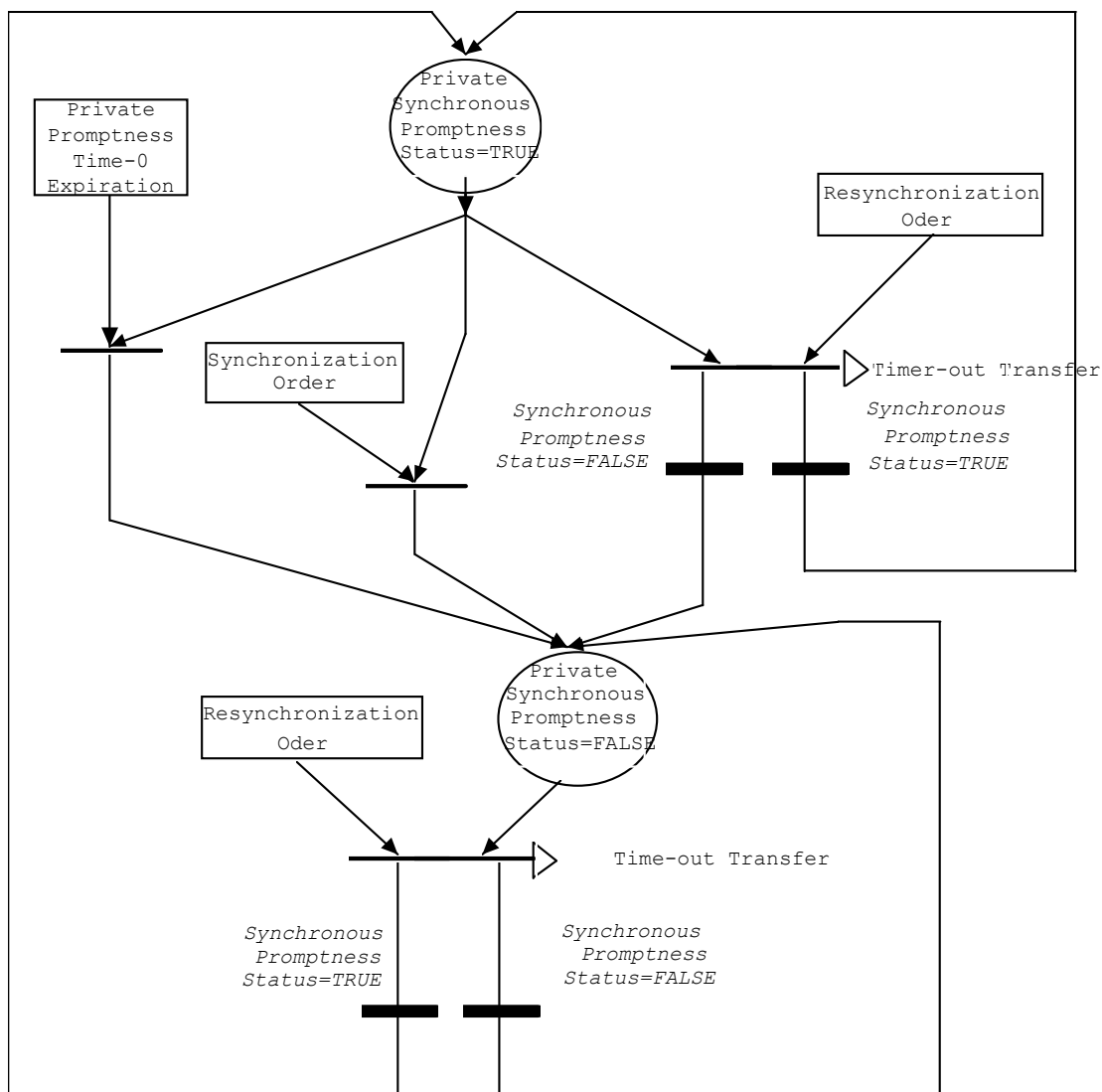


Figure 128 – Synchronous promptness private mechanism evaluation net

Table 481 – Synchronous promptness private mechanism events and actions

Initial state:	Private synchronous promptness status = FALSE
	State of the private promptness time-out = IDLE.
Requests:	<u>Private Promptness Time-out Expiration</u> Switch of the private time-out associated with the synchronous promptness from the RUNNING state to the IDLE state.
	<u>Resynchronization Order</u> Occurrence of a synchronization order associated with the resynchronization mechanism
	<u>Synchronization Order</u> Occurrence of a synchronization order associated with the promptness status and with the private synchronous promptness
Action:	<u>Time-outs Transfer:</u> Assignment of the attributes value State and Current value of the public synchronous promptness status, to the attributes State and Preset of the private timer associated with the synchronous promptness status.

12.2.1.3.6.2.3.6 Mechanism associated with the punctual promptness

The mechanism which elaborates the punctual promptness for a resynchronized variable may be described with two evaluation nets which one described the private punctual promptness status associated with the private value, and the other the punctual promptness status associated with the public variable value.

Public mechanism:

The public time-out associated with the punctual promptness is preset with the consumption time slot of the variable and set at the occurrence of the synchronization order associated with this status, as shown in Figure 129 and Table 482.

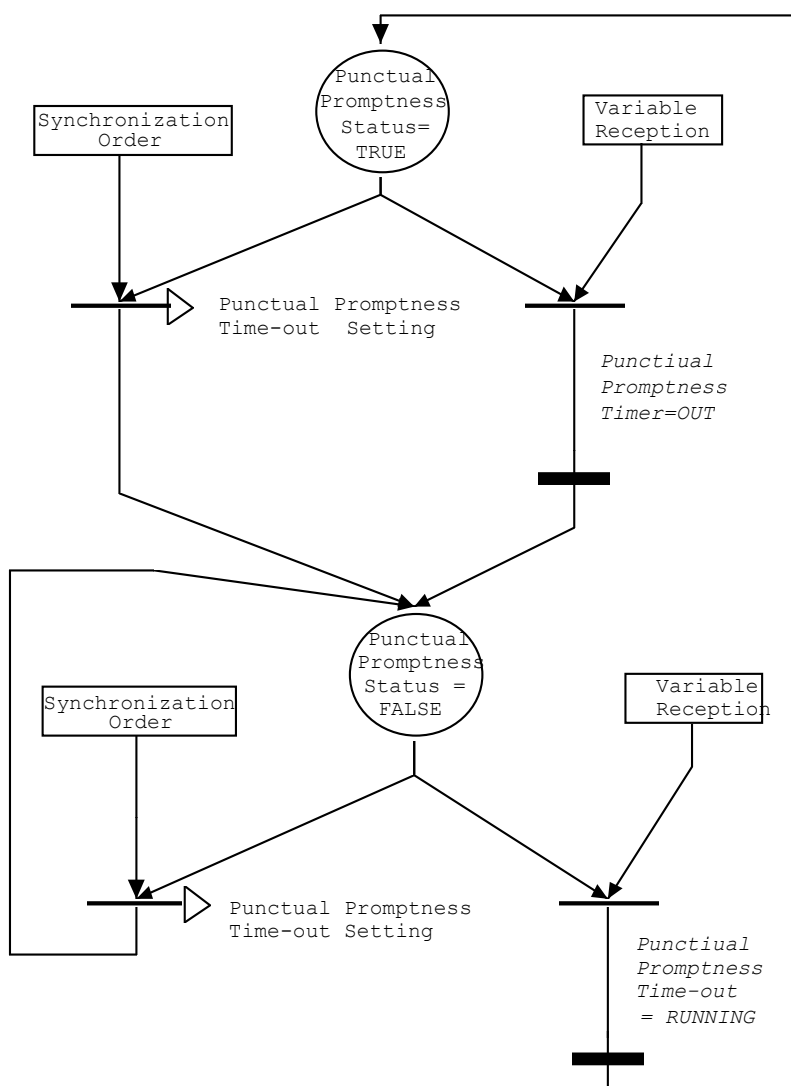


Figure 129 – Punctual promptness public mechanism evaluation net

Table 482 – Punctual promptness public mechanism events and actions

Initial state	Punctual promptness status = FALSE State of the promptness time-out = OUT
Requests:	<u>Variable Reception:</u> Updating of the variable value by the network. <u>Synchronization Order:</u> Occurrence of a synchronization order associated with the punctual promptness status and to the private punctual promptness status.
Action:	<u>Punctual Promptness Time-out Setting</u> Switch of the public time-out associated with the public punctual promptness status from the IDLE state to the RUNNING state, with the preset value equal to the <i>Consumption time slot</i> attribute value.

Private mechanism:

The private mechanism which elaborates the punctual promptness status of resynchronized variable is represented by the following evaluation net, shown in Figure 130 and Table 483.

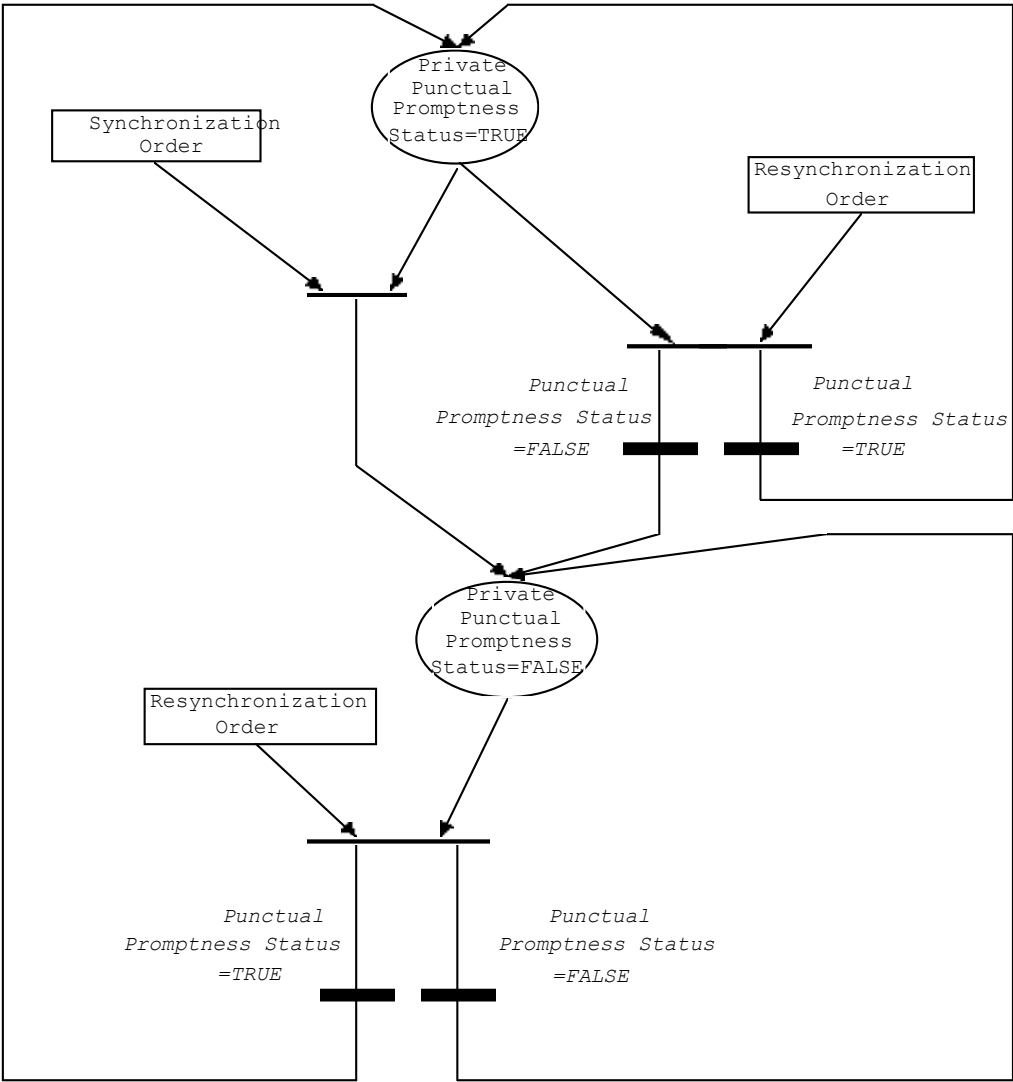


Figure 130 – Punctual promptness private mechanism evaluation net

Table 483 – Punctual promptness privatemechanism events and actions

Initial state	Private punctual promptness status = FALSE
Requests	<u>Resynchronization Order:</u> Occurrence of asynchronization order associated with the resynchronization mechanism. <u>Synchronization Order:</u> Occurrence of the synchronization order associated with the punctual promptness status and to the private punctual promptness status.

The elaboration of a synchronous promptness and a punctual promptness status requires specific precautions for respective choices of the synchronization variables associated with synchronous status and those associated with the resynchronization.

NOTE The specific choice consisting in using the same synchronization variable results in status that will be FALSE whatever the time of the network updating.

The choice of two distinct synchronization variables, for which transmission on the bus is separated by a time interval smaller than the consumption period or the consumption time slot, may lead to status FALSE if the time of the arrival of the resynchronization order precedes the updating time by the network.

12.2.1.4 Variable lists access sub-ASE

12.2.1.4.1 Concepts

12.2.1.4.1.1 List definition

The application layer supports definition and handling of variable lists.

A list is an ordered set of variables of NORMAL class. A variable list cannot include variables of SYNCHRONIZATION or DESCRIPTION classes.

A variable list is globally defined and instantiated at the application entity level. It is composed exclusively of consumed variables.

Several lists can be defined within a single system, but they shall all be separated two by two:

$$\forall i,j \text{ list}_i \cap \text{list}_j = \emptyset$$

However ; several instances of a same list can be defined, within a system, because several APs can be consumers of the same list.

A variable list defined in this way can only be handled by an appropriate read service permitting global access to the whole list. However, all the variables belonging to a list can also be handled individually by the variable read services.

NOTE There is no support for the concept of a recursive list at the level of the application services provided to the user. (Therefore, there is no list of list).

The variables declared by the user as belonging to a variable list cannot be grouped together into an structure, because it is necessary for them to be optimisation provided with individual validity information.

Most of the variable lists implemented in the MPS environment are composed of periodic variables having the same period, but lists of non periodic variables are also envisaged.

12.2.1.4.1.2 Status associated with a list

At the time of a list read, the user can get additional information relative to the variable consistency of this list. These status are elaborated or not according to the configuration associated to the list. They inform the user about the integrity of all the variables composing the list.

These status are returned by the list access service.

In the MPS environment the various status associated with the variable lists inform the user regarding:

- the TRANSMISSION CONSISTENCY of the list
- the PRODUCTION CONSISTENCY of the list
- the PUNCTUAL TRANSMISSION CONSISTENCY of the list
- the PUNCTUAL PRODUCTION CONSISTENCY of the list
- the SPATIAL CONSISTENCY of the list

All these status associated with a list by a user are elaborated using:

- information local to a consuming AE, and relative to the transmission validity of the variables of the list
- information elaborated by the remote producers of the variables of the list, and relative to their production validity.

The information relative to the transmission validity, involved in the elaboration of the consistency status for each of the list variables is:

- the asynchronous or synchronous promptness
- the punctual promptness

The information relative to the production validity involved in the elaboration of the consistency status for each of the list variables is:

- the asynchronous or synchronous refreshment
- the punctual refreshment

NOTE the production validity information elaborated within the producing entities of the list variables is carried on the network with the values of the variables.

12.2.1.4.1.3 Control elements and reliability

The reliability of transmission of the variables of a list can be enhanced by using a recovery mechanism ; This mechanism performs a retransmission request for a number of times limited to a maximum defined for each of the instances of this list.

The evaluation of the spatial consistency status is locally derived from a purely logical function based on the consistency variable values.

The concept of spatial consistency is based on:

- The existence of a consistency variable produced by every subscriber consuming a local image of the list, as well as on the existence of an interchange mechanism of these consistency variables between the different subscribers.
- The existence of a local mechanism of value management of the produced consistency variable.
- The definition of a certain sequential ordering action for the interchange of the variables of the list, and the consistency variables.

Sequential ordering of the variable interchange on network

In order to elaborate a spatial consistency status on a variable list, it is necessary to define a cycle corresponding to a transmission of all the list variables in order to be able to set up a temporal reference for the consistency variable management.

This cycle is delimited by a V_s synchronization order whose occurrence defines the beginning of a new variable interchange cycle. All the list variables of the list need to be received and all the consistency variables need to be interchanged between two consecutive occurrences of the V_s synchronization order. This is shown in Figure 131.

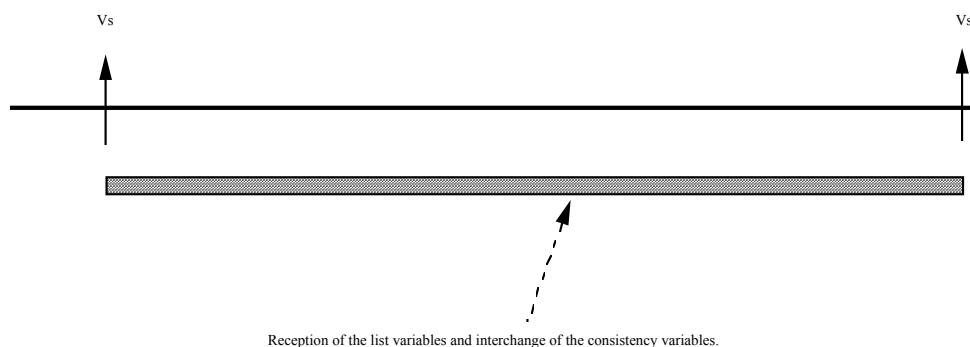


Figure 131 – Spatial consistency list variables interchange mechanism

Moreover, a condition of using the lists requires that the consistency variable interchange be performed after interchange of all the list variables, and within the current cycle, as shown in Figure 132.

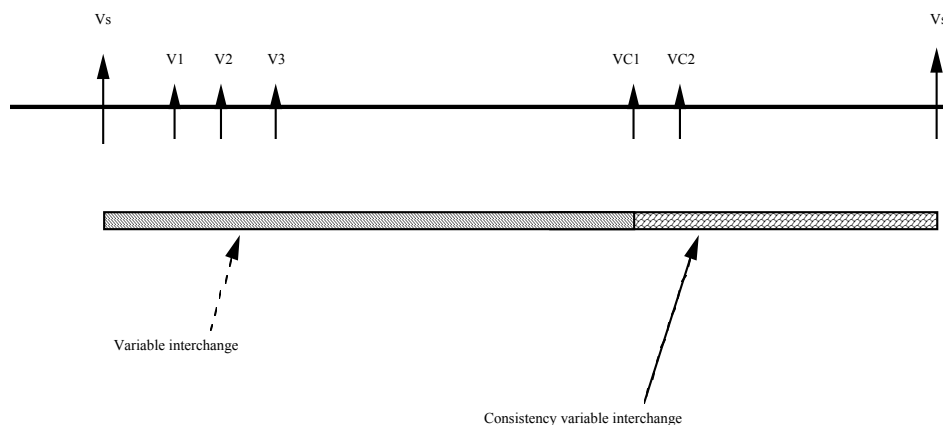


Figure 132 – Spatial consistency – consistency variable interchange mechanism

When the spatial consistency status elaboration mechanism is involved with a recovery mechanism, as shown in Figure 133, the eventual recovery on variables concerned by transmission errors need to be performed after the theoretical reception of all the list variables, and before the consistency variable interchange. This makes the recovery mechanism and the spatial consistency mechanism meaningful.

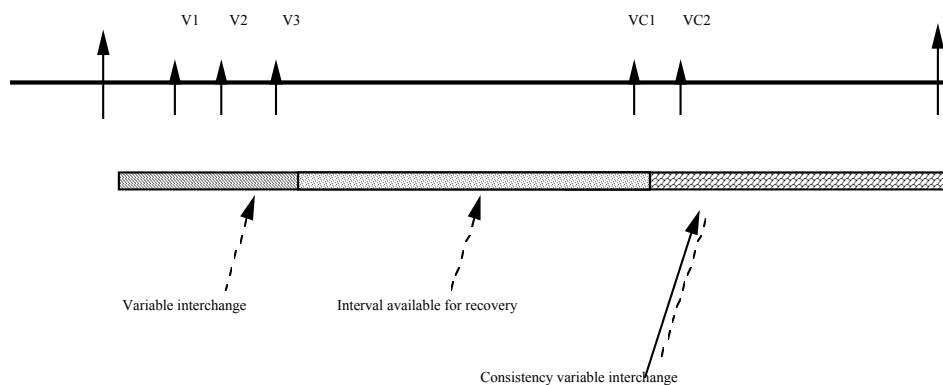


Figure 133 – Spatial consistency – list recovery mechanism

Finally, the spatial consistency status locally elaborated inside an AE and made available to the user is significant only when all the consistency variables have been interchanged. This leads to defining a validity interval of the spatial consistency status, as shown in Figure 134.

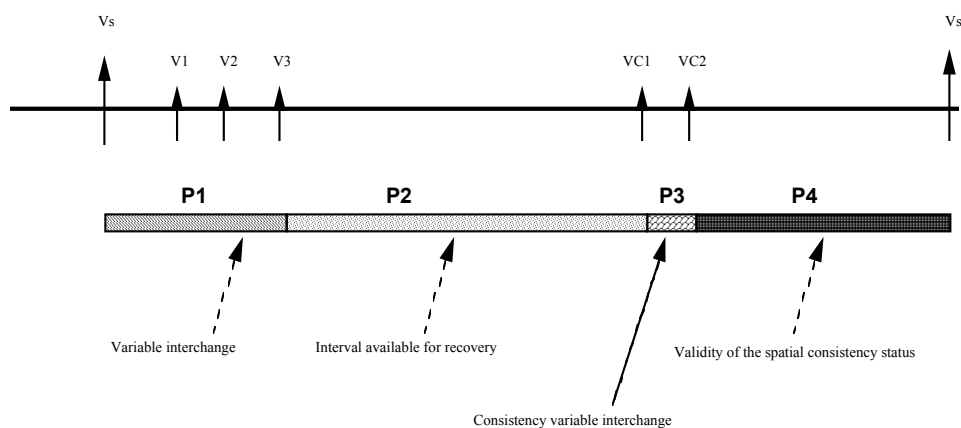


Figure 134 – Spatial consistency – validity of the spatial consistency status

12.2.1.4.1.4 Object model of a variable list

A variable list in the MPS environment is modelled by using the *Variable List Local Image* object which contains all the attributes required by an application entity consuming this variable list.

This object comes from the instantiation of the *Identified Variable List* class which contains all the attributes characterizing a variable list.

The common attributes of a variable list in the different entities, described in the *Identified variable List* class come from the instantiation of a *Variable List* generic class which describes the whole of the generic attributes of a variable list.

Moreover, it is pointed out that the *Variable List* generic class refers to the *Variable* generic class in order to describe the variables composing this variable list. This is all shown in Figure 135.

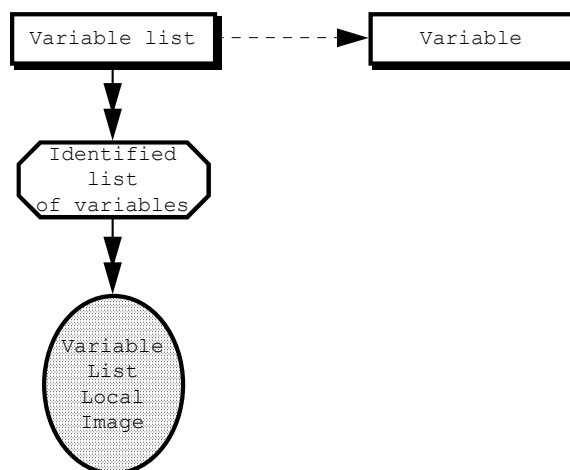


Figure 135 – Object model of a variable list

12.2.1.4.2 Variable list class specification

12.2.1.4.2.1 Variable List generic class specification

12.2.1.4.2.1.1 Variable list generic class model

FAL ASE:	MPS ASE
CLASS: Variable list	
CLASS ID:	not used
PARENT CLASS:	not used
<i>Generic Class:</i>	VARIABLE LIST
CLASS ATTRIBUTES:	
<i>Key-Attribute :</i>	A_Name
<i>Attribute :</i>	List of Reference Consumed Variable
<i>Attribute :</i>	Spatial consistency required [TRUE,FALSE]
<i>Attribute :</i>	Recovery required [TRUE,FALSE]
<i>Constraint :</i>	Spatial consistency required = TRUE OR Recovery required = TRUE
<i>Attribute</i>	: <i>Reference</i> (Synchronization) Variable
<i>Constraint</i>	: Spatial consistency required = TRUE
<i>Attribute :</i>	Inconsistency detection [TRANSITORY,PERMANENT,UNDEFINED]
<i>Attribute :</i>	<i>List of</i>
<i>Attribute:</i>	<i>Reference</i> (Consistency) Variable
<i>Constraint :</i>	Recovery required = TRUE
<i>Attribute</i>	: Recovery period
Instance Attributes:	
<i>Attribute :</i>	Transmission consistency required [TRUE,FALSE]
<i>Constraint :</i>	Transmission consistency required = TRUE
<i>Attribute</i>	: Transmission consistency status [TRUE,FALSE]
<i>Attribute :</i>	Production consistency required [TRUE,FALSE]
<i>Constraint :</i>	Production consistency required = TRUE
<i>Attribute</i>	: Production consistency status [TRUE,FALSE]
<i>Attribute :</i>	Punctual transmission consistency required [TRUE,FALSE]
<i>Constraint :</i>	Punctual transmission consistency required = TRUE
<i>Attribute</i>	: Punctual transmission consistency status [TRUE,FALSE]
<i>Attribute :</i>	Punctual production consistency required [TRUE,FALSE]
<i>Constraint :</i>	Punctual production consistency required = TRUE
<i>Attribute</i>	: Punctual production consistency status [TRUE,FALSE]
<i>Constraint :</i>	Spatial consistency required = TRUE

<i>Attribute</i>	: <i>Reference</i> (Consistency) Produced Variable
<i>Attribute</i>	: Spatial consistency status [TRUE,FALSE]
<i>Constraint</i> :	Recovery required = TRUE
<i>Attribute</i>	: Recovery list size
<i>Attribute</i>	: Recovery list contents
<i>Attribute</i>	: Recovery Nature [FRAGMENTED,INDIVISIBLE]
<i>Attribute</i>	: Recovery List Identifier
<i>Attribute</i>	: <i>Inherit from</i> Time-out

12.2.1.4.2.1.2 Attributes

A_Name:

This attribute supports unique identification at the interface between the application layer and the user. The A_Name of a variable list belongs to the MPS addressing space

Moreover, the maximum length of an A_Name of a variable list shall not exceed 14 characters.

List Of Reference Consumed Variable:

This attribute allows a variable list to refer to the constituent variables. This set of references between an *Identified Variable List* class and *Consumed variable* classes is identical for each local image.

Spatial consistency required:

The spatial consistency status associated with a variable list is optional.

A "TRUE" value for this boolean attribute indicates that a spatial consistency status shall be elaborated at the list level.

Recovery required:

The implementation of the recovery mechanism is optional.

A "TRUE" value for this boolean attribute indicates that a recovery mechanism shall be implemented for the variable list.

Reference (synchronization) Variable:

The variable lists refer to a synchronization order when a spatial consistency status is elaborated or when the recovery mechanism is implemented. This synchronization order is used on one hand by the service elements of various subscribers which elaborate the values of the consistency variables which are involved in the spatial consistency status elaboration, and on the other hand by the service elements that manage locally the recovery mechanism at the level of every application entity.

Inconsistency detection:

This attribute specifies the type of the consistency variables that are involved in the elaboration of the spatial consistency status.

The inconsistency detection is TRANSITORY when the consistency variable value type takes into account the transmission faults during the last interchange.

The inconsistency detection is PERMANENT when the consistency variable value type takes into account the transmission faults of the last 2^{16} previous interchanges.

The inconsistency detection is UNDEFINED when the consistency variable value type is freely defined by the user for an elaboration of a spatial consistency status in his user application.

Reference (consistency) Variable:

This set of references to consistency Variables locally supports the transmission status of the variables of the list within the other application entities consuming this list.

A consistency variable characterizes a variable list at the level of a particular application entity. It supplies information regarding the transmission validity of each variable of this list from the transmission of the last synchronization variable associated to the list.

Recovery period:

This attribute represents the maximum duration allowed between two reinitialisations of the contents of the recovery list.

This period is expressed in milliseconds.

Transmission consistency required:

Transmission consistency associated with a variable list is optional.

When it has the value "TRUE" this attribute indicates that a transmission consistency status shall be elaborated for this list.

Transmission consistency status:

The transmission consistency status provided to the user is of type boolean.

This status is elaborated at the level of every application entity receiving variables of a list defined with a transmission consistency status.

Production consistency required:

Production consistency associated with a variable list is optional.

When it has the value TRUE this attribute indicates that a production consistency status shall be elaborated for this list.

Production consistency status:

Production consistency is a status provided to the user.

This status of type Boolean is elaborated at the level of every application entity receiving the variables of a list with a production consistency status.

Punctual transmission consistency required:

Punctual transmission consistency associated with a variable list is optional.

When it has the value TRUE this attribute indicates that a punctual transmission consistency status shall be elaborated for this list.

Punctual transmission consistency status:

Punctual transmission consistency is a status provided to the user.

This status of type Boolean is elaborated at the level of every application entity receiving the variables of a list defined with a punctual transmission consistency status.

Punctual production consistency required:

The punctual production consistency associated with a variable list is optional.

This attribute of boolean type indicates that a punctual production consistency status shall be elaborated for this list when it has the value TRUE.

Punctual production consistency status:

Punctual production consistency is a status provided to the user.

This status of type boolean is elaborated at the level of every application entity receiving the variables of a list defined with a punctual production consistency status.

Spatial consistency status:

Spatial consistency is a status provided to the user.

This status of type boolean is elaborated at the level of every application entity receiving variables of a list defined with a spatial consistency status.

Reference (consistency) Produced Variable:

This reference designates the consistency variable locally produced and transmitted to the other application entities consuming the list in order to inform them about the transmission operation of the variables locally within the entity producing the consistency variable.

Recovery list size:

This attribute characterizes the maximum number of elements of the variable list for which a retransmission will be requested during the same cycle. The size of the recovery list is always smaller or equal to the number of elements which compose the variable list.

Recovery list contents:

This attribute characterizes a set of identifiers associated with elements of a variable list that were not successfully received after the last reception on the network of the synchronization variable associated to this list.

These contents are not accessible to the user, but they are directly interpretable by a data link service element to recover the failed transmissions during the last interchange.

The contents of a recovery list is used to perform a transmission recovery request of the maximum number of elements defined by the recovery list size.

The encoded value of the recovery list is empty in case of non recovery, i. e. when all the variables of the variable list have been correctly transmitted during the last interchange.

Recovery list identifier:

This attribute specifies the identifier with which the contents of the recovery list are associated. This identifier references in a unique way this recovery list associated to a local image of a variable list within a MPS application entity.

Inherit from Time-out:

The management of the recovery list contents takes into account the event associated with the time-out expiration which indicates a time which is too long without initialization of the recovery list, as well as an event associated with the occurrence of the synchronization order which initializes the recovery list contents. This time-out is set upon occurrence of the recovery list synchronization and reset automatically when it is over.

Recovery nature:

This attribute defines the quality of service associated with the recovery_mechanism on a variable list. When this attribute has value INDIVISIBLE, the recovery is performed immediately after transmission of the recovery list contents on the network. When this attribute has the value FRAGMENTED the recovery is delayed in a time slot reserved for aperiodic traffic ; moreover, it shall be performed only according to the network load.

12.2.1.4.2.2 **Identified Variable List class specification**

The *Identified Variable List* class comes from the instantiation of the *Variable List* generic class

12.2.1.4.2.3 **Variable List Local Image object specification**

The *Variable List Local Image* object comes from the instantiation of the *Identified Variable List* class

12.2.1.4.3 **Services**

12.2.1.4.3.1 **List read service**

12.2.1.4.3.1.1 **Function**

This function allows the user to perform a local read of all the values of the variables composing a list locally declared as consumed.

In addition to the list variable values, this service optionally provides various consistency status associated with this variable list.

12.2.1.4.3.1.2 **Service primitives**

The read of a variable list is performed using the A_READLIST service primitives, shown in Table 484:

A_READLIST.Rq (Arguments): List read request

A_READLIST.Cnf (Results): Return of the read value.

Table 484 – A_Readlist service parameters

Parameter name	Req	Cnf
Argument		
List specification	M	M(=)
Result (+)		S
Value		M
Transmission consistency status		C
Production consistency status		C
Punctual consistency status		C
Punctual production consistency status		C
Spatial consistency status		S
Result (-)		S
Type of error		M

Argument

List specification:

This argument corresponds to the A_Name attribute of the *Variable List Local Image* object which is to be accessed.

Result (+)

Value

List of the variable values belonging to the list which is read.

Transmission consistency status

Optional status describing the transmission consistency of all the variables of the list.

Production consistency status

Optional status describing the production consistency of all the variables of the list.

Punctual transmission consistency status

Optional status describing the punctual transmission consistency of all the variables of the list.

Punctual production consistency status

Optional status describing the punctual production consistency of all the variables of the list.

Spatial consistency status

Optional status describing the spatial consistency of all the variables of the list.

Result (-)**Error type**

Description of the various error types returned after a non successful read request:

Error_1: One of the buffers, where is located the value of a variable of the list is not accessible.

Error_2: The list specification does not correspond, within the addressing space, with a list identification declared as consumed in the AP in consideration.

Error_3: The list is declared locally as invalid.

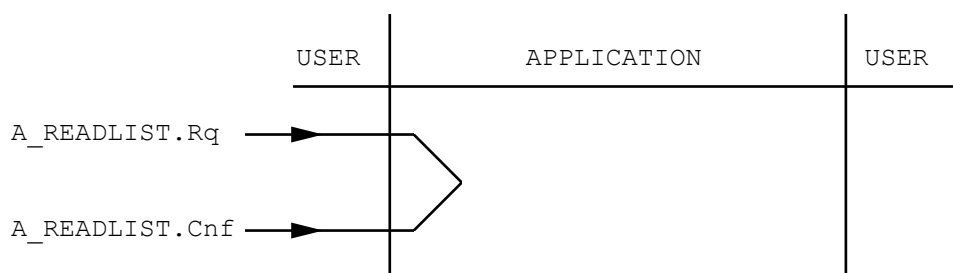
12.2.1.4.3.1.3 Service procedure

Figure 136 – A_Readlist service procedure

This service, shown in Figure 136, provides all the values locally available of the variables of a list at a given time.

The various elaborated status describe the various consistency levels applied to the variables of the list.

Read requests of a list are sequentially processed ; a new request can be initiated only after confirmation of the previous one.

12.2.1.4.4 Variable list consistency and recovery mechanisms**12.2.1.4.4.1 Transmission consistency****12.2.1.4.4.1.1 Description**

Information relative to the transmission consistency of a variable list can be elaborated within a consuming entity, and provided to the user.

This consistency information is provided with two possible semantics:

- the asynchronous transmission consistency informs the consuming user about the respect, for all the variables of this list, of their respective consumption period by the network.
- the synchronous transmission consistency informs the consuming user about the respect, for all the variables of this list, of an availability within their respective consumption period initialized upon occurrence of a single synchronization order associated with the list.

Moreover, these synchronous or asynchronous promptnesses are elaborated with a mandatory value of the *Consumption Period* attribute which is identical for all the variables of this list.

In the case of an asynchronous transmission consistency, all the variables composing this list shall elaborate an asynchronous promptness status.

In the case of a synchronous transmission consistency, all the variables composing this list shall elaborate a synchronous promptness status.

12.2.1.4.4.1.2 Composition rule

The transmission consistency status is the logical product of the promptness status of the different variables of the list. According to the synchronous or asynchronous characteristics of the promptness status associated with the variables of the list, the resulting transmission consistency status shall represent the transmission consistency which is respectively synchronous or asynchronous.

12.2.1.4.4.2 Production consistency

12.2.1.4.4.2.1 Description

Information relative to the production consistency of a variable list can be elaborated at the level of a consuming entity and made available to the user.

These consistency information are provided with two possible semantics:

- the asynchronous production consistency informs the consuming user about the respect, by all the users producing the variables of this list, of their respective production period.
- the synchronous production consistency informs the consuming user about the respect, by all the users producing the variables of this list, of a refreshment within their respective production period initialized by the producing entity upon occurrence of a synchronization order associated with the variable list.

In the case of an asynchronous production consistency, all the variables composing the list shall elaborate an asynchronous refreshment status, whereas in the case of a synchronous production consistency, all the variables composing the list shall elaborate a synchronous refreshment status. Moreover, these asynchronous and synchronous refreshment status are elaborated with a mandatory value of the *Production Period* attribute which is identical for all the variables of the list.

12.2.1.4.4.2.2 Composition rule

The production consistency status is the logical product of the refreshment status of the various variables of the list.

According to the synchronous or asynchronous characteristics of the refreshment status elaborated by the entities producing the variables of the list, the status shall represent the production consistency which is respectively synchronous or asynchronous.

12.2.1.4.4.3 Punctual transmission consistency

12.2.1.4.4.3.1 Description

Information relative to the punctual transmission consistency of a variable list can be elaborated within a consuming entity and provided to the user.

The punctual transmission consistency status informs the consuming user about the respect of an availability by the network of all the variables of this list within their respective consumption time slot.

These respective consumption time slots are initialized by the consuming entity upon occurrence of the synchronization order associated to the list.

All the variables composing the variable list elaborating a punctual transmission consistency status shall elaborate a punctual promptness status with the same value of the *consumption time slot* attribute.

12.2.1.4.4.3.2 Composition rule

The punctual transmission consistency status is the logical product of the punctual promptness status of all the variables composing this list.

12.2.1.4.4.4 Punctual production consistency

12.2.1.4.4.4.1 Description

Information relative to the punctual production consistency of a variable list can be elaborated within a consuming entity and provided to the user.

The punctual production consistency status informs the user consuming the variable list about the respect, by the all the users producing the variables of this list, of a production within their respective time slot.

All the production time slots of the variables of the list are initialized by the producing entities upon occurrence of the synchronization order associated with the list. The variables composing the variable list elaborating a punctual refreshment consistency status shall elaborate a punctual promptness status with the same value of the *production time slot* attribute.

12.2.1.4.4.4.2 Composition rule

The punctual production consistency status is the logical product of the punctual refreshment status of all the variables composing the list.

12.2.1.4.4.5 Spatial consistency

12.2.1.4.4.5.1 Description

A status relative to the spatial consistency of a variable list can be elaborated at the level of a consuming entity and made available to the user.

This spatial consistency status is a piece of information from which the user will be able to build a consistency status whose semantics are adapted to the needs of his application ; (For example, the semantics of the user consistency status can be the equality of the values of all the multiple copies of the list variables).

The elaboration mechanism of a spatial consistency status relies on the broadcast by all the entities consuming the same list declared with a spatial consistency characteristics of a consistency variable. This consistency variable reflects locally the transmission validity of each of the variables of the list.

A consistency variable is elaborated by each of the entities consuming a single variable list and provided to the other entities consuming this list.

The elaboration of this spatial consistency status requires that each of the AE consuming a single variable list is provided with a reception report of the list variables of the other consuming AEs. To provide reception report, additional interchanges allow every AE to broadcast to the others its consistency variable relative to this same list.

12.2.1.4.4.5.2 Composition rule

At the level of a entity consuming a variable list, the processing associated with the spatial consistency relies on the values of consistency variables associated with this list. the spatial consistency status is elaborated from the comparison of the values of the consistency variables associated with a list.

The spatial consistency status value is:

TRUE: when there is equality between all the values of the consistency variables associated with this list.

FALSE: in all the other cases.

12.2.1.4.4.5.3 Consistency variable specification

When a spatial consistency status is associated with a variable list, it necessary to elaborate a consistency variable at the level of every application entity consuming this variable list.

This consistency variable is provided to the other entities consuming this variable list.

A consistency variable in the MPS environment is modelled in the same way as for the variables by using the following objects:

- a (consistency) *Produced Variable Local Image* object which contains all the attributes required for a producing application entity.
- one or several (consistency) *Consumed Variable Local Image* objects which contain all the attributes required for consuming application entities.

These objects correspond to particular instantiation of the classes describing the variables for which the *Class* attribute of the object has the value 'SYNCHRONIZATION'.

A consistency variable is different from the other variable classes by a reference to the associated variable list, in order that the service element producing this variable can elaborate the corresponding value.

12.2.1.4.4.6 Predetermined values used by the list consistency mechanisms

12.2.1.4.4.6.1 Predetermined attribute value of the *Identified Variable* class

An *Identified Variable* class corresponding to a consistency variable is characterized by predetermined values for the following attributes:

Class:

This attribute has the value: SYNCHRONIZATION

Consistency Variable:

This attribute has the value: TRUE

12.2.1.4.4.6.2 Predetermined attribute value of the *Produced Variable Local Image* class

A *Produced Variable Local Image* object corresponding to a consistency variable is characterized by predetermined values for the following attributes:

Resynchronized Variable:

This attribute has the value: FALSE.

All the other non quoted attributes are not given predetermined values.

12.2.1.4.4.6.3 Predetermined attribute value of the *Consumed Variable Local Image* object

A *Consumed Variable Local Image* object corresponding to a consistency variable is characterized by predetermined values for the following attributes:

Resynchronized Variable:

This attribute has the value: FALSE.

All the other non quoted attributes are not given predetermined values.

12.2.1.4.4.6.4 Consistency variable type specification

12.2.1.4.4.6.4.1 Consistency variable functionality

The consistency variable values are assigned with a type issued from the instantiation of the *Type Constructor* class.

When the *Inconsistency Detection* attribute has the value 'TRANSITORY', the consistency variable type shall be 'K_SCONS'.

When the *Inconsistency Detection* attribute has the value 'FRAGMENTED', the consistency variable type shall be 'K_LCONS'.

When the *Inconsistency Detection* attribute has the value 'UNDEFINED', the consistency variable type can be described by any instance of the *Type Constructor* class.

12.2.1.4.4.6.4.2 Predetermined attribute value of the *Type Constructor* class.

Instances of the *Type Constructor* class associated with a consistency variable to detect a TRANSITORY spatial inconsistency:

A_Name:	K_SCONS
Construction	: STRUCTURE
Named field	: FALSE
Reference	: K_UN_16 -- Interchange Nr.
Reference	: K_BOOLAC64 -- Transmission status
A_Name	: K_UN_16
Construction	: SIMPLE
Class of type	UNSIGNED

```

Size           : 16
A_Name         : K_BOOLACnn
Construction   : ARRAY
  Compacted    : TRUE
  Dimension    : nn (list variable number)
  Reference    : K_BOOL
A_Name         : K_BOOL
Construction   : SIMPLE
  Class of type : BOOLEAN

```

Instances of the *Type Constructor* class associated with a consistency variable to detect a PERMANENT spatial inconsistency:

```

A_Name:        K_LCONS
Construction   : STRUCTURE
  Named field  : FALSE
  Reference    : K_UN_16 -- Interchange Nr.
  Reference    : K_UN_16ACnn -- Transmission_date
A_Name         : K_UN_16ACnn
Construction   : ARRAY
  Compacted    : TRUE
  Dimension    : nn (list variable number)
  Reference    : K_UN_16

```

12.2.1.4.4.6.4.3 Field semantics

The semantics of the consistency variable value fields when the *Inconsistency Detection* attribute does not have the value UNDEFINED is as follows:

Structure fields associated with the 'K_SCONS' type:

Interchange number:

The value of the variables of the list is characterized by the interchange number during which they have been updated by the network. This interchange number is a global data of the MPS environment produced by a specific application entity and broadcast to the entities elaborating spatial consistency information by using the synchronization variable associated with the list.

This interchange number is used to check that the processed consistency variable values dates are identical during the spatial consistency processing.

Transmission_status:

The transmission status of the variable list is an array whose elements represent respectively the transmission status for each of the list variables.

The variable transmission status at the level of a consuming application entity indicates whether this variable has been received since the last occurrence of the synchronization variable associated to the list.

Structure fields associated with the 'K_LCONS' type:

Interchange number:

This field is provided with the same semantics as that with the same name in the 'K_SCONS' type.

Transmission_date:

The transmission date of the variable list is an array whose elements represent respectively the number of the last interchange of each of the list variables.

The number of the last interchange of a variable at the level of a consuming application entity corresponds to that taken into account during the last reception of this variable.

This interchange number, taken into account for a variable during its reception, corresponds to the synchronization variable value of the list which this variable belongs to.

12.2.1.4.4.7 Elaboration mechanism specification of the consistency variable values

This mechanism is involved only when the *Inconsistency detection* attribute associated with the variable list has a value other than 'UNDEFINED'.

This service element elaborates the predetermined type value involved in the spatial consistency status elaboration of a variable list, as shown in Figure 137.

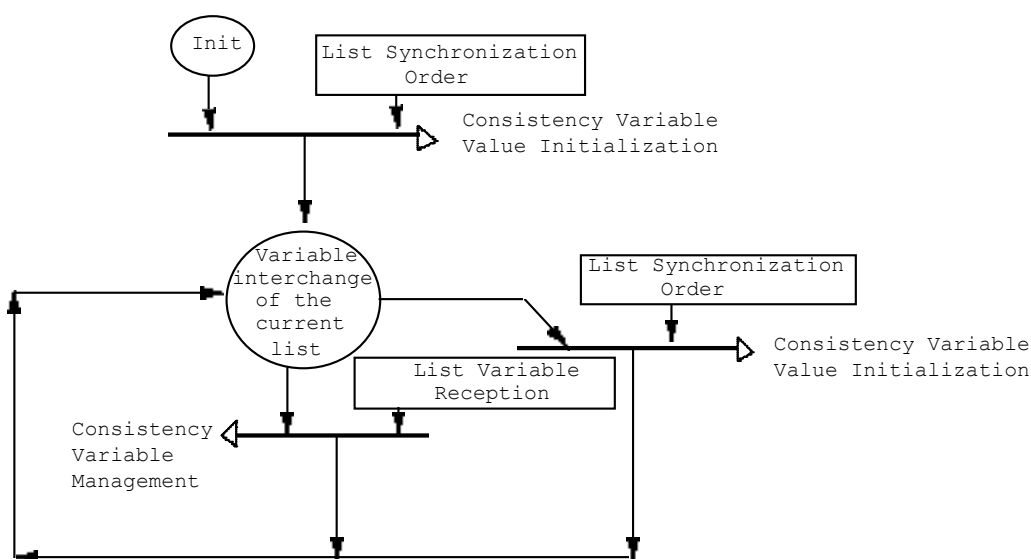


Figure 137 – Consistency variable value evaluation net

The initialization of the consistency variable value upon occurrence of the synchronization order associated with a variable list consists in giving its different fields a specific value that characterizes the implementation of the consistency processing.

The value of the consistency variable at the end of the initialization is as follows:

- the ' Interchange number ' field contains the interchange number corresponding to the synchronization variable value which triggered this initialization
- the ' Transmission status ' field contains, for each variable of the list, a transmission status 'FALSE'.
- the ' Transmission date ' field contains for each the variable composing the variable list with which the consistency variable is associated, an interchange number corresponding to that which triggered the initialization.

The initialization of a consistency cycle is performed upon occurrence of the synchronization order associated with the variable list. This initialization corresponds to an assignation of the consistency variable field ' *Interchange number* ' of the synchronization variable value which corresponds to the synchronization order of the list. The consistency variable value management upon reception of a variable consists in modifying the value of the Transmission_Status and Transmission_Date according to the selected predetermined type.

In the case of the `Transmission_Status` field, the value management consists in making a change of the transmission status of each of the variables of the received list from FALSE to TRUE.

In the case of the `Transmission_date` field, the management value consists in memorizing the current interchange number in the transmission date associated with the variable.

Example

Given the list {V1, V2, V3, V4, V5} and the associated synchronization variable: S_LIST

Interchange timing diagram:

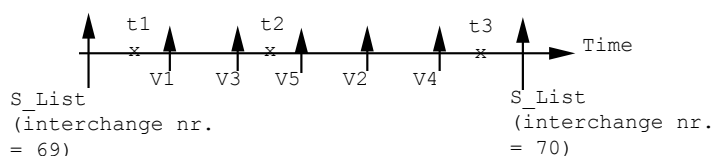


Figure 138 – Consistency interchange timing diagram

If we consider that all the variables were received during the previous interchange, the consistency variable associated with the various instants of the interchange has the following values:

t1 : Consistency variable value
Inconsistency Detection:= TRANSITORY {69; 0,0,0,0,0}
Inconsistency Detection:= PERMANENT {69; 68,68,68,68,68}

t2 : Consistency variable value
Inconsistency Detection:= TRANSITORY {69; 1,0,1,0,0}
Inconsistency Detection:= PERMANENT {69; 69,68,69,68,68}

t3 : Consistency variable value
Inconsistency Detection:= TRANSITORY {69; 1,1,1,1,1}
Inconsistency Detection:= PERMANENT {69; 69,69,69,69,69}

12.2.1.4.4.8 Access to the consistency variables.

The consistency variables are handled by a subset of the services provided to the user for normal variables.

The services permitting access to the consistency variables are those existing for the synchronization variables, with the following restriction:

A_WRITELOC: Only when the *Inconsistency detection* attribute has a value other than UNDEFINED.

12.2.1.4.4.9 Specification of the recovery mechanism

A variable list can be provided with a recovery mechanism whose implementation depends on this list according to the user needs concerning the reliability.

This recovery mechanism elaborates recovery list contents corresponding to an ordered set of references to elements of the variable list which were not successfully received since the last reception of the list synchronization variable, under the condition that this synchronization variable has arrived within a time interval smaller than the recovery period.

The sequence operation of the variable references in the recovery list is performed in the declaration order of the list variables.

The references to the list variables, stored in the recovery contents by this mechanism, corresponds to the variable identifiers of this list. These recovery contents which conditions the initiation of new interchanges are limited to a maximum number of references defined at the time of configuration of the variable list.

NOTE The use of the recovery mechanism, with a recovery period equivalent to the network period of the synchronization variable of the list, releases all the losses which appeared during the list synchronization variable transmission.

Figure 139 shows the evaluation net of the service element which elaborates the contents of the recovery list associated with a variable list at the application level.

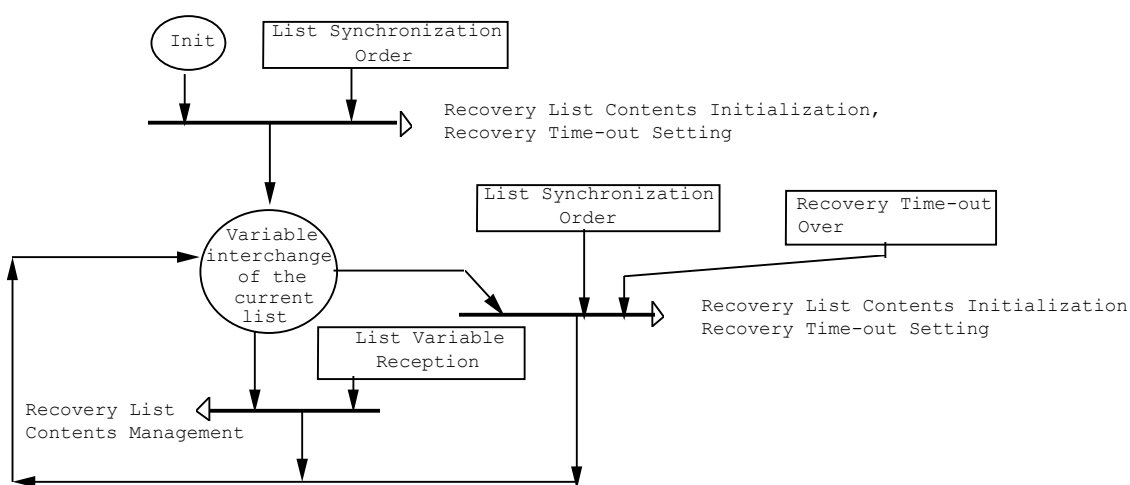


Figure 139 – Recovery mechanism evaluation net

- The initialisation of the recovery list contents, upon occurrence of the synchronization order associated with the variable list, consists in giving it the list of all the variable list identifiers as contents value.

As the synchronization variable associated with the list precedes all the list variables during an interchange, it shall be considered that all the list variables are not received at the instant of the synchronization order occurrence.

- The recovery list management upon reception of a variable, consists in removing the identifier associated with this variable of this recovery list.
- The setting of the recovery time-out is performed upon occurrence of the list synchronization order and also automatically when it is expired. When it is set, the value of the *Preselection* attribute of this time-out is that of the *Recovery period* attribute of the *Variable List Local Image* object.

Example:

Given the list {V1, V2, V3, V4, V5} and the associated synchronization variable: S_LIST. The maximum recovery list size is limited to 4 elements.

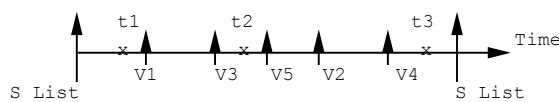


Figure 140 – Recovery interchange timing diagram

The recovery list, shown in Figure 140, has the following values at the different interchange instants:

t1 : Recovery list contents = { V1, V2, V3, V4 }

t2 : Recovery list contents = { V2, V4, V5 }

t3 : Recovery list contents = { $\emptyset$ }

12.2.1.5 Description of the configuration

The description specification of the application layer objects refers to static attributes of the objects:

- Variable of class normal, synchronization (by limiting to those attributes shared by the producer and the consumer plus those specific to the producer).
- Type of a variable
- Access to a variable
- Variable list (by limiting to those attributes shared by the consumers of this list).

12.2.1.5.1 Description variables

The configuration description accessible to the user anywhere on the network shall be addressed at the interface between the application layer and the user.

For this purpose, the configuration description associated with the various objects of the application layer is modelled by description variables.

These description variables are themselves subjected to particular constraints which endows them with the same configuration on the network, in order not to define recursive description variables describing the description variable configuration.

These particular constraints are translated at the application layer level, into predefined values of the *Consumed Variable Local Image* and *Produced Variable Local Image* objects associated to the description variables.

12.2.1.5.2 Specification of a description variable

12.2.1.5.2.1 Description variable overview

A description variable in the AL environment is modeled in the same way as the other variables using the following objects:

- A *Produced Variable Local Image* object, which includes all the attributes required for the definition of a description variable in a producing application entity.
- A *Consumed Variable Local Image* object, which includes all the attributes required for the definition of a description variable in a consuming application entity.

Both these object categories correspond to particular instantiation of the *Identified Variable* class, for which the *Class* attribute of the object has the value DESCRIPTION.

A *Produced Variable Local Image* or *Consumed Variable Local Image* object corresponding to a variable of class DESCRIPTION are different from those corresponding to 'normal' variables by predetermined values of some of its attributes.

12.2.1.5.2.2 Predetermined values of the *Identified Variable* class attributes.

An *Identified Variable* class corresponding to a description variable is characterized by values predetermined for the following attributes:

Transmitted Status:

This attribute has the value ' FALSE '.

Class:

This attribute has the value ' DESCRIPTION '.

All the non quoted attributes are not given predetermined values.

12.2.1.5.2.3 Predetermined values of the attributes of the *Produced Variable Local Image* object

A *Produced Variable Local Image* object corresponding to a description variable is characterized by values predetermined for the following attributes:

Resynchronized Variable:

This attribute has the value ' FALSE '.

Elaborated Refreshment:

This attribute has the value ' FALSE '.

Elaborated Punctual Refreshment:

This attribute has the value ' FALSE '.

Required Universal Services:

This attribute has the value ' FALSE '.

All the non quoted attributes are not given predetermined values.

12.2.1.5.2.4 Predetermined values of the attributes of the *Consumed Variable Local Image* object

A *Consumed Variable Local Image* object corresponding to a description variable is characterized by values predetermined for the following attributes:

Resynchronized Variable:

This attribute has the value ' FALSE '.

Required Universal Services:

This attribute has the value ' FALSE '.

All the non quoted attributes are not given predetermined values.

12.2.1.5.2.5 Description variable names

The description variable names handled at the interface between the application layer and the user, belong to the unique addressing space referencing the application objects. In this MPS addressing space, the standard defines four (reserved name sets) to address the description variables:

A_Name of the variable starting by:

"V_" Description of a variable

"A_" Description of a variable access

"T_" Description of a variable type

"L_" Description of a variable list.

12.2.1.5.2.6 Description variable types

12.2.1.5.2.6.1 Description variable type definition

The values of description variables are given a type which depends on the nature of the description, corresponding to predefined instances of the *Type Constructor* class.

12.2.1.5.2.6.2 Predefined instances of the *Type Constructor* class.

Four predefined instances provide semantics to the types of the description variables.

- variables of NORMAL, SYNCHRONIZATION class.
- renamed accesses to variables of NORMAL, SYNCHRONIZATION class.
- types of variables.
- lists of variables.

Instance of the *Type Constructor* class associated with the type of a description variable of a variable of NORMAL, SYNCHRONIZATION class:

A_Name : K_DVAR
Construction : PREDEFINED

Instance of the *Type Constructor* class associated with the type of a description variable of a renamed access to a variable:

A_Name : K_DACCESS
Construction : PREDEFINED

Instance of the *Type Constructor* class associated with the type of a description variable of a type:

A_Name : K_DTYPE
Construction : PREDEFINED

Instance of the *Type Constructor* class associated with the type of a description variable of a variable list:

A_Name : K_DLIST
Construction : PREDEFINED

NOTE It has to be stated that the predefined type description of the environment means only that they are predefined. The precise knowledge of the semantics of the predefined types in the AL environment is part of the conformance to the MPS standard.

12.2.1.5.2.6.3 Semantics of the predefined types of the description variables

The types of the description variables provide a semantics to these variables. These semantics reflect the values of the static attributes shared by all the instances of the described objects as well as some attributes specific to the producer of the object.

The semantics of description variable type of a variable of NORMAL or SYNCHRONIZATION class describe the values of the following attributes, if they exist:

A_Name
 Type Constructor Reference
 Transmitted Status
 Significant Status
 Identifier
 Transmission Mode
 Network Period
 Class
 Consistency Variable
 Variable List Reference
 Resynchronized Variable
 Synchronization Variable Reference
 Elaborated Refreshment
 Refreshment Characteristics
 Production Period
 Synchronization Variable Reference
 Elaborated Punctual Refreshment
 Time slot
 Synchronization Variable Reference

The semantics of the description variable type of a renamed access to a variable describe the values of the following attributes, if they exist:

A_Name
 Variable Reference
 Access Mode
 Access Path

The semantics of the description variable type of a variable type describe the values of the following attributes, if they exist:

A_Name
 Construction
 Primitive Type
 Size
 Compacted
 Dimension
 Reference Type Constructor
 Named Field
 Field Name
 Reference Type Constructor

The semantics of the description variable type of a list variable describe the values of the following attributes, if they exist:

A_Name
 List of Reference Variable
 Spatial Consistency Required
 Recovery Required
 Reference Synchronization Variable
 Inconsistency Detection
 List of Reference Consistency Variable
 Recovery Period.

12.2.1.5.2.6.4 Identifier of the description variables

The identifiers accepted for the description variables belong to a reserved subset defined by the network management standard.

Moreover, there exists an addressing relationship between identifiers of the variables and those of the description variables which is defined by the network management standard.

12.2.1.5.2.6.5 Access to the description variables

The description variables can be handled by a subset of services provided to the user for normal variables.

The services providing access to the description variables are:

- A_WRITELOC
- A_WRITEFAR
- A_READLOC
- A_READFAR
- A_SENT
- A_RECEIVED
- A_UPDATE

All other services, provided to the user for a normal variable, are not permitted for a description variable.

12.2.2 Virtual model of a device (VMD) ASE

12.2.2.1 Overview

The modelling of a Sub-MMS VMD within an application is similar to its equivalent MMS.

12.2.2.2 Concepts

12.2.2.2.1 Scope of objects

In order to simplify object identification, the listed classes of objects do not have the scope (in the MMS sense) explicitly specified.

In other words appurtenance of an object of a given class to another object of another class is implicitly known (variable belonging to a particular domain for example) and should not appear in the object identification.

12.2.2.2.2 Object identification

The identification of objects "variable", "variable-list", "pi", "domain" and "event" is done by an identifier capable of taking either the form of an integer value (Index), or the form of a character string (Name). The number of characters in the string is either undetermined or determined by the companion standards.

Taking into consideration the elimination of the concept of scope, the identification of an object shall be unique, either within its class or within a space common to all the classes. An object can be identified by an index and/or a name.

12.2.2.2.3 Environment management

12.2.2.2.3.1 Environment management services

The Sub-MMS environment management services are:

- 1- Initiate, allows negotiating the establishment of a Sub-MMS environment.
- 2- Conclude, allows proposing the closing of a negotiated Sub-MMS environment.
- 3- Abort, allows suddenly breaking a negotiated Sub-MMS environment.

The initiate and Abort services are compulsory for the devices supporting the management of the negotiated Sub-MMS environment. The Conclude service is negotiable.

The Reject service allows the notification of protocol errors. It is compulsory for all the equipment.

12.2.2.2.3.2 Type of communication

12.2.2.2.3.2.1 Supported communication types

A device can support the following types of communication:

- a) communication in a negotiated Sub-MMS environment,
- b) communication in a predefined Sub-MMS environment,
- c) communication without Sub-MMS environment.

12.2.2.2.3.2.2 Communication in a negotiated sub-MMS environment

The Sub-MMS environment is negotiated between a user of the calling Sub-MMS and a user of the called Sub-MMS.

The resources allocated to this environment are reserved for these two users.

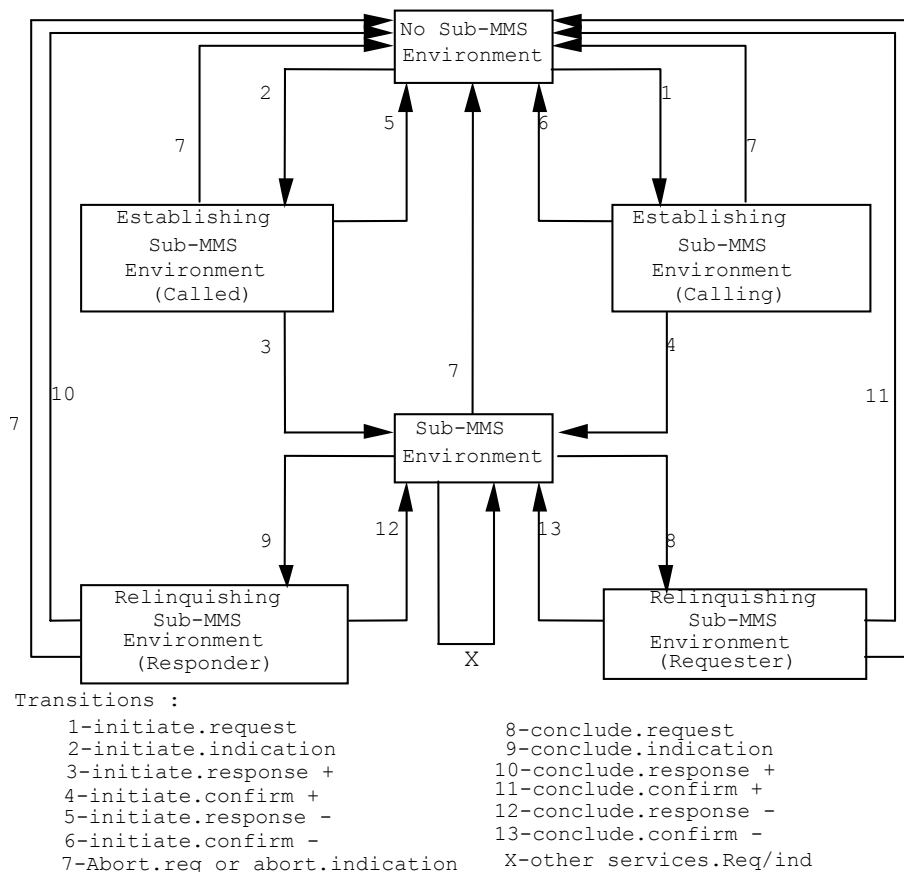


Figure 141 – Flowchart of the sub-MMS environment management state

A Sub-MMS environment known as association has several states, as shown in Figure 141. The initiate, conclude and abort services can act on the association.

The initial state for the calling Sub-MMS as well as the called shall be the state "No Sub-MMS Environment".

In the state "No Sub-MMS Environment", a user of the sub-MMS shall only invoke the request of the initiate service primitive.

In the "Establishing Sub-MMS Environment (Calling)" state, a user of the sub-MMS shall only invoke the request of the Abort service primitive.

In the "Establishing Sub-MMS Environment (Called)" state, a user of the sub-MMS can only invoke the response to the initiate service primitive or the request of the Abort service primitive.

In the "Relinquishing Sub-MMS Environment (Requester)" state, the only request for a primitive which a user of the sub-MMS can invoke is the request of the Abort service primitive. It is noted that the responses (+) or (-) can continue to be invoked.

In the "Relinquishing Sub-MMS Environment (Responder)" state, a user of the Sub-MMS can only invoke the request of the Abort service primitive and the response to the Conclude service primitive.

The only events which provoke the output of the "Sub-MMS environment" state are the invocations of the Abort service primitive request, the Conclude service primitive request as well as the reception of the indications of the Abort service primitive and the Conclude service primitive.

In the "Sub-MMS Environment" state, a user of the sub-MMS can invoke any of the requests or responses of the previously negotiated service primitives, on condition that the following points are observed:

- a) other sections of this document restraining the use of the services through sequencing constraints;
- b) a response to a primitive shall not be invoked unless a request of the primitive corresponding to a service indication has been received;
- c) the request of the initiate service primitive shall not be invoked.

12.2.2.2.3.2.3 Communication in a predefined sub-MMS environment

The SUB-MMS environment is established implicitly between two or more users. The resources allocated to this environment are reserved to its users.

The Sub-MMS environment management services (Initiate, Conclude, Abort) are not significant.

If an Initiate or Conclude Request PDU is received, a Reject PDU is issued with the parameters Reject PDU type equal to PDU ERROR and Reject Code equal to ILLEGAL-MAPPING.

If a Request PDU of a confirmed service not supported in this environment is received, a Reject PDU message is issued with the parameters Reject PDU Type equal to CONFIRMED.REQUEST PDU and Reject Code equal to UNRECOGNIZED SERVICE.

NOTE 1 A Sub-MMS environment established between more than two users only supports unconfirmed services.

NOTE 2 The reception of an Abort.Indication is ignored.

12.2.2.2.3.2.4 Communication without sub-MMS environment

No Sub-MMS environment is established between two or more users.

The resources allocated to this type of communication are available to all the potential remote users.

The Sub-MMS environment management services (Initiate, Conclude, Abort) are not supported.

If an Initiate or Conclude Request PDU is received, a Reject PDU is issued with the parameters Reject PDU Type equal to PDU ERROR and Reject code equal to ILLEGAL MAPPING.

If a Request PDU of a confirmed service not supported by the equipment is received, a Reject PDU is issued with the Reject PDU Type parameters equal to CONFIRMED.REQUEST PDU and Reject Code equal to UNRECOGNIZED-SERVICE.

NOTE The reception of an Abort.Indication is ignored.

12.2.2.2.3.3 Error types

12.2.2.3 VMD class specification

12.2.2.3.1 VMD class model

FAL ASE:	VMD ASE
CLASS: VMD	
CLASS ID:	not used
PARENT CLASS:	not used
Object: VMD	
(m) Key Attribute:	Executive Function
(m) Attribute:	Vendor Name
(m) Attribute:	Model Name
(m) Attribute:	Revision
(m) Attribute:	Logical Status (STATE-CHANGES-ALLOWED,NO-STATE-CHANGES-ALLOWED, OTHER)
(m) Attribute:	Physical Status (OPERATIONAL, PARTIALLY-OPERATIONAL, INOPERABLE, NEEDS-COMMISSIONING)
(m) Attribute:	List of Capabilities
(m) Attribute:	List of Program Invocations
(m) Attribute:	List of Domains
(m) Attribute:	List of Variables
(m) Attribute:	List of Variable Lists
(m) Attribute:	List of Events
(o) Attribute:	Additional Detail

12.2.2.3.2 Attributes

Executive function:

This attribute represents the set of software ensuring the service procedures.

Vendor Name:

This attribute identifies the constructor of the device.

Model Name:

This attribute identifies the device model which supports the VMD.

Revision:

This attribute identifies the version index of the device system supporting the VMD. The value of this attribute is assigned by the vendor.

Logical status:

Sub-MMS distinguishes two levels of functions which are described in this attribute. Other levels of functions could be defined eventually without altering the interoperability of the device.

1-STATE-CHANGES-ALLOWED:

In this status all the VMD supported services can be carried out,

2-NO-STATE-CHANGES-ALLOWED:

In this status only the following services can be carried out, if they are VMD supported:

- Initiate
- Conclude
- Abort
- Identify

- Status
- Read
- Get Alarm summary
- Get Name List
- Get Domain Attributes
- Get Program Invocation Attributes
- Get Variable Access Attributes
- Get Variable List Attributes
- Get Event Condition Attributes

3-OTHER:

Other statuses can be defined. These statuses should be described in the PICS.

Physical Status:

This attribute gives the hardware status of the device

List of capabilities:

This attribute enables defining a set of characteristics specific to the VMD.

List of Program Invocations:

This attribute gives the set of invocation programs belonging to the VMD. The program invocations can be predefined in the VMD or created dynamically by a specific Sub-MMS service or following a domain loading.

List of Domains:

This attribute gives the set of Domains belonging to the VMD. The domains can be predefined in the VMD or created dynamically by the loading services of the SUB-MMS domains.

List of Variables:

This attribute gives the set of objects of the variable class belonging to the VMD. The variables can be predefined in the VMD or created dynamically following a domain loading.

List of Variable Lists

This attribute gives the set of object variable lists belonging to the VMD. The variable lists can be predefined in the VMD or created dynamically by a specific Sub-MMS service or following a domain loading.

List of Events:

This attribute gives the set of events belonging to the VMD. The events could be predefined in the VMD or created dynamically following a domain loading.

Additional Detail:

This attribute can contain additional information

Besides the VMD object defined above, the Sub-MMS application layer also supports the objects mentioned below:

- Domain objects,
- Program Invocation objects,
- Variable objects,
- Variable list objects,
- Event objects...

The C.S can introduce other classes of objects.

12.2.2.4 Services

12.2.2.4.1 Confirmed initiate service

12.2.2.4.1.1 Functionality

The initiate service shall be used to establish the environment of the Sub-MMS (association) in order to authorize the exchange of information between two users of the Sub-MMS according to their resources and requirements.

12.2.2.4.1.2 Service primitives

The structure of the service is shown in Table 485.

Table 485 – Confirmed initiate service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument	M	M		
Quality of service calling	M	M=		
Extension Calling	C	C=		
Proposed Max Serv Outstanding Calling	M	M		
Proposed Max Serv Outstanding Called	M	M		
Proposed Data Structure Nesting	M	M		
Init Request Detail	M	M		
Result(+)			S	S=
Quality of service called			M	M=
Extension Called			C	C=
Negotiated Max Serv Outstanding Calling			M	M=
Negotiated Max Serv Outstanding Called			M	M=
Negotiated Data Structure Nesting			M	M=
Init Response Detail			M	M
Result(-)			S	S=
Error Type			M	M=

Argument

This parameter shall transport the service parameters specific to the initiate service request.

For the parameters of the request primitive, the Sub-MMS supplier in the calling system can reduce the values supplied by the calling Sub-MMS user. For the parameters of the indication primitive, the supplier of the Sub-MMS in the called system can reduce the values supplied by the InitiateRequestPDU except for the Indication Services Support Calling parameter. The reduction of the values shall follow the reduction rules as defined in the parameter descriptions. No other modification of these values is authorized.

NOTE The fact of authorizing the service suppliers to reduce the values proposed by the users, gives a mechanism which can be used to check the adequation of the values given by the user to the resources of the service supplier. Its implementation is a local problem.

Quality of the calling service:

This parameter shall indicate the choice of the service quality. This standard distinguishes between two service qualities. The level 1 service quality is reserved for communications without object access protection. Whereas the level 2 quality is that using object access protection. In this case, an access right verification mechanism is installed and checks the compatibility of the Password and Access Groups parameters provided during invocation of the Initiate service request, with the parameters Password, Access Groups and Access Rights of the objects which are accessed by the calling user.

The information necessary for the service 2 quality are those supplied in the Extension Calling parameter.

New service qualities can be specified. We recommend that the information necessary for these new service qualities are provided in the Extension calling parameter.

Extension Calling:

The structure of this parameter is shown in Table 486.

Table 486 – Detailed structure of the extension calling parameter

Parameter name: Extension Calling	Req	Ind
Quality of service 1	C	C=
Null	M	M=
Quality of service 2	C	C=
Password	M	M=
Access Groups	M	M=
Others Qualities of service	C	C=
Local Detail Calling	U	U=

Password:

This parameter, present for service 2 quality, defines the password, unique for any association. It will be used for access to all the objects of the called user on this association. If access by a password is not requested, this parameter shall take value 0.

Access Groups:

This parameter, present for a ,service 2 quality, defines the access groups to which the calling user belongs. This belonging is applied for the access to all the objects of the Server on this association.

Local Detail Calling:

This parameter can specify information concerning the service quality extensions.

Proposed Max Serv Outstanding Calling:

This parameter, proposed by the calling user, shall identify the maximum number of indications of confirmed services waiting for a response on the calling user side on this association.

The value of this parameter can be reduced by the supplier of the Sub-MMS service (this will then be different in the service indication).

Proposed Max Serv Outstanding Called:

This parameter, proposed by the calling user, shall identify the maximum number of indications of confirmed services awaiting response from the called user on this association.

The value of this parameter can be reduced by the supplier of the Sub-MMS service (this will then be different in the service indication).

Proposed Data Structure Nesting:

This parameter shall indicate the maximum number of overlapping levels in the data structures used on this association between the two users of the Sub-MMS. The value of this parameter can be reduced by the supplier of the Sub-MMS service. The value taken by this parameter shall be higher than or equal to 1.

Init Request Detail:

The structure of this parameter is shown in Table 487.

Table 487 – Detailed structure of the init request detail parameter

Parameter name: Init Request Detail	Req	Ind
Proposed Version Number	M	M
Proposed Parameter CBB	M	M
Indication Services Support Calling	M	M
Extension	U	U=
Request Services Support Calling	M	M=
Access by Name Support Calling	M	M=

Proposed Version Number:

This parameter contains the minor revision number of the Sub-MMS protocol services proposed by the calling user.

The value of this parameter can be reduced by the supplier of the Sub-MMS service (this will then be different in the service indication) but shall always be higher than 1.

Proposed Parameter CBB:

This parameter shall specify the negotiated set of conformity parameters relative to the variables (CBBs) which will be supported on this association. The value of this parameter in the service indication primitive shall represent the intersection between the set of CBBs supported by the calling user and the set of CBBs supported by the supplier of the Sub-MMS service.

Indication Services Support Calling:

This parameter defines the indications of services supported by the calling user. The value of this parameter can be reduced by the supplier of the Sub-MMS service supplier of the calling user.

Extension:

This parameter, if present, includes two items of information: Request Services Support Calling and Access by Name Support Calling.

Request Services Support Calling:

This parameter defines the requests of services supported by the calling user.

Access by Name Support Calling:

This parameter defines if the access to the objects by their name is supported by the calling user as requester or responder of the service.

Result(+)

This selection indicates that the requested service has been executed with success. In this case the following parameters shall be entered:

Quality of called service:

This parameter shall indicate the choice of the called user in terms of service quality. This standard distinguishes two qualities of service. The level 1 quality service is reserved for communications without access protection for the objects whereas the level 2 quality service is that using access protection for the objects. In this case, a mechanism to check the access rights is installed and checks the compatibility of the Password and Access Group parameters provided during the invocation of the initiate service request with the Password, Access Groups and Access Rights parameters of the objects which are accessed by the called user.

The information necessary for the quality of service 2 are supplied in the Extension Called parameter.

New service qualities can be specified. We recommend that the information necessary for these new service qualities are supplied in the Extension called parameter.

Extension Called:

The structure of this parameter is shown in Table 488.

Table 488 – Detailed structure of the extension called parameter

Parameter name: Extension Called	Rsp	Cnf
Quality of service 1	C	C=
Null	M	M=
Quality of service 2	C	C=
Password	M	M=
Access Groups	M	M=
Others Qualities of service	C	C=
Local Detail Called	U	U=

Password:

This parameter, present for a service 2 quality, defines the password, unique for all associations. It will be used for access to all the objects of the calling user on this association. If access with the help of the password is not requested, this parameter should take value 0.

Access Groups:

This parameter, present for a service 2 quality, defines the access groups to which the calling user belongs. This belonging applies for the access to all the objects of the calling user on this association.

Local Detail Called:

This parameter can specify the information concerning the service quality extensions.

Negotiated Max Serv Outstanding Calling:

This parameter shall identify the maximum number of indications of confirmed services waiting for a response on the calling side of this association.

The value of this parameter can be reduced in the response primitive with respect to the value contained in the indication primitive.

Negotiated Max Serv Outstanding Called:

This parameter shall indicate the maximum number of indications of confirmed services waiting for a response on the called side of this association.

The value of this parameter shall be reduced in the response primitive with respect to the value contained in the indication primitive.

Negotiated Data Structure Nesting:

This parameter shall indicate the maximum number of confirmed services waiting for a response on the side of the called party in this association.

The value of this parameter shall be reduced in the response primitive with respect to the value contained in the indication primitive.

Negotiated Data Structure Nesting:

This parameter shall indicate maximum number of nesting levels in the data structures used on this association between the two users of the Sub-MMS.

The value of this parameter can be reduced in the response primitive with respect to the value contained in the indication primitive. The value taken by this parameter shall be greater than or equal to 1.

Init Response Detail:

The structure of this parameter is shown in Table 489.

Table 489 – Detailed structure of the init request detail parameter

Parameter name: Init Response Detail	Req	Cnf
Negotiated Version Number	M	M=
Negotiated Parameter CBB	M	M=
Indication Services Support Called	M	M
Extension	C	C=
Request Services Support Called	M	M=
Access by Name Support Called	M	M=

Negotiated Version Number:

This parameter contains the minor revision number of the Sub-MMS protocol proposed by the called user.

The value of the parameter shall be less than or equal to the proposed value; but should always be higher than 1.

Negotiated Parameter CBB:

This parameter shall specify the negotiated set of CBBs which will be supported on this association. The value of this parameter in the service response primitive shall represent the intersection between the set of CBBs supported by the called user and the set of CBBs supplied in the indication primitive.

Indication Services Support Called:

This parameter defines the indications of services supported by the called. The value of this parameter can be reduced by the supplier of the Sub-MMS service of the called user.

Extension:

This parameter shall be present if the extension parameter is included in the request. This parameter contains two items of information: Request Services Support Called and Access by Name Support Called.

Requester Services Support Called:

This parameter defines the services supported by the called user.

Access by Name Support Called:

This parameter defines if the access to the objects by their name is supported by the called user as service requester or responder.

Result(-):

This parameter should indicate that the requested service is unsuccessful. The type of error is indicated by the Error Type parameter.

12.2.2.4.1.3 Service procedure

The called user transmits a positive response if he accepts the proposed values or if he is capable of proposing others in compliance with the rules for reduction of the values. Otherwise he transmits a negative response.

The success of the execution of the initial service shall result in the establishment of a Sub-MMS environment. If an Initiate Request PDU is received on an existing association, a Reject PDU shall be issued with the parameters Reject PDU Type equal to PDU-ERROR and Reject Code equal to ILLEGAL-MAPPING.

NOTE For reasons of simplicity, this document recommends a single abstract syntax and a single transfer syntax per association. Consequently, an Initiate Request PDU will be accepted or refused according to the proposed context.

12.2.2.4.2 Confirmed conclude service

12.2.2.4.2.1 Functionality

The Conclude service can be used to provoke the ordered termination of the Sub-MMS environment. A Sub-MMS user invokes the Conclude service to indicate that he has accomplished the requests which have been scheduled and that no other request will be invoked.

12.2.2.4.2.2 Service primitives

The structure of the Conclude service is shown in Table 490.

Table 490 – Conclude service parameter

Parameter Name	Req	Ind	Rsp	Cnf
Argument	M	M=		
Result(+)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument

There is no parameter specific to this service in the Conclude service request.

Result(+)

This parameter indicates that the requested service has been successfully executed.

Result(-)

This parameter shall indicate that the service request has failed. The Error Type parameter indicates the reason for this failure.

12.2.2.4.2.3 Service procedure

Calling procedure:

- 1 The invocation of the Conclude.Request primitive is forbidden on an association via which a domain loading is in progress and whose "State" attribute of the object domain has a value different from "READY", "IN-USE".
- 2 After the invocation of the Conclude.Request primitive:
 - a) the following actions shall be performed:
 - pass the Sub-MMS environment in the "Relinquishing Sub-MMS Environment" state (Requester),
 - continue to invoke the response primitives in order to serve the requests of his correspondent,
 - continue to process the received service confirmations.
 - b) the following constraint shall be observed:
 - no service request can be invoked besides Abort.

- 3 After reception of the Conclude.Cnf(+) primitive:
 - a) the following action shall be performed:
 - pass the Sub-MMS environment to the "No Sub-MMS Environment" state.
 - b) the following constraint shall be observed:
 - no service invocation shall be made on this association.
- 4- After reception of the Conclude.Cnf(-) primitive:
 - a) the following actions shall be performed:
 - pass the Sub-MMS environment to the "Sub-MMS Environment" state,
 - if necessary, continue to invoke the service primitives.

Called user procedure

- 1 Following reception of the Conclude.Indication primitive:
 - a) the following action shall be performed:
 - pass the Sub-MMS environment to the "Relinquishing Sub-MMS Environment" (Responder)state,
 - b) the following constraints shall be observed:
 - answer in priority to the Conclude.Ind primitive.
 - no service request can be invoked except Abort.
- 2- Invoke the Conclude.Response(+) primitive if the following constraints are observed:
 - no request is waiting for a response from its correspondent,
 - no request made by its correspondent is waiting for a response,
 - no domain loading is in progress.
 - no saving of a domain is in progress.
 - a) The following action shall be performed:
 - switch the Sub-MMS environment to the "No Sub-MMS Environment" state,
 - continue, if necessary, to invoke the primitive services.
 - b) the following constraint shall be observed:
 - no service invocation shall be made on this association.
- 3- Invoke the Conclude.Response(-) primitive if one of the following constraints is not respected:
 - at least one request is waiting for a response of its correspondent,
 - at least one request by its correspondent is waiting for a response,
 - at least one domain loading is in progress,
 - at least one domain saving is in progress.
 - a) the following actions shall be performed:
 - pass the Sub-MMS environment to the "Sub-MMS Environment" state,
 - if necessary, continue to invoke the service primitives.

NOTE We recommend reserving for the calling user the role of conclude service initiator.

12.2.2.4.3 Unconfirmed abort service

12.2.2.4.3.1 Functionality

The Abort service shall be used to quit the Sub-MMS environment without negotiation. The Sub-MMS user shall invoke the Abort request primitive to indicate that it wishes to stop the communications on the association immediately and without negotiation. The Sub-MMS Abort service comes from and uses the MCS A_ABORT service. (See projection of the Sub-MMS PDU on the Type 7 MCS services of IEC 61158-6).

NOTE The Abort indication service primitive can also be generated by the supplier of the Sub-MMS service.

12.2.2.4.3.2 Service primitives

The structure of the service is shown in Table 491.

Table 491 – Unconfirmed abort service parameters

Parameter Name	Req	Ind
Argument	M	M
Locally Generated		M
User Information	U	U

Argument

The argument shall transport the specific parameters of the Abort service.

Locally Generated

This parameter shall indicate if the Abort request has been generated locally by the supplier of the Sub-MMS services, or if the Abort request has been received. This parameter shall be filled in by the Sub-MMS supplier.

User Information

This parameter, when present, shall contain additional information concerning the reasons for this Abort.

12.2.2.4.3.3 Service procedure

In the case of an Abort initiated by the Sub-MMS user, the correspondent shall be informed of this Abort by the reception of an Abort service indication primitive and the Sub-MMS environment shall be deleted.

In the case of an Abort initiated by the Sub-MMS supplier, the two correspondents shall be informed of this Abort by the reception of Abort Service indication primitives (if this is possible) and the Sub-MMS environment shall be deleted.

After destruction of the Sub-MMS environment, all the objects with a specific link with the Sub-MMS environment in question are deleted.

12.2.2.4.4 Unconfirmed reject service

12.2.2.4.4.1 Functionality

The Reject service is a service initialized by the supplier following a protocol error.

The protocol errors of a unconfirmed service or a confirmed service response does not result in the issue of a Reject PDU.

12.2.2.4.4.2 Service primitives

The structure of the service is shown in Table 492.

Table 492 – Unconfirmed reject service parameters

Parameter Name	Ind
Argument	M
Detected here	M
Original Invoke ID	M
Reject PDU Type	M
Reject Code	M

Argument

This argument transports the specific parameters of the Reject service.

Detected here

This parameter, of the Boolean type, informs the user if the protocol error which resulted in the rejection has been detected by the local or remote Sub-MMS supplier. The true value signifies that the protocol error has been detected locally.

Original Invoke ID

This parameter shall indicate the invocation number of the PDU for which an error has been detected. This parameter is non significant if the reject concerns a ConcludeReqPDU.

Reject PDU Type

This parameter shall indicate the type of PDU which has caused the protocol error. The PDU-ERROR value shall be used when the PDU is not a valid Sub-MMS PDU. The possible values for a PDU-ERROR are a sub-assembly of those defined in the MMS.

The values of the Reject PDU Type selected are:

- CONFIRMED-REQUESTPDU,
- PDU-ERROR,
- CONCLUDE-REQUESTPDU.

Reject Code

This parameter shall indicate the reason for which the Reject has been issued as a function of the Reject PDU Type parameter values. The possible values for the Reject Code are a sub-set of those defined in MMS.

The Reject Code parameter values selected are:

1- For CONFIRMED-REQUESTPDU:

- OTHER
This code shall be used for errors other than those defined in the Sub-MMS standard;
- UNRECOGNIZED-SERVICE
This code shall be used when the service is not supported or is not recognized.
- INVALID-INVOKE-ID
This code shall be used when an invocation number does not follow the recommendations of this standard.
- INVALID-ARGUMENT
This code shall be used when an argument of a service does not follow the recommendations of this standard;

- MAX-SERV-OUTSTANDING-EXCEEDED

This code shall be used when the maximum number of confirmed negotiated services which can be waiting is exceeded on receiving a confirmed service request:

- MAX-RECURSION-EXCEEDED

This code shall be used when the received PDU, in the sense of overlapping of the data structures, exceeds that which has been negotiated.

- VALUE-OUT-OF-RANGE

This code shall be used when the received PDU contains one or more parameters whose values exceed those authorized by this standard.

2- For PDU-ERROR:

- UNKNOWN-PDU-TYPE

This code shall be used when the type of PDU received is not recognized or is not supported;

- INVALID-PDU

This code shall be used when the received PDU is syntactically incorrect and a more detailed processing is impossible because of the seriousness of the error.

- ILLEGAL-MAPPING

This code shall be used when the an Initiate.Request is received via an existing association.

3- For CONCLUDE-REQUESTPDU:

- OTHER

This code shall be used for errors other than those defined in the Sub-MMS standard;

- INVALID-ARGUMENT

This code shall be used when a service argument does not follow the recommendations of this standard.

12.2.2.4.4.3 Service procedure

If a Sub-MMS supplier receives a RejectPDU, it shall be shown by an indication of the Reject service to the Sub-MMS user. He can then use the Abort service to end the Sub-MMS environment abruptly.

A supplier of the Sub-MMS shall be capable of transmitting a RejectPDU if it receives a PDU which generates a protocol error.

12.2.2.4.5 Confirmed status service

12.2.2.4.5.1 Service primitives

The structure of the service is shown in Table 493.

Table 493 – Confirmed status service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
extended derivation	M	M=		
Result(+) (Comp)			S	S=
vmd logical status			M	M=
vmd physical status			M	M=
local detail			U	U=
Result(-)			S	S=
Error Type			M	M=

Argument

Extended Derivation:

This parameter indicates the method to be applied to provide the response to the Status request. Two distinct methods can be activated. Their definition is a local issue.

Result(+)

VMD Logical Status:

This parameter provides the VMD Logical Status attribute value.

VMD Physical Status:

This parameter provides the VMD Physical Status attribute value.

Local Detail:

This parameter can contain the additional information on the VMD status specific to a vendor.

Result(-)

The execution of the service ended in failure. The Error type parameter indicates the cause of the failure.

12.2.2.4.5.2 Service procedure

The procedures necessary for the elaboration of different Status are carried out, and a valid response is built.

12.2.2.4.6 Unconfirmed unsolicited status service

12.2.2.4.6.1 Service primitives

The structure of the service is shown in Table 494.

Table 494 – Unconfirmed unsolicited status service parameter

Parameter Name	Req	Ind
Argument (Comp)	M	M=
vmd logical status	M	M=
vmd physical status	M	M=
local detail	U	U=

Argument**VMD Logical Status:**

This parameter provides the VMD Logical Status attribute value.

VMD Physical Status:

This parameter provides the VMD Physical Status attribute value.

Local Detail:

This parameter can contain the additional information specific to a vendor concerning the VMD status.

12.2.2.4.6.2 Service procedure

A SUB-MMS user capable of detecting a change of his status, can take the initiative to transmit the new values of his Status without receiving a request.

12.2.2.4.7 Confirmed identify service**12.2.2.4.7.1 Service primitives**

The structure of the service is shown in Table 495.

Table 495 – Confirmed identify service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Result(+) (Comp)			S	S=
vendor name			M	M=
model name			M	M=
revision			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

No specific Parameter in the request.

Result(+)**Vendor Name:**

This parameter provides the VMD object Vendor Name attribute value.

Model Name:

This parameter provides the VMD object Model Name attribute value.

Revision:

This parameter provides the VMD object Revision attribute value.

Result(-)

The service execution ended in a failure. The Error type parameter indicates the cause of the failure.

12.2.2.4.7.2 Service procedure

The information necessary for the construction of the positive response are provided by a VMD object.

12.2.2.4.8 Confirmed get name list service

12.2.2.4.8.1 Service primitives

The structure of the service is shown in Table 496.

Table 496 – Confirmed get name list service paramaters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Object class	M	M=		
Continue after	M	M=		
Result(+) (Comp)			S	S=
List of Identification			C	C=
List of Description			C	C=
More follows			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Object Class

The Object class parameter defines the class of the object. The following values can be specified: Null, Domain, Program Invocation, Variable, Variable list, event and the classes of objects defined by the CS.

Continue After

This parameter gives the identification of the object from which the search should be done.

Result(+)

This selection indicates the successful accomplishment of the service.

List of Identification:

This parameter is present if the value of the parameter object class is not null.

This parameter gives a list of object identifications belonging to a specified class in the request.

NOTE If an object supports the double identification (name and index), only the index shall be restored.

List of object description:

This parameter is present if the value of the parameter class = null.

This parameter gives the description of each object read.

More Follows:

This parameter indicates if there are other objects to be identified.

Result(-)

This selection indicates the failure of the service

Error Type

This parameter indicates the cause of the failure

12.2.2.4.8.2 Service procedure

If all the operations are carried out successfully, the service provides, in its positive response, a list of object identifications that belong to the required class. If the class is not indicated (null) and if a common object identification space exists, the server then provides the description for each object. The first identification begins just after the required identification.

In case of failure of any operation the server provides the reason for the failure in a negative response.

Whatever may be the quality of the channel service via which this service is directed, no access restriction shall be applied by the object access protection.

12.2.3 Domain ASE

12.2.3.1 Overview

Sub-MMS defines a Domain object which can pre-exist or be created by a Sub-MMS service. The data or the data complement of a Domain object can be loaded or saved by sub-MMS services.

The domain management services are the following:

- 1 - Delete Domain
- 2 - Initiate Download Sequence
- 3 - Download Segment
- 4 - Terminate Download Sequence
- 5 - Initiate Upload Sequence
- 6 - Upload Segment
- 7 - Terminate Upload Sequence
- 8 - Get Domain Attribute

NOTE 1 The Delete Domain service can be locally executed.

NOTE 2 An already existing domain is spontaneously created either with all its data or with a part of its data.

12.2.3.2 Domain class specification

12.2.3.2.1 Domain class model

FAL ASE:	Domain ASE
CLASS: Domain	
CLASS ID:	not used
PARENT CLASS:	not used
Object: Domain	
(m) Key Attribute:	Domain name
(m) Key Attribute:	Domain index
(m) Attribute:	List of capabilities
(m) Attribute:	State (LOADING, COMPLETE, INCOMPLETE, READY, IN USE)
(m) Attribute:	Predefined (TRUE, FALSE)
(m) Attribute:	Sharable (TRUE, FALSE)
(m) Attribute:	List Of Program Invocation Identification
(m) Attribute:	Upload In Progress

- (o) Attribute: Domain Access Protection
 - (m) Attribute: Password
 - (m) Attribute: Access Groups
 - (m) Attribute: Access Rights
- (o) Attribute: Extension

12.2.3.2.2 Attributes

Domain name, index:

These attributes allow identifying in an unique way the object domain. The types of identifier supported are either the name or the index or both.

State:

This attribute specifies the state of the Domain object in compliance with the state chart described below.

Predefined:

This attribute can take the values TRUE or FALSE. The FALSE value authorises the service Delete Domain to delete the object. In a pre-existent Domain Object this attribute is initialized at TRUE.

Sharable:

This attribute takes the value TRUE if the Domain object can be used in several program invocations simultaneously.

List of Program Identifications:

This attribute contains the list of program invocations which use this Domain object. The list contains only a single identification of PI if the Sharable attribute is at FALSE.

Upload In Progress:

This attribute indicates the number of uploads being executed on this domain.

Domain Access Protection:

This attribute indicates if the object supports an access protection.

Password:

This attribute contains the value of the password accepted for the access rights.

Access Groups:

This attribute specifies if the object belongs to a group of users, as shown in Table 497. If one of the bits is set to 1 it indicates the number of the group to which the object belongs.

Table 497 – Access group attribute description for domain object

Bit name	Access group number
G0	Access group 8
G1	Access group 7
G2	Access group 6
G3	Access group 5
G4	Access group 4
G5	Access group 3
G6	Access group 2
G7	Access group 1

Access Rights:

This attribute contains the access rights associated with the Domain object, as shown in Table 498. Each bit, positioned to 1 indicates the acceptance conditions of the loading, saving or utilization services in a program invocation.

Table 498 – Access rights attribute description for domain object

Name	Significance
R	UPLOAD authorized to clients who have provided a password identical to the Password attribute of the object.
W	DOWNLOAD, Deleting authorized to clients who have provided a password identical to the Password attribute of the object
U	Utilization in a PI authorized to clients who have provided a password identical to the Password attribute of the object
Rg	Upload authorized to clients belonging to one of the groups specified in the Access Rights attribute of the object.
Wg	Download, Deleting authorized to clients belonging to one of the groups specified in the Access Rights attribute of the object
Ug	Utilization in a PI authorized to clients belonging to one of the groups specified in the Access Rights attribute of the object
Ra	Upload authorized to all the clients
wa	Download, Deleting authorized to all the clients
Ua	Utilization in a PI authorized to all the clients

Extension:

This attribute contains additional information.

12.2.3.3 Services**12.2.3.3.1 Confirmed delete domain service****12.2.3.3.1.1 Functionality**

This service allows a client, under certain conditions, to delete an existing domain.

12.2.3.3.1.2 Service primitives

The structure of the service is shown in Table 499.

Table 499 – Confirmed delete domain service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument**Domain identification:**

This parameter contains the identification by name or by index of a Domain object.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.3.3.1.3 Service procedure

The VMD should perform the following actions:

- check that there is an existing domain with the same identification,
- check that the predefined attribute is set to false,
- check that the Domain is in the ready state and that there is no uploading in progress,
- if the request is transmitted via a channel with the service quality 2, check the compatibility of the access rights of the channel with the password attributes, access group and access rights of the object,
- delete the object Domain in question.

If all the actions are executed with success, a positive report is transmitted, otherwise a negative report is transmitted explaining the reasons for the failure.

12.2.3.3.2 Confirmed initiate download sequence service

12.2.3.3.2.1 Functionality

There are two types of domains, those which are created dynamically by the Initiate Download Sequence service and those which are born partially or completely loaded at power up. In the latter case, the domain is said to be pre-existent.

The Initiate Download Sequence service allows a client to:

- 1 - initialize the loading of a pre-existent domain if it is in the "predefined" state,
- 2- create the domain object then initialize the loading if the domain is inexistent.

12.2.3.3.2.2 Service primitives

The structure of the service is shown in Table 500.

Table 500 – Confirmed initiate download sequence service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain Identification	M	M=		
List of capabilities	M	M=		
Sharable	M	M=		
Domain Access Protection	C	C=		
Password	M	M=		
Access Group	M	M=		
Access Right	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument

Domain identification:

This parameter contains the identification by name or by index of the domain to be downloaded.

List of Capabilities:

This parameter generally contains specific information concerning limitations of implementation. Its content can be empty.

Sharable:

The "sharable" parameter is a Boolean; if its value is true, the Domain can be divided by several PIs.

Domain Access Protection:

Its presence is compulsory if the service is transmitted via a channel with the quality of service (2).

This parameter contains the value to be assigned to the Password, Access Groups, Access Rights attributes of the object domain created. If the Domain is of the pre-existent type, this parameter is overlooked.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or is not accepted by the VMD. An error code is provided.

12.2.3.3.2.3 Service procedure

The VMD should:

- initialize the loading if the domain is Pre-existent on condition that its status has the "predefined" value,
- create an object domain then initialize the loading if the domain is inexistent,
- check the presence of the Domain Access Protection parameter if the request is transmitted via a channel having service quality 2;
- initialize the attributes of the domain as indicated below:
- initialize the attribute identification with the Name given in parameter or the value of the index if the object domain is created,
- initialize the "status" attribute at the "Loading" value,
- the predefined attribute is initialized at "true" if the domain is preexistent. Otherwise it takes the value "false".
- initialize the "Sharable" attribute in compliance with the sharable parameter;
- initialize the Password, Access Group and Access Right attributes of the created domain in compliance with the Domain Access Protection parameter if it is present in the request. If the domain is of the pre-existent type, its Password, Access Group and Access Right attributes remain unchanged.

If all the actions have been executed successfully, a positive report is transmitted. Otherwise a negative report shall be transmitted mentioning the reasons for the failure.

12.2.3.3.3 Confirmed download segment service**12.2.3.3.3.1 Functionality**

This service allows the Server to ask the Client for a segment of data of the domain.

12.2.3.3.3.2 Service primitives

The structure of the service is shown in Table 501.

Table 501 – Confirmed download segment service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain Identification	M	M=		
Result(+) (Comp)			S	S=
Load Data			M	M=
More Follow			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Domain identification:

This parameter contains the identification by name or by index of an object Domain.

Result(+)

Load data:

This parameter, of the OCTETSTRING type, contains the data to be loaded.

More follow:

This parameter of the Boolean type, if "true" informs the VMD that data remains to be loaded.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.3.3.3 Service procedure

The Client should perform the following actions:

- prepare a segment of data of the domain for the "Load data" parameter; the segmentation of the data is a local issue,
- correctly position the value of the "more follow" parameter.

If all the actions are successfully executed, a positive report including the "Load data" and "more follow" parameters is transmitted. If not a negative report shall be transmitted explaining the reasons for the failure.

12.2.3.3.4 Confirmed terminate download sequence service

12.2.3.3.4.1 Functionality

This service allows the Server:

- 1 to inform the Client that it is authoritatively stopping the loading because the received data are not consistent,
- 2 to inform the Client that it has received all the data of the domain and that it has checked their consistency.

12.2.3.3.4.2 Service primitives

The structure of the service is shown in Table 502.

Table 502 – Confirmed terminate download sequence service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain Identification	M	M=		
Discard	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument**Domain identification:**

This parameter contains the identification by name or by index of the Domain.

Discard:

This parameter informs the Client if the loading and consistency of all the data of the domain have been successful, otherwise it indicates the reasons for the failure.

Result(+)

The service is valid and has been correctly executed by the Client.

Result(-)

The service is invalid or has not been accepted by the Client. An error code is provided.

12.2.3.3.4.3 Service procedure

The Server should perform the following actions:

- check that all the data of the domain have been correctly received,
- check the consistency of the data of the domain.

If all the actions have been successfully executed, a request including the parameters "Discard = empty" is transmitted, otherwise a request shall be transmitted including the parameter "Discard = reasons for the failure".

In spite of the success of the loading, if the Client answers negatively, the Server shall force the status of the domain to the "predefined" value if the latter is of the "pre-existent domain" type. Otherwise it shall delete the domain object.

12.2.3.3.5 Confirmed initiate upload sequence service**12.2.3.3.5.1 Functionality**

This service allows a Client to upload the data of a domain belonging to a server. This uploading is carried out through an upload object (ULSM) created for this purpose.

12.2.3.3.5.2 Service primitives

The structure of the service is shown in Table 503.

Table 503 – Confirmed initiate upload sequence service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain Identification	M	M=		
Result(+) (Comp)			S	S=
ULSM Index			M	M=
List Of Capabilities			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Domain identification:

This parameter contains the identification by name or by index of the domain to be saved.

Result(+)

ULSM Index:

This parameter informs the Client of the identification under which the domain saving should take place.

A non-significant value can be transmitted in this parameter to signify that the saving procedure shall be performed with identification of the domain.

List of Capabilities:

This parameter generally contains specific information concerning the limitations of implementation. Its content can be empty.

Result(-)

The service is invalid or is not accepted by the VMD. An error code is supplied.

12.2.3.3.5.3 Service procedure

The VMD should perform the following actions:

- check the existence of the domain,
- check that its status has for value "Ready" or "In-Use",
- create an ULSM object and assigning it an identifier called "ULSM index". If the value of the ULSM index is not significant, the continuation of the upload should be done with the identifier forming part of the request.
- prepare the content of the "ULSM index" and "List of capabilities" parameters,
- increment the Upload in Progress attribute of the Domain object.

If all the actions have been executed with success, a positive report including the "ULSM index" and "List of capabilities" parameters is transmitted. Otherwise a negative report shall be transmitted including the reasons for the failure.

12.2.3.3.6 Confirmed upload segment service

12.2.3.3.6.1 Functionality

This service allows a client to obtain the content of a Domain.

12.2.3.3.6.2 Service primitives

The structure of the service is shown in Table 504.

Table 504 – Confirmed upload segment service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Upload Identification	M	M=		
ULSM Index	S	S		
Domain Identification	S	S		
Result(+) (Comp)			S	S=
Load Data			M	M=
More Follow			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument**Upload identification:**

This parameter indicates either the identification of the ULSM object of the upload created in the initialization phase or the identification (name or index) of the domain object to be uploaded.

Result(+)**Load Data:**

This parameter contains the data transmitted by the server.

More Follows:

This parameter of the Boolean type has the value "True" if the uploading operation shall continue.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.3.3.6.3 Service procedure

The Sub-MMS Server provides in the response the content of a data segment to be uploaded. The More Follow parameter is initialized at FALSE if there is no more information to be transferred.

12.2.3.3.7 Confirmed terminate upload sequence service**12.2.3.3.7.1 Functionality**

This service terminates an uploading sequence.

12.2.3.3.7.2 Service primitives

The structure of the service is shown in Table 505.

Table 505 – Confirmed terminate upload sequence service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Upload Identification	M	M=		
ULSM index	S	S=		
Domain Identification	S	S=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument

Upload identification:

This parameter indicates either the identification of the upload USLM object created in the initialization phase or the identification (name or index) of the domain object to be uploaded.

Result(+)

The uploading operation is terminated with success.

Result(-)

The service is invalid or has not been accepted by the VMD. An error code is provided.

12.2.3.3.7.3 Service procedure

The Sub-MMS server deletes the ULSM upload object dedicated to this operation. It decrements the value of the Upload in Progress attribute.

A negative response is transmitted if the Terminate Upload Request service is received whereas the last segment had been transmitted with the More Follow parameter equal to TRUE.

12.2.3.3.8 Confirmed get domain attribute service

12.2.3.3.8.1 Service primitives

The structure of the service is shown in Table 506.

Table 506 – Confirmed get domain attributes service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Domain Identification	M	M=		
Result(+) (Comp)			S	S=
List of Identification			M	M=
Object Class			M	M=
List Of Capabilities			M	M=
Domain State			M	M=
Predefined			M	M=
Sharable			M	M=
List Of Program Invocation			M	M=
Upload in Progress			M	M=
Domain Access Protection			C	C=
Password			M	M=
Access Group			M	M=
Access Right			M	M=
Extension			U	U=
Result(-)			S	S=
Error Type			M	M=

Argument**Domain identification:**

This parameter indicates the identification of a domain object (by name or by index) whose attributes shall be retrieved in the response.

Result(+)

This parameter indicates that the attribute retrieval operation has been successfully accomplished.

List of Identification:

This parameter gives the identification of the object

Object class:

This parameter indicates the object class

List Of Capabilities:

This parameter indicates the characteristics of the domain.

Domain State:

This parameter indicates the state of the domain.

Predefined:

This parameter indicates if the domain is predefined.

Sharable:

This parameter indicates if the domain can be shared by several PI objects.

List Of Program Invocation:

This parameter indicates the list of PIs sharing the domain.

Upload in Progress:

This parameter indicates if an upload is in progress.

Domain Access Protection:

This parameter is present if the object supports access protection. In this case it comprises the following information:

Password

This parameter indicates the password for the object.

Access Groups

This parameter indicates the groups to which the object belongs.

Access Rights

This parameter indicates the access rights required to have access to the object.

Extension:

This parameter gives additional information.

Result(-)

The service is invalid or has not been accepted by the VMD. An error code is provided.

12.2.3.3.8.2 Service procedure

The Sub-MMS server performs the following operations:

- to check that there is a domain with the same identification,
- read the attributes of the domain.

If all the operations have taken place correctly, the server provides in its positive response the values of the attributes read.

If one of the operations end in a failure, the server provides in the negative response the reasons for the failure.

Whatever the quality of service of the channel via which this service is routed, no access restriction shall be applied via the object access protections.

12.2.3.4 Mechanisms

12.2.3.4.1 State charts

Figure 142 shows the Domain management states and state transitions.

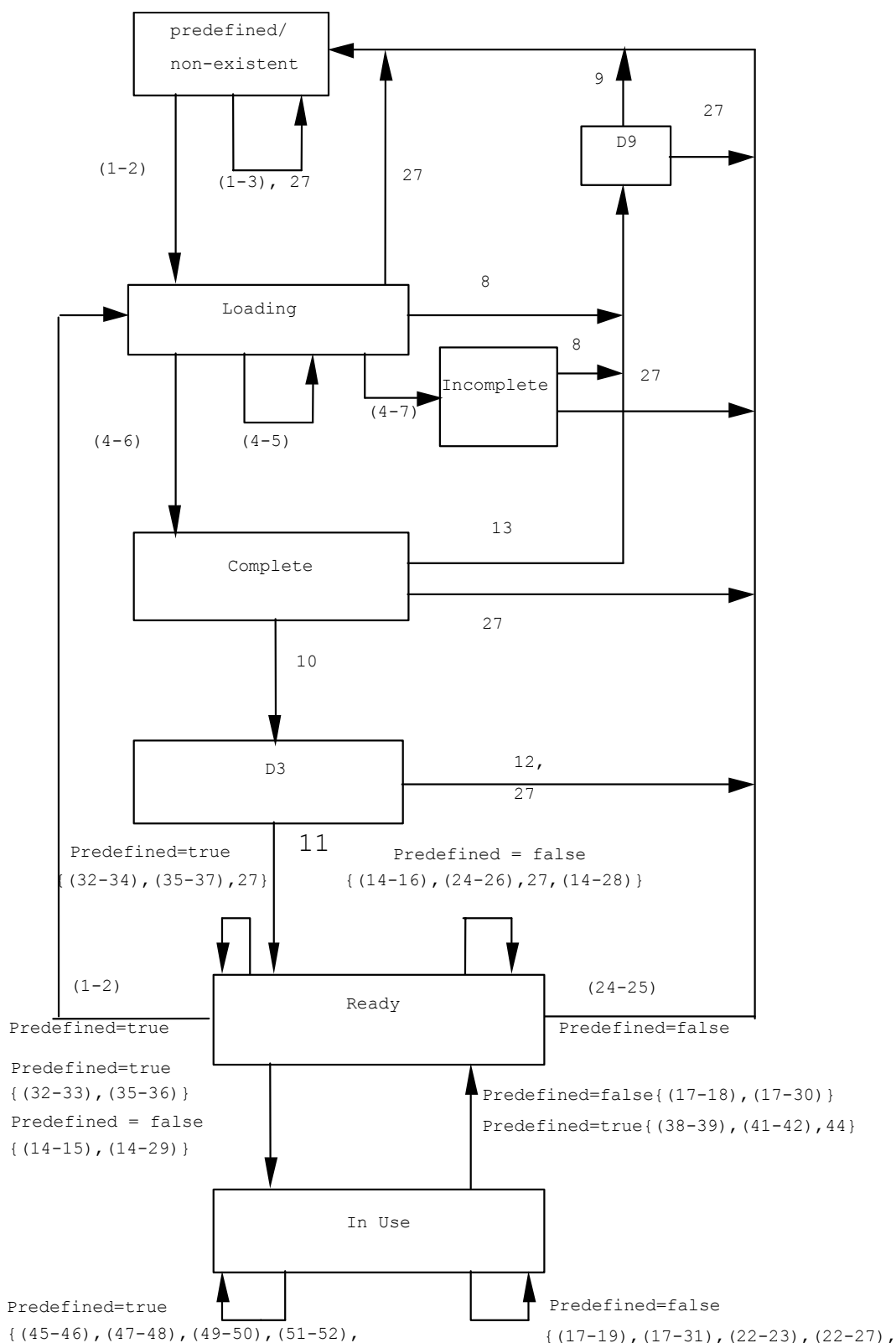


Figure 142 – Domain management state chart

NOTE This state chart is compatible with its MMS and FMS equivalent. The execution of a service indication is done automatically. Consequently only steady states are represented on the diagram. The reading of the attributes of a domain is possible only if the domain is completely loaded. Consequently, only the "ready" or "in-use" states are visible via the network

The transitions are as follows:

- 1- Initiate Download Sequence. Indication
- 2- Initiate Download Sequence. Response(+)
- 3- Initiate Download Sequence. Response(-)
- 4- Download Segment. Request
- 5- Download Segment. Confirm(+) More follow=true
- 6- Download Segment. Confirm(+) More follow=false
- 7- Download Segment. Confirm(-)
- 8- Terminate Download Sequence. Request Discard present
- 9- Terminate Download Sequence. Confirm(+)or (-)
- 10- Terminate Download Sequence. Request Discard not present
- 11- Terminate Download Sequence. Confirm(+)
- 12- Terminate Download Sequence. Confirm(-)
- 13- Terminate Download Sequence. Request Discard present
- 14- Create Program Invocation. Indication (program count=0)
- 15- Create Program Invocation. Response(+)
- 16- Create Program Invocation. Response(-)
- 17- Delete Program Invocation. Indication (program count=1)
- 18- Delete Program Invocation. Response(+)
- 19- Delete Program Invocation. Response(-)
- 20- Create Program Invocation. Indication (program count> 0)
- 21- Create Program Invocation. Response(+) or (-)
- 22- Delete Program Invocation. Indication (program count>1)
- 23- Delete Program Invocation. Response (+) or (-)
- 24- Delete Domain. Indication
- 25- Delete Domain. Response(+)
- 26- Delete Domain. Response(-)
- 27- Abort. Indication
- 28- Abort. Indication (program invocation creation failed)
- 29- Abort. Indication (program invocation creation succeeded)
- 30- Abort. Indication (program invocation deletion succeeded)
- 31- Abort. Indication (program invocation deletion failed).
- 32- Start.Indication (Program Count = 0)
- 33- Start.Response(+)
- 34- Start.Response(-)
- 35- Resume.Indication (Program Count = 0)
- 36- Resume.Response (+)
- 37- Resume.Response (-)
- 38- Stop.Indication (Program Count = 1)

- 39- Stop.Response (+)
- 40- Stop.Response (-)
- 41- Kill.Indication (Program Count = 1)
- 42- Kill.Response (+)
- 43- Kill.Response (-)
- 44- Program Stopped or End of Program (Program Count = 1)
- 45- Start.Indication (Program Count > 0)
- 46- Start.Response (+) or (-)
- 47- Resume.Indication (Program Count > 0)
- 48- Resume.Response (+) or (-)
- 49- Stop.Indication (Program Count > 0)
- 50- Stop.Response (+) or (-)
- 51- Kill.Indication (Program Count > 0)
- 52- Kill.Response (+) or (-)
- 53- Program Stopped or End of Program (Program Count > 0).

NOTE This flowchart is compatible with its MMS and FMS equivalent. The service indications for the server are executed automatically. Consequently, only the stable states are included in the flowchart.

Figure 143 shows the Domain upload states and state transitions.

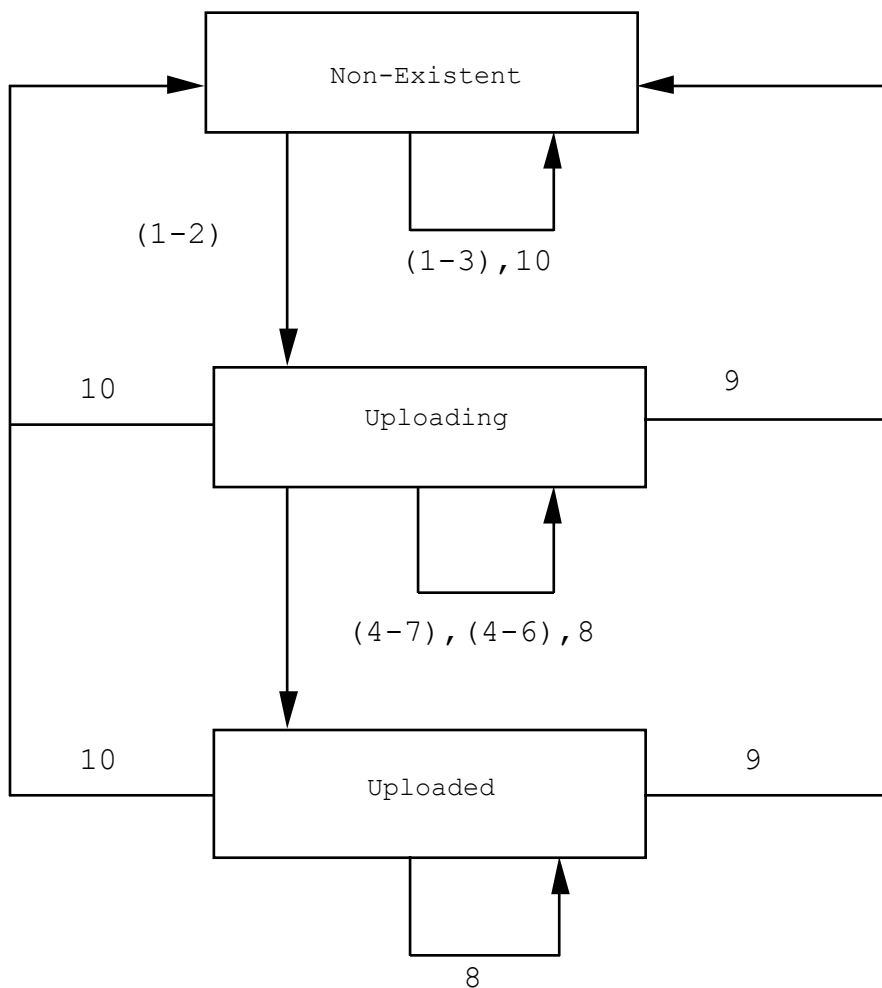


Figure 143 – Domain upload flowchart

Description of the Transitions

- 1- Initiate Upload Sequence.Indication
- 2- Initiate Upload Sequence.Response(+)
- 3- Initiate Upload Sequence.Response(-)
- 4- Upload Segment.Indication
- 5- Upload Segment.Response(+) more follows = false
- 6- Upload Segment.Response(+) more follows = true
- 7- Upload Segment.Response(-)
- 8- Terminate Upload Sequence.Indication
- 9- Terminate Upload Sequence.Response (+) or (-)
- 10-Abort.Indication or Abort.Request

12.2.3.4.2 Sequencing of the services

The Initiate Download Sequence, Download Segment and Terminate Download Sequence services shall observe the sequence shown in Figure 144.

CLIENT	SERVER
Initiate Download Sequence.Request	----->
<-----	Initiate Download Sequence.Response
<-----	Download Segment.Request
Download Segment.Response	----->
.....	
.....	
<-----	Terminate Download Sequence.Request
Terminate Download Sequence.Response	----->

Figure 144 – Domain download sequence diagram

The Initiate Upload Sequence, Upload Segment and Terminate Upload Sequence shall observe the sequence shown in Figure 145.

CLIENT	SERVER
Initiate Upload Sequence.Request	----->
<-----	Initiate Upload Sequence.Response
Upload Segment.Request	----->
<-----	Upload Segment.Response
.....	
.....	
Terminate Upload Sequence.Request	----->
<-----	Terminate Upload Sequence.Response

Figure 145 – Domain upload sequence diagram

12.2.4 Program invocation (PI) ASE

12.2.4.1 Overview

Sub-MMS defines an object "Program invocation" which can be created/deleted either by the adequate Sub-MMS services or by a local initiative.

The creation of a PI by the Create PI service and the deleting of a PI by the Delete or Kill services is done atomically (not interruptible by another service)

- 1 - Create Program Invocation
- 2 - Delete Program Invocation
- 3 - Start
- 4 - Stop
- 5 - Resume
- 6 - Reset
- 7 - Kill
- 8 - Get Program Invocation Attributes

12.2.4.2 Program invocation class specification

12.2.4.2.1 Program invocation model

FAL ASE: Program invocation ASE

CLASS: Program invocation

CLASS ID: not used

PARENT CLASS: not used

Object: Program Invocation

(m) Key Attribute: PI name

(m) key Attribute: PI index

(m) Attribute: PI state (IDLE, STARTING, RUNNING, STOPPING, STOPPED, RESUMING, RESETTING, UNRUNNABLE)

(m) Attribute: List of Domain identification

(m) Attribute: Sub-MMS Deletable (True, false)

(m) Attribute: Reusable(true, false)

(m) Attribute: Execution Argument

(o) Attribute: PI Access Protection

(m) Attribute: Password

(m) Attribute: Access Groups

(m) Attribute: Access Rights

(o) Attribute: Extension

12.2.4.2.2 Attributes

PI Name and index:

These attributes allow unique identification of a Program Invocation object. The types of identifier supported are either a name, or an index or both.

PI State:

This attribute specifies the state of the PI object in compliance with the status diagram described below.

List of Domain Identification:

This attribute contains the list of Domain objects which compose this program invocation.

Sub-MMS Deletable:

The attribute can take the TRUE or FALSE values. The TRUE value authorizes the Delete PI service to delete the object.

Reusable:

This attribute indicates if the program invocation passes after execution in the IDLE or UNRUNNABLE state. The TRUE value signifies a transition towards the IDLE state and the FALSE value a transition towards the UNRUNNABLE state.

Execution Argument:

This attribute contains information concerning execution of the program.

PI Access Protection:

This attribute indicates if the object supports the access protection.

Password:

This attribute contains the value of the password accepted for the access rights.

Access Groups:

This attribute specifies if the object belongs to a group of users, as shown in Table 507. If one of the bits is set to 1, it indicates a group number to which the object belongs.

Table 507 – Access group attribute details for program invocation object

Bit name	Access group number
G0	Access group 8
G1	Access group 7
G2	Access group 6
G3	Access group 5
G4	Access group 4
G5	Access group 3
G6	Access group 2
G7	Access group 1

Access Rights:

This attribute contains the access rights associated with the Program Invocation object, as shown in Table 508. Each bit set to 1 indicates the acceptance conditions for the services starting, stopping or deleting a program invocation.

Table 508 – Access rights attribute details for program invocation object

Bit name	Significance
S	The Start, Resume, Reset operation is authorized for clients who have provided a password identical to the password attribute of the object
H	The STOP operation is authorized for clients who have provided a password identical to the password attribute of the object
D	PI deletion (Kill, Delete PI) is authorized for clients who have provided a password identical to the password attribute of the object
Sg	The Start, Resume, Reset operation is authorized for clients who belong to one of the groups specified in the Access Groups attribute of the object
Hg	The STOP operation is authorized for clients who belong to one of the groups specified in the Access Groups attribute of the object
Dg	Deletion of the PI (Kill, Delete PI) is authorized for clients belonging to one of the groups specified in the Access Groups attribute of the object
Sa	The Start, Resume, Reset operation is authorized for all the clients
Ha	The STOP operation is authorized for all the clients
Da	Destruction of the PI (Kill, Delete PI) is authorized for all the clients

Extension:

This attribute contains additional information.

The status chart of a PI is identical to its MMS equivalent. However, in order to make the chart more legible, the intermediate states P1, P2, P3 and P4 concerning the KILL service are not included in the chart. We recall that the Create-PI, Delete-PI and Kill services shall be executed automatically (not interruptible by a service). The intermediate states are never visible via the network and their absence does not change the dynamic behaviour of a PI.

12.2.4.3 Services

12.2.4.3.1 Confirmed create program invocation service

12.2.4.3.1.1 Functionality

The purpose of this service is to allow a client to create, in a VMD, a PI object attached to several Domains.

12.2.4.3.1.2 Service primitives

The structure of the service is shown in Table 509.

Table 509 – Confirmed create program invocation service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Proposed PI Identification	M	M=		
List Of Domain Identification	M	M=		
Reusable	M	M=		
PI Access Protection	C	C=		
Password	M	M=		
Access Groups	M	M=		
Access Rights	M	M=		
Extension	U	U=		
Result(+) (Comp)			S	S=
Final PI Index			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Proposed PI identification:

This parameter contains the identification proposal per name or per index to be assigned by the VMD to the PI object after creation.

List of Domain Identification:

This parameter contains the identifications by name or by index of the domains to be attached to the created PI object. The limitation of the number (1 to n) of the domains attached to a local issue.

Reusable:

The "Reusable" parameter shall be a Boolean; if its value is true, PI can be executed several times.

PI Access Protection:

This parameter contains the value of the Password, Access Groups, Access Rights attributes to be assigned to the created PI object.

Its presence is mandatory if the service is transmitted via a channel with the quality of service (2).

Extension:

This parameter gives additional information.

Result(+)

Final PI index:

This parameter allows a server to transmit the definitive index value assigned to the created PI object. This index can take a non-significant value to indicate to the client that the identification proposed by name or by index is accepted.

A PI whose proposed identification is a name could always be identified by this name whatever the value of the Final-PI-index parameter.

Result(-)

This service is invalid or not accepted by the VMD. An error code is provided.

12.2.4.3.1.3 Service procedure

The VMD should:

- check that there is no PI with the same identification,
- check that all the domains to be attached to the PI exist and satisfy one of the two conditions: READY state, IN USE state, with the sharable attribute set to TRUE,
- If the request is transmitted via a channel with the service 2 quality, check the presence of PI Access Protection parameter and its compatibility with the password attributes, access groups and access rights of the related domains.
- create a PI object and initialize its attributes as indicated below:
- the identification attribute is initialized with the Name given in parameter or the index value selected (proposed or final) or with both,
- the "state" attribute is initialized to the IDLE value,
- the List of Domain identification attribute is initialized in compliance with the List of Domain Identification parameters.
- the Deletable Sub-MMS attribute is initialized to the TRUE value.
- the "reusable" attribute is initialized in compliance with the Reusable parameter,
- the execution attribute argument is initialized at one empty octet chain,
- the attributes Password, Access Groups and Access Rights attributes are initialized in compliance with the PI Access Protection parameter if it is present in the request.
- Initialize to the value IN USE the state of the domains related to the PI whose value is READY.

If all the actions have been executed with success, a positive report including, if necessary, a final PI index, is transmitted. Otherwise a negative report including the reasons for the failure shall be transmitted.

12.2.4.3.2 Confirmed delete program invocation service

12.2.4.3.2.1 Functionality

This service allows deletion of an object of the PI type in a VMD.

12.2.4.3.2.2 Service primitives

The structure of the service is shown in Table 510.

Table 510 – Confirmed delete program invocation service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.4.3.2.3 Service procedure

The VMD shall:

- check that there is a PI with the same identification,
- check that the PI is located in one of the following states: IDLE, UNRUNNABLE, STOPPED.
- check that the "Sub-MMS" deletable" attribute is true,
- check the compatibility of the access rights of the channel with the password, access group and access rights attributes of the object if the request is transmitted via a channel with the quality of service 2,
- delete the identification of this PI in the Domain objects composing it,
- delete the PI object in question.

If all the actions are executed successfully, a positive report is transmitted, otherwise a negative report indicating the reasons for the failure shall be transmitted.

12.2.4.3.3 Confirmed start service

12.2.4.3.3.1 Functionality

This service allows a Client to activate execution of a PI with a server.

12.2.4.3.3.2 Service primitives

The service structure is shown in Table 511.

Table 511 – Confirmed start service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Execution Argument	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=
Program Invocation State			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Execution argument:

The content of the parameter can be empty. The definition of its contents is a local issue.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid and has not been accepted by the VMD. An error code and the state of the PI are provided.

12.2.4.3.3.3 Service procedure

The VMD should:

- check that there is a PI with the same identification,
- check that the PI is in the "Idle" state,
- Initialize the Execution Argument attribute of the PI object with the Execution Argument parameter value.

If the request is transmitted via a channel with the quality of service 2, check the compatibility of the channel access rights with the password, access groups and access rights attributes of the object.

- start the execution of the PI object in question.

If all the actions are executed with success, a positive report is transmitted, otherwise a negative report shall be transmitted indicating the reasons for the failure.

12.2.4.3.4 Confirmed stop service

12.2.4.3.4.1 Functionality

This service allows a client to stop execution of a PI at a server.

12.2.4.3.4.2 Service primitives

The structure of the service is shown in Table 512.

Table 512 – Confirmed stop service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=
Program Invocation State			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or has been accepted by the VMD. An error code and the PI state are provided.

12.2.4.3.4.3 Service procedure

The VMD shall:

- check that there is a PI with the same identification,
- check that the PI is in the RUNNING state, it then transits to the STOPPING state,

- check the compatibility of the access rights of the channel with the password, access groups and access rights attributes of the object if the request is transmitted via a channel having the quality of service 2.
- stop the execution of the PI object in question.

If all the actions are executed with success, a positive report is transmitted and the object PI is placed in the STOPPED state. Otherwise a negative report shall be transmitted explaining the reasons for the failure and object PI response to the RUNNING state.

12.2.4.3.5 Confirmed resume service

12.2.4.3.5.1 Functionality

This service allows a Client to resume execution of a PI at a Server from its actual stopping point.

12.2.4.3.5.2 Service primitives

The structure of the service is shown in Table 513.

Table 513 – Confirmed resume service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Execution Argument	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=
Program Invocation State			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Execution argument:

The content of the parameter can be empty.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or has not been accepted by the VMD. An error code and the PI state are provided.

12.2.4.3.5.3 Service procedure

The VMD should:

- check that there is PI with the same identification,
- check that the PI is located in the STOPPED state, it then transits to the RESUMING state.
- check the compatibility of the access rights of the channel with the password attributes, access groups and access rights of the object if the request is transmitted via a channel with the quality of service 2,
- resume execution of the PI object in question.

If all the actions are executed with success, a positive report is transmitted and the object PI takes the RUNNING state, otherwise a negative report shall be transmitted explaining the reasons for the failure, the object PI can be placed in the UNRUNNABLE or STOPPED state as required.

12.2.4.3.6 Confirmed reset service

12.2.4.3.6.1 Functionality

This service allows resetting a PI at a server in view of a possible start-up on condition that the "Reusable" attribute has the value "TRUE".

12.2.4.3.6.2 Service primitives

The structure of the service is shown in Table 514.

Table 514 – Confirmed reset service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=
Program Invocation State			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Result(+)

The service is valid and has been correctly executed by the VMD.

Result(-)

The service is invalid or not accepted by the VMD. An error code and the state of the PI are provided.

12.2.4.3.6.3 Service procedure

The VMD shall:

- check that there is a PI with the same identification,
- check that the PI is in the STOPPED state, it then transits to the RESETTING state,
- check the compatibility of the access rights of the channel with the password, access groups and access rights attributes of the object if the request is transmitted via a channel with the quality of service 2,
- reset the PI object in question.

If all the actions are executed with success, a positive report is transmitted and the object is placed in the IDLE state. Otherwise a negative report shall be transmitted explaining the reasons for the failure. The PI can be placed in the UNRUNNABLE or STOPPED state as required.

12.2.4.3.7 Confirmed kill service

12.2.4.3.7.1 Functionality

This service allows forcing a PI at a server to the "Unrunnable" state in view of deletion.

12.2.4.3.7.2 Service primitives

The structure of the service is shown in Table 515.

Table 515 – Confirmed kill service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Program Invocation Identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument

PI identification:

This parameter contains the identification by name or by index of a PI object.

Result(+)

The service is valid and has been executed correctly by the VMD.

Result(-)

The service is invalid or has not been accepted by the VMD. An error code is provided.

12.2.4.3.7.3 Service procedure

The VMD shall:

- check that there is a PI with the same identification,
- check that the PI is located in one of the following states: IDLE, STARTING, RUNNING, STOPPING, RESUMING, STOPPED, RESETTING.
- check the compatibility of the access rights of the channel with the password, access groups and access rights attributes of the object if the request is transmitted via a channel with the quality of service 2,
- force the PI state to the "Unrunnable" value.

If all the actions are executed with success, a positive report is transmitted, otherwise a negative report shall be transmitted indicating the reasons for the failure.

Argument

PI identification:

This parameter contains the identification by name or by index of an object PI.

Result(+)

This selection indicates that the service has been correctly executed by the VMD.

List of identifications:

This parameter gives the identifiers of the PI object.

Object class:

This parameter indicates the class of the object.

Program Invocation State

This parameter indicates the state of the PI.

List Of Domain Identification

This parameter indicates the list of domains attached to the PI.

Sub-MMS Deletable

This parameter indicates if the PI is deletable.

Reusable

This parameter indicates if the PI is reexecutable.

Execution Argument

This parameter indicates the last value supplied as argument either by the Start service or by the Resume service.

PI Access Protection

This parameter is present if the object supports the access protections. In this case it includes the following information:

Password

This information indicates the password of the object.

Access Groups

This information indicates the groups to which the object belongs.

Access Rights

This information indicates the access rights required to obtain access to the object.

Extension:

This parameter gives additional information.

Result(-)

This selection indicates that the service is invalid or not accepted by the VMD.

Error Type

This parameter indicates the reason for the failure.

12.2.4.3.7.4 Service procedure

The VMD should:

- check that there is a PI with the same identification,
- read the PI attributes.

If all the actions are successfully executed, a positive report is transmitted, otherwise a negative report shall be transmitted explaining the reasons for the failure.

Whatever the quality of the service on the channel by which this service is conveyed, no access restriction shall be applied via the object access protections.

12.2.4.4 Mechanisms

Figure 146 shows the Program invocation states and state transitions.

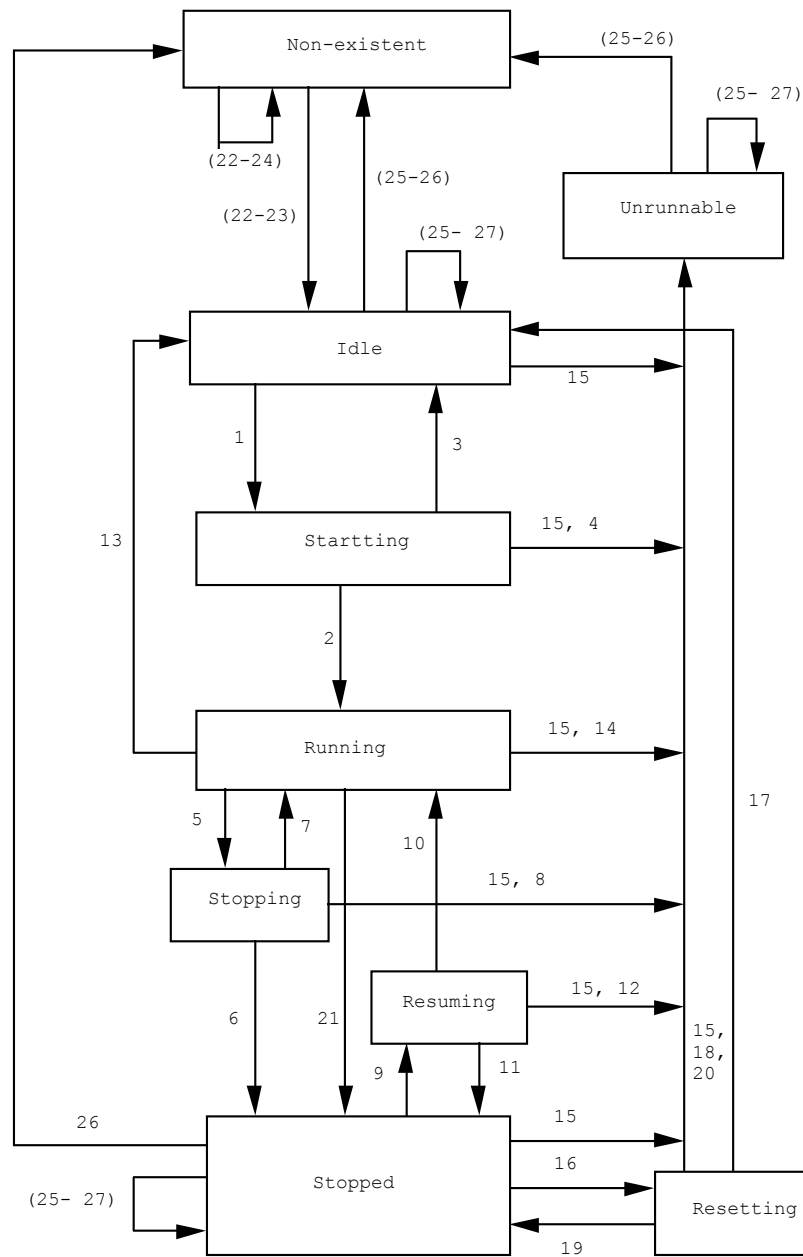


Figure 146 – Program invocation state chart

Description of the transitions:

- 1 -Start.Indication
- 2 -Start.Response(+)
- 3 -Start.Response(-) non-destructive
- 4 -Start.Response(-) destructive
- 5 -Stop.Indication
- 6 -Stop.Response(+)
- 7 -Stop.Response(-) non-destructive

- 8 -Stop.Response(-) destructive
- 9 -Resume.Indication
- 10 -Resume.Response(+)
- 11 -Resume.Response(-) non-destructive
- 12 -Resume.Response(-) destructive
- 13 -(end of program) & Reusable=true
- 14 -(end of program) & Reusable=false
- 15 -Kill.Response(+)
- 16 -Reset.Indication
- 17 -Reset.Response(+) Reusable=true
- 18 -Reset.Response(+) Reusable=false
- 19 -Reset.Response(-) non-destructive
- 20 -Reset.Response(-) destructive
- 21 -(program stopped)
- 22 -CreateProgramInvocation.Indication
- 23 -CreateProgramInvocation.Response(+)
- 24 -CreateProgramInvocation.Response(-)
- 25 -DeleteProgramInvocation.Indication
- 26 -DeleteProgramInvocation.Response(+)
- 27 -DeleteProgramInvocation.Response(-)

12.2.5 Variable ASE

12.2.5.1 Overview

The Sub-MMS defines two classes of objects:

- The Variable object which specifies the access to a simple or structured variable,
- The Variable-List object which specifies access to a series of variable objects.

These two objects can be identified either by name or by index or by both.

Five Sub-MMS services enable manipulating these objects. Total access and partial access are authorized on a Variable object.

The CS can introduce other objects.

The variable management services are as follows:

- 1 - Read (list of variables or a variable-list)
- 2 - Write (list of variables or a variable-list)
- 3 - Information Report (list of variables or a variable-list)
- 4 - Define Variable-List (a variable-list)
- 5 - Delete Variable-List (a variable-list)
- 6 - Get Variable Access Attributes
- 7 - Get Variable List Attributes

12.2.5.2 Variable class specification

12.2.5.2.1 Variable class model

A variable object is never created or deleted by Sub-MMS services. These operations are local issues.

FAL ASE:	Variable ASE
CLASS:	Variable
CLASS ID:	not used
PARENT CLASS:	not used
Object: Variable-	
(m) Key Attribute:	Variable Name
(m) key Attribute:	Variable Index
(m) Attribute:	Type Description
(o) Attribute:	Variable Access Protection
(m) Attribute:	Password
(m) Attribute:	Access groups
(m) Attribute:	Access rights
(o) Attribute:	Extension

12.2.5.2.2 Attributes

Variable name et index:

These attributes allow a unique identification of variable object. The types of identifier supported are either name, or index or both.

Type Description:

This attribute specifies the abstract type of the variable. The authorized types are those defined by "Type Data Specification" (except the Error Type) (See 12.2.5.2).

Variable Access Protection:

This attribute specifies if the object supports the access protection.

Password:

This attribute contains the value of the password accepted for the access rights.

Access Groups:

This attribute specifies if the object belongs to a group of users, as shown in Table 516. If one of the bits is set to 1, it indicates the number of a related group.

Table 516 – Access group attribute details for variable object

Bit name	access group number
G0	access group 8
G1	access group 7
G2	access group 6
G3	access group 5
G4	access group 4
G5	access group 3
G6	access group 2
G7	access group 1

Access Rights:

This attribute contains the access rights associated with the variable object, as shown in Table 517. Each bit set to 1 indicates the acceptable conditions of a read or write service.

Table 517 – Access rights attribute details for variable object

Bit name	Significance
R	Reading authorized to clients who have provided a password identical to the Password attribute of the object
W	Writing authorized to clients who have provided a password identical to the Password attribute of the object.
Rg	Reading authorized to clients belonging to one of the groups specified in the Access Groups attribute of the object.
Wg	Writing authorized to the clients belonging to one of the groups specified in the Access Groups attribute of the object
Ra	Reading authorized for all the clients
wa	Writing authorized for all the clients

Extension:

This attribute contains additional information.

12.2.5.2.3 Variable-list class specification

A Variable-List object allows access to a list of objects of the Variable class. Each element of the list is a variable which can be identified either by a name or by an index. The partial access to a variable is not authorized in the definition of an object of the Variable list class.

FAL ASE:	Variable ASE
CLASS:	Variable list
CLASS ID:	not used
PARENT CLASS:	not used
(m) Key Attribute:	Variable-List Name
(m) Key attribute:	Variable-List index
(m) Attribute:	Sub-MMS Deletable (True,false)
(m) Attribute:	List of Variable identification
(o) Attribute:	Variable-List Access Protection
(m) Attribute:	Password
(m) Attribute:	Access groups
(m) Attribute:	Access rights
(o) Attribute:	Extension

12.2.5.2.4 Attributes**Variable list Name and index:**

These attributes allow unique identification of a variable-list object by using either a name or an index, or both.

Sub-MMS Deletable:

This attribute can take true or false values. The true value authorizes the Delete Variable-List service to delete the object.

List of Variable Identification:

This attribute identifies all the variable objects which compose the object Variable-List. Each object can be identified either by a name or by an index or both.

Variable-List Access Protection

This attribute specifies if the object supports the access protection.

Password:

This attribute contains the value of the password accepted for the access rights.

Access Groups:

This attribute specifies if the object belongs to a group of users, as shown in Table 518. If one of the bits is set to 1, it indicates the number of group to which it belongs.

Table 518 – Access group attribute details for variable list object

Bit name	access group number
G0	access group 8
G1	access group 7
G2	access group 6
G3	access group 5
G4	access group 4
G5	access group 3
G6	access group 2
G7	access group 1

Access Rights:

This attribute contains the access rights associated with the variable-list object, as shown in Table 519. Each bit set to 1 indicates the acceptance condition of a read, write or delete service of a variable-list object.

Table 519 – Access right attribute details for variable list objects

Bit name	Significance
R	Reading authorized for clients who have provided a password identical to the Password attribute of the object
W	Writing authorized for clients who have provided a password identical to the Password attribute of the object
D	Deletion authorized for clients who have provided a password identical to the Password attribute of the object
Rg	Reading authorized for clients belonging to one of the groups specified in the Access Groups attribute of the object.
Wg	Writing authorized for clients belonging to one of the groups specified in the Access Groups attribute of the object
Dg	Deletion authorized for clients belonging to one of the groups specified in the Access Groups attribute of the object
Ra	Reading authorized for all the clients
wa	Writing authorized for all the clients
Da	Deletion authorized for all the clients

Extension:

This attribute contains additional information.

12.2.5.3 Services

12.2.5.3.1 Confirmed read service

12.2.5.3.1.1 Functionality

This service is used by a client to request from a VMD either the global/partial reading of one or several variables or the reading of a variable-list.

The limitation by the server of the number of variables to be read in one service invocation (1 to n) is a local issue.

12.2.5.3.1.2 Service primitives

The structure of the service is shown in Table 520.

Table 520 – Confirmed read service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Type With result	M	M=		
Variable Access Specification	M	M=		
Result(+) (Comp)			S	S=
List Of Access Result			M	M=
List Of Type Data Specification			C	C=
List Of Data Value			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Type With result:

This parameter indicates if the types of variables read should be supplied in the positive response.

Variable Access Specification:

This parameter describes the identification of several variables, elements of variables or of one variable-list to be accessed.

Result(+)

This selection specifies that at least one reading of variable has been carried out with success.

list of Access Results

The list of access results parameter contains the lists of failures/successes of all the variables read, observing the order given by the Variable Access Specification Parameter. In case of reading of a variable list object, the order of the results is consistent with the description of that object.

List of Type Data Specification:

The content of this parameter is empty if the parameter Type With result of the request is equal to false.

The parameter List of Type Data Specification contains the descriptors of the type of variables read with success or the descriptor of the ErrorType for the variables read with failure. This information is supplied observing the order given in the Variable Access Specification parameter. In case the variables are of the same type, the list can be

reduced to a single description. The authorized types are thus those defined by "Type Data Specification" (See 12.2.5.2).

List Of Data Values:

The List of Data Values parameter contains the data corresponding to the values of the variables read with success and the error data of the ErrorType for the variables read with failure. This information is supplied according to the order in the Variable Access Specification parameter or in the description of the Variable-List object. (See 12.2.5.2.3 for the authorized variable values).

Result(-)

The service is invalid or not accepted by the VMD. An error code is formed.

12.2.5.3.1.3 Service procedure

For each variable read, the VMD provides a data and a report of the operation, observing the order indicated in the request. It analyses the value of the parameter Type With Result to determine whether it should provide in the response, the description of the type of each variable read.

For a reading operation performed on a channel using access protections (quality of service 2) the VMD should check before each operation of elementary reading, the consistency of the access rights.

For each variable read with success, the VMD provides a positive indicator in the List of Access result, its descriptor type, if necessary, in List of Type data Specification and its value in List of Data Values.

For each variable read unsuccessfully, the VMD provides an error indicator in the List of Access Results, the descriptor of the Error Type in the List of Type data Specification and the value of the error in the list of data values.

12.2.5.3.2 Confirmed write service

12.2.5.3.2.1 Functionality

This service is used by a client to write the global or partial value of one or several variables or the value of a Variable List object.

The limitation by the server of the variable numbers (1 to n) to be written in a service invocation is a local issue.

12.2.5.3.2.2 Service primitives

The structure of the service is shown in Table 521

Table 521 – Confirmed write service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Variable Access Specification	M	M=		
List Of Type Data Specification	U	U=		
List Of Data Value	M	M=		
Result(+) (Comp)			S	S=
List Of Access Result			M	M=
List of Error Type			C	C=
Result(-)			S	S=
Error Type			M	M=

Argument**Variable Access Specification:**

This parameter describes the identification of several variables, elements of variables or of a Variable-List object to be written.

List Of Data specifications:

The parameter List of Data Specification contains the descriptors of the type of variables to be written, observing the order in the Variable Access Specification parameter. Its content can be empty when the variables are implicitly known. In case the variables are of the same type, the list can be reduced to a single description. The authorized types are those defined by "Type Data Specification" (except the Error Type) (See 12.2.5.2).

List Of Data Values:

The List of Data Value parameters contains the values of the variables to be written, observing the order in the Variable Access Specification parameter or in the description of the variable list object. The authorized values are those defined by "Data Value" (See 12.2.5.2.3).

Result(+)

This selection specifies that at least one writing has been carried out with success.

List Of Access Result:

The List of Access Result parameter contains the write operation reports of the variables, observing the order indicated in the Variable Access Specification parameter.

List Of Error Type:

The parameter List Of Error Type contains a value of the Error type for each writing of variable ending in a failure, observing the order indicated in the Variable Access Specification parameter.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.5.3.2.3 Service procedure

The VMD checks the consistency between the variables described in the Variable Access specification and the List of Type Data Specification parameter, if it is provided. In case of inconsistency on the type or the number of variables, a negative result is returned. If there is consistency, the VMD writes the values in each specified variable.

On a channel using the access protections, the VMD shall check before each elementary write operation the consistency of the access rights.

A report of the operation is provided in the List of Access Result parameter for each elementary write operation. An error code is given necessarily in the List of Error type parameter for each write failure or insufficiency of access rights.

12.2.5.3.3 Unconfirmed information report service

12.2.5.3.3.1 Functionality

This service is used by a Sub-MMS server to transmit either the global/partial values of one or several variables or the values of a Variable-List object.

The limitation by the server of the number of variables (1 to n) to be transmitted in one service invocation is a local issue.

This service is not confirmed.

12.2.5.3.3.2 Service primitives

The structure of the service is shown in Table 522.

Table 522 – Unconfirmed information report service parameters

Parameter Name	Req	Ind
Argument (comp)	M	M=
Local Detail	U	U=
Variable Access Specification	M	M=
List Of Type Data Specification	U	U=
List Of Data Value	M	M=

Argument

Local Detail:

This parameter of the OctetString type can contain information dependent on implementations. It can be used to transport a priority level.

Its content can be empty if no specific information is transmitted.

Variable Access Specification:

This parameter describes the identification of the variables, elements of variables or Variable-list object whose values are transmitted.

List of Type Data Specification:

This parameter contains the descriptors of the types of variables respecting the order given in the Variable Access Specification parameter. Its content can be empty when the variables are implicitly known. In case the variables are of the same type, the list can be reduced to a single description. The authorized types are those defined by "Type Data Specification" (except that of "Error Type") (See 12.2.5.2).

List Of Data Value:

This parameter contains the values of the variables, observing the order of the variables included in the Variable Access Specification parameter. The authorized values are those defined by "Data Value" (See 12.2.5.2.3).

12.2.5.3.3.3 Service procedure

No service procedure is specified by Sub-MMS.

12.2.5.3.4 Confirmed define variable list service

12.2.5.3.4.1 Functionality

The purpose of this service is to allow a client to create in a VMD a Variable-List object and attach to it a list of variable class objects.

Partial access to the attached variables is not possible.

The limitation by the server of the number of variables to be attached (1 to n) to a Variable-List class object is a local issue.

12.2.5.3.4.2 Service primitives

The structure of the service is shown in Table 523.

Table 523 – Confirmed define variable-list service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Proposed Variable-list Identification	M	M=		
List of Variable Identification	M	M=		
Variable-list Access protection	C	C=		
Password	M	M		
Access Groups	M	M		
Access Rights	M	M		
Extension	U	U=		
Result(+) (Comp)			S	S=
Final Variable-list Index			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Proposed Variable List identification:

This parameter contains the proposal for identification by name or by index to be assigned by the VMD to the object variable-list after creation.

List of variable Identification:

This parameter contains the identifications by name or by index of the variables linked to the object variable-list created. It also fixes the order of the variables composing the list.

Variable List Access Protection:

This parameter contains the value of the Password, Access Groups, Access Rights attributes to be assigned to the object variable-list created.

It is compulsory if the service is performed on a channel with the quality of service (2). Otherwise it is empty.

Extension:

This parameter gives additional information.

Result(+)

Final Variable-List index:

This parameter allows the server to transmit the value of the final index assigned to the created variable-list object. This index can take a non-significant value to inform the client that the identification proposed by name or by index is accepted.

A variable whose proposed identification is a name, could always be identified by this name whatever the value of the Final-VL-index parameter.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.5.3.4.3 Service procedure

If the parameters of the request are acceptable, an object variable-list is created and initialized.

The identification attribute is initialized with the Name given in parameter, or the index value selected (proposed or final) or with both.

The Sub-MMS Deletable attribute is assigned to the True value.

The values of the identifications of the variables composing the list are initialized from the List of Variable specification parameter.

On a channel using the quality of service 2, the Password, Access Rights and Access Group attributes are initialized with the value of the parameters.

After this initialization, a positive report is transmitted;

In a context using the quality of service with access protection, the absence of the Password parameters, access rights and access groups results in transmission of a negative result. Besides, the creation of the variable-List object is conditioned by the values of the access rights of each of the list variables which should be compatible with the access rights proposed by the service.

12.2.5.3.5 Confirmed delete variable-list service

12.2.5.3.5.1 Functionality

This service allow deletion in a VMD of a single object of the variable-list type.

12.2.5.3.5.2 Service primitives

The structure of the service is shown in Table 524.

Table 524 – Confirmed delete variable-list service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Variable-list Identification	M	M=		
Result(+) (Comp)			S	S=
Result(-)			S	S=
Error Type			M	M=

Argument**Variable-list identification:**

This parameter contains the identification by name or by index of a Variable-list object to be deleted.

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

Result(+)

The service is valid and has been correctly executed by the VMD.

12.2.5.3.5.3 Service procedure

If the identification of the Variable-list object corresponds to an existing Variable-list object and the Sub-MMS deletable attribute is true, the delete operation is performed and a positive response is transmitted.

On a channel with access protection, the access protection parameters provided during the opening of the association (negotiated or prenegotiated) are analyzed with respect to the values of the password, access rights and Access groups attributes of the object. If there is inconsistency, the operation is not performed and a negative report is transmitted.

12.2.5.3.6 Confirmed get variable access attributes service**12.2.5.3.6.1 Functionality**

This service allows a client to read, in a VMD, the attributes of a Variable object.

12.2.5.3.6.2 Service primitives

The structure of the service is shown in Table 525.

Table 525 – Confirmed get variable access attributes service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Variable Identification	M	M=		
Result(+) (Comp)			S	S=
List of identifications			M	M=
Object class			M	M=
Sub-MMS deletable			M	M=
Type description			M	M=
Variable Access Protection			C	C=
Password			M	M=
Access Groups			M	M=
Access Rights			M	M=
Extension			U	U=
Result(-)			S	S=
Error Type			M	M=

Argument**Variable Identification:**

This parameter specifies the identification of the object whose attributes are to be restored in the response.

Result(+)

This selection indicates that the service was correctly executed.

List of identification:

This parameter gives the identifiers of the variable object.

Object class:

This parameter indicates the class of the object.

Sub-MMS deletable

This parameter indicates if the object is deletable.

Type Description

This parameter indicates the type of variable. The types permitted are those defined by "Type Data Specification" (except Error Type) (See 12.2.5.2).

Variable Access Protection

This parameter is present if the object supports the access protections. In this case it includes the following information:

Password

This information indicates the object password.

Access Groups

This information indicates the groups to which the object belongs.

Access Rights

This information indicates the access rights required for access to the object.

Extension:

This parameter gives additional information

Result(-)

The service is invalid or not accepted by the VMD. An error code is supplied.

12.2.5.3.6.3 Service procedure

The server shall perform the following actions:

- check that there is a Variable object,
- read the object attributes.

If all the actions are performed correctly, a positive response is transmitted.

If one of the actions is unsuccessful, a positive response is transmitted.

Whatever the quality of service of the channel via which this service is transmitted, no access restriction will be applied by means of object access protection.

12.2.5.3.7 Confirmed get variable-list attributes service

12.2.5.3.7.1 Functionality

This service allows a client to read in a VMD the attributes of a variable-List object.

12.2.5.3.7.2 Service primitives

The structure of the service is shown in Table 526.

Table 526 – Confirmed get variable-list attributes service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Variable-list Identification	M	M=		
Result(+) (Comp)			S	S=
List of identification			M	M=
Object class			M	M=
Sub-MMS deletable			M	M=
List of Variable Identification			M	M=
Variable List Access Protection			C	C=
Password			M	M=
Access Group			M	M=
Access Rights			M	M=
Extension			U	U=
Result(-)			S	S=
Error Type			M	M=

Argument

Variable-list identification:

This parameter indicates the variable-list object whose attributes are to be retrieved.

Result(-)

The service is invalid or not accepted by the VMD. An error code is supplied.

Result(+)

The service is valid and has been correctly executed by the VMD.

List of identification:

This parameter gives the identifiers of the variable list object.

Object class:

This parameter indicates the class of the object.

Sub-MMS deletable

This parameter indicates if it is true that the object is destructible.

List of Variable Identification

This parameter indicates the variables attached to the Variable-List object.

Variable-List Access Protection

This parameter is present if the object supports the access protections. In this case it includes the following information:

Password

This information indicates the password of the object.

Access Groups

This information indicates the groups to which the object belongs.

Access Rights

This information indicates the access rights required for access to the object.

Extension:

This parameter gives additional information.

12.2.5.3.7.3 Service procedure

The server shall perform the following action:

- check that there is a variable-list object,
- read the attributes of the object.

If all the actions have been correctly executed, a positive response is sent.

If one of the actions is unsuccessful, a negative response is sent.

Whatever the quality of the service of the channel via which this service is conveyed, no access restriction shall be applied through the object access protection.

12.2.5.4 Additional concepts

12.2.5.4.1 Definition of the data specification type

The various descriptors of the type of data for variables are given by the above list. The CS and the sub-MMS users can define other types. See Table 527.

Table 527 – Data type specification

Parameter name	Req	Ind	CBB
Type Data Specification	M	M=	
Array type	S	S=	arr
Number Of element	M	M=	
Type Data Specification	M	M=	
Structure type	S	S=	str
List Of Type Data	M	M=	
Specification	S	S=	
Simple Boolean type	M	M=	
Boolean length	S	S=	
Simple Integer type	M	M=	
Integer length	S	S=	
Simple Unsigned type	M	M=	
Unsigned length	S	S=	
Simple Bitstring type	M	M=	
Bitstring length	S	S=	
Simple Boolean array type	M	M=	
Boolean Array length	S	S=	
Simple Visible type	M	M=	
Visible string length	S	S=	
Simple Octet String type	M	M=	
Octetstring length	S	S=	
Simple Time difference type	M	M=	
Time difference length	S	S=	
Simple Time of day type	M	M=	
Time of day length	S	S=	
Simple single floating type	M	M=	
Single floating length	S	S=	
Simple Double floating type	M	M=	
Double floating length	S	S=	
Simple BCD type	M	M=	
BCD length	S	S=	
Simple Error type	M	M=	
Error length			

The CBB (Conformance Building Block) parameter shows which are the types of variables negotiable at the opening of an association.

- 1- Array type is the description of the type of a table of elements of the same type.
- 2- Structure type is the description of the type of a structure of which each field can be of a distinct type.
- 3- Boolean type is the description of the type of a Boolean.
- 4- Integer type is the description of a type of an Integer.
- 5- Unsigned type is the description of the type of an integer whose values are positive.
- 6- Bitstring type is the description of the type of bit string. The first bit of the string is positioned at bit $2^{*}7$ of the first octet. The second bit is position at bit $2^{*}6$ of the first octet, and so on.
- 7- Boolean Array is the description of a table of bits. The first element of the table is positioned at bit $2^{*}0$ of the first octet. The second is positioned at bit $2^{*}1$ of the first octet, and so on.
- 8- Visiblestring type is the description of the type of a string of octets whose value is an ASCII character to be taken from {"A" to "Z", "a" to "z", "0" to "9", ".", "_"}.
- 9- Octetstring type is the description of the type of a string of octets.
- 10- Time difference type is the description of the Time difference type(Unsigned-32 expressing in milliseconds the time elapsed since midnight).
- 11- Time of day type is la description of the Time Of Day type
- 12- Single floating type is the description of the Single floating point (format IEEE 754).
- 13- Double floating type is the description of the Double floating point (format IEEE 754).
- 14- BCD type is the description of the type in digit string.
- 15- Error type is the description of the Error type.

12.2.5.4.2 Definition of the list of access results

The access result list is a series of indicators indicating the series of successes/failures relative to the access of several variables. This parameter shall be a BITSTRING. A bit set to TRUE (value 1) signifies access with success.

12.2.5.4.3 Variable access specification

12.2.5.4.3.1 Variable access specification parameter

The structure of the Variable Access Specification parameter is given in Table 528.

Table 528 – Variable access specification

Parameter name: Variable Access Specification	Req Rsp	Ind cnf
Kind of Variable	M	M=
Variable-List	S	S=
Identification	M	M=
List Of Variable	S	S=
variable Access description	M	M=

Kind of variable:

This parameter specifies an access to a series of objects of the Variable class with the possibility of obtaining partial access or access to a single object of the Variable-List class previously defined.

Variable List:

This parameter specifies the identification by name or by index of an object variable list.

List Of variables:

This parameter gives an enumeration of the access descriptions to variable class objects.

12.2.5.4.3.2 Variable access description parameter

The structure of the Variable Access description parameter is given in Table 529.

Table 529 – Variable access description attribute details

Parameter name: Variable Access description	Req Rsp	Ind cnf
Kind of Access	M	M=
Global Access	S	S=
Identification	M	M=
Partial Access	S	S=
Identification	M	M=
Path Selection	M	M=

Kind of Access:

This parameter indicates if the variable is accessed globally or partially.

Global Access:

This parameter exclusively contains the name or the index of the variable to be accessed globally.

Partial Access:

This parameter describes the identification of the variable object to be accessed partially followed by the description of the path to be followed to reach one or several elements of the variable.

12.2.5.4.3.3 Path selection parameter

The structure of the path selection parameter is given in Table 530.

Table 530 – Path selection parameters

Parameter name: Path Selection	Req Rsp	Ind cnf	CBB
Kind of selection	M	M=	
Select Access	S	S=	
Kind of access	M	M=	
(1)st level nesting access	S	S=	
Access selection	M	M=	
Component index	S	S=	Str
Element index	S	S=	Arr
(n≥1)th level nesting access	S	S=	
Access selection	M	M=	
List of components	S	S=	Str
Component index	M	M=	
List of Elements	S	S=	Arr
Element index	M	M=	
Range elements	S	S=	Arr
start element index	M	M=	
Number of elements	M	M=	
Select Alternate Access	S	S=	
Access selection	M	M=	
Component index	S	S=	Str
Element index	S	S=	Arr
Path selection	M	M=	

12.2.6 Event ASE

12.2.6.1 Overview

The objects, the services and the mechanisms of events should be capable of adapting themselves to the modest needs of sensors/actuators as well as to the greater demands of programmable controllers/work stations.

- 1- Sub-MMS standardizes the following actions:
 - a) Acknowledging an event resulting from a particular event object; this action uses the Acknowledge Event Notification service. The acknowledgement only has a sense for an event previously notified by the Event Notification service.
 - b) Validating/invalidating an event object; this action makes use of the Alter Event Condition Monitoring service. A valid object can generate events to be notified. An invalid object cannot generate events. The notification of events (resulting from several Event objects of the same type) uses the Event Notification service.
- 2- Sub-MMS authorizes the definition of specific actions by the users of Sub-MMS and/or by the Companion Standards. These actions use the Alter Event Condition Monitoring service.
- 3- Sub-MMS introduces the notion of the event detection mechanism common to several objects.

This detection takes place periodically during a period of time sufficiently short to consider that all these events have been detected at the same instant.

In this case a value which we shall call "Common Occurrence ID" is assigned to the "Event Occurrence ID" attribute (if supported) for all the objects for which an event was detected during this time interval.

The events thus detected resulting from objects of the same type are notified to clients in the same invocation of the "Event Notification" service with the value of the "Event Transaction Occurrence ID" parameter equal to "Common Occurrence ID". To save space, each of these events shall be represented by the truncated "Event Message" attribute of the "Event Occurrence ID" attribute (if it is supported) since its value is identical to the value of the Event Transaction Occurrence ID.

To meet the requirements of more or less complex event objects, Sub-MMS defines a configurable event object.

This configuration allows the creation of the following types of event objects:

- 1- Object not supporting any standard action or action specific to the user/C.S. In this case, the "Event Message" attribute is a structure with only one "Event Data" attribute. This object can collaborate in the detection of events common to several objects.
- 2- Object only supporting the acknowledgement action. In this case the "Event Message" attribute is a structure including three attributes: "Event State", "Event Occurrence ID" and "Event Data". This object can collaborate in the detection of events common to several objects.
- 3- Object only supporting the validation/invalidation action. In this case, the "Event Message" attribute is a structure with two attributes, "Event State" and "Event Data". This object can collaborate in the detection of events common to several objects.
- 4- Object supporting the acknowledgement and the validation/invalidation. In this case the "Event Message" attribute is a structure with three attributes, "Event State", "Event Occurrence ID" and "Event Data".

12.2.6.2 Event class specification

12.2.6.2.1 Event class model

FAL ASE:		Event ASE
CLASS:		Event
CLASS ID:		not used
PARENT CLASS:		not used
Object: Event		
(m)	Key Attribute:	Name
(m)	Key Attribute:	Index
(m)	Attribute:	Event type
	(m)	Attribute: Common event detection support(true, false)
	(m)	Attribute: Acknowledge action support(true, false)
	(m)	Attribute: Enable action support(true, false)
	(o)	Attribute: Extension action support
(m)	Attribute:	Event_Message
	(c)	Attribute: Event State
		(m) Attribute: ack action state(acknowledged, not acknowledged)
		(m) Attribute: enable action state(validated invalidated)
		(o) Attribute: extension actions states
	(c)	Attribute: Event Occurrence ID
	(m)	Attribute: Event data
(o)	Attribute:	Event Access Protection Support
	(m)	Attribute: Password
	(m)	Attribute: Access Group
	(m)	Attribute: Access Rights
(o)	Attribute:	Extension

12.2.6.2.2 Attributes

Key attributes

These attributes indicate if the object supports the identification by Name, Index, or Name and Index at the same time.

Event type

This attribute, of the BITSTRING-16 type, characterizes the object type. It indicates if the object collaborates with other Event-Objects with a common event detection mechanism. It also indicates if the object supports specific actions affecting the "Event State" attribute:

Common event detection support:

It indicates if it is true that the object collaborates with other objects for the same event detection mechanism.

Acknowledge action support:

This attribute indicates if the object supports the action of acknowledgement by using the Acknowledge Event Notification service.

Enable action support:

This attribute indicates if the object supports the "validation/invalidation" action using the Alter Event Condition Monitoring service.

Extension action support:

This attribute indicates if the object supports the specific actions, which are carried out by the Alter Event Condition Monitoring service. These actions could be in conformity with the C.S as well as specific to a user.

Event message

This attribute contains the dynamic information of the object.

Event state

This attribute, of the BITSTRING-16 type, is present if the object supports the actions. It indicates the last remote actions having acted on this object by the acknowledgement of the Acknowledge Event Notification and/or Alter Event Condition Monitoring services:

- a) the "Acknowledge action state" attribute stores the acknowledgement,
- b) the "Enable action state" attribute stores the action "validation/invalidation",
- c) the "Extension actions states" attribute stores the eventual extension actions.

Event occurrence ID

This attribute indicates the identification of the occurrence of an event specific to the object. The attribute is present if the acknowledgement is supported by the object.

The OCTETSTRING-2 type is permitted; the semantic of its contents should be described in the PICS.

Event data

This attribute indicates the value associated to the object Event. The types permitted are those defined by "Data Value" (See 12.2.5.2.3).

Event access protection support

This attribute if present contains the information relative to the protection of the object.

Password

This attribute contains the value of the accepted password for the access rights.

Access group

This attribute specifies if the object belongs to a group of users, as shown in Table 531. If one of the bits is positioned at 1, it indicates an access group number.

Table 531 – Access group attribute detail for event object

bit name	number of access group
G0	Access group 8
G1	Access group 7
G2	Access group 6
G3	Access group 5
G4	Access group 4
G5	Access group 3
G6	Access group 2
G7	Access group 1

Access rights

This attribute contains the access rights associated with the Event object, as shown in Table 532. Each bit positioned at 1 indicates the acceptance conditions of the Acknowledge Event Notification (acknowledgement) and Alter Event Condition Monitoring (validation/invalidation) services.

Table 532 – Access rights attribute details for event object

bit name	Signification
W	Acknowledgement and event reading (by the Acknowledge Event Notification, Get Alarm Summary services) are permitted to the clients having provided a password identical to the object Password attribute
D	Validation/Invalidation and event reading (by the Alter Event Condition Monitoring, Get Alarm Summary services) are permitted to clients having provided a password identical to the object Password attribute
Wg	Acknowledgement and event reading (by the Acknowledge Event Notification, Get Alarm Summary services) are permitted to clients belonging to one of the specified groups in the object Access Groups attribute
Dg	Validation/Invalidation and event reading (by the Alter Event Condition Monitoring, Get Alarm Summary services) are permitted to clients belonging to one of the specified groups in the object Access Groups attribute
Wa	Acknowledgement and event reading (by the Acknowledge Event Notification, Get Alarm Summary services) are permitted to all the clients
Da	Validation/Invalidation and event reading (by the Alter Event Condition Monitoring, Get Alarm Summary services) permitted to all the clients

Extension:

This attribute contains additional information.

The event management services are:

- 1- Event Notification; this service enables notifying the events.
- 2- Acknowledge Event Notification; this service enables event acknowledgement.
- 2- Alter Event Condition Monitoring; this service enables altering the event status.
- 3- Get Alarm Summary; this service enables reading the information associated with one or more events.
- 4- Get Event Condition Attribute: this service enables obtaining the values of certain attributes of an object Event.

12.2.6.3 Services

12.2.6.3.1 Unconfirmed event notification service

12.2.6.3.1.1 Functionality

This service allows the server to notify one or more events. Each even is represented by the Event_Message attribute of an object.

The limitation by the server of the number of events to be notified (0 to n) in one service invocation is a local issue.

When and how this service is used is a local issue.

12.2.6.3.1.2 Service primitives

The structure of the service is shown in Table 533 and Table 534.

Table 533 – Unconfirmed event notification service parameters

Parameter name	Req	Ind
Argument (Comp)	M	M=
Common Information	M	M=
Event Type	M	M=
Event Transaction Occurrence ID	M	M=
User Extension	U	U=
List of Event Identification	M	M=
List of Event_Message Type description	U	U=
Event State type description	C	C=
Event Occurrence ID type description	C	C=
Event Data type description	M	M=
List of Event_Message value	M	M=
Event State value	C	C=
Event Occurrence ID value	C	C=
Event Data value	M	M=

Table 534 – Event type parameter details

Parameter name: Event Type	Req	Ind
Common event detection support	M	M=
Acknowledge action support	M	M=
Enable action support	M	M=
Extension action support	U	U=

Argument

Common Information

This parameter, of the OCTETSTRING type, contains the information relative to the objects in the indicated order:

- a)- "Event Type" indicates a type common to all the object events. This type is characterized by the following information:
- "Commun event detection support",
 - "Acknowledge action support",
 - "Enable action support",

- "Extension action support".
- b)- "Event Transaction Occurrence ID" of OCTETSTRING-16 type, indicates the identification of the transaction occurrence.
- c)- User-Extension contains the information known to the user.

List Of Event Identification

This parameter gives the object list where an Event_Message appears in the service.

List Of Event_Message Type Description

The content of this parameter can be empty.

This optional parameter, when not empty, contains the descriptor of the Event_Message type for each object identified in the service. It is possible to reduce the list of descriptors to a single descriptor when they are identical. Each descriptor of the structure type contains the following information in the indicated order:

- the first item of information indicates the "Event State" type; this information is present only if the object supports actions;
- the second item of information indicates the "Event Occurrence ID" type; this event is present only if the "Common event detection support" parameter is false and the "Acknowledge action support" is true (objects supporting acknowledgement but not collaborating with a common event detection mechanism);
- the third item of information indicates the "Event Data" type; this information is always present.

The types permitted are those defined by the type Data Specification (except Error Type). (See 12.2.5.2).

List Of Event Message Value

This parameter contains the Event_Message value of each object identified in the service. Each value of the structure type contains the following information in the indicated order:

- the first item of information indicates the "Event State" value; this information is present only if the object supports actions;
- the second item of information indicates the "Event Occurrence ID" value; this information is present only if the "common event detection support" parameter is false and the "Acknowledge action support" parameter is true (objects supporting acknowledgement but not collaborating with a common event detection mechanism);
- the third item of information indicates the "Event Data" value; this information is always present.

(See 12.2.5.2.3 for the permitted data values).

12.2.6.3.1.3 Service procedure

The VMD having to indicate events concerning certain event class objects, shall carry out the following operations:

- 1- Initialize the "Event Occurrence ID" attribute (if present) of these objects either at a transaction identifying value if the "Common Event Detection Support" attribute has the real value, or at a value specific to event occurrence.
- 2- Transmission, in an invocation of the Event Notification service, of the detected events whose "Event-Type" attributes of the objects concerned are identical.

The event Objects whose "Acknowledge Action Support" attributes have the Real value shall be acknowledged by the Acknowledge Event Notification service if at least one event occurrence has been transmitted.

12.2.6.3.2 Confirmed acknowledge event notification service

12.2.6.3.2.1 Functionality

This service enable a Client to acknowledge several event occurrences. Each occurrence originates from an event object supporting the acknowledgement.

The limitation by the server of the number of objects to be acknowledged in a service invocation (1 to n) is a local issue.

12.2.6.3.2.2 Service primitives

The structure of the service is shown in Table 535.

Table 535 – Confirmed acknowledged event notification service parameter

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
List Of Event Identification	M	M=		
List Of Event Occurrence ID Type	U	U=		
Description	M	M=		
List Of Event Occurrence ID Values				
Result(+) (Comp)			S	S=
List of Access Result			M	M=
List Error Type			C	C=
Result(-)			S	S=
Error Type			M	M=

Argument

List Of Event Identification

This parameter gives the list of Event objects to be acknowledged.

List Of Event Occurrence Id Type Description

The content of this parameter can be empty.

This parameter, when not empty, contains for each of the objects to be acknowledged, the descriptor of the type of event occurrence. Since all the descriptors are identical, it is possible to reduce the list of descriptors to a single descriptor.

The types permitted are those defined by Type Data Specification (except Error Type). (See 12.2.5.2).

List Of Event Occurrence ID Values

This parameter contains the value of the event occurrences for each of the objects to be acknowledged. (See 12.2.5.2.3 for the permitted data values).

Result(+)

This selection informs the client that the server was able to successfully acknowledge at least one event object among those appearing in the request.

List Of Access Result

This parameter lists the event objects whose acknowledgement was not successful.

List Of Error Type

This parameter indicates the reason for failure for each acknowledgement that ended unsuccessfully.

Result(-)

This selection informs the client that the server was not able to execute the service successfully.

12.2.6.3.2.3 Service procedure

The VMD verifies the coherence between the Event objects described in the List of Event Identification and the List of Event Occurrence ID Type Description parameter, if it is provided. In case of incoherence of type or number a negative result is returned. If there is coherence, the VMD can use the Event Occurrence ID parameter to select the event occurrence(s) to be acknowledged from each of the objects identified by the List of Event Identification parameter.

On a channel using the access protections, the VMD shall verify the coherence of access rights before each acknowledgement operation.

An operation report is supplied in the List of access result parameters for each acknowledgement operation. An error code is obligatorily given in the List of Error type parameters for each acknowledgement failure or insufficiency of access rights.

12.2.6.3.3 Confirmed alter event condition monitoring service

12.2.6.3.3.1 Functionality

This service enables modifying the "Event State" attribute of several Event objects.

The limitation by the server of the number of objects to be modified (1 to n) in a service invocation is a local issue.

12.2.6.3.3.2 Service primitives

The structure of the service is shown in Table 536.

Table 536 – Confirmed alter event condition monitoring service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
List of Event State Action ID	M	M=		
Event Identification	M	M=		
Event State action selection	M	M=		
List Of Event State Type description	U	U=		
List Of Event State Value	M	M=		
Result(+) (Comp)			S	S=
List of Access Result			M	M=
List Error Type			C	C=
Result(-)			S	S=
Error Type			M	M=

Argument

List Of Event State Action ID

This parameter gives the event object list to be modified. Each modification is identified with the help of the two parameters mentioned above:

Event Identification

This parameter gives the identification of Event object.

Event State Action Selection

This parameter gives the identification of the chosen action.

List Of Event State Type Description

The content of this parameter can be empty.

This parameter, when not empty, contains the descriptors of the "Event State" type listed in this service. Since all the descriptors are identical, it is possible to reduce the list of descriptors to a single descriptor.

The types permitted are those defined by type Data Specification (except Error Type). (See 12.2.5.2).

List Of Event State Value

This parameter contains the "Event State" values listed in this service.

(See 12.2.5.2.3 for the permitted data values).

Result(+)

This selection indicates to the client that the server was able to modify successfully at least one element of the Event state attributes of the objects appearing in the request.

List Of Access Results

This parameter lists the "Event State" modifications which were not successfully carried out.

List Of Error Types

This parameter indicates the reason for failure for each unsuccessful modification.

Result(-)

This selection indicates to the client that the server was not able to carry out the service successfully.

12.2.6.3.3.3 Service procedure

The VMD checks the coherence between the Event objects described in the List of Event State Action ID and the List of Event State Type Description parameter, if it is provided. In case of incoherence on the type or number a negative result is returned. If there is coherence, the VMD carries out the modification of the "Event State" attributes.

On a channel using the access protection, the VMD should check the access right coherence before each modification operation.

An operation report is supplied in the List of access result parameter for each modification operation. An error code is mandatorily given in the List of Error type parameter for each modification failure or insufficiency of access rights.

12.2.6.3.4 Confirmed get alarm summary service**12.2.6.3.4.1 Functionality**

This service enables reading the Event_Message attribute of several Event objects. The limitation by the server of the number of objects (1 to n) to be read in a service invocation, is a local issue.

12.2.6.3.4.2 Service primitives

The structure of the service is shown in Table 537.

Table 537 – Confirmed get alarm summary service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Type with result	M	M=		
List of event Identification	M	M=		
Result(+) (Comp)			S	S=
List of access result			M	M=
List of Event_Message Type Description			C	C=
Event State Type Description			C	C=
Event Occurrence ID Type Description			C	C=
Event Data Type Description			M	M=
List of Event_Message Value			M	M=
Event State Value			C	C=
Event Occurrence ID value			C	C=
Event Data Value			M	M=
Result(-)			S	S=
Error Type			M	M=

Argument

Type with result

This parameter indicates if the Event_Message type attributes read shall be supplied in the positive response.

List Of Event Identification

This parameter describes the identification of many Event objects to be accessed.

Result(+)

This selection indicates that at least one "Event_Message" attribute object has been read successfully.

List Of access Results

The List of Access Result parameter contains the failure/success result of all the Event objects read, respecting the order of appearance in the List of Event Identification parameter.

List Of Event Message Type Descriptions

The content of this parameter is empty if the parameter Type With Result of the request is equal to false.

This parameter, when it is not empty, contains the descriptor type of Event_Message attribute of objects read successfully or the ErrorType descriptor for objects read unsuccessfully. This information is provided respecting the order appearing in the List Of Event Identification parameter. In case the "Event_Message" descriptor attributes of the object read are of the same type, the list can be reduced to the description of a single Event_Message.

Each Event_Message descriptor of the structure type includes in the indicated order, the descriptor of the Event State attribute if it is present, the descriptor of the Event Occurrence ID attribute if it is present and the descriptor of the Event Data attribute. The types permitted are those defined by Type Data Specification (except the Error Type). (See 12.2.5.2).

List Of Event Message Values

The List of Event_Message Values parameter contains the Event_Message attribute value of the objects read successfully and the ErrorType value for objects read unsuccessfully. This information is provided respecting the order in which it appears in the List Of Event Identification parameter.

Each Event_Message value in the indicated order consists of the Event State attribute value if it is present, the Event Occurrence ID attribute value if it is present and the Event Data attribute value. (See 12.2.5.2.3 for the permitted data values).

Result(-)

The service is invalid or not accepted by the VMD. An error code is provided.

12.2.6.3.4.3 Service procedure

The VMD provides each accessed object with a data and an operation report respecting the order indicated in the request. It analyses the value of the Type With Result parameter to determine if it should provide in the response the type description of each Event_Message attribute read.

In case of a read operation carried out on a channel using access protection (service 2 quality), the VMD shall check the coherence of access rights before each reading operation.

For each Event_Message attribute read successfully, the VMD provides a positive indicator in the List of Access result, its descriptor type, if necessary, in the List of Event_Message Type Description and its value in List of Event_Message Value.

For each unsuccessful Event_Message attribute read unsuccessfully, the VMD provides an error indicator in the List of Access Result, the ErrorType descriptor in the List of Event_Message Type Description and the error value in the List of Event_Message value.

12.2.6.3.5 Confirmed get event condition attribute service

12.2.6.3.5.1 Functionality

This service allows reading the attributes of an Event object

12.2.6.3.5.2 Service primitives

The structure of the service is shown in Table 538.

Table 538 – Confirmed get event condition attributes service parameters

Parameter Name	Req	Ind	Rsp	Cnf
Argument (Comp)	M	M=		
Event Identification	M	M=		
Result(+) (Comp)			S	S=
List of identification			M	M=
Object Class			M	M=
Event Type			M	M=
Event Message type description			M	M=
Event Access Protection			C	C=
Password			M	M=
Access Groups			M	M=
Access Rights			M	M=
Extension			U	U=
Result(-)			S	S=
Error Type			M	M=

Argument

Event Identification:

This parameter indicates the object whose attributes are to be retrieved.

Result(+)

List of identification:

This parameter indicates the object identifiers.

Object class:

This parameter indicates the class of the object.

Event Type:

This parameter indicates the event object type

Event Message type description:

This parameter indicates the type of Event-Message attribute of the Event object.

Event Access Protection:

This parameter contains three items of information if the object is protected.

Password:

This parameter indicates the password of the object.

Access Groups:

This parameter indicates the groups to which the object belong.

Access Rights:

This parameter indicates the protections of the object.

Extension:

This parameter gives additional information

Result(-)

The service is invalid or not accepted by VMD. An error code is provided.

12.2.6.3.5.3 Service procedure

The server carries out the following actions:

- check the existence of the object in question.
- read the object attributes.

If all the actions were performed correctly, the server transmits a positive response.

If an action ends in a failure, the server transmits a negative response.

Whatever the service quality of the channel via which this service is routed no access restriction shall be applied through the object access protection.

12.2.7 Directory ASE

To be completed later

12.3 ARs

12.3.1 Messaging common service (MCS) AR ASE

This subclause details a Messaging Common Service Element used in the AL environment for monitoring application associations (its role being equivalent to that of ACSE in the ISO environment), and to enable transfer of data units from the application layer in the associated and non-associated modes.

12.3.1.1 Overview

This application service element is intended for use in the AL environment application layer structure.

This common element has necessarily be integrated with the specific element or elements in the context of an application entity. The MCSE is, indeed, directly projected onto the DLL and provides a user interface for receiving a specific service element such as the SubMMS described.

The services offered by this element feature two additional possibilities constituted by transmission in the associated mode and the non-associated mode.

12.3.1.2 Concepts

12.3.1.2.1 Associated mode transmission

Associated mode transmission—establishment of an application association,

- transfer of data,
- termination of an application association.

In addition to these three stages, which correspond to the phases of its life, an application association has the following characteristics:

- it necessitates establishing and maintaining an agreement concerning the transfer of data between the two application entities communicating and the data link layer on which they are based,
- it enables negotiation, between the two application entities, of the user options and parameters which affect the transmission of data,
- it supplies a context to which the successive data units transmitted are logically related,

- it guarantees transfer by reserving resources and ensuring reliable transmission of the data units exchanged,
- it makes it possible to identify the communicating entities, thus avoiding transmitting their designations at each data transfer.

Associated mode transmissions thus more closely meet the requirements of the applications for long communications with a given interaction traffic, such as, for instance, the remote configuring of an item of equipment, and exchanges of control data between equipment in a distributed application. In such cases, the entities involved agree on the nature of the interactions and resources reserved before proceeding with exchanges of data units.

This agreement is obtained by:

- configuring the two entities involved in the association, in the case of a pre-negotiated association,
- negotiation exchanges between the two entities involved in the association, in the case of a negotiated association.

12.3.1.2.2 Non-associated mode transmission

Non-associated mode transmissions between application entities take place in a single data transmission phase, from the source entity to one or more recipient entities, without establishing an association.

This type of transmission, the lifetime of which is limited to that of the exchange of a data unit, has the following properties:

- it is necessary to attach to the data to be transmitted all the necessary information for relaying and interpreting it, such as, for instance, identification of the expected communicating entities, the quality of service desired, the application context to which the data units logically relate etc.,
- it makes no presumptions concerning the quality of the service and the contexts which may be applied, for one or more successive invocations of a service, between a pair or a set of entities.

Non-associated mode transmissions most closely meet the requirements of application for occasional spontaneous exchanges, without establishing, maintaining and terminating an association. For instance, for observing variables of an application by equipment randomly connected to the bus.

In such a case, the source entities involved need to have prior knowledge of the recipient entities to which they wish to address themselves, for the transfer of data to take place correctly.

12.3.1.2.3 Quality of service

12.3.1.2.3.1 Quality of service definition

The term quality of service relates to certain observable properties of a transmission, as described by different quality of service parameters.

Some of the quality of service parameters specifically relate to associated mode services and others to non-associated mode services, as shown in Table 539.

Table 539 – Classification of service quality parameters

Mode	Phase	Parameter
Associated	- Establishment - Transfer - Termination	- Establishment duration - PDU size - Transfer rate - Number of retries - Anticipation factor - SDU size - Termination duration
Non-associated	Data transfer	Priority

In the associated mode, the quality of service offered for the transfer of data results from the negotiation carried out during the establishment phase of the association.

The negotiation may result in reduction of the quality of service proposed by the user. The reduction in quality of service consists in acting on all or part of its parameters. The semantics of these actions are specific to each parameter, and specified in their definitions.

In the non-associated mode, the quality of service offered for the transfer of data is proposed by the source entity and is complied with by the service provider insofar as possible.

The quality of service parameters relating to the associated mode, whose values can be negotiated at the time of opening and association, are the following and are downgraded as indicated:

- PDU size becomes smaller,
- transfer rate becomes smaller,
- the number of retries becomes smaller,
- anticipation factor becomes smaller,
- SDU size becomes smaller.

Some of these negotiable parameters can effectively be negotiated for a specific association.

However, in the case of a specific association, the unnegotiated parameters will include the unnegotiable ones, the negotiable ones not supported and possibly some negotiable ones whose values are known implicitly by the correspondents.

Finally, in the case of a multipoint association which is necessarily pre-negotiated, only the PDU size and transfer rate parameters are supported.

NOTE In the context of a pre-negotiated association, the service quality parameters should be assigned values consistent with the properties of the devices participating in the association.

12.3.1.2.3.2 Duration of the establishment

The duration of the establishment is the maximum time interval between the association opening demand and confirmation of opening. This time interval includes the time taken for turning round the MCS user called from the correspondent, as well as the time taken for transfer of the request and opening response.

The turn-around time is an intrinsic property of the device, the value of which is utilised by the association opening monitoring mechanism.

NOTE The value of this duration of establishment, which is not proposed by the user, is obtained via network management functions.

12.3.1.2.3.3 PDU size

The size of the data units is the maximum size, as a number of bytes, of the protocol data units (MCS-APDU) of the application layer encoded which can be exchanged on an association.

The value of this parameter is negotiated on the basis of the intrinsic properties of the two devices participating in an association.

Indeed, each device globally offers, to all the associations it supports, an application layer protocol data unit maximum size possibility. The maximum size taken in an association is the least of the maximum sizes offered by the two devices participating in it.

The value of this quality of service parameter is thus accounted for by the association monitoring mechanism, which guarantees that all the application data units are smaller than this size.

12.3.1.2.3.4 Transfer rate

The transfer rate is determined, for both directions of transfer of an association, in terms of the guaranteed minimum number of data units of the application layer protocol which it is possible to transmit per unit time.

This transfer rate corresponds to that part of the total rate available in a device for association purposes.

NOTE The allocation of a transfer rate to an association at device level needs to be consistent with the total capacities and rates already allocated.

For reference, the definition of the total rate available in a device at the data link layer is given below. The abbreviations are defined as follows:

- Φ equip = Messaging flow available at the equipment level
- Φ cyc = messaging flow dependent on the cyclic messaging service available in a device
- Φ aper = messaging flow dependent on the aperiodic messaging service available in a device
- φ mac = messaging flow reserved by the microcycle in the various messaging windows. This flow is that which is shared between all system devices for aperiodic messaging.
- φ dem = message exchange request flow, for a given device. This corresponds to the sum of the flows offered by the various periodic identifiers supporting a message request in a device.

Definitions

$$\Phi \text{ equip} = \Phi \text{ cyc} + \Phi \text{ aper}$$

$$\Phi \text{ cyc} = \sum_i [1/T_i]$$

i = each F_MSGcyc queue of the device,

T_i = period of the ID_MSG associated to the queue

$$\Phi \text{ aper} = ((\varphi \text{ dem}) * (1 / \sum_j \varphi \text{ dem}(j))) * (\varphi \text{ mac})$$

j = each system device

$\varphi \text{ dem}$ = request flow of the device in question

It should be noted that:

- the cyclic flow depends only on the frequency of scanning of the equipment cyclic queues by the bus arbitrator,
- the aperiodic flow corresponds to the total available flow on the bus divided between the devices on the basis of the request flow.

NOTE The transfer rate thus allocated to an association in both directions is under network management surveillance.

12.3.1.2.3.5 Number of retries

The number of retries is the maximum number of retries allowed to the MCS service provider subsequent to an acknowledged transmission request made by the user in a point-to-point association.

This parameter is defined globally for both transfer directions and negotiated during the association establishment phase.

Retries continue until the value of this parameter is reached, as long as the acknowledged transmission has not been properly completed.

12.3.1.2.3.6 Anticipation factor

The anticipation factor is the maximum number of data transfers that an entity can have initiated at a given time in an association. This makes it possible to optimize the pass-band available for an association.

The anticipation factor concept is defined for each transfer direction. The anticipation factor thus defined is negotiated during the association establishment phase.

12.3.1.2.3.7 SDU size

The SDU size is the maximum size of a service data unit (MCS-ASDU) which can be submitted by an MCS user, to a given association, and it is designed for both directions of transfer.

This size can be negotiated during opening of an association and is expressed as a whole number of maximum size MCS protocol data units.

This parameter is only negotiated if the association implements segmentation, and in this case has a value greater than 1.

12.3.1.2.3.8 Termination duration

The termination duration is the maximum time interval between the association termination request and termination confirmation. This time interval corresponds to the MCS user turnaround time, as well as the times of request transfer and termination response.

This turnaround time is an intrinsic property of the device, the value of which is utilized by the association closure monitoring mechanism.

NOTE The value of this termination duration, which is not submitted by the user, is obtained via network management functions.

12.3.1.2.3.9 Priority

The concept of priority is specific to the non-association transmission mode.

In such transmissions, the priority assigned by the data transfer source concerns the relative importance of one data transfer relative to another so as to allow for the revocation of certain PDUs to free resources for others of higher priority.

NOTE This concept does not apply in associated mode transmissions as the quantity of resources implemented is reserved prior to a transfer of data. Furthermore, there is no global priority of resources of an association, making it possible to allow for its revocation, thus making it possible to free its resources for another association of higher priority. Only network management functions can allow early freeing of resources of an association despite user requirements.

12.3.1.2.4 Identification of correspondents

Each correspondent involved in an associated or non-associated mode transmission is unambiguously identified by the system by a four-part identifier consisting of:

- “AP title” to identify the AP to which the entity belongs in one of the system devices,
- “AE qualifier” which identifies the AE used in the AP context,
- “API identification” which identifies a specific activity of an AP in operation, specific use of the AE implemented for the requirements of a specific API.

During associated and non-associated mode transmissions, these indications are optionally transmitted. They are transmitted, during the establishment of an association or a non-associated mode data transfer, to make it possible for the user called or the destination or destinations to respectively:

[1] Inform them of the identification of the caller of the source.

This identification of the caller can be shortened to “AP title” and “AE qualifier”, the called entity being able to deduce this information from the link address of the entity which initiated the exchange.

In addition, the “API identification” can also not be transmitted if the entity initiating the exchange proceeds to establish a bilateral relationship between the APIs and the link addresses associated with the AE.

NOTE 1 This means that there is no multiplexing of the APIs at a given link address (of ADAE type).

[2] To verify whether the identification considered by this request is that localised by the link address utilised.

NOTE 2 Indeed, every AE of an AP is projected onto one or more link addresses at directory level prior to any transmission.

[3] Selection of one of the APIs to which the transmission is addressed.

NOTE 3 This can be avoided, in the particular case where any link address is used for transmissions to a single API.

[4] To propose an AEI identification, the creation of which is triggered by establishing an association at the called entity (only applies to the associated mode).

NOTE 4 The called entity can opt to accept or refuse this identification proposal made by the calling entity. In the case of refusal, the value attributed by the called entity is returned to the calling entity in the association opening request response.

The parameters relating to the identification are transmitted to meet the requirements indicated above are shown in Table 540.

Table 540 – Identification parameters

Parameter name	Role			
	[1]	[2]	[3]	[4]
Calling AP title	C			
AE qualifier	C			
API identification	C			
AEI identification	M			
Called AP title		M		
AE qualifier		M		
API identification			M	
AEI identification				M
NOTE In this table, M signifies mandatory and C signifies conditional. These correspond to the symbols defined in the ISO service conventions.				

12.3.1.2.5 Application context

Entities which intercommunicate use a set of rules to construct the PDUs sent and to interpret the PDUs received, these constitute the bases of the application context.

The bases of an application context comprise:

- a specification of one of the ASEs, which make up the AE supporting the application context.

NOTE This means that each association can only implement one of the ASEs which can contain an AE in the AL environment.

- a specification, for the ASE considered in this context, of one of the abstract syntax options supported by the latter in the framework of the AE supporting the context,
- a specification for one of the transfer syntax options supported by the AE.

For a specific application context, its bases have a name manipulated by the MCS services.

All ASE standards which offer MCS mapping, propose contingent universally recognised names. Any other application, whether defined on the basis of standards or not, has a scope specific to the distributed application in which it is implemented.

The name of an application context is structured as follows:

Name of context :: = {

- ASE type (Universal; Application)
- ASE name (Identification)
- Abstract syntax type (Universal; Application)
- Abstract syntax name (Identification)
- Transfer syntax type (Universal; Application)
- Transfer syntax name (Identification) }

NOTE The coding of a context name is proposed in the PDUs of MCS.

In associated mode transmissions, the context name is exchanged during the establishment of the association to enable negotiation of bases of the application context of the future association.

Unlike in non-associated mode transmissions, the context name is exchanged during each transfer contiguous to data.

12.3.1.3 Services

12.3.1.3.1 Overview

The nature of the services described below is shown in Table 541.

Table 541 – List of MCS AR ASE services

Service	Type
A_ASSOCIATE	Confirmed
A_RELEASE	Confirmed
A_ABORT	Unconfirmed
A_DATA	Unconfirmed (ack)
A_UNIDATA	Unconfirmed (ack)
NOTE (ack) = acknowledgement possible	

NOTE The A_DATA and A_UNIDATA are unconfirmed within the meaning of the ISO service convention but nevertheless feature local confirmation which is generated with allowance for acknowledgement or non-acknowledgement, depending on whether this has been requested or not.

12.3.1.3.2 A_ASSOCIATE service

12.3.1.3.2.1 Functionality

This service is used to establish an association to negotiate the parameters which will control the transfer of data between the two AEs participating in it.

12.3.1.3.2.2 Service primitives

The parameters of this service and description of its semantics are described in Table 542 and the accompanying text.

Table 542 – A_Associate service parameters

Parameter name	Req	Ind	Rsp	Cnf
Calling AEI access point	M	M(=)	M	M(=)
Called AEI access point	U	C(=)	M	M(=)
Conformity class	M	M(=)	M	M(=)
“Cyclic flow” option	U	C		
Quality of service	U	C	C	C(=)
Calling entity identification	U	C(=)		
Called entity identification	U	C(=)	U	C(=)
Application context	U	C(=)	U	C(=)
Negotiation result			M	M(=)
User information	M	M(=)	M	M(=)
For data link layer:				
Calling AE address	M	M(=)	M	M(=)
Called AE address	M	M(=)	M	M(=)

Calling AEI access point:

This parameter corresponds to the link address (of ADAEI type), making it possible to localise the AEI of the calling entity. The calling entity obtains it by means of its directory function.

Called AEI access point:

This parameter corresponds to the link address (of ADAEI type), making it possible to localise the AEI of the called entity. This access point can be proposed by the calling entity for use by the called entity. However, whether it is proposed or not, the called entity returns, in its response, the value of the access point used for its AEI.

Conformity class:

This parameter designates the MCS provider elements which are used in the context of a specific association.

The elements involved, which are designated by the associated conformity parameter names, are the following:

- association termination (PV_RE),
- acknowledgement (PH_AK),
- resend/antiduplication (PH_RA),
- segmentation/reassembly (PH_SR),
- flow control (PH_CF).

“Cyclic flow” option:

The cyclic flow option indicates the type of flow required for the association. Indeed, the flow offered by the association, depending on whether it is based on cycling messaging link services for both transmission directions or aperiodic services for either of the directions, will or will not be independent of the messaging requirements of the other system devices (12.3.1.2.3.4).

By means of this parameter, the calling entity can specify, as a condition of establishment, that the association offers a cyclic flow.

Quality of service:

For this parameter, quality of service is a subset of those defined earlier (see 12.3.1.2.3):

- transfer rate,
- number of retries,
- anticipation factor,
- SDU size.

These quality of service parameters are those which are negotiable on the basis of a value proposed by the calling entity.

Identification of the calling entity:

A calling entity is fully identified by four parameters:

- AP title,
- AE qualifier,
- API identifier,
- AEI identifier.

However, of these four parameters, the only ones present will be those which are necessary in view of the operating conditions of the system (see 12.3.1.2.4).

NOTE All these parameters may be absent. Furthermore, no modifications are made by the MCS provider.

Identification of the called entity:

A called entity is fully identified by four parameters:

- AP title,
- AE qualifier,
- API identifier,
- AEI identifier.

A called entity is fully or partially identified in an association establishment request, depending on the requirements for verification and selection adopted, in view of the operating conditions of the system (see 12.3.1.2.4).

NOTE All these parameters may be absent. Furthermore, no modifications are made by the MCS provider.

Application context:

This parameter corresponds to the name of the application context proposed by the calling entity.

The responder returns the same application context name or a different one, but compatible with that proposed.

Negotiation result:

This parameter makes it possible to report the result reserved for the association opening request, i.e. ACCEPT, REJECT_PROVIDER, REJECT_USER.

This result (acceptance or refusal) is exclusively related to the parameters exchanged in the establishment request.

User information:

The specific service elements which invoke this service use this parameter to negotiate other information characterising the association from their point of view.

For the data link layer:

List of the service parameters proposed by the calling entity for the requirements of the data link layer.

Called AE address:

This parameter corresponds to a link address (of the ADAE type) used to localise the AE of the calling entity (see 12.1.7.4.2).

Called AE address:

This parameter corresponds to the link address (of ADAE type) used to localise the AE of the called entity (see 12.1.7.4.2).

12.3.1.3.2.3 Service procedure

The chaining of the primitives leading to successful establishment of the association is shown in Figure 147.

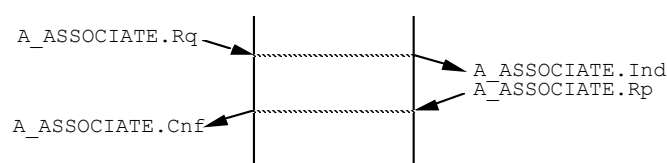


Figure 147 – A_Associate service procedure

This chaining diagram can fail:

- either due to the MCS provider being incapable of addressing the requirements of the calling entity,
- or due to disagreement by the called entity concerning its requirements,
- or due to transmission faults,
- or due to the implementing of ABORT services on this association during establishment.

NOTE In the first three cases, the service is only completed with negative confirmation, whereas in the last there is no confirmation.

The A_ABORT service is, indeed, the only which can be implemented on an association during establishment. This may be due to the action of:

- the calling entity awaiting confirmation of establishment,
- the called entity prior to its establishment response,
- the MCS provider, on specific faults or as a result of intervention at network management level.

NOTE Simultaneous opening is not considered possible as it is postulated that it is impossible to allocate, for two distinct entities, the same access points for localising the association ends. The network management is relied on to avoid this type of problem.

The A_ASSOCIATE service primitives, which are implemented on running the association establishment procedure, perform the following operations:

- A_ASSOCIATE.Rq primitive:

Of the primitives transiting via the calling entity, only the following are processed by the requester element before transmission in the encoded PDU:

- The “conformity class” can be reduced, provided the conformity constraints are complied with , depending on the capacities available.
- The “cyclic rate” option, when it has the value YES, can impose a pure cyclic messaging flow at link level. Consequently, one of the cyclic messaging identifiers for which this device is configured is adopted for the association in accordance with the criteria specific to the requesting element.
- The following quality of service parameters can be reduced:
 - “Transfer flow”
 - “Anticipation factor”
 - “SDU size”
 - “Number of retries”

- A_ASSOCIATE.Ind primitive:

Of the parameters received from the association establishment requester, only the following are processed by the responder element.

- The “conformity class” can be deduced, provided the conformity constraints are complied with , from the capacities available.
- The “cyclic rate” option, when it has the value YES, can impose a pure cyclic messaging flow at link level. Consequently, one of the cyclic messaging identifiers for which this device is configured is adopted for the association in accordance with the criteria specific to the responder element.
- The following quality of service parameters can be reduced:
 - “PDU size”
 - “Transfer flow”

“Anticipation factor”

“SDU size”

“Number of retries”

- The “called entity identification” is utilised to make the checks arising owing to the presence of the different elements of which it is composed (see 12.3.1.2.4). If the check reveals a discrepancy between this called entity identification considered by the calling entity and the called entities effectively residing in the device which receives the association establishment request, a negative result is returned in the association establishment response.
- The “Application context” can be reduced in accordance with the reduction rules specific to the service element (see 12.1.8.3). If reduction is not possible, a negative result is returned in the association establishment response.

• A_ASSOCIATE.Rp primitive:

Of the parameters which transit via the called entity, the result is the first that has to be considered. Depending on whether it is positive or negative, the nature of the processing carried out before transmission of the establishment response is different.

If the result is negative, any resources previously reserved on reception of the establishment request are freed.

If the result is positive, the quality of service parameters are considered and, for the following, the values are adjusted:

“Transfer rate”

“Anticipation factor”

“SDU size”

“Number of retries”

• A_ASSOCIATE.Cnf primitive:

Of the parameters received from the called entity, the result is the first which has to be considered. Depending on whether it is positive or negative, the nature of the processing performed is different.

If the result is negative, the resources previously reserved on receiving the establishment request are freed.

If the result is positive, the other parameters are considered and, for each of the following, the values are adjusted on the basis of the value received from the responder element.

The parameters adjusted are the following:

- The quality of service:

“PDU size”

“Transfer rate”

“Anticipation factor”

“SDU size”

“Number of retries”

12.3.1.3.3 A_RELEASE service

12.3.1.3.3.1 Functionality

This service is used to terminate an association and, consequently, to free the resources implemented by it, as soon as its current data transfers are completed. Success of this service depends on the willingness of the responder user of this service.

12.3.1.3.3.2 Service primitives

The parameters of this service and description of its semantics are described in Table 543 and the accompanying text.

Table 543 – A_Release service parameters

Parameter name	Req	Ind	Rsp	Cnf
User information Result	U	C(=)	U M	C(=) M(=)

User information:

The specific service elements which invoke this service can use this parameter to exchange additional information concerning revocation.

Result:

Simply indicates whether or not the responder user has accepted this association closure. This parameter takes the following values:

- Accepted
- Refused
- Other (definitions pending)

12.3.1.3.3.3 Service procedure

Chaining of the primitives leading to successful association termination is as indicated in Figure 148.

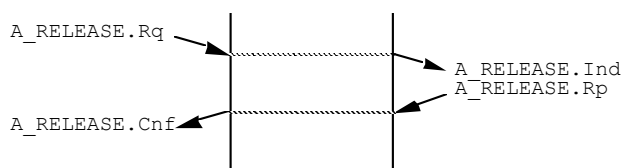


Figure 148 – A_Release service procedure

The termination request can be invoked by either the calling entity or the called entity of the association.

Simultaneous termination requests can occur when a termination request has been made by one of the correspondents of the association which receives a termination request from the other correspondent before it has received confirmation of its request.

In the case of simultaneous requests being detected by the provider, whether at called entity or calling entity level, the termination primitive chaining is broken. The provider proceeds with revocation of the association and freeing of the associated resources, after having invoked the ABORT.Rq service.

It is important to note that the requester of termination of an association cannot activate any primitive of the association other than A.ABORT.Rq until it has received confirmation of termination. Primitives can then be exchanged once more in the association if termination is refused by the corresponding entity.

NOTE It needs to however be borne in mind that a termination request can only be considered for a previously-established association, and that pre-negotiated associations can only be revoked by disabling their configuration under network management control.

Similarly, the responder to the termination request cannot activate any primitive other than A_ABORT.Rq in the association after it has received a termination indication. Primitives can once more be exchanged in the association if the responder local user refuses termination on this indication.

As soon as successful termination is confirmed, the association is revoked and the resources freed. In addition, either of the users of the association can always interrupt the running of the service by using the ABORT service, which revokes the association with the contingent loss of current data transfer.

NOTE Similarly, the running of this service can be interrupted if the network management opts to revoke the association during its creation.

12.3.1.3.4 A_ABORT service

12.3.1.3.4.1 Functionality

This service is invoked either by the user or directly by the MCS provider to revoke an association. Such revocation results in immediate freeing of the resources implemented by the association, which necessarily causes total loss of all current data transfers in it.

12.3.1.3.4.2 Service primitives

The parameters of this service and description of its semantics are shown in Table 544 and the accompanying text.

Table 544 – A_Abort service parameters

Parameter name	Req	Ind
Origin of invocation		M(=)
User information	U	C(=)

Origin of invocation:

Indicates the origin of invocation of this service, which can be invoked by the user or the provider, both at calling entity and called entity levels.

User information:

The specific application service elements which invoke this service utilise this parameter to exchange data units associated with the services that they offer. A service of this type does not guarantee their relaying.

12.3.1.3.4.3 Service procedure

The chaining of primitives relating to the revocation of an association is shown in Figure 149.

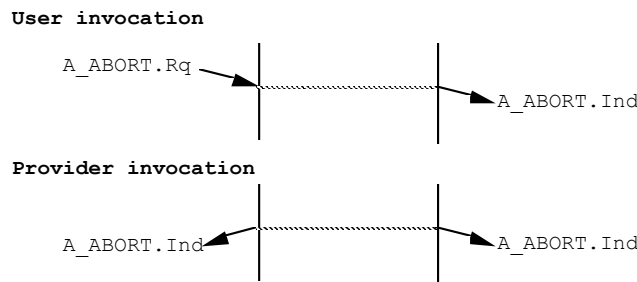


Figure 149 – A_Abort service procedure

A certain number of chaining variants, leading to special chaining, need to be considered:

- The simultaneous requesting of revocation by both the called user and the calling user results in the absence of a revocation indication at both ends.
- The simultaneous invoking of this service by the user and the provider results in there only being the option of invocation of the revocation request by the user who activates this revocation and indication of provider origin revocation at the other end of the association.

Furthermore, it is important to note that this service can be invoked by the provider if a collision should occur during termination of an association.

NOTE It should be borne in mind that a revocation request, whether at the initiative of the user or the provider, can only be considered for a pre-established association and that pre-negotiated associations can only be revoked by disabling their configuration under network management control.

12.3.1.3.5 A_DATA service

12.3.1.3.5.1 Functionality

This service enables exchange of MCS user data in an association. This transfer of data may or may not be acknowledged depending on the request, and only if the association uses the acknowledgement provider element.

In such cases, an acknowledgement provider element multipoint association cannot be used.

NOTE Acknowledgement may be requested systematically, occasionally or never in a given environment, depending on the desired trade-off between reliability of transmission and pass-band usage. However, in the case where segmentation is adopted and/or the anticipation factor is greater than 1, acknowledgement is implicitly utilised whatever the user requests.

12.3.1.3.5.2 Service primitives

The parameters of this service and description of its semantics are described in Table 545 and the accompanying text.

Table 545 – A_Data service parameters

Parameter name	Req	Ind	Cnf
Acknowledgement request	M	M(=)	
User information	M	M(=)	
Transfer result			M

Acknowledgement request:

The acknowledgement of a data unit transferred using this service can be requested using this parameter in the service. The value of this parameter indicates whether the acknowledgement is requested or not. The acknowledgement, if it is requested, is sent by the receiver MCS provider of the data transfer.

This parameter is meaningless when segmentation is adopted, as in this case the acknowledgement provider element is implicitly utilised.

User information:

The specific application service elements which invoke this service utilise this parameter to exchange data associated with the services that they offer.

Transfer result:

This parameter indicates whether transfer has taken place correctly or not. When an acknowledgement request is made, this result provides an indication concerning whether it has arrived in due time, whereas when it is not requested it is a local information item qualifying the sending process.

12.3.1.3.5.3 Service procedure

The chaining of primitives leading to a successful data transfer depending on whether acknowledgement is requested or not is shown in Figure 150.

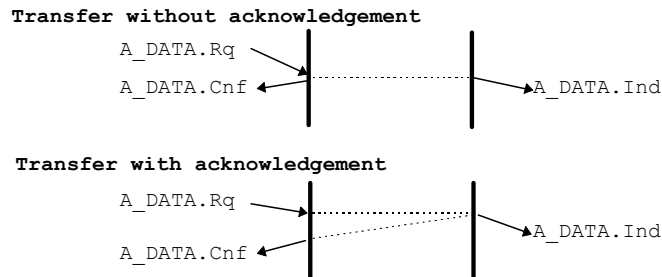


Figure 150 – A_Data service procedure

The chaining of these primitives cannot take place correctly except within the limits set by the quality of service adopted for the association.

NOTE It is thus that, for instance, exceeding of the data unit size by the sources, exceeding of the transfer rate adopted, or exceeding of the service size will result in non-completion of this chaining of primitives.

In the case of a transfer of data with acknowledgement, retries can occur to compensate for the loss of DLPDUs.

In addition, requests and acknowledgements of a number of transfers can be interlaced to optimise the bus traffic.

Finally, in the case of service requests of sizes greater than those of the protocol data units, they are segmented so that they can be relayed.

12.3.1.3.6 A_UNIDATA service

12.3.1.3.6.1 Functionality

This service enables the exchange of MCS user data outside an application association. It may or may not be the subject of an acknowledgement, depending on the operating conditions of the latter.

12.3.1.3.6.2 Service primitives

The parameters of this service and description of its semantics are described in Table 546 and the accompanying text.

Table 546 – A_Unidata service parameters

Parameter name	Req	Ind	Cnf
Acknowledgement request	M	M(=)	
Quality of service	M	M	
Source identification	U	C(=)	
Destination identification	U	C(=)	
Application context	U	C(=)	
User information	M	M(=)	
Transfer result			M
For the data link layer:			
Calling AE address	M	M(=)	
Called AE address	M	M(=)	

Acknowledgement request:

The acknowledgement of a data unit transferred with this service can be requested by means of this service parameter. The value of this parameter indicates whether acknowledgement is requested or not. This acknowledgement, if it is requested, is sent by the receiver MCS provider of the data transfer. Acknowledgement can only be requested for point-to-point data transfers.

Quality of service:

Quality of service is based on the parameters detailed in §4 of this section. Of these service quality parameters, during transfer of data in the non-associated mode, only one priority is transported contiguous with the data.

This priority is proposed by the source MCS user of this service, and determines the exchange of this service at provider level. The latter makes the commitment at source level to:

- account for invoking this service if the resources are free or occupied by other invocations of lower priority not yet transmitted,
- transmit this service invocation unless the invocation is that of lowest priority pending, should another invocation of higher priority occur and all the resources be occupied.

The latter makes the commitment at destination level to:

- receive an invocation of the service if the resources are free or occupied by invocations of lower priority not yet restored to the user,
- restore the invocation to the user, unless the invocation is that of lowest priority pending, when another of higher priority is received and all the resources are occupied.

Source identification:

The source identification identifies the initiator of the non-association data transfer service. A source in the context of the application layer architecture is fully identified by four parameters:

- AP title,
- AE qualifier,
- API identifier,
- AEI identifier.

Of these four parameters, the only ones present will be those necessary in view of the operating conditions of the system.

NOTE It is thus that the API identification is not necessary if each AE qualifier is operated by one single API.

Optionally, the value of this parameter is proposed by the user, which initialises a non-associated mode data transfer request. These parameters are present in the non-associated mode data transfer indication at called entity level, if they were present in the request. As for the value of these parameters in the data transfer indication, it is equal to that proposed by the source in the request.

Destination identification:

The destination identification identifies the receiver of the non-associated mode data transfer. A destination, in the context of the application layer architecture, is totally identified by four parameters:

- AP title,
- AE qualifier,
- API identifier,
- AEI identifier.

A called entity is fully or partially identified in a non-associated mode data transfer request, depending on the requirements of verification and selection adopted, in view of the system operating conditions.

The verification and selection possibilities offered at called entity level are described in 12.3.1.2.4.

NOTE For instance: (1) transit of the destination AP title makes it possible to verify that the AP localised by a link address is effectively that sought by the application; (2) transit of the API identifier makes it possible to select from it one of all those associated with the same AE qualifier.

Optionally, the value of this parameter is proposed by the operator who initialises a non-associated mode point-to-point data transfer request. It cannot be proposed in the case of multipoint data transfers.

Application context:

This parameter identifies the application context in which the source specifies to the destination that the non-associated data transfer shall be interpreted.

The destination, depending on whether or not it supports this context, may or may not carry out the service requested.

Optionally, this parameter is proposed by the user who initialises a non-associated mode data transfer.

NOTE This parameter can, for instance, be transmitted on an *ad hoc* basis during an initial exchange between two entities to enable the source to verify that the destination is capable of interpreting the data units transmitted.

User information:

The specific application service elements which invoke this service utilise this parameter to exchange data units associated with the services that they offer.

Transfer result:

This parameter indicates whether the transfer has taken place correctly or not. Depending on whether or not an acknowledgement request has been made, this indication respectively qualifies the effective arrival of the acknowledgement in due time or the proper local execution of the transmission.

In the event of failure, this information states the type of fault, so as to indicate whether it has occurred locally or at the remote correspondent.

NOTE A fault detected by a remote correspondent provider necessarily results in the latter not supplying data to its user.

For the data link layer:

List of the service parameters proposed by the source user of the service for the requirements of the data link layer.

Source AE address:

This parameter corresponds to a link address (of the ADAE type) used to localise the source AE of the service (see 12.1.7.4.2).

Destination AE address:

This parameter corresponds to the link address (of ADAE type) used to localise the destination AE of the service (see 12.1.7.4.2).

12.3.1.3.6.3 Service procedure

The primitive chaining leading to successful data transfer, depending on whether or not acknowledgement is requested, is shown in Figure 151.

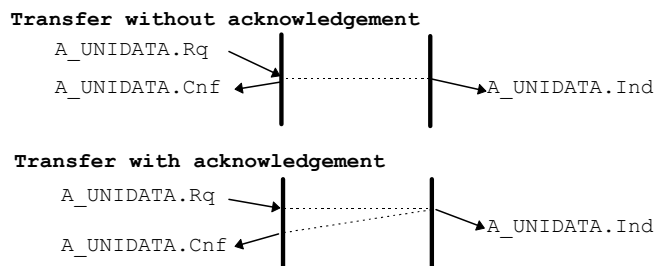


Figure 151 – A_Unidata service procedure

The primitive chaining can only take place correctly if the application context necessary for interpretation of the data is supported at destination level, and if the destination of which the identification is specified exists at receiver entity level.

When the priority requested by the source is not supported by the destination, the latter still carries out primitive chaining, using the priority level closest to that requested.

12.3.1.4 Mechanisms

12.3.1.4.1 MCS state machines

All MCS sequences which can take place, from the point of view of the user, are described by two state machines.

One machine for associated mode transmission services and the other for non-associated mode transmission services.

12.3.1.4.2 Associated mode primitive sequence

The service sequences, shown in Figure 152, depend on the state of the association with which they interact (see 12.1.6).

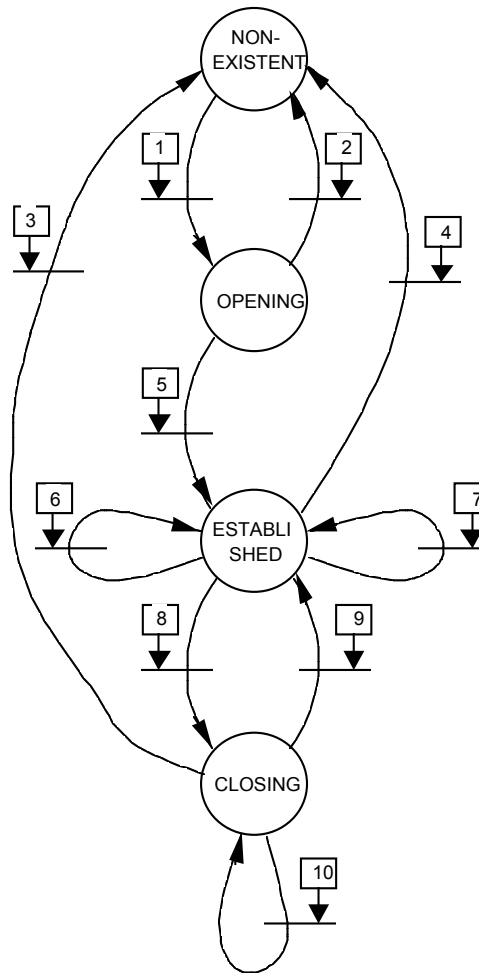


Figure 152 – Associated mode service state chart

List of events:

1. A_ASSOCIATE.Rq(); A_ASSOCIATE.Ind()
2. A_ASSOCIATE.Cnf(-); A_ASSOCIATE.Rp(-)
A_ABORT.Ind(Provider, User); A_ABORT.Rq()
3. A_RELEASE.Cnf(+); A_ABORT.Ind (Provider, User); A_ABORT.Rq()
4. A_ABORT.Rq(); A_ABORT.Ind(Provider, User)
5. A_ASSOCIATE.Cnf(+); A_ASSOCIATE.Rp(+)
6. A_DATA.Rq
7. A_DATA.Ind(); A_DATA.Cnf(+/-)
8. A_RELEASE.Rq(); A_RELEASE.Ind()
9. A_RELEASE.Cnf(-); A_RELEASE.Rp(-)
10. A_DATA.Ind(); A_DATA.Cnf(+/-)

12.3.1.4.3 Non-associated mode primitive sequence

These primitive sequences are not constrained and can be carried out within the limits of the resources available, as shown in Figure 153.

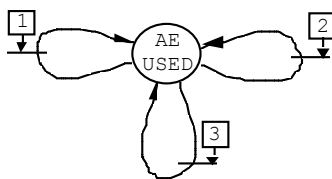


Figure 153 – Non-associated mode service state chart

List of events:

1. A_UNIDATA.Rq()
2. A_UNIDATA.Cnf(+/-)
3. A_UNIDATA.Ind()

13 Type 8 communication model specification

13.1 Concepts

13.1.1 Overview

The FAL is based on the concept described in Clause 4. Deviations and restrictions are described in the following subclauses.

The FAL is related to the Data Link Layer, defined in IEC 61158-3 and IEC 61158-4, as shown in Figure 154.

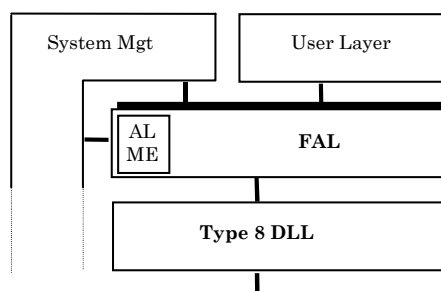


Figure 154 – Architectural positioning of the fieldbus Application Layer

NOTE IEC 61158-6 specifies a mapping of the application layer to the DLL.

13.1.2 Mapping of functional capabilities

Two communication mechanisms are supported:

- cyclic transmission of data in a high efficient manner using a publisher subscriber model
- non cyclic transmission of data using a client server communication model

AREPs, which act as push publisher or push subscriber, are used for cyclic transmission.

AREPs, which act as client and server (peer), are used for non cyclic data transmission. A specific transfer syntax is used for their APDUs. The non cyclic data transmission mechanism is described by a subset of those ASEs and services specified in 6.2.

13.1.3 Fundamental concept restrictions

13.1.3.1 Communication services

Communication Services as specified in 4.3.3.2 shall be applied with the following restrictions:

- Confirmed services are used to define request/response exchanges between APs, but the time between a request and a response is not limited.
- The Unconfirmed Send service is provided at a dedicated AR. It is used to take advantage of the native data transfer provided by the DLL.

13.1.3.2 AP interactions

AP Interactions as specified in 4.3.3.3 shall be applied with the following restrictions:

- Each publisher is related to a single subscriber.
- Only the "push" model applies.
- No timeliness is supported for the published data.

13.1.3.3 FAL presentation context

The FAL uses its specific transfer syntax rules specified in IEC 61158-6.

13.1.3.4 AR cardinality

AR Cardinality as specified in 4.3.7.4 shall be applied with the following restrictions:

- ARs that convey services between two APs have always a cardinality of 1-to-1.

13.1.3.5 AREP roles

AREP Roles as specified in 4.3.7.7 shall be applied with the following restrictions:

- The AREP Roles pull publishing manager, pull publisher and pull subscriber shall not be applied.

13.1.3.6 AREP timeliness

Timeliness (see 4.3.7.10) is not supported.

13.1.3.7 Identification of ARs

FAL AEs are accessed through data link addresses. ARs are identified by the data link addresses of the AREPs that participate in the AR.

AREPs supported by FAL only uses connection-oriented data link services. Therefore, a single DLCEP address is used to locate the receiving AREP. Group DL(SAP) addresses are not used.

Each device has a default DLSAP to locally access the DLE. This DLSAP is locally used by the ARPM to establish a DL connection. All DLCEPs exist within this DLSAP. As this DLSAP is only accessed locally, no DLSAP address is assigned to it.

13.1.3.8 Definition and creation of AREPs

Definition of AREPs as specified in 4.3.7.11 shall be applied with the following restriction:

- AREPs are always predefined.

13.1.3.9 Use of Data Link Layer addresses

DLCEP-addresses are used by the FAL to locate AR endpoints.

DL addresses are not used to identify objects accessed or referenced through the AR endpoint, and they are only used to locate the endpoint, not identify it.

13.1.3.10 Binding of AR endpoints to Data Link Layer service access points

Data Link Layer addresses locate the Data Link Layer connection endpoints (DLCEP) to which an AR endpoint may be bound. Therefore, an AR endpoint is located by its binding to a single DLCEP that may be identified by one or more DLL-addresses. Only one AR Endpoint may be bound to a specific DLCEP at a given time.

13.1.3.11 Data type ASE

Data type ASE as specified in Clause 5 shall be applied with the following restriction:

Only nesting level 1 and no embedded data types shall be used.

Only the following data types are supported:

- Boolean
- Integer8
- Integer16
- Integer32
- Unsigned8
- Unsigned16
- Unsigned32
- Float32
- BinaryDate
- BinaryDate2000
- TimeOfDay
- TimeDifference
- VisibleString
- OctetString
- BitString

13.1.4 Common FAL attributes

The common FAL attributes

- Key Attribute Numeric Identifier
- Key Attribute Name

are used as specified in 4.7.

13.2 ASEs

13.2.1 Overview

FAL uses the following subset of ASEs as specified in 6.2.

- Object Management ASE
- Application Process ASE
- Application Relationship ASE

Variable ASE

Function Invocation ASE

NOTE The following subclauses describe the subset of those ASEs used by the FAL. For each ASE the related classes (if classes are defined for that ASE) and the selected services are listed. If not all attributes of a class are supported, a class description is also included. If one or more of the conditional or user optional parameters of a service are not supported, a table for this service is included. Only the supported attributes or parameters are listed.

13.2.2 Object management ASE

13.2.2.1 Overview

A subset of the object management ASE specified in 6.2.1 is used, detailed description is given in the following subclauses.

13.2.2.2 FAL management model services

13.2.2.2.1 Get attributes service

The use of the service parameters is defined in Table 547.

Table 547 – Get Attributes service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
List of Objects and their Attributes	M	M(=)		
Key Attribute	M	M(=)		
List of Requested Attributes	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
More			M	M(=)
List of Responses			M	M(=)
List of Attribute Values			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

13.2.3 Application process ASE

13.2.3.1 Overview

A subset of the application process ASE specified in 6.2.2 is used, detailed description is given in the following subclauses.

13.2.3.2 Application process ASE class specification

13.2.3.2.1 Application process formal model class definition

Application Process formal model as specified in 6.2.2.2.1. The used attributes and services are summarised below :

FAL ASE: AP ASE**CLASS: AP****CLASS ID: 16****PARENT CLASS: TOP****IDENTIFY, GET STATUS SERVICE ATTRIBUTES:**

1	(m)	Attribute:	List Of AP Service Attributes
1.1	(m)	Attribute:	Manufacturer Identifier
1.1.1	(m)	Attribute:	Vendor Name
1.1.2	(m)	Attribute:	Model Identifier
1.1.3	(m)	Attribute:	Vendor Revision
1.2	(m)	Attribute:	AP Descriptor Reference
1.3	(m)	Attribute:	Network Access State
1.3.1	(m)	Attribute:	Service Access Status
1.3.2	(m)	Attribute:	Operational Status

SYSTEM MANAGEMENT ATTRIBUTES:

2	(m)	Attribute:	List Of System Management Attributes
2.1	(m)	Attribute:	List Of AR Endpoint Info
2.1.1	(m)	Attribute:	Numeric Id
2.1.2	(m)	Attribute:	Configured Max FAL PDU Size Sending
2.1.3	(m)	Attribute:	Configured Max FAL PDU Size Receiving
2.1.4	(m)	Attribute:	Configured Max Outstanding Services Requesting
2.1.5	(m)	Attribute:	Configured Max Outstanding Services Responding
2.1.6	(m)	Attribute:	List of Supported Services
2.2	(m)	Attribute:	List Of In Use AR Endpoint Info
2.2.1	(m)	Attribute:	Context State
2.2.6	(m)	Attribute:	Outstanding Services Requesting Counter
2.2.7	(m)	Attribute:	Outstanding Services Responding Counter

OBJECT MANAGEMENT ATTRIBUTES:

3	(m)	Attribute:	List Of Managed AP Attributes
3.1	(m)	Attribute:	List Of Supported APO Classes and Objects
3.1.1	(m)	Attribute:	List Header
3.1.1.1	(o)	Attribute:	Numeric Identifier = 0
3.1.1.2	(m)	Attribute:	ROM / RAM Flag (TRUE, FALSE)
3.1.1.3	(m)	Attribute:	Max Name Length
3.1.1.4	(m)	Attribute:	Access Protection Supported (TRUE, FALSE)
3.1.1.5	(m)	Attribute:	Version Of List
3.1.1.6	(m)	Attribute:	Local Reference
3.1.2	(c)	Constraint	Data Type Supported
3.1.2.1	(o)	Attribute:	Numeric Identifier = 1
3.1.2.2	(m)	Attribute:	Number Of Objects (Data Types)
3.1.2.3	(m)	Attribute:	Local Reference
3.1.3	(c)	Constraint	Variable
3.1.3.1	(o)	Attribute:	Static Object Numeric Identifier
3.1.3.2	(m)	Attribute:	Number Of Static Objects (Variables)
3.1.3.3	(m)	Attribute:	Static Object Local Reference
3.1.5	(c)	Constraint	Function Invocation Supported
3.1.5.1	(o)	Attribute:	Function Invocation Numeric Identifier
3.1.5.2	(m)	Attribute:	Number Of Function Invocation Objects
3.1.5.3	(m)	Attribute:	Function Invocation Local Reference

SERVICES:

2	(o)	Ops Service:	Identify
3	(o)	Ops Service:	Get Status
5	(o)	Ops Service:	Initiate
6	(o)	Ops Service:	Terminate
8	(o)	Ops Service:	Reject

13.2.3.3 Application process ASE attributes

13.2.3.3.1 Identify, get status attributes

List Of AP Service Attributes as specified in 6.2.2.2.2 with the following restriction:

- Serial Number is not supported.

13.2.3.3.2 System management attributes

List Of System Management Attributes as specified in 6.2.2.2.3 with the following restriction:

- Only non-segmenting ARs are supported
- Data structure nesting is not supported

The attributes

- Configured Max Data Structure Nesting Level
- Negotiated Max FAL PDU Size Sending
- Negotiated Max FAL PDU Size Receiving
- Negotiated Max Outstanding Services Responding
- Negotiated Max Data Structure Nesting Level
- Negotiated List of Supported Services

are not supported.

13.2.3.3.3 Object management attributes

List Of Managed AP Attributes as specified in 6.2.2.2.4 with the following restriction:

The attributes

- List of Supported APO Classes and ObjectsList of DLSAP Addressess
- List of APO Classes Supported
- List of Event Summary Selection Criteria

are not supported.

13.2.3.3.4 Support attributes

List of Supported Attributes is not supported.

13.2.3.4 Application process ASE service specification

13.2.3.4.1 General

This subclause contains the definition of services that are unique to this ASE.

13.2.3.4.2 Identify service

Identify as specified in 6.2.2.3.3.

13.2.3.4.3 Get status service

Get status as specified in 6.2.2.3.4.

13.2.3.4.4 Initiate service

Initiate as specified in 6.2.2.3.6. The use of the service parameters is defined in Table 548.

Table 548 – Initiate service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Version Object Definitions Calling	M	M(=)		
AP Descriptor Calling	M	M(=)		
Access Protection Supported Calling	M	M(=)		
Password Calling	M	M(=)		
Access Groups Calling	M	M(=)		
Result(+)			S	S(=)
AREP			M	M(=)
Version OD Called			M	M(=)
AP Descriptor Called			M	M(=)
Access Protection Supported Called			M	M(=)
Password Called			M	M(=)
Access Groups Called			M	M(=)
Result(-)			S	S(=)
AREP			M	M(=)
Error Code			M	M(=)
Max PDU Length Sending Called				C
Max PDU Length Receiving Called				C
FAL Features Supported Called				C

13.2.3.4.5 Terminate service

Terminate as specified in 6.2.2.3.7.

13.2.3.4.6 Reject service

Reject as specified in 6.2.2.3.9.

13.2.4 Application relationship ASE**13.2.4.1 Overview****13.2.4.1.1 General**

A subset of the Application Relationship ASE specified in 6.2.3 is used, detailed description is given in the following subclauses.

13.2.4.1.2 Endpoint context

13.2.4.1.2.1 Endpoint context overview

Endpoint Context as in 6.2.3.1.2.

13.2.4.1.2.2 Endpoint role

The role of an AREP determines the permissible behavior of an AP at the AREP. An AREP role may be that of a client, server, peer (client and/or server), push publisher, push subscriber.

NOTE See concepts subclause for description of clients and servers, and the push and pull models for publishers and subscribers.

Table 549 and Table 550 summarize the characteristics and combinations of each of the AREP roles.

Table 549 – Conveyance of service primitives by AREP role

	Client	Server	Peer	Push Publisher	Push Subscriber
Send Service Req	X		X	X	
Recv Service Req		X	X		X
Send Service Rsp		X	X		
Recv Service Rsp	X		X		

Table 550 – Valid combinations of AREP roles involved in an AR

AREP Roles by Interaction Model	Client	Server	Peer	Push Publisher	Push Subscriber
Client/server					
Client		Yes	Yes		
Server			Yes		
Peer			Yes		
Publisher/subscriber					
Push Publisher					Yes
Push Subscriber					

13.2.4.1.2.3 Dedicated AR endpoints

Dedicated AR Endpoints as in 6.2.3.1.2.2.

13.2.4.1.2.4 Cardinality

The cardinality of an AR specifies, from the point of view of a client or publisher endpoint, how many remote application processes are involved in an AR. Cardinality is never expressed from the viewpoint of a server or a subscriber.

When expressed from the viewpoint of a client or peer endpoint, ARs are always 1-to-1. Clients are never capable of issuing a request and waiting for responses from multiple servers.

13.2.4.1.2.5 Conveyance model

Conveyance model as in 6.2.3.1.2.4.

13.2.4.1.3 Underlying communications services

13.2.4.1.3.1 Overview

The AR ASE conveys FAL APDUs using the capabilities of the underlying Data Link Layer. Several characteristics are used to describe these capabilities. 13.2.4.1.3 provides a description of each. These characteristics are specific to the Data Link Mapping defined in IEC 61158-6. Their precise specification can be found there.

13.2.4.1.3.2 Connection-oriented services

The underlying layer may support AR endpoints by providing connection-oriented services. Endpoints are pre-configured to be open.

13.2.4.1.3.3 Buffered and queued services

The underlying layer may support AR endpoints by providing buffered or queued services. These services can be used to implement buffers or queues required by certain classes of endpoints.

13.2.4.1.3.4 Cyclic and non cyclic transfers

The underlying layer may support AR endpoints by providing cyclic or non cyclic services.

The underlying layer may also provide for the non cyclic transfer of APDUs concurrently with cyclic transfer of APDUs between the same endpoints.

13.2.4.1.4 AR establishment

For an AR endpoint to be used by an application process, the corresponding AR has to be active. When an AR is activated, it is referred to as "established".

AR establishment can occur in one of two ways.

First, ARs can be pre-established. Pre-established means that the AE that maintains the endpoint context is created when the AP is connected to the network. In this case, communications among the applications involved in the AR may take place without first having to explicitly establish the AR. Any AR may be defined to be pre-established

Second, ARs can be pre-defined, but not pre-established. Pre-defined means that the characteristics of the AR are known to each endpoint, but that their contexts have not been synchronized for operation. In this case, the use of the FAL Establish service is required to synchronize them and open them for data transfer.

13.2.4.1.5 Application relationship classes

AREPs are defined with a combination of characteristics to form different classes of ARs.

13.2.4.2 Application relationship ASE class specifications

13.2.4.2.1 AR endpoint formal model class definition

AR Endpoint formal model as specified in 6.2.3.2.1. The used attributes and services are summarised below :

FAL ASE:		AR ASE
CLASS:	AR ENDPOINT	
CLASS ID:	32	
PARENT CLASS:	TOP	
SYSTEM MANAGEMENT ATTRIBUTES:		
1	(m)	Attribute: Local AP
2	(m)	Attribute: FAL Revision
3	(m)	Attribute: Dedicated (TRUE, FALSE)
4	(o)	Attribute: Transfer Syntax
SERVICES:		
1	(o)	OpsService: AR-Unconfirmed Send
3	(o)	OpsService: AR-Establish
5	(o)	OpsService: AR-Abort
14	(o)	OpsService: AR-Data-Send-Acknowledge

13.2.4.3 Application relationship ASE attributes

13.2.4.3.1 System management attributes

List Of System Management Attributes as specified in 6.2.3.2.2 with the following restriction:

List Of Supported Attributes is not supported

Transfer Syntax

This optional attribute identifies the encoding rules to be used on the AR. It is mandatory and shall be set to Transfer Syntax description (see IEC 61158-6).

13.2.4.4 Application relationship ASE service specification

13.2.4.4.1 AR-unconfirmed send service

AR-Unconfirmed Send as specified in 6.2.3.3.2. The use of the service parameters is defined in Table 551.

Table 551 – AR-Unconfirmed Send service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
FAL Service Type	M	M(=)
FAL APDU Body	M	M(=)

13.2.4.4.2 AR-establish service

AR-Establish as specified in 6.2.3.3.4. The use of the service parameters is defined in Table 552.

Table 552 – AR-Establish service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
User Data	U	U(=)		
Result			M	M(=)
AREP			M	M(=)
User Data			U	U(=)

13.2.4.4.3 AR-abort service

AR-Abort as specified in 6.2.3.3.6.

13.2.4.4.4 AR-data-send-acknowledge service

This service is defined for the FAL to provide the FAL user a service for transparent data transfer. This service can be used to exchange arbitrary data over a DLL.

For this service no specific ARPM state machine is available. The data flow is controlled by the FAL user.

Table 553 – AR-Data-Send-Acknowledge service parameters

Parameter name	Req	Ind	Cnf
Argument	M	M(=)	
Destination Address	M	M(=)	
Source Address	M	M(=)	
Destination Node	M	M(=)	
Source Node	M	M(=)	
Destination Subnode	M	M(=)	
Source Subnode	M	M(=)	
Protocol Code	M	M(=)	
Block Number	M	M(=)	
Data	M	M(=)	
Result(+)			M
Destination Address			M
Source Node			M
Source Subnode			M
Result(-)			M
Destination Address			M
Source Node			M
Source Subnode			M
Error Info			M

Argument

The argument contains the parameters of the service request.

Destination Address

This parameter contains the logical destination address of the requested device. The broadcast address is FF FF FF.

Source Address

This parameter contains the logical source address of the local device.

Destination Node

This parameter contains the destination node of the requested device.

Source Node

This parameter contains the destination node of the local device.

Destination Subnode

This parameter contains the destination subnode of the requested device.

Source Subnode

This parameter contains the destination subnode of the local device.

Protocol Code

This parameter specifies the FAL user protocol.

Block Number

This parameter is the number of the PDU segment.

Data

This parameter contains the PDU that has to be exchanged.

13.2.5 Variable ASE

13.2.5.1 Overview

A subset of the Variable ASE specified in 6.2.4 is used, detailed description is given in the following subclauses.

13.2.5.2 Variable ASE class specification

13.2.5.2.1 Simple variable formal model class definition

Simple Variable Formal Model as specified in 6.2.4.3.1.1. The used attributes and services are summarised below.

FAL ASE:		VARIABLE ASE	
CLASS:	SIMPLE VARIABLE		
CLASS ID:		7	
PARENT CLASS:		TOP	
ATTRIBUTES:			
2	(m) Attribute:	Data Type	
3	(m) Attribute:	Length	
5	(m) Attribute:	Access Privilege	
5.1	(m) Attribute:	Password	
5.2	(m) Attribute:	Access Groups	
5.3	(m) Attribute:	Access Rights	
6	(m) Attribute:	Local Detail	

SERVICES:

1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
5	(o)	OpsService:	Information Report

13.2.5.2.2 Array variable formal model class definition

Array Variable formal model as specified in 6.2.4.3.2.1. The used attributes and services are summarised below.

FAL ASE:			VARIABLE ASE
CLASS:			ARRAY VARIABLE
CLASS ID:			8
PARENT CLASS:			TOP
ATTRIBUTES:			
2	(m)	Attribute:	Data Type
2.1	(m)	Attribute:	Data Type ID
3	(c)	Constraint:	Data Type Format = FIXED-LENGTH STRING
3.1	(m)	Attribute:	Element Length
5	(m)	Attribute:	Number of Elements
6	(m)	Attribute:	Access Privilege
6.1	(m)	Attribute:	Password
6.2	(m)	Attribute:	Access Groups
6.3	(m)	Attribute:	Access Rights
7	(m)	Attribute:	Local Detail
SERVICES:			
1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
5	(o)	OpsService:	Information Report

13.2.5.2.3 Record variable formal model class definition

Record Variable formal model as specified in 6.2.4.3.3.1. The used attributes and services are summarised below.

FAL ASE:			VARIABLE ASE
CLASS:	RECORD VARIABLE		
CLASS ID:			9
PARENT CLASS:			TOP
ATTRIBUTES:			
2	(m)	Attribute:	Data Type
2.1	(m)	Attribute:	Data Type ID
3	(m)	Attribute:	Access Privilege
3.1	(m)	Attribute:	Password
3.2	(m)	Attribute:	Access Groups
3.3	(m)	Attribute:	Access Rights
4	(m)	Attribute:	List of Local Detail
SERVICES:			
1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
5	(o)	OpsService:	Information Report

13.2.5.3 Variable ASE service specification

13.2.5.3.1 Read service

Read as specified in 6.2.4.4.2. The use of the service parameters is defined in Table 554.

Table 554 – Read service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variable Specifier	M	M(=)		
Numeric Identifier	S	S		
Name	S	S		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Value			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

13.2.5.3.2 Write service

Write as specified in 6.2.4.4.4. The use of the service parameters is defined in Table 555.

Table 555 – Write service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)	M(=)	M(=)
AREP	M	M(=)		
Invoke ID	M	M(=)		
Variable Specifier	M	M(=)		
Numeric Identifier	S	S(=)		
Name	S	S(=)		
Value	M	M(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)

13.2.5.3.3 Information report service

Information report as specified in 6.2.4.4.6. The use of the service parameters is defined in Table 556.

Table 556 – Information Report service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Variable Specifier	M	M(=)
Numeric Identifier	S	S(=)
Name	S	S(=)
Value	M	M(=)

13.2.6 Function invocation ASE

13.2.6.1 Overview

A subset of the Function Invocation ASE specified in 6.2.7, detailed description is given in the following subclauses.

13.2.6.2 Function invocation ASE class specification

13.2.6.2.1 Function invocation formal model class definition

Function Invocation Formal Model as specified in 6.2.7.2.1.2. The used attributes and services are summarized below.

FAL ASE:		FUNCTION INVOCATION ASE	
CLASS:		FUNCTION INVOCATION	
CLASS ID:		3	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1	(m)	Attribute:	Access Privilege
1.1	(m)	Attribute:	Password
1.2	(m)	Attribute:	Access Groups
1.3	(m)	Attribute:	Access Rights
2	(m)	Attribute:	Deletable (TRUE,FALSE)
3	(m)	Attribute:	Reusable (TRUE,FALSE) Default : TRUE
4	(m)	Attribute:	Function Invocation State
5	(m)	Attribute:	Number of Related Objects In Use
SERVICES:			
1	(o)	OpsService:	Start
2	(o)	OpsService:	Stop
3	(o)	OpsService:	Resume
4	(o)	OpsService:	Reset

13.2.6.3 Function invocation ASE service specification

13.2.6.3.1 Start service

Start as specified in 6.2.7.3.2. The use of the service parameters is defined in Table 557.

Table 557 – Start service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Numeric Identifier	S	S(=)		
Name	S	S(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

13.2.6.3.2 Stop service

Stop as specified in 6.2.7.3.3. The use of the service parameters is defined in Table 558.

Table 558 – Stop service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Numeric Identifier	S	S(=)		
Name	S	S(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

13.2.6.3.3 Resume service

Resume as specified in 6.2.7.3.4. The use of the service parameters is defined in Table 559.

Table 559 – Resume service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Numeric Identifier	S	S(=)		
Name	S	S(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

13.2.6.3.4 Reset service

Reset as specified in 6.2.7.3.5. The use of the service parameters is defined in Table 560.

Table 560 – Reset service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
Key Attribute	M	M(=)		
Numeric Identifier	S	S(=)		
Name	S	S(=)		
Result (+)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
AREP			M	M(=)
Invoke ID			M	M(=)
Error Info			M	M(=)
Function Invocation State			C	C(=)

13.3 Application relationships

13.3.1 Queued user-triggered bi-directional with flow control (QUB-FC) AR endpoint formal model

Queued User-triggered Bi-directional with Flow Control as specified in 6.3.4 with the following restriction:

- The “Remote Address Configuration Type” attribute shall only have the value “LINKED”.
- The attribute “Dedicated” of the parent class shall always be set to FALSE.

13.3.2 Buffered network-scheduled uni-directional (BNU) AR endpoint formal model

Buffered Network-Scheduled Uni-directional as specified in 6.3.7 with the following restriction:

- If present the “ Duplicate Pdu Detection Supported” attribute is always set to FALSE.
- The attribute “Dedicated” of the parent class shall always be set to TRUE.

13.3.3 Queued user-triggered bi-directional transparent mode (QUB-TM) AR endpoint formal model

13.3.3.1 Description

This class is defined to support asynchronous operation of the publisher/subscriber model using queues. It provides a conveyance path for arbitrary data that are exchanged over a communication system.

The behavior of this class is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR-Data-Send-Acknowledge Service to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity. The latter will be ignored if the outstanding service counter has reached the negotiated maximum.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user, in the order that it was received.

The following summarizes the characteristics of this AREP class.

Roles	PUSH PUBLISHER PUSH SUBSCRIBER
Cardinality	1-to-1
Timeliness	No

13.3.3.2 Formal class definition

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional Transparent Mode AREP
CLASS ID:	50
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1	(m) Attribute: Role (PUSH Publisher, PUSH Subscriber)
2	(m) Attribute: Maximum Outstanding Requests Calling
3	(m) Attribute: Maximum Outstanding Requests Called
4	(m) Attribute: Transmit DL Mapping Reference
5	(m) Attribute: Receive DL Mapping Reference
SERVICES:	
1	(m) OpsService: AR-Data-Send-Acknowledge

13.3.3.3 Network management attributes

13.3.3.3.1 Role

This attribute specifies the role of the AREP. The valid values are:

PUSH PUBLISHER Endpoints of this type publish their data issuing AR-Data-Acknowledge service Request-APDUs.

PUSH SUBSCRIBER Endpoints of this type receive data published in AR-Data-Acknowledge service Request-APDUs.

13.3.3.3.2 Max outstanding requests calling

This attribute indicates the maximum number of requests that the AREP may be expecting from the remote AREP.

13.3.3.3.3 Max outstanding requests called

This attribute indicates the maximum number of requests that the AR Endpoint may be expecting from its local user.

13.3.3.3.4 Transmit DL mapping reference

This attribute provides a reference to the underlying Data Link Layer mapping for the transmit conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

13.3.3.3.5 Receive DL mapping reference

This attribute provides a reference to the underlying Data Link Layer mapping for the receive conveyance path for this AREP. DL mappings for the Data Link Layer (IEC 61158-3) are specified in IEC 61158-6.

13.3.3.4 Services

13.3.3.4.1 AR-data-send-acknowledge

This service is used to exchange the PDU on an AR.

13.4 Permitted FAL services by AREP role

Table 561 defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 561 – Services by AREP role

	Unc	Cnf	Client	Server	Push Publ	Push Subsc
FAL Services			req rcv	req rcv	req rcv	req rcv
Mgt ASE						
Get Attributes		X	X	X		
AP ASE						
Identify		X	X	X		
Get Status		X	X	X		
Initiate		X	X	X		
Terminate		X	X	X		
Reject		X	X			
AR ASE						
AR-Unconfirmed Send					X	X
AR-Establish					X	X
AR-Abort					X	X X
AR-Data-Send-Acknowledged					X	X
Variable ASE						
Read		X	X	X		
Write		X	X	X		
Information Report	X		X	X		
Function Invocation ASE						
Start		X	X	X		
Stop		X	X	X		
Resume		X	X	X		
Reset		X	X	X		

14 Type 9 communication model specification

14.1 Concepts

There are no additional concepts for the Communications Model.

14.2 Common parameters

Many services have the following parameters. Instead of defining them with each service, the following common definitions are provided.

AREP ID

This parameter contains sufficient information to locally identify the AR used to convey the service.

VCR

This parameter contains sufficient information to locally identify the VCR and AR used to convey the service.

Invoke ID

This attribute identifies the Transaction Object within the VCR.

Error Type

This parameter contains information about the reason for the unsuccessful execution of the service.

14.3 ASEs

14.3.1 Virtual field device ASE

14.3.1.1 Virtual field device class specification

14.3.1.1.1 Class overview

The Virtual Field Device (VFD) is an abstract model for the description of the data and the behavior of an Application Process. VFDs contain APOs. The attributes of an APO are described by object descriptions. The collection of a VFD's object descriptions is defined as the Object Dictionary (OD). For each VFD there is exactly one Object Dictionary. Services are defined for accessing a VFD's OD and its APOs, as shown in Figure 155.

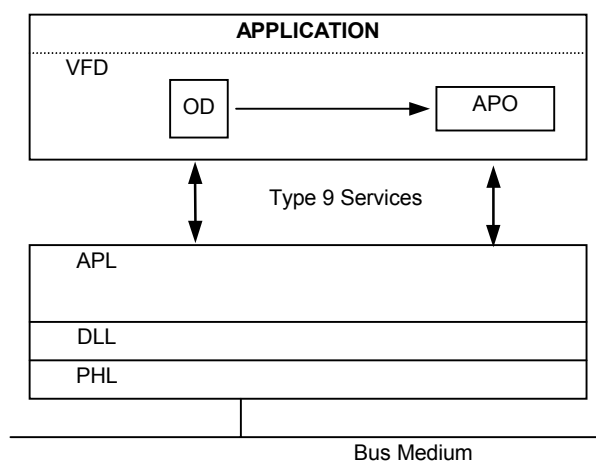


Figure 155 – VFD model

The services define no concrete interface for an implementation. They describe in an abstract form which functions may be used.

The application is not a subject of this standard. It shall only be indicated, how the abstractly described services may be made available to the application.

Access to VFD objects is defined by Virtual Communication Relationships (VCR). Several VFD Objects may be present in a device, as shown in Figure 156.

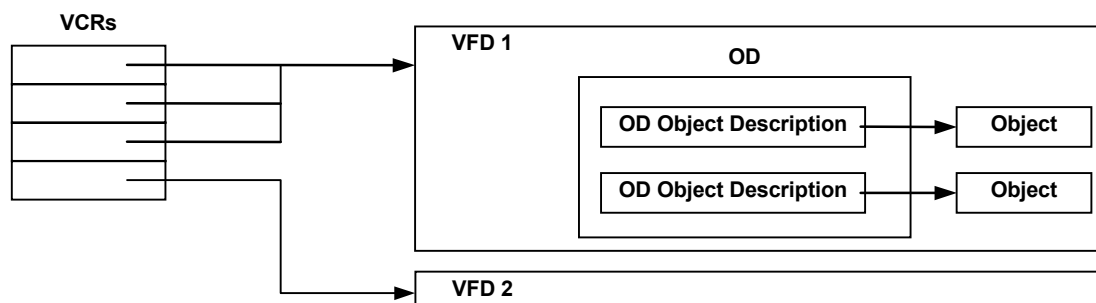


Figure 156 – Abstract model of an automation system (VFD)

14.3.1.1.2 Formal model

The VFD class specifies the attributes and services defined for application processes. Its parent class "top" indicates the top of the class tree.

ASE:	VFD ASE
CLASS:	VFD
CLASS ID:	—
PARENT CLASS:	TOP
1. (m) Key Attribute:	implicit through the VCR
2. (m) Attribute:	VendorName
3. (m) Attribute:	ModelName
4. (m) Attribute:	Revision
5. (m) Attribute:	ProfileNumber
6. (m) Attribute:	LogicalStatus
7. (m) Attribute:	PhysicalStatus
8. (m) Attribute:	List of VFD Specific Objects
SERVICES:	
1. (m) Ops Service:	Status
2. (m) Ops Service:	Unsolicited Status
3. (m) Ops Service:	Identify

14.3.1.1.3 Attributes

Implicit through the VCR

The VCR which is entered in the VCR List points to the assigned VFD Object.

VendorName

This attribute identifies the vendor of the device.

ModelName

This attribute contains the model name of the device. The value of this attribute is defined by the vendor.

Revision

This attribute describes the revision level of the device. The value of this attribute is defined by the vendor.

ProfileNumber

This attribute identifies the profile of the VFD. The profile value is defined by an external administrative authority. If the device does not correspond to a profile, then this attribute has a null value.

Logical Status

This attribute contains information about the state of the communication capabilities of the device. Its values are shown in Table 562.

Table 562 – Logical status

Logical status	Meaning												
Ready for communication	All services may be used normally.												
Limited number of services	Limited number of services may be supported for some cases.												
OD-LOADING-NON-INTERACTING	If the Object Dictionary is in the state OD-LOADING-NON-INTERACTING, it is not allowed to execute the service InitiatePutOD.												
OD-LOADING-INTERACTING	If the Object Dictionary is in the state OD-LOADING-INTERACTING, then all connections, except for the connection over which the InitiatePutOD Service was received, are locked and the establishment of a further connection shall be rejected. On this connection only the following services are allowed: <table> <tr> <td>Initiate</td><td>PhysWrite</td></tr> <tr> <td>Abort</td><td>GetOD</td></tr> <tr> <td>Reject</td><td>InitiatePutOD</td></tr> <tr> <td>Status</td><td>PutOD</td></tr> <tr> <td>Identify</td><td>TerminatePutOD</td></tr> <tr> <td>PhysRead</td><td></td></tr> </table>	Initiate	PhysWrite	Abort	GetOD	Reject	InitiatePutOD	Status	PutOD	Identify	TerminatePutOD	PhysRead	
Initiate	PhysWrite												
Abort	GetOD												
Reject	InitiatePutOD												
Status	PutOD												
Identify	TerminatePutOD												
PhysRead													

Physical Status

This attribute gives a coarse summary of the state of the real device. Its values are:

- operational
- partially operational
- not operational
- needs maintenance

List of VFD Specific Objects

Contains all VFD Specific Objects.

14.3.1.1.4 Services**Status**

This service is used to request the network visible state from the AP.

Unsolicited Status

This service is used by the AP to report its network visible state.

Identify

This service is used to request manufacturer information from the AP.

14.3.1.2 Virtual field device ASE service specification**14.3.1.2.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Status
- Unsolicited Status
- Identify

14.3.1.2.2 Status service**14.3.1.2.2.1 Service overview**

The device/user state is read with the Status service.

14.3.1.2.2.2 Service primitives

The service parameters for this service are shown in Table 563.

Table 563 – Status

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Result(+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Logical Status			M	M(=)
Physical Status			M	M(=)
Local Detail			U	U(=)
Result(-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Logical Status

This parameter contains the value of the Logical Status attribute of the VFD.

Physical Status

This parameter contains the value of the Physical Status attribute of the VFD.

Local Detail

This parameter contains the user defined state of the application.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.1.2.2.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.1.2.3 Unsolicited status service

14.3.1.2.3.1 Service overview

The Status Notification service is used by the AP to report its state spontaneously.

14.3.1.2.3.2 Service primitives

The service parameters for this service are shown in Table 564.

Table 564 – Unsolicited status

Parameter name	Req	Ind
Argument	M	M(=)
VCR	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
LogicalStatus	M	M(=)
Physical Status	M	M(=)
Local Detail	U	U(=)

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and the Remote Address Configuration Type is FREE. It gives the remote address to which the Status Notification should be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and the Remote Address Configuration Type is FREE. This parameter indicates the remote address from which the indicated Status Notification is to be sent.

Logical Status

This parameter contains the value of the Logical Status attribute of the VFD.

Physical Status

This parameter contains the value of the Physical Status attribute of the VFD.

Local Detail

This parameter contains the user defined state of the application.

14.3.1.2.3.3 Service procedure

The Unconfirmed Service Procedure specified in Clause 4 applies to this service.

14.3.1.2.4 Identify service

14.3.1.2.4.1 Service overview

Information to identify an AP is read with this service.

NOTE The attribute ProfileNumber of the AP is transmitted with the Initiate service of the Context Management ASE.

14.3.1.2.4.2 Service primitives

The service parameters for this service are shown in Table 565.

Table 565 – Identify

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Result(+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Vendor Name			M	M(=)
Model Identifier			M	M(=)
Vendor Revision			M	M(=)
Result(-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Vendor Name

This parameter contains the value of the Vendor Name attribute of the VFD.

Model Identifier

This parameter contains the value of the Model Identifier attribute of the VFD.

Vendor Revision

This parameter contains the value of the Vendor Revision attribute of the VFD.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.1.2.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.2 Object dictionary (OD) ASE

14.3.2.1 Overview

The ASE Models each specify one or more related APO classes as a collection of attributes and services.

The collection of attributes for the APOs of an AP is referred to as the Object Dictionary (OD). The attributes for a single APO is referred to as its object description. To access the object descriptions of APOs, this ASE defines common OD services. These services operate using the client/server model.

The Object Dictionary (OD) contains the Object Descriptions of the following Communication Objects:

- DataType
- Domain
- SimpleVariable
- Record
- Event
- DataTypeStructureDescription
- ProgramInvocation
- Array
- VariableList

An object description "Object Dictionary" (OD object description) is uniquely assigned to the Object Dictionary. The OD object description contains information about the structure of the Object Dictionary.

The object descriptions are marked with an unique index of type Unsigned16.

A name of type visible string may also be assigned to the following objects:

- Domain
- SimpleVariable
- Record
- Event
- ProgramInvocation
- Array
- VariableList

The index or the name serves as a key to the object and the object description.

The services address on principle the object of the server, because the real object exists there.

Before a remote communication partner can access an object, he shall know its description. The logical address (Index) or the name may be used, when objects are accessed remotely.

The object descriptions may be specified during system configuration, but they may also be transferred between stations at any later time.

The definition of the object description takes place at that station, in which the object really exists (Source OD). The length of an object description may be different for the individual objects. The communication partners maintain a complete or a partial copy of the Remote Object Description (Remote OD). This is illustrated in Figure 157.

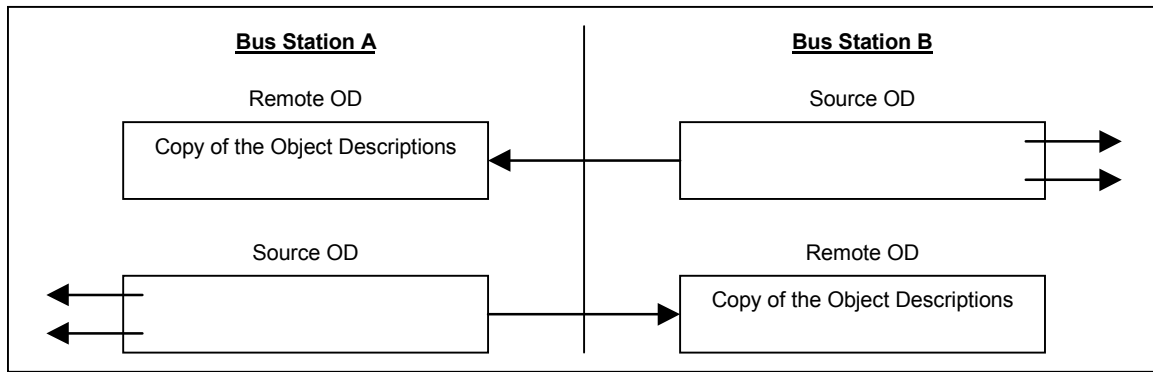


Figure 157 – Source OD/remote OD

Thus every station may possess a Source OD for locally existing communication objects and one or more Remote ODs. The Remote ODs are connection specific, i.e. for each connection another Remote OD may be valid.

The OD model provides corresponding services for reading and writing of Source ODs. If a Source OD is modified by a remote communication partner, then this shall be notified to all other communication partners which use the same data base (Source OD). For this purpose, the user releases the connections to all other communication partners. After the OD has been modified, the connections may be established again. During the connection establishment, the new version number (Version OD) is given to each communication partner to notify that the corresponding Source OD has been modified. The addition of object descriptions is permissible without connection release.

The structure of the OD described as follows does not prescribe a concrete implementation of the OD. It only defines the structure of the object descriptions which are transmitted via the bus. Even the existence of an OD is not necessary, if the needed information can be obtained in an other way (e.g. by an algorithm). Detailed attributes and transfer syntaxes are specified in the appropriate models.

- The Object Dictionary (OD) is divided into several parts, as shown in Table 566:
- The Object description of the "Object Dictionary" (OD object description) contains structural information about the OD.
- Static Type Dictionary (ST-OD) contains Data Types and Data Type Structure Description.
- Static Object Dictionary (S-OD) contains object descriptions of Simple Variables, Arrays, Records, Domains and Events.
- Dynamic List of Variable Lists (DV-OD) contains object descriptions of variable lists.
- Dynamic List of Program Invocations (DP-OD) contains object descriptions of Program Invocations.

Table 566 – Structure of the object dictionary

Index	Object Dictionary (OD)
0	OD object description (OD-ODES)
1 ... i	Static List of Types (ST-OD)
k ... n	Static Object Dictionary (S-OD)
p ... t	Dynamic List of Variable Lists (DV-OD)
v ... z	Dynamic List of Program Invocations (DP-OD)

14.3.2.1.1 Static list of types (ST-OD)

The Static List of Types (ST-OD) contains the object descriptions of Data Types and Data Type Structure Descriptions for the Variable Access objects of the S-OD, as shown in Table 567.

Table 567 – Structure of the static list of types

Index	Static List of Types (ST-OD)
1 ... c	contains objects: Data Types
d ... i	contains objects: Data Types and Data Type Structure Descriptions

Exactly one object description is assigned to every index. This assignment is static and may not be deleted. The ST-OD may be loaded by the remote partner.

The Static List of Types always begins with the index 1 in the Object Dictionary. The remaining object descriptions shall follow with increasing index. The number of assignments "index <=> object description" is entered as length of the ST-OD in the Object Dictionary.

The ST-OD is divided in two segments. The first segment contains the object descriptions of standardized Data Types (Standard Data Types) and begins with index 1. The second segment follows immediately and contains object descriptions of Data Types and Data Type Structure Descriptions, which can be defined individually. This segment may be further specified by user groups with the help of profiles.

14.3.2.1.2 Static object dictionary (S-OD)

The Static Object Dictionary (S-OD) contains the object descriptions of Simple Variables, Arrays, Records, Domains and Events, as shown in Table 568.

Table 568 – Structure of the static object dictionary

Index	Static Object Dictionary (S-OD)
k ... n	contains objects: Simple Variables, Arrays, Records, Domains and Events

An object description is assigned to each index. The first index, which is associated to an object description, is entered in the object description of the Object Dictionary. The remaining object descriptions shall follow with increasing index. The number of assignments "index <=> object description" is entered as length of the S-OD in the object description of Object Dictionary.

Besides the assignment "index <=> object description", an additional assignment "name <=> object description" may exist. The length of the name may be from 0 to 32 octets. It is entered in the Name Length Field of the OD object description. If this field contains a "0", no names are present. The objects which are defined in the S-OD may be provided with access rights. Whether Access Rights are possible is entered in the field AccessProtectionSupported of the OD object description.

An object description in the S-OD contains an extension, which may contain further object specific definitions.

14.3.2.1.3 Dynamic list of variable lists (DV-OD)

The Dynamic List of Variable Lists (DV-OD) contains the object description of the Variable Lists, as shown in Table 569.

Table 569 – Structure of the dynamic list of variable lists

Index	Dynamic List of Variable Lists (DV-OD)
p ... t	contains objects: Variable List

An object "Variable List" and its object description in the DV-OD is dynamically created with the service DefineVariableList and may be deleted with the service DeleteVariableList. Upon creation of the object access rights may be assigned to it. Moreover it is possible, to create these objects by loading the whole OD.

The first index, which has the object description of a Variable List assigned to it, and the number of available indices are entered in the OD object description.

Fundamental information is entered in the object description about the Variable List, for example access rights, the number of Variable Access objects and the logical addresses (indices) of Variable Access objects in the S-OD.

Besides the assignment "index <=> object description", an additional assignment "name <=> object description" may exist.

14.3.2.1.4 Dynamic list of program invocations (DP-OD)

The Dynamic List of Program Invocations (DP-OD) contains the object descriptions of the objects Program Invocation, as shown in Table 570.

Table 570 – Structure of the dynamic list of program invocations

Index	Dynamic List of Program Invocations (PD-OD)
V ... W	Predefined Program Invocations
X ... Z	Dynamic Program Invocations

An object Program Invocation and its object description in the DP-OD is dynamically created with the service Create Program Invocation and deleted with the service Delete Program Invocation. Upon creation of the object access rights may be assigned to it. Moreover it is possible to create these objects by loading the whole OD.

The first index, which has the object description of a Program Invocation assigned to it, and the number of available indices are entered in the OD object description.

The object description of a Program Invocation contains information such as access rights and the number of Domain objects and their logical addresses (indices).

Besides the assignment "index <=> object description", an additional assignment "name <=> object description" may exist.

The DP-OD may contain a segment with pre-configured Program Invocations.

14.3.2.2 OD class specifications

14.3.2.2.1 Class overview

Every object description contains an index, an object code, object attributes, system specific references to the real object and, if need be, a name and an extension.

The object is addressed by the index and identified by the object description. The object code is the identifier of the object and indicates which class this object belongs to. The other object attributes are object specific. The content of the extension field is application-specific.

The local address serves the internal addressing of the object. Besides the index, optionally a name for the addressing of the object may also be used. The extension contains a length information for the extension and optionally further object attributes. The value "0" for the length means: no further object attributes.

14.3.2.2.2 OD description class specification

14.3.2.2.2.1 Formal model

The structure of the Object Dictionary is described by the first entry in the OD. This has the index "0" in the Object Dictionary. By reading the OD object description the structure description and the version number of the OD are made available.

ASE:	Object Dictionary ASE
CLASS:	OD Description
CLASS ID:	—
PARENT CLASS:	TOP
ATTRIBUTES:	
1. (m) Key Attribute:	OD Index
2. (m) Attribute:	ROM/RAM Flag
3. (m) Attribute:	Name Length
4. (m) Attribute:	Access Protection Supported
5. (m) Attribute:	Version OD
6. (m) Attribute:	Local Address OD-ODES
7. (m) Attribute:	ST-OD Length
8. (m) Attribute:	LocalAddress ST-OD
9. (m) Attribute:	First Index S-OD
10. (m) Attribute:	S-OD Length
11. (m) Attribute:	Local Address S-OD
12. (m) Attribute:	First Index DV-OD
13. (m) Attribute:	DV-OD Length
14. (m) Attribute:	LocalAddress DV-OD
15. (m) Attribute:	First Index DP-OD
16. (m) Attribute:	DP-OD Length
17. (m) Attribute:	Local Address DP-OD

SERVICES:

1. (m) OpsService: Get OD
2. (m) OpsService: Initiate Put OD
3. (m) OpsService: Put OD
4. (m) OpsService: Terminate Put OD

14.3.2.2.2.2 Attributes

OD Index

OD Index of the OD Object Description, here always = 0.

ROM/RAM Flag

This attribute of type Boolean describes whether modifications in the OD are allowed.

false <=> no modifications in the OD are permitted

true <=> modifications in the OD are permitted

Name Length

This field gives the length of the name and may take only the values "0..32":

0 <=> no names are used

Access Protection Supported

This attribute states, whether access rights for pass-word, access groups and all communication partners are supported (true), or whether the access to all objects is permitted for every communication partner (false).

Version OD

Gives the version of the OD.

Local Address OD-ODES

This attribute is a system specific reference to the real OD object description and serves the internal addressing. If no representation of this kind is used, this attribute is set to the value FFFFFFFF hex.

ST-OD Length

This attribute gives the maximal number of available entries in the Static List of Types.

NOTE The first available index of the Static List of Types is always = 1, therefore the highest available index in the Static List of Types is equal to its length.

Local Address ST-OD

This attribute is a system specific reference to the real ST-OD and serves the internal addressing. If no representation of this kind takes place, this attribute is set to the value FFFFFFFF hex.

First Index S-OD

The first available index in the Static Object Dictionary.

S-OD Length

This attribute gives the maximum number of available entries in the Static Object Dictionary.

NOTE The highest available index is S-OD-Length = FirstIndex-S-OD + S-OD-Length - 1 (done)

Local Address S-OD

This attribute is a system specific reference to the real S-OD and serves the internal addressing. If no representation of this kind takes place, this attribute is set to the value FFFFFFFF hex.

First Index DV-OD

First available index in the Dynamic List of Variable Lists.

0 <=> no Dynamic List of Variable Lists.

DV-OD Length

This attribute gives the maximum number of available entries in the Dynamic List of Variable Lists.

NOTE The highest available index is $DV-OD-Length = FirstIndex-DV-OD + OD-OD-Length - 1$

Local Address DV-OD

This attribute is a system specific reference to the real DV-OD and serves the internal addressing. If no representation of this kind takes place, this attribute is set to the value FFFFFFFF hex.

First Index DP-OD

First available index in the Dynamic List of Program Invocations.

0 <=> no Dynamic List of Program Invocations.

DP-OD Length

This attribute gives the maximum number of available entries in the Dynamic List of Program Invocations.

NOTE The highest available index is $DP-OD-Length = FirstIndex-DP-OD + DP-OD-Length - 1$

Local Address DP-OD

This attribute is a system specific reference to the real DP-OD and serves the internal addressing. If no representation of this kind takes place, this attribute is set to the value FFFFFFFF hex.

14.3.2.2.2.3 Services

All services defined for this class are mandatory.

Get OD

This confirmed service is used to read entries of the OD.

Initiate Put OD

This confirmed service is used to begin the process of writing entries of the OD.

Put OD

This confirmed service is used to write entries of the OD.

Terminate Put OD

This confirmed service is used to terminate the process of writing entries of the OD.

14.3.2.2.2.4 Empty object dictionary

The Object Dictionary is loadable. Therefore if an Object Dictionary is not yet loaded, at least an empty Object Dictionary shall exist.

An empty Object Dictionary is defined by pre-setting the attributes of the OD object description shown in Table 571:

Table 571 – Empty object dictionary

Attribute	Value
ROM/RAM-Flag	True
NameLength	0
AccessProtectionSupported	False
VersionOD	0
ST-OD-Length	0
FirstIndex-S-OD	0
S-OD-Length	0
FirstIndex-DV-OD	0
DV-OD-Length	0
FirstIndex-DP-OD	0
DP-OD-Length	0

14.3.2.2.3 OD object formal model

14.3.2.2.3.1 Formal model

The structure of the Object Dictionary is described by the first entry in the OD. This has the index "0" in the Object Dictionary. By reading the OD object description the structure description and the version number of the OD are made available.

ASE: Object Dictionary ASE

CLASS: OD Object

CLASS ID: —

PARENT CLASS: TOP

ATTRIBUTES:

1. (m) Key Attribute: implicit by VFD
2. (m) Attribute: OD object description
3. (m) Attribute: List of object description
4. SERVICES:
 1. (m) OpsService: Get OD
 2. (m) OpsService: Initiate Put OD
 3. (m) OpsService: PutOD
 4. (m) OpsService: Terminate Put OD

14.3.2.2.3.2 Attributes

implicit by VFD

The VCR, which is entered in the VCR List points to the assigned Object Dictionary.

OD object description

Contains the object description of the Object Dictionary.

List of object description

Contains object descriptions of the following objects:

- Domain
- ProgramInvocation
- SimpleVariable
- Array
- Record
- VariableList
- DataType
- DataTypeStructureDescription
- Event

14.3.2.2.3.3 Services

All services defined for this class are mandatory.

Get OD

This confirmed service is used to read entries of the OD.

Initiate Put OD

This confirmed service is used to begin the process of writing entries of the OD.

Put OD

This confirmed service is used to write entries of the OD.

Terminate Put OD

This confirmed service is used to terminate the process of writing entries of the OD.

14.3.2.3 Object dictionary model services

14.3.2.3.1 Supported services

OD services are specified to read or write the object descriptions of Domain, Program Invocation, Simple Variable, Array, Record, Variable List, Event, Object Dictionary, Data Type and Data Type Structure Description objects. The attributes of an APO which can be written are specified as part of the attribute definition.

Because unexpected changes to the object definitions within an AP can affect operation of the AP, the Initiate Put OD and the Terminate Put OD services are defined. The Initiate Put OD service is used to request the remote AP for permission to change the OD. The Terminate Put OD service is used to indicate to the remote AP that changes to its OD are complete and that it can resume normal operation.

With the GetOD service one or several object descriptions are read. With the PutOD service one or several object descriptions are written. The number of object descriptions, which are transferable with one PutOD or GetOD Service, depends on their length and the maximum available PDU size.

With the PutOD services one or several object descriptions are written. Before the beginning of a modification of the OD an user may take appropriate measures (e.g. to put a process into a safe state). The communication does not test the plausibility of the new Object Dictionary.

Writing in the OD is performed with the sequence InitiatePutOD service, one or several PutOD services and a TerminatePutOD service. For this it is distinguished between a loading which is free of interactions and a loading which is not free of interactions (OD-LOADING-NON-INTERACTING and OD-LOADING-INTERACTING). The loading which is not free of interactions differentiates between a new loading (complete) and a post loading (partial). If need be, new communication objects and their state machines are created with the TerminatePutOD.

Loading free of interactions (OD-LOADING-NON-INTERACTING):

The client communicates to the server with the parameter Consequence = 0, whether the intended modifications or extensions are free of interactions for the other communication partners. In this case the VCRs are continued. For example, modifications free of interactions are:

- Addition of new object descriptions in the static segment
- Deletion of private object descriptions for stations which are no longer in the communication system

The communication does not check whether the write to the OD is really free of interactions.

The loading free of interaction is initiated with the service InitiatePutOD with the parameter Consequence = 0. After that object descriptions may be loaded with PutOD. As long as the loading process is not terminated with TerminatePutOD, every further InitiatePutOD is rejected.

Loading not free of interactions (OD-LOADING-INTERACTING):

The loading not free of interactions is initiated with the service InitiatePutOD with the Parameter Consequence = 1 (post-loading) or Consequence = 2 (new-loading). In this case the user may abort all active VCRs, except the one over which the InitiatePutOD is received to this VFD. After that the object description may be loaded with PutOD. It is also the task of the client user to enter the new version number in the OD. The loading process is terminated with the service TerminatePutOD. After that all VCRs may be re-established. With the re-establishment of the VCRs the new version number of the OD is transferred. The communication partners may then recognize that the corresponding OD has been modified. Whereas for post-loading solely parts of the OD are loaded, the existing OD is deleted and subsequently loaded completely anew upon receipt of an InitiatePutOD with Consequence = 2.

14.3.2.3.2 Get OD service

14.3.2.3.2.1 Service overview

The service distinguishes between a short and a long form. The long form of the GetOD service is optional. To read a single object description, the index or the name shall be given.

To read several or all object descriptions, the index of the first required object description (StartIndex) shall be given. To read the whole OD, the GetOD service shall be used repeatedly.

14.3.2.3.2.2 Service parameters

The service parameters for this service are shown in Table 572.

Table 572 – Get OD service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
All Attributes	M	M(=)		
Access Specification	M	M(=)		
Index	S	S(=)		
Variable Name	S	S(=)		
Variable List Name	S	S(=)		
Domain Name	S	S(=)		
PI Name	S	S(=)		
EventName	S	S(=)		
Start Index	S	S(=)		
Result(+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
List of Object Description			M	M(=)
More Follows			M	M(=)
Result(-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

All Attributes

This parameter states whether the object description shall be transferred in the short form (false) or in the long form (true).

The following object attributes are not contained in the short form:

Password
 AccessGroups
 AccessRights
 LocalAddress
 Name
 LocalAddress-OD-ODES
 LocalAddress-ST-OD
 LocalAddress-S-OD
 LocalAddress-DV-OD
 LocalAddress-DP-OD
 Extension

Access Specification

This parameter states which object description is accessed.

Index

Index of the object description.

Variable Name

Name of a variable.

Variable List Name

Name of a Variable List.

PI Name

Name of a Program Invocation.

Domain Name

Name of a Domain.

Event Name

Name of an Event.

StartIndex

Index, starting with which the object descriptions shall be read. When this option is selected, the responding VFD sends object descriptions of indexes greater than StartIndex in the order of increasing Index. When all of requested object descriptions are not sent in the response, the MoreFollows flag shall be set.

Result(+)

This selection type parameter indicates that the service request succeeded.

List of object description

For logical access and for access by name one, otherwise several complete object descriptions. The number depends on the maximum possible PDU size and the length of the single object descriptions. If access rights are not supported (Access Protection = false), the attributes Password, Access Groups and Access Rights are meaningless.

More Follows

States whether further object descriptions exist after reading several object descriptions selecting StartIndex for Access Specification. This parameter is set true when the responding VFD has at least one object description of Index greater than that of the last object description in the response. It is set false when the responding VFD has no more object descriptions. For the access to single object descriptions selecting other options for Access Specification, descriptions this parameter is always set to false.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.2.3.2.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

If the user is able to access the specified object descriptions, and if they can be conveyed in a single APDU, they are returned in the response (+) primitive.

If they cannot be conveyed in a single APDU, the user returns those that can and sets the more flag in the response (+) primitive. The requester is then responsible for submitting additional requests identifying the remaining attributes to be retrieved.

If the user is unable to get the description for the first object in the list, the service fails and the user issues a response (-) primitive indicating the reason.

14.3.2.3.3 InitiatePutOD

14.3.2.3.3.1 Service overview

The InitiatePutOD opens the process of loading the OD free of interaction or not free of interaction.

14.3.2.3.3.2 Service parameters

The service parameters for this service are shown in Table 573.

Table 573 – Initiate Put OD service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Consequence	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Consequence

This parameter states whether the intended modifications are free of interaction for the other communication partners, not free of interaction with partial reload of the OD, or not free of interaction with completely new load of the OD.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.2.3.3.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.2.3.4 Put OD service

14.3.2.3.4.1 Service overview

With the PutOD service one or several object descriptions are written into the OD.

14.3.2.3.4.2 Service parameters

The service parameters for this service are shown in Table 574.

Table 574 – Put OD service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
List of Object Description	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

List of Object Description

One or several object descriptions which shall be entered into the OD. If access rights are not supported (Access Protection = false), the attributes Password, Access Groups and Access Rights are meaningless.

An object description may also be deleted with the PutOD service. In this case, an empty object description with the following structure is transferred:

- Index of the object description to be deleted
- Object Code = Null Object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.2.3.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.2.3.5 Terminate put OD service

14.3.2.3.5.1 Service overview

The Terminate Put OD service terminates the process of loading of the OD. The objects with their state machines are generated.

14.3.2.3.5.2 Service parameters

The service parameters for this service are shown in Table 575.

Table 575 – Terminate Put OD service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)
Index			C	C(=)

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Index

This parameter gives the Index of the object, for which the generation was not successful.

14.3.2.3.5.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.2.3.6 Put OD services state machine

The state machine is shown in Figure 158.

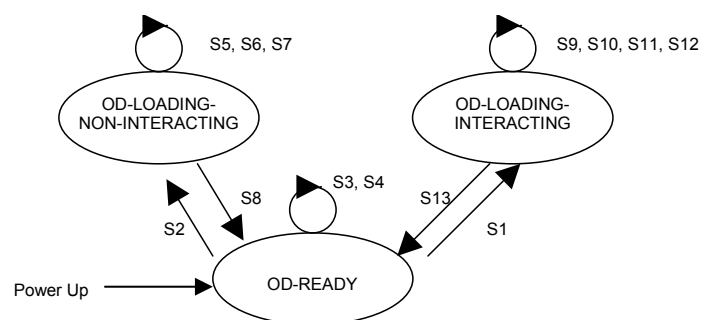


Figure 158 – Put OD state machine

14.3.2.3.6.1 States

OD-LOADING-NON-INTERACTING

In this state the execution of the service InitiatePutOD is not permitted. In this state the service PutOD on the DV-OD or the DP-OD is not permitted.

OD-LOADING-INTERACTING

Except for the connection over which the InitiatePutOD service was received, all connections are locked and the establishment of a further connection shall be rejected. On this connection only the following services are allowed:

- Initiate
- Abort
- Reject
- Status
- Identify
- PhysRead
- PhysWrite
- GetOD
- InitiatePutOD
- PutOD
- TerminatePutOD

OD-READY

In this state all connections and services may be used normally.

14.3.2.3.6.2 State transitions

Table 576 specifies the state transitions related to PutOD.

Table 576 – Put OD state transitions

#	Current State	Events	Actions	Next State
S1	OD-READY	InitiatePutOD.ind && consequence <> loading-free-of-interaction	InitiatePutOD.rsp(+){ }	OD-LOADING-INTERACTING
S2	OD-READY	InitiatePutOD.ind && consequence = loading-free-of-interaction	InitiatePutOD.rsp(+){ }	OD-LOADING-NON-INTERACTING
S3	OD-READY	PutOD.ind	PutOD.rsp(-){ ErrorType = object-state-conflict }	OD-READY
S4	OD-READY	TerminatePutOD.ind	TerminatePutOD.rsp(-){ ErrorType = object-state-conflict }	OD-READY
S5	OD-LOADING-NON-INTERACTING	InitiatePutOD.ind	InitiatePutOD.rsp(-){ ErrorType = object-state-conflict }	OD-LOADING-NON-INTERACTING
S6	OD-LOADING-NON-INTERACTING	PutOD.ind	PutOD.rsp(+){ }	OD-LOADING-NON-INTERACTING
S7	OD-LOADING-NON-INTERACTING	TerminatePutOD.ind && OD is not usable	TerminatePutOD.rsp(-){ ErrorType = operational-problem }	OD-LOADING-NON-INTERACTING
S8	OD-LOADING-NON-INTERACTING	TerminatePutOD.ind && OD is okay	TerminatePutOD.rsp(+){ }	OD-READY
S9	OD-LOADING-INTERACTING	InitiatePutOD.ind && consequence <> loading-free-of-interaction	InitiatePutOD.rsp(+){ }	OD-LOADING-INTERACTING
S10	OD-LOADING-INTERACTING	InitiatePutOD.ind && consequence = loading-free-of-interaction	InitiatePutOD.rsp(-){ ErrorType = object-state-conflict }	OD-LOADING-INTERACTING
S11	OD-LOADING-INTERACTING	PutOD.ind	PutOD.rsp(+){ }	OD-LOADING-INTERACTING
S12	OD-LOADING-INTERACTING	TerminatePutOD.ind && OD is not usable	TerminatePutOD.rsp(-){ ErrorType = operational-problem }	OD-LOADING-INTERACTING
S13	OD-LOADING-INTERACTING	TerminatePutOD.ind && OD is okay	TerminatePutOD.rsp(+){ }	OD-READY

14.3.3 Context management ASE

14.3.3.1 Overview

The context management services are used to explicitly establish a VCR, to release a VCR and to reject improper services. The data necessary for this are stored in the OD object description and in the VCR object. Transaction objects are provided for each VCR. The transaction objects are used for the management of confirmed service primitives.

14.3.3.2 Context management class specifications

14.3.3.2.1 VCR list class specification

14.3.3.2.1.1 VCR list formal model

The Virtual Communication Relationship List (VCR List) contains the descriptions of all VCRs of a device. A VCR consists of static part and dynamic part. This subclause gives the definition to FMS specific part of VCR. Refer Network Management Specification for the definition of other part of VCR.

ASE:	Context Management ASE
CLASS:	VCR List
CLASS ID:	—
PARENT CLASS:	TOP
VCR STATIC ATTRIBUTES:	
1.1 (m) Attribute:	Static VCR ID
1.2 (m) Attribute:	Max APDU Size Sending
1.3 (m) Attribute:	Max APDU Size Receiving
1.4 (m) Attribute:	Features Supported Length
1.5 (m) Attribute:	Features Supported
1.6 (m) Attribute:	Max Outstanding Services Sending
1.7 (m) Attribute:	Max Outstanding Services Receiving
1.8 (m) Attribute:	VFD ID
VCR DYNAMIC ATTRIBUTES:	
2.1 (m) Attribute:	Dynamic VCR ID
2.2 (m) Attribute:	Outstanding Services Counter Client
2.3 (m) Attribute:	Outstanding Services Counter Server
2.4 (m) Attribute:	VCR State
2.5 (m) Attribute:	Actual Max Outstanding Services Sending
2.6 (m) Attribute:	Actual Max Outstanding Services Receiving
SERVICES:	
1. (m) Ops Service:	Initiate
2. (m) Ops Service:	Abort
3. (m) Ops Service:	Reject

14.3.3.2.1.2 Attributes

VCR STATIC ATTRIBUTES:

Static VCR ID

This attribute is the index in the Object Dictionary for the static part of this VCR.

Max APDU Sending

This attribute specifies the maximum length of the APDU that may be sent on this VCR.

Max APDU Receiving

This attribute specifies the maximum length of the APDU that may be received on this VCR.

Features Supported Length

This attribute defines the number of octets present in the Features Supported field.

Features Supported

This attribute identifies the services and features supported on this VCR, as shown in Table 577. This subclause specifies bit assignment of fixed part and the method for extension of services and features. The Features Supported is of the length specified by Features Supported Length. It consists of two concatenated bit strings of the same length. The first bit string identifies services and features as a client and the second bit string identifies services and features as a server. For each optional service there are two bits, one in the first bit string and another in the second bit string. For the optional feature there are two bits, one in the first bit string, specifying the optional feature is permissible. The extension of the attribute can be made by using unassigned bits or adding another octet at the end of each bit string. All bits within a byte, which are not specified, have the value 0.

Table 577 – Attribute FMS features supported

Service	Primitive of client functions	bit(n)	Primitive of server functions	bit(n)
GetOD (Long form)	.req, .cnf	0	.ind, .rsp	0
UnsolicitedStatus	.req	1	.ind	1
InitiatePutOD	.req, .cnf	2	.ind, .rsp	2
PutOD	.req, .cnf	"	.ind, .rsp	"
TerminatePutOD	.req, .cnf	"	.ind, .rsp	"
InitiateDownloadSequence	.req, .cnf	3	.ind, .rsp	3
DownloadSegment	.ind, .rsp	"	.req, .cnf	"
TerminateDownloadSequence	.ind, .rsp	"	.req, .cnf	"
InitiateUploadSequence	.req, .cnf	4	.ind, .rsp	4
UploadSegment	.req, .cnf	"	.ind, .rsp	"
TerminateUploadSequence	.req, .cnf	"	.ind, .rsp	"
RequestDomainDownload	.req, .cnf	5	.ind, .rsp	5
RequestDomainUpload	.req, .cnf	6	.ind, .rsp	6
CreateProgramInvocation	.req, .cnf	7	.ind, .rsp	7
DeleteProgramInvocation	.req, .cnf	"	.ind, .rsp	"
Start	.req, .cnf	8	.ind, .rsp	8
Stop	.req, .cnf	"	.ind, .rsp	"
Resume	.req, .cnf	"	.ind, .rsp	"
Reset	.req, .cnf	"	.ind, .rsp	"
Kill	.req, .cnf	9	.ind, .rsp	9
Read	.req, .cnf	10	.ind, .rsp	10
Write	.req, .cnf	11	.ind, .rsp	11
ReadWithType	.req, .cnf	12	.ind, .rsp	12
WriteWithType	.req, .cnf	13	.ind, .rsp	13
PhysRead	.req, .cnf	14	.ind, .rsp	14
PhysWrite	.req, .cnf	15	.ind, .rsp	15
InformationReport	.req	16	.ind	16
InformationReportWithType	.req	17	.ind	17
DefineVariableList	.req, .cnf	18	.ind, .rsp	18
DeleteVariableList	.req, .cnf	"	.ind, .rsp	"
EventNotification	.req	19	.ind	19
EventNotificationWithType	.req	20	.ind	20
AcknowledgeEventNotification	.req, .cnf	21	.ind, .rsp	21
AlterEventConditionMonitoring	.req, .cnf	22	.ind, .rsp	22
Addressing with Name	.req	23	.ind	23
Extensions	Primitive of client functions	bit[n]	Primitive of server functions	bit[n]
GenericInitiateDownloadSequence	.req, .cnf	24	.ind, .rsp	24
GenericDownloadSegment	.req, .cnf	"	.ind, .rsp	"
GenericTerminateDownloadSequence	.req, .cnf	"	.ind, .rsp	"
Explanation: [n] : 0 to (23+8x); without extension ==> x = 0; with extension ==> x = 1,2, ...				

Following services are mandatory for all VCRs to open and close them:

- Initiate - Abort

For consistency, the server supports at least the following basic services:

- Identify - Status
- Reject - GetOD

Max Outstanding Services Sending (MaxSCC)

This attribute specifies the maximum number of outstanding confirmed services capable in this station as the requester side of this VCR.

Max Outstanding Services Receiving (MaxRCC)

This attribute specifies the maximum number of outstanding confirmed services capable in this station as the responder side of this VCR.

VFD Reference

This attribute identifies the VFD that uses the VCR.

VCR DYNAMIC ATTRIBUTES:

Outstanding Services Counter Client (OSCS)

This attribute shall specify the number of currently outstanding confirmed services on this VCR as a requester.

Outstanding Services Counter Server (OSCR)

This attribute shall specify the number of outstanding services that are currently being processed on this VCR as a responder.

VCR State

This attribute shall specify the state of the VCR. A connection-oriented VCR may be in one of the following states:

- CONNECTION-NOT-ESTABLISHED
- CONNECTION-ESTABLISHING (CALLING)
- CONNECTION-ESTABLISHING (CALLED)
- CONNECTION-ESTABLISHED

A connectionless VCR may be in one of the following states:

- CONNECTIONLESS-CLIENT
- CONNECTIONLESS-SERVER

Actual Max Outstanding Services Sending (ActualMaxSCC)

This attribute specifies the maximum number of outstanding confirmed services allowed in this station as the requester side of this VCR. Its value shall be negotiated during initialization of this VCR.

Actual Outstanding Services Receiving (ActualMaxRCC)

This attribute specifies the maximum number of outstanding confirmed services allowed in this station as the responder side of this VCR. Its value shall be negotiated during initialization of this VCR.

14.3.3.2.1.3 Services

Initiate

This optional service is used to open the VCR.

Abort

This optional service is used to close the VCR.

Reject

This optional service is used to reject a service primitive submitted to the VCR.

14.3.3.2.2 The transaction object class specification

14.3.3.2.2.1 Formal model

A Transaction Object shall be created upon receipt of a confirmed service indication primitive. The Transaction Object shall be uniquely related to the corresponding service primitive. The Transaction Object shall be deleted after the corresponding response primitive has been sent. The Transaction Object shall be uniquely identified by the combination of Invoke ID and Static VCR ID.

The maximum number of Transaction Objects for each VCR of the server shall be specified by the Max Outstanding Services Receiving attribute in the VCR entry. The following services act on the Transaction Object:

ASE:	Context Management ASE
CLASS:	Transaction Object
CLASS ID:	—
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Invoke ID & VCR ID
2 (m) Attribute:	Confirmed service indication

14.3.3.2.2.2 Attributes

Invoke ID & VCR ID

Invoke ID

This attribute shall identify the Transaction Object within the VCR.

VCR ID

This attribute shall identify the VCR on which the Transaction Object is used.

Confirmed service indication

Service identifier and argument of the pending service.

14.3.3.2.2.3 Services

Initiate

This optional service is used to open the VCR.

Abort

This optional service is used to abruptly close the VCR.

Reject

This service is initiated internally by the AP ASE to indicate that a confirmed service request APDU has been received with a protocol error.

14.3.3.2.2.4 State machine

14.3.3.2.2.4.1 State machine description

The state machine is shown in Figure 159.

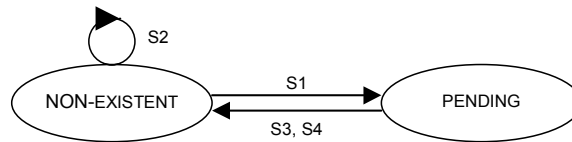


Figure 159 – Transaction object state machine

NON-EXISTENT

The Transaction Object does not exist.

PENDING

A confirmed service indication primitive has been received and the corresponding response primitive has not yet been sent.

14.3.3.2.4.2 State transitions

Table 578 specifies the state transitions related to transaction objects.

Table 578 – Transaction object state transitions

#	Current State	Events	Actions	Next State
S1	NON_EXISTENT	Confirmed service.ind && OSC<R = Act MaxRCC	(no actions taken)	PENDING
S2	NON_EXISTENT	Confirmed service.ind && OSC<R = Act MaxRCC	(no actions taken)	NON_EXISTENT
S3	PENDING	Confirmed service.rsp	(no actions taken)	NON_EXISTENT
S4	PENDING	Abort.ind	(no actions taken)	NON_EXISTENT

14.3.3.3 Context management ASE service specification

14.3.3.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Initiate

Abort

Reject

14.3.3.3.2 Initiate service

14.3.3.3.2.1 Service overview

This service shall be used to establish a connection between two communication partners and to exchange information regarding the supported services, the supported options, the maximum PDU length and the current version of the OD.

When the type of a VCR is such that FMS Initiate PDU is not exchanged, the request is locally confirmed and no information on the communication partner is available in confirmation

14.3.3.3.2.2 Service primitives

The service parameters for this service are shown in Table 579.

Table 579 – Initiate service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Version OD Calling	M	M(=)		
Profile Number Calling	M	M(=)		
Access Protection Supported Calling	M	M(=)		
Password Calling	M	M(=)		
Access Groups Calling	M	M(=)		
Destination DL-Address	C			
Result(+)			S	S(=)
VCR			M	M(=)
Version OD Called			M	M(=)
Profile Number Called			M	M(=)
Access Protection Supported Called			M	M(=)
Password Called			M	M(=)
Access Groups Called			M	M(=)
Result(-)			S	S(=)
VCR			M	M(=)
Error Code			M	M(=)
Max APDU Length Sending Called				C
Max APDU Length Receiving Called				C
Features Supported Called				C

Argument

This parameter carries the parameters of the service invocation.

Version OD Calling

This parameter specifies the version of the client's OD. Its value is null if the client does not contain an OD.

Profile Number Calling

This parameter contains the value of the client's Profile Number attribute if it has one. If it does not, its value is null.

Access Protection Supported Calling

This parameter contains the value of the client's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password Calling

This parameter specifies the password to be used for the access to all objects of the server on this AR. If access with password is not to be used on this AR, its value is null.

Access Groups Calling

This parameter specifies the client's membership in specific access groups. The membership applies for the access to all objects of the server on this AR.

Destination DL-Address

This parameter exists only when the corresponding AR supports it and the Remote Address Configuration Type is FREE. It gives the remote address to which the connection shall be established.

Result(+)

This selection type parameter indicates that the service request succeeded.

Version OD Called

This parameter specifies the version of the server's OD.

Profile Number Called

This parameter contains the value of the server's Profile Number attribute.

Access Protection Supported Called

This parameter contains the value of the server's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password Called

This parameter specifies the password to be used for the access to all objects of the client on this VCR. If access with password is not to be used on this VCR, its value is null.

Access Groups Called

This parameter specifies the server's membership in specific access groups. The membership applies for the access to all objects of the client on this VCR.

Result(-)

This selection type parameter indicates that the service request failed.

Error Code

This parameter shall provide the reason for the failure as specified in Table 580.

Table 580 – Failure reasons

Reason	Meaning
Max APDU Size Insufficient	The maximum APDU length is not sufficient for the communication.
Feature Not Supported	The requested service or option is not supported by the server.
User Initiate Denied	The user refuses to establish the connection.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already a VCR established with the same password, or the password is not valid.
Profile number incompatible	The client's profile number is not supported by the server.
Other	Reason other than any of those identified above.

Max PDU Length Sending Called

This parameter specifies the maximum length of the APDU to be sent by the server on this VCR.

Max PDU Length Receiving Called

This parameter specifies the maximum length of APDUs to be received by the server on this VCR.

Features Supported Called

This parameter identifies the optional services and options supported by the server on this VCR.

14.3.3.3.2.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.3.3.3 Abort service

14.3.3.3.3.1 Service overview

This service shall be used to release an existing AR between APs, or to terminate a subscriber/receiver AREP's involvement in an AR.

14.3.3.3.3.2 Service primitives

The service parameters for this service are shown in Table 581.

Table 581 – Abort service parameters

Parameter name	Req	Ind
Argument	M	M(=)
VCR	M	M(=)
Locally Generated		M
Abort Identifier	M	M(=)
Reason Code	M	M(=)
Abort Detail	U	U(=)

Argument

This parameter carries the parameters of the service invocation.

Locally Generated

This parameter shall indicate whether the termination was generated locally or by the communication partner.

The value false is not allowed if the Abort Identifier parameter has the value APO ASE and the Reason Code parameter has the value VCR Error.

Abort Identifier

This parameter shall indicate where the reason for the termination has been detected. Possible values are:

USER

APO ASE

AR ASE

DLL

Reason Code

This parameter shall specify the reason for the termination.

If the Abort Identifier parameter has the value USER, the values listed in Table 582 shall apply.

Table 582 – User abort reasons

Reason	Meaning
Disconnection	The connection is released by the user.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already an AR established with the same password, or the password is not valid.
Profile Number incompatible	The server's profile number is not supported by the client.
Limited Services Permitted	The AP is in the Logical Status LIMITED-SERVICES-PERMITTED.

If the Abort Identifier parameter has the value APO ASE, the values listed in Table 583 shall apply.

Table 583 – APO ASE abort reasons

Reason	Meaning
VCR Error	Faulty VCR
User Error	Improper, unknown or faulty service primitive received from the user
APDU Error	Unknown or faulty APDU received
Connection State Conflict AR ASE	Improper AR ASE service primitive
AR ASE Error	Unknown or faulty AR ASE service primitive
APDU Size	APDU length exceeds maximum APDU length
Feature Not Supported	SERVICE-REQ PDU received from the AR ASE and service or option not supported as a server (see Features Supported attribute in the VCR)
Invoke ID Error Response	Confirmed service.rsp received from the user and Invoke ID does not exist or CONFIRMED_SERVICE-RSP_PDU received from the AR ASE and Invoke ID does not exist
Max Services Overflow	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Outstanding Services Counter Receiving $\geq$ Max Outstanding Services Receiving
Connection State Conflict	INITIATE-REQ_PDU received from the AR ASE
Service Error	The service in the response does not match the service in the indication or the service in the confirmation does not match the service in the request.
Invoke ID Error Request	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Invoke ID already exists.

If the Abort Identifier parameter has the value AR ASE or DLL, the Reason Code parameter shall be supplied by the AR ASE.

Abort Detail

This optional parameter contains additional information about the abort reason (max. 16 octets). In case of error reports from the application, the meaning is defined in the profile.

14.3.3.3.3 Service procedure

The Unconfirmed Service Procedure specified in Clause 4 applies to this service.

14.3.3.3.4 Reject service

14.3.3.3.4.1 Service overview

This service is used to inform the remote endpoint that a protocol error has been detected. It is generated by the AP ASE if an APDU has been received with an invalid service type code. When another APO ASE detects a protocol error, it internally requests the AP ASE to generate a Reject PDU. This service is not used to indicate that a service error has been detected by the user.

14.3.3.3.4.2 Service primitives

The service parameters for this service are shown in Table 584.

Table 584 – Reject service parameters

Parameter name	Ind
Argument	M(=)
VCR	M(=)
Detected Here	M(=)
Original Invoke ID	C(=)
Reject APDU Type	M(=)
Reject Code	M(=)

Argument

The argument contains the parameters of the service request.

Detected Here

This parameter indicates if the error has been detected locally (TRUE). The value FALSE is allowed only if the Reject PDU Type parameter has the value Confirmed-Response-PDU and the Reject Code parameter has the value PDU Size.

Original Invoke ID

This parameter contains the invoke id of the rejected APDU, if it has one.

Reject APDU Type

This parameter shall indicate the type of the rejected PDU. The permissible values are as follows:

- Confirmed-Request-PDU
- Confirmed-Response-PDU
- Unconfirmed-PDU
- Unknown type of PDU

Reject Code

This parameter indicates the reason for rejection as listed in Table 585.

Table 585 – Reject APDU reasons

Reason	Meaning
Invoke-ID-Exists	Confirmed service.req received from the user and Invoke ID already exists
Max-Services-Overflow.	Confirmed service.req received from the user and OSGS = ActualMaxSCC
Feature-Not-Supported-Connection-Oriented	Service.req received from the user and service or option not supported as a client (see Features Supported attribute in the VCR)
Feature-Not-Supported-Connectionless.	Unconfirmed service.req received from the user and service or option not supported as a client (see Features Supported attribute in the VCR) or confirmed service.req received from the user
APDU-Size	APDU length exceeds maximum APDU length
User-Error-Connectionless	Improper or faulty service primitive received from the user
Other	Reason other than any of those identified above

14.3.3.3.4.3 Service procedure

If any APO ASE receives an APDU that contains one of the above errors, the AP ASE delivers a Reject indication primitive.

If any APO ASE receives a confirmed request primitive that contains one of the above errors, the AP ASE delivers a Reject indication primitive.

Upon receipt of the Reject Request APDU, the receiving AP ASE delivers an AR-Reject.indication primitive to the requesting user.

14.3.3.4.1 Context test in the AE

Upon receipt of an INITIATE-REQ_PDU the called AE shall check whether the context of the communication partner (remote Context) is compatible with its own context (local context) as defined for this connection in this VCR, as shown in Table 586. The local context is assumed to be correct. The compatibility of the local with the remote context is defined by the following table. In this table, meaningless fields are left blank (e.g. the combination of local context "Max APDU Sending " with remote context "Features Supported"). Those combinations shall not be checked

Table 586 – Compatibility of the local context to the remote context

Remote Content		Local Content					
		Max APDU		Feature Supported			
				Req [n]		Ind [n]	
		Sending	Receiving	0	1	0	1
Max APDU Sending			≥				
Max APDU Receiving		≤					
Features Supported	.req [n] 0					X	X
	.req [n] 1					-	X
	.ind [n] 0			X	-		
	.ind [n] 1			X	X		

Explanation:

.req 0 : Feature is not used as Client
.req 1 : Feature is used as Client
.ind 0 : Feature is not supported as Server
.ind 1 : Feature is supported as Server
≤ : local value smaller than or equal remote value
≥ : local value larger than or equal remote value
X : compatible
- : not compatible (error case)
[n] : 0 to (23+8x); without extension ==> x= 0;
with extension ==> x= 1,2, ...

The length of the Features-Supported is not limited. So it can happen, that the bitstring length of the Features-Supported is different between the bitstring length of the Features-Supported from the communication partners. To check the context it is necessary to approximate the both bitstring length, which shall be compared. The rule is, that the bitstring with the shorter length has to be extended with bits of the value 0, until both bitstrings are equal. Note that the bitstring of the Features Supported consists of a bitstring for the primitive of client functions and a bitstring for the primitive of server functions. The extension has to be symmetrical between the bitstring for the primitive of client functions and the bitstring for the primitive of server functions. This is shown in Figure 160.

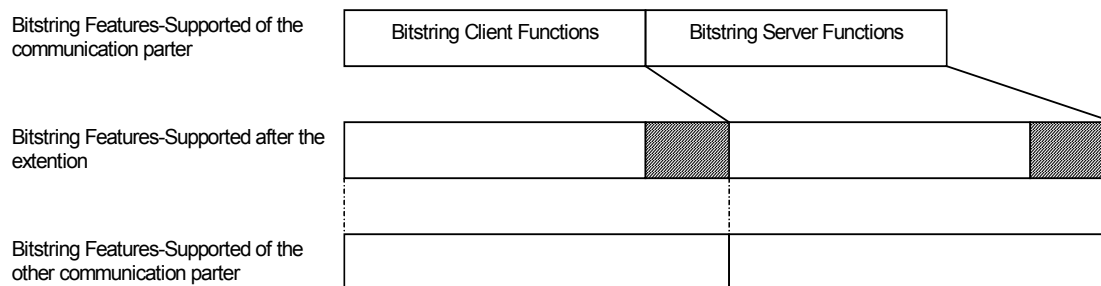


Figure 160 – Context test of two features-supported with different bitstring length

MaxRCC and MaxSCC of initiating endpoint are shown in INITIATE-REQ_PDU. Upon reception of this PDU, the called AE shall store the smaller one of MaxRCC in the PDU and its own MaxSCC to its ActualMaxSCC. It shall also store the smaller one of MaxSCC in the PDU and its own MaxRCC to its ActualMaxRCC. The called AE shall return the resultant ActualMaxRCC and ActualMaxSCC in the MaxRCC and MaxSCC fields of INITIATE-RSP_PDU. Upon reception of this PDU, the initiating FMS shall store them in its own Act MaxSCC and ActualMaxRCC, respectively. If the ActualMaxRCC is greater than MaxRCC or ActualMaxSCC is greater than MaxSCC, the initiating AE shall abort this connection.

14.3.3.4.2 Context test in the user

- Password Test. If the user supports access with password, it shall check if the password is unambiguous. That means, if there is a password, it shall be different from the passwords of all other VCRs.
- If the password is not unambiguous and the user is the server of the Initiate service, it shall issue an Initiate.rsp primitive with the Result(-) parameter and with Error Code "Password Error".
- If the password is not unambiguous and the user is the client of the Initiate service, it shall release the connection with the Abort service, with Reason Code "Password Error".
- Test of Version OD. If the user has a remote OD for this connection, it may check if the received parameter "Version OD" is compatible with the attribute "Version OD" of the associated remote OD.
- If they are incompatible and the user is the server of the Initiate service, it may issue an Initiate.rsp primitive with the Result(-) parameter and with Error Code "Version OD incompatible".
- If they are incompatible and the user is the client of the Initiate service, it may release the connection with the Abort service, with Reason Code "Version OD incompatible".
- Test of Profile Number. The user may check if the received parameter "Profile Number" is compatible with the attribute "Profile Number" of the VFD.
- If they are incompatible and the user is the server of the Initiate service, it may issue an Initiate.rsp primitive with the Result(-) parameter and with Error Code "Profile Number incompatible".
- If they are incompatible and the user is the client of the Initiate service, it may release the connection with the Abort service, with Reason Code "Profile Number incompatible".

14.3.4 Application relationship ASE

14.3.4.1 Overview

The AR ASE supports three types of ARs, all derived from the top level AREP class shown below.

14.3.4.2 Application relationship endpoint class specification

14.3.4.2.1 Formal model

ASE:	AR ASE
CLASS:	AR ENDPOINT
CLASS ID:	
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m)	Key Attribute: Numeric Identifier
SERVICES:	
1 (m)	OpsService: AR-Associate
2 (m)	OpsService: AR-Abort
3 (o)	OpsService: Confirmed Send
4 (o)	OpsService: Unconfirmed Send
5 (o)	OpsService: Compel
6 (o)	OpsService: Get-Buffered-Message
7 (o)	OpsService: AR-Status

14.3.4.2.2 Attributes

Numeric Identifier

Numeric Identifier of the AREP.

14.3.4.2.3 Services

AR-Associate

This service is used to establish an AR.

AR-Abort

This service is used to abruptly de-establish an AR.

Confirmed Send

This service is used to send a confirmed service on the specified AR.

Unconfirmed Send

This service is used to send an unconfirmed service on the specified AR.

Compel

This optional service is used to evoke acyclic communication out on a BNU AREP.

Get-Buffered-Message

This optional service is used to retrieve the current contents of the associated buffer.

AR-Status

This optional service is used to report status of the AR.

14.3.4.3 Application relationship ASE service specifications

14.3.4.3.1 Unconfirmed send service

14.3.4.3.1.1 Service overview

This service is used to send an unconfirmed service on the specified AR.

14.3.4.3.1.2 Service primitives

The service parameters for this service are shown in Table 587.

Table 587 – Unconfirmed Send service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Remote DLSAP Address	C	C(=)
Duplicate FAL PDU Body		C
FAL APDU Body	M	M(=)
Local Timeliness		C
Remote Timeliness		C

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Remote DLSAP Address

This conditional parameter is present if the ConfigurationType attribute of the associated AREP is Free. If it is present, this parameter carries the DLSAP address associated with the remote AREP. If the ConfigurationType attribute is Linked, this parameter is not present.

Duplicate FAL PDU Body

This conditional parameter is present if the DuplicatePduDetectionSupported attribute is True. If it is present, it indicates whether or not the FAL APDU Body is a duplicate of the FAL APDU Body that has been received and delivered in a previous service indication.

This parameter takes a value of True if the same FAL APDU Bodies have been received successively. Otherwise, it takes a value of False.

FAL APDU Body

This parameter contains an unconfirmed service request APDU.

Local Timeliness

This conditional parameter, if it is present and True, indicates that the FAL APDU Body parameter has met the receiving timeliness criteria defined for the DLL. If its value is False, at least one of the timeliness criteria has not been met.

This parameter is present if the value of theSubscriberD1TimelinessClass attribute is other than None and is not present if it is None.

Remote Timeliness

This conditional parameter, if it is present and True, indicates that the FAL APDU Body parameter has met the Publisher's and transmitting DL's timeliness criteria. If its value is False, at least one of the timeliness criteria has not been met.

This parameter is present if the value of thePublisherD1TimelinessClass attribute is other than None and is not present if it is None.

14.3.4.3.2 Confirmed send service

14.3.4.3.2.1 Service overview

This service is used to send a confirmed service on the specified AR.

14.3.4.3.2.2 Service primitives

The service parameters for this service are shown in Table 588.

Table 588 – Confirmed Send service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
AREP	M	M(=)		
Invoke ID	M	M(=)		
FAL APDU Body	M	M(=)		
Result			M	M(=)
AREP			M	M(=)
Invoke ID			M	M(=)
FAL APDU Body			M	M(=)

Argument

The argument contains the parameters of the service request.

FAL APDU Body

This parameter contains an confirmed service request APDU.

Result(+)

FAL APDU Body

This parameter contains an confirmed service response APDU.

14.3.4.3.3 AR-abort service

14.3.4.3.3.1 Service overview

This service is used to abruptly de-establish an AR.

14.3.4.3.3.2 Service primitives

The service parameters for this service are shown in Table 589.

Table 589 – AR-Abort service parameters

Parameter name	Req	Ind
Argument	M	M(=)
AREP	M	M(=)
Locally Generated		M
Identifier	M	M(=)
Reason Code	M	M(=)
Additional Detail	U	U(=)

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Locally Generated

This parameter indicates whether the Abort service was initiated at the local or remote AREP.

Identifier

This parameter indicates whether the Abort service was initiated at the local or remote AREP.

NOTE In cases when the Abort.request or Abort.indication is generated by the AREP, the Additional Detail parameter does not exist. If the Abort.indication is generated by the local AREP, the Identifier and Reason Code parameters are set locally.

Reason Code

This parameter specifies the reason for the AR abort.

Additional Detail

This optional parameter contains additional information about the AR release.

14.3.4.3.4 Compel service**14.3.4.3.4.1 Service overview**

This optional service is used to evoke acyclic communication out on a BNU AREP.

14.3.4.3.4.2 Service primitives

The service parameters for this service are shown in Table 590.

Table 590 – Compel service parameters

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result		S
AREP		M
Status		M

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Result**AREP**

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Status

This parameter indicates the result of the service request. The following three status codes provided by the DLL are defined:

- success
- failure – inappropriate request
- failure – reason unspecified

14.3.4.3.5 Get buffered message service

14.3.4.3.5.1 Service overview

This optional service is used to retrieve the current contents of the associated buffer.

14.3.4.3.5.2 Service primitives

The service parameters for this service are shown in Table 591.

Table 591 – Get Buffered Message service parameters

Parameter name	Req	Cnf
Argument	M	
AREP	M	
Result(+)		S
AREP		M
Duplicate APDU Body		C
FAL PDU Body		M
Local timeliness		C
Remote timeliness		C
Result (-)		S
AREP		M

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Result(+)

AREP

This parameter contains sufficient information to locally identify the AR used to convey the service.

Duplicate FAL PDU Body

This conditional parameter is present if the DuplicatePduDetectionSupported attribute is True. If it is present, it indicates whether or not the FAL APDU Body is a duplicate of the FAL APDU Body that has been received and delivered in a previous service indication.

This parameter takes a value of True if the same FAL APDU Bodies have been received successively. Otherwise, it takes a value of False.

FAL APDU Body

This parameter contains an unconfirmed service request APDU.

Local Timeliness

This conditional parameter, if it is present and True, indicates that the FAL APDU Body parameter has met the receiving timeliness criteria defined for the DLL. If its value is False, at least one of the timeliness criteria has not been met.

This parameter is present if the value of theSubscriberD1TimelinessClass attribute is other than None and is not present if it is None.

Remote Timeliness

This conditional parameter, if it is present and True, indicates that the FAL APDU Body parameter has met the Publisher's and transmitting DL's timeliness criteria. If its value is False, at least one of the timeliness criteria has not been met.

This parameter is present if the value of thePublisherD1TimelinessClass attribute is other than None and is not present if it is None.

Result(-)

This parameter indicates that the service request failed. When this parameter is returned, it means that the buffer attached to the specified AREP does not contain any FAS-PDUs.

AREP

This parameter contains sufficient information to locally identify the AR used to convey the service.

14.3.4.3.6 AR-status service

14.3.4.3.6.1 Service overview

This optional service is used to report status of the AR.

14.3.4.3.6.2 Service primitives

The service parameters for this service are shown in Table 592.

Table 592 – AR-Status service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument		M		
AREP		M		
Status code		M		

Argument

The argument contains the parameters of the service request.

AREP

This parameter contains sufficient information to locally identify the AR to be used to convey the service.

Status Code

This parameter specifies the event which is reported from the DLL. The following event is specified:

Buffer-Sent: The DLL reported that the buffer content had just been sent.

14.3.5 Variable ASE

14.3.5.1 Overview

The Variable ASE is an extended subset of the Type 1 Variable ASE. Extensions incorporate a new class, new services, additional class attributes, and in some cases, restrictions on attribute and parameter values.

The classes contain a subset of the Type 1 attributes and services, and the services contain a subset of the Type 1 parameters.

Therefore, except for the extensions, only the class templates and service tables are shown to reflect the selected subsets. See Type 1 for the definitions of the classes, attributes, and services. The Type 1 attribute and service numbers have been retained in the class template below.

14.3.5.2 Variable model class specifications

14.3.5.2.1 Simple variable class specification

14.3.5.2.1.1 Formal model

The Simple Variable Object represents a single, simple variable which is characterized by a defined Data Type. The Object Description of the Variable Access Object Simple Variable is stored statically in the Object Dictionary (S-OD). The mapping of a Simple Variable to a real Simple Variable, which exists in the application system, is defined by the object description of the Simple Variable Object. Only one single Data Type of the set of configured Data Types is allowed in the Object Description of the Simple Variable Object.

ASE:	VARIABLE ASE
CLASS:	SIMPLE VARIABLE
CLASS ID:	7
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Numeric Identifier
2 (m) Attribute:	Data Type Index
3 (m) Attribute:	Length
4 (m) Attribute:	Password
5 (m) Attribute:	Access Groups
6 (m) Attribute:	Access Rights
7 (m) Attribute:	Local Address
8 (m) Attribute:	Extension
SERVICES:	
1 (o) OpsService:	Read
2 (o) OpsService:	Write
3 (o) OpsService:	Information Report

14.3.5.2.1.2 Attributes

Numeric Identifier

Identifies an instance of this object class.

Data Type Index

Logical address of the corresponding Data Type in the Static Type Dictionary (ST-OD).

Length

Length in octets of the data.

Password

This attribute contains the password for the Access Rights.

Access Groups

This attribute relates the object to specific Access Groups as shown in Table 593. The object is a group member when the corresponding bit is set.

Table 593 – Simple variable access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute contains information about the Access Rights as shown in Table 594. The specific Access Right exists when the corresponding bit is set.

Table 594 – Simple variable access rights membership

Bit	Name	Meaning
7	R	Right to Read for the registered Password
6	W	Right to Write for the registered Password
3	Rg	Right to Read for Access Groups
2	Wg	Right to Write for Access Groups
15	Ra	Right to Read for all Communication Partners
14	Wa	Right to Write for all Communication Partners

Local Address

This attribute is a system specific address of the real object. It serves the internal addressing of the object. If no mapping of this kind is performed, this attribute shall have the value FFFFFFFF hex.

Extension

This attribute contains profile specific information.

14.3.5.2.1.3 Services

Read

This service permits a client to read the value of a variable.

Write

This service permits a client to read the value of a variable.

Information Report

This service permits the VFD to send the value of a variable.

14.3.5.2.2 Array variable class specification

14.3.5.2.2.1 Formal model

The Array Object is used to define a constructed variable in which all elements have the same Data Type and length.

The Object Description of the Variable Access Object Array is stored statically in the Object Dictionary (S-OD). The mapping of a Array Variable to a real array, which exists in the application system, is described by the object description of this object. The object description for Array Objects specifies the number of elements in the array, the Data Type and Length of the elements.

The Array Object may be accessed completely, or element by element using subindexes. Subindex 1 accesses the first element of the object.

ASE:	VARIABLE ASE
CLASS:	ARRAY VARIABLE
CLASS ID:	8
PARENT CLASS:	SIMPLE VARIABLE
ATTRIBUTES:	
1 (m)	Attribute: Number of Elements

14.3.5.2.2.2 Attributes

Number of Elements

States how many elements are contained in the Array.

14.3.5.2.3 Record variable class specification

14.3.5.2.3.1 Formal model

The Record Object consists of a collection of Simple Variables of different Data Types.

The Object Description of the Variable Access Object Record is stored dynamically in the Object Description (S-OD).

The mapping of a Record Variable to a real record, which exists in the application system, is described by the object description of this object. The object description states the relation of the object to a configured Data Type Structure Description.

The Record Object may be accessed completely or element-wise. If the Object shall be accessed completely the index shall be used for the service. If an element of the object shall be accessed then the subindex shall be used together with the index for the service. Subindex 1 accesses the first element of the object.

ASE:	VARIABLE ASE
CLASS:	RECORD VARIABLE
CLASS ID:	9
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m)	Key Attribute: Numeric Identifier
2 (m)	Attribute: Data Type Index
3 (m)	Attribute: Password
4 (m)	Attribute: Access Groups
5 (m)	Attribute: Access Rights
6 (m)	Attribute: List of Local Address
6.1 (m)	Attribute: Local Address
7 (m)	Attribute: Extension
SERVICES:	
1 (o) OpsService:	Read
2 (o) OpsService:	Write
3 (o) OpsService:	Information Report

14.3.5.2.3.2 Attributes

Numeric Identifier

Identifies an instance of this object class.

Data Type Index

Logical address of the corresponding Data Type in the Static Type Dictionary (ST-OD).

Length

Length in octets of the data.

Password

This attribute contains the password for the Access Rights.

Access Groups

This attribute relates the object to specific Access Groups as shown in Table 595. The object is a group member when the corresponding bit is set.

Table 595 – Array variable access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute contains information about the Access Rights as shown in Table 596. The specific Access Right exists when the corresponding bit is set.

Table 596 – Array variable access rights membership

Bit	Name	Meaning
7	R	Right to Read for the registered Password
6	W	Right to Write for the registered Password
3	Rg	Right to Read for Access Groups
2	Wg	Right to Write for Access Groups
15	Ra	Right to Read for all Communication Partners
14	Wa	Right to Write for all Communication Partners

List of Local Address

This attribute is a list of addresses, one for each element in the record. The order in the list is defined by the order of the elements in the record.

Local Address

This attribute is a system specific address of the real object. It serves the internal addressing of the object.. If the real objects are stored sequentially without gaps, then only the Local Address of the first element is given. If no mapping of this kind is performed, this attribute shall have the value FFFFFFFF hex.

Extension

This attribute contains profile specific information.

14.3.5.2.3.3 Services

Read

This service permits a client to read the value of a variable.

Write

This service permits a client to read the value of a variable.

Information Report

This service permits the VFD to send the value of a variable.

14.3.5.2.4 Variable list class specification

14.3.5.2.4.1 Formal model

The Variable List Object consists of a collection of Simple, Array, and Record Variable Objects.

The Object Description of the Variable Access Object Variable List is stored dynamically in the Object Dictionary (DV-OD). This Object Description contains an index list of the contained variable objects from the S-OD.

A Variable List may be created and deleted with the DefineVariableList and DeleteVariableList services.

The Access Rights shall be declared for the DefineVariableList service. The Access Rights to the single objects are checked on creation (DefineVariableList) of a Variable List.

The Variable List Object is only created if the requested Access Rights do not exceed the Access Rights to the single objects.

If the same Variable List exists with the requested Access Rights, no new object is created but the Logical Address of the existing object is given to the client. In the other case the Variable List is created with the requested Access Rights.

Only the authorized client with the proper Access Rights may delete a Variable List Object (DeleteVariableList).

If the Object Dictionary is freshly loaded all Variable List Objects are deleted.

ASE:	VARIABLE ASE
CLASS:	VARIABLE LIST
CLASS ID:	10
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m)	Key Attribute: Numeric Identifier
2 (m)	Attribute: Number of Elements
3 (m)	Attribute: Password
4 (m)	Attribute: Access Groups
5 (m)	Attribute: Access Rights
6 (c)	Attribute: Deletable
7 (m)	Attribute: List Of Element Index
7.1 (m)	Attribute: Index
8 (m)	Attribute: Extension

SERVICES:

- 1 (o) OpsService: Read
- 2 (o) OpsService: Write
- 3 (o) OpsService: Information Report
- 4 (o) OpsService: Define Variable List
- 5 (o) OpsService: Delete Variable List

14.3.5.2.4.2 Attributes**Numeric Identifier**

Identifies an instance of this object class.

Number Of Elements

This parameter states the number of Variable Objects in the Variable List.

Password

This attribute contains the password for the Access Rights.

Access Groups

This attribute relates the object to specific Access Groups as shown in Table 597. The object is a group member when the corresponding bit is set.

Table 597 – Variable list access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute contains information about the Access Rights as shown in Table 598. The specific Access Right exists when the corresponding bit is set.

Table 598 – Variable list access rights membership

Bit	Name	Meaning
7	R	Right to Read for the registered Password
6	W	Right to Write for the registered Password
5	D	Right to Delete for the registered Password
3	Rg	Right to Read for Access Groups
2	Wg	Right to Write for Access Groups
1	Dg	Right to Delete for Access Groups
15	Ra	Right to Read for all Communication Partners
14	Wa	Right to Write for all Communication Partners
13	Da	Right to Delete for all Communication Partners

Deletable

This attribute indicates if the Variable List Object may be deleted (true) or not (false) using the DeleteVariableList service.

List of Element Index

This attribute contains the indices of the Variable Access Objects which are combined to this Variable List.

Extension

This attribute contains profile specific information.

14.3.5.2.4.3 Services

Read

This service permits a client to read the value of a variable list.

Write

This service permits a client to read the value of a variable list.

Information Report

This service permits the VFD to send the value of a variable list.

Define Variable List

This service creates a variable list object.

Delete Variable List

This service deletes a variable list object.

14.3.5.2.5 Variable data type class specification

14.3.5.2.5.1 Formal model

The Data Type defines the syntax, the range of the variables and their presentation inside the communication system. The Data Type of an object is indicated by a corresponding attribute in the Object Description. Type definitions may not be made remotely for reasons of efficiency. They have a fixed configuration in the Static Type Dictionary (ST-OD).

This class defines free configurable Data Types and Standard Data Types. They are stored beginning with Index 1 in the Static Type Dictionary (ST-OD). Standard Data Types which are not used are marked as not configured in the ST-OD.

ASE:	VARIABLE ASE
CLASS:	DATA TYPE
CLASS ID:	
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Numeric Identifier
2 (m) Attribute:	Description
2.1 (m) Attribute:	Symbolic Name Length
2.2 (m) Attribute:	Symbolic Name

14.3.5.2.5.2 Attributes

Numeric Identifier

Identifies an instance of this object class.

Description

This attribute contains a textual description of the Data Type. It consists of the Symbolic Name Length and the Symbolic Name.

Symbolic Name Length

This attribute contains the length of the symbol. The value 0 means that no symbol is used.

Symbolic Name

This attribute contains a visible string of 0 to 32 octets as a symbolic description.

14.3.5.2.5.3 Services

None.

14.3.5.2.6 Variable data type structure class specification**14.3.5.2.6.1 Formal model**

The Data Type Structure Object characterizes the size and the structure of records.

The Object Description of the Data Type Structure Object has a fixed configuration in the Static Type Dictionary. It contains a list of elements, consisting of Data Type and Length of the corresponding record element. As Data Types for the single elements only Data Types in the set of configured Data Types of the Static Type Dictionary are allowed.

ASE:		VARIABLE ASE
CLASS:		DATA TYPE STRUCTURE
CLASS ID:		
PARENT CLASS:		TOP
ATTRIBUTES:		
1	(m) Key Attribute:	Numeric Identifier
2	(m) Attribute:	Number Of Elements
3	(m) Attribute:	List Of Elements
3.1	(m) Attribute:	Data Type Index
3.2	(m) Attribute:	Length

14.3.5.2.6.2 Attributes**Numeric Identifier**

Identifies an instance of this object class.

Number Of Elements

Number of elements of the structured Data Type.

List Of Elements

This attribute contains a list of elements, consisting of Data Type and Length of the corresponding record element.

Data Type Index

Index in the ST-OD of the Data Type of element n.

Length

Length of the element n in octets.

14.3.5.2.6.3 Services

None.

14.3.5.3 Variable ASE service specification**14.3.5.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Read
Write
Information Report
Define Variable List
Delete Variable List

14.3.5.3.2 Read service

14.3.5.3.2.1 Service overview

This service permits a client to read the value of a variable or a variable list object.

14.3.5.3.2.2 Service primitives

The service parameters for this service are shown in Table 599.

Table 599 – Read service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Variable Index	M	M(=)		
Variable Subindex	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Data			S	S(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Variable Index

This parameter is the OD index of the variable or variable list.

Variable Subindex

This optional parameter identifies an individual element in an array or record variable by its position within the variable, starting at 1.

Result(+)

Data

This parameter contains the value of the variable or variable list.

14.3.5.3.3 Write service

14.3.5.3.3.1 Service overview

This service permits a client to write the value of a variable or a variable list object.

14.3.5.3.3.2 Service primitives

The service parameters for this service are shown in Table 600.

Table 600 – Write service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Variable Index	M	M(=)		
Variable Subindex	U	U(=)		
Data	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Variable Index

This parameter is the OD index of the variable or variable list.

Variable Subindex

This optional parameter identifies an individual element in an array or record variable by its position within the variable, starting at 1.

Data

This parameter contains the value of the variable or variable list.

14.3.5.3.4 Information report service

14.3.5.3.4.1 Service overview

This service permits the VFD to send the value of a variable or a variable list object.

14.3.5.3.4.2 Service primitives

The service parameters for this service are shown in Table 601.

Table 601 – Information Report service parameters

Parameter name	Req	Ind
Argument	M	M(=)
VCR	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Duplicated APDU Body		C
Variable Index	M	M(=)
Variable Subindex	U	U(=)
Data	U	U(=)
On Change	U	U(=)

Argument

This parameter carries the parameters of the service invocation.

Destination DL-Address

This parameter exists only when type of the corresponding VCR is QUU and the remote partner has FREE endpoint. It gives remote address to which the requested Data should be sent.

Source DL-Address

This parameter exists only when type of the corresponding VCR is QUU and has FREE endpoint. It indicates address from which the indicated data was sent.

Duplicated APDU Body

This parameter exists only when FAS provides it for BNU VCR and indicates whether or not the data parameter has been received and delivered in a previous service indication.

Variable Index

This parameter is the OD index of the variable or variable list.

Variable Subindex

This optional parameter identifies an individual element in an array or record variable by its position within the variable, starting at 1.

Data

This parameter contains the value of the variable or variable list.

On Change

This optional Boolean parameter indicates, when TRUE, that the variable value is being reported because it has changed, or because it has not changed within an externally specified time period.

14.3.5.3.5 Define variable list service

A Variable List Object is created at the communication partner using this service. The Number Of Elements attribute is calculated automatically by the server for the Define Variable List service. The client user shall ensure that the data of the Variable List Service may be transmitted in one single APDU. The Deletable attribute of a Variable List which is created by the Define Variable List service is set to the value TRUE.

14.3.5.3.5.1 Service overview

This service creates a variable list object.

14.3.5.3.5.2 Service primitives

The service parameters for this service are shown in Table 602.

Table 602 – Define Variable List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Password	M	M(=)		
Access Groups	M	M(=)		
Access Rights	M	M(=)		
List of Variable Indexes	M	M(=)		
Extension	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
OD Index			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Password

This parameter states to which value the Password attribute is to be set.

Access Groups

This parameter states to which value the attribute Access Groups is to be set.

Access Rights

This parameter states to which value the attribute Access Rights is to be set.

List of Variable Indexes

This parameter contains the OD indexes of the variables which are combined in this Variable List.

Extension

This attribute contains profile specific information to be set.

Result(+)

The Result(+) parameter shall indicate that the service was completed successfully.

OD Index

OD Index of the newly created Variable List object.

Result(-)

The Result(-) shall indicate that the service request was not completed successfully.

14.3.5.3.5.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.5.3.6 Delete variable list service

14.3.5.3.6.1 Service overview

This service deletes a variable list object.

A Variable List Object is deleted with this service if there is an Access Right for this Object. Only Variable List objects having the value true for the Deletable attribute may be deleted.

14.3.5.3.6.2 Service primitives

The service parameters for this service are shown in Table 603.

Table 603 – Delete Variable List service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
OD Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

OD Index

This parameter identifies the variable list by its OD Index.

Result(+)

The Result(+) parameter shall indicate that the service was completed successfully.

Result(-)

The Result(-) shall indicate that the service request was not completed successfully.

14.3.5.3.6.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.6 Event ASE

14.3.6.1 Overview

The Event serves for sending an important message from one device to another (or in multicast mode to selected devices) as shown in Figure 161. The detection of the conditions that cause an event is responsibility of the user. The application program invokes the Event Notification service when the conditions are met. The monitoring and checking of the (optional) acknowledgment is also the responsibility of the user.

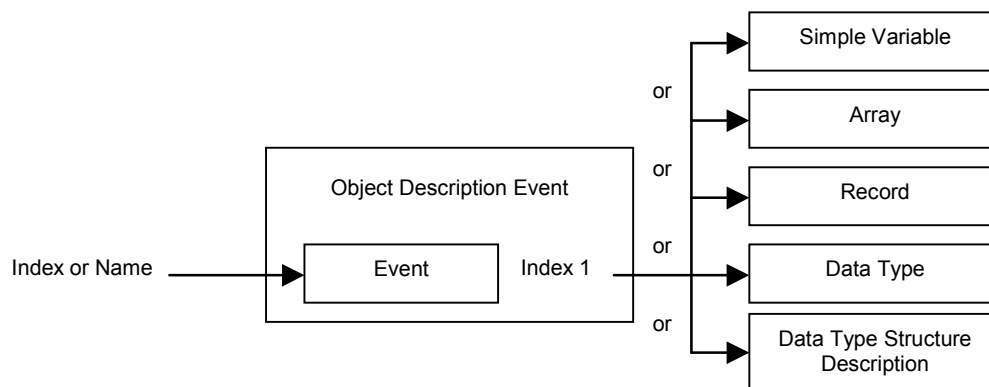


Figure 161 – Overview of event

A user sends an event notification with the EventNotification service.

The user transmits with the EventNotification service a counter value (Event Number) and data (Event Data), e.g. measurement value, status, units.

The management (increment) of the counter value is the responsibility of the user. The Event Notification may represent a collective alarm, where the data may contain information about which channel has triggered the collective alarm. Also in this case the logical operations for the triggering conditions and for the data are part of the application program.

The receiver of the Event Notification may acknowledge the Event-Notification by using the AcknowledgeEventNotification service.

The counter value is transmitted with the AcknowledgeEventNotification service. The counter value serves to reliably correlate the EventNotification and the AcknowledgeEventNotification. The monitoring and the checking of the acknowledgment is the responsibility of the user.

The receiver of the EventNotification may lock or unlock the EventNotification using the AlterEventConditionMonitoring service.

14.3.6.2 Event model class specifications

14.3.6.2.1 Class overview

This subclause provides formal class definitions for event objects. Each class definition specifies a type of event class as a combination of its attributes and the services defined for them.

14.3.6.2.2 Formal model

ASE: **EVENT ASE**

CLASS: **EVENT**

CLASS ID: **4**

PARENT CLASS: **TOP**

ATTRIBUTES:

- | | | | |
|---|-----|------------|---------------------|
| 1 | (m) | Attribute: | Numeric Identifier |
| 2 | (m) | Attribute: | Index of Event Data |
| 3 | (m) | Attribute: | Length |
| 4 | (m) | Attribute: | Password |
| 5 | (m) | Attribute: | Access Groups |
| 6 | (m) | Attribute: | Access Rights |
| 7 | (m) | Attribute: | Enabled |
| 8 | (m) | Attribute: | Extension |

SERVICES:

- | | | | |
|---|-----|------------|----------------------------------|
| 1 | (m) | OpService: | Event Notification |
| 2 | (m) | OpService: | Acknowledge Event Notification |
| 3 | (m) | OpService: | Alter Event Condition Monitoring |

14.3.6.2.3 Attributes

Numeric Identifier

Identifies an instance of this object class.

Index of Event Data

This attribute is the OD Index for event data associated with this event object. It may be the OD Index of a variable object or the OD index of the data type of the event data.

Length

If the Index Event Data points to a Data Type then this attribute contains the length in octets of the Event Data. This attribute is not significant otherwise.

Password

This attribute contains the password for the access rights. Its value is null if it is not used.

Access Groups

This attribute relates the object to specific Access Groups as shown in Table 604. The object is a group member when the corresponding bit is set.

Table 604 – Event access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute contains information about the Access Rights as shown in Table 605. The specific Access Right exists when the corresponding bit is set.

Table 605 – Event access rights membership

Bit	Name	Meaning
6	W	Right to Acknowledge for the registered Password
5	D	Right to Enable/Disable for the registered Password
2	Wg	Right to Acknowledge for Access Groups
1	Dg	Right to Enable/Disable for Access Groups
14	Wa	Right to Acknowledge for all Communication Partners
13	Da	Right to Enable/Disable for all Communication Partners

Enabled

This Boolean attribute indicates, when TRUE, that the event is enabled (unlocked).

Extension

This attribute contains profile specific information.

14.3.6.2.4 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be selected.

Event Notification

This service is provided to permit notifier object to report the occurrence of one or more events in a single service invocation.

Acknowledge Event Notification

This optional confirmed service is provided to permit a subscriber of an event to acknowledge receipt of an event message.

Enable Event Condition Monitoring

This optional service is provided to permit event reporting to be enabled for an event.

14.3.6.3 Event ASE service specifications**14.3.6.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Event Notification

Acknowledge Event

Enable Event Condition Monitoring

14.3.6.3.2 Event notification**14.3.6.3.2.1 Service overview**

This unconfirmed service is used to send an Event Notification, indicating that an event has been detected. This service is unconfirmed.

14.3.6.3.2.2 Service primitives

The service parameters for this service are shown in Table 606.

Table 606 – Event Notification service parameters

Parameter name	Req	Ind
Argument	M	M(=)
VCR	M	M(=)
Destination DL-Address	C	
Source DL-Address		C
Event Index	M	M(=)
Event Number	M	M(=)
Event Data	M	M(=)

Argument

The argument contains the parameters of the service request.

Destination DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the destination address to which the requested Event Notification is to be sent.

Source DL-Address

This parameter exists only when the corresponding AREP supports it and is configured with a Remote Address Configuration Type of FREE. It indicates the source address from which the indicated Event Notification is to be sent.

Event Index

This parameter identifies the event being reported by this service using its OD index.

Event Number

This parameter reports the event count for the occurrence being reported.

Event Data

This parameter contains the data. The data of the object shall be mapped onto the parameter data according to the description of the Data Types. The Index is stored as Index Event Data in the Object Description of the Event Object.

14.3.6.3.2.3 Service procedure

The Unconfirmed Service Procedure specified in Clause 4 applies to this service.

14.3.6.3.3 Acknowledge event notification

14.3.6.3.3.1 Service overview

This service allows the acknowledgment of a reported event.

14.3.6.3.3.2 Service primitives

The service parameters for this service are shown on Table 607.

Table 607 – Acknowledge Event Notification service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Event Index	M	M(=)		
Event Number	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the parameters of the service request.

Event Index

This parameter identifies the event object that is being acknowledged by its OD Index.

Event Number

This parameter reports the event count for the occurrence being reported.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.6.3.3.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

The association between the event identified in this service and the corresponding event object in the receiving AP is performed by the user.

14.3.6.3.4 Alter event condition monitoring

14.3.6.3.4.1 Service overview

This service allows the enabling or disabling of the Event Object.

14.3.6.3.4.2 Service primitives

The service parameters for this service are shown in Table 608.

Table 608 – Alter Event Condition Monitoring service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Event Index	M	M(=)		
Enabled	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Event Index

This parameter identifies the event object by its OD index.

Enabled

This Boolean parameter, when TRUE, indicates that the event object is to be enabled, and when FALSE, that the event object is to be disabled. It is used to set or clear the attribute value of the same name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.6.3.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.6.4 Event notification state machine

14.3.6.4.1 State machine description

14.3.6.4.1.1 States

The states are shown in Figure 162.

LOCKED

No Event Notifications are send in the LOCKED state (Enabled = false).

UNLOCKED

Event Notifications are sent normally in the UNLOCKED state (Enabled = true).

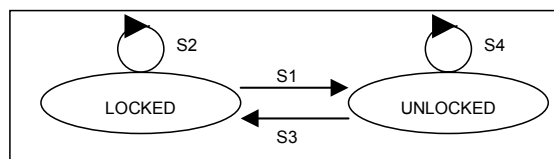


Figure 162 – Event state machine

14.3.6.4.1.2 State transitions

Table 609 specifies the state transitions related to events.

Table 609 – Event state transitions

#	Current State	Events	Actions	Next State
S1	LOCKED	AlterEventConditionMonitoring.ind && Enabled = true	AlterEventConditionMonitoring.rsp(+){ }	UNLOCKED
S2	LOCKED	AlterEventConditionMonitoring.ind && Enabled = false	AlterEventConditionMonitoring.rsp(+){ }	LOCKED
S3	UNLOCKED	AlterEventConditionMonitoring.ind && Enabled = false	AlterEventConditionMonitoring.rsp(+){ }	LOCKED
S4	UNLOCKED	AlterEventConditionMonitoring.ind && Enabled = true	AlterEventConditionMonitoring.rsp(+){ }	UNLOCKED

14.3.7 Domain ASE

14.3.7.1 Overview

The Domain ASE is an extended subset of the Type 1 Load Region ASE. Extensions incorporate a new class, new services, additional class attributes, and in some cases, restrictions on attribute and parameter values.

The classes contain a subset of the Type 1 attributes and services, and the services contain a subset of the Type 1 parameters.

Therefore, except for the extensions, only the class templates and service tables are shown to reflect the selected subsets. See Type 1 for the definitions of the classes, attributes, and services. The Type 1 attribute and service numbers have been retained in the class template below.

14.3.7.2 Domain model specification

14.3.7.2.1 Formal model

ASE:	LOAD REGION ASE
CLASS:	LOAD REGION
CLASS ID:	2
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Numeric Identifier
2 (m) Attribute:	Max Octets
3 (m) Attribute:	Password
4 (m) Attribute:	Access Groups
5 (m) Attribute:	Access Rights
6 (m) Attribute:	Local Address
7 (m) Attribute:	Domain State
8 (m) Attribute:	Upload State
9 (m) Attribute:	Counter

10 (m) Attribute: Extension

SERVICES:

- 1 (m) Ops Service: Generic Download Services
- 2 (m) Ops Service: Download Services
- 3 (m) Ops Service: Upload Services

14.3.7.2.2 Attributes

Numeric Identifier

Identifies an instance of this object class.

Max Octets

This attribute specifies the max. number of octets of the Domain.

Password

This attribute shall specify the password for the access rights.

Access Groups

This attribute shall specify the object's membership in specific access groups as shown in Table 610. The object is member of an access group if the corresponding bit is set.

Table 610 – Domain access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute shall specify the rights to access the object as shown in Table 611. The respective access is allowed if the corresponding bit is set.

Table 611 – Domain access rights membership

Bit	Name	Meaning
7	R	Right to Read the registered Password
6	W	Right to Write the registered Password
5	U	Right to Use in a PI for the registered Password
3	Rg	Right to Read for Access Groups
2	Wg	Right to Write for Access Groups
1	Ug	Right to Use in a PI for Access Groups
15	Ra	Right to Read for all Communication Partners
14	Wa	Right to Write for all Communication Partners
13	Ua	Right to Use in a PI for all Communication Partners

Local Address

This attribute is a system specific reference to the real object. It may be used internally for addressing the object. If a reference of this kind is not applied, the Local Address attribute shall have the value FFFFFFFF hex.

Domain State

This attribute specifies the state of the Domain Object. The values of this attribute are:

- 1 <=> EXISTENT
- 2 <=> LOADING
- 3 <=> INCOMPLETE
- 4 <=> COMPLETE
- 5 <=> READY
- 6 <=> IN-USE

Upload State

This attribute shall specify the state of the State Machine for Upload of the Domain.

- 0 <=> NON-EXISTENT
- 1 <=> UPLOADING
- 2 <=> UPLOADED

Counter

This attribute shall specify the number of Program Invocations which currently use this Domain. The Program Invocation Management maintains this Counter, while the Domain Management only interprets the Counter. If the Counter attribute has a value greater than 0, the Domain is in use and may not be overwritten with a Download service.

Extension

This attribute contains profile specific information.

14.3.7.2.3 Services**Generic Download Services**

This service set provides for the “push” download of a domain.

Download Services

This service set provides for the “pull” download of a domain.

Upload Services

This service set provides for the “pull” upload of a domain.

14.3.7.3 Domain ASE service specifications**14.3.7.3.1 Supported services**

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

- Generic Initiate Download Sequence
- Generic Download Segment
- Generic Terminate Download Sequence
- Initiate Download Sequence
- Download Segment
- Terminate Download Sequence
- Request Domain Download
- Initiate Upload Sequence
- Upload Segment
- Terminate Upload Sequence

14.3.7.3.2 Request Domain UploadGeneric download services

14.3.7.3.2.1 Overview

The Generic Download Services shall be used to load data from the client into the server's Domain using the “push” model.

14.3.7.3.2.2 GenericInitiateDownloadSequence

14.3.7.3.2.2.1 Service overview

The GenericInitiateDownloadSequence service shall be used to begin the loading of the Domain whose Index or Domain Name is included in the service request.

14.3.7.3.2.2.2 Service primitives

The primitives and parameters of this service are shown in Table 612 and the accompanying text.

Table 612 – GenericInitiateDownloadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.2.3 GenericDownloadSegment

14.3.7.3.2.3.1 Service overview

The GenericDownloadSegment service shall be used to transfer one data segment into the server's Domain. The segment is transmitted in the service request.

14.3.7.3.2.3.2 Service primitives

The primitives and parameters of this service are shown in Table 613 and the accompanying text.

Table 613 – GenericDownloadSegment

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Load Data	M	M(=)		
More Follows	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Load Data

This parameter shall contain the data to be downloaded.

More Follows

This parameter shall indicate whether or not any additional data remains to be transmitted.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.2.4 GenericTerminateDownloadSequence

14.3.7.3.2.4.1 Service overview

The GenericTerminateDownloadSequence service shall be used to finish the loading of the Domain whose Index or Domain Name is included in the service request.

14.3.7.3.2.4.2 Service primitives

The primitives and parameters of this service are shown in Table 614 and the accompanying text.

Table 614 – GenericTerminateDownloadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Final Result			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Final Result

This parameter shall inform the client whether or not the server successfully finished the Download.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.2.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.7.3.3 Download services

14.3.7.3.3.1 Overview

The Download Services shall be used to load data from the client into the server's Domain.

14.3.7.3.3.2 InitiateDownloadSequence

14.3.7.3.3.2.1 Service overview

The InitiateDownloadSequence service shall be used to begin the loading of the Domain whose Index or Domain Name is included in the service request.

14.3.7.3.3.2.2 Service primitives

The primitives and parameters of this service are shown in Table 615 and the accompanying text.

Table 615 – InitiateDownloadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.3.3 DownloadSegment**14.3.7.3.3.3.1 Service overview**

The DownloadSegment service shall be used to transfer one data segment into the server's Domain. The segment is transmitted in the service response.

14.3.7.3.3.3.2 Service primitives

The primitives and parameters of this service are shown in Table 616 and the accompanying text.

Table 616 – DownloadSegment

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Load Data	M	M(=)		
More Follows	M	M(=)		
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Load Data

This parameter shall contain the data to be downloaded.

More Follows

This parameter shall indicate whether or not any additional data remains to be transmitted.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.3.4 TerminateDownloadSequence

14.3.7.3.3.4.1 Service overview

The TerminateDownloadSequence service shall be used to finish the loading of the Domain whose Index or Domain Name is included in the service request.

14.3.7.3.3.4.2 Service primitives

The primitives and parameters of this service are shown in Table 617 and the accompanying text.

Table 617 – TerminateDownloadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Final Result			M	M(=)
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Final Result

This parameter shall inform the client whether or not the server successfully finished the Download.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.3.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.7.3.3.5 RequestDomainDownload**14.3.7.3.3.5.1 Service overview**

The RequestDomainDownload service shall be used by a server to request the client to perform a Download into the Domain whose Index or Domain Name is included in the service request.

The service response with Result(+) is not sent until the Download Sequence has been completely finished.

14.3.7.3.3.5.2 Service primitives

The primitives and parameters of this service are shown in Table 618 and the accompanying text.

Table 618 – RequestDomainDownload

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Additional Information			M	M(=)
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Additional Information

This parameter shall contain additional information on the Domain. It may for example contain a file name. The correlation between Domain and file name is a local matter and is the responsibility of the application.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.3.5.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.7.3.4 Upload services

14.3.7.3.4.1 Overview

The Upload Services shall be used to transmit the data from the server's Domain to the client.

14.3.7.3.4.2 InitiateUploadSequence

14.3.7.3.4.2.1 Service overview

The InitiateUploadSequence service shall be used to begin the Upload of the Domain whose Index or Domain Name is included in the service request.

14.3.7.3.4.2.2 Service primitives

The primitives and parameters of this service are shown in Table 619 and the accompanying text.

Table 619 – InitiateUploadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.4.3 UploadSegment**14.3.7.3.4.3.1 Service overview**

The UploadSegment service shall be used to transfer one data segment of the server's Domain to the client.

14.3.7.3.4.3.2 Service primitives

The primitives and parameters of this service are shown in Table 620 and the accompanying text.

Table 620 – UploadSegment

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Load Data	M	M(=)		
More Follows	M	M(=)		
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Load Data

This parameter shall contain the data to be uploaded.

More Follows

This parameter shall indicate whether or not any additional data remains to be transmitted.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.4.4 TerminateUploadSequence

14.3.7.3.4.4.1 Service overview

The TerminateUploadSequence service shall be used to finish the Upload Sequence.

14.3.7.3.4.4.2 Service primitives

The primitives and parameters of this service are shown in Table 621 and the accompanying text.

Table 621 – TerminateUploadSequence

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.4.4.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.7.3.4.5 RequestDomainUpload**14.3.7.3.4.5.1 Service overview**

The RequestDomainUpload service shall be used by a server to request that the contents of the specified Domain be uploaded to the client.

Remark: The service response with Result(+) is not sent until the Upload Sequence has been completely finished.

14.3.7.3.4.5.2 Service primitives

The primitives and parameters of this service are shown in Table 622 and the accompanying text.

Table 622 – RequestDomainUpload

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Domain Index	M	M(=)		
Additional Information			M	M(=)
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

The argument contains the service specific parameters of the service request.

Domain Index

This parameter is the OD index of the domain object.

Additional Information

This parameter shall contain additional information on the Domain. It may for example contain a file name. The correlation between Domain and file name is a local matter and is the responsibility of the application.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.7.3.4.5.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.7.4 State machine description

14.3.7.4.1 State machine for genericDownload/download

14.3.7.4.1.1 States

The states are shown in Figure 163.

EXISTENT

The Domain exists, but the content is not defined.

LOADING

GenericDownload/Download on the Domain is in progress.

INCOMPLETE

Download failed, the content is incomplete.

COMPLETE

The Domain content has been transmitted, but the Domain is not yet released for use.

READY

The Domain is released for use.

IN-USE

The Domain is being used by a Program Invocation. GenericDownload/Download is not allowed in this state.

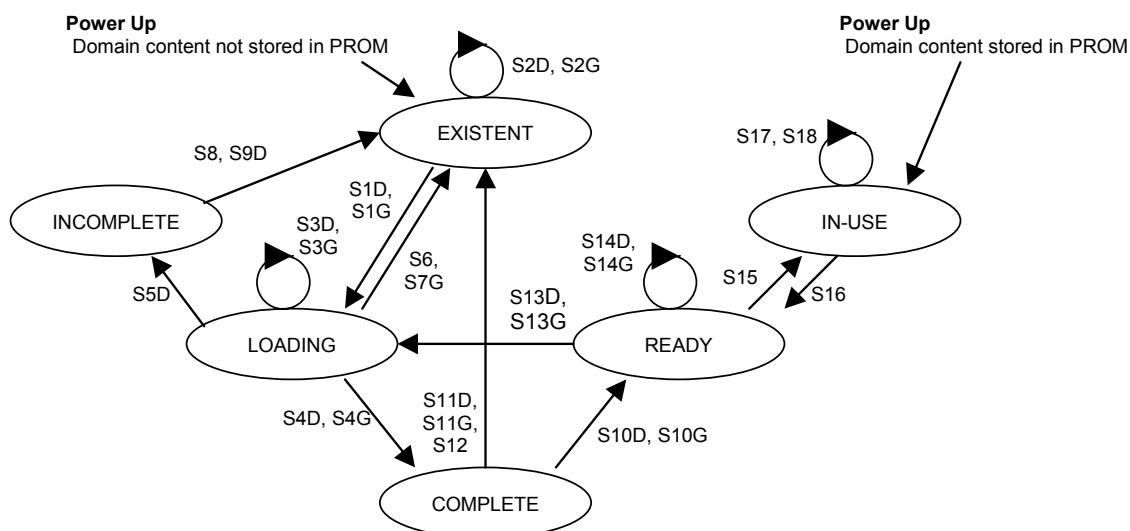


Figure 163 – Domain GenericDownload/Download state machine (server)

GenericDownload, Download and Upload may not operate on the same Domain at the same time.

State transition numbers with “G” indicate they are transitions for Domain GenericDownload, while transition numbers with “D” indicate they are transitions for Download.

14.3.7.4.1.2 State transitions

Table 623 specifies the server-side state transitions related to download.

Table 623 – Domain GenericDownload/Download state machine (server)

#	Current State	Events	Actions	Next State
S1D	EXISTENT	InitiateDownloadSequence.ind && Upload state = NON-EXISTENT	InitiateDownloadSequence.rsp(+){ }, DownloadSegment.req{ }	LOADING
S1G	EXISTENT	GenericInitiateDownloadSequence.ind && Upload state = NON-EXISTENT	GenericInitiateDownloadSequence.rsp(+) { }	LOADING
S2D	EXISTENT	InitiateDownloadSequence.ind && Upload state <> NON-EXISTENT	InitiateDownloadSequence.rsp(-){ ErrorType := object-constraint- conflict }	EXISTENT
S2G	EXISTENT	GenericInitiateDownloadSequence.ind && Upload state <> NON-EXISTENT	GenericInitiateDownloadSequence.rsp(-) { ErrorType := object-constraint- conflict }	EXISTENT

#	Current State	Events	Actions	Next State
S3D	LOADING	DownloadSegment.cnf(+) && more follows = true	store received data, DownloadSegment.req{ }	LOADING
S3G	LOADING	GenericDownloadSegment.ind && more follows = true && GenericDownloadSegment successfully executed	GenericDownloadSegment.rsp(+){ }	LOADING
S4D	LOADING	DownloadSegment.cnf(+) && more follows = false	store received data, TerminateDownloadSequence.req{ Final Result := true }	COMPLETE
S4G	LOADING	GenericDownloadSegment.ind && more follows = false && GenericDownloadSegment successfully executed	GenericDownloadSegment.rsp(+){ }	COMPLETE
S5D	LOADING	DownloadSegment.cnf(-)	TerminateDownloadSequence.req{ Final Result := false }	INCOMPLETE
S6	LOADING	Abort.ind	(no actions taken)	EXISTENT
S7G	LOADING	GenericDownloadSegment.ind && more follows = true/false && GenericDownloadSegment failed	GenericDownloadSegment.rsp(-){ ErrorType := reason of failure }	EXISTENT
S8	INCOMPLETE	Abort.ind	(no actions taken)	EXISTENT
S9D	INCOMPLETE	TerminateDownloadSequence.cnf(+/-)	(no actions taken)	EXISTENT
S10D	COMPLETE	TerminateDownloadSequence.cnf(+)	(no actions taken)	READY
S10G	COMPLETE	GenericTerminateDownloadSequence. ind && GenericDownloadSequence successfully executed	GenericTerminateDownloadSequence. rsp(+){ Final Result := true }	READY
S11D	COMPLETE	TerminateDownloadSequence.cnf(-)	(no actions taken)	EXISTENT
S11G	COMPLETE	GenericTerminateDownloadSequence. ind && GenericTerminateDownload Sequence failed	GenericTerminateDownloadSequence. rsp(-){ ErrorType := reason of failure }	EXISTENT
S12	COMPLETE	Abort.ind	(no actions taken)	EXISTENT
S13D	READY	InitiateDownloadSequence.ind && Upload state = NON-EXISTENT	InitiateDownloadSequence.rsp(+){ }, DownloadSegment.req{ }	LOADING
S13G	READY	GenericInitiateDownloadSequence.ind && Upload state = NON-EXISTENT	GenericInitiateDownloadSequence. rsp(+){ }	LOADING
S14D	READY	InitiateDownloadSequence.ind && Upload state <> NON-EXISTENT	InitiateDownloadSequence.rsp(-){ ErrorType := object-constraint- conflict }	READY
S14G	READY	GenericInitiateDownloadSequence.ind && Upload state <> NON-EXISTENT	GenericInitiateDownloadSequence. rsp(-){ ErrorType := object-constraint- conflict }	READY
S15	READY	IncrementDomainCounter()	counter := 1	IN-USE
S16	IN-USE	DecrementDomainCounter() && counter = 1	counter := 0	READY
S17	IN-USE	IncrementDomainCounter()	counter := counter + 1	IN-USE
S18	IN-USE	DecrementDomainCounter() && counter > 1	counter := counter - 1	IN-USE
NOTE IncrementDomainDounter() and DecrementDomainCounter() are actions in PI state machine.				

14.3.7.4.2 State machine for upload

14.3.7.4.2.1 States

The states are shown in Figure 164.

NON-EXISTENT

The Domain exists, no Upload in progress.

UPLOADING

Upload on the Domain is in progress, Download is not allowed.

UPLOADED

The Domain content has been completely transmitted, but the TerminateUploadSequence service is still to be executed.

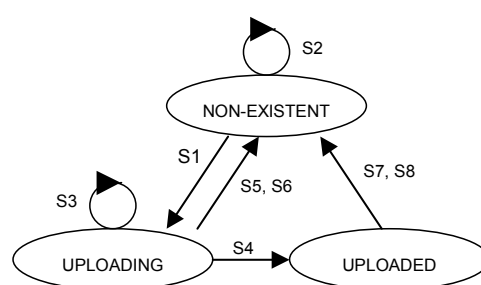


Figure 164 – Domain Upload state machine (server)

Download, Upload and GenericDownload may not operate on the same Domain at the same time.

14.3.7.4.2.2 State transitions

Table 624 specifies the server-side state transitions related to upload.

Table 624 – Domain Upload state machine (server)

#	Current State	Events	Actions	Next State
S1	NON-EXISTENT	InitiateUploadSequence.ind && Download state machine of this domain is in one state of {EXISTENT, READY, IN-USE}	InitiateUploadSequence.rsp(+){ }	UPLOADING
S2	NON-EXISTENT	InitiateUploadSequence.ind && Download state machine of this domain is in one state of {LOADING, INCOMPLETE, COMPLETE}	InitiateUploadSequence.rsp(-){ ErrorType := object -constraint-conflict }	NON-EXISTENT
S3	UPLOADING	UploadSegment.ind && more follows = true	UploadSegment.rsp(+){ more follows := true }	UPLOADING
S4	UPLOADING	UploadSegment.ind && more follows = false	UploadSegment.rsp(+){ more follows := false }	UPLOADED
S5	UPLOADING	TerminateUploadSequence.ind	TerminateUploadSequence.rsp(-){ ErrorType := object-state-conflict }	NON-EXISTENT
S6	UPLOADING	Abort.ind	(no actions taken)	NON-EXISTENT
S7	UPLOADED	TerminateUploadSequence.ind	TerminateUploadSequence.rsp(+){ }	NON-EXISTENT
S8	UPLOADED	Abort.ind	(no actions taken)	NON-EXISTENT

14.3.8 Program invocation ASE

14.3.8.1 Overview

The Program Invocation ASE is an extended subset of the Type 1 Function Invocation ASE. Extensions incorporate a new class, new services, additional class attributes, and in some cases, restrictions on attribute and parameter values.

The classes contain a subset of the Type 1 attributes and services, and the services contain a subset of the Type 1 parameters.

Therefore, except for the extensions, only the class templates and service tables are shown to reflect the selected subsets. See Type 1 for the definitions of the classes, attributes, and services. The Type 1 attribute and service numbers have been retained in the class template below.

14.3.8.2 Program invocation model class specifications

14.3.8.2.1 Program invocation class specification

14.3.8.2.1.1 Formal model

ASE:	PROGRAM INVOCATION ASE
CLASS:	PROGRAM INVOCATION
CLASS ID:	3
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Numeric Identifier
2 (m) Attribute:	Number of Domains
3 (m) Attribute:	Password
4 (m) Attribute:	Access Groups
5 (m) Attribute:	Access Rights
6 (m) Attribute:	Deletable (TRUE,FALSE)
7 (m) Attribute:	Reusable (TRUE,FALSE) Default : TRUE
8 (m) Attribute:	Program Invocation State
9 (m) Attribute:	List of Domain Index
9.1 (m) Attribute:	Domain Index
10 (m) Attribute:	Extension
SERVICES:	
1 (m) OpsService:	Start
2 (m) OpsService:	Stop
3 (m) OpsService:	Resume
4 (m) OpsService:	Reset
5 (m) OpsService:	Kill
6 (m) OpsService:	Create Program Invocation
7 (m) OpsService:	Delete Program Invocation

14.3.8.2.1.2 Attributes

Numeric Identifier

Identifies an instance of this object class.

Number of Domains

Number of the domain indices entered in this Program Invocation. The number is limited by the maximum PDU length of the Create Program Invocation service.

Password

This attribute contains the password for the Access Rights.

Access Groups

This attribute relates the object to specific Access Groups as shown in Table 625. The object is a group member when the corresponding bit is set.

Table 625 – Program invocation access group membership

Bit	Meaning
7	Access Group 1
6	Access Group 2
5	Access Group 3
4	Access Group 4
3	Access Group 5
2	Access Group 6
1	Access Group 7
0	Access Group 8

Access Rights

This attribute contains information about the Access Rights as shown in Table 626. The specific Access Right exists when the corresponding bit is set.

Table 626 – Program invocation access group membership

Bit	Name	Meaning
7	S	Right to Start the PI for the registered Password (Start, Resume, Reset)
6	H	Right to Stop the PI for the registered Password (Stop)
5	D	Right to Delete the PI for the registered Password (Kill, Delete Program Invocation)
3	Sg	Right to Start for Access Groups
2	Hg	Right to Stop for Access Groups
1	Dg	Right to Delete for Access Groups
15	Sa	Right to Start for all Communication Partners
14	Ha	Right to Stop for all Communication Partners
13	Da	Right to Delete for all Communication Partners

Deletable

This attribute indicates, when TRUE, that the Program Invocation can be deleted using the Delete Program Invocation service. The value of this attribute is always TRUE for objects dynamically created with the Create Program Invocation Service.

Reusable

This attribute indicates if the Program Invocation transits after execution to the state IDLE or to the state UNRUNNABLE.

true <=> transits to the state IDLE

false <=> transits to the state UNRUNNABLE.

PI State

This attribute describes the state of the program.

1 ⇔ UNRUNNABLE

2 ⇔ RUNNING

3 ⇔ STARTING

4 ⇔ RESUMING

5 ⇔ IDLE

6 ⇔ STOPPED

7 ⇔ STOPPING

8 ⇔ RESETTING

List of Domain Index

This attribute contains the indices of the domains, which are combined to this Program Invocation. The ordering of the entries of the domain indices in the Program Invocation corresponds to the ordering in the Create Program Invocation service.

The first domain in the list of the indices shall contain an executable program.

Domain Index

This attribute is the OD Index for the domain.

Extension

This attribute contains profile specific information.

14.3.8.2.1.3 Services

Create Program Invocation

This service is used to create a program invocation and relate it to one or more load regions.

Delete Program Invocation

This service is used to delete a program invocation.

Start

This service is used to start a program invocation.

Stop

This service is used to stop (suspend) a program invocation.

Resume

This service is used to resume a stopped (suspended) a program invocation.

Reset

This service is used to terminate a stopped (suspended) program invocation and prepare it to be started again.

Kill

This service is used to abort a program invocation without preparing it to be restarted.

14.3.8.3 Program invocation model service specifications

14.3.8.3.1 Create program invocation service

14.3.8.3.1.1 Service overview

With this service, domains (e.g. programs, data regions), that have been defined in the OD, are combined to a program and are defined on-line in the DP-OD. This program is addressed by a logical address or a name. The Deletable attribute of a Program Invocation, that is created by the Create Program Invocation service, is set to true. The attribute Number of Domains in the object description is created automatically by the server.

The first domain of the list of domains shall contain an executable program.

The Program Invocation is only created, if the rights for the use of the domains exist. Otherwise a Result(-) is responded.

If the same Program Invocation with the same Access Rights already exists, no new object is created. In this case, the logical address (Index) of the existing object is given to the client.

14.3.8.3.1.2 Service primitives

The service parameters for this service are shown in Table 627.

Table 627 – Create Program Invocation service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Password	M	M(=)		
Access Groups	M	M(=)		
Access Rights	M	M(=)		
Reusable	M	M(=)		
List of Domains	M	M(=)		
Domain Index	M	M(=)		
Program Invocation Name	U	U(=)		
Extension	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
OD Index			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Password

This parameter states to which value the Password attribute is to be set.

Access Groups

This parameter states to which value the attribute Access Groups is to be set.

Access Rights

This parameter states to which value the attribute Access Rights is to be set.

Reusable

This parameter states to which value the Reusable attribute is to be set.

List of Domains

This parameter contains the OD Indexes of the Domains that contain the code and data for this Program Invocation.

Domain Index

This parameter is the OD Index of the domain object.

Program Invocation Name

This parameter contains the name of the program invocation. It is unused.

Extension

This parameter contains the Extension attribute.

Result(+)

The Result(+) parameter shall indicate that the service was completed successfully.

OD Index

OD Index of the newly created Program Invocation object.

Result(-)

The Result(-) shall indicate that the service request was not completed successfully.

14.3.8.3.1.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.8.3.2 Delete program invocation service

14.3.8.3.2.1 Service overview

With the Delete Program Invocation service programs that have been defined in the DP-OD are deleted. Program Invocation Objects may only be deleted if their attribute Deletable has the value true.

14.3.8.3.2.2 Service primitives

The service parameters for this service are shown in Table 628.

Table 628 – Delete Program Invocation service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be deleted.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.8.3.2.3 Service procedure

The Confirmed Service Procedure specified in Clause 4 applies to this service.

14.3.8.3.3 Start service**14.3.8.3.3.1 Service overview**

A program invocation is started with this service. It runs from the beginning. The counters of the used domains are incremented. Before that all corresponding domains are checked if they are in the READY or in the IN USE state.

14.3.8.3.3.2 Service primitives

The service parameters for this service are shown in Table 629.

Table 629 – Start service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)
Program Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be started.

Execution Argument

This optional parameter of type octet string contains data that is given to the Program Invocation for the start.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

PI State

This parameter indicates the state of the Program Invocation.

14.3.8.3.4 Stop service

14.3.8.3.4.1 Service overview

A running program invocation is stopped (suspended) so that it can be resumed. It is not set to the beginning. The counters of the used domains are decremented.

14.3.8.3.4.2 Service primitives

The service parameters for this service are shown in Table 630.

Table 630 – Stop service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)
Program Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be stopped.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

PI State

This parameter indicates the state of the Program Invocation.

14.3.8.3.5 Resume service

14.3.8.3.5.1 Service overview

This confirmed service is used to resume the execution of a Program Invocation that has been stopped. Execution is resumed using the context saved when the Program Invocation was stopped, as follows.

The stopped program invocation is set to the state RUNNING. It is not reset. The counters of the used domains are incremented. Before that all corresponding domains are checked if they are in the READY or in the IN USE state.

14.3.8.3.5.2 Service primitives

The service parameters for this service are shown in Table 631.

Table 631 – Resume service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Execution Argument	U	U(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)
Program Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be resumed.

Execution Argument

This optional parameter of type octet string contains data that is given to the Program Invocation for the resume.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

PI State

This parameter indicates the state of the Program Invocation.

14.3.8.3.6 Reset service

14.3.8.3.6.1 Service overview

This confirmed service is used to reset a stopped Program Invocation with its initial context. Its initial context is defined as the context of the Program Invocation set by its initialization procedures, as follows.

A stopped program having the attribute Reusable = true is set to the beginning. The Program Invocation transits to the state IDLE. If the attribute Reusable is equal to false the Program Invocation Object transits to the state UNRUNNABLE.

14.3.8.3.6.2 Service primitives

The service parameters for this service are shown in Table 632.

Table 632 – Reset service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)
Program Invocation State			C	C(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be reset.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

PI State

This parameter indicates the state of the Program Invocation.

14.3.8.3.7 Kill service

14.3.8.3.7.1 Service overview

This confirmed service is used to abort the execution of a Program Invocation so that it may not be restarted or resumed, as follows.

A Program Invocation is set to the state UNRUNNABLE. This is independent of the current state. If the state of the Program Invocation was RUNNING or STOPPING, the counters of the used domains are decremented.

14.3.8.3.7.2 Service primitives

The service parameters for this service are shown in Table 633.

Table 633 – Kill service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument	M	M(=)		
VCR	M	M(=)		
Invoke ID	M	M(=)		
Program Invocation Index	M	M(=)		
Result (+)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Result (-)			S	S(=)
VCR			M	M(=)
Invoke ID			M	M(=)
Error Type			M	M(=)

Argument

This parameter carries the parameters of the service invocation.

Program Invocation Index

This parameter specifies the OD Index of the Program Invocation object to be killed.

Execution Argument

This optional parameter of type octet string contains data that is given to the Program Invocation for the resume.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

14.3.8.4 Program invocation state machine

14.3.8.4.1 States

The states are shown in Figure 165.

NON-EXISTENT

The Program Invocation is after a power-up, if not otherwise defined, or after deletion, non-existent.

IDLE

A Program Invocation transits to the state IDLE by entering it in the DP-OD. After a successful Reset service and after the end of the program the Program Invocation transits also to the state IDLE. Program Invocations may be predefined. These Program Invocations transit after power-up to the state IDLE.

STARTING

Intermediate state for the preparation of the program start.

RUNNING

The program is running in this state. The domains that are defined in this Program Invocation are now in the state IN USE.

STOPPING

Intermediate state for the preparation of the stop of the program.

STOPPED

The program is stopped in this state.

RESUMING

Intermediate state for the preparation of the resumption of the program.

RESETTING

Intermediate state in which the program is set to a well defined initial state.

UNRUNNABLE

In this state the program is stopped but it may not be started again. It may only be deleted.

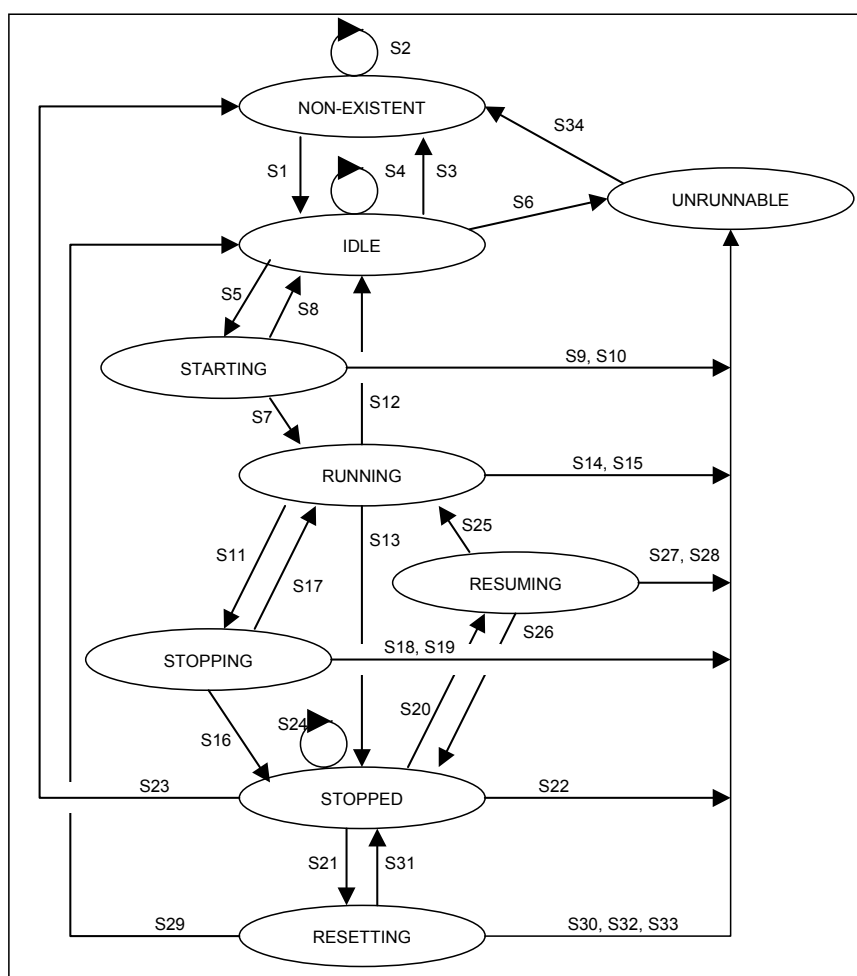


Figure 165 – State diagram

14.3.8.4.2 State transitions

Table 634 specifies the state transitions related to program invocation.

Table 634 – Program invocation state machine

#	Current State	Events	Actions	Next State
S1	NON-EXISTENT	CreateProgramInvocation.ind && Domains available && Access to Domains allowed	CreateProgramInvocation.rsp(+){ } NON-EXISTENT	IDLE
S2	NON-EXISTENT	CreateProgramInvocation.ind && (Domains not available Access to Domains not allowed)	CreateProgramInvocation.rsp(-){ ErrorType := object-constraint-conflict }	NON-EXISTENT
S3	IDLE	DeleteProgramInvocation.ind && Deletable = true	DeleteProgramInvocation.rsp(+){ }	NON-EXISTENT
S4	IDLE	DeleteProgramInvocation.ind && Deletable = false	DeleteProgramInvocation.rsp(-){ ErrorType := object-constraint-conflict }	IDLE
S5	IDLE	Start.ind	(no actions taken)	STARTING
S6	IDLE	Kill.ind	Kill.rsp(+){ }	UNRUNNABLE
S7	STARTING	Start successfully executed && Domains in State READY or IN-USE	IncrementDomainCounter(), Start.rsp(+){ } (note)	RUNNING

#	Current State	Events	Actions	Next State
S8	STARTING	Start failed, non-destructive (e.g. Domains not in State READY or IN-USE)	Start.rsp(-){ ErrorType := object-constraint-conflict }	IDLE
S9	STARTING	Start failed, destructive	Start.rsp(-){ ErrorType := object-access-denied }	UNRUNNABLE
S10	STARTING	Kill.ind	Kill.rsp(+){ }	UNRUNNABLE
S11	RUNNING	Stop.ind	(no actions taken)	STOPPING
S12	RUNNING	End of Program && Reusable = true	DecrementDomainCounter()	IDLE
S13	RUNNING	Program stop	DecrementDomainCounter()	STOPPED
S14	RUNNING	End of Program && Reusable = false	DecrementDomainCounter()	UNRUNNABLE
S15	RUNNING	Kill.ind	DecrementDomainCounter(), Kill.rsp(+){ }	UNRUNNABLE
S16	STOPPING	Stop successfully executed	DecrementDomainCounter(), Stop.rsp(+){ }	STOPPED
S17	STOPPING	Stop failed, non-destructive	Stop.rsp(-){ ErrorType := object-constraint-conflict }	RUNNING
S18	STOPPING	Stop failed, destructive	DecrementDomainCounter(), Stop.rsp(-){ ErrorType := object-access-denied }	UNRUNNABLE
S19	STOPPING	Kill.ind	DecrementDomainCounter(), Kill.rsp(+){ }	UNRUNNABLE
S20	STOPPED	Resume.ind	(no actions taken)	RESUMING
S21	STOPPED	Reset.ind	(no actions taken)	RESETTING
S22	STOPPED	Kill.ind	Kill.rsp(+){ }	UNRUNNABLE
S23	STOPPED	DeleteProgramInvocation.ind && Deletable = true	DeleteProgramInvocation.rsp(+){ }	NON-EXISTENT
S24	STOPPED	DeleteProgramInvocation.ind && Deletable = false	DeleteProgramInvocation.rsp(-){ ErrorType := object-constraint-conflict }	STOPPED
S25	RESUMING	Resume successfully executed && Domains in State READY or IN-USE	IncrementDomainCounter(), Resume.rsp(+){ }	RUNNING
S26	RESUMING	Resume failed, non-destructive (e.g. Domains not in State READY or IN-USE)	Resume.rsp(-){ ErrorType := object-constraint-conflict }	STOPPED
S27	RESUMING	Resume failed, destructive	DecrementDomainCounter(), Stop.rsp(-){ ErrorType := object-access-denied }	UNRUNNABLE
S28	RESUMING	Kill.ind	Kill.rsp(+){ }	UNRUNNABLE
S29	RESETTING	Reset successfully executed && Reusable = true	Reset.rsp(+){ }	IDLE
S30	RESETTING	Reset successfully executed && Reusable = false	Reset.rsp(+){ }	UNRUNNABLE
S31	RESETTING	Reset failed, non-destructive	Reset.rsp(-){ ErrorType := object-constraint-conflict }	STOPPED

#	Current State	Events	Actions	Next State
S32	RESETTING	Reset failed, destructive	Reset.rsp(-){ ErrorType := object-access-denied }	UNRUNNABLE
S33	RESETTING	Kill.ind	Kill.rsp(+){ }	UNRUNNABLE
S34	UNRUNNABLE	DeleteProgramInvocation.ind	DeleteProgramInvocation.rsp(+){ }	NON-EXISTENT
NOTE IncrementDomainCounter() and DecrementDomainCounter() are actions for Download state machine.				

14.4 ARs

14.4.1 Overview

The AREPs used are the following, from Type 1:

Queued User-triggered Uni-directional (QUU) AR Endpoint

Queued User-triggered Bi-directional Connection-Oriented (QUB-Co) AR Endpoint

Buffered Network-Scheduled Uni-directional (BNU) AR Endpoint

14.4.2 Queued user-triggered unidirectional AREP class specification

14.4.2.1 Formal model

ASE: **AR ASE**
CLASS: **QueuedUser-triggeredUnidirectionalAREP**
CLASS ID:
PARENT CLASS: **AR Endpoint**
ATTRIBUTES:
1 (m) Attribute: Role (Client, Server, Peer)
2 (m) Attribute: RemoteAddressConfigurationType (Free, Linked)
3 (m) Attribute: State
SERVICES:
1 (m) OpsService: Unconfirmed Send

14.4.2.2 Attributes

Role

This attribute specifies possible role of this end point. The possible values are Source and Sink.

Source AREPs of this type convey their data by issuing a Data Transfer Unconfirmed PDU.

Sink AREPs of this type receive data transmitted by a Source AREP.

RemoteAddressConfigurationType

This attribute specifies how the remote address of the AREP, that is used to establish an AR, is specified.

Free The value of “Free” indicates that the destination of the FAS-PDUs from the Source AREP is provided during the DTU service invocations.

Linked The value of “Linked” indicates that the destination of the FAS-PDUs is configured with the RemoteDisapAddress attribute of the QuuCI class defined later.

Initiator

This conditional attribute is present if the Role attribute has the value of either Client or Peer. The value of True means that this AREP can issue but cannot receive an Associate request PDU. The value of False means that this AREP can receive but cannot issue an Associate request PDU.

State

This attribute indicates the current state of the Application Relationship Protocol Machine (ARPM) that is defined in detail in the later section. The possible values for this attribute are specified there.

14.4.2.3 Services

Unconfirmed Send

This service is used to send an unconfirmed service on the specified AR.

14.4.3 Queued user-triggered bidirectional AREP class specification

14.4.3.1 Formal model

ASE: **AR ASE**
CLASS: **QueuedUser-triggeredBidirectionalAREP**
CLASS ID:
PARENT CLASS: **AR Endpoint**
ATTRIBUTES:

1	(m)	Attribute:	Role (Client, Server, Peer)
2	(m)	Attribute:	RemoteAddressConfigurationType (Free, Linked)
3	(m)	Attribute:	Role = Peer Client
3.1	(m)	Attribute:	Initiator (True, False)
4	(m)	Attribute:	State

SERVICES:

1	(m)	OpsService:	Confirmed Send
---	-----	-------------	----------------

14.4.3.2 Attributes

Role

This attribute specifies possible role of this end point. The possible values are Client, Server, and Peer.

Peer AREPs of this type may perform as either Clients or Servers.

Client AREPs of this type issue Request-FAS-PDUs to Servers and receive their corresponding Response-FAS-PDUs.

Server AREPs of this type receive Request-FAS-PDUs from Clients and issue corresponding Response-FAS-PDUs to them.

RemoteAddressConfigurationType

This attribute specifies how the remote address of the AREP, that is used to establish an AR, is specified.

Free The value of “Free” indicates that the destination address is provided with the Associate service request.

Linked The value of “Linked” indicates that the destination address of the Associate service request is specified by the RemoteDlcepAddress attribute of the QubCo class defined later.

Initiator

This conditional attribute is present if the Role attribute has the value of either Client or Peer. The value of True means that this AREP can issue but cannot receive an Associate request PDU. The value of False means that this AREP can receive but cannot issue an Associate request PDU.

State

This attribute indicates the current state of the Application Relationship Protocol Machine (ARPM) that is defined in detail in the later section. The possible values for this attribute are specified there.

14.4.3.3 Services**Confirmed Send**

This service is used to send a confirmed service on the specified AR.

14.4.4 Buffered network-scheduled unidirectional AREP class specification**14.4.4.1 Formal model**

ASE:	AR ASE
CLASS:	QueuedUser-triggeredUnidirectionalAREP
CLASS ID:	
PARENT CLASS:	AR Endpoint
ATTRIBUTES:	
1 (m)	Attribute: Role (Client, Server, Peer)
2 (m)	Attribute: DuplicatePduDetectionSupported (True, False)
3 (m)	Attribute: State
SERVICES:	
1 (m)	OpsService: Unconfirmed Send
2 (o)	OpsService: Compel
3 (o)	OpsService: Get-Buffered-Message
4 (o)	OpsService: AR-Status

14.4.4.2 Attributes**Role**

This attribute specifies possible role of this end point. The possible values are Publisher and Subscriber.

Publisher AREPs of this type publish their data issuing unconfirmed service request-PDUs.

Subscriber AREPs of this type receive data published in unconfirmed service request-PDUs.

DuplicatePduDetectionSupported

This attribute specifies when True that the AREP is capable of relaying status information to the AR user indicating whether or not the DLL has delivered a duplicate DL-PDU to the AREP.

State

This attribute indicates the current state of the Application Relationship Protocol Machine (ARPM) that is defined in detail in the later section. The possible values for this attribute are specified there.

14.4.4.3 Services**Unconfirmed Send**

This service is used to send an unconfirmed service on the specified AR.

Compel

This optional service is used to evoke acyclic communication out on a BNU AREP.

Get-Buffered-Message

This optional service is used to retrieve the current contents of the associated buffer.

AR-Status

This optional service is used to report status of the AR.

14.5 Summary of classes

This subclause contains a summary of the defined Classes. Table 635 provides a summary of the classes.

Table 635 – Class summary

ASE	Class	Class ID
VFD	VFD	—
Data Type	Fixed Length & String Data Type	5
	Structure Data Type	6
Object Dictionary	OD Description	—
	Object Dictionary	—
Context Management	VCR List	—
Application Relationship	AREP	32
	QUB-Co	34
	QUU	36
	BNU	38
Variable	Simple Variable	7
	Array Variable	8
	Record Variable	9
	Variable List	10
	Data Type	
	Data Type Structure	
Event	Event	4
Domain	Domain	2
Program Invocation	Program Invocation	3

14.6 Permitted services by AREP role

Table 636 defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 636 – Services by AREP role

	Unc	Cnf	Client	Server	Push Publ	Push Subsc	Pull Publ Mgr	Pull Publ	Pull Subsc	Report Src	Report Sink
Services			req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv	req rcv
VFD ASE											
Status		X	X	X							
Unsolicited Status	X		X	X							
Identify		X	X	X							
Object Dictionary ASE											
Get OD		X	X	X							
Initiate Put OD		X	X	X							
Put OD		X	X	X							
Terminate Put OD		X	X	X							
Context Mgt ASE											
Initiate		X	X	X							
Abort		X	X	X							
Reject		X	X	X							
AR ASE											
AR-Confirmed Send			X	X							
AR-Unconfirmed Send			X X	X X	X	X					
AR-Associate			X	X							
AR-Abort			X X	X X	X	X X			X	X	
AR Compel			X	X		X					
AR-Get Buffered Msg			X	X		X					
AR-Status			X	X							
Variable ASE											
Read		X	X	X			X X	X			
Write		X	X	X							
Information Report	X				X	X				X	X
Define Variable List		X	X	X							
Delete Variable List		X	X	X							
Event ASE											
Event Notification	X				X	X				X	X
Ack Event		X	X	X							
Alter Event Cond Mon		X	X	X							
Domain ASE											
GenInitDwnldSeq		X	X	X							
GenDownloadSeq		X	X	X							
GenTermDwnldSeq		X	X	X							
InitiateDwnldSeq		X	X	X							
DwnldSeq		X	X	X							
TermDwnldSeq		X	X	X							
ReqDomainDwnld		X	X	X							
InitiateUploadSeq		X	X	X							
UploadSeq		X	X	X							
TermUploadSeq		X	X	X							
ReqDomainUpload		X	X	X							
Program Inv ASE											
Start		X	X	X							
Stop		X	X	X							
Resume		X	X	X							
Reset		X	X	X							
Kill		X	X	X							
Create Program Inv		X	X	X							
Delete Program Inv		X	X	X							

15 Type 10 communication model specification

15.1 Type 10 Concepts

15.1.1 Overview

The type 10 specification provides an advanced FAL object model necessary for distributed automation between smart automation devices having equal rights and having a facet for the remote engineering of the system. These two facets are well defined within the type 10 FAL application entity.

This section introduces the basic concept of the type 10 FAL application entity (FAL AE). The type 10 FAL is composed of type 10 specific APO ASEs and one AR ASE. The APO ASEs provide different services and attributes which are structured into groups referred to as interfaces. Each interface provide a certain view on the object instance and is uniquely referenced via a local address or a reference referred to as Interface Pointer. The type 10 APO ASEs have a well defined relationship among each other – also within a device- referred to as Runtime Object Model. This basic type 10 model is described in subclause 15.1.5.

The type 10 FAL AE maps the type 10 ASEs via one AR ASE onto a set of underlying services and protocols modeled as an abstract ORPC model. These services are referred to as ORPC services and the protocol is referred to as ORPC wire protocol. Any existing and future ORPC services and protocols (e.g. in form of specific middleware) that can be mapped to the defined abstract services can be used. The abstract ORPC model provides services and protocols that may create an object instance from an globally unique identified object class or interface from that class, provides and manages all binding information to the remote object instance by means of Interface Pointers, provides at least one confirmed service to convey defined type 10 ASE services, and finally provides data marshaling for all service parameters by means of a well defined interface description referred to as interface definition language (IDL).

15.1.2 Objectives of the Type 10 FAL AE

One objective of type 10 FAL AE is to provide services for distributed automation by means of ORPC services. This allows type 10 automation devices (e.g. controller, drives, field devices, I/O devices, HMI devices, ...) to be connected and exchange data values both as consumer and/or provider in an optimized way.

Another objective of type 10 FAL AE is to provide services for the engineering of automation devices and automation functions by means of ORPC services. This allows one or more engineering devices to configure an automation system via interconnecting data items as source or sink and invoke services to save that connection information persistent within each consumer part of an automation device. Furthermore, engineering clients may use services to read diagnosis information at different levels from automation devices. The engineering is also supported by a well defined navigation procedure through the linked object instances of the APO ASE classes with the Physical Device Class as the navigation anchor.

Another objective is the seamless business integration. The type 10 FAL ASE in combination with the chosen ORPC model is able to communicate seamlessly with common office application e.g. as HMI tools.

Another objective of type 10 FAL AE is to connect type 10 devices with type 3 devices by means of a proxy device. Therefore, distributed applications are supported on devices that are connected via different networks.

In summary, the type 10 FAL AE enables:

- distributed automation systems to be constructed and engineered from type 10 devices,

- remote applications (e.g. enterprise resource planning systems, office applications) may access field devices or controller of any type through ORPC services using the type 10 communication model,
- engineering applications to interconnect data items in a common way,
- type 3 or other networks to be linked via a type 10 proxy device.

15.1.3 Devices types

Type 10 FAL devices are FAL devices that contain a type 10 FAL AE with exactly one instance of the Physical Device Class (PDev), with one or more instances of the Logical Device Class (LDev), with exactly one instance of the Active Connection Control Object Class (ACCO) per LDev instance, and with one or more instances of RT-Auto Class per LDev instance. These type 10 FAL devices are also referred to as automation devices.

Another class of type 10 devices is capable to manage the connection of data items of instances of RT-Auto classes, read diagnosis information from automation devices, and in summary is capable to use the services from the FAL objects as a client. This type of FAL device is referred to as engineering device.

Another class of type 10 devices is capable to connect a type 3 network or other networks to a type 10 network and to provide the type 10 object model as a proxy for type 3 or other devices. This type of FAL device is referred to as proxy device.

A device manufacturer may also implement different type 10 device classes inside one product as mixed implementation.

15.1.4 Type 10 FAL ASEs

Figure 166 shows the type 10 FAL ASEs from the communication perspective. The application process uses ASE service primitives to influence remote objects. The services are grouped by means of interfaces. The only address information is the Interface Pointer. The Interface Pointer contains implicit binding information (e.g. IP address and reference to an interface on a particular object instance on that node). These services are all mapped to the AR-Call service. The AR-CoCreateInstance service is directly called by the application process to achieve a binding to a remote PDev. The AR ASE conveys the services by means of the services provided by the ORPC model. In addition to the unified service parameter the AR ASE adds a reference to the IDL description. This reference is necessary for the marshalling of service parameter performed by the ORPC sublayer. Eventually, the application layer protocol data unit (APDU) is build by the ORPC wire protocol and conveyed to the remote host.

NOTE This communication procedure may also be used for local communication within the device as a local matter.

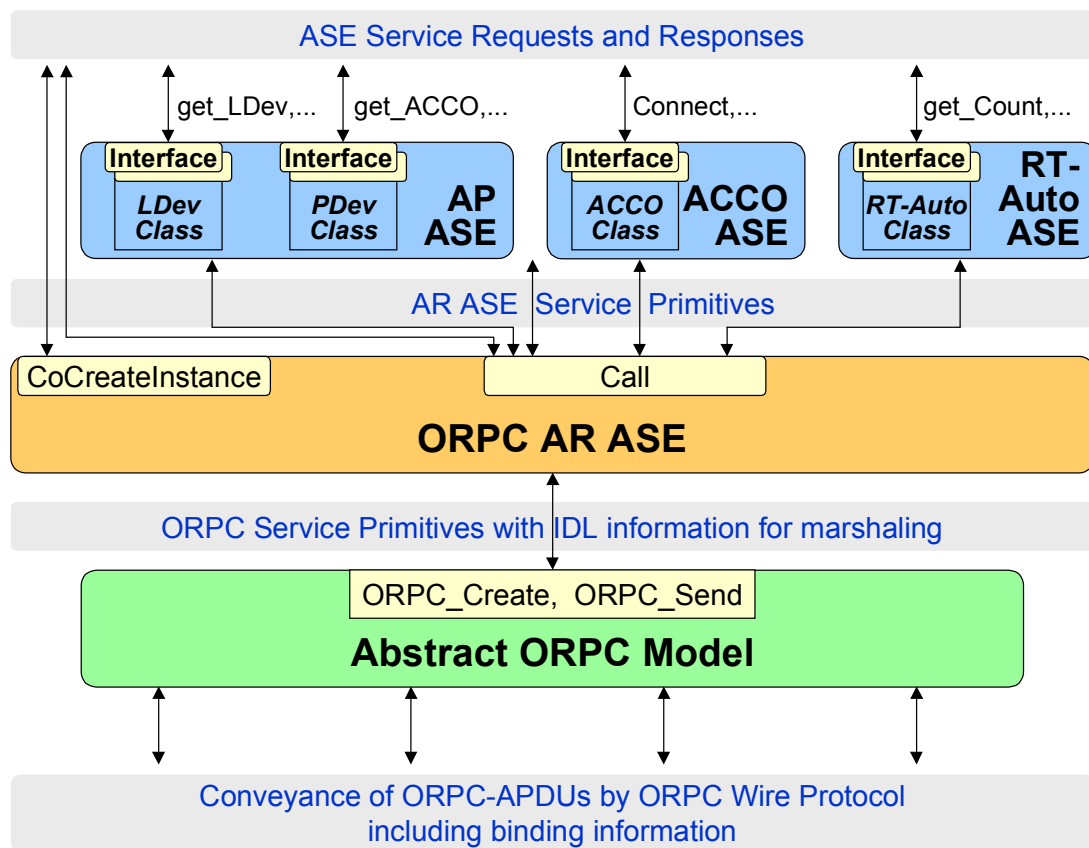


Figure 166 — FAL ASEs Communication Architecture

The relationships and linking between the FAL ASEs is provided by the Runtime Object Model that is defined in the subclause below.

15.1.5 Runtime Object Model

The type 10 runtime object model represents the objects that exist in a type 10 automation device, together with their interfaces grouping services and attributes. The model also describes the interrelationship between the individual objects.

The Figure 167 specifies the runtime object model of type 10.

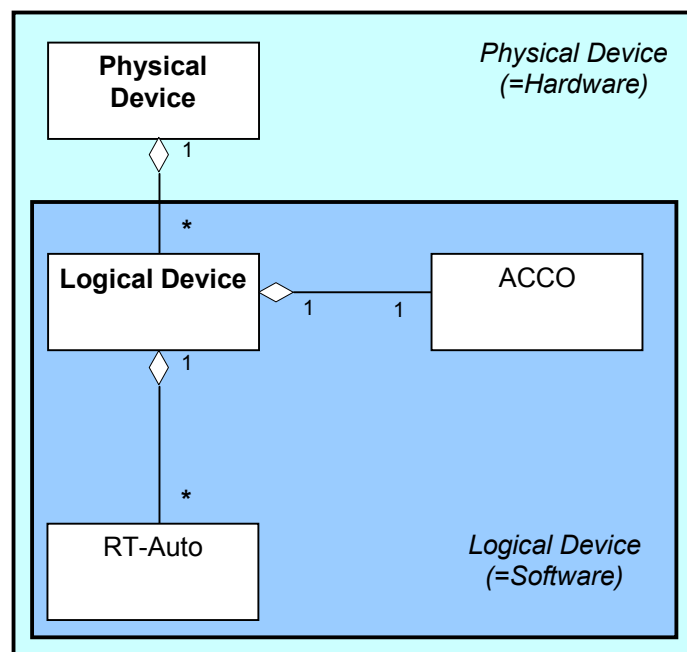


Figure 167 — Runtime object model

In the runtime object model, the Physical Device (PDev) represents a physical component (i.e. hardware). It offers a network access to one or more IP networks. The PDev exposes the physical properties of the component via the ICBAPhysicalDevice interface. Exactly one instance of the PDev exists in each hardware component (e.g. PLC, drive, PC).

A PDev references one or more Logical Devices (LDev). An LDev represents a software component or a firmware component as an autonomous self-contained unit. The LDev represents the functions of a device for example an actuator, sensor, controller or CNC. The devices solve the distributed automation task.

The PDev is able to navigate to all contained LDev by their names. It offers browsing functionality for the names of the LDevs.

The PDev shall have the type 10 ClassID (defined in 15.3.2.1.1) that is the same for all devices participating in type 10 fieldbus. A communication partner without any knowledge of the destination hardware/software can obtain an Interface Pointer to the PDev object. This object is used for navigating to any further object.

In most cases, there is a firm one to one allocation between PDev and LDev. This means that there is exactly one firmware related to a hardware component. Examples are PLC, drives or field devices.

In addition, a one to n allocation is possible. It is used for devices that offer several independent functions in a hardware. Examples are free computing power within a PC with SlotPLC and SoftPLC, or a PC with HMI and PLC functionality.

The LDev comprise the components operating system and applications. The operating system consists of two parts: A standardized part and an enhanced part that is used for special definitions of the LDev programmer.

The type 10 specification uses the LogicalDevice and ACCO class to describe the functionality that is the same throughout all LDevs.

The LogicalDevice class provides general automation functions such as identification or diagnosis, and is used as a navigation anchor for all other objects of the operating system. In addition, it is able to navigate to all referenced RT-Auto objects by their names and it offers browsing functionality for the names of the RT-Auto objects.

The ACCO (Active Control Connection Object) class implements a configurable connection of RT-Autos objects.

For both, functionality that is specified by type 10 is required. In the following, this functionality is marked as mandatory or optional. Mandatory interfaces shall be offered, optional interfaces may be offered. If a functionality is offered that is specified by an optional interface, this shall be done using the described interface.

The LDev and ACCO may provide manufacturer specific functions that go beyond type 10 specification. This functionality may be defined by a manufacturer and/or a trade organization. This enhanced functionality shall be expressed in form of interfaces. It shall be defined by an interface definition in the form of an IDL. The QueryInterface mechanism supports inquiry whether a specific device supports optional functionality.

The RT-Auto ASE represents the automation functionality in the form of a process-related component. Examples are boiler, controller, etc. These automation functionalities of an application may exist in the form of pre-tested, self-contained and universally useable components. Using the interconnecting functionality that is defined by ACCO ASE, automation functionalities can be configured as a distributed application.

The Figure 168 specifies the relationship between Engineering and Runtime.

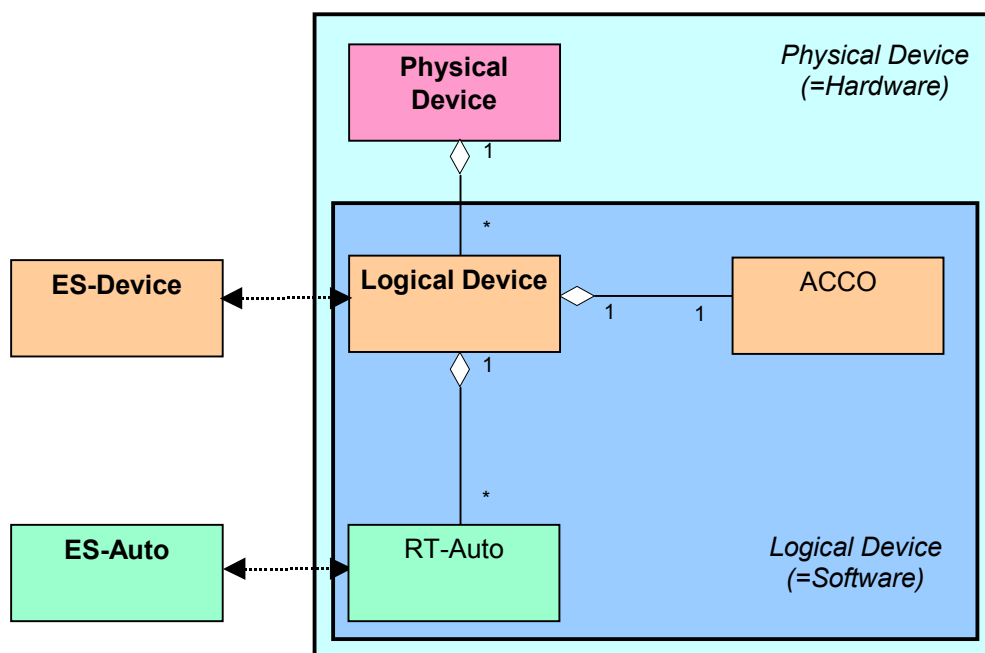


Figure 168 — Relationship between Engineering and Runtime

Each RT-Auto object has a representative in Engineering. All the other objects of the LDev together have one Engineering System device as representative in the engineering. This representative hides the internal object structure and the producer-specific features of the enhanced LDev towards Engineering.

The following basic rule applies to separating a device of fixed functionality into RT-Auto and LDev objects: If the function of a LDev object shall be interconnectable in an application, this function shall be implemented via an RT-Auto object of a fixed functionality. The producer-specific properties of the LDev chiefly contain the functionality that is used for hardware configuration, debugging and other similar services. The properties of the LDev can only be used by the LDev's engineering representative.

The LDev shall have a functioning default so that it can be used immediately after installation in a basic scope and without parameter value assignment. In particular, this concerns definitions with regard to a default configuration for the IP stack.

15.1.6 Navigation in the Object Model

Two different procedures may be used for navigating to an object of the runtime object model:

The navigation to the PDev of a type 10 device is performed via IP address or DNS name using the ORPC AR service CoCreateInstance (see 15.4.2.2.1).

The navigation within the runtime object model takes place via attributes (properties) at the individual objects (as shown in Figure 169).

If these are one to n relations, the navigation between PDev and LDev, and between LDev and RT-Auto objects are based on names. PDev offers the LogicalDevice property and LDev offers the RTAuto property for this purpose.

Each of the LDev and RT-Auto objects offers the Name property for self-information.

The tag and the parent property that are available at the PDev and LDev objects shall solely be used for supporting identification systems, it is not used for navigation.

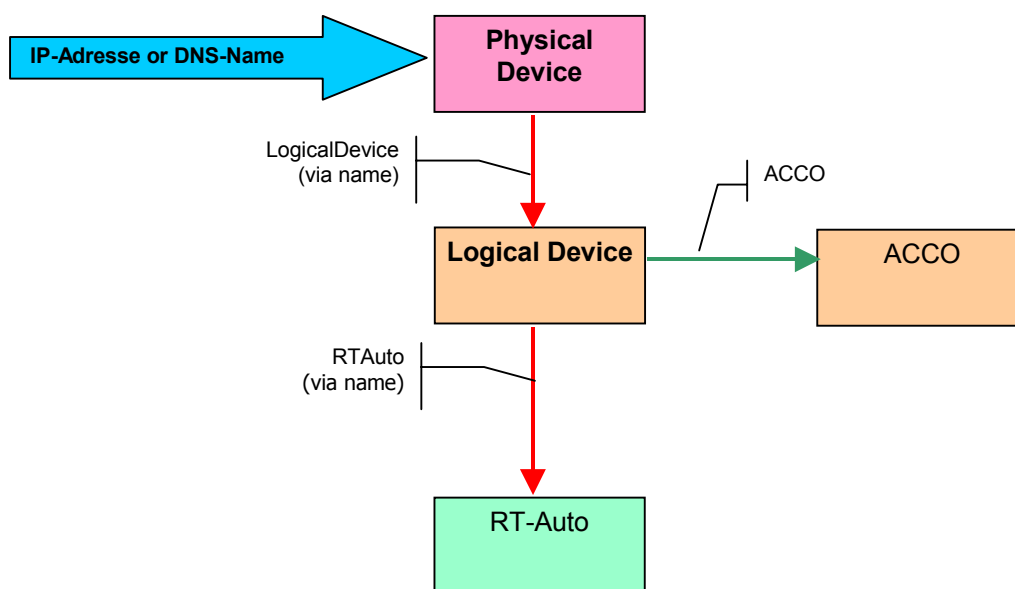


Figure 169 — Navigation in the runtime object model

15.2 ASE Data Types

The data types supported by type 10 are a subset of those defined in clause 5. They are:

ACCESSRIGHTSDEF
 ADDCONNECTIONIN
 ADDCONNECTIONOUT
 BSTR
 char
 CONNECTIN
 CONNECTOUT
 currency
 date
 DISPPARAMS
 double
 EXCEPINFO
 HRESULT
 FILETIME
 float
 GETCONNECTIONOUT
 GETIDOUT
 GROUPEXCEPTIONDEF
 Interface Pointer
 ITEMQUALITYDEF
 long
 LPWSTR
 PERSISTDEF
 READITEMOUT
 SAFEARRAY
 short
 STATEDEF
 struct (see note below)
 unsigned char
 unsigned long
 unsigned short
 UUID
 VARIANT
 VARTYPE
 VT_BOOLEAN
 WRITEITEMIN
 WRITEITEMQCDIN

NOTE The term struct is used to address user defined structured data types that shall be built according to 5.2.1. using the base data types defined in type 10.

15.3 ASEs

15.3.1 Object Management ASE

15.3.1.1 Overview

The object management ASEs provides a common set of management services applicable to all classes of objects, except for the Data Type classes. This management services operate on FAL APOs and their attributes. FAL APO classes are defined within the FAL ASEs that are specified in the remaining clauses. Each FAL APO class defined is described by a set of attributes and a set of services.

Within the FAL Type 10 ASE the term interface is used to structure attributes and services into different views of a FAL Class. This interface or attribute structure of a class is uniquely identified by a globally unique identifier (UUID) referred to as Interface ID. The parameter Interface Pointer contains the local address of the interface itself during runtime and defines an unambiguous reference. The reference is volatile and the value shall be always obtained online by means of specific type 10 FAL services. It is only valid (e.g. indicated by specific HRESULTS) until the server fails in the sense of communication (e.g. power down, communication failure). The reference stays invalid, even if the server hosting the object with former referenced interface is up and running again. Therefore, the client gets a valid Interface Pointer reestablishing the Interface Pointer anew. There are three possibilities to do this. The Interface Pointer is returned by the object instance of the PDev Class accessed via the AR service CoCreateInstance using IP address or DNS name, ClassID and Interface ID. A second alternative is to ask the IUnknown interface of an object for another interface of the same class via the management service QueryInterface. A third alternative is to use specific services (e.g. get_LogicalDevice) to navigate through the objects.

The type 10 FAL services are utilizing an abstract object remote procedure call (ORPC) model that defines exactly one AREP for the FAL services. Therefore, the local addressing of a FAL service is implicit and the service parameter AREP is omitted. The conveyance of the FAL service parameter, network addressing and, e.g. managing of network connections is part of the underlying ORPC wire protocol and goes beyond the FAL service specification.

The FAL Type 10 Object Management ASE provides two object management attribute and service groups: the IUnknown attribute and the IDispatch attribute.

The IUnknown attribute is the fundamental interface and mandatory for each type 10 FAL Class. It supports resource management (AddRef and Release), type coercion (QueryInterface) and identity. The IUnknown interface is the root of all interfaces. This means that it is not derived from any further interface. All other interfaces are derived either directly from IUnknown or from another interface that is in turn derived from IUnknown or from another interface. IUnknown contains exactly three services: QueryInterface, AddRef and Release. Since each interface needs to eventually be derived from IUnknown, these methods can be invoked from any interface of an object. IUnknown itself is a separate interface with its own interface identifier ID. Thus, a client can obtain an Interface Pointer via QueryInterface that points specifically to IUnknown.

The IDispatch attribute is a second interface supported by the Physical Device, Logical Device, and RT-Auto FAL Classes. IDispatch is an interface that is designed in such a way that it can call virtually any other FAL class interfaces if supported by the particular class. Type 10 FAL Classes supporting the IDispatch interface shall contain all attributes and all services of the object instance.

NOTE 1 The IDispatch interface can be used for the integration into script languages. Essentially, it permits a symbolic access to an Type 10 FAL object to be made.

NOTE 2 Scripts use only one access to Type 10 FAL objects – via the IDispatch interface that is returned by the QueryInterface service. To enable scripts to access all attributes and services of an object, the implementation of the IDispatch interface contains all members of all interfaces of the object that have been derived from IDispatch. In a way, the IDispatch interface aggregates all attributes and services of all interfaces to one "union set".

15.3.1.2 Class Specifications

1. (m) Key Attribute: implicit
2. (m) Attribute: IUnknown
- 2.1. (m) MgtService: QueryInterface
- 2.2. (m) MgtService: AddRef
- 2.3. (m) MgtService: Release
3. (m) Attribute: IDispatch
- 3.1. (m) MgtService: GetTypeInfoCount
- 3.2. (m) MgtService: GetTypeInfo
- 3.3. (m) MgtService: GetIDsOfNames
- 3.4. (m) MgtService: Invoke

15.3.1.3 Attributes

IUnknown

This attribute specifies the IUnknown interface of the object. It is a structured attribute composed of the following attributes and services.

IDispatch

This attribute specifies the IDispatch interface of the object. It is a structured attribute composed of the following attributes and services.

15.3.1.4 Service Specification

15.3.1.4.1 QueryInterface

The QueryInterface service reads the Interface Pointer (address) of a requested interface of an object instance. The arguments of the service are shown in Table 637.

Table 637 – QueryInterface

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Interface ID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
ppvObject			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

Interface ID

This parameter specifies the unique interface identifier of the requested interface.

Parameter Type: UUID

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

ppvObject

This parameter contains the address of the requested interface of the object instance.

Parameter Type: Interface Pointer

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOINTERFACE

15.3.1.4.2 AddRef

The AddRef service increments and reads the reference count for a requested interface of an object. The reference count is a local attribute that can be used to reflect the using of an object interface. It does not have any influence on the lifetime of the object.

The arguments of the service are shown in Table 638.

Table 638 – AddRef

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result			M	M(=)
Interface Pointer			M	M(=)
Reference Count			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

Result:

This parameter indicates that the service request was executed.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

Reference Count

This parameter contains the new reference count.

Parameter Type: unsigned long

Allowed values: any value different than zero

15.3.1.4.3 Release

The Release service decrements and reads the reference count for an requested interface on an object. The reference count is a local attribute that can be used to reflect the using of an object interface. It does not have any influence on the lifetime of the object. The arguments of the service are shown in Table 639.

Table 639 – Release

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result			M	M(=)
Interface Pointer			M	M(=)
Reference Count			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

Result:

This parameter indicates that the service request was executed.

Interface Pointer

This parameter specifies the address of the IUnknown interface of the requested object instance.

Parameter Type: Interface Pointer

Reference Count

This parameter contains the new reference count.

Parameter Type: unsigned long

Allowed values: any value different than zero

15.3.1.4.4 GetTypeInfoCount

The GetTypeInfoCount service determines whether type information is available for an object or not. The arguments of the service are shown in Table 640.

Table 640 – GetTypeInfoCount

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pcTInfo			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pcTInfo

This parameter specifies whether this object provides type information or not.

Parameter Type: unsigned short

Allowed values: 0 (the object does not provide type information), 1 (the object provides type information)

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL

15.3.1.4.5 GetTypeInfo

The GetTypeInfo service retrieves the type information for an object, which can then be used to get the type information for an interface.

NOTE The type information is derived from the deployed ORPC model and beyond the scope of the FAL service specification. It contains the description of attributes and services in order to support a late binding.

The arguments of the service are shown in Table 641.

Table 641 – GetTypeInfo

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
iTInfo	M	M(=)		
Icid	U	U(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
ppTInfo			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

ITInfo

This parameter specifies the type information to return. Zero indicates the IDispatch interface.

Parameter Type: unsigned short

Allowed values: 0

Icid

This parameter contains a language identifier for the type information. An object may be able to return different type information for different languages.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

ppTInfo

This parameter contains the requested type information.

Parameter Type: ITypeInfo

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: DISP_E_BADINDEX, TYPE_E_ELEMENTNOTFOUND

15.3.1.4.6 GetIDsOfNames

The GetIDsOfNames service maps a single attribute name or service name and an optional set of parameter names to a corresponding set of integer dispatch identifiers, which are used on subsequent calls of the Invoke service. The arguments of the service are shown in Table 642.

Table 642 – GetIDsOfNames

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
riid	M	M(=)		
List of rgszNames	M	M(=)		
cNames	M	M(=)		
lcid	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of rgDispId			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

riid

This parameter is reserved and shall contain UUID_NULL.

Parameter Type: UUID

List of rgszNames

This parameter refers the names to be mapped.

Parameter Type: LPWSTR

cNames

This parameter contains the number of the names to be mapped.

Parameter Type: unsigned short

lcid

This parameter contains a language identifier to indicate the context in which the names are to be interpreted. An object may be able to provide different languages.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

List of rgDispld

This parameter contains the requested identifiers (ID) corresponding to the names passed in the List of rgpszNames. The first identifier represents the attribute name respective service name. The subsequent identifiers represent the service parameter names.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, DISP_E_UNKNOWNNAME, DISP_E_UNKNOWNLCID

15.3.1.4.7 Invoke

The Invoke service provides access to attributes and calls services exposed by an object. The arguments of the service are shown in Table 643.

Table 643 – Invoke

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
displdMember	M	M(=)		
riid	M	M(=)		
lcid	M	M(=)		
wFlags	M	M(=)		
pDispParams	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVarResult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pExcepInfo			C	C(=)
puArgErr			C	C(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

dispIdMember

This parameter specifies the identifier of the attribute or service to invoke.

Parameter Type: long

riid

This parameter is reserved and shall contain UUID_NULL.

Parameter Type: UUID

lcid

This parameter contains a language identifier to indicate the context in which the names are to be interpreted. An object may be able to provide different languages.

Parameter Type: unsigned long

wFlags

This parameter specifies the context of the invoke service.

Parameter Type: unsigned short

Allowed values: DISPATCH_METHOD, DISPATCH_PROPERTYGET, DISPATCH_PROPERTYPUT, DISPATCH_PROPERTYPUTREF

pDispParams

This parameter contains an array of arguments, an array of dispatch identifiers for named arguments and the number of elements in these arrays.

Parameter Type: DISPPARAMS

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVarResult

This parameter contains the result of the invoked attribute or service.

Parameter Type: VARIANT

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the IDispatch interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: DISP_E_BADPARAMCOUNT, DISP_E_BADVARTYPE,
DISP_E_EXCEPTION, DISP_E_MEMBERNOTFOUND, DISP_E_NONAMEDARGS,
DISP_E_OVERFLOW, DISP_E_PARAMNOTFOUND, DISP_E_TYPEMISMATCH,
DISP_E_UNKNOWNINTERFACE, DISP_E_UNKNOWNLCID,
DISP_E_PARAMNOTOPTIONAL

pExcepInfo

This parameter is only valid for *hresult=DISP_E_EXCEPTION* and contains the exception information.

Parameter Type: EXCEPINFO

puArgErr

This parameter is only valid for *hresult=DISP_E_TYPEMISMATCH* or *hresult=DISP_E_PARAMNOTFOUND* and contains the index of the first argument containing an error.

Parameter Type: unsigned short

15.3.2 AP ASE

15.3.2.1 Physical Device

15.3.2.1.1 Overview

In the runtime object model, the PhysicalDevice (abbreviated PDev) represents a physical component (i.e. hardware). It offers a network access to one or more IP networks. The PDev exposes the physical properties of the component via the ICBAPhysicalDevice interface. Exactly one instance of the PDev exists in each type 10 automation device (e.g. PLC, drive, PC).

15.3.2.1.2 Physical Device Class Specifications

15.3.2.1.2.1 Formal Definition

- | | | | |
|--------|-----|----------------|-----------------------|
| 1. | (m) | Key Attribute: | implicit |
| 2. | (m) | Attribute: | ICBAPhysicalDevice |
| 2.1 | (m) | Attribute: | Producer |
| 2.2 | (m) | OpsService: | get_Producer |
| 2.3 | (m) | Attribute: | Product |
| 2.4 | (m) | OpsService: | get_Product |
| 2.5 | (m) | Attribute: | SerialNo |
| 2.6 | (m) | OpsService: | get_SerialNo |
| 2.7 | (m) | Attribute: | ProductionDate |
| 2.8 | (m) | OpsService: | get_ProductionDate |
| 2.9 | (m) | OpsService: | Revision |
| 2.10 | (m) | Attribute: | List of LogicalDevice |
| 2.10.1 | (m) | Attribute: | Name |
| 2.10.2 | (m) | Attribute: | Interface Pointer |
| 2.11 | (m) | OpsService: | get_LogicalDevice |
| 3. | (m) | Attribute: | ICBABrowse |
| 3.1 | (m) | Attribute: | Count |
| 3.2 | (m) | OpsService: | get_Count |
| 3.3 | (m) | OpsService: | BrowseItems |
| 4. | (m) | Attribute: | ICBAPersist |
| 4.1 | (m) | OpsService: | Save |
| 5. | (m) | Attribute: | Dispatchable |

15.3.2.1.2.2 Attributes

ICBAPhysicalDevice

This attribute specifies the physical device interface of the object. It is a structured attribute composed of the following attributes.

Producer

This attribute specifies the name of the device producer.

Attribute Type: BSTR (maximum length of 128 usable characters)

Product

This attribute specifies the name of the device type in a producer-specific and producer-unambiguous way.

Attribute Type: BSTR (maximum length of 128 usable characters)

SerialNo

This attribute specifies the serial number of the Physical Device.

Attribute Type: VARIANT

ProductionDate

This attribute specifies the production date of the Physical Device.

Attribute Type: date

List of LogicalDevice

This attribute specifies a list of references to all related Logical Device objects. A single reference can be read via the service `get_LogicalDevice`. It is used for navigating in the object model. The attribute is composed of the following list elements.

Name

This attribute specifies the name of the Logical Device object.

Attribute Type: BSTR

Interface Pointer

This attribute specifies the local address of the `ICBALogicalDevice` interface of the Logical Device object.

Attribute Type: Interface Pointer

ICBABrowse

This attribute specifies the browse interface of the object. It is a structured attribute composed of the following attributes.

Count

This attribute specifies the number of elements in the browse database.

Attribute Type: long

ICBAPersist

This attribute specifies the persistence interface of the object. It is a structured attribute composed of the following attributes.

Dispatchable

This attribute specifies the availability of type information for the FAL Class that is provided by the `IDispatch` common interface. The Physical Device class provides that kind of information.

Attribute Type: Boolean

Allowed values: 1

15.3.2.1.3 Physical Device Behavior

For type 10 FAL ASE object instances, the `AR-CoCreateInstance` service is only permitted for the `PhysicalDevice` class. All other objects do not support this service and are "noncreatable".

The `PhysicalDevice` class shall be implemented according to the Singleton pattern.

NOTE Singleton is a class that possesses exactly one object copy that was instantiated beforehand, and provides a well-defined access point to it. The Singleton pattern ensures that only one instance of a class can be created, and that this instance is globally accessible. The function `Instance(s)` is (are) used to access the unique instance of the class.

The `PhysicalDevice` class is instantiated by the `AR-CoCreateInstance` service with the `ClassID (UUID_PhysicalDevice)`, that is well-defined by type 10 specification, and an IP address or a DNS name. This service returns an Interface Pointer to the – according to the Singleton pattern – previously instantiated object (i.e. `PDev`). Any other object of the RT object model can only be reached via navigation procedure (see 15.1.6).

15.3.2.1.4 Physical Device Service Specification

15.3.2.1.4.1 `get_Producer`

The `get_Producer` service reads the name of the device producer. The arguments of the service are shown in Table 644. This service reads the attribute `Producer`.

Table 644 – `get_Producer`

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
<code>hresult</code>			M	M(=)
<code>pVal</code>			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
<code>hresult</code>			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the `ICBAPhysicalDevice` interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the `ICBAPhysicalDevice` interface of the requested object instance.

Parameter Type: Interface Pointer

`hresult`

This parameter contains the result code.

Parameter Type: `HRESULT`

Allowed values: `S_OK`

`pVal`

This parameter contains the name of the producer.

Parameter Type: `BSTR`

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the `ICBAPhysicalDevice` interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.2 get_Product

The get_Product service reads the name of the device type in a producer-specific and producer-unambiguous way. The arguments of the service are shown in Table 645. This service reads the attribute product.

Table 645 – get_Product

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the name of the product.

Parameter Type: BSTR

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.3 get_SerialNo

The get_SerialNo service reads the serial number of the PDev. The arguments of the service are shown in Table 646. This service reads the attribute SerialNo.

Table 646 – get_SerialNo

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the SerialNo.

Parameter Type: VARIANT

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.4 get_ProductionDate

The get_ProductionDate service reads the production date of the PDev. The arguments of the service are shown in Table 647. This service reads the attribute ProductionDate.

Table 647 – get_ProductionDate

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the ProductionDate.

Parameter Type: date

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.5 Revision

The Revision service reads the revision level of the PDev (i.e. the hardware version) as a major and minor revision. The arguments of the service are shown in Table 648.

Table 648 – Revision

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pMajor			M	M(=)
pMinor			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pMajor

This parameter contains the major part of the revision.

Parameter Type: short

pMinor

This parameter contains the minor part of the revision.

Parameter Type: short

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY

15.3.2.1.4.6 get_LogicalDevice

The get_LogicalDevice service reads the reference to a LDev object via its name. The arguments of the service are shown in Table 649. This service reads one element of the FAL Class attribute List of LogicalDevice.

Table 649 – get_LogicalDevice

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Name	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)
PpLDev			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Name

This parameter contains the name of the referenced LDev object.

Parameter Type: BSTR

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

ppLDev

This parameter contains the address of the ICBALogicalDevice interface of the referenced LDev object. Other LDev interfaces can be obtained using the QueryInterface service.

Parameter Type: Interface Pointer

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPhysicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, CBA_E_UNKNOWNOBJECT

15.3.2.1.4.7 get_Count

The get_Count service reads the number of elements in the browse database. The arguments of the service are shown in Table 650. This service reads the attribute Count.

Table 650 – get_Count

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal = Count			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the number of Browse items.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.8 BrowseItems

The BrowseItems service reads the list of available LDevs. The arguments of the service are shown in Table 651.

Table 651 – BrowseItems

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Offset	M	M(=)		
MaxReturn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)
Pitem			M	M(=)
pDataType			U	U(=)
pAccessRight			U	U(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Offset

This parameter specifies the offset of the first LDev item in the List of LogicalDevice that shall be returned.

Parameter Type: long

MaxReturn

This parameter specifies the maximum number of LDev items that shall be returned.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pltem

This parameter contains the name or list of names of the available LDevs.

Parameter Type: VARIANT

Allowed values: Tag=VT_SAFEARRAY_BSTR

pDataType

This parameter is reserved and shall be filled with the corresponding result code.

Parameter Type: VARIANT

Allowed values: Tag=VT_ERROR, Value=DISP_E_PARAMNOTFOUND.

pAccessRight

This parameter is reserved and shall be filled with the corresponding result code.

Parameter Type: VARIANT

Allowed values: Tag=VT_ERROR, Value=DISP_E_PARAMNOTFOUND.

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.1.4.9 Save

The Save service saves all connection information (inside the consumer) of all ACCO ASEs of the automation device. The connection information shall be stored in non volatile memory and shall be available after a power on.

NOTE The PDev is responsible to store the connection information of all ACCO ASEs at the same time. The access to the connection information is a local matter.

The arguments of the service are shown in Table 652.

Table 652 – Save

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAPersist interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAPersist interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAPersist interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, CBA_E_PERSISTRUNNING, CBA_E_DEFECT, CBA_E_NOTAPPLICABLE

15.3.2.2 Logical Device**15.3.2.2.1 Overview**

The LogicalDevice object provides general automation functions such as identification or diagnosis, and is used as navigation anchor for all further objects of the automation device. In addition, it is able to navigate to all contained RT-Auto objects by their names and it offers browsing functionality for the names of the RT-Auto objects. A LDev object represents a software package or a firmware package as an autonomous self-contained unit.

15.3.2.2.2 Logical Device Class Specifications**15.3.2.2.2.1 Formal Definition**

- | | | | |
|--------|-----|----------------|--------------------|
| 1. | (m) | Key Attribute: | Implicite |
| 2. | (m) | Attribute: | ICBALogicalDevice |
| 2.1 | (m) | Attribute: | Name |
| 2.2 | (m) | OpsService: | get_Name |
| 2.3 | (m) | Attribute: | Producer |
| 2.4 | (m) | OpsService: | get_Producer |
| 2.5 | (m) | Attribute: | Product |
| 2.6 | (m) | OpsService: | get_Product |
| 2.7 | (m) | Attribute: | SerialNo |
| 2.8 | (m) | OpsService: | get_SerialNo |
| 2.9 | (m) | Attribute: | ProductionDate |
| 2.10 | (m) | OpsService: | get_ProductionDate |
| 2.11 | (m) | OpsService: | Revision |
| 2.12 | (m) | Attribute: | ACCO |
| 2.13 | (m) | OpsService: | get_ACCO |
| 2.14 | (m) | Attribute: | List of RT-Auto |
| 2.14.1 | (m) | Attribute: | Name |
| 2.14.2 | (m) | Attribute: | Interface Pointer |
| 2.15 | (m) | OpsService: | get_RTAuto |
| 3. | (m) | Attribute: | ICBAState |
| 3.1 | (m) | Attribute: | State |
| 3.2 | (m) | OpsService: | get_State |
| 3.3 | (m) | OpsService: | Activate |
| 3.4 | (m) | OpsService: | Deactivate |
| 3.5 | (m) | OpsService: | Reset |
| 3.6 | (m) | OpsService: | AdviseState |
| 3.7 | (m) | OpsService: | UnadviseState |
| 4. | (o) | Attribute: | ICBATime |
| 4.1 | (m) | Attribute: | Time |
| 4.2 | (m) | OpsService: | get_Time |
| 4.3 | (m) | OpsService: | put_Time |
| 5. | (m) | Attribute: | ICBABrowse |

5.1	(m)	Attribute:	Count
5.2	(m)	OpsService:	get_Count
5.3	(m)	OpsService:	BrowseItems
6.	(o)	Attribute:	ICBAGroupError
6.1	(m)	OpsService:	GroupError
6.2	(m)	OpsService:	AdviseGroupError
6.3	(m)	OpsService:	UnadviseGroupError
7.	(o)	Attribute:	List of ICBACustom
7.1	(o)	Attribute:	List of Custom Attributes
7.2	(o)	Attribute:	List of Custom Services
8.	(m)	Attribute:	Dispatchable

15.3.2.2.2 Attributes

ICBALogicalDevice

This attribute specifies the logical device interface of the object. It is a structured attribute composed of the following attributes.

Name

This attribute specifies the name of the LDev.

Attribute Type: BSTR

Producer

This attribute specifies the name of the device producer.

Attribute Type: BSTR (maximum length of 128 usable characters)

Product

This attribute specifies the name of the device type in a producer-specific and producer-unambiguous way.

Attribute Type: BSTR (maximum length of 128 usable characters)

SerialNo

This attribute specifies the serial number of the Logical Device.

Attribute Type: VARIANT

ProductionDate

This attribute specifies the production date of the Logical Device.

Attribute Type: date

ACCO

This attribute specifies the local address of the IUnknown interface of the related ACCO object and is used for navigating in the object model.

Attribute Type: Interface Pointer

List of RT-Auto

This attribute specifies a list of references to all related RT-Auto objects. A single reference can be read via the service get_RTAuto. It is used for navigating in the object model. The attribute is composed of the following list elements.

Name

This attribute specifies the name of the RT-Auto object.

Attribute Type: BSTR

Interface Pointer

This attribute specifies the local address of the ICBARTAuto interface of the RT-Auto object.

Attribute Type: Interface Pointer

ICBAState

This optional attribute specifies the state interface of the object. It is a structured attribute composed of the following attributes.

State

This attribute specifies the operating state of the Logical Device.

Attribute Type: STATEDEF

The allowed operating states are:

Non Existent

The device is without power, communication is not possible.

Defect

There is a malfunction in PDev or LDev that cannot be eliminated without intervention. Typical defects are hardware or firmware faults. A restart of the LDev is only possible by cycling the power off and back on. Depending on the type of defect, communication with the LDev may be restricted.

Initializing

The LDev performs its initialization after power off/on or reset. The LDev stays in this state until the automation device holds all necessary parameter or configuration data (e.g. self-containing or via download).

Ready

The LDev object is ready for operation; the inner activity of the RT-Auto objects is stopped. The outputs of the LDev object are in a safe state.

Operating

The LDev object performs its normal operation. The RT-Auto objects are free with respect to their inner activity.

NOTE 1 The defined operating states may be extended with substates if necessary. The substates may be conveyed by means of manufacturer specific extensions deploying the CBACustom interface.

NOTE 2 The client may wish to be automatically informed about state changes of a LDev. Therefore, it registers itself by means of a AdviceState service delivering an Interface Pointer referencing a client specific StateEvent interface. This interface is beyond the scope of this standard. However, it is assumed that the client provides an "OnStateChanged" service with the "NewState" and "OldState" as service parameters that the server calls in case of a state change.

ICBATime

This optional attribute specifies the time interface of the object. It is a structured attribute composed of the following attributes.

Time

This attribute specifies the time and date of the Logical Device.

Attribute Type: date

ICBABrowse

This attribute specifies the browse interface of the object. It is a structured attribute composed of the following attributes.

Count

This attribute specifies the number of elements in the browse database.

Attribute Type: LONG

ICBAGroupError

This attribute specifies the group error interface of the object. It is a structured attribute composed of the following attributes.

NOTE The client may wish to be automatically informed about group error changes. Therefore, it registers itself by means of the AdviceGroupError service delivering an Interface Pointer referencing a client specific GroupErrorEvent interface. This interface is beyond the scope of this standard. However, it is assumed that the client provides an "OnGroupErrorChanged" service with the "NewState" and "OldState" as service parameters that the server calls in case of a group error change.

List of ICBACustom

This optional attribute specifies a list of one or more custom interfaces of the object and shall be defined according to manufacturer specific extensions (e.g. to deliver substates of the LDev). The number of custom interfaces is not limited and the actual contents is beyond the scope of this standard.

List of Custom Attributes

This optional attribute specifies a list of possible attributes per custom interface. They shall be defined task specific by the manufacturer and are beyond the scope of this standard.

List of Custom Services

This optional attribute specifies a list of possible services per custom interface. They shall be defined task specific by the manufacturer and are beyond the scope of this standard.

Dispatchable

This attribute specifies the availability of type information for the FAL Class that is provided by the IDispatch common interface. The Logical Device class provides that kind of information.

Attribute Type: Boolean

Allowed values: 1

15.3.2.2.3 Logical Device Behavior

15.3.2.2.3.1 Operating state

The operating state graph shown in Figure 170 describes the state behavior of the LDev. The ICBASState interface supports the reading of the current operating state by means of the get_State service. State transitions can be triggered by means of the services Activate, Deactivate, and Reset. The meaning of the particular states is specified in 15.3.2.2.2.2.

The operating state of the LDev affects the behavior of the RT-Auto objects. The RT-Auto objects shall only be active (“running”) in the LDev state “Operating”. In all other states the activity of the RT-Auto objects is stopped. Therefore, the outputs of the LDev related RT-Auto objects shall be in the safe state.

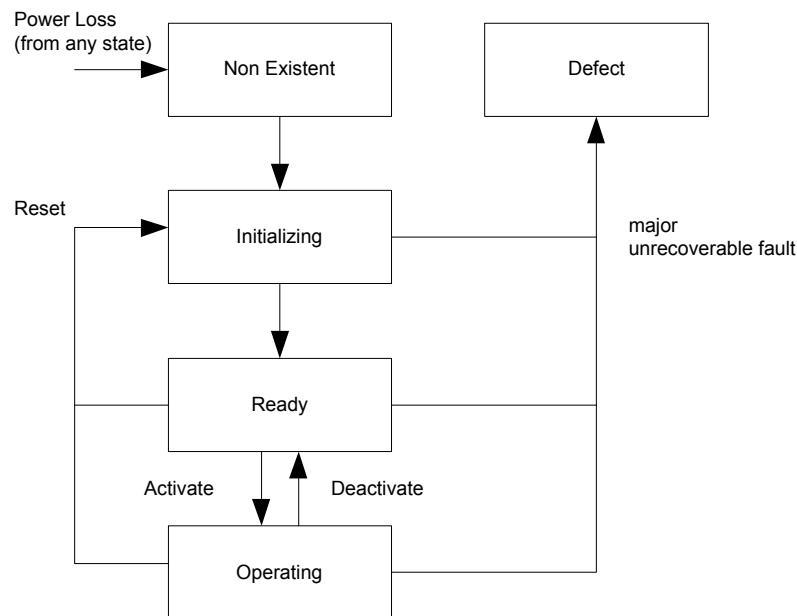


Figure 170 — Operating state block diagram

NOTE The defined operating states may be extended with substates if necessary.

15.3.2.2.3.2 Diagnosis

The common diagnosis specifies the diagnoses contents and the access of the diagnosis information via services at the ICBAGroupError interface. Figure 171 shows the common diagnoses model.

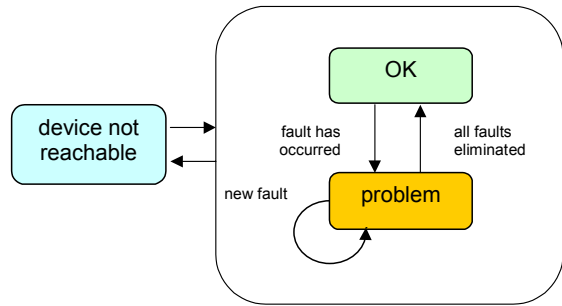


Figure 171 — Device status model for the common diagnosis

The common diagnosis is the top level diagnosis information of the device. On this level, there is no further categorization in corresponding error types.

NOTE The common diagnosis of a device corresponds for example to a common fault LED of a controller.

The common diagnosis can be fetched via the GroupError service. This service reads the current error status and a change indicator referred to as cookie. The cookie can be used by the client to determine the changing of the group error. The cookie itself shall be modified by the server object (e.g. increment) if the group error changes.

NOTE An external client may not derive any information from the internal cookie generation procedure. The cookie is not hold persistent over the power off of the unit, i.e. the same cookie can appear before power off and after power off by different error states. Since the ICBAGroupError interface instance is invalid after power on, a client should easily be able to detect this condition.

15.3.2.2.4 Logical Device Service Specification

15.3.2.2.4.1 get_Name

The get_Name service reads the name of the Logical Device. The name is used for navigation. The arguments of the service are shown in Table 653. This service reads the attribute Name.

Table 653 – get_Name

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the name of the Logical Device.

Parameter Type: BSTR

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.2 get_Producer

The get_Producer service reads the name of the device producer. The arguments of the service are shown in Table 654. This service reads the attribute Producer.

Table 654 – get_Producer

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the name of the producer.

Parameter Type: BSTR

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.3 get_Product

The get_Product service reads the name of the device type in a producer-specific and producer-unambiguous way. The arguments of the service are shown in Table 655. This service reads the attribute Product.

Table 655 – get_Product

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the name of the product.

Parameter Type: BSTR

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.4 get_SerialNo

The get_SerialNo service reads the serial number of the LDev. The arguments of the service are shown in Table 656. This service reads the attribute SerialNo.

Table 656 – get_SerialNo

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the SerialNo.

Parameter Type: VARIANT

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.5 get_ProductionDate

The get_ProductionDate service reads the production date of the LDev. The arguments of the service are shown in Table 657. This service reads the attribute ProductionDate.

Table 657 – get_ProductionDate

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the ProductionDate.

Parameter Type: date

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.6 Revision

The Revision service reads the revision level of the LDev (i.e. the software version) as a major and minor revision. The arguments of the service are shown in Table 658.

Table 658 – Revision

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pMajor			M	M(=)
pMinor			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pMajor

This parameter contains the major part of the revision.

Parameter Type: short

pMinor

This parameter contains the minor part of the revision.

Parameter Type: short

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY

15.3.2.2.4.7 get_ACCO

The get_ACCO service supports the navigation procedure and reads the FAL Class attribute ACCO containing a reference to the ACCO object. This reference is deployed to address ACCO services. The arguments of the service are shown in Table 659.

Table 659 – get_ACCO

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
ppACCO			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

ppACCO

This parameter contains the address of the IUnknown interface of the referenced ACCO object. Other ACCO interfaces can be obtained using the QueryInterface service.

Parameter Type: Interface Pointer

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.8 get_RTAuto

The get_RTAuto service reads the reference to a RT-Auto object via ist name. The arguments of the service are shown in Table 660. This service reads one element of the FAL Class attribute List of the RT-Auto object.

Table 660 – get_RTAuto

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Name	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
ppAuto			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

Name

This parameter contains the name of the referenced RT-Auto object.

Parameter Type: BSTR

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

ppAuto

This parameter contains the address of the ICBARTAuto interface of the referenced RT-Auto object. Other RT-Auto interfaces can be obtained using the QueryInterface service.

Parameter Type: Interface Pointer

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBALogicalDevice interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, CBA_E_UNKNOWNOBJECT

15.3.2.2.4.9 get_State

The get_State service reads the current operating state of the LDev. The arguments of the service are shown in Table 661. This service reads the FAL Class attribute State.

Table 661 – get_State

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the state of the Logical Device.

Parameter Type: STATEDEF

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.10 Activate

The Activate service is used to set the LDev to the “Operating” state. This is only possible if the LDev is in “Ready” state and there is no specific startup obstacle. The arguments of the service are shown in Table 662.

Table 662 – Activate

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_NOTAPPLICABLE, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.11 Deactivate

The Deactivate service is used to set the LDev to the “Ready” state. This is only possible if the LDev is in “Operating” state. The arguments of the service are shown in Table 663.

Table 663 – Deactivate

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_NOTAPPLICABLE, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.12 Reset

The Reset service is used to reset the LDev to the “Initializing” state. This is only possible if the LDev is not in “Defect” state. The arguments of the service are shown in Table 664.

Table 664 – Reset

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_NOTAPPLICABLE, E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.13 AdviseState

The AdviseState service requests the server to inform the client if the attribute State of the LDev changes.

NOTE The client may wish to be automatically informed about state changes of a LDev. Therefore, it registers itself by means of a AdviceState service delivering an Interface Pointer referencing a client specific StateEvent interface. This interface is beyond the scope of this standard. However, it is assumed that the client provides an "OnStateChanged" service with the "NewState" and "OldState" as service parameters that the server calls in case of a state change.

The arguments of the service are shown in Table 665.

Table 665 – AdviseState

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
pStateEvent	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pCookie			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBASState interface of the requested object instance.

Parameter Type: Interface Pointer

pStateEvent

This parameter contains the address of the event interface.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBASState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

pCookie

This parameter contains the event interface handle.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBASState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.14 UnadviseState

The UnadviseState service clears the event advise. The arguments of the service are shown in Table 666.

Table 666 – UnadviseState

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Cookie	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

Cookie

This parameter contains the handle to the event interface.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAState interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Allowed values: E_INVALIDARG, E_OUTOFMEMORY, CBA_E_INVALIDCOOKIE

Parameter Type: HRESULT

15.3.2.2.4.15 get_Time

The get_Time service reads the current time of the LDev. The arguments of the service are shown in Table 667. This service reads the attribute Time.

Table 667 – get_Time

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the current time of the LDev.

Parameter Type: date

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY, E_NOTIMPL

15.3.2.2.4.16 put_Time

The put_Time service sets the current time of the LDev. The arguments of the service are shown in Table 668. This service writes the attribute Time.

Table 668 – put_Time

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
NewVal	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

NewVal

This parameter contains the new time for the LDev.

Parameter Type: date

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBATime interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_TIMEVALUEUNSUPPORTED, E_INVALIDARG, E_OUTOFMEMORY, E_NOTIMPL

15.3.2.2.4.17 get_Count

The get_Count service reads the number of elements (attribute count) in the browse database. The arguments of the service are shown in Table 669. This service reads the attribute Count.

Table 669 – get_Count

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the number of elements.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.18 BrowseItems

The BrowseItems service reads the list of available RT-Auto objects. The arguments of the service are shown in Table 670.

Table 670 – BrowseItems

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Offset	M	M(=)		
MaxReturn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pltem			M	M(=)
pDataType			U	U(=)
pAccessRight			U	U(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Offset

This parameter specifies the offset of the first item in the List of RT-Auto that shall be returned.

Parameter Type: long

MaxReturn

This parameter specifies the maximum number of RT-Auto items that shall be returned.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pltem

This parameter contains the name or list of names of the available RT-Auto objects.

Parameter Type: VARIANT

Allowed values: Tag=VT_SAFEARRAY_BSTR

pDataType

This parameter is reserved and shall be filled with the corresponding result code.

Parameter Type: VARIANT

Allowed values: Tag=VT_ERROR, Value=DISP_E_PARAMNOTFOUND.

pAccessRight

This parameter is reserved and shall be filled with the corresponding result code.

Parameter Type: VARIANT

Allowed values: Tag=VT_ERROR, Value=DISP_E_PARAMNOTFOUND.

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.19 GroupError

The GroupError service reads the error state of the LDev. The returned cookie specifies whether or not the group error state of the device has changed since the last invocation. The arguments of the service are shown in Table 671.

Table 671 – GroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
pMagicCookie			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the group error state of the LDev.

Parameter Type: GROUPEXCEPTIONDEF

pMagicCookie

This parameter contains the group error state change indicator.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.20 AdviseGroupError

The AdviseGroupError service is used to advise the server to issue an OnGroupErrorChanged service (ICBAGroupErrorEvent interface at the clients site) if the group error changes. The arguments of the service are shown in Table 672.

Table 672 – AdviseGroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
pGroupErrorEvent	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pCookie			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

pGroupErrorEvent

This parameter contains the address of the event interface.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

pCookie

This parameter contains the event interface handle.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.2.2.4.21 UnadviseGroupError

The UnadviseGroupError service clears the event advise specified by cookie. The arguments of the service are shown in Table 673.

Table 673 – UnadviseGroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Cookie	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

Cookie

This parameter contains the handle to the event interface.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY, CBA_E_INVALIDCOOKIE

15.3.3 ACCO ASE

15.3.3.1 Overview

The ACCO (Active Control Connection Object) objects implements a configurable interconnection of RT-Autos.

15.3.3.2 ACCO class specifications

15.3.3.2.1 Formal Definition

- | | | | |
|------|-----|----------------|--------------------|
| 1. | (m) | Key Attribute: | Implicite |
| 2. | (m) | Attribute: | ICBAAccoMgt |
| 2.1 | (m) | OpsService: | AddConnections |
| 2.2 | (m) | OpsService: | RemoveConnections |
| 2.3 | (m) | OpsService: | ClearConnections |
| 2.4 | (m) | OpsService: | SetActivationState |
| 2.5 | (m) | OpsService: | GetInfo |
| 2.6 | (m) | OpsService: | GetIDs |
| 2.7 | (m) | OpsService: | GetConnections |
| 2.8 | (m) | OpsService: | ReviseQoS |
| 2.9 | (m) | Attribute: | PingFactor |
| 2.10 | (m) | OpsService: | get_PingFactor |
| 2.11 | (m) | OpsService: | put_PingFactor |
| 2.12 | (m) | Attribute: | CDBCcookie |
| 2.13 | (m) | OpsService: | get_CDBCcookie |
| 3. | (m) | Attribute: | ICBAAccoServer |
| 3.1 | (m) | OpsService: | Connect |
| 3.2 | (m) | OpsService: | Disconnect |
| 3.3 | (m) | OpsService: | DisconnectMe |
| 3.4 | (m) | OpsService: | SetActivation |
| 3.5 | (m) | OpsService: | Ping |
| 4. | (m) | Attribute: | ICBAAccoCallback |
| 4.1 | (m) | OpsService: | OnDataChanged |
| 5. | (m) | Attribute: | ICBAAccoSync |
| 5.1 | (m) | OpsService: | ReadItems |
| 5.2 | (m) | OpsService: | WriteItems |
| 5.3 | (m) | OpsService: | WriteItemsQCD |
| 6. | (o) | Attribute: | ICBAGroupError |
| 6.1 | (m) | OpsService: | GroupError |
| 6.2 | (m) | OpsService: | AdviseGroupError |
| 6.3 | (m) | OpsService: | UnadviseGroupError |
| 7. | (m) | Attribute: | Dispatchable |

15.3.3.2.2 Attributes

ICBAAccoMgt

This attribute specifies the configuration data base interface of the object. It is a structured attribute composed of the following attributes.

PingFactor

This attribute specifies the current PingFactor that is used for determining the maximum dead time of a provider (PingFactor * minimum QoS).

Attribute Type: unsigned short

Allowed values are shown in Table 674.

Table 674 – PingFactor values

PingFactor	Description
0 – 65535	valid values
10	default value
0	disable pinging

CDBCcookie

This attribute specifies the Configuration Data Base change indicator.

Attribute Type: long

ICBAAccoServer

This attribute specifies the provider for connections interface of the object. It is a structured attribute composed of the following attributes.

ICBAAccoCallback

This attribute specifies the consumer for connections interface of the object. It is a structured attribute composed of the following attributes.

ICBAAccoSync

This attribute specifies the synchronous reading or writing interface of the object. It is a structured attribute composed of the following attributes.

ICBAGroupError

This attribute specifies the common diagnosis interface of the object. It is a structured attribute composed of the following attributes.

NOTE The client may wish to be automatically informed about group error changes. Therefore, it registers itself by means of the AdviceGroupError service delivering an Interface Pointer referencing a client specific GroupErrorEvent interface. This interface is beyond the scope of this standard. However, it is assumed that the client provides an "OnGroupErrorChanged" service with the "NewState" and "OldState" as service parameters that the server calls in case of a group error change.

Dispatchable

This attribute specifies the availability of type information for the FAL Class that is provided by the IDispatch common interface. The ACCO ASE does not provide that kind of information and the Dispatchable attribute shall contain the value 0.

Attribute Type: Boolean

15.3.3.3 ACCO ASE behavior

15.3.3.3.1 Interconnecting RT-Autos – ACCO

The ACCO (Active Control Connection Object) is a general purpose FAL Class that shall have only one object instance per LDev. The main purpose of this class is to provide optimized and dynamically configurable (during runtime) data exchange between automation task specific attributes (inside custom interfaces) of the RT-Auto objects. These class attributes are not part of this specification because they are automation task specific. The ACCO ASE defines a way to collect such attribute values from the local RT-Auto objects (local matter or local object behavior) and provide the attribute values to remote RT-Auto objects via the remote ACCO object. Therefore, the ACCO ASE introduces a consumer and provider facet. The ACCO ASE offers connection management services to define such connections during runtime. It conveys

data between RT-Auto objects using connection information. The access to the RT-Auto object data items is local matter.

A RT-Auto ASE offers data (attributes or properties) and events (services or methods) as information. Consequently, the ACCO ASE offers data connections and event connections.

15.3.3.3.2 Architecture

The ACCO ASE is responsible for the following tasks:

- Establishing configured or dynamically created (for OS connections, for example) connections via symbolic addressing with the properties
Many to many - this means that an information may also be distributed to several consumers; a consumer can receive information from different providers
A component producer need not program anything for the connections
- Transportation of data and events along these connections with the configured quality (Quality of Service)
- Ensuring the transparency of the connection between RT-Auto objects (local and remote are handled identically)
- Implementing failure strategies for data and event connections (specific to the individual runtime environments)
- Optimizing the transport of data and events
- Enabling various communication procedures (and their expansions) to be used - also under the aspect of real-time communication procedures
- Implementing simple operators to data and event connections

The following terms are used in the description:

A Provider is the source of the data. It produces the information of an connection.

A Consumer is the sink of the data. It receives the information of an connection.

The basic rule of an connection defines that the consumer shall always ensure that the information is obtained; the provider merely makes the information available. This has an effect on the configuration of the connections (always at the consumer's device) and on the fault strategy.

Figure 172 shows the internal structure of the ACCO ASE and the operation in relation to object instances of the RT-Auto ASE.

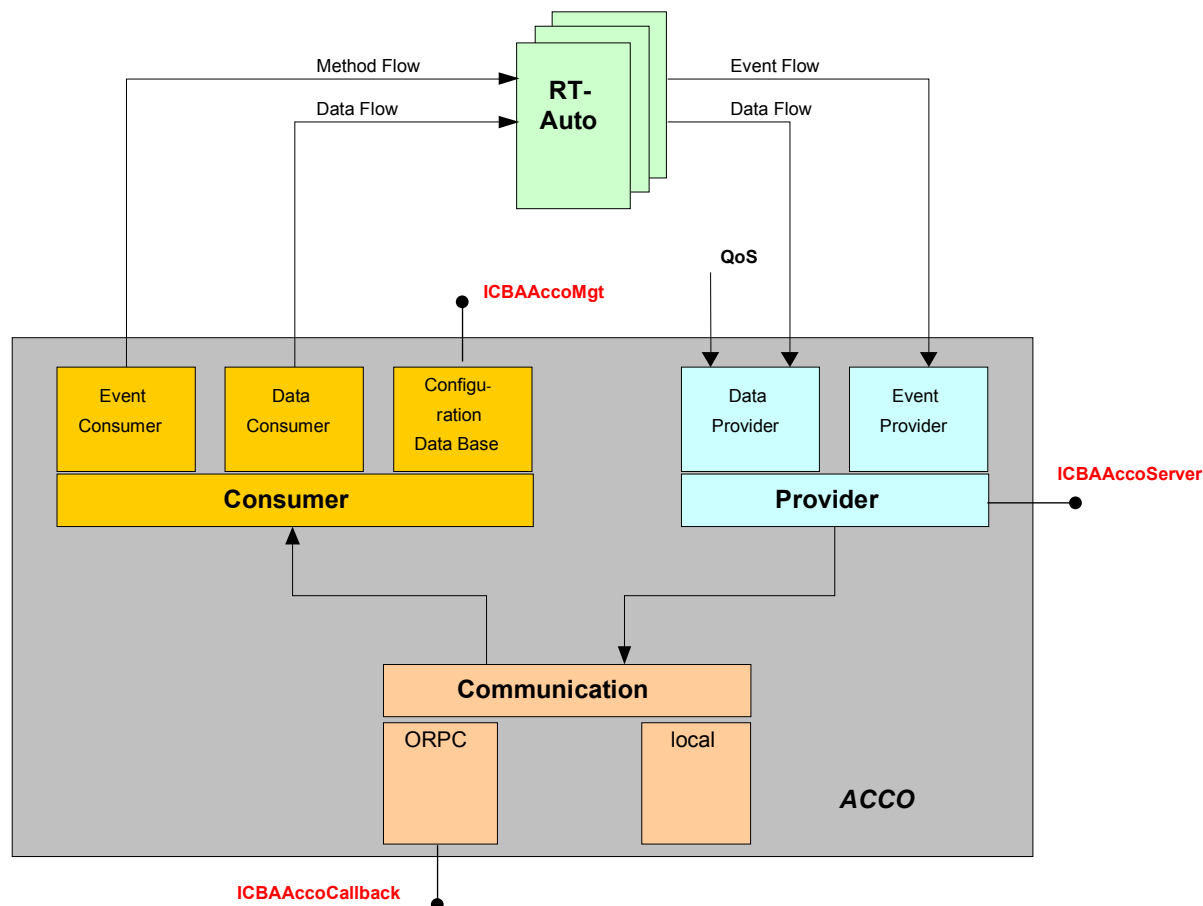


Figure 172 — ACCO ASE structure

The ACCO ASE consists of the following components:

The Consumer provides the functions required for configuring connections and connection sinks.

The Provider provides the functions required for connection sources.

The Communication abstracts the deployed communication path for provider and consumer

15.3.3.3.3 Rules for Connections

An connection defines a link between a data or event source and the corresponding data or method sink and shall obey the following rules:

- An connection can only be established between attributes (properties), services (events and methods) at ICBACustom interfaces of RT-Auto objects.
- An connection can only be established between partners of the same signature:

Data connections shall use identical data types in source and sink (e.g. signed long). A data connection can only be established between attributes (properties) at ICBACustom interfaces of RT-Auto objects. The source of the connection shall provide a service to read the attribute (IDL attribute [propget]). The sink of the connection shall provide services to write and read (for debug purposes) the attribute (IDL attributes [propput] and [propget]).

Event connections shall use identical parameter profiles of operational services. It means that the service parameter shall be equal according their data types and IDL attributes (such as [in] or [out]). The operational service shall be an event at the source and a method at the sink. For example an connection of the event *ReactorFull([in] long a)* and the method *Drain([in] long z)* can be established.

NOTE The relation between the RT-Auto object and the ACCO object receiving data and events and issuing data and methods is a local matter.

- The Engineering System should only load connections of identical signature into the consumer. A partial check for signature differences is asynchronously performed by the provider when the connection is established later.
- Merging data (i.e. connecting several data sources with one data sink) is not permitted. Routing multiple events to one method is permitted.
- Splitting a source to several sinks is always possible (for data and for events).
- Interconnecting a source with itself being the sink is not permitted.

NOTE The definition of the type 10 event connection is very flexible and variable supporting also a link between single event items instead of linking whole interfaces as other models force to do.

15.3.3.3.4 Sequence model

15.3.3.3.4.1 Establishing a Connection

An connection is established by transferring the corresponding configuration information into the Configuration Data Base of the consumer (connection sink) using the AddConnections service. Depending on the initial state this information is conveyed to the provider by means of the Connect service of the ICBAAccoServer interface.

NOTE In future extensions the consumer may negotiate with the provider the best communication path according to the QoS.

15.3.3.3.4.2 Productive Operation of Data Connections

The provider transmits the productive data of the connection via the communication channel to the consumer using the OnDataChanged service of the ICBAAccoCallback interface.

A data connections works according to the block diagram in Figure 173.

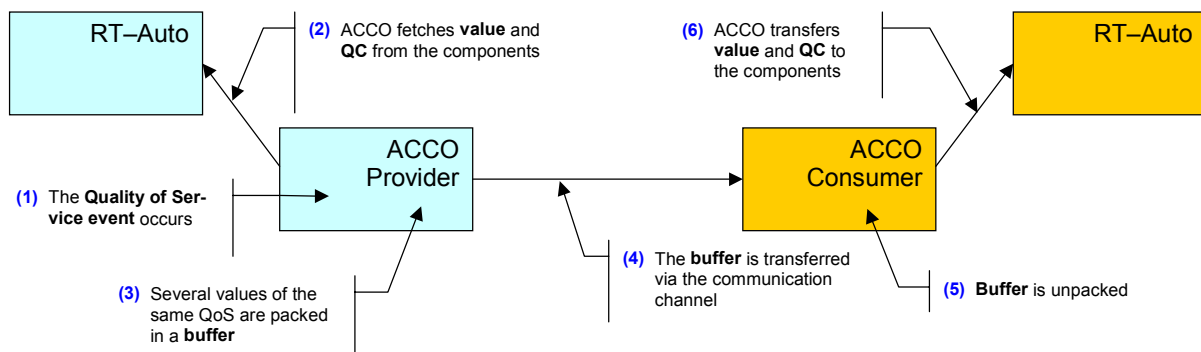


Figure 173 — Productive operation of data connections

The productive operation is performed in the following steps:

1. An event for the configured Quality of Service (e.g. a time interval of 100 ms expires) occurs at the provider. The way in which that is signaled to the provider depends on the deployed QoS.
2. The connection data pending for this QoS are read via the local access methods at the RT-Auto objects.
3. The connection data are filtered for changes and packed into a buffer.
4. The data item is transferred to the consumer via the communication channel.
5. The buffer is unpacked.
6. The connection data is transferred to the RT-Auto objects via the local access methods.

The productive operation shall only take place if the state of the LDev is **Operating**.

NOTE Since the operating states of individual LDevs are not coordinated with each other, provider and consumer may be in different operating states.

When the state of the LDev changes from **Operating** to **Ready**, the values of the provider and the corresponding Quality Code are transferred for the last time.

15.3.3.3.4.3 Productive Operation of Event Connections

The provider transmits the productive data of the connection via the communication channel to the consumer using the OnDataChanged service of the ICBAAccoCallback interface.

An event connection works according to the block diagram in Figure 174.

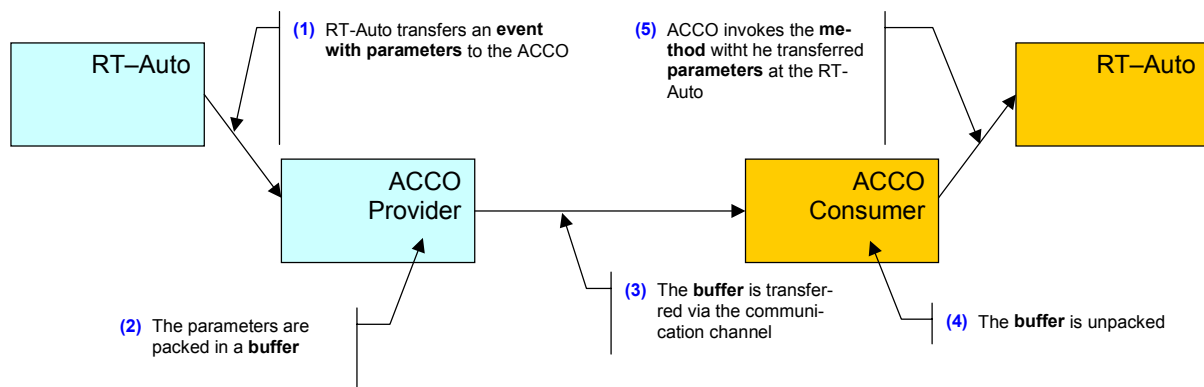


Figure 174 — Productive operation of event connections

Productive operation is performed in the following steps:

1. The RT-Auto object transfers the event to the provider.
2. The event parameters are transferred to the consumer via the communication channel.
3. The consumer deploys these parameters to invoke the method of the RT-Auto object.

The productive operation shall only take place if the state of the LDev is **Operating**.

NOTE Since the operating states of individual LDevs are not coordinated with each other, provider and consumer may be in different operating states. However, the inner activities of the RT-Auto object are inactive in the Ready state no events can be triggered anyway.

15.3.3.3.5 Configuration Information

The configuration information describes the connections between the individual RT-Auto objects. This information is generated during the configuration of the application. It is transferred via the ICBAAccoMgt interface. Only the ACCO object of a configuration sink obtains the configuration data.

A data or event connection between two RT-Auto objects is characterized by the following information:

- Identification of the connection source (i.e. the identification of the property or event at the RT-Auto object of a specific LDev),
- Identification of the connection sink,
- Initial state of the connection (active or inactive); i.e. does the connection run immediately or only after an explicit activation.

A data connection requires the following additional information:

- Quality of Service is characterized by type and qualifier; i.e. when data are to be transferred,
- Epsilon value for delimiting the transmission frequency, e.g. for analog values,
- Substitute value, for applying specific values in case of a failure of the communication relationship.

If the initial state of the connection is inactive, the connection is actually established but productive operation will not take place before the connection has been activated by the SetActivationState service provided by the ACCO management interface.

15.3.3.3.6 Quality of Service

The Quality of Service (QoS) specifies the quality of a data item as it is available at the sink of a data connection.

The following standard QoS have been defined within the scope of the type 10 specification:

Cyclically every n (milli)seconds for changes:

- The provider shall test the connection source at intervals of at least n (milli)seconds for changes that are outside the range that is defined by the epsilon value. The value shall only be transferred to the consumer if the changes are outside this range.
- This QoS specifies the maximum latency that elapses from the moment the value of an connection source changes until the moment this change is transmitted.
- If a reconstruction of a frequency is desired, the user shall apply the sampling theorem for this purpose. This means that a QoS of at least half the frequency shall be specified.
- This QoS is mainly used to limit the transfer rate of a data item and to decouple two resources (the input values exist when an object is activated). This reduces the network load, in particular with slowly changing data.
- The value shall always be conveyed to the consumer if the quality code (QC) of a QC-aware RT-Auto object changes, even if the value itself has not changed.

On Demand

- The value is transferred when it is required for activating the RT-Auto object. This QoS is only deployed in local communication.

A QoS is not defined for event connections. The rule "best effort" applies here.

The definition in Table 675 applies to the QoSType and QoSValue values that are required at the ACCO class interfaces.

Table 675 – QoS Types and Values

Quality of Service (QoS)	QoSType	QoSValue
Cyclically every n milliseconds for changes	0	Number of msec
Cyclically every seconds for changes	1	Number of sec
On demand	10	not relevant

With respect to the QoS, a device has subset capabilities in the following points:

Supported QoS types

Minimum resolution of the QoS types

Maximum number of groups (i.e. the number of different QoS types and QoS values it supports in parallel).

15.3.3.3.7 Dead Band and Epsilon Value

For a QoS that defines a transmission only upon a change, an epsilon value is additionally transferred when an connection is established. The dead band defines a new transfer that takes place only after a percentaged deviation of the value from the variable's range, exceeding low limit or high limit.

The change is only passed on if the absolute change of the variable exceeds this value. This means that the following inequality shall be fulfilled:

$$\text{ABS}(\text{Last_Cached_Value} - \text{Current_Value}) > \text{Deadband} * (\text{High_Limit} - \text{Low_Limit})$$

This is also used for reducing data from the network. It mainly prevents an "oscillation" of a floating point value from being transferred to the line.

Since the right-hand side of the above inequality is a constant value, the epsilon value is defined in the following way in order to delimit the multitude of parameters to a certain extent.

$$\epsilon = \text{Deadband} * (\text{High_Limit} - \text{Low_Limit})$$

When the connection is set up, merely the epsilon value (calculated by the Engineering System) is transferred. It describes the absolute change of the value of the variable that leads to a new transfer. To transfer the value again, the following inequality shall therefore be fulfilled:

$$\text{ABS}(\text{Last_Cached_Value} - \text{Current_Value}) > \epsilon$$

An empty value is used for the epsilon value if all data changes shall be transferred. An epsilon value may also not be expedient for a given connection – which is true for structures, arrays and strings. In both cases the transmitted variant is then of the data type VT_EMPTY, which leads to transmission on every change.

Checking a value against the epsilon value can be omitted in the case of local connections. For a corresponding destination platform this permits a swift optimization of the connection of local data.

15.3.3.3.8 ACCO ASE – Consumer facility

15.3.3.3.8.1 Overview

The consumer implements the configuration of connection and the connection sinks. It has the following tasks:

- It shall manage the configuration information and provides for its persistence.
- It shall establish the connections based on the configuration information.
- It shall maintain the connection sinks of data and event connections – initiated by the reception of new connection data from a provider. It routes the incoming connection data to the corresponding methods in the interconnected RT-Auto objects.
- It shall monitor the configured provider for failures and responds accordingly.
- It shall initiate a specific reaction to a corresponding Quality Code.

NOTE It initiates a bus-specific optimization procedures and negotiates an optimized communication channel with the provider for this purpose in future extensions.

15.3.3.3.8.2 Establishing Connections

By means of the AddConnections service the configuration process downloads the configuration information into the individual ACCO ASEs. Each ACCO ASE only gets the section of the configuration information for which it controls the connection ends. It does not obtain the configuration information of the entire system.

The Engineering System shall group its AddConnections service calls by providers and deployed QoS to avoid extensive sorting at the consumer.

The consumer initializes its internal data with the connection sinks (i.e. an identification of the method that is to be called that was resolved in an internal form). A ConsumerID that is assigned by the consumer unambiguously identifies (within the local scope of this ACCO ASE) the internal data of a configured connection sink. This can be an array index, for example. The unambiguity shall be ensured locally by the implementation.

The connection sinks are initialized with value and QC regarding to the substitute value strategy (see subclause 15.3.3.3.8.9 for details).

The consumer shall respond with an error result to an AddConnection service in case of an error during the setup of the connections. The client can synchronously see in the error results that the connection could not be set up. Consequently, resources have not been created either for this connection.

The consumer shall deny the request for a further data connection to an already used connection sink when the connection is established.

NOTE A dual connection would mean that several sources write to a destination data item and is therefore a consistency problem.

Next - and asynchronously to the AddConnections service - the consumer shall establish the connections to the configured providers. It shall send the Connect service (via ICBAAccoServer interface) to the provider. The interconnecting sources, QoS and the ConsumerIDs that were set up beforehand are transferred as parameters. Connection errors for individual connections are stored in the ACCO ASE of the consumer. These errors can be read by using the GetIDs service of the ACCO management interface.

Irrespective of the error that was returned at the connect service, a new connect will be attempted after a certain time has elapsed. This is done even if the object or a specific property could not be found. An error is returned at GroupError until the fault(s) has (have) been eliminated. In addition, an event is produced upon each change. Faults of the ORPC model can also be considered as one fault. For newly loaded connections, the ICBAGroupErrorEvent interface at the ACCO only reports an error after the first connect attempt and its general or partial failure. A newly loaded connection will carry the state CBA_S_ESTABLISHING during this first establishment phase (see also subclause 15.3.3.3.11).

NOTE For the first stage, the time that elapses until the connect is retried is set to 30 seconds (analog to the time-out time for ORPC calls). For a further stage it shall be considered whether this time will be configured together with the time-out time (as one value) or whether several separate values are to be provided.

15.3.3.3.8.3 Changing Connections

An connection can be changed at any time and independently of the operating state (as long as the device is operational) of the programmable controller (i.e. it can be set up, cleared, or changed in its activation state). However, a synchronization has not been defined that permits simultaneous changes of connections concerned on several devices.

15.3.3.3.8.4 Clearing Connections

The DeleteConnections service (ICBAAccoMgt) can be used for clearing connections. The ClearConnections service (ICBAAccoMgt) deletes all consumer connections. In this process, the acceptance of the configured values is deactivated immediately and the configured substitute value is applied.

If an error result is returned by the response for clearing an connection, the connection could not be cleared. This means that this connection basically did not exist.

By means of the Disconnect service (ICBAAccoServer) the corresponding connections will be cleared in the provider. Values for this connection that are sent via the communication channel shall be ignored until this connection can be cleared in the provider. The connection state is inactive. Hence, the consumer ID that was allocated to this closed connection shall not be allocated anew in the meantime.

15.3.3.3.8.5 Activating and Deactivating Connections

Each connection carries a status that defines whether it is active or inactive. The initial status of the connection is transferred via the AddConnections service (ICBAAccoMgt) when the connection is established. The status can be changed using the SetActivationState service (ICBAAccoMgt).

Although an inactive connection has already been established in the resources of consumer and provider, connection data is not transferred. With respect to the data, it has the same effect as if it were not established.

If an connection is inactive, the acceptance of the configured values is deactivated synchronously to the invocation of the method in the consumer, and the configured substitute value is applied.

The connection status is transferred to the provider by means of the SetActivation service (ICBAAccoServer).

15.3.3.3.8.6 Productive Operations of Connections

The consumer receives data packets via the established communication channel, unpacks the individual connection data and deploys the information to invoke the corresponding method with the transferred parameters and/or stores the corresponding property. The format of the transmitted packet is specified in the service description of the OnDataChanged service.

15.3.3.3.8.7 Version of an Connection

To assist the Engineering System on keeping the local existing connection information in an actual state, and simultaneously to reduce the information that is to be exchanged between the Engineering System and the runtime, there is a version change procedure of the connections. This version procedure enables all partners to detect changes in the connections.

The ACCO ASE provides two services at the ICBAAccoMgt interface that can be used for loading connection information into the Engineering System:

The GetConnections service transfers the entire connection information - i.e. the rather static information such as connection source and connection target, and the dynamic information (the current state, for example).

The GetIDs service, merely transfers the dynamic information. The service request assumes that the static information is already stored in the Engineering System.

Both services provide a special service parameter refer to as version per ConsumerID that enables the client (e.g. the Engineering System) to recognize whether still the same connection is involved when the service GetIDs is invoked. The parameter version should be stored locally in the client (e.g. Engineering System). When a GetID service is requested by the client, it can be validated whether the information is still related to the same connection - in this case the transferred version is identical to the stored one.

To support this, the ACCO ASE shall change the version, when

- a new connection is set up (AddConnections), or
- an connection is cleared (RemoveConnections, ClearConnections).

The ACCO ASE shall take precautions that ensures that the version of a newly established connection differs from the version of an connection that was established earlier under the same ConsumerID.

NOTE Since an unsigned short is used for the version, "earlier" is rather be interpreted as back in the past for a sufficiently long time. This can also be reached by a retention of the versions for unassigned (i.e. established and subsequently cleared) connections.

15.3.3.3.8.8 Persistence

The restart without an explicit intervention of the client (e.g. Engineering System) shall be supported by the automation device. Hence, the consumer as part of the ACCO ASE shall maintain the connection information as non volatile when power is switched off. The Physical Device class provides the Save service within the ICBAPersist interface to request the persistent save (e.g. in flash memory or EEPROM) for all connection information of the whole automation device, independent, which client was adding the connection before. There is no concept of an initial state of an connection, always the current active information is saved.

If the connection to the automation device was interrupted during the connection download, the client (e.g. Engineering System) shall check the current state of the connections when the connections are re-established.

NOTE The automation device could be switched off during the interruption and could thus have forgotten the newly entered connections that had not yet been saved. This is especially true for deleting connections via the services RemoveConnections or ClearConnections of the ICBAAccoMgt interface without saving them to the persistence data base. Powering off the device and power it on again "forgets" all the changes in between.

The usage model for changing the connections is therefore to do all changes and then to "save" them.

To support this model, each connection carries an identification, whether the connection is currently backed up in the persistence. This information can be fetched with the GetConnections service. The Persistence information has three states:

- CBAVolatile – the connection will never be persistent.
- CBAPendingPersistent – the active information of the connection is not equivalent to the information in the persistence, i.e. a call to the Save service of the ICBAPersist interface is needed to bring them in sync.
- CBAPersistent – the active information of the connection is equivalent to its information in the persistence.

If a connection is established via the AddConnections service of the ICBAAccoMgt interface, all connections which where requested to be persistent, carry the information CBAPendingPersistent until the Save service of the ICBAPersist interface stores the current state into the persistence data base. Then all connections are set to CBAPersistent. The next change to an connection via the SetActivationState service of the ICBAAccoMgt interface brings the connection back to the state CBAPendingPersistent.

If an error appears during the Save service, all persistent connections of the currently stored LDev will be set to CBAPendingPersist and the configuration data base will be set to invalid. This leads to an error returned by the Save service, nevertheless the save continues for all other LDevs. The client saving the connections is able to detect the failed connections via CBAPendingPersistent, he is forced to retry the operation (whereas the error is likely to reappear).

The information that is to be kept persistent includes about all request parameters of the AddConnections service and, in addition, the ConsumerID defined above - in order to enable established connections to be identified via the same ID after a restart. Therefore a client (e.g. Engineering System) can store this ConsumerID as a valid key.

This requires the following connection information to be retentive:

Currently valid ping factor and
for each connection

- Consumer ID
- Identifier for the provider's LDev
- Identifier for the connection source
- Identifier for the connection sink
- Current status (active or inactive)
- Type and qualifier of the QoS
- Substitute value
- Epsilon
- Version number

NOTE 1 The number of connections a consumer/provider pair is able to establish mainly depends on the memory space available in the target platform. Since it can be assumed, that the non volatile memory space available is usually smaller than the memory space that is managed via the dynamic memory management, the bottleneck is with the consumer.

NOTE 2 In order to permit a swift replacement of a defective module, the persistence information should be stored on a medium that can be swapped when the defective module is replaced (e.g. an EPROM card).

Simultaneous modifying access to the connections is not possible as long as the Save service is outstanding and processed by the server. Therefore, the server shall negative respond incoming AddConnections, RemoveConnections, ClearConnections, and SetActivationState services with CBA_E_PERSISTRUNNING error during this phase. The client application that wishes to install connections should repeat the setup attempt after a short delay. This means that this error should only be returned to the user after several unsuccessful attempts have been made.

15.3.3.3.8.9 Substitute Values

The substitute value (or fail safe value) provide a safe value for the application process at the information sink (consumer) in case the connection or the information source fails. The behavior differs among various application areas, mainly between factory automation (discrete production) and process automation (continuous or batch process).

Depending on the requirements during creating connections, the following basic procedures shall be supported:

- Applying a fixed substitute value ("Substitute Set").
This procedure is mainly used in job production to define a safe state. The default substitute value used in the Engineering System should be "0".
- Holding the last known value ("Last Usable Value").
This procedure is mainly used for the conti/batch process. If possible, this process should not be interrupted. Since values in process control industry change at a rather slow rate, holding the last value is the better solution.

Using the AddConnections service the user configures the substitute value strategy for an connection when the connection is established. A VARIANT is used for transferring the substitute value. The content of the VARIANT - mainly the variant type - determines the substitute value strategy that is valid for this connection:

- Applying a fixed substitute value ("Substitute Set") by a value in VARIANT of the corresponding variant type (e.g. VT_I4 - this includes the safe "0").
- Holding the last known value ("Last Usable Value") by an empty VARIANT of the variant type VT_EMPTY.

15.3.3.3.8.10 Quality Code

15.3.3.3.8.10.1 Overview

A value shall have status information about its state (such as good, bad, simulated, substitute value, ...) that can be used by consumers for reacting and/or displaying the state of this variable.

In the Quality Code concept, an additional attribute is assigned to a value which stores the quality code. The quality code consists of three portions:

- Quality contains the state of quality of data (Bad, Uncertain, Good Non Cascade, Good Cascade)
- Quality Substatus defines a set of substatus values for each quality
- Limits contains limit information for every quality and substatus (OK, low limited, high limited, constant)

15.3.3.3.8.10.2 Using Quality Codes

With respect to the quality codes two classes of component producers can be distinguished:

1. Producers of components that take the quality code (and in most cases the time stamp, too) into account in their components. This is typically the case in the environment of process engineering. This type of component requires an applicative integration of quality control and time stamp to be provided. Based on the quality code that is transferred to the RT-Auto object, this application is able to implement a corresponding strategy. This means it can also take a value that is preconceived in the RT-Auto object as a substitute value.
2. Producers of components that do not take the quality code into account in their components. This is typically the case in the environment of production engineering. This application cannot distinguish whether a valid value or a substitute value has been applied to the component.

On the basis of these two classes, the following two implementation types of RT-Autos are distinguished in type 10 specification:

Quality Code aware RT-Auto ("QC-aware")

At its input side, the RT-Auto object processes the quality codes that are supplied from the outside, and creates a quality code for the properties that are provided at the outside.

Creating and processing quality codes is within the responsibility of the RT-Auto object.

The method in which the quality code is delivered to a QC-aware RT-Auto object shall be defined in a target-specific way. From the type 10 perspective, there is no requirement of being able to read the quality code at an interface of the RT-Auto object. It can be determined via the ICBAAccoSync interface (synchronous reading or writing). The ACCO object shall know, where the Quality Code of the RT-Auto object comes from and/or how it can set it. This is defined in a destination-system-specific way (i.e. it is a function of the system adaptation interface).

Otherwise, this request would require a structure (value and quality code) for a property in order to be able to read or write these two values consistently. This would only permit to connect one QC-aware RT-Auto object with another one.

Quality Code unaware RT-Auto ("QC-unaware")

The RT-Auto object does not know any quality code. Acting as a substitute, the ACCO object creates and processes quality codes and the RT-Auto object is provided with the values that are compatible with the transferred quality code.

Only the Quality Codes listed in Table 676 are generated and used by the type 10 specification. They are a subset of all defined Quality Codes.

Table 676 – Quality Codes

Quality	Sub-status	Value	Comment
Good (NC)	ok	GoodNonCascOk	The value is OK.
Good (C)	local override	GoodCascLocalOverride	The value is forced.
Uncertain	last usable value	UncertainLastUsableValue	The last valid value exists. This means that the last known value is held.
Uncertain	substitute-set	UncertainSubstituteSet	The value is a configured substitute value.
Uncertain	initial value	UncertainInitialValue	The value is an initial value.
Bad	out of service	BadOutOfService	The value is bad since the component is currently not processed.

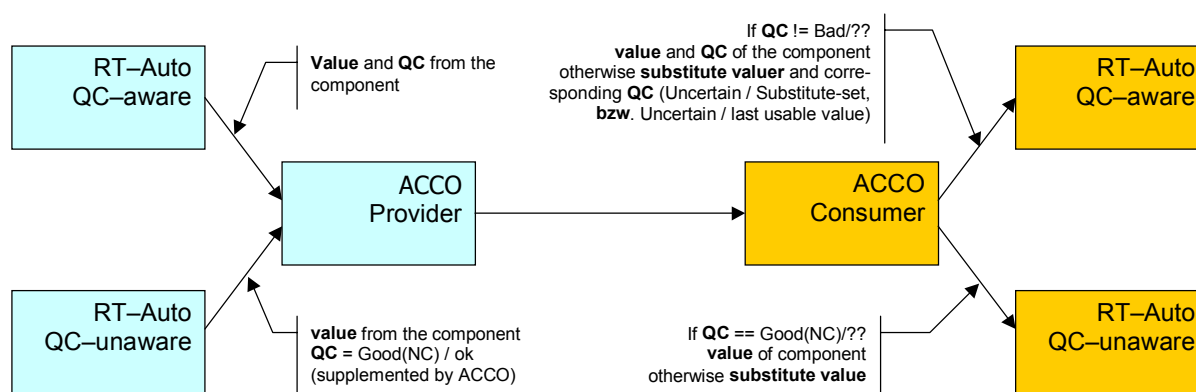
The definition of the behavior for a component-overlapping quality code has an effect on an connection to an input (necessary for a dynamic diagram where the QC of the provider can be passed on).

In order to support monitoring (i.e. the dynamic diagram), a quality code of an interconnected data item shall be passed on and/or can only get worse.

15.3.3.3.8.10.3 Standard behavior for Quality Codes

Any mixture of QC-aware and QC-unaware RT-Auto objects can exist. Therefore, the quality code is always transferred between the ACCOs as a qualifier of a variable.

Figure 175 represents the standard behavior of the Quality Code transfer.

**Figure 175 — Quality Code transfer – standard behavior**

The behavior of the provider is defined by the implementation of the RT-Auto.

If the RT-Auto object is QC-aware, the provider accepts its Quality Code but for the scenarios that are identified explicitly below.

If the RT-Auto object is QC-unaware, the provider always sets the quality code to "Good(NC) / ok".

The behavior of the consumer is also characterized by the implementation of the RT-Auto.

If the RT-Auto object is QC-aware, the value specified by the provider and the associated Quality Code are transferred transparently to the RT-Auto object - provided that the Quality Code is different than "Bad".

Depending on the configured substitute value strategy, a Quality Code "Bad" leads to the application of a fixed substitute value with the Quality Code "Uncertain / Substitute-set" or to holding the last known value with "Uncertain / last usable value" assigned.

If the RT-Auto object is QC-unaware, the RT-Auto object has only one value assigned. This value depends on the quality code transmitted by the provider. The transmitted value is transferred if it is "Good (NC)" with any substatus.

Any other quality code leads to applying a fixed substitute value or to holding the last known value (depends on the configured substitute value strategy).

The difference in the behavior of QC-aware and QC-unaware RT-Auto objects is in the assessment of values with the Quality Code with the status "Uncertain" or "Good(C)". Here, the ACCO object alone cannot be used for deciding whether or not they are useful. Thus, they are sent to a QC-aware RT-Auto object which can decide for itself from considering the Quality Code. For a QC-unaware RT-Auto object, however, they are considered as being bad.

The only exception of this is rule is the Quality Code "Uncertain/sensor conversion not accurate". Here always the transmitted value gets delivered to a QC-unaware RT-Auto object.

15.3.3.3.8.10.4 Behavior in case of a communication fault

Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy).

As shown in Figure 176 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

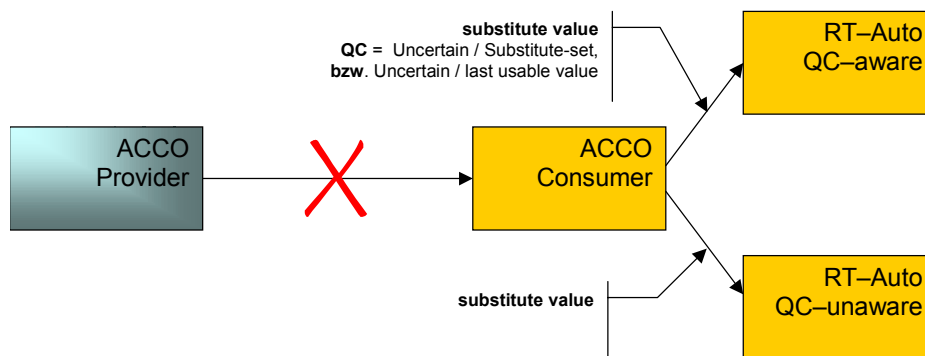


Figure 176 — Quality Code with communication fault

15.3.3.3.8.10.5 Behavior in case of connection is cleared

Once an connection has been cleared, the connection sink shall obtain a defined value again. Otherwise it would hold the last transferred value and quality code. The substitute value strategy defined by the connection is used for this new initialisation.

Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy).

As shown in Figure 177 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

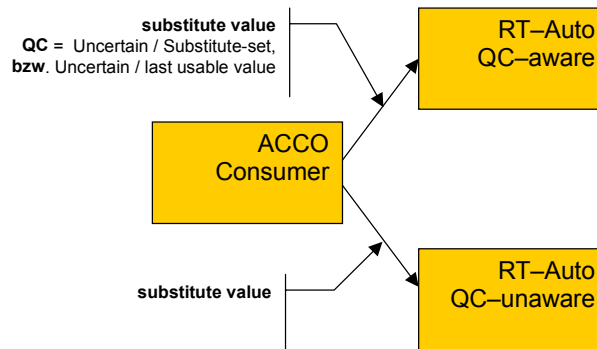


Figure 177 — Quality Code when a connection is cleared

15.3.3.3.8.10.6 Behavior in case of connection is deactivated

Deactivating a connection corresponds qualitatively to clearing a connection (with holding the resources at the same time). The connection sink shall again obtain a defined value. Otherwise it would hold the last transferred value and quality code. The substitute value strategy defined by the connection is used for this new initialization.

Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy).

As shown in Figure 178 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

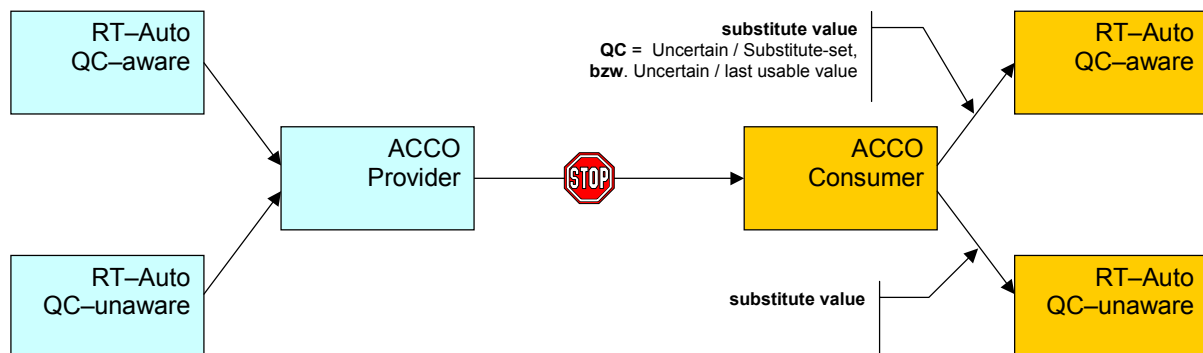


Figure 178 — Quality Code when a connection is deactivated

15.3.3.3.8.10.7 Behavior in case of "incorrect" connection data

"Incorrect" connection data can occur in the following cases:

- The expected length of the connection data does not correspond to the transferred length. This can happen, for example, in a connection between structures if the provider's structure does not correspond to the consumer's structure.
- Transfer of connection data that exceed consumer limits. For example this is possible for RT-Auto objects that have implemented a maximum upper limit for a string, or expect an array of a fixed size.
- Transfer of incorrectly serialized connection data.

Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy). In addition, the error "Type Mismatch" or "Limit violation" is logged for the connection.

As shown in Figure 179 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

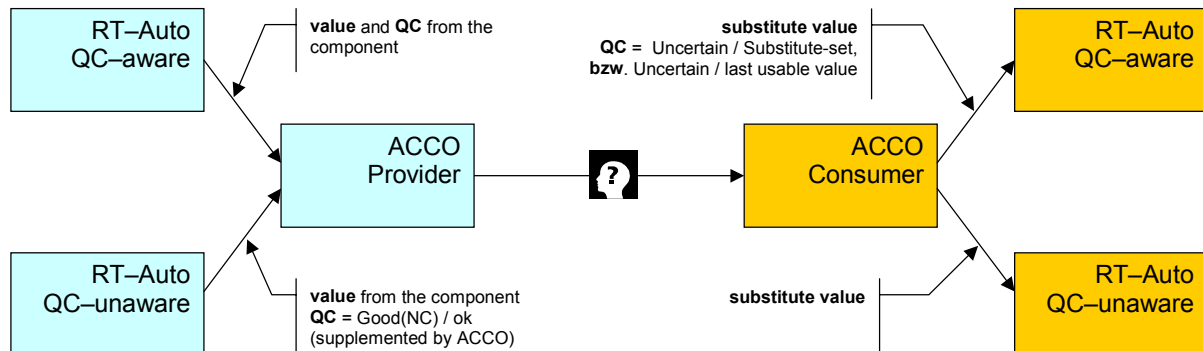


Figure 179 — Quality Code during the transfer of "incorrect" connection data

15.3.3.3.8.10.8 Behavior in case of the provider in "Ready" state

If the provider is in the "Ready" state, the RT-Auto objects can no longer supply valid values. Their values do not change at the side of the provider. In this situation, the provider replaces the QC of a QC-aware component and/or adds the Quality Code "Bad / out of service" to the Quality Code of a QC-unaware component, and transfers this code (as usual) to the consumer.

For the consumer, the behavior corresponds to the standard behavior described above: Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy).

As shown in Figure 180 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

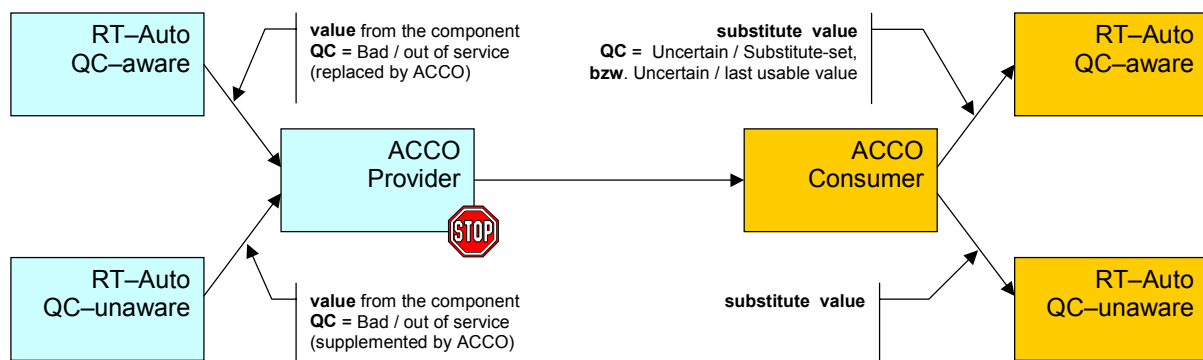


Figure 180 — Quality Code for provider in "Ready" state

15.3.3.3.8.10.9 Behavior in case of clearing an object from the provider

If an object with an active connection is cleared from the provider, the provider transfers the last known value together with the quality code "Bad / out of service".

For the consumer, the behavior corresponds to the standard behavior described above: Irrespective of the QC-awareness of the RT-Auto object, a fixed substitute value is applied or the last value is held (depending on the substitute value strategy).

As shown in Figure 181 depending on the substitute value strategy, a QC-aware RT-Auto object has the quality code "Uncertain / Substitute-set" or "Uncertain / last usable value" assigned.

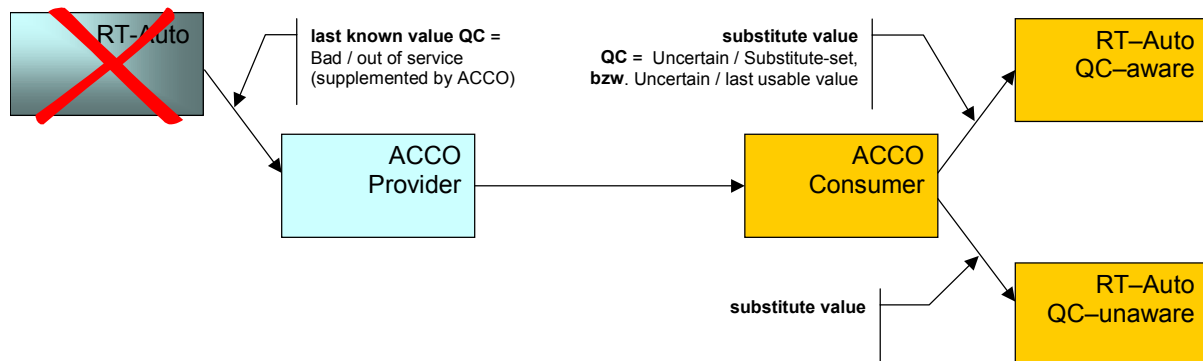


Figure 181 — Quality Code when clearing an object from the provider

15.3.3.3.8.10.10 Behavior in case of connection is forced

Forcing an connection holds the connection sink at a fixed value that is specified from the outside.

Irrespective of the QC-awareness of the RT-Auto object, the force value is assigned instead of the transferred value.

A QC-aware RT-Auto object has additionally the "Good (C) / local override" quality code assigned. Figure 182 shows the behavior.

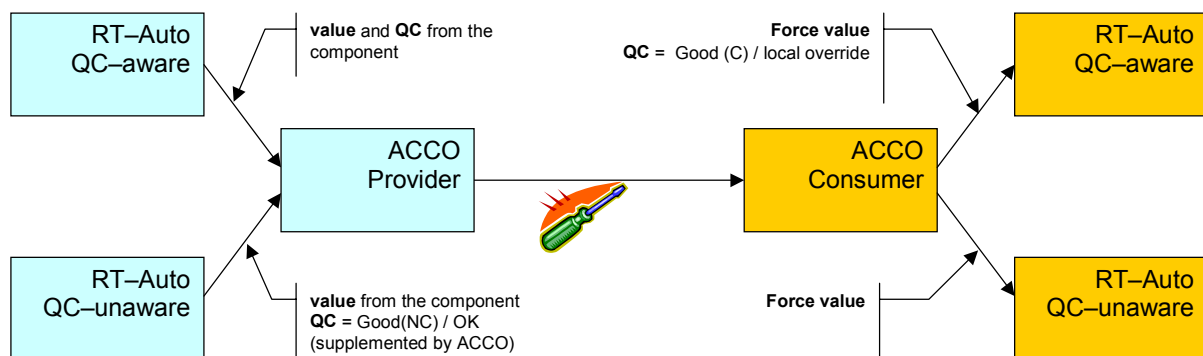


Figure 182 — Quality Code when a connection is forced

15.3.3.3.8.10.11 Behavior in case of QoS violation

If the provider is no longer able to maintain a Quality of Service for an connection (i.e. if it is no longer able to maintain the requested time intervals due to a temporary or permanent work overload), the values can no longer be provided at the required quality. In this situation, the provider replaces the QC of a QC-aware component and/or adds the Quality Code "Uncertain / sensor conversion not accurate" to the Quality Code of a QC-unaware component, and transfers this code (as usual) to the consumer.

If a QC-aware component already specifies a Quality Code that is - according to the priority Table 677 - worse than "Uncertain / sensor conversion not accurate", the Quality Code that is specified by the component side shall be retained and transferred.

For the consumer, the behavior corresponds to the standard behavior described in Figure 183. Resulting from the transmitted quality code, a QC-unaware RT-Auto object gets the transmitted value without any notification of the problem. A QC-aware RT-Auto object has the transferred value with the corresponding quality code assigned. Therefore only a QC-aware RT-Auto object is capable to detect this situation.

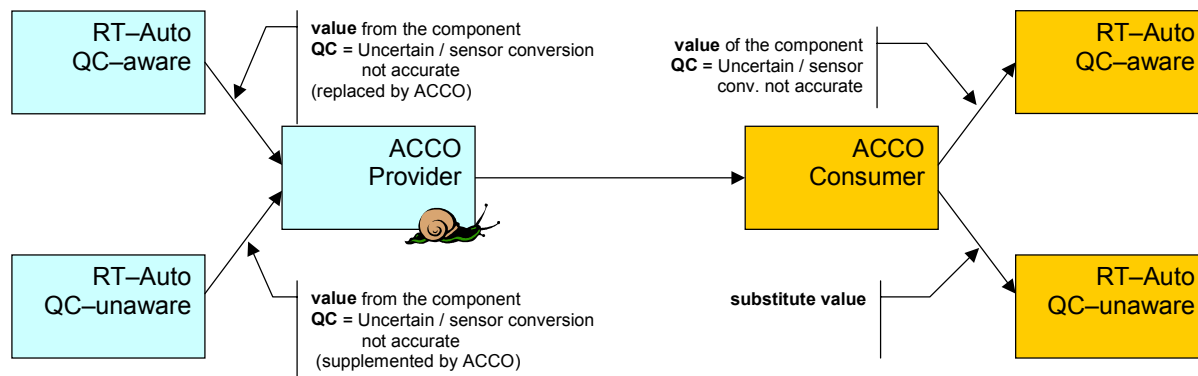


Figure 183 — Quality Code at QoS violation

15.3.3.3.8.10.12 Initial value

With a QC-aware RT-Auto object, the Quality Code "Uncertain / initial value" is transferred until the first value can be transferred via an connection. The initial value of the RT-Auto object is defined by the substitute value strategy, either by setting the fixed substitute value or the last known value. The latter is defined by the device (for a PLC, for example, after booting this is the buffered value last used before a power-off), as shown in Figure 184.

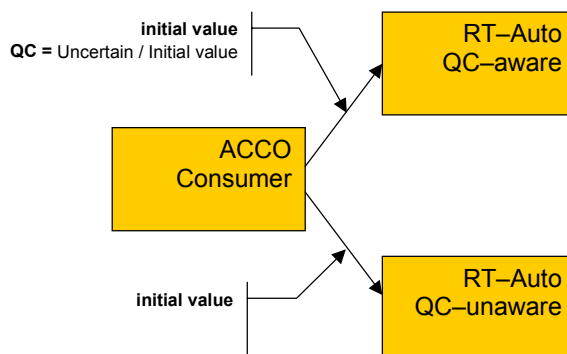


Figure 184 — Quality Code for initial value

15.3.3.3.8.10.13 Write access to values via put-services or WriteItem services

If RT-Auto object values are written via the ORPC put-services (PropertyPut) or the ACCO service WriteItem (ICBAACcoSync interface), the Quality Code "Good(NC) / ok" will always be added in the event of a QC-aware RT-Auto object.

15.3.3.3.8.10.14 Component-Overlapping Quality Code

At an exact point of time, only one quality code can exist at a variable. This quality code contains exactly the state of the variable at this moment. The quality code for a variable is generated exactly at several points, not only at one point. A variable in a device obtains a quality code that is the result from the verification of the state of the variable in the device (a substitute value is passed on, for example, if a broken wire is detected). Then the variable will be processed furthermore. A quality code for the variable is generated there based on specific verifications (does, for example, the variable exceed a configured upper limit?).

In general it can be said that a quality code for a variable can be produced in any component in which variables are produced and/or processed. A rule is therefore required that says how the quality code is handled across the components:

- In the processing hierarchy, the quality code may only deteriorate, never improve. Each subsequent component that produces quality code determines the component-specific quality code according to its rules. Next, a priority table (see Table 677) is used for checking whether this component-specific quality code is better or worse than the one that already exists at the variable.

NOTE A component that produces variables can, of course, improve the quality code (from bad to good, for example). A subsequent component is a component that processes existing variables and their quality codes.

Two cases are possible with subsequent components:

- The component-specific quality code is better than or equal to the one that exists at the variable: Leave the quality code at the variable unchanged.
- The component-specific quality code is worse than the one that exists at the variable: Overwrite the quality code at the variable with the component-specific one.

This ensures that the quality code at the variable always maintains the worse state, irrespective of number and kind of the components in which the variable was processed.

The priority table has been defined in Table 677.

Table 677 – Quality Code priority table

Quality	Substatus
Good (NC)	OK
Good (NC)	Maintenance Required
Good (NC)	Active Update Event
Good (NC)	Active Advisory Alarm
Good (NC)	Active Critical Alarm
Good (NC)	Unack Udate Event
Good (NC)	Unack Advisory Alarm
Good (NC)	Unack Critical Alarm
Good (C)	OK
Good (C)	Initialization Acknowledge
Good (C)	Initialization Request
Good (C)	Not invited
Good (C)	Do Not Select
Good (C)	Local override
Good (C)	Initiate fail safe
Uncertain	Non-specific
Uncertain	Last Usable Value (LUV)
Uncertain	Substitute
Uncertain	Initial Value
Uncertain	Sensor Conversion not Accurate
Uncertain	Engineering Unit Violation
Uncertain	Sub-Normal
Uncertain	Configuration Error
Uncertain	Sensor Calibration
Uncertain	Simulated Value
Bad	non-specific
Bad	Configuration Error
Bad	Not Connected
Bad	Sensor Failure
Bad	Device Failure
Bad	No Comm, with LUV
Bad	No Comm, no LUV
Bad	Out of Service

The quality code of the lowest priority (the best quality code) is at the top of the table. The one with the highest priority (the worst) is at the bottom.

15.3.3.3.8.11 Operating state

A consumer in **Ready** state can still receive data packets from providers. It can either store these data packets in a buffer for the individual current values, or reject them with a corresponding negative result. The buffered data are only transferred to the application after the transition to **Operating** has been made.

15.3.3.3.8.12 Power-Off

The consumer shall re-establish all connections after a power-off. The consumer first invokes the DisconnectMe service (ICBAAccoServer interface) to all persistent configured providers. The provider shall then discard all information that is related to this consumer if it has not yet detected the power-off state of the consumer.

The consumer shall then re-establish all connections based on its retentive data by invoking the Connect service (ICBAAccoServer interface).

15.3.3.3.8.13 Online–/Offline Comparison of the Connections

There is a procedure defined for the on-line / off-line comparison of the connections. The consistence shall be determined by means of the get_CDBCcookie service (ICBAAccoMgt interface). This service reads a brief version of the current state of the persistent information of the Configuration Data Base referred to as cookie. The cookie shall be stored by the client (e.g. Engineering System) to determine the change of connection information without uploading the whole Configuration Data Base. The cookie itself shall be maintained (e.g. incremented) by the ACCO ASE (consumer) every time the Save service for the PDev or LDev is called.

The Engineering System shall upload all connections and compare them against the off-line database if the cookies are different. The Engineering System shall accept the new cookie that exists on-line if all connections that were configured off-line by this client exist in the on-line database. Therefore, connections that shall be added by a second Engineering System are accepted.

15.3.3.3.8.14 Synchronous Access

Custom interfaces of the RT-Auto Class permit access to data. Their corresponding property access services can be used for reading or writing the value of a property. The quality code cannot directly be retrieved from a QC-unaware RT-Auto since it is generated and managed by the ACCO ASE. Only an inconsistent access (since two operations are required) is possible for a QC-aware RT-Auto.

The ACCO Class provides with its services on the ICBAAccoSync interface a consistent access to the value and quality code. This interface enables a client to perform consistent read and/or write access procedures to value, Quality Code and time stamp - irrespective of the QC-awareness of the RT-Auto object. The interface permits symbolic access to the values.

15.3.3.3.8.15 ICBAAccoMgt – Configuration Data Base

The Configuration Data Base – the persistent database for the connection information – provides the mandatory ICBAAccoMgt interface as external service interface.

15.3.3.3.9 ACCO ASE – Provider facility

15.3.3.3.9.1 Overview

The provider part of the ACCO ASE provides the necessary services for connection sources. It has the following tasks:

- It actively performs the data connections according to the Quality of Service in cooperation with the operating system – for example via timers, interrupts, etc.
- It passively performs event connections via initiation by RT-Auto object.
- It determines the quality code of a QC-unaware RT-Auto object.
- It transfers the connection data to the consumer of the connection.
- It optimizes data connections by automatic grouping and optimizes event connections by filtering.

It responds adequately to a consumer failure.

15.3.3.3.9.2 Establishing Connections

The consumer determines the best communication channel via negotiation with the provider. The Connect service (ICBAAccoServer interface) is used to indicate the establishment of a connection. The provider checks whether the given connection sources exist and store the necessary information in its internal data structures.

On the basis of its internal information (e.g. the interface description information delivered by a typelibrary), the provider recognizes via the custom interfaces of the RT-Auto objects that are to be interconnected whether the connection source is an event or a data item.

For data sources, the provider shall determine the sending of data time triggered according to the Quality of Service parameter.

For event sources, the provider shall determine the sending of data when the appropriate event is triggered.

NOTE The local event recognition between custom interfaces of the RT-Auto ASE and the provider part of the ACCO ASE is a local matter. However, the provider may use a local event model of a component object model if present in the automation device. Therefore, it creates a new instance of a special implementation of the IDispatch interface and deploys this instance to log on with the corresponding ConnectionPoint of the RT-Auto object, using IConnectionPoint:Advice.

With the response of the Connect service the provider delivers for each requested connection source a HRESULT and a ProviderID. The ProviderID unambiguously identifies the internal data of the provider. It is used for identifying the connection sources at all other services of the ICBAAccoServer interface.

During the cyclic data exchange, the ConsumerIDs that are transferred by the consumer will always be transferred together with the value of the requested variable. When a consumer receives a data packet, it is able to unambiguously identify the internal data on the basis of the contained ConsumerID, and can transfer the values to the corresponding access function.

The provider checks for signature identity by means of evaluating the data types when it establishes a data connection. If there is no signature identity, the provider rejects the connection and a corresponding error (Type Mismatch) is reported. The consumer then creates the substitute value.

The following special treatments are necessary for the signature identity of arrays and structures:

- Due to its structure, the Safearray data type has more freedom. It is neither restricted in the dimension nor in the data type or the number of elements of the individual dimensions. Its definition does not make it particularly suitable for automation applications (e.g. in a PLC programmed to IEC 61131-3) that typically assume an array with defined dimension, data type and number of elements per dimension. In order to express the Safearray's expectation of a fixed layout, and since the Automation data type VT_SAFEARRAY does not explicitly contain the base data type - this is a feature of the Safearray (as structure element) during the transfer - it is defined that the "VT_ARRAY | base data type" flag pair is transmitted for adjusting the type information. Any errors due to the dimension or the number of the elements of a dimension can only be detected during the transfer of the data, and will then be rejected by the consumer with the corresponding error message (Type Mismatch).
- If a structure (VT_USERDEFINED) is expected, a check for signature identity – but for the fact that a structure is expected – can only be performed at the consumer side when the data is transmitted. This can only be restricted via the length of the data that is serialized via the communication channel. A stringent check for signature identity would require the transfer of the complete type information since the declaration of the structure can be of any complexity. The data is deserialized accordingly if the length of the transferred structure agrees with the length of the expected structure.

NOTE It is anticipated that the check for signature identity of event connections will be included in a future edition of this International Standard.

15.3.3.3.9.3 Productive Operation for Data Connections

For the data sources, the data provider possesses lists that are sorted by communication channel and QoS. On the basis of these lists and the connection status ("active"), it identifies the data that shall newly be determined. This data is read via the configured access methods of the related objects.

For the QoS "cyclically on changes", this value shall be compared with the last value in a local cache for deviations by $\pm \varepsilon$. A new value will only be transferred after a change.

The values are transferred via the communication channel together with the corresponding ConsumerID, quality code and - optionally – a time stamp. The transmission path depends on the communication process. The procedure is shown in Figure 185.

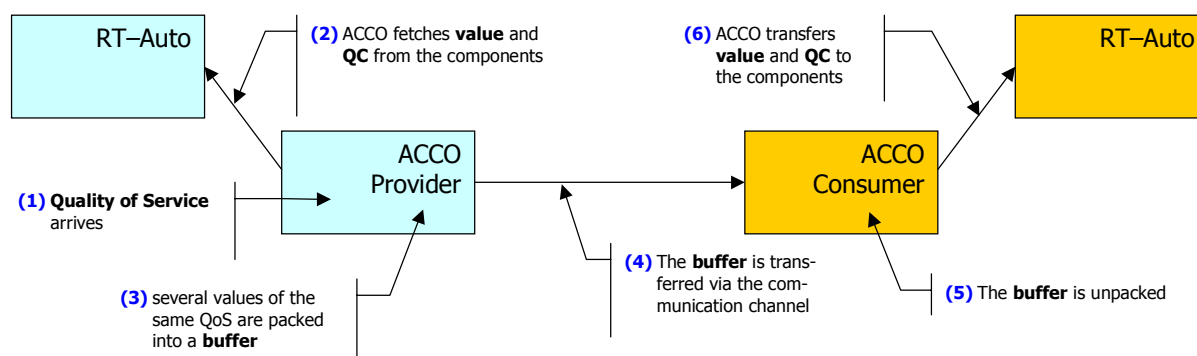


Figure 185 — Productive operation of data connections

This packet is transmitted to the consumer according to the definitions of the communication channel using the OnDataChanged service (ICBAAccoCallback interface).

The provider part of the ACCO ASE automatically groups data connections of the same QoS and communication channel to reduce the communication loading since multiple data items are bundled and transmitted in one packet (ORPC-PDU).

The provider has to transfer the actual data of connections independent of the QoS in the following cases:

- Operating state transition from Ready to Operating, respective Operating to Ready with the appropriate Quality Codes. This affects all data connections.
- Establishing new connections by ICBAAccoServer::Connect. This affects the new connections.
- Changing the connection state by ICBAAccoServer::SetActivation with the appropriate Quality Codes. This affects the changed connections.
- Clearing an RTAuto from the provider with the appropriate Quality Codes. This affects the connections regarding that RTAuto.

15.3.3.3.9.4 Productive Operation for Event Connections

The behavior of event connections is shown in Figure 186.

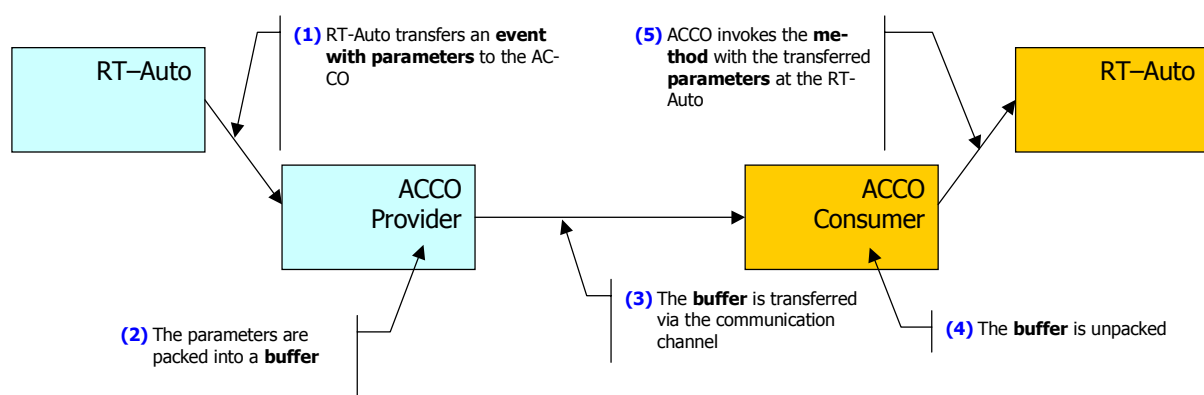


Figure 186 — Productive operation of event connections

The provider part of the ACCO ASE receives and handles RT-Auto events on established event connections. The procedure is a local matter and not described within this specification.

NOTE Depending on the internal modeling, this is done either via a proprietary way or via the methods IConnectionPointContainer and IConnectionPoint.

The event provider is informed of triggered events (e.g. via a proprietary way or via the invocation of IDispatch (Invoke) at an instance of the interface) and provides the following functions:

- It identifies the ProviderID and the related event connection via the instance data of the invoked interface.
- It filters events that are not needed.
- It packs the call parameters according to the rules of the communication process and transfers them to the consumer via the communication channel.

15.3.3.3.9.5 Operating state

The provider transmits the values and the corresponding Quality Code until the LDev state changes from **Operating** to **Ready** (last transfer in case of state transition). Subsequently, it still responds to the pinging of the consumer, but does not transfer any further data to it. Local value changes of the provider are transferred again when the state changes back to **Operating**.

The Connect service (ICBAAccoServer interface) causes the first transmission of all valid data. This enables a consumer to be linked also in the Ready state. In this case the consumer is provided with the current values.

It appears to a consumer as if the configured connection values would not change during the Ready State.

15.3.3.3.9.6 Power-Off

The provider remains passive after a power-off. The consumer shall re-establish the connections. This means, that the provider do not provides any retentiveness.

15.3.3.3.9.7 ICBAAccoServer – Provider for Connections

The mandatory interface ICBAAccoServer provides services to manage the connection sources.

15.3.3.3.10 ACCO ASE – Communication facility

15.3.3.3.10.1 Overview

The communication part of the ACCO ASE has the following tasks:

- It abstracts from the actual access to the communication medium.
- It offers various communication procedures and can be enhanced with respect to new communication procedures.
- It automatically optimizes the communication between consumer and provider with respect to the best communication procedure possible.
- It observes the communication channel and causes fault reactions.

The following terms are defined for the description that follows below:

- A **communication procedure** is a possible communication path ("**type**") between a provider and a consumer. (AR Class Definition)
- A **communication channel** is an established communication path ("**instance**") between a provider and a consumer. It is described by the instance data.

15.3.3.3.10.2 Negotiating the Communication Channel

Type 10 defines a procedure that can be used by consumer and provider for negotiating a communication channel that matches the given situation. The negotiation is used in the following two scenarios:

- Provider and consumer optimize each other automatically to a communication procedure with the least possible base loading.
- A special QoS requires a special communication procedure.

Within the scope of the configuration, additional QoS, that requires a special communication procedure, can be specified over and beyond the type 10 standard.

15.3.3.3.10.3 ORPC Communication Procedure

The ORPC communication procedure is mandatory.

The ICBAAccoServer interface (the Connect service in particular) is used as interface definition for the ORPC communication procedure. The communication channel is established by transferring the Callback Interface Pointer with the Connect service from the consumer to the provider.

The provider uses the OnDataChanged service of the ICBAAccoCallback interface for the data transfer via the ORPC communication procedure. The data are conveyed as a buffer.

If different interface instances of the callback pointer are transmitted upon repeated invocations of the Connect service by the same consumer (can be recognized by the transmitted dstLDev), the provider only deploys the instance last transmitted. Consequently it is able to automatically group connection data and transmit it in an optimized way.

The Ping service (ICBAAccoServer interface) is used by the consumer for observing that the provider is alive. The Ping service shall be invoked by the consumer if there was no communication to a given provider after a certain dead time (i.e. if that provider has not supplied any data to the consumer during this interval). This ping is valid for all the connections that this provider delivers to the consumer.

The Engineering System configures this dead time by means of the SetPingFactor service (ICBAAccoMgt interface). It is specified as a factor. The n-fold dead time of the least Quality of Service for a repeated occurrence of data via OnDataChanged service is allowed for each provider.

The following definitions are made for the ping factor:

- The default value is fixed to 10.
- Setting the ping factor to 0 disables pinging.

The observing that the consumer is alive does not require any additional procedure. A provider is able to detect the consumer's failure synchronously, since a necessary data transfer to the consumer cannot be realized with results in a negative confirmation of the OnDataChanged service.

15.3.3.3.10.4 Fault scenarios

15.3.3.3.10.4.1 Overview

The fault handling is a simple error handling procedure. Any fault is handled as a device failure of the type 10 device (power-off or defect). The fault elimination procedure does not distinguish between line problems or the failure of a device. This means that temporary line problems are handled like a failure.

15.3.3.3.10.4.2 Failure of the Provider in Productive Operation

The consumer detects the failure of a provider during productive operation by means of the monitoring procedure. It requested to set all data sinks that are supplied by this provider to the configured substitute value or to hold their last values (depends on the configuration). At the same time, the quality code is set to "Substitute-set" or "last usable value".

The consumer tries to re-establish the connection to the provider at a certain time interval (e.g. 30 seconds).

Once the provider is available again, the consumer shall clear all connections with the DisconnectMe service and shall re-established all connections by means of the Connect service. using. This clears the situation in which the provider has not yet detected a communication failure, see Figure 187.

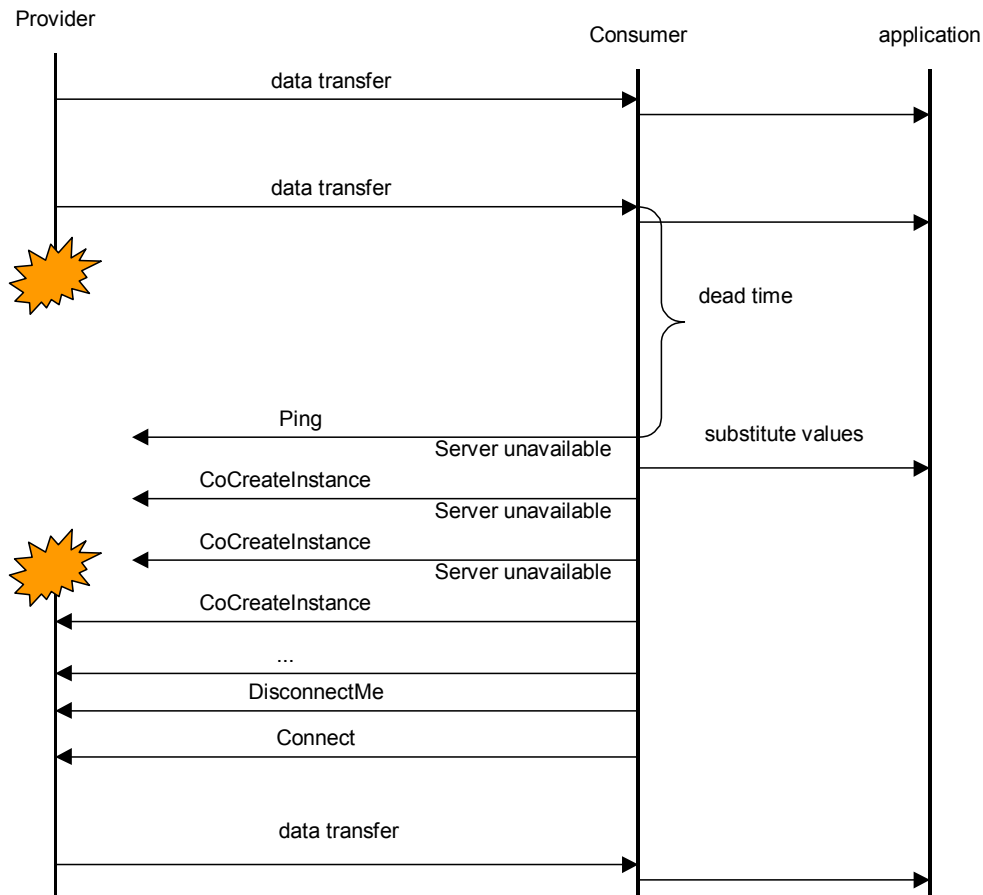


Figure 187 — Failure of the provider in productive operation

15.3.3.3.10.4.3 Failure of the Consumer in Productive Operation

If the provider detects the failure of the consumer, the provider relinquishes all the connections to this consumer. As shown in Figure 188, it deletes all connection information because the consumer is responsible for re-establishing the connections again.

Subsequently, the consumer re-establishes – as defined above – the connections.

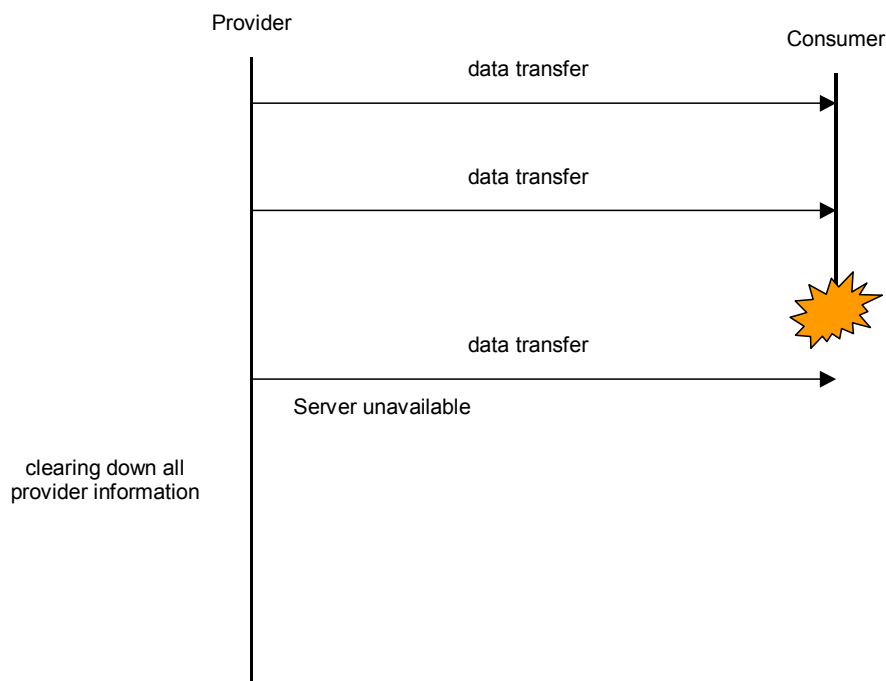


Figure 188 — Failure of the consumer

The provider shall respond with a negative result in the event of an indication of a Ping service of the failed consumer (e.g. after a short failure of the link). This informs the consumer about the fact that all its connections are cleared.

15.3.3.3.10.4.4 Unilateral Recognition of the Communication Failure during Productive Operation

The fault scenarios defined above and their reaction requires a unilateral recognition of a communication failure to be discussed separately. A unilateral recognition can occur due to a transient router failure or communication fault (poor connector contact, ...).

The provider discards all connections to the consumer concerned when it detects a communication failure. If the consumer subsequently – when communication is back to normal – tests the connections via the Ping service, the provider shall return a negative acknowledgement with CBA_S_NOCONNECTION if it does not maintain any connections to the nominated consumer. The consumer then follows the standard procedure for establishing all its connections.

A special form of this scenario results if - due to unilateral recognition of a communication failure - the provider discarded its connections, and this consumer subsequently and before its next ping established new connections via Connect service. The subsequent Ping service would supply S_OK since the ping does not check the quantity of the connections, but whether or not there are any connections to the consumer.

The result value "FirstConnect" at the Connect service exists for this purpose. It shows the consumer that the established connections are the first ones the provider has from this consumer. Depending on the expectation of the consumer, it can recognize this situation and follow the normal error scenario (i.e. DisconnectMe service and new establishment of all connections). If, for example, the consumer thinks that connections have already been established in this provider, the Connect service shall return FirstConnect == false.

15.3.3.3.10.4.5 Failure of the Provider when Changing the Connection Status

If the consumer detects the failure of a provider when it deactivates or activates a connection (via an error return from ORPC model (RPC_*)), the consumer shall try to establish the connection to the provider at intervals that remain to be defined.

Once the provider is available again, all connections at the provider's side are cleared by the DisconnectMe service. Subsequently, all connections are re-established, including the ones that are newly to be established. This clears the situation in which the provider has not yet detected a communication failure.

This procedure also becomes effective if the connection is interrupted during a outstanding Connect service.

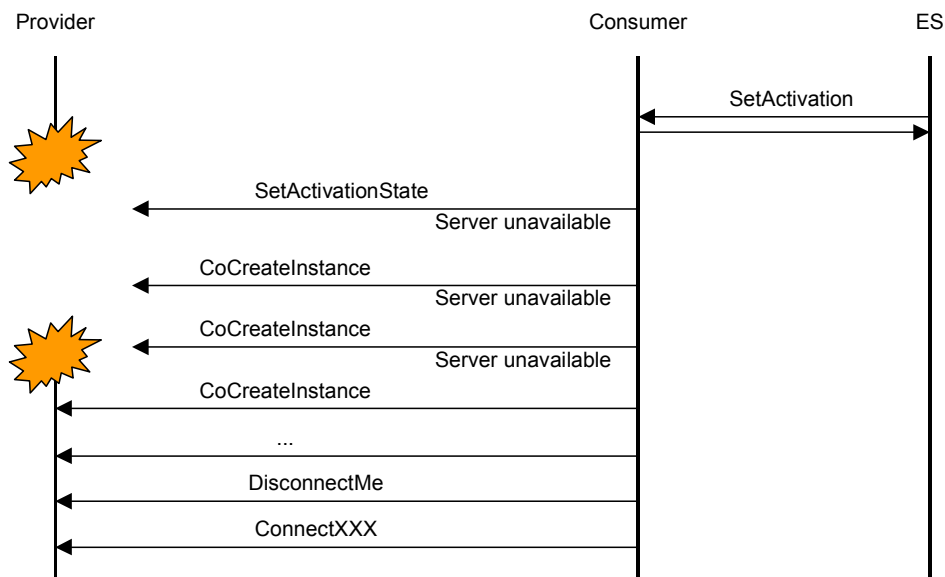


Figure 189 — Failure of the provider when changing the connection status

15.3.3.3.10.4.6 Failure of the Provider when Changing the Interconnection Status

If the consumer detects the failure of a provider when it deactivates or activates an interconnection (via an error return from ORPC_*), the consumer shall try to establish the connection to the provider at intervals that remain to be defined. See Figure 190.

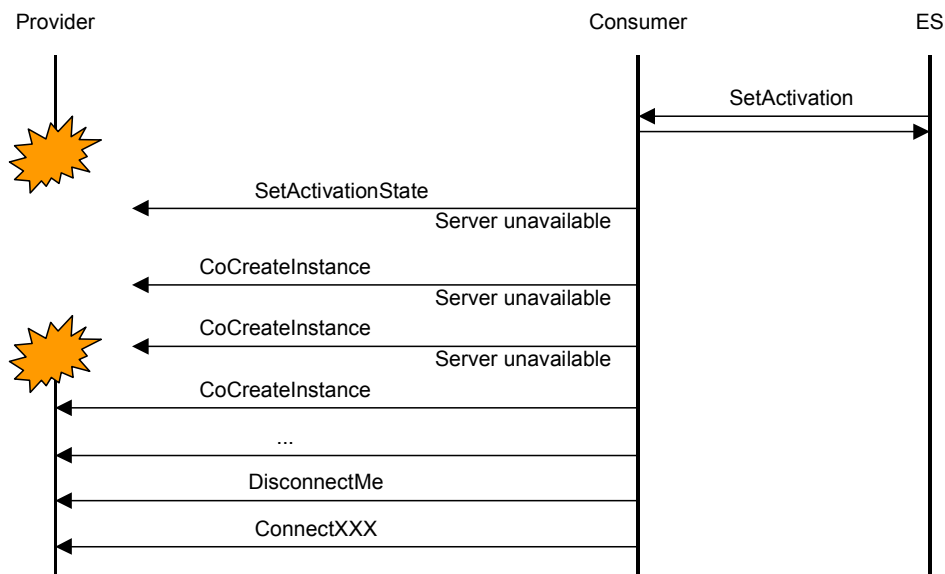


Figure 190 – Failure of the provider when changing the interconnection status

Once the provider is available again, all interconnections at the provider's side are cleared by the DisconnectMe service. Subsequently, all interconnections are re-established, including the ones that are newly to be established. This clears the situation in which the provider has not yet detected a communication failure.

This procedure also becomes effective if the connection is interrupted during a outstanding Connect service.

15.3.3.3.10.4.7 Failure of the Provider when Clearing Connections

If the consumer detects the failure of a provider when it clears a connection (via an error return from ORPC model (RPC_*)), the consumer shall try to establish the connection to the provider at intervals that remain to be defined.

Once the provider is available again, all connections at the provider's side are cleared by the DisconnectMe service. Subsequently, all connections are re-established, including the ones that are newly to be established. This clears the situation in which the provider has not yet detected a communication failure.

This procedure also becomes effective if the connection is interrupted during a outstanding Connect service.

The resources for the connections in the consumer may only be released after the connection data in the provider has been cleared up. If the provider were not notified of the failure of the communication relationship between provider and consumer, it could at any time receive data for the specific (first cleared and subsequently re-assigned) item and distribute connection data to a wrong RT-Auto object.

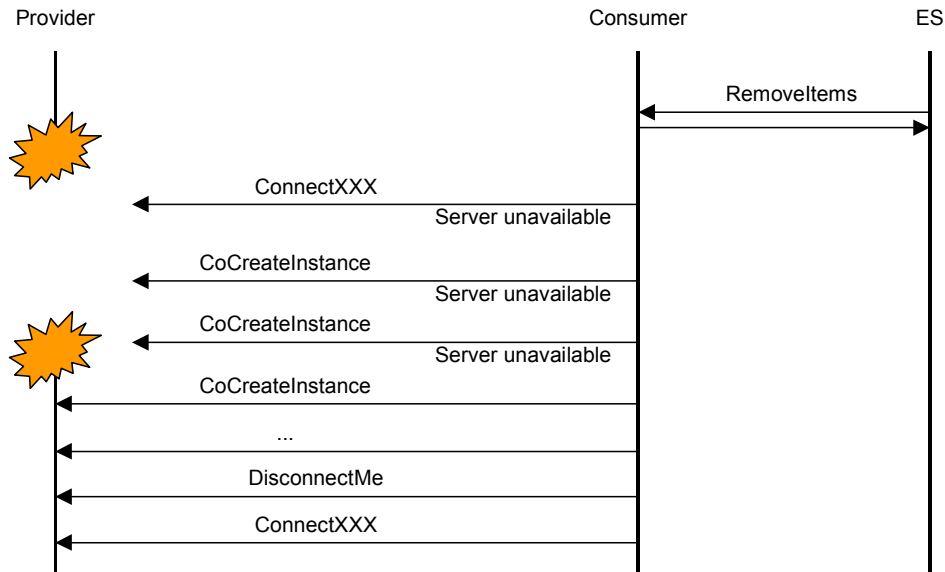


Figure 191 — Failure of the provider when clearing connections

An connection to a provider can only be cleared when the provider participates in this process. Connection "corpses" at the consumer side can be left over if a provider is removed completely from the distributed automation system without having cleared the connections.

15.3.3.3.10.4.8 Transmission of mutilated data

If the consumer is not able to decode the transmitted connection data, he reacts with an error return to the provider and applies substitute values to all connection sinks delivered by this provider. The provider itself drops all connections established in his reaction to the error code. The consumer then reestablishes the connections anew.

15.3.3.3.11 Connection Diagnosis

Like device diagnosis, different information levels exist for connection diagnosis as shown in Figure 192.

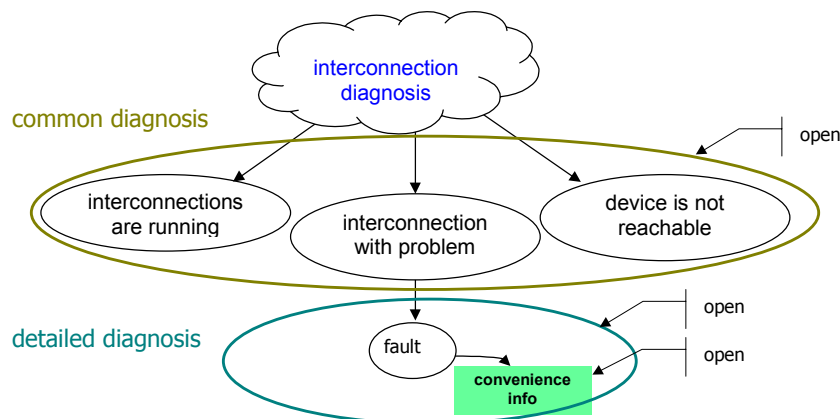


Figure 192 — Information levels

Status Model for the Common Diagnosis

The common diagnosis of the ACCO ASE is shown in Figure 193.

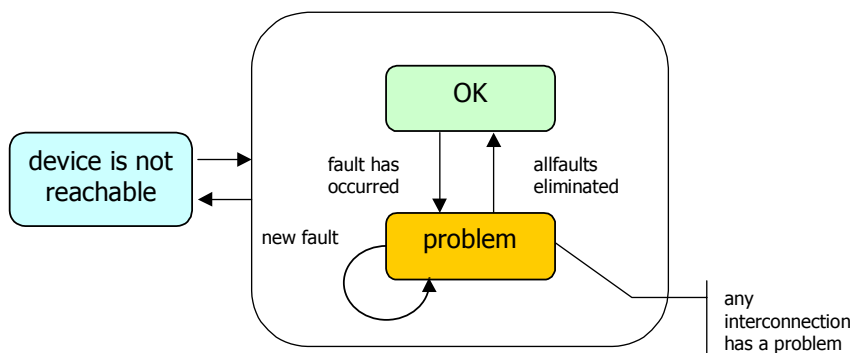


Figure 193 — ACCO ASE status model for the common diagnosis

The ACCO ASE common diagnosis uses the same interfaces that are used for the common diagnosis of the device (i.e. synchronous and asynchronous variant). Furthermore, the same rules apply to the distribution of the asynchronous notifications.

Status Model for the Detailed Diagnosis

The representations of the connections in a dynamic diagram requires information about the state of a specific connection to be obtained in order to be able to graphically edit this information according to the requirements. For the ACCO ASE, this information is understood under detailed diagnosis.

The Figure 194 shows the status model of an connection for the detailed diagnosis:

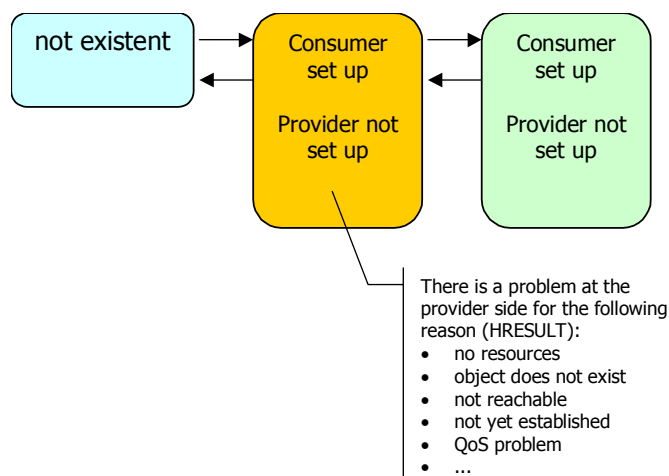


Figure 194 — ACCO ASE status model for the detailed diagnosis

Due to the ACCO ASE definition and its response to faults, the status model can easily be selected:

- Faults that occur during the setup of the consumer (no resources or object cannot be found) are returned **synchronously** when the connection is established via the AddConnections service. In this case, the connection is not yet established and remains in the "not existent" state.
- Faults that occur during the setup of the provider (no resources, object cannot be found, communication errors) are returned via detailed diagnosis and lead to a display of a group error. The consumer tries after a specific time (set to 30 seconds in the first stage) to purge the error by automatically reconnecting the connections. If an error is permanent, the user has to clear this connection explicitly to avoid traffic and in consequence to clear the group error. This applies also to local connections, i.e. connections with provider and

consumer in the same logical device. In consequence also the error specifying an connection from an identifier to itself will also be reported via detailed diagnosis.

- If an object is cleared that is consumer of one or more connections, those associated connections are cleared automatically. Although this is a malfunction of the Engineering System (an object should only be cleared after all the connections to this object have been cleared), it avoids "corpses" that would probably never be cleared by the Engineering System.

This is why the complete error information can be reduced to errors regarding the provider. These errors are represented by an error code (standard ORPC error code, i.e. HRESULT). The consumer of an connection consequently holds the current error information.

Table 678 shows the possible faults. The table shows also in the column group error, whether a specific fault results in displaying a group error on the ACCO ASE. A group error is displayed, if one or more connections have an error code with severity set to "error" (characterized by E_ in the error code's literal). The group error is cleared, if none of the connections have such an error code.

Table 678 – Error codes for the ACCO ASE detailed diagnosis

Fault description	Error code	GroupError
The provider's resources prove insufficient for establishing the connection.	E_OUTOFMEM	yes
The SrcItem cannot be found in the provider	CBA_E_UNKNOWNOBJECT	yes
The SrcItem has a different data type	CBA_E_TYPEMISMATCH	yes
The provider cannot provide the data with the necessary QoS	CBA_E_QOSTYPEUNSUPPORTED CBA_E_QOSVALUEUNSUPPORTED	yes
Communication error	RPC_E_*	yes
It is not allowed to specify an connection from an identifier to itself.	CBA_E_INVALIDCONNECTION	yes
The connection is not yet been established completely	CBA_S_ESTABLISHING	no

The synchronous form of the detailed diagnosis is integrated into the existing ACCO class interface ICBAACoMgt with the services GetIDs or GetConnections.

15.3.3.3.12 Definition of the Identifiers

The identifier of an LDev is defined by the following command:

"PDev!LDev"

An exclamation mark is used as delimiter.

PDev

is used for identifying the hardware in the IP sense. The IP address or the domain name can be used here. Using the domain name requires a DNS server that is able to convert the name into the corresponding IP address.

LDev

corresponds to the name of the LogicalDevice.

In order to facilitate identification systems of all variants with respect to the nomenclature of the LDev, all characters are permitted for LDev (as mentioned above). This requires a special interpretation of the identifier by first extracting from the left-hand side to the "!" PDev. The remainder is the identification of the LDev.

The identification for the source or sink of a data or event connection is performed in plain text via an identifier. The identifier unambiguously describes a property, an event or a method in an interface of an RT-Auto Class. The syntax of the identifier has been defined as follows:

"Object.Member"

The delimiter between object and member is a point.

Object

corresponds to the name of the RT-Auto object that can also be used at the enumeration of RT-Auto objects.

In order to facilitate identification systems of all variants with respect to the nomenclature of RT-Auto objects, all characters are permitted for Object (as mentioned above). This requires a special interpretation of the identifier in which the method name (defined in the IDL) is extracted from the right-hand side up to the ".". The method name has been defined with a restricted character set. The remainder is the identification of the RT-Auto object.

Member

For a data connection, member unambiguously describes the put or get method used for accessing the corresponding property. For an event connection, it describes the method used for the event.

The member follows the identification system that is defined by the IDL.

With respect to possible language variants of the IDispatch interface, only the member name defined under the "Neutral" language (LANG_NEUTRAL) is supported (which is the name supported in the IDL). The configuration of the connections shall therefore be performed in a language-independent way.

An example for an interface declaration for this purpose follows below:

```
EXAMPLE    [
            object,
            uuid(7f41e860-ba54-11d1-912c-00a02442df7d),
            oleautomation,
            dual,
            pointer_default(unique)
        ]
interface IReaktor : IDispatch
{
    [propput] HRESULT InflowValve([in] VARIANT_BOOL newVal);
    [propget] HRESULT InflowValve([out, retval] VARIANT_BOOL
*pVal);
    [propput] HRESULT OutflowValve([in] VARIANT_BOOL newVal);
    [propget] HRESULT OutflowValve([out, retval] VARIANT_BOOL
*pVal);
    [propget] HRESULT Level([out, retval] long *pVal);
};

[
    object,
    uuid(7f41e861-ba54-11d1-912c-00a02442df7d),
    oleautomation,
    dual,
    pointer_default(unique)
]
interface ITemp : IDispatch
{
    [propget] HRESULT CurrentTemp([out, retval] float *pVal);
};

...

[
    uuid(7f41e862-ba54-11d1-912c-00a02442df7d),
    noncreatable
]
coclass Reaktor
{
    [default] interface IReaktor;
    interface ITemp;
};
```

If there is an RT-Auto object "Reaktor 1" of the Reaktor class, the source of the "OutflowValve" property (i.e. the *propget* method) is identified by the designator

"Reaktor 1.OutflowValve"

To interconnect the "CurrentTemp" property, the designator

"Reaktor 1.CurrentTemp"

shall be used.

The identification of a constant is performed via a special identifier. A constant is only permitted as the source of a data connection. The syntax of the identifier has been defined as follows:

"DELValue"

DEL is used as the representative of the character with the code 127 from ISO 8859-1. This character is declared invalid for being used as object name, therefore no collisions with "normal" object names can appear. The *DEL* character is used as the first character of the identification.

Value defines the value of the constant in plain text. This value shall be converted according to the connection destination. In contrast to an connection with a productive data value, the constant is handled only locally by the consumer.

NOTE A constant of the data type VT_I2 (or VT_I4 and further integer types) is, for example, designated as "DEL16", a constant of the data type VT_FLOAT is designated as "DEL1.24", and a constant of the data type VT_BSTR is designated as "DELabc".

Object names (for LDev and RT-Auto objects) and method names are considered as identifiers.

The ISO 8859-1 character set (ISO Latin-1 Character Set) is used for object names (i.e. LDev and RT-Auto objects). This character set permits a West-European notation of identifiers. Thus, at least the characters of languages like English, German, Italian, Spanish or French can be used. One byte is used for storing the characters.

Supporting more advanced character sets (such as DBCS or Unicode for Kanji or something similar) is disapproved since this would mean the introduction of local information in runtime.

To permit all variants with respect to the nomenclature to be used, all "printable" characters from the ISO 8859-1 character set may - from the runtime perspective - be used for an object name. The following characters may not be used:

characters with the code 0 – 31

characters with the code 127 – 159.

There are no further restriction with respect to special characters such as "\", "–", etc.

The definitions of the character set permits identifiers to be stored in a compressed format. This means that Wide Character Strings need not be stored, these can be reconstructed from a normal character string by just adding a zero for the second byte of each wide character. At the interface level always Wide Character Strings are transported by using the data type BSTR.

The ASCII (ISO 10646-1) Row 00, or equivalently the "lower half" of the ISO 8859-1 character set, that is, Code 0 – 127), is defined for method names.

Only letters ("A"–"Z", "a"–"z"), digits ("0"–"9") and underscore ("_") may be used for method names. A method name shall always begin with a letter.

Identifiers are stored in a case-preserving way, but the comparison of all identifiers is case insensitive.

NOTE A case-insensitive comparison for ISO 8859-1 can easily be implemented since capital and small letters are mapped in parallel code areas. The characters of the following character strings are converted into the corresponding capital letters by merely subtracting 32: 97 (a) – 122 (z), 224 (à) – 246 (ö), 248 (ø) – 254 (þ). All the other characters are retained in their original form.

The maximum length of identifiers shall be defined in order to permit an optimum implementation of the persistence of identifiers in the ACCO ASE context. Throughout the system, a maximum of 32 usable characters are defined for object names, tag names and method names.

15.3.3.4 ACCO ASE service specification

15.3.3.4.1 AddConnections

The AddConnection service adds one or multiple connection records to the ACCO object. Table 679 shows the parameters of the service.

Table 679 – AddConnections

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Provider	M	M(=)		
QoSType	M	M(=)		
QoSValue	M	M(=)		
State	M	M(=)		
Count	M	M(=)		
List of pAddConnectionIn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppAddConnectionOut			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Provider

This parameter contains the identifier of the LDev of the provider.

Parameter Type: LPWSTR

QoSType

This parameter contains the designation of the QoS type for all connections.

Parameter Type: unsigned short

QoSValue

This parameter contains the designation of the QoS value for all connections.

Parameter Type: unsigned short

State

This parameter contains the initial state of the connection; i.e. data transmission immediately and/or after the invocation of SetActivationState service.

Parameter Type: VT_BOOLEAN

Count

This parameter contains the number of connections that are to be added.

Parameter Type: unsigned long

List of pAddConnectionIn

This parameter contains the connection information. The number of elements is determined by the parameter Count.

Parameter Type: ADDCONNECTIONIN

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppAddConnectionOut

This parameter contains the connection results. The number of elements is determined by the parameter Count.

Parameter Type: ADDCONNECTIONOUT

Allowed values for ErrorState (hresult = S_OK): S_OK

Allowed values for ErrorState (hresult = S_FALSE): S_OK, CBA_E_MALFORMED, CBA_E_UNKNOWNOBJECT, CBA_E_UNKNOWNMEMBER, CBA_E_INUSE, CBA_E_QOSTYPEUNSUPPORTED, CBA_E_QOSVALUEUNSUPPORTED, CBA_E_INVALIDEPSILON, CBA_E_PERSISTRUNNING, E_INVALIDARG, E_OUTOFMEMORY, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.2 RemoveConnections

The RemoveConnection service removes one or multiple connection records from the ConsumerIDs that were obtained via AddConnections. Table 680 shows the parameters of the service.

Table 680 – RemoveConnections

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
List of pConsumerID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of connections that are to be removed.

Parameter Type: unsigned long

List of pConsumerID

This parameter contains the ConsumerIDs of the connections. The number of elements is determined by the parameter Count.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the connection specific faults. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_INVALIDID, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_PERSISTRUNNING, E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.3 ClearConnections

The ClearConnections service clears all connections in the ACCO object. Table 681 shows the parameters of the service.

Table 681 – ClearConnections

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_PERSISTRUNNING, E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.4 SetActivationState

The SetActivationState service influences the activation state (enable or disable) of one or multiple connections. Table 682 shows the parameters of the service.

Table 682 – SetActivationState

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
State	M	M(=)		
Count	M	M(=)		
List of pConsumerID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

State

This parameter contains the state of the connection (i.e. whether data shall be transferred for this connection).

Parameter Type: VT_BOOLEAN

Count

This parameter contains the number of connections that are to be activated or deactivated.

Parameter Type: unsigned long

List of pConsumerID

This parameter contains the ConsumerIDs of the connections. The number of elements is determined by the parameter Count.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the connection specific faults. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_INVALIDID, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_E_PERSISTRUNNING, E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.5 GetInfo

The GetInfo service reads the maximum number of possible and currently used connections. The maximum number of possible connections is based on the current resource situation and the assumption, that all following connections loaded are of scalar type, i.e. from *VT_BOOLEAN* up to *double*. Table 683 shows the parameters of the service.

Table 683 – GetInfo

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pMax			M	M(=)
pCurCnt			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pMax

This parameter contains the maximum possible number of connections. If the ACCO object is not capable to provide a reasonable estimation, 0xFFFFFFFF is returned instead.

Parameter Type: unsigned long

pCurCnt

This parameter contains the currently used number of connections.

Parameter Type: unsigned long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_FAIL, E_INVALIDARG

15.3.3.4.6 GetIDs

The GetIDs service reads the major connection information. Table 684 shows the parameters of the service.

Table 684 – GetIDs

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pCount			M	M(=)
List of ppGetIDOut			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pCount

This parameter contains the number of connections.

Parameter Type: unsigned long

List of ppGetIDOut

This parameter contains the connection specific properties. The number of elements is determined by the parameter pCount.

Parameter Type: GETIDOUT

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.7 GetConnections

The GetConnections service reads all connection information. Table 685 shows the parameters of the service.

Table 685 – GetConnections

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
List of pConsumerID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppGetConnectionOut			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of connections.

Parameter Type: unsigned long

List of pConsumerID

This parameter contains the ConsumerIDs of all connections of this ACCO object. The number of elements is determined by the parameter Count.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

List of ppGetConnectionOut

This parameter contains connection specific properties. The number of elements is determined by the parameter Count.

Parameter Type: GETCONNECTIONOUT

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.8 ReviseQoS

The ReviseQoS service allows to verify a QoS value. Table 686 shows the parameters of the service.

Table 686 – ReviseQoS

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
RTAuto	M	M(=)		
QoSType	M	M(=)		
QoSValue	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pRevisedQoSValue			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

RTAuto

This parameter contains the identifier of the RT-Auto object.

Parameter Type: LPWSTR

QoSType

This parameter contains the designation of the QoS type for all connections.

Parameter Type: unsigned short

QoSValue

This parameter contains the designation of the QoS value for all connections.

Parameter Type: unsigned short

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pRevisedQoSValue

This parameter contains revised additional information.

Parameter Type: unsigned short

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.9 get_PingFactor

The get_PingFactor service reads the current PingFactor that is used for determining the maximum dead time of a provider. This service reads the FAL Class attribute PingFactor. After the PingFactor gets changed via the put_PingFactor service up to reflecting that change to the persistence data base, this service returns CBA_S_PERSISTPENDING to inform the caller. Table 687 shows the parameters of the service.

Table 687 – get_PingFactor

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the PingFactor.

Parameter Type: unsigned short

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_S_PERSISTPENDING, E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.10 put_PingFactor

The put_PingFactor service sets the PingFactor that is used for determining the maximum dead time of a provider. This service puts the FAL Class attribute PingFactor. Table 688 shows the parameters of the service.

Table 688 – put_PingFactor

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
NewVal	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

NewVal

This parameter contains the dead time within which a message is expected from a provider (PingFactor * minimum QoS).

Parameter Type: unsigned short

Allowed values: see Table 674.

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.11 get_CDBCcookie

The get_CDBCcookie service reads the current Configuration Data Base Cookie. It changes if the Configuration Data Base changes. Table 689 shows the parameters of the service.

Table 689 – get_CDBCcookie

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the Configuration Data Base change indicator.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoMgt interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_NOTIMPL

15.3.3.4.12 Connect

The Connect service adds one or multiple connection sources to the ACCO object. Table 690 shows the parameters of the service.

Table 690 – Connect

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Consumer	M	M(=)		
QoSType	M	M(=)		
QoSValue	M	M(=)		
State	M	M(=)		
PICBAAccoCallback	M	M(=)		
Count	M	M(=)		
List of pConnectIn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)
PFirstConnect			M	M(=)
List of ppConnectOut			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
Hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

Consumer

This parameter contains the identifier of the consumer's LDev in the form of "PDev!LDev".

Parameter Type: LPWSTR

QoSType

This parameter contains the designation of the QoS type for all connections.

Parameter Type: unsigned short

QoSValue

This parameter contains the designation of the QoS value for all connections.

Parameter Type: unsigned short

State

This parameter contains the initial state of the connection; i.e. data transmission immediately and/or after the invocation of the SetActivationState service.

Parameter Type: VT_BOOLEAN

pICBAAccoCallback

This parameter contains the reference to the callback interface that is used for transferring connection data.

Parameter Type: Interface Pointer

Count

This parameter contains the number of connections that are to be connected.

Parameter Type: unsigned long

List of pConnectIn

This parameter contains the connection information. The number of elements is determined by the parameter Count.

Parameter Type: CONNECTIN

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

pFirstConnect

This parameter contains information whether this is the first established connection.

Parameter Type: VT_BOOLEAN

List of ppConnectOut

This parameter contains the connection specific results. The number of elements is determined by the parameter Count.

Parameter Type: CONNECTOUT

Allowed values for ErrorState (hresult = S_OK): S_OK

Allowed values for ErrorState (hresult = S_FALSE): S_OK, CBA_E_MALFORMED, CBA_E_UNKNOWNOBJECT, CBA_E_UNKNOWNMEMBER, CBA_E_INUSE, CBA_E_QOSTYPEUNSUPPORTED, CBA_E_QOSVALUEUNSUPPORTED, CBA_E_INVALIDEPSILON, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.13 Disconnect

The Disconnect service clears one or multiple connection blocks from the ACCO object. Table 691 shows the parameters of the service.

Table 691 – Disconnect

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
List of pProviderID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of connections that are to be disconnected.

Parameter Type: unsigned long

List of pProviderID

This parameter contains the provider IDs of the connections. The number of elements is determined by the parameter Count.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the connection specific faults. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_INVALIDID, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.14 DisconnectMe

The DisconnectMe service clears all connection blocks of a consumer. Table 692 shows the parameters of the service.

Table 692 – DisconnectMe

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Consumer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

Consumer

This parameter contains the identifier of the consumer's LDev in the form of "PDev!LDev".

Parameter Type: LPWSTR

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, CBA_S_NOCONNECTION

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.15 SetActivation

The SetActivation service influences the activation state (enable or disable) of one or multiple connections. Table 693 shows the parameters of the service.

Table 693 – SetActivation

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
State	M	M(=)		
Count	M	M(=)		
List of pProviderID	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

State

This parameter contains the State of the connection; i.e. data transmission immediately and/or after the invocation of the SetActivationState service.

Parameter Type: VT_BOOLEAN

Count

This parameter contains the number of connections that are to be activated or deactivated.

Parameter Type: unsigned long

List of pProviderID

This parameter contains the Provider Ids of the connections. The number of elements is determined by the parameter Count.

Parameter Type: unsigned long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the connection specific faults. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_INVALIDID, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_FAIL, E_INVALIDARG

15.3.3.4.16 Ping

The Ping service is invoked by the consumer for heartbeat monitoring if the provider does not send a message to the consumer within a specified dead time. Table 694 shows the parameters of the service.

Table 694 – Ping

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Consumer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

Consumer

This parameter contains the identifier of the consumer's LDev in the form of "PDev!LDev".

Parameter Type: LPWSTR

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoServer interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: CBA_S_NOCONNECTION

15.3.3.4.17 OnDataChanged

The OnDataChanged service is invoked by a provider part of an ACCO object in order to transmit new connection data via a byte stream of the structure shown in Figure 195.

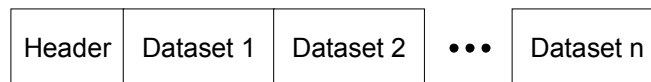


Figure 195 — Structure of the transmitted connection data

Table 695 shows the parameters of the service.

Table 695 – OnDataChanged

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Length	M	M(=)		
pBuffer	M	M(=)		
Header	M	M(=)		
Version	M	M(=)		
Flags	M	M(=)		
Count	M	M(=)		
List of Datasets	M	M(=)		
Length	M	M(=)		
ConsumerID	M	M(=)		
Data Connection Data	S	S(=)		
QualityCode	M	M(=)		
Value	M	M(=)		
Event Connection Data	S	S(=)		
List of Parameter values	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoCallback interface of the requested object instance.

Parameter Type: Interface Pointer

Length

This parameter contains the overall length of the transferred connection data in octets.

Parameter Type: long

pBuffer

This parameter contains the transferred connection data. The data shall be serialized according the used data types with least significant octet first and without any padding octets.

Header

This parameter contains information about the transferred data sets.

Version

This parameter contains the hexadecimal value of 0x01 to indicate version 1.

Parameter Type: unsigned char

Flags

This parameter describes the structure of the data within the connection data sets. Therefore, that field is reserved for future use and shall be set to zero.

Parameter Type: unsigned char

Allowed values: 0

Count

This parameter specifies the number of the subsequent data sets that are to be transmitted.

Parameter Type: unsigned short

List of Datasets

This parameter contains the list of individual data sets. The number of data sets is determined by the parameter Count in the header.

Length

This parameter contains the length of the data including the length itself.

Parameter Type: unsigned short

ConsumerID

This parameter contains identification of the connection sink data in the consumer.

Parameter Type: unsigned long

Data Connection Data

This parameter contains the new data in the case of a data connection.

QualityCode

This parameter contains the Quality Code of the data value.

Parameter Type: ITEMQUALITYDEF

Value

This parameter contains the value itself with the format according to the appropriate data type.

Parameter Type: see Table 696

Event Connection Data

This parameter contains the new data in the case of an event connection. It corresponds

List of Parameter values

This parameter contains the list of parameter values of the connection source according to the [IN] tags defined by IDL in IEC 61158-6 Type 10. It refers the defined data types and the order of definition.

Parameter Type: see Table 696

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoCallback interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoCallback interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

The following table contains the allowed data types for Value and Parameter value.

Table 696 – Data Type Names for Values

Data Type Names
VT_BOOLEAN
char
short
long
unsigned char
unsigned short
unsigned long
float
double
date
currency
BSTR
SAFEARRAY
Struct

15.3.3.4.18 ReadItems

The ReadItems service is used for consistent reading of value, quality code and time stamp. Table 697 shows the parameters of the service.

Table 697 – ReadItems

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
List of pReadItem	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppReadItemOut			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of items that are to be read.

Parameter Type: unsigned long

List of pReadItem

This parameter contains the identifiers of the items. The number of elements is determined by the parameter Count.

Parameter Type: LPWSTR

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppReadItemOut

This parameter contains the item informations. The number of elements is determined by the parameter Count.

Parameter Type: READITEMOUT

Allowed values for ErrorState (hresult = S_OK): S_OK

Allowed values for ErrorState (hresult = S_FALSE): S_OK, CBA_E_MALFORMED, CBA_E_UNKNOWNOBJECT, CBA_E_UNKNOWNMEMBER, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_INVALIDARG

15.3.3.4.19 WriteItems

The WriteItems service is used for writing data via an identifier. Table 698 shows the parameters of the service.

Table 698 – WriteItems

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
List of pWriteItem	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
List of ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of items that are to be written.

Parameter Type: unsigned long

pWriteItem

This parameter contains the identifiers of the items and their values. The number of elements is determined by the parameter Count.

Parameter Type: WRITEITEMIN

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the item specific errors. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_MALFORMED, CBA_E_UNKNOWNOBJECT, CBA_E_UNKNOWNMEMBER, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_INVALIDARG

15.3.3.4.20 WriteltemsQCD

The WriteltemsQCD service is used for consistent writing of value, quality code and time stamp. Table 699 shows the parameters of the service.

Table 699 – WriteltemsQCD

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Count	M	M(=)		
pWriteltemQcdIn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
ppError			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

Count

This parameter contains the number of items that are to be written.

Parameter Type: unsigned long

List of pWriteltemQcdIn

This parameter contains the identifiers of the items and their values. The number of elements is determined by the parameter Count.

Parameter Type: WRITEITEMQCDIN

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

List of ppError

This parameter contains the item specific errors. The number of elements is determined by the parameter Count.

Parameter Type: HRESULT

Allowed values (hresult = S_OK): S_OK

Allowed values (hresult = S_FALSE): S_OK, CBA_E_MALFORMED, CBA_E_UNKNOWNOBJECT, CBA_E_UNKNOWNMEMBER, E_FAIL

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAAccoSync interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY, E_INVALIDARG

15.3.3.4.21 GroupError

The GroupError service reads the error state of the ACCO object. The returned cookie specifies whether or not the group error state of the device has changed since the last invocation. The arguments of the service are shown in Table 700.

Table 700 – GroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
pMagicCookie			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the group error state of the ACCO object.

Parameter Type: GROUPERRORDEF

pMagicCookie

This parameter contains the group error state change indicator.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.3.4.22 AdviseGroupError

The AdviseGroupError service advises an event to the event interface ICBAGroupErrorEvent at the server. The arguments of the service are shown in Table 701.

Table 701 – AdviseGroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
pGroupErrorEvent	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pCookie			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

pGroupErrorEvent

This parameter contains the address of the event interface.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK, S_FALSE

pCookie

This parameter contains the event interface handle.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.3.4.23 UnadviseGroupError

The UnadviseGroupError service clears the event advise specified by cookie. The arguments of the service are shown in Table 702.

Table 702 – UnadviseGroupError

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Cookie	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

Cookie

This parameter contains the handle to the event interface.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBAGroupError interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY, CBA_E_INVALIDCOOKIE

15.3.4 RT-auto ASE**15.3.4.1 Overview**

The runtime automation object (RT-Auto object) ASE represents the automation functionality in the form of a process-related component. Examples are boiler, controller, etc. These automation functionalities of an application may exist in the form of pre-tested, self-contained and universally useable components. Using the interconnecting functionality that is defined by ACCO ASE, automation functionalities can be configured as a distributed application.

15.3.4.2 RT-auto class specifications**15.3.4.2.1 Formal Definition**

- | | | | |
|-----|-----|----------------|---------------------------|
| 1. | (m) | Key Attribute: | Implicite |
| 2. | (m) | Attribute: | ICBARTAuto |
| 2.1 | (m) | Attribute: | Name |
| 2.2 | (m) | OpsService: | get_Name |
| 2.3 | (m) | OpsService: | Revision |
| 3. | (m) | Attribute: | ICBABrowse |
| 3.1 | (m) | Attribute: | Count |
| 3.2 | (m) | OpsService: | get_Count |
| 3.3 | (m) | OpsService: | BrowseItems |
| 4. | (m) | Attribute: | List of ICBACustom |
| 4.1 | (m) | Attribute: | List of Custom Attributes |
| 4.1 | (m) | Attribute: | List of Custom Services |
| 5. | (m) | Attribute: | Dispatchable |

15.3.4.2.2 Attributes**ICBARTAuto**

This attribute specifies the RT-Auto interface of the object. It is a structured attribute composed of the following attributes.

Name

This attribute specifies the name of the RT-Auto object.

Attribute Type: BSTR

ICBABrowse

This attribute specifies the browse interface of the object. It is a structured attribute composed of the following attributes.

Count

This attribute specifies the number of elements in the browse database.

Attribute Type: long

List of ICBACustom

This attribute specifies a list of one or more custom interfaces of the object that shall be defined according to the automation function of a specific RT-Auto Class. The number of custom interfaces is not limited and the actual contents is beyond the scope of this standard. It is a structured attribute composed of the following attributes.

List of Custom Attributes

This attribute specifies a list of possible attributes per custom interface. They shall be defined task specific by the manufacturer and are beyond the scope of this standard. These attributes define data that are issue of data connections managed by the ACCO ASE. Therefore, the possible data types for those attributes shall be only a subset of possible data types.

The Table 703 defines the data types that are valid for a data connection, and the parameter types that are valid for an event connection.

Table 703 – Allowed Data Types

Data Type Names
VT_BOOLEAN
char
short
long
unsigned char
unsigned short
unsigned long
float
double
date
currency
BSTR
SAFEARRAY
structure

Only data of the same data type can be connected with each other. For an array, in particular, this means that dimension, length and base data type of source and destination shall be compatible with each other. With a structure, the layout of source and destination shall be compatible (i.e. the data types of the members).

List of Custom Services

This optional attribute specifies a list of possible services per custom interface. They shall be defined task specific by the manufacturer and are beyond the scope of this standard. However, these services define events that are issue of event connections managed by the ACCO ASE.

Dispatchable

This attribute specifies the availability of type information for the FAL Class that is provided by the IDispatch common interface. The RT-Auto ASE provides that kind of information and the Dispatchable attribute shall contain the value 1.

Attribute Type: Boolean

15.3.4.3 RT-Auto ASE Behavior

The RT-Auto object represents the automation functionality as a process-related component. These automation functions exist in the form of pre-tested, self-contained and universally deployable components. Using the interconnecting functionality defined by ACCO ASE, they can be configured as a distributed application.

This functionality, usable for an application, can be subdivided into a fixed and a loadable functionality. A fixed functionality is a function implemented invariably in a LDev. A loadable functionality is programmed in the Engineering System and will be loaded into the LDev in a separate step. From the runtime perspective, there is no difference in the outside view onto the components.

15.3.4.4 RT-auto ASE service specification

15.3.4.4.1 get_Name

The get_Name service reads the name of the RT-Auto object. The name is used for navigation. The arguments of the service are shown in Table 704. The service reads the FAL Class attribute Name.

Table 704 – get_Name

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the name of the RT-Auto object.

Parameter Type: BSTR

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.4.4.2 Revision

The Revision service reads the revision level of the RT-Auto (i.e. the software version) as a major and minor revision. The arguments of the service are shown in Table 705.

Table 705 – Revision

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pMajor			M	M(=)
pMinor			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pMajor

This parameter contains the major part of the revision.

Parameter Type: short

pMinor

This parameter contains the minor part of the revision.

Parameter Type: short

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBARTAuto interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_OUTOFMEMORY

15.3.4.4.3 get_Count

The get_Count service reads the number of elements (attribute count) in the browse database. The arguments of the service are shown in Table 706. The service reads the FAL Class attribute Count.

Table 706 – get_Count

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
pVal	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pVal			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pVal

This parameter contains the number of elements.

Parameter Type: long

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.3.4.4.4 BrowseItems

The BrowseItems service reads the list of available items of the RT-Auto object. The arguments of the service are shown in Table 707.

Table 707 – BrowseItems

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Offset	M	M(=)		
MaxReturn	M	M(=)		
Result (+)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)
pItem			M	M(=)
pDataType			M	M(=)
pAccessRight			M	M(=)
Result (-)			S	S(=)
Interface Pointer			M	M(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

Offset

This parameter contains the offset of the first requested RT-Auto item.

Parameter Type: long

MaxReturn

This parameter contains the maximum number of received items.

Parameter Type: long

Result (+):

This selection type parameter indicates that the service request succeeded.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

pltem

This parameter contains the name or list of names of the available attributes.

Parameter Type: VARIANT

pDataType

This parameter contains the data type or list of data types of the available attributes.

Parameter Type: VARIANT

Allowed values: Tag=VT_SAFEARRAY_LONG, Value according VARTYPE

pAccessRight

This parameter contains the access privileges or list of access privileges of the available attributes.

Parameter Type: VARIANT

Allowed values: Tag=VT_SAFEARRAY_LONG, Value according ACCESSRIGHTSDEF

Result (-):

This selection type parameter indicates that the service request failed.

Interface Pointer

This parameter specifies the address of the ICBABrowse interface of the requested object instance.

Parameter Type: Interface Pointer

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: E_INVALIDARG, E_OUTOFMEMORY

15.4 ARs**15.4.1 Overview**

The type 10 FAL provides exactly one AR Class referred to as ORPC AR. This AR type provides many-to-many, bi-directional, user-triggered and queued application relation utilizing the client server model in both roles. Each device shall implement exactly one instance of that class to support the transmission of all type 10 ASE services by means of the AR-Call service. Therefore, it multiplexes and de-multiplexes all communication paths. The path or binding information derived from the common service parameter Interface Pointer is maintained by the abstract ORPC instance as the lower part of the application layer. By means of the AR-CoCreateInstance service an application process gets this binding information to a remote object instance with the appropriate Interface Pointer within the response. The host and socket information is stored by the abstract ORPC sublayer and is no subject of type 10 AR or APO ASE's. The type 10 object model only allows the usage of the AR-CoCreateInstance service for the Physical Device class (PDev). Following the type 10 navigation procedure it is possible to get the binding information to other type 10 ASE objects (LDev, ACCO, RT-Auto).

15.4.2 ORPC AR class specification**15.4.2.1.1 Formal Definition**

- | | | | |
|----|-----|-------------|--------------------------|
| 1. | (m) | Attribute: | IDL Marshaling Reference |
| 2. | (m) | Attribute: | ORPC Mapping Reference |
| 3. | (m) | Attribute: | ORPC Binding Reference |
| 4. | (m) | OpsService: | CoCreateInstance |
| 5. | (m) | OpsService: | Call |

15.4.2.1.2 Attributes

IDL Marshaling Reference

This attribute specifies the usage of the interface definition language (IDL) to marshal the service parameter into the ORPC-PDUs as defined in the type 10 protocol specification.

ORPC Mapping Reference

This attribute specifies the mapping of the AR services into the ORPC services of the abstract ORPC model as defined in the type 10 protocol specification.

ORPC Binding Reference

This attribute specifies the binding of the Interface Pointer of the AR services into the additional host and socket information that are maintained by the abstract ORPC model. The used abstract ORPC model provides the management of that information for the convenience of APO ASEs and AR ASEs.

15.4.2.2 ORPC AR service specification

15.4.2.2.1 CoCreateInstance

The CoCreateInstance service creates a binding to a remote instance of a FAL ASE. The arguments of the service are shown in Table 708.

Table 708 – CoCreateInstance

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Host address	M	M(=)		
Class ID	M	M(=)		
Interface ID	M	M(=)		
Result (+)			S	S(=)
hresult			M	M(=)
Interface Pointer			M	M(=)
Result (-)			S	S(=)
hresult			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Host Address

This parameter contains the address information of the remote node as IP address or DNS name.

Parameter Type: VARIANT

Class ID

This parameter contains the unique class identifier of the remote object. Only the Class ID of the Physical Device Class is valid.

Parameter Type: UUID

Allowed Values: UUID_PhysicalDevice

Interface ID

This parameter specifies the unique interface identifier of the requested interface of the remote object.

Parameter Type: UUID

Result (+):

This selection type parameter indicates that the service request succeeded.

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Allowed values: S_OK

Interface Pointer

This parameter specifies the address of the requested interface of the remote object.

Parameter Type: Interface Pointer

Result (-):

This selection type parameter indicates that the service request failed.

hresult

This parameter contains the result code.

Parameter Type: HRESULT

15.4.2.2.2 Call

The Call service transports all type 10 ASE services. The arguments of the service are shown in Table 709.

Table 709 – Call

Parameter name	.req	.ind	.rsp	.cnf
Argument	M	M		
Interface Pointer	M	M(=)		
Service Name	M	M(=)		
List of Unified Service In-Parameter	M	M(=)		
Result (+)			S	S(=)
hresult			M	M(=)
Service Name			M	M(=)
List of Unified Service Out-Parameter			M	M(=)
Result (-)			S	S(=)
hresult			M	M(=)
Service Name			M	M(=)
List of Unified Service Out-Parameter			M	M(=)

Argument

The argument shall convey the service specific parameters of the service request.

Interface Pointer

This parameter specifies the address of the requested interface.

Parameter Type: Interface Pointer

Service Name

This parameter contains the name or method number of the type 10 ASE service that is used to convey the application data.

Parameter Type: VARIANT

List of Unified Service In-Parameter

This parameter contains all parameters that are issued by the type 10 ASE service. By means of the IDL information and the service and interface name the abstract ORPC model provides the marshaling of the service parameter and codes the ORPC-PDU.

Parameter Type: VARIANT

Result (+):

This selection type parameter indicates that the service request succeeded.

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Service Name

This parameter contains the name or method number of the type 10 ASE service that is used to convey the application data.

Parameter Type: VARIANT

List of Unified Service Out-Parameter

This parameter contains all parameters that are issued by the type 10 ASE service. This parameter is interpreted according the called type 10 ASE service parameter definition.

Parameter Type: VARIANT

Result (-):

This selection type parameter indicates that the service request failed.

hresult

This parameter contains the result code.

Parameter Type: HRESULT

Service Name

This parameter contains the name or method number of the type 10 ASE service that is used to convey the application data.

Parameter Type: VARIANT

List of Unified Service Out-Parameter

This parameter contains all parameters that are issued by the type 10 ASE service. This parameter is interpreted according the called type 10 ASE service parameter definition.

Parameter Type: VARIANT

15.5 Summary of FAL classes

This subclause contains a summary of the defined fieldbus AL Classes.

Table 710 shows the fieldbus AL Class Summary.

Table 710 – Fieldbus AL class summary

AL ASE	Class
AP	Physical Device Logical Device
ACCO	ACCO
RT-Auto	RT-Auto

15.6 Summary of FAL services

Table 711 gives an overview of the assignment of services to the Client and the Server.

Table 711 – Assignment of the services to client and server

Service	Client		Server	
	req	con	ind	rsp
Physical Device				
QueryInterface	x	x	x	x
AddRef	x	x	x	x
Release	x	x	x	X
GetTypeInfoCount	x	x	x	x
GetTypeInfo	x	x	x	x
GetIDsOfNames	x	x	x	X
Invoke	x	x	X	x
get_Producer	x	x	x	x
get_Product	x	x	x	x
get_SerialNo	x	x	x	x
get_ProductionDate	x	x	x	x
Revision	x	x	x	x
get_LogicalDevice	x	x	x	x
get_Count	x	x	x	x
BrowseItems	x	x	x	x

Service	Client		Server	
	req	con	ind	rsp
Save	x	x	x	x
Logical Device				
QueryInterface	x	x	x	x
AddRef	x	x	x	x
Release	x	x	x	X
GetTypeInfoCount	x	x	x	x
GetTypeInfo	x	x	x	x
GetIDsOfNames	x	x	x	X
Invoke	x	x	X	x
get_Name	x	x	x	x
get_Producer	x	x	x	x
get_Product	x	x	x	x
get_SerialNo	x	x	x	x
get_ProductionDate	x	x	x	x
Revision	x	x	x	x
get_ACCO	x	x	x	x
get_RTAuto	x	x	x	x
get_State	x	x	x	x
Activate	x	x	x	x
Deactivate	x	x	x	x
Reset	x	x	x	x
AdviseState	x	x	x	x
UnadviseState	x	x	x	x
get_Time	x	x	x	x
put_Time	x	x	x	x
get_Count	x	x	x	x
BrowseItems	x	x	x	x
GroupError	x	x	x	x
AdviseGroupError	x	x	x	x
UnadviseGroupError	x	x	x	x
ACCO				
QueryInterface	x	x	x	x
AddRef	x	x	x	x
Release	x	x	x	X
AddConnections	x	x	x	x
RemoveConnections	x	x	x	x
SetActivationState	x	x	x	x
GetInfo	x	x	x	x
GetIDs	x	x	x	x
GetConnections	x	x	x	x
ReviseQoS	x	x	x	x
get_PingFactor	x	x	x	x
get_CDBCcookie	x	x	x	x
Connect	x	x	x	x
Disconnect	x	x	x	x
DisconnectMe	x	x	x	x
SetActivation	x	x	x	x
Ping	x	x	x	x
OnDataChanged	x	x	x	x
ReadItems	x	x	x	x
WriteItems	x	x	x	x
GroupError	x	x	x	x
AdviseGroupError	x	x	x	x
UnadviseGroupError	x	x	x	x

Service	Client		Server	
	req	con	ind	rsp
RT-Auto				
QueryInterface	x	x	x	x
AddRef	x	x	x	x
Release	x	x	x	X
GetTypeInfoCount	x	x	x	x
GetTypeInfo	x	x	x	x
GetIDsOfNames	x	x	x	X
Invoke	x	x	X	x
get_Name	x	x	x	x
Revision	x	x	x	x
get_Count	x	x	x	x
BrowseItems	x	x	x	x



Standards Survey

The IEC would like to offer you the best quality standards possible. To make sure that we continue to meet your needs, your feedback is essential. Would you please take a minute to answer the questions overleaf and fax them to us at +41 22 919 03 00 or mail them to the address below. Thank you!

Customer Service Centre (CSC)

International Electrotechnical Commission

3, rue de Varembe
1211 Genève 20
Switzerland

or

Fax to: **IEC/CSC** at +41 22 919 03 00

Thank you for your contribution to the standards-making process.

A Prioritaire

Nicht frankieren
Ne pas affranchir



Non affrancare
No stamp required

RÉPONSE PAYÉE

SUISSE

Customer Service Centre (CSC)
International Electrotechnical Commission
3, rue de Varembe
1211 GENEVA 20
Switzerland



Q1 Please report on **ONE STANDARD** and **ONE STANDARD ONLY**. Enter the exact number of the standard: (e.g. 60601-1-1)

.....

Q2 Please tell us in what capacity(ies) you bought the standard (tick all that apply). I am the/a:

- purchasing agent ☐
 librarian ☐
 researcher ☐
 design engineer ☐
 safety engineer ☐
 testing engineer ☐
 marketing specialist ☐
 other

Q3 I work for/in/as a:
(tick all that apply)

- manufacturing ☐
 consultant ☐
 government ☐
 test/certification facility ☐
 public utility ☐
 education ☐
 military ☐
 other

Q4 This standard will be used for:
(tick all that apply)

- general reference ☐
 product research ☐
 product design/development ☐
 specifications ☐
 tenders ☐
 quality assessment ☐
 certification ☐
 technical documentation ☐
 thesis ☐
 manufacturing ☐
 other

Q5 This standard meets my needs:
(tick one)

- not at all ☐
 nearly ☐
 fairly well ☐
 exactly ☐

Q6 If you ticked NOT AT ALL in Question 5 the reason is: (tick all that apply)

- standard is out of date ☐
 standard is incomplete ☐
 standard is too academic ☐
 standard is too superficial ☐
 title is misleading ☐
 I made the wrong choice ☐
 other

Q7 Please assess the standard in the following categories, using the numbers:

- (1) unacceptable,
 (2) below average,
 (3) average,
 (4) above average,
 (5) exceptional,
 (6) not applicable

- timeliness
 quality of writing.....
 technical contents.....
 logic of arrangement of contents
 tables, charts, graphs, figures.....
 other

Q8 I read/use the: (tick one)

- French text only ☐
 English text only ☐
 both English and French texts ☐

Q9 Please share any comment on any aspect of the IEC that you would like us to know:

.....



ISBN 2-8318-6973-0



9 782831 869735

ICS 25.040; 35.100; 35.240.50
